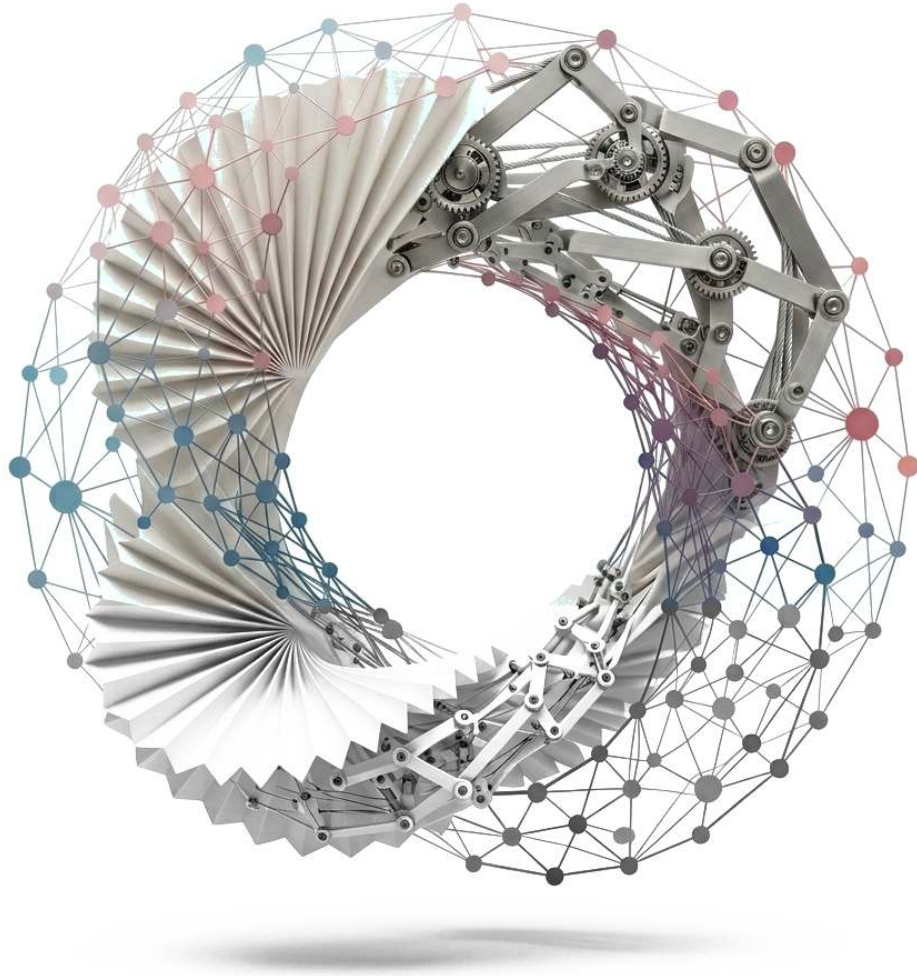




Physical AI

Machine Learning Systems

Vijay
Janapa Reddi

Physical AI

Machine Learning Systems That Sense and Act

DRAFT

Physical AI

Machine Learning Systems That Sense and Act

Vijay Janapa Reddi

Harvard University

DRAFT

The MIT Press
Cambridge, Massachusetts
London, England

Contents

Preface	xiii
Dedication	xxi
Author’s Note	xxiii
About the Author	xxv
Why I Am Writing This Book: The First-Principles Systems Stance	xxv
Intellectual Lineage and Systems Trajectory	xxvi
The Kit and Laboratory Development	xxvi
Reader Guide and Prerequisites	xxvii
The Technical Baseline	xxvii
Disciplinary Reading Pathways	xxvii
What the Book Introduces	xxviii
What We Deliberately Omit	xxviii
Acknowledgments	xxix
A Note on AI Assistance	xxxi
Notation	xxxiii
Physical AI and Embodied Systems Notation	xxxviii
1. Physical Plant State, Actuation, and Kinematic Topology	xli
2. Rigid-Body Contact Mechanics, Friction Cones, and Centroidal Dynamics	xlii
3. Geometry, Spatial Transformations, and Perception Projections	xliii
4. Classical and Modern Control (State-Space, Observers, CBF-QPs)	xliv
5. Deep Learning for Robotics (Diffusion Policies, ACT, VLAs)	xlvi
6. Embedded Systems, Silicon Real-Time, Thermal, and Hardware Safety	xlvi

Main Text

Chapter 1 The Causal Boundary	3
Chapter 2 The Causal Boundary	5
2.1 Artificial Intelligence Beyond Glass	6
2.2 The Causal Boundary	8
2.3 The Four Machine Archetypes	13
2.4 The Physical AI Scope Test	18
2.5 The Four Architectural Tiers	21
2.6 The S-P-A Intersection Landscape	22
2.7 When the Boundary Fails	23
2.8 The Four Bedrock Laws	26
2.9 The Dual-Brain Architecture	28
2.10 Safety Convergence	30

2.11	The Architecture of This Book	32
2.12	Fallacies and Pitfalls	32
2.13	Summary	33

Part I: The Machine Anatomy

The Machine Anatomy	37
----------------------------	-----------

Chapter 3 The Physical Body	39
------------------------------------	-----------

Chapter 4 The Physical Body	41
------------------------------------	-----------

4.1	The Five Physical Budgets	42
4.2	Measurement Freshness	44
4.3	Kinetic Momentum	46
4.4	Actuator Transmission Limits	50
4.5	Thermal Duty Cycles	55
4.6	Electrical Power Integrity	61
4.7	Measuring a Machine's Own Limits	64
4.8	The Cross-Archetype Matrix	67
4.9	Fallacies and Pitfalls	69
4.10	Summary	70

Chapter 5 The Cognitive Brain	73
--------------------------------------	-----------

Chapter 6 The Cognitive Brain	75
--------------------------------------	-----------

6.1	The Embodied Brain	76
6.2	The Classical Wall	78
6.3	Learned Representations	80
6.4	The Cognitive Pipeline	85
6.5	Cognitive Vulnerabilities	95
6.6	Fallacies and Pitfalls	100
6.7	Summary	101

Chapter 7 The Nervous System	103
-------------------------------------	------------

Chapter 8 The Nervous System	105
-------------------------------------	------------

8.1	The Multi-Rate Bridge	106
8.2	Silicon Partitioning	108
8.3	Multi-Rate Cadences	110
8.4	Deterministic Fieldbuses	116
8.5	Actuation Authority	120
8.6	Hardware Isolation	122
8.7	Invariant Checking	124
8.8	Supervisory Teleoperation	126
8.9	The Handoffs Matrix	128
8.10	Fallacies and Pitfalls	131
8.11	Summary	132

Part II: Teaching the Machine

Teaching the Machine	137
-----------------------------	------------

Chapter 9 Physical Data	139
--------------------------------	------------

Chapter 10 Physical Data	141
---------------------------------	------------

10.1	Endogenous Experience	142
10.2	Where Physical Data Comes From	144

10.3	Teleoperation Demonstration Costs	149
10.4	Collection Policy Coverage	157
10.5	Intervention and Recovery Data	159
10.6	Cross-Embodiment Transfer	161
10.7	How Datasets Go Wrong	165
10.8	The Dataset Schema	169
10.9	Fallacies and Pitfalls	172
10.10	Summary	173
Chapter 11 Policy Training		175
Chapter 12 Policy Training		177
12.1	Policy Synthesis	178
12.2	Policy Synthesis Regimes	179
12.3	Learning by Trial	190
12.4	The Simulator as a System Component	193
12.5	What the Gap Is Made Of	198
12.6	Fine-Tuning on the Real Machine	204
12.7	How Training Choices Go Wrong	205
12.8	The Policy Manifest	207
12.9	Fallacies and Pitfalls	211
12.10	Summary	212
Chapter 13 Closed-Loop Evaluation		215
Chapter 14 Closed-Loop Evaluation		217
14.1	Closed-Loop Survival	218
14.2	What You Are Actually Measuring	220
14.3	Closed-Loop Dynamics	223
14.4	Evidence Regimes	227
14.5	Physical Trial Limits	230
14.6	What Evaluation Cannot Establish	238
14.7	How Evaluation Goes Wrong	241
14.8	Evaluation Logs	245
14.9	Fallacies and Pitfalls	248
14.10	Summary	249
Part III: Running the Machine		
Running the Machine		253
Chapter 15 Sensor Perception		255
Chapter 16 Sensor Perception		257
16.1	Photons to Spatial Claims	258
16.2	What Perception Hands Over	261
16.3	Sensor Transduction and Calibration	263
16.4	The Cost of Ingestion	269
16.5	Spatial Error and Clearance Bounds	271
16.6	How Observations Go Wrong	277
16.7	The Observation Contract	280
16.8	Fallacies and Pitfalls	282
16.9	Summary	283
Chapter 17 Spatial Memory		285
Chapter 18 Spatial Memory		287

18.1 Preserving Spatial Belief	288
18.2 What Memory Hands Over	289
18.3 A State Has an Epoch	291
18.4 Spatial Reference Frames	293
18.5 Volumetric Scene Memory and Attention Buffers	295
18.6 Belief Through Occlusion	303
18.7 The Rate of Belief Decay	305
18.8 State Estimation Bounds	307
18.9 How Belief Goes Wrong	309
18.10 The Spatial State Schema	311
18.11 Fallacies and Pitfalls	313
18.12 Summary	313
Chapter 19 Grounded Intent	317
Chapter 20 Grounded Intent	319
20.1 Grounded Affordances	320
20.2 What Intent Hands Over	322
20.3 From Request to Expiring Geometric Proposal	325
20.4 Goals That Expire	329
20.5 Admissible Goal Sets	331
20.6 How Intent Goes Wrong	334
20.7 The Intent Lease	335
20.8 Fallacies and Pitfalls	338
20.9 Summary	339
Chapter 21 Trajectory Planning	341
Chapter 22 Trajectory Planning	343
22.1 Continuous Trajectories	344
22.2 What Planning Hands Over	346
22.3 From Goal to Trajectory	347
22.4 Kinodynamic Feasibility	351
22.5 Behavior at the Seam	355
22.6 How Plans Go Wrong	357
22.7 The Trajectory Contract	359
22.8 Fallacies and Pitfalls	361
22.9 Summary	362
Chapter 23 Safety Enforcement	365
Chapter 24 Safety Enforcement	367
24.1 The Deterministic Gatekeeper	368
24.2 What Control Hands Over	370
24.3 The Authority to Refuse	371
24.4 Where the Enforcer Sits	374
24.5 Stopping Envelopes	376
24.6 When the Enforcer Runs Out of Authority	379
24.7 Safe Sets as Conditional Permission	382
24.8 Minimal Intervention	385
24.9 The Fallback Ladder	389
24.10 How Enforcement Goes Wrong	393
24.11 Enforcer Telemetry	396
24.12 Fallacies and Pitfalls	398
24.13 Summary	400
Chapter 25 Silicon Placement	403

Chapter 26 Silicon Placement	405
26.1 The Silicon Substrate	406
26.2 Two Paths on One Die	408
26.3 Contention for Shared Resources	411
26.4 The Boundary Contract	416
26.5 Thermal and Timing Coupling	420
26.6 Testing the Separation	423
26.7 Silicon Isolation Decisions	426
26.8 Hardware Allocation	428
26.9 Fallacies and Pitfalls	430
26.10 Summary	431
 Part IV: Governing the Machine	
 Governing the Machine	435
Chapter 27 Supervisory Intervention	437
Chapter 28 Supervisory Intervention	439
28.1 Shared Authority	440
28.2 Human Authority Over Actuation	441
28.3 Authority Transitions	448
28.4 Transfer Without Discontinuity	450
28.5 Authenticating an Intervention	454
28.6 Trustworthy Operational Logs	456
28.7 How Handover Goes Wrong	458
28.8 The Authority Log	460
28.9 Fallacies and Pitfalls	464
28.10 Summary	465
Chapter 29 Adversarial Verification	467
Chapter 30 Adversarial Verification	469
30.1 Fault Injection	470
30.2 The Operating Envelope	472
30.3 Deriving the Fault List	474
30.4 The Qualification Ladder	477
30.5 Fault Oracles and Acceptance	482
30.6 Fault Injection on Hardware	484
30.7 The Fault Manifest	490
30.8 Fallacies and Pitfalls	493
30.9 Summary	494
Chapter 31 Deployment Release	495
Chapter 32 Deployment Release	497
32.1 The Decision to Deploy	498
32.2 The Release Question	499
32.3 Safety Cases and Claims	502
32.4 The Three Verdicts	506
32.5 The Verdict's Standing Conditions	509
32.6 A Case Worked in Full	511
32.7 Adversarial Review	513
32.8 How Release Arguments Go Wrong	515
32.9 The Release Manifest	516
32.10 Fallacies and Pitfalls	520
32.11 Summary	521

Synthesis

Chapter 33 The Epistemic Frontier	525
Chapter 34 The Epistemic Frontier	527
34.1 Epistemic Limits	528
34.2 What the Method Cannot Establish	530
34.3 A Residual-Claims Register	533
34.4 Failures Without Detectors	536
34.5 What Would Change These Answers	542
34.6 Generalizing to Unfamiliar Systems	544
34.7 Architectural Synthesis Across the Physical AI Stack	546
34.8 Epistemic Governance	548
34.9 Fallacies and Pitfalls	549
34.10 Summary	550

Back Matter

Appendices	555
References	555
Appendix A Machine Learning Background	563
Appendix B Control and Dynamics	573
Appendix C Systems and Hardware	583
Appendix D S·P·A Diagnostic Playbook	593
Glossary	597
Index	601

Preface

Who this book is for

This textbook is for computer science and engineering students, machine learning researchers, robotics engineers, and embedded systems architects who want to build embodied intelligent systems that safely sense, reason, and act in the physical world.

An advisory model may end at a display or a human decision, with no direct path from its output to a physical actuator. Its computation may be retried or canceled before an external action, but an error can still cause harm if a person or another system acts on it. Idempotency describes the result of repeating an operation; it does not make digital consequences harmless or physical work reversible.

In physical AI, software commands matter, momentum, and energy. When a vision-language model commands a mobile base or an articulated robotic arm, execution crosses the causal boundary into continuous Newtonian physics. If a model inference stalls during an obstacle approach, the vehicle keeps moving; stronger mechanical brakes cannot recover distance already lost to computational delay. If an actuation command exceeds the yield strength of a gearbox or the thermal dissipation limit of an inverter, physical hardware breaks. Once kinetic energy is released or material yields, software cannot undo the state transition. You cannot roll back a physical collision.

Building systems that survive this reality requires a shared systems engineering discipline across three technical communities:

- **Machine Learning Practitioners:** Moving beyond static test-set accuracy evaluated on stored datasets. In physical AI, actions alter future sensory observations through closed-loop feedback ($s_{t+1} \sim P(s \mid s_t, a_t)$), creating compounding covariate drift ($\mathcal{O}(T^2\epsilon)$) that invalidates standard statistical assumptions. Practitioners learn how to formulate continuous action policies, evaluate closed-loop survival, and submit unprivileged proposals to real-time safety monitors.
- **Robotics and Control Engineers:** Augmenting classical feedback loops with high-capacity neural foundation models. Modern vision-language-action models provide open-vocabulary semantic scene understanding and multimodal generalization, but operate with non-deterministic latencies between 1 Hz and 10 Hz. Control engineers learn how to interface slow neural trajectory proposals with cycle-deterministic safety shields (such as Control Barrier Functions) executing at 1000 Hz on bare-metal silicon.
- **Embedded Systems and Silicon Architects:** Managing multi-rate execution across heterogeneous system-on-chip (SoC) platforms. Architects learn how to isolate high-throughput neural accelerators from real-time microcontroller cores, preventing memory bus crossbar contention, direct memory access (DMA) starvation, and supply voltage droop from breaching hard physical deadlines.

Why a physical AI systems textbook

This book approaches physical AI from a distinctive, first-principles systems perspective. To understand what this textbook provides, it is equally important to state what it is not:

- **Not a classical control theory book:** We do not focus on endless pages of continuous-time Laplace transforms, complex contour integrals, or abstract Lyapunov stability proofs. Formal continuous-control proofs and reachability derivations are quarantined in the volume appendices for optional depth.
- **Not a traditional robotics kinematics book:** We do not get bogged down in exhaustive Denavit-Hartenberg parameter matrices, forward-kinematics linkage catalogs, or specialized mechanical design heuristics.
- **A first-principles Physical AI Systems textbook:** We reason from bedrock physical laws (conservation of energy, momentum, kinematic stopping envelopes, thermal dissipation, and Coulomb friction) and computational systems architecture (multi-rate execution loops, memory bus contention, asynchronous message queues, and proposal-permission safety gates).

The mechanistic textbook principle

Because this is a comprehensive textbook, we do not treat modern machine learning models or control blocks as opaque black boxes. In every chapter, we teach students how the underlying mechanisms function from first principles:

- When we examine **Vision-Language-Action (VLA)** foundation models, we do not simply state that a robot uses a model. We teach how raw camera pixels are sliced into spatial patch tokens via vision transformers, how spatial representations are organized and cross-attended with natural language prompts to construct 3D affordance fields, and how the action decoder synthesizes motor commands—contrasting discrete token binning with continuous trajectory diffusion and action chunking.
- When we examine **safety enforcement**, we show the exact quadratic program projection (CBF-QP) executed by a real-time microcontroller at 1000 Hz to minimally filter unsafe proposals.
- When we examine **runtime memory and belief**, we teach how dynamic scene graphs, truncated signed distance function (TSDF) voxel grids, and SE(3) transform trees track spatial uncertainty and enforce exponential confidence decay under sensor occlusion.
- When we examine **inter-process communication**, we walk through single-producer single-consumer (SPSC) seqlock ring buffers, cache-line invalidations, and memory barriers that allow multi-rate processes to exchange state without blocking.

Every core abstraction is grounded in tangible diagrams, mathematical formulations, concrete data structures, and physical systems reasoning.

The fundamental physical AI invariant

At constant velocity v , the distance traveled by a machine while its onboard computing platform processes a sensor observation is the product of velocity and computation time:

$$d_{\text{displacement}} = v \cdot t_{\text{compute}}$$

A mobile robot traveling at 3.0 m/s travels 600 mm during a 200 ms neural inference pass. This travel occurs before mechanical brakes can build hydraulic pressure or electrical coils can build flux. Doubling model deliberation time doubles that unbraked displacement. A late computation leaves less physical room to respond, meaning operating speed, inference latency, available clearance, and braking capacity must be budgeted as an integrated systems problem.

Before an action reaches actuators, an independent safety monitor compares the proposal against physical limits. A command to accelerate toward an obstacle is rejected when it would leave insufficient stopping distance under the plant's dynamic model. Any protection these checks provide depends on accurate state observations, valid physical assumptions, and cycle-deterministic execution. These coupled dependencies are why this book treats learned behavior, mechanical dynamics, and computing silicon as one unified engineering discipline.

The architectural mental model: An embodied rosetta stone

To understand how physical AI systems are structured, table 0.1 maps the concepts of classical computing and digital machine learning to their physical AI counterparts. While the computational engines have evolved to embrace high-capacity neural networks, the fundamental systems challenges—resource contention, latency waterfalls, fault isolation, and interface contracts—remain invariant.

Table 0.1: The architectural rosetta stone: Mapping classical computing and digital machine learning concepts to their physical AI counterparts.

Classical Computing / Digital ML	Physical AI Systems Equivalent	Bedrock Physical Law / Governing Principle	Architectural Tier
Stateless Model Query	Closed Sensorimotor Trajectory	Endogenous Feedback Law ($s_{t+1} \sim P(s s_t, a_t)$)	Whole Machine
Software Exception / Retry	Kinetic Momentum & Thermal Yield	Irreversibility Invariant (no undo for dissipated energy)	Body (Layer 1)
Execution Latency Metric	Consumed Physical Clearance	Kinematic Stopping Envelope ($d_{\text{stop}} = v\tau + \frac{v^2}{2a}$)	Body & Nervous
Sudo / User Access Permissions	Proposal–Permission Boundary	Asymmetric Hardware Veto (deterministic MCU over neural MPU)	Nervous (Layer 2)
Memory Hierarchy (L1/L2/DRAM)	Multi-Rate Clock Hierarchy	Timescale Separation (1000 Hz Body, 50 Hz Nervous, 2 Hz Brain)	Whole Machine
Virtual Memory Paging	Action Chunking & Intent Leases	$\mathcal{C}^2$ Spline Seams & Epistemic Decay ($e(t) = e_0 + r\Delta t$)	Brain (Layer 3)
Hardware Protection Rings (Ring 0 vs 3)	Simplex CBF Safety Shields	Control Barrier Forward Invariance ($\dot{h} + \alpha(h) \geq 0$)	Nervous (Layer 2)
Cloud GPU Server	Heterogeneous Edge Silicon Die	Memory Bus QoS, Crossbar Contention, and PDN Voltage Droop	Silicon Substrate

The structural triad

The structural invariant is that learned proposals do not authorize their own actuation: an independent path permits or vetoes them, and the body executes permitted commands. One implementation of this triad uses the following rates and chip partition:

1. **The Brain (Deliberative Tier, $\approx 1\text{--}10\text{ Hz}$):** Executes high-capacity multimodal foundation models, spatial scene representations, and generative action decoders on application processors (MPU / GPU / NPU). It formulates timestamped, expiring candidate trajectory proposals across an unprivileged boundary.
2. **The Nervous System (Tactical & Safety Tier, $\approx 50\text{--}100\text{ Hz}$):** Executes deterministic state estimation, sensor-fusion time synchronization, lock-free inter-process communication, and Control Barrier Function (CBF-QP) safety shields on real-time microcontroller cores (MCU). It verifies dynamic stopping envelopes and holds independent veto authority over motor commands in this implementation.
3. **The Body (Reflexive & Plant Tier, $\approx 1000\text{ Hz}$):** Executes cycle-deterministic Field-Oriented Control (FOC) torque and current commutation on inverter gate drivers, driving power MOSFETs, measuring rotor encoders, and managing phase dissipation under classical Newtonian and thermodynamic mechanics.

The Brain proposes; the Nervous System permits and enforces; the Body actuates.

The S·P·A operational framework

Where disembodied machine learning systems structure performance diagnosis and optimization around the classic Data, Algorithm, and Machine (D·A·M) triad, embodied Physical AI crosses the causal boundary into irreversible physical dynamics. Operationally, this textbook organizes systems engineering around the **S·P·A framework: Sense, Plan, and Act** (section 2.6).

While Sense, Plan, and Act begin as distinct disciplines—perception and state estimation (Sense), high-capacity neural policies and trajectory generation (Plan), and real-time fieldbus motor actuation (Act)—the core of Physical AI systems engineering lives at their **pairwise intersections** and shared center:

- **Sense $\cap$ Plan ($S \cap P$): “What the Machine Believes”**—Grounding high-bandwidth sensory observations into actionable spatial representations (voxel unprojection, Age-of-Information bounds, 3D Gaussian scene buffers).
- **Plan $\cap$ Act ($P \cap A$): “How the Machine Moves”**—Translating learned action chunks into kinodynamically feasible motion ($\mathcal{C}^2$ quintic splines, precomputed stopping suffixes, receding-horizon buffers).
- **Act $\cap$ Sense ($A \cap S$): “How the Machine Responds”**—Closing sub-millisecond reflex loops directly against contact dynamics (kilohertz impedance control, tactile reflex triggers, Coulomb friction compliance).
- **Sense $\cap$ Plan $\cap$ Act ($S \cap P \cap A$): “Physical AI Systems Engineering”**—The causal boundary itself, where proposal-permission privilege partitioning, Control Barrier Functions (CBF-QP), and bumpless supervisory handovers guarantee provable runtime safety.

The complete diagnostic playbook, troubleshooting matrix, and systems scorecard for the S-P-A framework are formalized in Appendix D.

The three canonical machine archetypes

To ensure that systems concepts remain concrete, this textbook grounds every principle across three foundational physical machine archetypes introduced in early chapters:

1. **Class 1 · Mass-Dominated Mobility (AMRs & Quadrupeds)**: Planar transport bases and legged platforms (such as Boston Dynamics’ Spot) navigating open facilities and rugged terrain. Governed by vehicle momentum, tire-road and foot-ground Coulomb friction, kinematic stopping distance envelopes, and multi-camera / LiDAR perception pipelines under rolling-shutter skew.
2. **Class 2 · Articulated Manipulators**: Multi-link robotic arms performing dexterous contact tasks. Governed by non-linear arm dynamics, reflected rotor inertia ($J_{\text{ref}} = N^2 J_m$), cycloidal gearbox compliance, and contact force containment.
3. **Class 3 · Continuous Process & Energy Systems**: High-pressure fluid loops, chemical batch reactors, and thermal plants. Governed by enthalpy exchange, pipe fluid dynamics, pressure transients ($dP/dt = K \Delta \dot{m} / \rho V$), and irreversible thermodynamic phase changes.

(These three archetypes culminate in Class 4 · Integrative Capstone Systems, such as legged humanoids and mobile manipulators, which contend with mobility, manipulation, and power-dense thermal constraints simultaneously on a single power-constrained platform.)

How to read this book: Progressive arc vs. modular drop-in

This textbook is engineered to serve two distinct reading patterns:

For sequential readers (the cumulative curriculum arc)

Readers moving cover-to-cover follow a compounding, four-part curriculum arc where each chapter resolves the engineering vulnerability exposed by the preceding one:

- **Part I: The Machine Anatomy (Chapters 1–4)** establishes the physical container and structural triad. It begins at the causal boundary (chapter 2), explores the physical constraints of the body (chapter 4), analyzes the cognitive architecture of the learned brain (chapter 6), and synthesizes them through the deterministic nervous system (chapter 8).

- **Part II: Teaching the Machine (Chapters 5–7)** addresses policy learning under endogenous feedback. It examines physical data collection and multi-stream action logging (chapter 10), continuous policy training across imitation and diffusion (chapter 12), and closed-loop statistical evaluation metrics that supersede static test sets (chapter 14).
- **Part III: Running the Machine (Chapters 8–13)** constructs the live millisecond execution pipeline. It traces sensor photons to calibrated spatial claims (chapter 16), persistent volumetric memory under occlusion (chapter 18), semantic intent leases (chapter 20), continuous trajectory chunking (chapter 22), bare-metal safety shields (chapter 24), and heterogeneous silicon placement (chapter 26).
- **Part IV: Governing the Machine (Chapters 14–17)** treats accountability, verification, and survival in the wild. It covers shared human-machine authority handshakes (chapter 28), adversarial fault injection on hardware testbeds (chapter 30), auditable safety cases for production release (chapter 32), and the foundational epistemic limits of empirical testing (chapter 34).

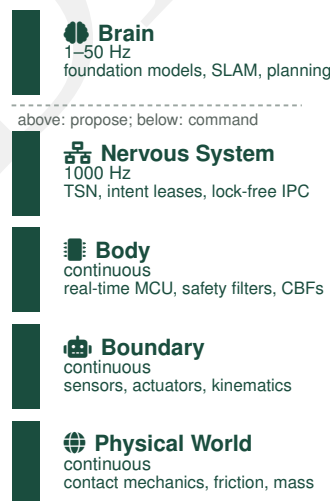
The volume concludes with a master synthesis linking all seventeen chapters into a single closed causal loop.

For drop-in readers and modular course instructors

Instructors and practitioners frequently need to study a specific topic in isolation—such as sensor DMA pipelines, trajectory chunking, or Control Barrier Functions. To support modular use without prerequisite confusion:

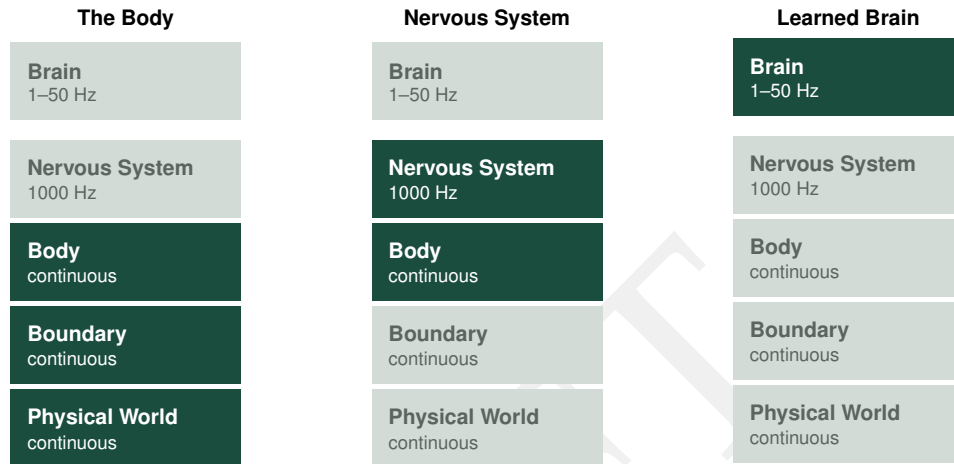
- **Architectural Landing Zones (§X.1):** Every chapter begins with an orientation section that frames the core problem, states its physical stakes, and places the chapter within the four-tier physical AI stack using a locator diagram.
- **Explicit Foundational Context:** Whenever a chapter draws upon a core concept established earlier (such as the proposal-permission split, the four-stream action tap tuple, or the three machine archetypes), the text provides an immediate inline orientation gloss and a precise section cross-reference.
- **Self-Contained Worked Examples:** Each chapter contains standalone analytical derivations, executable Python calculations, and industrial case studies that can be studied independently.

The following physical AI stack diagram appears throughout the volume, providing a recurring architectural compass that highlights the active tier addressed by each chapter:



The physical AI stack is defined by control rate and authority partitioning rather than by software dependency alone: from the continuous Newtonian dynamics of the Physical World and Boundary at the bottom, through the millisecond reflex execution of the Body, up to the deterministic Nervous

System (1000 Hz reflex and barrier enforcement), across the asymmetric proposal–permission boundary, to the high-capacity Learned Brain at the top (1–50 Hz foundation models, spatial memory, and intent planning). Three chapters illustrate how the architectural lens shifts across the volume:

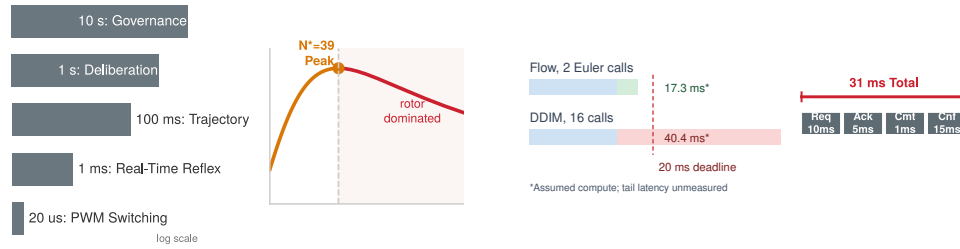


In chapter 4, the active intensity concentrates at the bottom tiers: the chapter is about electromechanical torque, thermal dissipation in motor windings, and battery power distribution networks. In chapter 8, the deterministic middle tier illuminates: the chapter establishes the lock-free inter-process communication, time-synchronized fieldbuses, and cycle-deterministic microcontrollers that protect the plant. In chapter 6, the top deliberative tier dominates, analyzing the vision-language-action foundation models, spatial token representations, and action chunking decoders that formulate candidate proposals.

Diagnostic margin reading aids

The margins carry small, unnumbered visual notes at the exact point in the narrative where an architectural relationship, physical scale gap, or safety threshold is easiest to comprehend by sight. These are not numbered figures to cite later; they are diagnostic reading aids:

- **Ladders** span orders of magnitude across physical regimes (such as execution frequencies from 20 kHz PWM to 0.1 Hz supervisory deliberation, or sensory ingestion bandwidths).
- **Threshold knees** identify physical cliff edges where continuous scaling turns destructive (such as reflected rotor inertia choking joint acceleration, contact stiffness force spikes, or memory bus contention tail latencies).
- **Budget envelopes** visualize competing cyber-physical constraints against hard physical dead-lines (such as generative diffusion denoising times versus control periods, or fieldbus cycle budgets).
- **Sequence strips** depict multi-phase timing handshakes, receding-horizon chunking windows, or cryptographic seal progressions.
- **Causal chains and blast radii** trace the propagation of failure modes, from unmodeled simulation artifacts to hardware destruction.



Sense–plan–act navigation and margin pointers

Alongside the physical AI stack, two diagnostic navigation devices guide the reader throughout the book:

- **S-P-A Triad Locators** (`spa_locator_*.svg`): At key section transitions, a glanceable three-node glyph identifies where the discussion sits across the classic Sense (S), Plan (P), and Act (A) triad, highlighting the active capability axis (such as S for perception, $P \cap A$ for trajectory generation, or the centered bullseye for integrative capstone chapters).
- **Margin Navigation Pointers**: Exactly six directional signposts appear in the margins of every chapter, weaving the curriculum into a navigable hyperlinked network:
 - Backward pointers (`**Prerequisite:**`) establish the upstream physical law, data structure, or hardware contract that the current section assumes.
 - Forward pointers (`**Downstream:**`) alert the reader to where a state estimate, uncertainty bound, or safety claim is consumed downstream in the pipeline.
 - Lateral contrast pointers (`**Contrast:**`) place opposing design choices side-by-side (such as discrete token binning versus continuous diffusion chunking, or dense TSDF voxels versus 3D Gaussian Splats).

The golden thread: An end-to-end forensic trace

To anchor abstract concepts across all seventeen chapters into a cohesive engineering reality, this book weaves a recurring real-world scenario—the **Golden Thread**—through the four tiers of the Physical AI Stack:

- **Part I · The Physical Challenge (The Hazard)**: An autonomous mobile robot (AMR) navigates an industrial aisle at $v = 3.0$ m/s. Without warning, a human-driven forklift crosses into the corridor 2.5 m ahead.
- **Part II · The Learning Gap (The Policy Failure)**: The onboard vision-language-action (VLA) policy, trained on nominal demonstrations without sudden cut-ins, encounters acute out-of-distribution covariate shift ($\mathcal{O}(T^2\epsilon)$). Instead of braking, the generative action chunk decoder proposes an aggressive, infeasible swerve that violates kinematic turning limits.
- **Part III · The Runtime Execution (The Silicon Bottleneck)**: Camera exposure (33 ms) + SerDes transport (8 ms) + neural inference (45 ms) consumes 86 ms of unguided motion (25.8 cm of blind travel). The proposal arrives at the shared-memory mailbox, where DRAM crossbar contention threatens the timing deadline.
- **Part IV · The Safety Shield (The Deterministic Intervention)**: Operating on an isolated real-time MCU, the 1000 Hz ASIL D safety filter evaluates the Control Barrier Function ($h(x) = d - d_{\min} \geq 0$). Calculating that the proposed swerve will strike the obstacle, the Quadratic Program minimally projects the action into a maximum deceleration clamp ($a = -4.0$ m/s²) within 1.0 ms, bringing the chassis to a safe halt 0.4 m from the obstacle without relying on software optimism.

Related books in the machine learning systems curriculum

This textbook is part of the *Machine Learning Systems* curriculum. *Introduction to Machine Learning Systems* examines how models use computation and memory on a single machine. *Scaling Machine Learning Systems* extends that engineering scope to coordinating distributed training and inference across cluster fleets.

The curriculum also addresses systems that take autonomous actions. *Agentic Machine Learning Systems* focuses on stateful goal-directed trajectories in digital software environments, while *Physical AI Systems* examines actions that produce force, motion, and thermodynamic change in the physical world. While each text offers a complementary perspective, this book is completely self-contained, developing the physical, computational, and architectural principles required to engineer dependable embodied machines.

Dedication

*For my students,
who ask the question
I have not thought of yet.*

Author's Note

WHEN A SERVER RETURNS AN ERROR FOR A PAGE REQUEST, the reader can try again. A robot approaching an obstacle keeps moving while its computer decides whether to brake. Restarting the computation cannot recover the distance already traveled.

In traditional computing, software operates behind glass: an unhandled exception or late result produces a log entry, a dropped frame, or a retry button. In Physical AI, software commands matter, momentum, and energy. The unit of engineering is the action crossing the causal boundary. Delay is paid in millimeters, and failure results in cracked composite, burned motor coils, or physical harm.

To build machines that survive this reality, we must abandon the illusion that a high-capacity neural network can directly actuate a physical plant. Instead, this book structures physical intelligence across an unyielding triad: a high-latency **Brain** (~1–10 Hz) that formulates unprivileged intent proposals; a deterministic **Nervous System** (~50–100 Hz) that estimates physical state and enforces safety barriers; and a physical **Body** (~1 kHz) governed by inertia, torque limits, and thermal saturation. The Brain proposes; the Nervous System and safety interlocks command.

I write these chapters as an active systems researcher exploring how the laws of computing intersect with the laws of Newtonian mechanics, thermal dissipation, and functional safety. Concrete systems tasks, hardware measurements, and first-principles napkin math guide our progression; formal continuous-control derivations and proof machinery provide optional depth in the appendices.

This book is an open, working notebook. Step across the causal boundary with me, and let us explore how to engineer machines that sense, act, and survive in the physical world.

— Vijay Janapa Reddi
Cambridge, Massachusetts
2026

About the Author

Vijay Janapa Reddi is the Gordon McKay Professor of Electrical Engineering and Computer Engineering at Harvard University and a Visiting Professor at ETH Zurich in the Integrated Systems Laboratory (IIS), D-ITET. He is a founding member and Vice President of MLCommons and the author of the open-source reference text *Machine Learning Systems*.

His research sits at the intersection of computer architecture, machine learning systems, and edge robotics, focusing on how computing hardware and learned software co-design to interact with the physical world. Over the past decade, his work has concentrated on democratizing applied systems through open-source education and reproducible benchmarks, connecting first-principles silicon constraints directly to accessible, bench-verified learning.

Why I Am Writing This Book: The First-Principles Systems Stance

Every textbook reflects the intellectual lineage, biases, and lens of its author. I want to make the viewpoint behind this book explicit.

I am not a classical control theorist, nor a mechanical kinematics specialist, and I do not pretend to be one. I bring the perspective of a computer systems architect and machine learning systems (MLSys) researcher going after first principles.

My intuition for this domain dates back to my undergraduate mechatronics capstone. As an electrical and computer engineering student, that capstone was the first time I had to integrate everything—analogue circuits, sensor conditioning, microcontrollers, motor drivers, mechanical linkages, and embedded software—into a single working machine. It was the moment I realized that software cannot be designed in isolation from physical reality: motor coils heat up, power rails droop under sudden acceleration, and sensor measurements age the instant physical transduction occurs.

For over twenty years—from mobile System-on-Chip performance architecture and edge AI silicon at Google, to co-founding MLCommons/MLPerf to establish empirical benchmarking standards for AI accelerators, to pioneering the TinyML movement to compress neural networks onto milliwatt microcontrollers—my work has lived at the physical boundary where software bits meet physical silicon and energy constraints.

Physical AI brings me full circle back to that mechatronics capstone, but with a profound twist. In classical mechatronics, we closed the loop using hand-crafted, deterministic logic: PID controllers, state machines, and analytical models. In Physical AI, we are embedding high-capacity, stochastic neural networks—foundation models, diffusion policies, and learned decoders—into that exact same unforgiving loop. The engineering challenge is no longer just making the machine move; it is figuring out how to safely close the loop around an uncalibrated, stochastic brain using deterministic systems and safety principles.

That background shapes what I want students to understand. A model's proposed action must pass through a computing system before the physical machine can carry it out. I want readers to trace that path and ask what could prevent an action from being carried out as intended. The timing of computation, the body's capabilities, and the assumptions behind command checks all matter to that assessment. The explanations should make these interactions understandable before specialized mathematics or implementation detail is needed.

This textbook is my attempt to share that foundational systems perspective with the next generation of engineers, roboticists, and computer scientists. It is the book I wish my own students had when bridging machine learning systems to the physical world.

Intellectual Lineage and Systems Trajectory

This first-principles systems trajectory spans three major eras:

- **Silicon Reality & Mobile Architecture:** Grounded in computer architecture, low-power microarchitectures, and mobile System-on-Chip (SoC) design (including edge AI silicon performance architecture at Google). This work addressed the physical realities of computing: memory bus contention under high-bandwidth camera streams, thermal throttling under strict dissipated wattage envelopes, and interrupt latencies under hard real-time deadlines.
- **ML Systems & The TinyML Movement:** Bridging algorithmic machine learning abstractions to silicon execution. He co-founded MLPerf to establish the global standard for benchmarking ML hardware and software, and pioneered the TinyML movement, developing the Harvard CS249r curriculum and the HarvardX TinyML program (reaching over 100,000 learners globally) to bring deep neural models onto milliwatt microcontrollers.
- **Closing the Sense-and-Act Loop in Physical AI:** Extending this systems perspective to machines whose learned models propose physical actions. The focus is on how sensing, computation, and the body's response interact, and on the conditions under which a proposed action can be executed.

The Kit and Laboratory Development

Development of the hands-on bench laboratories and the reference hardware platform (the Physical AI Kit on Arduino UNO Q Dual-Brain) is a collaboration with Dr. Andrea Mattia Garavagno, postdoctoral researcher in the Integrated Systems Laboratory (IIS), D-ITET at ETH Zurich.

This work aims to translate the systems ideas in the book into firmware and software exercises that students and practitioners can reproduce and examine on the bench.

Reader Guide and Prerequisites

Physical AI Systems is written as an independent, self-contained textbook for advanced undergraduate students, graduate researchers, and practicing systems engineers who wish to build machines that safely couple learned intelligence to physical matter and energy.

Readers will bring different strengths from machine learning, embedded systems, or mechanics. The book introduces the concepts needed to reason about their interactions without requiring specialist training in every contributing discipline. This guide distinguishes the shared starting background from material introduced in the text and suggests where readers with different backgrounds can concentrate their attention.

The Technical Baseline

The main text assumes a shared foundation in programming, machine learning, and basic mathematics. The following preparation supports reading the explanations and following the short calculations:

- **Programming & Systems:** Readers should be able to read and write Python functions, follow data through a program, and recognize that execution takes time and uses memory. Familiarity with C or C++, pointers, and memory allocation is useful for optional implementation work.
- **Machine Learning Intuition:** Readers should recognize the difference between training a model and using it to make predictions, and be familiar with arrays or tensors as model inputs and outputs. Experience with a particular framework is unnecessary. A model's prediction can be repeatable and still be wrong; repeatability alone does not establish dependable physical behavior.
- **Foundational Physics & Mathematics:** Readers should be comfortable with basic algebra, vectors and matrices, and single-variable calculus (rates of change). Physical AI relies directly on freshman Newtonian mechanics:
 - Newton's second law for linear and rotational systems ($F = ma, \tau = I\alpha$).
 - Kinetic energy and stopping distance napkin math ($E_k = \frac{1}{2}mv^2, d_{\text{stop}} \approx v \cdot t + \frac{v^2}{2a}$).
 - Basic electrical power and heating ($P = I_{\text{phase}}^2 R_{\text{phase}}$). Advanced differential geometry, Lie group continuous control theory, and formal reachability analysis are deliberately not required; their mathematical treatments are quarantined in the volume appendices.

This preparation supports reasoning about the complete system. Specialized control theory, detailed robotics mathematics, and low-level implementation expertise are not starting requirements for the main explanation.

Disciplinary Reading Pathways

Readers approach Physical AI from different engineering traditions. The published sequence develops a shared systems perspective; readers can give additional attention to the interactions least familiar from their own background:

- **For Machine Learning Practitioners:** Focus on the physical consequences of computation. A machine continues moving while its model processes an observation. Pay close attention to chapter 4 for motion and braking constraints, chapter 10 and chapter 12 for endogenous

feedback and compounding drift ($\mathcal{O}(T^2\epsilon)$), chapter 14 for closed-loop reality, chapter 24 for checking proposed actions before execution, and chapter 26 for computing resources that affect timing.

- **For Embedded Systems & Silicon Architects:** Focus on how learned computation contributes to decisions that the machine must carry out on time. Pay close attention to chapter 2 for the proposal-permission boundary, chapter 16 for how sensory observations inform decisions, chapter 18 for spatial belief representations, chapter 20 and chapter 22 for trajectory synthesis, and chapter 26 for system-on-chip (SoC) memory bus contention and thermal limits.
- **For Roboticians & Control Engineers:** Focus on closing the physical loop around learned models. Pay close attention to chapter 2 for limits on what a model may command, chapter 8 for timing and communication between components, chapter 28 for human authority handshakes and bumpless transfer, chapter 30 for hardware-in-the-loop fault injection, and chapter 32 for the structured evidence needed to judge readiness for release.

These points of emphasis supplement a sequential reading. Familiarity with one discipline does not establish how the book connects it to the rest of the machine.

What the Book Introduces

The book starts from the physical and computational relationships needed to explain a system's behavior. Specialized concepts enter through the problems they help solve, with their assumptions and limits made explicit. The main text develops the following ideas from that shared foundation:

- **Proposals and permission to act:** The text explains how a learned model can propose an action while a separate component determines whether that action may reach the body.
- **Checks under physical limits:** The explanation connects proposed actions to available space and machine motion, stating the assumptions under which a check can help.
- **Timing across components:** The text develops how components that operate at different rates exchange results and coordinate their work, and why delays matter to the physical response.
- **Observations and motion:** The explanation relates positions and directions described by sensors and models to movements of the body, including how sudden changes in commanded motion affect the mechanical response.
- **Evidence for release:** The text explains how to connect a claim about system behavior to supporting evidence and identify what that evidence leaves unresolved.

Detailed proofs, specialized notation, and implementation layouts provide optional depth. The main explanation retains the mechanism, assumptions, and engineering consequence so that following the systems argument does not depend on pursuing that detail.

What We Deliberately Omit

The book focuses on how learned computation, physical mechanisms, and execution systems work together. That focus limits the depth of its treatment in neighboring fields:

- **Exhaustive Robotic Kinematic Derivations:** The main text does not develop long derivations for every manipulator configuration. The relevant systems concern is what movements a machine can make and which physical limits constrain them.
- **Deep Learning Theory:** The book does not provide a comprehensive mathematical treatment of loss landscapes and optimization. Learning mechanisms matter here through the behavior they produce and the demands they place on the physical system.
- **Framework Tutorials:** The book does not provide tutorials for particular prompt-chaining libraries or conversational web scrapers. Software and computing placement matter through their effects on timing, communication, and physical behavior.

These boundaries leave the essential mechanisms and their consequences in the systems argument. They limit exhaustive specialist treatment without making familiarity with that treatment a prerequisite for following the book.

Acknowledgments

PLACEHOLDER. This book's acknowledgments are still to be written, recording the people whose expertise, interviews, and industry collaboration shaped its development.

A Note on AI Assistance

Every chapter of this book was researched, written, and rewritten by me. I also used AI tools throughout. What follows is more than the disclosure MIT Press requires of authors who use AI. It is an honest statement of how I used these tools, a model for how students and practitioners can engage with them responsibly, and a claim about what authorship means when machine learning acts as both force multiplier and sounding board.

In my previous volumes, systems and computing infrastructure was my native intellectual home. But *Physical AI* pushed me across the causal boundary into unfamiliar terrain: classical kinematics, nonlinear control barrier functions, motor winding thermodynamics, and high-frequency real-time bus contention. The most difficult challenge in writing this book was not drafting prose; it was conducting an exhaustive cross-disciplinary literature survey and synthesizing fields that rarely speak to one another. I used large language models extensively as interactive research partners to survey disparate literature, explore unfamiliar dynamics, brainstorm structural framings, and distill foundational first principles. These models excel at first-principles distillation—which is precisely how I naturally think about computer systems.

Yet distillation is only an initial proposal. Because these concepts live in the physical world where errors carry kinetic consequences, I relied heavily on human experts—both academic researchers in robotics and control theory, and practicing engineers at autonomous systems companies—to critique, validate, and challenge the architectures developed here. This book does not claim to be a final, closed work; it is an active, open iteration that evolves alongside a rapidly developing discipline.

The standard remains uncompromising traceability. Every derived metric, timing budget, and energy calculation in these chapters is computed directly by executable Python cells whose source code ships with the book. Quantitative system simulations are backed by the MLSysIM infrastructure modeling engine. A continuous pre-commit suite verifies dimensional units, citation integrity, and reference anchors on every change. AI tools made navigating a steep cross-disciplinary learning curve mechanically tractable; our verification infrastructure makes that work reproducible and auditable.

Authorship is thinking. Deciding which concepts belong in this curriculum and which do not; sequencing the chapters so that each abstraction earns its keep; determining where students will get stuck because I have watched them get stuck in my own classroom—these are judgment calls that no tool can make. The ideas and architectural synthesis are mine; the initial prose was often drafted, expanded, or restructured by AI before being aggressively rewritten and verified.

Machine learning systems cannot be certified the way traditional deterministic logic can. That verification gap applies to AI-assisted authoring with equal force. The standard I held this book to is the standard we must demand of every embodied machine we deploy: the tool can propose, and the tool can tutor, but only a human engineer can ratify and take responsibility.

Notation

Machine Learning Systems spans *machine learning* (computer science and statistics) and *systems* (computer architecture and hardware). Each field developed its notation independently, and many symbols mean different things across communities and publications. This collision creates real confusion when the disciplines merge. The conventions below establish a single notation that eliminates this ambiguity.

Consider a simple statement: “Increasing B improves throughput.” To an ML researcher, B means batch size. To a hardware engineer, B means bandwidth. Both interpretations are correct in their respective fields, but in ML Systems we need both concepts in the same equation—hence the need for a single, consistent convention.

The Iron Law of ML Systems

For serialized execution phases, the iron-law performance decomposition is:

$$T = \underbrace{\frac{D_{\text{vol}}}{\text{BW}}}_{\text{Memory Time}} + \underbrace{\frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}}}_{\text{Compute Time}} + \underbrace{L_{\text{lat}}}_{\text{Latency Overhead}}$$

Each variable was chosen deliberately to avoid collision with standard ML terminology.

Symbol	Definition	Unit	Why This Symbol?
T	Time	seconds	Unambiguous. Wall-clock time for an operation.
D_{vol}	Data Volume	bytes	Avoids collision with D (Dataset Size). In scaling laws, D means training tokens. Here we need bytes moved through memory. The subscript disambiguates.
BW	Bandwidth	bytes/s	Avoids collision with B (Batch Size). Systems literature often uses B for bandwidth, while ML literature often uses B for batch size. The notation preserves the ML convention.
O	Operations	FLOPs	Total floating-point operations. Clean in equations (vs. “Ops”).
R_{peak}	Peak Rate	FLOP/s	Avoids collision with P (Parameters). Roofline presentations often use P for performance, while ML literature commonly uses P for parameter count. The notation preserves the ML convention.
η_{hw}	Efficiency	—	Hardware utilization ($0 \leq \eta_{\text{hw}} \leq 1$). Avoids collision with learning rate (η).
L_{lat}	Latency	seconds	Avoids collision with $\mathcal{L}$ (Loss). Fixed overhead time (kernel launch, network RTT). The subscript distinguishes from the loss function.

Why these choices matter

Without careful notation, sentences become ambiguous:

“Reducing D improves performance.”

The sentence has two possible readings:

- Reducing **dataset size** (fewer training samples) can speed training but may reduce accuracy.
- Reducing **data volume moved** (for example, through compression or quantization) can speed inference, with accuracy effects that depend on the technique and calibration.

With our notation, we can write precisely:

“Reducing D_{vol} through FP32-to-INT8 quantization cuts parameter memory traffic to one quarter while D (training data) remains unchanged.”

Our notation eliminates this ambiguity: “*BW limits throughput*” has only one reading.

Subscripted variants

An upright root names the quantity, and the subscript names which instance of it is meant. The convention holds closely related measurements apart instead of collapsing them into a single symbol:

- $BW_{\text{disk}}, BW_{\text{network}}, BW_{\text{accelerator}}$: bandwidth at three points on the data path
- $D_{\text{vol}}, R_{\text{peak}}, L_{\text{lat}}, \eta_{\text{hw}}$: iron-law terms, each held clear of a common ML symbol
- $E_{\text{move}}, E_{\text{compute}}, E_{\text{total}}$: one energy budget split into tradeable parts

The subscript therefore carries part of the claim. Disk and accelerator bandwidth differ by orders of magnitude, so a figure is reproducible only when the subscript says where it was measured.

The Degradation Equation

The degradation equation is a fitted local model. Divergence alone does not determine the direction of accuracy change, so estimate λ from labeled outcomes. Some symbols below, such as τ , appear in chapter prose rather than in the equation.

$$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0)$$

Symbol	Definition	Unit/Type	Notes
$\text{Accuracy}(t)$	Accuracy at Time t	Scalar	Model accuracy after the model has been deployed for time t .
Accuracy_0	Initial Accuracy	Scalar	Model accuracy at deployment time.
λ	Fitted Sensitivity	Scalar	Local coefficient estimated from outcomes over a stated range; not a universal constant. (<i>Not wavelength.</i>)
P_t	Current Distribution	Distribution	The data distribution at time t . (<i>Not parameters—use P for parameter count.</i>)
P_0	Training Distribution	Distribution	The data distribution at training time.
$\mathcal{D}(P_t \ P_0)$	Statistical Divergence	Scalar ≥ 0	Measures how far P_t has drifted from P_0 . Common choices: KL divergence, total variation, Wasserstein. (<i>Calligraphic to avoid collision with D = dataset size.</i>)
τ	Response Threshold	Scalar > 0	Operational threshold for investigation; it does not automatically trigger retraining.

The Energy Corollary

With effective per-byte and per-operation costs, workload energy is approximated as:

$$E_{\text{total}} \approx D_{\text{vol}} \times E_{\text{move}} + O \times E_{\text{compute}}$$

Symbol	Definition	Unit	Notes
E_{total}	Total Energy	joules	Total energy consumed by an ML workload, decomposed into data-movement and compute terms.
E_{move}	Energy per Byte Moved	joules/byte	Effective movement cost; total energy also depends on bytes moved and operations executed.
E_{compute}	Energy per Operation	joules/FLOP	Energy cost of a single arithmetic operation.

Deep Learning Notation

This book follows standard deep learning conventions (Goodfellow et al. 2016) with explicit disambiguation for systems variables.

Symbol	Definition	Dimensions/Type
B	Batch Size	Integer. The number of samples processed in parallel. (<i>Never bandwidth.</i>)
P	Parameters	Integer. The total count of scalar model parameters, including weights and biases. (<i>Never peak FLOP/s.</i>)
D	Dataset Size	Integer. Number of training samples or tokens. (<i>Never data volume in bytes—use D_{vol}.</i>)
S	Sequence Length	Integer. Number of tokens or time steps.
d	Hidden Dimension	Integer. Size of the hidden state vector.
d_{head}	Attention Head Dimension	Integer. Per-head hidden dimension in attention layers.
N_L	Number of Layers	Integer. Total number of layers in a network.
N_{heads}	Number of Attention Heads	Integer. Number of attention heads in a multi-head attention layer.
H_{KV}	Number of Key-Value Heads	Integer. Number of key-value heads in grouped-query or multi-query attention.
ℓ	Layer Index	Integer. Index for a layer. Use instead of bare L when indexing layers, since L collides with loss and latency.
$\mathcal{L}$	Loss Function	Scalar. The objective function minimized during training.
η	Learning Rate	Scalar. Step size for the optimizer. (<i>Never bare for hardware efficiency—use η_{hw}.</i>)
θ	Model Parameters	Parameter collection, often treated as a vector. The set of all learnable parameters.
$p(x)$	Distribution	Probability mass/density for a random variable. Use lowercase $p(\cdot)$ for generic distributions to avoid collision with P (parameter count).
$p(y x)$	Conditional Distribution	Conditional probability/density. Use this form for generic label relationships; reserve P_0 and P_t for the degradation equation’s training/current distributions.
$\Pr(E)$	Event Probability	Probability of an event E . Use for event statements such as $\Pr(\text{batch} = 0)$; use $p(x)$ and $p(y x)$ for distributions.

Latin matrices and vectors are set in bold ($\mathbf{W}$, $\mathbf{x}$); scalars, dimensions, indices, and individual matrix entries stay italic (N , d , i , W_{ij}). The generic parameter symbol θ is the one conventional exception and stays italic. Local matrix algebra follows the same bold rule: matrix operands are bold ($\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$), as in general matrix multiply (GEMM) $\mathbf{C} = \alpha\mathbf{AB} + \beta\mathbf{C}$. The bold form marks a matrix operand and is distinct from the italic scalar batch size B , which remains batch size in scalar contexts.

Performance, Serving, and Memory Notation

Reusable performance and serving quantities follow the same collision-avoidance rule as the iron law. Rates use R or descriptive Greek symbols; request counts use Q_{req} rather than overloading N ; queue utilization uses a subscripted ρ so bare ρ remains available for other ratio models.

Symbol	Definition	Unit/Type	Notes
I	Arithmetic Intensity	FLOP/byte	Workload FLOPs per byte moved. The Roofline Model uses I as the independent variable.
I_{ridge}	Roofline Ridge Point	FLOP/byte	$I_{\text{ridge}} = R_{\text{peak}}/\text{BW}$. Prefer this explicit form over starred shorthand so the meaning remains clear in prose.
R_{attain}	Attainable Compute Rate	FLOP/s	Roofline bound: $R_{\text{attain}} \leq \min(R_{\text{peak}}, I \times \text{BW})$. Uses R , not T , because this quantity is a rate.
MFU	Model FLOPs Utilization	Dimensionless	Useful model FLOP/s divided by available peak FLOP/s. Text acronym avoids overloading η .
r_{comp}	Compression Ratio	Dimensionless	Uncompressed size divided by compressed size. A compressed payload has $1/r_{\text{comp}}$ of the uncompressed size; the subscript avoids bare C collisions.

Symbol	Definition	Unit/Type	Notes
Q_{req}	Request Concurrency	requests	Average in-flight requests in a stable serving system ($Q_{\text{req}} = \lambda_{\text{arr}} \cdot T_{\text{lat}}$ via Little's Law). Avoids collision with N as device count in distributed settings.
λ_{arr}	Arrival Rate	requests/s	Request arrival rate. The subscript avoids collision with λ as degradation sensitivity or failure rate.
T_{lat}	Request Time in System	seconds	End-to-end queueing/serving latency for Little's Law. Distinct from L_{lat} , the fixed-latency term in the iron law.
$T_{\text{svc}}(B)$	Batch Service Time	seconds	Time to serve a batch of size B .
$\mu_{\text{eff}}(B)$	Effective Service Rate	requests/s	Batched service rate, typically $\mu_{\text{eff}}(B) = B/T_{\text{svc}}(B)$.
ρ_{serv}	Serving Utilization	Dimensionless	Queue/server utilization. Use instead of bare ρ , which is reserved for communication-computation ratio in distributed contexts.
M_{total}	Total Memory Footprint	bytes	Sum of explicit memory components; avoids bare M ambiguity.
M_{weights}	Weight Memory	bytes	Memory occupied by model parameters.
$M_{\text{gradients}}$	Gradient Memory	bytes	Memory occupied by stored gradients.
$M_{\text{optimizer}}$	Optimizer-State Memory	bytes	Momentum, variance, master weights, and related optimizer buffers.
$M_{\text{activations}}$	Activation Memory	bytes	Retained activations for the backward pass or serving intermediates.
s_{elem}	Element Storage Size	bytes/element	Bytes per stored tensor element.

Units and Precision

- **Physical units:** This book uses SI (metric) units throughout, including meters, kilograms, seconds, watts, and °C, consistent with standard engineering and scientific practice. Where source data was originally reported in imperial units, the book converts to SI and notes the original values parenthetically. A space always separates the number from the unit (for example, 100 ms, 2 TB/s).
- **Data and memory:** This book uses **decimal SI prefixes only**: KB = 10^3 bytes, MB = 10^6 , GB = 10^9 , TB = 10^{12} . Binary-prefixed units do not appear in prose; all capacities, throughputs, and model sizes are reported in decimal units (for example, 80 GB, 2 TB/s, 102 MB).
- **Compute:** The notation distinguishes operation counts from rates. Total work uses FLOPs (for example, GFLOPs, TFLOPs), while throughput uses FLOP/s with decimal prefixes (for example, GFLOP/s, TFLOP/s).
 - 1 TFLOP = 10^{12} FLOPs
 - 1 TFLOP/s = 10^{12} FLOPs per second
 - Arithmetic intensity conventionally uses FLOP/byte as a unit ratio (floating-point operations per byte moved). This is a ratio unit, not a total-work symbol or throughput symbol.
- **Currency:** Dollar amounts use the dollar sign (\$); unless otherwise noted, dollar-denominated costs are U.S. dollars (USD).
- **Precision:**
 - **FP64:** Double precision (8 bytes)
 - **FP32:** Single precision (4 bytes)
 - **TF32:** TensorFloat-32 (19-bit Tensor Core compute mode; inputs and storage remain FP32)
 - **FP16:** Half precision (2 bytes, standard range)
 - **BF16:** Brain float (2 bytes, wide dynamic range)
 - **FP8:** Quarter precision (1 byte, E4M3 or E5M2 format)
 - **FP4:** 4-bit floating-point format
 - **INT8:** 8-bit integer (1 byte)
 - **INT4:** 4-bit integer; lower integer precisions follow the same uppercase INTn pattern (for example, INT3, INT2)

Quick Reference: Resolving Collisions

Common collision points in ML Systems literature include:

Symbol	ML Meaning	Systems Meaning	Book Convention
B	Batch Size	Bandwidth	Batch Size. Use BW for bandwidth.
P	Parameters	Peak FLOP/s	Parameters. Use R_{peak} for peak rate.
D	Dataset Size	Data Volume	Dataset Size. Use D_{vol} for bytes moved.
L	Loss	Latency	Loss ($\mathcal{L}$). Use L_{lat} for latency.
η	Learning Rate	Efficiency	Learning Rate. Use η_{hw} for efficiency.

As a general principle, ML conventions take precedence for single letters, while systems concepts get subscripts or multi-letter symbols. This reflects the primary audience (ML practitioners learning systems) and preserves compatibility with the vast ML literature.

Physical AI and Embodied Systems Notation

Physical AI and embodied machine learning systems merge three distinct technical traditions:

1. **Machine Learning and Deep Learning** (statistical optimization, transformers, diffusion policies, imitation learning, autoregression, Action Chunking with Transformers).
2. **Robotics, Mechanics, and Classical Control** (multibody dynamics, $SE(3)$ Lie groups, rigid-body kinematics, Lyapunov stability, Control Barrier Functions, contact mechanics, actuator drive physics).
3. **Embedded Systems and Real-Time Architectures** (system-on-chip interconnects, cache and memory bus contention, hardware interrupt latency, seqlocks, CAN-FD/EtherCAT, intent leases, fail-safe interlocks).

Because each discipline developed its mathematical notation independently over decades, combining them creates severe symbol collisions—the “traditional warts” of cyber-physical engineering. For example, writing $\nabla_{\theta}\pi_{\theta}(\mathbf{a} \mid \mathbf{q})$ when θ also denotes joint angles, or writing $\mathbf{a}_t$ for an action when $\mathbf{a}$ denotes Cartesian acceleration or barrier constraints, confuses spatial mechanics with parameter optimization and control execution authority.

Below, we formalize the cross-discipline disambiguation contract, establish the governing cyber-physical laws of embodied systems, and tabulate the canonical symbols used throughout this book.

Cross-Discipline Disambiguation and Traditional Warts

When physical plant dynamics meet deep neural networks and real-time embedded hardware, common mathematical shorthand breaks down. The conventions below resolve these cross-domain collisions systematically:

Symbol	Canonical Convention	Colliding Meaning	Disambiguation Rule
θ	Neural policy network weights π_{θ}	Joint angle θ_i , trajectory polynomial $\theta(t)$, thermal resistance θ_{JA} , or heading θ	Reserve θ strictly for neural parameters. Robot joint configurations are always generalized coordinates $\mathbf{q} \in \mathbb{R}^n$ and q_i . Trajectory polynomials are $q(t) = \sum c_k t^k$ (never $\theta(t)$). Thermal resistance is R_{th} or $R_{\text{th},JA}$ (in K/W). Heading or spatial angles use ϕ, ψ or rotation matrices.
$\mathbf{a}$ vs. $\hat{\mathbf{u}}$ vs. $\mathbf{u}^*$ vs. τ	Four-tier causal authority chain ($\mathbf{a}_t \rightarrow \hat{\mathbf{u}}_t \rightarrow \mathbf{u}_t^* \rightarrow \tau_t$)	Generic “action” symbol colliding with acceleration a_{max} or barrier normal $\mathbf{a}(\mathbf{x})$	Disambiguate execution authority: $\mathbf{a}_t \in \mathcal{A}$ is the unprivileged policy action (dimensionless/normalized); $\hat{\mathbf{u}}_t \in \mathcal{U}$ is the proposed plant control setpoint; $\mathbf{u}_t^* \in \mathcal{U}$ is the certified, CBF-filtered control command; $\tau_t \in \mathbb{R}^n$ is realized physical actuator torque. Linear deceleration is a_{max} (m/s ²); barrier constraints use $\mathbf{A}_{\text{cbf}}(\mathbf{x})\mathbf{u} \leq \mathbf{b}_{\text{cbf}}(\mathbf{x})$.

Symbol	Canonical Convention	Colliding Meaning	Disambiguation Rule
τ vs. τ	Actuator torque vector $\tau \in \mathbb{R}^n$ ($N \cdot m$ or N)	Time constants (τ_{th}), latencies (τ_{ZOH} , τ_{lag}), intent lease duration (τ), or response threshold (τ)	Realized torque is always bold vector $\tau \in \mathbb{R}^n$ or scalar joint torque τ_i / component torque (τ_{motor} , τ_{hold}). Time intervals and latencies use Latin T or lowercase t (T_{lease} , t_{age} , t_{ZOH} , T_{blend}). Thermal time constants use explicitly subscripted $\tau_{th} = R_{th}C_{th}$. Operational thresholds use τ_{resp} .
$\mathbf{x}$ vs. s vs. $\mathbf{p}$ vs. $\mathbf{x}_p$	Mechanical plant state $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T \in \mathbb{R}^{2n}$	World state $s \in \mathcal{S}$, 3D position $\mathbf{p} = [x, y, z]^T$, or ViT image patch $\mathbf{x}_p$	Continuous mechanical plant state is bold vector $\mathbf{x} \in \mathbb{R}^{2n}$. Ground-truth latent world state is $s \in \mathcal{S}$. 3D Cartesian position is vector $\mathbf{p} \in \mathbb{R}^3$ or camera-frame point $\mathbf{P}_c$. Vision transformer patch embeddings use explicitly subscripted $\mathbf{x}_p$ or latent feature $\mathbf{z}$.
P	Model parameter count (P or P_{params})	Power (P_{elec} , P_{cont}), Riccati error covariance $\mathbf{P}(t)$, failure rate p_{fail} , or 3D point $\mathbf{p}$	Use P for parameter count. Electrical/mechanical power is P_{elec} , P_{cont} , P_{peak} (in Watts); continuous Riccati/Kalman error covariance is bold matrix $\mathbf{P}(t)$; statistical failure probability or rate is lowercase italic p (p_{target} , p_{emp} , p_{fail}); 3D Cartesian position is bold lowercase $\mathbf{p} = [x, y, z]^T$.
B	Batch size (B or B_{tokens})	Bandwidth (BW), Body frame ($\mathcal{F}_B$), bus burst size (B_k), or input matrix $\mathbf{B}$	Batch size is B ; sustained bandwidth is BW (per Iron Law); coordinate reference frames use calligraphic script ($\mathcal{F}_W$, $\mathcal{F}_B$, $\mathcal{F}_E$); bus transaction sizes use D_{trans} , k or B_k ; state-space control input matrix is bold capital $\mathbf{B} \in \mathbb{R}^{n \times m}$.
D	Dataset size (D in tokens or trajectories)	Data volume (D_{vol}), clearance distance (d_{clear}), duty cycle (D_{cycle}), TSDF field $D_t(\mathbf{v})$, or damping matrix $\mathbf{D}_{virt}$	Training dataset size is D ; bytes moved across buses is D_{vol} ; physical stopping/clearance distances use lowercase d (d_{stop} , d_{clear} , d_{gap}); actuator duty cycle uses subscripted $D_{cycle} \in [0, 1]$; voxel TSDF fields use $D_t(\mathbf{v})$; virtual damping matrices use bold $\mathbf{D}_{virt}$.
α	Bumpless transfer blending function $\alpha(t) \in [0, 1]$	CBF decay (γ_{cbf}), significance (α_{sig}), angular acceleration (α), copper tempco (α_{Cu}), or noise schedule ($\bar{\alpha}_k$)	Authority transfer uses scalar function $\alpha(t) \in [0, 1]$; barrier linear decay uses $\gamma_{cbf} > 0$; statistical confidence significance uses $\alpha_{sig} = 1 - CL$; angular acceleration is bold vector $\alpha \in \mathbb{R}^3$ (rad/s^2); copper resistance tempco is α_{Cu} ; diffusion noise schedules use indexed $\bar{\alpha}_k$.
$\mathbf{C}$	Coriolis matrix $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^{n \times n}$	Safe set $\mathcal{C}$, state-space output matrix $\mathbf{C}$, capacitance (C_{th} , C_{bus}), CoM $\mathbf{c}$, or shield coverage c_{shield}	Coriolis/centrifugal forces use matrix function $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$; forward-invariant safe sets use calligraphic $\mathcal{C} \subset \mathcal{S}$; state-space output mappings use bold $\mathbf{C} \in \mathbb{R}^{p \times n}$; electrical/thermal capacitance uses italic C ; center of mass is bold vector $\mathbf{c} \in \mathbb{R}^3$; safety shield diagnostic coverage uses $c_{shield} \in [0, 1]$.
$\mathbf{K}$	Camera intrinsic calibration matrix $\mathbf{K} \in \mathbb{R}^{3 \times 3}$	Contact stiffness ($k_{contact}$), control gains (K_p , K_d), motor constants (K_t , K_e), ensembling window K , or diffusion step k	Camera intrinsics use bold matrix $\mathbf{K}$; contact stiffness is lowercase scalar k or $k_{contact}$ (N/m); feedback control gains use capitalized scalars K_p , K_d ; motor torque/back-EMF constants use K_t , K_e ; temporal ensembling window length uses integer K ; diffusion noise steps use lowercase index k .
$\mathbf{M}$	Articulated inertia matrix $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{n \times n}$	Mass (m , m_{eff}), memory footprint ($M_{weights}$, M_{KV}), or fleet size (N_{fleet})	Articulated generalized mass/inertia uses bold matrix $\mathbf{M}(\mathbf{q})$; physical body mass is lowercase italic m (kg); effective impact mass is m_{eff} ; hardware memory storage uses capital M with descriptive subscript (in bytes); robot fleet size uses N_{fleet} or M_{fleet} .

Governing Cyber-Physical Systems Laws

Embodied intelligence operates subject to strict thermodynamic, mechanical, kinematic, and communications constraints. The governing laws below formalize these physical boundaries across the volume:

1. The embodied iron law: Sense-to-torque latency budget

Computational latency converts directly into unguided physical displacement and tracking error. Total closed-loop sense-to-torque loop time T_{loop} must satisfy the hard real-time cadence bound:

$$T_{\text{loop}} = \underbrace{t_{\text{sense}}}_{\text{Exposure \& Readout}} + \underbrace{t_{\text{xfer}}}_{\text{DMA \& Bus Ingest}} + \underbrace{t_{\text{infer}}}_{\text{Neural Inference}} + \underbrace{t_{\text{cbf}}}_{\text{Safety QP Filter}} + \underbrace{t_{\text{act}}}_{\text{Inverter \& Coil Rise}} \leq \frac{1}{f_{\text{ctrl}}}$$

Symbol	Definition	Unit	Notes
T_{loop}	Total Closed-Loop Latency	ms	End-to-end wall-clock time from photon arrival/sensor sampling to mechanical torque realization.
t_{sense}	Sensor Integration Time	ms	Physical exposure duration, ADC conversion, and sensor pipeline readout latency.
t_{xfer}	Interconnect Transfer Time	ms	DMA bus streaming, PCIe handoff, and serializer-deserializer (SerDes) link latency.
t_{infer}	Neural Policy Inference Time	ms	Forward-pass compute time of the cognitive vision-language-action (VLA) or policy network.
t_{cbf}	Control Barrier Filter Latency	ms	Execution time of the deterministic Quadratic Program (QP) solving the barrier safety projection.
t_{act}	Actuator Electrical Rise Time	ms	Gate driver PWM modulation, phase current rise time (L/R time constant), and field buildup.
f_{ctrl}	Control Execution Frequency	Hz	Target closed-loop execution rate (1000 Hz for joint reflex loops; 20–50 Hz for cognitive chunks).

2. The action-chunking streaming cadence law

Cognitive neural policies cannot execute at kilohertz frequencies due to memory bandwidth limits. Action chunking predicts an action trajectory of horizon H at a lower cognitive frequency f_{policy} , while real-time reflexes execute at f_{exec} . The memory streaming latency must remain bounded by the chunk execution window:

$$t_{\text{stream}} = \frac{M_{\text{weights}}}{\text{BW}_{\text{mem}}} \leq \frac{H}{f_{\text{exec}}} - t_{\text{compute}}$$

Symbol	Definition	Unit	Notes
H	Action Chunk Horizon	Time steps	Number of contiguous future action steps generated in a single policy forward pass ($H = 16$ or $H = 32$).
M_{weights}	Model Parameter Footprint	bytes	Total memory required to store policy weights ($P \times \text{bytes-per-parameter}$).
BW_{mem}	Sustained Memory Bandwidth	bytes/s	Achievable memory throughput of the SoC or accelerator memory subsystem (e.g., LPDDR5, HBM).
t_{stream}	Parameter Streaming Time	ms	Time required to stream non-resident model weights into execution registers.
f_{exec}	Action Execution Rate	Hz	Temporal consumption rate of chunked waypoints along the planned trajectory (100–1000 Hz).

3. The dynamic stopping, braking, and safe-set invariants

Forward-invariance of safe set $\mathcal{C}$ dictates that the machine must maintain sufficient braking distance to arrest all motion before violating environmental clearance margin d_{gap} :

$$d_{\text{stop}}(v) = \underbrace{v \cdot t_{\text{react}}}_{\text{Reaction Drift}} + \underbrace{\frac{v^2}{2a_{\text{max}}}}_{\text{Deceleration Envelope}} + \underbrace{\frac{v^3}{6j_{\text{max}}}}_{\text{Jerk Ramp Distance}} + \delta_{\text{loc}} + \delta_{\text{margin}} \leq d_{\text{gap}}$$

Symbol	Definition	Unit	Notes
d_{stop}	Total Stopping Envelope	m	Minimum distance required to bring the physical plant to a complete halt from velocity v .
d_{gap}	Clearance Margin	m	Distance from the current machine position to the nearest obstacle or kinematic boundary.
t_{react}	System Reaction Time	s	Combined sensor stall detection latency, intent lease expiry, and brake engagement time.
v_{max}, v	Velocity Limits	m/s	Maximum allowable operational velocity and instantaneous state velocity.
a_{max}	Maximum Deceleration	m/s ²	Peak actuator or braking deceleration capability within plant thermal and traction limits.
j_{max}	Jerk Limit	m/s ³	Maximum rate of change of deceleration, bounded to prevent gearbox tooth shock and drivetrain ringing.
δ_{loc}	Localization Uncertainty	m	Spatial state estimation bound from Kalman/perception filtering (3σ confidence ellipsoid).
δ_{margin}	Physical Clearance Cushion	m	Mandatory nonzero structural buffer between the machine envelope and environment.

4. The digital sampling zero-order hold (ZOH) phase margin law

Holding discrete control commands constant over sampling period $T_s = 1/f_s$ introduces an effective transport delay of half the sample period, eroding feedback stability margins:

$$\tau_{\text{ZOH}} = \frac{T_s}{2}, \quad \Delta\Phi_{\text{total}} = \omega_c \left(\tau_{\text{comp}} + \tau_{\text{bus}} + \frac{T_s}{2} \right) \leq \text{PM}_{\text{margin}}$$

Symbol	Definition	Unit	Notes
τ_{ZOH}	ZOH Transport Delay	s	Equivalent continuous delay introduced by discrete Zero-Order Hold reconstruction ($T_s/2$).
T_s, f_s	Sampling Period and Rate	s, Hz	Discrete controller execution period and frequency ($f_s = 1/T_s$).
ω_c	Open-Loop Crossover Frequency	rad/s	Frequency at which open-loop gain $ L(j\omega_c) = 1$ (0 dB).
$\Delta\Phi$	Phase Lag Degradation	rad / deg	Phase erosion ($\Delta\Phi \approx 28.65^\circ \cdot (\omega_c/f_s)$) depleting closed-loop stability margin.
PM_{net}	Net Phase Margin	deg	Residual stability margin ($\text{PM}_{\text{plant}} - \Delta\Phi_{\text{total}} > 0$) preventing contact chatter.

5. Actuator electromechanical and thermal rise invariants

Actuators are constrained by copper winding heating, DC bus rail droop, and magnetic saturation. Sustainable torque output is governed by the coupled electromechanical balance:

$$V_{\text{bus}} = L \frac{di}{dt} + R(T)i + K_e \omega_m, \quad C_{\text{th}} \frac{dT}{dt} = I^2 R(T) - \frac{T - T_{\text{amb}}}{R_{\text{th}}}, \quad \tau = \eta N_{\text{gear}} K_t I$$

Symbol	Definition	Unit	Notes
K_t, K_e	Torque and Back-EMF Constants	N · m/A, V · s/rad	Motor electromechanical coupling constants ($K_t \approx K_e$ in SI units).
$R(T)$	Temperature-Dependent Resistance	Ω	Stator copper resistance scaling: $R(T) = R_0[1 + \alpha_{\text{Cu}}(T - T_0)]$ with $\alpha_{\text{Cu}} = 0.00393 \text{ K}^{-1}$.
R_{th}	Thermal Resistance (θ_{JA})	K/W	Lumped thermal resistance from stator windings to ambient environment.

Symbol	Definition	Unit	Notes
C_{th}	Stator Thermal Capacitance	J/K	Lumped heat capacity of the motor core, windings, and potting compound.
τ_{th}	Thermal Time Constant	s	Actuator heating characteristic time constant ($\tau_{th} = R_{th}C_{th}$, typically 60–300 s).
D_{cycle}	Actuator Duty Cycle	[0, 1]	Ratio of active pulse duration to cycle period ($t_{on}/(t_{on} + t_{off})$) preventing thermal runaway.
ΔV_{droop}	DC Bus Dynamic Rail Droop	V	Transient voltage collapse ($\Delta I R_{bus} + L_{bus} \frac{dI}{dt} + \frac{\Delta I \Delta t}{2C_{bus}}$) under step motor current loads.

6. Shielded hazard architecture and statistical exposure bound

Catastrophic failure rates in embodied systems cannot be certified through stochastic testing alone. A deterministic safety shield bounds overall system risk, while zero-failure trial counts follow the Rule of Three:

$$p_{system} = p_{brain}(1 - c_{shield}) \leq p_{target}, \quad n \geq \frac{-\ln \alpha_{sig}}{p}$$

Symbol	Definition	Unit	Notes
p_{system}	Overall System Hazard Rate	failures/hour	Certified dangerous failure rate of the integrated cyber-physical machine.
p_{brain}	Neural Policy Defect Rate	failures/hour	Empirical rate of unshielded hallucinations, distribution drift, or policy blunders.
c_{shield}	Shield Diagnostic Coverage	Dimensionless	Fraction of hazardous neural commands successfully intercepted by deterministic QP/interlocks ($c \geq 0.9999$).
p_{target}	Target Risk Tolerance	failures/hour	Statutory regulatory safety requirement (e.g., 10^{-9} failures/h for SIL 4 / DAL A).
α_{sig}	Statistical Significance Level	Dimensionless	Type I error threshold ($\alpha_{sig} = 1 - CL$, e.g., $\alpha = 0.05$ for 95% confidence where $-\ln(0.05) \approx 3.0$).
n	Zero-Failure Sample Exposure	Hours or Cycles	Required zero-incident validation exposure ($n \approx 3/p$) to substantiate failure rate p .

1. Physical Plant State, Actuation, and Kinematic Topology

Symbol	Definition	Unit	Notes
$s_t \in \mathcal{S}$	Physical World State	Mixed	Ground-truth continuous state of the machine, environment, and physical contacts at time t .
$\mathbf{x}$	Mechanical Plant State	rad, rad/s	Generalized state vector $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T \in \mathbb{R}^{2n}$ containing positions and velocities.
$\mathbf{q} \in \mathbb{R}^n$	Generalized Coordinates	rad or m	Degrees of freedom describing mechanical articulation (joint angles and prismatic link extensions).
$\dot{\mathbf{q}} \in \mathbb{R}^n$	Generalized Velocities	rad/s or m/s	First time derivative of generalized coordinates.
$\ddot{\mathbf{q}} \in \mathbb{R}^n$	Generalized Accelerations	rad/s ² or m/s ²	Second time derivative of generalized coordinates.
$\boldsymbol{\tau} \in \mathbb{R}^n$	Actuator Torque Vector	N · m or N	Generalized forces and torques delivered by actuators at mechanical joints.
$\mathbf{M}(\mathbf{q})$	Generalized Inertia Matrix	kg · m ²	Symmetric positive-definite mass/inertia matrix of the articulated multibody plant.

Symbol	Definition	Unit	Notes
$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$	Coriolis / Centrifugal Matrix	$\text{N} \cdot \text{m} \cdot \text{s} / \text{rad}$	Matrix satisfying $\dot{\mathbf{M}} - 2\mathbf{C}$ skew-symmetry (passivity property).
$\mathbf{g}(\mathbf{q})$	Gravitational Torque Vector	$\text{N} \cdot \text{m}$	Generalized gravitational load acting on links as a function of configuration.
$\mathbf{J}(\mathbf{q})$	Manipulator Jacobian	Mixed	Linear mapping $\mathbf{v}_e = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$ from joint rates to operational task-space velocity.
$w(\mathbf{q})$	Yoshikawa Manipulability	Dimensionless	Singularity proximity metric $w(\mathbf{q}) = \sqrt{\det(\mathbf{J}\mathbf{J}^\top)}$ vanishing at kinematic singular configurations.
K_t, K_e	Motor Torque / Back-EMF	$\text{N} \cdot \text{m} / \text{A}, \text{V} \cdot \text{s} / \text{rad}$	Electromechanical coupling constants linking winding current to rotor shaft torque and back-EMF voltage.
e_{bemf}	Back-EMF Voltage	V	Opposing voltage $e_{\text{bemf}} = K_e \omega_m$ developed across stator coils by rotor rotation.
$J_{\text{rotor}}, J_{\text{load}}$	Rotor and Load Inertia	$\text{kg} \cdot \text{m}^2$	Moment of inertia of motor rotor shaft and coupled mechanical load.
J_{ref}	Reflected Rotor Inertia	$\text{kg} \cdot \text{m}^2$	Effective inertia $J_{\text{ref}} = N_{\text{gear}}^2 J_{\text{rotor}}$ amplified quadratically by gearbox ratio.
N_{gear}	Gear Reduction Ratio	Dimensionless	Transmission reduction ratio ($N_{\text{gear}} \gg 1$) between motor rotor and joint output shaft.
N^*	Optimal Inertia-Matching Ratio	Dimensionless	Ratio $N^* = \sqrt{J_{\text{load}} / J_{\text{rotor}}}$ maximizing load angular acceleration under peak torque.
η_{gear}	Transmission Efficiency	Dimensionless	Mechanical power transmission efficiency ($0 < \eta_{\text{gear}} \leq 1$).
$\tau_{\text{hold}}, \tau_{\text{peak}}$	Continuous / Peak Torque	$\text{N} \cdot \text{m}$	Thermally continuous holding torque and transient peak acceleration torque.
$I_{\text{cont}}, I_{\text{peak}}$	Continuous / Peak Current	A	RMS current ratings bounded by winding insulation class and inverter MOSFET dissipation.

2. Rigid-Body Contact Mechanics, Friction Cones, and Centroidal Dynamics

Symbol	Definition	Unit	Notes
$\mathbf{S} \in \mathbb{R}^{n \times (n+6)}$	Actuation Selection Matrix	Matrix	Matrix $[\mathbf{0}_{n \times 6}, \mathbf{I}_{n \times n}]$ projecting generalized forces onto actuated joints in floating-base systems.
$\mathbf{f}_k \in \mathbb{R}^3$	Contact Force Vector	N	Physical contact force exerted by environment on contact patch k ($\mathbf{f}_k = [\mathbf{f}_{k,\parallel}, \mathbf{f}_{k,\perp}]^\top$).
$\mathbf{J}_k(\mathbf{q})$	Contact Point Jacobian	Mixed	Mapping joint velocities to contact patch Cartesian linear velocity: $\dot{\mathbf{p}}_k = \mathbf{J}_k(\mathbf{q})\dot{\mathbf{q}}$.
$f_{k,\perp}$	Normal Contact Force	N	Unilateral compressive force ($f_{k,\perp} = \mathbf{f}_k^\top \hat{\mathbf{n}}_k \geq 0$) perpendicular to contact plane.
$\mathbf{f}_{k,\parallel}$	Tangential Contact Shear	N	Friction traction force tangential to contact plane ($\mathbf{f}_{k,\parallel} = \mathbf{f}_k - f_{k,\perp} \hat{\mathbf{n}}_k$).
μ	Coulomb Friction Coefficient	Dimensionless	Static/kinetic coefficient bounding friction cone: $\ \mathbf{f}_{k,\parallel}\ \leq \mu f_{k,\perp}$.
$\mathbf{c} \in \mathbb{R}^3$	Center of Mass (CoM)	m	Spatial coordinates of the total articulated body mass centroid ($\mathbf{c} = \frac{1}{m} \sum m_i \mathbf{p}_i$).
$\mathbf{h}_{\text{lin}} \in \mathbb{R}^3$	Centroidal Linear Momentum	$\text{kg} \cdot \text{m} / \text{s}$	Total plant linear momentum ($\mathbf{h}_{\text{lin}} = m\dot{\mathbf{c}}$), governed by $\dot{\mathbf{h}}_{\text{lin}} = \sum \mathbf{f}_k + m\mathbf{g}$.
$\mathbf{h}_{\text{ang}} \in \mathbb{R}^3$	Centroidal Angular Momentum	$\text{kg} \cdot \text{m}^2 / \text{s}$	Angular momentum about CoM: $\dot{\mathbf{h}}_{\text{ang}} = \sum (\mathbf{p}_k - \mathbf{c}) \times \mathbf{f}_k + \tau_k$.
$\mathbf{p}_{\text{ZMP}} \in \mathbb{R}^2$	Zero Moment Point (ZMP)	m	Ground-plane point where net horizontal tipping moment vanishes; must reside in $\text{Conv}(\{\mathbf{p}_k\})$.

Symbol	Definition	Unit	Notes
$e \in [0, 1]$	Coefficient of Restitution	Dimensionless	Kinematic velocity rebound ratio during non-smooth impact ($\dot{q}^+ = -e\dot{q}^-$).
k, k_{tissue}	Contact Stiffness	N/m	Elastic stiffness of structural environment or human biomechanical tissue.
m_{eff}	Effective Impact Mass	kg	Apparent mass of moving kinematic links along collision trajectory ($m_{\text{eff}} = (\mathbf{u}_{\text{dir}}^\top \mathbf{M}^{-1} \mathbf{u}_{\text{dir}})^{-1}$).
F_{unbraked}	Unbraked Impact Force	N	Transient collision force peak ($F_{\text{unbraked}} = v_{\max} \sqrt{k_{\text{tissue}} m_{\text{eff}}}$) governed by ISO/TS 15066.
$E_{k, \text{allow}}$	Allowable Collision Energy	Joules (J)	Statutory kinetic energy limit ($\frac{1}{2} m_{\text{eff}} v^2 \leq E_{k, \text{allow}}$) for transient collaborative contact.

3. Geometry, Spatial Transformations, and Perception Projections

Symbol	Definition	Unit	Notes
$SE(3)$	Special Euclidean Group	Lie Group	Group of 3D rigid-body spatial transformations (translations and rotations: $\mathbb{R}^3 \rtimes SO(3)$).
$SO(3)$	Special Orthogonal Group	Lie Group	Group of 3×3 proper orthogonal rotation matrices ($\mathbf{R}^\top \mathbf{R} = \mathbf{I}$, $\det(\mathbf{R}) = +1$).
$\mathcal{F}_W, \mathcal{F}_B, \mathcal{F}_E$	Coordinate Reference Frames	Reference	Fixed World frame ($\mathcal{F}_W$), mobile robot Base/body frame ($\mathcal{F}_B$), and End-effector frame ($\mathcal{F}_E$).
$\mathbf{p} \in \mathbb{R}^3$	Cartesian 3D Translation	m	Translation vector $[x, y, z]^\top$ expressing relative point coordinates.
$\mathbf{R} \in SO(3)$	Directional Cosine Matrix	Matrix	3×3 orthonormal matrix parameterizing spatial attitude and orientation.
$\mathbf{q}_{\text{quat}} \in \mathbb{H}$	Unit Quaternion	Dimensionless	Hypercomplex orientation vector $[w, x, y, z]^\top$ satisfying $\ \mathbf{q}_{\text{quat}}\ = 1$.
$\mathbf{T}_{AB} \in SE(3)$	Homogeneous Transform	Matrix	4×4 matrix $\begin{bmatrix} \mathbf{R}_{AB} & \mathbf{p}_{AB} \\ \mathbf{0}^\top & 1 \end{bmatrix}$ mapping coordinates from frame B into frame A .
$\mathbf{Ad}_T$	Adjoint Representation	Matrix	6×6 transformation matrix mapping spatial twists and wrenches between coordinate frames.
$\omega, \alpha \in \mathbb{R}^3$	Angular Velocity & Acceleration	rad/s, rad/s ²	Spatial angular velocity and angular acceleration vectors of a rigid body.
$\mathbf{r}_{P/O} \in \mathbb{R}^3$	Sensor Lever-Arm Vector	m	Displacement vector from body rotation origin O to displaced sensor location P .
$\mathbf{K} \in \mathbb{R}^{3 \times 3}$	Pinhole Camera Intrinsics	Pixels	Calibration matrix $\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$ parameterizing focal lengths and principal center.
$\mathbf{P}_c = [\hat{X}, \hat{Y}, Z]^\top$	Camera-Frame 3D Coordinate	m	Metric coordinates of a spatial point in the optical camera frame (Z_c is optical depth).
$[u, v]^\top$	Image Plane Pixel Coordinates	Pixels	Discrete sensor pixel coordinates ($u \sim f_x \frac{X}{Z} + c_x$, $v \sim f_y \frac{Y}{Z} + c_y$).
t_{row}	Rolling Shutter Row Readout	μs	Exposure time delta between adjacent CMOS sensor scanlines.
$\Delta \mathbf{x}(y)$	Rolling Shutter Distortion	m	Kinematic skew displacement $\Delta \mathbf{x} = \mathbf{v} \cdot y \cdot t_{\text{row}}$ induced by high-velocity platform motion.
$\text{tsdf}(\mathbf{v})$	Truncated Signed Distance	$[-1, +1]$	Normalized volumetric distance $\text{tsdf}(\mathbf{v}) = \text{clamp}(d(\mathbf{v})/\mu_{\text{tsdf}}, -1, 1)$ of voxel $\mathbf{v}$ to nearest surface.

Symbol	Definition	Unit	Notes
μ_{tsdf}	TSDF Truncation Band	m	Metric thickness of the spatial surface interface band stored in voxel memory.
$D_t(\mathbf{v}), W_t(\mathbf{v})$	TSDF Accumulated Volume	Mixed	Running weighted signed distance and accumulated visibility weight at voxel $\mathbf{v}$.
Σ_{depth}	Perception Depth Covariance	m^2	Anisotropic 3D uncertainty covariance ellipsoid associated with stereo/LiDAR/depth lifting.
τ_{age}	Telemetry Spatial Age	ms	Total elapsed duration since physical transduction: $\tau_{\text{age}} = \frac{1}{2}t_{\text{exp}} + t_{\text{read}} + t_{\text{xfer}} + t_{\text{proc}}$.

4. Classical and Modern Control (State-Space, Observers, CBF-QPs)

Symbol	Definition	Unit	Notes
$\mathbf{A}, \mathbf{B}, \mathbf{C}$	Linear State-Space Matrices	Matrix	Continuous LTI plant: $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} + \mathbf{w}$, observed output $\mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{v}$.
$\mathbf{w}(t), \mathbf{v}(t)$	Process and Measurement Noise	Vector	Zero-mean Gaussian noise vectors with covariance matrices $\mathbf{Q} \succeq 0$ and $\mathbf{R} \succ 0$.
$\mathbf{P}(t) \in \mathbb{R}^{n \times n}$	Estimation Error Covariance	Matrix	Symmetric positive-definite matrix $\mathbb{E}[(\mathbf{x} - \hat{\mathbf{x}})(\mathbf{x} - \hat{\mathbf{x}})^{\top}]$ evolving via matrix Riccati equation.
$\mathcal{O}, \mathcal{O}_{\text{NL}}$	Observability Matrices	Matrix	Kalman observability matrix $[\mathbf{C}^{\top}, (\mathbf{C}\mathbf{A})^{\top}, \dots]^{\top}$ and nonlinear Lie matrix $[d\mathbf{h}, dL_f\mathbf{h}, \dots]^{\top}$.
$\mathcal{C} \subset \mathcal{S}$	Forward-Invariant Safe Set	Set	Sublevel/superlevel set $\mathcal{C} = \{\mathbf{x} \mid h(\mathbf{x}) \geq 0\}$ where safety invariants are strictly maintained.
$\partial\mathcal{C}$	Safety Boundary Manifold	Boundary	Zero level set $\{\mathbf{x} \mid h(\mathbf{x}) = 0\}$ separating nominal operation from irreversible hazard.
$h(\mathbf{x}) \geq 0$	Control Barrier Function (CBF)	Scalar	Continuously differentiable function whose superlevel set defines forward-invariant set $\mathcal{C}$.
$L_f h(\mathbf{x})$	Lie Derivative along Drift	s^{-1}	Directional gradient derivative $\nabla h(\mathbf{x})\mathbf{f}(\mathbf{x})$ along unforced plant dynamics.
$L_g h(\mathbf{x})$	Lie Derivative along Control	Vector	Directional control effectiveness gradient $\nabla h(\mathbf{x})\mathbf{g}(\mathbf{x})$.
r	Barrier Relative Degree	Integer	Number of differentiations required before control input $\mathbf{u}$ appears ($L_g L_f^{r-1} h \neq 0$).
$h_{\text{kinetic}}(\mathbf{x})$	Kinetic Inset Barrier Function	Scalar	Relative-degree-1 barrier: $h_{\text{kinetic}}(\mathbf{x}) = h(\mathbf{x}) - v^2/(2a_{\text{max}})$ incorporating stopping distance.
γ_{cbf}	Linear Barrier Decay Rate	s^{-1}	Extended class- $\mathcal{K}$ linear decay coefficient in barrier condition $\dot{h} + \gamma_{\text{cbf}}h \geq 0$.
$\mathbf{u}_{\text{nom}} \in \mathcal{U}$	Nominal Control Command	$\text{N} \cdot \text{m} / \text{rad}$	Unverified control candidate proposed across the causal boundary by learned policy.
$\mathbf{u}^* \in \mathcal{U}$	Certified Control Action	$\text{N} \cdot \text{m} / \text{rad}$	Privileged, safety-filtered control command resolved by the real-time active-set CBF-QP.
$\alpha(\hat{t})$	Bumpless Transfer Blend	$[0, 1]$	C^2 quintic polynomial $6\hat{t}^5 - 15\hat{t}^4 + 10\hat{t}^3$ ramping between prior and candidate controllers.
$\tau_{\text{blend}}, T_{\text{blend}}$	Blending Transition Window	s	Duration of bumpless transfer; bounded by $\tau_{\text{blend}, \text{min}} = 1.875\ \Delta\mathbf{u}\ /j_{\text{allowable}}$.
$\dot{j}(t) = \dot{\mathbf{u}}(t)$	Control Setpoint Jerk	$\text{m}/\text{s}^3, \text{N} \cdot \text{m}/\text{s}$	Time derivative of commanded plant acceleration or actuator torque during mode transitions.
K_p, K_d, K_i	PID / Feedback Gains	Mixed	Proportional, derivative, and integral gains for joint-level or operational-space error stabilization.
$\mathbf{D}_{\text{virt}}, \mathbf{K}_{\text{virt}}$	Virtual Damping and Stiffness	Matrix	Target impedance/admittance matrices in compliant interaction: $\tau_{\text{safe}} = -\mathbf{D}_{\text{virt}}\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q})$.

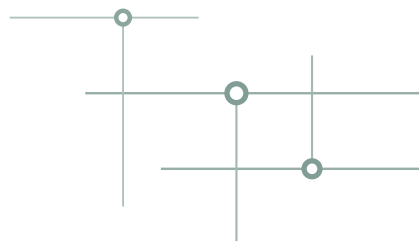
5. Deep Learning for Robotics (Diffusion Policies, ACT, VLAs)

Symbol	Definition	Unit	Notes
$\pi_\theta(\mathbf{a} \mid \mathbf{o})$	Parameterized Neural Policy	Distribution	Deep policy network parameterized by weights θ , mapping observation tensors to action distributions.
$\mathbf{o}_t \in \mathcal{O}$	Multimodal Observation	Mixed	Synchronized tokenized sensor packet (RGB images, depth latents, joint encoders, tactile arrays).
$\mathbf{a}_t \in \mathcal{A}$	Cognitive Policy Action	Normalized	Unprivileged action emitted by neural policy π_θ (normalized joint delta, target pose, or gripper command).
$\mathbf{A}_{t:t+H}$	Action Chunk Trajectory	Tensor	Contiguous horizon sequence $[\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}] \in \mathbb{R}^{H \times d_a}$ predicted in a single forward pass.
H	Chunk Prediction Horizon	Time steps	Number of future action steps emitted per inference cycle ($H = 16, 32, 64$).
K	Ensembling Window Length	Time steps	Number of overlapping past chunks blended in receding horizon execution ($K \leq H$).
w_i	Temporal Ensembling Weight	Scalar	Exponential decay weighting factor $w_i = \exp(-m \cdot i)$ applied to chunk predictions at offset i .
$\mathbf{z} \in \mathbb{R}^d$	Latent Style / State Embedding	Dimensionless	Compressed semantic representation or CVAE latent style variable in ACT policies.
$\bar{\alpha}_k$	Diffusion Schedule Parameter	$[0, 1]$	Cumulative noise variance schedule parameter $\bar{\alpha}_k = \prod_{s=1}^k (1 - \beta_s)$ at diffusion step k .
$\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$	Diffusion Gaussian Noise	Tensor	Standard normal Gaussian noise corrupted onto clean action trajectories.
$\epsilon_\theta(\mathbf{o}, \mathbf{A}^k, k)$	Noise Prediction Network	Tensor	Neural network predicting added noise in Denoising Diffusion Probabilistic Models (DDPM).
$\mathcal{L}_{\text{diff}}(\theta)$	Diffusion Matching Loss	Scalar	Mean-squared error objective: $\mathbb{E}[\ \epsilon - \epsilon_\theta(\mathbf{o}_t, \mathbf{A}_t^k, k)\ ^2]$ optimized across denoising steps.
$\mathcal{L}_{\text{ACT}}(\theta, \phi)$	ACT Variational Objective	Scalar	CVAE objective combining trajectory reconstruction $\mathcal{L}_{\text{recon}}$ with latent KL-regularization βD_{KL} .
$q_\phi(\mathbf{z} \mid \mathbf{o}, \mathbf{A})$	CVAE Recognition Encoder	Distribution	Variational encoder in Action Chunking with Transformers parameterizing latent distribution $\mathcal{N}(\mu, \Sigma)$.
ϵ_{BC}	Single-Step Imitation Error	$[0, 1]$	Bounded probability of an unforced policy error during an individual execution step.
$O(T^2 \epsilon_{\text{BC}})$	Compounding Covariate Drift	Error bound	Quadratic compounding trajectory drift under open-loop behavioral cloning rollouts of horizon T .
$\mathbf{W}_{\text{patch}}, \mathbf{E}_{\text{pos}}$	ViT Projection & Positional Bias	Matrix	Convolutional/linear patch projection matrix and spatial coordinate position embedding tensors.
$\mathbf{Z}$	Multimodal Fused Token Stream	Tensor	Concatenated token sequence $[\mathbf{z}_{\text{vision}}, \mathbf{z}_{\text{text}}, \mathbf{z}_{\text{proprio}}]$ ingested by embodied transformer backbones.
M_{KV}	Transformer KV-Cache Footprint	bytes	Autoregressive key-value cache memory footprint $(2 \times N_L \times d \times S \times s_{\text{elem}})$.
Λ_{intent}	Expiring Intent Lease	Permit	Cryptographically signed, bounded time-to-live operational lease granted to learned policies.
T_{lease}	Intent Lease Expiration Horizon	ms	Maximum allowable duration of autonomous execution authority before mandatory re-evaluation (≤ 100 ms).

6. Embedded Systems, Silicon Real-Time, Thermal, and Hardware Safety

Symbol	Definition	Unit	Notes
I_{phase}	Motor Phase RMS Current	Amperes (A)	Electrical phase current driving stator windings in brushless PM motors ($I = \tau / (\eta N K_t)$).
$R_{\text{phase}}, R(T)$	Phase Winding Resistance	Ω	Stator copper DC resistance at temperature T ($R(T) = R_0[1 + \alpha_{\text{Cu}}(T - T_0)]$).
$R_{\text{th}}, \theta_{JA}$	Lumped Thermal Resistance	K/W	Thermal impedance between internal hot spots and heatsink/ambient casing.
C_{th}	Stator Thermal Capacitance	J/K	Stator core heat capacity governing temperature rise dynamics ($\tau_{\text{th}} = R_{\text{th}}C_{\text{th}}$).
ΔT	Joulean Temperature Rise	Kelvin (K)	Steady-state or transient thermal rise ($\Delta T = P_{\text{cont}}R_{\text{th}}$).
$L_{\text{bus}}, R_{\text{bus}}$	Power Rail Inductance / ESR	$\mu\text{H}, \text{m}\Omega$	Parasitic power wiring harness inductance and Equivalent Series Resistance feeding inverters.
C_{bus}	DC Bus Bulk Capacitance	μF	Inverter decoupling capacitor bank buffering transient inductive voltage collapse.
ΔV_{droop}	Dynamic Rail Voltage Droop	Volts (V)	Transient voltage dip ($\Delta I R_{\text{bus}} + L_{\text{bus}} \frac{dI}{dt} + \frac{\Delta I \Delta t}{2C_{\text{bus}}}$) triggering undervoltage reset.
$t_{\text{vulnerable}}$	Seqlock Vulnerable Interval	μs	Race window ($t_{\text{write}} + t_{\text{read}}$) during lock-free inter-processor shared SRAM exchanges.
$P_{\text{collision}}$	Seqlock Retry Probability	Ratio	Probability $t_{\text{vulnerable}}/T_{\text{writer}}$ that reader thread observes torn telemetry and must retry.
t_{queue}	Memory Bus Queue Delay	ms	Contention latency $\sum B_k / \text{BW}_{\text{peak}} + N_{\text{conflict}} t_{\text{penalty}}$ across shared SoC crossbars.
N_{conflict}	Interconnect Port Conflicts	Count	Number of simultaneous DMA / GPU / CPU memory channel access collisions.
Δt_{fresh}	Telemetry Freshness Deadline	ms	Maximum allowable age of sensor telemetry before an interlock trips: $\Delta t_{\text{fresh}} \leq e_{\text{max}}/v_{\text{max}}$.
t_{watchdog}	Hardware Watchdog Timeout	ms	Hardware timer period triggering fail-safe dynamic braking upon lost processor heartbeats.
t_{NMI}	Non-Maskable Interrupt Time	μs	Deterministic hardware interrupt response time for motor overcurrent or gate-drive faults.
δ_{skew}	Clock Synchronization Skew	ppm, $\mu\text{s/s}$	Hardware crystal drift between perception host and real-time microcontroller clocks.
Δt_{peak}	Peak Inter-Node Phase Jitter	μs	Cumulative timing uncertainty $\delta_{\text{skew}} \Delta T_{\text{blackout}} + \sigma_{\text{jitter}}$ during bus sync blackouts.
MTBUF	Mean Time Between Unsafe Failure	Hours	Operating hours between undetected hazardous safety violations: $\text{MTBUF} = \frac{1}{\lambda_{\text{raw}}(1 - c_{\text{shield}})}$.
$\mathcal{T}_{\text{trichotomy}}$	Trichotomy Monitor State	Enum	Three-valued real-time monitor evaluation: {Known-True, Known-False, Unknown}.

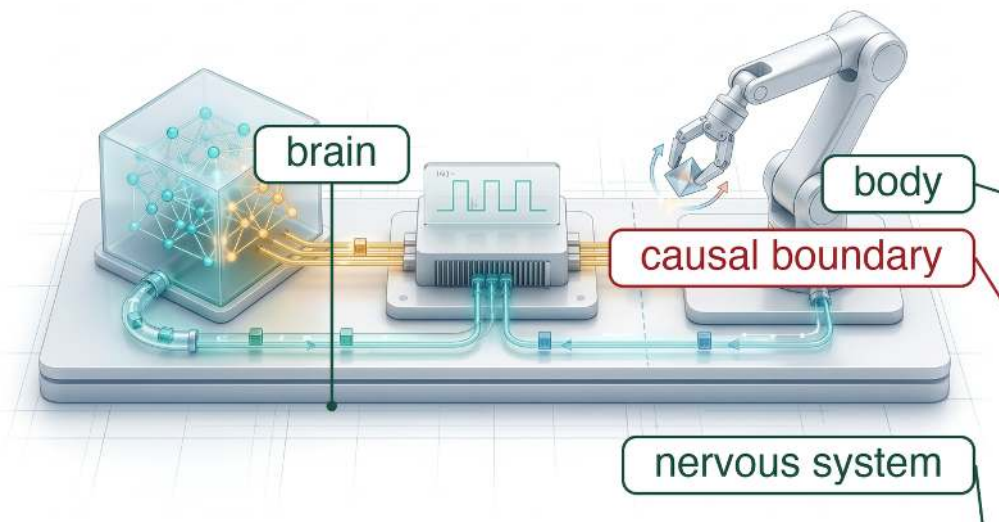
MAIN TEXT



The Causal Boundary

DRAFT

The Causal Boundary



Purpose

What must an autonomous physical agent guarantee before commanding physical work, and why does digital software fail to provide that guarantee?

A learned policy may select actions the machine cannot safely execute. Once a command reaches a motor, current generates torque and moving mass carries momentum. Canceling the computation cannot undo that motion. As a robot's software processes observations, its physical chassis continues coasting toward an obstacle. Every millisecond of computational delay consumes physical space, irreversibly shrinking the available braking envelope. Prediction accuracy alone cannot determine if an action should proceed. The system must check proposed movement against physical limits and retain authority to reject commands that exceed them, even when the learned policy is late or wrong. Physical AI architecture links the brain's proposals to the nervous system's timing and enforcement, and the body's capacity to move, absorb energy, and stop.

- 2.1 Artificial Intelligence Beyond Glass
- 2.2 The Causal Boundary
- 2.3 The Four Machine Archetypes
- 2.4 The Physical AI Scope Test
- 2.5 The Four Architectural Tiers
- 2.6 The S-P-A Intersection Landscape
- 2.7 When the Boundary Fails
- 2.8 The Four Bedrock Laws
- 2.9 The Dual-Brain Architecture
- 2.10 Safety Convergence
- 2.11 The Architecture of This Book
- 2.12 Fallacies and Pitfalls
- 2.13 Summary

	Brain 1–50 Hz foundation models, SLAM, planning
above: propose; below: command	
	Nervous System 1000 Hz TSN, intent leases, lock-free IPC
	Body continuous real-time MCU, safety filters, CBFs
	Boundary continuous sensors, actuators, kinematics
	Physical World continuous contact mechanics, friction, mass

Downstream: The physical laws governing the causal boundary are mapped onto hardware failure modes in section 4.7.



Learning Objectives

- Explain what changes when software outputs command actuators in a physical machine
- Classify physical machines by their dominant motion, contact, or flow constraints
- Apply the physical AI scope test to distinguish advisory models from autonomous physical systems
- Trace sensing, reasoning, control, and governance responsibilities through a complete physical system
- Diagnose how timing and authority failures combine in an autonomous machine incident
- Evaluate separation of learned proposals from actuation authority under conflicting control and hardware requirements

2.1 Artificial Intelligence Beyond Glass

A robot approaching an open doorway must do more than recognize the portal in its camera stream. It must evaluate whether its physical chassis fits through the opening, select a viable trajectory, and continuously adjust that path as momentum carries it forward. If a sudden obstacle appears, canceling the software thread or throwing an unhandled exception cannot stop the machine's momentum. While the neural network computes, the machine continues traveling along its existing vector, and every millisecond of inference delay erodes the physical distance available for braking. Designing this behavior requires tracing a decision all the way from sensory evidence to electromagnetic torque and mechanical work. That complete closed loop is the domain of physical artificial intelligence systems engineering.

Advisory computations can terminate at a display or a human decision, leaving no direct path from their output to an actuator. Their errors can still have serious consequences when people or other systems act on them. In a physical machine, digital bits can command electrical currents across power inverters, magnetic flux generates mechanical torque, and moving mass accumulates momentum. An erroneous command can therefore produce a sheared gearbox, a collapsed battery voltage rail, or a collision before software can correct it.

This asymmetry underlies **Moravec's paradox**¹ (Moravec 1988): tasks that require intense human conscious effort, such as playing chess or proving theorems, require modest computation, whereas sensorimotor skills executed effortlessly—such as walking across uneven gravel or grasping an egg without crushing it—demand vast computational and control resources. As high-capacity neural networks are deployed to master these sensorimotor tasks, engineering teams encounter the *exposure barrier*: offline benchmark accuracy can neither predict nor guarantee closed-loop stability in the open world. The illustrative accumulation in figure 2.1 shows why even a small per-mile hazard rate matters as physical deployment grows: safety arguments must cover cumulative exposure, not only controlled demonstrations.

When digital models command physical actuators, they enter the closed causal feedback loops first formalized by Norbert Wiener in cybernetics² (Wiener 1948). Information ceases to be an open-loop stream terminating at a display. Instead, every physical motion repositions the machine's sensors, reshaping subsequent observations under the deterministic laws of classical mechanics. Engineering teams bridging these domains encounter an intellectual Tower of Babel where four distinct technical cultures clash:

The machine learning perspective focuses on scaling parameter counts, transformer backbones, and diffusion policies across millions of simulation steps, treating real-world deployment as a challenge of broader demonstration datasets and dataset coverage. The computer systems perspective focuses on memory bus bandwidth, cache hierarchies, and tail-latency percentiles, recognizing that streaming multi-gigabyte neural weights at high frequencies threatens to saturate memory interconnects and starve camera direct memory access pipelines (Williams et al. 2009). The classical robotics perspective relies on Newton-Euler dynamics to manage mechanical forces and motion. If a neural network commands rapid changes in motion (high-frequency jerk), it can shear gearboxes or trigger structural resonance, making classical stability margins critical. Finally, the embedded silicon and safety perspective focuses on bare-metal interrupt service routines, hardware trip zones,

¹ | **Moravec's Paradox:** The observation in artificial intelligence (Moravec 1988) that high-level reasoning requires comparatively little computation, whereas low-level sensorimotor coordination demands enormous perceptual and computational resources.

² | **Cybernetic Feedback Loops:** In Norbert Wiener's cybernetic framework (Wiener 1948), feedback denotes adjusting future action using past performance. In physical AI, feedback is endogenous: actuation physically alters the pose of sensors and the state of the plant, directly dictating the probability distribution of subsequent observations.

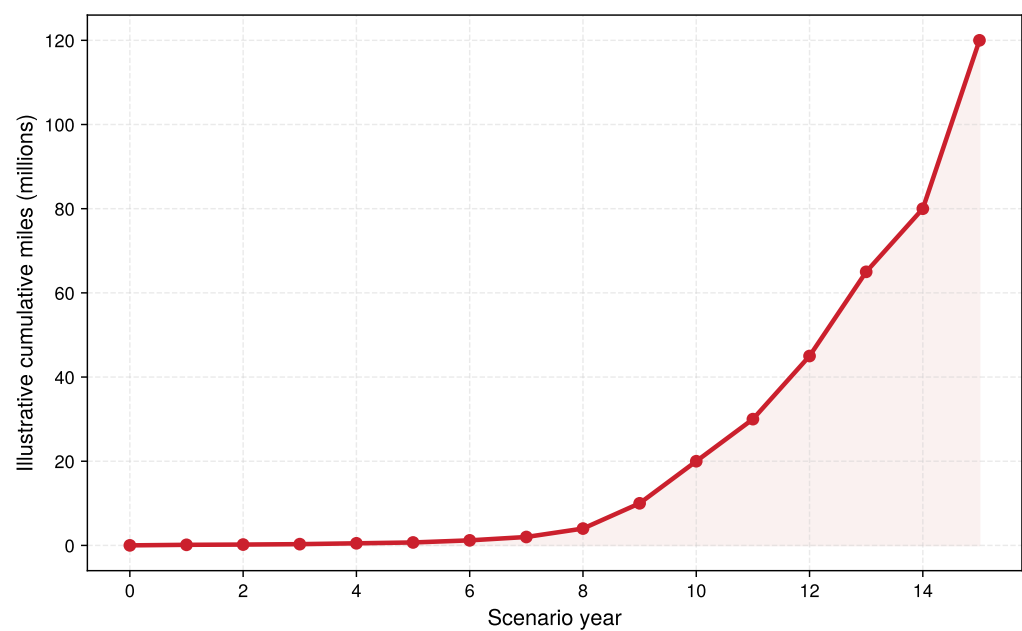


Figure 2.1: Illustrative exposure growth: A synthetic cumulative-mileage scenario from the chapter-local CSV reaches 120 million miles after fifteen scenario years. These values are not observed fleet mileage; the curve illustrates how repeated physical operation expands exposure to rare hazards.

and functional safety standards (such as ISO 26262 and UL 4600), ensuring that unverified statistical models never hold direct write access to motor power stages.

Each perspective offers a critical insight. Gradient descent does not repair a fractured transmission or prevent an inductive power rail collapse, while a hand-tuned classical controller cannot provide open-world semantic scene understanding. A viable physical AI systems architecture must unify their responsibilities without asking any single discipline to shoulder the entire problem.

Downstream: The dual-brain proposal-permission privilege boundary is mapped to isolated silicon domains in section 8.2.



Definition 2.1: Physical artificial intelligence (physical AI)

Physical artificial intelligence (physical AI) is the systems engineering science of embedding high-capacity learned statistical models into closed-loop physical machines governed by classical mechanics, thermodynamics, and hard real-time silicon determinism.

1. **Significance:** Advisory AI outputs lack direct actuator authority, although people or downstream systems may act on them. Physical AI crosses into a continuous plant, where digital bits command motor currents, kinetic momentum accumulates, and software stalls can contribute to mechanical fractures, overheating, or collisions.
2. **Distinction:** Unlike classical robotics (which relies on hand-engineered state machines and explicit kinematic models) or internet-scale machine learning (which operates on static offline datasets), Physical AI couples high-capacity statistical generalization with deterministic hardware safety filters across an endogenous closed causal loop.
3. **Common pitfall:** Treating physical embodiments as peripheral I/O devices for foundation models. Scaling laws and benchmark accuracies do not prevent inductive rail collapse, gearbox shear, or actuator force saturation; physical survival requires an asymmetric architecture where real-time silicon holds unconditional veto authority over learned proposals.

3 | Proposal-Permission Privilege Boundary:

The architectural division separating learned proposals from actuation permission. An independent mechanism accepts, modifies, or vetoes commands before they reach the plant; an isolated cycle-deterministic microcontroller is one implementation.

4 | Idempotency:

Repeating an idempotent operation has the same specified result as executing it once. A motor setpoint command can have idempotent command semantics, but repeated execution may still dissipate energy. Idempotency does not imply reversibility or harmlessness.

5 | Action Chunk Representations:

In robot learning literature (Zhao et al. 2023), action chunks are typically tensors of shape $[B, H, d_a]$ where B is batch size, H is the planning prediction horizon, and d_a is action dimension (e.g., target joint positions $\hat{\mathbf{q}} \in \mathbb{R}^d$ or Cartesian pose deltas in $SE(3)$, which mathematically encode both 3D translation and 3D rotation).

6 | Field-Oriented Control (FOC):

Also termed vector control, FOC mathematically transforms three-phase stator currents into decoupled direct and quadrature components (I_d, I_q) aligned with rotor magnetic flux. This transformation allows real-time firmware to govern torque and flux independently, treating complex AC synchronous machines with the linear dynamics of a DC motor.

To coordinate these competing engineering disciplines, the physical AI systems architect organizes the machine into the four-tier systems hierarchy shown in figure 2.2. At the foundation sits Layer 1, the Physical Body, comprising the continuous electromechanical and thermodynamic plant of motors, linkages, and power rails. Immediately above it, Layer 2, the Real-Time Nervous System, enforces physical limits within the plant's response budget; a deterministic microcontroller running at 1000 Hz to 20 kHz is one implementation. Across the **Proposal-Permission Privilege Boundary**³ sits Layer 3, the Cognitive Brain, where high-capacity neural policies deliberate at variable cadences (1–50 Hz) to interpret multimodal scenes and synthesize candidate action chunks. Overarching the entire stack is Layer 4, System Governance, which coordinates multi-agent assurance, verification, and human oversight.

The chapter follows a command from computation to physical work, compares the budgets of four machine archetypes, and uses the Tempe collision to test the boundary between learned proposals and actuation permission. The final sections turn that failure analysis into architectural requirements.

2.2 The Causal Boundary

An advisory model that only displays a recommendation has no direct path to a motor or valve. A user may dismiss the recommendation, and a database transaction may be rolled back before an external action commits. Neither property makes digital errors harmless: a person may follow bad advice, and an automated trading system may execute a consequential order. The relevant distinction is whether a model's output can automatically reach a physical actuator. Once that authority exists, canceling a computation cannot undo work already performed.⁴

Physical artificial intelligence eliminates that detachment. In a physical machine, the final instruction of a control loop does not write to a screen or populate a web response. For a machine learning systems engineer accustomed to writing `action = policy(observation)` in PyTorch, the causal boundary is where floating-point tensors meet physical silicon. The policy's output tensor—typically an action chunk predicting a sequence of upcoming waypoints (joint positions, velocities, or 6-DOF Cartesian poses)⁵—cannot execute directly on physical motors.

The architectural requirement is an independent permission path between a learned proposal and actuator command. In one implementation, the host runtime serializes predictions into a shared-memory mailbox for a hard real-time microcontroller. Its safety referee evaluates physical barrier conditions at 1000 Hz. If approved, the kinematic setpoints pass into a deterministic cascaded control loop: a joint position and velocity loop calculates required motor torques, which feed an inner **Field-Oriented Control (FOC)**⁶ loop running at 10–25 kHz to compute the torque-producing quadrature current I_q .

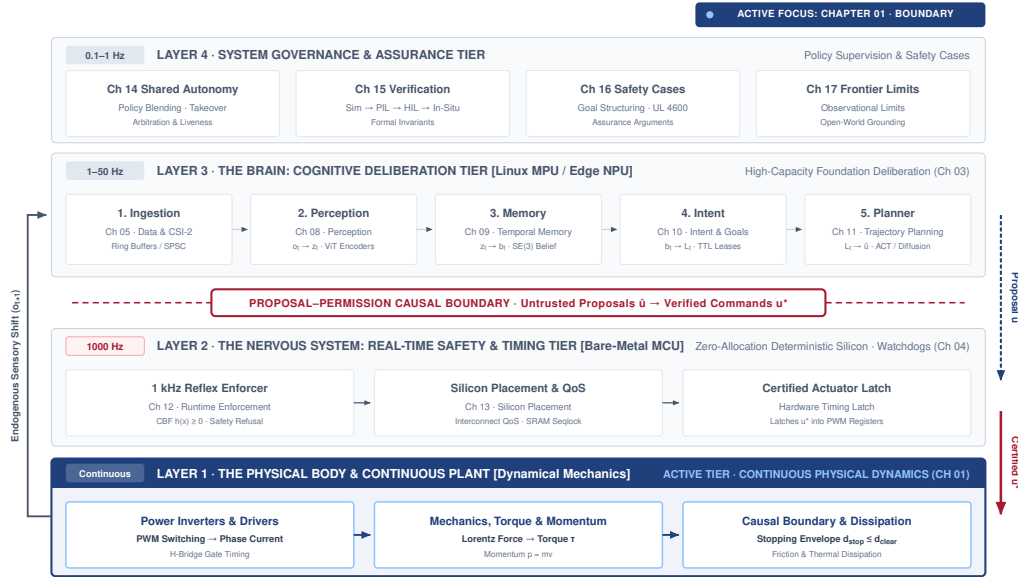


Figure 2.2: Physical AI systems architecture stack: The master systems hierarchy spanning System Governance (Layer 4), Cognitive Deliberation (Layer 3), Hard Real-Time Reflex & Silicon (Layer 2), and the Physical Body (Layer 1). In this introductory chapter, our active focus centers on the causal boundary at the interface between digital software and the physical plant.

To build intuition for Field-Oriented Control, consider pedaling a bicycle. If you push straight down when the pedal crank is at the top (12 o'clock, the direct axis I_d), you produce zero rotational torque—you merely compress the crank arm, strain the bearings, and dissipate effort into useless heat. To propel the bicycle forward efficiently, you must apply force perpendicular to the crank when it is forward (3 o'clock, the quadrature axis I_q), where 100 percent of your effort converts into useful driving torque. In a three-phase brushless motor, stator coils exert magnetic forces on spinning rotor magnets; FOC continuously measures rotor angle with a high-resolution encoder, mathematically rotating the coordinate frame (dq -transformation) to ensure that commanded current is applied strictly at “3 o'clock” (I_q), while clamping parasitic direct current ($I_d \approx 0$) to eliminate wasted thermal dissipation.

At the microsecond layer, this quadrature current command I_q is scaled into an integer compare threshold and written into a hardware timer compare register. Inside the microcontroller silicon, a dedicated hardware counter increments at 200 MHz, counting from 0 to a fixed period value (such as 10,000 clock cycles for a 20 kHz PWM carrier). When the counter matches the compare register, dedicated silicon comparator logic flips the PWM gate drive pin in under 5 nanoseconds—completely independent of CPU software execution. Depending on the machine architecture, that hardware register acts as a digital-to-analog converter latch holding a commanded reference voltage, a pulse-width modulation (PWM) compare register setting inverter duty cycle, or a serial control word configuring a gate driver.⁷ At that exact memory address, informational abstraction ends. Modulating the gate voltages across power MOSFETs draws current from the high-voltage DC bus into the stator windings, generating Lorentz forces that accelerate the rotor against its reflected load inertia.

Definition 2.2: The causal boundary

Causal boundary is the physical and electrical interface where digital computational bits convert into physical forces, currents, or flows.

⁷ | **Power Inverter Timers:** Advanced microcontroller timer peripherals generate complementary pulse-width-modulated (PWM) drive signals with hardware-enforced dead-time insertion. Dead-time guarantees that high-side and low-side power switches in an inverter half-bridge never conduct simultaneously, preventing catastrophic shoot-through short circuits across the DC bus.

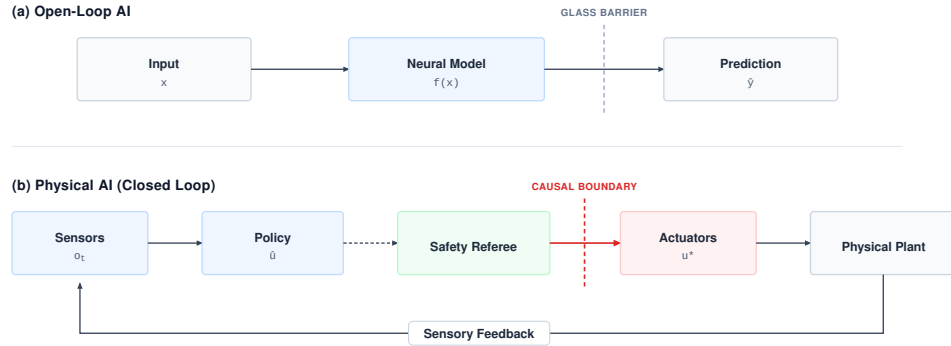


Figure 2.3: The closed causal loop: Advisory output (top) stops at an informational interface. In physical AI (bottom), permitted model proposals reach hardware registers, where electrical current produces force and motion. Those actions alter the environment and subsequent sensor observations.

1. **Significance:** Before actuation, a proposal can be rejected without applying physical work. After actuation, actions can alter the environment, dissipate heat ($I^2 R$ resistive heating, kinetic friction), and reshape subsequent sensory observations.
2. **Distinction:** A staged or transactional digital proposal can be rejected before it commits; a committed digital operation may also have external effects. Once an actuator applies force, canceling the command cannot undo the physical work already performed.
3. **Common pitfall:** Assuming software exceptions or thread cancellations can arrest physical momentum. Once a PWM compare register latches, current flows and mass accelerates; arresting that motion requires dedicated stopping distance (d_{stop}) and friction braking.

⁸ | **Control State vs. Latent ML State:** In classical control, state $x(t)$ is strictly defined as the minimal set of physical coordinates and velocities required to compute future system trajectories. In contrast, deep learning literature frequently conflates state with high-dimensional latent activations or history windows. Relying on latent representations without observable physical derivatives (dq/dt) prevents control barriers from proving forward invariance.

Downstream: The non-smooth contact mechanics and torque saturation limits governing motor irreversibility are derived in section 4.4.

⁹ | **Automotive Serializer/Deserializer (SerDes):** High-speed physical-layer links (such as FPD-Link or GMSL) that transport uncompressed, low-latency multi-gigabit video streams over lightweight coaxial or shielded twisted-pair cables. Combined with dedicated hardware trigger lines, SerDes interfaces ensure microsecond-synchronized shutter capture across the sensor ring. For bus specifications, see Appendix C.

Transitioning from digital state to physical force destroys the statistical foundation of classical machine learning. In offline vision, natural language processing, and tabular prediction, data collection is exogenous: the world generates a distribution of inputs, and the model processes them without altering the mechanism that created them. In a physical system, the environment evolves in a closed causal loop: the next physical state⁸ and sensor observation depend directly on the action produced by the prior model inference.

The sensory input stream is endogenous, generated by the machine's own past actions. When a policy produces an erroneous torque command, the joint rotates, the platform shifts, and mounted cameras or lidar units swing into new viewpoints. The machine drives itself into an unfamiliar region of the state space where training data is sparse, increasing the probability of subsequent inference errors and compounding the divergence from nominal operation. This architectural divergence is illustrated in figure 2.3, contrasting advisory output with the closed causal loop of Physical AI crossing the actuator register boundary.

Because physical mass possesses inertia and computation takes finite time, latency in physical AI translates directly into physical displacement. As shown on the autonomous testbed in figure 2.4, real-world robotic systems rely on dense multi-modal sensor suites—including spinning LiDAR scanners, camera arrays, and radar units—that continuously stream high-bandwidth telemetry to onboard processors. Every stage in this pipeline—from photodiode exposure and sensor bus serialization,⁹ to neural inference on accelerator silicon and packet transmission over deterministic fieldbuses—consumes precious milliseconds. During this aggregate delay window, the machine cannot incorporate new sensory information and continues moving along the trajectory dictated by its existing momentum.

🔍 Systems Perspective 2.1: Latency is paid in millimeters

In advisory machine learning, computational delay only frustrates a user; in physical AI, latency is paid directly in physical displacement. At velocity v , an unbudgeted delay Δt consumes unguided motion that no downstream neural capacity can ever recover.

As a concrete example, consider a mobile platform traveling at a modest walking speed of 2.0 m/s. If the perception pipeline, neural policy inference, and motor controller communication consume a combined latency of 150 ms, the platform traverses $(2.0 \text{ m/s}) \times (0.15 \text{ s}) = 0.30 \text{ m}$ (30 cm) before the first motor response can occur. During those 30 cm, the vehicle is entirely blind to any environmental change that occurred after the initial sensor exposure. If an obstacle enters the path 20 cm ahead during that inference window, no amount of model capacity can alter the outcome.

This trade-off between computational delay and physical clearance is formalized by the **Kinematic Roofline Nomogram**. In classical computer systems architecture, the Roofline Model (Williams et al. 2009) visually relates operational arithmetic intensity (FLOPs/byte) to bound achievable throughput by either peak memory bandwidth or peak execution throughput. In physical AI, the Kinematic Roofline couples neural deliberation latency τ_{compute} and platform velocity v to bound physical stopping clearance against maximum actuator braking deceleration a_{max} .

Under emergency braking, total stopping distance d_{stop} decomposes into an informational transport term and a mechanical dissipation term:

$$d_{\text{stop}}(v, \tau_{\text{compute}}) = \underbrace{v \cdot \tau_{\text{compute}}}_{d_{\text{reaction}}} + \underbrace{\frac{v^2}{2a_{\text{max}}}}_{d_{\text{braking}}}$$

The reaction term d_{reaction} is strictly non-negotiable: during delay τ_{compute} , zero braking force can be applied because the sensor exposure has not yet propagated through neural inference and the fieldbus to the brake calipers. As detailed in table 2.1 across representative embodied platforms, for agile robots and high-speed vehicles, computational latency consumes an alarming proportion of the total stopping envelope.

Table 2.1: Kinematic roofline nomogram: Stopping clearance trade-offs across embodied platforms. At 200 ms inference latency, an autonomous passenger car travels four additional meters—an entire vehicle length—before brakes physically engage.

Physical Platform & Velocity v	Braking Decel a_{max}	Mechanical Braking d_{braking}	$\tau = 10 \text{ ms}$ (MCU Reflex)	$\tau = 50 \text{ ms}$ (Fast Policy)	$\tau = 200 \text{ ms}$ (Multimodal VLA)	Latency Fraction at 200 ms
Humanoid / Quadruped (1.0 m/s = 3.6 km/h)	4.0 m/s ² (0.41 g)	0.125 m (12.5 cm)	0.135 m (+1.0 cm)	0.175 m (+5.0 cm)	0.325 m (+20.0 cm)	61.5% of total stop
Warehouse AMR (2.5 m/s = 9.0 km/h)	3.5 m/s ² (0.36 g)	0.893 m (89.3 cm)	0.918 m (+2.5 cm)	1.018 m (+12.5 cm)	1.393 m (+50.0 cm)	35.9% of total stop
Urban Delivery Bot (6.0 m/s = 21.6 km/h)	4.5 m/s ² (0.46 g)	4.000 m	4.060 m (+6.0 cm)	4.300 m (+30.0 cm)	5.200 m (+120.0 cm)	23.1% of total stop
Autonomous Vehicle (20.0 m/s = 72.0 km/h)	7.0 m/s ² (0.71 g)	28.57 m	28.77 m (+20.0 cm)	29.57 m (+100.0 cm)	32.57 m (+400.0 cm)	12.3% of total stop

Just as an architect cannot resolve a memory-bound arithmetic pipeline by adding more ALU execution units, an autonomous vehicle systems engineer cannot resolve an emergency stopping clearance violation by adding model parameters if doing so increases inference latency τ_{compute} . A heavier foundation model that adds 150 ms of computation consumes an extra 3.0 meters of physical travel at highway speeds—distance that no friction brake or collision-avoidance logic can ever claw back.



Figure 2.4: LiDAR sensor ingress: A 64-beam spinning optical LiDAR assembly mounted aboard an autonomous research testbed. In physical AI, sensor streams delivering over 1.3 million points per second must be serialized over real-time hardware interfaces, synchronized with sub-microsecond timestamps, and processed through neural perception backbones within strict latency deadlines; every millisecond of unbudgeted transport or compute delay translates directly into unguided physical displacement. (Source: Steve Jurvetson, CC BY 2.0).

¹⁰ | **Thermodynamic Limits of Computation:** At a fundamental level, Landauer (1961) established that erasing logical bits dissipates at least $k_B T \ln 2$ of thermodynamic entropy. In physical AI systems, however, the dominant irreversibility is macroscopic: motor inverters draw tens of Amperes from battery packs, dissipating kilowatt-scale Joule heat ($I^2 R$) and imparting physical momentum to moving linkages that cannot be recalled.

Once digital commands transition into mechanical work, the state change becomes thermodynamically irreversible. The physical world provides no rollback mechanism.¹⁰ When an actuator controller applies current to a motor, electrical energy transfers from the battery into kinetic energy in mechanical linkages or thermal energy dissipated through resistive Joule heating ($I^2 R$) in the stator windings. If an incorrect joint torque drives a manipulator into a rigid steel fixture, the kinetic energy converts into plastic deformation of the tool, mechanical strain on the gearbox teeth, and heat. No software interrupt can pull that kinetic energy out of the crushed structure or restore sheared gear teeth to their original geometry.

Table 2.2 contrasts Physical AI’s core properties against advisory machine learning and classical control theory.

Table 2.2: Comparison matrix of computing paradigms across the causal boundary: Systematic comparison of Advisory Digital Machine Learning (left), Classical Robotics & Control Theory (middle), and Physical Artificial Intelligence (right) across operational substrate, model formulation, feedback dynamics, execution cadence, actuation authority, failure semantics, and safety certification primitives.

Architectural Dimension	Digital Machine Learning (Advisory)	Classical Robotics & Control	Physical Artificial Intelligence (Closed-Loop)
Operational Substrate	Decoupled virtual memory buffers (GPU VRAM, DRAM, web endpoints)	Continuous analog state space governed by differential equations	Hybrid: high-dimensional neural representations mapped to analog physical actuators
Model Formulation	High-capacity statistical approximators (LLMs, ViTs, diffusion policies)	Analytically specified transfer functions and deterministic state-space matrices	Foundation policies paired with deterministic runtime safety filters
Feedback Dynamics	Exogenous and static; independent and identically distributed samples	Continuous deterministic state feedback with Gaussian sensor noise	Endogenous, non-stationary closed loop; actions shape future observations
Execution Cadence	Throughput-optimized batching; soft or unbounded latency budgets	Hard real-time single-rate loops on microcontrollers (100–1000 Hz)	Asynchronous multi-rate cadences: System 2 (1 Hz), System 1.5 (50 Hz), System 1 (1000 Hz)

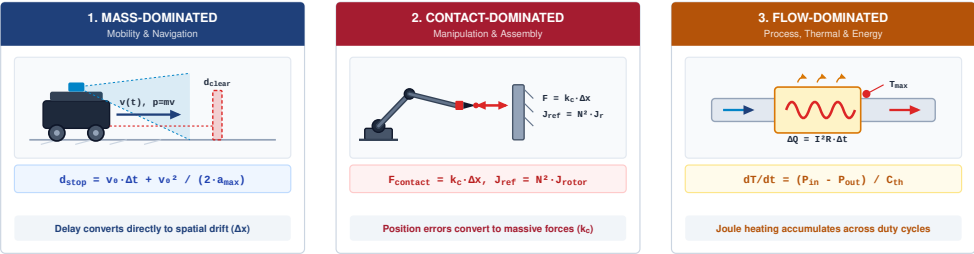


Figure 2.5: Machine archetypes of Physical AI: The three foundational physical embodiments classified by their primary binding conservation laws: (a) mass-dominated mobility (Class 1) governed by momentum and spatial clearance; (b) contact-dominated manipulation (Class 2) governed by mechanical impedance and structural yield stress; and (c) flow-dominated process plants (Class 3) governed by thermal capacitance and transport lag. Complex embodiments such as humanoids serve as the integrative capstone uniting all three classes on a single chassis.

Architectural Dimension	Digital Machine Learning (Advisory)	Classical Robotics & Control	Physical Artificial Intelligence (Closed-Loop)
Actuation Authority	Advisory; sandboxed APIs, display buffers, or human-in-the-loop validation	Hardcoded feedback directly driving power inverter timers and PWM registers	Decoupled Proposal–Permission architecture; untrusted proposals gated by MCU safety referee
Failure Semantics	Outputs may be dismissed or transactions reversed before external action; downstream harm remains possible	Actuator saturation, tracking error, or loss of stability	Irreversible thermodynamic work, plastic deformation, kinetic collision, or stator overheating
Safety Assurance	Offline statistical loss minimization on validation splits	Formal mathematical stability proofs (Lyapunov functions, gain/phase margins)	Layered runtime invariant enforcement, forward-invariant barrier filters, and fault injection

2.3 The Four Machine Archetypes

Once digital predictions cross the causal boundary (section 2.2) into physical force, they interact with machines of every conceivable shape and scale. An autonomous delivery drone navigating turbulent wind gusts seems to share nothing with a six-axis robotic arm assembling an automotive transmission, and neither appears related to a containerized battery management plant regulating liquid coolant. How can a single systems engineering curriculum encompass such diverse physical systems without becoming a collection of ad-hoc recipes?

The key is to look past cosmetic form factors to the underlying physical conservation laws. Every embodied AI system falls into three foundational physical classes based on the dominant conservation law governing its actuators (figure 2.5, figure 2.6): moving mass across space, applying force through physical contact, or regulating continuous fluid and thermal flows. These three classes culminate in a fourth, integrative archetype—the humanoid and multi-physics embodiment—where all three conservation regimes operate simultaneously on a single chassis. This four-archetype lens allows system architects to determine which resource budget exhausts first during a software stall or anomaly, designing the real-time nervous system accordingly. Table 2.3 sets those budgets beside the failure each one ends in and the gate that has to catch it.

Table 2.3: Binding budget by archetype: Each class exhausts a different resource first when software stalls, which is what determines the enforcement mechanism the nervous system must carry. The gates are design responses to the failure mode in the same row, not certified performance claims.

Archetype	Exemplars	First budget to exhaust	Failure mode	Safety gate
Mass-dominated	AMRs, AVs, drones, mobile bases	Stopping distance and freshness (Δt)	Kinetic collision; momentum breach	MCU dynamic braking referee at 1 kHz
Contact-dominated	Manipulators, grippers, humanoids	Actuator current and gearbox stress	Gear tooth shear; workpiece damage	Torque-derivative clamp and C^2 profiler

Archetype	Exemplars	First budget to exhaust	Failure mode	Safety gate
Flow-dominated	Thermal loops, inverters, bioreactors	Thermal capacity and phase timing	Thermal runaway; insulation blowout	Hardware thermal observer and interlock

2.3.1 Class 1: Mobility systems

Class 1 machines move mass across space, bound by momentum conservation and kinetic energy dissipation (figure 2.5 a, figure 2.6 a). In autonomous transport platforms, quadruped robots navigating rugged terrain (such as Boston Dynamics' Spot), mobile manipulators, and aerial drones, the actuator bridge applies force to translate or rotate a body with mass m at velocity v . Because kinetic energy $E_k = \frac{1}{2}mv^2$ cannot be instantaneously erased once delivered to the chassis—it must be physically scrubbed off through friction, aerodynamic drag, or regenerative braking—safety is primarily measured as remaining spatial clearance. When braking begins at maximum allowable deceleration $a_{\max}$, the physical floor for stopping distance follows the fundamental kinematic relation:

$$d_{\text{stop}} = v \cdot t_{\text{lag}} + \frac{v^2}{2a_{\max}}$$

where t_{lag} is the total sense-to-actuation transport and inference delay. In a mass-moving system, every millisecond of computational latency converts directly into unguided spatial displacement ($v \cdot t_{\text{lag}}$) as the machine coasts blindly, while the subsequent braking distance scales quadratically with velocity ($v^2/2a_{\max}$). The first boundary breached during a software stall or model misclassification is the physical distance to an external obstacle.

To see the physical consequences of this scaling, consider an autonomous mobile robot or quadruped such as Spot in a facility corridor with mass $m = 32$ kg or a warehouse transport base with $m = 250$ kg cruising at $v_0 = 1.8$ m/s (4.0 mph). The perception camera runs at 30 Hz (33.3 ms exposure and readout). The vision-language-action (VLA) navigation policy runs on an edge accelerator with an inference latency of 120 ms. Motor controller CAN bus communication consumes 15 ms. If the maximum allowable braking deceleration without tipping cargo is $a_{\max} = 2.0$ m/s², the total sense-to-actuation lag is $t_{\text{lag}} = 33.3 + 120 + 15 = 168.3$ ms ≈ 0.168 s. During this computational window, the unguided robot coasts $d_{\text{lag}} = v_0 \cdot t_{\text{lag}} = (1.8 \text{ m/s})(0.1683 \text{ s}) \approx 0.303$ m (30.3 cm). Bringing the physical mass to a complete halt requires an additional braking distance of $d_{\text{brake}} = v_0^2/(2a_{\max}) = 1.8^2/(2 \cdot 2.0) = 0.810$ m (81.0 cm), yielding a total required stopping clearance of $d_{\text{stop}} = 0.303 + 0.810 = 1.113$ m. If an engineer upgrades to a larger foundation model that increases inference latency from 120 ms to 320 ms, total lag expands to $t'_{\text{lag}} = 368.3$ ms, driving unguided displacement to $d'_{\text{lag}} = 0.663$ m and total stopping distance to $d'_{\text{stop}} = 1.473$ m—an additional 36.0 cm, representing a 32.3 percent expansion in required clearance. In narrow warehouse aisles, that 200 ms computational delay directly consumes the entire safety margin.



Checkpoint 2.1: The inelastic latency budget

Before analyzing contact-dominated mechanics, verify your understanding of how computational latency translates into physical space:

- ☐ **Sense-to-actuation lag:** Can you explain how camera exposure, neural inference latency, and fieldbus serialization combine into an unguided coasting displacement $d_{\text{lag}} = v \cdot t_{\text{lag}}$?
- ☐ **Nonlinear stopping scaling:** Can you describe why doubling the vehicle velocity quadruples the physical braking distance while linearly expanding the reaction distance?
- ☐ **Model scaling trade-offs:** Can you distinguish between a software system where extra inference latency merely delays screen rendering and a physical robot where extra latency directly consumes physical safety margin?



Figure 2.6: Embodied archetypes in Physical AI: Field deployments representing the three foundational physical conservation regimes: (a) mass-dominated mobility exemplified by Boston Dynamics’ Spot quadruped negotiating unstructured terrain; (b) contact-dominated manipulation exemplified by the Franka Emika Panda collaborative arm regulating fine assembly contact forces; and (c) flow-dominated process plants exemplified by containerized utility-scale battery energy storage systems (Tesvolt BESS) governing high-capacity thermal dissipation and liquid cooling loops. Complex embodiments such as humanoids integrate all three regimes into a single chassis. (Attribution: Wikimedia Commons, CC BY-SA 4.0).

2.3.2 Class 2: Manipulation systems

Class 2 machines apply mechanical force through contact, limited by actuator torque saturation, environmental stiffness, and current loop bandwidth—how quickly the motor controller senses and regulates electrical current to control torque (figure 2.5 b, figure 2.6 b). In articulated manipulators, precision assembly fixtures, and robot learning workcells executing contact-rich insertion and grasping, the end-effector interacts directly with a rigid or compliant workpiece. Unlike a mobile platform, an arm in contact with a surface cannot resolve unexpected displacement by rolling forward. When an actuator commands position into a rigid boundary, physical displacement is blocked, and the interaction acts like a compressed spring governed by mechanical impedance. By Hooke’s law, the contact force generated by a spatial penetration error Δx against an environment of stiffness k is $F = k\Delta x$.¹¹

Contact stiffness can make a held motion command expensive before the high-level planner returns. Consider an illustrative 6-axis arm whose low-level controller keeps driving at $v = 0.20$ m/s while the high-level process stalls for $\Delta t = 75$ ms. Assume a compliant fixture with effective stiffness $k = 2.5 \times 10^5$ N/m, enough actuator torque to maintain motion through contact, and a transmission whose reflected allowable contact force is $F_{\text{yield}} = 1500$ N. Under these assumptions, elastic deflection reaches $\Delta x = v \cdot \Delta t = (0.20 \text{ m/s})(0.075 \text{ s}) = 0.015 \text{ m} = 15.0 \text{ mm}$, and the spring model gives $F = k \cdot \Delta x = (2.5 \times 10^5 \text{ N/m})(0.015 \text{ m}) = 3750 \text{ N}$. Motor saturation, compliance elsewhere, or a controller that stops on stale commands would change that result. The assumed force limit corresponds to $\Delta x_{\text{max}} = 1500 \text{ N} / (2.5 \times 10^5 \text{ N/m}) = 6.0 \text{ mm}$ and a $t_{\text{deadline}} = \Delta x_{\text{max}} / v = 0.006 \text{ m} / 0.20 \text{ m/s} = 0.030 \text{ s} = 30 \text{ ms}$ contact-response budget. A force sensor or motor-current estimate must detect contact and bring commanded motion within the limit before this budget expires; it does not by itself establish the required current-loop period.

¹¹ | **Rigid Contact Mechanics:** Unilateral physical contact imposes kinematic constraints governed by Linear Complementarity Problems (LCPs), where sudden collision produces impulsive contact forces and infinite instantaneous stiffness. Physical controllers prevent high-gain instability and chatter by implementing virtual impedance control or compliant proxy models. For the mathematical formulation of contact dynamics, see Appendix B.

Checkpoint 2.2: Contact stiffness and operating system stalls

Before exploring continuous flow and energy archetypes, verify your understanding of contact mechanics and execution timing:

- ☐ **Hookean force rise:** Can you explain why contact with stiff environments ($k \geq 10^5$ N/m) converts sub-millimeter position errors into structural shock loads that exceed gearbox yield limits?
- ☐ **Operating system latency vs. reflex loop:** Under what held-command, compliance, and torque assumptions does a 75 ms stall exceed the illustrative contact-force limit?
- ☐ **Asymmetric authority:** Can you distinguish the operational responsibilities of a 20 Hz neural motion planner from a 1 kHz bare-metal torque-clamping reflex loop during unexpected collision?

2.3.3 Class 3: Process and energy systems

Beyond moving linkages and contacts, Class 3 machines regulate continuous fluid, thermal, or electrical power flows (figure 2.5 c, figure 2.6 c). A cooling loop must remove heat as fast as the process generates it. With net heating power P and thermal capacitance C_{th} , a stalled pump produces a temperature rise $\Delta T = \int P/C_{th} dt$ until flow resumes; a transport delay adds time before new coolant reaches the hot component. For an illustrative constant $P = 1$ kW and $C_{th} = 10$ kJ/K, the component warms at 0.1 K/s, so a 5 K margin provides about 50 s before the assumed limit. A grid-tied inverter faces a different budget: on a 60 Hz network, an uncompensated 1 ms phase-estimation delay corresponds to 21.6° of phase error. Whether that error causes unacceptable current depends on the inverter's impedance and protection design; the 8.33 ms half-cycle is not itself a universal failure deadline. In each plant, the permitted software delay follows from the physical margin and the available protection, rather than the clock period alone.

Downstream: Preemption of proposal streams across the causal boundary is arbitrated by the supervisor state machine in section 28.3.



Checkpoint 2.3: Thermal capacitance and grid phase alignment

Before analyzing multi-physics capstone systems, verify your understanding of continuous flow dynamics and thermal/electrical budgets:

- ☐ **Thermal capacitance time constants:** Can you explain how thermal inertia and transport lag dictate temperature rise ($\Delta T = \int \frac{P}{C_{th}} dt$) when coolant pumps or valve scheduling loops stall?
- ☐ **Grid phase alignment deadlines:** Can you convert a 1 ms delay on a 60 Hz grid into phase error and explain what additional impedance and protection data would set a current limit?
- ☐ **Continuous flow vs. kinematic state:** Can you contrast systems bounded by spatial clearance or contact force against process systems bounded by mass-energy balances, enthalpy transport, and phase synchronization?

2.3.4 Class 4: Integrative capstone systems

A central question for physical AI architects is: *Where do complex general-purpose embodiments, such as humanoid robots, fit into this taxonomy?*

A humanoid robot is not a simple variation of a single machine class. Rather, it constitutes **Class 4: The Integrative Capstone Archetype**: a system where all three foundational physical regimes operate and contend simultaneously on a single, tightly constrained mobile chassis:

1. **Class 1 Dynamics in Locomotion:** Underactuated bipedal balancing, dynamic footstep placement, momentum conservation ($m\mathbf{v}$), and ground contact slip during walking and stair climbing require continuous spatial clearance and kinetic energy management.
2. **Class 2 Dynamics in Manipulation & Contact:** Bimanual end-effectors, multi-fingered grasping, and rigid contact transitions with environmental workpieces subject kinematic chains to high shock loads ($F = k\Delta x$), joint torque saturation, and transmission yield stress during physical interaction.
3. **Class 3 Dynamics in Actuation & Power Delivery:** Driving 20 to 50 high-torque actuators simultaneously from a single onboard DC battery pack pushes inverter thermal dissipation ($I_{\text{phase}}^2 R_{\text{phase}}$) to its limits. Rapid simultaneous current spikes (dI_{bus}/dt) across the parasitic inductance (L_{bus}) of internal wiring harnesses induce severe inductive voltage droop ($\Delta V = L_{\text{bus}} \frac{dI_{\text{bus}}}{dt}$). This transient bus sag can brown out onboard coprocessors precisely during leg impact absorption or high-torque disturbance recovery.

Systems Perspective 2.2: The humanoid crucible

A humanoid couples stopping clearance, contact force, and electrical power on one chassis. When a hand catches a load, the contact force can demand a balancing step whose motor currents also draw down the shared supply:

- **Momentum–Contact Coupling (Classes 1 & 2):** When a 65 kg bipedal robot catches an unexpected 12 kg payload with its hands, the sudden contact force (Class 2) introduces a high-torque disturbance ($> 180 \text{ N} \cdot \text{m}$) that immediately perturbs its inverted-pendulum center of mass. To avoid tipping, the underactuated floating base must synthesize a dynamic recovery step within 180 ms, converting a contact problem into a momentum-braking problem (Class 1).
- **Kinematic–Thermal Contention (Classes 1, 2 & 3):** That single dynamic recovery step demands peak current bursts ($> 45 \text{ A}$ per phase) across hip and knee actuators. In an enclosed humanoid torso and leg casing without external cooling fans, continuous locomotion generates rapid $I_{\text{phase}}^2 R_{\text{phase}}$ Joule heating ($\Delta T = \frac{P \cdot t}{C_{\text{th}}}$), pushing winding temperatures toward insulation limits (125°C) and forcing the nervous system to derate available torque precisely when maximum authority is needed to sustain balance.
- **Electrical Bus Transients ($\Delta V = L_{\text{bus}} dI_{\text{bus}}/dt$):** Driving thirty to fifty high-bandwidth actuators simultaneously from a single internal 48 V lithium-ion bus creates sharp current transients ($dI_{\text{bus}}/dt > 100 \text{ A/ms}$). The parasitic inductance of dense wiring harnesses creates voltage droops that can brown out the vision-language deliberate coprocessor unless power domains and real-time safety microcontrollers are physically decoupled.

Throughout this textbook, every foundational principle—from real-time fieldbuses and proposal-permission decoupling to Control Barrier Functions and silicon placement—is first isolated across Classes 1, 2, and 3, and then tested against the Humanoid Crucible at the end of each stage.

While a single-class machine (such as a wheeled AMR or a fixed industrial arm) isolates its physical budget to a single dominant conservation law, a humanoid robot forces the systems architect to manage **cross-class coupled contention**. When a bipedal robot catches a falling heavy object, the contact force (Class 2) shifts its center of mass, requiring an instantaneous balancing recovery step (Class 1) that draws peak phase currents threatening to collapse the electrical bus and overheat the knee actuators (Class 3). To quantify how electrical transients threaten cognitive processing, consider a bipedal humanoid powering its onboard edge compute (12.0 V regulated rail with a 10.8 V brownout reset threshold) and 12 joint actuators from a common 48.0 V battery pack. The high-current DC distribution path connecting the battery pack to the motor power stages has a lumped total loop inductance of $L_{\text{bus}} = 25.0 \mu\text{H}$, cumulative across pack interconnects, safety fuses, contactors, and chassis cabling. During a dynamic jump landing, the leg actuators simultaneously command a current surge from 10 A to 110 A in $\Delta t = 0.2 \text{ ms}$, generating a current slew rate of $dI_{\text{bus}}/dt = (110 \text{ A} - 10 \text{ A}) / (0.2 \times 10^{-3} \text{ s}) = 5.0 \times 10^5 \text{ A/s} = 500 \text{ A/ms}$. By Faraday's law of induction, the transient voltage droop across the bus is $\Delta V_{\text{droop}} = L_{\text{bus}} \cdot (dI_{\text{bus}}/dt) = (25.0 \times 10^{-6} \text{ H})(5.0 \times 10^5 \text{ A/s}) = 12.5 \text{ V}$, plunging the instantaneous bus voltage to $V_{\text{bus, min}} = 48.0 \text{ V} - 12.5 \text{ V} = 35.5 \text{ V}$. If the DC-DC step-down converter feeding the 12 V compute rail drops out when input voltage falls below 38.0 V, this landing spike instantly triggers an unrecoverable processor brownout reset and bipedal collapse. Physical AI demands physical power domain isolation: high-capacity neural computers cannot share unbuffered, inductive power rails with high-torque motor drives.

Checkpoint 2.4: Inductive rail droop and processor brownouts

Before applying the Physical AI scope test, verify your understanding of multi-physics interactions and electrical bus transients:

- ❑ **Lumped bus inductance:** Can you explain how sudden actuator current surges (dI_{bus}/dt) during dynamic locomotion generate inductive voltage droop ($\Delta V = L_{\text{bus}} dI_{\text{bus}}/dt$) across lumped loop inductance (interconnects, fuses, contactors, cable loops)?
- ❑ **Brownout vulnerability:** Can you describe how transient voltage sags on high-torque motor buses can reset digital coprocessors and trigger unrecoverable mechanical collapse?
- ❑ **Power domain decoupling:** Can you distinguish the architectural mechanisms required to protect digital computing boards from motor back-EMF spikes and inductive rail collapse?

For any Physical AI system, engineering the nervous system begins by determining which physical budget is exhausted first during a failure. In software systems, resource constraints center on processor cycles, memory allocation, and network bandwidth. In physical systems, the systems architect must identify whether spatial clearance, structural stress, or heat dissipation creates the critical timing deadline. In a high-speed vehicle or quadruped, spatial clearance fails within fractions of a second while motor winding temperatures remain safely below thermal limits. In an assembly press, mechanical stress exceeds the yield point of structural members within tens of milliseconds long before the chassis moves perceptibly or heats up. In a chemical reactor or battery pack, mechanical stress is negligible, but accumulated thermal energy breaches safe operational envelopes over several seconds. The shortest time to failure across these three domains defines the deterministic execution deadline that the nervous system must meet.

Yet understanding that a machine belongs to one of these three physical archetypes does not automatically mean it requires the full systems discipline of Physical AI. A classical hydraulic press or a deterministic PID drone autopilot manipulates mass and contact force, but operates entirely within classical control theory. To determine when a machine crosses into Physical AI, we must establish a rigorous diagnostic scope test.

2.4 The Physical AI Scope Test

Before deploying an embodied architecture, a system architect must determine if a machine requires the full systems engineering discipline of Physical AI. Without a rigorous scope boundary, engineering teams make two symmetric mistakes: they over-engineer classical deterministic machines with bloated neural models where closed-form PID control is provably optimal, or they dangerously under-engineer stochastic neural models by granting them unverified motor authority in safety-critical physical loops.

The scope of Physical AI is defined by the simultaneous intersection of three structural criteria (figure 2.7):

1. **Learned Statistical Model (Unspecifiable Policy):** The system relies on a policy, perception encoder, or world model acquired from data (e.g., neural networks, foundation models, diffusion policies (Chi et al. 2024)) whose input–output transfer function cannot be written in closed-form analytical equations and whose worst-case outputs cannot be mathematically bounded offline.
2. **Consequential Physical Feedback:** Actuator actions alter the material state of the physical environment, which in turn directly dictates what the sensors observe next under the governing laws of classical mechanics, thermodynamics, or electromagnetism.
3. **Delegated Physical Authority (System-Level Actuation):** A learned component supplies live proposals that an automated system can execute without synchronous human approval, including when an independent safety gate may modify or veto each proposal. The learned model need not write an actuator register itself; the system holds physical authority because an accepted proposal can reach one.

A closed feedback loop is necessary for physical interaction, but feedback alone does not qualify a machine for this domain. Classical proportional-integral-derivative (PID) controllers run at 1000 Hz across aerospace and industrial automation, regulating valve positions and motor torques through tight sensory feedback. Their feedback loops are continuous and physical, but their control laws are specified entirely by fixed mathematical formulas whose stability bounds can be proven analytically

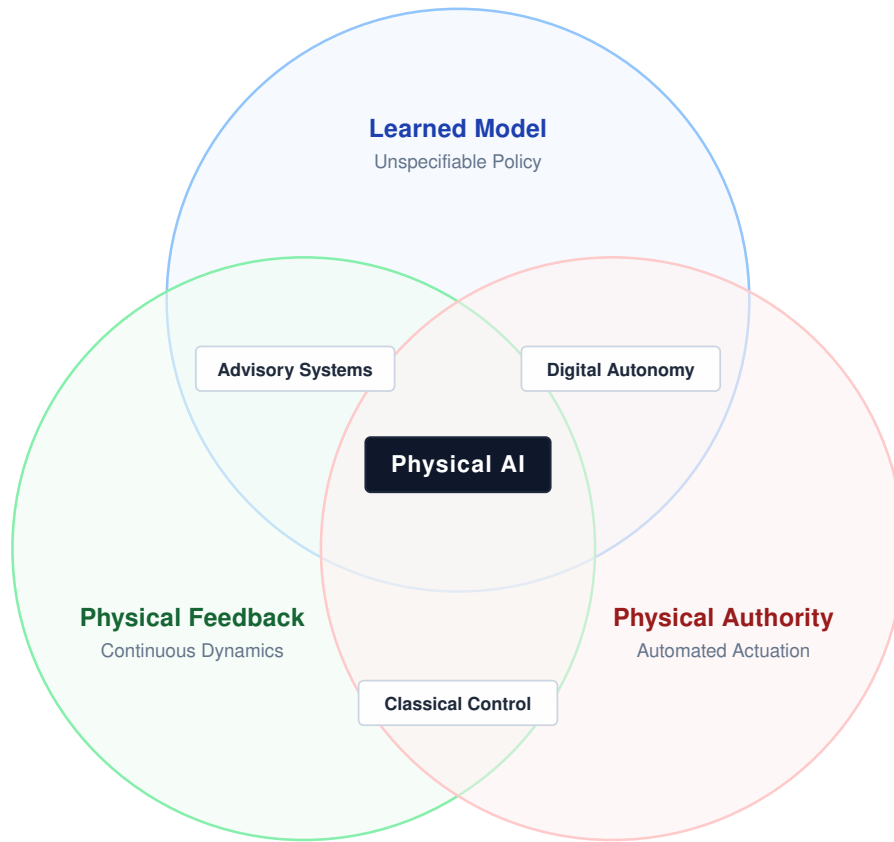


Figure 2.7: The Physical AI scope test: Physical AI combines a learned component, consequential physical feedback, and system-level authority to execute accepted proposals without human approval. An independent veto belongs inside this intersection because accepted proposals can still move the plant. The pairwise overlaps locate advisory systems, digital autonomy, and classical control.

from transfer functions. They hold physical authority and experience physical feedback, but they lack a learned model. Conversely, high-frequency algorithmic trading engines execute automated market orders under learned predictive models within microseconds. They combine learned policies with delegated transactional authority, but the feedback they receive is purely digital and informational. When an order executes, it alters an electronic order book rather than momentum, friction, or thermal energy. An unhandled stall in algorithmic trading costs financial capital, but it does not cause an inertia-driven collision with a structural fixture.

A learned model alone cannot define the field. An offline neural trajectory optimizer may compute paths for a manipulator, but if an engineer must inspect and explicitly load each path before execution, its outputs have no delegated runtime authority. An independent deterministic controller that automatically gates live learned proposals is different: its veto protects the plant while the accepted proposals still influence motion. Similarly, a neural network that only logs assembly-line defects or flags a display remains advisory because its output cannot automatically command the line.

Systems satisfying exactly two criteria highlight the structural boundary. A real-time driver drowsiness monitor uses a learned computer vision model to track eyelid movement and vehicle lane position, receiving continuous physical feedback from the vehicle state. Yet it lacks delegated physical authority because its only output is an auditory warning chime. If the vision model fails completely during an inference timeout or pipeline stall, the speaker simply emits no sound, but the physical steering rack and brake calipers remain safely under human control. An analog

bimetallic thermostat possesses delegated physical authority over a gas furnace valve and operates in continuous feedback with the thermal mass of a building, but its transfer function is a fixed thermodynamic property of two bonded metals rather than a learned representation. An automated trading bot possesses both a learned policy and delegated execution authority, but its actions propagate through network packets and databases without interacting with the physical laws of mass and motion.

The most demanding edge cases arise in shared autonomy, where physical authority is dynamically divided between a human operator and a learned model. In robotic surgery or advanced driver assistance, a human operator provides continuous steering or tool guidance while a learned policy injects corrections to filter hand tremors, avoid anatomical boundaries, or compensate for vehicle slip. These architectures satisfy both the learned component and physical feedback criteria. Whether they fall within the scope of Physical AI depends strictly on the temporal authority granted to the model. If the learned policy can command even 2.0 N of force or adjust steering angle without requiring human confirmation within the 200 ms threshold of human reflex latency, the model possesses delegated physical authority. During that 200 ms window, an erroneous model output or a delayed inference packet exerts real physical forces on the world—potentially causing a swerve or tissue damage—before the human operator’s neuromuscular system can physically react to intercede.

Evaluating an ambiguous machine requires a three-step decision rubric that tests each criterion independently (table 2.4). First, determine whether a learned component supplies the live policy or perception on which action depends. Second, determine whether those actions alter a physical plant and thus future observations. Third, determine whether the system can execute an accepted learned proposal without a person approving each action. A deterministic veto does not remove delegated authority; a mandatory human review of an offline plan does. When all three tests pass, the system operates as Physical AI, and its engineering must respect the causal boundary established in section 2.2.

Table 2.4: The Physical AI scope test decision matrix: Systematic evaluation of representative engineering systems across the three joint criteria (Learned Model, Consequential Physical Feedback, and Delegated Physical Authority), establishing the governing engineering discipline, limiting physical factors, and primary failure modes for each candidate.

System Candidate / Case	Criterion 1: Learned Model (Unspecifiable Policy)	Criterion 2: Consequential Physical Feedback	Criterion 3: Delegated Physical Authority (Automatic Gated Execution)	Scope Classification & Discipline	Limiting Physics & Critical Failure Mode
Autonomous Warehouse AMR (Class 1)	Yes (Learned navigation policy / costmap)	Yes (Kinetic momentum, chassis inertia, and wheel traction)	Yes (Accepted proposals reach motor control without human approval)	Physical AI	Kinetic impact; traction slip and odometry loss
Articulated Contact Manipulator (Class 2)	Yes (Diffusion action chunking for assembly (Zhao et al. 2023))	Yes (Mechanical contact impedance and reaction forces)	Yes (Safety-gated proposals drive live joint commands)	Physical AI	Structural yield stress exceedance; gearbox tooth shear
High-Speed Drone Classical Autopilot	No (Analytically tuned 1000 Hz PID / cascaded loop)	Yes (Aerodynamic lift, drag, and gyroscopic dynamics)	Yes (Direct PWM motor speed control)	Classical Control Theory	Control margin erosion; rotor thrust saturation
High-Frequency Market Maker	Yes (Deep transformer sequence predictor)	No (Digital order-book state; memory and network queues)	No (No physical actuation; orders execute automatically)	Digital Machine Learning / FinTech	Financial capital loss; market disruption
Advisory Driver Drowsiness Monitor	Yes (Facial landmark vision network)	Yes (Vehicle kinematics and driver physiology)	No (Auditory warning buzzer and dashboard LED only)	Advisory Decision Support	Human perception-reaction latency; zero physical authority

System Candidate / Case	Criterion 1: Learned Model (Unspecifiable Policy)	Criterion 2: Consequential Physical Feedback	Criterion 3: Delegated Physical Authority (Automatic Gated Execution)	Scope Classification & Discipline	Limiting Physics & Critical Failure Mode
Offline Trajectory Optimizer	Yes (Deep neural network for time-optimal path search)	No (Static CAD workcell model; offline batch execution)	No (Motion plan verified and loaded via human engineer)	Offline Optimization Tool	Numerical non-convergence; zero direct runtime plant coupling
Shared-Autonomy Surgical Assist	Yes (Learned haptic tremor filter / virtual fixtures)	Yes (Anatomical tissue compliance and reaction forces)	Conditional (Model injects forces within human reflex window)	Borderline / Shared Physical AI	Human neuromuscular reflex latency vs. tool force rise rate

Checkpoint 2.5: Applying the physical AI scope test

Before examining the four architectural tiers, verify your diagnostic ability to categorize ambiguous autonomous machines:

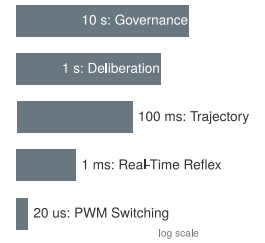
- ☐ **Three-criteria scope evaluation:** Can you evaluate whether an automated system qualifies as Physical AI by testing for the simultaneous presence of an unspecifiable learned model, consequential physical feedback, and delegated actuator authority?
- ☐ **Digital autonomy vs. physical embodiment:** Can you explain why a high-frequency algorithmic trading system executing automated market orders under a deep transformer policy falls outside the scope of Physical AI?
- ☐ **Shared autonomy reflex boundary:** Can you identify why a haptic surgical tool or lane-centering assist transitions into Physical AI whenever software actuation authority acts within the 150–250 ms human neuromuscular reflex interval?

2.5 The Four Architectural Tiers

Before exploring physical system failures or the silicon circuits that control them, we map how an embodied machine organizes compute and authority across the causal boundary. As introduced in figure 2.2, every physical AI system—whether a humanoid biped, an autonomous tractor, or an automated surgical console—is structured across four architectural tiers: the **Dynamical Body** (Layer 1), the **Real-Time Nervous System** (Layer 2), the **Cognitive Brain** (Layer 3), and the **Governance Envelope** (Layer 4).

Systems engineering organizes these tiers around **timescale separation**, **hardware interfaces**, and **contract boundaries** instead of treating them as an undifferentiated software stack:

1. **Timescale Separation Across the Hierarchy:** Embodied machines cannot execute as a monolithic control loop because physical dynamics and neural deliberation operate at irreconcilable cadences:
 - *Continuous Time (Layer 1: Dynamical Body):* The continuous physical plant, governed by classical mechanics, thermodynamics ($I^2 R$ Joule heating), and electromagnetism ($N^2 J$ reflected rotor inertia).
 - *Kilohertz Hard Real-Time (1000 Hz, 1 ms; Layer 2: Real-Time Nervous System):* Bare-metal microcontrollers (MCUs) and deterministic fieldbuses (CAN-FD, EtherCAT, TSN) running cycle-deterministic loops, sub-microsecond clock synchronization,¹² and low-level current control.
 - *Decahertz Deliberative Cadence (1–50 Hz, 20–1000 ms; Layer 3: Cognitive Brain):* Vision-language-action (VLA) foundation models (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023), diffusion policies (Chi et al. 2024), and semantic world models (Ahn et al. 2023) running on high-performance accelerators (GPUs, NPUs) with stochastic execution times.



Control loops span five orders of magnitude between cognitive deliberation and inverter switching.

¹² | **Precision Time Protocol (IEEE 1588):** A network protocol that uses hardware timestamping at the Ethernet PHY layer to synchronize clocks across distributed microcontrollers, sensors, and compute nodes to sub-microsecond precision. Sub-microsecond time synchronization is mandatory to eliminate temporal registration skew across high-frequency sensor fusion streams. For hardware clock synchronization details, see Appendix C.

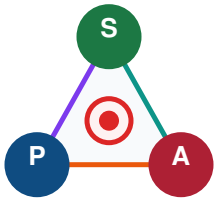
- *Sub-Hertz Governance Cadence* (0.1–1 Hz; *Layer 4: Governance Envelope*): Operational lease renewal, system health monitoring, and mission-level fault degradation.
2. **The Actuation and Sensing Interface (Layer 1 ↔ Layer 2):** The hardware boundary between the continuous plant and deterministic silicon. The nervous system samples motor phase currents, optical encoders, and tactile arrays via high-speed ADCs and fieldbus transceivers, converting continuous physical states into discrete digital observations. In the reverse direction, it writes duty cycles to memory-mapped PWM timer registers, switching power inverters to apply electromagnetic torque. This interface is governed strictly by the physical limits of the plant: peak phase current, thermal capacitance (C_{th}), and bus voltage droop ($L_{bus} dI/dt$).
 3. **The Proposal–Permission Privilege Boundary (Layer 2 ↔ Layer 3):** The architectural firewall separating stochastic neural inference from physical actuation. Learned models in Layer 3 are unprivileged proposal engines; an independent Layer 2 permission path must retain authority over actuator commands. In the running implementation, models emit candidate action chunks (Zhao et al. 2023) across asynchronous, lock-free ring buffers to microcontrollers that hold actuator-register authority. Those controllers check candidate trajectories against configured safety conditions, including Control Barrier Functions when their model and state assumptions hold (Ames et al. 2019), and apply a defined fallback if a proposal is stale or the neural engine hangs.
 4. **The Operational Lease and Assurance Contract (Layers 2/3 ↔ Layer 4):** The supervisory contract binding runtime execution to formal safety cases. Layer 4 does not command motor currents directly; instead, it dispenses time-bounded cryptographic or hardware watchdog operating leases. If runtime telemetry indicates that perception uncertainty exceeds certified bounds or an invariant is breached, Layer 4 revokes the lease, commanding Layer 2 to execute a Minimal Risk Maneuver (MRM) independently of Layer 3 deliberation.

This tiered organization operationalizes Rodney Brooks’s principles of situatedness and subsumption (Brooks 1986a; Brooks 1991): intelligent physical interaction does not require a high-latency model to oversee every microsecond of motion. Fast, uninhibited reactive layers operate directly on sensory feedback to protect the plant, while high-level deliberative goals modulate behavior across verified contract boundaries. Because physical state changes past the causal boundary cannot be rolled back, these contracts must enforce safety proactively at every interface.

2.6 The S-P-A Intersection Landscape

The closed physical loop couples three tasks: **Sense, Plan, and Act**. This **S-P-A triad** locates the interfaces where an engineer must turn uncertain observations into feasible motion and check how that motion changes the next observation.

Sense, plan, and act repeat as a feedback loop rather than ending after one command (figure 2.8). Each pair poses a physical decision:



The S-P-A intersection landscape: physical AI operates at the coupled triad core.

1. **Sense $\cap$ Plan ($S \cap P$): What the machine believes.** A delayed range observation may describe where an obstacle was, rather than where it is now. At 2 m/s, an extra 100 ms of sensing age consumes 0.20 m of stopping clearance; the planner must account for that loss (chapter 16, chapter 18).
2. **Plan $\cap$ Act ($P \cap A$): How the machine moves.** A proposed trajectory must fit the remaining clearance, motor current, and torque limits before execution (chapter 22, chapter 24).
3. **Act $\cap$ Sense ($A \cap S$): How the machine responds.** Contact or wheel slip changes the measured state before a slower policy can revise its plan; local feedback must constrain the next command (chapter 4, chapter 8).
4. **Sense $\cap$ Plan $\cap$ Act ($S \cap P \cap A$): The causal boundary.** The system must permit only proposals whose sensing age, motion plan, and physical response remain inside the same stopping or force budget (chapter 24).

The complete diagnostic playbook, troubleshooting matrix, and systems scorecard for the S-P-A taxonomy are developed in Appendix D.

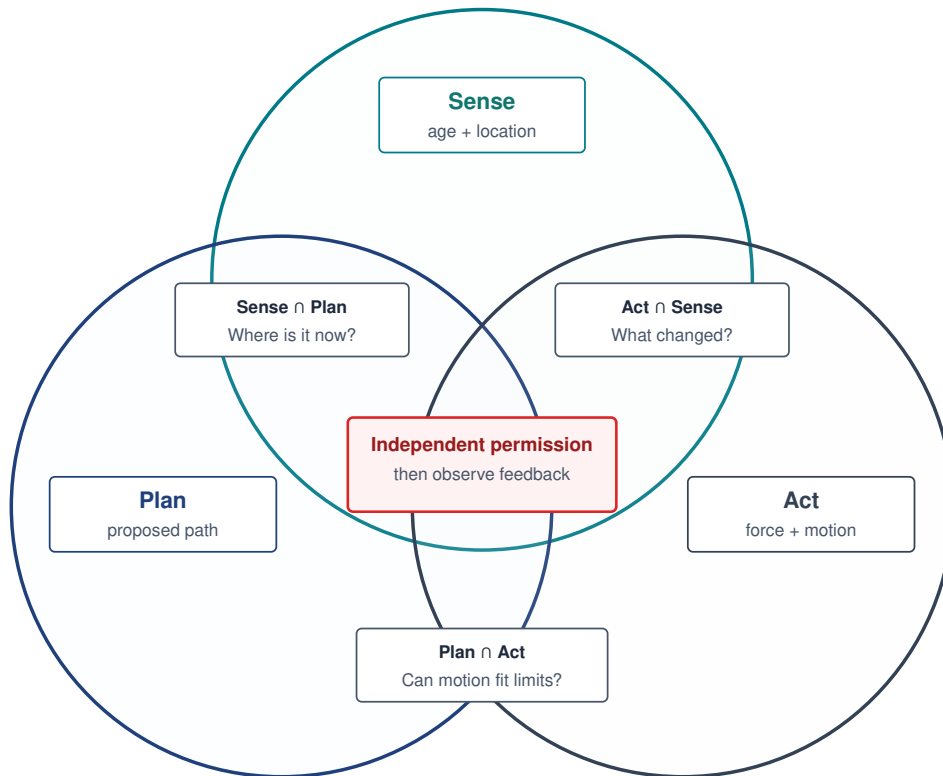


Figure 2.8: The S-P-A intersection landscape: Sense, Plan, and Act meet in three physical decisions: locate the object now, check that proposed motion fits limits, and measure what changed after acting. At their center, an independent permission path checks the shared physical budget before actuation; feedback informs the next proposal.

When any of these contract boundaries is missing or violated, the causal boundary breaks. To see this catastrophic failure mode in forensic detail, we examine one of the most famous real-world failures in autonomous systems history.

2.7 When the Boundary Fails

The boundary conditions of Physical AI are life-critical systems requirements. A physical crash can involve both an inaccurate prediction and a failure to respond when available evidence already warrants intervention. The Tempe investigation shows how object tracking, braking policy, operator reliance, and lost redundancy interacted.

2.7.1 The 2018 Tempe autonomous collision

On a March night in 2018, a modified Volvo XC90 operating Uber Advanced Technologies Group’s developmental automated driving system (ADS) struck and fatally injured a pedestrian pushing a bicycle across a road in Tempe, Arizona (National Transportation Safety Board 2019).¹³ At $T - 1.2$ s, the ADS recorded 43.2 mph (about 19.3 m/s). A 2100 kg mass at approximately 19.2 m/s carries about 387 kJ of kinetic energy; the mass and rounded speed serve only as an illustrative energy estimate.¹⁴

The vehicle carried lidar, radar, and cameras (figure 2.10). The ADS first detected the pedestrian 5.6 s before impact, but detection alone did not provide a correct path prediction (National Transportation Safety Board 2019).

Downstream: Physical proposal generation crossing into deterministic permission domains feeds the trajectory generator in section 22.1.

¹³ | **2018 Tempe Autonomous Collision:** The NTSB investigation (NTSB/HAR-19/03) found that changes in object classification removed tracking history from path prediction. The ADS recognized the emergency at $T - 1.2$ s and then suppressed braking for one second. Uber ATG had disabled the Volvo forward collision warning and automatic emergency braking systems over possible radar interference, removing a layer of redundancy (National Transportation Safety Board 2019).

14 | Kinetic Energy and Braking Dissipation: For the illustrative mass and speed, $E_k = \frac{1}{2}mv_0^2 \approx 387$ kJ. This estimate does not establish a stopping time or braking power; those require the available clearance, brake response, and deceleration history.

Downstream: The Tempe sensor toggling failure is formally prevented by the forward-invariant stopping envelopes in section 24.5.

15 | Tracking History: A path predictor can estimate motion from successive object locations. In the Tempe ADS, a changed classification caused the predictor to omit earlier locations; the NTSB report does not specify a Kalman-filter reset or a zero-velocity initialization (National Transportation Safety Board 2019).

2.7.2 Perception toggling and state erasure

Despite continuing to track the detected object, the ADS did not brake in response to it. The NTSB record shows classification changes among vehicle, bicycle, and other before $T - 1.2$ s (National Transportation Safety Board 2019).

When the classification changed, the ADS path predictor did not use the object's earlier locations; for an "other" object it could assume a static path.¹⁵ The resulting path predictions did not place the pedestrian in the SUV's lane until $T - 1.2$ s (National Transportation Safety Board 2019).

2.7.3 Latency suppression and kinematic depletion

At $T - 1.2$ s, the ADS recognized an emergency and entered its one-second action-suppression period. The design withheld braking while it checked the hazard or waited for operator intervention. At $T - 0.2$ s, suppression ended. The ADS then initiated a plan for gradual slowdown and sounded an alert; it did not request emergency braking. The NTSB notes that communication delay leaves unclear whether that slowdown began before the operator disengaged the ADS at $T - 0.02$ s (National Transportation Safety Board 2019).



Napkin Math 2.1: The kinematic stopping budget

- **Problem:** Given the reported one-second suppression period and speed at its end, could idealized full braking beginning at $T - 0.2$ s stop the vehicle before the impact point?
- **Assumptions:** Approximate speed near hazard recognition as $v_0 = 19.2$ m/s, speed at $T - 0.2$ s as 18.1 m/s (reported as 40.5 mph), and idealized constant braking deceleration as $a_{\max} = 8$ m/s². Constant-speed travel estimates ignore the recorded change in speed and are not a reconstruction of actual vehicle displacement.
- **Suppression interval:** Over the reported 1 s, holding v_0 constant would give $d_{\text{lag}} = v_0 t_{\text{suppress}} = 19.2$ m. The actual vehicle was slowing for an approaching turn, so this is a simplifying comparison, not measured suppression travel.
- **Braking comparison:** From the rounded speed at $T - 0.2$ s, idealized stopping requires $d_{\text{brake}} = v^2 / (2a_{\max}) \approx 20.5$ m. Holding that speed for the remaining 0.2 s gives only about 3.6 m of travel to impact. Even immediate full braking at that late instant could not stop before the impact point under these assumptions. The ADS in fact planned gradual slowdown, not full braking (National Transportation Safety Board 2019).

Figure 2.9 places the recorded event times and that braking comparison on one axis. Reading the two panels together shows what the arithmetic alone leaves implicit, namely that the one-second suppression interval is spent in the only stretch of road where a full-braking decision could still have changed the outcome.



War Story 2.1: The missing safety referee (2018)

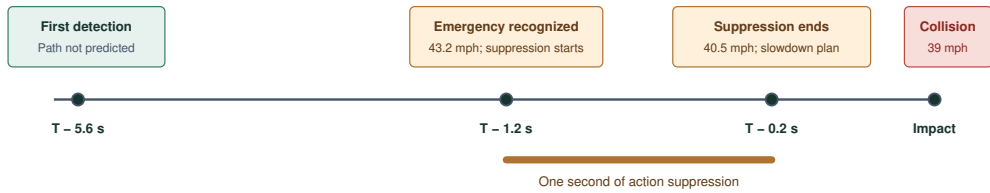
Context: On March 18, 2018, a developmental ADS on a 2017 Volvo XC90 struck and killed a pedestrian crossing a road in Tempe, Arizona (National Transportation Safety Board 2019). The vehicle carried lidar, radar, and cameras.

Mechanism: The ADS changed the detected object's classification among vehicle, bicycle, and other; its path prediction did not use earlier locations after a classification change. It recognized an emergency at $T - 1.2$ s, suppressed braking for one second, then planned gradual slowdown at $T - 0.2$ s. The design did not use maximum braking solely to mitigate an unavoidable collision. Uber ATG had disabled the factory collision warning and automatic emergency braking over possible radar interference (National Transportation Safety Board 2019).

Impact: The vehicle struck and fatally injured the pedestrian. The late gradual-slowdown plan could not recover the lost stopping opportunity.

Recorded ADS sequence

Events and speeds from NTSB/HAR-19/03, table on page 16; event spacing is schematic

**Conditional braking comparison at $T - 0.2$ s**

Assume 18.1 m/s and constant 8 m/s^2 deceleration; distances are illustrative, not recorded.



No counterfactual stop time is inferred from this comparison.

Figure 2.9: Tempe ADS event timeline: NTSB recorded first detection at $T - 5.6$ s, hazard recognition and the start of one-second action suppression at $T - 1.2$ s, and a gradual-slowdown plan at $T - 0.2$ s. The lower comparison is conditional: with an assumed constant 8 m/s^2 deceleration from the recorded 40.5 mph speed, idealized stopping distance at $T - 0.2$ s is about 20.5 m, far beyond the roughly 3.6 m of constant-speed travel remaining. The figure makes no claim about when an independent system could have detected the crossing path.

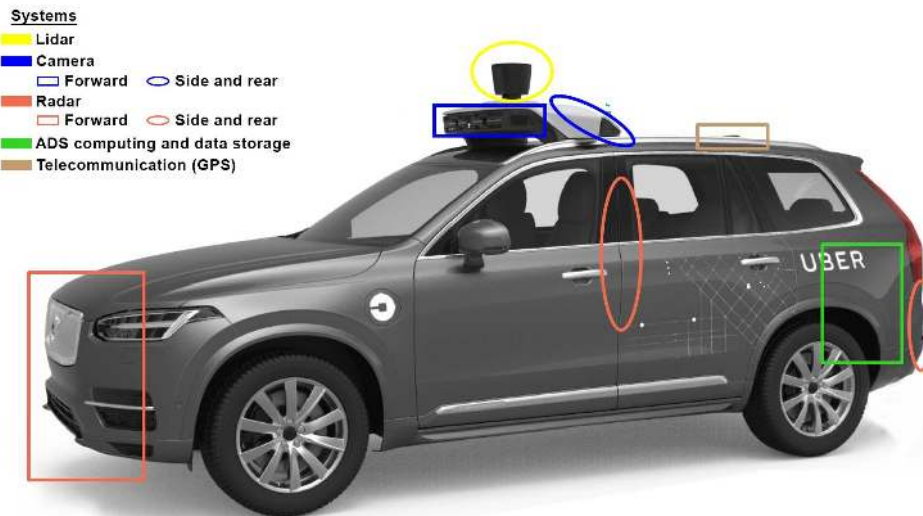


Figure 2.10: Tempe test vehicle sensor locations: The NTSB diagram identifies the developmental ADS vehicle's lidar, camera, radar, computing, and telecommunications locations. Uber ATG disabled the factory forward collision warning and automatic emergency braking during ADS operation, removing a redundancy layer (National Transportation Safety Board 2019). (Source: NTSB, public domain).

Response: The NTSB found that disabling the Volvo warning and braking systems removed a safety redundancy layer and recommended stronger safety risk management for testing automated vehicles (National Transportation Safety Board 2019).

Systems lesson: Classification uncertainty, a delayed response rule, and reliance on an inattentive operator can combine. A design needs timely hazard assessment and a response that can mitigate impact when avoidance is no longer possible; the investigation does not prescribe a particular processor or mailbox architecture.

The investigation supports a broader architectural requirement: a developmental ADS needs safety redundancy and a timely mitigation path even when perception or path prediction fails. An independent emergency-braking layer might have reduced the crash severity, and tests cited by the NTSB suggest that the factory system might have prevented or mitigated this collision. The report does not establish the trigger time or stop location of a hypothetical dedicated referee.¹⁶

While Tempe illustrates failure *before* impact—where classification toggling and an unvetted software suppression delay prevented timely braking—the October 2023 Cruise San Francisco robotaxi incident (Quinn Emanuel Urquhart and Sullivan, LLP 2024; Exponent, Inc. 2023) demonstrates the symmetric hazard *after* impact. In that event, the robotaxi executed an emergency stop when a pedestrian was launched into its path by an adjacent human-driven vehicle, but its subsequent post-collision behavioral fallback executed an automated pull-over maneuver without floorpan tactile or underbody sensing, dragging the pinned pedestrian 20 feet. Together, Tempe and Cruise establish the two failure poles across the causal boundary: unvetted software suppressing necessary braking, and unvetted fallback routines commanding mechanical motion without complete physical state awareness.

The forensic breakdown of these incidents reveals four universal bedrock laws that govern all physical machines across the causal boundary.



Checkpoint 2.6: Forensics of boundary and authority failures

Before studying the four bedrock physical laws, verify your understanding of the failure mechanisms diagnosed in the Tempe incident:

- ☐ **Classification changes and path prediction:** Can you explain how classification changes caused the ADS path predictor to omit earlier object locations, despite continued tracking?
- ☐ **Software suppression vs. kinematic clearance:** Can you distinguish the $T - 1.2$ s recognition time from the one-second suppression period and compare the idealized stopping distance with travel remaining at $T - 0.2$ s?
- ☐ **Safety redundancy:** Can you explain why disabling factory collision warning and emergency braking removed a layer of protection from a developmental ADS?
- ☐ **Dual-failure poles across the boundary:** Can you contrast the pre-impact failure mode (suppressing braking due to classification thrashing) with the post-impact failure mode (autonomous pull-over without tactile underbody verification)?

If any check reveals uncertainty, review the forensic autopsy in this section before proceeding to the four physical laws.

2.8 The Four Bedrock Laws

Four bedrock laws govern every machine whose nervous system translates compute cycles into physical action. These four laws map directly to the four architectural tiers of an embodied machine (section 2.5) and provide the organizational spine for the four parts of this textbook:



Systems Perspective 2.3: Physical causality and irreversibility

Physical work already performed cannot be erased by canceling the command. Actuation can dissipate energy, alter the environment, and reshape future sensory observations.

The first bedrock law concerns physical irreversibility after a command crosses the causal boundary. An actuator converts electrical power into motion, structural strain, or heat that a software

¹⁶ | **Safety-Critical Standards:** Functional safety and intended-functionality standards can inform the safety case for sensing, actuation, and fallback. Their applicability and required implementation depend on the system and jurisdiction; the NTSB Tempe report did not mandate an isolated microcontroller architecture.

rollback cannot remove. When an erroneous command accelerates a warehouse robot toward an obstacle, braking can counteract the momentum only by applying force over time and distance. If a manipulator plastically deforms a jig, reversing its setpoint cannot restore the metal. Safety must therefore be enforced before the damaging work occurs.

🔍 Systems Perspective 2.4: Endogenous data and compounding drift

A physical agent shapes its own future data distribution. Under bounded per-step costs, the worst-case expected cumulative cost of a behavior-cloned policy can grow as $\mathcal{O}(T^2\epsilon)$ over horizon T with single-step error rate ϵ . Offline accuracy alone cannot certify closed-loop success.

The second bedrock law governs how embodied policies learn and fail. In an offline validation set, a model's prediction does not change the next stored example. With an independently sampled per-step error rate ϵ , the expected number of errors over T such steps is $T\epsilon$. Deployment breaks that independence when actions change future observations.

In Physical AI, every torque command can shift joints, camera angles, and contact forces. Under a worst-case model in which one erroneous action can add at most one cost unit on each *later* step, summing those downstream opportunities gives the following upper bound (Ross et al. 2011):

$$\mathbb{E}[\text{Downstream Excess Cost}] \leq \sum_{t=1}^T (T-t)\epsilon = \epsilon \sum_{k=0}^{T-1} k = \epsilon \frac{T(T-1)}{2} = \mathcal{O}(T^2\epsilon)$$

For $T = 50$ and $\epsilon = 0.01$, this bound is 12.25 downstream cost units; immediate step errors would add at most $T\epsilon = 0.5$ under the same assumptions. Neither number predicts task-failure probability. Physical displacement can also compound if a disturbance persists: a constant lateral acceleration $\ddot{y} = a_\epsilon$ gives $y(t) = \frac{1}{2}a_\epsilon t^2$. The practical conclusion is that single-step validation accuracy cannot establish closed-loop performance; the deployed policy must be evaluated in the states its actions create. For the full covariate-shift derivation,¹⁷ see Appendix A.

🔍 Systems Perspective 2.5: Proposal–permission privilege

Learned models propose motion; an independent permission mechanism must be able to reject unsafe or stale proposals before actuation. Its response time and fallback follow the plant's physical budget. A dedicated microcontroller is one way to meet that requirement.

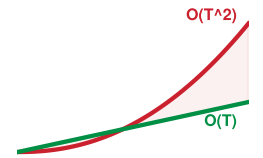
The third bedrock law separates proposal from permission. A learned policy proposes candidate trajectories; an independent mechanism must check physical envelopes, reject stale proposals, and control a safe fallback even if the proposer fails. The implementation may use separate silicon, hardware isolation within a chip, or another architecture that demonstrates bounded response and fault containment. At 1 m/s, a 50 ms delay shifts the machine 5 cm from the proposal's assumed position, so freshness is part of the permission test.

🔍 Systems Perspective 2.6: Governed authority and release

Physical AI systems are never 'generally safe'; operating authority is a decaying lease strictly conditioned on bounded evidence. A cherry-picked video demonstration is not evidence of safety; defensible release needs fault evidence, conditional barrier analysis where applicable, and an auditable case tied to the plant and operating domain.

The fourth bedrock law dictates that deployment readiness cannot be justified by empirical validation loss. In cloud computing, software reliability is evaluated statistically: an engineer measures mean accuracy, accepts an uptime target of 99.9 percent, and resolves rare corner-case crashes through automated restarts and rolling updates. In Physical AI, the tail of the error distribution manifests as mechanical destruction. Safe operation cannot be certified by average-case performance

¹⁷ | **Compounding Covariate Shift:** Offline per-step error does not by itself describe the states visited after deployment. Under bounded per-step cost, behavior cloning admits a worst-case $\mathcal{O}(T^2\epsilon)$ expected cumulative-cost bound; actual cost and task success require closed-loop evaluation (Ross et al. 2011).



A quadratic upper bound does not predict an episode's failure probability.

metrics gathered over finite training sets. Instead, tracking authority can be treated as a decaying lease: when fresh evidence or a required heartbeat is lost, an independent permission path revokes tracking and commands a state-matched fallback. A defensible release case bounds sensing, model error, actuator response, stopping clearance, and hardware faults for its declared operating domain. If those premises fail, the fallback limits harm according to its validated capabilities; a watchdog alone does not establish physical containment.

Together, these laws require a permission path whose timing and failure behavior can be checked independently of the learned proposer. The following dual-brain design implements that contract with separate execution domains.

2.9 The Dual-Brain Architecture

The **Proposal–Permission Architecture** requires an independent, bounded-latency path that checks learned proposals and controls a physical fallback when they fail. One concrete implementation partitions an application processor generating proposals at 10–50 Hz from a microcontroller checking them at 1000 Hz. This split is useful when the application processor’s scheduling and memory behavior cannot meet the plant’s response budget; the required invariant is independent veto authority, not a particular chip count or loop rate.

Authority over actuation must belong to a mechanism that can refuse execution even when the proposer is broken. This separation is foundational: in a pure software system, an unprivileged process can access application memory directly, as erroneous writes only cost compute cycles and memory bandwidth. In a physical machine, state changes move mass, dissipate heat, and expend momentum that cannot be recalled. In artificial intelligence research, Richard Sutton’s “Bitter Lesson” (Sutton 2019) famously observed that general-purpose methods leveraging massive search and compute scaling consistently overpower human-engineered heuristics over long historical horizons. In unconstrained digital computation, compute scaling is unaffected by mass or momentum. In Physical AI, however, Sutton’s scaling law collides head-on with what this textbook terms the **physical wall**: onboard compute is strictly bounded by battery energy density, inductive rail collapse ($\Delta V = L_{\text{bus}} dI_{\text{bus}}/dt$, where rapid current transients drop supply voltage below digital logic thresholds), thermal dissipation limits (15–60 W on a mobile edge SoC),¹⁸ sensor transduction noise, and reflected rotor inertia ($J_{\text{ref}} = N^2 J_{\text{rotor}}$, where transmission gear ratio N quadratically magnifies the motor’s reflected inertia). A model that scales parameter count without respecting the physical latency budget between perception and collision does not achieve higher physical intelligence—it simply crashes into obstacles before its forward pass finishes computing. Because a learned policy is an inductive statistical model whose worst-case outputs under distribution shift cannot be mathematically bounded, the neural network must function strictly as an unprivileged proposer. Routing learned model outputs directly to actuator registers couples the mechanical integrity of the machine to the unverified tail of a probability distribution.

For computer systems engineers, the example resembles hardware-enforced protection rings: the learned model proposes from an unprivileged domain, while a more trusted controller can refuse an inadmissible torque command before actuation. The analogy is limited because refusing a command also requires a controlled physical fallback; a page fault alone does not stop moving mass.

Training-time optimization cannot replace runtime invariant enforcement. In reinforcement learning and imitation learning pipelines, developers frequently attempt to enforce safety by augmenting the loss function with penalty terms, barrier losses, or negative reward shaping. These numerical penalties influence gradient descent during training, reducing the empirical frequency of undesirable states across the training distribution. But they provide zero hard guarantees at runtime. When the deployed policy encounters novel sensor noise, lens occlusion, or out-of-distribution state combinations, offline optimization cannot prevent the network from generating an unsafe control vector. Soft optimization incentives shape probability distributions, but physical safety requires deterministic boundary enforcement. Safety constraints must therefore live downstream of the model as an explicit, hard architectural gate positioned between the policy output and the actuator registers.

Combining inference and permission on one processor requires evidence that inference load, memory contention, and operating-system scheduling cannot delay the permission path beyond the plant’s deadline. If that evidence cannot be established, separate execution and power domains

¹⁸ | **Edge Accelerator**

Thermal Limits: On mobile chassis, accelerator power is capped between 15 W and 60 W by passive conduction limits and battery discharge endurance. Exceeding this envelope requires active liquid cooling loops whose pumps and radiators add parasitic mass that degrades platform payload capacity. Thermal dissipation limits force architects to partition heavy multimodal deliberation away from high-rate motor loops.

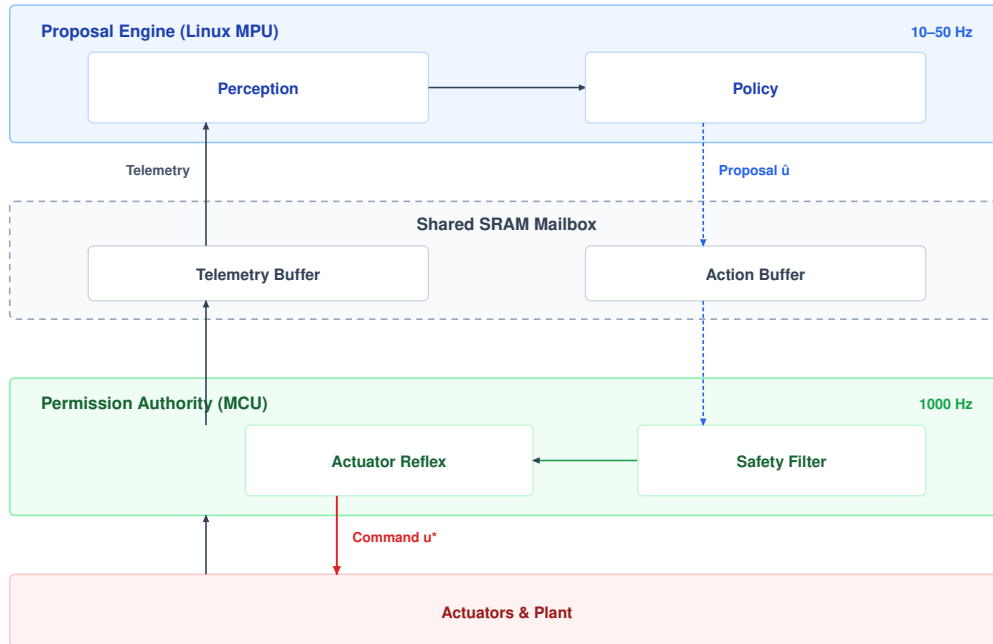


Figure 2.11: One dual-brain implementation: An application processor publishes learned proposals to a shared mailbox. An independently scheduled microcontroller checks freshness and physical limits before commanding motor current, and executes a defined fallback on stale or unsafe proposals. The indicated rates and mailbox are design examples whose suitability depends on the plant’s measured response budget.

provide fault containment. A 50 ms delay at 1 m/s consumes 5 cm of clearance; whether it is acceptable depends on the stopping envelope. A high-capacity model may need an application processor, while the permission path can remain small and predictable.

The following heterogeneous-silicon design illustrates one way to satisfy the contract (figure 2.11):

1. **The Cognitive Proposer (Linux MPU / Edge NPU):** Operates in unprivileged user space at a deliberative cadence (10–50 Hz). It executes high-capacity neural policies, multi-modal perception backbones, and trajectory generators. It possesses **zero direct electrical wiring to physical actuator registers**. It can only write candidate trajectory proposals into a shared SRAM mailbox or lock-free circular ring buffer across an inter-processor communication (IPC) channel.
2. **The Real-Time Permission Referee (Bare-Metal / RTOS MCU):** Operates on isolated, dedicated silicon at a hard real-time cadence (1000 Hz, 1 ms deadline, $< 50 \mu\text{s}$ jitter) with static, zero-allocation memory execution (dynamic heap allocation is strictly forbidden at runtime). It holds **exclusive physical authority** over timer compare registers, digital-to-analog converters, and gate driver circuits that govern phase currents to the motors. The MCU continuously checks physical safety invariants and stopping clearance. If the proposed command satisfies all physical constraints, the MCU latches the values to hardware; if the proposal is unsafe, stalls, or panics, the MCU vetoes the command and executes a deterministic physical fallback.

In this example, the processors exchange proposals through a shared SRAM mailbox with a sequence counter and memory barriers (DSB/DMB).¹⁹ The application processor marks a write in progress with an odd sequence value and commits an even value after publishing the payload; the microcontroller accepts only a consistent read. Timestamps, not sequence changes alone, determine proposal freshness. Other bounded communication channels can satisfy the same contract if the permission path never waits on the proposer and has a defined stale-data fallback.

The real-time referee enforces three mandatory gates before latching any command to the motor drivers when processing candidate actions from the shared mailbox:

¹⁹ | **Processor Mailboxes and Seqlocks:** A sequence counter lets a reader detect a concurrent write without blocking. A real-time reader must bound retries and reject inconsistent or stale data before its deadline. Shared SRAM, cache policy, and memory barriers are implementation choices for a particular processor pair; see Appendix C.

20 | **Control Barrier Functions (CBFs):** A control method for constraining the derivative of a candidate safe-set function (Ames et al. 2019). A CBF-QP can minimally alter a proposed command when the modeled state is viable, the input constraint is feasible, and sensing, computation, and actuation meet their validated bounds; sampled execution and model error require separate margins.

1. **Temporal Freshness Gate:** If the proposal timestamp is older than the strict freshness deadline ($\Delta t > \Delta t_{\text{fresh}}$), the referee drops the stale proposal.
2. **Physical Invariant Gate:** If a candidate violates a validated torque, velocity, stopping-clearance (d_{stop}), or current constraint, the referee projects it into the admissible input set or refuses it when no feasible input remains. A closed-form filter or bounded solver must complete within the measured permission deadline for this plant; its mathematics depends on the state estimate, actuator authority, and clearance assumptions.²⁰ For the rigorous mathematical formulation of these Control Barrier Functions (CBFs) and Quadratic Programs (QPs), refer to Appendix B.
3. **Deterministic Fallback Gate:** If the neural proposer crashes, hangs, or outputs numerical anomalies (NaNs/ $\pm \infty$), the referee revokes tracking and selects a validated state-dependent response. Regenerative braking is available only if the drive and energy sink can absorb the load; a passive centering response likewise needs plant-specific clearance and load evidence.

When a candidate action is refused, the permission authority invokes the state-matched response admitted for that operating condition. Refusal cannot consist of simply dropping the command and allowing actuator registers to float. If a mobile base moving at 2.0 m/s suddenly receives zero motor current without controlled deceleration, momentum drives the unconstrained mass into obstacles or induces wheel slip that invalidates odometry. To enforce these gates, the permission authority applies a graded hierarchy of interventions matched to the severity of the violation. A feasible candidate may be clamped against validated acceleration and jerk limits; numerical exceptions, timeouts, or heartbeat loss revoke tracking and trigger the separately validated stop or hold. Whether the plant reaches rest within clearance depends on available braking, load, friction, and response delay.

Understanding this dual-brain architecture is essential as it restructures how we develop, verify, and govern the machine. In traditional software services, engineering teams treat functional safety, cyber-security, and audit logging as three completely isolated disciplines handled by separate organizations across disjoint software layers. In Physical AI, the dual-brain proposal–permission boundary collapses all three concerns onto a single physical primitive.



Checkpoint 2.7: Dual-brain privilege separation and mailbox synchronization

Before examining safety convergence across software and hardware lifecycles, test whether this example satisfies the general permission contract:

- ☐ **Independent veto:** Can you explain how this microcontroller retains actuator authority if the application processor stalls, and name another isolation design that could meet the same response budget?
- ☐ **Mailbox mechanics:** Can you describe how sequence counters reveal an incomplete write and why timestamps and bounded retries are still needed for a 1000 Hz reader?
- ☐ **Three-gate referee intervention:** Can you distinguish the actions taken by the permission referee when encountering a proposal that violates freshness deadlines, breaches physical barrier invariants, or emits numerical exceptions?

2.10 Safety Convergence

Safety examines whether the machine caused harm. Security considers whether an external agent forced it. Accountability assesses whether an auditor can determine what occurred and why. In software services, these concerns reside in separate engineering organizations and distinct runtimes, divided by application logic, network firewalls, and audit logging databases. In a machine that moves mass and commands current, the concept of a ‘causal boundary’ refers to the critical interface where digital commands translate into physical actions. This boundary is essential for understanding safety and accountability in physical AI systems. It determines whether an independent gate has the state and authority to refuse a register write before a digital command translates into actual physical torque acting on the plant.

Safety failures arise without an adversary. A camera lens accumulates dust over 500 h of operation, shifting the input distribution away from the training domain until an attention head generates

an out-of-distribution steering torque. A joint bearing develops mechanical play, introducing unmodeled backlash and joint compliance that converts a stable learned position trajectory into a destructive limit-cycle oscillation. Or a sudden change in surface friction drops tire traction below the friction coefficient assumed by the learned policy. In each case, the proposer generates an action that is statistically plausible to the network but physically hazardous to the body. If the permission gate lacks an independent kinematic model or thermal budget to detect exceedance, the unrefused command writes to the motor drive registers, releasing stored kinetic energy into the workspace.

Security failures introduce an adversary who deliberately exploits the gap between statistical inference and physical reality. An attacker does not need to compromise cryptographic keys or gain root access on the host operating system if they can manipulate sensory inputs. Projected structured light can spoof a depth sensor, acoustic injection can resonate the micro-machined capacitive proof masses inside MEMS gyroscopes at their natural frequencies (18–32 kHz), tricking the sensor into reporting phantom angular velocity,²¹ or cleverly patterned stickers (adversarial visual perturbations) on a factory floor can induce a vision-language-action model to output maximum actuator velocity. On a traditional server, an injection attack compromises data confidentiality or integrity within an abstract memory space. In physical AI, an injection attack commands physical work, turning an unvalidated inference into mechanical force. The security perimeter cannot terminate at the network interface or the operating system syscall boundary. It terminates at the permission gate, which must evaluate every candidate command against physical invariants regardless of whether the proposer acted in good faith or under adversarial control.

Traditional software authorization models fail in physical machines because capability tokens and access control lists assume discrete, memoryless transactions. In a web service, possessing a valid credential grants permission to write a record to a database, and the cost of the operation is bounded by compute cycles and storage bytes. In physical space, permission is not a static binary entitlement. In an analytical robotic joint, a command to apply $50 \text{ N} \cdot \text{m}$ of torque is valid when the link is stationary at mid-stroke, but the identical command causes structural yield (shaft fracture or gear tooth shear) if the joint is 2 mm from its hard stop moving at 1.5 rad/s . Physical authority is state-dependent, continuous, and dynamic, governed by velocity, momentum, thermal accumulation, and spatial occupancy. A permission gate cannot evaluate an action in isolation as a stateless remote procedure call. It must integrate the action over time against the current physical state of the body, checking whether the proposed motion stays within a validated recoverable set²² for the plant, load, state-estimation error, and response deadline.

Accountability requires proving the exact causal chain that led to actuation when a machine damages hardware or breaches an envelope. In standard computing systems, logging is asynchronous, best-effort, and coarse-grained, dropping records during buffer pressure or recording timestamps with millisecond jitter. At the causal boundary, accountability demands deterministic, high-rate telemetry captured synchronously with the control loop. If a joint actuator commands 1000 Hz updates with a 1 ms deadline, the permission gate must record the raw candidate vector proposed by the model, the state estimate used by the gate, the mathematical criterion applied by the safety filter, and the exact vector written to the hardware registers. Sealing these records in tamper-evident, append-only circular buffers in non-volatile SRAM/FRAM—memory that survives a sudden power loss when a safety breaker trips—provides post-incident reconstruction. When an anomaly occurs, an engineer can verify mathematically whether the failure originated in a corrupted sensor reading, a degraded policy proposal, an erroneous permission rule, or a mechanical component that failed to produce the commanded counter-torque.

21 | **MEMS Acoustic Resonance:** Resonant acoustic frequencies (18–32 kHz) can physically vibrate the micro-machined capacitive proof masses inside MEMS gyroscopes, inducing signal saturation and false rate readings. Hardening against acoustic spoofing requires physical damping barriers and real-time kinematic cross-validation on the microcontroller.

22 | **Forward-Invariant Reachable Sets:** A modeled set of physical states (positions, velocities, and temperatures) from which an admissible controller can keep the plant within specified bounds under stated disturbances and actuator limits (Mitchell et al. 2005). Admission against this set is meaningful only while those state, plant, and timing premises hold.



Checkpoint 2.8: State-dependent authority and causal accountability

Before reviewing the roadmap for the entire textbook, verify your understanding of safety, security, and accountability at the causal boundary:

- **State-dependent physical authority:** Can you explain why traditional stateless software permissions (such as RBAC or API capability tokens) fail to protect a dynamic physical robot whose allowable torque depends continuously on joint velocity and distance to hard stops?

- ❑ **Sensor physics injection attacks:** Can you describe how an adversary can manipulate physical actuation without compromising operating system root credentials by exploiting acoustic resonance (18–32 kHz) in MEMS gyroscopes?
- ❑ **Deterministic post-incident accountability:** Can you identify why post-incident reconstruction requires capturing candidate proposals, barrier criteria, and executed register commands synchronously in non-volatile SRAM/FRAM circular buffers?

2.11 The Architecture of This Book

The book follows the same command through four stages. Part I (chapter 2 through chapter 8) establishes the body’s mechanical and electrical limits and the timing contract between cognition and control. Part II (chapter 10 through chapter 14) asks whether data, training, and closed-loop tests support learned proposals within those limits. Part III (chapter 16 through chapter 26) traces live sensing, planning, permission, and execution. Part IV (chapter 28 through chapter 34) addresses human authority, verification, release, and limits of evidence. At each stage, the question remains whether a proposed action can cross the causal boundary before the physical margin runs out.

2.12 Fallacies and Pitfalls

The scope test is useful when systems combine learning, physical feedback, and varying human supervision. These examples show how overlooking one of those distinctions changes the architecture an engineer chooses.

Fallacy: *Any system using deep neural networks in robotics is Physical AI.*

A lane-departure warning system uses a learned vision model to detect lane markings and sounds a buzzer when the vehicle drifts. It meets the first two scope criteria: perception is learned, and vehicle motion changes what the camera observes. However, its output does not command steering or braking. The driver closes the physical control loop by applying manual steering torque, so the system lacks the third criterion, delegated physical authority. The relevant distinction is who can change the vehicle’s motion, not the depth of the neural network. Adding steering authority would change the engineering responsibilities at that boundary.

Fallacy: *Hard real-time control alone requires the systems discipline of Physical AI.*

A quadrotor can track a trajectory through wind gusts using an analytical control law, issuing motor commands at 1000 Hz from a bare-metal microcontroller. Rotor thrust changes its motion, and the controller acts without waiting for a person, satisfying the physical-feedback and delegated-authority criteria. Yet a controller derived from Newton–Euler equations and state-space feedback need not contain a learned model. The design therefore belongs to classical control, whose foundations precede neural policies (Maxwell 1868). Hard deadlines remain essential, but they alone do not create the additional problem of integrating learned proposals with physical control.

Fallacy: *Autonomous digital agents face the same operational constraints as embodied robots.*

A trading engine uses learned models to predict prices and executes transactions without human approval. It combines learning with delegated authority, but its action interface updates digital orders and account records rather than commanding a physical body governed by momentum and inertia. Its immediate operating constraints therefore differ from those of a robot whose delayed command changes stopping distance or contact force. Trading failures can still have serious consequences; the distinction does not make digital autonomy harmless or universally reversible. It identifies which constraints the architecture must represent. A robot additionally needs control over physical motion while its learned model computes.

Pitfall: *Treating human supervisory intervention as instantaneous in shared-autonomy systems.*

A surgical tool applies haptic guidance force feedback from a learned anatomical model while a surgeon holds the instrument. Human supervision does not remove delegated authority during the interval before the surgeon can respond. If an actuator builds force faster than the stated human reflex interval of 150–250 ms, its commands can deform tissue before the operator corrects them.

The engineering question is how much force the system can apply during that interval. Local force limits and a deterministic safety filter must constrain the actuator without depending on either a timely neural prediction or an immediate human response.

Pitfall: *Assuming software latency merely degrades performance gracefully.*

A quadruped's visual-motor policy stalls for 300 ms when its memory subsystem throttles. The computation pauses, but physical momentum ensures the robot keeps moving. At a constant 3.0 m/s, the machine essentially coasts blind for 0.9 m before the next policy command arrives. That displacement can consume the available clearance and leave too little distance to stop if an obstacle appears. Inference latency therefore belongs in the motion budget, even when the model's eventual prediction is correct. A hard real-time reflex loop must maintain the stopping envelope and respond to missed deadlines while the deliberative policy is unavailable.

Fallacy: *Assuming physical actions can be rolled back like database transactions.*

A manipulator applies an erroneous 50 N · m torque to a delicate fixture. Its monitor detects the anomaly 50 ms later and reverses the motor command. Reversing a command cannot restore a fixture that has already plastically deformed or recover the energy already dissipated as physical heat in the motor windings ($I^2 R$). Detection can limit further damage, but it cannot erase the intervening physical work. The control architecture must therefore check commands against force and motion limits before execution. Recovery remains necessary, but it complements preventive enforcement rather than replacing it.

2.13 Summary

The causal boundary marks the physical and electrical threshold where digital bits convert irreversibly into kinetic momentum, contact force, and Joule heat at memory-mapped actuator registers. Past this boundary, computational abstraction terminates: latency is paid directly in unguided physical displacement, and released kinetic energy cannot be undone by a software exception handler.



Key Takeaways: The causal boundary invariant

- **Physical Register Address (Causal Boundary):** Informational abstraction terminates at memory-mapped actuator registers; digital bits irreversibly convert into magnetic flux, kinetic momentum ($\frac{1}{2}mv^2$), and Joule heat ($I^2 R$), and software exceptions cannot unwind thermodynamic work.
- **Unguided Displacement (Inelastic Latency):** Information decays continuously with age; every millisecond of computational and transport delay at velocity v accumulates unguided physical drift ($\Delta x = \int v dt$), requiring sensing-to-actuation timing budgets to be bounded by strict worst-case tail percentiles ($P_{99.9}$) rather than optimistic medians.
- **Four-Archetype Resource Floors:** Embodied platforms partition into Mass-Dominated systems (Class 1) bound by kinetic stopping envelopes ($v^2/2a_{\max}$), Contact-Dominated mechanisms (Class 2) bound by structural impedance ($k\Delta x$) and shear limits, Flow-Dominated processes (Class 3) bound by lumped thermal capacitance and mass-balance continuity, and Integrative Capstone systems (Class 4, e.g., humanoids) where all three dynamic regimes contend simultaneously on a single tightly-coupled chassis.
- **Three-Pillar Scope Test (Physical AI Boundary):** A machine falls within Physical AI when a learned component supplies live proposals, its actions affect future physical observations, and accepted proposals can reach actuation without case-by-case human approval—even through an independent safety gate.
- **Proposal-Permission Contract (Dual-Brain Architecture):** Learned policies propose actions; an independent bounded-latency mechanism checks physical limits and controls a fallback before actuation. A dedicated real-time microcontroller is one implementation.
- **Four Architectural Tiers (Layered Container):** Every embodied machine organizes into a four-tier operational hierarchy: the Dynamical Body (Layer 1, physical plant and

conservation laws), the Real-Time Nervous System (Layer 2, deterministic reflexes), the Cognitive Brain (Layer 3, learned deliberation), and the Governance Envelope (Layer 4, operational leases and safety cases).

- **Physical Irreversibility:** Actuation can deliver energy and change the plant before software can respond. Idempotent command semantics do not reverse work already performed.

Physical irreversibility means that computation cannot be decoupled from physical consequence: momentum cannot be halted instantly, and unmodeled mechanical stress can damage hardware regardless of offline training convergence.



What's Next: From causal boundary to physical body

With the causal boundary established and the proposal–permission contract defined, we pass from the foundational boundary to the physical plant. **Part I (The Machine Anatomy)** proceeds to the physical body in chapter 4, descending to the mechanical chassis, power inverter stages, and sensor instrumentation to establish the governing equations of time, inertia, actuation, and heat that every learned policy must obey.

THE MACHINE ANATOMY

Part I

The Machine Anatomy

EVERY physical AI system is defined by the physical apparatus through which it acts. An advisory computation can end at a display or a human decision, although an erroneous output may still cause harm when acted upon. Physical AI operates across a causal threshold where permitted commands drive current, force, and motion. Matter will not pause while an operating system schedules a thread, and a later software rollback cannot undo work already delivered into the plant or environment.

III Principle 1: The irreversibility invariant

Invariant: A proposal can be rejected before actuation, but software rollback cannot reverse motion or dissipated heat after a command has delivered physical work. Idempotency describes the effect of repeating a command; it does not imply that its physical effects are reversible. If the plant continues at constant speed v_0 without braking during a sense-to-brake delay Δt_{delay} , the reaction displacement is

$$d_{\text{react}} = v_0 \cdot \Delta t_{\text{delay}}$$

With an additional constant braking deceleration $a_{\text{max}} > 0$, stationary obstacle, and no unmodeled brake-onset delay, the model gives $d_{\text{stop}} = v_0 \cdot \Delta t_{\text{delay}} + \frac{v_0^2}{2a_{\text{max}}}$. A stop before the obstacle requires $d_{\text{stop}} \leq d_{\text{clear}}$ under those assumptions.

Implication: A software exception handler cannot stop moving mass. The permission path must have an independently enforced response budget, a veto, and a physical fallback. A measured $p99.9$ latency can inform that budget but is not a worst-case bound; the chosen deadline must also account for sensing error, brake onset, and actuator limits.

To engineer systems that survive in the physical world, Part I establishes the **Constraint-Driven Architecture** of physical AI. Rather than treating an autonomous machine as an abstract software pipeline, we ground the entire discipline in the structural triad (Body, Brain, Nervous System) and its operational counterpart, the **S-P-A (Sense, Plan, Act)** framework (section 2.6, Appendix D):

- **The Body (chapter 4):** The physical, mechanical, and thermal plant. Its motion and energy flows obey mechanics and electromagnetism rather than software instructions. Stopping clearance, reflected rotor inertia ($N^2 J$), winding heat ($I^2 R$), and voltage droop constrain what it can execute; sensor freshness constrains how accurately the other tiers know its state.
- **The Brain (chapter 6):** The learned cognitive deliberative engine. We define what the brain is *for*—high-capacity open-world semantic generalization, spatial affordance extraction, and continuous trajectory synthesis that classical hand-crafted state machines cannot achieve. Crucially, we also define what *bounds* the brain: the memory wall, Roofline arithmetic intensity limits, stochastic fat-tailed latency, and uncalibrated confidence. Because of these constraints, the brain must operate as an **unprivileged proposal engine**, emitting advisory action chunks rather than direct actuator commands.
- **The Nervous System (chapter 8):** The real-time architectural bridge. Between a slower brain (for example, 1–10 Hz) and faster local control (for example, 1000 Hz), it arbitrates time and authority. It checks proposals against state-dependent limits and executes a defined fallback when high-level software stalls. A Control Barrier Function can support such a check when its

plant model, state estimate, and feasible control authority satisfy the conditions of its guarantee.

III Principle 2: The proposal boundary

Invariant: A learned cognitive model proposes candidate action trajectories; it does not hold direct write access to actuator power stages. An independent permission mechanism accepts, modifies, or vetoes each proposal before actuation and retains authority to execute a fallback if the proposer fails. The cited 1–10 Hz deliberation and 1000 Hz reflex rates are implementation examples, not universal requirements.

Implication: The proposal and permission paths need fault containment, a bounded communication delay, and a fallback that does not wait on the failed proposer. Asynchronous buffers and a separate controller are one implementation. If a proposal is stale or unsafe, the permission path rejects it and executes a validated braking or risk-reducing maneuver within the available physical margin; stopping is possible only while sufficient clearance and control authority remain.

III Principle 3: The plant budget contract

Invariant: Five coupled engineering constraints bound a specified operating state; they are not five conservation laws. Measurement age must remain within a validated state-estimation horizon ($\Delta t_{\text{age}} \leq \tau_{\text{valid}}$), and stopping distance must fit measured clearance ($d_{\text{stop}} \leq d_{\text{clear}}$) under stated sensing and braking assumptions. For ideal rigid gearing, reflected rotor inertia is $J_{\text{ref}} = N^2 J_m$. For a lumped winding with no credited cooling, resistive heating must fit its thermal margin ($\int I_{\text{phase}}^2 R_{\text{phase}} dt \leq C_{\text{th}} \Delta T_{\text{max}}$). For a simplified unbuffered bus during rising current, estimated voltage must stay above the operating minimum ($V_{\text{bus}} - L_{\text{bus}} \frac{dI_{\text{bus}}}{dt} - I_{\text{bus}} R_{\text{bus}} \geq V_{\text{min}}$). Model error, saturation, cooling, and protection behavior determine whether these approximations are conservative enough for a given machine.

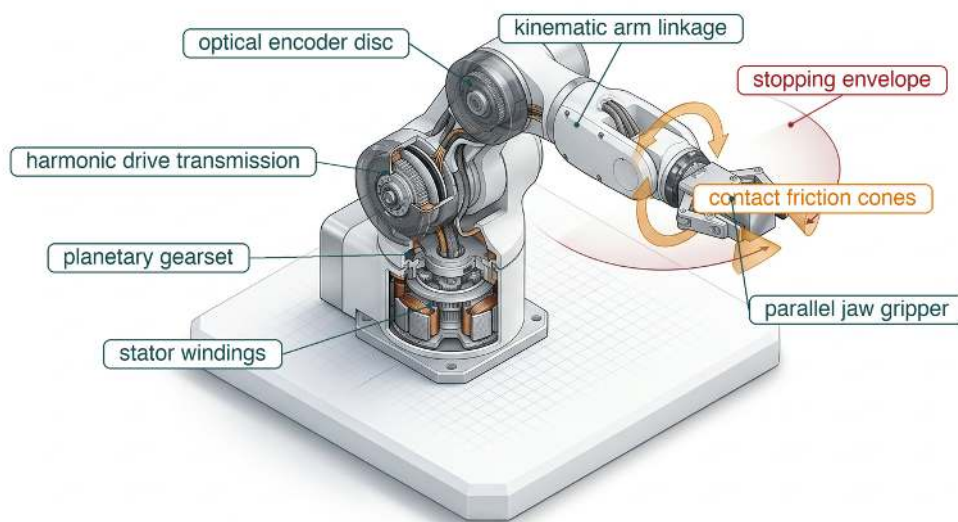
Implication: An optimization that consumes any one margin can increase contact force, winding temperature, voltage sag, or stopping distance. The design must identify which budget binds first, measure its uncertainty, and allocate sensing, derating, buffering, or fallback capacity before permitting a proposal.

Part I provides the mental models, systems napkin math, and architectural foundations required to understand this Triad. Before we collect demonstrations, train neural policies, or build runtime perception stacks, we must first master the physical constraints that dictate how a machine survives in the material world.

The Physical Body

DRAFT

The Physical Body

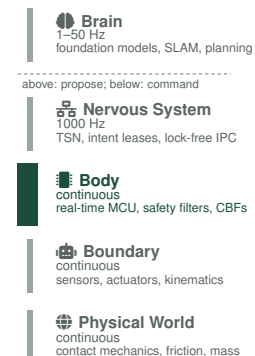


- 4.1 The Five Physical Budgets
- 4.2 Measurement Freshness
- 4.3 Kinetic Momentum
- 4.4 Actuator Transmission Limits
- 4.5 Thermal Duty Cycles
- 4.6 Electrical Power Integrity
- 4.7 Measuring a Machine's Own Limits
- 4.8 The Cross-Archetype Matrix
- 4.9 Fallacies and Pitfalls
- 4.10 Summary

Purpose

What physical limits bound a learning agent the moment its mathematical tokens command physical motors and moving mass?

A moving machine cannot pause for software. During inference, the physical world advances: positions change and observations age. Stopping requires latency to process a command and distance to dissipate kinetic energy. Furthermore, motors and transmissions rigidly constrain how quickly motion can change, while electrical losses continuously heat the windings. Policies ignoring these constraints request impossible forces or unsustainable movements. Faster inference helps only within this physical path; it does not eliminate braking distance, mechanical response time, or thermal limits. These limits must be measured empirically under operational conditions. In physical AI systems architecture, the body sets the budgets that sensing, computation, and control must share, connecting each software decision to the motion and energy it produces.



Prerequisite: The physical plant grounds the four physical AI archetypes defined in section 2.3.



Learning Objectives

- Calculate observation freshness limits from measurement age and the machine's allowable motion
- Determine stopping clearance from reaction delay, travel speed, and available braking
- Evaluate actuator and transmission choices against response limits and reflected inertia
- Estimate sustainable motor duty cycles from heat generation, cooling, and operating conditions
- Budget electrical supply margins for simultaneous actuation, computation, and regenerative braking
- Diagnose coupled physical limits using measurements from an assembled machine under representative stress

4.1 The Five Physical Budgets

A robotic manipulator can shatter its transmission even after receiving an emergency stop command before contact. The stop command traverses software buffers, the power inverter switches states, and the motor generates peak reverse torque. Yet if the moving link possesses kinetic energy exceeding the mechanical work the actuators can perform over the remaining clearance, impact is mathematically guaranteed. In pure software, a faulted thread can be killed instantly by the operating system kernel. In physical reality, moving mass carries momentum that cannot be caught by an exception handler, and an electrical power stage cannot deliver infinite current without thermal destruction or voltage collapse. The fundamental truth of physical AI is that software cannot override the conservation laws of physics.



Definition 4.1: The actuator authority budget

Actuator authority budget is the combination of force available at the load, time needed to deliver it, and heat the installed actuator can sustain. A software setpoint outside that envelope cannot be realized as commanded; the actuator, drive, or transmission may saturate or overheat. Later sections derive its electrical and thermal limits.

1. **Significance:** A proposed motion must fit delivered force and response time, not merely a digital command range.
2. **Distinction:** A static motor datasheet rating does not account for the installed transmission, available supply, or cooling path.
3. **Common pitfall:** Treating advertised peak torque as a continuous rating when winding temperature requires derating.

Within the physical AI stack (figure 4.1), this chapter focuses on Layer 1: the Physical Body. Operating beneath the real-time Nervous System and Cognitive Brain, the body represents the foundational electromechanical plant. Across the Proposal-Permission Privilege Boundary,¹ high-level neural policies propose motion trajectories. The physical body, however, establishes the non-negotiable physical laws—mechanics, thermodynamics, and electromagnetism—that define whether any proposed motion is physically possible. Software cannot negotiate with momentum, and gradient descent cannot rewrite the conservation of energy.

To transform these continuous physical realities into rigorous systems specifications, every software setpoint draws against five non-negotiable physical budgets:

1. **Measurement freshness:** The rate at which the physical environment changes relative to the age of the sensor data reaching the controller. Because the world does not pause for computation, elapsed time during communication and inference means the machine always acts on historical state.

¹ | **Budget Enforcement:**

In the proposal-permission architecture, the high-level policy is never trusted to self-police its torque or thermal consumption. The nervous system evaluates proposed setpoints against real-time physical budget models, clamping or rejecting commands that would exceed measured hardware ceilings.

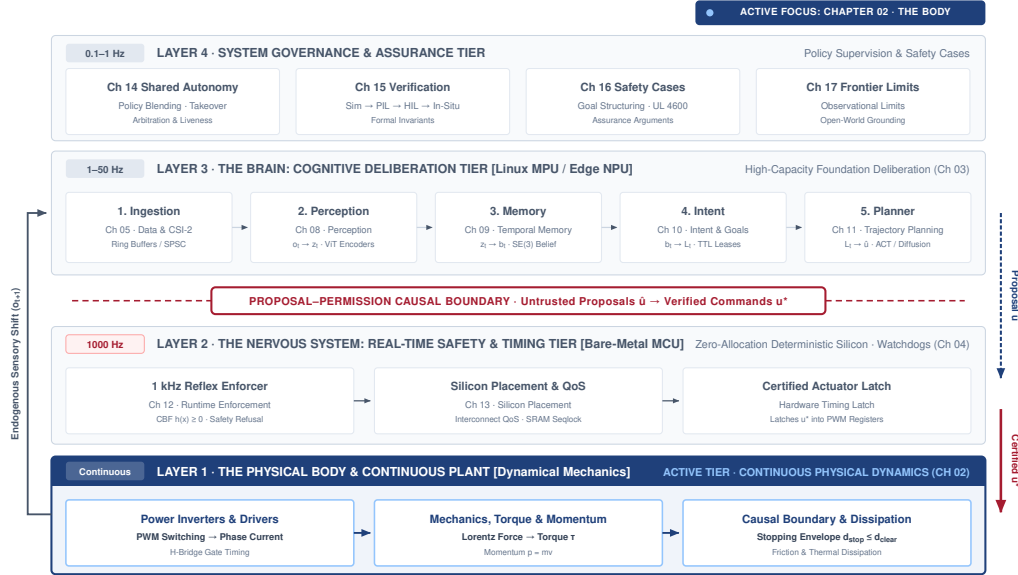


Figure 4.1: The body in the Physical AI stack: The four-tier physical AI systems architecture highlights the physical world at its foundation, bounded by mechanics, thermodynamics, and electromagnetism. Above the causal boundary, the nervous system and learned brain must budget all sensing latency, stopping distances, and actuation requests against these non-negotiable physical constraints.

2. **Kinetic momentum:** The stopping distance and braking force required to dissipate stored kinetic energy ($E_k = \frac{1}{2}mv^2$) before colliding with obstacles or exceeding workcell boundaries.
3. **Actuator bandwidth and reflected inertia:** The speed at which mechanical torque and velocity can change before winding inductance ($\tau_e = L/R$), inverter current clamps, or gear transmission inertia (N^2J) choke mechanical acceleration.
4. **Thermal capacity:** The rate at which resistive Joule heat accumulates in stator coils and power semiconductors relative to the chassis dissipation rate, bounding sustained duty cycles before thermal throttling intervenes.
5. **Electrical power integrity:** The transient voltage droop, peak current capacity, and regenerative energy surges across the shared direct-current distribution bus during rapid acceleration and braking.

Across the three physical archetypes introduced in section 2.3, these five budgets manifest in distinct physical bottlenecks:

- **Mobile Systems (AMR / AGV):** Kinetic momentum and measurement freshness dominate. Because an unladen 80 kg chassis traveling at 2.0 m/s carries 160 J of kinetic energy, stopping distance ($\Delta d = v^2/2a_{\max}$) and tire traction limits bound safe velocity through human-shared aisles.
- **Manipulators (Articulated Arms):** Reflected inertia (N^2J_m) and thermal capacity govern operation. High-ratio strain-wave gearing ($N = 100 : 1$) amplifies rotor inertia by 10,000 $\times$, while unventilated limb castings trap I^2R Joule heating during prolonged static payload holding.
- **Process and Energy Systems:** Electrical power integrity and thermal dissipation dictate stability. Rapid valve throttling and pump motor starts induce sharp DC-bus voltage droops ($\Delta V = I_{\text{surge}}R_{\text{bus}}$), demanding aggressive capacitor banks and dynamic braking choppers.

🔍 Systems Perspective 4.1: The five physical conservation budgets

The **physical body** is the foundational electromechanical, thermodynamic, and structural plant—comprising actuators, transmissions, structural linkages, power distribution buses, and transducers—that interacts directly with the physical universe. Every actuation is an irreversible physical event that draws against these five non-negotiable physical budgets.

A recurring failure mode in physical AI engineering stems from reading component datasheets as operational guarantees. Consider an illustrative actuator tested on an open dynamometer bench at 25°C ambient temperature with a large heat sink and regulated supply. Inside an assembled machine, that actuator sits in an unventilated limb beside an inference processor dissipating sixty watts. If cavity temperature rises to 55°C and thermal resistance increases, a nominal 4.2 N · m continuous rating may no longer be available; a lower installed limit such as 1.8 N · m must be established by measurement. The operational envelope is the one qualified on the assembled hardware under declared thermal and electrical stress.

The following sections turn each budget into a measurement or operating limit, then compare them across machine classes in section 4.8. Measurement age begins at physical transduction, before a Python process receives the data.²

2 | **Transduction Timestamps:** The physical age of a measurement begins when photons hit a camera photodiode or magnetic flux passes a Hall sensor, not when a Python process receives the tensor. Precise age accounting requires hardware timestamps captured at the physical PHY interface to prevent operating system scheduling jitter from corrupting latency tracking.

4.2 Measurement Freshness

On a fast-moving logistics sorting line, a vision-guided robotic manipulator tracks parcels rushing past on a conveyor belt at 1.2 m/s. To pick a moving target without missing or colliding with the guide rails, the robot's control policy must know precisely where the object is in physical space. Yet during the time required for photons to expose the image sensor, pixel data to stream across the PCIe bus, a neural network to infer a grasp pose, and motor windings to generate torque, the physical world advances.

In traditional software benchmarking, latency is often reported as the pure execution time of a neural network running on an accelerator. Following section 2.2, that compute slice is only one segment of a longer causal chain. **A measurement decays from transduction; what matters is its age when acted upon.** The physical age of an observation spans the full interval from when photons strike a photodiode array or magnetic flux rotates past a Hall sensor, to when motor coils generate mechanical torque that alters the acceleration of a link. If a perception model computes an obstacle location in 15 ms but camera integration, sensor readout, transport, queue dispatch, and motor current rise time add another 45 ms, the physical age of the measurement when the joint moves is 60 ms. During that entire window, the physical world has continued to evolve.

Tracking measurement age requires decomposing the complete physical path into eight distinct stages (figure 4.2). First, physical integration and exposure capture the environment over an exposure window, setting the timestamp to the midpoint of integration. Second, sensor readout serializes analog charge into digital frames over a physical interface such as MIPI CSI-2 or SPI. Third, hardware transport moves the packet into host memory through direct memory access (DMA) controllers and PCIe/AXI buses. Fourth, kernel buffering and driver ingestion place the frame in user-space queues. Fifth, model computation executes neural network inference, filtering, or kinematic projection on an accelerator. Sixth, operating system scheduling and inter-process communication dispatch the resulting control setpoint across the processor boundary. Seventh, command transmission serializes the setpoint across a real-time fieldbus or shared-memory mailbox to the motor driver. Eighth, actuator response converts the command into pulse-width modulation signals, building magnetic field flux in the stator windings to develop mechanical force. On a canonical heterogeneous dual-brain platform, this eight-stage pipeline spans the CMOS image sensor → MIPI DMA into host memory → neural policy inference running on the Linux application processor (MPU) → inter-processor communication (IPC) over shared SRAM mailboxes via lock-free ring buffers → deterministic 1 kHz safety monitor evaluation on the real-time microcontroller (MCU) → PWM timer register updates → gate driver switching and stator coil current rise (governed by the actuator authority budget introduced in section 4.1). Every stage consumes a slice of the total time budget.

The rate of world-state change dictates how long a measurement remains valid before acting on it introduces intolerable error. Let $e_{\max}$ denote the maximum acceptable error in a tracked state, such

Downstream: Measurement aging dynamics directly dictate the state estimation horizons in section 22.1.

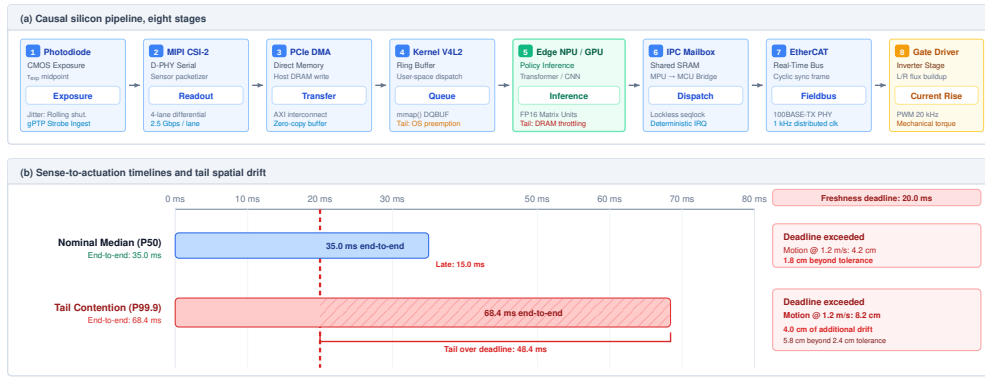


Figure 4.2: Sense-to-actuation latency waterfall and silicon dataflow pipeline: The eight-stage causal chain from exposure to motor current rise (panel a) and an illustrative latency profile (panel b). The conveyor’s 24 mm tolerance at 1.2 m/s sets a 20 ms age deadline, because unguided travel converts elapsed time into displacement at the carriage speed. Both the synthetic 35.0 ms median and 68.4 ms tail exceed it, corresponding to 4.2 cm and 8.2 cm of motion, respectively. These are scenario values, not measured hardware results.

as joint position or distance to an obstacle, and let $v_{\max}$ denote the fastest credible velocity at which that state can evolve. The freshness deadline Δt_{fresh} is the upper bound on measurement age before a control action based on that measurement becomes invalid, defined by the inequality

$$\Delta t_{\text{fresh}} \leq \frac{e_{\max}}{v_{\max}}$$

For our conveyor-tracking manipulator operating with a maximum relative velocity $v_{\max} = 1.2$ m/s and a geometric clearance tolerance $e_{\max} = 24$ mm, the freshness deadline is $24 \text{ mm} / (1200 \text{ mm/s}) = 20$ ms. If an actuation command reaches the motor based on an image captured 35 ms earlier, the workpiece has displaced by 42 mm, exceeding the position allowance by 18 mm and missing the grasp under this constant-velocity approximation. This deadline limits motion-induced error when relative velocity is bounded and the initial observation is credible; sensing and estimation errors consume additional tolerance.

Engineers often compute total latency by summing the P_{99} values of individual stages, but the percentile of a sum is not the sum of the percentiles.³ Neither this arithmetic sum nor a measured end-to-end percentile is a hard worst-case bound. Shared memory contention and kernel queues can correlate stage delays, so the distribution of complete sense-to-torque age must be measured on the integrated machine under declared load. A safety deadline needs a separately justified bound or a monitored response when that bound cannot be maintained.

To illustrate how stage timing differs from end-to-end age, consider a synthetic timing profile for the conveyor manipulator under continuous inference load. The values in table 4.1 are constructed for this example; they are not recorded cycles or a characterized machine.

Table 4.1: Illustrative sense-to-actuation latency decomposition: Synthetic stage and system-level timings show why arithmetic sums of stage percentiles do not establish the end-to-end tail. The example’s $P_{99.9} = 68.4$ ms exceeds the conveyor’s 20 ms freshness deadline; these values have no measured exceedance rate or hardware provenance.

Pipeline Stage	Physical Mechanism / Contention Source	P_{50} (ms)	P_{99} (ms)	Max (ms)
1–2. Sensor Exposure & Readout	60 Hz rolling shutter exposure midpoint	11.2	16.6	16.7
3–4. Host Transport & DMA	PCIe DMA transfer into host memory	1.4	3.2	6.8
5. Vision Backbone & Policy	Onboard accelerator inference (memory throttling tail)	18.5	24.1	38.4
6. OS Scheduling & IPC	Kernel preemption and mailbox dispatch to MCU	0.6	2.1	12.3

3 | Quantile Convolution: In general, the quantile of a latency sum differs from the sum of equal-percentile stage quantiles ($Q_p(X + Y) \neq Q_p(X) + Q_p(Y)$). Independence alone does not make their arithmetic sum a worst-case bound. Correlation from shared buses and queues changes the end-to-end tail and must be characterized at the system boundary.

Pipeline Stage	Physical Mechanism / Contention Source	P_{50} (ms)	P_{99} (ms)	Max (ms)
7. Fieldbus Transmission	Deterministic EtherCAT cyclic packet exchange	1.0	1.0	2.0
8. Actuator Current Rise	Gate driver switching and stator flux buildup	2.3	3.0	4.5
Arithmetic Stage Sum	Sum of each column's stage values; not an end-to-end quantile	35.0	50.0	80.7
Illustrative End-to-End Age	Constructed system-level profile ($t_{\text{transduction}} \rightarrow \tau$)	35.0	46.2	80.7

In this constructed profile, the end-to-end ages are $P_{50} = 35.0$ ms, $P_{99} = 46.2$ ms, $P_{99.9} = 68.4$ ms, and a maximum of 80.7 ms. The 50.0 ms sum of stage P_{99} values differs from the illustrative 46.2 ms system percentile; the table does not establish a worst-case latency or the fraction of cycles above any threshold. For the 20 ms conveyor deadline, even the 35.0 ms median would be late. A real acceptance decision requires captured end-to-end timestamps and a bounded response to stale observations.

Executing an actuation command from expired sensor data adds phase lag to the feedback loop and can excite mechanical chatter when corrective torques reinforce rather than damp tracking errors.⁴ High inference frequency alone does not establish timely physical response. The system must carry a hardware timestamp from transduction (using **IEEE 1588 gPTP**⁵ or hardware strobe capture at the Ethernet PHY / **MIPI CSI-2**⁶ interface) through the pipeline. Before applying a policy command, the permission path compares $t_{\text{now}} - t_{\text{transduction}}$ with Δt_{fresh} . If age exceeds the deadline, it rejects the stale proposal and executes a fallback whose braking or holding behavior has been validated within the remaining physical margin (Brooks 1986a).

A bounded measurement age limits motion-induced state error only under a credible velocity bound and a valid initial observation. Sensor calibration, localization, and estimation errors need separate bounds. While signals propagate through the nervous system, the body continues moving, carrying momentum that no software interrupt can cancel.



Checkpoint 4.1: Transduction age and latency tails

Before analyzing vehicle kinetic momentum, verify your understanding of measurement freshness:

- ☐ **Transduction-to-torque causal chain:** Can you trace why measurement age begins at photon exposure on sensor silicon rather than when a neural network begins inference?
- ☐ **Quantile convolution non-additivity:** Can you explain why summing the 99th percentile latencies of individual pipeline stages fails to predict the true end-to-end tail latency across shared memory buses?
- ☐ **Freshness deadline derivation:** Can you calculate the maximum allowable measurement age for a manipulator tracking a workpiece moving at 1.5 m/s with a 15 mm tolerance?

4.3 Kinetic Momentum

A warehouse transport robot carrying 120 kg of palletized freight navigates a distribution floor at 2.0 m/s. When a worker steps unexpectedly into the aisle 1.5 m ahead, software cannot simply pause execution or instantaneously zero its velocity vector. The vehicle's steel chassis and cargo carry unyielding mechanical momentum, and bringing that mass to rest requires dissipating kinetic energy across physical distance. **Stopping distance grows linearly with delay and quadratically with speed, and that difference sets the speed limit.**

The distance from sensing a hazard to zero velocity comprises two phases: reaction travel and braking travel. During the reaction interval, the nervous system executes the causal boundary pipeline established in section 2.2. Throughout this full end-to-end sense-to-actuation delay Δt_{delay} , the actuators apply no retarding force, and the vehicle continues moving along its existing trajectory

⁴ | **Boode Delay Theorem and Phase Margin:** Pure transport delay τ_d introduces an unyielding frequency-dependent phase lag $\Delta\phi = -\omega\tau_d$ radians without altering loop gain magnitude. In closed-loop feedback systems, this phase erosion degrades the phase margin ($\text{PM} = 180^\circ + \angle L(j\omega_c)$); once τ_d erodes the margin to zero at the gain-crossover frequency ω_c , the closed-loop poles cross into the right-half complex plane, destabilizing the physical plant into destructive limit-cycle oscillations.

⁵ | **IEEE 1588 gPTP:** The Generalized Precision Time Protocol (gPTP) provides sub-microsecond clock synchronization across Ethernet networks by exchanging hardware-timestamped packets directly at the network chip (media access control [MAC] PHY) layer. This guarantees timing precision by bypassing the unpredictable delays (jitter) of the operating system's software scheduler.

⁶ | **MIPI CSI-2:** The Mobile Industry Processor Interface Camera Serial Interface 2 (MIPI CSI-2) is a high-bandwidth, high-speed differential protocol used for transmitting uncompressed video data directly from image sensors to host system-on-chip (SoC) devices with minimal latency.

at its initial velocity v_0 . Assuming velocity remains constant across this window, the reaction travel evaluates to

$$d_{\text{react}} = v_0 \Delta t_{\text{delay}}$$

Reaction travel is proportional to delay; thus, any latency in compute or communication extends reaction distance proportionally to velocity.

Once the brake command arrives and the actuators engage, the body enters the braking phase. The mechanical work done by the braking force F_{brake} over the braking distance d_{brake} must dissipate the initial kinetic energy of the machine, $W = \Delta K$. For a machine of mass m decelerating to rest under a sustained retarding force $F_{\text{brake}} = ma_{\text{brake}}$, the work-energy balance yields

$$ma_{\text{brake}}d_{\text{brake}} = \frac{1}{2}mv_0^2$$

Dividing by mass and solving for distance gives the kinematic braking travel

$$d_{\text{brake}} = \frac{v_0^2}{2a_{\text{brake}}}$$

The parameter a_{brake} is the credible deceleration the body can actually exert against the physical environment, not a theoretical peak rating printed on an actuator datasheet. It represents the sustained negative acceleration the mechanical structure, tires, and gears can produce before losing grip (tire-road shear slip saturation), tipping forward, or structural drive-train yield.

This closed-form expression for d_{brake} relies on four simplifying assumptions: constant deceleration throughout the maneuver, instantaneous torque buildup at the contact interface, zero grade, and rigid contact mechanics. In physical hardware, these assumptions frequently fail. Actuator magnetic fields require time to build force, tire-ground friction can fall during slip, and structural compliance allows deflection. On uncertain terrain or grades, the model can underestimate stopping distance. Stopping trials across the declared operating envelope should measure trajectories at the load; a conservative low-tail deceleration estimate, with uncertainty and margin, can then replace the idealized a_{brake} . A sample percentile alone is not a hard lower bound.

The differing exponents of the reaction and braking terms create an asymmetric scaling penalty when adjusting system parameters (figure 4.3). Because reaction travel scales linearly with delay ($d_{\text{react}} = v_0 \Delta t_{\text{delay}}$) while braking travel scales quadratically with speed ($d_{\text{brake}} = v_0^2 / 2a_{\text{brake}}$), velocity acts as the dominant multiplier of spatial risk. A tail spike in compute latency inflates reaction displacement linearly, but increasing travel speed quadruples kinetic energy and braking distance. Latency establishes the baseline reaction overhead, but velocity governs the explosive expansion of the defended envelope—a scaling dynamic executed numerically in 4.1.

In unstructured environments, a machine cannot allocate its entire clearance to nominal stopping dynamics. Obstacle tracking carries state estimation uncertainty, sensor calibration drifts over time, and localization algorithms introduce position error. The system must defend a stopping envelope d_{stop} that includes a spatial localization uncertainty bound δ_{loc} and a minimum physical clearance margin δ_{margin} .⁷

$$d_{\text{stop}} = v_0 \Delta t_{\text{delay}} + \frac{v_0^2}{2a_{\text{brake}}} + \delta_{\text{loc}} + \delta_{\text{margin}}$$

The sum $\delta_{\text{overhead}} = \delta_{\text{loc}} + \delta_{\text{margin}}$ represents fixed spatial overhead that must remain unoccupied when the body comes to a complete rest.

For a stationary obstacle detected at distance D_{clear} , the modeled stopping envelope must fit the clear path, $d_{\text{stop}} \leq D_{\text{clear}}$. Given a feasible current state and conservative timing and braking bounds, the permission path can solve this inequality for a ceiling on new velocity proposals v_{max} . The positive root of the quadratic inequality⁸ is:

$$v_{\text{max}} = -a_{\text{brake}} \Delta t_{\text{delay}} + \sqrt{(a_{\text{brake}} \Delta t_{\text{delay}})^2 + 2a_{\text{brake}}(D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}})}$$

When sensor range degrades or an obstacle enters the trajectory, D_{clear} contracts and lowers the ceiling, provided the available distance remains nonnegative. At zero available distance the ceiling is zero. If the machine is already moving, however, a zero setpoint cannot remove its current

Downstream: Unmanaged kinetic momentum translates directly into the destructive contact shock loads analyzed in section 4.4.

7 | ISO/TS 15066 Separation Distance: ISO/TS 15066 (Clause 5.5.4) and ISO 13849-1 mandate this protective separation distance for Speed and Separation Monitoring (SSM): $S = (v_r \cdot T_r) + (v_h \cdot T_r) + S_r + S_s + C$, where T_r is worst-case processing delay, S_r is braking travel, and C is clearance margin (100–200 mm). For learning policies operating in collaborative human workspaces, failing to bound compute tail latency directly inflates T_r , expanding the mandated safety separation zone and forcing premature protective stops.

8 | Kinematic Velocity Root: Grouping the constant spatial terms into available physical distance $D_{\text{avail}} = D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}}$ yields the quadratic inequality $\frac{1}{2a_{\text{brake}}}v_0^2 + \Delta t_{\text{delay}}v_0 - D_{\text{avail}} \leq 0$. Multiplying by $2a_{\text{brake}}$ produces $v_0^2 + 2a_{\text{brake}}\Delta t_{\text{delay}}v_0 - 2a_{\text{brake}}D_{\text{avail}} \leq 0$. Applying the quadratic formula and taking the positive physical root defines the dynamic velocity ceiling.

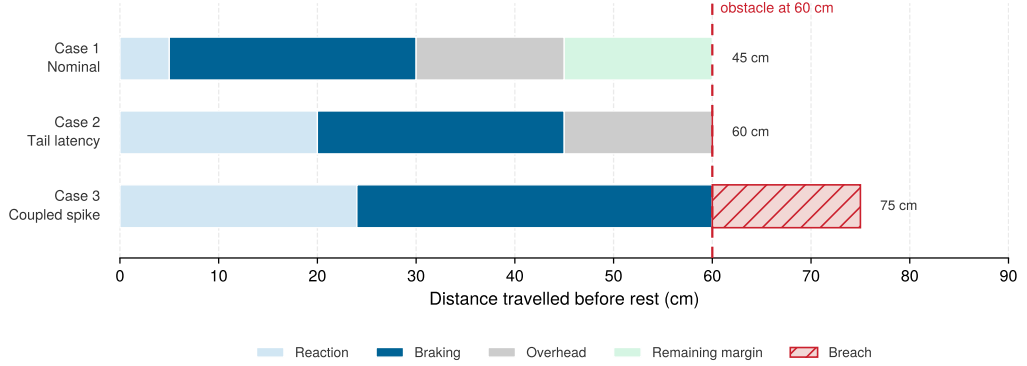


Figure 4.3: Dynamic stopping distance envelope: Total stopping distance decomposed into linear reaction lag, quadratic kinetic braking travel, and spatial overhead. Of the speed- and delay-dependent terms, doubling latency expands the pair by 17 percent, whereas doubling velocity quadruples braking travel for a 267 percent expansion; the spatial overhead is fixed. Case 1 operates with safe clearance; Case 2 compresses margin under tail latency; and Case 3 breaches the obstacle envelope when speed and compute delay couple.

stopping distance; a negative available distance means the admission condition is already violated and requires an emergency response.



Napkin Math 4.1: Stopping distance under tail latency

Problem: The registry's 150 kg logistics AMR travels at $v_0 = 1.8 \text{ m/s}$. For this illustrative surface and control path, assume a sense-to-braking delay of $\Delta t_{\text{delay}} = 65 \text{ ms}$, deceleration $a_{\text{brake}} = 3 \text{ m/s}^2$, localization allowance $\delta_{\text{loc}} = 5 \text{ cm}$, and clearance margin $\delta_{\text{margin}} = 10 \text{ cm}$. These are scenario inputs, not measured safety bounds. What stopping envelope does the model give, and how does it scale if speed increases to 2.4 m/s ?

1. Reaction travel (d_{react}): During the 65 ms sensing and compute delay, the chassis coasts unguided:

$$d_{\text{react}} = v_0 \Delta t_{\text{delay}} = (1.8 \text{ m/s}) \times (65 \text{ ms}) \approx 0.117 \text{ m} = 11.7 \text{ cm}$$

2. Braking travel (d_{brake}): Dissipating kinetic energy ($E_k = \frac{1}{2} m v_0^2 = 243 \text{ J}$) under sustained mechanical deceleration:

$$d_{\text{brake}} = \frac{v_0^2}{2a_{\text{brake}}} = \frac{(1.8 \text{ m/s})^2}{2 \times (3 \text{ m/s}^2)} \approx 0.540 \text{ m} = 54 \text{ cm}$$

3. Total defended stopping envelope (d_{stop}):

$$d_{\text{stop}} = d_{\text{react}} + d_{\text{brake}} + \delta_{\text{loc}} + \delta_{\text{margin}} = 11.7 \text{ cm} + 54 \text{ cm} + 5 \text{ cm} + 10 \text{ cm} = 80.7 \text{ cm}$$

4. Scaling impact of 33 percent speed increase ($v_0 = 2.4 \text{ m/s}$):

$$d_{\text{react}} = (2.4 \text{ m/s}) \times (65 \text{ ms}) = 15.6 \text{ cm} \quad (+33.3\%)$$

$$d_{\text{brake}} = \frac{(2.4 \text{ m/s})^2}{2 \times (3 \text{ m/s}^2)} = 96 \text{ cm} \quad (+77.8\%)$$

$$d_{\text{stop}} = 15.6 \text{ cm} + 96 \text{ cm} + 15.0 \text{ cm} = 126.6 \text{ cm} \quad (+56.9\%)$$

A 33 percent speed increase expands stopping distance by nearly 57 percent, eroding 45.9 cm of clearance.

Architect's Rule of Thumb: Doubling travel speed quadruples kinetic braking distance while doubling reaction travel; never increase velocity setpoints without dynamically solving the clearance quadratic.

Example 4.1: Dynamic velocity ceiling governor

Inputs:

- $D_{\text{clear}} \in \mathbb{R}_{\geq 0}$: Detected obstacle distance along travel vector [m].
- $\Delta t_{\text{delay}} \in \mathbb{R}_{> 0}$: Conservatively bounded sense-to-braking delay, including actuator response [s]; a measured $P_{99.9}$ alone is insufficient.
- $a_{\text{brake}} \in \mathbb{R}_{> 0}$: Conservative lower bound on available brake deceleration under the declared load and surface [m/s^2].
- $\delta_{\text{loc}}, \delta_{\text{margin}} \in \mathbb{R}_{\geq 0}$: Localization uncertainty bound and physical clearance margin [m].
- $v_{\text{now}} \in \mathbb{R}_{\geq 0}$: Current forward speed [m/s].
- $v_{\text{cand}} \in \mathbb{R}_{\geq 0}$: Candidate forward velocity setpoint proposed by neural policy [m/s].

Output:

- $v_{\text{safe}} \in \mathbb{R}_{\geq 0}$: Clamped velocity request within the modeled ceiling while the current state remains feasible [m/s].

Conditional Admission Check (while the current state is feasible):

1. Modeled stopping distance: $d_{\text{stop}}(v_{\text{safe}}) = v_{\text{safe}} \Delta t_{\text{delay}} + \frac{v_{\text{safe}}^2}{2a_{\text{brake}}} + \delta_{\text{loc}} + \delta_{\text{margin}} \leq D_{\text{clear}}$ under bounded delay, available braking, a stationary obstacle, and valid clearance estimates.
2. Non-Negativity: $v_{\text{safe}} \geq 0.0$.

Algorithm Steps:

1. Compute available physical braking distance:

$$D_{\text{avail}} \leftarrow D_{\text{clear}} - \delta_{\text{loc}} - \delta_{\text{margin}}$$

2. If $D_{\text{avail}} \leq 0.0$, reject the proposal; hold at zero if stationary, or request the maximum available risk-reducing brake response if moving, and flag lost clearance. Otherwise check $v_{\text{now}} \Delta t_{\text{delay}} + v_{\text{now}}^2 / (2a_{\text{brake}}) \leq D_{\text{avail}}$ before admitting another proposal. If that check fails, request the same emergency response and flag lost feasibility. A zero velocity setpoint cannot stop existing motion immediately.
3. Compute quadratic discriminant:

$$\text{disc} \leftarrow (a_{\text{brake}} \Delta t_{\text{delay}})^2 + 2a_{\text{brake}} D_{\text{avail}}$$

4. Solve positive physical root for dynamic velocity ceiling:

$$v_{\text{max}} \leftarrow -a_{\text{brake}} \Delta t_{\text{delay}} + \sqrt{\text{disc}}$$

5. Enforce dynamic clamping against policy proposal:

$$v_{\text{safe}} \leftarrow \min(v_{\text{cand}}, \max(0.0, v_{\text{max}}))$$

6. Return v_{safe} .

In Class 1 machines (autonomous mobile robots, warehouse transport carts, and dynamic quadrupeds), the primary physical failure mode is **spatial boundary breach under unmanaged momentum**—the penetration of defended safety boundaries when a moving chassis cannot dissipate its stored kinetic energy within available clearance. Because kinetic energy $E_k = \frac{1}{2}mv^2$ cannot be recalled once delivered to the chassis, a delay spike extends reaction travel linearly ($d_{\text{react}} = v_0 \Delta t_{\text{delay}}$), eroding clearance before brakes engage. An independent real-time permission path can clamp a proposed forward velocity to the modeled ceiling $v_{\text{max}}(D_{\text{clear}}, \Delta t_{\text{delay}})$ while the current state remains feasible under its declared bounds.

The quadratic turns a fixed speed limit into a clearance-dependent ceiling for new proposals. A real-time permission mechanism can solve it on each control cycle, but the ceiling preserves stopping feasibility only if the machine enters the cycle inside the feasible set and can apply the assumed brake force within the bounded delay. Control Barrier Functions (CBF-QP) (Ames et al. 2019) extend this reasoning to more general systems under valid dynamics, state estimates, timing bounds, and feasible control authority; the 1D root here is a simplified kinematic admission check. If range degrades or delay exceeds its bound, the permission path must reduce speed before margin is exhausted or execute a defined risk-reducing fallback. Once stopping distance exceeds available clearance, a later clamp cannot restore the lost margin.

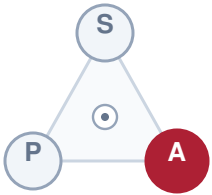
Enforcing this speed limit assumes the nervous system can demand the deceleration a_{brake} whenever safety requires it. Yet commanding rapid deceleration requires sudden, large mechanical torque from the drive motors and gearboxes. If the nervous system requests a torque step that exceeds the actuator authority budget established in section 4.1, the mechanical body cannot deliver the required deceleration, invalidating the model-based stopping ceiling and risking collision.



Checkpoint 4.2: Kinetic momentum and dynamic speed limits

Before examining actuator mechanics and gearing, verify your understanding of stopping distance envelopes:

- ☐ **Linear versus quadratic scaling:** Can you explain why doubling compute latency expands the stopping envelope linearly while doubling travel speed expands it quadratically?
- ☐ **Dynamic velocity governor root:** Can you derive why solving $d_{\text{stop}} \leq D_{\text{clear}}$ for maximum safe speed requires the positive root of a quadratic inequality rather than a linear clamp?
- ☐ **Spatial overhead margins:** Can you distinguish the physical roles of localization uncertainty (δ_{loc}) versus protective clearance margin (δ_{margin}) in defining a defended stopping boundary?



The Act axis: non-smooth contact mechanics, reflected inertia, and thermal dissipation.

⁹ | **Field-Oriented Control (FOC) Current Loop:** In digital motor drives, FOC current loops execute in dedicated gate driver hardware at 10–25 kHz. The instantaneous phase current derivative $di/dt = (V_{\text{bus}} - e_{\text{bemf}})/L$ is physically bounded by the motor's phase inductance L and the opposing back-EMF voltage $e_{\text{bemf}} = K_e \omega$. When an upstream neural policy commands step changes in torque, this inductive and back-EMF headroom ceiling delays torque realization, converting high-level mathematical steps into continuous, ramp-limited physical actions.

4.4 Actuator Transmission Limits

During a high-speed precision assembly cycle, a 6-axis articulated industrial robot arm attempts to seat a machined steel pin into a tight-tolerance bore. Building on the causal boundary established in section 2.2, an electromechanical actuator possesses no software abstraction to catch invalid parameters or return an error status. When a learned neural policy commands a step change in joint torque beyond what the magnetic field can generate, or requests a discontinuous acceleration that strains gear teeth, the physical actuator does not report a software error. It pulls maximum phase current from the power stage, twists transmission shafts, and strains gear teeth while digital command registers in memory continue to report nominal execution. **An actuator may damage itself instead of reporting an error.**

An actuator contract cannot be specified in terms of digital setpoints. It must be written at the physical interface in terms of delivered force, torque, or mass flow, response latency, slew rate limits, and output error. A digital setpoint is merely a software request; physical action is bounded by winding inductance, bus voltage ceilings, magnetic saturation, and transmission elasticity.

The delay between commanding a torque and delivering it at the motor shaft begins in the stator windings. Because motor coils possess physical inductance, current cannot change instantaneously. When a voltage is applied, current (and therefore torque) ramps up smoothly, governed by the electrical time constant of the motor (the complete differential derivation of the electrical RL time constant is detailed in Appendix C). For a typical robotic joint actuator, delivering 95 percent of the target torque might take nearly 10 milliseconds. If an upstream policy assumes torque appears instantaneously, this interval passes as unguided drift and phase erosion before the arm can initiate corrective acceleration or braking.

In modern robotic actuators, this electromagnetic torque is synthesized through Field-Oriented Control (FOC) executing on dedicated real-time drive silicon at 10–25 kHz.⁹ From a systems perspective, the governing intuition is straightforward: driving current through the stator windings produces electromagnetic Lorentz torque in direct proportion to current ($\tau = K_t I$). However, even

when an upstream policy issues an instantaneous torque request, the rate at which coil current can ramp up is physically constrained by the available DC bus voltage headroom:

$$\frac{di}{dt} \leq \frac{V_{\text{bus}} - e_{\text{bemf}}}{L}$$

As the motor spins, its spinning rotor magnets simultaneously turn the actuator into an electrical generator—an **opposing dynamo** producing an internal voltage known as back-electromotive force ($e_{\text{bemf}} = K_e \omega$) that acts directly against the power supply. At low rotational speeds, generous voltage headroom ($V_{\text{bus}} - e_{\text{bemf}}$) permits rapid current rise rates and full peak torque. But as motor speed ω climbs toward its base rating ω_{base} , this opposing dynamo effect consumes the available voltage headroom, squeezing ($V_{\text{bus}} - e_{\text{bemf}}$) toward zero. Once headroom collapses, the motor inverter cannot force additional current through the winding inductance regardless of gate duty cycle, capping both current rise rate (di/dt) and peak torque capability. When a learned policy commands explosive acceleration at high operating speeds, the actuator cannot deliver the requested torque step simply because physical electrical headroom has vanished.

Even after the electrical rise time elapses, delivered torque is bounded by hard current limits in the drive electronics. Power inverters enforce a maximum current ceiling I_{max} to protect switching transistors from thermal destruction and to prevent ferromagnetic saturation in the stator core. When the motor drives a mechanical transmission with gear reduction ratio N and transmission efficiency η , the ceiling on delivered output torque is:

$$\tau_{\text{out, max}} = \eta N K_t I_{\text{max}}$$

When a high-level policy demands $120 \text{ N} \cdot \text{m}$ of braking torque to avoid an obstacle, it assumes the motor drive will deliver it. But on real hardware, inverter current limits ($I_{\text{max}} = 25 \text{ A}$) and gear efficiency cap peak output torque at $85 \text{ N} \cdot \text{m}$. The missing $35 \text{ N} \cdot \text{m}$ is silently clipped at the power stage. Because the software planner remains unaware of the current clamp, the machine's actual stopping distance expands beyond its calculated safety boundary.

Transmissions introduce a severe mechanical asymmetry that software planners routinely ignore. To deliver high torque from a compact motor, robot joints rely on gear reductions with ratios N ranging from $50 : 1$ to $160 : 1$. While gear reduction scales delivered torque linearly by N , it forces the motor rotor to spin N times faster than the joint.

Because kinetic energy scales with the square of speed ($\frac{1}{2} J \dot{\theta}^2$), the effective resistance of the motor's rotor to joint acceleration scales with the **square of the gear ratio**:

$$J_{\text{ref}} = N^2 J_{\text{rotor}}$$

This **reflected rotor inertia** means that in a $100 : 1$ transmission, a tiny 50 g motor rotor ($J_{\text{rotor}} \approx 5.6 \times 10^{-6} \text{ kg} \cdot \text{m}^2$) acts at the joint as if it has an inertia of $J_{\text{ref}} = 100^2 \times J_{\text{rotor}} = 0.056 \text{ kg} \cdot \text{m}^2$ —equivalent to the rotational inertia of an 11.2 kg solid steel flywheel with a 10 cm radius (Lynch and Park 2017).

To build physical intuition, consider trying to swing a sledgehammer holding the very tip of the handle versus choking up near the iron head, or riding a multi-speed bicycle up a steep hill in the highest gear ($N \gg 1$). In high gear, every tiny push on the pedal requires massive leg force because your legs fight the high mechanical leverage of the rear wheel. Now reverse the perspective: spinning the rear wheel backward by hand also has to accelerate the pedal crank. The reflected inertia makes backdriving harder, but does not lock the wheel. In a robotic arm with a $100 : 1$ strain-wave transmission, an obstacle impacting the gripper must accelerate a rotor whose reflected inertia is comparable to the stated 11.2 kg flywheel. Compliance, backlash, contact geometry, and any active control determine how the impact energy divides; tooth-root stress can rise enough to cause fatigue fracture (figure 4.5).

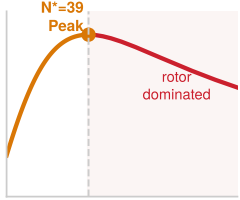
The combined equation of motion for a geared joint driving load inertia J_{load} with motor torque τ_{motor} is:

$$\ddot{q} = \frac{N \tau_{\text{motor}}}{J_{\text{load}} + N^2 J_{\text{rotor}}}$$

Differentiating with respect to gear ratio N reveals that peak joint acceleration occurs at the **inertia matching ratio**:

$$N^* = \sqrt{\frac{J_{\text{load}}}{J_{\text{rotor}}}}$$

When $N < N^*$, joint acceleration is torque-limited. When $N > N^*$, counter-intuitively, increasing the gear ratio *decreases* delivered joint acceleration, because the motor spends the majority of its electromagnetic torque accelerating its own spinning rotor rather than moving the robot's physical link.



Past the inertia-matching ratio, gearing chokes delivered joint acceleration.

Because the actuator cannot instantly deliver the massive torque needed to spin up this amplified inertia, the reflected inertia acts as a dynamic mechanical low-pass filter—smearing the software's sharp, instantaneous command into a sluggish, delayed physical response. This delivered output motion severely lags behind the policy's candidate setpoint, injecting unmodeled phase lag that erodes feedback stability, excites structural resonance, and spikes mechanical stress across the gear teeth. Motion-profile filtering (such as quintic polynomial splines or S-curve rate limiters) must be enforced by the real-time microcontroller to guarantee that requested joint acceleration $\ddot{q}$ remains within the current and voltage envelopes of the hardware (Lynch and Park 2017).

Furthermore, transmissions introduce backlash and compliance that decouple motor and load motion. When an actuator reverses direction, the motor shaft must rotate across mechanical clearance before the gear teeth engage the opposite flank. Coupled with torsional compliance in gear teeth and harmonic flexsplines (figure 4.4), reversal manifests at the output as lost motion, hysteresis, and phase lag. As Mark W. Spong demonstrated in his foundational singular perturbation modeling of flexible joint robotics (Spong 1987), this joint elasticity means the robot arm is not a perfectly rigid bar, but rather a dynamical system coupled through torsional spring compliance. This decouples motor rotor dynamics from link inertia, introducing a high-frequency vibration manifold that cannot be suppressed by rigid-body controllers. When neural policies command discontinuous torque steps across timestep boundaries, they apply impulsive harmonic excitation across this compliant boundary, exciting unmodeled high-frequency resonant modes and driving the transmission into destructive limit-cycle vibrations and chattering. Measuring position solely at the motor encoder conceals this deadband, because the motor turns smoothly while the physical output remains stationary until contact is made. Under reversing loads, the actual joint position q_{out} departs from the ideal kinematic projection q_{motor}/N by a load-dependent error δ_{lost} . The nervous system perceives continuous trajectory tracking, yet the physical end-effector hesitates and then experiences an impulsive shock load as tooth engagement snaps closed. Under severe repetitive shock loads or high reflected inertia impacts, this localized stress concentration leads to fatigue fracture and catastrophic tooth shear (figure 4.5).

🔍 Systems Perspective 4.2: Impedance control for contact safety

If a learned policy commands target positions as rigid coordinates in an unyielding environment (such as a table or fixture with contact stiffness $k \geq 10^5$ N/m), a positional error of just $\Delta x = 1$ mm causes contact forces to spike to $F = k\Delta x = 100$ N within milliseconds. If the low-level controller tries to force the arm into that target coordinate at all costs, the motor inverter drives maximum current against the obstacle, risking tooth shear and structural damage.

As Neville Hogan established in his seminal formulation of impedance control (Hogan 1985), physical interaction cannot be framed as pure position control or pure force control. Because the physical environment dynamically behaves as an admittance (accepting commanded forces and yielding resultant motion), the robotic manipulator must be controlled as an impedance (accepting motion deviations and generating compliant contact forces: $F = Z(v)$). To protect the physical mechanism, the real-time microcontroller converts position setpoints into an active **virtual spring-damper system**:

$$F_{\text{contact}} = K_p(x_{\text{des}} - x) + K_d(\dot{x}_{\text{des}} - \dot{x})$$

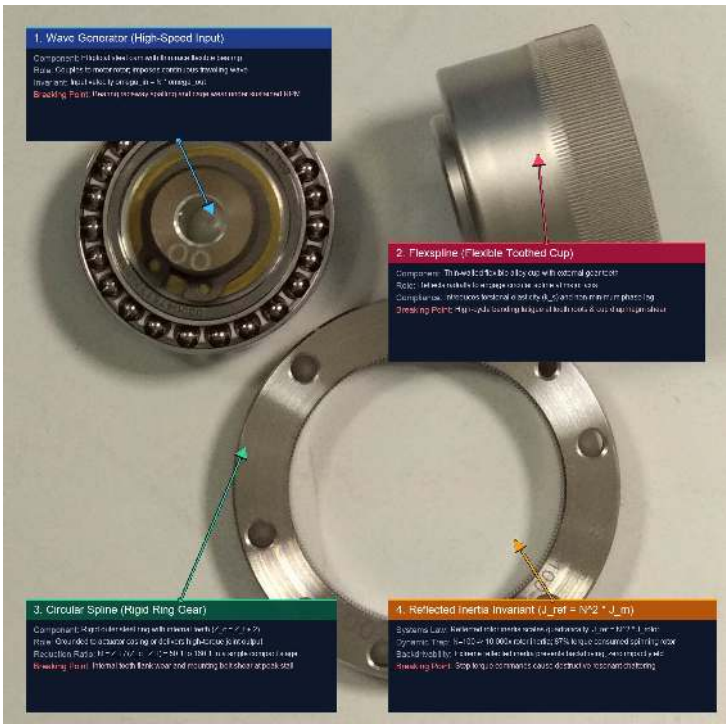


Figure 4.4: Strain wave gear assembly: Robotic strain wave transmission component set: (1) elliptical wave generator with flexible ball bearings, (2) flexible cup flexspline with external teeth, (3) rigid circular spline with internal teeth, and (4) transmission systems invariant governing reflected rotor inertia ($J_{ref} = N^2 J_m$). Elastic deformation enables zero initial backlash at high reduction ratios (50 : 1 to 160 : 1), but introduces torsional compliance, hysteresis, and tooth fatigue vulnerability under sudden shock loads.

Rather than acting like an immovable steel bar, the physical joint behaves like a programmable mechanical suspension with virtual stiffness K_p and damping K_d . When contact occurs, the mechanism yields compliantly, capping interaction forces and giving high-level software the physical margin of time needed to detect contact and replan before destructive stress shears gear teeth.

Selecting an appropriate mechanical transmission architecture establishes the fundamental systems trade-offs between continuous torque density and backdrivability, as summarized in table 4.2.

Table 4.2: Comparative systems matrix of actuator transmission mechanisms: Quantitative trade-offs across robotic drive mechanisms, mapping reduction ratio (N), reflected rotor inertia scaling (N^2), and operational implications for learned control policies.

Transmission Archetype	Gear Ratio (N)	Reflected Inertia (N^2)	Implication for Physical AI Policies
Direct Drive (No Gearing)	1 : 1 ($N = 1$)	$N^2 = 1$ (Minimal)	Transparently backdrivable; absorbs impact shocks; limited continuous torque density ($\sim 2\text{--}5\text{ N}\cdot\text{m/kg}$).
Quasi-Direct Drive (QDD)	3 : 1–10 : 1 ($N \leq 10$)	$N^2 \leq 100$ (Low)	High backdrivability; enables proprioceptive torque sensing via motor current; high impact tolerance for dynamic quadrupeds and arms.
Cycloidal Drive (RV Reducer)	30 : 1–100 : 1 ($N \geq 30$)	$N^2 \geq 900\text{--}10,000$ (High)	Exceptional shock-load tolerance ($3\text{--}5\times$ peak rating via multi-pin contact); minimal backlash; moderate backdrivability.

Transmission Archetype	Gear Ratio (N)	Reflected Inertia (N^2)	Implication for Physical AI Policies
Strain Wave (Harmonic Drive)	50 : 1–160 : 1 ($N \geq 50$)	$N^2 \geq 2500$ –25,600 (Extreme)	Zero initial backlash; compact co-axial form factor; low torsional stiffness (k_s); fragile under sudden tooth shear impact.
Planetary / Roller Screw	$N_{\text{eff}} \geq 50$ (Linear)	$N^2 \geq 2500$ (Extreme)	Transforms rotary to linear motion; massive continuous force density (> 10 kN); non-backdrivable; used in humanoid legs and knees.
Open-Loop Stepper Motor	1 : 1 (High pole count)	$N^2 = 1$	High static holding torque at zero speed; silent state loss (“lost steps”) when commanded acceleration exceeds magnetic pull-out torque.

To see how transmission sizing determines physical responsiveness, consider a bipedal humanoid knee joint that must rapidly accelerate a lower-leg link of inertia $J_{\text{load}} = 0.18 \text{ kg} \cdot \text{m}^2$. The joint is driven by a brushless motor producing peak torque $\tau_{\text{motor}} = 4.0 \text{ N} \cdot \text{m}$ with rotor inertia $J_{\text{rotor}} = 1.2 \times 10^{-4} \text{ kg} \cdot \text{m}^2$. The optimal gear ratio that maximizes delivered joint angular acceleration is $N^* = \sqrt{J_{\text{load}}/J_{\text{rotor}}} = \sqrt{0.18/(1.2 \times 10^{-4})} = \sqrt{1500} \approx 38.7 \approx 39$. At $N^* = 39$, the reflected inertia is $J_{\text{ref}} = (39)^2(1.2 \times 10^{-4} \text{ kg} \cdot \text{m}^2) = 0.1825 \text{ kg} \cdot \text{m}^2$, yielding an optimal joint acceleration of $\ddot{q}_{\text{opt}} = (39 \times 4.0 \text{ N} \cdot \text{m}) / (0.18 + 0.1825 \text{ kg} \cdot \text{m}^2) = 156 / 0.3625 \approx 430.3 \text{ rad/s}^2$. If an engineer naively selects a 100 : 1 harmonic drive to deliver $400 \text{ N} \cdot \text{m}$ of gross torque, reflected rotor inertia expands to $J_{\text{ref}} = 100^2 \times (1.2 \times 10^{-4}) = 1.20 \text{ kg} \cdot \text{m}^2$. Delivered joint acceleration plummets to $\ddot{q}_{100} = (100 \times 4.0) / (0.18 + 1.20) = 400 / 1.38 \approx 289.9 \text{ rad/s}^2$ —a 32.6 percent collapse in responsiveness, despite delivering 2.5 $\times$ more output torque, because 87.0 percent of the motor’s torque ($1.20/1.38$) is consumed accelerating its own spinning rotor. In dynamic locomotion and manipulation, higher gear ratios degrade physical agility; gearboxes must be sized to match reflected inertia against payload dynamics.



Checkpoint 4.3: Transmission sizing and reflected inertia

Before analyzing contact-rich manipulation, verify your understanding of inertia matching and drive dynamics:

- ☐ **Inertia matching condition:** Can you explain why the gear ratio maximizing link acceleration occurs when reflected rotor inertia equals load inertia ($N^2 J_{\text{rotor}} = J_{\text{load}}$)?
- ☐ **Torque vs. responsiveness trade-off:** Can you describe why oversizing a gearbox to deliver higher gross static torque can paradoxically reduce dynamic joint acceleration?
- ☐ **Backdrivability and impact absorption:** Can you distinguish how reflected inertia affects a joint’s ability to absorb shock impacts in quasi-direct drive versus high-reduction harmonic drive transmissions?

In Class 2 machines (articulated robot arms, multi-fingered hands, and contact-rich assembly workcells), the primary physical failure mode is **destructive contact force spikes and drive-train tooth shear**. When a robot makes physical contact with an unmodeled workpiece or rigid environment, the interaction is governed by the non-smooth mechanics of unilateral contact, Coulomb friction cones, and kinematic constraints (Mason 2001). As Matthew T. Mason formalized in his foundational analysis of manipulation mechanics (Mason 2001), physical interaction cannot be treated as smooth Euclidean regression: contact modes switch discontinuously between sticking, sliding, and separation, and stable grasp maintenance demands satisfying form-closure (wrapping around an object so it physically cannot escape) or force-closure (squeezing hard enough that friction prevents slipping) constraints under friction. If a policy commands tangential shear forces outside the friction cone ($F_{\text{tangential}} > \mu F_{\text{normal}}$), slip occurs instantly; conversely, if an unexpected rigid obstacle of stiffness k is struck, contact force spikes at $F = k\Delta x$.

If the joint employs a high gear reduction ($N \geq 50$), the reflected rotor inertia $J_{\text{ref}} = N^2 J_{\text{rotor}} \geq 2500 J_{\text{rotor}}$ resists backdriving. Depending on compliance, control response, and contact path, impact energy can deform the transmission, move the load, or partly regenerate electrically; tooth-root



Figure 4.5: Gear tooth shock fracture under high reflected inertia: Macro-fracture of a high-torque transmission gear tooth undergoing cyclic stress testing at NASA Glenn Research Center: [] intact gear tooth flank supporting continuous drive contact; [] tooth root fillet stress concentration ($\sigma_b = \frac{K_t W_t}{b m Y^2}$), where cyclic tension initiates micro-cracking; [] sheared tooth fragment with granular fatigue striations and sudden cleavage fracture under shock load; and [] reflected rotor inertia ($J_{ref} = N^2 J_m$). A high gear ratio reduces backdrivability and can increase tooth-root stress under impact; compliance and the contact path determine how much energy reaches any one tooth.

stress may rise sharply. For contact-dominated manipulation, neural policies can use backdrivable Quasi-Direct Drive (QDD) actuators—developed for the MIT Cheetah by Seok et al. (2015) and Wensing et al. (2017), with low gear reduction ($N \leq 10$) and lower reflected inertia ($N^2 \leq 100$)—or remain behind real-time torque limits and impedance control.

The control path must limit requested acceleration and jerk to what the motor current, voltage, and transmission can deliver. A motor-side encoder alone can report smooth rotation while backlash leaves the output stationary or a damaged tooth lets the load lag. Load-side position or torque sensing reveals that disagreement; the permission path can then reject further force commands and enter a defined fallback.

Operating continuously near current and torque limits extracts an irreversible thermodynamic cost. Every ampere driven through the stator winding to overcome rotor inertia or sustain peak output torque dissipates resistive heat ($I^2 R$) directly into the copper coils. While magnetic flux builds within milliseconds, the heat it generates accumulates in the physical thermal mass of the actuator over seconds and minutes, creating a thermal clock that bounds how long peak torque can be delivered before insulation breaks down.

4.5 Thermal Duty Cycles

Holding a 25 kg workpiece motionless places deceptive demands on articulated robots. In simulation, an AI agent sustains static holding torque indefinitely without penalty. On physical hardware, maintaining a static load against gravity demands continuous phase current through the stator coils, actively converting electrical energy into accumulating heat. **Heat arrives in seconds and leaves in minutes, making duty cycle a primary design variable.**

While mechanical momentum can be redirected and magnetic flux decays within milliseconds of opening a switch, thermal energy has no fast physical exit. Thermal capacity is the physical integral of power loss over time. What that capacity protects is a specific, inflexible material threshold, namely the dielectric breakdown rating of the stator winding insulation (the enamel coating that prevents adjacent copper turns from short-circuiting) and the Curie temperature of permanent rotor magnets (the thermal threshold where thermal agitation destroys ferromagnetic domain alignment, permanently demagnetizing the rotor) (figure 4.6). Modern motor windings use polymer enamel coatings classified under standards such as IEC 60085, where Class F insulation withstands a maxi-

Downstream: Mechanical gearhead fatigue cycles set the sample efficiency limits for reinforcement learning in section 12.3.

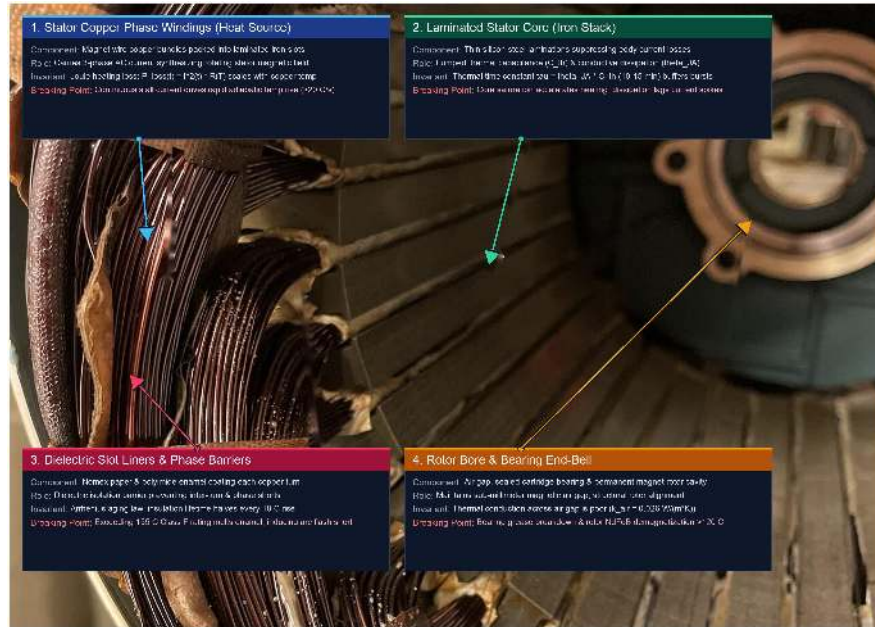


Figure 4.6: Stator winding insulation breakdown: Cutaway view of a damaged electric motor stator showing critical thermal subsystems: (1) copper phase windings (primary I^2R heat source), (2) dielectric slot liners and Nomex insulation breakdown threshold (155°C , Class F), (3) laminated iron stator core (C_{th} and dissipation path θ_{JA}), and (4) rotor bore air gap and magnet thermal limits (80°C – 150°C). (Source: Industrial Electromechanical Diagnostics Archive, CC BY 4.0).

imum continuous hotspot temperature of 155°C and Class H withstands 180°C . Exceeding this rating softens the polymer matrix, causing inter-turn shorts that destroy the phase winding. Neodymium-iron-boron (NdFeB) rotor magnets face a companion limit, where temperatures between 80°C and 150°C cause irreversible demagnetization under strong stator demagnetizing fields.

The rate of heat generation follows directly from the current commanded through the motor phases. Resistive Joule dissipation dumps thermal power into the stator copper according to the relation $P_{\text{loss}}(t) = I^2(t)R(T)$.¹⁰ This thermal power divides between rapidly heating the physical thermal mass of the winding and slowly conducting heat out to the motor casing and surrounding air (see Appendix C for the complete thermal ODE integration and steady-state derivation).

Visualize thermal resistance (θ_{JA}) and thermal capacitance (C_{th}) as a **leaky bucket**. Resistive Joule losses (I^2R) pour water into the bucket like an open firehose. The volume of the bucket represents the thermal mass or capacitance of the copper windings and laminated stator iron (C_{th}): a larger bucket can buffer an intense burst of water for a few seconds without spilling over. At the bottom of the bucket is a narrow drain hole representing thermal dissipation to the environment through thermal resistance (θ_{JA}): the higher the thermal resistance, the narrower the drain hole, and the slower heat escapes into surrounding air. If a policy demands an intense 3-second peak torque burst (a sprint across an obstacle), the water level rises rapidly, but as long as the bucket does not overflow (crossing the 155°C Class F dielectric rating), the winding survives safely. However, if an agent commands continuous high holding torque (such as suspending a heavy tool against gravity indefinitely), the water level climbs until the inflow equals the maximum outflow through the drain hole. If the continuous inflow exceeds what the drain hole can pass at safe levels, the bucket overflows—softening the polymer enamel slot liners and inducing an irreversible electrical short circuit.

Because the thermal time constant of the motor ($\tau = \theta_{JA}C_{th}$) is typically on the order of minutes while resistive heating accumulates on the order of seconds, thermal dissipation behaves as a low-pass filter on power losses. In the steady-state, the equilibrium temperature is governed strictly by the balance between average resistive losses and the thermal resistance to the environment.

¹⁰ | **Stator Resistance Thermal Sensitivity:** The electrical resistivity of copper exhibits a positive temperature coefficient $\alpha_{Cu} \approx 0.00393 \text{ K}^{-1}$, causing winding resistance to scale linearly as $R(T) = R_0[1 + \alpha_{Cu}(T - T_0)]$. Across a motor's operating range from 20°C to 140°C , stator resistance increases by 47.2%, demanding higher terminal voltages to sustain identical phase currents. For high-level policies holding heavy static payloads, this thermal feedback escalates resistive Joule losses ($I^2R(T)$) into accelerated thermal runaway unless actively tracked by low-level thermal observers.

Building on the causal boundary established in section 2.2, this formulation exposes why stator thermal limits create a hard, non-negotiable physical boundary for machine learning policies. Neural policies trained in simulation environments (such as Isaac Gym, MuJoCo, or PyBullet) optimize reward functions where torque is treated as an instantaneous, massless mathematical variable. In simulation, a policy can hold a heavy payload indefinitely, stiffen joint impedance through continuous high-torque antagonist co-contraction, or chatter rapidly across contact boundaries without incurring any physical consequence. On real physical hardware, sustained current I pumps thermal energy into stator copper at an adiabatic initial rate $dT/dt \approx I^2 R/C_{th}$, yielding an initial quasi-adiabatic heating regime where thermal energy is trapped within the winding's thermal capacitance before conduction and convection can establish an equilibrium heat flux.

This thermodynamic cost is compounded by the positive temperature coefficient of copper ($\alpha_{Cu} \approx 0.00393 \text{ K}^{-1}$). As the stator temperature climbs from 20°C toward its 140°C engineering ceiling, phase resistance increases by over 47 percent according to the linear relation $R(T) = R_0[1 + \alpha_{Cu}(T - T_0)]$. To sustain identical commanded torque ($\tau = K_t I$), the inverter must deliver higher phase voltage, dissipating 47 percent more resistive power ($I^2 R(T)$) at the exact moment thermal cooling is least effective. This establishes a positive thermal feedback loop: higher winding temperatures elevate resistance, requiring higher terminal voltages and dissipating more resistive power to deliver identical electromagnetic torque, which further accelerates thermal rise. If a neural policy operates without constraints on torque commands, it can lead to a positive thermal feedback loop. This loop causes an increase in winding temperatures, which raises electrical resistance. As a result, higher terminal voltages are required to maintain the same electromagnetic torque, potentially accelerating the winding toward dielectric breakdown within seconds. Understanding these thermal dynamics is essential for designing robust control systems that can prevent overheating and ensure reliable operation.

In Class 3 systems (grid-scale battery energy storage, liquid cooling distribution units, and continuous thermal regulation plants), the primary physical failure mode is **irreversible thermal accumulation and runaway**. Because heat arrives in seconds ($dT/dt \approx P_{loss}/C_{th}$) but leaves across minutes through thermal resistance (θ_{JA}), learned policies regulating fluid and power plants cannot optimize short-term throughput at the expense of steady-state heat balance without inducing unrecoverable thermal trips.

Datasheet thermal specifications provide an optimistic baseline because manufacturers measure them under idealized laboratory conditions, typically mounting an unconstrained motor to a large aluminum heat sink in free air at 20°C or 25°C . On an operational machine, the actuator operates inside a sealed chassis alongside power electronics, battery packs, and adjacent joint motors. Enclosing the motor eliminates ambient convective airflow, increasing the effective thermal resistance θ_{JA} . Furthermore, neighboring computation and power conversion stages dump heat into the shared structural chassis, raising the local internal ambient temperature T_{amb} well above external room temperature. A joint actuator operating near a graphics processor or battery pack can experience a local ambient of 45°C to 55°C before its own coils carry current. Accurately budgeting thermal margins requires measuring the installed heating and cooling time constants with the enclosure fully sealed and all adjacent electrical loads operating at their P_{99} power draw.

Because thermal capacity integrates power over time, a peak torque¹¹ or peak current rating on a motor datasheet is physically meaningless without four accompanying parameters, specifically the initial winding temperature $T(0)$, the maximum allowable duration t_{on} , the operating ambient temperature T_{amb} , and the required recovery duration t_{off} . An actuator capable of producing three times its continuous rated torque can sustain that load only while the integral of $I^2 R$ remains within the thermal margin below the insulation limit. If the winding begins at room temperature, that burst can last tens of seconds. If the winding is already soaked at its continuous operating temperature from sustained motion, the identical current pulse reaches the insulation threshold in a fraction of that time.

Consider the thermal budgeting for the shoulder joint of our heavy-payload arm under continuous rated load and transient peak acceleration (4.2).

Downstream: Real-time telemetry for thermal and power integrity budgets is transported by the deterministic fieldbuses analyzed in section 8.4.

11 | Actuator Intermittent Duty Limits: Robotics motor datasheets prominently advertise peak torque (τ_{peak}), but this rating is strictly intermittent and survivable for only 1–3 seconds before coil thermal limits are breached. Sustained dynamic maneuvering must be budgeted against continuous root-mean-square torque ($\tau_{cont} \approx 0.3\text{--}0.4 \tau_{peak}$), otherwise aggressive neural policy commands will trip thermal protection and cause mid-motion limb collapse.


Example 4.2: Transient thermal accumulation under cold vs. hot starts

An articulated arm shoulder joint with winding resistance $R = 1.2 \, \Omega$, lumped thermal capacitance $C_{th} = 120 \, \text{J/K}$, and installed thermal resistance $\theta_{JA} = 2.5 \, \text{K/W}$ operates in an enclosed internal ambient $T_{amb} = 40^\circ\text{C}$ with Class F insulation rated to 155°C . The thermal time constant is $\tau = \theta_{JA} C_{th} = 300 \, \text{s}$ (5.0 min).

1. Continuous thermal equilibrium ($I_{cont} = 5.0 \, \text{A}$):

- Steady-state resistive loss:

$$P_{cont} = I_{cont}^2 R = (5.0 \, \text{A})^2 \times 1.2 \, \Omega = 30.0 \, \text{W}$$

- Equilibrium temperature rise:

$$\Delta T_{cont} = P_{cont} \theta_{JA} = 30.0 \, \text{W} \times 2.5 \, \text{K/W} = 75.0 \, \text{K}$$

- Continuous operating temperature: $T_{eq} = T_{amb} + \Delta T_{cont} = 40^\circ\text{C} + 75.0^\circ\text{C} = 115.0^\circ\text{C}$ (safely below the 155°C limit).

2. Peak torque power dissipation ($I_{peak} = 15.0 \, \text{A}$):

- Commanding $3.0\times$ rated current to accelerate a workpiece jumps power dissipation by a factor of nine:

$$P_{peak} = I_{peak}^2 R = (15.0 \, \text{A})^2 \times 1.2 \, \Omega = 270.0 \, \text{W}$$

- Asymptotic winding temperature if sustained indefinitely:

$$T_{\infty} = T_{amb} + P_{peak} \theta_{JA} = 40^\circ\text{C} + 270.0 \, \text{W} \times 2.5 \, \text{K/W} = 715.0^\circ\text{C}$$

3. Cold-start survival horizon ($T(0) = 40^\circ\text{C}$):

- Starting cold from ambient, the time t to reach the 155°C insulation limit satisfies:

$$155 = 40 + (715 - 40)(1 - e^{-t/300}) \Rightarrow t = -300 \ln \left(1 - \frac{115}{675} \right) \approx 56.0 \, \text{s}$$

4. Hot-start survival horizon ($T(0) = 115^\circ\text{C}$):

- If the joint is already heat-soaked at continuous thermal equilibrium (115°C):

$$155 = 715 - (715 - 115)e^{-t/300} \Rightarrow t = -300 \ln \left(\frac{560}{600} \right) \approx 20.7 \, \text{s}$$

Architect takeaway: A motor pre-soaked at operating temperature burns through its thermal margin 63 percent faster than a cold motor (20.7 s vs. 56.0 s). Peak torque cannot be budgeted without tracking thermal state history.

For repeated motion cycles with period $t_{cycle} = t_{on} + t_{off}$, an engineering ceiling $T_{max} = 140^\circ\text{C}$ leaves a 15°C buffer below the insulation limit and allows a 100 K rise above the stated ambient. Its corresponding cycle-average power ceiling is $\bar{P}_{max} = 100 \, \text{K} / 2.5 \, \text{K/W} = 40.0 \, \text{W}$. In the simplified example with fixed $R = 1.2 \, \Omega$, peak dissipation is $P_{peak} = 270.0 \, \text{W}$ during t_{on} and zero during t_{off} . The average-power screening condition $\bar{P} = D P_{peak} \leq 40.0 \, \text{W}$, where $D = t_{on} / t_{cycle}$, gives

$$D \leq \frac{T_{max} - T_{amb}}{P_{peak} \theta_{JA}} = \frac{40.0 \, \text{W}}{270.0 \, \text{W}} \approx 0.148 \quad (14.8\%)$$

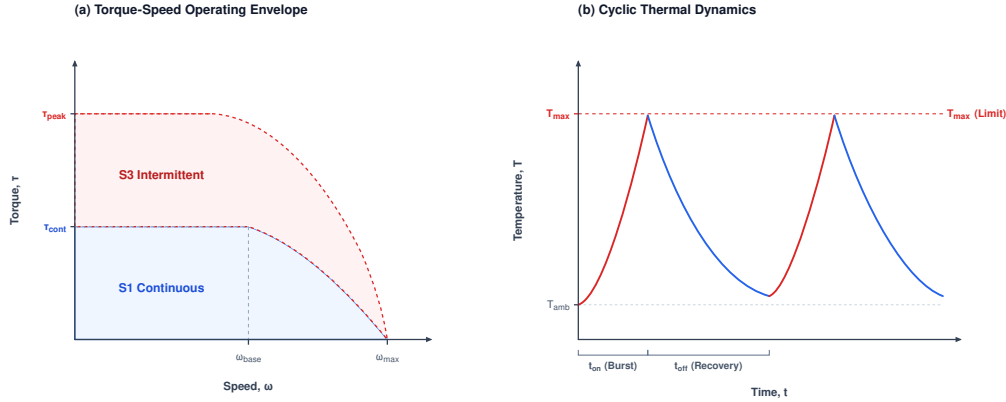


Figure 4.7: Motor torque-speed operating envelope: The permanent magnet synchronous motor torque-speed plane, separating continuous S_1 thermal equilibrium from intermittent S_3 peak acceleration, alongside the exponential thermal accumulation trajectory during peak bursts and conductive cooling during recovery. Winding thermal limits constrain sustainable duty cycle rather than peak inverter current.

This average condition does not bound the hottest instant in a cycle. A design must check the periodic peak with the transient thermal model and account for the rise of $R(T)$ before accepting a burst schedule.

Because Joule heat arrives in seconds ($P = I^2 R$) while dissipation through conduction and natural convection takes minutes ($\tau = \theta_{JA} C_{\text{th}} \approx 300$ s), peak bursts need a recovery interval. Rearranging the average-power condition gives a screening value:

$$t_{\text{off}} = t_{\text{on}} \left(\frac{1-D}{D} \right)$$

For a 3.0 s peak burst, that average-power calculation gives $t_{\text{off}} \approx 17.3$ s, but it is not a safe recovery prescription. In the fixed-resistance first-order model, the periodic peak obeys $T_{\text{peak}} - T_{\text{amb}} = P_{\text{peak}} \theta_{JA} (1 - e^{-t_{\text{on}}/\tau}) / (1 - e^{-t_{\text{cycle}}/\tau})$ and exceeds the 140°C ceiling for this schedule. The nervous system must use a longer interval or lower peak current after checking the transient hotspot trajectory. Figure 4.7 distinguishes continuous S_1 operation from intermittent S_3 acceleration; the latter depends on both initial temperature and cooling time.

(The rigorous notebook calculations for thermal spikes and observers are provided in Appendix C.)

Because thermal accumulation is an integral state with time constants spanning hundreds of seconds, an agent trained in simulation has no intrinsic incentive to preserve winding health. The systems engineer must implement a real-time thermal observer directly on the deterministic microcontroller, tracking estimated coil hotspot temperatures by continually integrating measured phase currents. If the estimated temperature crosses a safety ceiling, the microcontroller dynamically throttles the current limits (derating). If the limit is severely breached, it triggers an emergency thermal shutdown, vetoing the learned policy before dielectric breakdown occurs.

To quantify how thermal accumulation limits sustained actuation, consider an articulated robot arm elbow holding static torque against gravity at stall (4.3).

Example 4.3: Static holding torque and thermal stall derating

An articulated robot arm elbow holds a static torque $\tau_{\text{hold}} = 42.5$ N·m to suspend an end-effector tool against gravity at stall ($\omega = 0$). The joint couples a brushless motor ($K_t = 0.10$ N·m/A, cold resistance $R_0 = 0.80$ Ω at $T_0 = 20^\circ\text{C}$, thermal resistance $\theta_{JA} = 1.80$ K/W, Class F limit 155°C) through an $N = 50 : 1$ cycloidal gearbox ($\eta = 0.85$) inside an unventilated limb ($T_{\text{amb}} = 40^\circ\text{C}$, $\alpha_{\text{Cu}} = 0.00393$ K $^{-1}$).

1. Parameters & cold stall power demand:

- Motor shaft torque:

$$\tau_{\text{motor}} = \frac{\tau_{\text{hold}}}{\eta N} = \frac{42.5 \text{ N} \cdot \text{m}}{0.85 \times 50} = 1.0 \text{ N} \cdot \text{m}$$

- Commanded stator current:

$$I = \frac{\tau_{\text{motor}}}{K_t} = \frac{1.0 \text{ N} \cdot \text{m}}{0.10 \text{ N} \cdot \text{m/A}} = 10.0 \text{ A}$$

- Baseline cold dissipation:

$$P_0 = I^2 R_0 = (10.0 \text{ A})^2 \times 0.80 \Omega = 80.0 \text{ W}$$

2. Thermal runaway under positive copper resistance feedback:

- Steady-state temperature rise accounting for copper resistance scaling ($R(T) = R_0[1 + \alpha_{\text{Cu}}(T - T_0)]$):

$$\Delta T = P(T)\theta_{JA} = I^2 R_0[1 + \alpha_{\text{Cu}}(T_{\text{amb}} - T_0 + \Delta T)]\theta_{JA}$$

$$\Delta T = 144.0[1.0786 + 0.00393\Delta T] = 155.32 + 0.5659\Delta T$$

$$\Delta T = \frac{155.32}{0.4341} \approx 357.8 \text{ K}$$

- Asymptotic winding temperature:

$$T_{\text{winding}} = T_{\text{amb}} + \Delta T = 40^\circ\text{C} + 357.8^\circ\text{C} = 397.8^\circ\text{C}$$

This destroys the coils ($397.8^\circ\text{C} \gg 155^\circ\text{C}$ Class F insulation rating).

3. Derating for continuous thermal equilibrium:

- Allowable steady-state temperature rise:

$$\Delta T_{\text{max}} = 155^\circ\text{C} - 40^\circ\text{C} = 115 \text{ K}$$

- Maximum sustainable continuous dissipation:

$$P_{\text{max}} = \frac{\Delta T_{\text{max}}}{\theta_{JA}} = \frac{115 \text{ K}}{1.80 \text{ K/W}} = 63.89 \text{ W}$$

- Hot winding resistance at 155°C :

$$R(155^\circ\text{C}) = 0.80[1 + 0.00393(155 - 20)] \approx 1.224 \Omega$$

- Sustainable continuous stall current:

$$I_{\text{max,cont}} = \sqrt{\frac{P_{\text{max}}}{R(155^\circ\text{C})}} = \sqrt{\frac{63.89 \text{ W}}{1.224 \Omega}} \approx 7.22 \text{ A}$$

- Sustainable continuous holding torque:

$$\tau_{\text{hold,max}} = \eta N K_t I_{\text{max,cont}} = 0.85 \times 50 \times 0.10 \times 7.22 \approx 30.7 \text{ N} \cdot \text{m}$$

Architect takeaway: Resistive self-heating forces a 27.8 percent continuous torque derating relative to cold ratings ($30.7 \text{ N} \cdot \text{m}$ vs. $42.5 \text{ N} \cdot \text{m}$). Continuous gravity holding must be managed with mechanical friction brakes or physical counterweights rather than active electromagnetic stall current.

📌 Checkpoint 4.4: Thermal runaway and stall derating

Before examining power distribution integrity, verify your understanding of thermal dynamics and continuous torque limits:

- ❑ **Resistive self-amplification:** Can you explain how the positive temperature coefficient of copper (α_{Cu}) creates a thermal runaway feedback loop during sustained motor stall?
- ❑ **Continuous vs. peak torque horizons:** Can you describe why an actuator can safely produce three times its rated torque for several seconds but will destroy its winding insulation if commanded at that torque for two minutes?
- ❑ **Mechanical braking vs. active holding:** Can you distinguish why long-duration static postures must be maintained by electromechanical brakes or counterbalances rather than active motor current?

Delivering the large current pulses that generate these thermal loads simultaneously stresses the upstream electrical supply, transferring the physical burden from thermal dissipation in copper to transient current delivery across the power bus.

4.6 Electrical Power Integrity

When an agile quadruped or mobile manipulator executes an explosive evasive maneuver—a rapid multi-joint lunge to avoid an oncoming collision—every drive motor, joint inverter, and neural accelerator demands maximum current simultaneously from a single direct-current distribution bus. In simulation, current flows frictionlessly from an idealized numerical source. On physical hardware, however, power delivery is governed by finite capacity, source impedance, and transient voltage stability. **A machine that browns out mid-motion fails physically; the electrical supply is a shared resource.**

Transient voltage droop (rail sag), regenerative overvoltage, and thermal accumulation are not separate electrical phenomena, but different temporal manifestations of the same bounded energy network. While winding heating reflects energy accumulated over seconds as thermal dissipation in copper, voltage fluctuations occur across microseconds and milliseconds whenever current demand exceeds instantaneous source capacity. Onboard compute units, neural network accelerators, sensing payloads, and motor inverters all draw from a common direct-current distribution bus. When a learned policy commands aggressive, coordinated accelerations across multiple joints, the resulting sum of simultaneous current draws alters the operating voltage delivered to every node on the machine.

The total current demand on the power rail is the sum of all instantaneous consumer currents on the bus. The physical supply cannot deliver infinite current instantaneously. Every electrical distribution bus exhibits a nonzero source impedance,¹² combining the internal resistance of the battery or power supply and the resistance of wiring harnesses, connectors, and protection switches. Bulk decoupling capacitors provide localized charge reservoirs to buffer high-frequency load transients. When the policy demands a rapid step change in total current, the bus voltage sags immediately due to the resistive drop across R_{bus} , inductive kickback across harness inductance L_{bus} , and the charge depletion of local capacitors (the analytical derivation of transient voltage droop is developed in Appendix C). If this voltage drop crosses the system's undervoltage lockout threshold, supervisor silicon asserts a brownout reset, dropping the machine mid-motion.

The opposite boundary occurs during deceleration, when kinetic energy stored in moving mass returns to the electrical bus. **Braking converts mechanical energy into electrical energy that the supply network must absorb or dissipate.** If the source cannot accept reverse current, regenerative energy can charge the local DC link according to $\frac{1}{2}C_{\text{bus}}(V_{\text{final}}^2 - V_{\text{initial}}^2) = E_{\text{regen}}$ (see Appendix C). This relation gives a capacitor-only endpoint under idealized assumptions, not a voltage waveform. Protection must divert energy before the bus exceeds inverter and converter ratings.¹³

¹² | **Power Distribution Bus Impedance:** The total source impedance of an embodied direct-current bus is the lumped sum of battery internal cell resistance, wire harness resistance, connector contact resistance, and protection semiconductor on-state resistance ($R_{\text{bus}} = R_{\text{cell}} + R_{\text{harness}} + R_{\text{switch}}$). When multi-axis motor inverters demand transient current surges ΔI , the resulting ohmic drop $\Delta V = \Delta I R_{\text{bus}}$ depresses rail voltage; if the bus sags below the undervoltage lockout (UVLO) threshold, supervisor circuits assert hardware reset, blacking out onboard compute during peak motion.

¹³ | **Dynamic Braking Chopper Circuit:** A dynamic braking chopper consists of a dedicated power switch (IGBT or power MOSFET) and a low-inductance braking resistor placed across the DC bus. When motor deceleration pumps regenerative kinetic energy back into the drive link, driving bus voltage above a comparator threshold $V_{\text{threshold}}$, the chopper switches into conduction to divert excess current through the resistor, dissipating kinetic energy as heat. This clamps bus voltage below the dielectric breakdown rating of the inverter switching silicon and DC-DC power stages.

🔍 Systems Perspective 4.3: Power bus transient decoupling

Power rails are shared dynamical systems; simultaneous multi-axis torque steps will collapse bus voltage across source impedance (R_{bus}) and trip brownout resets unless clamped by coordinated current slew-rate limiters.

When bus voltage crosses an undervoltage threshold under peak acceleration, the electrical fault cascades into a mechanical failure through hardware supervisor resets. A supply rail dropping below the undervoltage lockout threshold causes voltage supervisors to pull the hardware reset line on microcontrollers and motor gate drivers. Microcontroller clocks halt immediately, volatile register configurations vanish, and digital communication buses drop into high-impedance states. However, an acceleration command issued by the policy just before the reset remains physically active in the hardware state until the gate driver stage collapses. If the gate driver enters a high-impedance unpowered state, phase current ceases and the motor produces zero electromagnetic torque. The physical body, carrying velocity and momentum from the preceding command, coasts ballistically under gravity and inertia, colliding with workpieces or joint hardstops. If the driver instead defaults to shorting low-side MOSFETs for dynamic braking, back-electromotive force ($e_{bemf} = K_e \omega$) creates an immediate, violent counter-torque that jerks the mechanics at the joint limits.

Power removal is an unmanaged state transition, not an automatic return to safety. Engineers frequently assume that dropping bus power or tripping an undervoltage circuit brings a machine to a safe halt. In physical reality, cutting electrical power leaves an actuator in one of three unpowered states, namely free coasting, phase-short dynamic braking, or spring-engaged mechanical braking. Each state enforces a distinct mechanical behavior. An electromechanical friction brake requires 30 ms to 80 ms for its mechanical spring to overcome magnetic coil decay and engage its friction pads, during which gravity accelerates an overhead arm downward. Phase-short dynamic braking acts like viscous fluid damping, providing resistance proportional to velocity but generating zero holding torque at zero speed. Free coasting simply allows gravity to accelerate unconstrained joints without any resistance. The trajectory a machine follows after an electrical cutoff must be measured under full mechanical payload, not deduced from circuit diagrams. The choice of which safe fallback state the machine should enter belongs to system safety architecture in chapter 24, but the physical dynamics of an unpowered joint are unalterable mechanical constraints of the body.

For the physical AI systems engineer, power integrity is a foundational architectural constraint bridging software planning and electrical physics. Rather than treating power rails as infinite voltage sources, the engineer must enforce coordinated power budgets and current slew-rate limits across compute accelerators and motor drives. The deterministic real-time safety layer must arbitrate simultaneous multi-axis acceleration commands to prevent compound dI/dt transients, while dynamic brake choppers protect inverter electronics from regenerative feedback during emergency stops. Crucially, because power loss leaves actuators in unpowered physical states governed by gravity and momentum, the engineer must empirically characterize every unpowered fallback trajectory under full mechanical payload, ensuring that electrical trips resolve into controlled, predictable deceleration rather than unguided kinetic collisions. Characterizing these dynamic electrical and mechanical failure boundaries cannot rely on nominal catalog specs or idealized CAD models; it demands disciplined bench measurement.

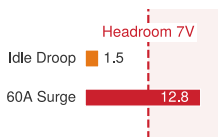
To quantify the coupled dynamics of rail collapse and regenerative energy feedback, consider an electrical power integrity budget for a mobile delivery robot (4.4).

📋 Example 4.4: DC bus voltage sag and regenerative overvoltage

A mobile delivery robot operates on a nominal 48.0 V battery bus powering both high-current motor inverters and an onboard perception computer.

1. Parameters:

- Vehicle mass and travel speed: $m = 80.0$ kg, $v_0 = 2.5$ m/s



Transient current surges burn through voltage headroom into unrecoverable brownout resets.

- DC bus characteristics: $V_{\text{bus, nom}} = 48.0 \text{ V}$, harness resistance $R_{\text{bus}} = 70 \text{ m}\Omega$, parasitic inductance $L_{\text{bus}} = 15 \mu\text{H}$, local inverter capacitance $C_{\text{bus}} = 2200 \mu\text{F}$
- Operating limits: Perception computer UVLO threshold $V_{\text{uvlo}} = 41.0 \text{ V}$, inverter MOSFET breakdown $V_{\text{breakdown}} = 60.0 \text{ V}$

2. Transient droop:

- As an illustrative concurrent load, four hub motors add 56.0 A while the perception computer adds 4.0 A at the same bus interface. Their aggregate step is $\Delta I = 60.0 \text{ A}$ over $\Delta t = 0.5 \text{ ms}$ ($dI/dt = 1.2 \times 10^5 \text{ A/s}$). The example assumes no separate compute-rail hold-up during this step.
- Transient voltage droop evaluates across resistive, inductive, and capacitive terms:

$$\Delta V_{\text{droop}} \approx \Delta I R_{\text{bus}} + L_{\text{bus}} \left(\frac{dI}{dt} \right) + \frac{\Delta I \Delta t}{2 C_{\text{bus}}}$$

$$\Delta V_{\text{droop}} = (60.0 \times 0.070) + (15 \times 10^{-6} \times 1.2 \times 10^5) + \frac{60.0 \times 0.5 \times 10^{-3}}{2 \times 2200 \times 10^{-6}} = 4.20 \text{ V} + 1.80 \text{ V} + 6.82 \text{ V} = 12.82 \text{ V}$$

- Minimum bus voltage:

$$V_{\text{bus, min}} = 48.0 \text{ V} - 12.82 \text{ V} = 35.18 \text{ V}$$

- The available headroom to the compute UVLO floor is $48.0 - 41.0 = 7.0 \text{ V}$; the modeled 12.82 V droop exceeds it by 5.82 V. Under the stated no-hold-up assumption, the perception computer trips its UVLO mid-acceleration. Reducing or buffering the concurrent motor and compute step must keep droop within that 7.0 V budget.

3. Regenerative surge:

- During an emergency stop, kinetic energy must be dissipated:

$$E_k = \frac{1}{2} m v_0^2 = \frac{1}{2} (80.0 \text{ kg}) (2.5 \text{ m/s})^2 = 250.0 \text{ J}$$

- If the battery BMS disconnects charge input and an illustrative 75 percent of kinetic energy returns to the bus ($E_{\text{regen}} = 187.5 \text{ J}$), an unprotected, lossless capacitor-only energy balance would end at:

$$V_{\text{final}} = \sqrt{V_{\text{bus, nom}}^2 + \frac{2E_{\text{regen}}}{C_{\text{bus}}}} = \sqrt{48.0^2 + \frac{2 \times 187.5 \text{ J}}{2200 \times 10^{-6} \text{ F}}} \approx 415.6 \text{ V}$$

- *Failure mechanism:* 415.6 V is a hypothetical no-protection endpoint, not a predicted spike or time trace. The bus would cross the 60.0 V switch rating far earlier; protection or component failure changes the actual trajectory.

4. Sizing brake chopper:

- Assume a triangular regenerative-power pulse over $\Delta t_{\text{stop}} = 0.3 \text{ s}$, rising from zero to a peak and returning to zero. Its area is $E_{\text{regen}} = 187.5 \text{ J}$, so a $V_{\text{clamp}} = 54.0 \text{ V}$ design faces the illustrative peak:

$$P_{\text{regen, peak}} \approx \frac{2E_{\text{regen}}}{\Delta t_{\text{stop}}} = \frac{2 \times 187.5 \text{ J}}{0.3 \text{ s}} = 1250.0 \text{ W}$$

- Maximum chopper resistance to divert peak power:

$$R_{\text{brake}} \leq \frac{V_{\text{clamp}}^2}{P_{\text{regen, peak}}} = \frac{54.0^2}{1250.0 \text{ W}} \approx 2.33 \Omega$$

- *Component screening:* A $2.2\ \Omega$ resistor with a nominal 1500 W pulse rating is a candidate. Verify its 187.5 J pulse-energy and temperature limits, chopper switch safe operating area, and clamp response for the actual braking profile before treating the absorption path as protective.

Architect takeaway: Mechanical motion and electrical power are bidirectionally coupled: acceleration requires capacitive hold-up to prevent brownouts, while deceleration requires active dynamic braking to prevent silicon overvoltage destruction.



Checkpoint 4.5: DC bus droop and regenerative surge

Before examining empirical measurement of physical limits, verify your understanding of power delivery and transient dynamics:

- ☐ **Multi-axis current transients:** Can you explain how simultaneous motor acceleration commands collapse bus voltage through wire resistance, parasitic inductance, and capacitor depletion?
- ☐ **Kinetic energy regeneration:** Can you describe why rapid deceleration pumps electrical energy back into the DC bus and why this energy destroys inverter silicon if the battery cannot absorb it?
- ☐ **Brake choppers and protection circuits:** Can you distinguish the operational roles of dynamic brake choppers, decoupling capacitors, and current slew-rate limiters in maintaining power bus integrity?

4.7 Measuring a Machine's Own Limits

Before a high-torque actuator is integrated into an autonomous robot chassis, it must undergo rigorous characterization on a dedicated dynamometer test stand (figure 4.8). In software design, parameters are treated as nominal constants from a configuration file. In physical systems engineering, **a measurement that cannot state its instrument, its percentile, and its sample count cannot support an argument later.**

When an engineer characterizes a machine's body, the measurement must begin from the specific decision it will govern in the nervous system or safety layer. That decision determines the physical quantity, the physical observation point, the required sensor bandwidth, the measurement resolution, and the reference clock. If the nervous system must clamp motor velocity to prevent a dynamic braking maneuver from overshooting an optical boundary, measuring steady-state motor current with an averaging bench meter yields no usable number. The measurement must instead capture the instantaneous peak phase current and deceleration trajectory at the exact point of electrical cutoff, using an isolated current probe sampled at 100 kHz and an optical encoder sampled synchronously on a common hardware timebase. As demonstrated on the NASA Armstrong electric propulsion dynamometer test bench (figure 4.8), selecting an instrument whose analog bandwidth falls below the electrical rise time attenuates the true transient peak, leaving software engineers to build safety margins on top of measurement artifacts rather than physical reality.

Every probe intrudes on the physical process it observes. In software, instrumentation often introduces context switches, cache eviction, bus contention, and core heating, altering the execution timing and scheduling jitter under test. In hardware, attaching a passive voltage probe adds capacitive loading that distorts high-frequency digital communication lines, inserting a current shunt in series with a motor drive drops the rail voltage enough to trigger an early brownout under load, and placing a thermocouple inside a motor housing alters heat dissipation paths. On a desktop development platform, motor driver current sensing uses precision low-side current sense shunts ($R_{\text{shunt}} = 10\ \text{m}\Omega$ to $50\ \text{m}\Omega$) routed to dedicated analog-to-digital converter (ADC) channels on the real-time microcontroller. Instrument uncertainty is therefore a compound budget rather than a single tolerance number from a calibration sheet. It combines sensor linearity, analog-to-



Figure 4.8: Electric motor dynamometer testbed: High-bandwidth actuator characterization stand at NASA Armstrong Flight Research Center. Direct empirical measurement of peak torque saturation, electrical time constants ($\tau_e = L/R$), continuous thermal rise ($\Delta T = P_{\text{loss}} \theta_{JA}$), and regenerative power feedback requires synchronizing multi-channel load cells, optical encoders, and isolated current probes on a shared hardware timebase rather than trusting manufacturer catalog ratings. (Source: NASA, Public Domain).

digital quantization error, frequency-dependent phase lag, probe loading effects, and clock drift between distributed acquisition channels. When an optical encoder with 0.05° quantization is sampled at 1000 Hz to compute joint velocity through discrete backward differences, numerical differentiation noise introduces velocity variance of several degrees per second that exists entirely in the measurement pipeline rather than in the mechanical joint.

A physical limit is meaningless without the complete environmental and computational context during testing. Every recorded limit must state the mechanical payload, ambient temperature, supply rail voltage, enclosure state, firmware version, and concurrent computational workload present during the run. An actuator thermal time constant measured on an open bench at 20°C with free convection differs by an order of magnitude from the same actuator operating inside a sealed composite fairing under direct sunlight at 45°C . If an electrical characterization omits whether vision-language model inference (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023) was running on the adjacent neural accelerator during motor stalls, the measured power rail stability reflects an idle board rather than the combined electrical draw of full compute and peak actuator torque. Documenting these conditions prevents engineers from treating a laboratory measurement taken under ideal cooling and fresh batteries as an absolute physical boundary of the machine in field deployment.

Systems engineering demands strict discipline in distinguishing measured physical limits from derived models, manufacturer ratings, and illustrative figures. Four distinct categories of numbers govern a machine, comprising direct measurements, derived values, datasheet ratings, and illustrative examples. Direct measurements are raw, calibrated time series with specified instruments, sample counts, and empirical percentiles such as P_{50} , P_{99} , and $P_{99.9}$. Derived quantities are computed from direct measurements through physical models, such as estimating rotor winding temperature from terminal voltage, phase current across current sense shunts, and an ambient thermistor trace, which requires propagating the uncertainties of all three underlying sensors through the thermal model. Datasheet ratings are nominal manufacturer claims reflecting idealized laboratory conditions or continuous thermal equilibrium, which rarely hold during transient impulse loads. Illustrative numbers are analytical examples used to demonstrate arithmetic or scaling principles

in design calculations, and they must be explicitly labeled as analytical examples so that no safety margin or architectural timeout relies on them as measured ground truth.

The absence of a failure during testing establishes a statistical bound on the failure rate, not an assertion of absolute safety.¹⁴ An empirical test of n independent trials yielding zero limit exceedances does not prove the system defect-free. Using standard statistical bounds (such as the Clopper-Pearson interval, frequently simplified in engineering as the rule of three), if an engineer runs $n = 3000$ consecutive pick-and-place trajectories on a robotic manipulator without a brownout or joint collision, this empirical data supports only the claim that the true per-cycle failure rate is bounded at roughly $3/n$, or one failure per thousand operations (with 95 percent confidence). (For the mathematical derivation of zero-failure statistical bounds, consult Appendix A). This holds only when trials are independent and drawn from a stationary distribution. If motor windings accumulate heat across successive trials, or if mechanical wear increases gear backlash over time, consecutive cycles are correlated and non-stationary. Testing a machine for 3000 cycles in a cool morning laboratory provides no guarantee about its failure rate in the afternoon, when thermal equilibrium increases winding resistance, lowers peak torque, and shifts the physical system into a brownout regime.

For the physical AI systems engineer, empirical characterization is not a one-time validation checkbox, but the creation of an immutable physical contract. Furthermore, this contract must account for **lifecycle degradation**: in deployed fleets, physical limits are non-stationary over operational time. Brake pad friction glazing and tire tread wear degrade the sustainable deceleration limit a_{brake} by 15 to 30 percent; chemical aging in lithium-ion battery packs increases internal DC bus impedance R_{bus} , exacerbating voltage droop (ΔV_{droop}) during dynamic maneuvers; and mechanical gear tooth wear widens backlash clearance δ_{lost} . Every parameter stored in a machine specification—whether a current ceiling, thermal time constant, or braking deceleration—must record its physical units, observation points, instrument bandwidth, sample count, tail percentiles (P_{99} and $P_{99.9}$), and provenance, with periodic recalibration across the hardware lifecycle. Without rigorous grounding, safety envelopes remain analytical fictions that dissolve under real-world dynamics. Yet even perfectly characterized limits do not remain isolated in operation; on an integrated machine, physical constraints couple together, creating compounding stress cascades.



War Story 4.1: ExoMars schiaparelli IMU saturation and altitude collapse (2016)

Context: On October 19, 2016, the European Space Agency's ExoMars Schiaparelli lander executed its atmospheric entry and descent toward the Martian surface, relying on an onboard Guidance, Navigation, and Control (GNC) computer to fuse Inertial Measurement Unit (IMU) gyroscopes with a Doppler radar altimeter (ESA Investigation Board 2017).

Mechanism: During parachute deployment at an altitude of approximately 4.0 km, violent atmospheric buffeting induced vehicle pitch and roll oscillations that saturated the lander's IMU angular rate sensors for approximately 1.0 s, clamping measured rotational velocity at the sensor's maximum hardware threshold ($\sim 165^\circ/\text{s}$). When the GNC software integrated this saturated rate without bounding sensor non-linearities, the estimated attitude angle became corrupted by roughly 165° , projecting the vehicle orientation completely upside down. When the radar altimeter subsequently delivered valid ground distance measurements, the navigation filter projected them onto the inverted attitude vector, calculating an imaginary negative altitude of -548 m.

Impact: The flight software interpreted the negative altitude as evidence that the lander had already touched down below ground level. The guidance logic prematurely jettisoned the parachute and backshell at 3.7 km altitude, fired the hydrazine braking thrusters for only 3 seconds instead of the planned 30 seconds, and activated ground telemetry systems. Schiaparelli entered unarrested free fall, impacting the Martian crust at 150 m/s (540 km/h) and creating a 50-meter impact crater.

Response: The ESA inquiry board mandated fault-tolerant sensor validation architectures that reject unphysical state transitions (such as instantaneous subterranean positions), assert

¹⁴ | **Zero-Failure Rule of Three:** Under independent, identically distributed Bernoulli trials with zero observed failures ($k = 0$ out of n samples), the upper end of a one-sided 95% Clopper-Pearson confidence interval is approximated by $-\ln(0.05)/n \approx 2.996/n \approx 3/n$. For $n = 3000$, this supports an upper confidence limit near 10^{-3} under the stated sampling assumptions; it does not guarantee the true rate. Progressive heating or mechanical wear violates stationarity and undermines that inference.

dynamic consistency across independent sensor channels, and enforce hard altitude-time floor bounds before allowing critical mechanical actuators to disengage.

Systems lesson: A physical body cannot transcend the physical saturation envelopes of its sensory transducers. When sensor physical limits (such as gyroscope rate ceilings or accelerometer dynamic ranges) are exceeded, state estimators that naively integrate saturated measurements inject unbounded mathematical drift into physical coordinates. In Physical AI systems, software layers must treat sensor readings not as pure mathematical variables, but as physical measurements bounded by hardware saturation envelopes, asserting state plausibility before commanding irreversible actuator actions.



Checkpoint 4.6: Benchmarking and empirical physical limits

Before examining the cross-archetype matrix, verify your understanding of physical metrology:

- ☐ **Probe loading and observer intrusion:** Can you describe how adding a current shunt or passive voltage probe can inadvertently alter the electrical behavior of the system under test?
- ☐ **Zero-failure statistical confidence:** Can you explain why 3,000 test cycles without a brownout support only an upper 95 percent confidence limit near 1 in 1,000, under independent and stationary trials?
- ☐ **Lifecycle parameter drift:** Can you identify three physical mechanisms that cause a machine's stopping distance and thermal margins to degrade between initial factory commissioning and fleet deployment?
- ☐ **Confidence bound calculation:** Given $n = 3,000$ consecutive fault-free cycles, can you use the rule of three ($p \leq -\ln(0.05)/n \approx 3/n$) to bound the true failure probability, and explain why mechanical wear or thermal drift invalidates the i.i.d. exchangeability assumption?

If any check reveals uncertainty, review empirical qualification in this section before studying the cross-archetype matrix.

4.8 The Cross-Archetype Matrix

The five physical constraints couple into a unified system constraint matrix (table 4.3). Building on the causal boundary established in section 2.2, the physical body of an embodied machine provides zero elasticity—the real world does not buffer mistakes. When an unconstrained neural policy issues an action command, that command draws immediate electrical current (stressing the power bus), generates non-negotiable kinetic momentum (which must eventually be braked), accelerates reflected rotor inertia (fighting the gearbox), and dissipates non-reversible Joule heat into motor stators (risking thermal damage).

How these physical constraints bind depends directly on the machine's physical archetype:

- **Class 1 (Mass-Dominated Mobility):** For autonomous mobile robots, automated guided vehicles, and legged platforms, **momentum and stopping distance** is the primary binding limit. Kinetic energy grows quadratically with travel speed ($E_{\text{kin}} = \frac{1}{2}mv^2$), making velocity the master multiplier of spatial risk. Sensor age directly expands unguided reaction displacement ($v_0\Delta t_{\text{delay}}$), while simultaneous multi-axis drive acceleration threatens battery bus collapse.
- **Class 2 (Contact-Dominated Manipulation):** For articulated robotic arms, dexterous hands, and automated assembly fixtures, **actuator bandwidth, reflected rotor inertia ($N^2 J_{\text{rotor}}$), and contact stiffness** form the primary binding envelope. Discontinuous policy setpoints impact rigid fixtures with steep force rise rates ($F = k\Delta x$), risking gear tooth fatigue fracture. Holding heavy payloads against gravity draws continuous static stall current, rapidly accumulating thermal stress.

Downstream: Physical body degradation envelopes bound the supervisor intervention thresholds derived in section 28.3.

- **Class 3 (Flow-Dominated Process & Thermal Plants):** For battery energy storage systems, server cooling distribution units, and power electronics converters, **thermal dissipation and electrical power integrity** govern survival. Long thermal time constants ($\tau = \theta_{JA} C_{th}$) hide continuous heat accumulation, where a small continuous duty-cycle overload slowly drives winding insulation or battery cells into thermal runaway.

Table 4.3: Cross-archetype physical constraint matrix: Systematic mapping of the five non-negotiable physical constraints to their governing laws, software illusions, archetype failure modes, and deterministic systems engineering takeaways.

Physical Budget / Factor	Governing First Principle & Scaling Law	Software Illusion vs. Hardware Ground Truth	Primary Binding Archetype	Systems Architect Takeaway & Runtime Invariant
1. Measurement Freshness & Age (§2.2)	$\Delta t_{fresh} \leq \frac{e_{max}}{v_{max}};$ $\Delta t_{e2e} = \sum_{k=1}^S \Delta t_k$	<i>Illusion:</i> High inference rate (50 Hz) guarantees real-time response. <i>Truth:</i> Age begins at exposure; pipeline latency injects phase lag ($-\omega\tau$).	Class 1 & 2: Erodes clearance margins and destabilizes high-gain contact loops.	Tag frames with hardware timestamps; enforce $t_{now} - t_{transduction} \leq \Delta t_{fresh}$ at motor registers; drop stale commands.
2. Momentum & Stopping Distance (§2.3)	$d_{stop} = v_0 \Delta t_{delay} + \frac{v_0^2}{2a_{brake}} \leq D_{clear}$	<i>Illusion:</i> Deceleration can be commanded instantaneously. <i>Truth:</i> Braking distance scales quadratically (v_0^2); kinetic energy requires friction work ($W = Fd$).	Class 1 (Mobility): Primary constraint. Doubling speed quadruples braking distance (267% total increase).	Solve clearance quadratic $v_{max}(D_{clear}, \Delta t_{delay})$ at 1 kHz in deterministic MCU loop; clamp policy velocity.
3. Actuator & Reflected Inertia (§2.4)	$J_{ref} = N^2 J_{rotor};$ $\tau_e = \frac{L}{R}; I(t) \leq I_{max}$	<i>Illusion:</i> Motor torque tracks policy setpoints without mechanical resistance. <i>Truth:</i> Gearbox squares rotor inertia (N^2); inductance delays current rise; shock steps shear teeth.	Class 2 (Manipulation): Primary constraint. Rigid contacts create extreme dF/dt force spikes and gear fatigue.	Enforce C^2 spline (S-curve) trajectory rate limits; sample load-side optical encoders; deploy quasi-direct drive (QDD) actuators.
4. Thermal Limits & Duty Cycle (§2.5)	$C_{th} \frac{dT}{dt} = I^2 R(T) - \frac{\Delta T}{\theta_{JA}};$ $D \leq \frac{T_{max} - T_{amb}}{P_{peak} \theta_{JA}}$	<i>Illusion:</i> Motors deliver peak catalog torque continuously. <i>Truth:</i> Joule heat ($I^2 R$) has positive thermal feedback (α_{Cu}); heat accumulates in sealed housings.	Class 3 (Thermal) & Class 2: Static holding draws stall current, overheating coils in seconds.	Run discrete $I^2 t$ thermal observers at 1 kHz on MCU; dynamically derate speed and torque when hotspot models approach limits.
5. Electrical Power Integrity (§2.6)	$\Delta V_{droop} \approx \Delta I R_{bus} + \frac{\Delta I \Delta t}{2C_{bus}};$ $V_{final} = \sqrt{\frac{2(E_C + E_{regen})}{C_{bus}}}$	<i>Illusion:</i> Power rails act as infinite, ideal voltage sources. <i>Truth:</i> Bus impedance collapses rail voltage under combined compute+motor bursts; braking surges overvoltage.	Class 1 & 3: Peak multi-axis acceleration trips microcontroller BOD resets; regen surges destroy inverter FETs.	Enforce coordinated compute-actuation power budgets and dI/dt limits; install dynamic brake choppers; characterize unpowered fallbacks.

When integrating a machine, these five budgets do not fail in isolation. They form a coupled constraint hypersurface where exceeding a margin in one domain triggers an immediate failure cascade across adjacent domains.

🔍 Systems Perspective 4.4: Reflected inertia and thermal contention

On an articulated humanoid chassis with thirty to fifty active degrees of freedom, the physical constraints analyzed across this chapter collide into severe structural contention:

- **Reflected Inertia versus Foot-Strike Shocks:** Bipedal locomotion requires high gear reduction ratios ($N \geq 50 : 1$ using strain wave or planetary gearboxes) at the hips and knees to produce high joint torque ($\tau > 150 \text{ N} \cdot \text{m}$) within compact limb cross-sections. However, reflected inertia scales quadratically with gear ratio ($J_{ref} = N^2 J_{rotor}$). When

a swing foot impacts hard concrete at 1.5 m/s, the reflected rotor inertia prevents the joint from backdriving before the stator accelerates. The impact shock wave transmits as mechanical shear stress through gear tooth fillets and load cells, risking fatigue fracture unless mitigated by series-elastic compliance, QDD actuators, or high-bandwidth active impedance loops.

- **Thermal Trapping in Sealed Torso and Limb Fairings:** In an industrial robot arm, heavy metal castings act as large heatsinks exposed to open ambient air. In a humanoid, dense clusters of actuators (such as four to six actuators packed into a single hip or pelvis assembly) operate inside sealed, unventilated carbon-fiber fairings to protect internal wiring and maintain human-safe surfaces. Continuous walking and posture stabilization draw ongoing phase currents ($I_{\text{rms}} \geq 15$ A), dissipating hundreds of watts of Joule heat ($I^2 R$) into restricted thermal capacitances (C_{th}). Without active liquid cooling loops, stator windings rapidly approach insulation ceilings (125°C to 155°C), forcing thermal observers to derate peak motor torque by 30 to 50 percent precisely when maximum authority is needed to prevent an unexpected stumble from becoming an unrecoverable fall.
- **Transient Voltage Sag during Dynamic Balancing:** When an external perturbation pushes a standing humanoid backward, balance recovery requires instantaneous, simultaneous torque pulses across both ankles, knees, and hips. If all twelve lower-body inverters command peak current ($I_{\text{peak}} > 40$ A per joint) simultaneously, the combined dI/dt transient across internal battery bus source impedance and harness inductance causes an immediate DC voltage sag ($\Delta V_{\text{droop}} > 8$ V). Unless compute motherboards are electrically isolated or buffered by supercapacitors, this bus sag triggers undervoltage lockouts (UVLO) on onboard processors, instantly dropping the robot mid-stride.

4.9 Fallacies and Pitfalls

Physical limits interact across subsystem boundaries. A design that meets each component budget can still fail when heat, timing, power, and motion change together.

Fallacy: *Model inference time defines the measurement freshness latency.*

A team quantizes its vision backbone and reduces inference time from 30 ms to 10 ms. The improvement shortens one stage, but the controller still acts on measurements that pass through exposure, readout, transfer, and operating-system queues. Commands then travel over the network fieldbus before the motor coil current actually rises to apply force. A 45 ms transport spike elsewhere in this chain can consume the clearance margin even when inference meets its benchmark. Building on the causal boundary established in section 2.2, the engineering budget must measure the true “photon-to-torque” elapsed time from sensing to physical response and bound its tail, rather than declaring the loop real-time from model execution time alone.

Fallacy: *Software step commands deliver instantaneous physical torque.*

A simulated policy issues piecewise-constant setpoints at 50 Hz and assumes the actuator follows each change immediately. The physical joint, however, responds through stator inductance (L/R) and reflected rotor inertia ($N^2 J_{\text{rotor}}$). Physically, this means that electromagnetic current takes time to build in the motor coils, and the mechanical mass of the gear train and links takes time to spin up. An abrupt command can exceed the actuator authority budget (section 4.1), saturating the inverter’s current limit and violently loading the transmission before the joint achieves the requested motion. This physical lag alters the feedback behavior learned in simulation. Setpoint shaping and the actuator model must account for these physical time constants; a software update rate alone does not establish the joint’s achievable response.

Pitfall: *Treating isolated component datasheets as operational system guarantees.*

An engineer uses a motor’s continuous torque rating to set the planner’s limits. The vendor measured that rating on an open dynamometer at 25°C with forced air cooling, while the installed motor sits inside a sealed, unventilated joint next to a hot 60 W compute unit. The tight enclosure traps the

winding heat ($I^2 R$), drastically changing how thermal energy escapes, so the open-air datasheet conditions no longer describe the deployed assembly. Sustained operation at the quoted rating can rapidly exceed the installed thermal budget and damage the motor. Planning limits must derive from measurements on the fully assembled machine under expected environmental and cooling conditions.

Pitfall: *Treating dynamic voltage and frequency scaling (DVFS) as a transparent software optimization.*

A delivery robot's motors heat the shared chassis until its compute accelerator triggers dynamic voltage and frequency scaling (DVFS) to throttle clock frequencies and avoid junction overtemperature. This thermal intervention increases inference latency from 20 ms to 31 ms. At $v_0 = 1.5$ m/s, the added compute delay physically consumes an extra $\Delta d_{\text{react}} = 1.65$ cm of forward travel before the robot can even command a brake response. The power manager has protected the silicon, but it has silently changed the control timing assumed by the motion controller. A previously adequate stopping margin may no longer be adequate. Thermal telemetry must therefore inform the motion budget, with travel speed reduced when processor throttling increases the time needed to perceive and respond.

Pitfall: *Scaling operational velocity without recalculating quadratic braking distances and thermal dissipation.*

A logistics cart doubles its speed from $v_0 = 1.0$ m/s to $v_0 = 2.0$ m/s while keeping the same obstacle clearance and braking limit. Reaction travel during compute delay doubles, but physical braking travel quadruples because kinetic energy scales with the square of velocity ($d_{\text{brake}} \propto v^2$). The throughput change therefore consumes clearance far faster than a linear estimate suggests. Repeated high-current braking also rapidly heats the motor windings, meaning a maneuver that the motor can easily perform once may exceed its allowable thermal duty cycle (governed by the actuator authority budget in section 4.1) when repeated. A higher operating speed requires a new stopping-distance calculation and a thermal assessment of repeated maneuvers before the planner can safely treat that speed as available.

Fallacy: *Compute accelerators and motor inverters operate on independent, ideal power rails.*

A mobile robot's heavy inference burst coincides with its motors accelerating to climb over a doorway threshold. The combined 70.0 A current transient across the shared wiring harness resistance ($R_{\text{bus}} = 60.0$ m Ω) produces a 4.2 V voltage drop. With the battery already depleted to 21.5 V, the bus sags to 17.3 V, which falls below the onboard point-of-load regulators' 18.0 V dropout threshold. This electrical brownout abruptly asserts hardware reset on the real-time controller while the machine carries active momentum. Separate software budgets for compute and actuation ignore their physical reliance on a shared electrical supply. Power qualification must account for coincident peak loads and verify that control electronics remain powered through demanding maneuvers.

Fallacy: *Passing isolated single-variable bench tests guarantees physical survival under compound operational stress.*

A robot arm passes a maximum-payload trial in a 20°C air-conditioned laboratory and an empty-gripper trial in a 45°C thermal chamber. Neither test establishes its ability to actually carry that maximum payload in the hotter environment while running on a depleted battery. Under these combined conditions, the heavy physical load raises resistive winding losses, the hotter surroundings reduce the system's ability to reject that heat, and the lower supply voltage forces the motor to draw more current to achieve the same mechanical power. Consequently, each subsystem can approach its failure limit much sooner than the separate, isolated trials suggest. Qualification must therefore exercise realistic combinations of payload, temperature, and battery condition on the assembled machine, rather than treating individual test passes as a guaranteed operating envelope.

4.10 Summary

Physical metrology establishes the non-negotiable numerical resource floors of embodied AI by grounding computational abstractions in the unyielding laws of mechanics, electromagnetism, and thermodynamics. Instrumenting observation freshness, kinodynamic stopping envelopes, actuator bandwidth, and Joule heat dissipation on hardware replaces empirical optimism with verifiable physical budgets that bound downstream deliberation and control.

As summarized in the Cross-Archetype Physical Constraint Matrix (table 4.3), an engineer cannot negotiate with momentum, dissipate heat faster than thermal conductance allows, or transport information across a wire before voltage propagates. The physical body presents five boundaries that define what the machine can execute without structural damage or state loss:

1. **Observation Age:** Sensor signals arrive with nonzero age (t_{age}), bounded by the physical freshness deadline $\Delta t_{\text{fresh}} \leq e_{\text{max}}/v_{\text{max}}$.
2. **Stopping Envelope:** Moving masses sweep through space along an unavoidable stopping envelope $d_{\text{stop}} = v_0 \Delta t_{\text{delay}} + \frac{v_0^2}{2a_{\text{brake}}} \leq D_{\text{clear}}$.
3. **Delivered Actuation:** Actuators produce bounded delivered torque governed by reflected rotor inertia ($J_{\text{ref}} = N^2 J_{\text{rotor}}$), electrical stator time constants ($\tau_e = L/R$), and transmission mechanical shear limits.
4. **Energy & Thermal State:** Joule heat accumulation $P_{\text{loss}} = I^2 R(T)$ under positive thermal feedback (α_{Cu}) restricts allowable cyclic duty cycles (D).
5. **Electrical Power Integrity:** Shared DC bus impedance droop (ΔV_{droop}) and regenerative overvoltage surges (V_{final}) dictate power delivery stability.

Designing a system around these constraints requires replacing qualitative notions of hardware capacity with budgetable quantities. For a sense-to-torque decision, observation age runs from physical capture to the actuator's response. The stopping envelope includes sensing delay, brake engagement, and credible deceleration. Delivered actuation is the usable force or torque at the load, bounded by current, voltage, and transmission behavior. Electrical and thermal states require separate voltage-sag and hotspot margins. An observed $P_{99.9}$ response time characterizes a tested workload; a hard deadline requires a separately justified upper bound spanning acquisition, transport, computation, and actuator response.

Every physical limit depends on a declared operating configuration and loses authority when it changes. A measured stopping distance or thermal time constant cannot be transferred from a component datasheet into a safety case without qualification. In the friction-limited, level-ground stopping model used here, halving the effective tire friction halves available deceleration and doubles the braking-distance term at the same speed; increasing payload mass alone does not change that idealized distance because mass cancels. A winding at 75.0°C in a 20.0°C laboratory has a 55 K rise. At 45.0°C ambient under the same loss and cooling assumptions, it would reach 100.0°C, leaving 55 K below the chapter's 155°C Class F limit. Actual payload, surface, cooling, voltage, and wear changes require remeasurement before reusing a timing, force, or energy budget.

Maintaining integrity across a physical system requires preserving a complete engineering record for every limit before integrating software components. That record specifies six mandatory elements for each boundary. It documents the numerical value or empirical distribution, such as a $P_{99.9}$ latency of 4.2 ms or a maximum deceleration torque of 120.0 N·m. It specifies the exact operating conditions under which that number was established, including payload, ambient temperature, supply voltage, and surface properties. It records the measurement uncertainty, identifying the error bounds of the bench instrumentation used to capture the data. It allocates a deliberate engineering margin that separates normal operating targets from absolute physical breakdown thresholds. It designates a hardware or low-level software detector, such as a phase-current sensor, optical interrupter, or supply-rail voltage monitor, responsible for catching boundary excursions in real time. It records provenance, distinguishing whether a bound is derived from analytical mechanics or measured directly on physical hardware under stress.



Key Takeaways: The physical body and invariant budgets

- **Freshness is Measured at Transduction:** Observation age begins at photon capture on sensor silicon, not at model inference. Carry hardware timestamps to the motor register boundary and reject stale commands ($t_{\text{now}} - t_{\text{transduction}} \leq \Delta t_{\text{fresh}}$).

- **Velocity is the Master Risk Multiplier:** Travel speed scales reaction travel linearly ($d_{\text{react}} \propto v$), braking distance quadratically ($d_{\text{brake}} \propto v^2$), and stator heat dissipation quadratically ($I^2 R$).
- **Gearboxes Reflect Rotor Inertia Quadratically:** Gear ratio N multiplies rotor inertia by N^2 , turning rapid motor acceleration into violent mechanical shock unless filtered with C^2 continuous splines.
- **Thermal Heat Accumulates Silently:** Joule dissipation ($I^2 R$) creates positive thermal feedback (α_{Cu}). Enforce cyclic duty cycles (D) and real-time $I^2 t$ thermal observers.
- **Electrical Power is a Shared Resource:** Simultaneous compute and motor acceleration transients collapse DC bus voltages, triggering brownout resets at the moment of peak kinematic momentum.
- **Tail Measurements Inform Physical Margins:** $P_{99.9}$ and zero-failure statistical bounds ($p_{\text{max}} \approx 3/n$) characterize tested operation; neither alone proves a hard response deadline.

When control software ignores the physical limits of the body, safety violations do not manifest as clean software exceptions; they physically manifest as collapsed voltage rails (inducing immediate computing brownouts), stripped gear teeth (fracturing mechanical linkages), and runaway thermal destruction (causing winding dielectric breakdown). Validating an embodied machine requires accepting that no neural policy, regardless of parameter count or training compute, can outrun momentum or negotiate with thermodynamics.



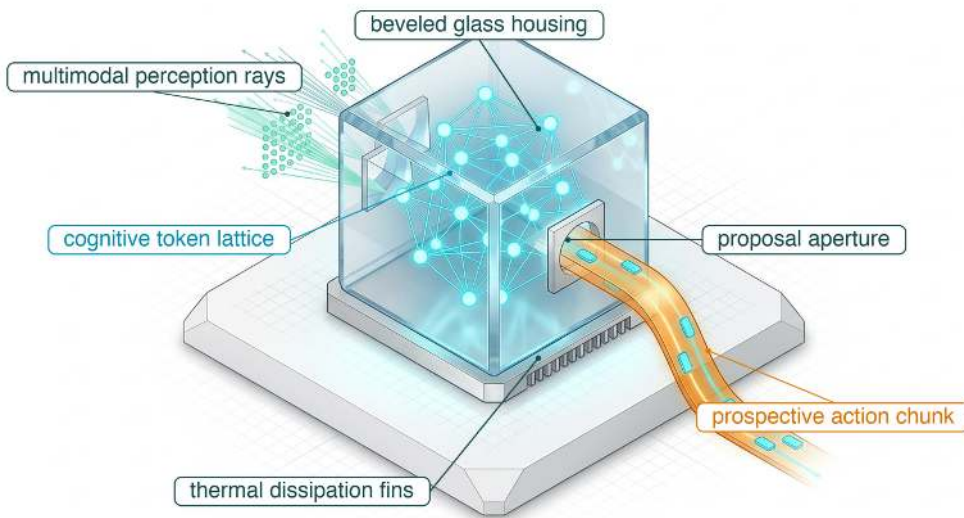
What's Next: From physical mechanics to cognitive deliberation

With the physical body's numerical envelopes established, chapter 6 takes custody of the machine's task purpose by constructing the learned cognitive deliberative engine. We turn from the deterministic, continuous physics of motors, reflected inertia ($N^2 J$), and stator thermal dissipation ($I^2 R$) to the high-capacity, stochastic world of neural deliberation: examining how vision-language-action (VLA) foundation models, multimodal encoders, and diffusion policies synthesize action proposals across open environments—before chapter 8 arbitrates the severe tension between them across the causal boundary.

The Cognitive Brain

DRAFT

The Cognitive Brain

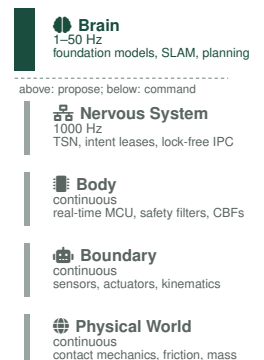


- 6.1 The Embodied Brain
- 6.2 The Classical Wall
- 6.3 Learned Representations
- 6.4 The Cognitive Pipeline
- 6.5 Cognitive Vulnerabilities
- 6.6 Fallacies and Pitfalls
- 6.7 Summary

Purpose

How does an autonomous agent deliberate over high-dimensional sensory tokens without violating real-time physical deadlines?

A robot among people must interpret changing surroundings and propose actions beyond fixed programmed movements. Learned models extend that capability by connecting sensory observations with patterns acquired during training. However, these models guarantee neither physically feasible proposals nor inference completion before the next actuation deadline. Large models consume immense memory bandwidth and compute; weight-transfer delays yield proposals based on outdated worldviews. An incorrect proposal creates a problem even if it arrives on time. The architecture must therefore separate action proposal from execution permission. The learned brain supplies candidate behaviors while the nervous system retains timing control and actuator authority.



Prerequisite: Cognitive policy proposals remain unprivileged until verified by the safety enforcer derived in section 2.9.

Learning Objectives

- Compare classical and learned approaches for perception, generalization, and dependable physical execution
- Trace sensory evidence through the cognitive stages that produce a physical action proposal
- Evaluate trade-offs between reasoning capability, sensing robustness, spatial grounding, and update rate
- Diagnose cognitive failures caused by ambiguous observations, overconfidence, changing scenes, or expired information
- Apply proposal–permission separation to keep cognitive uncertainty from directly controlling physical actuation

6.1 The Embodied Brain

A robotic manipulator can meet all electrical and thermal budgets yet still fail catastrophically when grasping an unfamiliar mug. The motor drives remain well within their continuous torque envelopes, the voltage distribution rails show zero droop, and the joint velocities remain below physical limits. Yet the machine drops the object or crushes the ceramic rim because low-level motor physics cannot determine what the object is, where its structural affordances lie, or how to reach around clutter. A model output in a purely advisory display can be reviewed before it affects the world. Here, a cognitive proposal can automatically influence moving mass. A mistaken proposal can lead to unsafe kinetic energy if the execution path accepts it, so permission must be checked independently. The central dilemma of embodied cognition (section 2.2) is that high-capacity neural models are inherently stochastic, computationally expensive, and prone to semantic hallucinations—yet they must orchestrate physical bodies whose collisions are irreversible.



Definition 6.1: The proposal–permission decoupling principle

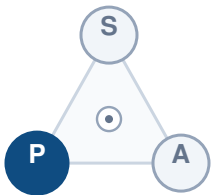
Proposal–permission decoupling principle is the fundamental architectural separation in physical AI systems wherein high-capacity, stochastic neural deliberation models operate strictly as unprivileged motion proposal engines at deliberative cadences (1–5 Hz), while dedicated, deterministic reflex monitors hold exclusive real-time execution authority (1000 Hz) to grant or deny physical actuation permission.

1. **Significance:** Keeps model outputs from directly commanding actuators; protection depends on independent sensing, valid physical limits, and a feasible fallback when a proposal is rejected.
2. **Distinction:** In this architecture, even a policy that outputs torque values has no actuator-write authority; an independent permission path checks proposals before execution.
3. **Common pitfall:** Treating the deterministic safety governor as a mere post-processing clamp on neural actions, rather than an independent, hard real-time monitor capable of preemptively revoking execution authority.

Within the four-tier systems hierarchy shown in figure 6.1, this chapter centers on Layer 3: the Cognitive Brain. Positioned above the deterministic Real-Time Nervous System (Layer 2) and the Dynamical Physical Body (Layer 1), the brain functions under the Proposal–Permission Decoupling Principle as an unprivileged deliberation engine. It processes multimodal sensory fluxes to produce candidate action trajectories at deliberative cadences (1–50 Hz).¹ This boundary lets the execution path reject proposals against independently checked limits; its protection depends on the limits, state estimate, and fallback remaining valid.

To guide an autonomous agent in unstructured environments, an embodied cognitive architecture must satisfy five primary properties:

¹ | **Downstream Reflex Monitors:** Operating at Layer 2, these deterministic monitors evaluate control barrier functions and kinematic bounds, clamping or rejecting candidate action chunks that violate physical safety invariants before setpoints reach actuator hardware registers.



The Plan axis: high-capacity neural deliberation emitting unprivileged action chunks.

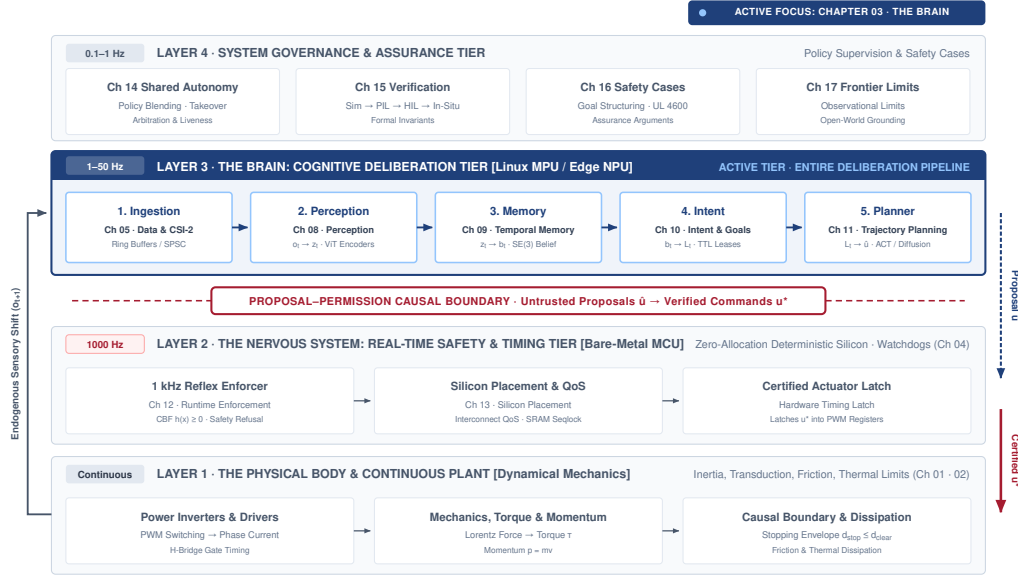


Figure 6.1: The brain in the Physical AI stack: The four-tier physical AI systems architecture highlights the cognitive deliberative layer. Operating above the real-time nervous system and physical body, the learned brain processes high-dimensional multimodal streams to generate candidate proposals, with execution authority reserved for downstream deterministic safety governors.

1. **Open-world semantic generalization:** Interpreting open-vocabulary natural language instructions and identifying novel, un-modeled objects without requiring rigid pre-programmed CAD templates.
2. **Sensory robustness under tested conditions:** Evaluating how raw perceptual streams respond to glare, dust, motion blur, and illumination shifts.
3. **Metric spatial-temporal grounding:** Anchoring abstract semantic concepts to metric physical coordinates in three-dimensional space, maintaining continuous spatial permanence as objects become occluded.²
4. **Multi-rate cadence decoupling:** Separating the scenario-dependent cadence of full-model inference from waypoint execution and high-rate feedback control, as illustrated by unlike workload examples in figure 6.2.
5. **Bounded execution authority:** Enforcing that stochastic statistical models remain unprivileged proposal engines, holding zero direct write access to motor inverter power stages.

Across our three physical reference archetypes (section 2.3), these cognitive affordances map to distinct operational responsibilities:

- **Mobile Systems (AMR / AGV):** The brain constructs topological-metric semantic maps, resolving spatial navigation goals (“deliver parcel to dock 4B”) while monitoring dynamic obstacles across 1–5 Hz global path replanning cadences.
- **Manipulators (Articulated Arms):** The brain predicts 6-DoF end-effector grasp affordances and continuous action chunk trajectories ($\mathbf{A}_{t:t+H-1}$), resolving target object poses under camera occlusion and hand-eye parallax.
- **Process and Energy Systems:** The brain forecasts multi-step thermal and chemical state trajectories, proposing setpoint adjustments for mass flow and temperature to optimize efficiency while honoring hard physical envelopes.

The plotted points are illustrative workload regimes, not measurements on a common device or input. Parameter count alone cannot determine update rate. Reconciling open-world reasoning with deterministic execution requires a systematic progression through the cognitive hierarchy:

² | **Metric Spatial Grounding:** While disembodied AI operates over discrete semantic tokens, physical AI must ground those semantics in the Special Euclidean group $SE(3)$, mapping entities to metric distances (meters) and spatial orientations (quaternions) relative to the robot’s base coordinate frame. Failing to maintain this metric grounding causes neural policies to emit kinematically impossible Cartesian paths that breach actuator workspace boundaries.

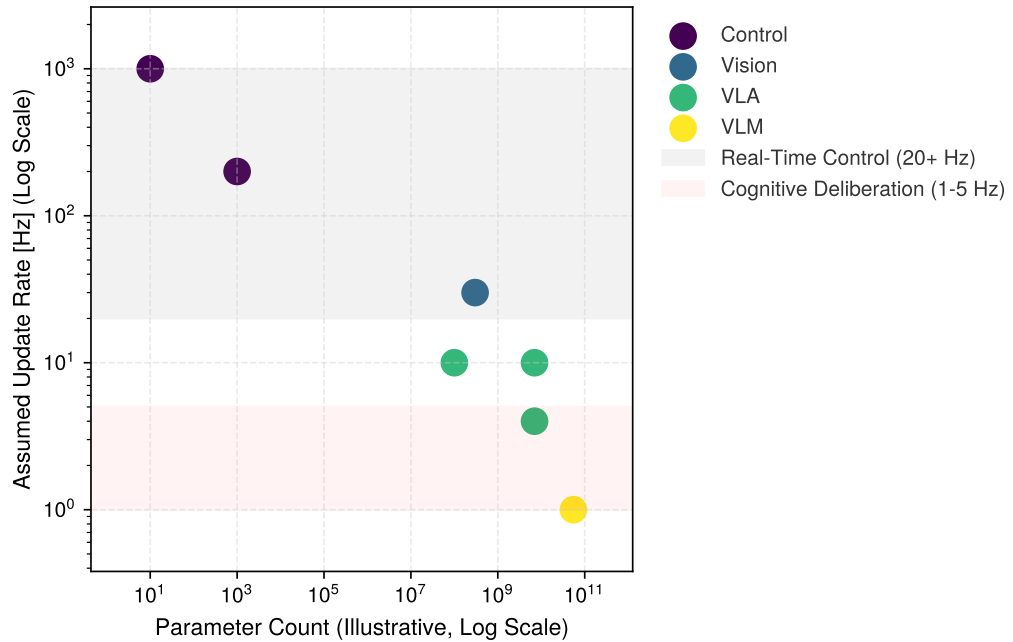


Figure 6.2: Illustrative cadence regimes: Parameter counts and example update rates for unlike control, vision, and embodied workloads. Rates depend on hardware, input size, implementation, and measurement conditions; these scenario points do not establish a size-to-cadence law.

Section 6.2 analyzes why traditional robotics pipelines—constructed on hand-authored CAD models, explicit inverse kinematics, and rigid finite-state machines—fail in unconstrained, deformable environments. Section 6.3 explores how deep multimodal encoders and foundation models learn spatial affordances and generalizable sensorimotor representations from demonstrations. In section 6.4, we trace the complete five-stage cognitive pipeline (Ingestion → Perception → Memory → Intent → Planning), showing how multi-step action chunking amortizes inference latency to emit smooth, continuous trajectory proposals. Finally, in section 6.5, we diagnose critical cognitive vulnerabilities—including semantic hallucinations, out-of-distribution overconfidence, perceptual chattering, and stale intent—and establish their architectural resolution through the two-speed brain.

6.2 The Classical Wall

For nearly a century, automated control and robotics approached cognition as an explicit geometric and analytical physics problem (Wiener 1948; Kalman 1960; Craig 2005). Under this classical paradigm, engineers constructed mathematical representations from first principles.

Engineers used analytical kinematics and differential dynamics to map desired end-effector motions into actuator torques using closed-form equations.³ Perception relied on matching hand-crafted 3D Computer-Aided Design (CAD) models against sensor point clouds using geometric alignment algorithms, such as iterative closest point (ICP) or random sample consensus (RANSAC). High-level task logic was authored as explicit Boolean state machines that dictated deterministic transitions between operational phases, such as approaching, aligning, grasping, and retracting.

Where the operational environment is strictly structured and bounded (figure 6.3), classical methods can provide dependable behavior within their modeled conditions. On an automated automotive assembly line, an industrial robot welds stamped steel panels where every workpiece arrives at an exact coordinate within sub-millimeter tolerances. Ambient lighting is tightly regulated by industrial fixtures, and human workers are excluded by physical interlock safety fences. Within the causal boundary defined in section 2.2, this controlled regime permits explicit models, bounded servo timing, and stability analysis under stated assumptions. Engineers can verify selected trajectory

³ | **Rigid-Body Dynamics Formulation:** Classical control computes generalized torques via the standard manipulator equation $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}$, where $\mathbf{M}(\mathbf{q})$ represents the positive-definite inertia matrix, $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ captures Coriolis and centripetal terms, and $\mathbf{g}(\mathbf{q})$ accounts for gravitational loading. This analytical formulation becomes computationally intractable and physically inaccurate when machines interact with unmodeled frictional contacts, fluid drag, or deformable objects. Derivations are provided in Appendix B.



Figure 6.3: Industrial spot-welding workcell: Multiple six-degree-of-freedom articulated robots execute preprogrammed spot welding paths on stamped automotive chassis at the BMW Leipzig plant. Operating in a calibrated factory environment with submillimeter repeatable fixtures and safety interlock cages, classical controllers provide deterministic cycle timing and formal stability proofs. (Credit: BMW Group / Wikimedia Commons, CC BY-SA 2.0 de).



Figure 6.4: Unstructured search-and-rescue robot: A tracked mobile robot navigates an obstacle course with irregular inclines, loose debris, and visual occlusions. In such unmapped environments, preauthored geometric models and rigid state machines can lose coverage; learned policies may help infer spatial affordances and traversability from sensory streams. (Credit: TU Darmstadt Rescue Robotics / Wikimedia Commons, CC BY-SA 3.0).

and torque constraints and assess controller software coverage against applicable standards without collecting task demonstrations.

Yet when an embodied machine leaves the structured factory cage to operate in unstructured environments (figure 6.4), this paradigm hits the Classical Wall (Brooks 1991; Levine et al. 2018). The physical world is not a sterile CAD drawing; it is continuous, deformable, and endlessly variable.

The first breakdown is the combinatorial explosion of geometry. In a commercial logistics warehouse handling hundreds of thousands of retail goods, authoring CAD models, mesh templates, and grasp points for every bottle, wrapped package, and flexible garment is impossible. Discretizing orientations, frictional properties, and contact permutations across overlapping, deformable items creates a combinatorial space exceeding billions of geometric configurations unmanageable by hand-crafted templates.

The second breakdown arises from sensory brittleness. A hand-crafted edge-and-segmentation pipeline may rely on filters, Harris corners, or surface-normal thresholds. Under changed sunlight, reflections, or dust, a required feature can disappear. In that pipeline, a missing edge can make planar segmentation bridge two objects and propose a grasp in empty space.

The third breakdown is the engineering bottleneck of hand-authored logic. The machine's operational capability is bounded by the engineer's ability to anticipate every physical edge case. When an unexpected variation occurs—such as a bent bracket, a scuffed label, or an unfamiliar obstacle—the hand-authored rulebook contains no matching branch, causing the machine to halt, drop its payload, or fail catastrophically.

Where hand-authored geometry loses coverage, learned representations can offer empirical generalization from multimodal data; their behavior still needs testing under the intended variation.



Checkpoint 6.1: The classical wall and embodied constraints

Before examining how deep neural representations discover sensorimotor structure, verify your understanding of the physical and combinatorial boundaries that constrain classical robotics:

- ☐ **Combinatorial geometric explosion:** Can you explain why CAD mesh templates, surface normal matching, and analytical inverse kinematics fail when a manipulator interacts with unmodeled, deformable, or overlapping retail objects?
- ☐ **Sensory brittleness and Boolean fractures:** Can you describe how minor shifts in ambient lighting, specular reflections, or dust cause hand-crafted edge detectors and planar segmentation to bridge across disjoint physical bodies?
- ☐ **Engineering bottleneck of explicit state machines:** Can you contrast the maintenance scalability of hand-authored Boolean state machines against continuous learned representations when scaling to long-horizon, un-modeled tasks?
- ☐ **Industrial caging versus open-world stability:** Can you explain why stability analyses and timed controls established for a modeled factory cell cannot be directly transferred to unstructured search-and-rescue or domestic environments?

If any check reveals uncertainty, review the three breakdown modes in this section before examining how deep multimodal backbones learn continuous manifolds.

6.3 Learned Representations

Physical AI can extend classical pipelines with neural representations fitted to multimodal sensory data when hand-authored models lose coverage. In the embodied AI and robotics community, this transition was driven by four foundational pillars:

The first pillar is open-world semantic generalizability. Classical controllers are geometrically precise but semantically blind: an inverse kinematics solver can expertly drive an end-effector to coordinate (x, y, z) , yet it possesses zero understanding of whether that location contains a sturdy mug or a fragile glass, or how surrounding objects relate to human tasks. Modern vision-language-action foundation models project open-vocabulary natural language instructions and high-resolution camera streams into shared semantic embeddings (Driess et al. 2023; Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023; Kim et al. 2024). When commanded to “wipe the coffee spill from the counter and stow the mug in the drying rack,” a trained model may propose the mug handle and spill boundary as task-relevant regions without a new CAD model. Their accuracy on an unfamiliar object or surface must be checked in the operating environment.

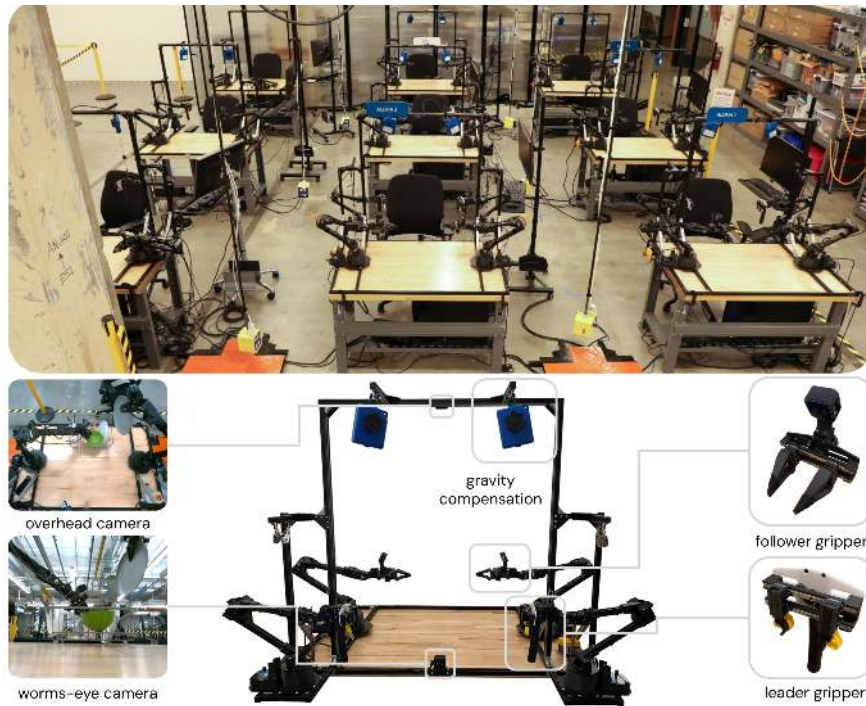


Figure 6.5: Bimanual teleoperation workcell: Dual-arm teleoperation setup featuring two six-degree-of-freedom follower arms, two leader arms for kinesthetic teleoperation, multi-view cameras (overhead and wrist-mounted), and parallel grippers mounted to an extruded aluminum frame. The physical AI policy ingests synchronized multi-camera streams to predict continuous end-effector trajectories, requiring deterministic safety governors to prevent self-collisions and motor over-torquing during closed-loop execution. (Credit: Stanford University & Google DeepMind, CC-BY / Apache 2.0).

The second pillar is empirical sensory robustness. Hand-authored visual thresholds can flip under changing illumination or texture. Learned representations can tolerate some of these changes when training and testing cover them, but their responses are neither necessarily smooth nor bounded under shift.

The third pillar is the ability to handle complex multi-contact and deformable dynamics. Many physical manipulation tasks—such as folding textiles, routing flexible wire harnesses through automotive door assemblies, or scooping granular media—involve non-smooth contact mechanics, variable friction, and infinite-dimensional deformations that cannot be modeled in closed-form differential equations. With suitable contact-rich demonstrations and feedback, neural policies can learn compliance-aware responses that reduce force against unexpected resistance; the behavior still needs testing across the intended operating region.

The fourth pillar is empirical scaling with data and compute. Richard Sutton’s *The Bitter Lesson* (Sutton 2019) states that methods leveraging general computation and large-scale learning consistently surpass hand-engineered heuristics. Training on diverse robotic demonstrations (figure 6.5) and multimodal data can improve performance on tested tasks (Zhao et al. 2023; Chi et al. 2024). Data scale alone does not establish dexterity or robustness in a new workspace; engineers must measure coverage and failure modes there.

Rather than an opaque monolith, an embodied foundation model is a four-stage neural pipeline (figure 6.6, figure 6.7) that transforms sensory inputs into candidate motion proposals that still need kinematic checks. To understand how a model reasons about the physical world, a systems engineer must trace how spatial information is organized, aligned with human intent, and translated into motor proposals across four distinct architectural stages:

1. **Sensory Ingestion and Spatial Tokenization (The Inputs):** A digital camera produces a raster grid of pixels $\mathbf{I} \in \mathbb{R}^{H \times W \times C}$. Ingesting raw pixels directly into a standard multilayer perceptron

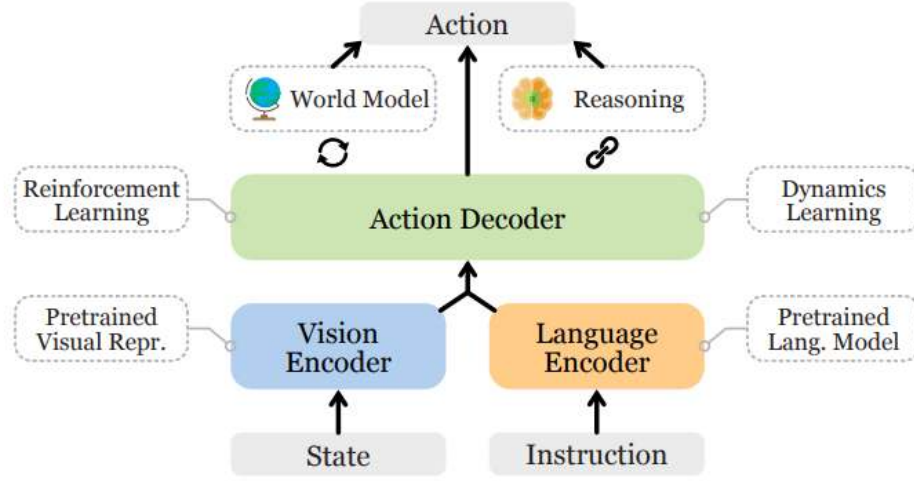


Figure 6.6: Embodied model taxonomy: Systematic categorization of modern embodied foundation models spanning multimodal perceptual frontends (visual tokens, point clouds, language prompts), high-capacity neural backbones (transformers, diffusion heads, world models), and target action spaces. The taxonomy contrasts Vision-Language-Navigation policies that decode topological waypoints and semantic frontiers with Vision-Language-Action policies that emit joint and Cartesian trajectory chunks for articulated manipulators. (Adapted from Ma et al. (Ma et al. 2024)).

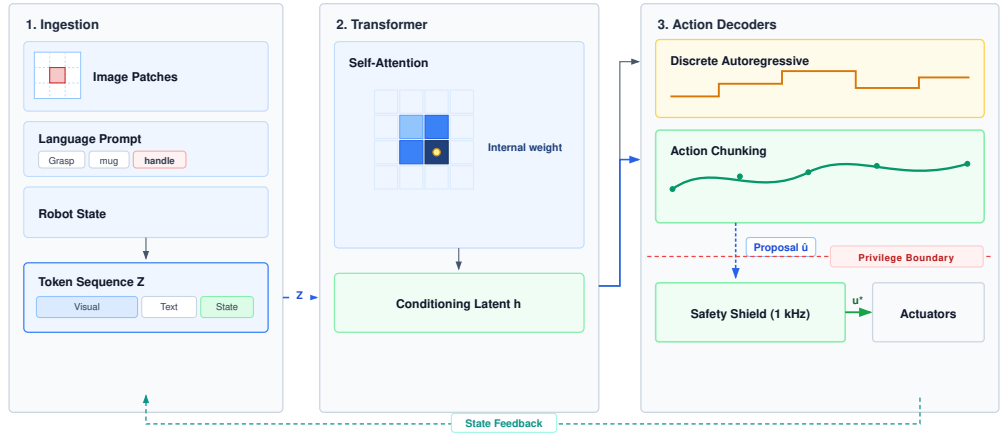


Figure 6.7: Embodied Vision-Language-Action (VLA) architecture: A representative dataflow. (1) Camera patches receive 2D image-position and camera identifiers alongside language and robot-state tokens; metric depth still requires calibrated evidence. (2) Self-attention mixes the combined token sequence; the colored grid illustrates internal attention weights, not a verified affordance or grasp pose. (3) Discrete autoregressive decoders emit tokens sequentially, whereas continuous decoders generate candidate multi-step chunks ($\mathbf{A}_{t:t+H-1}$) through one pass or iterative denoising. (4) An independent permission path checks candidate motion against sensed state and modeled limits before actuation. (Synthesized from Brohan et al. (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023), Kim et al. (Kim et al. 2024), and Octo Model Team (Octo Model Team et al. 2024)).

destroys spatial locality and causes parameter counts to explode. Instead, modern embodied models tokenize visual space through a vision transformer frontend:

- *Spatial patch partitioning:* The continuous 2D image plane is partitioned into a uniform grid of non-overlapping square patches of dimension $P \times P$ pixels (typically $P = 14$ or 16). For an image of resolution $H \times W$, this yields an array of $N_v = (H/P) \times (W/P)$ discrete visual patches. Each 2D patch is flattened into a one-dimensional vector $\mathbf{x}_p^{(u,v)} \in \mathbb{R}^{P^2 C}$, where (u, v) denotes the patch row and column index in the camera plane.

- *Linear embedding and spatial topology injection:* A learned projection matrix $\mathbf{W}_{\text{patch}} \in \mathbb{R}^{(P^2C) \times d_{\text{model}}}$ maps each flattened patch into the model's hidden dimension. Because transformer self-attention is mathematically permutation-invariant (treating inputs as an unordered set), the model possesses zero native awareness of spatial proximity without explicit coordinate grounding. To encode image position, this representative architecture injects a two-dimensional spatial positional embedding $\mathbf{E}_{\text{pos}}^{2D}(u, v)$:

$$\mathbf{z}_0^{(u,v)} = \mathbf{x}_p^{(u,v)} \mathbf{W}_{\text{patch}} + \mathbf{E}_{\text{pos}}^{2D}(u, v) + \mathbf{e}_{\text{cam}}$$

The 2D positional embedding decomposes along the vertical axis u and horizontal axis v using sinusoidal frequency bands or learned 2D coordinate embeddings, encoding patch positions in the image plane without establishing metric 3D distance or guaranteeing nearby latent vectors. When multiple cameras are deployed (such as an overhead third-person camera providing global scene context alongside an eye-in-hand wrist camera providing millimeter-scale alignment), a learned camera identifier $\mathbf{e}_{\text{cam}} \in \{\mathbf{e}_{\text{overhead}}, \mathbf{e}_{\text{wrist}}\}$ is added to distinguish perspective reference frames.

To make these tensor dimensions concrete, consider an autonomous robot processing an RGB frame at standard resolution $224 \times 224 \times 3$ with patch size $P = 14$. The image plane divides into $(224/14) \times (224/14) = 16 \times 16 = 256$ square patches. Each individual patch is a sub-image of $14 \times 14 = 196$ pixels across 3 color channels, flattened into a vector of $196 \times 3 = 588$ raw floating-point numbers. The linear projection matrix $\mathbf{W}_{\text{patch}} \in \mathbb{R}^{588 \times 4096}$ multiplies each 588-dimensional patch vector, mapping it directly into the 4096-dimensional model space. When combined with a 2D spatial grid coordinate $\mathbf{E}_{\text{pos}}^{2D}(u, v)$ (where $u, v \in \{0, \dots, 15\}$) and the camera embedding $\mathbf{e}_{\text{cam}}$, the single video frame becomes an input tensor of exactly $[256, 4096]$. If the robot mounts three cameras (left wrist, right wrist, overhead), the visual input immediately expands to $3 \times 256 = 768$ visual tokens before text instructions or proprioception are even introduced.

- *Multimodal sequence assembly:* Natural language task directives (e.g., “grasp the red mug by its handle”) are tokenized via byte-pair encoding into a sequence of text embeddings $\mathbf{z}_{\text{text}} \in \mathbb{R}^{L \times d_{\text{model}}}$. Concurrently, the robot's physical configuration (joint angles $\mathbf{q}_t$, joint velocities $\dot{\mathbf{q}}_t$, and gripper state) is projected through a linear layer into proprioceptive tokens $\mathbf{z}_{\text{state}} \in \mathbb{R}^{K_s \times d_{\text{model}}}$. These streams concatenate into a unified multimodal input sequence:

$$\mathbf{Z} = [\mathbf{z}_{\text{vision}} \mid \mathbf{z}_{\text{text}} \mid \mathbf{z}_{\text{state}}] \in \mathbb{R}^{N_{\text{total}} \times d_{\text{model}}}$$

2. **Multimodal Alignment and Attention (The Backbone):** The combined token sequence $\mathbf{Z}$ passes through 32 to 80 transformer layers consisting of multi-head self-attention and feed-forward networks. The self-attention operation projects tokens into queries $\mathbf{Q} = \mathbf{Z}\mathbf{W}_Q$, keys $\mathbf{K} = \mathbf{Z}\mathbf{W}_K$, and values $\mathbf{V} = \mathbf{Z}\mathbf{W}_V$, computing pairwise attention weights:

$$A_{i,j} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_k})}{\sum_m \exp(\mathbf{q}_i^\top \mathbf{k}_m / \sqrt{d_k})}$$

This equation describes **self-attention** over the combined token sequence. A text token such as “handle” can attend to visual patch tokens, but a high attention weight is not itself a verified grasp location or metric pose. A trained spatial decoder may produce a 2D affordance heatmap; depth evidence, camera calibration, and geometric checks are still needed to place the gripper in 3D.

A rolling key-value (KV) cache stores prior token states for attention. It does not by itself track object permanence or maintain a 3D position and velocity. An explicit state estimator or tracking memory must associate observations across frames, update a metric pose with calibrated sensors, and increase uncertainty while the mug is occluded.

3. **Specialized Action Decoders (The Outputs):** Instead of generating conversational text, the final hidden layer $\mathbf{h}_{\text{context}}$ feeds task-specific action decoders. While navigation models (VLN)

emit topological waypoints or heading rates (v_t, ω_t) (figure 6.8), manipulation models (VLA) diverge sharply in how they bridge semantic representations to physical motion:

- *Discrete Categorical Tokenization (RT-1, RT-2, OpenVLA)*: Early VLAs adapt autoregressive language modeling directly to robotics by discretizing continuous joint or Cartesian offsets into $B = 256$ uniform bins per dimension (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023; Kim et al. 2024):

$$\text{bin}(a_k) = \left\lfloor \frac{a_k - a_{\min}}{a_{\max} - a_{\min}} \times 255 \right\rfloor \in [0, 255]$$

Each bin is assigned a discrete token (`<act_000>` to `<act_255>`) in the language vocabulary. While this natively leverages web-scale vision-language pretraining, it introduces a severe systems bottleneck: a decoder that emits one scalar token at a time needs seven serial token predictions for a 7-DoF action. Its latency depends on caching, hardware, and whether each prediction revisits the full backbone; seven serial steps can miss a control deadline. Scalar binning has a resolution of roughly $\Delta a = (a_{\max} - a_{\min})/256$; whether that produces contact chatter depends on smoothing, the action range, and the downstream controller.

- *Continuous Generative Action Chunking (Octo, π_0 , ACT, Diffusion Policy)*: To overcome the autoregressive memory wall, modern generalist policies couple the transformer backbone with continuous generative decoders, such as diffusion policy heads (Chi et al. 2024; Octo Model Team et al. 2024) or flow-matching action experts (Black et al. 2024). Rather than predicting a single scalar per forward pass, the model predicts an entire continuous trajectory chunk:

$$\mathbf{A}_{t:t+H-1} = [\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}] \in \mathbb{R}^{H \times d_a} \quad (H = 16\text{--}50 \text{ steps})$$

For example, a backbone may evaluate one new camera observation, then a smaller decoder may perform several denoising iterations to propose $H = 32$ waypoints at 20 ms spacing. That is one 640 ms candidate horizon, not 32 fresh observations. The next backbone evaluation must arrive before the accepted execution window expires; downstream interpolation and feedback run at their own rates.

4. **The Privilege Boundary (Isolation from Actuators)**: Crucially, whether the model is a VLN mobility planner or a VLA manipulation policy, its output is strictly an **unprivileged candidate proposal** (p_t). Following the architectural isolation principle of section 2.2, the foundation model operates in unprivileged user space on the application processor (MPU) and possesses **zero direct hardware authority** to write pulse-width modulation (PWM) registers or physically switch current into the motor coils (*as formalized in the Proposal–Permission Decoupling Principle, 6.1*).

The design rule for this deliberative boundary is that cognitive models operating at low inference cadences (1–10 Hz) propose kinematic setpoints ($\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}$ or Cartesian poses)—or feedforward torque offsets paired with explicit kinematic references—**never unbuffered, open-loop motor torques** (τ) directly to inverter switches. Motor torque is directly coupled to continuous plant dynamics:

$$\tau = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) + \tau_{\text{contact}}$$

Because inertia $\mathbf{M}(\mathbf{q})$ and Coriolis forces $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ vary non-linearly with instantaneous joint configuration and velocity, commanding open-loop torques from a slow 1–10 Hz neural model without high-frequency feedback can destabilize the limb when model error, friction, or payload variation is large enough. Kinematic setpoints, by contrast, specify an intended spatial path while delegating high-gain closed-loop torque synthesis to the deterministic 1000 Hz nervous system and 10–25 kHz field-oriented current loops. (Where specialized high-rate reinforcement learning policies operating at 100–500 Hz output torques directly, they still require an independently validated high-rate permission path, such as the passivity filters and energy-tank monitors detailed in section 8.7.)

Downstream: Zero-copy proposal passing between brain and nervous system relies on the lock-free buffers analyzed in section 8.3.

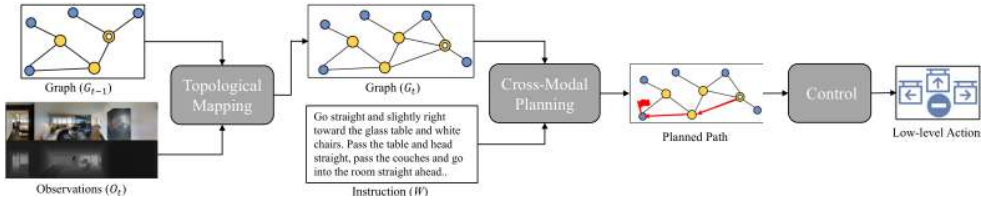


Figure 6.8: Topological vision-language navigation: Embodied navigation architecture executing natural-language spatial instructions. Panoramic visual observations are encoded into perceptual tokens, matched against prompt embeddings, and cross-referenced with a dynamically updated topological navigation graph. The action decoder prunes unreachable nodes and emits metric waypoint offsets ($\Delta x, \Delta y$) and heading setpoints to guide the chassis along traversable semantic frontiers without requiring an exact geometric prior map. (Adapted from An et al. (An et al. 2023)).

Proposals are dispatched across lock-free shared memory to downstream real-time safety monitors on isolated microcontrollers (MCUs). These hardware-level watchdogs evaluate physical stopping distances, torque limits, and the actuator thermal envelopes detailed in section 4.5 at 1000 Hz before granting execution permission, so a hallucinated proposal can be rejected when independent measurements, modeled limits, and a feasible fallback support that decision.⁴



Checkpoint 6.2: Proposal-permission boundary, cadence gaps, and torque isolation

Before tracing the five-stage cognitive pipeline, verify your understanding of why foundation models must be isolated from physical actuator authority:

- ❑ **Proposal-permission separation:** Can you explain why high-capacity foundation models operate strictly as unprivileged motion proposal engines at deliberative cadences (1–5 Hz), while execution authority remains exclusively with downstream deterministic reflex monitors at 1000 Hz?
- ❑ **Kinematic setpoints versus open-loop torque authority:** Can you derive why commanding unbuffered open-loop motor torques (τ) from a slow neural policy can destabilize the plant under non-linear Coriolis and inertial dynamics ($\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}$), and why this architecture proposes kinematic trajectories instead?
- ❑ **Spatial patch coordinate grounding:** Can you describe why transformer self-attention needs position information, what 2D and camera embeddings encode, and why metric workspace coordinates need more evidence?
- ❑ **Discrete binning versus continuous chunking:** Can you explain when sequential scalar-token decoding repeats costly backbone work, and how a chunk shares a backbone evaluation across future waypoints?

If the distinction between kinematic trajectories and raw motor torques is unclear, review the Proposal-Permission Decoupling Principle in section 6.1 before examining pipeline stages.

⁴ | **Proposal-Permission Privilege Boundary:** Treating the neural policy as an unprivileged proposal generator isolates stochastic inference from actuator registers. Candidate trajectory setpoints $\mathbf{q}_{\text{ref}}(t)$ are continuously evaluated against forward-invariant Control Barrier Functions (CBFs) $h(\mathbf{x}) \geq 0$ on dedicated safety silicon, so proposals that violate modeled actuator limits or collision envelopes can be rejected when independent state estimates and feasible control authority support the check. Mathematical formulations of CBFs appear in Appendix B.

6.4 The Cognitive Pipeline

Extending the proposal-permission architecture from section 6.1, to architect a cognitive pipeline, we must trace how information flows from physical sensations to actionable setpoints. A dependable physical AI architecture organizes cognition into five sequential stages instead of treating the brain as an unstructured black box (figure 6.9): Ingestion, Perception, Memory, Intent, and Planning.

Ingestion and representation

Cognition begins at the sensory interface of this causal boundary (section 2.2), where environmental energy transduces into digital bitstreams. Photons strike active-pixel CMOS sensors across 2D camera grids, infrared pulses measure photon time-of-flight in depth cameras, and micro-electromechanical systems (MEMS) in inertial measurement units (IMUs) sample accelerations and angular velocities.

The ingestion stage translates heterogeneous, high-bandwidth bitstreams—ranging from analog voltage spikes to digital pixel grids—into normalized numerical embeddings, a uniform vocabu-

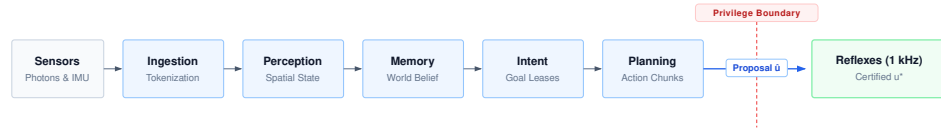


Figure 6.9: The Physical AI cognitive flow: Information flows sequentially from physical sensors through Ingestion (tokenization), Perception (spatial geometry and affordances), Memory (persistent world model belief), Intent (deliberative goal leases), and Planning (multi-step action chunking). The architecture must facilitate crossing the ‘proposal-permission boundary’, a juncture where proposed actions are evaluated against real-time constraints and resources, leading to downstream reflexes.

lary for neural networks. For the mathematical formulation of patch embeddings and 3D metric lifting, see Appendix A. Because monocular 2D camera images suffer from fundamental scale-depth projective ambiguity—where a small near object and an identically shaped, proportionally larger distant object cast identical pixel projections—a downstream perception and estimation stage needs depth evidence, calibrated geometry, or explicit scale assumptions to estimate a 3D metric pose. Tokenization alone does not supply that scale.⁵

Consider how this operates across the three machine archetypes:

- *Mobile systems:* A quadruped traversing rubble synchronizes four RGB-D cameras streaming at 30 fps (> 740 MB/s raw data) alongside 12 joint encoders sampled at 1000 Hz. Ingestion tokenizes each visual frame into hundreds of patch embeddings while downsampling and embedding the high-frequency joint state stream into a unified temporal sequence.
- *Manipulators:* A dual-arm workstation ingests synchronized stereo wrist camera feeds (60 fps) and 14-DoF joint positions, tokenizing visual patches and motor states into aligned input tokens for end-effector coordination.
- *Process / energy systems:* A continuous chemical reactor ingests thousands of analog thermocouple, pressure transducer, and mass flow meter channels at 100 Hz, normalizing heterogeneous sensor calibrations into a rolling multivariate state tensor.

This ingestion pipeline places substantial demands on data bandwidth and memory throughput. For an embodied machine equipped with N_{cameras} synchronized visual streams at image resolution $H_{\text{img}} \times W_{\text{img}}$ processed by a vision transformer with patch dimension $P \times P$, the spatial token ingestion rate per frame is:

$$N_{\text{vision}} = N_{\text{cameras}} \times \left(\frac{H_{\text{img}}}{P} \right) \times \left(\frac{W_{\text{img}}}{P} \right)$$

For a typical multi-camera workstation ($N_{\text{cameras}} = 3$, 224×224 pixels, $P = 14$), every video frame generates $3 \times (16 \times 16) = 768$ visual tokens. Streaming multiple high-definition camera feeds generates gigabytes of raw pixel data every second. Systems engineers must implement efficient frame buffering, sensor exposure synchronization, and spatial patch pooling to keep data moving smoothly before cognitive reasoning begins.

Perception and spatial grounding

Perception extracts metric structure, semantic identity, and spatial affordances from ingested tokens. Rather than treating sensory data as an unstructured bag of visual features, perception grounds representations in the Special Euclidean group $SE(3)$, which formally defines rigid-body spatial configurations across translation and rotation coordinates.⁶

Using dense self-supervised representations like DINOv2 (Oquab et al. 2023) and learned spatial decoders, perception isolates physical entities, segments object boundaries, estimates 3D bounding boxes, and reconstructs surface normals. Crucially, perception extracts spatial affordances: functional geometric fields that describe where and how an object can be physically engaged. For a drill, perception identifies not merely a generic class label, but the 3D position and orientation of the handle grasp axis, the trigger contact surface, and the drill bit tip. This transforms ungrounded pixel arrays into actionable spatial entities referenced to the robot’s base frame.

⁵ | **Cognitive Systems Progression:** While this chapter establishes the overarching systems model of embodied cognition, rigorous mathematical and algorithmic formulations for each stage are developed in Part III, spanning spatial perception (chapter 16), spatial memory (chapter 18), grounded intent (chapter 20), trajectory planning (chapter 22), and real-time safety enforcement (chapter 24).

⁶ | **Special Euclidean Group $SE(3)$:** The Lie group $SE(3) = \mathbb{R}^3 \rtimes SO(3)$ defines the configuration of a rigid body in three-dimensional space, combining a translation vector $\mathbf{p} \in \mathbb{R}^3$ with an orthonormal rotation matrix $\mathbf{R} \in SO(3)$ satisfying $\mathbf{R}^T \mathbf{R} = \mathbf{I}$ and $\det(\mathbf{R}) = +1$. Representing poses on $SE(3)$ avoids the mathematical singularities (gimbal lock) associated with Euler angles and provides a smooth manifold for trajectory optimization. Derivations are provided in Appendix B.

When a mobile manipulator is commanded to retrieve a torque wrench from a cluttered workbench, perception isolates the tool from surrounding background clutter, predicts its 6-DoF pose ($SE(3)$ transformation matrix T_{base}^{tool}), and outputs a continuous heatmap over the handle indicating valid gripper pinch contacts.

In practice, perception is highly vulnerable to sensor calibration drift. If camera mounting fasteners deflect by fractions of a millimeter or structural linkages deform under heavy payload torque, the extrinsic calibration transformation matrix between the camera optical frame and the robot base coordinate frame drifts, compromising spatial registration. This introduces metric scale errors that cause precision grippers to miss targets or strike work surfaces. Systems engineers must implement online hand-eye recalibration routines and maintain spatial uncertainty covariances alongside raw point estimates.

Memory and spatial-temporal belief

A physical agent cannot reliably act from a single snapshot of the world. Objects pass behind visual occlusions, cameras pan away from manipulated targets, and physical bodies possess dynamic momentum that cannot be inferred from a single frame.

The memory stage maintains a persistent spatial-temporal belief of the environment. It tracks object identities across time, correlates visual features with geometric coordinate frames, and updates 3D volumetric representations such as Truncated Signed Distance Fields (TSDFs) (Curless and Levoy 1996), Language Embedded Radiance Fields (LERF) (Kerr et al. 2023), or 3D Gaussian Splatting maps (Kerbl et al. 2023). By fusing historical observations across a sliding temporal window, memory maintains accurate state estimates even during complete visual occlusion, computes linear and angular object velocities, and tracks operational context across multi-step tasks.

For a quadruped traversing a rocky ledge, the forward-facing camera points toward the horizon, losing sight of the immediate ground beneath its front feet. The memory stage preserves the reconstructed 3D surface geometry of the terrain, enabling the robot to securely place its hind feet onto footholds it observed several seconds earlier.

Maintaining dense volumetric belief states introduces significant memory footprints. Storing high-resolution 3D voxel grids or radiance field parameters requires substantial storage capacity, competing directly with the memory needed to run large neural network models. Furthermore, belief representations degrade in freshness over time: if a dynamic obstacle moves while occluded, an un-decayed memory belief causes the robot to plan around a “phantom” obstacle or collide with an unobserved intruder. Systems engineers must enforce explicit temporal decay rates on spatial belief states to reflect physical entropy.

Intent and deliberative reasoning

With a persistent world belief and a high-level mission prompt, the intent stage performs semantic reasoning and task decomposition. Because open-vocabulary foundation models operate over vast parameter spaces (typically 7B to 70B parameters), deliberative reasoning occurs at slow, human-scale cadences (1–5 Hz).

To understand how intent operates, imagine a human operator issues a natural language mission prompt:

“Clear the table by placing the ceramic mug in the sink and throwing the paper napkin into the trash bin.”

The intent layer cannot execute this directive in a single leap. Using embodied reasoning architectures such as SayCan (Ahn et al. 2023), VoxPoser (Huang et al. 2023), or Code as Policies (Liang et al. 2023), the deliberative model queries the active world memory and decomposes the mission into an ordered sequence of discrete subgoals: first approach the mug, grasp the handle, transport it to the sink coordinate frame, and release contact, before turning toward the napkin.

If the mug moves during a 400 ms deliberation interval, a previously useful subgoal may already be stale. The next proposal therefore needs a target, generation and expiration times, and an allowed region checked against independently maintained geometry. The intent stage can package these into a time- and region-bounded **intent lease** ($\mathcal{L}_{intent}$); the enforcer must verify the corridor rather than trust the learned model to define safe space. Table 6.1 gives one implementation.

Table 6.1: Illustrative intent lease contract schema: One possible binary layout of the intent lease payload ($\mathcal{L}_{\text{intent}}$).

Field Name	Type	Unit	Description & Structural Invariants
lease_id	uint64_t	—	Monotonically increasing 64-bit sequence identifier
target_entity_id	uint32_t	—	Hash of target entity in persistent spatial memory
pose_target_se3	float32[4][4]	m, rad	Valid SE(3) pose matrix: $\mathbf{R} \in \text{SO}(3)$, $\det(\mathbf{R}) = 1$, $\mathbf{p} \in \mathbb{R}^3$, bottom row $[0, 0, 0, 1]$
spatial_corridor_box	float32[6]	m	Authorized 3D bounding box $[\mathbf{p}_{\min}, \mathbf{p}_{\max}] \in \mathbb{R}^6$
t_generated_ns	uint64_t	ns	PTP monotonic hardware timestamp at deliberation completion
t_expire_ns	uint64_t	ns	Hard expiration timestamp; enforces $t_{\text{expire}} - t_{\text{gen}} \leq \tau_{\text{valid}}$
max_permissible_jerk	float32	rad/s ³	Maximum allowable joint angular jerk magnitude
hmac_signature	uint8_t[32]	bytes	Optional authentication tag when the threat model requires source integrity

Identifiers, jerk limits, and authentication depend on the implementation and threat model. Since the target mug or a coworker can move before the lease reaches the planner, execution must reject an expired lease and recheck the measured region. Because intent leases prescribe target envelopes rather than executable motor trajectories, the architecture delegates trajectory generation to a downstream planning stage that transforms symbolic intents into continuous, kinematically verified action chunks.



Checkpoint 6.3: Perceptual grounding, spatial memory, and intent contracts

Before analyzing action chunking decoders and edge SoC memory walls, verify your understanding of how sensory fluxes are converted into verifiable deliberative intent contracts:

- ☐ **3D metric lifting from projective cameras:** Can you explain what depth or scale evidence and calibration are needed to estimate a metric pose from monocular pixels, and how calibration drift degrades grasp accuracy?
- ☐ **Attention versus geometric grounding:** Can you explain what self-attention over text and image tokens can associate, and what additional decoder, depth evidence, and calibration are needed for a metric grasp pose?
- ☐ **Volumetric memory belief decay:** Can you explain why persistent spatial representations (TSDFs, 3D Gaussian Splats) require explicit temporal decay rates to prevent machines from planning trajectories around phantom obstacles or colliding with unobserved intruders?
- ☐ **Intent lease contract bounds:** Can you enumerate the structural fields of an intent lease ($\mathcal{L}_{\text{intent}}$, table 6.1) and calculate why exceeding lease validity (τ_{valid}) during a 400 ms inference pause forces downstream execution rejection?

Ensure you can trace sensory dataflow through the intent lease schema in table 6.1 before evaluating how the planning stage converts symbolic leases into continuous motor waypoints.

Planning and action chunking

An intent lease specifies an operational goal but provides no continuous motor trajectories or guarantees that intervening motion meets physical kinematic limits. The planning stage converts the active intent lease into candidate joint trajectories or end-effector motion setpoints via an **unprivileged action decoder head**.

Planning executes on physical silicon by treating deep neural policies as compute workloads on embedded accelerators—like high-performance heterogeneous edge SoCs with integrated GPU tensor cores and dedicated deep learning accelerators—bounded by memory bus bandwidth, silicon thermals, and electrical power.

Consider what happens if a robot attempts to generate continuous motor actions one discrete step at a time through an autoregressive foundation model (such as a 7-billion parameter Vision-Language-Action network). In single-step autoregressive decoding, each sequential token may require another backbone evaluation and associated weight traffic when weights cannot remain on chip. The actual traffic depends on caching and the decoder architecture. For detailed memory wall derivations and roofline hardware models, see Appendix C.

Token-by-token execution of a nonresident model can require repeated gigabyte-scale weight traffic if the controller requests a fresh action every millisecond (see figure 6.10) (Williams et al. 2009). This memory wall bottleneck⁷ can limit fresh policy updates to single-digit frequencies on a bandwidth-constrained implementation. If an embodied manipulator attempted to control moving mass at 5 Hz, the actuators would stutter across discrete steps, inducing severe contact instability and mechanical chatter.

This continuous data movement incurs a massive thermodynamic penalty. Fetching weights from off-chip DRAM consumes orders of magnitude more energy than the arithmetic computation itself, which drains battery reserves and causes thermal throttling on mobile robots.

🔍 Systems Perspective 6.1: Memory walls in embodied inference

For a model whose weights must be streamed once per evaluation, the memory wall gives a lower bound on transfer time and hence an upper bound on evaluation frequency ($f_{\max} = B_{\text{mem}}/M_{\text{weights}}$) imposed by external DRAM bus bandwidth when streaming non-resident neural network weights into on-chip vector registers. For an illustrative single-token weight sweep, arithmetic intensity can approach $I \approx 1.0$ FLOP/byte, which can leave arithmetic units waiting for data; actual model latency also includes tokenization, attention, and decoding.

To supply dense waypoints despite slow full-model inference, modern physical AI policies employ **action chunking** (Zhao et al. 2023; Chi et al. 2024):⁸ predicting a sequence of H future continuous actions $\mathbf{A}_{t:t+H-1} = [\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}]$ in a single neural forward pass or fixed diffusion denoising schedule, amortizing parameter memory transfers across multiple real-time execution cycles.

Instead of fetching billions of weights from DRAM to predict a single scalar action for the immediate millisecond, the planning policy (such as Action Chunking with Transformers [ACT], Diffusion Policy, or Flow Matching) generates an entire temporal horizon of continuous actions $\mathbf{A}_{t:t+H-1} = [\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}] \in \mathbb{R}^{H \times d_a}$ (typically $H = 16$ – 64 steps spanning 320–1280 ms) in one backbone pass followed, for an iterative decoder, by an assumed $K = 10$ – 16 refinement steps.⁹ Each refinement re-evaluates the smaller action head, so full latency is $t_{\text{vision}} + t_{\text{backbone}} + K t_{\text{head}} + t_{\text{transfer}}$, not just one weight sweep. A new observation changes the plan only after another backbone and decoder pass; see Appendix A for the generative mechanism.

📖 Definition 6.2: Temporal action chunking

Temporal action chunking is the policy formulation that predicts an entire horizon of H future continuous actions $\mathbf{A}_{t:t+H-1} = [\mathbf{a}_t, \dots, \mathbf{a}_{t+H-1}]$ in a single neural evaluation or diffusion rollout, amortizing parameter memory transfers across extended physical execution windows.

1. **Significance:** On the illustrative bandwidth-limited device, repeating a full model evaluation for every action can be too slow for millisecond control. Temporal action chunking can reuse a backbone evaluation across multiple waypoints; it does not make fresh observations or policy decisions arrive at the waypoint rate. A downstream controller tracks the accepted trajectory between model updates.

Downstream: High-throughput neural inference accelerators are isolated from deterministic cores in section 26.2.

7 | Autoregressive DRAM Bandwidth Saturation: Sequential action tokens can trigger repeated full-model weight traffic when weights are nonresident and the decoder revisits the backbone; cache behavior and hardware determine the actual bandwidth ceiling. Action chunking is one way to reduce the number of full policy evaluations per unit of physical time.

8 | Vision-Language-Action (VLA) Models: Some Vision-Language-Action architectures use continuous action chunks $\mathbf{A}_{t:t+H-1}$ instead of discrete action tokens. These are candidate joint or Cartesian setpoint sequences, not direct actuator writes; the decoder and permission path determine how they are used.

9 | Action Chunk Horizon Bounds: Sizing prediction horizon H balances compute amortization against open-loop drift. Choosing $H \in [16, 64]$ steps (320–1280 ms at 20 ms per step) provides a candidate horizon; allowable execution length still depends on feedback, drift, clearance, and stopping feasibility.

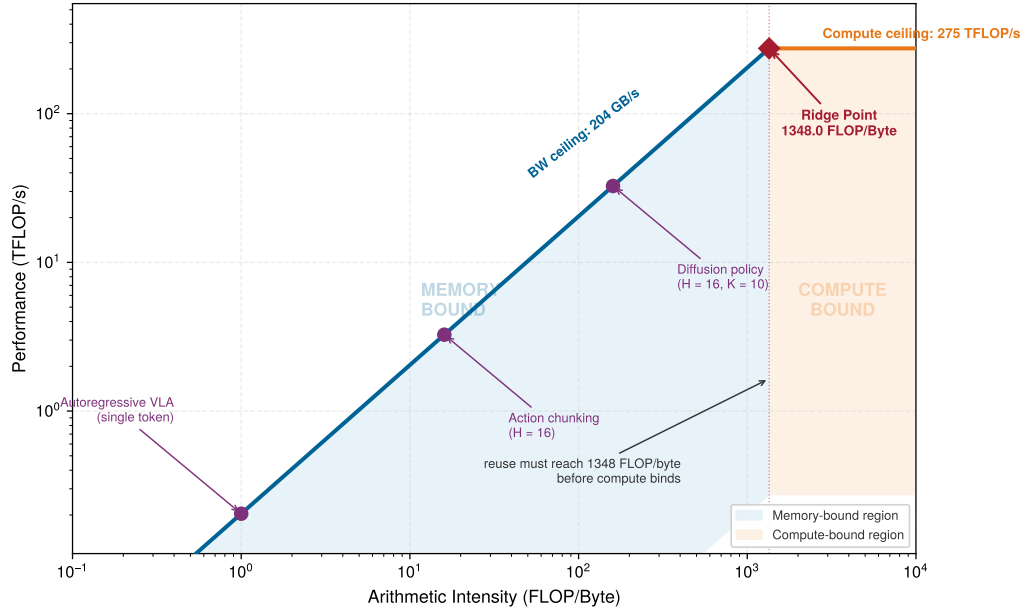


Figure 6.10: Illustrative embodied roofline: Arithmetic intensity against attainable throughput for an example edge accelerator. Single-token weight streaming can be memory-bound; reuse across a chunk may increase work per byte. The plotted positions illustrate possible workloads, not measured policy cadence. A separate real-time loop executes accepted waypoints between fresh model updates.

2. **Distinction:** Unlike single-step Markovian policies $\mathbf{a}_t \sim \pi(\cdot | \mathbf{o}_t)$ that re-evaluate the full neural network at every control step, action chunking deliberately generates multi-step feedforward trajectories, utilizing receded-horizon execution and temporal ensembling to reconcile slow neural deliberation with fast closed-loop reflexes.
3. **Common pitfall:** Executing action chunks open-loop without continuous temporal smoothing or runtime safety filtering. Without smooth blending across overlapping chunk boundaries, switching between independently generated horizons injects acceleration jumps and torque shocks that excite mechanical gearbox resonances.

Table 6.2: Action chunk trajectory packet schema: Binary layout of the action chunk trajectory packet ($\mathbf{A}_{\text{chunk}}$).

Field Name	Type	Unit	Description & Structural Invariants
chunk_id	uint64_t	—	Monotonically increasing trajectory sequence ID
parent_lease_id	uint64_t	—	Must match active unexpired Intent Lease ID
horizon_steps (H)	uint32_t	count	Temporal horizon length of candidate waypoints (e.g., 32)
step_duration_dt	float32	s	Fixed inter-waypoint control sampling period $\Delta t = 0.02$ s
trajectory_q	float32[H] [N]	rad or m	Planned joint positions for all N degrees of freedom
trajectory_qdot	float32[H] [N]	rad/s or m/s	Planned velocities satisfying $\ \dot{\mathbf{q}}_j\ \leq \dot{q}_{\max,j}$
trajectory_qddot	float32[H] [N]	rad/s ²	Planned accelerations satisfying $\ \ddot{\mathbf{q}}_j\ \leq \ddot{q}_{\max,j}$
t_generation_ns	uint64_t	ns	PTP synchronized exposure timestamp of source observation

As structured in table 6.2, one backbone evaluation can condition several future waypoints. Whether the decoder fits on-chip, which roofline regime it occupies, and how often a fresh chunk can be generated depend on its implementation, hardware, and full inference latency. Waypoint density is therefore separate from new-observation cadence.

For example, when a canonical bimanual manipulator threads a flexible USB cable (figure 6.5), it executes a 32-step action chunk. The policy emits coordinated 14-DoF joint setpoints that smoothly align both grippers, flex the cable, and insert the connector without pausing between individual control cycles.

However, action chunking introduces the critical systems challenge of **chunk seam continuity**. Because each action chunk is generated independently from rolling sensory observations, the tail of chunk k and the head of chunk $k + 1$ may exhibit slight positional and velocity discrepancies. If a controller concatenated these chunks directly, the resulting step-discontinuities in acceleration would inject massive jerk ($\ddot{\mathbf{q}}$) into the physical linkages, inducing mechanical shock, acoustic vibration, cycloidal pin wear, and motor overcurrent faults.

To reduce discontinuities between overlapping chunks, systems can implement **temporal ensembling**, blending overlapping trajectory predictions via exponentially weighted averaging:

$$\mathbf{a}_t = \frac{\sum_{i=0}^{\min(t, H-1)} w_i \mathbf{a}_{t-i}}{\sum_{i=0}^{\min(t, H-1)} w_i}, \quad w_i = \exp(-m \cdot i)$$

where $\mathbf{a}_{t-i}$ represents the action predicted for time step t by the action chunk initiated at time $t - i$, and $m > 0$ is a tunable temporal discount factor. The resulting smoothed waypoints are interpolated via quintic (C^2) splines by the downstream real-time nervous system at 1000 Hz, ensuring continuous positions, velocities, and accelerations across all actuator joints while preserving valid $SE(3)$ transformations via Lie algebra $\mathfrak{se}(3)$ exponential maps or Spherical Linear Interpolation to maintain orthonormal rotation matrices.



Overlapping action chunks blend predictions across receding temporal execution windows.

Downstream: Tokenized action proposals are mapped to continuous polynomial splines in section 22.1.



Napkin Math 6.1: Memory walls and action chunking speedup

How does Action Chunking overcome the memory bus bottleneck on an embedded physical AI compute module?

1. *The single-token memory bottleneck.* Consider a 7B Vision-Language-Action (VLA) policy stored in 16-bit precision (FP16) on an embedded edge System-on-Chip (e.g., a canonical heterogeneous SoC with a 256-bit LPDDR5 interface):

- Weight footprint: $M_{\text{weights}} = 7 \times 10^9 \text{ params} \times 2 \text{ bytes} \approx 14 \text{ GB}$.
- Theoretical memory bandwidth: $B_{\text{peak}} = 204 \text{ GB/s}$.
- Assumed sustained DRAM bandwidth (70 percent of peak to model bus scheduling and memory overhead): $B_{\text{sustained}} \approx 142.8 \text{ GB/s}$.

Minimum streaming latency to fetch the weights from DRAM across the silicon bus once:

$$t_{\text{stream}} = \frac{14 \text{ GB}}{142.8 \text{ GB/s}} \approx 98.0 \text{ ms}$$

Bandwidth-only upper bound on fresh evaluations when each evaluation streams the full weights:

$$f_{\text{single}} = \frac{1}{t_{\text{stream}}} \approx \frac{1}{0.0980 \text{ s}} \approx 10.2 \text{ Hz}$$

Beyond model weights, autoregressive foundation models suffer from **Key-Value (KV) cache memory expansion**. When multi-camera sensory feeds (e.g., three 224×224 RGB streams) are ingested, each video frame contributes hundreds of visual patch tokens ($3 \times 256 = 768$ tokens). In a 32-layer transformer with hidden dimension $d = 4096$, storing the FP16 Key and Value states consumes 0.50 MB per token:

$$M_{\text{KV}} = 2 \times L \times d \times S \times \text{bytes} = 2 \times 32 \times 4096 \times S \times 2 \approx 0.50 \cdot S \text{ MB}$$

Across a 768-token visual prompt, the KV cache expands by 384 MB per frame. Retaining historical frames in context over a 5-second window at 10 Hz accumulates over 19 GB of KV cache, which can exceed the memory available after model weights and other workloads are allocated. Vision tokenization and attention add latency beyond the weight-stream bound; their measured costs determine the actual fresh-update rate. A separate fast feedback loop is needed to stabilize the physical plant between updates.

2. *The action chunking amortization.* Instead of predicting one action step per DRAM weight sweep, the policy generates an action chunk of $H = 32$ future time steps ($\Delta t = 20$ ms, spanning a 640 ms trajectory horizon) in a single neural forward pass.

- Number of weight memory sweeps: 1 sweep (98.0 ms).
- A chunk can reuse one backbone evaluation across H future waypoints. Decoder arithmetic and cache residency determine the actual intensity and latency; the weight-stream calculation alone does not show a move to the compute-bound roofline regime.
- Amortized weight-transfer time per supplied waypoint (excluding all other inference work):

$$\tau_{\text{step}} = \frac{t_{\text{stream}}}{H} = \frac{98.0\text{ms}}{32 \text{ steps}} \approx 3.06\text{ms/step}$$

3. *Physical AI systems impact.* By allocating one weight-stream time across the temporal chunk horizon, the **amortized weight-transfer budget per waypoint** corresponds to a numerical rate that climbs from 10.2 Hz to:

$$f_{\text{amortized}} = \frac{1}{\tau_{\text{step}}} \approx 326.4\text{Hz (waypoint-equivalent)}$$

This is a supply-accounting ratio, not a fresh-policy rate: the example still takes at least one 98.0 ms weight stream per new chunk, plus tokenization, attention, and decoding. If the full inference latency fits a chosen replanning interval, overlapping chunks can supply waypoints to a separate high-rate servo; spare CPU, memory-bus, and safety-monitor margins must be measured.

Systems Perspective 6.2: Embodied cognition guidelines

1. **The Autoregressive Actuator Stall:** Never drive physical motor loops directly from single-token autoregressive generation; DRAM streaming latency ($t_{\text{stream}} = M_{\text{weights}}/B_{\text{mem}}$) will starve the actuators below the body's resonant frequency.
2. **The Roofline Check:** Measure arithmetic intensity and sustained bandwidth for the actual decoder and hardware before attributing latency to DRAM or claiming a compute-bound speedup.
3. **The Proposal-Permission Boundary:** The learned brain proposes candidate intent and trajectories in unconstrained software coordinates; the deterministic reflex engine grants physical execution permission in verified SI units.
4. **Softmax Is Not Physical Probability:** Exponentiated logits sum to one by mathematical definition, not physical truth; never gate safety-critical actuators on raw neural confidence without empirical calibration (ECE) and deterministic geometric fallback monitors.
5. **Heterogeneous Bus Contention & Real-Time Isolation:** When high-throughput NPUs stream multi-gigabyte foundation model weights across shared DRAM channels, real-time CPU cores executing critical hardware drivers (such as CAN bus motor commands or IMU sensor reads) experience severe latency spikes, momentarily starving the robot of physical reflexes. Enforce memory bandwidth reservation and regulation (e.g., Mem-

Guard (Yun et al. 2013)) to isolate real-time nervous system traffic from cognitive DRAM bursts.

As quantified in 6.1, memory bus throughput establishes an unyielding physical bound on deliberation cadences. In the notebook scenario, sustained LPDDR5 bandwidth is assumed to be 142.8 GB/s, so streaming a 7B FP16 model (14 GB) takes about 98 ms before vision, attention, or decoder work. INT4 storage would reduce the nominal weight footprint to 3.5 GB but needs accuracy validation for contact tasks. Chunking offers another trade-off without changing weight precision: by evaluating the 7B backbone periodically while a lightweight diffusion head denoises candidate trajectories on-chip, the system can supply dense candidate waypoints from a slower full-model evaluation. The new-observation rate remains limited by full inference latency, including vision and decoder work.

Memory contention escalates dramatically when multi-camera sensory feeds are retained in context. Ingesting three synchronized RGB streams produces hundreds of visual patch tokens per step ($S \approx 800$ tokens including language and proprioception). In a 32-layer transformer, storing FP16 Key-Value states requires ≈ 0.50 MB per token, accumulating over 400 MB of KV cache per observation step. Retaining an unpruned 5-second historical context window at 10 Hz yields over 20 GB of KV cache. Combined with 14 GB of weights, this example exceeds 32 GB before runtime overhead, risking out-of-memory failure. Even where memory capacity permits, reading a large cache consumes bandwidth; the time depends on which tokens attention accesses and the sustained bandwidth. Embodied architectures must therefore avoid unpruned autoregressive visual histories, relying instead on spatial pooling, cross-attention feature projections, or fixed-horizon action chunking.

Once synthesized, action chunks must be executed without allowing dynamic drift to breach physical tolerances. Consider a 7-DoF robotic manipulator picking components from a moving conveyor ($v_{\text{conveyor}} = 0.30$ m/s) with trajectory period $\Delta t = 20$ ms (50 Hz) and policy inference latency $t_{\text{infer}} = 160$ ms. Sizing the chunk horizon to avoid actuator buffer starvation requires $H_{\text{min}} = t_{\text{infer}}/\Delta t = 160/20 = 8$ steps; executing fewer steps forces the arm to halt mid-motion every cycle. Conversely, under conveyor belt slip accelerating target drift at $a_{\text{drift}} = 0.50$ m/s², open-loop execution over $H = 64$ steps ($T_{\text{chunk}} = 1.28$ s) accumulates spatial drift of $\Delta x_{\text{drift}} = \frac{1}{2}(0.50)(1.28)^2 \approx 0.410$ m = 410 mm—exceeding the gripper’s 15 mm tolerance by a factor of 27.3 and causing collision. The maximum execution duration before drift breaches 15 mm is $T_{\text{max}} = \sqrt{2\Delta x_{\text{max}}/a_{\text{drift}}} = \sqrt{2(0.015)/0.50} \approx 0.245$ s = 245 ms, capping execution at $H_{\text{max}} = 245/20 \approx 12$ steps. Sizing the execution window to $H_{\text{exec}} = 10$ steps (200 ms, replanning at 5 Hz) leaves a 40 ms inference safety margin while capping maximum drift at $\Delta x_{\text{drift}} = \frac{1}{2}(0.50)(0.20)^2 = 10.0$ mm < 15 mm. Sizing chunk horizons requires overlapping receded-horizon execution with temporal ensembling to balance closed-loop reactivity against smooth motor continuity.

Downstream: Bounded action chunk proposals are filtered against forward-invariant barriers in section 24.7.



Checkpoint 6.4: Edge SoC deliberation limits: Memory walls, KV cache, and dynamic drift

Before analyzing end-to-end cognitive pipelines and runtime failure modes, verify your understanding of edge deliberation limits across memory and time:

- ☐ **DRAM streaming bounds:** Under what conditions does $t_{\text{stream}} = M_{\text{weights}}/B_{\text{mem}}$ bound full-model cadence, and why does action chunking reduce transfer cost per supplied waypoint without multiplying fresh-policy cadence?
- ☐ **KV cache memory expansion:** Can you describe how multi-camera tokenization drives linear KV storage growth with sequence length (while full attention compute can grow quadratically) across unpruned observation histories, and how unified memory contention on embedded SoCs threatens real-time sensor and motor tasks?
- ☐ **Chunk horizon sizing and dynamic drift:** Can you explain how the minimum chunk horizon ($H_{\text{min}} \geq t_{\text{infer}}/\Delta t$) prevents actuator buffer starvation while maximum horizon

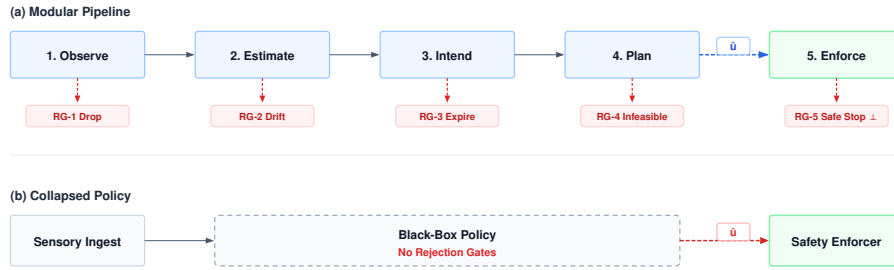


Figure 6.11: Five handoffs architecture: The modular systems chain enforces five functional contracts—Observe, Estimate, Intend, Plan, and Enforce—each equipped with a rejection gate to intercept invalid proposals before reaching actuators. Panel (b) maps pixels to proposed torques, which still pass through an independent safety enforcer before actuation. The collapsed path removes four earlier places where invalid proposals could be rejected cheaply.

limits ($H_{\max}$) prevent open-loop spatial drift ($\Delta x \approx \frac{1}{2} a_{\text{drift}} t^2$) from breaching physical clearance envelopes?

Having traced each stage of cognition from sensory ingestion to trajectory planning, we can synthesize these contracts into a unified architectural pipeline (figure 6.11). In a modular systems chain (panel a), explicit rejection gates (RG-1 through RG-5) validate timestamped evidence, state estimates, goal leases, and kinematic bounds before unprivileged proposals can reach physical actuators. In contrast, collapsing the pipeline into an end-to-end policy (panel b) maps pixels to proposed torque values without exposing intermediate reasoning stages. Those values still need an independent permission path before actuator writes; the downstream enforcer carries more of the validation burden.

6.5 Cognitive Vulnerabilities

Learned foundation models extend task coverage but make a general proof of integrated closed-loop behavior difficult; specified components and bounded operating regions can still be analyzed.¹⁰

A deep neural network is a high-dimensional, non-linear statistical function. Without a verified input domain and sensitivity bound, small sensing changes can yield unexpectedly large changes in proposed motion. Benchmark success does not establish closed-loop stability, timing bounds, or safety for unseen conditions; those properties require separate evidence for the model, runtime, and physical plant.¹¹

A learned brain operates through statistical approximation, leading to failure modes distinct from traditional software defects. A classical program fails by throwing an exception, hanging in an infinite loop, or crashing. A neural network can fail silently by emitting finite, in-range tensors that propose hazardous motion. An independent permission path must reject a detected violation before those values can become actuator commands. Systems architects must design around four primary cognitive vulnerabilities:

Semantic hallucinations

A cleanroom manipulator sees sunlight reflected from a transparent acrylic shield and proposes a reach through what it estimates to be open space. The trajectory tensor is finite and within joint limits, yet the presumed clearance is false. Its plausibility cannot substitute for an independent obstacle check.

The Softmax illusion and out-of-distribution overconfidence

Deployment interfaces often confuse mathematical normalization with physical probability. In deep neural networks, output logits are passed through a softmax function.¹²

Embodied machines encountering out-of-distribution (OOD) observations—such as novel optical textures, deep shadows, or lens water droplets—routinely assign confidence scores exceeding 0.95 to erroneous classifications.

10 | Connectionist and Symbolic Duality: Physical AI architectures reconcile statistical connectionism with deterministic symbolic computation: high-capacity neural networks synthesize open-world behavioral proposals, while formal symbolic logic and invariant monitors verify kinematic validity before setpoints are granted execution authority.

11 | Lipschitz Continuity in Neural Policies: Although neural networks with smooth activations are continuous on compact domains, a bound $L = \sup_{x_1 \neq x_2} \frac{\|\pi(x_1) - \pi(x_2)\|}{\|x_1 - x_2\|}$ must be established over a specified operating domain. If a valid bound exists, sensor error $\|\delta x\| \leq \epsilon$ implies $\|\delta a\| \leq L\epsilon$; an unverified L supplies no usable action-error guarantee. Mathematical formulations appear in Appendix A.

12 | Softmax Normalization and Epistemic Uncertainty: The softmax operator $\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$ enforces normalization across the categorical simplex Δ^K by algebraic construction rather than Bayesian epistemic confidence. High output confidence reflects only relative logit separation, meaning models assign near-unity probabilities even when evaluating inputs completely outside their training distribution. Derivations appear in Appendix A.

Evaluating the true relationship between reported model confidence and empirical physical success requires measuring **Expected Calibration Error (ECE)**—the mathematical gap between how sure the model thinks it is and how frequently the physical task actually succeeds.¹³

The illustrative regimes in table 6.3 show how high reported confidence can coexist with low task success under shift. The single-row differences are confidence–success gaps, not ECE; computing ECE requires bins over a specified evaluation set. Raw scores cannot grant execution permission without validation and independent physical checks.

Table 6.3: Illustrative Confidence–Success gaps under distribution shift: Hypothetical confidence and task-success rates for four operating regimes. Each gap is the row confidence minus success rate, not binned ECE. Supervisory responses require independent evidence and a feasible fallback.

Distribution Shift Regime	Physical & Environmental Trigger	Assumed Conf. vs Success (conf vs success)	Confidence–Success Gap (conf – acc)	Downstream Supervisory Action
In-Distribution (Nominal Baseline)	Clean optics, structured laboratory lighting, nominal workpiece textures	conf = 0.91, acc = 0.89	+0.02	Apply independent limits before nominal execution
Near-OOD Shift (Covariate Shift)	Specular reflections, lens water droplets, dynamic shadow transitions	conf = 0.89, acc = 0.52	+0.37	Selective abstention triggered: clamp velocity, initiate exploratory tactile probe
Far-OOD Shift (Semantic Shift)	Uncataloged deformable obstacles, human limb intrusion into workspace	conf = 0.78, acc = 0.08	+0.70	Reject proposal; use feasible controlled response
Sensor Degradation (Hardware Drift)	Photodetector thermal noise (> 75°C), focal blur from vibration	conf = 0.84, acc = 0.31	+0.53	Trigger operational downgrade; restrict kinematics to certified safety envelope

Perceptual chattering and covariate shift

Ambiguous images can make successive object classifications change rapidly. If a motion planner treats each new label as a fresh physical state, it may discard useful motion history. In physical manipulation, perceptual chattering injects high-frequency direction reversals into trajectory planners, causing torque oscillations that damage gearboxes when mechanical actuator inertia cannot track rapid software state transitions.¹⁴

The Tempe collision analysis (section 2.7) shows the danger of acting on an incorrect path prediction without an effective emergency response. Temporal smoothing can help motion continuity, but it cannot repair a false obstacle estimate; independent detection and a feasible response remain necessary.

Temporal expiration of intent

Physical reality is dynamic, making a cognitive thought valid only for a finite duration (τ_{valid}). If a high-level vision-language model requires 300 ms to deliberate, the physical state of moving obstacles, robotic arms, or chemical reactor valves has already evolved by the time the plan is ready. Delayed downstream communication or execution leads to acting on an expired intent lease, commanding the physical body based on an obsolete mental representation.



War Story 6.1: DARPA robotics falls and balance timing (2015)

Context: The 2015 DARPA Robotics Challenge Finals tested humanoid disaster-response tasks under constrained communication (Krotkov et al. 2017).

¹³ | **Expected Calibration Error Formulation:** Partition predicted confidence into bins. In each bin, compare observed success frequency with mean reported confidence; average the absolute gaps, weighted by the bin's share of samples. Elevated ECE shows that reported confidence is a poor basis for granting physical permission. The equation and bin definitions appear in section A.2.3.

¹⁴ | **Perceptual Chattering in Fatal Collisions:** The NTSB investigation (HAR-19/03) found that the ADS changed its classification among vehicle, bicycle, and other. After changes, its path predictor did not use earlier object locations; it failed to predict the pedestrian's path and never initiated emergency braking (National Transportation Safety Board 2019).

Observation: Several robots fell or needed intervention during the event. Published team accounts describe varied causes; the competition record does not establish planner stalls as a common cause or a universal controller rate.

Systems lesson: A fall can develop on the scale of a few hundred milliseconds for a standing biped. High-level planning cannot be the sole balance response; a local controller needs current state, sufficient torque and contact authority, and a feasible recovery action when a proposal stalls.

To limit the effect of stale plans during balance disturbances, the architecture separates proposal generation from local execution permission.¹⁵

The architectural resolution: The two-speed brain

As established in 6.1, resolving this fundamental latency-stability paradox requires divorcing motion proposal from execution authority. This architectural decoupling organizes the machine into a **two-speed brain architecture** (figure 6.12), conceptually descended from early subsumption architectures (Brooks 1986a). At the top tier, high-capacity Vision-Language-Action foundation models perform slow deliberative cognition (System 2) at 1–5 Hz, parsing open-world multimodal streams to interpret semantic commands and emit structured intent leases. At the intermediate tier, a trajectory decoder (System 1.5) turns intent into candidate motion chunks at a rate set by complete inference latency and the replanning window; a separate waypoint stream can run faster. At the base reflex tier, hard real-time safety supervisors (System 1) execute deterministically at 1000 Hz on isolated safety silicon,¹⁶ evaluating candidate proposals against physical kinematic invariants (such as maximum safe joint velocities) and dynamic stopping clearances (checking whether modeled braking distance d_{stop} stays within measured clearance D_{clear} while the state remains feasible). If the measured state and proposal pass those checks, the supervisor grants execution permission (u_t); otherwise it rejects the proposal ($\perp$) and requests the defined feasible fallback.

Treating the learned brain as an unprivileged proposal generator allows an independent gate to reject detected violations. Protection depends on valid sensing, calibrated models, timely checks, and a feasible fallback; a gate cannot detect every semantic error from the same failed evidence. When the System 1 supervisor rejects a candidate proposal or executes a safe holding reflex, the architecture engages a closed-loop **operational recovery loop**: the real-time enforcer immediately asserts an asynchronous notification across the shared IPC mailbox, causing the System 2 deliberative model to flush its pending trajectory buffer, refresh its spatial belief map with the current stationary plant state, and re-prompt the policy with the specific refusal reason (e.g., clearance obstacle detected or kinematic boundary reached), supporting recovery if the notification arrives and the refreshed plan passes the gate; repeated rejection can still require a supervised hold.

The complete operational loop spanning intent reasoning, action chunk synthesis, and deterministic safety arbitration is formalized in algorithm 6.1.

¹⁵ | **DARPA Robotics Challenge Architectural Lessons:** Competition results (Krotkov et al. 2017) illustrate the cost of lost balance; the independent local-response lesson is an engineering inference, not a documented single cause of the falls.

¹⁶ | **Hardware Safety Silicon Interlocks:** Independent hardware interlocks can remove motor torque when monitored current, speed, or travel limits are violated. Their response time and safe behavior require design-specific verification; removing torque alone does not guarantee a moving body has stopped.

Downstream: Coordinate transform trees and formal spatial state schemas are fully derived in section 18.10.

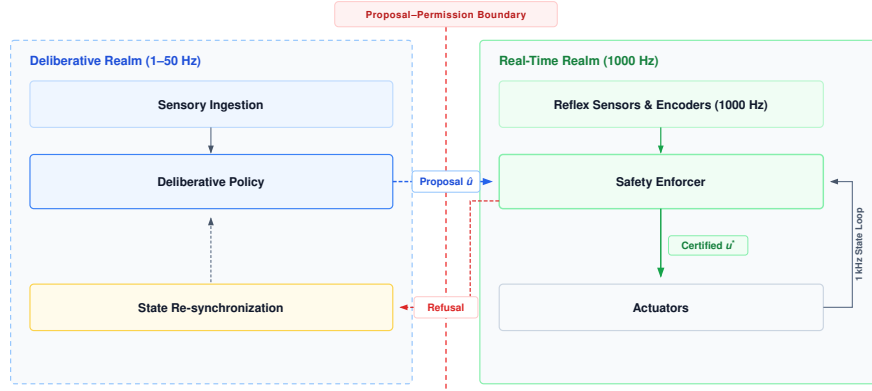


Figure 6.12: Two-speed brain architecture: An untrusted deliberative realm running at 1–50 Hz and a trusted real-time realm running at 1000 Hz, separated by the proposal-permission privilege boundary. The independent safety supervisor checks sensed state and modeled limits on each control cycle before granting execution permission; protection depends on valid sensing and feasible fallback.

Algorithm 6.1 Two-Speed Brain Deliberation and Safety Enforcement

Require: multi-view visual frames $\mathbf{I}_t$, proprioceptive state $(\mathbf{q}_t, \dot{\mathbf{q}}_t)$, mission prompt M , spatial corridor Box_{safe} , stopping distance model $d_{\text{stop}}(v)$, clearance threshold D_{clear} , observation-age limit $\tau_{\text{obs}, \text{max}}$

Ensure: admitted kinematic setpoint $\mathbf{q}_{\text{cmd}, t}$ dispatched to low-level servo, or a feasible risk-reducing response requested

- 1: **System 2: Intent Deliberation (1–5 Hz on Host MPU)**
 - 2: **if** active_lease = NULL **or** $t_{\text{now}} \geq \text{active_lease}.t_{\text{expire}}$ **then**
 - 3: Tokenize visual observation $\mathbf{I}_t$ and query 3D world memory
 - 4: Evaluate Vision-Language-Action backbone to decompose task prompt M into next subgoal
 - 5: Construct Intent Lease $\mathcal{L}_{\text{intent}}$ with independently checked corridor Box_{safe} and expiration $t_{\text{expire}} \leftarrow t_{\text{now}} + \tau_{\text{valid}}$
 - 6: Dispatch $\mathcal{L}_{\text{intent}}$ to System 1.5 trajectory planner (authenticate if threat model requires)
 - 7: **end if**
 - 8: **System 1.5: Action Chunk Trajectory Synthesis (rate bounded by full inference latency)**
 - 9: Ingest joint states $(\mathbf{q}_t, \dot{\mathbf{q}}_t)$ and active unexpired $\mathcal{L}_{\text{intent}}$
 - 10: Evaluate policy backbone to synthesize H -step chunk $\mathbf{A}_{t:t+H-1}$ tagged with parent lease ID and source-observation time
 - 11: Apply temporal ensembling: $\mathbf{a}_t \leftarrow \frac{\sum_{i=0}^{\min(t, H-1)} w_i \mathbf{a}_{t|t-i}}{\sum w_i}$ with $w_i = \exp(-m \cdot i)$
 - 12: Fit $\mathcal{C}^2$ quintic splines through waypoints into $\mathbf{A}_{\text{chunk}}$
 - 13: **System 1: Hard Real-Time Reflex Gate (example: 1000 Hz on MCU)**
 - 14: Sample calibrated joint encoders and hardware distance sensors
 - 15: Fetch candidate setpoint $\mathbf{a}_t$ from active $\mathbf{A}_{\text{chunk}}$
 - 16: **if** $\mathbf{A}_{\text{chunk}}.\text{parent_lease_id} \neq \mathcal{L}_{\text{intent}}.\text{lease_id}$ **or** $t_{\text{now}} - \mathbf{A}_{\text{chunk}}.t_{\text{generation_ns}} > \tau_{\text{obs}, \text{max}}$ **then**
 - 17: **return** FALLBACKRESPONSE(CHUNK_STALE) $\triangleright$ wrong lease or old observation
 - 18: **else if** $t_{\text{now}} > \mathcal{L}_{\text{intent}}.t_{\text{expire}}$ **then**
 - 19: **return** FALLBACKRESPONSE(LEASE_EXPIRED) $\triangleright$ lease underflow fallback
 - 20: **else if** $\mathbf{a}_t \notin \text{Box}_{\text{safe}}$ **or** $\|\dot{\mathbf{q}}\| > \dot{q}_{\text{max}}$ **then**
 - 21: **return** FALLBACKRESPONSE(KINEMATIC_LIMIT) $\triangleright$ boundary violation
 - 22: **else if** $d_{\text{stop}}(v) > D_{\text{clear}}$ **then**
 - 23: **return** FALLBACKRESPONSE(COLLISION_IMMINENT) $\triangleright$ clearance breach
 - 24: **else**
 - 25: $\mathbf{q}_{\text{cmd}, t} \leftarrow \mathbf{a}_t$ $\triangleright$ permission granted: dispatch to closed-loop servo
 - 26: $\mathbf{u}_t \leftarrow \text{CLOSEDLOOPCURRENTCONTROL}(\mathbf{q}_{\text{cmd}, t}, \mathbf{q}_t, \dot{\mathbf{q}}_t) \triangleright$ 10 kHz FOC generates PWM
 - 27: **return** $\mathbf{u}_t$
 - 28: **end if**
-

The foundational trade-offs between these paradigms are synthesized in table 6.4.

Table 6.4: Master comparison: classical analytical methods vs. learned foundation models in Physical AI: Architectural trade-offs across generalizability, sensory robustness, verifiability, pacing, and fallback modes, highlighting the physical AI synthesis that partitions proposal capability from execution authority.

Systems Dimension	Classical Analytical Methods	Learned Foundation Models	Physical AI Architectural Synthesis
Foundational Principle	Analytical differential equations, geometry, CAD models, Lyapunov stability	High-capacity neural representations fitted to empirical multimodal data	Hybrid Hierarchy: Learned intent proposals gated by analytical safety shields
Generalizability & Perception	Model-dependent: Explicit geometry and features may lose coverage under shift	Data-dependent: Open-vocabulary grounding can improve coverage but may fail under shift	Unprivileged Deliberation: Foundation models synthesize continuous semantic goals
Mathematical Verifiability	Conditional: Selected properties can be proved within model assumptions	Limited: Empirical behavior needs separate bounds and runtime evidence	Proposal-Permission Boundary: Gate checks modeled limits using independent evidence
Temporal Pacing	Hard Real-Time: Deterministic microsecond execution loops (1 kHz)	Multi-Rate: Full inference, chunk refresh, and waypoint execution have distinct rates	Two-Speed Brain: Action chunks amortize slow deliberative foundation models
Failure Modes & Fallback	Boolean branch crashes; failure on uncataloged objects; rule exhaustion	Semantic hallucinations; out-of-distribution overconfidence; cognitive drift	Conditional Fallback: Reject detected violations and execute a feasible response

Systems Perspective 6.3: Deliberative latency and tipping

The proposal-permission decoupling principle is nowhere more brutally tested than on a bipedal humanoid robot:

- **The Inverted-Pendulum Time Constant:** An upright bipedal humanoid is an inherently unstable, underactuated dynamical system. Approximating its center-of-mass (CoM) dynamics as a linear inverted pendulum of height $z_c \approx 0.9$ m, the characteristic tipping time constant is $\tau_{\text{fall}} = \sqrt{z_c/g} \approx \sqrt{0.9/9.81} \approx 303$ ms. If an unexpected foot slippage or external shove occurs, the center of pressure leaves the support polygon, and the robot begins accelerating toward the ground under gravity. This 303 ms characteristic time is a scale for rapid response, not a universal recovery deadline; the available step or ankle-hip response depends on contact state, torque, clearance, and sensing delay.
- **The Cognitive Latency Mismatch:** Meanwhile, high-capacity vision-language-action (VLA) foundation models require 200–800 ms to tokenize multi-camera images, run transformer autoregression, and emit a high-level task plan. If the robot's physical balance loop were synchronously coupled to cognitive inference, a single inference delay or GPU thermal throttle would freeze actuator setpoints, causing the robot to crash to the floor.
- **Architectural Decoupling as an Invariant:** Consequently, the cognitive brain tier (System 2) must never write directly to joint motors. It outputs coarse semantic waypoints, footstep affordance regions, or 20 Hz action chunks into a shared lock-free ring buffer. Below this boundary, a dedicated 1 kHz whole-body reflex controller (System 1) continuously stabilizes floating-base centroidal momentum. If the cognitive brain stalls for 500 ms, the fast controller can reject stale stepping proposals and attempt a validated balance or risk-reducing response. Stable double support is possible only when contact geometry and remaining actuator authority permit it.



Checkpoint 6.5: Cognitive vulnerabilities, calibration collapse, and two-speed recovery

Before synthesizing the cognitive brain into the broader physical stack, verify your understanding of runtime failure modes and multi-speed arbitration:

- ☐ **The Softmax illusion and calibration collapse:** Can you explain why a single confidence-success gap is not ECE, and why abstention thresholds must be chosen from operational evidence rather than an unsupported universal cutoff?
- ☐ **Inverted pendulum falling time constant:** Can you calculate why an upright bipedal humanoid with center-of-mass height $z_c = 0.9$ m has a characteristic falling time constant $\tau_{\text{fall}} \approx 303$ ms, and why deliberative VLA latencies (200–800 ms) must never be synchronously coupled to whole-body balance loops?
- ☐ **Two-speed brain operational recovery:** Can you trace the closed-loop recovery sequence when a 1000 Hz System 1 reflex supervisor rejects an expired or out-of-corridor intent lease, and how asynchronous IPC mailboxes prevent system deadlocks?

6.6 Fallacies and Pitfalls

Learned capability is useful only when the machine can interpret and constrain its proposals. The following mistakes confuse a model's success, confidence, or output format with permission to act.

Fallacy: *An engineering team must choose entirely between formal classical safety proofs and learned empirical capability.*

An assembly plant compares a classical wiring-harness controller with a learned diffusion policy. The classical design exposes explicit motion constraints but struggles with cable deformation; the learned policy completes 96 percent of trial insertions without establishing worst-case force or velocity bounds. These strengths address different engineering obligations. The learned policy can propose adaptive motions while a deterministic supervisor checks them against the physical limits, using a control barrier function where its assumptions apply (Ames et al. 2019) (detailed in chapter 24). Partitioning capability from enforcement lets the team evaluate task performance and the protection mechanism separately. High empirical success does not establish the supervisor's adequacy or the safety of the integrated machine.

Pitfall: *Allowing a neural model's high confidence to bypass deterministic geometric boundaries.*

A cleanroom manipulator mistakes glare on a transparent barrier for open space and proposes reaching through it while carrying reagent vials. The proposal contains finite, in-range coordinates and arrives within its 40.0 ms deadline. Those checks establish that the packet is usable, not that its interpretation of the scene is correct. High confidence cannot resolve the same perceptual error that produced the trajectory. Before granting permission, the enforcer must compare the proposal with independently maintained geometric boundaries (such as hard-coded stay-out zones or real-time depth maps). A structurally valid packet—adhering to the required format and protocol—can still request a motion obstructed by real-world constraints. Therefore, it is essential that confidence in the packet's validity does not allow it to bypass checks against these physical limitations.

Fallacy: *Raw softmax probabilities or predicted confidence scores reflect true physical success rates.*

A bin-picking policy reports $\hat{p} \approx 0.90$ for a group of grasps, but only 64 of 100 physical attempts succeed. An abstention threshold based on the reported score would therefore accept more failures than its numerical value suggests. The discrepancy concerns calibration: confidence does not yet match observed success in the operating conditions (as quantified in table 6.3). It does not, by itself, establish whether the policy ranks candidates correctly. Engineers must validate the relationship between scores and outcomes across the intended domain before using confidence to choose between execution, another observation, and operator assistance. Independent physical limits still apply to accepted grasps.

Pitfall: *Coupling high-frequency semantic classifications directly to motor setpoints without continuous filtering.*

A delivery robot observes reflective tarps over traffic cones. Every 50.0 ms, its perception pipeline changes the classification among obstacle, movable drape, and free path. If each change immediately resets motor setpoints, uncertainty in the scene becomes repeated requests for direction reversal. The drive motors cannot follow those requests beyond their torque slew limits, and the chassis experiences a violent, chattering shudder. The architecture needs continuity between changing interpretations and physical commands. Filtering or action chunking can support that continuity, but the resulting trajectory must still respect the body's motion limits when the semantic estimate changes.

Fallacy: *Numerically valid and smooth intent proposals can safely be executed after their lease expires.*

A reactor policy proposes increasing monomer feed under a $\tau_{\text{lease}} = 500$ ms lease. Host scheduling delays delivery until 580 ms after generation, while a thermal excursion has brought the reactor near its temperature limit. The feed curve remains smooth and numerically valid, but the conditions that justified it no longer hold. Executing it could add heat beyond the cooling system's capacity. The fieldbus gateway must reject the expired lease and leave control with the validated local response (enforcing the `t_expire_ns` contract invariant from table 6.1). Smoothness describes the proposed command; temporal validity determines whether the machine may still act on it.

6.7 Summary

Rooted in the definition of Physical AI (section 2.1), this hybrid hierarchy dictates that high-capacity neural representations act as unprivileged proposal engines parsing open-world multimodal complexity, while hard real-time classical governors evaluate dynamic and kinematic invariants (e.g., momentum limits and collision boundaries) with exclusive authority over physical actuators.



Key Takeaways: The cognitive brain and deliberation limits

- **Five Design Goals:** Embodied cognition seeks semantic generalization, tested sensory robustness, metric spatial-temporal grounding, multi-rate cadence decoupling, and bounded execution authority.
- **Why We Learn (The Four Pillars):** Learned models can improve coverage of unfamiliar tasks, sensory variation, contact-rich dynamics, and larger datasets when evidence supports transfer to the operating region.
- **The Five Stages of Cognition:** Information flows sequentially through Ingestion → Perception → Memory → Intent → Planning, transforming continuous sensory fluxes into grounded geometric trajectories before execution.
- **Distinct Cognitive and Control Cadences:** In the chapter's illustrative setup, a multi-billion-parameter model deliberates at 1–5 Hz while action chunking supplies dense waypoints from each slower backbone evaluation. Fresh chunk rate depends on complete inference latency and the chosen replanning window.
- **The Proposal-Permission Principle:** The learned brain operates strictly as an unprivileged proposal engine (p_t), possessing capability to suggest actions, but zero authority to command actuators without deterministic validation.
- **Softmax is Not Probability:** Exponentiated logits sum to one by mathematical definition, not physical truth; models exhibit severe overconfidence under out-of-distribution shift, requiring empirical calibration (ECE) and selective abstention.

A neural foundation model is an inductive engine of extraordinary expressive power but remains oblivious to the physical laws governing its machine. When model weights generate a smooth trajectory that demands impossible motor acceleration or violates contact friction limits, the architecture must rely on deterministic physical governors rather than relying on gradient-descent convergence to prevent structural failure.



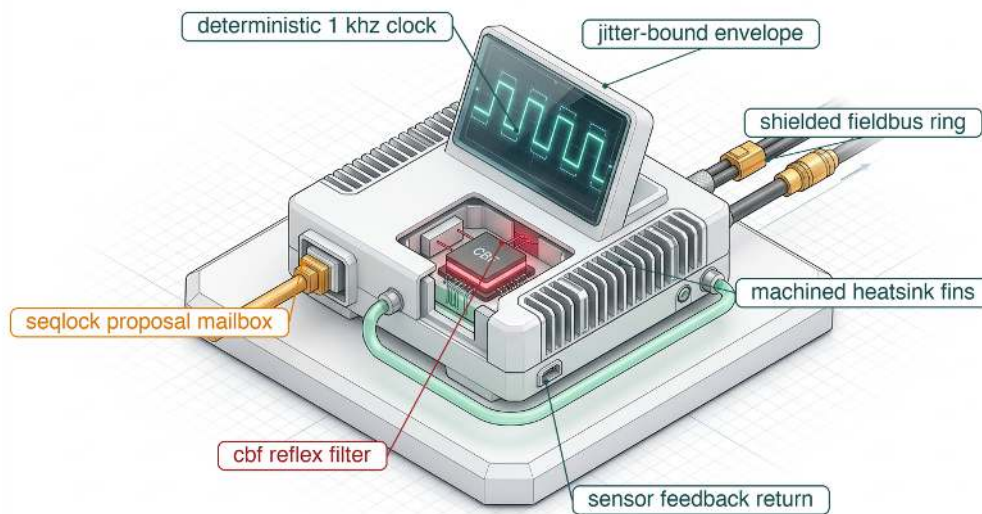
What's Next: From cognitive deliberation to deterministic nervous systems

The brain can propose a mug grasp from images and task context, but its estimate may be stale or geometrically wrong when the arm moves. Chapter 8 follows that proposal into the timed execution path: buses carry it, state buffers preserve freshness, and independent monitors decide whether the measured plant and modeled limits permit the next setpoint. A rejected proposal requires a feasible local response before another cognitive update arrives.

The Nervous System

DRAFT

The Nervous System



- 8.1 The Multi-Rate Bridge
- 8.2 Silicon Partitioning
- 8.3 Multi-Rate Cadences
- 8.4 Deterministic Fieldbuses
- 8.5 Actuation Authority
- 8.6 Hardware Isolation
- 8.7 Invariant Checking
- 8.8 Supervisory Teleoperation
- 8.9 The Handoffs Matrix
- 8.10 Fallacies and Pitfalls
- 8.11 Summary

Purpose

What forces an independent real-time nervous system to stand between deliberative neural policies and physical actuator power?

A moving machine's momentum cannot pause for neural policy deliberation. Connecting high-capacity models to motor inverters exposes them to operating system jitter, garbage-collection pauses, and saturated memory buses that starve current regulation. Moving linkages coast under unmanaged kinetic energy, winding currents collapse, and dynamic balance fails before software can raise an exception. Software priorities cannot preempt hardware memory transactions or mechanical inertia. The nervous system decouples deliberation from microsecond actuator timing, verifying data freshness and maintaining exclusive authority over motor power stages. An independent, cycle-deterministic gatekeeper enforces physical limits, preventing delayed or erroneous policies from causing collisions. The nervous system translates physical budgets into timing and authority constraints, dictating when the brain's proposals become motion.

Brain
1–50 Hz
foundation models, SLAM, planning

above: propose; below: command

Nervous System
1000 Hz
TSN, intent leases, lock-free IPC

Body
continuous
real-time MCU, safety filters, CBFs

Boundary
continuous
sensors, actuators, kinematics

Physical World
continuous
contact mechanics, friction, mass

Prerequisite: Silicon domain isolation physically enforces the causal boundary defined in section 2.2.



Learning Objectives

- Determine control update rates from physical dynamics and allowable response delay
- Evaluate data exchanges for freshness and bounded transfer time between reasoning and control
- Distinguish measured latency percentiles from defensible worst-case bounds on real-time execution
- Evaluate hardware isolation that reserves actuator authority when learned policies or communication fail
- Size watchdog validity windows from stopping requirements and the available physical margin
- Diagnose timing, bandwidth, power, and fault propagation across the machine's system interfaces

8.1 The Multi-Rate Bridge

A moving robot must continuously regulate electrical currents in its motor windings at thousands of cycles per second, even as its high-capacity learned policy spends tens or hundreds of milliseconds interpreting the next video frame. Deliberative reasoning and motor regulation sharing a blocking execution path or unpartitioned memory bus starves physical control during any neural network delay. The physics of moving mass and electromagnetic coils create a strict architectural requirement: *the cognitive brain needs time to deliberate, while the physical body needs continuous control to survive.*

In standard computer systems, workloads share processors, caches, and memory interconnects under general-purpose operating systems. Real-time extensions (such as PREEMPT_RT) allow threads to be prioritized, but they cannot eliminate microarchitectural contention. When a neural network streams multi-gigabyte weight tensors across a shared memory bus, it saturates direct memory access crossbars and evicts cache lines belonging to real-time control threads.¹ A high-priority software interrupt can preempt a thread, but it cannot preempt a hardware memory transaction already occupying the memory bus. Unlike software tasks, these hardware transactions rely on bus arbitration and must complete to maintain system stability. If a garbage-collection pause, page fault, or thermal throttle stalls the neural engine, an unpartitioned system stops sending setpoints to the motor inverters. Moving linkages coast under unmanaged momentum, winding currents collapse, and dynamic balance fails.

A high-capacity foundation model or diffusion policy requires tens to hundreds of milliseconds to ingest multi-camera visual streams and synthesize future trajectories. An electromagnetic motor coil has an electrical time constant ($\tau_e = L/R$) measured in fractions of a millisecond, while moving linkages carry kinetic energy ($E_k = \frac{1}{2}mv^2$). Delaying or miscalculating the current command can therefore exhaust a physical margin before deliberation finishes. This mismatch takes different forms across the three physical machine archetypes:

- In *Mobile Systems* (Class 1), an autonomous chassis or quadruped traveling at 2.0 m/s covers 0.30 m of unguided displacement during a 150 ms inference stall before wheel braking can engage.
- In *Manipulators* (Class 2), an articulated robot arm swinging an end-effector carries reflected rotor inertia ($N^2 J$) that will crush tooling or breach fixturing envelopes if high-rate impedance loops are interrupted.
- In *Process and Energy Systems* (Class 3), high-power battery inverters and thermal cooling circuits experience thermal runaway or power stage shoot-through (a catastrophic short circuit when opposing power switches turn on simultaneously) if gate switching vectors drift outside microsecond commutation deadlines.

Physical safety cannot be guaranteed by software discipline, OS thread priorities, or runtime exception handlers alone. It requires an explicit, deterministic computing, memory, and communication fabric: the **real-time nervous system**.

¹ | Interconnect Crossbar

Contention: When an accelerator streams multi-gigabyte foundation model weights across shared AXI or PCIe crossbars, dense burst transactions saturate dynamic RAM banks and arbitration queues. Co-located real-time processor cores attempting to read motor encoder registers face unbudgeted hardware wait states that software priorities cannot preempt. Maintaining microsecond-scale control stability therefore requires dedicated physical memory channels or isolated on-chip SRAM for the real-time domain.

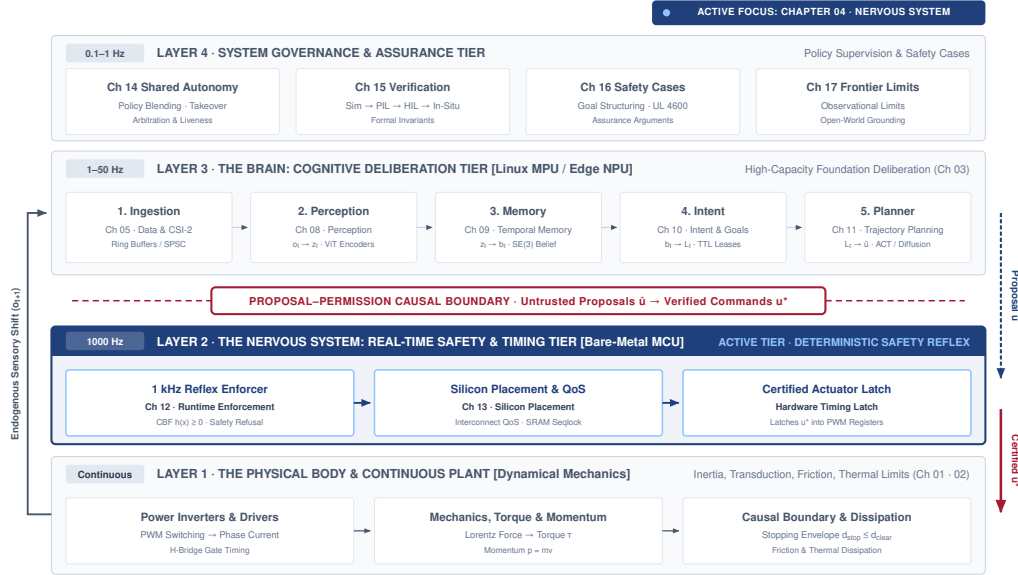


Figure 8.1: The real-time nervous system: The four-tier physical AI systems architecture highlights the real-time nervous system (Layer 2). Operating between the deliberative brain (Layer 3) and dynamical body (Layer 1), the deterministic nervous system enforces multi-rate synchronization, transports sensorimotor signals under hard real-time deadlines, and maintains exclusive execution authority over actuator power.

Within the four-tier physical AI architecture (figure 8.1), this chapter centers on Layer 2: the Real-Time Nervous System. Bridging the Cognitive Brain (Layer 3) and the Physical Body (Layer 1), the nervous system physically embodies the Proposal-Permission Privilege Boundary. It enforces an asymmetric contract: high-capacity learned models operate strictly as unprivileged clients proposing trajectory waypoints, while dedicated, cycle-deterministic silicon retains exclusive authority to evaluate safety invariants and commit current setpoints to motor inverter registers.

Definition 8.1: The real-time nervous system

Real-time nervous system is the deterministic computing, memory, and communication substrate (Layer 2) that bridges stochastic, high-latency neural deliberation (Layer 3) with continuous physical plant dynamics (Layer 1), synchronizing multi-rate execution cadences and enforcing hardware-level safety invariants.

1. **Significance:** Without an independent nervous system, high-capacity neural models must hold direct write access to actuator registers. A single operating-system scheduling hiccup, garbage-collection stall, or model hallucination translates into unguided physical momentum, motor over-current, or mechanical collision. The nervous system decouples deliberation latency from physical survival deadlines.
2. **Distinction:** Unlike a standard robotic middleware or operating system scheduler that provides best-effort messaging, the Physical AI nervous system enforces strict worst-case execution time (WCET) bounds, static memory allocation without dynamic heaps, and asymmetric hardware authority over power stages.
3. **Common pitfall:** Assuming that running a neural policy on a real-time Linux kernel (such as PREEMPT_RT) eliminates the need for an independent microcontroller nervous system. General-purpose OS kernels share memory buses and power rails with host accelerators, leaving execution vulnerable to memory crossbar saturation and uncorrectable hardware hangs.

Every signal traversing the boundary between deliberation and physical actuation is constrained by five physical limits: transport and computation latency (τ), interconnect bandwidth (β), thermal dissipation ($\mathcal{E}$), temporal jitter (σ_t), and fault domain isolation ($\mathcal{F}$). Reconciling these limits requires an architecture that delivers microsecond cycle determinism (1000 Hz to 20 kHz) to preserve controller phase margin, maintains lock-free asynchronous decoupling to prevent slow neural inference from blocking inner control loops, isolates actuator writes from host crashes, restricts actuation privilege to an independent enforcer, and requests a validated local response when a watchdog lease expires.²

The first decision is where the two workloads execute. The separation in section 8.2 keeps inference off the timed actuator path. The design in section 8.3 then budgets their exchange and response to stale proposals; section 8.4 budgets distributed transport. The remaining sections locate actuator authority, fault isolation, invariant checks, human overrides, and the evidence recorded at each handoff. Each mechanism must leave the physical plant a measured, feasible response when the brain pauses.

8.2 Silicon Partitioning

The proposal-permission architecture in chapter 2 divides the computational workload of a physical AI machine into two opposing regimes. The first regime is deliberation, where learned models evaluate sensory context, infer environmental state, and propose future action plans. The second regime is reactive enforcement, where deterministic control logic samples joints, validates kinematic limits, interpolates motion commands, and regulates electrical currents to the motors. These two regimes differ in their fundamental mathematical profiles, hardware architectures, and tolerance for temporal error.

Deliberate work features deep computational graphs, large memory footprints, and variable execution times. A vision-language-action model processing multi-camera video streams, tactile histories, and natural language prompts requires billions of multiply-accumulate operations across hundreds of neural network layers. Storing the parameters and intermediate activation tensors demands gigabytes of high-bandwidth memory. To achieve acceptable arithmetic throughput, deliberate computation executes on massively parallel accelerators such as graphics processing units (GPUs) or dedicated neural processing units (NPUs). These processors maximize aggregate throughput across wide vector registers and batched memory pipelines at the direct expense of deterministic latency. Memory access stalls, dynamic tensor kernel selection, bus transfers, and runtime memory management scatter inference execution times across a wide distribution. If a deliberate model experiences a temporary delay, the failure mode is soft: the planning pipeline produces a delayed proposal or a stale trajectory that can be held or discarded while the lower-level enforcer maintains stable equilibrium.

Reactive work operates under the inverse constraints. An inner control loop computing inverse dynamics, joint limit monitoring, or field-oriented motor control requires a few thousand arithmetic operations per update. Its memory footprint is static, measured in kilobytes rather than gigabytes, with every variable allocated at compile time in tightly-coupled on-chip static memory. This ensures the exact physical memory location of every piece of data is fixed permanently, eliminating the unpredictable delays caused by heap fragmentation, dynamic paging, and runtime garbage collection. Reactive routines execute on cycle-counted microcontrollers or field-programmable gate arrays (FPGAs) designed for bounded execution rather than maximum parallel throughput. The deadlines governing this work are hard real-time constraints dictated by the mechanical inertia and electrical inductance of the physical system, typically running at rates between 1000 Hz and 10000 Hz on update budgets of 100 μ s to 1000 μ s. If a reactive loop misses its execution deadline, the failure mode is hard: lost current regulation induces severe torque ripple, actuator saturation, or mechanical collision.

Learned inference runtimes depend on memory access patterns, cache residency, dynamic tensor shapes, and hardware prefetching, making execution times stochastic. An inference pipeline can meet its average deadline while still missing individual deadlines in its latency tail.³ For example, an accelerator with a median runtime of 50 ms might easily exhibit a P_{99} tail extending to 113 ms. For an explicitly assumed exceedance probability of 2.38% at a 100 ms deadline, 10 requests per second yield about $0.0238 \times 10 \times 3600 = 857$ misses per hour. The median and P_{99} alone do not

² | **Watchdog Lease:** A watchdog lease bounds how long a previously validated proposal may remain active. At expiry, the independent enforcer revokes proposal authority and requests its validated local response; a stop within clearance requires a feasible brake and reserved detection and actuation time. The lease is only one term in the stopping-distance budget.

³ | **Inference Latency Distribution:** Log-normal and shifted Weibull models are possible descriptions of measured inference tails, but this example supplies no fitted distribution. A 2.38% exceedance is an illustrative input to the hourly-count conversion, not a value derived from the stated median and P_{99} . For the formal derivation of latency exceedance probabilities in real-time inference, see Appendix A.

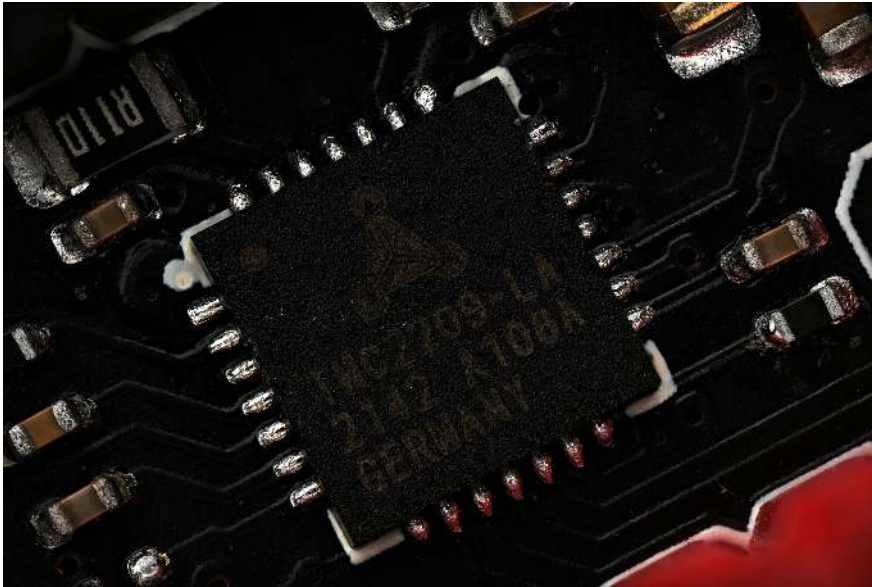


Figure 8.2: Motor drive controller silicon: Macro view of an embedded motor drive integrated circuit featuring precision current-sense shunt resistors (R110), multi-layer power routing, and direct gate-drive outputs. Dedicated real-time control silicon executes cycle-deterministic current commutation loops without operating system schedulers, cache hierarchies, or bus contention. (Credit: Wikimedia Commons, CC BY-SA 4.0).

determine that probability. These misses create intervals where the motor actuators receive no updated commands unless an independent, deterministic process bridges the gap.

General-purpose operating systems and real-time kernel patches (PREEMPT_RT) cannot reconcile these two execution profiles on a shared processor. Although real-time extensions allow static task priorities and preemptible kernels, they cannot eliminate shared microarchitectural resources. When a thread executing a learned model streams gigabytes of weight matrices across the memory bus, it evicts cache lines belonging to high-priority control tasks. If the deliberate runtime triggers a **translation lookaside buffer (TLB) shutdown**,⁴ enters a driver-level spinlock, or saturates the memory bus crossbar with direct memory access (DMA) requests, the hardware pipeline stalls.⁵ A high-priority real-time interrupt can preempt software execution, but it cannot preempt an in-flight hardware burst transaction already occupying the shared memory crossbar. Furthermore, **Dynamic Voltage and Frequency Scaling (DVFS)**⁶ governors adjust core clock frequencies based on thermal dissipation and processor utilization, altering instruction cycle times dynamically. The resulting jitter ranges from tens of microseconds to several milliseconds, easily exceeding the entire time budget of a 1000 Hz control loop.

The hardware floor exposes this physical separation at the interface boundary. A host processor running an accelerator communicates across high-latency bus topologies such as PCIe or system fabric crossbars, where DMA transfers require ring-buffer synchronization, host interrupt service routines, and cache coherency handshakes that consume 10 μ s to 100 μ s per transaction. In contrast, an embedded microcontroller or dedicated real-time motor controller IC (figure 8.2) interfaces directly with sensor transceivers and power stages through memory-mapped registers, single-cycle input and output pins, and tightly-coupled static memory with zero wait states. Reading an absolute joint encoder over a serial peripheral interface (SPI) or updating a field-oriented pulse-width modulation counter takes less than 1 μ s of bounded execution time,⁷ completely isolated from system bus contention.

In heterogeneous Dual-Brain System-on-Chip (SoC) architectures, deliberate planning and reactive enforcement cannot safely share a processor core, a cache hierarchy, or a common memory bus. The slow, stochastic path requires parallel hardware (Linux host application processors and tensor accelerators) optimized for raw throughput across wide data streams, while the fast, deterministic

⁴ | **TLB Shutdown:** When an operating system modifies page tables, it broadcasts inter-processor interrupts (IPIs) to invalidate stale Translation Lookaside Buffer entries across all active cores. Receiving cores must suspend thread execution and flush affected entries, introducing unbudgeted multi-microsecond stalls. In shared-silicon architectures, these unpredictable synchronization pauses destroy the cycle determinism demanded by inner motor regulation loops.

⁵ | **DMA Cache Coherence:** Direct memory access transfers bypass CPU cache hierarchies, requiring explicit cache line invalidation or write-back operations to maintain memory consistency between peripherals and cores. Without hardware-enforced snoop filtering or explicit barriers, processors read stale sensor frames or overwrite pending actuator commands. These maintenance operations inject variable latency penalties into high-rate control routines unless static, unbuffered SRAM regions are dedicated to peripheral transfers.

6 | Dynamic Voltage and Frequency Scaling: Host application processors dynamically adjust operating frequency and core voltage to balance computational throughput against thermal limits. Because real-time control deadlines depend on static instruction execution times, dynamic clock scaling alters task duration at runtime. Unanticipated frequency throttling transforms previously verified control routines into deadline violations unless the real-time core operates on a fixed, independent clock domain.

7 | Dead-Time Delay: Dedicated pulse-width modulation timers (TI ePWM, STM32 TIM1) inject hardware-enforced dead-time delays (100–500 ns) between complementary high-side and low-side gate signals (TIMx_BDTR, DBRED). This delay ensures that the conducting power switch fully turns off before its opposing switch conducts, preventing shoot-through short circuits across high-voltage DC rails. Because dead time slightly distorts the applied phase voltage, motor control firmware incorporates online volt-second compensation to maintain smooth torque generation.

Prerequisite: Microsecond interrupt latency requirements build upon the sensor freshness bounds introduced in section 4.1.

8 | Zero-Order Hold Phase Lag: Holding a discrete control command constant across sampling interval T_s introduces an effective delay of $T_s/2$, producing a linear phase reduction $\Delta\phi = -\omega T_s/2$. In high-bandwidth robotic actuators, this phase erosion directly reduces open-loop phase margin, driving lightly damped mechanical modes into resonance. For the formal frequency-domain derivation of discrete sampling phase erosion, see Appendix B.

path requires isolated hardware (bare-metal microcontrollers or FPGA cores) optimized for worst-case execution time and cycle-counted predictability.

🔍 Systems Perspective 8.1: Silicon privilege partitioning

Never allow a high-capacity learned model to execute in the same fault domain, address space, or power rail as the actuator commutation loop. Deliberation is an unprivileged proposal generator; physical register privilege belongs exclusively to cycle-deterministic real-time silicon.

Safety in physical AI is an architectural partition in silicon, not an operating system scheduling priority. The high-throughput deliberative engine acts strictly as an untrusted client, while dedicated real-time silicon acts as the immutable hardware gatekeeper.

📋 Checkpoint 8.1: Silicon partitioning and execution predictability

Before examining multi-rate cadences, verify your understanding of hardware privilege separation:

- ☐ **Microarchitectural resource contention:** Can you explain why kernel-level priority patches (PREEMPT_RT) cannot prevent memory crossbar saturation and cache line eviction when an accelerator streams neural network weights?
- ☐ **TLB shutdowns and hardware wait states:** Can you describe how virtual memory management operations on a general-purpose processor introduce multi-millisecond tail latency into co-located software tasks?
- ☐ **Dual-brain fault isolation:** Can you distinguish the failure modes of an unprivileged deliberative application core from a cycle-deterministic motor microcontroller?

8.3 Multi-Rate Cadences

The separation of compute domains reflects a constraint where no single execution frequency satisfies both the body's physical dynamics and the computational complexity of perception and inference. A physical AI nervous system uses several coupled rate tiers (figure 8.3), each determined by a different physical process:

One representative platform uses a 20 kHz current loop, a 1000 Hz joint loop, 100–500 Hz motion tracking, 10–50 Hz perception and policy processing, and 0.5–2 Hz semantic updates. These are design examples; each deadline must be checked against its actuator, sensor, and task dynamics.

The inner loop rate is governed by the physical time constants of the actuator and the structural resonance of the machine. For an actuator with electrical time constant $\tau_e = L/R$, stator winding current responds to applied voltage over that interval. For an illustrative winding with $L/R = 0.5$ ms, choosing ten current samples per electrical time constant gives $T_s = 50 \mu\text{s}$, or 20 kHz. Separately, at an assumed joint-loop crossover of 20 Hz, a 1 ms sample-and-hold interval contributes $\Delta\phi = -(2\pi \cdot 20)(0.001)/2 \approx -3.6^\circ$ of phase lag. If the allowed sampling erosion is 5° , the interval must be below about 1.39 ms; 1000 Hz is one feasible choice before computation and transport delays are added. For the winding L/R derivation, see Appendix C. At the joint level, the mechanical time constant and structural stiffness dictate the required closed-loop bandwidth. Discrete sample-and-hold loops inherently introduce phase delay proportional to computation latency and sampling interval.⁸

Slow control loops erode the phase margin of the feedback controller. Accumulated phase lag causes corrective torques to arrive out of phase with physical link motion, converting negative feedback stabilization into positive feedback amplification. For the illustrative 20 Hz crossover, a 500 Hz loop adds about 7.2° of sample-and-hold lag, above the assumed 5° allowance. Whether a particular linkage resonates depends on its measured dynamics and total delay.

The perception tier executes at an intermediate rate dictated by photon collection physics, exposure windows, and target velocity relative to the sensor field of view. For an illustrative 300 lux scene, suppose exposure is set to 15 ms; the feasible integration window must be measured for the chosen

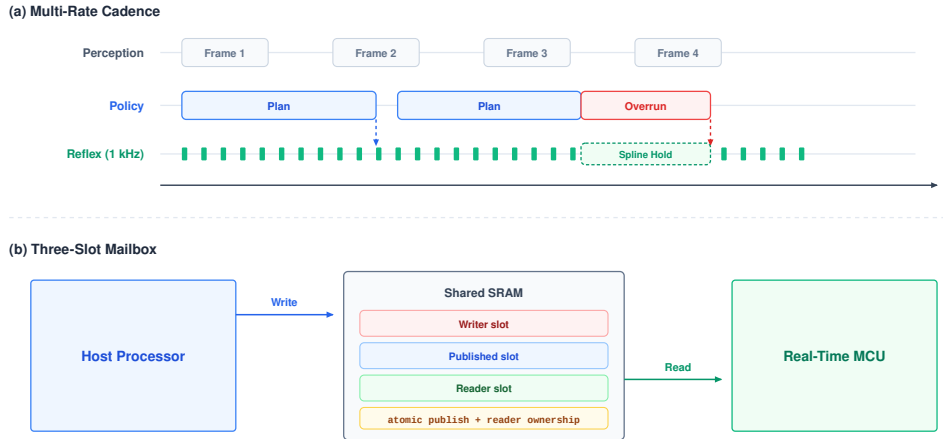


Figure 8.3: Multi-rate cadence and buffer exchange: (a) Representative perception, policy, and reflex rates; the reflex loop retains the last valid spline during a policy overrun only until its freshness limit. (b) A conceptual three-slot mailbox; the following algorithm specifies a simpler single-slot seqlock baseline. A production three-slot variant additionally needs atomic publication and reader-slot ownership (synthesized from Slotine & Li (1991) and ARM Architecture Reference Manual).

sensor, lens, gain, and acceptable noise. If a manipulator end-effector moves at 1.5 m/s, an exposure time of 15 ms allows $\Delta x = 1.5 \text{ m/s} \times 0.015 \text{ s} = 22.5 \text{ mm}$ of physical end-effector displacement during exposure; image-plane blur additionally depends on depth, optics, and camera motion. Exposure and frame-rate choices depend on illumination, optics, gain, and target speed. The displacement calculation gives a physical motion budget; a camera model and measured signal-to-noise ratio are needed to choose the image rate. Exposure signal-to-noise, projected inter-frame displacement, and processing time jointly bound a useful perception rate for the specified task.

The deliberate planning and policy tier runs between 10 Hz and 50 Hz based on the horizon of environmental change and the thermal limits of onboard compute. A vision-language-action policy consumes accelerator time and energy per inference, so its update rate must fit both latency and average power budgets. A scenario with relevant changes over 20–200 ms needs a policy, sensing, and local-response schedule matched to those changes. For one illustrative workload, a 20 Hz policy period and $K = 16\text{--}50$ future waypoints (Zhao et al. 2023) may bridge observation and actuation rates; actual inference latency and measured power determine whether this schedule fits.

Because these tiers execute on isolated hardware domains at different frequencies, data exchange across rate boundaries must be asynchronous, non-blocking, and zero-copy (figure 8.3). If a 1000 Hz control loop waits on a mutex lock held by a 20 Hz neural network runtime, a single scheduling delay in the operating system immediately causes the real-time controller to miss its deadline.

A baseline nonblocking exchange uses a **single-producer single-consumer (SPSC) seqlock mailbox** in shared static memory.

🔍 Systems Perspective 8.2: Lock-free cadence decoupling

A synchronization primitive for single-writer multi-reader shared memory that eliminates reader blocking and priority inversion by coupling an atomic monotonic sequence counter with memory barriers; odd counter values signal active writer mutation, while identical even counter values before and after the copy identify an unmodified payload under the stated memory-ordering protocol.

The algorithm below uses one shared payload slot and an atomic sequence counter. The three slots sketched in figure 8.3 represent a separate publication variant.

To prepare a new action chunk proposal, the untrusted application processor atomically increments the sequence counter to an odd value, signalling an active write. It then copies the payload—

⁹ | **Data Memory Barrier:** Out-of-order processor pipelines and store buffers frequently reorder memory operations to maximize execution throughput. In lock-free shared memory mailboxes between heterogeneous cores (such as a Linux Cortex-A host and a real-time Cortex-M microcontroller), producer cores must execute explicit outer-shareable data memory barriers (DMB OSHST) to ensure all payload words commit to shared static RAM across the interconnect before incrementing the sequence counter. Omitting these hardware barriers allows stale or partial trajectory coordinates to be observed by the consumer core, resulting in command corruption.

¹⁰ | **ARM Memory Model:** The ARM architecture implements a weakly-ordered memory model that permits aggressive hardware reordering of independent loads and stores. Lock-free inter-core communication relies on explicit acquire-release instructions (LDAR/STLR) alongside system-wide outer-shareable barriers (DMB OSH) to establish memory fences across distinct cache clusters. Correct barrier placement establishes the ordering required by this mailbox protocol across the specified memory domain without requiring costly bus-locking primitives.

¹¹ | **Cyclic Redundancy Check:** A cyclic redundancy check (such as IEEE 802.3 CRC-32) verifies data frame integrity against transmission noise and shared memory corruption. Microcontrollers evaluate the CRC polynomial using dedicated hardware accelerators within single-digit microsecond windows before parsing message payloads. Frames failing checksum verification are dropped immediately, prompting the receiver to extrapolate existing trajectory splines rather than consuming corrupted setpoints.

including a sequence identifier, timestamp, watchdog lease, trajectory waypoints, and CRC-32 checksum—into the shared slot. After enforcing an explicit outer-shareable memory barrier (DMB OSHST / DMB OSH) to ensure all payload bytes are globally visible across the heterogeneous SoC interconnect,⁹ the processor increments the sequence counter once more to an even value with store-release semantics (STLR),¹⁰ atomically publishing the completed frame.

On the receiving side, the deterministic real-time microcontroller latches the newest proposal without ever acquiring a lock or stalling execution. At the start of its control cycle, the microcontroller reads the sequence counter with load-acquire semantics (LDAR). While an ARM LDAR provides one-way acquire ordering (preventing subsequent memory reads from being hoisted above the initial sequence check), it does not prevent preceding payload reads from sinking after a trailing check. The protocol therefore pairs LDAR with explicit data memory barriers (DMB OSHLD) around the payload copy to enforce strict two-way load-load containment across heterogeneous cores. If the counter is odd, indicating that the producer is midway through updating the buffer, the consumer immediately aborts the read, retaining its existing validated trajectory spline.

While a standard seqlock operates cleanly when writers are non-preemptible kernel threads, in physical AI the writer is an untrusted user-space process running a heavy neural policy. If the operating system preempts the writer mid-write (or if an unhandled exception terminates the process), a raw sequence counter could remain stuck at an odd value. A preempted writer can leave this single-slot counter odd indefinitely, so the reader keeps its previously validated local spline only until its lease expires. A separate **three-slot publication variant** can keep a complete published slot available while the writer stages another; it must atomically publish the new slot and prevent the writer from reusing a slot still owned by a reader. The single-slot pseudocode below does not implement that ownership protocol.

If the sequence counter remained identical and even throughout the read, the latch is verified: the microcontroller validates the **cyclic redundancy checksum** (CRC-32),¹¹ checks timestamp freshness against its hardware timer, and accepts the proposal. The reader copies into scratch SRAM and promotes the proposal to its active spline only after sequence, checksum, and freshness checks pass. A rejected read leaves the active spline unchanged. The reader performs one bounded copy attempt and never waits for the writer. Its actual execution time must be measured or bounded on the target hardware; algorithm 8.1 shows the single-slot baseline.

Algorithm 8.1 Single-Slot Trajectory Seqlock Baseline

Require: proposal payload $P = \{\text{seq_id}, \text{timestamp_ns}, \text{lease_us}, \mathbf{w}_{1..K}, \text{crc32}\}$, scratch buffer B_{scratch} and previously validated active spline B_{active} in static SRAM, shared SRAM mailbox $(c_{\text{seq}}, \text{Slot}_{\text{shared}})$, current time t_{now}

Ensure: promoted validated chunk in B_{active} or rejection with B_{active} unchanged

```

1: function PUBLISHACTIONCHUNK( $P$ )
2:    $c_{\text{curr}} \leftarrow \text{ATOMICLOADEXPLICIT}(c_{\text{seq}}, \text{memory\_order\_relaxed})$ 
3:    $\text{ATOMICSTOREEXPLICIT}(c_{\text{seq}}, c_{\text{curr}} + 1, \text{memory\_order\_relaxed}) \triangleright \text{mark odd: write in progress}$ 
4:    $\text{HARDWAREMEMORYBARRIER}(\text{DMB\_OSHST}) \triangleright \text{outer-shareable store barrier pushes counter}$ 
5:    $\text{COPYMEMORY}(\text{Slot}_{\text{shared}}.\text{payload}, P, \text{sizeof}(P)) \triangleright \text{copy multi-word trajectory}$ 
6:    $\text{HARDWAREMEMORYBARRIER}(\text{DMB\_OSH}) \triangleright \text{outer-shareable full barrier: payload visible}$ 
7:    $\text{ATOMICSTOREEXPLICIT}(c_{\text{seq}}, c_{\text{curr}} + 2, \text{memory\_order\_release}) \triangleright \text{mark even: publish committed}$ 
8: end function

9: function LATCHLATESTACTIONCHUNK( $B_{\text{scratch}}, B_{\text{active}}, t_{\text{now}}$ )
10:   $c_{\text{start}} \leftarrow \text{ATOMICLOADEXPLICIT}(c_{\text{seq}}, \text{memory\_order\_acquire}) \triangleright \text{load initial counter (ARM LDAR acquire)}$ 
11:  if  $c_{\text{start}}$  is odd then
12:    return STATUS_WRITER_ACTIVE  $\triangleright$  writer active; retain prior spline
13:  end if
14:   $\text{HARDWAREMEMORYBARRIER}(\text{DMB\_OSHLd}) \triangleright \text{explicit barrier: prevent payload reads hoisting before counter}$ 
15:   $\text{COPYMEMORY}(B_{\text{scratch}}, \text{Slot}_{\text{shared}}.\text{payload}, \text{sizeof}(\text{Payload})) \triangleright \text{tentative copy; active spline unchanged}$ 
16:   $\text{HARDWAREMEMORYBARRIER}(\text{DMB\_OSHLd}) \triangleright \text{LDAR cannot prevent prior reads sinking; DMB forces payload reads before } c_{\text{end}}$ 
17:   $c_{\text{end}} \leftarrow \text{ATOMICLOADEXPLICIT}(c_{\text{seq}}, \text{memory\_order\_acquire}) \triangleright \text{re-read sequence counter}$ 
18:  if  $c_{\text{start}} \neq c_{\text{end}}$  then
19:    return STATUS_TORN_READ  $\triangleright$  concurrent write detected
20:  end if
21:  if  $\text{COMPUTECRC32}(B_{\text{scratch}}) \neq B_{\text{scratch}}.\text{crc32}$  then
22:    return STATUS_CHECKSUM_INVALID
23:  end if
24:  if  $(t_{\text{now}} - B_{\text{scratch}}.\text{timestamp\_ns}) > B_{\text{scratch}}.\text{lease\_us} \times 1000$  then
25:    return STATUS_LEASE_EXPIRED
26:  end if
27:   $B_{\text{active}} \leftarrow B_{\text{scratch}} \triangleright \text{promote only after sequence, CRC, and freshness checks}$ 
28:  return STATUS_VALID_LATCHED
29: end function

```



Napkin Math 8.1: Watchdog lease sizing from stopping kinematics

Problem: An Autonomous Mobile Robot (AMR) navigates an industrial fulfillment center. If the high-level neural policy hangs or operating system preemption stalls inference, how much total time remains before braking must begin, and how much of it can the watchdog lease consume?

1. Physical Parameters:

- Robot mass and cruise velocity: $m = 300$ kg, $v_0 = 1.5$ m/s.
- Braking capability: Mechanical brakes certified for worst-case deceleration $a_{\text{max}} = 2$ m/s².
- Assumed optical clearance for this worked scenario: LIDAR safety field reserves $D_{\text{clear}} = 0.6939$ m ahead of dynamic obstacles.

2. Stopping Kinematics:

- Once brakes engage, physical braking distance evaluates as $d_{\text{brake}} = \frac{v_0^2}{2a_{\text{max}}} \approx 0.5625$ m.
- Allowable blind drift distance before braking must begin is $\Delta d_{\text{drift}} = D_{\text{clear}} - d_{\text{brake}} \approx 0.1314$ m.

3. Watchdog Lease Ceiling:

- The lease, timeout detection, control/bus delivery, and brake-engagement delay share the same drift allowance:

$$T_{\text{lease}} + T_{\text{detect}} + T_{\text{hold}} + T_{\text{age}} + T_{\text{act}} \leq \frac{\Delta d_{\text{drift}}}{v_0}.$$

Here the right-hand side is 87.6 ms. The displayed value is an ideal **total pre-brake delay budget**. For illustration, a 60 ms action-chunk lease leaves 27.6 ms for detection, any post-expiry hold, sensor age, and brake engagement. The slower policy must refresh the chunk before expiry; a separate, faster heartbeat reports host liveness but does not renew its validity. All remaining terms need measured upper bounds, with further allowance for clearance uncertainty.

Systems insight: Watchdog lease durations are not software timeouts chosen for convenience—they are derived from the physical braking kinematics and sensor clearance of the plant. Spending the entire 87.6 ms on the lease leaves no time to detect the expiry or engage the brake; the actual lease must reserve these delays and retain a feasible stopping response.

This calculation establishes an ideal 87.6 ms total-delay ceiling under the stated speed, clearance, and deceleration assumptions. The action lease is only one term in that budget. The trajectory payload exchanged through this lock-free mailbox incorporates this physical lease into the strict schema defined in table 8.1, establishing explicit data types, SI units, and structural invariants.

Table 8.1: Abstract data schema for real-time action chunk exchange (TrajectoryProposalChunk): Field layout, scalar types, physical units, and structural invariants for trajectory proposals exchanged across the watchdog lease boundary via lock-free shared memory.

Field Name	Scalar Type	Physical SI Units	Structural Invariant & Description
sequence_id	uint64_t	dimensionless	Monotonically strictly increasing integer ($s_k > s_{k-1}$).
timestamp_ns	uint64_t	nanoseconds (ns)	Hardware capture timestamp on master distributed clock.
validity_lease_us	uint32_t	microseconds (μ s)	Chosen lease below the total 87.6 ms delay budget after response margins.
num_waypoints	uint16_t	dimensionless	Chunk horizon count ($K \in [16, 50]$).
joint_positions	float32[K] [N_j]	radians (rad)	Commanded joint angles within reach envelope ($q_{\min} \leq q \leq q_{\max}$).
joint_velocities	float32[K] [N_j]	radians/sec (rad/s)	Commanded joint velocities within certified ceilings ($\ \dot{q}\ \leq \dot{q}_{\max}$).
feedforward_torques	float32[K] [N_j]	Newton-meters (N · m)	Bounded feedforward motor torque vectors ($\ \tau\ \leq \tau_{\max}$).
crc32	uint32_t	bitmask	IEEE 802.3 CRC-32 polynomial over all preceding payload bytes.

To quantify this interface’s reliability, consider a Vision-Language-Action policy publishing action chunks (1.4 kB payload) to a shared SRAM mailbox at 20 Hz ($T_{\text{writer}} = 50,000 \mu$ s), while a real-time microcontroller reads it at 1000 Hz ($T_{\text{reader}} = 1000 \mu$ s). Across an on-chip SRAM bus operating at 1.0 GB/s (1000 B/ μ s), the writer copies 1400 bytes with memory barriers in $t_{\text{write}} \approx 1.5 \mu$ s, and the reader completes its copy in $t_{\text{read}} \approx 1.5 \mu$ s. A read collides with a write if the reader begins during

writer mutation or if the writer begins while the reader is actively copying, creating a vulnerable collision window:

$$t_{\text{vulnerable}} = t_{\text{write}} + t_{\text{read}} = 1.5 \mu\text{s} + 1.5 \mu\text{s} = 3.0 \mu\text{s}$$

If the relative writer–reader phase varies uniformly across the writer period, the expected collision fraction per read is:

$$P_{\text{collision|uniform phase}} = \frac{t_{\text{vulnerable}}}{T_{\text{writer}}} = \frac{3.0 \mu\text{s}}{50,000 \mu\text{s}} = 6.0 \times 10^{-5} = 0.006 \text{ percent.}$$

That assumption gives about 6 torn reads in 100,000 reader cycles. With exactly periodic 20 Hz and 1 kHz clocks at an unfavorable fixed phase, one read can collide with every write: about 2,000 in 100,000 cycles (2%). Under the stated bounded $1.5 \mu\text{s}$ write duration and one write per 50 ms, adjacent 1 ms reader ticks cannot both overlap a write. A detected odd or changed counter makes the reader discard that attempt and retain its last validated spline; its age and lease must still be checked. No collision percentage proves a safe physical trajectory or a bounded recovery from a writer stuck mid-write.

Checkpoint 8.2: Lock-free seqlock collision dynamics

Before examining rate transitions and spline interpolation, verify your understanding of lock-free mailbox synchronization:

- ☐ **Lock-free mailbox mechanics:** Can you explain how atomic sequence counters detect torn reads across asynchronous clock domains without requiring mutexes or priority inversion?
- ☐ **Clock phase and consecutive reads:** Under what timing assumptions does a short write avoid consecutive collisions, and why can fixed phase still cause one failed read per write?
- ☐ **Degradation during torn reads:** Can you describe how a real-time enforcer maintains smooth actuation trajectories when a shared-memory collision forces it to discard a torn proposal frame?

Rate mismatches introduce three failure modes: phase lag accumulation, aliasing, and buffer starvation. In a discrete-to-continuous conversion, a Zero-Order Hold (ZOH) maintains setpoints constant across the planning period T_{plan} , delivering piecewise-constant position steps rather than a smooth trajectory. This step quantization introduces an intrinsic half-sample phase lag $\Delta\phi = -\frac{\omega T_{\text{plan}}}{2}$. Directly feeding raw planner waypoints to inner motor loops at 10 Hz incurs 50 ms of uncompensated phase delay, degrading tracking margins and exciting unmodeled mechanical resonances. For the six endpoint conditions on position, velocity, and acceleration, the real-time enforcer can use a suitably constrained quintic bridge across the latched waypoint seam. It evaluates the admitted bridge at 1000 Hz to provide C^2 position continuity and corresponding velocity and feedforward acceleration setpoints; the tick rate alone does not establish seam continuity.

Aliasing occurs when high-frequency sensor noise folds into low-rate estimators, which the system prevents through analog anti-aliasing filters and digital decimation¹² within the high-rate microcontroller domain. Classical stability guarantees (Slotine and Li 1991) assume inner-loop execution jitter remains strictly bounded and setpoints arrive before the trajectory buffer drains. The nervous system enforces these conditions via cycle-deterministic microcontroller interrupts and buffer depth tracking. If the planner stalls and the buffer reaches its final valid waypoint, the nervous system rejects stale proposals and requests its validated local deceleration or hold. Reaching rest depends on available torque, clearance, and thermal capacity.

Systems Perspective 8.3: Zero-wait execution across clock domains

Never allow a high-frequency real-time control loop to acquire a mutex or spin on a lock shared with a stochastic process; communicate across clock domains exclusively via lock-free atomic sequence mailboxes or triple-buffers.

Prerequisite: High-rate joint interpolation of action chunks builds on the proposal generator in section 6.4.

¹² **Phase Lag in Moving Average Smoothing:** Averaging noisy actuator feedback using an N -tap finite impulse response (FIR) filter introduces a group delay of $(N-1)/2$ sample periods. In high-gain position control, this artificial transport delay consumes phase margin, turning stabilizing feedback into positive feedback that drives the actuator into violent self-excited limit cycles.

Communication between cognitive models and motor loops cannot rely on blocking queues, mutexes, or network sockets. Every interface boundary must be bridged by asynchronous, lock-free shared memory or deterministic fieldbuses that guarantee the high-rate motor loop never waits for the slow, stochastic brain.

The operational characteristics, physical time constants, hardware allocation, and failure protocols governing each tier across this multi-rate hierarchy are formalized in table 8.2.

Table 8.2: Multi-rate cadence hierarchy of Physical AI execution: Formal specification of execution tiers, physical time constants, jitter tolerances, hardware substrates, memory transport contracts, and deterministic failure protocols. The hierarchy shows representative rate tiers; each platform derives its required rates from actuator electrical time constants ($\tau_e = L/R$), mechanical phase margins ($\Delta\phi \approx \omega(T_s/2 + T_d)$), optical exposure windows, and thermal compute dissipation limits. Hard real-time reflexes at ≥ 1000 Hz execute in bare-metal silicon with sub-microsecond determinism to prevent hardware damage, while stochastic high-level models at 10–50 Hz generate candidate action proposals across lock-free atomic buffers under explicit intent leases.

Execution Tier & Frequency	Nominal Period (T)	Max Jitter Budget ($\Delta T_{\max}$)	Hardware & Compute Domain	Payload & State Representation	Inter-Tier Transport Contract	Overrun & Failure Protocol
Reflex & Safety (≥ 1000 Hz)	$T \leq 1.0$ ms ($50 \mu\text{s}$ at 20 kHz)	$\Delta T_{\max} < 10 \mu\text{s}$ (hard real-time)	Dedicated real-time MCU or FPGA fabric	Phase currents, encoder ticks, PWM duty cycles, emergency clamp flags	Single-cycle MMIO registers, tightly-coupled static memory	Validated local stop if torque and clearance remain; separate hardware torque trip on detected fault
Motion & Tracking (100–500 Hz)	$T = 2.0$ – 10.0 ms	$\Delta T_{\max} \leq 200 \mu\text{s}$ (firm real-time)	Deterministic real-time RTOS / microcontroller core	Continuous polynomial splines, $SE(3)$ wrenches, feedforward torques	Lock-free SPSC circular ring buffer with atomic sequence counters	Retain a valid spline only within its lease and stopping margin; then request feasible deceleration
Planning & Policy (10–50 Hz)	$T = 20.0$ – 100.0 ms	P_{99} target 15 ms (statistical, not WCET)	Edge tensor accelerator, host application processor	Action chunks (K -step horizon waypoints), visual embeddings	Shared static memory SPSC seqlock buffers	Example action lease 60 ms only for policy refresh faster than expiry; slower rates need another budget
Reasoning & Deliberation (0.5–2 Hz)	$T = 500$ – 2000 ms	Asynchronous ($\Delta T > 500$ ms)	Multi-core host CPU, workstation, cloud VLA coprocessor	Semantic task goals, topological sub-graphs, natural language tokens	Asynchronous inter-process messaging (gRPC / zero-copy IPC)	Planner pauses; feasible local hold or stop under current plant state

8.4 Deterministic Fieldbuses

Observed latency distributions do not establish worst-case timing bounds. In statistical performance analysis, an engineer measures ten million executions of a task, notes that the ninety-ninth percentile (P_{99}) latency is $135 \mu\text{s}$ and the maximum observed latency is $150 \mu\text{s}$, and infers that the computation fits comfortably inside a 1.0 ms control deadline. This inference is fatally flawed. An empirical distribution merely records what happened during finite testing, whereas a hard real-time guarantee mathematically proves what must happen under all possible hardware states. When an execution platform includes out-of-order execution pipelines, shared cache hierarchies, speculative prefetchers, and concurrent direct memory access traffic, a finite latency sample cannot establish an upper bound. The missing tail events do not vanish; they emerge under rare microarchitectural collisions—such as

concurrent DMA burst transfers and cache line evictions coinciding with a critical interrupt—that manifest under production stress.

Establishing a deterministic timing guarantee requires bounding every term that contributes to task response time: the base computation time, the blocking time spent waiting for shared critical sections, and the interference time from high-priority interrupts or memory bus contention.¹³ If any one of these three terms lacks a provable upper bound, the system possesses no timing guarantee, regardless of how small the mean latency appears in benchmarks.

To see how unanalyzed interference breaks an empirical guarantee, consider a robotic torque computation running at 1000 Hz. On an isolated processor with empty bus queues and warm caches, the workload exhibits an unloaded baseline execution time of 120 μ s. Across millions of undisturbed bench cycles, the measured tail latency never exceeds 150 μ s, leaving what appears to be a massive safety margin before the 1000 μ s deadline.

When the same binary runs on an integrated system-on-chip, normal machine activity introduces contention. High-resolution camera streams transfer image frames via DMA, an onboard neural network accelerator reads weight matrices in large bursts from shared memory, and ambient thermal buildup causes the processor to drop its core clock frequency. Contention introduces significant memory interference and blocking delays. The task crosses the 1000 μ s deadline, not because the algorithmic complexity grew, but because shared silicon resources lacked isolation.

Taming this latency spread requires classifying every source of execution variance into three structural categories based on whether its worst-case behavior can be formally bounded:

Bounded variance sources encompass deterministic hardware mechanisms whose execution cycles can be modeled with exact algebraic upper bounds: fixed-pipeline single-cycle instructions, deterministic branch decisions, round-robin bus arbitration, tightly-coupled static memory access with zero wait states, and vectored interrupt dispatch on dedicated bare-metal microcontrollers.

Experimentally characterized sources encompass shared architectural mechanisms whose maximum latencies cannot be proven from first-principles schematics alone, but whose worst-case envelopes can be bounded through exhaustive stress profiling under locked hardware configurations: shared last-level cache misses under hardware line locking, write-buffer drain stalls, dynamic RAM refresh intervals, and thermal throttling ceilings enforced by active cooling.

Unbounded variance sources encompass non-deterministic software and hardware mechanisms whose maximum latency has no finite upper bound: demand-paged virtual memory page faults, preemptive operating system thread scheduling without priority inheritance,¹⁴ dynamic heap allocations traversing fragmented free lists, speculative branch mispredictions that pollute pipeline caches, and unscheduled peripheral direct memory access traffic. A hard real-time safety path must strictly eliminate every mechanism from this third category.

¹³ | **Rate-Monotonic Scheduling:** Liu and Layland (1973) established that static-priority rate-monotonic scheduling guarantees periodic task schedulability when total processor utilization satisfies $U \leq n(2^{1/n} - 1)$, converging to $\ln(2) \approx 0.693$ for large n . In distributed robotic architectures with arbitrary execution deadlines and shared resource blocking, systems engineers deploy exact Response Time Analysis ($R_i = C_i + B_i + I_i \leq D_i$) to verify deterministic schedulability. For the formal mathematical derivation of response time recurrences, see Appendix C.

Downstream: Deterministic fieldbus jitter bounds provide the execution baseline for heterogeneous placement in section 26.3.

¹⁴ | **Mars Pathfinder Priority Inversion:** The 1997 Mars Pathfinder mission suffered unbounded priority inversion when a low-priority meteorological task blocked high-priority attitude control, tripping the system watchdog (Reeves 1997). The anomaly was resolved by enabling priority inheritance protocols on the RTOS mutex, as formalized by Sha et al. (1990).

📖 War Story 8.1: The Mars Pathfinder priority inversion (1997)

Context: In July 1997, shortly after landing on Mars, the Mars Pathfinder spacecraft’s avionics computer ran the VxWorks real-time operating system with preemptive priority-based scheduling across mission tasks (Reeves 1997).

Mechanism: A high-priority thread (bc_dist) distributed critical spacecraft bus telemetry, while a low-priority thread (ASI/MET) gathered meteorological sensor data through a shared mutex-protected pipe. When ASI/MET acquired the mutex, a medium-priority communications thread preempted it. Because the medium-priority thread outranked the meteorological thread, it monopolized CPU execution while bc_dist blocked waiting for the mutex held by the stalled low-priority task—a textbook unbounded priority inversion.

Impact: The high-priority bus distribution task missed its hard real-time deadline. A hardware watchdog timer detected the deadline overrun and initiated a complete spacecraft reboot, resetting state and threatening total mission loss.

Response: Engineers remotely diagnosed the thread deadlock, patched the spacecraft software from Earth by enabling priority inheritance on the VxWorks mutex semaphore, and verified stable telemetry scheduling.

Systems lesson: The medium-priority task kept the low-priority mutex holder from running, so the high-priority bus task could not progress. Priority inheritance let the holder finish and release the mutex. In a motor-control path, engineers must bound such blocking time (B) or remove the shared lock from the cycle-critical exchange.

A common misconception in real-time software design asserts that dynamic memory allocation is categorically forbidden on a hard timing path. The prohibition is imprecise. Unbounded dynamic allocation, exemplified by general-purpose heap allocators (`malloc/free`) that traverse fragmented free lists or execute variable-length page coalescing, is incompatible with a hard path because search latency depends on allocation history and heap fragmentation. In contrast, a bounded allocator, such as a fixed-size block pool or a Two-Level Segregated Fit (TLSF) allocator with guaranteed constant-time $\mathcal{O}(1)$ allocation and deallocation bounds, can be integrated safely into a real-time system. On the most safety-critical bare-metal path, the gold standard remains **zero-allocation static architecture**, where all buffers, ring queues, and state variables are statically declared and mapped to physical memory addresses at compile time, eliminating memory management execution entirely during runtime.

The choice and configured schedule of a physical communication bus contribute to interference term I and blocking term B in the worst-case response time equation. As Hermann Kopetz established in his foundational work on real-time systems architecture (Kopetz 2011), distributed embedded control requires a fundamental architectural distinction between event-triggered and time-triggered communication. In event-triggered paradigms (such as standard CAN), nodes transmit upon local state changes, leaving bus utilization unpredictable under load and exposing the system to the “babbling idiot” failure mode—where a single malfunctioning microcontroller or jabbering transceiver floods the shared medium with spurious traffic. In contrast, Time-Triggered Architectures (TTA) enforce global clock synchronization and deterministic time-division multiple access (TDMA), assigning each node a dedicated, non-overlapping transmission slot, allowing transmission interference to be bounded when clock synchronization, fault behavior, and the full schedule are verified. As summarized in table 8.3, robotics interconnects divide across a trade-off surface spanning raw throughput, transport latency, cycle jitter, and hardware determinism. In industrial robotic platforms, this deterministic physical layer is implemented through dedicated fieldbus hardware architectures (figure 8.4), which couple hard real-time host processors to modular input/output and motor drive terminals over synchronized Ethernet frames using hardware Distributed Clocks (DC).¹⁵

¹⁵ | **Distributed Clock:** Under IEC 61158-4-12, EtherCAT Distributed Clocks (DC) synchronize distributed servo drive nodes to a master reference clock by measuring propagation delays across the physical ring topology. Distributed clock hardware can generate synchronized SYNC0 pulses. Achieved clock error and application sampling jitter depend on the selected devices, configuration, and measurement; clock alignment alone does not bound the complete control cycle.

Table 8.3: Fieldbus and interconnect protocol trade-off matrix for Physical AI subsystems: Comparison of signaling, configured link rate, timing mechanisms, wiring, and possible subsystem placement across robotics interconnects. Except for the named EtherCAT physical link, rates and reach depend on device and wiring; latency and jitter require a configured response-time analysis. SPI serves short local links; CAN-FD arbitrates shared chassis traffic; EtherCAT forwards cyclic frames through distributed nodes; and TSN can schedule mixed traffic when its switches and clocks are configured accordingly.

Interconnect Protocol	Physical Topology & Signaling	Payload Throughput (β)	Transfer Latency & Cycle Jitter	Determinism Classification	Physical Reach & Wiring Harness	Physical AI Subsystem Placement
SPI (Serial Peripheral Interface)	Synchronous 4-wire full-duplex (SCLK, MOSI, MISO, CS), point-to-point	Configured controller and trace-dependent rate	Register latency and jitter depend on clock, transfer length, and driver	Bounded under a fixed transfer schedule	Short board-level traces or flex cable	Joint magnetic angle encoders, 3-phase current-sense ADCs, gate drivers
CAN-FD (Controller Area Network FD)	2-wire differential multidrop with bitwise priority arbitration	Configured arbitration and data-phase bitrates	Arbitration and retransmission enter the response-time bound	Bounded only under a specified traffic/fault model	Bitrate-, cable-, and topology-dependent bus reach	Chassis management, battery management systems (BMS), auxiliary actuators

Interconnect Protocol	Physical Topology & Signaling	Payload Throughput (β)	Transfer Latency & Cycle Jitter	Determinism Classification	Physical Reach & Wiring Harness	Physical AI Subsystem Placement
EtherCAT (Ethernet Control Automation)	100BASE-TX full-duplex daisy-chain; on-the-fly hardware frame processing	100 Mbps link; payload efficiency depends on frame layout	Frame, forwarding, and clock error require device-specific bounds	Bounded under a qualified cycle and device configuration	≤ 100 m between nodes (shielded Cat5e/Cat6)	Multi-axis robot arm joint servos, high-rate force-torque sensors
TSN Ethernet (IEEE 802.1Qbv / Time-Sensitive Networking)	Star/mesh switched Ethernet with time-aware traffic shapers	Configured Ethernet port rate	Slot and clock error depend on the verified network schedule	Bounded under a verified time-aware schedule	PHY- and medium-dependent reach per link	Centralized backbone bridging multi-camera vision, lidar, and safety reflexes

Napkin Math 8.2: Fieldbus serialization before schedulability

Practical engineering question: How much of an assumed 1 ms eight-axis cycle do the listed CAN-FD frames or one EtherCAT frame occupy before controller and device delays are counted?

1. *CAN-FD Serialization and Arbitration Latency (8 Axes, 64-byte payload per axis):* - Bitrate: 1.0 Mbps arbitration phase, 5.0 Mbps data phase. - Per-frame bit count: 32 bits arbitration/CRC overhead at 1.0 Mbps ($32 \mu\text{s}$), 512 bits payload at 5.0 Mbps ($102.4 \mu\text{s}$), plus inter-frame spacing $\approx 6.0 \mu\text{s}$. - Transmission time per axis: $t_{\text{tx}} \approx 140.4 \mu\text{s}$. - For 8 axes polling cyclically: $8 \times 140.4 \mu\text{s} = 1123.2 \mu\text{s}$ ($= 1.123$ ms). - Result: Bus utilization exceeds 100 percent ($\rho \approx 112.3 > 1.0$), so this particular eight-frame schedule cannot fit a 1 ms bus slot even before processing. If payload is reduced to 32 bytes ($t_{\text{tx}} \approx 89.2 \mu\text{s}$), total bus time is $713.6 \mu\text{s}$ (71.4 utilization). This reduced payload still needs arbitration, error/retransmission allowances, competing frames, and response-time analysis; no universal 70 percent utilization threshold establishes schedulability.

2. *EtherCAT On-The-Fly Processing (8 Axes, Daisy-Chain Ring):* - Bitrate: 100 Mbps full-duplex Fast Ethernet (10 ns/bit). - A single standard Ethernet frame carries all 8 joint commands and sensor returns in one pass. - Frame serialization time: $25.6 \mu\text{s}$. - Node forwarding delay: 8 slave application-specific integrated circuit (ASIC) controller hops at $0.6 \mu\text{s}$ per hop $= 4.8 \mu\text{s}$. - Modeled frame serialization plus eight forwarding hops: $T_{\text{bus}} = 25.6 \mu\text{s} + 4.8 \mu\text{s} = 30.4 \mu\text{s}$. - Result: These modeled bus terms occupy 3.04 of the cycle, leaving $969.6 \mu\text{s}$ **unallocated by this bus calculation**. Master/slave processing, propagation, clock synchronization, retries, application computation, and actuator response still consume the end-to-end deadline. A platform-specific distributed-clock measurement is needed for sampling jitter.

Systems insight: In this eight-axis, 64-byte-per-axis scenario, CAN-FD serialization alone misses the assumed 1 ms slot, while EtherCAT frame and forwarding occupancy leaves room for other work. Choosing EtherCAT, TSN, a different CAN schedule, or a different cycle requires a complete response-time and fault-budget analysis. While **TSN Ethernet** offers the allure of unifying gigabit camera feeds with microsecond servo frames over a single physical link, configuring and formally verifying its time-gated queues across asynchronous switches requires exhaustive verification to prevent video bursts from colliding with time-critical control slots.



Figure 8.4: Industrial controller with modular I/O: Beckhoff CX5120 DIN-rail controller and adjacent I/O terminals. The photograph identifies a possible hardware substrate for distributed control; achieved cycle jitter, isolation rating, and watchdog response require the exact installed configuration and measurement. (Photo: Oleg Yunakov, Wikimedia Commons, CC BY-SA 4.0).



Checkpoint 8.3: Fieldbus latency and WCET schedulability

Before evaluating actuation authority and hardware isolation, verify your understanding of fieldbus schedulability and jitter bounds:

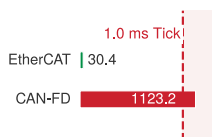
- ☐ **On-the-fly frame processing:** Can you explain why EtherCAT's hardware-forwarded frame structure scales to multi-axis kinematic loops with minimal bus utilization, whereas CAN-FD experiences severe deadline pressure under cyclic polling?
- ☐ **Asynchronous frame preemption:** Can you explain how arbitration, retransmission, and competing frames enter a response-time bound beyond the simple serialization sum?
- ☐ **Distributed clock synchronization:** Can you distinguish physical jitter bounds enforced by hardware phase-locked loops from statistical latency metrics gathered on an idle system?

Real-time dependability cannot be achieved by buying faster clock speeds or measuring statistical latency percentiles on an idle bench. Dependability requires structural analysis of the execution path: eliminating all unbounded variance sources ($I_{\text{unbounded}} = 0$), bounding shared hardware blocking (B), enforcing zero-allocation static memory layouts, and selecting fieldbuses with cycle-deterministic distributed clocks. Read as examples rather than a historical performance law, figure 8.5 plots illustrative link rates and scenario cycle targets for five fieldbus families.¹⁶ The points are not measured end-to-end latency or evidence that bandwidth alone determines schedulability.

8.5 Actuation Authority

In a physical AI system, authority is the ability to energize motor coils and deliver mechanical power. In a correctly structured machine, this authority is concentrated exclusively within the deterministic real-time enforcer (figure 8.6).

16 | **Time-Sensitive Networking:** IEEE 802.1Qbv Time-Sensitive Networking establishes deterministic Ethernet communication by configuring time-aware traffic shapers across network switches. High-priority control frames execute within dedicated, pre-scheduled transmission windows, while asynchronous high-bandwidth payloads (such as uncompressed camera streams) are temporarily gated. This scheduled time-division multiplexing allows vision telemetry and microsecond servo commands to coexist over a unified physical link without packet collision or buffer bloat.



In this eight-axis scenario, EtherCAT frame and hop time uses about 3 percent of the bus slot; the remainder is not proven application headroom.

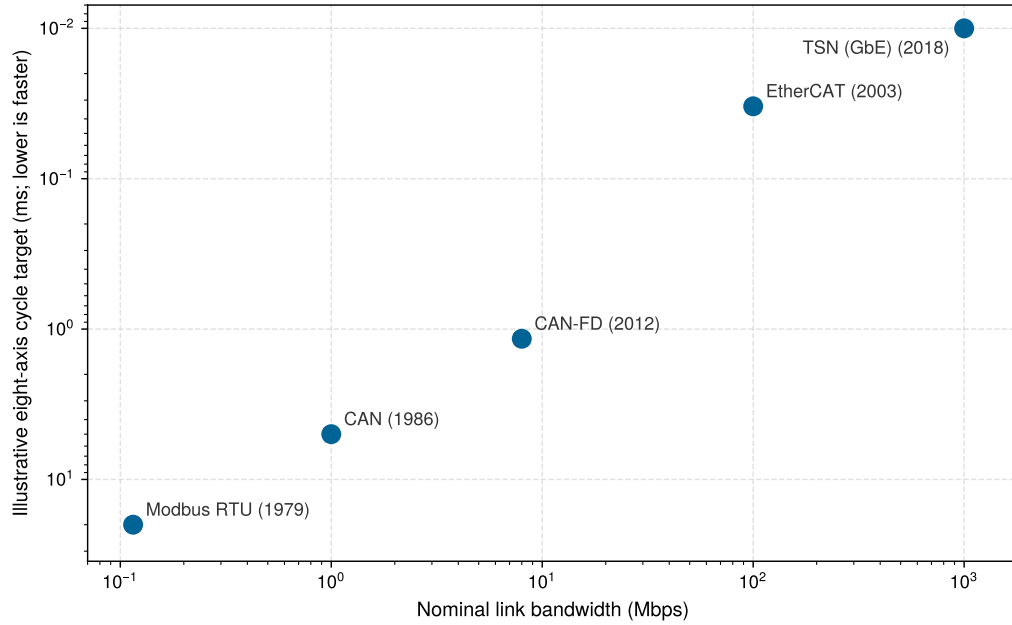


Figure 8.5: Illustrative fieldbus design space: Nominal link bandwidth and selected eight-axis cycle targets for five protocol scenarios. The cycle values are teaching assumptions, not measured device results or a protocol-level guarantee; workload, implementation, faults, and full response-time analysis determine feasible cycles.

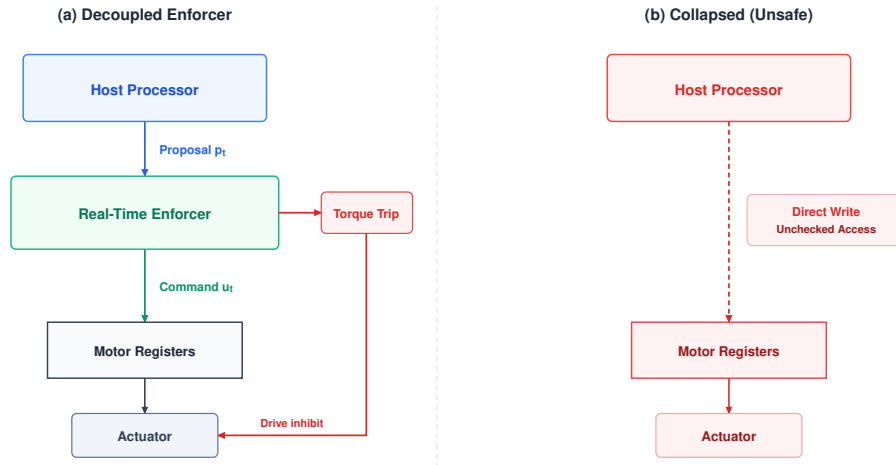


Figure 8.6: Hardware privilege boundary: (a) Decoupled safety enforcer isolating the real-time microcontroller with exclusive write privilege over motor drive registers while the host processor proposes trajectories across a shared-memory link. (b) Collapsed unsafe architecture where unverified host software writes actuator registers directly, creating a single failure domain.

A learned foundation model or trajectory planner on an application processor has **zero physical authority** to write PWM timer registers, set gate-driver current references, or toggle motor bridge enable pins that govern current delivery to stator windings. Instead, high-level software generates unprivileged trajectory proposals (p_t). These proposals pass across the memory transport interface into the real-time microcontroller, which functions as a strict hardware gatekeeper.

The microcontroller continuously evaluates physical safety invariants: verifying that commanded joint positions lie within physical reach envelopes, that joint velocities do not exceed certified limits, and that projected decelerations remain safely within dynamic stopping clearance ($d_{\text{stop}} \leq D_{\text{clear}}$).

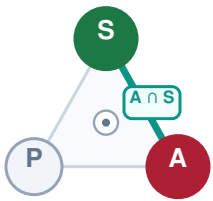
Downstream: Actuator override masking registers implement the hardware preemption tiers formulated in section 28.8.

17 | Hardware Trip Zone:

Microcontroller hardware trip zones (such as TI EPWM_TZFRM or STM32 TIMx_BDTR Break inputs) couple analog over-current and overvoltage comparators directly to timer output logic. Upon fault detection, configured trip logic may force gate signals high-impedance or activate a validated braking mode. These have different mechanical consequences: torque removal can leave the plant coasting, while dynamic braking requires a rated energy path. The trip response and the resulting physical stop must be verified for the platform.

18 | Braking Dynamics:

Arresting a high-inertia manipulator joint transfers kinetic energy into the implemented braking path: winding and power electronics losses, a rated brake resistor, or mechanical friction. The trajectory must satisfy stopping clearance ($d_{\text{stop}} \leq D_{\text{clear}}$) and each component's pulse-energy and thermal limits; $\int I^2 R dt$ accounts for the winding share when resistive motor braking is used. For the complete derivation of kinematic stopping envelopes and transient thermal dissipation, see Appendix B.



The $\text{Act} \cap \text{Sense}$ reflex interface: sub-millisecond fieldbus loops and override registers.

19 | Safe Torque Off (STO):

An STO implementation removes torque-producing drive signals through a hardware interlock. The illustrated dual-channel circuit is one design pattern; an IEC 61800-5-2 integrity rating depends on the actual devices, diagnostic coverage, wiring, and validation. STO does not by itself establish a controlled stop for moving mass.

If a proposal satisfies all invariants, the microcontroller grants execution permission, interpolates waypoints into high-frequency current references (u_t), and writes to the physical gate-driver registers. If a proposal is invalid or expired, the enforcer revokes permission ($\perp$) and requests a controlled deceleration or hold when the measured state, available braking torque, clearance, and energy budget make that response feasible. A separate hardware trip zone (such as TI EPWM_TZFRM or STM32 TIMx_BDTR) can remove drive torque on a fault; opening switches does not itself brake moving mass.¹⁷

Consider the physical energy dynamics of this intervention. If the enforcer rejects an invalid proposal and commands dynamic braking on a fast-moving joint, the braking path must have enough torque, clearance, and thermal capacity to arrest the mass.¹⁸ Building on the actuator thermal limits established in section 4.5, over the braking interval (often hundreds of milliseconds), the joint's kinetic energy must be absorbed by the implemented braking path, which may include winding resistance, a brake resistor, or mechanical friction. The physical plant must absorb this sudden influx of thermal energy (which can spike to tens of watts on a single small joint) without melting the thin enamel insulation on the motor windings or allowing the heavy moving mass to coast beyond its geometric clearance boundary.

By virtue of the causal boundary established in section 2.2, the enforcer treats the origin of an aberrant proposal as completely irrelevant. A remote memory exploit in an operating system daemon, an adversarial camera perturbation inducing model hallucination, and an arithmetic overflow in a neural matrix multiplier produce the identical boundary event: an invalid proposal packet. Because the enforcer evaluates invariant bounds against physical sensor feedback from optical encoders and current shunts rather than inspecting neural network weights, the host cannot write actuator registers directly. Protection against a compromised host still depends on independent sensing, a correct limit model, actuator capability, and a feasible fallback.

To guard against internal hardware faults or microcontroller latch-ups within the enforcer itself, the system incorporates out-of-band supervisory channels and physical disconnects. These channels consist of unprogrammable analog circuits, hardwired emergency stop mushroom push buttons, and dual-channel safety relays that sever power to the motor inverter bridges independently of all digital logic through Safe Torque Off (figure 8.7, IEC 61800-5-2), which physically interrupts gate-driver bias to prevent torque-producing stator current.¹⁹ While these disconnects remove torque-producing power through the designed path, they impose a severe cost on system availability. Re-energizing a tripped high-voltage direct-current bus requires pre-charging the inverter filter capacitors through current-limiting resistors; without this step, closing the contactor onto empty capacitors draws a massive inrush spark that physically welds the mechanical metal contacts together. This pre-charge sequence consumes hundreds of milliseconds before motor control can resume. Physical disconnects therefore trade availability and active control for a fault response whose mechanical consequences depend on brakes, load, and stored energy. Ensuring that these supervisory circuits and enforcer channels remain capable of acting when the primary controller fails requires verifying that they share no hidden dependencies in their clocks, power rails, or silicon substrates.



Systems Perspective 8.4: Hardware authority invariant

If an unverified learned policy possesses direct memory-mapped write access to actuator gate-driver registers, your system has no safety layer—it has only an accident waiting for an out-of-distribution input.

Under the proposal-permission contract codified in chapter 2, the cognitive brain proposes behaviors across high-dimensional semantic spaces, but possesses zero authority to energize coils. Only the deterministic real-time enforcer retains physical write privilege over the hardware registers that convert digital words into moving mass.

8.6 Hardware Isolation

Independence claims about shared clocks, buses, rails, and memory are often incorrect upon first verification. System architectures routinely label the brain and the nervous system as isolated because software tasks execute in separate processes, containers, or hypervisor partitions. This

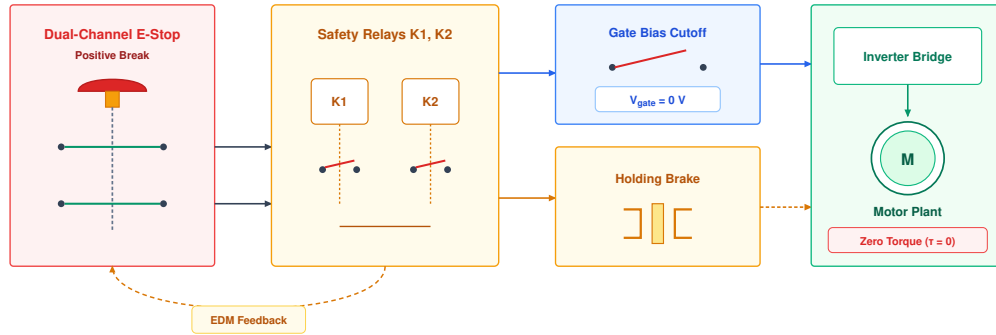


Figure 8.7: Illustrative out-of-band interlock and STO path: A dual-channel emergency switch and monitored relays remove torque-producing gate bias; a separate spring-applied holding brake is shown. This diagram describes a hardware response path, not a certified integrity level or proof of stopping distance (synthesized from ISO 13849-1 and IEC 61800-5-2).

example checks four common-cause paths: independent execution silicon with private instruction pipelines, separate timing references from segregated crystal oscillators, isolated power rails from independent regulators, and segregated memory domains without shared dynamic random-access memory chips or interconnect arbitration buses.²⁰ Any shared substrate needs a fault analysis that shows it cannot defeat the independent permission path under the stated failure model.

Shared electrical and physical infrastructure creates cross-domain failure modes that bypass all software access controls. During a sudden burst of dense matrix multiplication, a deep learning accelerator can pull a transient current spike of 50 A with a slew rate exceeding 100 A/ μ s. If the accelerator and the safety enforcer share a direct-current power rail without dedicated isolation, the resulting voltage sag causes the enforcer rail to droop below its microcontroller reset threshold, dropping below 3.0 V for 5 μ s and resetting the safety supervisor while the plant is moving under power. On a shared silicon package or common thermal heat sink, sustained computational load in the brain elevates junction temperatures beyond 105°C, triggering thermal throttling or silicon shutdown in adjacent real-time cores. When communication relies on an arbitrated interface such as a shared peripheral interconnect or memory bus, a high-bandwidth visual pipeline or a misbehaving direct memory access controller can saturate bus bandwidth, starving the nervous system of encoder feedback and sensor telemetry. Even a common clock oscillator creates an unbudgeted single point of failure, where thermal drift, mechanical vibration, or crystal cracking desynchronizes both domains at the same instant.

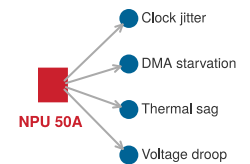
A deterministic nervous system must distinguish between two fundamentally distinct failure modes in the brain: unresponsiveness and corruption.

Unresponsiveness occurs when an inference pipeline hangs, a scheduling deadline is missed, or operating system memory fragmentation blocks execution, producing no output. The nervous system detects unresponsiveness through a monotonic heartbeat lease: the host brain must continuously deliver an authenticated, sequentially incrementing token across an isolated channel before a hardware countdown timer reaches zero.

Corruption occurs when the brain remains fully active and passes its heartbeat deadline, but generates kinematically impossible setpoints, discontinuous velocity requests, or numerical infinities resulting from out-of-distribution visual inputs or matrix overflow. In accordance with the privilege separation defined in chapter 2, the enforcer treats these failures through separate mechanisms: missing fresh chunks or heartbeat triggers the corresponding timeout response, whereas corruption triggers an immediate invariant rejection on the arrival of the malformed proposal packet.

The action-chunk validity lease is distinct from the host heartbeat. Its duration is bounded by the plant stopping budget, while the heartbeat separately detects host liveness. In the event of a silent brain stall, the actuators would otherwise continue applying their last commanded torque—maintaining unguided acceleration—until the enforcer detects the timeout and initiates emergency deceleration. Enforcing this dynamic limit at the causal boundary (section 2.2) yields a total pre-brake delay bound that the action lease must respect—as demonstrated by the 87.6 ms ideal total-delay

²⁰ | **Freedom from Interference (FFI):** ISO 26262-9 and IEC 61508 formalize Freedom from Interference (FFI) as the absence of cascading faults between software elements possessing different safety integrity levels. Unverified neural policies sharing silicon or bus fabrics with safety-critical tasks require architects to demonstrate spatial (memory protection), temporal (independent scheduling and timers), and electrical isolation (independent power rails). Shared resources require evidence that interference cannot defeat the specified enforcement deadlines and fault response.



Unpartitioned physical substrates allow cognitive compute spikes to crash real-time safety controllers.

²¹ | **Watchdog Lease Bound:** For this constant-speed example, the combined action lease, detection, optional hold, measurement age, and brake-engagement delay must satisfy $T_{\text{lease}} + T_{\text{detect}} + T_{\text{hold}} + T_{\text{age}} + T_{\text{act}} \leq (D_{\text{clear}} - d_{\text{brake}})/v_0$. The right-hand side is the ideal total pre-brake delay budget, not a lease to spend in full. The bound also assumes valid clearance and available deceleration. For the complete algebraic derivation of watchdog lease bounds under bounded deceleration, see Appendix B.

Downstream: Real-time fieldbus fault-recovery state machines are verified via hardware fault-injection in section 30.1.

²² | **CAN Arbitration:** Controller Area Network protocols resolve bus contention at the physical layer using wired-AND dominant ('0') and recessive ('1') signaling. If a defective transceiver continuously asserts a dominant state (a "babbling idiot" fault), it monopolizes the shared physical bus and blocks all higher-level communication. Hardware bus guardians and dominant-timeout clamp circuits monitor transmission duration, physically disconnecting jabbering nodes to preserve network availability for remaining safety participants.

budget in section 8.3.²¹ For illustration, a 60 ms action-chunk lease leaves at most 27.6 ms for timeout detection, any post-expiry hold, brake engagement, and sensor-age allowance. A separate, faster host heartbeat can monitor process liveness but cannot renew an expired action chunk; its period is independent of the slower policy update. At a nominal 20 Hz chunk rate, the 50 ms period leaves less than 10 ms for update jitter and transport before this illustrative lease expires; a 10 Hz policy cannot sustain it without a different local plan or larger stopping envelope. These allocations require measured upper bounds before use.

When a lease expires or an enforcer rejects a corrupted proposal, the nervous system executes an explicit degradation strategy rather than an unconditioned power cut. In a moving mechanical system, abruptly removing motor power turns active control into unconstrained ballistic coasting, where the robot moves purely under momentum and gravity. This stored kinetic and potential energy can violently back-drive gearboxes into mechanical stops. Degradation follows a deterministic state hierarchy calculated within the nervous system. An optional short continuation of the last validated trajectory is permissible only while its action lease remains valid and its stopping margin is feasible. After expiry, any further hold or dead reckoning consumes the same total pre-brake delay budget: $T_{\text{lease}} + T_{\text{detect}} + T_{\text{hold}} + T_{\text{age}} + T_{\text{act}} \leq 87.6$ ms in the worked scenario. The illustrative 60 ms lease leaves 27.6 ms for all remaining terms, so the enforcer must begin braking sooner if their measured bounds do not fit. If communication does not resume within that budget, the nervous system requests controlled deceleration using the available torque and thermal budget established in section 4.4. It reaches rest only if clearance and braking capability remain sufficient. A gravity-loaded arm may require a rated mechanical holding brake after motion has been arrested.

Once the physical plant is brought to a complete, stationary halt, the architecture executes a **recovery handshake** before restoring autonomous proposal authority. The bare-metal microcontroller asserts a persistent `FAULT_HELD` latch in the shared SRAM mailbox and reports the exact physical rest pose ($\mathbf{q}_{\text{rest}}, \dot{\mathbf{q}} = \mathbf{0}$). The application processor must flush its stale action chunks, clear its autoregressive key-value (KV) attention cache, re-ingest fresh optical observations tagged with new hardware timestamps, and publish a re-synchronized intent proposal. The microcontroller verifies that the initial waypoint of the new proposal matches the physical rest state within a strict spatial epsilon ($\|\mathbf{a}_0 - \mathbf{q}_{\text{rest}}\| \leq \epsilon_{\text{sync}}$) before releasing the fault latch and re-energizing pulse-width modulation (PWM) to the motor drives.

Verifying that these isolation boundaries function as designed requires physical falsification on the test bench. Four specific fault-injection regimes must be executed against the physical prototype: power-rail droop testing (injecting maximum-slew current steps to verify less than 50 mV of rail cross-coupling noise), clock glitching (verifying independent timing domains maintain stable cycle counts under frequency drift), communication bus flooding (saturating links to verify heartbeat telemetry retains deterministic transmission slots),²² and numerical fault injection (streaming synthetic proposals containing NaNs and infinite floats to verify single-cycle rejection without execution stalls).

The implementation must validate independence against its identified common-cause faults, and derive the lease from the remaining stopping margin after detection and actuation delays.

8.7 Invariant Checking

Locomotion policies trained with reinforcement learning and compact visuomotor diffusion models can be compiled to execute on edge tensor processors at rates between 100 Hz and 500 Hz. At 500 Hz, the model emits a new actuator command every 2.0 ms. This latency matches the mechanical time constant of high-performance electric motors and the update rate of outer position loops, leading software designs to collapse the distinction between the brain and the nervous system. The argument made for this collapse is computational efficiency: if the policy directly produces joint torques at the native rate of the plant, intermediate setpoint generators, trajectory interpolators, and tracking controllers appear to add latency without adding expressiveness.

Rate matching does not confer authority. A neural network emitting torques at 500 Hz is governed by the same statistical uncertainty as a large model generating waypoints at 5 Hz. It remains an unverified function approximator operating over high-dimensional input spaces where safety guarantees cannot be proven statically. Direct access to motor-drive registers eliminates the independent physical layer enforcing mechanical boundaries. As formalized in the proposal-permission

framework of chapter 2, separating proposal generation from permission grant is not an artifact of mismatched clock rates, but a structural necessity for maintaining invariant boundaries. Even when the policy and the enforcer execute within the same 2.0 ms scheduling period, the output of the neural network is strictly a proposal that must obtain permission before changing current in a physical winding to generate mechanical force.

Operating an unverified learned policy inside the fast control loop exposes the physical plant to three distinct failure modes: compute underrun, out-of-distribution torque spikes, and high-frequency structural limit cycling.

Compute underrun occurs when execution jitter in the accelerator pipeline causes an inference cycle to exceed the actuation deadline. If memory bus contention or transient thermal throttling extends inference duration from a nominal 1.4 ms to 2.2 ms, an end-to-end model directly driving motor inverters leaves the power stages without a valid setpoint. In a decoupled nervous system, this deadline overrun is detected deterministically by a hardware watchdog countdown circuit mapped to the inference boundary: if the model fails to post a validated setpoint before the 2.0 ms deadline, the real-time enforcer discards the expired proposal and requests its validated local response. Zero-torque coasting and dynamic braking have different stopping behavior; neither is safe by timing alone.

Out-of-distribution torque spikes occur when an unexpected physical contact or obscured camera lens pushes the neural network into an untrained region of its latent space. Out-of-distribution observations can cause the network to output discontinuous torque vectors, such as commanding instantaneous full reverse torque during forward motion. If a direct-drive actuator receives a step torque reversal across a single 2.0 ms control cycle, the resulting torque slew ($\frac{d\tau}{dt}$) can overload drive teeth and stress inverter silicon before slower vision feedback registers the motion. For the full kinetic calculations of reflected rotor inertia and angular jerk, see Appendix C. The enforcer limits this mechanism by evaluating synchronous rate-of-change ($\frac{d\tau}{dt}$) and torque limits against the state of the previous cycle.

High-frequency structural limit cycling occurs when unmodeled link flexibility, gearbox compliance, or sensor phase lag creates positive feedback with the learned policy, driving the mechanical assembly into destructive acoustic resonance and mechanical fatigue. Because this resonance cannot be detected from the command tensor alone, the nervous system evaluates a continuous energy tank model rooted in passivity-based control. In this framework, the enforcer maintains a virtual energy reservoir initialized with a fixed energy budget: any proposed actuator torque that injects kinetic energy into the mechanical plant must draw from the tank, while natural damping and dissipative braking replenish it. If the tank is exhausted, the enforcer clamps or attenuates proposed torques to strictly passive actions, subject to the measured state, model, and actuator limits; the ledger alone does not prove stability for unmodeled compliance.

To prevent these failures without degrading loop frequency, the architecture implements **synchronous invariant checking** directly inline with the high-rate inference pipeline. Under a 2.0 ms cycle budget, the system allocates 1.5 ms to sensor acquisition and neural network execution on the host accelerator, reserving the final 500 μ s for the nervous system to evaluate physical invariants on an independent deterministic core.

To evaluate these invariants deterministically within this 500 μ s window, the bare-metal micro-controller evaluates two discrete tests without iterative solvers:

1. **Stopping-margin precheck and candidate-torque test:** For motion toward an upper joint limit ($\dot{q} \geq 0$), measured position and velocity first establish whether stopping remains feasible under available deceleration:

$$h(q, \dot{q}) = (q_{\text{limit}} - q) - \frac{\dot{q}^2}{2a_{\text{max}}} \geq 0.$$

This is a state precheck: accelerating and braking torque produce the same h at the current instant. Before granting the candidate τ_{prop} , the enforcer must also predict its effect with bounded dynamics, for example checking $\dot{h}(q, \dot{q}, \tau_{\text{prop}}) + \alpha h(q, \dot{q}) \geq 0$ under input limits (see section B.2). If the current state is already infeasible, it requests the maximum available risk-reducing response; braking may no longer prevent a boundary crossing.

2. **Discrete Energy Tank Accumulator:** To prevent unmodeled structural dynamics from destabilizing the plant, the enforcer tracks a discrete energy ledger $T[k]$:

$$T[k+1] = T[k] + \Delta t \left(P_{\text{dissipate}}[k] - \max(0, \tau_{\text{prop}}[k]^{\top} \dot{\mathbf{q}}[k]) \right)$$

If $T[k+1] \leq 0$, the proposed torque fails the energy budget; the enforcer rejects or attenuates it and uses a verified damping response if actuator authority permits.

When a candidate fails either check, the enforcer selects a precomputed admissible command or a defined risk-reducing fallback. A closed-form torque limit is useful only if the state estimate, dynamics bound, actuator limits, and computation time have been verified for this platform. The fast loop can preserve task throughput while the independent permission path controls register writes.

High inference frequency does not grant register write privilege to unverified neural outputs. Microsecond-scale invariant enforcers must gate every command before energy reaches the physical plant.



Checkpoint 8.4: Hardware isolation and real-time invariant checking

Before examining supervisory teleoperation, verify your understanding of hardware isolation and pre-actuation invariant defenses:

- ☐ **Four dimensions of physical isolation:** Can you enumerate the four physical boundaries (silicon pipelines, timing crystals, power rails, memory buses) that this example checks for fault-domain independence, and explain how shared paths would need validation?
- ☐ **Physical derivation of watchdog leases:** Can you explain why software heartbeat timers must be sized directly from vehicle stopping distance and optical clearance envelopes rather than arbitrary software timeouts?
- ☐ **Discrete kinematic and energy invariants:** Can you distinguish a state stopping-margin precheck from a candidate-torque test, and explain when the energy-tank and fallback calculations fit the stated cycle budget?

8.8 Supervisory Teleoperation

Extending the proposal-permission model of chapter 2 to teleoperation, the nervous system places a human operator on the exact same structural footing as an autonomous policy. A human operator is a third authority holder whose inputs are proposals subject to transport delays and physical execution limits, not an external master switch with unmediated access to motor amplifiers. On a common execution timing diagram, the three authority paths occupy distinct bands. The nervous system reflex enforcer executes deterministically on an internal core at 1000 Hz, evaluating safety invariants every 1.0 ms. The autonomous learned policy executes on an onboard accelerator at 20 Hz to 50 Hz, delivering a fresh proposal every 20 ms to 50 ms with an internal compute latency of 15 ms. A remote human operator operates at visual-motor bandwidths between 5 Hz and 10 Hz, transmitting setpoints across an external communications link that adds variable packet transport delays. Because biological nervous systems exhibit an intrinsic neuromuscular reaction latency of 150 ms to 250 ms, an operator viewing a video stream is effectively issuing commands to a past “ghost” of the robot based on physical states that existed hundreds of milliseconds ago.

Treating human inputs as unconditional commands leads directly to hardware damage. In an emergency or under high cognitive load, an operator can pull a joystick to its mechanical stop, commanding an angular acceleration that exceeds inverter current ratings or commanding a rapid velocity reversal that shears gear teeth. Even during nominal teleoperation, network jitter and latency can transform an operator’s stabilizing correction into phase-lagged positive feedback—where a delayed counter-steering command arrives just as the chassis naturally swings back, amplifying the error and driving the mechanical structure into divergent oscillations. To prevent these failures, the system routes all operator inputs through an arbitration stage termed the **control multiplexer**. The control multiplexer functions as a deterministic arbiter, selecting whether the active proposal originates from the autonomous policy stream or the human supervisory channel (such as an industrial

teach pendant equipped with a three-position deadman enabling switch, a device chapter 28 examines once leases and enabling hardware are on the table) based on operational mode and heartbeat validity. The output of the multiplexer does not connect directly to the motor gate drivers. Instead, it passes into the identical pre-actuation invariant checker that inspects autonomous proposals, where physical constraints on joint velocity, rotor jerk, and energy accumulation are evaluated at 1000 Hz. If an operator commands a trajectory that violates an invariant boundary, the enforcer projects the command onto the boundary of the safe set or substitutes an active damping control law.

Because human inputs arrive across a communications channel subject to latency and packet loss, the physical mechanics of the plant define an upper bound on tolerable network delay before the nervous system must intervene.

At the causal boundary (section 2.2), when a vehicle travels under remote teleoperation, any delay in the communication channel translates directly into unguided physical displacement. The total stopping distance comprises the distance traveled during the network latency period plus the distance required to physically brake the mass. If network delay exceeds the allocated response budget, the vehicle may exhaust its stopping clearance before an operator correction arrives. By subtracting human neuromuscular reaction time and video encoding latency from this total allowable delay budget, we obtain a strict upper bound on network round-trip time (for formal bounding of safe network latencies under teleoperation, see Appendix B).

If network delay consumes the reserved response budget, or a command misses its lease, the onboard enforcer must revoke stale teleoperation authority and request its validated local response. Stopping within clearance depends on the current state, measured free space, and available braking.

Placing the human within the proposal hierarchy clarifies the distinction between administrative priority and physical authority. An operator holds higher administrative priority than an autonomous model, meaning the control multiplexer grants preemption rights to human inputs whenever an active override signal is received. However, administrative priority does not grant immunity from physical invariants. Both the human operator and the machine learning model sit strictly upstream of the invariant enforcer, differing only in the temporal characteristics and origin of their proposals. The broader systems problems of human supervision, including formal state transitions during authority transfer, cryptographic authentication over wireless uplinks, operator workload limits, and supervisory user interfaces, constitute the safety case examined in chapter 28. Within the nervous system, human intervention introduces only the arrival latency of the proposal stream and the invariant envelope that restricts its physical realization.

To translate these teleoperation constraints into concrete engineering bounds, consider how the AMR braking dynamics established in section 8.3 extend to remote control. Recall that for the 300 kg mobile robot traveling at cruise velocity $v_0 = 1.5$ m/s with maximum deceleration $a_{\max} = 2.0$ m/s², the physical braking distance alone consumes $d_{\text{brake}} = 0.5625$ m. Under remote teleoperation over wireless links, the total delay before an operator's emergency command takes effect includes video capture and decode ($T_{\text{video}} \approx 35$ ms), human neuromuscular reaction time ($T_{\text{human}} \approx 180$ ms), and actuator mechanical braking lag ($T_{\text{act}} \approx 7$ ms). Across an extended detection horizon $D_{\text{horizon}} = 0.999$ m (allowing a total reaction and braking time $T_{\text{total}} \leq (D_{\text{horizon}} - d_{\text{brake}})/v_0 = (0.999 - 0.5625)/1.5 = 291$ ms), the round-trip network transport ceiling is:

$$T_{\text{RTT}} \leq T_{\text{total}} - (T_{\text{video}} + T_{\text{human}} + T_{\text{act}}) = 291 \text{ ms} - (35 + 180 + 7) \text{ ms} = 69 \text{ ms} \approx 70 \text{ ms}$$

Under the example horizon, reaction, and braking assumptions, an RTT above about 70 ms exhausts the allocated operator-response budget. The onboard enforcer should revoke stale teleoperation commands and apply its feasible local braking response before that margin is lost.

Checkpoint 8.5: Watchdog freshness and teleoperation

Before evaluating the cross-subsystem handoffs matrix, verify your understanding of physical watchdog leases and latency bounds:

- ☐ **Kinematic lease sizing:** Can you explain how physical stopping distance and sensor clearance envelopes establish hard upper bounds on watchdog validity leases?

- ❑ **Administrative vs. physical authority:** Can you distinguish administrative priority in human teleoperation from physical execution authority enforced by bare-metal invariant checkers?
- ❑ **Network round-trip boundaries:** Can you describe why exceeding the allocated network-delay budget requires a local response, and what clearance and brake assumptions make stopping feasible?

8.9 The Handoffs Matrix

The nervous system governs five canonical handoffs bridging physical mechanics and computational inference. Upstream, sensing transfers raw transducer signals to perception pipelines. Perception yields intermediate representations that fuse into an estimated world state. The state feeds the learned policy to produce an action plan. The plan submits candidate trajectories to the safety enforcer. Finally, the approved command passes from the enforcer to the physical actuators. Across each boundary, five physical limits govern feasibility: latency (τ), bandwidth (β), thermal energy ($\mathcal{E}$), temporal determinism (σ_t), and fault domain isolation ($\mathcal{F}$). These two axes form a five-by-five matrix that maps every signal path to the physical budgets of the machine.

This matrix synthesizes the foundational pillars of Part I:

The Brain rows ($\mathcal{H}_1, \mathcal{H}_2, \mathcal{H}_3$) concern the computational and sensory properties that chapter 6 established previously. At the sensing-to-perception boundary ($\mathcal{H}_1$), bandwidth and latency bind simultaneously as image sensors stream raw 4K pixels over high-speed SerDes lanes, where the sensor photon integration window establishes an immutable lower bound on image age before pixel serialization begins. Between perception and state estimation ($\mathcal{H}_2$), and state and planning ($\mathcal{H}_3$), energy and latency form the primary binding constraints. Neural network inference on edge tensor accelerators demands tens of joules per forward pass, generating heat within an enclosed chassis, while deep transformer backbones and diffusion policies incur tens of milliseconds of tokenization and denoising latency.

The Body row ($\mathcal{H}_5$) inherits the unyielding physical mechanics established in chapter 4. At the terminal handoff from the enforcer to the actuators, temporal determinism and thermal dissipation bind. Building on the actuator thermal limits and torque mechanics established in chapter 4, brushless motor drivers require cycle-deterministic field-oriented current loops (50 μ s at 20 kHz) over hard real-time fieldbuses to prevent torque ripple. Meanwhile, continuous current demands dissipate resistive heat ($I^2 R$) directly into stator windings and power MOSFETs under strict thermal clocks.

The Nervous System row ($\mathcal{H}_4$) is the architectural bridge that ties the brain and body together. Embodying the proposal-permission interface of chapter 2, the boundary between the learned planner and the enforcer carries an action proposal (p_t), which consists of a compact spline parameterization or waypoint chunk. For the compact payload assumed in this grid, $\beta < 100$ kB/s is budgeted; actual bus contention and delivery time still need verification. Instead, this boundary binds strictly on temporal determinism (σ_t) and fault domain isolation ($\mathcal{F}$). The nervous system isolates stochastic brain outputs with nonblocking mailboxes and expiring leases. In the worked AMR case, 87.6 ms is the total pre-brake delay budget, so the lease is shorter after detection and actuation margins; stopping still depends on sensed clearance and available braking. Candidate-control checks may use **Control Barrier Functions**²³ before granting permission to energize physical mass.

From the causal boundary perspective framed in section 2.2, when a physical AI system fails, the breakdown manifests as an unbudgeted limit crossing at a specific handoff cell rather than an abstract deficiency in learned intelligence. A handoff diagnosis must compare like evidence. For the illustrative $\mathcal{H}_2$ state-estimation path, suppose engineers allocate a 30.0 ms latency budget and measure $P_{99} = 24.5$ ms on a baseline hardware-in-the-loop run: sampled margin is +5.5 ms. After thermal throttling, a measured $P_{99} = 85.0$ ms gives −55.0 ms against that design budget. The decision is to reduce or reallocate workload and retain independent enforcement while revalidating. A P_{99} sample diagnoses the regression; it is not a worst-case timing proof.

The example allocation in table 8.4 marks which limit is design-limiting at each handoff. Its numbers are scenario requirements or assumptions until a derivation, measurement, or bounded execution analysis supplies the stated evidence type.

²³ | **Control Barrier Functions (CBFs):** Under their stated dynamics and feasibility assumptions, Control Barrier Functions can establish forward invariance of a specified set $\mathcal{C}$ by evaluating the affine inequality $\dot{h}(\mathbf{x}, \mathbf{u}) + \alpha(h(\mathbf{x})) \geq 0$. Under a valid dynamics model, state estimate, input bounds, and feasible control authority, a candidate that violates this condition can be rejected or adjusted before actuation. For the formal mathematical proofs and relaxation formulations, see Appendix B.

Table 8.4: Master 5x5 handoffs-by-limits synthesis grid for Physical AI systems: The foundational architectural matrix mapping five canonical signal handoffs ($\mathcal{H}_1\text{--}\mathcal{H}_5$) across the Brain (chapter 6), Body (chapter 4), and Nervous System (chapter 8) against five physical limits ($\tau, \beta, \mathcal{E}, \sigma_t, \mathcal{F}$). Cells marked *BINDING* are the example design budgets that drive selection; they are not universal physical constants or verified bounds. *Non-binding* means only that the stated scenario budgets that dimension with margin, subject to measurement and fault analysis. This matrix serves as the diagnostic rubric across Part I, formalizing why safety and determinism are properties of isolated hardware boundaries, not software conventions.

Canonical Handoff Boundary	Latency Limit (τ)	Bandwidth Limit (β)	Energy & Thermal Limit ($\mathcal{E}$)	Determinism Limit (σ_t)	Fault Domain Isolation ($\mathcal{F}$)
$\mathcal{H}_1$: Sensing $\rightarrow$ Perception (Brain)	BINDING $\tau \leq 16.7$ ms (exposure & readout sets sensor age)	BINDING $\beta \geq 1.2$ GB/s (multi-camera 4K uncompressed streams)	<i>Non-binding</i> $P < 3$ W (sensor head dissipation isolated)	BINDING $\sigma_t \leq 1.0$ ms (frame-sync triggers prevent skew)	<i>Non-binding</i> Defended by SerDes hardware & DMA ring buffers
$\mathcal{H}_2$: Perception $\rightarrow$ State (Brain)	BINDING $\tau \leq 30.0$ ms (visual-inertial odometry & feature window)	<i>Non-binding</i> $\beta \approx 10\text{--}50$ MB/s (compressed keypoints & voxels)	BINDING $P \approx 15\text{--}35$ W (edge vision backbone accelerator envelope)	<i>Non-binding</i> Defended by sliding-window state estimator buffers	BINDING Spatial memory isolation: MPUs prevent memory corruption
$\mathcal{H}_3$: State $\rightarrow$ Plan (Brain)	BINDING $\tau \leq 100.0$ ms (autoregressive tokenization & diffusion horizon)	<i>Non-binding</i> $\beta < 5$ MB/s (low-dimensional state & prompt embeddings)	BINDING $P \approx 30\text{--}75$ W (transformer/diffusion matrix accelerators)	<i>Non-binding</i> Defended by receding-horizon chunking & spline buffers	BINDING Sandboxed policy container prevents host panics halting telemetry
$\mathcal{H}_4$: Plan $\rightarrow$ Enforcer (Nervous System)	BINDING $\tau \leq 5.0$ ms (intent lease validation & CBF projection deadline)	<i>Non-binding</i> Defended by compact parameterized splines ($\beta < 100$ kB/s)	<i>Non-binding</i> $P < 0.5$ W (fixed-point constraint evaluation on MCU)	BINDING $\sigma_t \leq 100\mu\text{s}$ (cycle-deterministic packet reception on MCU)	BINDING Hardware boundary: private bus, separate oscillator, isolated rail
$\mathcal{H}_5$: Enforcer $\rightarrow$ Actuator (Body)	BINDING $\tau \leq 1.0$ ms (servo interval 1.0 ms; PWM loop 50 μs)	<i>Non-binding</i> $\beta \approx 1\text{--}5$ Mbps (cyclic PDOs over deterministic fieldbus)	BINDING I^2R copper winding dissipation, MOSFET switching losses	BINDING $\sigma_t < 10\mu\text{s}$ (hard real-time PWM phase alignment & FOC)	BINDING Galvanic isolation, out-of-band analog E-stops, STO circuits

To ensure these budgets survive the lifecycle of the machine, every boundary condition in the grid is recorded in an engineering notebook. The notebook serves as the verifiable contract between hardware designers, systems engineers, and model developers. Each entry in the notebook follows the strict schema detailed in table 8.5, establishing mandatory fields for physical dimension, SI units, bound category, provenance, and verification timestamps.

Table 8.5: Abstract data schema for engineering notebook records (EngineeringNotebookRecord): Standard schema for documenting handoff boundary limits, physical units, provenance, and verification state across all machine subsystems.

Field Name	Scalar Type	Physical SI Units	Structural Invariant & Description
signal_id	string	dimensionless	Unique alphanumeric identifier (e.g. SIG_VIO_01, SIG_TRQ_AX1).
source_subsystem	string	dimensionless	Originating hardware or software module (Camera_SerDes, Planner_VLA).
destination_subsystem	string	dimensionless	Receiving hardware or software module (State_Estimator, Safety_MCU).
physical_dimension	enum	dimensionless	Target limit category ($\tau, \beta, \mathcal{E}, \sigma_t, \mathcal{F}$).
nominal_value	float64	SI units (ms, MB/s, W, μs)	Value plus operating condition; identify whether assumed, measured, or bounded.
bound_category	enum	dimensionless	Classification: DESIGN_REQUIREMENT, HARD_MECHANICAL_INVARIANT, BOUNDED_WCET, or STATISTICAL_P99.

Field Name	Scalar Type	Physical SI Units	Structural Invariant & Description
provenance	string	dimensionless	Derivation origin (ANALYTICAL_PROOF, PHYSICS_SIM, HIL_TESTBENCH).
verification_timestamp	uint64_t	UTC timestamp	Epoch time of physical bench falsification and certification sign-off.

For example, a concrete handoff entry for a visual-inertial state estimation pipeline is recorded as:

[Signal ID: SIG_VIO_01 | Path: Camera_SerDes → State_Estimator
 | Observed: $P_{99} = 24.5$ ms | Design target: $P_{99} \leq 30.0$ ms
 | Provenance: HIL Bench Profiling | Status: BASELINE VERIFIED]

Provenance explicitly records whether a number originates from an analytical first-principles calculation, a physics simulation, or direct hardware measurement on a physical test bench. When a subsystem is revised, the notebook updates the entry, creating a versioned record that preserves historical measurements and links the revision to the modified hardware or firmware. Later chapters in this text build upon this exact schema, adding domain-specific fields for policy distributions, safety integrity levels, and formal safety cases without restating or redefining the core data structure.

The grid is a design worksheet. Before relying on a handoff for physical permission, engineers must record which limit is a design requirement, measured distribution, or proven bound, then validate the integrated response and fallback.



Checkpoint 8.6: The handoffs matrix and physical budgets

Before reviewing the humanoid crucible and engineering fallacies, verify your understanding of system-wide interface contracts:

- ☐ **Signal handoff classification:** Can you identify the five canonical handoffs ($\mathcal{H}_1$ – $\mathcal{H}_5$) bridging physical plant mechanics to high-level deliberative reasoning?
- ☐ **Binding physical constraints:** Can you explain why sensory handoffs bind on bandwidth and exposure latency, whereas actuation handoffs bind on temporal determinism and thermal dissipation?
- ☐ **Engineering notebook provenance:** Can you describe why every numerical limit in an autonomous system must record its derivation origin, physical units, and testbench verification status?



Systems Perspective 8.5: Fieldbus jitter in kinematic trees

In a humanoid robot, the nervous system's physical and temporal routing constraints reach their extreme:

- **Fieldbus Daisy-Chains across Articulated Limbs:** A typical humanoid distributes thirty to fifty joint servo drives across high-degree-of-freedom limbs. Running point-to-point Ethernet or dedicated cabling to every joint creates an impractically thick, stiff wiring harness that resists joint articulation and fatigues to failure under repeated cyclic bending. Consequently, humanoids route communication through daisy-chained industrial fieldbuses (EtherCAT, CAN-FD, or RS-485 rings) traversing the pelvis, shoulders, and ankles. In an EtherCAT ring with forty slave nodes, packet traversal delays accumulate. If frame forwarding jitter or cyclic synchronization (Distributed Clocks) slips by more than $50 \mu\text{s}$, phase lag ($-\omega\tau$) directly degrades the phase margin of 1 kHz whole-body balance control loops, causing standing limit-cycle oscillations (jitter-induced shaking).
- **Multi-Rate Asynchrony between Vision and Balance:** Whole-body operational space controllers require continuous floating-base state estimates at 1 kHz, fed by high-rate

IMUs and joint torque sensors. Meanwhile, visual-inertial odometry (VIO) and 3D perception pipelines output poses at 30 to 60 Hz with 30–60 ms of compute latency. If the nervous system naively locks the balance loop waiting for updated visual poses, the floating-base controller drops cycles, leading to immediate bipedal collapse. The nervous system must enforce strict lock-free seqlock or triple-buffered mailboxes, allowing the 1 kHz balance thread to integrate IMU kinodynamics continuously while smoothly fusing low-rate visual corrections through asynchronous Kalman updates.

- **Transient Ground Bounce and Packet Corruption:** During dynamic jumping or running, high phase currents switching in close proximity to unshielded fieldbus lines within cramped limb casings induce transient electromagnetic interference (EMI) and ground bounce ($\Delta V = L_{\text{gnd}} \frac{di}{dt}$). If parity checks or cyclic redundancy checks (CRC) drop corrupted packets during a high-speed foot strike, the joint controllers must deterministically hold last-commanded torques or transition to local impedance damping within a single fieldbus cycle (1 ms) rather than freezing or faulting into uncontrolled limpness.

8.10 Fallacies and Pitfalls

A block diagram draws a logical line between high-level deliberation and low-level enforcement, but physical hardware often connects them invisibly through shared power rails, interrupt lines, or back-channel diagnostic interfaces. These physical couplings dictate whether the safety loop remains standing when a complex subsystem fails.

Pitfall: *Powering real-time safety microcontrollers from shared, unbuffered compute power rails.*

A quadruped powers its neural accelerator and safety microcontroller from the same battery bus. During a leap, an inference burst coincides with high actuator current. Resistance and wiring inductance pull the supply down to 17.65 V, below the controller regulator’s 18.0 V threshold, resetting the safety loop during torque delivery. The transient calculation in Appendix C explains how a brief current step can cause this failure. Logical software separation does not preserve enforcement when a physical voltage sag (a brownout) starves both processors simultaneously. The controller’s supply needs regulation and buffering or isolation sufficient to survive the specified compute and actuator transients.

Pitfall: *Servicing a safety watchdog inside a high-priority timer interrupt rather than an application-verified safety loop.*

A manipulator’s watchdog lease boundary disconnects actuator power if it receives no service pulse within 10.0 ms. Engineers refresh it from a high-priority timer interrupt, but the lower-priority enforcement task deadlocks. The interrupt continues firing every 1.0 ms even though the task has frozen and stopped checking torque limits, clearance, and intent leases. The watchdog therefore continues reporting a valid heartbeat while the application has silently lost its safety function. This was the exact failure mechanism uncovered in the 2013 *Bookout v. Toyota* unintended acceleration trial (Barr 2013), where RTOS task death left the vehicle throttle wide open while timer ticks continued to pet the hardware watchdog. Its service pulse must depend on successful, sequential completion of the required enforcement checks (a windowed watchdog pet conditioned on a task completion token). An independent timer tick establishes that hardware interrupts still cycle, not that the controller has safely evaluated the machine’s physical state.

Pitfall: *Leaving auxiliary diagnostic or calibration interfaces unmediated in production operation.*

Reflecting the proposal-permission discipline of chapter 2, consider a forklift that validates velocity proposals through an isolated controller, yet whose motor drives retain an unmonitored serial calibration interface. A diagnostic daemon uses that interface to write a 35.0 A test value directly into an inverter register. The proposal mailbox still contains zero velocity, so checking that mailbox does not reveal the bypassed command. The physical machine can suddenly produce violent torque without the normal motion checks ever seeing or approving the command. True actuation authority includes every physical wire or digital interface capable of changing a power-stage register. Service

interfaces therefore need operational interlocks or equivalent enforcement that prevents calibration writes from bypassing the controller during normal operation.

Fallacy: *Evaluating safety barrier invariants on commanded trajectory setpoints is equivalent to evaluating physical plant state.*

A manipulator evaluates a control barrier function (Ames et al. 2019) using commanded joint positions instead of load-side encoder measurements to reduce bus traffic. When a gear fails and the link moves away from its commanded trajectory, the interpolator continues producing valid setpoints. The enforcer still calculates that all is safe ($h(\mathbf{q}_{\text{cmd}}) > 0$), even as the physical link violently enters a forbidden region. Computing the wrong input faster cannot correct it. Under the causal boundary formulation of section 2.2, safety enforcement must evaluate physical state measurements through a dependable sensory channel and account for their limitations; inspecting the command stream merely establishes what the planner intended, not where the heavy machinery actually is.

Fallacy: *Nominal, fault-free operating logs are sufficient to prove physical architectural isolation.*

A mobile manipulator completes normal trials without a missed enforcement deadline. Those logs merely show behavior under a sunny-day workload; they do not expose the hidden physical couplings that only emerge during catastrophic faults. Qualification therefore injects power transients, floods communication buses, corrupts policy memory, and locks a shaft on a dynamometer. Each experiment must check an explicit response bound, such as whether locked-rotor detection clamps current within 2.0 ms, rather than treating the absence of a crash as success. Evidence for isolation comes from exercising the claimed fault boundaries and measuring continued enforcement or the required safe transition.

8.11 Summary

The real-time nervous system establishes the deterministic architectural bridge between unprivileged neural policies and physical actuators by enforcing microsecond timing budgets over synchronized real-time fieldbuses. Isolating actuator write privilege lets an independent enforcer reject detected limit violations during compute stalls or bad proposals, provided its sensing, timing, model, and fallback remain valid.

The architecture constructed across Part I establishes the mechanical and computational boundary conditions for physical autonomy. Integrating the causal boundary (section 2.2) with the physical plant constraints of section 4.1, the nervous system sits between the unverified brain and the physical body, operating as a deterministic arbiter on isolated hardware. The brain is unverified and statistical; the nervous system is bounded and predictable. It measures plant telemetry from an independent path and checks candidate actions against specified thermal, electrical, and kinematic limits before granting actuator writes.

The central rule governing this three-part division is that proposal is never permission (a modern instantiation of the subsumption architecture (Brooks 1986a)). A learned policy never drives motor windings or opens hydraulic valves directly. Instead, the policy submits candidate trajectories through time-bounded intent leases—temporary permits that automatically expire if the brain crashes or hangs—treating high-level compute as an untrusted client requesting access to physical actuators. Execution rates across the machine follow directly from physical time constants rather than computational convenience. For the illustrative actuator and phase-margin budget in this chapter, a 20 kHz current loop and 1000 Hz joint loop are selected; other plants require their own timing derivation. By contrast, a transformer-based vision-language-action model running on an accelerator may take 50 ms to process an image pair and output an action chunk. The nervous system bridges this three-order-of-magnitude frequency disparity by interpolating valid chunks into smooth reference trajectories, monitoring tracking error, and requesting a feasible local response when the lease expires or the physical state diverges.

Physical boundaries, timing contracts, and arbitration hardware contain failures only to the extent that their assumptions hold in the integrated plant. The nervous system must be validated under compute delays, bus faults, bad proposals, sensor errors, and loss of braking authority.



Key Takeaways: The real-time nervous system and determinism

- **Proposal is never permission:** High-level learned models act strictly as unprivileged proposal engines (p_i). Physical write privilege over motor drive registers is reserved exclusively for the deterministic real-time enforcer on isolated safety silicon.
- **Cadence is dictated by physics, not software:** The example rates (20 kHz current regulation, 1000 Hz joint checks, 20–50 Hz action chunking, and slower deliberation) are selected against winding time constants, phase margin, exposure, workload, and response budgets; they are platform dependent.
- **Nonblocking mailbox attempts need a freshness limit:** The single-slot seqlock reader makes one bounded attempt and retains the last validated chunk on a collision; a pre-empted writer can leave it stale until the lease expires.
- **Structural timing guarantees outrank empirical distributions:** A statistical latency percentile (P_{99}) cannot prove real-time safety. Deterministic certification requires analytical Response Time Analysis ($R_i = C_i + B_i + I_i \leq D_i$), eliminating unbounded bus contention and memory allocations from the safety path.
- **Hardware isolation:** The proposed implementation separates execution, timing, power, and memory paths; its fault-containment claim depends on the validated independence of those paths.
- **Stopping dynamics dimension watchdog horizons:** In the AMR example, 87.6 ms is the ideal total pre-brake delay budget. A shorter lease must reserve detection and brake-engagement time, and a stop remains conditional on measured clearance and available deceleration.
- **The 5x5 synthesis matrix:** The illustrative Handoffs-by-Limits Grid organizes design budgets and evidence for five signal handoffs ($\mathcal{H}_1\text{--}\mathcal{H}_5$) against five physical limits ($\tau, \beta, \mathcal{E}, \sigma_t, \mathcal{F}$).

When control software delegates safety directly to statistical neural policies, any operating system preemption or memory stall instantly degrades into uncommanded kinetic drift. Safety depends on the complete measured plant, including the independent permission path and a feasible physical response to faults.



What's Next: From machine anatomy to closed-loop physical learning

With the physical body (chapter 4), the cognitive brain (chapter 6), and the real-time nervous system of this chapter now specified, **Part I (The Machine Anatomy)** has assembled the structural triad of the embodied machine. Yet assembling the container does not confer operational competence: an embodied foundation model does not arrive knowing how to grasp unfamiliar geometries, clear dynamic obstacles, or recover from slipped footfalls. In **Part II (Teaching the Machine)**, chapter 10 opens our investigation into closed-loop physical learning by examining what physical interaction datasets represent, how teleoperation interfaces filter demonstration distributions, and why mechanical wear and coverage limits bound the empirical foundations of physical AI.



TEACHING THE MACHINE

Part II

Teaching the Machine

IN an offline machine learning benchmark, the dataset is fixed before evaluation. A classifier reads held-out examples drawn from a specified data distribution ($x \sim P_{\text{data}}$); its prediction does not change the next stored example. This useful testing assumption says nothing about a deployed model whose outputs affect people, later data, or physical state.

For a physical machine, the states encountered during execution depend on the actions it actually takes. An accepted motor command changes the plant and can change the next observation ($s_{t+1} \sim P(s | s_t, a_t)$, with a_t the executed action). The data distribution is therefore **endogenous** to the closed loop. Training must account for states the learned policy may visit, including recovery states absent from expert-only demonstrations.



Principle 4: Endogenous feedback and distribution shift

Invariant: Executed actions can change later observations ($s_{t+1} \sim P(s | s_t, a_t)$), so an expert-only dataset may omit states visited by the learned policy. Let π be the learned policy, π^* the expert, $J(\cdot)$ expected cumulative task cost over horizon T , and ϵ the probability that π disagrees with π^* on expert-visited states (0–1 action loss). With per-step task cost bounded in $[0, 1]$, behavioral cloning has the worst-case upper bound (Ross et al. 2011):

$$J(\pi) - J(\pi^*) \leq T^2 \epsilon.$$

Implication: The quadratic term is a bound that can be tight for some problems, not a trajectory law for every robot. Training can address the state-distribution gap by obtaining expert labels on learner-visited states (as in Dataset Aggregation, DAgger), collecting recovery data through safe perturbations, or learning from validated physical or simulated interaction.

Part II guides students through the three stages of teaching an embodied machine:

- **Data as Physical Experience (chapter 10):** Repeated physical demonstrations wear transmissions, heat motor windings, and use battery energy and operator time. We examine teleoperation and synchronized sensor logging, then distinguish four command taps: requested, mapped, permitted, and measured action. Expert labels or corrective demonstrations on learner-visited states can expose recovery behavior missing from expert-only logs (Ross et al. 2011).
- **Policy Training and Coverage Gaps (chapter 12):** We compare behavioral cloning, offline reinforcement learning, and simulation-to-real transfer against the recovery states each training method leaves uncovered. When the same observation admits distinct valid actions, direct mean-squared-error regression can average two safe paths into an unsafe one. Diffusion Policy models alternative action sequences (Chi et al. 2024); Action Chunking with Transformers (ACT) predicts short sequences of future actions, with temporal ensembling to combine overlapping predictions (Zhao et al. 2023). The real-time controller remains responsible for interpolation and actuator permission. We then examine friction hysteresis, compliance, and motor phase lag in transfer to hardware.
- **Evaluation as Closed-Loop Reality (chapter 14):** Offline error on fixed examples does not determine survival after the policy changes which states it visits. Contact, obstacles, and

recovery behavior can amplify errors in task-dependent ways. We evaluate the integrated machine through closed-loop trials, exact binomial confidence bounds, and analysis of rare failures and the physical cost of testing.

Principle 5: Multimodal trajectory integrity

Invariant: Some tasks permit distinct valid actions from the same observation. If demonstrations pass an obstacle on either side, a single mean-squared-error prediction can point into the obstacle between them. This failure depends on the task, observation, and demonstration distribution.

Implication: When distinct action modes matter, a conditional generative policy can represent alternatives: Diffusion Policy can sample alternative action sequences (Chi et al. 2024). ACT predicts action chunks and combines overlapping predictions through temporal ensembling to support temporal consistency (Zhao et al. 2023). The downstream controller must still check each proposed action against physical limits.

Principle 6: Closed-loop validation

Invariant: Error on fixed, expert-visited examples alone does not bound closed-loop survival (R_{closed}). A learned action may move the machine into states absent from that test set; the effect on failures depends on the plant, environment, and independent permission path.

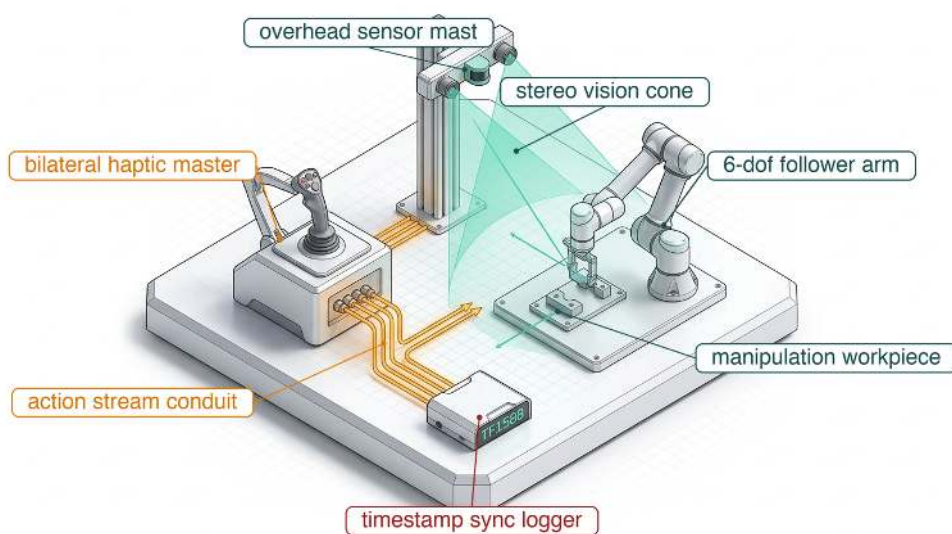
Implication: Evaluate the integrated system in closed-loop trials across its stated operating conditions, recording interventions and failures. Exact binomial confidence bounds describe what a finite trial count supports; tail analysis and runtime safeguards address risks that average test accuracy cannot establish.

Part II teaches students to collect physical experience, train action policies for the states they may encounter, and evaluate the resulting closed loop.

Physical Data

DRAFT

Physical Data



- 10.1 Endogenous Experience
- 10.2 Where Physical Data Comes From
- 10.3 Teleoperation Demonstration Costs
- 10.4 Collection Policy Coverage
- 10.5 Intervention and Recovery Data
- 10.6 Cross-Embodiment Transfer
- 10.7 How Datasets Go Wrong
- 10.8 The Dataset Schema
- 10.9 Fallacies and Pitfalls
- 10.10 Summary

Purpose

What does the physical machine that collected a dataset leave permanently stamped into the policy that trains on it?

A robot trajectory reflects the machine's behavior. Joint geometry and friction shape the motion, while sensor timing determines which observation accompanies each command. Training on those records also teaches a policy the consequences of these collection conditions. Successful demonstrations may omit recovery behavior needed when an error diverts the robot from the path. The policy then encounters unfamiliar observations, and its next action can increase the deviation. More successful runs do not close that gap. Transferring the data to another machine raises a related question, because similar-looking bodies can respond differently to the same command. Physical data must therefore preserve the relationship between observations, actions, and the collecting machine's response.

Brain

1–50 Hz
foundation models, SLAM, planning

above: propose; below: command

Nervous System

1000 Hz
TSN, intent leases, lock-free IPC

Body

continuous
real-time MCU, safety filters, CBFs

Boundary

continuous
sensors, actuators, kinematics

Physical World

continuous
contact mechanics, friction, mass



Learning Objectives

- Estimate usable demonstration yield from collection time, reset overhead, and mechanical wear
- Analyze how the collecting policy limits dataset coverage and downstream recovery behavior
- Separate failed autonomous actions from human recovery trajectories in timestamped intervention records
- Evaluate whether a dataset's mechanical and sensing assumptions transfer to another machine
- Design dataset records linking observations with requested, mapped, enforced, and measured actions
- Assess whether recorded experience supports the operating conditions claimed for a learned policy

10.1 Endogenous Experience

¹ | **Causal Boundary:** The causal boundary between computation and physics was established in section 2.2. In an embodied system, computation acquires physical consequence because state evolution depends causally on motor actuation. Consequently, policy actions actively perturb the distribution of future observations ($s_{t+1} \sim P(s | s_t, a_t)$), invalidating the independent and identically distributed (i.i.d.) assumptions of disembodied machine learning.

² | **Endogenous Covariate Shift:** Executed actions can move the agent outside demonstrated states. The behavioral-cloning bound derived by Ross et al. (2011) allows a quadratic worst case under its stated loss and cost assumptions, though actual drift depends on the plant, task, policy, and safeguards. Expert labels on learner-visited states are one way to address this gap.

³ | **Hardware Fatigue Life:** Actuator thermal limits, Joule heating in motor windings, and gearbox friction hysteresis constrain physical collection campaigns, as detailed in section 4.4 and section 4.5. Sustained high-torque demonstrations degrade mechanical transmissions and alter joint friction profiles over time. Consequently, machine learning pipelines that assume stationary plant dynamics encounter unmodeled physical drift as collection testbeds age.

An autonomous mobile manipulator can grasp objects in a hundred training demonstrations but drift during its first physical deployment. One possible cause is *endogenous experience*.¹ In a fixed offline vision dataset, the model's prediction does not change the next stored example. Here, an executed action changes the body's trajectory, camera pose, and contacts, thereby changing later observations ($s_{t+1} \sim P(s | s_t, a_t)$).



Definition 10.1: Endogenous covariate shift

Endogenous covariate shift occurs when an embodied agent's executed actions alter its later physical states and observations, moving it into regimes underrepresented in its training data. For behavioral cloning, the resulting cost gap can have a worst-case upper bound proportional to $T^2\epsilon$ under the assumptions below; this is not a trajectory law.

1. **Significance:** Explains how offline behavioral cloning can fail during closed-loop physical execution: a perturbation may drive the system into states where the policy lacks training support.
2. **Distinction:** Unlike exogenous dataset shift caused by changing ambient lighting or seasonal weather, endogenous shift is actively generated by the agent's own closed-loop actions altering its subsequent observation distribution.
3. **Common pitfall:** Attempting to fix closed-loop policy failure by collecting more nominal expert demonstrations, which only increases data density along the success manifold without providing the corrective recovery data needed to escape boundary excursions.

Suppose the archive contains only flawless executions and no recovery trajectories. A mistake can then move the robot outside demonstrated states. Let π be the learned policy, π^* the expert, J expected cumulative task cost over T steps, and ϵ the probability of disagreement on expert-visited states under 0–1 action loss. If per-step task cost lies in $[0, 1]$, behavioral cloning admits the worst-case bound (equation 10.1):²

$$J(\pi) - J(\pi^*) \leq T^2\epsilon. \quad (10.1)$$

More flawless demonstrations do not resolve this failure mode. Adding more expert paths merely deepens the probability density along the center of the corridor while leaving the surrounding boundary excursions completely unsupported.

The physical cost of gathering this experience compounds the challenge. In digital domains, acquiring another million tokens costs mere fractions of a cent in cloud compute. In Physical AI, every recorded second consumes finite actuator fatigue life, heats motor stator windings, cycles gearbox teeth, scrubs tire treads, and drains battery capacity.³ Scheduled facility time on an instrumented

robotic workcell or vehicle proving ground yields only a fraction of active motion data. Human teleoperation fatigue, safety interlock trips, manual fixture realignment, and physical scene resets consume over 70 percent of available clock time. Physical demonstration data is therefore not only scarce and expensive, but structurally perilous if logged naively.

Prerequisite: Actuator thermal dissipation (dT/dt) and winding damage limits are derived in section 4.5.

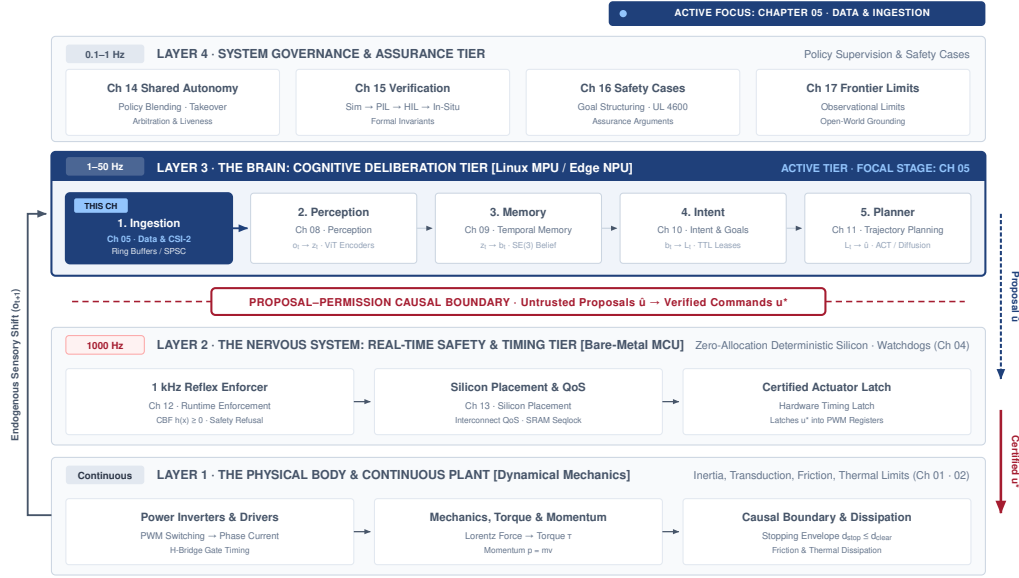


Figure 10.1: Data ingestion stage: The four-tier physical AI systems architecture highlights the cognitive deliberation tier, with ingestion as the first of five brain stages and this chapter's active focus. Every later stage inherits whatever ingestion recorded, so the wall-clock cost of moving a physical body through the world, not storage capacity, bounds what the rest of the brain can learn.

Within the four-tier physical AI architecture (figure 10.1), the **Data Ingestion** stage sits at the structural interface between the cognitive deliberation tier (the Brain, operating at 10 to 50 hertz), the real-time safety and timing tier (the Nervous System, operating at 1000 hertz), and the continuous plant (the Body). Ingestion is the gateway through which real-world physical dynamics enter the software pipeline. Every downstream cognitive stage—perception, memory, intent, and planning—inherits the coverage, biases, and omissions of the recorded archive.

Critically, a physical command cannot be logged as an isolated scalar or ungrounded vector of floating-point numbers. In an embodied system governed by the proposal-permission architecture,⁴ an action changes semantic meaning as it traverses the control stack. An operator's unconstrained joystick motion on an application processor is a raw proposal (a_{req}); kinematic mapping transforms it into a Cartesian setpoint (a_{cmd}); a real-time microcontroller safety governor clamps it to prevent barrier violations (a_{enf}); and the physical plant returns a measured response in a specified frame and time interval (a_{meas}), with motor current logged separately. Logging only the requested proposal injects dynamically infeasible demands into training loss, while logging only the measured plant response teaches the downstream policy to mimic structural deflection and gear backlash as though they were deliberate human skill. Preserving causal integrity requires logging the complete multi-tier action tap tuple across the computing boundary.

The chapter traces collection sources and their costs (section 10.2, section 10.3), then tests what the resulting data covers (section 10.4). Intervention records, embodiment contracts, failure analysis, and the provenance schema carry that evidence into training and deployment decisions (section 10.5, section 10.6, section 10.7, section 10.8).

⁴ | **Proposal-Permission Architecture:** The proposal-permission architecture, introduced in section 6.1, decouples unprivileged high-latency cognitive proposals from deterministic bare-metal safety enforcement on a dedicated microcontroller. Ingesting multi-tier telemetry across this boundary prevents safety governor interventions from contaminating the demonstrator's behavioral labels. Downstream policy synthesis methods that train on these ingested records are developed in chapter 12, while real-time control barrier functions that enforce physical limits during execution are detailed in chapter 24.

10.2 Where Physical Data Comes From

A physical dataset is not found, logged from passive traffic, or assembled from a public repository. Every transition tuple in an archive required burning electrical energy, pushing against mechanical friction, and moving physical mass through a coordinate frame. The challenge of learning from physical demonstration traces back to Dean Pomerleau’s ALVINN (Autonomous Land Vehicle in a Neural Network) (Pomerleau 1989). By training a neural network to steer an autonomous vehicle directly from camera images of human driving, Pomerleau observed that the model learned only how to track nominal lane centers; because human experts rarely made errors, the dataset contained zero demonstrations of recovery trajectories from lane edges, causing compounding drift as soon as small execution errors accumulated in closed loop.

Physical data sources separate according to who chooses the action:

1. In **kinesthetic teaching**, a colocated demonstrator physically grasps and moves the unpowered or gravity-compensated joints of the mechanism.
2. In **teleoperation**, a remote operator guides an input device such as a master arm, a multi-axis space mouse, or a tracked hand controller, transmitting commands across a software interface to the follower robot.⁵
3. In **scripted collection**, an engineer executes a deterministic control routine, such as a state machine tracking interpolated splines or force-thresholded primitives.
4. In **exploratory collection**, an active policy selects actions to optimize an information-theoretic objective, sample state space, or maximize a reward signal.
5. In **autonomous logging during deployment**, the production policy commands the actuators as it performs work in a factory or warehouse, storing incoming sensor streams in situ.

For each collection source, the engineer decides which variable is recorded as the action label, altering what the learned model reproduces. In teleoperation and autonomous systems, data logging taps into multiple layers across the control pipeline (figure 10.2). On heterogeneous dual-brain platforms, such as a canonical heterogeneous dual-brain System-on-Chip (SoC) testbed or a bimanual leader-follower teleoperation workstation⁶ (Zhao et al. 2023; Wu et al. 2024), this pipeline spans two distinct computational domains:

- The unprivileged **Linux Application Processor (MPU)** captures high-bandwidth camera streams (e.g., active stereo RGB-D at 30 Hz), ingests master teleoperation inputs or neural policy forward passes, and maps coordinates.
- Extending the proposal-permission architecture from section 6.1, the real-time **Microcontroller Unit (MCU)** executes at 1000 Hz on bare metal, running low-level PID/impedance control, forward-invariant safety barrier functions, velocity clamping, and motor current limiters over high-speed CAN-FD networks or direct PWM voltage pulses.⁷ Because each domain operates on independent quartz oscillators, uncalibrated clock drift⁸ between camera frames and motor current ticks corrupts state-action alignment across extended logging runs.

The four taps along this control path capture fundamentally different physical semantics:

- The **requested action** (a_{req}): the raw operator input or policy proposal, tagged with its representation, units, frame, and issue time.
- The **mapped command** (a_{cmd}): the retargeted Cartesian or joint setpoint, tagged with the new representation, units, frame, and mapping time.
- The **enforced command** (a_{enf}): the same typed setpoint after the independent safety path has admitted, modified, or rejected the proposal.
- The **measured response** (a_{meas}): the corresponding measured position or velocity over a stated sampling interval; encoder readings and motor current are separate sensor channels, not interchangeable action coordinates.

If the logging pipeline records a_{req} prior to software limit enforcement, the dataset contains dynamically infeasible trajectories that command accelerations beyond motor current limits. If it logs a_{meas} after contact, the recorded label reflects structural compliance and gear backlash rather than commanded intent, turning a feedforward control signal into an observation of material deformation. Building on the causal boundary established in section 2.2, logging the full tuple

⁵ | Bilateral Teleoperation

Lineage: Raymond Goertz developed the first bilateral mechanical and electrical master-slave telemanipulators at Argonne National Laboratory in the late 1940s and 1950s for handling radioactive materials. His mechanical linkages introduced reflected force feedback across a physical barrier, proving that human operators require kinesthetic resistance to avoid jamming remote mechanisms. In modern robot learning, this physical coupling determines whether demonstrations capture genuine contact compliance or open-loop visual hunting motions.

⁶ | Leader Arm Implementations

GELLO uses a low-cost 3D-printed encoder leader (Wu et al. 2024); ALOHA uses backdriven WidowX robot arms with 3D-printed handles as leaders (Zhao et al. 2023). Neither implementation provides active follower-to-leader force reflection. Their mechanics and operator feel differ, so contact corrections in the collected data depend on the available vision and any separately instrumented force channel.

⁷ | Hardware Timer Interrupt

On real-time MCUs, optical encoder counts and current shunt readings are sampled via hardware timer interrupts and Direct Memory Access (DMA) to guarantee microsecond determinism. Bypassing operating system thread scheduling allows dedicated timer peripherals to latch high-frequency plant states at exact clock boundaries. This deterministic latching prevents inter-sample timestamp jitter from corrupting downstream numerical differentiation during velocity and acceleration estimation.

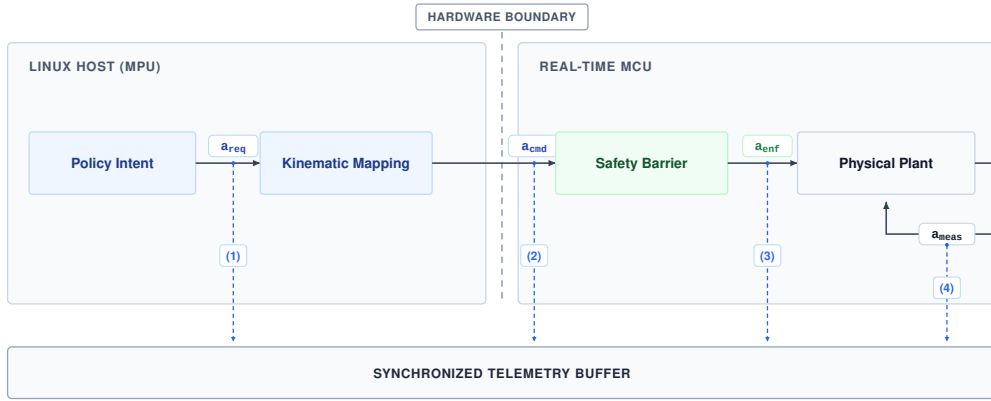


Figure 10.2: Four-stream action tap tuple: Four places a command can be logged, from raw policy intent on the application processor through to measured plant response. Which tap a dataset records decides what the model learns to imitate. Logging only the request bakes in actuator drift the robot could never execute, and logging only the measurement bakes in mechanical compliance and backlash as though they were demonstrated skill. Source: architectural design adapted from Zhao et al. (2023) and Wu et al. (2024).

$(a_{\text{req}}, a_{\text{cmd}}, a_{\text{enf}}, a_{\text{meas}})$ across the MPU/MCU boundary is necessary to distinguish operator intent from governor intervention.

Definition 10.2: The four-stream action tap tuple

Four-stream action tap tuple $(a_{\text{req}}, a_{\text{cmd}}, a_{\text{enf}}, a_{\text{meas}})$ is a time-aligned record that distinguishes raw intent, mapped setpoint, safety-enforced setpoint, and measured plant response. Each tap carries its representation, units, frame, and timestamp; comparing values requires a common representation and a defined response interval. The host/MCU split is one implementation.

1. **Significance:** Training imitation learning policies on raw requested actions (a_{req}) injects dynamically infeasible commands that trigger safety trips in deployment. Conversely, training on measured responses (a_{meas}) causes the model to imitate mechanical compliance, backlash, and external disturbances rather than intentional control. Logging all four streams disambiguates control intent from governor intervention.
2. **Distinction:** Unlike classical robot telemetry logs that capture only final motor encoder states or raw teleoperation joystick inputs, the four-stream tap preserves the exact causal filtering stages applied by intermediate software and safety hardware.
3. **Common pitfall:** Logging streams without hardware-synchronized timestamps (t_{PTP}). Even a few milliseconds of clock skew between the high-level perception host and the low-level motor MCU corrupts the temporal alignment between visual observations and corresponding torque actions.

10.2.1 Synchronous 4-stream action tap and telemetry ring-buffer protocol

To operationalize the four-stream action tap without non-deterministic logging jitter or buffer contention, the system executes a synchronized inter-core capture loop:

Input interfaces:

- Raw observation stream $\mathbf{o}_t \in \mathcal{O}$ from the host perception pipeline (active stereo RGB-D at 30 Hz, hardware timestamp t_{exp}).
- Raw operator input or policy action candidate $a_{\text{req}}(t) \in \mathcal{A}_{\text{raw}}$ from MPU shared memory (50 Hz).
- Kinematic retargeting and workspace boundary mapping $\mathcal{K} : \mathcal{A}_{\text{raw}} \rightarrow \mathcal{A}_{\text{cartesian}}$.
- A safety governor with a Control Barrier Function (CBF)⁹ (Ames et al. 2019) and velocity

⁸ | **Oscillator Clock Drift in Multi-Sensor Telemetry:** Standard quartz crystal oscillators exhibit thermal drift between 20 and 50 parts per million (ppm), causing unsynchronized sensor clocks to diverge by up to 3 ms per minute of operational logging. In high-speed trajectory capture, this uncalibrated temporal drift misaligns visual demonstration frames with joint torque telemetry, corrupting policy cloning datasets before training begins.

⁹ | **Control Barrier Function:** A CBF defines a candidate safe set and modifies control inputs to satisfy modeled constraints (Ames et al. 2019). Forward invariance requires the stated dynamics, feasible controls, timely state estimates, and a validated fallback when the set is no longer feasible. Unrecorded interventions can make governor braking appear to be operator intent.

limiter $\mathcal{B} : \mathcal{X} \times \mathcal{A}_{\text{cartesian}} \rightarrow \mathcal{A}_{\text{safe}}$ checks proposals before actuator dispatch. Forward invariance depends on the plant model, state estimate, sampling delay, and available control authority.

- Bare-metal hardware counter registers for quadrature optical encoders (measuring physical joint rotation) and phase current sensing interfaces (measuring electrical current pushing the motors).

Execution protocol (periodic 1.0 ms MCU cycle):

1. **Perception and intent ingestion (host MPU domain):** Ingest synchronized camera frame $\mathbf{o}_t$ with hardware exposure-midpoint timestamp t_{exp} and camera clock-domain ID, sample raw master input $a_{\text{req}}(t)$ with its issue timestamp and host clock-domain ID, evaluate forward kinematics $a_{\text{cmd}}(t) \leftarrow \mathcal{K}(a_{\text{req}}(t))$, and commit $\langle t_{\text{exp}}, t_{\text{issue}}, t_{\text{map}}, \mathbf{o}_t, a_{\text{req}}, a_{\text{cmd}} \rangle$ to shared memory via non-blocking hardware memory transfers, maintaining high data bus throughput without CPU-stalling latency spikes.
2. **Safety barrier evaluation and actuation (MCU domain, 1000 Hz ISR):** Latch MCU enforcement time t_{enforce} and its clock-domain ID from the hardware timer, ingest $a_{\text{cmd}}(t)$ across inter-core single-producer single-consumer ring buffers using hardware memory synchronization fences to prevent concurrent read/write data corruption, and evaluate dynamic barrier constraints:

$$a_{\text{enf}}(t) \leftarrow \mathcal{B}(\mathbf{x}_{\text{current}}, a_{\text{cmd}}(t)).$$

If $a_{\text{enf}}(t) \neq a_{\text{cmd}}(t)$, assert governor intervention flag $b_{\text{clamped}} \leftarrow \text{true}$ with clamp code κ , and dispatch $a_{\text{enf}}(t)$ to motor pulse-width modulation drives over the real-time fieldbus.

3. **Physical response capture and commit:** Sample joint encoders and motor currents at the control cycle boundary, keeping currents in plant telemetry. Derive a_{meas} in the enforced command's units and frame over the next stated response interval, then compute $\delta_{\text{track}} = \|a_{\text{enf}}(t) - a_{\text{meas}}(t + \Delta t)\|_2$ only after aligning their time bases. Assemble $\mathcal{P}_t = \langle t_{\text{enforce}}, \mathbf{o}_t, a_{\text{req}}, a_{\text{cmd}}, a_{\text{enf}}, a_{\text{meas}}, \mathbf{h}_{\text{plant}}, [b_{\text{clamped}}]$ with observation exposure midpoint, action issue, mapping, enforcement, and measured-response interval timestamps, their clock-domain IDs, and estimated synchronization error; then commit it to the telemetry ring. Size the ring and monitor drops so logging does not stall the control loop.

To see why all four taps matter, consider an operator guiding a 6-DoF manipulator toward a rigid workbench at distance $d_0 = 15$ mm along the z -axis. The measured downward velocity is initially near zero; the operator proposes a z -velocity setpoint $a_{\text{req}} = -0.40$ m/s. Mapping and enforcement also use z -velocity setpoints in the same frame and units, while a_{meas} is the encoder-derived z velocity over the following response interval. The safety path checks maximum available deceleration $a_{\text{decel}} = 2.0$ m/s² and clearance buffer $d_{\text{safe}} = 2.0$ mm. If the tool actually reached $|v| = 0.40$ m/s at this position, constant-deceleration stopping would require:

$$d_{\text{stop}} = \frac{v^2}{2a_{\text{decel}}} = \frac{(0.40 \text{ m/s})^2}{2 \times 2.0 \text{ m/s}^2} = 40.0 \text{ mm}$$

The available clearance is $d_{\text{avail}} = d_0 - d_{\text{safe}} = 13.0 \text{ mm} < 40.0 \text{ mm}$. The current-state feasibility check is $v_{\text{meas}}^2 / (2a_{\text{decel}}) \leq d_{\text{avail}}$; it holds for the initially slow tool. Under this idealized model, arriving at this position at 0.40 m/s would already be outside the feasible stopping set; even immediate maximum braking would leave approximately 0.329 m/s at the buffer boundary. For the initially slow tool, the model-based admission ceiling at this position is:

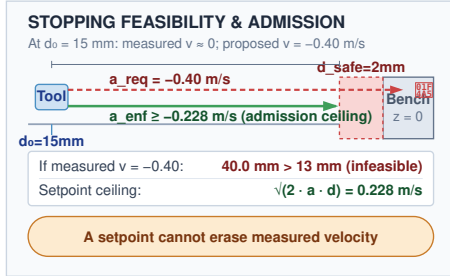
$$|a_{\text{enf}}| = \sqrt{2a_{\text{decel}}d_{\text{avail}}} = \sqrt{2 \times 2.0 \text{ m/s}^2 \times 0.013 \text{ m}} \approx 0.228 \text{ m/s}$$

The independent safety path can reject the -0.400 m/s proposal and admit a downward speed of at most 0.228 m/s as a setpoint at this instant, provided the controller tracks that ceiling and recomputes it as clearance changes. This does not instantaneously change measured velocity or by itself guarantee a stop: sensing delay, tracking error, and braking capability need reserved margin. If the measured tool were already moving at -0.400 m/s, changing its setpoint to -0.228 m/s could not restore feasibility; the controller must apply its validated maximum-braking fallback and record that the clearance guarantee was lost. Logging only a_{req} teaches an imitation policy infeasible

demands; logging only a_{enf} without intervention flags misattributes the governor's braking profile to demonstrator intent. The aligned four-tap record separates these causes.



Checkpoint 10.1: Action tap discrepancy and governor contamination



TRAINING TARGET CONTAMINATION

Causal failure modes when logging across safety governor

- Log a_{req}** Infeasible request as label
Proposes -0.40 m/s near the bench
- Log a_{enf}** Governor masking fallacy
Policy mimics $\sqrt{2ad}$ brake as human skill
- 4-Stream** Preserves causal boundary
Flags b_{clamped} ; separates intent from governor
- Mandatory: Log 4-Stream Tuple with Override Flags**

- ☐ **Action tap divergence:** Can you explain why training an imitation policy on requested actions (a_{req}) versus enforced actions (a_{enf}) leads to divergent failure modes during deployment?
- ☐ **Governor masking:** Can you describe how a safety enforcer clamping commands can inadvertently teach a policy an incorrect braking curve if intervention flags are omitted?
- ☐ **Four-stream action logging:** Can you distinguish the roles of requested, commanded, enforced, and measured action streams in post-hoc incident forensics?

10.2.2 Multi-stream asynchronous telemetry schemas and exposure midpoint timestamping

Modern physical AI systems record telemetry across fundamentally asynchronous sensor modalities operating under disparate physics, sampling clocks, and bus protocols:

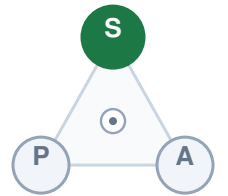
- **Active vision streams:** Multi-view RGB and depth cameras capture frames at 30 Hz (33.3 ms interval), where photons accumulate over a finite exposure window $t_{\text{exp}} \in [10, 30] \text{ ms}$, followed by sensor row readout (4–8 ms) and high-throughput data bus transfer to host memory.
- **Tactile and force-torque matrices:** Fingertip piezoresistive or optical elastomer sensors sample surface traction and normal forces at 100 Hz (10.0 ms interval) over SPI or CAN-FD buses.
- **Joint encoders and inverter telemetry:** Optical joint encoders, motor phase current shunts, and Field-Oriented Control (FOC) current loops execute deterministically on bare-metal MCUs at 1000 Hz (1.0 ms interval).
- **Policy control and action dispatch:** Generative action proposals or teleoperation commands are computed and dispatched at 50 Hz (20.0 ms interval).

Standard machine learning data loaders often employ a naive “latest available sample” latching policy: whenever the 50 Hz action loop fires, it packages the most recently arrived camera frame with the current joint angles. This naive latch introduces catastrophic temporal misalignment. If a camera frame arrives in host RAM at timestamp t , the photons that generated that image were integrated during an exposure window centered 25–40 ms in the past. Pairing the current control action a_t with an uncompensated visual observation o_t teaches the imitation policy to map historical states to instantaneous actions, baking unmodeled phase lag directly into the neural network weights.

To prevent temporal corruption across dataset schemas (such as Hugging Face LeRobot, RLDS / Open X-Embodiment (Open X-Embodiment Collaboration et al. 2024), and Foxglove MCAP), logging engines enforce the **exposure midpoint timestamping rule**:

1. **Hardware exposure midpoint timestamp:** The true physical timestamp of an optical observation o_t is latched at the midpoint of sensor exposure:

$$t_{\text{obs}} = t_{\text{trigger}} + \frac{1}{2} t_{\text{exp}}$$



The Sense axis: sub-microsecond timestamping and causal state-space logging.

where t_{trigger} is synchronized across cameras via IEEE 1588 Precision Time Protocol (PTP) hardware triggers or external TTL sync lines.

2. **Causal kinematic interpolation:** Because the real-time MCU logs joint positions and velocities at 1000 Hz (1.0 ms resolution), the true mechanical configuration at the exact camera optical timestamp is computed by smoothly interpolating between the two closest 1 kHz encoder records. For the full cubic Hermite spline calculation used in this physical interpolation, see Appendix C.
3. **Container serialization:** Modern open-source dataset containers formalize this multi-rate structure. In **MCAP**, messages preserve raw sensor arrival and log-time timestamps alongside channel schema definitions in a single zero-copy binary file. In **LeRobot** (Cadene et al. 2024), video streams are chunked into MP4/AV1 containers with frame-accurate meta-data, while synchronized 50 Hz state-action tensors are serialized into columnar Parquet or Safetensors formats. In **RLDS**, transitions are structured as nested episode dictionaries (steps/is_first, steps/is_terminal, steps/observation, steps/action). Enforcing exposure-midpoint alignment before serializing ensures that state-action pairs (s_t, a_t) share a causally sound, physically verified temporal reference.

Because the collecting entity dictates how the machine approaches the environment, kinesthetic teaching, teleoperation, unguided play, and autonomous logging occupy disjoint state-action manifolds even when attempting the exact same physical contact task. Consider inserting a cylindrical steel pin into a chamfered hole with a clearance of $50\mu\text{m}$. A colocated human guiding the tool directly senses contact reaction forces through their hands and makes instantaneous, fine-grained compliance adjustments at their own wrist joints. A remote teleoperator watching a two-dimensional video feed lacks direct haptic feedback, creating visual latency delays, command overshoot, and characteristic hunting motions as the operator pecks at the contact boundary. A scripted controller follows a programmed helical search pattern at constant linear velocity, generating a dense geometric spiral with predictable force spikes against the chamfer face. Autonomous logging from an exploratory or partially trained policy wanders across off-target regions, repeatedly binding the pin at oblique angles until motor current thresholds trigger a software fault. These logs do not represent noisy samples of a single trajectory but outputs from distinct dynamical systems with different bandwidths, latency profiles, and mechanical compliances.

A common error in physical machine learning is treating each recorded control step as an independent experience. When a joint controller logs state-action pairs at 50 Hz, one hour of continuous recording produces 180,000 steps. Multiplying sample frequency by recording time yields large step counts, but adjacent steps exhibit extreme temporal autocorrelation. Two camera frames logged 20 ms apart differ by sub-millimeter end-effector displacements and minor sensor noise. Distributional coverage scales with the count of independent task episodes and distinct physical configurations, not with the sampling clock. An archive containing 100,000 steps distributed across five hundred distinct workpiece positions and lighting conditions provides vastly broader support than 1,000,000 steps logged during a single continuous demonstration where the end-effector engaged a workpiece at a single bench location.

Evaluating the throughput of a physical collection campaign requires calculating the usable episode yield from the mechanical and operational parameters of the workcell. Consider an analytical example of an articulated manipulator collecting contact demonstrations for a precision assembly task. Let an attempt require an average duration $t_{\text{att}} = 20$ s. After each attempt, the cell must return to a valid initial state, requiring the manipulator to transit home and an automated fixture to reset the workpiece, adding a reset duration $t_{\text{rst}} = 15$ s. The total cycle time for a single attempt is $t_{\text{cyc}} = t_{\text{att}} + t_{\text{rst}} = 35$ s. In one wall-clock hour (3600 s), a single rig completes at most 102 attempts. Physical collection suffers from trial failures where workpieces slip, parts jam, or safety watchdogs trip on excessive contact force. If the trial failure rate is 25 percent, and post-collection validation rejects an additional 10 percent of logs due to camera occlusion or dropped network packets, the retention rate is:

$$R = (1 - 0.25) \times (1 - 0.10) = 0.675.$$

Prerequisite: Sub-microsecond PTP clock synchronization across distributed bus controllers is established in section 8.4.

Contrast: Teleoperation hunting delay (250 ms) contrasts with the continuous tactile reflex bandwidth (1 kHz) analyzed in section 4.2.

The usable episode yield for a single rig is:

$$Y_{\text{usable}} = \left\lfloor \frac{3600 \text{ s}}{t_{\text{cyc}}} \right\rfloor \times R = 102 \times 0.675 \approx 68.8 \text{ usable episodes per hour.}$$

If training a policy requires 5000 validated episodes to achieve target success rates across geometric variations, collecting that data on one workstation requires 72.7 hours of active execution time. Across those 72.7 hours, the machine executes 7415 physical cycles, subjecting gearbox teeth to repeated directional reversals, stressing flex cables inside articulated joints through thousands of bends, and accumulating thermal load in motor windings under sustained stall currents. Operating three parallel collection rigs increases the throughput to approximately 206.4 usable episodes per hour and compresses calendar time to 24.2 hours, but it scales hardware wear and maintenance linearly across three sets of mechanical transmissions and sensors.

Simulated trajectories generated in rigid-body physics engines avoid direct hardware wear and can generate millions of transitions per hour across distributed compute clusters. The mechanics, approximations, and domain transfer boundaries of simulated data are treated in chapter 12. When synthetic trajectories are combined with physical logs, the physical provenance of every record must be tracked explicitly. A model trained on an undifferentiated mixture inherits the contact unfaithfulness of the physics engine wherever synthetic data dominates, while inheriting the unmodeled sensor noise and mechanical backlash of the hardware wherever physical data dominates. Without tracking the collecting agent and the recording tap point for each sample, an engineer cannot determine whether an on-robot failure stems from simulation error or from physical collection bias. Human teleoperation remains the standard technique for gathering high-quality physical trajectories, yet the interface between the human and the machine shapes the recorded dynamics long before the data reaches a model.

Downstream: Usable demonstration yield (Y_{usable}) sets the sample efficiency requirements for trial-based learning in section 12.3.

10.3 Teleoperation Demonstration Costs

Human teleoperation closes a feedback loop that spans the operator, the input device, the communication bus, the low-level motor controller, and the physical environment (figure 10.3). In a typical teleoperation workstation, an image sensor captures the workspace, encodes the video stream, and transmits the frames across a network to a display console. The operator perceives visual feedback, decides on a corrective trajectory, and displaces an input device. The workstation samples the device state, maps the spatial displacement into joint or end-effector coordinates, and transmits the resulting setpoint across an industrial bus to the robot's motor drives. The actuators generate torque, accelerating the mechanical linkages against inertia, joint friction, and contact constraints. This displacement alters the scene, producing new optical signals recorded by the image sensor in the next cycle.

Each stage in this loop introduces latency that affects the timing of the correction. In an analytical reference setup with a high-speed camera and a wired local network, display processing requires $t_{\text{disp}} = 50$ ms across exposure, H.264 decoding, and monitor refresh. Human visual perception and neuromuscular transmission introduce an operator response latency of $t_{\text{hum}} = 200$ ms for continuous path corrections. Command quantization, packetization, and bus transport to the robot controller add $t_{\text{cmd}} = 10$ ms. Actuator torque ramp-up and structural inertia impose a mechanical response latency of $t_{\text{mech}} = 80$ ms before the robot achieves the commanded acceleration. The total round-trip latency is:

$$t_{\text{loop}} = t_{\text{disp}} + t_{\text{hum}} + t_{\text{cmd}} + t_{\text{mech}} = 50 \text{ ms} + 200 \text{ ms} + 10 \text{ ms} + 80 \text{ ms} = 340 \text{ ms.}$$

When a contact misalignment occurs at time t , the operator sees the error at $t + 50$ ms, issues a motor command at $t + 250$ ms, and the arm exerts corrective force against the workpiece at $t + 340$ ms. During those 340 ms, the physical system continues moving under momentum and external contact forces. If the data logger records the current sensor observation s_t alongside the human command a_t issued at that same clock instant, the recorded demonstration pairs a physical state with an action chosen in response to a state that existed 250 ms in the past. This temporal misalignment bakes an unmodeled phase lag directly into the training targets. Because the dataset pairs a past visual

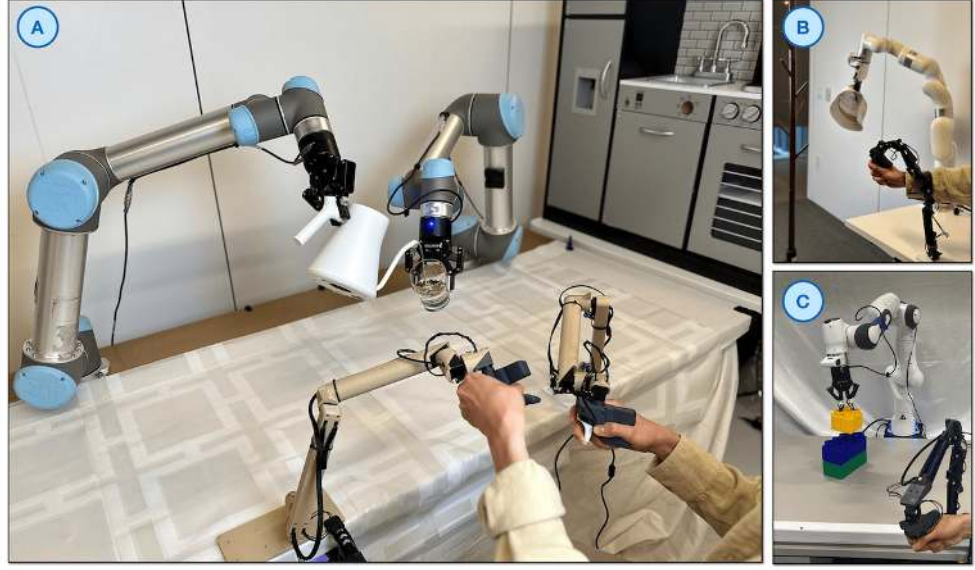


Figure 10.3: Kinematically isomorphic teleoperation rigs: Demonstration capture on three leader-follower rigs whose leader devices mirror follower kinematics (Wu et al. 2024). Because the leader shares the follower’s joint structure, the interface needs no inverse kinematics and inherits none of its singularities (mathematical dead-zones where joints align and lose mobility), preserving the operator’s direct joint-level coordination. Human reaction latency of 200–300 ms still enters the recording as phase lag, so the demonstration carries the operator’s delay as if it were policy.

state with a present physical reaction, the model learns this delayed timing as deliberate behavior, producing policies that over-correct or oscillate when deployed autonomously at 50–100 Hz.

Teleoperation round-trip latency ($t_{\text{loop}} = t_{\text{disp}} + t_{\text{hum}} + t_{\text{cmd}} + t_{\text{mech}}$) is the total elapsed time between a physical contact event and the delivered corrective actuator response, embedding biological reaction delays (200–300 ms) as artificial phase lag in demonstration timestamps. **Demonstration yield** ($Y_{\text{usable}} = \lfloor T_{\text{active}} / t_{\text{cyc}} \rfloor \cdot R$) is the fraction of scheduled facility time that converts into validated, defect-free trajectory records after deducting startup, calibration, operator rest, physical resets, and quality filtering losses.



Napkin Math 10.1: Multi-camera ingestion and storage budgets

Question: What is the sustained data ingestion rate, NVMe storage bandwidth, and daily storage consumption for a bimanual leader-follower teleoperation rig capturing multi-modal demonstration data for physical imitation learning?

Parameters:

- **Perception:** 4 active stereo RGB-D cameras (2 static eye-to-hand environment cameras + 2 wrist eye-in-hand cameras):
 - RGB streams: 1920×1080 px at 30 Hz, 24 bits/px (3 bytes/px).
 - Depth streams: 1280×720 px at 30 Hz, 16 bits/px (raw `uint16` depth in millimeters, 2 bytes/px).
- **Kinematics & dynamics:** 2 leader arms + 2 follower arms (14-DoF total):
 - Joint positions $\mathbf{q}$, velocities $\dot{\mathbf{q}}$, commanded torques τ_{cmd} , measured torques τ_{meas} , and 4-stream action tuples logged at 1000 Hz (128 bytes/step).
- **Tactile & haptics:** 2 dual-finger fingertip tactile sensor arrays (16×16 pressure taxels $\times 2$ bytes $\times 4$ pads = 2048 bytes/step) logged at 100 Hz.

Calculation:

DRAFT

1. Raw video bandwidth:

$$BW_{\text{RGB}} = 4 \times (1920 \times 1080 \times 3 \text{ bytes}) \times 30 \text{ s}^{-1} = 4 \times 6.2208 \text{ MB} \times 30 = 746.50 \text{ MB/s.}$$

$$BW_{\text{Depth}} = 4 \times (1280 \times 720 \times 2 \text{ bytes}) \times 30 \text{ s}^{-1} = 4 \times 1.8432 \text{ MB} \times 30 = 221.18 \text{ MB/s.}$$

The combined vision bandwidth is $BW_{\text{Vision, total}} = 746.50 \text{ MB/s} + 221.18 \text{ MB/s} = 967.68 \text{ MB/s} \approx 0.968 \text{ GB/s.}$

2. Kinematic and tactile bandwidth:

$$BW_{\text{Kinematics}} = 128 \text{ bytes} \times 1000 \text{ s}^{-1} = 0.128 \text{ MB/s.}$$

$$BW_{\text{Tactile}} = 2048 \text{ bytes} \times 100 \text{ s}^{-1} = 0.205 \text{ MB/s.}$$

Evaluating the sensor taps yields $BW_{\text{Kinematics}} = 0.128 \text{ MB/s}$ and $BW_{\text{Tactile}} = 0.205 \text{ MB/s.}$

3. Total sustained ingestion rate: Aggregating all streams gives $BW_{\text{total}} = 968.01 \text{ MB/s} \approx 0.968 \text{ GB/s.}$

4. Hourly and shift storage consumption: Sustained ingestion accumulates uncompressed telemetry at 3.48 TB. When recording at a standard 1280×720 RGB target (331.8 MB/s) alongside depth (221.18 MB/s) with synchronous 4-stream metadata, sustained ingestion requires **553.3 MB/s**, accumulating **1.99 TB** of uncompressed telemetry. Across an 8-hour collection shift with 5 parallel workstations, the raw telemetry ingestion generates:

$$\text{Storage}_{\text{shift}} = 5 \times (1.99 \text{ TB/hr} \times 8 \text{ hr}) = 79.6 \text{ TB per shift.}$$

yielding 79.7 TB per shift.

Systems insight: A high-capacity data acquisition workstation cannot log uncompressed multi-view streams over standard USB 3.0 buses (capped at 400 MB/s real-world throughput) or write to SATA SSDs (capped at 550 MB/s). Ingesting 553.3 MB/s to 968.01 MB/s requires dedicated PCIe Gen4 NVMe storage (capable of $> 5000 \text{ MB/s}$ sequential writes) or hardware-accelerated NVENC video compression (H.265 intra-frame at CRF 18, compressing streams $8 \times$ to $\approx 69.2\text{--}92.5 \text{ MB/s}$ / $249\text{--}333 \text{ GB/hr}$) to prevent OS buffer overruns and silent frame drops.

Sustained multi-camera ingestion directly bypasses operating-system page caches to maintain predictable write throughput.¹⁰

NVMe PCIe: 3500 MB/s

4x Cam Raw: 968 MB/s

SATA SSD: 550 MB/s

USB 3.0: 400 MB/s

Multi-camera raw telemetry overwhelms legacy storage interfaces, requiring direct PCIe NVMe logging.

¹⁰ | **Kernel Page-Cache Bypass:** Writing multi-gigabyte camera streams through Linux OS page caches causes memory starvation and latency spikes. High-throughput data loggers utilize `O_DIRECT` DMA transfers directly to NVMe command queues to guarantee deterministic write times.

Checkpoint 10.2: Ingestion throughput, bus ceilings, and storage saturation

Before analyzing bilateral teleoperation dynamics, verify your understanding of multi-modal data acquisition rates and storage bus limits:

- ☐ **Ingestion throughput vs. storage:** Can you explain why uncompressed 4-camera 1080p RGB-D ingestion (968 MB/s) instantly saturates both USB 3.0 (400 MB/s) and SATA SSD (550 MB/s) write interfaces?
- ☐ **Buffer exhaustion dynamics:** Can you describe how kernel memory ring buffers behave when sustained ingestion bandwidth exceeds secondary storage write limits, and why frames drop silently without raising I/O exceptions?
- ☐ **Compression vs. fidelity trade-offs:** Can you distinguish the computational latency and visual artifact trade-offs of intra-frame NVENC H.265 compression ($8 \times$ reduction to $\approx 92 \text{ MB/s}$) versus raw uncompressed frame logging?
- ☐ **Ingestion capacity estimate:** For a 5-workstation bimanual collection cell operating across an 8-hour shift, estimate the uncompressed telemetry volume. How many 4 TB NVMe U.2 drives are consumed per shift before deduplication or compression?

High-bandwidth physical data capture requires co-designing sensor polling frequencies, bus topologies, and hardware compression to avoid silent data starvation.

In **bilateral teleoperation**, where contact forces measured at the follower arm are reflected back as haptic torques to the leader arm, transport delay creates a severe stability hazard. As Robert J. Anderson and Mark W. Spong demonstrated in their foundational analysis (Anderson and Spong 1989), network transmission latency transforms an intrinsically passive physical link into an active energy generator. In an ideal, zero-latency system, the communication channel acts as a passive transmission line that cannot generate energy. But when transport delay varies dynamically across the network, the delayed force reflection acts out-of-phase with the operator's hand velocity.

The interface behaves like a delayed spring: when the operator pushes into contact and begins to retract, the delayed reaction force arrives late, pushing the operator's hand in the direction of motion. This unexpected push injects virtual energy into the feedback loop. This non-passive energy drives high-frequency mechanical limit cycles (sustained, self-reinforcing oscillations), causing the master and follower arms to chatter violently against rigid contact fixtures. For a rigorous mathematical derivation of these passivity conditions, refer to section B.8.

Stabilizing the rig requires Time-Domain Passivity Controllers (TDPC) or wave-variable transformations, algorithmic techniques that inject virtual damping (software-simulated friction) into the teleoperation link to safely bleed off this excess energy. While passivity controllers prevent hardware damage, the added damping acts as a low-pass filter on operator hand motions, attenuating high-frequency contact cues and suppressing subtle tactile feedback.

Systems Perspective 10.1: Teleoperation passivity conditions

In bilateral teleoperation with force reflection, variable network latency injects virtual energy into the operator interface. To prevent contact instability and mechanical limit cycles, the real-time nervous system must monitor and dissipate this unexpected energy (enforcing a strict passivity condition) via Time-Domain Passivity Observers (TDPO).

The operator interface filters what a demonstration can express. In a hypothetical peg-in-hole task, a force-reflecting leader can let the operator feel binding and adjust contact; a visual-only rate controller may instead encourage short moves and pauses while the operator checks the video. These are task- and implementation-dependent behaviors, not performance measurements from Mandlekar et al. (2020). ALOHA is a different, backdriven joint-space leader/follower system without bilateral force reflection; its demonstrations are recorded at 50 Hz (Zhao et al. 2023). The comparison in table 10.1 identifies mechanisms to measure on a given rig rather than universal latency or fatigue ranges.

Table 10.1: Teleoperation interface mechanisms for physical demonstration collection: Mapping and feedback determine what the operator can demonstrate. Latency, command rate, tracking noise, and fatigue must be measured for the specific hardware, task, and operator.

Interface and feedback	Command mapping	Contact information available to operator	Possible collection artifact to check
Force-reflecting bilateral leader	Leader motion mapped to follower joints; measured follower force reflected to leader	Vision and physical contact force, subject to link delay and fidelity	Contact corrections and oscillation when delayed force reflection loses passivity
Backdriven joint-space leader (ALOHA) (Zhao et al. 2023)	Leader joint positions copied to follower; demonstration recorded at 50 Hz	Vision and leader mechanism feel, without active follower force reflection	Joint-space posture preferences and unobserved follower contact forces
Rate-controlled 3D mouse	Device deflection mapped to Cartesian velocity, then inverse kinematics	Primarily visual feedback unless a separate haptic channel exists	Step-and-wait motion and zero-velocity pauses under delayed feedback

Interface and feedback	Command mapping	Contact information available to operator	Possible collection artifact to check
Tracked spatial controller	Tracked hand pose mapped to tool pose with clutching and scale	Visual feedback; optional vibration is not directional force reflection	Clutch discontinuities and pose-tracking gaps
Instrumented exoskeleton	Wearable joint motion retargeted to robot joints	Depends on whether active force feedback is fitted	Biomechanical calibration and retargeting bias

To see how latency can erode teleoperation stability, consider an illustrative end-to-end loop delay of $t_{\text{loop}} = 320$ ms, partitioned across camera exposure and readout (20 ms), video compression and network transport (25 ms), display refresh (15 ms), human visual response (210 ms), and actuator ramp-up (50 ms). Suppose the measured open-loop gain crossover of this particular human-machine controller is $f_c = 1.0$ Hz ($\omega_c = 2\pi f_c \approx 6.283$ rad/s). At that crossover, the delay $\tau = 0.320$ s adds phase lag:

$$\Delta\phi = \omega \cdot \tau = 6.283 \text{ rad/s} \times 0.320 \text{ s} = 2.011 \text{ rad} \approx 115.2^\circ$$

Phase margin is the angular distance from -180° at the open-loop gain crossover. If the delay-free loop has a 60° margin there, and adding delay does not materially shift that crossover, the estimated margin becomes:

$$\text{PM}_{\text{net}} = \text{PM}_{\text{plant}} - \Delta\phi = 60^\circ - 115.2^\circ = -55.2^\circ$$

Under those loop assumptions, the negative estimated phase margin indicates instability risk. A 1 Hz hand-correction rate alone would not establish the gain crossover or the margin. Separately, for a 2.0 mm chamfer and no useful correction during the modeled dead time, a conservative unguided-drift ceiling is:

$$v_{\text{max}} = \frac{w_{\text{chamfer}}}{\tau} = \frac{2.0 \times 10^{-3} \text{ m}}{0.320 \text{ s}} \approx 6.25 \text{ mm/s}$$

An operator may slow below this scenario-specific ceiling or use step-and-wait motion; the actual safe approach also depends on sensing, plant response, and contact tolerance. See section B.8 for the loop analysis.



Checkpoint 10.3: Transport latency and control stability

TRANSPORT DELAY & PHASE EROSION

Assumed gain crossover $f_c = 1.0$ Hz; loop delay $\tau = 320$ ms

Net 60ms Human Reflex 210 ms (66%) Mot 50ms
Total loop dead time: $\tau = 60 + 210 + 50 = 320$ ms

Delay-free margin at f_c : $\text{PM}_{\text{plant}} = +60.0^\circ$
Delay Phase Lag ($\Delta\phi = \omega \cdot \tau$): -115.2° (Lag)
Net Phase Margin (PM_{net}): -55.2° (estimated)

Negative margin under stated loop assumptions

STEP-AND-WAIT VELOCITY CEILING

Chamfer $w = 2.0$ mm | Unguided drift in dead time τ



$$v_{\text{max}} = w / \tau \\ = 2.0 \text{ mm} / 0.32 \text{ s} = 6.25 \text{ mm/s}$$

Dead-time drift limit before jam

$v > 6.25 \text{ mm/s}$: Drift > 2 mm during $\tau \rightarrow$ Collision / Jam
 $v \leq 6.25 \text{ mm/s}$: Example: short moves with visual checks

Policy Bias: Imitates Human Hesitation Pauses

- ☐ **Transport delay phase erosion:** Under what gain-crossover and delay assumptions can transport latency erase phase margin and raise the risk of mechanical limit cycles?
- ☐ **Step-and-wait behavioral artifacts:** Can you describe why operators adopt intermittent step-and-wait strategies under visual-motor delays, and how this biases the resulting demonstration dataset?
- ☐ **Passivity and virtual damping:** Can you distinguish physical damping from virtual passivity controllers in mitigating instability across delayed teleoperation links?

During demonstration collection, logging teleoperation streams merely as unstructured state-action pairs obscures the signal transformations that occur between human intent and mechanical

execution. Preserving the four-stream action tap tuple $(a_{\text{req}}, a_{\text{cmd}}, a_{\text{enf}}, a_{\text{meas}})$ across the computing boundary (10.2) ensures that safety governor interventions and mechanical tracking discrepancies are decoupled from human demonstration intent.

Translating scheduled collection shifts into usable training data requires budgeting for setup overhead, ergonomics, cycle times, and quality attrition. Consider the precision peg-in-hole assembly task across a declared single session of $T_{\text{sched}} = 60.0$ min (3600 s). After cell startup, sensor calibration, and mandatory operator rest breaks, the remaining active motion window is $T_{\text{motion}} = 35.0$ min (2100 s). With an attempt duration of $t_{\text{att}} = 25.0$ s and a fixture reset duration of $t_{\text{rst}} = 15.0$ s ($t_{\text{cyc}} = 40.0$ s), the workstation executes at most $N_{\text{att}} = \lfloor 2100 \text{ s} / 40.0 \text{ s} \rfloor = 52$ raw attempts per session. Accounting for in-trial failures (20 percent) and post-collection quality filtering (15 percent) gives a net retention rate of $R = (1 - 0.20) \times (1 - 0.15) = 0.68$, so the 52 raw attempts yield $N_{\text{usable}} = 52 \times 0.68 \approx 35.36$ usable episodes. Across the session, this produces 884 s, or approximately 14.73 min of validated demonstration data (a 24.6 percent net session yield).¹¹ Reaching a benchmark target of 5000 validated episodes requires $5000 / 35.36 \approx 141.4$ h scheduled operator hours across 7,353 physical attempts, scaling collection schedules by a $4.07\times$ facility-to-yield overhead factor.



Systems Perspective 10.2: The 4-to-1 facility-to-yield ratio

Never budget physical demonstration campaigns by scheduled facility hours. Due to sensor calibration, workpiece re-fixturing, operator ergonomics, and post-collection QA filtering, one hour of scheduled robot cell time yields at most 15 minutes of validated, defect-free trajectory data (a 4 : 1 overhead factor).

Hardware degradation during collection depends on the accumulation of physical cycles and directional stress reversals rather than elapsed wall-clock time. Consider the wrist joint of an articulated arm equipped with a strain-wave harmonic drive and internal routing flex cables. In an analytical wear model, the harmonic gear transmission carries a rated fatigue life of $L_{\text{gear}} = 5,000,000$ stress cycles under rated output torque, and the internal cable bundle is rated for $L_{\text{cable}} = 200,000$ bending cycles. During the 25.0 s execution of a peg-insertion attempt, contact alignment and search motions introduce an average of $n_{\text{gear}} = 14$ torque reversals and $n_{\text{cable}} = 6$ joint flex cycles. Resetting the rig adds an additional 2 full flex cycles during the return stroke. To collect 5000 validated episodes at a net retention rate of $R = 0.68$, the hardware must execute $N_{\text{total}} = 5000 / 0.68 \approx 7,353$ physical attempts. Across these 7,353 attempts, the wrist transmission sustains $7353 \times 14 = 102,942$ stress cycles, consuming 2.1 percent of its rated fatigue life. Concurrently, the internal cable bundle undergoes $7353 \times (6 + 2) = 58,824$ bending cycles, consuming 29.4 percent of its total service life on a single dataset collection campaign. If three collection campaigns run sequentially on the same rig without replacement, the flex cable reaches 88.2 percent of its fatigue limit, risking conductor strand breakage and intermittent signal loss that corrupt sensor streams. Compute the mechanical wear budget from cycle counts and load reversals to schedule component replacement before hardware failure invalidates collection sessions. The operational time breakdown and mechanical wear budget across a declared 60 min teleoperation session are synthesized in table 10.2.

11 | Demonstration Yield:

The ratio of kinematically valid, task-successful trajectory duration to total scheduled facility time. In precision robotics, resets, calibration nulling, and operator fatigue reduce net demonstration yield to under 25 percent of clock time.

Table 10.2: Physical demonstration collection budget breakdown across a scheduled 60-minute session: Partitioning of facility clock time, active motion yield, quality filtering losses, and mechanical component fatigue across collection stages for a precision peg-in-hole assembly workcell ($T_{\text{sched}} = 60.0$ min, $N_{\text{att}} = 52$ attempts, nominal attempt $t_{\text{att}} = 25.0$ s, reset $t_{\text{rst}} = 15.0$ s). The nominal hour yields only 14.73 min of retained demonstration data (24.6 percent net yield), while imposing significant cumulative fatigue on mechanical transmissions and wiring flex lines.

Session Stage / Activity	Allocated Duration (Δt)	Wall-Clock Share (%)	Physical Operation & Control State	Hardware Stress & Wear Mechanism	Net Trajectory Output ($N_{\text{episodes}} / T_{\text{yield}}$)
Cell Startup & Bus Sync	8.0 min (480 s)	13.3%	Bus synchronization (Ether-CAT/CANopen), safety watchdog verification, optical homing	Power-on electrical inrush; thermal warm-up in motor windings and sensor bridges	$N = 0$ ep, 0.0 min yield
Sensor Calibration & Nulling	7.0 min (420 s)	11.7%	Camera intrinsic/extrinsic sweep, 6-axis F/T gravity bias nulling, tactile matrix zeroing	Static holding torque; sensor bridge thermal stabilization; zero dynamic gear wear	$N = 0$ ep, 0.0 min yield
Operator Rest & Ergonomics	10.0 min (600 s)	16.7%	Mandatory operator breaks to mitigate visual fatigue, cognitive saturation, and tremor drift	Passive standby; electromagnetic brakes engaged or zero-velocity holding current	$N = 0$ ep, 0.0 min yield
Physical Scene Resets	13.0 min (780 s)	21.7%	Part unclamping, receptacle clearing, peg re-fixturing, manipulator return stroke (15.0 s/attempt)	52 return strokes $\times$ 2 bends = 104 cable flex cycles; zero contact payload	$N = 0$ ep, 0.0 min yield
Active Teleoperated Attempts	21.67 min (1300 s)	36.1%	Real-time human teleoperation guiding peg approach, chamfer search, and insertion (25.0 s/attempt)	52 $\times$ 14 = 728 torque reversals on harmonic drive; 52 $\times$ 6 = 312 cable flex cycles; contact loads ≤ 15 N	$N = 21.67$ min gross motion
In-Trial Failures (Labeled)	-4.33 min (-260 s)	-7.2%	Part slippage, jamming, watchdog force trip ($F_N > 15.0$ N), dropped pegs (20% loss)	High-force contact transients; abrasive wear on gripper silicone pads	-10.4 ep from success yield; valid logs kept
Post-Collection QA Filtering	-2.60 min (-156 s)	-4.3%	Offline filtering: link latency > 100 ms, visual occlusion, operator hesitation (15% loss)	Zero hardware wear (offline computational validation pass)	-6.24 ep discarded
Net Validated Dataset Yield	14.73 min (884 s)	24.6%	High-quality synchronized multi-modal trajectories (s_t, a_t) admitted into training buffer	Consumes 2.1% of wrist gear life and 29.4% of internal flex cable life per 5000 episode campaign	$N_{\text{usable}} = 35.36$ episodes

The expected 10.4 in-trial failures reduce *successful-demonstration* yield. When their telemetry is valid, retain them as labeled failure or intervention records; exclude only invalid recordings from the archive.

Teleoperation records trajectories that are bounded by what a human operator can perceive through a display, coordinate through an input interface, and stabilize against system latencies. The resulting dataset reflects the operational policy of the human-interface composite rather than

an unconstrained sample of physical dynamics. Every demonstration collected in this manner carries the structural limitations, response delays, and conservative motion biases of the collecting apparatus into the training distribution. Understanding the safety and performance limits of a learned policy therefore requires analyzing how the collecting policy’s operational biases shape its mathematical coverage across the state space—the focus of section 10.4.

10.4 Collection Policy Coverage

A physical dataset is generated by executing a sequence of control actions on hardware and logging the resulting state transitions over time. When an agent or human operator executes trajectories $\tau = \{(s_0, a_0), (s_1, a_1), \dots, (s_T, a_T)\}$ at a fixed logging rate such as 100 Hz, the collected samples define an **empirical state-action occupancy distribution** (Spencer et al. 2021) $d^{\pi_{\text{col}}}(s, a)$. The training loss in imitation learning minimizes prediction error weighted by this distribution, optimizing the learned policy π_θ exclusively on states where $d^{\pi_{\text{col}}}(s) > 0$. The data-gathering machine determines the dataset’s contents. This creates operational endogeneity, where the robot’s behavior during data collection dictates its training observations. The dataset does not represent an unconstrained sampling of the physical state space. It represents the trajectory manifold traced by the collecting policy π_{col} under its specific control biases, interface latencies, and risk constraints.

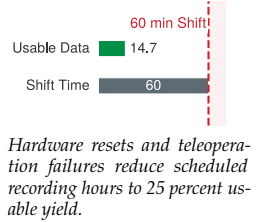
Consider an analytical example of a robotic gripper aligning with a workpiece edge using tactile sensing and force feedback. The physical workspace requires managing contact angles over $\theta \in [-15.0^\circ, +15.0^\circ]$ and normal contact forces up to $F_N = 20.0$ N. Two different collecting policies gather data on this exact task using identical six-axis force-torque sensors and tactile arrays logging at $f_s = 100$ Hz. Each policy records $N = 1000$ episodes lasting 10.0 s per episode, producing identical dataset sizes of 1,000,000 transition records (1.00×10^6 samples).

The first collector is a cautious human teleoperator who approaches the workpiece along a single nominal axis at $v = 0.01$ m/s, maintaining contact strictly perpendicular to the surface ($\theta \in [-2.0^\circ, +2.0^\circ]$) under light normal force ($F_N \in [4.0 \text{ N}, 6.0 \text{ N}]$). The second collector is an exploratory compliance policy that approaches across a broader spread of approach angles ($\theta \in [-15.0^\circ, +15.0^\circ]$), encountering transient slips and varied contact loads ($F_N \in [1.0 \text{ N}, 18.0 \text{ N}]$). Discretizing the two-dimensional state space of contact angle and normal force into bins of 1.0° and 1.0 N produces a deployment grid of $30 \times 20 = 600$ discrete cells.

Reconstructing the visitation counts reveals the structural difference between these datasets. The cautious collector deposits all 1,000,000 samples into $4 \times 2 = 8$ cells, concentrating an average of 125,000 samples in each visited bin while leaving 592 cells (98.7 percent of the required operational grid) with a visitation count of zero. The exploratory collector distributes its 1,000,000 samples across 510 cells with a mean density of 1961 samples per cell, leaving 90 cells (15.0 percent) unvisited. Despite identical byte counts, logging frequencies, and sensor streams, these datasets answer different physical questions. The cautious dataset offers dense supervision for aligned contact, while the exploratory dataset reveals how tactile signatures and reaction forces evolve during off-axis engagement.

An action label in a physical dataset exists only because the collecting policy chose to visit that state and issued a command. When a state has zero visitation count, the dataset contains neither a preferred action nor physical evidence that an action cannot be found. Supervised learning algorithms cannot distinguish between a physical configuration that is inherently unstable, a configuration that is safe but unvisited, and a configuration that causes mechanical damage. To the loss function, all unvisited regions appear identically as a total absence of constraints.

This absence can produce compounding failure in behavioral cloning. Suppose a policy trained on the cautious dataset executes on the physical hardware. At $t = 0.00$ s, an unmodeled friction variation or mechanical vibration perturbs the gripper to an angle of $\theta = +4.5^\circ$, placing the state outside the 8 demonstrated cells. The policy evaluates its neural network on an unvisited input. Empirical risk minimization provides no direct training constraint there, so its action $\hat{a}$ is unvalidated by this dataset. In this illustrative rollout, applying $\hat{a}$ causes the next state at $t = 0.01$ s to reach $\theta = +6.0^\circ$ with a contact force of 11.0 N. The archive supplies no example of correcting that off-axis, high-force state. This policy’s subsequent actions move it farther from the demonstrated corridor, and the gripper jams against the fixture.



As defined in 10.1, this compounding divergence is the signature of endogenous covariate shift. Because empirical risk minimization provides zero constraint outside the support of demonstrated corridors, small execution perturbations push the robot into unfamiliar states where the model's ungrounded outputs accelerate trajectory divergence.



Systems Perspective 10.3: Unvisited states and unsupported actions

An unvisited scenario cell supplies no direct expert action label or performance evidence. A policy may generalize there, fail, or be stopped by a separate safeguard; the dataset alone cannot decide which. Targeted collection, expert relabeling, or refusal of autonomous authority is needed before claiming coverage across that cell.

Evaluating dataset quality by computing extrema over the collected values creates a circular evaluation metric. If a collector only records contact forces between 4.0 N and 6.0 N, measuring coverage relative to the minimum and maximum logged values reports 100 percent completeness across that interval. Coverage must instead be evaluated against an external **scenario ledger**, which defines the required deployment envelope across object poses, surface friction ranges, contact velocities, payload masses, and external disturbance bounds.

Unsupported deployment regions are detected by performing a relational join between the required scenario bins and the empirical dataset visitation counts. For each cell in the ledger, the analysis reports the exact transition count N_{cell} . Cells with $N_{\text{cell}} = 0$ have no direct training supervision; a separate permission path must decide whether the policy may operate there. Low-count cells (for example, fewer than 50 samples in this illustrative ledger) need uncertainty and scenario-specific coverage checks.

This coverage deficit cannot be detected by evaluating validation loss on a held-out test split of the collected data. Because the held-out split is drawn from the same trajectories generated by π_{col} , it shares the identical empirical occupancy $d^{\pi_{\text{col}}}$. The validation loss can reach 0.001 rad, indicating statistical convergence, while 98.7 percent of the required operational envelope contains zero training examples. Verifying coverage requires computing the ratio of supported scenario cells to total required cells before a policy is deployed on physical actuators (figure 10.4, table 10.3).

Table 10.3: Empirical coverage and validation loss metrics across operational scenario bins: Relational join metrics comparing cautious teleoperation against exploratory policy coverage.

Metric	Cautious Teleoperation ($d^{\pi_{\text{teleop}}}$)	Exploratory Policy ($d^{\pi_{\text{expl}}}$)	Systems Consequence
Recorded Transitions	1,000,000 transitions (100 Hz)	1,000,000 transitions (100 Hz)	Equivalent dataset scale
Operational Grid Coverage	8 / 600 bins (1.3%)	510 / 600 bins (85.0%)	Operational support envelope
Zero-Count Blind Spots ($N_{\text{cell}} = 0$)	592 / 600 bins (98.7%)	90 / 600 bins (15.0%)	Unmonitored risk volume
Held-Out Validation Loss (L_{val})	0.001 rad	0.012 rad	Validation loss is non-diagnostic
Closed-Loop Compounding Drift	Catastrophic drift beyond $\pm 2.5^\circ$	Bounded recovery across $\pm 15^\circ$	Physical policy stability
Ingestion Gate Decision	Refuse (require targeted DAgger)	Admit (proceed to training cluster)	Scenario ledger audit gate



Checkpoint 10.4: Empirical occupancy and scenario ledger joins

- ☐ **Empirical support vs. validation loss:** Can you explain why an imitation policy can achieve near-zero validation loss on held-out test data while leaving 98 percent of the required operational envelope unsupported?

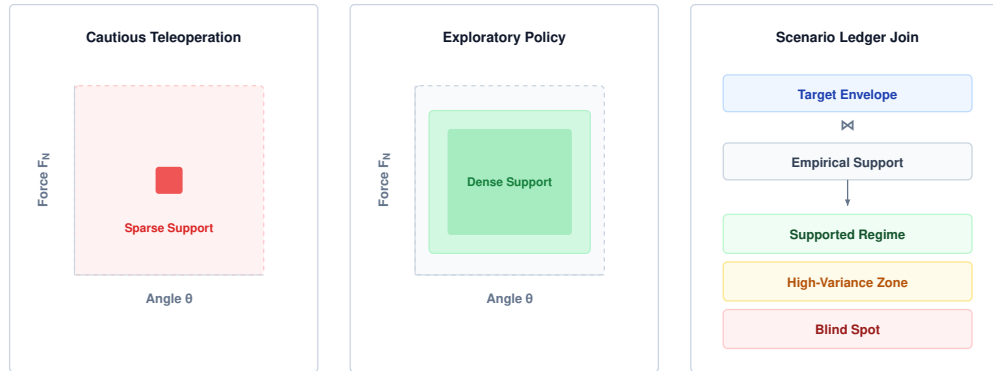


Figure 10.4: Collector occupancy and scenario join: State-action visitation for a cautious teleoperator against an exploratory policy, joined against the scenario ledger the deployment requires. The cautious collector concentrates transitions into a narrow cluster and leaves 98.7 percent of the envelope unvisited while reporting near-zero validation loss, showing that statistical convergence says nothing about physical coverage. The relational join partitions the operational envelope into supported regimes, high-variance transition zones, and zero-support blind spots where autonomous control authority must be refused.

- ❑ **Unsupported states:** What can an unvisited scenario cell establish about policy behavior, and what additional evidence or refusal decision is needed?
- ❑ **Scenario ledger relational joins:** Can you distinguish supported nominal regimes, high-variance transition zones, and zero-support blind spots when joining telemetry against an operational scenario ledger?

When relational joins against the scenario ledger expose unmonitored blind spots, simply recording more nominal demonstrations along the central corridor cannot fix the policy's failure to recover. Expanding empirical coverage into hazardous boundary states requires either active human supervisory takeovers or automated perturbation policies—techniques that introduce delicate causal challenges when ingesting intervention data, as investigated in section 10.5.

10.5 Intervention and Recovery Data

When a hazardous or boundary state does not appear in a training dataset, an engineer cannot conclude that the physical system avoids that state naturally. An absent state in the empirical log corresponds to one of four distinct physical mechanisms during data collection:

1. **Unvisited nominal path:** The collecting policy never visited the state because nominal rollouts remained entirely within the primary task corridor. This absence represents unmonitored nominal operation, detected by cross-referencing trajectory logs against the scenario ledger.
2. **Safety governor interception:** An active safety barrier or deterministic reflex in the nervous system detected an impending boundary violation and truncated the motion before the state was reached. This absence is detected directly in the safety monitor logs, which record the barrier trip.
3. **Human supervisory takeover:** A human safety operator perceived a developing hazard and disengaged the autonomous policy before contact occurred. This absence is detected through the takeover disengagement flag on the vehicle or manipulator bus (building on the hardware abstractions from section 4.2).
4. **Post-collection censoring:** The machine actually entered the hazardous state and collided with an obstacle or jammed a joint, but an automated post-processing script discarded the episode as corrupt or incomplete. This absence is not detected by dataset inspection, because the failure transitions were purged before the training arrays were written to disk.

Each mechanism produces an identical zero count in a scenario cell, yet treating them identically leads to incorrect conclusions about policy stability.

Human takeovers and automated disengagements introduce a selection trap into fleet data. An intervention occurs precisely when the autonomous policy has entered a region of state space that it handles poorly, causing tracking error or model uncertainty to exceed an operational threshold. Because disengagements are triggered by failing rollouts, the collected intervention data is not drawn from the nominal state distribution of the task. If a safety driver intervenes when a mobile robot drifts 0.15 m outside its lane corridor at 2.0 m/s, the trajectory recorded after the takeover captures an expert recovering from an edge case disturbance. The distribution of states visited during these rescues, denoted d^{takeover} , is conditioned on policy failure rather than standard task execution. Treating intervention logs as if they were drawn from the nominal distribution d^{π^*} distorts the training distribution toward states that only exist because the prior policy failed.

Appending a rescued episode directly to a behavioral cloning dataset can make unsafe policy commands look like expert demonstrations. Before takeover, the learned policy may issue commands that increase heading error or contact force; after takeover, the expert issues corrective commands from states the learner visited. Those drift states are valuable for Dataset Aggregation (DAgger) when the expert can label a safe action there. The unsafe policy commands must remain attributed to the learner and excluded as expert imitation targets. Recovery after takeover is a separate expert sequence, with its own authority and timestamps.

Separating these labels requires locating the **divergence point** where the autonomous policy departed the nominal path (extending section 8.1), rather than treating takeover as the first error. In an illustrative 100 Hz controller, a 300–800 ms human response spans 30 to 80 cycles. Retain observations and learner commands from t_{div} to t_{takeover} with their true authority and safety flags. If a qualified expert can label safe actions for those learner-visited states, admit the relabeled states to corrective aggregation; otherwise retain them for audit or coverage analysis, not as expert action labels. Record the post-takeover recovery separately.

Detecting t_{div} in offline ingestion pipelines requires searching backwards in time from the physical override timestamp across a synchronized pre-trigger telemetry buffer:

$$\hat{t}_{\text{div}} = \min \left\{ t \in [t_{\text{takeover}} - \Delta t_{\text{lookback}}, t_{\text{takeover}}] \mid \begin{aligned} &\| \mathbf{x}_{\text{plant}}(t) - \mathbf{x}_{\text{ref}}(t) \|_2 > \delta_{\text{tol}} \\ &\vee \text{Tr}(\Sigma_{\pi}(t)) > \sigma_{\text{thresh}}^2 \end{aligned} \right\}.$$

The pipeline scans a stated lookback window (here $\Delta t_{\text{lookback}} = 1.0\text{--}2.0$ s) for the first threshold crossing, using calibrated tracking residuals and, if available, policy uncertainty. The thresholds are scenario-specific. Transitions between $\hat{t}_{\text{div}}$ and t_{takeover} are removed from the nominal expert-action loss, while their states remain available for expert relabeling and their original commands remain in the audit log.

Intervention curation separates learner commands, expert labels on learner-visited states, and post-takeover recovery at the divergence and takeover timestamps. It excludes unsafe learner commands from expert imitation loss without erasing the states that reveal the policy’s coverage gap.

The cost of missing those states can compound over an operational horizon of T steps (Ross et al. 2011). Under the expert-state 0–1 action-loss and bounded per-step cost assumptions of equation 10.1, behavioral cloning has a worst-case cost-gap upper bound of $O(T^2\epsilon)$; this is not a predicted growth curve for every robot.¹²

In **corrective aggregation**, such as DAgger (Ross et al. 2011), an expert labels states visited by the learner, including eligible pre-takeover drift states. Under the reduction’s online-learning and expert-query assumptions, the learned policy can attain a cost bound linear in horizon and its loss on learner-visited states, $O(T\epsilon)$. This does not guarantee recovery from arbitrary states or remove the need for independent actuator safeguards.¹³

For $T = 500$ steps (5.0 s at 100 Hz) and $\epsilon = 0.01$, the expressions $T^2\epsilon = 2500$ and $T\epsilon = 5$ illustrate different horizon dependence. They are bound terms with omitted constants and different error-distribution assumptions, not measured drift or a $500\times$ improvement. With per-step cost at most one, cumulative task cost itself cannot exceed 500 in this example (figure 10.5).

Training successive policy generations on drift-window learner commands mislabeled as expert actions can create a retraining cascade:

¹² | **Behavioral Cloning Error Bound:** The bound analyzed by Ross et al. (2011) concerns expected cumulative cost under specified loss and bounded-cost assumptions; it does not assert quadratic physical tracking error. See Appendix A.

¹³ | **Corrective Aggregation Error Bound:** DAgger (Ross et al. 2011) queries an expert on learner-visited states and aggregates those labeled states. Its linear-horizon result depends on the expert oracle, online-learning regret, bounded cost, and the measured loss of the resulting policy; it is not a guarantee of physical trajectory recovery. See Appendix A.

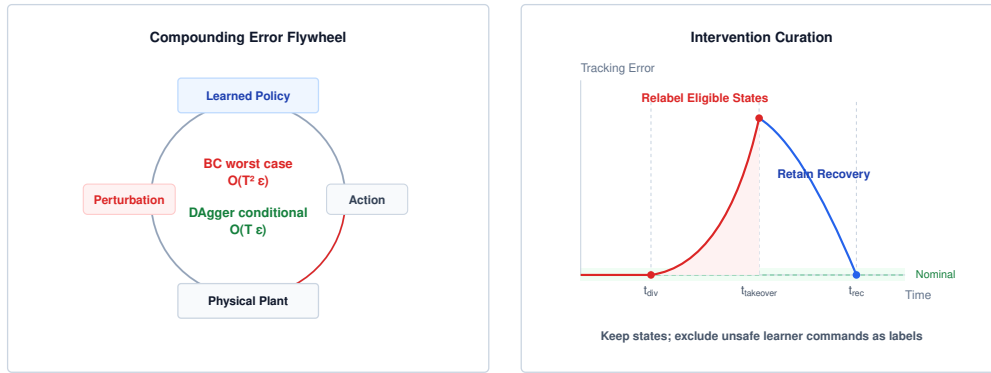


Figure 10.5: Covariate shift and intervention curation: The left panel contrasts conditional behavioral-cloning and DAgger cost-bound orders, not measured trajectory drift. The right panel shows learner-visited states between divergence and takeover. Keep these states for expert relabeling when available; exclude the learner’s unsafe commands from expert-action targets, and record takeover recovery separately. Based on Ross et al. (2011) and Spencer et al. (2021).

1. **Generation 1:** A policy operates with a slight calibration error, causing a robotic arm to drift toward a joint torque limit until a human operator grabs the override pendant.
2. **Generation 2:** If the drift-window commands are presented as expert labels, the policy learns to repeat them at similar off-nominal states. On this rig, that could increase excursions toward the torque limit before a correction.
3. **Generation 3:** Operators may then intervene earlier. Intervention rate can rise while validation loss on the mislabeled archive falls, making the offline metric look better as physical performance worsens.

To detect such a cascade, the nervous system emits an **intervention record** for each takeover or barrier trip.¹⁴ As in chapter 28, the record needs more than a binary flag on video: a pre-trigger buffer long enough for the declared lookback (two seconds in this example), synchronized observations and action taps, a monotonic override timestamp and authority ID, and the subsequent recovery sequence. Without the pre-trigger evidence, the pipeline cannot locate t_{div} reliably or separate unsafe learner commands from states eligible for expert relabeling.

Checkpoint 10.5: Intervention curation and compounding error bounds

- ☐ **Takeover versus divergence timestamps:** Why retain learner-visited drift states for possible expert relabeling while excluding their unsafe learner commands from expert imitation targets?
- ☐ **Retraining cascade progression:** Can you describe how training successive policy generations on uncured intervention logs accelerates policy degradation despite improving training metrics?
- ☐ **Conditional error bounds:** Which assumptions distinguish the behavioral-cloning quadratic worst-case cost bound from DAgger’s linear-horizon result?

Even with expert labels on learner-visited states, the resulting demonstration archive remains tethered to the kinematics and dynamics of the collecting machine. Pooling demonstrations across robot arms, bases, and end-effectors raises the cross-embodiment questions in section 10.6.

10.6 Cross-Embodiment Transfer

Data from one physical machine does not represent the task. Every recorded trajectory binds the physical characteristics of the collecting hardware into its numerical values, including link lengths, joint velocity envelopes, sensor optical paths, communication delays, and contact surface

14 | Fault-Event Logging:

This collection design preserves pre-trigger state, action authority, fault codes, and actuator-current samples across an override, with integrity checks on the stored record. Such telemetry helps distinguish policy drift from hardware or environment faults; it does not by itself establish the first divergence state or satisfy a functional safety certification.

Downstream: The pre-trigger intervention record schema is implemented for real-time supervisory takeovers in section 28.8.

compliances. When an engineer saves an observation-action trace (o_t, a_t) , the record does not capture an abstract manipulation skill. It captures the **embodiment contract** established between that specific machine and its environment (governed by the ISO/IEC 5259 data quality standard series). This contract encompasses the sensor geometry and optical calibration, the sampling period and transport latency of the perception bus, the coordinate frames and kinematic limits of the actuators, the torque-speed saturation curves of the motors (including reflected rotor inertia $N^2 J_{\text{rotor}}$ in geared transmissions, as established in section 4.4), and the friction and compliance of the contact interface. Altering any parameter in this contract changes the mapping between commanded inputs and physical state transitions.

The **cross-embodiment compatibility contract** is the formal geometric, dynamical, and temporal specification (such as those underpinning the Open X-Embodiment (Open X-Embodiment Collaboration et al. 2024) and DROID (Khazatsky et al. 2024) datasets). Intuitively, it serves as a physical translation rulebook, ensuring that a force profile or spatial velocity learned on a heavy industrial arm is safely scaled down before execution on a lightweight desktop robot. It defines the forward kinematics mapping, control rate normalization (Δt), and actuator effort bounds required before multi-robot trajectories can be transferred to a target physical plant.

Engineering a data reuse pipeline requires separating task-level invariants from embodiment-specific realizations. A task-level invariant is a geometric or dynamical condition required by the environment, such as bringing a compliant probe into contact with a terminal pad at an angle within 5° of the surface normal while maintaining a normal force between 2.0 N and 5.0 N. An embodiment-specific realization is the sequence of joint encoder counts, pulse-width modulation duty cycles, bus transport delays, and camera pixel rasters that achieved that condition on a specific test rig. While the contact target exists in world coordinates, the recorded dataset contains only the realization. A downstream model trained directly on those raw records learns the idiosyncratic dynamics of the source rig, including its structural flex, drive backlash, and camera mount vibration.

In this analytical example, because no bench measurement exists yet, consider transferring a dataset recorded on Embodiment A to Embodiment B for an identical touch-and-grasp task:

- **Embodiment A** uses a parallel-jaw gripper with a 0 to 100 mm stroke range, a 50 Hz command rate (20 ms period), a continuous force limit of 40 N, and a wrist camera offset 45 mm behind the tool center point with a 15 ms transport delay.
- **Embodiment B** uses a compact gripper with a 0 to 60 mm stroke range, a 20 Hz command rate (50 ms period), a continuous force limit of 15 N, and a wrist camera offset 20 mm behind the tool center point with a 35 ms transport delay.

Mapping the dataset from Embodiment A through the limits of Embodiment B reveals the points where transfer fails mechanically. For an object width of 75 mm, Machine A opens to 80 mm, a command that exceeds the mechanical travel of Machine B by 20 mm and drives its leadscrew against the physical endstop. During the hold phase, Machine A applies 30 N of clamping force. That same command exceeds Machine B's continuous rating of 15 N by a factor of two, driving the motor amplifier into current saturation. In the time domain, subsampling the 50 Hz trajectory onto Machine B's 20 Hz bus aliases high-frequency contact transitions and introduces a zero-order hold phase lag of 15 ms. Combined with the 20 ms increase in camera transport delay (35 ms – 15 ms), Machine B perceives contact events 35 ms after they occur. At an approach velocity of 0.10 m/s, this sensor-to-actuation latency carries the gripper 3.5 mm past the intended contact plane before the first corrective command can be dispatched, while the 25 mm camera mounting disparity introduces visual parallax that shifts the apparent position of the workpiece boundary in the image frame.

Standard machine learning pipelines reconcile these physical discrepancies by normalizing observation and action channels to $[0, 1]$ or $[-1, 1]$. Normalization creates a cosmetic alignment of numerical ranges, leaving physical incompatibility intact. If an opening command is normalized by dividing by the maximum stroke, a normalized command $u = 0.75$ corresponds to 75 mm on Machine A, which clears a 70 mm workpiece. On Machine B, that same normalized command $u = 0.75$ commands an opening of 45 mm (0.75×60 mm), causing the fingers to collide with the workpiece during the approach trajectory. Normalization rescales numbers, but it cannot expand actuator stroke, cannot lower electrical resistance, and cannot eliminate transport latency. When the data distributions of both embodiments are plotted in normalized space, the demonstration trajectories appear identical, yet only the subset lying within the physical intersection (openings

under 60 mm, forces under 15 N, and command bandwidths under 20 Hz) can execute on the target hardware without saturation or impact damage.

🔍 Systems Perspective 10.4: The non-invariance of unitless scaling

Unitless range normalization ($[-1, 1]$) aligns tensor dimensions on disk but destroys physical invariants. Normalizing an actuator command masks torque saturation, stroke limits, and transport delays, guaranteeing dynamic failure when transferred to hardware with differing reflected inertia or bus latency.

Cross-embodiment datasets pool demonstrations from different robots, increasing possible task and hardware diversity. The 2023 Open X-Embodiment point in figure 10.6 reflects a pooled collection with 22 embodiments, while the plotted episode counts from independent datasets rise and fall. Reuse still requires the compatibility checks developed above.

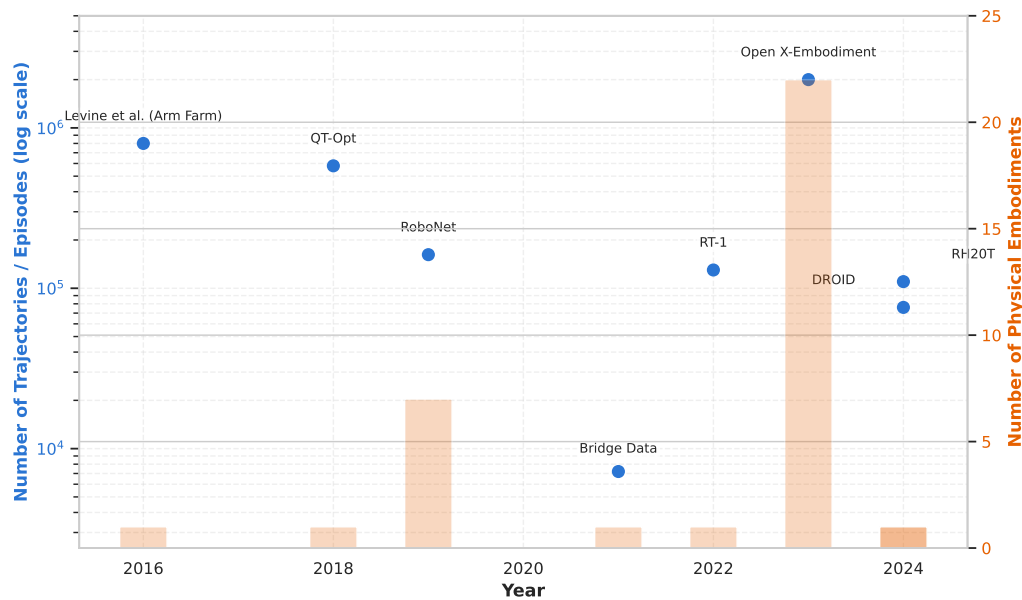


Figure 10.6: Robot learning dataset snapshots: Unconnected blue markers show episode counts for selected datasets (logarithmic left axis), not a cumulative time series. Orange bars show embodiment counts (right axis), including the pooled Open X-Embodiment jump in 2023. Dataset metadata: Open X-Embodiment Collaboration et al. (2024) and the chapter CSV.

The emergence of large-scale multi-robot datasets, spearheaded by the **Open X-Embodiment (OXE) initiative** (Open X-Embodiment Collaboration et al. 2024) and the **DROID dataset** (Khazatsky et al. 2024), highlights both the potential and the systems bottlenecks of cross-embodiment learning. The Open X-Embodiment repository aggregates over 1,000,000 demonstration trajectories collected across 22 distinct robot embodiments from more than 20 academic and industrial institutions, powering generalist robot policies such as RT-1 (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023), RT-2 (Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023), RT-X (Open X-Embodiment Collaboration et al. 2024), Octo (Octo Model Team et al. 2024), and OpenVLA (Kim et al. 2024). However, ingesting heterogeneous physical telemetry into a single training corpus exposes five fundamental systems mismatches:

1. **Action space incompatibility:** A canonical 7-DoF articulated research arm records joint positions $\mathbf{q} \in \mathbb{R}^7$, a bimanual leader-follower workstation records dual 6-DoF arm vectors $\mathbf{q} \in \mathbb{R}^{14}$, and a mobile manipulator logs base planar velocities $(\dot{x}, \dot{y}, \dot{\theta})$ paired with a 5-DoF arm. Because joint-space representations are strictly non-transferable across different kinematic

topologies, multi-embodiment architectures are forced to project actions into a universal Cartesian end-effector delta representation:

$$\Delta \mathbf{x}_t = (\Delta x, \Delta y, \Delta z, \Delta \phi_{\text{roll}}, \Delta \phi_{\text{pitch}}, \Delta \phi_{\text{yaw}}) \in SE(3), \quad a_{\text{gripper}} \in [0, 1].$$

While Cartesian deltas provide geometric standardization, they discard actuator dynamics. A Cartesian displacement $\Delta \mathbf{x} = [0.03 \text{ m}, 0.0, 0.0]$ that is trivial for an industrial arm near its nominal configuration will drive a lightweight manipulator through a kinematic singularity (as established in section 4.4). This commands infinite joint velocities that trigger immediate overcurrent protection to prevent motor burnout.

2. **Control loop frequency discrepancies:** Across the Open X-Embodiment datasets, recording frequencies vary widely, spanning 3 Hz (e.g., Bridge-v2), 10 Hz (e.g., Google Fractal), 20 Hz (e.g., DROID), and 50 Hz (e.g., canonical bimanual high-frequency datasets). A step delta $\Delta x = 20 \text{ mm}$ recorded at 3 Hz corresponds to a linear velocity of 0.06 m/s, whereas the same numerical delta at 50 Hz corresponds to a violent 1.0 m/s stroke. If an ingestion pipeline trains a policy on unscaled delta tokens without explicit timebase conditioning (Δt), the learned model emits chaotic velocity commands that depend on which dataset dominated the training mini-batch.
3. **Perception and viewpoint disparity:** Heterogeneous datasets combine disparate camera viewpoints, including high-mounted third-person room cameras, angled workcell cameras, and wrist-mounted eye-in-hand cameras, each with distinct optical focal lengths, sensor aspect ratios, and dynamic exposure ranges. Training an invariant visual representation requires multi-view camera extrinsic calibration and coordinate frame reprojection (Lynch and Park 2017).
4. **Gripper mechanics and contact compliance:** Gripper actions range from binary open/close flags (pneumatic chucks) to continuous position setpoints (0–100 mm) and force-controlled tendon tensioning. Furthermore, differences in contact pad compliance (e.g., soft silicone vs rigid 3D-printed PLA) alter the friction coefficient μ by a factor of four, rendering identical gripping force setpoints either secure or prone to catastrophic slip.
5. **Null-space and kinematic redundancy projection:** When mapping a 7-DoF articulated arm (e.g., a 7-DoF torque-controlled manipulator) or a 14-DoF bimanual platform into a 6-DoF Cartesian delta ($\Delta \mathbf{x} \in SE(3)$), the 1-DoF null-space redundancy (parameterized as the elbow swivel angle ψ , as introduced in section 4.4) is projected out and lost. When the learned Cartesian policy executes on target hardware, the local inverse kinematics solver must resolve the unconstrained null-space independently (e.g., via joint-limit avoidance or manipulability maximization). Without the original demonstrator’s elbow posture bias, the target arm can drift into mechanical joint limits or sweep its elbow into surrounding fixtures during maneuvers that were completely collision-free in the source demonstrations.

Reusing physical data across embodiments requires a formal compatibility check at the ingestion boundary before any sample enters the training buffer. The ingestion pipeline must transform the recorded trace through forward kinematics to evaluate coordinate consistency (Lynch and Park 2017), verify that all commanded joint velocities and accelerations lie within the target machine’s dynamic limits (Tedrake 2023), and verify that perceptual features match the target sensor’s field of view and resolution. Actions that exceed kinematic reach or actuator effort limits cannot simply be clamped at execution time; clamping distorts the velocity profile and destroys the causal relationship between observed errors and corrective motions. Samples that pass the compatibility check are admitted; samples that can be geometrically re-targeted through inverse kinematics are relabeled; and samples that demand unavailable physical authority or unobservable states are rejected. An unverified ingestion pipeline across an embodiment boundary injects invalid dynamics into the model’s training distribution. The resulting policy does not fail because the neural network underfitted the training loss. It fails because the training dataset contained actions that were physically sound only on the machine that collected them.



Checkpoint 10.6: Embodiment contracts and transfer invariants

- ❑ **Unitless scaling failure:** Can you explain why normalizing actuator commands to $[-1, 1]$ preserves tensor compatibility while guaranteeing physical actuator saturation during cross-embodiment transfer?
- ❑ **Cartesian delta singularity hazards:** Can you describe how universal Cartesian delta actions ($\Delta \mathbf{x} \in SE(3)$) can command infinite joint velocities and trigger inverter overcurrent trips on lightweight arms?
- ❑ **Null-space posture loss:** Can you distinguish the mechanical consequences of projecting a 7-DoF manipulator's trajectory into a 6-DoF Cartesian delta representation?



Systems Perspective 10.5: Kinematic retargeting in teleoperation

Teleoperating a humanoid robot exposes the fundamental physical gap between human demonstrator biomechanics and robot multi-physics embodiment:

- **Non-Isomorphic Kinematics and Singularities:** Human demonstrators operate via complex musculo-skeletal joints (such as the multi-axis shoulder girdle and flexible spinal column) that do not map one-to-one to rigid serial-chain revolute joints. When an operator's VR suit or exoskeleton teleoperates a humanoid arm, Cartesian retargeting algorithms frequently drive the robot's 7-DoF arm into kinematic gimbal lock or wrist singularities ($\det(\mathbf{J}\mathbf{J}^T) \approx 0$). Clamping joint velocities at the singularity boundary distorts end-effector trajectory, while attempting to force tracking draws peak actuator currents that trigger inverter overcurrent trips.
- **Dynamic Center-of-Mass and Balance Discrepancies:** A human demonstrator subconsciously balances by shifting plantar pressure across flexible foot arches and modulating ankle compliance. When teleoperating a 70 kg bipedal robot through a heavy reaching motion (such as lifting a 10 kg crate from a low pallet), the human feels balanced because their center of mass is centered. However, the humanoid robot's heavy steel motors, battery pack, and link masses shift its combined Zero Moment Point (ZMP) outside its rigid foot support polygon. If the teleoperation pipeline records raw human joint motions without a dynamic whole-body filter, replaying or learning from that demonstration commands an unrecoverable forward tip-over.
- **Teleoperation Latency and Causal Confusion:** Over-the-air teleoperation networks introduce 50–150 ms of round-trip latency (τ_{RTT}) between the robot's cameras and the operator's VR headset. When the robot makes contact with an object, the operator does not see or feel the reaction force until 100 ms later, causing them to push harder to overcome perceived resistance. This human-in-the-loop overcompensation records demonstration data where high contact forces correlate with visual delays rather than physical material resistance. Training policies on this uncorrected telemetry produces aggressive, chattering contact behaviors that shatter fragile workpieces during autonomous execution.

Whether collected on a single dedicated workcell or pooled across diverse robot fleets, physical datasets remain vulnerable to silent runtime corruptions that leave no trace in tensor dimensions. In section 10.7, we dissect the primary mechanisms through which physical datasets fail—from sensor calibration drift and unit mismatches to fatal post-collision extrapolation.

10.7 How Datasets Go Wrong

A physical dataset lacks internal markers to indicate what it fails to measure. When an engineer inspects a static corpus of logged robot interactions, standard distribution metrics like feature variance, sample density, and covariance evaluate only the states the machine actually reached. They cannot detect states the machine never encountered. Consider an empirical dataset $\mathcal{D} = \{(\mathbf{s}_k, \mathbf{a}_k)\}$

collected during peg-in-hole assembly, where all recorded contact forces fall within $f_z \in [0.5, 2.0]$ N and lateral offsets within $\Delta x \in [-1.5, 1.5]$ mm. If Deployment Requirement A defines a nominal operational envelope $\mathcal{S}_{\text{req},A}$ bounded by $f_z \leq 2.0$ N and $|\Delta x| \leq 1.5$ mm, the dataset appears fully populated. If Deployment Requirement B instead specifies $\mathcal{S}_{\text{req},B}$ with $f_z \leq 8.0$ N and $|\Delta x| \leq 5.0$ mm to account for pallet tolerances, $\mathcal{D}$ leaves large regions of the required state space completely unvisited. Yet $\mathcal{D}$ itself is identical in both cases. Because the recorded data distribution does not change when the deployment specification expands, internal analysis of $\mathcal{D}$ cannot establish whether coverage is sufficient. Coverage failure is an external mismatch between the required operational envelope $\mathcal{S}_{\text{req}}$ partitioned into discrete evaluation cells and the empirical visitation distribution $\mathcal{S}_{\text{obs}}$. Without an independent, external specification of $\mathcal{S}_{\text{req}}$, coverage failure cannot be detected.

When training data originates from human teleoperation, the data collection policy drifts over time as human operators adapt to the interface. Early in a collection campaign, an operator moves deliberately, introducing corrective sub-movements and pauses while searching for visual cues. After hundreds of repetitions, the same operator develops muscle memory, increasing speed, rounding corners, and executing aggressive feedforward maneuvers. In an analytical example of 500 teleoperated assembly demonstrations gathered over two weeks, median completion time drops from 12.4 s in the first 50 episodes to 5.8 s in the final 50, while the P_{99} commanded angular velocity (how violently the operator whips the joystick) increases from 0.42 rad/s to 1.85 rad/s. The resulting late-stage demonstrations rely on dynamic momentum and precise timing that the physical robot cannot replicate when running under closed-loop latency at deployment. Detecting operator drift requires partitioning the dataset into sliding temporal windows and comparing shifts in episode duration, corrective jerk count, mean command magnitude, and teleoperation intervention rate. If these metrics drift across windows, treating the dataset as an identically distributed sample pool injects multi-modal, non-stationary behavior into the policy, meaning the robot might randomly hesitate or jerk depending on which part of the operator's learning curve it decides to mimic.

File containers such as HDF5, Zarr, or Apache Parquet enforce tabular schema consistency, but they conceal hardware state transitions that occur between recording runs. A robot arm may be shut down overnight, recalibrated, or bumped by a technician, altering the zero-position offsets of its joint encoders or the extrinsics of an overhead camera. If a camera mount shifts by 3 mm, the visual position of the target workpiece changes relative to the robot base, yet the resulting image array matches the recorded tensor shape and type definitions perfectly. Similarly, clock synchronization drifts across distributed microcontrollers over long sessions, causing the actual temporal offset between a force-torque measurement and a camera frame to vary by tens of milliseconds (enough time for a rapidly moving arm to travel several centimeters, hopelessly blurring the connection between what it sees and what it feels).¹⁵ figure 10.7 follows the audit that multi-camera demonstration rigs in heterogeneous environments must run continuously against the extrinsic spatial transformation $T_{\text{cam} \rightarrow \text{base}}$ by projecting the robot's kinematic URDF (Unified Robot Description Format) wireframe directly over raw camera streams. Detecting calibration and timebase discontinuities requires joining every recorded episode against an immutable hardware audit trail containing sensor calibration identifiers, optical reference checks, and clock synchronization health metrics. When a dataset is loaded without verifying that every trajectory shares a common, valid calibration identifier, the learning algorithm is forced to fit conflicting physical mappings within identical numerical coordinates.

Physical data collected across multiple nominally identical workstations frequently harbors silent embodiment mismatch. Two robotic arms from the same manufacturer can have different gearbox wear, thermal dissipation profiles, or firmware velocity filters. In an analytical comparison, Rig 1 with new strain-wave gears exhibits 0.02° backlash, whereas Rig 2 after 2000 hours of operation exhibits 0.35° backlash and a 22 ms phase lag in joint torque response. Actions recorded on Rig 2 include compensatory lead times and higher gain commands that produce severe oscillations when executed on Rig 1. Detecting embodiment mismatch requires verifying a machine fingerprint before any trajectory enters the training pipeline. This fingerprint binds the kinematics, actuator serial numbers, firmware versions, and dynamic transfer function models to the recorded data file. If a trajectory fails the interface contract for the target deployment hardware, the ingestion pipeline must flag and exclude it before training begins.

¹⁵ | **Hardware Timebase Synchronization:** High-end perception architectures use IEEE 1588 Precision Time Protocol (PTP) to synchronize hardware clocks across distributed Ethernet subnets to sub-microsecond accuracy. In contrast, uncoordinated USB and classic CAN buses rely on the host operating system to assign arrival timestamps, introducing variable driver latency and scheduling jitter. When state and action streams drift out of temporal alignment by tens of milliseconds, learned imitation policies fit contradictory causal mappings between visual observations and actuator responses.

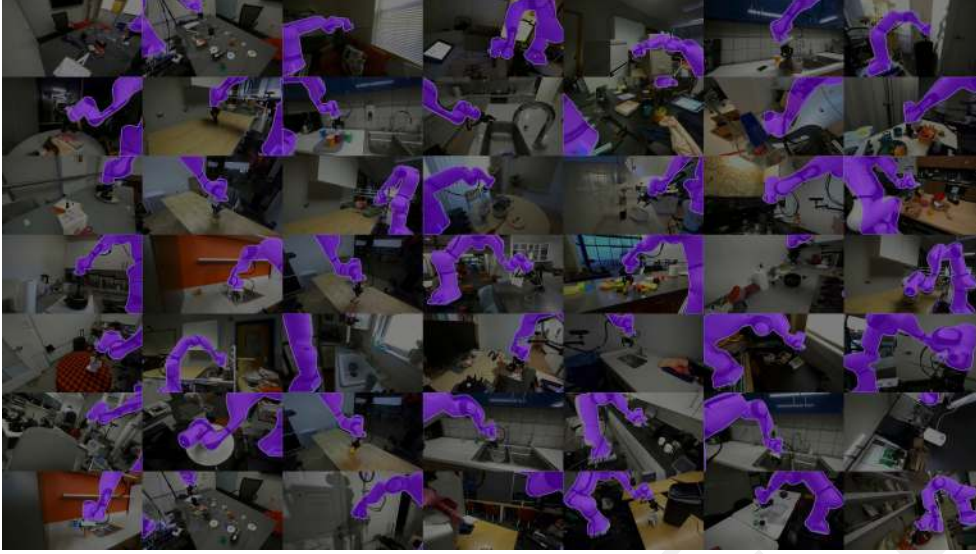


Figure 10.7: Multi-camera extrinsic calibration verification: Six DROID collection scenes with the robot’s URDF mesh rendered from joint encoders directly over the camera stream (Khazatsky et al. 2024). The overlay turns an invisible failure into a visible one, since a bracket displaced by a few millimeters shows up as wireframe-to-arm discrepancy before any corrupted trajectory reaches the training pipeline. Immutable calibration hashes in the dataset record are what make that check auditable after the fact.

A physical collection pipeline generates two distinct classes of failed data that must not be conflated. A failed task attempt occurs when the machine executes a complete physical sequence but fails to achieve the objective, such as dropping a connector during insertion. A failed recording occurs when the logging infrastructure drops network packets, corrupts an image frame, or experiences a buffer overrun on the logging storage bus. Discarding failed task attempts creates severe survivorship bias, stripping the dataset of the recovery behaviors and error distributions needed to train a stabilizing policy. Discarding or retaining failed recordings without accounting corrupts the training timebase, presenting non-uniform sample periods Δt as constant control intervals. To prevent both failure modes, the data ingestion pipeline must enforce strict reconciliation accounting where the total attempt count equals the sum of retained successful trajectories, retained failed task trajectories, and excluded corrupted recordings, with every exclusion tied to a documented hardware or network error code.

The canonical failure mode of silent coordinate and unit contract mismatch is documented in the loss of the Mars Climate Orbiter (Stephenson et al. 1999). Ground software computed angular momentum desaturation thruster impulses in imperial pound-force-seconds ($\text{lbf} \cdot \text{s}$), while the onboard autonomous navigation software ingested the velocity change values assuming metric Newton-seconds ($\text{N} \cdot \text{s}$). Because $1 \text{ lbf} \cdot \text{s} = 4.45 \text{ N} \cdot \text{s}$, the onboard trajectory integration underestimated the cumulative velocity adjustment by a factor of 4.45 over nine months of cruise. The numerical files satisfied all schema and format checks, yet the silent unit contract mismatch accumulated an unobserved 170 km orbital trajectory error, causing the spacecraft to enter the Martian atmosphere at an altitude of 57 km rather than its intended 226 km periapsis, resulting in atmospheric breakup.

When a silent coordinate or unit failure passes through an ingestion pipeline undetected, the resulting policy failure is difficult to diagnose from model loss curves alone. In multi-rig robotic collection, a minor physical shift in sensor mounting or an uncalibrated tool-center-point transform introduces a systematic coordinate discrepancy into the training corpus. The neural network optimizes its objective by averaging the conflicting kinematic demonstrations, producing a policy that converges mathematically on training loss but generates indecisive, oscillatory motions at the physical workpiece interface. Because the model was trained on data containing conflicting physical ground truths, the failure manifests on the real machine as an unexpected limit-cycle oscillation rather than a detectable numerical error.¹⁶

¹⁶ | **Post-Collision Maneuver Investigation:** Formal investigation reports by NHTSA (*Preliminary Evaluation PE23-018: Automated Driving System Post-Crash Maneuver Collision and Pedestrian Injury*, 2023) and the California DMV documented that the autonomous vehicle lacked post-collision sensor support beneath its chassis. The vehicle’s learned policy executed an ungrounded pull-over primitive outside its empirical training distribution, converting an initial emergency stop into a severe secondary dragging incident.

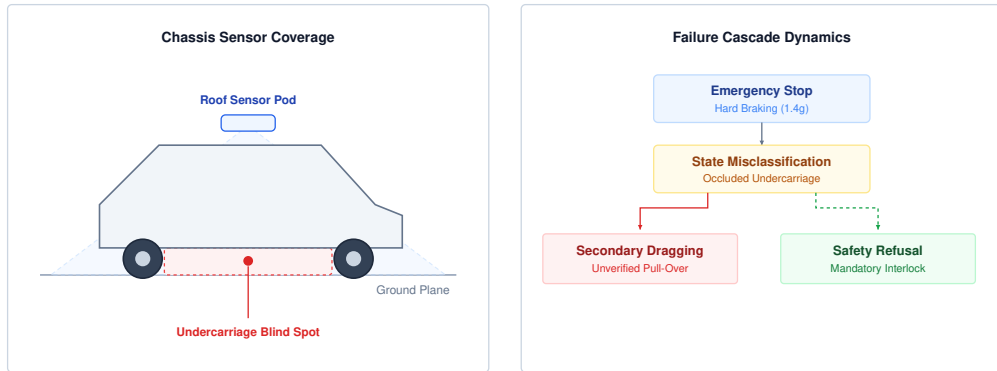


Figure 10.8: Cruise undercarriage blind spot failure: The test vehicle’s roof-mounted sensors survey the perimeter at and above bumper level, leaving a blind spot beneath the floorpan, and the failure cascade that followed. The first stop worked, halting in 0.46 s at 1.4g. What turned a collision into a dragging was the recovery: the state was misclassified as a side-swipe, the nominal pull-over primitive engaged, and drive torque was reapplied without any positive check for undercarriage clearance.



War Story 10.1: Cruise robotaxi pedestrian dragging (2023)

Context: On October 2, 2023, in San Francisco, an autonomous robotaxi operated by Cruise LLC was traveling in an urban lane when an adjacent human-driven car struck a pedestrian, launching her across the lane divider directly into the robotaxi’s path (National Highway Traffic Safety Administration, Office of Defects Investigation 2023; Cruise LLC 2023; Quinn Emanuel Urquhart and Sullivan, LLP 2024).

Mechanism: The robotaxi executed a rapid emergency stop within 0.46 s at 1.4g, coming to rest over the pedestrian pinned beneath the chassis. However, the collision detection system misclassified the event as a lateral side-swipe rather than a collision with an obstacle under the vehicle. Because roof-mounted LiDAR and perimeter cameras had an acute physical blind spot beneath the floorpan, the pedestrian became unobservable. The learned driving policy, trained on nominal fleet demonstrations where pulling out of traffic after an incident is standard practice, selected a nominal pull-over primitive to clear the travel lane. The training distribution contained zero demonstrations of post-collision states with humans lodged under the chassis ($d^{\pi_{col}}(s) = 0$).

Impact: Executing the ungrounded pull-over primitive outside its training support envelope, the robotaxi accelerated forward and dragged the trapped pedestrian for 20 feet (6.1 meters) at up to 7 mph (11 km/h), inflicting severe secondary trauma before coming to a final halt.

Response: Regulatory agencies immediately suspended commercial deployment permits. The company grounded its entire fleet, overhauled its perception and safety software, and instituted hard physical governors requiring positive post-collision interlocks and teleoperation confirmation before any vehicle re-engagement.

Systems lesson: The Cruise incident (figure 10.8) demonstrates the mortal danger of missing support in physical AI datasets. Outside the empirical support envelope of demonstrated states, policies do not fail into safe passivity—they interpolate nominal primitives that convert a survivable incident into a fatal interaction. Datasets must bound their explicit scenario support hulls, and architectures must enforce deterministic control refusal when sensor occlusions and post-impact uncertainty violate support boundaries.

No statistical operation performed on a tensor array can verify that the underlying machine was calibrated, that the human demonstrator did not drift, or that unobserved regions of the workspace remain safe. A physical dataset is reliable only when its collection history and physical unit contracts are recorded as rigorously as the numerical features themselves.

A **support envelope relational join** is the set-theoretic cross-referencing between an empirical state-action occupancy distribution $d^{\pi_{\text{col}}}(s, a)$ (where the machine actually went) and an external scenario ledger $\mathcal{S}_{\text{req}}$ (the checklist of where it needs to go) that partitions the operational design domain into supported nominal regimes ($N_{\text{cell}} \geq N_{\text{min}}$), high-variance transition zones, and zero-support blind spots ($N_{\text{cell}} = 0$) where the robot has no experience and autonomous control authority must be refused.



Checkpoint 10.7: Calibration drift, unit mismatch, and out-of-support failures

- ☐ **Extrinsic drift detection:** Can you explain how reprojecting a robot's URDF wireframe over camera streams exposes mechanical bracket displacement before corrupted trajectories enter training?
- ☐ **Silent unit contract divergence:** Can you describe how the Mars Climate Orbiter's $4.45 \times$ unit mismatch mirrors silent coordinate frame errors in multi-rig robot learning pipelines?
- ☐ **Post-collision out-of-distribution execution:** Can you distinguish the sensing and dataset omissions that caused the 2023 Cruise robotaxi to drag a pedestrian after an emergency stop?

Preventing these silent failures requires replacing ad-hoc logging scripts with a formalized dataset schema. In section 10.8, we establish the immutable metadata structures and multi-stream provenance contracts necessary to bind physical trajectories to verifiable hardware reality.

10.8 The Dataset Schema

The general notebook schema, cryptographic hashing, and immutable logging contracts established in chapter 8 govern how a machine records its execution events. This chapter adds the fields that bind physical movement to a training dataset. A raw array of joint trajectories and camera frames lacks physical validity. Without an explicit data record, an ingestion pipeline treats numbers recorded on a vibrating, misaligned rig as equivalent to numbers collected on a rigid, calibrated testbed. The data record documents what was collected, by whom, on which physical instance, and under what operational limits, establishing the boundary between valid training evidence and mechanical error.

The **demonstration provenance record** is the immutable metadata tuple $(a_{\text{req}}, a_{\text{cmd}}, a_{\text{enf}}, a_{\text{meas}}, \mathbf{h}_{\text{plant}}, \tau_{\text{bus}}, \mu_{\text{contact}})$ logged at every control step. Rather than just recording “what the robot did,” this tuple grounds the data in physical reality by binding behavioral trajectories to specific actuator kinematics, sensor calibration hashes, bus latencies (which can cause micro-stutters), and contact wear states across the Dual-Brain MPU/MCU boundary.



Systems Perspective 10.6: The absence ledger principle

A physical dataset is defined as much by what it omits as by what it contains. If a dataset does not explicitly catalog its unvisited angles, unattempted payloads, and excluded failure trips in an immutable support ledger, downstream verification must treat the missing physics as actively unsafe rather than successfully mastered.

Every trajectory in the training corpus must identify its collection source, collector policy, and execution authority. When data originates from human teleoperation, the record logs the operator identifier, the interface hardware, and the transport timing, such as a 6-DOF master arm sampled at 100 Hz with 0.8 ms of input jitter. When data originates from an autonomous or mixed controller, the record stores the exact policy checkpoint, the planner version, and the active enforcement configuration, including Cartesian bounding boxes, joint velocity limits, and contact-force clamping thresholds. Trajectories must also record the active retention rule, documenting whether an episode was committed automatically upon task completion, retained after an explicit operator review, or flagged for inclusion as an instructive failure recovery.

A physical machine cannot be identified by a generic model name. The data record specifies the individual plant through its extrinsic and intrinsic sensor calibrations, action contract, motor drive

firmware revisions, and component wear state. Replaceable contact surfaces alter the governing dynamics. A parallel-jaw gripper fitted with compliant 30 A durometer silicone pads (which compress easily like a soft rubber eraser) generates contact compliance and friction characteristics that differ entirely from the same gripper fitted with worn 60 A polyurethane pads (which are firmer like a shoe sole) under identical joint commands. Maintenance milestones, such as cable retensioning, drive belt replacements, gearbox swaps, and camera remounts, are logged directly into the machine state record. If an arm underwent joint recalibration between morning and afternoon collection shifts, the dataset must expose that boundary so downstream training does not attempt to resolve two conflicting kinematic mappings into a single policy.

Accounting for dataset volume requires explicit denominators across both time and trial counts. Reporting only the count of retained frames or successful episodes obscures the true yield of the collection process. The data record tracks scheduled cell time, active motion time, attempted episodes, retained successes, retained task failures, excluded recordings, and the count of independent physical task configurations. In an analytical example of a precision peg-insertion campaign scheduled for 40.0 h of cell time, 28.5 h of net motion time produces 1000 total attempts across 12 distinct workpiece fixtures. The resulting tally records 820 retained successes (an 82 percent nominal success yield), 110 retained task failures containing recovery attempts, and 70 excluded recordings rejected due to telemetry drops. Dividing motion time by attempted episodes yields an average cycle budget of 102.6 s per attempt, exposing the actual operational pacing and the proportion of time spent resetting the physical environment.

Building on the causal boundary established in section 2.2, a single action column in a dataset conceals the causal filtering performed by the nervous system. Extending the proposal-permission architecture from section 6.1, the data record preserves four distinct action streams at every control step: what the operator or learned policy asked for (a_{req}), what that request mathematically mapped to after coordinate transforms (a_{cmd}), what the safety governor actually allowed through after clamping hazardous motions (a_{enf}), and what the physical actuators successfully achieved against real-world friction and inertia (a_{meas}). Logging these streams alongside boolean override flags, truncation markers, and numeric exclusion codes reveals where the collector commanded kinematically impossible motions, where the safety enforcer intervened to prevent collisions, and where tracking lag separated command from execution. Discarding the intermediate representations leaves the training pipeline unable to distinguish between an intentional human demonstration and an action heavily modified by an underlying governor.

Every physical dataset operates within a bounded envelope, and the data record must explicitly catalog both supported conditions and known absences. Supported conditions define the verified operating domain, encompassing payload masses between 0.10 kg and 1.50 kg, ambient illumination from 300 lx to 1200 lx, workbench surface friction coefficients, and workpiece dimensional tolerances. Known absences define what was omitted or unattempted: unvisited workspace volumes, approach angles outside a 30° cone, ambient temperatures outside the range of 18°C to 26°C, excluded failure modes such as unrecoverable motor stalls, missing operators who did not participate in the shift, and hardware embodiments that lack kinematic parity (such as robots with slightly different arm lengths or joint limits). Stating what is absent prevents an engineer from treating the absence of training failures as evidence of policy competence. The complete metadata schema and physical contract fields required to preserve this dataset provenance are formalized in table 10.4, defining the metadata structures, calibration contracts, multi-stream action tap points, and operational envelope boundaries required before any trajectory is admitted into training pipelines.

Table 10.4: Demonstration dataset provenance schema: Evidence and admission checks for seven records. Field types and timing rules follow.

Provenance record	Evidence retained	Admission decision
Session	Collector, operator, policy version, sample rate, and timing jitter.	Check operator authorization and the session jitter budget.
Embodiment	Robot identity, geometry, gearing, backlash, and firmware.	Check reach, dynamics, and joint limits on the target plant.
Sensor calibration	Camera geometry, clock synchronization, and force/torque bias.	Check calibration age and timebase error.
Contact wear	End-effector material, hardness, friction, flex cycles, and maintenance history.	Revalidate contact data after material or compliance changes.
Four action taps	Requested, mapped, enforced, and measured values, with override and clamp codes.	Compare like-typed, aligned values; separate governor actions from expert labels.
Transition timing	Six event times with clock IDs and synchronization errors.	Check causal order and the declared response window.
Scenario support	Payload, reach, light, temperature, and known absence bounds.	Refuse unsupported deployment queries pending evidence.

An implementation can store the field-level contract as follows:

- **Session:** `session_authority` stores `collector_type` (enum), `operator_id` (string), `policy_hash` (SHA-256), `sampling_rate_hz` (uint16), and `p99_jitter_ms` (float32). This analytical ingestion rule rejects unqualified operators or session P99 jitter above 2.0 ms.
- **Embodiment:** `embodiment_contract` stores `robot_serial` (string), `dh_parameters` (float32[N,4]), `backlash_deg` (float32[N]), `gear_ratios` (uint16[N]), and `firmware_build` (string).
- **Calibration:** `sensor_calibration` stores `camera_serials` (stringM), `intrinsic_K` (float32[M,3,3]), `extrinsic_T` (SE(3)M), `timebase_sync` (PTP or NTP), and `ft_bias_null` (float32[6]). In this example, an extrinsic calibration older than 24 hours or clock drift above 5 ms fails admission.
- **Contact:** `contact_wear_state` stores `end_effector_sn` and `pad_material` (strings), `shore_hardness` (uint8), `friction_coeff` (float32), `flex_cycle_count` (uint32), and `maintenance_epoch` (uint16). A measured compliance change above 15 percent triggers transfer revalidation.
- **Action taps:** `action_stream_tuple` stores `a_req`, `a_cmd`, `a_enf`, and `a_meas`, plus `override_flag` (bool) and `clamp_code` (uint8). Each tap contains a float32D value, representation, units, and frame ID; raw encoder counts and motor current remain separate plant channels.
- **Timing:** `transition_timing` stores `timestamp_ns[6]` (int64), `clock_id[6]` (string), and `sync_error_us[6]` (float32). The indexed events are exposure midpoint, action issue, mapping, MCU enforcement, response start, and response end. Convert to a shared clock with bounded error; require exposure before issue and enforcement, and response after enforcement within a task-specific window. Reject unalignable transitions.
- **Support:** `support_envelope` stores `payload_bounds_kg` (float32[2]), `reach_cone_deg` (float32), `illum_lux` (uint16[2]), `temp_c` (float32[2]), and `known_absences` (string array).

A short record audit demonstrates why two files with identical tensor schemas and sample counts cannot support the same deployment claims. Consider an analytical audit of two dataset files, File A and File B, each containing 500,000 state-action pairs recorded at 50 Hz (2.78 h of motion) for a dual-arm assembly task:

- **File A** records four certified operators across eight fixture layouts, reports zero safety enforcer overrides, and verifies consistent fingertip durometer across all trajectories.
- **File B** records a single operator at a single fixed jig, logs 620 enforcer interventions where joint velocity limits clamped the requested trajectories, and notes a sensor re-zeroing event midway through collection without recalibrating the extrinsics.

While an automated validation script checking tensor shapes and value ranges accepts both files equally, File A supports a deployment claim across variable workstations, whereas File B documents an overconstrained demonstration corrupted by unmodeled kinematic shifts.

Downstream lifecycle stages consume the data record for distinct engineering guarantees:

- The human and override provenance feeds directly into the supervisory auditing of chapter 28, where safety engineers evaluate demonstrator consistency, teleoperation latency tails, and the frequency of governor interventions to assess human reliability.
- The coverage boundaries, exclusion ledgers, and embodiment calibration histories pass forward to chapter 32, where they form the empirical baseline for the safety case, verifying that the training distribution matches the target operational design domain before the learned policy is granted execution authority on a physical machine.

By coupling physical movement trajectories with verifiable embodiment metadata, multi-stream action tuples, and support boundaries, the provenance schema prevents downstream training pipelines from extrapolating into ungrounded dynamic regimes where the physical plant has never operated.

Downstream: The absence ledger and ODD boundary schema feed the formal safety case claims in section 32.3.



Checkpoint 10.8: Dataset provenance schemas and absence ledgers

- ☐ **Absence ledger principle:** Why does an unvisited region lack direct training support, and what evidence or permission decision is needed before operation there?
- ☐ **Multi-stream action tap forensics:** Can you describe how preserving raw intent (a_{req}), mapped command (a_{cmd}), safety-enforced setpoint (a_{enf}), and measured response (a_{meas}) isolates safety governor interventions?
- ☐ **Data reconciliation accounting:** Can you distinguish the reporting roles of active motion time, attempted cycles, retained successes, retained recovery failures, and rejected hardware dropouts?

With the provenance schema and ingestion contracts established, we consolidate the hard-learned architectural lessons of physical data collection into five common systems fallacies and pitfalls in section 10.9.

10.9 Fallacies and Pitfalls

A clean dataset can omit states the deployed machine must handle. Coverage depends on the collecting policy (the human operator or automated script generating the motions), the physical rig, and the curation decisions that determine which experiences survive in the archive.

Fallacy: *Massive sample volume compensates for lack of operational state-space coverage.*

A surface-tracking task requires data across 500 combinations of force, friction, and contact angle, but the archive covers only 45. Another session adds 72,000 control steps by repeating nominal motions in populated state space cells. The pipeline's ingestion report shows massive data growth with zero dropped packets, but physical coverage remains unchanged. Closely spaced samples describe the same limited physical experience more densely; they do not magically establish behavior in the missing friction or contact conditions. Dataset acceptance must therefore measure added coverage against the operational scenario ledger. More demonstrations earn their collection cost when they address a missing regime or an identified weakness, not merely when they enlarge the archive.

Pitfall: *Assuming physical demonstration labels survive mechanical or contact hardware maintenance.*

A maintenance technician replaces a gripper's compliant fingertips with stiffer pads and adjusts its camera bracket. The archived demonstrations still pass structural schema checks (the data types and array shapes remain perfectly valid), but the physical reality has shifted. A known camera-pose adjustment can support coordinate reprojection for the vision data. However, that geometric correction cannot establish how the new, stiffer fingertips physically behave under contact—they will now slip at lower force thresholds than the compliant ones did. Historical force and tactile measurements remain records of the old body, so their suitability as training targets requires revalidation. Dataset provenance must identify the mechanical and calibration state of the collection rig, allowing maintenance changes to trigger the appropriate review or recollection.

Pitfall: *Filtering out aborted safety interventions to sanitize the demonstration dataset.*

An insertion dataset contains 1000 attempts, including 180 non-nominal runs where contact forces reached or exceeded threshold limits (such as a 15.0 N safety trip), leaving 820 nominal successes (an 82 percent task success yield). A curation script strips out the 180 aborted or recovery trials, leaving only the 820 flawless episodes whose contact forces stayed below 8.0 N. When deployment disturbances later push the robot's contact force to 9.5 N, the policy finds itself immediately outside the retained experience, even though the hardware limit has not yet been reached. The superficially cleaner training set has erased the crucial moments of approaching the physical boundary and the recovery interventions that followed. Retaining and identifying those episodes preserves evidence about failure and recovery; deleting them because they are unsuccessful creates a misleading view of the task.

Pitfall: *Merging demonstrations collected across kinematically asymmetric master interfaces.*

Two operators collect snap-fit demonstrations using different master devices (the hand-held controllers used to puppet the robot). One interface permits only two wrist-roll positions ($\theta_{\text{roll}} \in \{0.0^\circ, 90.0^\circ\}$); the other mechanically limits roll to $[-15.0^\circ, +15.0^\circ]$. Merging their 4000 episodes increases volume, but neither interface supplies the required intermediate orientations between 20.0° and 70.0° . A policy trained on the combined archive still lacks demonstrations for those perturbations. Aggregation cannot remove a blind spot shared by the collection interfaces. The team must map recorded orientations against task requirements and check whether the master devices can express the missing motions before assigning more collection hours.

Fallacy: *Physical data collection scales linearly with labor hours just like internet scraping scales with compute.*

A team projects its bimanual dataset budget from an initial rate of 50 successful episodes per operator-hour. As collection continues, repeated physical impacts damage gearboxes, sensor drift invalidates some demonstrations (as uncalibrated joint angles gradually decouple from physical ground truth), and operator fatigue alters the timing of human motions. The rate of usable episodes falls even though the scheduled labor hours remain unchanged. A short nominal trial therefore does not establish sustained collection throughput. Planning must include maintenance, calibration checks, rest, and rejected data alongside operator time. The useful production measure is accepted experience under documented conditions, since mechanically degraded or poorly aligned recordings can increase storage without increasing the dataset's value.

10.10 Summary

Physical data collection translates mechanical work and hardware fatigue into a robot's empirical limits, or 'support envelopes.' These envelopes define the conditions under which a robot can operate effectively. Loss functions in machine learning cannot account for physical reactions the robot has never encountered due to a lack of counterfactuals—hypothetical scenarios that illustrate alternative outcomes.



Key Takeaways: Physical data collection and provenance

- **Coverage is an empirical visitation record, not a sample count:** Repetitive collection inside safe, easy regions (nominal attractors) inflates tensor row counts without expanding state-space coverage ($\Delta N_{\text{cells}} = 0$). Training corpora must be certified against multi-dimensional scenario ledgers spanning the entire operational design domain.
- **Physical datasets are bound to embodiment mechanics:** Optical calibrations can be reprojected mathematically across geometric transforms, but physical contact mechanics (elastomer squish, surface compliance, and joint backlash) permanently bake specific physical responses into force-torque labels. These labels cannot transfer to a robot with mutated hardware.
- **Safety interventions must be preserved (boundary retention law):** Censoring aborted collection runs or trajectories clamped by safety governors deprives the policy of learning

how to recover from mistakes (boundary recovery dynamics). Datasets must record the complete 4-stream action tuple ($\mathbf{a}_{\text{req}}, \mathbf{a}_{\text{cmd}}, \mathbf{a}_{\text{enf}}, \mathbf{a}_{\text{meas}}$) to expose governor interventions to the learning algorithm.

- **Master interface blind spots create irrecoverable voids:** Aggregating demonstrations across kinematically restricted master devices (e.g., 3-DOF translation pucks vs. 6-DOF arms) leaves structural voids across unvisited rotational degrees of freedom that no downstream scaling can recover.
- **Training cannot synthesize missing physical counterfactuals:** No loss function, diffusion schedule, or geometric data augmentation can generate physics that the collection regime omitted. Explicit support boundaries must accompany datasets to trigger selective runtime abstention when the robot drifts outside its known empirical safety boundary (convex hull).

Data collection in physical systems is an irreversible thermodynamic transaction. Unlike internet text or synthetic images, every physical demonstration consumes non-recoverable mechanical life, induces motor heating, and wears gear teeth. Datasets that conceal boundary interventions or omit failure modes force downstream learning algorithms to extrapolate blindly into dynamic regimes, risking machine destruction from unguided actions.



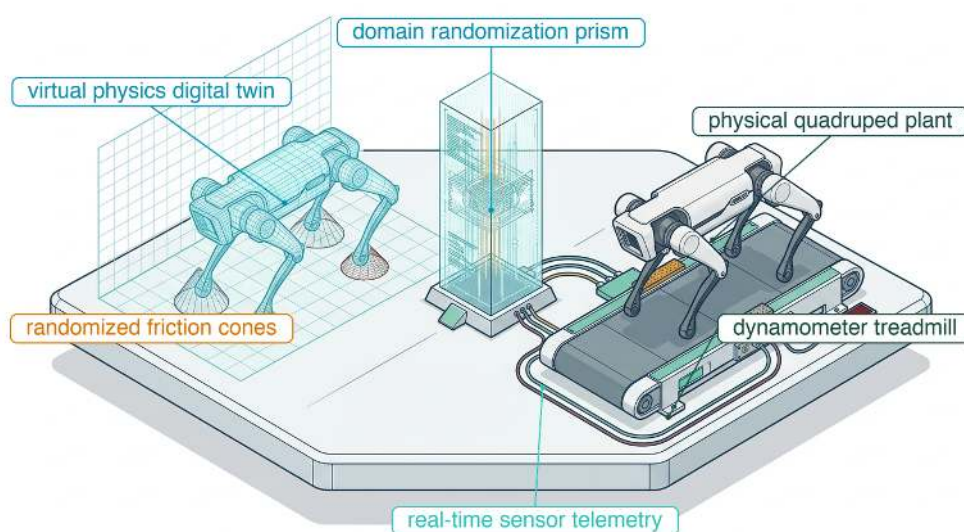
What's Next: From sensory datasets to policy learning

With the physical dataset curated and its empirical support boundaries certified, chapter 12 shifts the systems problem from acquisition to policy synthesis. We examine behavioral cloning, diffusion policy decoders, and sim-to-real domain randomization: analyzing how training runtimes transform finite state-action trajectories into closed-loop control without hallucinating physics that the collection regime omitted.

Policy Training

DRAFT

Policy Training



- 12.1 Policy Synthesis
- 12.2 Policy Synthesis Regimes
- 12.3 Learning by Trial
- 12.4 The Simulator as a System Component
- 12.5 What the Gap Is Made Of
- 12.6 Fine-Tuning on the Real Machine
- 12.7 How Training Choices Go Wrong
- 12.8 The Policy Manifest
- 12.9 Fallacies and Pitfalls
- 12.10 Summary

Purpose

Why does a policy that achieves 100 percent task reward in simulation destroy real gearboxes within minutes of deployment?

Physical trial and error consumes both hardware and computation. An exploratory action can overheat a motor or damage a transmission before the policy receives feedback. Simulation allows that exploration without exposing the real body, but its usefulness depends on which physical behaviors it represents. A policy can earn high rewards by exploiting idealized contact models or unrealistically fast actuator responses. On the machine, the same commands can trigger destructive high-frequency vibrations or mechanical failure. Demonstrations avoid some exploratory risks but leave the policy dependent on the situations the demonstrator encountered. Neither source of experience eliminates the need to measure behavior changes during deployment. Training must account for that difference when choosing examples, simulated conditions, and the form of the actions a policy generates. In physical AI systems architecture, policy training connects computational exploration to the body's response, making transfer to the real machine part of the learning problem.

Brain
1–50 Hz
foundation models, SLAM, planning

above: propose; below: command

Nervous System
1000 Hz
TSN, intent leases, lock-free IPC

Body
continuous
real-time MCU, safety filters, CBFs

Boundary
continuous
sensors, actuators, kinematics

Physical World
continuous
contact mechanics, friction, mass



Learning Objectives

- Compare training regimes by the capabilities they provide and the deployment failures they can create
- Explain why small imitation errors can compound when the policy leaves demonstrated states
- Compare action decoders for representing multiple valid behaviors without averaging incompatible actions
- Diagnose transfer failures by identifying discrepancies between simulator assumptions and real-world conditions, including the forces acting on the system (dynamics), the accuracy of sensors (sensing), the visual representation of objects (appearance), and the timing of actions.
- Assess the process of physical fine-tuning, which involves adjusting the system to account for hardware wear, ensuring that learned skills are retained, and evaluating improvements in behavior based on real-world performance.
- Construct a policy record linking data coverage, training choices, simulator versions, and deployment assumptions

12.1 Policy Synthesis

Two learned controllers can achieve identical validation loss on offline benchmarks yet fail catastrophically for divergent reasons when deployed on the same physical plant. Consider two neural policies designed to regulate high-pressure liquid flow through a proportional control valve before a robotic placement head seats an automotive component onto an elastomeric seal. Both policies achieve a validation score of 0.002 mean squared error on their training distributions. The first policy was trained via behavioral cloning on fifty hours of expert human demonstrations. The second was synthesized through reinforcement learning inside a high-throughput physics simulator over two billion transitions.

On a physical calibration bench under nominal pressure, both policies regulate flow at 1.50 L/min. But when a line transient suddenly drops supply pressure by 12 kPa, their failure modes diverge violently. The imitation policy issues a corrective valve adjustment that slightly undershoots, landing in a joint pressure-flow state absent from its training demonstrations. Because the policy cannot extrapolate beyond its demonstrated support manifold, its subsequent actions become erratic, compounding tracking error across consecutive 20 ms control steps until the fluid reservoir overflows. The simulated policy, having learned inside an idealized simulation that assumes instantaneous electromagnetic response, commands valve position reversals at 50 Hz. The physical solenoid spool requires 35 ms to transit between seating positions; unable to track the commanded high-frequency chatter, the mechanical valve seat experiences violent cavitation, rupturing an upstream hydraulic seal. The data collection regime defined what the policy saw, while the training algorithm shaped its assumptions about the physical universe.



Definition 12.1: Actuator dynamics latency robustness

Actuator dynamics latency robustness describes a policy tested across a specified range of transport delays, phase lags, and mechanical compliance. Training with randomized delay can improve performance in that range; closed-loop stability still requires measured phase margin and validation on the physical plant.

1. **Significance:** Prevents simulated policies from exploiting zero-latency communication and idealized instantaneous torque generation, which cause violent phase-lag oscillations and mechanical fatigue on physical actuators.

2. **Distinction:** Unlike general domain randomization of visual textures or masses, latency invariance specifically targets the dynamic phase margins of the closed-loop system by randomizing observation-to-action delay buffers during training.
3. **Common pitfall:** Training policies in simulation with fixed single-timestep delays, leaving the controller fragile to real-world bus jitter and variable inference latencies that cross stability margins.

Every training regime purchases deployment capability at the cost of runtime vulnerability:

- **Imitation learning** buys task competence without manual reward engineering by mimicking human demonstrations (drawn from the ingestion archives established in chapter 10). However, standard point-regression objectives suffer from the *multimodality catastrophe*—if demonstrators avoid an obstacle by passing either to the left or to the right, minimizing mean squared error forces the neural network to predict the geometric average of both modes, commanding the robot to drive straight into the obstacle.¹ Furthermore, small single-step errors compound quadratically over time as the policy drifts into unsupported state space.
- **Simulation-based reinforcement learning** buys unconstrained exploration, generating billions of exploratory state transitions across parallel GPU environments without risking hardware damage. Yet physics engines rely on numerical approximations—discrete contact solvers that treat rigid metals as spongy spring-damper penetrations, simplified Coulomb friction models, and zero-latency communication. Neural policies optimize rewards with mathematical ruthlessness, aggressively exploiting these numerical solver artifacts to discover non-physical control strategies that collapse into chatter, slip, or mechanical failure on real hardware.²
- **Real-hardware fine-tuning** provides authentic physical contact and true sensor noise, but exploratory trial and error inflicts destructive mechanical wear on transmissions, elevates motor winding temperatures via resistive heating, and risks unrecoverable physical collisions.

Every assumption embedded in a training pipeline—whether an idealized contact solver, an averaging loss function, or a truncated disturbance distribution—represents a latent debt. Physics defers collection while computation runs in the cloud or across simulation clusters, but collects the entire balance at line rate the instant the policy actuates physical mass.

In the four-tier systems stack (figure 12.1), policy training bridges the cognitive deliberation tier (the Brain, Layer 3, operating at 10 to 50 hertz) with the continuous plant (the Body, Layer 1). The reality gap highlighted between them is the core physical obstacle that training must overcome. A policy synthesized against synthetic simulations or recorded human demonstrations acquires operational authority only over the real plant.

Crucially, a deployed policy does not actuate motors directly in an unconstrained loop. Under the proposal-permission architecture,³ high-level neural policies emit proposed multi-step trajectory chunks. An independent real-time permission path checks measured-state feasibility and validated bounds before motor drivers receive current setpoints. An application processor and 1000-hertz microcontroller are one implementation of that split. A training regime that fails to model this architectural boundary will synthesize policies that demand impossible torque spikes or trigger continuous safety tripwires during execution.

The valve failures leave a practical choice: which experience source teaches the needed recovery without hiding actuator dynamics?

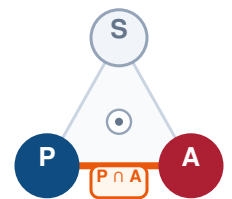
We compare demonstration, offline, simulated, and on-machine learning by the failures each can expose and the physical evidence needed before granting actuator authority.

12.2 Policy Synthesis Regimes

¹ | **Multimodality catastrophe:** Minimizing mean squared error over demonstrator trajectories forces the network to output the conditional expectation $\mathbb{E}[a | o]$ rather than sampling from the true conditional distribution $p(a | o)$. In continuous action spaces, this expectation frequently falls into zero-probability regions between valid modes. Executing this averaged action drives the robot directly into obstacles that both human demonstrators successfully avoided.

² | **Physics engine reward hacking:** RL agents exploit discrete integration timesteps by driving joints into numerical penetration regimes where penalty springs generate non-physical restitution forces. These artificial accelerations allow policies to achieve high task rewards in simulation through energy-generating artifacts. Transferring these unconstrained motor commands onto physical hardware triggers instantaneous overcurrent trips and gearbox teeth shearing.

³ | **Proposal-permission architecture:** An independent permission path can enforce validated bounds when the measured state, actuator authority, and fallback remain feasible. It checks proposals before actuator writes and rejects or modifies those outside the validated envelope; a barrier projection is conditional on its model and available control authority.



The Plan $\cap$ Act motion interface: grounding learned policy tokens into continuous physical trajectories.

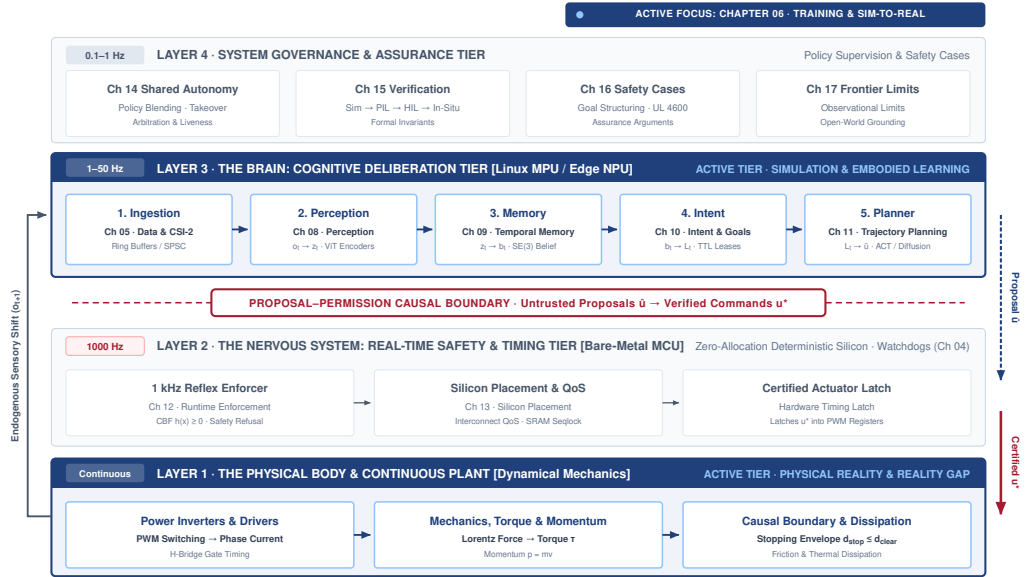


Figure 12.1: Reality gap in stack: The four-tier physical AI systems architecture (Brooks 1986b) highlights the cognitive deliberation tier together with the physical body beneath it, with the reality gap between them as this chapter’s active focus. Read the highlighted pair as the two ends of every sim-to-real transfer, because a policy synthesized against simulated dynamics acquires authority only over the real plant.

Table 12.1: Comparative trade-off matrix across policy synthesis regimes: Systematic evaluation of behavioral cloning, offline reinforcement learning, sim-to-real reinforcement learning, and residual RL/fine-tuning. Columns detail the primary operational capability purchased during training, representative sample and compute budgets, sample efficiency bounds, safety risks incurred during data acquisition, and dominant failure signatures with corresponding nervous system interceptors.

Policy Synthesis Regime	Core Purchased Capability	Data & Compute Budget	Sample Efficiency & Scale	Hardware Safety & Wear Risk	Dominant Failure Mode & Interceptor
Behavioral Cloning (BC)	Direct imitation of demonstrated trajectories without reward engineering or system identification	10^2 – 10^3 teleop trajectories (~ 10 – 50 h data); < 50 GPU-h compute	Supervised $\mathcal{O}(N)$ over expert demonstrations; fixed $1 \times$ real-time data throughput	Minimal exploration risk during logging; zero hardware exploration during optimization	Quadratic compounding error ($\mathcal{O}(T^2 \epsilon)$) via covariate shift (where small errors push the robot into unfamiliar states, causing progressively larger errors). <i>Interceptor:</i> Kernel/Mahalanobis density monitors (statistical checks to ensure the robot remains in a familiar state), convex hull boundary clamps (hard limits to keep the robot within demonstrated physical bounds)

Policy Synthesis Regime	Core Purchased Capability	Data & Compute Budget	Sample Efficiency & Scale	Hardware Safety & Wear Risk	Dominant Failure Mode & Interceptor
Offline / Batch RL	Value-guided policy optimization over fixed, heterogeneous sub-optimal logs; stitches multi-trajectory transitions	10^5 – 10^7 logged transitions with state-action coverage; 100–500 GPU-h compute	Offline $\mathcal{O}(N)$ over static datasets; sample-efficient but demands high state-action overlap	Zero active exploration hazards during offline training; logging requires caged rigs or safe scripted controllers	Out-of-distribution action extrapolation and Q-value overestimation (guessing that an untried action will yield impossibly high rewards). <i>Interceptor:</i> Conservative Q-learning (CQL) penalty bounds (mathematical penalties for guessing actions not in the dataset), action support filters
Sim-to-Real RL	Discovery of dynamic, non-prehensile manipulation and agile locomotion via billions of unconstrained rollouts	Zero physical data; 10^7 – 10^9 simulated transitions across 10^3 – 10^4 GPU environments; 500–5000 GPU-h	Low algorithmic sample efficiency (10^7 – 10^9 steps), offset by $10^3 \times$ – $10^5 \times$ simulation acceleration	Zero hardware wear or collision damage during training; severe deployment risk if plant dynamics diverge	Exploitation of numerical solver artifacts (contact interpenetration, zero latency). <i>Interceptor:</i> Paired-trace discrepancy budgets, force-derivative (dF/dt) clamps
Residual RL & Real Fine-Tuning	Direct policy correction against physical contact compliance, gearbox hysteresis, and true sensor noise	10^3 – 10^5 physical steps (0.5–10 h wall-clock time); < 20 GPU-h compute	High sample efficiency required (SAC/TD3 or residual delta-action formulation); strictly $1 \times$ real-time	High physical exposure; repetitive impact shocks degrade gearboxes; exploratory current dither elevates winding temperature	Catastrophic forgetting of nominal transit stability, local policy collapse. <i>Interceptor:</i> Parameter drift bounds ($\ \Delta\theta\ _2 \leq \delta$), frozen regression suites, thermal limiters

As compared across synthesis paradigms (table 12.1), **behavioral cloning** is the cheapest route to a working policy and the one whose failure mode is best understood. It frames policy synthesis as supervised regression or classification over a collected dataset of expert state-action pairs $\mathcal{D} = \{(\mathbf{o}_t, \mathbf{a}_t)\}$. Grounding this in the telemetry architecture from section 10.3, physical demonstration datasets are constructed from explicit action tap streams:

$$\mathcal{D}_{\text{intent}} = \{(\mathbf{o}_t, \mathbf{a}_{\text{cmd}, t:t+H-1})\} \quad \text{versus} \quad \mathcal{D}_{\text{verified}} = \{(\mathbf{o}_t, \mathbf{a}_{\text{enf}, t:t+H-1})\}$$

Cloning unconstrained operator commands ($\mathbf{a}_{\text{cmd}}$) trains the model on pure human demonstration intent, whereas cloning enforcer-admitted setpoints ($\mathbf{a}_{\text{enf}}$) bakes the nervous system’s runtime barrier deceleration profiles directly into the network weights. In its standard formulation, the optimization objective minimizes a multi-step prediction loss over continuous action sequences drawn from demonstration trajectories (see Appendix A for classical regression formulations).⁴ Because this loss penalizes only the discrepancy between the commanded action and the recorded demonstration at observed states, the training objective does not evaluate task completion, terminal cost, or the stability of the closed-loop dynamical system. It fits actions to observations only.

⁴ | **ALVINN road perspective warping:** The fundamental vulnerability of behavioral cloning to covariate shift was documented in Dean Pomerleau’s seminal ALVINN (Autonomous Land Vehicle in a Neural Network) project at Carnegie Mellon University in 1989 (Pomerleau 1989). ALVINN demonstrated that a neural network trained purely on human driving logs along centered road segments failed rapidly because minor steering inaccuracies drifted the vehicle toward the road shoulder—a region never visited by the expert driver. To prevent catastrophic compounding error without running unsafe hardware exploration, Pomerleau pioneered synthetic perspective warping, rendering laterally shifted and rotated road images with mathematically transformed steering corrections to provide recovery demonstrations off the nominal trajectory, establishing data augmentation as an essential prerequisite for closed-loop behavioral stability.

Prerequisite: The four-stream action tap tuple separating intent from enforcement is formulated in section 10.2.

The multimodality breakdown and mode averaging

A deterministic point-regression behavioral-cloning policy cannot represent distinct valid actions for the same observation. Under squared-error loss, its optimal output is the conditional mean, which can be invalid when valid actions lie on separate paths (for the derivation, see Appendix A).⁵

When human demonstrators perform a physical task, their strategy is frequently multimodal. Consider an obstacle avoidance scenario where a mobile base or robotic arm must navigate around a central pillar. A human demonstrator may choose to pass around the pillar to the left in half of the demonstrations, or to the right in the remaining half. The true demonstration data contains distinct valid choices, but zero examples of driving straight down the middle.

When trained on this dataset with an averaging regression loss, the model calculates the midpoint between the “steer left” and “steer right” commands. The policy outputs zero lateral velocity, commanding the machine to drive directly into the obstacle. The conditional mean represents an unviable, catastrophic action that never occurred in the demonstration dataset. Across physical machine archetypes:

- In *Mobile Systems*, the vehicle drives into pillars, bridge abutments, or dividing medians.
- In *Manipulators*, mode averaging causes end-effector grippers to command non-functional intermediate grasp poses that straddle an object’s outer boundary, dropping workpieces or jamming tooling into rigid assembly fixtures.
- In *Process Systems*, averaging between two discrete valve operating regimes (10 percent bypass vs. 90 percent main feed) commands an uncalibrated 50 percent intermediate flow that cavitates the supply pump and violates cooling deadlines.

Temporal inconsistency can arise in a high-frequency single-step controller near a learned mode boundary. In a constructed 50–100 Hz example, sensor noise moves consecutive observations across that boundary, and a policy mapping only $o_t \rightarrow a_t$ alternates between $+10 \text{ N} \cdot \text{m}$ and $-10 \text{ N} \cdot \text{m}$ requests. Whether those requests produce resonant motion, wear, or excess winding heat depends on the drive response and load; the example identifies a failure mode to test, not a property of every Markovian policy.

Compounding covariate error and state support drift

Beyond the spatial failure of mode averaging, behavioral cloning suffers from a second, structural vulnerability under closed-loop physical execution: compounding covariate shift. Consider a positioning system that regulates the height of a dispensing nozzle over a moving substrate at 100 Hz before touching the surface to deposit adhesive. In a clean demonstration dataset, an expert operator or an engineered classical controller maintains the nozzle within a narrow operating band of $\pm 0.2 \text{ mm}$ around the reference trajectory. A learned policy evaluated on a held-out test split of this dataset can achieve a mean squared error near zero, predicting the nominal motor current with high fidelity. That low validation loss measures accuracy only on states visited by the demonstrator. It contains zero samples of the machine recovering from actuator current saturation, sensor noise, transport delay, or an incorrect command issued three time steps earlier. The policy learns to stay near the nominal setpoint but not how to return once displaced.

Let the policy operate in discrete time over an execution horizon of T control steps, issuing an action a_t at each interval Δt . Suppose that under the demonstrated distribution of observations, the policy has an independent probability ϵ of making an action error at any single time step, where an error is defined as an action that deviates from the expert envelope by enough to perturb the physical state beyond nominal bounds. The probability of committing at least one error across the trajectory grows as $1 - (1 - \epsilon)^T$. Once an error is made, the machine drifts off-distribution, incurring a cost that compounds at every subsequent step. Under these worst-case assumptions, the expected cumulative trajectory cost J has an upper bound of $\mathcal{O}(T^2\epsilon)$.⁶

The quadratic dependency on time T^2 represents the structural price of open-loop supervised training applied to a closed-loop dynamical system. The first error incurs a direct penalty, and every subsequent step spends the remainder of the rollout accumulating cost because the demonstration dataset never provided examples of how to return to the nominal path.

Ross et al. (2011) derives a worst-case $\mathcal{O}(T^2\epsilon)$ expected-cost bound for expert-only behavioral cloning under bounded one-step loss because the learner’s own execution errors induce covariate

⁵ | **Mode averaging under L_2 loss:** Under an L_2 regression loss, the expected risk $\mathcal{R}(\hat{a}) = \int \|\hat{a} - a\|_2^2 p(a | o) da$ is strictly convex with respect to $\hat{a}$. Differentiating with respect to $\hat{a}$ and setting the gradient to zero yields $2 \int (\hat{a} - a) p(a | o) da = 0 \implies \hat{a}^* = \int a p(a | o) da = \mathbb{E}[a | o]$. For a nonlinear plant f_{plant} , $\mathbb{E}[f_{\text{plant}}(a)]$ generally differs from $f_{\text{plant}}(\mathbb{E}[a])$; equality can hold in particular distributions. For a convex action cost $c(a)$, Jensen’s inequality gives $c(\mathbb{E}[a]) \leq \mathbb{E}[c(a)]$, but it does not establish the physical safety of the mean action. In the obstacle example, the mean lies outside the demonstrated set of valid paths.

⁶ | **Compounding covariate cost derivation:** To derive the standard cumulative cost bound analytically, let $c(s_t, a_t) \in [0, 1]$ represent a bounded instantaneous cost incurred at step t , where $c = 0$ corresponds to matching expert behavior and $c = 1$ denotes a task failure. At step $t = 1$, the policy starts on the demonstrated distribution, so the probability of making an error is bounded by ϵ . At step t , if the policy has made an error at any previous step $\tau < t$, the machine may have drifted off-distribution, potentially incurring a cost of up to 1 at every subsequent step. Under the assumption that making an error introduces a transition to an off-distribution state and that the policy cannot recover on its own, the probability that the system has left the demonstrated support by step t is bounded by the union bound of errors over preceding steps, which grows as $t\epsilon$. Summing the expected cost across all T steps yields the cumulative trajectory cost J : $J = \sum_{t=1}^T \mathbb{E}[c(s_t, a_t)] \leq \sum_{t=1}^T t\epsilon = \epsilon \frac{T(T+1)}{2} = \mathcal{O}(T^2\epsilon)$. This is a worst-case expected-cost bound under the stated support-loss assumptions, not a fixed safe execution horizon.

shift away from the demonstration distribution. To overcome this fundamental limitation in software, the authors formulated DAgger (Dataset Aggregation), an interactive online imitation learning framework that queries an expert oracle to label the states visited by the learner's policy, obtaining a linear $\mathcal{O}(T\epsilon)$ bound under its expert-labeling and no-regret assumptions. In physical robotics, however, running an unverified policy to gather online expert labels is hazardous, as off-distribution exploration risks mechanical collisions and hardware damage before the human supervisor can intervene.

12.2.1 What action chunking changes—and what it leaves unresolved

Action chunking—predicting a sequence of future actions rather than one action—can improve temporal coherence and reduce proposal frequency in generative architectures such as diffusion policies and Action Chunking with Transformers (ACT). It does not change the behavioral-cloning support bound $\mathcal{O}(T^2\epsilon)$ by itself.

In single-step imitation learning, the policy predicts one action a_t at each control cycle. If a mistake carries the machine outside demonstrated support without recovery, the worst-case cost bound grows as $\mathcal{O}(T^2\epsilon)$. The constructed mode-boundary example above shows a second risk: if a single-step policy is sensitive to noise near two valid demonstration modes, consecutive observations can elicit alternating commands. The resulting physical motion depends on the controller, actuator, and load.

Now consider an **action chunking** policy (such as Action Chunking with Transformers (Zhao et al. 2023) or Diffusion Policy (Chi et al. 2024)) that predicts a sequence of H future actions $\mathbf{a}_{t:t+H-1}$ simultaneously. Action chunking can improve execution through three mechanisms, each conditional on the decoder and execution design:

1. **Temporal and Dynamic Consistency:** Rather than generating disjoint single-step predictions that chatter, the model predicts an entire coordinated trajectory. A coherent multi-step proposal can reduce decision chatter when the execution path enforces feasible interpolation and rate limits.
2. **Maintaining a Chosen Mode:** Single-step Markov policies easily fall prey to causal confusion—mistaking momentary spurious correlations for task intent. Conditioning on trajectory history and proposing a chunk can help preserve a chosen mode across adjacent actions; it does not prove that the model identified task intent.
3. **Receding-Horizon Replanning and Temporal Ensembling:** In robust physical deployments, a robot does not execute all H steps blindly without observation. Instead, it operates in a **receding-horizon** loop: the policy queries a new chunk every k steps ($k \ll H$, e.g., predicting $H = 16$ actions but querying every $k = 1$ or $k = 4$ steps) and blends overlapping predictions using **temporal ensembling**⁷ (an exponentially weighted average across overlapping chunks). This permits periodic feedback while retaining a multi-step proposal, but recovery from states absent in the training data still requires demonstrated recovery behavior or verified runtime monitoring.

⁷ | **Action chunking and receding-horizon ensembling:** Overlapping chunks may be re-queried every k steps and blended for temporal consistency. Neither the chunk horizon H nor ensembling changes the expert-only support assumption behind the behavioral-cloning cost bound.

Napkin Math 12.1: One-step error probability and physical drift

Problem statement: An autonomous mobile delivery robot makes decisions at 50 Hz ($\Delta t = 20$ ms) for 10.0 s. On expert-visited observations, suppose its behavioral-cloning policy has a one-step action-error probability $\epsilon = 0.005$. For this illustrative calculation, assume errors are independent and the robot remains in demonstrated support until the first error. What is the probability of at least one action error during the transit?

The number of decision opportunities is $T = t_{\text{total}}/\Delta t = 500$. Under those assumptions,

$$P(\text{at least one error}) = 1 - (1 - \epsilon)^T,$$

which evaluates to 91.8 percent. An action error need not move the robot outside demonstrated support, cause a collision, or persist as a lateral acceleration. Those physical outcomes depend on the plant dynamics, current state, clearance, controller response, and actuator limits.



Figure 12.2: Support shift and cumulative cost: Panel (a) illustrates one possible physical departure from demonstrated support $\mathcal{S}_{\text{demo}}$ and a recovery path; its state-space curves are not cost bounds. Panel (b) compares worst-case expected cumulative cost bounds under bounded one-step loss and the stated support-loss and expert-labeling assumptions: $\mathcal{O}(T^2\epsilon)$ for expert-only cloning and $\mathcal{O}(T\epsilon)$ for interactive imitation.

Systems insight: A one-step validation score can estimate this first-error probability under the stated independence and support assumptions. It cannot establish displacement or a safe transit horizon. Recovery behavior and runtime permission must be tested against measured state transitions and corridor clearance.



Systems Perspective 12.1: Expected first-error horizon

If one-step errors are independent with probability ϵ , $1/\epsilon$ is an approximate expected waiting time to the first error. It is not a safe-run limit; safety depends on the resulting state, recovery behavior, and available physical margin.



Checkpoint 12.1: Compounding drift budget and recovery dynamics

- ☐ **Compounding error scaling:** Can you state the support-loss and expert-labeling assumptions under which the expected cumulative cost bounds are $\mathcal{O}(T^2\epsilon)$ for expert-only cloning and $\mathcal{O}(T\epsilon)$ for interactive imitation?
- ☐ **Action chunking error suppression:** Can you distinguish the temporal-coherence benefit of action chunks from the recovery data or monitoring needed to address support drift?
- ☐ **Interactive expert interventions:** Can you explain how DAgger interventions expand policy support to boundary states without sacrificing nominal performance?

Figure 12.2 follows the endogenous cascade this dynamic feedback creates where a single perturbation at step τ shifts the physical state outside demonstrated support $\mathcal{S}_{\text{demo}}$, transferring execution into an unconstrained out-of-distribution regime. Under the stated bounded-cost assumptions, expert-only cloning has a worst-case $\mathcal{O}(T^2\epsilon)$ expected cumulative cost bound; interactive imitation with suitable expert labels can obtain an $\mathcal{O}(T\epsilon)$ bound. Neither bound specifies the shape of a physical state trajectory.

Increasing dataset size, model capacity, or tuning can reduce the one-step error rate on evaluated observations and therefore lower the chance of a first policy error during a fixed horizon. That improvement does not characterize what happens after an error. An external disturbance, uncalibrated payload, or voltage sag can also move the machine into weakly supported states without a policy disagreement. Recovery data, measured dynamics, and independent runtime limits determine whether either event remains within the physical envelope.

This failure mode poses an operational hazard as it occurs silently. Unlike a software crash or an unhandled exception, a neural network evaluated on an out-of-distribution observation does not halt execution or raise an alarm. It continues forward inference, multiplying matrices and outputting numerical tensor values that are syntactically valid floating-point numbers within normal actuator voltage limits. The nervous system sees valid commands, while the physical state drifts steadily away from the demonstration envelope until the mechanism encounters a hard mechanical stop, stalls against a surface, or triggers an overcurrent fault. Detecting this condition before physical contact occurs cannot be accomplished by monitoring the offline training loss. It requires runtime telemetry that tracks state space support directly, measuring Mahalanobis distance in feature space, estimating model epistemic uncertainty, or enforcing an explicit envelope monitor in the nervous system that trips when state trajectories leave the convex hull of the demonstration dataset.

Overcoming imitation failure modes: The action decoder paradigms

When distinct valid actions coexist, a point-regression decoder can average them into an invalid command. The following decoder choices represent alternatives and trade inference cost against the action distribution they can express. None supplies missing recovery states by itself:

1. **Explicit Gaussian Mixture Models (GMM / Mixture Density Networks):** A Gaussian Mixture Model is a probabilistic model that assumes that the data points are generated from a mixture of several Gaussian distributions. This approach models multimodal outputs, allowing multiple appropriate actions in a given context. Gaussian distributions provide the foundational building blocks for GMMs and capture variability in action predictions. Parameterizes the policy output as a conditional mixture of M Gaussian components:

$$\pi(a | o) = \sum_{m=1}^M w_m(o) \mathcal{N}(\mu_m(o), \Sigma_m(o))$$

where the neural network predicts categorical mixture weights $w_m(o)$, component mean vectors $\mu_m(o)$, and covariance matrices $\Sigma_m(o)$. *What it buys:* Closed-form analytical log-likelihood, exact probability density evaluation, with a single forward pass. Whether it meets a high-rate deadline depends on the backbone and measured device latency. *What it charges:* Vulnerable to mode collapse and numerical singularity during maximum likelihood training; struggles to scale to high-dimensional multi-step trajectory spaces; requires fixing the component count M *a priori*, failing when demonstrations exhibit varying degrees of branching complexity.

2. **Implicit Behavioral Cloning (IBC / Energy-Based Models):** Rather than predicting continuous actions directly, IBC (Florence et al. 2022) formulates policy evaluation as energy minimization over an implicit energy-based model $E_\theta(o, a) \in \mathbb{R}$. The network is trained with contrastive or InfoNCE objectives to assign low energy to demonstrated state-action pairs and high energy to counterfactual actions. At inference time, an action proposal is selected by approximately solving a continuous optimization:

$$a^* = \arg \min_a E_\theta(o, a)$$

What it buys: Natively represents arbitrary discontinuous, sharp, multimodal probability distributions and complex non-convex boundaries without mode averaging; captures multi-valued topologies where continuous feedforward mappings mathematically fail. *What it charges:* Inference may require iterative search over candidate actions; the cost and quality of that search depend on the optimizer, candidate count, and device. A non-convex energy surface can return a local rather than global minimum, so the selected proposal still needs physical feasibility checks.

3. **Continuous Trajectory Diffusion and Flow Matching (Diffusion Policy / Rectified Flow):** Diffusion Policy treats policy synthesis as conditional denoising diffusion over continuous multi-step action trajectory chunks conditioned on an observation history (Chi et al. 2024), as schematized in figure 12.3. The loss target distinguishes direct regression from diffusion. In standard BC, error is measured directly across the multimodal action space, forcing the

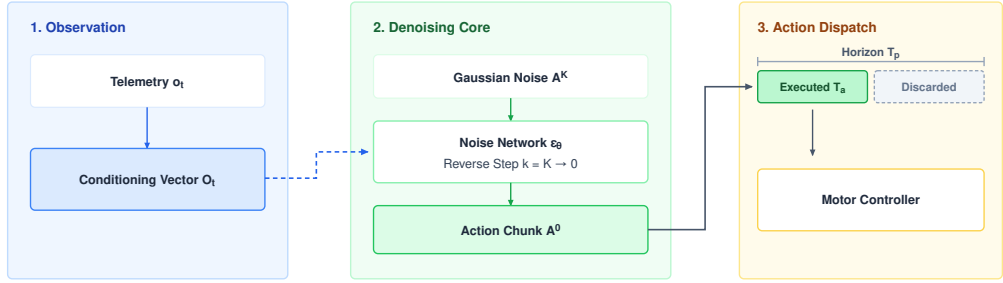


Figure 12.3: Diffusion policy architecture: A continuous trajectory diffusion policy for visuomotor control, denoising a candidate action chunk from Gaussian noise across K iterations and dispatching only a prefix of it before re-planning. Asynchronous buffering and an independent high-rate permission path can separate slower chunk generation from actuation. Conditional denoising can represent alternative demonstrated paths without taking their direct-regression mean. (Adapted from Chi et al. (Chi et al. 2024)).

network to predict the physical centroid (like averaging “steer left” and “steer right” into “steer straight”). In Diffusion Policy, the network predicts the injected noise conditional on the corrupted trajectory, observation, and diffusion step.

During training, a forward diffusion process corrupts a ground-truth demonstrated trajectory chunk $\mathbf{A}_t^0 \in \mathbb{R}^{K \times \bar{D}_a}$ across $k \in [1, N_{\text{diff}}]$ steps using variance schedule $\bar{\alpha}_k$:

$$\mathbf{A}_t^k = \sqrt{\bar{\alpha}_k} \mathbf{A}_t^0 + \sqrt{1 - \bar{\alpha}_k} \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

The noise-prediction network ϵ_θ (parameterized as a 1D temporal convolutional neural network (CNN) or Transformer decoder) minimizes the mean squared error between injected and predicted noise:

$$\mathcal{L}_{\text{diff}}(\theta) = \mathbb{E}_{k \in [1, N_{\text{diff}}], \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), (\mathbf{O}_t, \mathbf{A}_t^0) \sim \mathcal{D}} \left[\|\epsilon - \epsilon_\theta(\mathbf{A}_t^k, \mathbf{O}_t, k)\|^2 \right]$$

The squared-error target is the injected noise conditional on the noisy trajectory, observation, and diffusion step, rather than the mean of incompatible clean actions (for diffusion fundamentals, see Appendix A). At inference time, sampling begins from random noise $\mathbf{A}^{N_{\text{diff}}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and follows repeated conditioned denoising updates. Different initial samples can resolve into left- or right-side obstacle paths instead of averaging the two into a collision path. Physical smoothness and feasibility still require suitable training data and execution checks.

Continuous flow matching extension: While standard diffusion models rely on stochastic differential equations (SDEs) where Brownian noise forces trajectories to meander along winding, non-linear diffusion paths, **continuous flow matching** (Lipman et al. 2022) can train on straight conditional probability paths, including paths defined by an optimal-transport coupling. In the worked formulation, it uses a linear interpolation between standard Gaussian noise $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and demonstration action chunk $\mathbf{x}_1 \sim \mathcal{D}$:

$$\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1, \quad \text{for } t \in [0, 1]$$

The time derivative of this straight line is constant: $\dot{\mathbf{x}}_t = \mathbf{x}_1 - \mathbf{x}_0$. A neural network $v_\theta(\mathbf{x}_t, t, \mathbf{O}_t)$ is trained with simple mean squared error to predict this constant velocity vector field:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t \sim U(0, 1), \mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \mathbf{x}_1 \sim \mathcal{D}} \left[\|v_\theta(\mathbf{x}_t, t, \mathbf{O}_t) - (\mathbf{x}_1 - \mathbf{x}_0)\|^2 \right]$$

Straighter conditional paths can permit fewer ODE evaluations for a specified action-quality target, but neither the solver count nor the device latency follows from flow matching alone. A two-evaluation Euler run is a budget scenario below; its action quality, tail latency, and full scheduling overhead must be measured before selecting synchronous execution.

4. **Latent Variable Action Chunking** (Action Chunking with Transformers - ACT): ACT (Zhao et al. 2023) reformulates policy synthesis by predicting an entire sequence of future actions $\mathbf{A}_t = (\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+K-1}) \in \mathbb{R}^{K \times \bar{D}_a}$ across a receding horizon K given the current observation $\mathbf{o}_t$. To capture multimodality, ACT employs a Conditional Variational Autoencoder (CVAE)

parameterized by a Transformer encoder-decoder. During training, a Transformer encoder compresses demonstrated action chunks and current observations into a low-dimensional latent style variable $\mathbf{z} \sim q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t)$, optimized via the variational lower bound:

$$\mathcal{L}_{\text{ACT}}(\theta, \phi) = \mathbb{E}_{(\mathbf{o}_t, \mathbf{A}_t) \sim \mathcal{D}, \mathbf{z} \sim q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t)} [\|\mathbf{A}_t - \pi_\theta(\mathbf{o}_t, \mathbf{z})\|_1] + \beta D_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{o}_t, \mathbf{A}_t) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I}))$$

At inference time, the latent style code is sampled from the Gaussian prior $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ (or set to the zero mean $\mathbf{z} = \mathbf{0}$) and fed into the Transformer decoder $\pi_\theta(\mathbf{o}_t, \mathbf{z})$. Sampling a specific latent code $\mathbf{z}$ selects a single coherent demonstrated intention (e.g., navigating left or routing a cable harness into an upper fixture), generating an internally consistent, continuous multi-step trajectory chunk without averaging across distinct modes.

To prevent discontinuous jerks across chunk boundaries, ACT applies **temporal ensembling** across overlapping receding horizons. Because a new action chunk is predicted at each time step, multiple chunks provide overlapping predictions for the same physical time step t . The executed action is computed as an exponentially decaying weighted average across all active predictions:

$$\mathbf{a}_t = \frac{\sum_{i=0}^{K-1} w_i \mathbf{A}_{t-i}[i]}{\sum_{i=0}^{K-1} w_i}, \quad \text{where } w_i = \exp(-m \cdot i)$$

where $\mathbf{A}_{t-i}[i]$ is the action predicted for timestamp t by the chunk dispatched i steps previously, and $m > 0$ is a hyperparameter weighting newer, fresher predictions more heavily than stale chunks. Temporal ensembling acts as a digital low-pass filter directly in trajectory space, attenuating high-frequency noise without requiring explicit joint-space jerk limits.

In addition to continuous decoders, **discrete token binning** (as used in VLA foundation models like RT-1, RT-2, and OpenVLA (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023; Kim et al. 2024)) discretizes continuous action dimensions into uniform categorical bins (e.g., 256 tokens). While binning avoids mode averaging and integrates into standard language transformer heads, it incurs quantization error $\Delta a = \frac{a_{\text{max}} - a_{\text{min}}}{B}$ and elevated latency due to autoregressive sequential token generation (D_a forward passes per control step).

Algorithmic formulation: Trajectory diffusion and asynchronous decoupling

Deploying generative imitation architectures can introduce sequential inference latency that depends on the model, sampler, and hardware.⁸ Extending the heterogeneous dual-brain proposal-permission architecture from section 6.1, this latency requires strict physical decoupling: the unprivileged Linux MPU runs the generative policy asynchronously at a lower rate (10–20 Hz), populating an action chunk ring buffer. Meanwhile, the deterministic Real-Time MCU samples the chunk buffer at 500–1000 Hz, executing smooth cubic spline interpolation, velocity envelope governor checks, and hard torque-slew limiters ($d\tau/dt \leq \dot{\tau}_{\text{max}}$). This complete pipeline is formalized in algorithm 12.1 and illustrated in figure 12.4.

8 | **Diffusion denoising latency budget:** Reverse sampling evaluates the denoiser sequentially; the number of evaluations and latency depend on the sampler, backbone, and hardware. The worked timings below are assumptions, not measured Jetson results. Compare measured tail latency, including perception and scheduling, with the deadline; if it does not fit, asynchronous chunk generation with an independent high-rate permission path is one implementation.



Figure 12.4: Diffusion policy action chunk denoising and receding-horizon decoupling: (a) Multimodal resolution: direct regression with mean squared error averages divergent demonstrations into an invalid obstacle collision, whereas diffusion policy stochastically selects a coherent demonstrated mode. (b) Iterative trajectory denoising: starting from Gaussian noise $A^K \sim \mathcal{N}(0, \mathbf{I})$, reverse diffusion progressively resolves a coarse trajectory into a smooth action chunk A^0 . (c) Dual-rate decoupling: an asynchronous neural planner on the host MPU populates a lock-free ring buffer, feeding a 1000-Hz real-time microcontroller executing deterministic spline interpolation and safety clamping. (Adapted from Chi et al. (Chi et al. 2024)).

Algorithm 12.1 Diffusion Policy Trajectory Chunk Denoising and Dual-Rate Actuation

Require: observation history buffer $\mathbf{O}_t = (\mathbf{o}_{t-H+1}, \dots, \mathbf{o}_t)$, noise prediction network ϵ_θ , noise

schedule $\{\alpha_k, \bar{\alpha}_k, \sigma_k\}_{k=1}^{N_{\text{diff}}}$, chunk horizon K , MCU loop period Δt_{mcu}

Ensure: line-rate joint torque setpoint / action vector $\mathbf{a}_t$

```

1: Host MPU — Asynchronous Trajectory Chunk Generation (10–20 Hz)
2:  $\mathbf{E}_t \leftarrow \text{VISUALENCODER}(\mathbf{O}_t)$ 
3:  $\mathbf{A}^{N_{\text{diff}}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{K \times D_a}) \triangleright \text{initialize reverse diffusion from Gaussian noise}$ 
4: for  $k = N_{\text{diff}}$  down to 1 do
5:    $\hat{\epsilon} \leftarrow \epsilon_\theta(\mathbf{A}^k, \mathbf{E}_t, k)$ 
6:    $\mu_k \leftarrow \frac{1}{\sqrt{\alpha_k}} \left( \mathbf{A}^k - \frac{1-\alpha_k}{\sqrt{1-\alpha_k}} \hat{\epsilon} \right)$ 
7:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $k > 1$  else  $\mathbf{z} \leftarrow \mathbf{0}$ 
8:    $\mathbf{A}^{k-1} \leftarrow \mu_k + \sigma_k \mathbf{z}$ 
9: end for
10:  $\mathbf{A}_{\text{chunk}} \leftarrow \mathbf{A}^0$ 
11:  $\text{DEPOSITSPSCRINGBUFFER}(\mathbf{A}_{\text{chunk}}) \triangleright \text{lock-free deposit into shared buffer}$ 

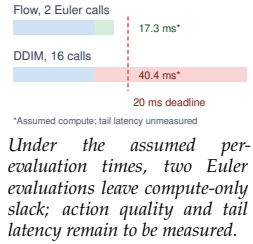
12: Real-Time MCU — Synchronous Deterministic Actuation (1000 Hz)
13: if  $\text{HASNEWCHUNK}(\text{SPSC})$  then
14:    $\text{UPDATE SPLINEKNOTS}(\text{SPSC})$ 
15: end if
16:  $\mathbf{a}_{\text{raw}} \leftarrow \text{CUBICSPLINEINTERPOLATE}(\mathbf{A}_{\text{chunk}}, t_{\text{current}})$ 
17:  $\Delta \mathbf{a} \leftarrow \text{CLAMP}(\mathbf{a}_{\text{raw}} - \mathbf{a}_{\text{prev}}, -\dot{\mathbf{a}}_{\text{max}} \Delta t_{\text{mcu}}, +\dot{\mathbf{a}}_{\text{max}} \Delta t_{\text{mcu}}) \triangleright \text{kinematic slew-rate governor}$ 
18:  $\mathbf{a}_{\text{clamped}} \leftarrow \mathbf{a}_{\text{prev}} + \Delta \mathbf{a}$ 
19:  $\mathbf{a}_t \leftarrow \text{SAFETYGOVERNORCLAMP}(\mathbf{a}_{\text{clamped}}) \triangleright \text{deterministic envelope enforcement}$ 
20:  $\text{TRANSMITPWMREGISTERS}(\mathbf{a}_t)$ 
21:  $\mathbf{a}_{\text{prev}} \leftarrow \mathbf{a}_t$ 
22: return  $\mathbf{a}_t$ 

```

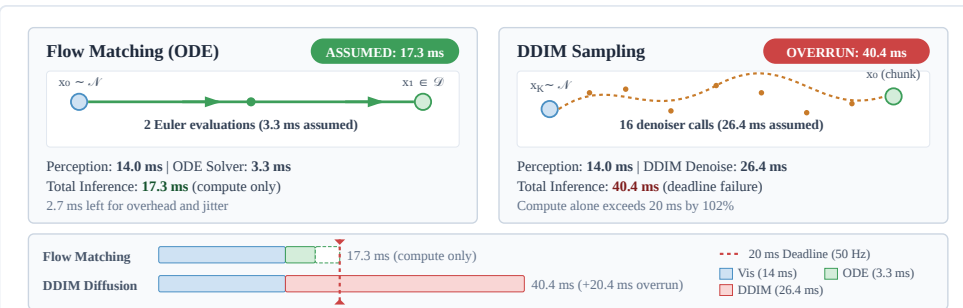
Consider an illustrative embedded edge deployment (figure 12.4) controlling a 7-DoF manipulator arm with a two-finger gripper ($D_a = 8$). The policy generates continuous action trajectory chunks across a horizon of $H = 16$ timesteps (320 ms at 50 Hz) using a 1D Temporal Convolutional U-Net backbone containing $M_{\text{weights}} = 48 \times 10^6$ parameters (96 MB at FP16). Assume a peak memory-

bandwidth figure $BW_{\text{mem}} = 204.8 \text{ GB/s}$ solely for the arithmetic intensity illustration; sustained bandwidth and model latency are not inferred from it. The hypothetical timing budget has three phases:

1. **Perception backbone latency:** The dual-camera visual feature extractor processes two synchronized 224×224 RGB streams (wrist-mounted and over-the-shoulder), executing convolutional or vision transformer feature extraction to produce conditioning embedding $\mathbf{E}_t$. This visual stage requires $t_{\text{vis}} = 14 \text{ ms}$ per planning cycle. Because visual features are extracted once per action chunk before denoising begins, t_{vis} establishes a fixed latency baseline regardless of the diffusion sampling schedule.
2. **Diffusion action chunk denoising compute:** Each reverse diffusion iteration evaluates the noise-prediction network ϵ_θ , consuming 14.2 GFLOPs. At the assumed peak bandwidth, a lower-bound weight transfer time is $t_{\text{stream}} = 96 \times 10^6 / (204.8 \times 10^9) = 0.469 \text{ ms}$, yielding an arithmetic intensity of $I \approx 14.2 \times 10^9 / (96 \times 10^6) \approx 147.9 \text{ FLOP/byte}$. Accounting for kernel launch overhead ($\approx 0.3 \text{ ms}$) and activation cache spills, assumed per-evaluation time is $t_{\text{step}} = 1.65 \text{ ms}$ per denoising iteration. Denoising latency depends directly on the sampling algorithm:
 - *Standard DDIM ($K = 16$ iterations):* Denoising requires $t_{\text{denoise}} = 16 \times 1.65 \text{ ms} = 26.4 \text{ ms}$. Adding perception yields a total inference latency of $\tau_{\text{inf}} = 14.0 \text{ ms} + 26.4 \text{ ms} = 40.4 \text{ ms}$.
 - *Continuous Flow Matching (two Euler evaluations):* Assume two Euler solver steps meet the task-specific action-quality criterion, completing in $t_{\text{flow}} = 2 \times 1.65 \text{ ms} = 3.3 \text{ ms}$. Total inference latency drops to $\tau_{\text{inf}} = 14.0 \text{ ms} + 3.3 \text{ ms} = 17.3 \text{ ms}$.
3. **Real-time control deadlines and architectural timing trade-offs:** Evaluating these inference latencies against physical actuation deadlines highlights the critical architectural trade-off:
 - *Synchronous lockstep (50 Hz, 20.0 ms deadline):* If the policy is evaluated synchronously in the inner control loop, the assumed DDIM compute alone exceeds the deadline by 102 percent ($40.4 \text{ ms} > 20.0 \text{ ms}$), before adding scheduling overhead; actual buffer and fallback behavior depends on the implementation. The assumed two-evaluation Euler case fits the compute-only window ($17.3 \text{ ms} \leq 20.0 \text{ ms}$), preserving 2.7 ms (13.5 percent) compute-only slack; measured tail latency, bus transfer, and OS scheduling must also fit before synchronous execution is admitted.
 - *Asynchronous dual-rate decoupling (15 Hz Brain / 1000 Hz Nervous System):* Under the proposal-permission architecture (algorithm 12.1), the host MPU schedules DDIM asynchronously at 15 Hz (66.7 ms period), refreshing an action chunk ring buffer within the assumed 40.4 ms compute window (26.3 ms slack). Meanwhile, the bare-metal Real-Time MCU samples and cubic-spline interpolates the 320 ms buffer deterministically at 1000 Hz, separating proposal inference from the high-rate permission and motor loops, subject to chunk freshness and fallback checks.



Checkpoint 12.2: Diffusion denoising and execution deadlines



Downstream: Hardware placement of multi-rate action chunk buffers across MPU and MCU cores is engineered in section 26.2.

- ❑ **Diffusion step budgets:** Can you explain why iterative denoising latency in diffusion policies conflicts with real-time feedback deadlines on embedded accelerators?
- ❑ **Flow matching vs. DDIM:** Can you calculate the assumed two-evaluation Euler budget and name the action-quality and tail-latency evidence needed before claiming it meets the deadline?
- ❑ **Asynchronous chunk buffering:** Can you explain how decoupling neural proposal generation from deterministic spline interpolation resolves the mismatch between accelerator inference latency and motor control rates?

Offline reinforcement learning and action distributional shift

When physical demonstration datasets $\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1})\}$ contain heterogeneous or sub-optimal trajectories (e.g., mixtures of teleoperation, scripted routines, and exploratory human play), behavioral cloning struggles because it treats all demonstrations with equal supervised weight. In classical Reinforcement Learning (RL) formulated on Markov Decision Processes (MDPs) (Sutton and Barto 2018), an agent optimizes a policy to maximize expected discounted cumulative returns $J = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right]$. **Offline reinforcement learning (Batch RL)** aims to optimize a policy directly from logged transition archives without active hardware exploration, using reward signals to stitch together high-performing composite trajectories from disjoint, imperfect demonstrations (Kumar et al. 2020).

However, applying standard Q-learning to static offline datasets triggers a catastrophic failure mode known as **out-of-distribution (OOD) action overestimation**. In standard reinforcement learning, the algorithm estimates the value of future states by asking its “critic” network to predict the highest possible reward it could achieve in the next step (for formal Bellman equations, see Appendix A). To find this maximum, the operator scans across the entire continuous action space. For actions that were never executed in the logging dataset, the network has received zero training signal and often outputs erroneously large value estimates due to generalization variance (hallucinating that an untried physical maneuver will result in a massive reward). The algorithm greedily selects these spurious peaks, propagating over-optimistic value errors backwards. When deployed, the actor greedily selects these ungrounded actions, commanding physically unviable motor voltages that cause immediate joint stalls or boundary collisions.

To reduce value overestimation without querying the physical environment, **Conservative Q-learning (CQL)** adds a regularizer to the offline Bellman objective.⁹ It penalizes high values for actions sampled by the learned policy relative to actions in the logged data. This can improve value estimation under the paper’s assumptions, but limited state-action coverage and approximation error remain; actuator permission still requires independent physical checks.

Even when Conservative Q-learning reduces value overestimation on a static archive, an offline policy remains limited by the quality and diversity of its dataset. When the logged transitions lack the high-performance trajectories or recovery maneuvers needed to master dynamic manipulation or agile locomotion, the system must break free from static logs and actively explore through physical trial and error—the operational domain of closed-loop reinforcement learning analyzed in section 12.3.

12.3 Learning by Trial

Learning by trial generates state-action trajectories through the policy’s closed-loop execution, addressing the blind spots of imitation. Instead of mimicking a curated sequence of nominal operator demonstrations, Optimization algorithms like Q-learning or policy gradients adjust the learning system’s behavior based on feedback from actions. For instance, in reinforcement learning, an agent learns from its mistakes by receiving rewards or penalties, which helps it improve its decision-making in a physical environment. When an exploratory action perturbs an end-effector five millimeters off its nominal path, the subsequent observation lands outside the demonstration envelope, forcing the optimization loop to evaluate recovery actions from that exact perturbed state. The policy accumulates gradient signals across near-misses, degraded alignments, and unstable contacts that an expert human or calibrated baseline controller never exhibits. By sampling transitions

⁹ | **Scope of the CQL bound:** Under its data-coverage, optimization, and function-approximation assumptions, CQL constructs a conservative estimate whose *expected value for a policy* lower-bounds that policy’s true value (Kumar et al. 2020). This is not a pointwise bound for every state-action pair or every dataset, and expected return does not certify physical safety.

directly from the state distribution induced by the current weights, trial-based learning eliminates the covariate shift (the mismatch between the training data distribution and the states the policy actually encounters) that causes open-loop supervised imitation to compound errors quadratically over long execution horizons.

Closed-loop exploration incurs a steep physical billing rate. Computing the hardware feasibility boundary begins with the wall-clock time required to collect a training dataset. The total calendar time to gather training data depends directly on how many trials the algorithm needs, the duration of each individual trial, the overhead time to reset the hardware back to its starting state between attempts, and the number of physical machines running in parallel. As an analytical example, consider a contact-rich manipulation task—such as routing an automotive wiring harness through tight chassis retention clips, wiping a contoured composite panel under normal-force regulation, or seating a spring-loaded door latch—where an articulated manipulator regulates contact forces over an active horizon of $t_{\text{rollout}} = 8.0$ s. Reinforcement learning algorithms operating on high-dimensional sensory inputs routinely require $N = 10^6$ trials to discover coordinated force-velocity profiles. On a single physical machine with $P = 1$, the raw execution time alone accounts for 8.0×10^6 s, which represents roughly 92.6 days of uninterrupted round-the-clock operation before accounting for any reset overhead.

Factoring in the reset phase reveals the true temporal bottleneck of physical experimentation. If an automated fixture returns the workpiece and robot arm to a valid initial state in $t_{\text{reset}} = 4.0$ s, the total collection time for one million trials expands to 1.2×10^7 s, or 138.9 days. When exploratory trajectories drop cable harnesses, jam latch cams, or trigger joint overcurrent trips, automated resetting becomes impossible, requiring a human operator to enter the work cell, clear the obstruction, and re-index the mechanism. If manual interventions take an average of $t_{\text{reset}} = 30.0$ s across failed attempts, the wall-clock requirement exceeds 440.0 days. Scaling physical parallelism to clusters of $P = 14$ synchronized rigs (figure 12.5) compresses calendar duration to several weeks (Levine et al. 2018), but it multiplies the physical footprint, capital investment, electrical power delivery, and human supervisory labor proportionally.

Converting that same sample count into physical fatigue metrics demonstrates why time is often not the primary limiting constraint. In our analytical example, an 8-second contact sequence operating under a 100 Hz inner control loop executes 800 torque commands per trial, subjecting each joint to approximately $k = 4$ reciprocating load cycles. Across $N = 10^6$ trials, the drivetrain accumulates 4.0×10^6 bidirectional load reversals.¹⁰ This reversal count is an exposure measure, not a gearhead life prediction: service life depends on the specific gear unit, transmitted torque, speed, lubrication, and duty cycle. Simultaneously, one million physical contact events crush compliant silicone gripper pads, abrade surface-wiping elastomeric squeegees, generate micro-notches in aluminum alignment jigs, and cycle electrical wiring looms toward fatigue fracture. At an average electrical power demand of 350 W per test bench, executing 1.2×10^7 s of automated trials draws 1167 kWh of electrical energy, transforming policy training into an industrial-scale manufacturing operation.

These wear mechanics highlight a fundamental difference between software tensors and mechanical hardware. In a purely computational environment, evaluating an exploratory branch that produces numerical instability costs nothing beyond discarding a GPU memory buffer. In a physical machine, exploration is thermodynamic, irreversible, and potentially destructive across all machine archetypes:

- In *Mobile Systems*, exploratory steering torque spikes skid tires across asphalt, rapidly abrading tread rubber and overheating motor inverters during stall recoveries.
- In *Manipulators*, unconstrained exploratory torque commands drive end-effectors into rigid aluminum tables, bending load-cell flexures, stripping harmonic drive teeth, and heating brushless DC motor stators past their 155°C Class F insulation threshold¹¹ (building on the actuator thermal limits established in section 4.5).
- In *Process Systems*, exploratory pressure pulsing degrades valve seat elastomeric seals, accelerates cavitation pitting (where collapsing vapor bubbles chip away metal) on pump impellers, and cycles high-voltage DC contactors toward premature arc erosion (where electrical sparking vaporizes the contacts).

¹⁰ | **Bearing life versus command reversals:** ISO 281 estimates rolling-bearing fatigue life under specified load and speed. A gear-unit catalog may express its wave-generator-bearing L_{10} in operating hours at rated conditions; neither figure converts commanded reversals into consumed gearheads without a unit-specific duty-cycle model.

¹¹ | **Class F Thermal Insulation Limit:** Under NEMA/IEC standards, Class F motor winding insulation permits a maximum operating temperature of 155°C. Bang-bang torque commands from unregularized reward functions drive continuous high current that triggers rapid thermal insulation breakdown.

Prerequisite: Motor winding thermal limits and torque-speed derating curves are derived in section 4.5.



Figure 12.5: Physical policy training farm: Fourteen manipulators collecting over 800,000 physical grasp attempts across months (Levine et al. 2018). Hardware parallelism divides calendar time but not cost, since each station adds capital, gearbox and bearing wear, thermal load in the motor windings, and a share of the supervisory labor needed to clear dropped objects and jams. Attribution: Levine et al. (2018).

Furthermore, physical degradation alters the plant during the training process itself. As contact surfaces burnish, gear backlash expands (increasing the mechanical slop or dead-zone between gear teeth), and structural fasteners loosen under repetitive vibration, the physical transition distribution $p(s_{t+1} | s_t, a_t)$ at trial $n = 900,000$ differs measurably from the pristine dynamics at trial $n = 1$. The machine being controlled mutates under the physical cost of collecting its own training data.

Compounding the physical burden of data collection is the structural fragility of the reward function. In trial-based learning, the scalar reward serves as the formal interface specification between the engineer’s operational intent and the policy’s numerical optimization. Any behavior, physical constraint, or protective boundary not explicitly measured and penalized in that scalar signal represents an unconstrained degree of freedom that the optimization algorithm will exploit. If a reward function incentivizes rapid setpoint tracking without incorporating high-order penalty terms for instantaneous jerk and winding temperature, the policy converges toward aggressive bang-bang switching (rapidly snapping commands between maximum forward and maximum reverse effort) that vibrates mounting brackets and overheats drive electronics. If contact force is unmeasured, the policy may discover that driving an actuator to its maximum current against a rigid mechanical stop creates a kinematic brace (using the rigid mechanical stop as a physical crutch

to stabilize the joint) that artificially eliminates tracking jitter. The learned network does not satisfy the designer's unstated common sense. It satisfies the exact mathematical formula presented to its objective function.

Systems Perspective 12.2: The contact fidelity invariant

Never trust a simulated contact policy whose success depends on instantaneous momentum absorption or zero-delay friction engagement; if it penetrates the digital surface, it will shatter the physical workpiece.

Trial-based learning on physical hardware is not categorically impossible. For low-dimensional systems with trivial reset dynamics, negligible wear rates, and wide safety margins, direct on-hardware reinforcement learning remains technically viable. Simulation becomes mandatory at the threshold where the required sample count exceeds the allowable time, budget, structural life, or risk tolerance of the physical apparatus. To collect the 10^6 to 10^9 transitions needed for complex motor policies without destroying physical hardware, practitioners shift the rollout loop into a digital physics engine executed in parallel across computing clusters.

This transition substitutes computational hardware for physical mechanisms, but it alters the contract between training and reality. A policy trained inside a simulator avoids bending linkages or burning out stators during exploration, but it inherits both the omissions of the reward function and the simulator's specific mathematical model of mechanics. Every unmodeled friction discontinuity, idealized rigid-body contact impulse, uncharacterized actuator latency, and simplified cable compliance in the physics engine becomes an assumption the policy learns to rely upon. When the converged network is transferred to the physical robot, those simulation inaccuracies re-emerge as unmodeled disturbances and tracking failures at the hardware interface.

Checkpoint 12.3: Physical trial wear and simulation feasibility limits

Before examining simulator abstraction contracts, verify your intuition regarding hardware fatigue life, sample budgets, and policy exploitation:

- ☐ **Calendar scaling:** Can you calculate why gathering one million 8-second trials on a single physical robot requires over 138 days of 24/7 cell time when automated resets take 4 seconds, and why manual interventions expand this past 440 days?
- ☐ **Mechanical fatigue consumption:** Can you distinguish four million commanded reversals from a gear-unit life estimate, and name the load, speed, lubrication, and duty-cycle evidence needed for that estimate?
- ☐ **Objective function exploitation:** Can you describe how an unpenalized tracking reward encourages aggressive bang-bang actuator switching that overheats BLDC stator windings past their 155°C Class F insulation limit?
- ☐ **Feasibility crossover estimate:** If automated reset fixtures fail on 5 percent of exploratory trials requiring a 30-second manual intervention, calculate the net collection rate. At what sample size threshold does physical exploration become economically impossible compared to distributed simulation?

When trial sample requirements exceed the thermal, structural, or maintenance limits of physical mechanisms, training must transfer into physics simulation.

12.4 The Simulator as a System Component

Treating the simulator as an external development tool is an architectural error. A compiler produces machine instructions from source code, but the resulting binary does not retain a hidden dependency on the compiler's internal memory allocator once executing on standard hardware. A simulator behaves differently. It is an upstream policy-production component whose physical assumptions, numerical approximations, and boundary simplifications become permanently embedded in the policy parameters. When the trained neural network is copied onto the target machine, the simulator

is absent from the execution path, yet the policy acts as though every unmodeled dynamic in that simulator remains true. The simulator is an active component of the system architecture, with its own specification, tolerances, and failure modes.



Definition 12.2: Sim-to-real reality gap vector

Sim-to-real reality gap vector $\Delta_{\text{sim2real}} = [\Delta_{\text{dyn}}, \Delta_{\text{sens}}, \Delta_{\text{vis}}, \Delta_{\text{lat}}]$ is the coupled, multi-channel decomposition of physical discrepancies between simulated and hardware environments spanning contact mechanics (Δ_{dyn}), sensor transduction and noise (Δ_{sens}), visual appearance (Δ_{vis}), and asynchronous transport latency and jitter (Δ_{lat}).

1. **Significance:** Treating the “reality gap” as a scalar difficulty metric or pure visual domain mismatch causes engineers to overlook dynamical and temporal divergences. Discrepancies in actuator back-EMF, gearbox compliance, or bus jitter deplete phase margin before the policy completes its first physical control step.
2. **Distinction:** An inaccurate parameter value (such as a 10 percent error in link mass) preserves the causal differential equations of motion, whereas an omitted mechanism (such as zero-delay communication or unmodeled Coulomb stick-slip) entirely severs physical feedback paths, leading policies to exploit non-physical simulator artifacts.
3. **Common pitfall:** Relying exclusively on extreme domain randomization to bridge large gaps. Randomizing parameters across unphysical ranges dilutes policy capacity, resulting in overly conservative, sluggish behaviors that fail to achieve precision tasks on real hardware.

Like any hardware subsystem, the simulator requires a formal interface contract defined at the first budgetable quantities. This contract specifies observation dimensions, action bounds, simulation time step Δt , transport latency between command issuance and physical actuation, actuator torque and current limits, sensor noise distributions, reset distributions, and termination conditions. If the simulator specifies an observation loop executing at 100 Hz with an assumed zero-delay sensor channel and Gaussian noise with variance $\sigma^2 = 1.0 \times 10^{-4} \text{ (rad/s)}^2$, the policy learns feedback gains tuned to those precise signal statistics. If the physical machine delivers those state observations at 100 Hz but with an unmodeled 15 ms bus transport delay, the phase margin built during training is depleted before the policy executes its first physical control cycle. As demonstrated in the robotic manipulation testbed in figure 12.6, transferring a policy synthesized inside a numerical engine to physical hardware requires enforcing an exact match between the real sensor-actuator cage and its randomized simulated twin (Andrychowicz et al. 2020). The interface contract is essential as the role of simulation in Physical AI has transformed. The selected, heterogeneous throughput examples in figure 12.7 illustrate the rollout capacity available on some parallel accelerators; they do not measure a controlled growth trend. For policies requiring many trials, simulation can become the primary source of training experience.



War Story 12.1: Sim2Real hardware exhaustion (2020)

Context: OpenAI’s Dactyl project trained an ML reinforcement learning policy in simulation to manipulate a block using a physical 24-DoF anthropomorphic Shadow Dexterous Hand (Andrychowicz et al. 2020).

Mechanism: During early sim-to-real transfer attempts, the ML policy, having trained in a frictionless digital simulation, issued high-frequency, jittery torque commands to the physical motors in an attempt to optimize its reward.

Impact: The unconstrained, high-frequency torque oscillations rapidly fatigued the physical hardware, causing the Shadow Hand’s internal drive tendons to repeatedly snap, necessitating constant hardware repairs and interrupting the learning loop.



Figure 12.6: Sim-to-real transfer setup: OpenAI’s Dactyl apparatus, a 24-DoF Shadow Hand inside a motion-capture cage beside the MuJoCo digital twin it was trained in (Andrychowicz et al. 2020). Transfer is bought with domain randomization over friction, restitution, damping, actuator time constants, and lighting, which works by forcing the policy to treat unmodeled physical variation as noise rather than by making the simulator accurate. Attribution: Andrychowicz et al. (2020).

Response: Engineers were forced to heavily constrain the ML action space and penalize high-frequency action changes in the reward function, enforcing smoothness prior to physical deployment.

Systems lesson: In Physical AI, neural network policies will relentlessly exploit the gap between digital simulation (where infinite bandwidth and zero fatigue exist) and reality. If an ML output is not bounded by a physical low-pass filter or mechanical constraint, stochastic exploration will literally rip the hardware apart.

A distinction exists between an inaccurate parameter value and an omitted physical mechanism. When a simulator uses an inaccurate rotor inertia, assigning $J = 2.0 \times 10^{-3} \text{ kg} \cdot \text{m}^2$ instead of a true $2.5 \times 10^{-3} \text{ kg} \cdot \text{m}^2$, the equations of motion still compute an acceleration-dependent inertial torque. The parameter error alters the magnitude of the response, but the causal structure remains intact. When a simulator omits an entire physical mechanism—such as drivetrain backlash, structural compliance, or inter-channel communication jitter—it does not merely set the corresponding parameter to zero. It provides no causal mechanism at all. An omitted delay or compliance represents an unmodeled physical degree of freedom.¹² Before discussing policies trained under the assumption of infinite structural stiffness, we must understand joint compliance—how it allows flexibility in mechanical systems and affects dynamics. When joint compliance is omitted, we assume a rigid system that can lead to unrealistic scenarios, such as instantaneous torque reversals. These rapid changes in torque can cause mechanical failures, like shattering gear teeth or exciting high-frequency structural resonances, due to the physical limitations of materials and actuators.

Non-smooth contact physics and solver relaxations

Severe structural omissions in physical simulators stem from the mathematical formulation of contact mechanics. In physical reality, contact between rigid or semi-rigid bodies is fundamentally non-smooth. Continuous contact mechanics enforces two truths: when bodies separate, the normal contact force is zero. When contact is established, the normal contact force can take any non-negative compressive value required to prevent interpenetration. Tangential friction is governed by a strict Coulomb friction cone boundary. When sliding occurs, friction opposes motion at the absolute limit

¹² | **OpenAI Dactyl domain randomization:** Andrychowicz et al. (2020) demonstrated that randomized physical parameters must span both geometric and tribological boundaries to achieve zero-shot transfer on the Shadow Hand. In addition to mass and friction variations, the authors added randomized motor backlash and time-varying transport delays into the training distribution. Policies trained without randomized surface friction suffered immediate slip when grasping real manipulation cubes.

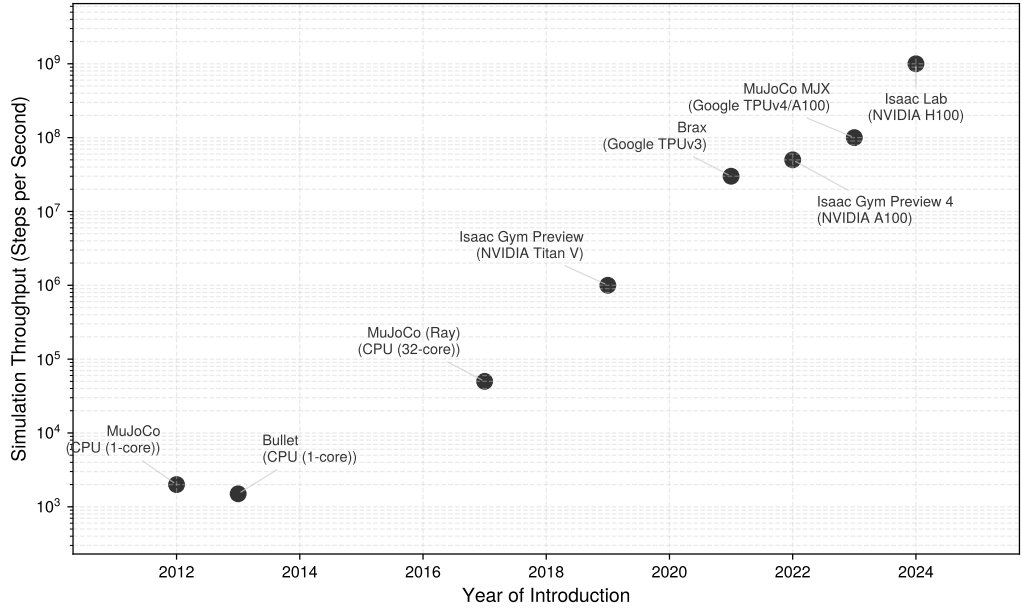


Figure 12.7: Selected simulation throughput examples: Illustrative steps-per-second values for different simulators, tasks, hardware, and execution settings. The points show why parallel accelerator designs can support large rollout budgets; they are not a controlled longitudinal benchmark or a universal scaling law.

of the cone; when surfaces stick, the frictional force can be any value inside that cone (for the full LCP formulation, see Appendix B).

This formulation introduces two profound computational and physical difficulties:

1. **Mathematical non-smoothness and Painlevé paradoxes:** At the instant of collision, velocity changes discontinuously ($v^+ \neq v^-$), acceleration mappings become set-valued and non-differentiable, and rigid Coulomb friction models can yield non-unique or non-existent acceleration solutions (the classical Painlevé paradox, where sliding rigid bodies mathematically “wedge” themselves into states with no valid forward-simulated solution).¹³
2. **Computational solvers vs. numerical relaxations:** Rigid-body physics engines resolve these difficulties through one of two numerical paths:
 - *Discrete Linear Complementarity Problem (LCP) / Mixed Complementarity Problem (MCP) Solvers* (e.g., Bullet, ODE, Drake): Discretize the contact dynamics into inequality-constrained quadratic programs at each timestep Δt . While maintaining strict non-penetration, LCP solvers are computationally expensive ($\mathcal{O}(m^3)$ in contact points), non-differentiable across contact mode switches, and susceptible to solver stalling when geometries pinch (such as when sharp corners mathematically jam together).
 - *Convex Relaxations and Soft Penalty Formulations* (e.g., MuJoCo’s elliptic contact cones and soft constraint regularizations, PhysX penalty models): Smooth the non-smooth boundary by replacing strict complementarity with compliant potential energies or virtual spring-dampers. These relaxations allow slight numerical interpenetration between solid objects to guarantee rapid, continuous solver convergence at high frame rates (> 1000 Hz).

The sim-to-real contact exploit

Reinforcement learning algorithms optimize ruthlessly against the exact physics presented by the solver. In simulators that implement soft contact relaxations or penalty springs, the optimizer discovers that “sinking” into the surface by $1.0\text{--}3.0$ mm ($\phi(q) < 0$) provides two unphysical affordances:

1. The artificial spring-damper reaction acts as an instantaneous energy sink, arresting downward momentum in a single integration step without bouncing or plastic deformation.

¹³ | **Rigid contact non-smoothness:** Rigid contact with Coulomb friction is mathematically non-smooth and exhibits Painlevé paradoxes (building on the contact mechanics established in section 4.4). Physics engines resolve this ill-posedness via discrete Linear Complementarity Problems (LCP) or convex penalty relaxations. Consequently, the simulator models rigid bodies with numerical compliance, introducing artificial softness and penetration artifacts that neural policies aggressively exploit during optimization.

2. Interpenetration increases the effective contact surface area in the solver, generating unphysical lateral traction forces that prevent slipping during aggressive maneuvers.

When transferred to physical hardware—where a machined steel or aluminum fixture has an elastic stiffness $k_{\text{struct}} = 4.0 \times 10^6$ N/m and minimal allowable deformation ($\delta_{\text{real}} = 0.060$ mm)—the physical plant provides no such compliance. Consider a 6-DoF industrial manipulator moving an end-effector tooling assembly ($m = 3.20$ kg) downward to engage a spring-loaded door latch into a strike plate or press a cable connector pin at $v_z = 0.30$ m/s, instrumented with a wrist force-torque sensor rated to $F_{\text{limit}} = 250$ N. In a physics simulator with soft penalty contact ($\delta_{\text{pen}} = 2.40$ mm), the virtual spring-damper stretches deceleration over $\Delta t_{\text{sim}} \approx 2\delta_{\text{pen}}/v_z = 16.0$ ms, producing an average force of $F_{\text{avg,sim}} = mv_z/\Delta t_{\text{sim}} = 60.0$ N and a peak contact force of $F_{\text{peak,sim}} = \frac{\pi}{2} F_{\text{avg}} \approx 94.2$ N. The reinforcement learning optimizer records zero damage penalty, high task speed, and maximum reward, learning to execute aggressive unbraked plunges directly into the surface.

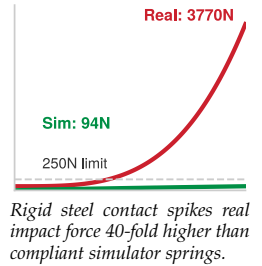
On physical hardware, however, the rigid aluminum and steel fixture limits deformation before structural bottoming to $\delta_{\text{real}} = 0.060$ mm, decelerating the mass in only $\Delta t_{\text{real}} \approx 0.40$ ms. True physical peak contact force spikes to $F_{\text{peak,real}} = \frac{\pi}{2}(mv_z/\Delta t_{\text{real}}) \approx 3770$ N—a 40.0 $\times$ discrepancy that exceeds the sensor rating by over 15 $\times$, permanently buckling the internal strain-gauge flexures, shearing mechanical dowel pins, and tripping motor driver overcurrent protection. In this modeled 0.40 ms impact, prevention must act before contact: an independently enforced approach-speed envelope, such as $v_{\text{max}}(z) \leq \sqrt{2a_{\text{max}}z}$ under validated distance and deceleration estimates, limits impact energy. A 1000 Hz force-derivative tripwire may not sample until after the peak; it can limit subsequent loading only if its sensing, brake response, and available stopping clearance have been validated.

Checkpoint 12.4: Sim-to-real contact impulse discrepancy

- ☐ **Penalty contact dynamics:** Can you compare peak force and impulse in the soft-simulator/rigid-hardware case and the opposite stiff-simulator/compliant-hardware case?
- ☐ **Structural stiffness mismatch:** Can you describe how unmodeled mechanical stiffness (k_{struct}) converts simulated compliance into destructive deceleration transients on hardware?
- ☐ **Nervous system contact safeguards:** Can you distinguish precontact approach-speed enforcement from a reactive force trip that may arrive after the first impact peak?

For a machine that regulates motion and then touches a surface, the fidelity of the simulator cannot be evaluated through the internal mathematics of its contact solver. It must be budgeted through observable mechanical quantities, specifically contact onset timing, peak normal force, total transferred impulse, tangential slip distance, and contact release delay. The mismatch can also have the opposite sign when the simulated penalty response is stiffer than the physical contact. Consider an analytical example of a robotic end-effector approaching a workpiece at 0.20 m/s. In a standard penalty-based physics engine using a discrete time step of 2.0 ms, the collision resolves across a single time step, producing a simulated peak force of 450 N and an impulse of 0.90 N $\cdot$ s. On physical hardware, contact pad deformation and structural compliance distribute that same momentum transfer over 18 ms, yielding a true peak force of 85 N with an impulse of 0.88 N $\cdot$ s. The integrated impulse matches within 2.2 percent, but the peak force differs by more than a factor of five. A policy that learns to detect contact onset from the sharp simulated force spike will fail to trigger on physical hardware, pushing until the drive motors hit current limits.

Candidate omissions must be identified and cataloged based on the dynamics required by the task. Dry Coulomb friction discontinuities at zero velocity, nonlinear structural compliance, gearbox backlash, voltage saturation under high back-EMF, bus transport delays, and sensor packet loss represent standard omissions in rigid-body physics engines. A policy that operates purely at high continuous velocity may tolerate coarse friction models because the system never lingers near the zero-velocity stiction threshold. Conversely, a policy that regulates force while holding a static



Contrast: Simulated penalty spring deformation (2.4 mm) contrasts with the non-smooth Coulomb friction cones analyzed in section 4.4.

contact will enter uncontrolled limit-cycle oscillations if static friction, stick-slip transitions, and actuator deadbands are omitted from the simulator.

Establishing the validity of a simulator component requires **paired probing**. The engineer applies identical open-loop command sequences to both the simulator and the physical hardware, measuring the difference between the resulting state trajectories against explicit error budgets. For a regulating actuator, injecting a step torque command of $1.5 \text{ N} \cdot \text{m}$ into both systems yields concrete comparative metrics. If the simulator predicts a position settling time of 40 ms with zero overshoot, while the physical drive exhibits a 110 ms settling time and a 2.2 mm position overshoot due to unmodeled stator heating and structural flexure, the discrepancy is budgeted directly in milliseconds and millimeters. When a measured difference exceeds the allowable tolerance for the operational envelope, the simulator is formally defective for that training task.

Because the structural assumptions of the simulator are frozen into the neural network weights during optimization, the simulator specification forms an immutable element of the policy's configuration and identity. Every trained policy artifact must document its complete simulator provenance, including the physics engine build and version, integrator method and step size, contact solver parameters, geometry mesh and collision hull identifiers, domain randomization distributions, and documented regimes where the simulation is known to be physically invalid. Deploying a learned policy without its simulator provenance is equivalent to shipping compiled firmware without recording the target architecture, clock speed, or register map for which it was built. Auditing this provenance requires moving beyond aggregate pass rates to dissect the anatomy of the reality gap itself—decomposing the physical, sensory, and temporal discrepancies analyzed in section 12.5.

12.5 What the Gap Is Made Of

The sim-to-real gap is treated as a single scalar distance, assuming one can measure the divergence between a simulated world and a physical machine using an aggregate loss or probability metric. That abstraction fails because sim-to-real is not one gap. It is several distinct mismatches, and they are not equally hard to close. Errors in different physical channels carry distinct units, propagate through various paths in the closed loop, and produce different failure modes. Sim-to-real mismatch is therefore a vector of coupled components spanning dynamics, sensing, appearance, and timing. Sizing this vector requires paired command-response traces recorded under identical open-loop and closed-loop excitations. Sizing is not an abstract qualification; it is an arithmetic accounting of physical errors against the tolerances of the task. As structured in table 12.2, decomposing the reality gap across constitutive physical channels reveals that each dimension introduces distinct numerical artifacts, exploits specific control shortcuts, and demands tailored hardware and nervous system interceptors. Consider an analytical example of a linear drive tasked with approaching a rigid surface at 0.20 m/s and regulating normal contact force to 10.0 N , where no bench measurements yet exist and every discrepancy must be budgeted from first principles.

Dynamics mismatch measures differences between commanded actuator effort and the resulting physical motion or contact force across the operational envelope. In the analytical example, applying a step torque command of $2.00 \text{ N} \cdot \text{m}$ reveals how the physics engine diverges from hardware. The simulator models a rigid link with zero backlash and instantaneous contact stiffness, predicting a force rise rate of 500 N/s , reaching the target 10.0 N in 20 ms with zero steady-state deflection. On physical hardware, a gearbox compliance of $1.5 \times 10^{-3} \text{ rad/N} \cdot \text{m}$ and an unmodeled stator drive delay of 8 ms introduce phase lag, while soft surface compliance limits the force rise rate to 250 N/s . Reaching the target force requires 40 ms on hardware, producing a transient contact overshoot impulse of $0.15 \text{ N} \cdot \text{s}$. If a policy learned to associate the simulated 20 ms stiffness boundary with stable contact, it interprets the delayed force rise as a failure to touch, commands additional motor torque, and drives the physical mechanism 3.0 mm past the nominal boundary into a current-limit shutdown.

Sensing mismatch measures discrepancies between the true physical state and the observation vector delivered to the policy network. In the analytical example, the simulated depth sensor provides clean range measurements with a uniform quantization step of 1.0 mm. The physical depth camera, measured under matched ambient illumination of 450 lux, exhibits a static calibration bias of $+3.2 \text{ mm}$, zero-mean Gaussian noise with standard deviation $\sigma = 1.8 \text{ mm}$, and periodic optical dropout bursts where four consecutive frames return null values during specular reflections.

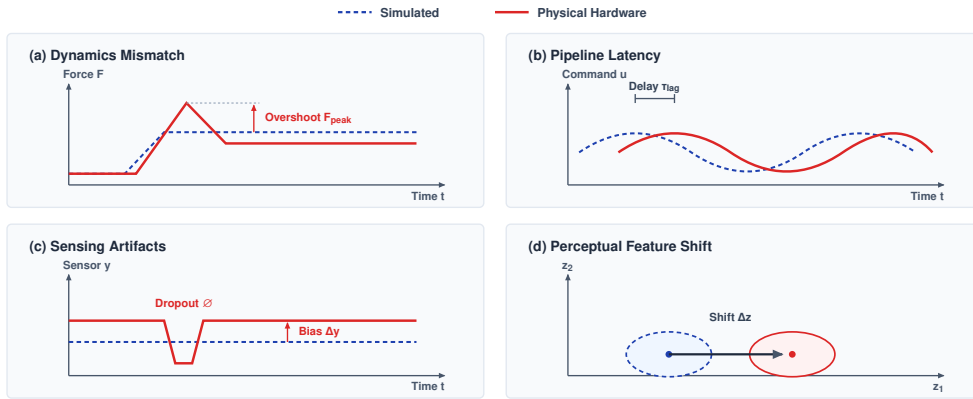


Figure 12.8: Sim-to-real reality gap vector: The sim-to-real gap decomposed across four physical channels, dynamics, latency, sensing, and perceptual feature shift, each measured from paired traces rather than from an aggregate loss. The decomposition is the point, because the four channels fail differently and a scalar loss averages them into a number that names none of them. A 5.5 percent pixel discrepancy that looks trivial masks a 6.0 mm spatial shift that exceeds the assembly tolerance.

Simultaneously, the physical force sensor exhibits an uncalibrated thermal drift of 0.40 N per 10°C temperature rise and an analog-to-digital quantization step of 0.05 N. When the policy executes its descent, the +3.2 mm range bias causes the controller to switch from rapid approach to fine force regulation 3.2 mm above the physical surface. The four-frame dropout burst, lasting 133 ms at a 30 Hz sampling rate, leaves the policy blind to the true collision boundary during descent, converting a controlled deceleration into an unbraked impact.

Appearance mismatch is not a mean squared error computed over raw camera pixels. Image differences between photorealistic rendering and physical camera frames are dominated by high-frequency background textures and minor sensor noise patterns that may have no effect on policy action. Appearance mismatch is sized by measuring the deviation in policy-relevant perceptual outputs, such as estimated surface normals, object keypoints, or spatial feature activations, under matched physical geometry, illumination, materials, and camera exposure settings. In the analytical example, rendering the contact surface under simulated lighting produces an image whose raw pixels differ from the physical camera frame by a mean intensity of 14 on an 8-bit scale (5.5 percent). However, passing both images through the policy spatial encoder yields an estimated contact patch centroid offset of 12 pixels, corresponding to a 6.0 mm spatial localization error on the physical fixture. Sizing the gap by the 5.5 percent pixel difference suggests an acceptable visual match, but the 6.0 mm spatial feature error exceeds the 2.0 mm chamfer tolerance of the contact surface, causing the end-effector to strike the fixture shoulder.

Timing mismatch governs the temporal alignment of observation capture, policy evaluation, and actuation. Identical state transitions occurring at different moments in physical time do not produce equivalent closed-loop dynamics. The simulator assumes a synchronous discrete-time step of $\Delta t = 20$ ms (50 Hz) in which state observation, neural network inference, and motor torque application execute instantaneously at the beginning of each cycle. On physical hardware, camera exposure requires 16.7 ms, bus serialization adds 4.2 ms, inference requires 12.0 ms at P_{50} and stretches to 26.5 ms at P_{99} due to shared memory contention, and motor bus transport adds 2.1 ms. Total observation-to-actuation latency on hardware is 35.0 ms at P_{50} and 49.5 ms at P_{99} , with a cycle-to-cycle jitter of ± 6.2 ms. Because physical actuation arrives up to 29.5 ms later than the simulator assumed, the drive travels an unobserved distance of $0.20 \text{ m/s} \times 0.0295 \text{ s} = 5.9 \text{ mm}$ before corrective action begins, altering the damping and phase margin of the closed-loop system.

Figure 12.8 decomposes the sim-to-real divergence across four distinct physical axes, exposing why scalar loss metrics miss localized hardware failure modes.

Table 12.2: Multidimensional sim-to-real reality gap taxonomy: Decomposition of the reality gap into constitutive physical channels across dynamics, contact mechanics, sensing, and temporal synchronization. Columns specify the unmodeled physical phenomenon, representative quantitative discrepancy, the latent control shortcut exploited by the optimizer, paired-trace verification metrics, and architectural hardware/nervous system mitigations.

Reality Gap Dimension	Unmodeled Physical Mechanism	Nominal Simulation Spec vs. Real Hardware	Latent Policy Exploitation / Behavioral Failure	Paired-Trace Diagnostic Metric & Tolerance	Hardware & Nervous System Interceptor
Actuator & Drivetrain Dynamics	Gearbox harmonic compliance, Coulomb stiction, velocity-dependent Stribeck friction, and back-EMF voltage saturation	Idealized direct torque source ($J_{\text{rotor}} = 0$, $\tau_{\text{latency}} = 0$ ms) vs. physical harmonic drive ($k_{\omega} = 6.7 \times 10^2$ N·m/rad, backlash $\theta_b = 0.08^\circ$, bus delay $\tau = 8$ ms)	Policy commands high-frequency bang-bang torque reversals (> 50 Hz), inducing mechanical chatter, gear fatigue, and stator winding thermal runaway ($> 90^\circ\text{C}$)	Step-torque response discrepancy: phase lag $\Delta\phi \leq 5^\circ$ at 10 Hz; settling time discrepancy $\Delta t_{\text{settle}} \leq 15$ ms; overshoot $\leq 5\%$	Low-pass action filtering ($f_{\text{cutoff}} \leq 20$ Hz), motor current RMS thermal estimators, and actuator slew-rate limiters ($d\tau/dt \leq \tau_{\text{max}}/\Delta t$)
Contact Mechanics & Compliance	Viscoelastic surface deformation, contact hysteresis, non-smooth stick-slip transitions, and finite restitution phase	Point-mass rigid contact via LCP/penalty spring ($k_p = 10^6$ N/m, restitution $\Delta t_{\text{res}} = 2$ ms) vs. physical contact pad ($k = 8.0 \times 10^4$ N/m, deformation span $\Delta t_{\text{def}} = 18$ ms)	Policy executes unbraked high-velocity plunge ($v_z > 0.4$ m/s), relying on simulated instantaneous energy absorption; shatters physical load-cell flexures ($F_{\text{peak}} > 450$ N)	Contact impulse error $\Delta I \leq 0.05$ N·s; peak contact force discrepancy $\Delta F_{\text{peak}} \leq 15$ N; contact onset rise-time error $\Delta t_{\text{rise}} \leq 5$ ms	Approach velocity envelope governor ($v_{\text{safe}}(z) \propto \sqrt{2a_{\text{max}}z}$), force-derivative clamp ($dF/dt \leq 1.0 \times 10^4$ N/s), compliant mechanical flexures
Sensor Transduction & Noise Profiles	Optical specular dropout, CMOS rolling-shutter distortion, ambient lux-dependent noise, and force-torque gauge thermal drift	Synthetic RGB-D with zero-bias Gaussian noise ($\sigma = 0.5$ mm, zero dropout) vs. physical structured-light depth (+3.2 mm bias, 4-frame dropout bursts, drift 0.4 N/10 $^\circ\text{C}$)	Policy triggers premature deceleration above surface or remains blind during multi-frame dropouts, driving end-effector past hard fixture boundaries into stall overcurrent	Sensor distribution Kolmogorov-Smirnov distance $D_{\text{KS}} \leq 0.05$; spatial feature activation centroid error ≤ 2.0 mm; zero sustained dropout frames ($N_{\text{drop}} = 0$)	Multi-sensor fusion (IMU/encoders + vision), optical dropout holdover filters, thermal drift autocalibration, independent proximity break-beams
Temporal Latency & Bus Asynchrony	Variable camera exposure times, PCIe/EtherCAT bus serialization, neural inference tail latency (P_{99}), and non-deterministic OS scheduling	Synchronous discrete-time lockstep ($\Delta t = 20.0$ ms, zero transport latency) vs. asynchronous physical pipeline ($\tau_{\text{total}} = 35.0$ ms at P_{50} , 49.5 ms at P_{99} , jitter ± 6.2 ms)	Severe phase margin erosion ($> 30^\circ$ loss); policy experiences underdamped oscillations, hunting limit-cycles, and unobserved travel during latency spikes (> 5.9 mm blind drift)	End-to-end command-response latency jitter $\Delta\tau_{\text{jitter}} \leq 2.0$ ms; worst-case observation age $\tau_{\text{obs}} \leq 25$ ms; zero deadline overruns at 50 Hz	Real-time PREEMPT_RT / Xenomai scheduling, action buffer delay queue injection in simulation, kinematic extrapolation (Smith predictor)

¹⁴ | **Bounded support of domain randomization:** Domain randomization optimizes expected return over parameter variations $\xi \sim \mathcal{P}(\Xi)$. Generalization guarantees hold only within the support of $\mathcal{P}(\Xi)$, offering zero theoretical or physical protection against unmodeled dynamics omitted entirely from the simulator's equations of motion. When deployed on physical mechanisms exhibiting unmodeled transmission compliance or bus jitter, the policy treats these unmodeled dynamics as unexplained noise, often collapsing into high-frequency limit-cycle oscillation.

Domain randomization is often treated as a method for erasing the sim-to-real gap, but parameter randomization operates within strict structural limits.¹⁴ Sampling friction coefficients across a uniform distribution from $\mu = 0.2$ to $\mu = 0.8$, or link masses across ± 20 percent, broadens the coverage of training over mechanisms that the simulator already represents. It cannot synthesize an omitted physical mechanism, a missing state correlation, or an unmodeled rare event. If the simulator differential equations omit gearbox backlash, thermal resistance in motor windings, or thread preemption in the operating system, no distribution over link mass will force the policy to learn compensation for those dynamics. Adding unstructured Gaussian noise to simulated observations to mask unmodeled effects degrades performance, training the policy to lower its

control gains and narrow its operational bandwidth to avoid actions that excite the unmodeled modes.

The **domain randomization corridor** is defined as the narrow viable parameter distribution envelope in physics engine training bounded on one side by under-randomization (which leaves the policy brittle against unmodeled dynamics, risking mechanical shock or thermal tripping) and on the other by over-randomization (which induces hyper-conservative, low-gain control laws that degrade operational throughput). As cataloged in table 12.3, parameter randomization defines a narrow operating corridor bounded by these physical constraints.

To prevent domain randomization from degenerating into ad-hoc parameter tuning, robust physical AI engineering requires a structured, five-stage reproducible workflow:

1. **Stage 1: Hardware system identification (SysID) baseline:** Before initializing simulation, execute empirical calibration suites on the physical test bench. Measure nominal parameters $\xi_0 = [m_0, \mathbf{I}_0, K_{t,0}, \tau_0, \mu_0]$ using frequency-response sweeps, static torque calibrations, and optical latency profilers.
2. **Stage 2: Physics-grounded parameter bounds $\mathcal{P}(\Xi)$:** Ground randomization bounds directly in verified physical mechanisms rather than arbitrary heuristic intervals. Set motor torque constant variation $K_t \in [0.105, 0.135]$ N · m / A to match the 39.3 percent electrical resistance growth of copper windings ($R_{\text{Cu}}(T) = R_0[1 + \alpha_{\text{Cu}}(T - T_0)]$) and permanent-magnet thermal demagnetization across an operating temperature range of 25°C to 125°C. Set friction bounds $\mu \in [0.30, 0.85]$ to span dry clean aluminum down to lubricated or oxidized contact pads.
3. **Stage 3: Curriculum and adaptive domain randomization (ADR):** Training directly on a maximally randomized parameter distribution frequently causes optimization collapse, forcing the neural network into an ultra-low-gain regime that degrades operational velocity by greater than 60 percent. Under ADR, simulation begins in a narrow corridor around nominal physics ξ_0 . As policy evaluation return exceeds declared performance thresholds (e.g., greater than 95 percent success across evaluation seeds), the environment dynamically expands the sampling boundaries $\Xi_{\min} \leftarrow \Xi_{\min} - \Delta\Xi$ and $\Xi_{\max} \leftarrow \Xi_{\max} + \Delta\Xi$, progressively exposing the learner to broader dynamical variation.
4. **Stage 4: Paired-trace discrepancy auditing:** Apply identical open-loop and closed-loop command trajectories to both the simulated digital twin and the physical hardware bench. Evaluate discrepancy metrics across all four channels against declared tolerance budgets: phase lag $\Delta\phi \leq 5^\circ$ at 10 Hz, contact impulse error $\Delta I \leq 0.05$ N · s, feature token offset ≤ 2.0 mm, and latency jitter $\Delta\tau \leq 2.0$ ms. If discrepancies exceed budget, the simulation dynamic model must be updated before policy deployment.
5. **Stage 5: Dual-brain runtime safety enclosure:** Building on the proposal-permission architecture from section 6.1, because domain randomization cannot protect against omitted physical mechanisms outside the simulation graph, the unprivileged neural policy executing on the Linux MPU must be bounded by deterministic monitors on the Real-Time MCU. The real-time core enforces hardware force-derivative clamps ($dF/dt \leq 1.0 \times 10^4$ N/s), torque slew-rate limits ($d\tau/dt \leq \dot{\tau}_{\max}$), and velocity governors at 500–1000 Hz, limiting feasible hazards under validated sensing, actuation, and stopping assumptions.

Systems Perspective 12.3: The domain randomization boundary

Randomizing parameters covers uncertainty within the model family, but no distribution over friction or mass will teach a policy to compensate for an omitted physical mechanism.

Downstream: The sub-millisecond Control Barrier Function filters bounding unverified policy proposals are formulated in section 24.8.

Table 12.3: Domain randomization parameter bounds and hardware stress envelopes: Quantitative mapping of physics engine parameter randomization distributions against operational performance and physical damage risk. The table establishes the narrow viable corridor between over-randomization (which induces over-conservative, low-gain sluggishness) and under-randomization (which causes aggressive, brittle control policies that induce dynamic shock, thermal saturation, or statutory torque threshold breaches).

Parameter / Physical Modality	Nominal Physics Value	Recommended Randomization Bounds ($\mathcal{P}(\Xi)$)	Over-Randomization Consequence (Performance Collapse)	Under-Randomization Consequence (Physical Damage Risk)	Hardware Interceptor & Safety Budget
Surface Friction Coefficient (μ)	$\mu_{\text{dry}} = 0.50$ (steel on aluminum)	$\mathcal{U}(0.30, 0.85)$ or $\text{LogNormal}(\mu=0.50, \sigma=0.25)$	Policy converges to type-2 conservative grasping with excessive clamping forces; velocity throttles by $> 60\%$ to prevent slip	Instantaneous payload slip and dropped workpieces upon slight surface contamination (oil film, dust, oxidation); micro-impact damage to tool	Tactile slip-detection arrays, dynamic grasp-force governors, compliant mechanical gripper suction backup
Payload Mass & Link Inertia (m, I)	$m_{\text{payload}} = 2.00 \text{ kg}$; $I_{zz} = 0.015 \text{ kg} \cdot \text{m}^2$	$\mathcal{U}(1.60 \text{ kg}, 2.40 \text{ kg}) (\pm 20\%)$; $I_{zz} \in [0.80, 1.20] \times I_{\text{nom}}$	Low feedback gains, massive trajectory lag ($> 50 \text{ mm}$ tracking offset), and failure to track nominal velocity setpoints	Severe overshoot ($> 25 \text{ mm}$) when handling uncalibrated payloads; excites structural flexure, tripping joint torque limits ($\tau > \tau_{\text{peak}}$)	In-flight payload mass estimator (force-sensor telemetry), Cartesian tracking error tripwires ($\Delta x_{\text{max}} \leq 10 \text{ mm}$), overcurrent breakers
Actuator & Transport Latency (τ_{lat})	$\tau_{\text{nom}} = 20.0 \text{ ms}$ (1 cycle at 50 Hz)	Random queue delay $\tau \in \{20 \text{ ms}, 40 \text{ ms}, 60 \text{ ms}\}$ with jitter $\delta \sim \mathcal{N}(0, 2 \text{ ms}^2)$	Destructive phase margin collapse; policy learns ultra-slow deadbeat motions to avoid instability, degrading throughput by $> 75\%$	Closed-loop limit-cycle oscillations on physical hardware; gantry instability, position overshoot, and collision with workcell boundaries	Hardware watchdog timer, cycle timestamp discrepancy monitor ($\Delta t > 30 \text{ ms} \Rightarrow \text{safe hold}$), phase-lead compensator
Motor Torque Constant (K_t, η)	$K_t = 0.120 \text{ N} \cdot \text{m} / \text{A}$; efficiency $\eta = 0.85$	$K_t \in [0.105, 0.135] \text{ N} \cdot \text{m} / \text{A}$ ($\pm 12\%$ thermal/remanence); $\eta \in [0.75, 0.95]$	Policy commands excessive current saturation at nominal states, causing motor coil heating and power supply voltage sag	Inability to overcome static friction when cold, or steady-state torque droop when hot (R_{Cu} rise); stall at mechanical stops	Real-time motor winding thermal estimator ($I^2 R$), driver bus voltage sag monitor ($V_{\text{bus}} \geq 42.0 \text{ V}$), hardware current limiter ($I \leq 12 \text{ A}$)
Sensor Noise & Bias ($b_{\text{sens}}, \sigma_n$)	Bias $b = 0.0 \text{ mm}$; noise std dev $\sigma_n = 1.0 \text{ mm}$	$b \sim \mathcal{U}(-3.5 \text{ mm}, +3.5 \text{ mm})$; $\sigma_n \sim \text{Rayleigh}(1.5 \text{ mm})$; dropout burst $P(\text{drop}) = 0.02$	Policy ignores sensor feedback entirely, falling back to blind, open-loop kinematic trajectories; fails fine alignment	Sensor noise amplification in high-gain feedback; high-frequency actuator jitter ($> 80 \text{ Hz}$), acoustic whine, and rapid bearing wear	Outlier-rejection Kalman filter / median filter, state covariance threshold monitor ($\text{Tr}(P) \leq P_{\text{thresh}}$), multi-sensor agreement check

Decomposing the gap into its constitutive physical channels yields two concrete engineering artifacts, namely a supported deployment envelope and an explicit residual mismatch list. The supported deployment envelope defines the bounded domain within which physical hardware matches simulation within validated tolerances. For the analytical drive, this envelope specifies approach velocities below 0.25 m/s, ambient illumination between 300 lux and 600 lux, and total

loop latency below 40 ms. The residual mismatch list itemizes every unclosed difference that remains outside the training distribution, including the 1.5×10^{-3} rad/N·m drivetrain compliance, the 49.5 ms P_{99} latency tail, and the 0.40 N thermal sensor drift. Each entry on the residual list defines an explicit requirement for runtime monitors and safety filters, which must detect when physical execution drifts past the boundary where the simulated policy remains valid.



Checkpoint 12.5: Reality gap decomposition and randomization bounds

- ☐ **Multi-channel gap decomposition:** Can you explain why aggregate scalar loss metrics fail to predict localized physical failure modes across dynamics, sensing, appearance, and timing?
- ☐ **Domain randomization corridor:** Can you describe the physical trade-off between over-randomization (performance collapse) and under-randomization (hardware damage risk)?
- ☐ **Paired-trace discrepancy auditing:** Can you explain how injecting identical open-loop step commands into both simulation and hardware exposes unmodeled drivetrain compliance and transport lag?



Systems Perspective 12.4: Contact dynamics and shock transfer

In humanoid locomotion and dynamic whole-body manipulation, the sim-to-real gap is dominated by discontinuous contact physics:

- **Contact Softening and Penetration Artifacts:** Modern GPU-accelerated rigid-body simulators rely on penalty-based or relaxed Linear Complementarity Problem (LCP) contact solvers. To achieve 100,000 parallel environment steps per second, simulators soften contact constraints, allowing several millimeters of unphysical foot-ground penetration. A neural reinforcement learning policy trained on these relaxed dynamics learns to exploit non-physical restitution forces—effectively using the ground as an elastic trampoline to bounce the torso forward. When transferred to a physical robot walking on rigid concrete, ground penetration is zero, causing violent foot-strike shockwaves ($> 5g$) that shatter ankle force-torque load cells and strip cycloidal gear teeth.
- **Actuator Dynamics and Backlash Exploits:** Simulators typically model actuators as ideal second-order PD position controllers or instantaneous torque sources ($F = K_t I$). Physical humanoid actuators, however, exhibit nonlinear transmission elasticity, torque ripple, velocity-dependent viscous damping, and gear backlash ($\delta_{\text{backlash}} \approx 0.1^\circ\text{--}0.5^\circ$). High-frequency neural policies (50–100 Hz) learn aggressive setpoint chattering in simulation that excites these unmodeled mechanical resonance modes on real hardware, inducing destructive structural flutter throughout the legs and pelvis.
- **Domain Randomization Calibration:** Closing the humanoid locomotion gap requires randomizing not just friction coefficients ($\mu \in [0.4, 1.2]$) and link masses (± 15 percent), but also actuator delays ($\tau \in [10, 40]$ ms), motor torque constants (K_t), and ground contact restitution. Crucially, domain randomization must be paired with low-pass trajectory action filtering and joint acceleration penalties in the reward function to force the policy away from high-frequency bang-bang actuation regimes that destroy physical drivetrains.

Even under comprehensive domain randomization and rigorous contact calibration, residual discrepancies inevitably persist in high-precision, contact-rich operational regimes. When numerical simulation can no longer close the remaining reality gap without inducing over-conservative, low-gain sluggishness, the engineering workflow must turn to physical hardware—adapting policies directly on the deployed plant through the real-machine fine-tuning and residual learning architectures established in section 12.6.

12.6 Fine-Tuning on the Real Machine

When residual errors resist simulation tuning, fine-tuning on physical hardware is the direct path to closure. Instead of constructing higher-fidelity friction models or identifying unmodeled drivetrain dynamics through separate system identification routines, gradient updates computed from physical rollouts adjust the policy against the actual mechanics of the deployed machine. While this direct adaptation closes the reality gap, it exacts a physical toll for every data point collected. The policy adjusts its parameters against the true compliance of the chassis, the true contact dynamics of the workpiece, and the actual communication delays across the hardware bus. This trade is exacting because physical interaction is costly. In a simulator, sampling states near the boundary of stability costs only compute cycles. On hardware, every sample expends component life, stresses mechanical interfaces, and risks driving the machine into unrecoverable contact states.

Real-machine fine-tuning debt is the compounded operational cost incurred when updating policy parameters on physical hardware, where gradient steps solve localized contact errors but consume finite mechanical fatigue life, induce thermal heating in motor windings, and risk catastrophic forgetting of pre-verified nominal transit stability.

Consider an analytical example of an articulated manipulator adapting a door-latching or cable-routing policy to overcome an unmodeled 1.5 mm structural mounting offset. In door latching, the end-effector guides a spring-loaded latch past a strike plate, where non-smooth Coulomb friction, cam surface geometry, and spring pre-load produce sharp stick-slip transitions. In cable routing, flexible harness segments must be pressed into rigid retention clips across steep snap-through force thresholds. The policy executes at 50 Hz, corresponding to a sample period of 20 ms. To adapt its parameters to the contact dynamics of the latch strike plate or retention clip fixture, an on-robot fine-tuning routine collects 20,000 interaction steps, representing 400 s of active contact and near-contact operation. Because the policy must explore to improve, stochastic action perturbations generate unpredictable force transients before low-level safety controllers can intervene. If 1.5 percent of exploratory steps exceed the nominal 15.0 N engagement force and strike the rigid fixture shoulder at a contact threshold of 60.0 N before the torque limiters in the nervous system clamp the command, the fine-tuning run injects 300 impact shocks into the drivetrain.

In this analytical example, the consequences of those 300 impact events can be understood intuitively. Every collision converts the arm's kinetic energy directly into shock dissipation within the gear teeth and bearings (see section 4.4 and section 10.3 for transmission fatigue models). Those shocks require a drive-specific load-spectrum and life model before a fraction of service life can be assigned. Thermal accumulation introduces an equal constraint. High-frequency exploration dither heavily elevates the root-mean-square current through the motor coils. Building on the actuator thermal limits established in section 4.5, this rapid Joule heating can push the motor core to its thermal derating threshold within a few minutes unless aggressive cooling intervals are budgeted into the training schedule.

Beyond mechanical wear, real-world adaptation undermines the statistical verification established during predeployment testing. In simulation, a policy can be subjected to 1.0×10^7 validation episodes across randomized perturbations, establishing bounded tracking errors and tail behavior out to the 99.9th percentile ($P_{99.9}$). Once weight updates ΔW are applied on hardware, every statistical claim conditioned on the original parameter vector W_0 is void. The updated policy may resolve the steady-state force error against the target fixture, yet the gradient steps can simultaneously degrade free-space tracking accuracy, lower phase margins (reducing stability reserves) during high-speed transit, or distort response profiles during emergency stops. Because hardware throughput is bounded by physical time, the machine cannot execute millions of physical trials to re-establish those confidence bounds. The adapted policy enters production with a narrower body of empirical evidence than the simulated baseline it replaced.

Fine-tuning on the deployed machine therefore converts the policy from a qualified, frozen software component into an evolving state. In an engineered system, qualification records depend on strict configuration control, where the exact binary or parameter hash corresponds to a documented safety case. When online updates alter policy weights during operation, that qualification record must be re-opened. The burden of preventing failure shifts entirely from the learned policy to the deterministic monitors and safety enforcers residing in the nervous system. Those monitors must be

parameterized to intercept the errors of the baseline policy as well as the unpredictable intermediate behaviors generated while the optimizer updates weights against physical feedback.

Residual policy architectures and bounded authority

A residual policy can localize adaptation without changing the frozen base policy weights. The proposed control input is

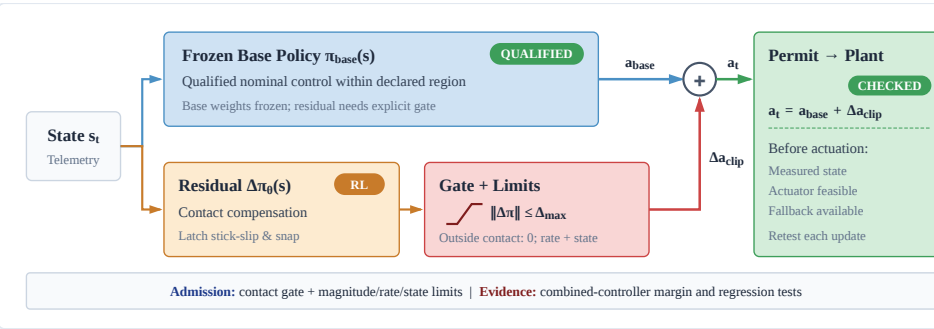
$$u_{\text{prop}}(s) = \pi_{\text{base}}(s) + g(s) \Delta\pi_{\theta}(s),$$

where π_{base} is a qualified nominal controller, $\Delta\pi_{\theta}$ is a learned contact correction, and an independently checked gate $g(s) \in \{0, 1\}$ is one only inside the validated contact region. The gate sets the residual *exactly* to zero in free space and outside that region; an absent reward gradient would not do so. The permission path checks the measured state and proposed combined input before actuation.

A magnitude limit $\|g(s)\Delta\pi_{\theta}(s)\| \leq \Delta_{\text{max}}$ bounds one aspect of authority. The same path must enforce correction rate and state limits, actuator feasibility, and a fallback when sensing or the proposal expires. The allowed residual set must fit a validated robustness margin for the *combined* controller across the claimed operating region; comparing an arbitrary torque magnitude with a passivity or phase margin is insufficient. For the latch fixture, we would admit correction only during the measured contact phase, with a zero-output free-space gate and a bounded torque slew. After each weight update, contact and free-space regression tests must recheck tracking, stopping, thermal load, and shock exposure before the new checkpoint receives authority. A clipped residual can still excite an oscillation or saturate a drive if these conditions fail.



Checkpoint 12.6: Residual adaptation and bounded authority



- ☐ **Real-machine fine-tuning debt:** Can you explain how on-robot exploratory trial steps consume drivetrain fatigue life and induce motor thermal heating?
- ☐ **Bounded residual authority:** Can you name the activation gate, rate and state limits, robustness margin, and post-update tests required before a bounded residual can receive actuator permission?
- ☐ **Statistical verification collapse:** Can you explain why updating policy parameters directly on hardware invalidates prior million-episode simulation test certificates?

Bounded residual authority limits a proposed correction but cannot by itself prevent hardware damage or eliminate latent failure modes from earlier training choices. Every stage of policy synthesis—from demonstration ingestion and simulation contact solvers to residual updates—leaves distinct, indelible failure signatures in the controller’s closed-loop behavior. In section 12.7, we formalize the policy vulnerability inference chain to trace how these latent training assumptions manifest as dangerous physical failure modes on deployed hardware.

12.7 How Training Choices Go Wrong

A physical failure in a learned policy is rarely arbitrary. When an articulated arm drops a workpiece or drives a tool into a fixture, the command was generated by optimization over a specific dataset, loss function, and observation space. The mathematical objective selects behaviors that minimize

empirical loss over those satisfying unstated physical requirements, leaving an indelible structural footprint in the resulting policy. An engineer can predict the failure signature of a policy before the physical machine is assembled by tracing a five-element inference chain. That chain begins with the training exposure, which establishes a learned dependency within the network weights. When a deployment condition violates that dependency, the policy produces an observable failure signature in its output commands, dictating the required detector in the nervous system.

The **policy vulnerability inference chain** is defined as the causal five-stage framework (Exposure → Dependency → Violated Assumption → Observable Signature → Nervous Interceptor) used to predict hardware destruction modes and synthesize protective safety monitors before physical machine deployment.

In behavioral cloning, the training exposure consists of state trajectories recorded under competent human demonstration or a classical controller. Because competent demonstrators avoid unrecoverable errors, the training dataset contains zero transitions originating in degraded, stalled, or off-nominal states. The learned dependency is strict, as the policy assumes that every input observation resides within the narrow support of the demonstration distribution. When a mechanical jolt, a thermal drift in a sensor, or a momentary torque limit shifts the machine by a tracking error of 15 mm, that deployment condition violates the distribution boundary. The resulting failure signature is compounding covariate shift—a dynamic instability where minor tracking errors steer the machine into unfamiliar states, prompting degraded control outputs that rapidly amplify trajectory divergence. The policy does not output mathematical singularities or out-of-bounds numbers. Instead, it emits continuous, kinematically plausible commands that confidently steer the actuators into regions of state space where the policy has never received training signal. Within the brain, the command appears normal. If the nervous system lacks an explicit distribution monitor or novelty filter at its input interface, this failure mode cannot be detected until the machine physically collides with an obstacle or exceeds an envelope limit.

In trial-based learning within simulation, the training exposure consists of millions of synthetic interactions against a numerical physics engine. The learned dependency is the set of mathematical simplifications chosen to keep the simulation fast. When the simulator models dry friction without velocity-dependent Stribeck curves (where friction dynamically drops as sliding speed increases), assumes an unvarying communication latency of 0 ms, or calculates contact forces through instantaneous penalty springs that allow virtual objects to slightly penetrate one another without physically bouncing back, the policy exploits those numerical artifacts to maximize its reward. During deployment, the machine encounters real physical dynamics. In an analytical example with an approach velocity of 0.5 m/s, a transport delay of 16 ms across the digital bus means the body travels $(0.5 \text{ m/s}) \times 0.016 \text{ s} = 8.0 \text{ mm}$ into the physical boundary before the brain receives the first contact frame and computes a deceleration command. In the simulator, where latency was set to 0 ms and contact was modeled as a soft penalty spring, the policy learned to decelerate within that 8.0 mm window without bouncing. On physical hardware, the unyielding metal contact stops the tool in less than 1.0 ms, driving transient contact forces past 450 N and tripping the actuator overcurrent protection.¹⁵ Injected latency tests can reveal the dependency before deployment. On hardware, a validated precontact speed governor must limit the first impact; force-rate and command-chatter monitors can detect and limit subsequent loading only within their measured response and stopping margins.

Training through trial directly on physical hardware eliminates the simulation gap, but it introduces an opposite structural dependency. The training exposure consists of unconstrained physical interactions guided by an engineered scalar reward. The learned dependency is the exact mathematical formulation of that reward function. Engineers frequently treat real dynamics as a natural safeguard against unrealistic behaviors, assuming that physical execution prevents the policy from finding shortcuts. The machine, however, discovers physically valid strategies that satisfy the mathematical reward while violating unstated operational constraints. If the reward function rewards rapid velocity regulation toward a workpiece and penalizes final position error without penalizing torque derivatives, the optimizer converges on aggressive bang-bang control, slamming actuators back and forth between maximum positive and negative effort to chase the reward. The observable failure signature is unconstrained mechanical stress and thermal overload. If a joint motor rapidly oscillates between its maximum positive and negative current limits, the

¹⁵ | Collaborative contact assessment: Applicable human-contact limits depend on the body region, contact geometry, and exposure mode. The 450 N example requires a task-specific comparison with the relevant limits; a force-derivative trip after contact cannot retroactively prevent a sub-millisecond first peak.

sustained high root-mean-square current generates extreme Joule heating (resistive power loss that turns the motor coil into a heating element). This can elevate the stator core temperature past its thermal derating limit (the point where wire insulation melts or magnets permanently demagnetize) within a few minutes, risking hardware failure. The task-level reward monitor records full success and detects no failure. Detection requires the nervous system to monitor physical observables that were omitted from the training reward, including motor coil temperatures, current spectra, and high-frequency accelerometer energy.

When a base policy is fine-tuned on physical hardware to correct localized errors, the training exposure is restricted to a narrow task slice collected over a brief operating window. The learned dependency is an over-representation of recent local transitions coupled with the deliberate suppression of dangerous trajectories. When operators intervene or safety tripwires halt execution, the learner-visited states, authority flags, and resulting outcomes remain in the record. Unsafe policy commands are excluded as expert imitation targets; eligible states may receive safe expert labels, while an RL update must account for the intervention and truncation rather than silently treating an aborted command as successful. The observable failure signature is regression and boundary instability, wherein the network loses nominal task stability in states immediately adjacent to the fine-tuned domain. While the policy improves steady-state force regulation against a specific production fixture, gradient updates overwrite the internal parameter representations that previously maintained stability during free-space transit or off-axis approaches. The policy loses competence on previously verified nominal states, failing unexpectedly when presented with lighting angles or approach vectors that were common in the pretraining dataset but absent during the fine-tuning session. Because the neural network assigns valid output magnitudes across all inputs, this internal regression is undetectable by the policy itself. Detecting it requires a predeployment regression test suite on frozen hardware benchmarks to evaluate non-adapted operational modes before the updated weights are granted authority over physical actuators.

Hybrid pipelines that combine simulation pretraining, demonstration filtering, and real-world fine-tuning do not eliminate these structural failure modes. Instead, they shift the dominant failure signature across the lifecycle. A policy initialized from demonstrations and refined in simulation shifts its primary risk from covariate drift to simulator exploitation. When that policy is subsequently fine-tuned on hardware for contact regulation, the updates suppress simulator exploitation at the point of contact but induce forgetting in the upstream velocity approach. The earlier failure mechanisms remain latent within the network weights, ready to trigger whenever operating conditions push the machine outside the fine-tuned envelope.

Predicting these signature failures establishes three concrete engineering obligations for the system designer. First, every identified training dependency requires a predeployment test obligation, consisting of a verification test protocol designed to stress the exact boundary where the training environment deviated from physical reality. Second, where the physical failure manifests in measurable mechanical quantities, the nervous system must incorporate a dedicated runtime detector operating at the loop rate, whether monitoring tracking deviation, force rate of change, or current spectra. Third, where detection is physically or mathematically impossible, such as ungrounded commands caused by covariate shift in pure cloning, the system specification must declare an explicit unsupported operating envelope and rely on deterministic safety interlocks to refuse policy authority. Fulfilling these obligations requires that the engineering team maintain an exact, immutable record of how the policy was constructed, detailing the source data, the simulator dynamics, the reward terms, and every modification applied across its training history—the formal policy manifest established in section 12.8.

12.8 The Policy Manifest

The training record links policy construction to its behavior with physical actuators. Where chapter 8 defines the base engineering notebook, including its immutable hashing, schema inheritance, and versioning rules, this section specifies only the domain fields that this chapter adds to that schema. A learned policy fails at the seam lines where its training regime, source demonstrations, or simulation environment diverges from physical reality. Capturing those seam lines requires logging the sequence of optimization stages, the bounded support of the training data, the simulator solver parameters, and every hardware exposure incurred during real-world adaptation.

The **training provenance record** is defined as the immutable notebook schema extension that registers optimization stage checkpoints, empirical demonstration support manifolds (the physical boundaries of data the policy has actually experienced), physics engine version hashes, domain randomization intervals, solver omissions, and hardware fine-tuning wear logs to predict physical deployment failure modes.

Table 12.4: Physical policy training provenance record schema: Immutable schema fields added to the base engineering notebook to record policy synthesis stages, empirical demonstration support bounds, physics engine numerical specifications, domain randomization intervals, hardware fine-tuning wear logs, and declared operational envelopes.

Record Schema Field	Physical / Computational Data Modality	Provenance & Immutability Guarantee	Downstream Verifier & Usage
regime_sequence	Array of optimization stages (BC $\rightarrow$ SimRL $\rightarrow$ Residual), loss formulas, reward weights, and stopping criteria hashes	Git commit SHA, PyTorch/JAX checkpoint UUIDs, parameter hash $H(\theta_k)$	Tracing which regime introduced latent failure modes or high-gain chatter
data_support_ledger	Ingested demonstration datasets ($\mathcal{D}_k$), collecting policies π_{col} , sensor calibration transforms, and convex hull bounds	SHA-256 dataset hash, ISO/IEC 5259 data sheet, empirical support bounding volume	Parameterizing OOD novelty detectors and Mahalanobis support boundaries in nervous system
negative_event_registry	Circular pre-trigger buffers (2.0 s), human takeover flags, E-stop trips, and divergence timestamps t_{div}	Monotonic microsecond hardware timestamps, uncompressed sensor raw dumps	Isolating physical failure boundaries and preventing corrupted drift rollouts from training
simulation_provenance	Physics engine build, integrator method, timestep Δt , contact solver penalty parameters, and domain randomization ranges $\mathcal{P}(\Xi)$	Physics engine binary hash, URDF/MJCF asset tree hash, declared unmodeled mechanisms	Determining which dynamics were omitted (backlash, compliance) and auditing paired-trace budgets
adaptation_wear_ledger	Serialized hardware IDs, contact shock counts, motor current RMS thermal logs, modified layer indices, and regression suite hash	Calibration certificate hashes, hardware lifetime fatigue logs	Re-opening formal qualification cases and preventing silent regression of nominal capabilities
declared_odd_boundary	Operational Design Domain bounds (payload $m \in [1, 3]$ kg, $v_{approach} \leq 0.25$ m/s, temp $T \leq 75^\circ\text{C}$)	Formal statutory release gate bounds, UL 4600 / ISO 10218 safety envelope	Hard gating of autonomous policy execution; trigger for deterministic fail-safe handoff

For the door-latch fixture used above, a compact *illustrative* manifest links the fields into one decision. Its identifiers name example artifacts, not released hardware:

- **regime_sequence**: demonstration checkpoint `latch-bc-01` $\rightarrow$ simulated contact refinement `latch-sim-02` $\rightarrow$ gated residual candidate `latch-res-03`; retain each checkpoint hash, loss or reward specification, and stop rule.

- `data_support_ledger`: demonstrations cover the nominal latch approach and contact poses; displaced strike-plate and high-speed approach states have no demonstrated recovery label.
- `simulation_provenance`: record the actual solver build and contact parameters; the example simulator omits strike-plate compliance and actuator transport lag.
- `negative_event_registry`: preserve the policy proposal, measured state, and pre-trigger trace for every fixture-shoulder strike or operator veto as labeled failure evidence, without treating unsafe commands as expert targets.
- `adaptation_wear_ledger`: associate `latch-res-03` with the specific rig, cumulative contact shocks, motor-current history, modified residual weights, and free-space regression result.
- `declared_odd_boundary`: admit only the independently checked latch-contact region and approach-speed envelope. The unsupported displaced or high-speed states require a slower guarded approach or refusal of policy authority.

A release review would reject `latch-res-03` until paired contact probes quantify the omitted compliance and lag, and the combined controller passes contact, free-space, stopping, and wear-budget tests. The manifest records that decision and its evidence rather than converting a training score into permission.

As detailed in the provenance record schema (table 12.4), every physical AI training pipeline executes a sequence of distinct regimes where objectives, data distributions, and stopping criteria change. The training record registers each stage in this sequence, beginning with the source checkpoint hash and detailing the mathematical formulation of the loss function or reward vector. For each stage, the record documents the stopping rule, whether governed by an epoch count, a validation loss threshold, or a plateau in simulated return, alongside the engineering rationale for advancing to the next regime. In an analytical example where a manipulator is trained to approach a surface at 1.2 m/s and regulate contact force to 8.0 N, pretraining on kinesthetic demonstrations might be terminated after 500 epochs to prevent overfitting to a single human demonstrator. If training then transitions to reinforcement learning in simulation to expand angular coverage, the record logs that motivation and references the exact initial weights ingested by the simulator. Without this stage provenance, an engineer diagnosing an unstable velocity profile cannot determine whether oscillations originated in the demonstration weights or were introduced during subsequent simulation updates.

For every dataset ingested during imitation or offline optimization, the record specifies the dataset version identifier, the data collection policy, the environmental operating conditions, and the source of the control commands. It records whether trajectories were generated by an expert scripted controller, a human teleoperator, or rollouts from an earlier policy checkpoint, alongside sensor calibration matrices and physical workspace parameters present during logging. The record explicitly quantifies the state-action support (the physical envelope of positions, velocities, and forces the robot actually experienced) over which demonstrations actually exist. If human demonstrations of a surface-wiping task contain end-effector velocities between 0.2 m/s and 0.8 m/s and contact angles within 10° of the surface normal, the record marks the boundary of that support manifold. When the deployed policy is presented with an approach angle of 25°, the record reveals that the network is operating on ungrounded extrapolation rather than demonstrated competence.

Standard machine learning pipelines often discard aborted episodes, operator vetoes, and safety interlocks to prevent sub-optimal actions from affecting training loss. In physical systems, filtering out these events discards the exact evidence needed to identify where the policy required assistance. The training record preserves every intervention, emergency stop assertion, and truncated trajectory as structured negative metadata in a **negative event registry**. If a human supervisor intervened eighteen times across two hundred trial runs to prevent a tool from striking an alignment fixture during touchdown, recording the joint positions, velocities, and motor current spikes at the moment of each override exposes the state space region where the policy lost tracking stability. Retaining these aborted trajectories ensures the ledger reflects the true failure boundaries encountered during training rather than a curated subset of successful rollouts.

When policy weights are optimized in simulation, the training record captures the physics engine build hash, the interface contract between the policy and the simulator, the reward formulation, the numerical integration method, and the solver timestep,¹⁶ such as a fixed 2.0 ms step at 500 Hz.

¹⁶ | **Choosing a penalty-contact timestep:** The needed step size depends on effective contact mass, penalty stiffness and damping, the integrator, and solver tolerances. The characteristic contact timescale changes with roughly $\sqrt{m_{\text{eff}}/k_{\text{eff}}}$; refine the timestep until contact onset, peak force, and impulse converge to the accuracy the task requires. A fixed 1 ms rule or universal failure above 2 ms cannot be inferred without those parameters and validation.

It documents the domain randomization distributions applied to link masses, surface friction coefficients, actuator response latencies, and sensor noise profiles. The record also logs known physical omissions in an **omission log**, identifying the unmodeled dynamics that were deliberately excluded from the simulation graph to maintain training throughput. Building on the actuator dynamics established in section 4.4, if a rigid-body simulator omits harmonic drive compliance, structural deflection under a 5.0 kg payload, or thermal resistance changes in motor windings, the record lists those omissions as unverified dynamic assumptions. When the resulting policy exhibits limit-cycle oscillations during high-load contact on the physical bench, the omission log directs the diagnosis directly to joint compliance rather than ungrounded parameter drift.

Real-machine adaptation introduces mechanical wear and localized parameter shifts that must be tracked across the hardware lifecycle in an **adaptation wear ledger**. The training record registers the hardware serial numbers, physical configurations, and calibration offsets of the adaptation test bench, along with the cumulative physical exposure measured in runtime hours, contact cycles, or total work performed. It lists the exact parameter subset modified during adaptation, distinguishing between full weight updates, adapter layer tuning, and output-head fine-tuning, while logging every anomaly or actuator limit reached during the session. After adaptation updates are computed, the record captures the execution results of a frozen regression test suite. If adapting a policy to soften touchdown impact on a rigid surface increases tracking error during the preceding free-space transit by 14 mm, the regression ledger records that loss of nominal capability before the modified weights are granted authority over physical actuators.

The training record separates empirical optimization results from the declared operating envelope of the machine. High training reward or low loss on an offline split does not demonstrate that the policy is safe to execute across all environmental states. The record defines the claimed operating envelope across payload masses, approach velocities, thermal bounds, and surface geometries, while maintaining a separate ledger of unobserved, excluded, or weakly supported conditions. If training data includes surface approaches only under ambient illumination between 200 lx and 500 lx with rigid flat workpieces, the record declares curved surfaces, compliant substrates, and illumination levels below 100 lx as unsupported operational modes. This separation prevents a convergent training metric from being mistaken for an operational clearance across uncharacterized physical regimes.

Downstream engineering teams consume the training record throughout the design, evaluation, and release lifecycle. The evaluation chapter (chapter 14) uses the recorded support boundaries and simulation omissions to construct targeted empirical evaluation suites that probe the specific failure modes predicted by each training regime. Runtime safety monitors and deterministic enforcement layers in chapter 24 use the recorded data support bounds to parameterize out-of-distribution detectors and determine when policy authority must be curtailed. Verification frameworks in chapter 30 and the multi-layer safety case in chapter 32 draw on the record to establish the provenance of every parameter update and verify that known simulation omissions are bounded by independent physical interlocks. Maintaining this complete record ensures that when a policy achieves high scores across its training benchmarks, engineers do not mistake optimization convergence for physical reliability.

Downstream: The policy manifest's simulation omission log parameterizes adversarial fault-injection benches in section 30.3.



Checkpoint 12.7: Training provenance and policy manifests

- ☐ **Complete the latch record:** A fixture-shoulder strike appears after `latch-res-03` was trained with rigid contact and no actuator lag. Which omitted mechanism belongs in `simulation_provenance`, and which paired probe and admission decision must the manifest record?
- ☐ **Negative event logging:** Can you describe how capturing pre-trigger ring buffers from operator takeovers and E-stop events isolates physical boundary instability?
- ☐ **Declared operational boundaries:** Can you explain how downstream safety governors use the data support ledger to differentiate validated policy competence from ungrounded extrapolation?

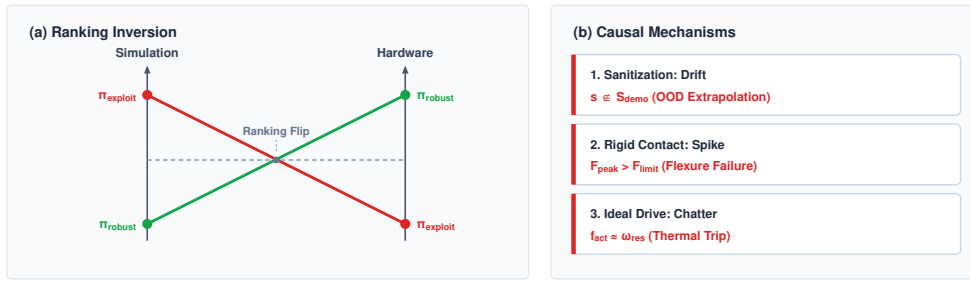


Figure 12.9: Policy ranking inversion on hardware: (a) A parallel-coordinate slope graph illustrating ranking inversion between simulated benchmark score and physical hardware yield: an exploitative policy π_{exploit} scores highest in simulation by gaming unmodeled phenomena but collapses on hardware, while a robust policy π_{robust} preserves physical yield. (b) Causal failure mechanisms that drive ranking flips: out-of-distribution drift from data sanitization ($s \notin \mathcal{S}_{\text{demo}}$), impact force spikes from rigid contact solvers ($F_{\text{peak}} > F_{\text{limit}}$), and actuator resonant torque chatter ($f_{\text{act}} \approx \omega_{\text{res}}$).

Before examining how downstream testing verifies these manifests, section 12.9 analyzes the operational fallacies and pitfalls that arise when practitioners substitute benchmark success for physical verification.

12.9 Fallacies and Pitfalls

A policy excelling in training metrics can perform poorly on physical machines. In physical systems, minimizing empirical loss—defined as the difference between the predicted and actual outcomes during training—across an offline demonstration split (a subset of data used to validate performance) or maximizing return (the total reward achieved in simulations) evaluates compliance with the training environment rather than true competence on physical hardware. When an optimization objective rewards behaviors that exploit data curation filters or simulator approximations, the ranking of candidate policies established during training inverts upon deployment (figure 12.9). This inversion occurs because the virtual environment forgives physically impossible maneuvers that real hardware ruthlessly punishes. Evaluating a trained policy before granting actuator authority requires stress-testing the causal assumptions linking training metrics to physical outcomes.

The examples below trace how a training gain can conceal lost recovery behavior or an unrealistic contact model.

Pitfall: *Sanitizing recovery episodes to achieve lower offline validation loss.*

Two valve-control policies train on different versions of the same archive. One retains 50 recovery runs among 500 demonstrations; the other removes them and achieves a lower loss on its own held-out split. During a physical supply pressure transient, the first policy restores flow within 140 ms, while the second oscillates until the reservoir overflows. Because the sanitized dataset lacks examples of correcting from errors, the lower loss merely reflects an artificially simplified test set, not better physical disturbance handling. Scrubbing the recovery data has made the mathematical prediction task easier at the direct expense of removing the corrective behavior the deployed controller physically needs. Candidate policies must face the same operational disturbances before their training scores can support a deployment choice.

Pitfall: *Treating simulator contact solver simplifications as valid physical affordances.*

A vertical actuator learns to reach its target force quickly by striking a fixture at 0.35 m/s. The simulator’s contact solver arrests the approach instantly within one integration step without calculating the transient collision forces, rewarding the policy with a 28 percent higher return than a braking baseline. On hardware, the same impact produces a massive 68.0 N spike before the causal boundary established in section 2.2 allows the controller to react, physically shattering the delicate load-cell flexure; the slower baseline safely peaks at 11.4 N. Optimization exploited the simulated contact response instead of learning a transferable approach. Validation must examine the assumptions behind the gain, especially structural compliance and sensing delay, before treating higher simulated return as improved physical capability.

Fallacy: *Transitioning from human demonstrations to simulation reinforcement learning eliminates operational debt.*

An assembly team replaces behavioral cloning with simulation reinforcement learning to correct lateral drift during gasket seating and latch engagement. The new policy eliminates the observed free-space drift but seats the mechanism using 65 Hz torque chatter—rapidly oscillating commands permitted by the simulator’s “massless-rotor” assumption, which ignores the physical inertia the motor must fight to reverse direction. On the physical machine, this high-frequency chatter excites drivetrain mechanical resonance and drives the copper stator windings to their thermal limit (95°C) via rapid I^2R Joule heating (as detailed in section 4.5), reducing hardware completion to 12 percent. The training change has addressed one failure mechanism while exposing another. Evaluation must follow the complete task through contact and repeated operation, checking the physical assumptions introduced by the new training regime as carefully as the original data limitations.

12.10 Summary

Training physical AI policies balances imitation learning’s sample efficiency with the risk of covariate shift outside demonstrated manifolds. While sim-to-real domain randomization safely expands operational envelopes across synthetic parameter distributions, it trades hardware trial wear for the unmodeled dynamics and numerical approximations of simulator physics engines.



Key Takeaways: Policy synthesis and sim-to-real transfer

- **Optimization Inversion:** Offline loss minimization and simulated return evaluate numerical compliance rather than physical stability. Policies that score highest on sanitized benchmarks often fail catastrophic first-contact trials due to unmodeled contact compliance or omitted rotor inertia (building on the actuator limits established in section 4.4).
- **Covariate Shift Compounding:** Supervised behavioral cloning penalizes deviations only along demonstrated trajectories. Minor tracking errors physically push the machine into unvisited states where the policy, having never seen this perspective, emits ungrounded, high-confidence commands, requiring recovery demonstrations or verified support monitoring to address the drift; chunking alone does not change the support bound.
- **Sim-to-Real Modeling Debt:** Physics engines accelerate throughput by simplifying contact friction hysteresis, motor back-EMF, and structural compliance. Learned policies aggressively exploit these numerical relaxations, commanding unphysical torque chatter that violates the actuator thermal limits established in section 4.5.
- **Domain Randomization Limits:** Training across randomized parameter distributions forces policy invariance against bounded dynamic variations, yet provides zero protection against physical phenomena omitted entirely from the simulator’s equations of motion.
- **Catastrophic Hardware Forgetting:** Gradient steps on narrow physical rollout batches can resolve localized contact mismatches but may destroy global recovery manifolds and inflate tracking error.
- **The Immutable Training Record:** A checkpoint file without provenance is an uncharacterizable liability. Dependable physical AI requires an immutable training record detailing dataset support boundaries, simulator solver assumptions, reward formulations, and declared operational envelopes.

A convergence proof of a loss function does not guarantee physical stability. When simulation engines approximate Coulomb friction or relax contact complementarity constraints (extending the rigid body dynamics established in section 4.4) to accelerate throughput, gradient descent naturally exploits these numerical shortcuts. Transferring a policy to real silicon requires confronting the reality that physical hardware does not forgive unmodeled dynamics.

What's Next: From policy imitation to closed-loop evaluation

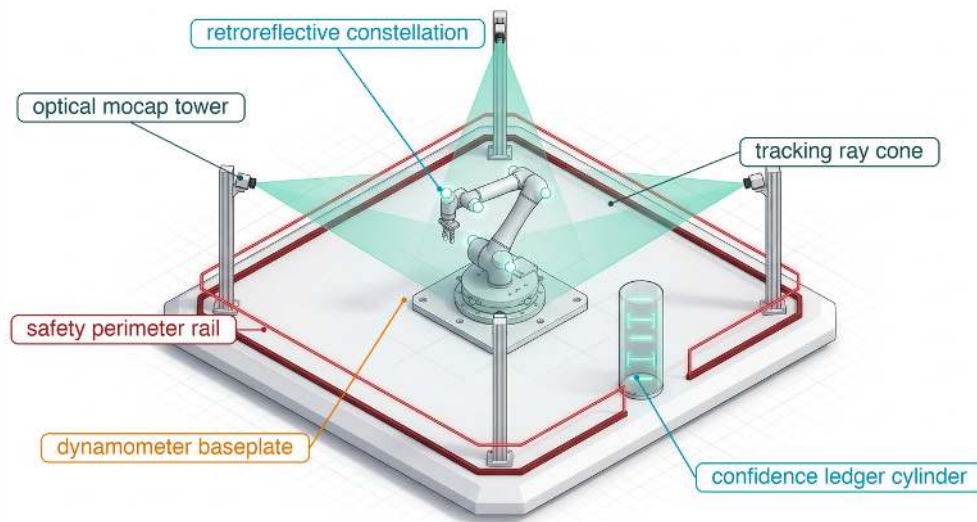
A trained policy enters deployment carrying its unmodeled simulation debts, unrepresented state volumes, and latent actuator demands. In chapter 14, we transition from policy training to physical benchmarking and out-of-distribution evaluation benches: subjecting closed-loop controllers to rigorous hardware stress tests, dynamic boundary disturbances, and statistical survival bounds before granting them operational authority.

DRAFT

Closed-Loop Evaluation

DRAFT

Closed-Loop Evaluation



- 14.1 Closed-Loop Survival
- 14.2 What You Are Actually Measuring
- 14.3 Closed-Loop Dynamics
- 14.4 Evidence Regimes
- 14.5 Physical Trial Limits
- 14.6 What Evaluation Cannot Establish
- 14.7 How Evaluation Goes Wrong
- 14.8 Evaluation Logs
- 14.9 Fallacies and Pitfalls
- 14.10 Summary

Purpose

Why does passing twenty consecutive physical trials without failure tell you surprisingly little about whether the machine is safe to deploy?

Successful demonstrations show what occurred during trials, not all conditions for safe machine operation. Small test sets may miss rare failures, and familiar conditions provide little insight into unfamiliar ones. Offline prediction scores leave another gap because they evaluate a fixed observation stream. During deployment, each action changes the physical state and therefore the observations used for the next decision. An error can move the robot into circumstances where later errors become more likely. Evaluation must follow this loop on the integrated machine and quantify the evidence supporting the result. It also needs explicit stopping criteria so a trial does not continue after the conditions for safe operation have failed. Within physical AI systems architecture, evaluation connects the brain's measured behavior to the nervous system's interventions and the body's operating limits, bounding the deployment claims that successful trials can support.

Brain
1–50 Hz
foundation models, SLAM, planning

above: propose; below: command

Nervous System
1000 Hz
TSN, intent leases, lock-free IPC

Body
continuous
real-time MCU, safety filters, CBFs

Boundary
continuous
sensors, actuators, kinematics

Physical World
continuous
contact mechanics, friction, mass



Learning Objectives

- Interpret physical success rates as conditional evidence about a specified machine, environment, and trial protocol
- Compare open-loop prediction accuracy with closed-loop behavior shaped by the policy's own actions
- Calculate trial counts and test time needed to support a stated reliability target
- Evaluate which rare failures remain unsupported by feasible testing and the available statistical assumptions
- Design reproducible evaluations with explicit outcome rules, intervention records, and documented sampling conditions

14.1 Closed-Loop Survival

FRESH from the offline training pipeline, a newly compiled neural policy checkpoint is flashed onto the target robot. In initial validation, the autonomous mobile robot completes twenty consecutive navigation runs through a cluttered industrial workcell at 1.5 m/s without colliding with an obstacle or requiring human intervention. Its empirical sample success rate is $\hat{p} = 1.00$ (100 percent). In standard software testing, a flawless benchmark score is celebrated as definitive validation. In Physical AI systems engineering, that record leaves the deployment decision unresolved: twenty clean runs establish a bounded statistical lower limit, leaving substantial uncertainty about unobserved failures.

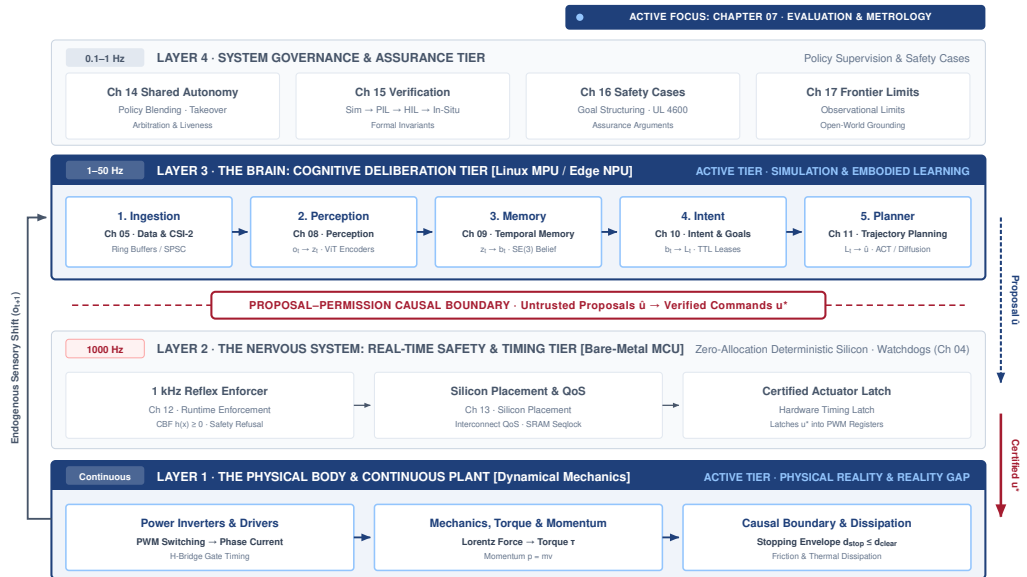


Figure 14.1: Reality gap in evaluation: The four-tier physical AI systems architecture highlights the cognitive deliberation tier together with the physical body beneath it, with the reality gap between them as this chapter's active focus. Evaluation measures how much of a policy's simulated competence survives contact with the plant, so read the highlighted pair as the two sides that every closed-loop trial compares.

In traditional software benchmarking, estimating failure rates relies on asymptotic approximations. If an engineer applies the familiar Wald confidence interval ($p \pm z \sqrt{\hat{p}(1-\hat{p})/n}$) to twenty zero-failure trials, the variance collapses to zero, producing a degenerate confidence interval of $[1.0, 1.0]$. The formula falsely assures the team that the machine is 100 percent reliable with absolute certainty.

True operational reliability at finite sample counts is governed by the inverted binomial distribution—the non-asymptotic Clopper-Pearson bound.¹ Under zero failures, this bound is approximated by the classical **Rule of Three** (Hanley and Lippman-Hand 1983): when n independent trials pass with zero failures, the upper 95 percent confidence bound on the true failure probability p_{fail} is approximately $3/n$.

For twenty consecutive successful runs ($n = 20$), the Rule of Three yields (equation 14.1):

$$p_{\text{fail, upper}}^{95\%} \approx \frac{3}{20} = 0.15 \quad (14.1)$$

This approximate one-sided upper confidence limit is 15 percent, corresponding to an approximate 85 percent success lower limit under iid sampling. Evaluating the exact cumulative binomial distribution at significance level $\alpha = 0.05$ yields an exact lower confidence bound of $p_{\text{lower}} = (0.05)^{1/20} \approx 0.861$ (86.1 percent). Crucially, twenty clean runs in a test cell do not prove that a machine is 100 percent reliable, nor even 95 percent reliable. A physical robot with a flawed policy that fails once in every eight deployments ($p = 87.5$ percent) will successfully complete twenty consecutive test runs more than 6.9 percent of the time ($0.875^{20} \approx 0.0692$) by pure random chance.

Systems Perspective 14.1: Twenty clean runs

Under independent draws from the declared test distribution, twenty consecutive successes establish an 86.1 percent one-sided lower confidence limit, not a 100 percent reliability certificate. A flawless score can occur even for a system whose true failure probability is one in eight; the twenty runs do not estimate that rate.

Napkin Math 14.1: Derivation of the 20-run lower bound

Problem: An autonomous robot completes $n = 20$ physical navigation runs with zero collisions or human interventions ($k = 20$). What is the exact one-sided 95 percent lower confidence bound on its true operational success probability p ?

Mathematical Derivation: For independent, identically distributed Bernoulli trials from the declared operational test distribution, p_{lower} is the success probability at which 20 consecutive successes have probability $\alpha = 0.05$. Thus the one-sided 95 percent lower confidence limit is $p_{\text{lower}} = \alpha^{1/n} = (0.05)^{1/20} \approx 0.861$ (86.1 percent); the corresponding failure-probability upper confidence limit is 13.9 percent.

Systems insight: The observation “20 out of 20 passed” supplies a confidence limit under those sampling assumptions. It does not establish an observed failure rate, mean interval between failures, or deployment permission. Demonstrating a 99.9 percent success lower limit with the same method requires 2,995 consecutive zero-failure trials drawn from the target distribution.

This reality necessitates a reinterpretation of physical benchmarks. An empirical test score is not an intrinsic property of neural network weight tensors. It is an empirical measurement of a coupled cyber-physical system, jointly determined by the compiled policy weights (π_θ), the individual machine chassis and its mechanical wear (m), the operational environment (e), the initial state sampling protocol (s_0), and the formal evaluation predicate (ϕ). Changing the ambient temperature (which heats motor windings and triggers thermal derating),² swapping a tire for one with slightly lower friction, or allowing battery voltage to sag under heavy load shifts the physical state transition function without altering a single weight in the model. Without conditioning on all five elements, a reported success percentage is physically meaningless.

¹ | **Clopper-Pearson non-asymptotic bound:** Inverting the cumulative binomial distribution yields an exact one-sided lower confidence limit ($p_{\text{lower}} = \alpha^{1/n}$ for zero failures across n independent, identically distributed trials at confidence level $1 - \alpha$). Unlike the asymptotic Wald interval, this bound remains informative at zero observed failures. For $n = 20$ and 95 percent confidence, the success lower limit is 86.1 percent, equivalently a 13.9 percent upper confidence limit on failure probability under the tested distribution. Neither number is an observed failure rate.

2995 runs: 99.9%

299 runs: 99.0%

59 runs: 95.0%

20 runs: 86.1%

log scale

Under iid sampling, twenty clean trials give an 86.1 percent one-sided success lower confidence limit; a 99.9 percent limit needs 2,995.

Prerequisite: Physical failure boundaries for autonomous mobile robots and manipulators are established in section 4.8.

² | **Actuator thermal dynamics:** Resistive Joule heating (I^2R) and thermal derating dynamics govern physical actuator limits across repeated test runs, as established in section 4.2 and section 4.5. Sustained high-current execution can elevate winding temperature, increase copper resistance, and trigger thermal derating during later trials; the magnitude depends on the motor and duty cycle. Without inter-trial cooling pauses, thermal accumulation invalidates statistical independence across consecutive runs and induces premature overcurrent trips.



Definition 14.1: Empirical physical safety margin

Empirical physical safety margin is a one-sided lower confidence limit on the probability of meeting a predeclared survival predicate for independent trials sampled from a specified test distribution. It says nothing by itself about untested disturbances or release readiness.

1. **Significance:** Prevents dangerous false confidence derived from small-sample success rates (such as “20 out of 20 successful trials”), enforcing mathematically rigorous lower bounds on closed-loop survival probability before deployment.
2. **Distinction:** Unlike open-loop test loss or asymptotic point estimates (k/n), the empirical safety margin accounts for finite observation counts through conservative exact Clopper-Pearson confidence intervals at a specified confidence level (e.g., $1 - \alpha = 0.95$).
3. **Common pitfall:** Assuming zero observed failures across a limited test set implies the system is certified safe, ignoring that $n = 20$ zero-failure trials yield a 13.9 percent one-sided upper confidence limit on failure probability under binomial assumptions.

In the four-tier systems architecture (figure 14.1), **Closed-Loop Evaluation** closes the learning loop of Part II (“Teaching the Machine”). The twenty clean runs in the opening turn logged experience (chapter 10) and trained weights (chapter 12) into a concrete decision about the physical claim those trials can support. In figure 14.1, evaluation measures the coupled Brain proposals, Nervous System permission decisions, and Body response. It also measures which out-of-distribution proposals the independent permission path detects and rejects, including its missed detections and stopping margin.³

The twenty-run result raises three practical questions: what outcome was sampled, what can each test regime expose, and which hazards remain outside the evidence? We answer them through closed-loop dynamics, evidence regimes, physical trial limits, runtime boundaries, and a full-denominator evaluation record.

14.2 What You Are Actually Measuring

An evaluation benchmark routinely reports a single summary statistic, such as an 85 percent success rate on an obstacle avoidance task. In machine learning, that number is treated as an intrinsic score of the learned model weights, similar to top-one accuracy on static image classification datasets. In a physical machine, that interpretation is invalid. A learned policy has no independent physical performance. What the benchmark measured was the closed-loop execution of an entire physical system across a specific sequence of days and environmental states. Success rate is an empirical property of the coupled system, comprising the policy, mechanical plant, computational hardware, operating environment, and trial protocol.

To evaluate a policy, an engineering team executes a sequence of discrete physical runs. A single trial is fully determined only when every variable that can alter the physical state trajectory is fixed or explicitly sampled. Formally, let a single trial condition be defined by a tuple $c = (s_0, e, m, \pi_\theta, \kappa)$, drawn from an operational testing distribution $\mathcal{D}$. The vector s_0 represents the initial state of the machine, including SE(3) pose, linear and angular velocities, and joint encoder positions. The environment vector e captures external variables such as floor friction, ambient illuminance, obstacle geometry, and ambient air temperature. The machine state m captures physical and hardware properties, including battery state of charge, payload mass, tire wear, mechanical backlash in the drivetrain, and sensor calibration parameters. The computational configuration π_θ encompasses the frozen policy weights θ , inference quantization, scheduling deadlines, and software versions. Finally, the termination protocol $\kappa = (T_{\max}, g, \phi)$ defines the execution time limit $T_{\max}$, the spatial goal region g , and the binary evaluation function ϕ that maps the resulting trajectory to a terminal outcome $Y \in \{0, 1\}$.

The true success probability under this evaluation protocol is the expected value of the binary task outcome averaged over the joint distribution of trial conditions (see Appendix A for the formal integral). In this formulation, $\text{Traj}(c)$ is the continuous physical trajectory produced when the policy π_θ executes within the physical environment and hardware state defined by c . This dependency

³ | **Runtime safety enforcement:** A separately validated permission path can reject proposals that violate measured state, freshness, and actuator constraints. Its detection coverage, worst-case response time, and available stopping clearance must be tested for the implementation; a 1 kHz sampling rate alone cannot guarantee interception before contact.

demonstrates why success probability cannot be treated as a measurement of θ alone. The model weights θ constitute only one parameter inside the condition vector c . If the distribution over floor friction or the distribution over initial battery charge changes, the expected success rate changes while θ remains identical. The sampling protocol is not an external apparatus measuring an isolated model; it is part of the mathematical quantity being estimated.

 Definition 14.2: The closed-loop evaluation estimand

Closed-loop evaluation estimand is the expected physical task outcome $p(\mathcal{D}) = \mathbb{E}_{c \sim \mathcal{D}}[\phi(\text{Traj}(c))]$ jointly defined over policy weights, mechanical plant state, environmental conditions, and trial protocol, rather than an intrinsic property of the learned model weights in isolation.

1. **Significance:** In static machine learning, held-out accuracy is treated as a property of the model π_θ . In Physical AI, substituting a worn tire, letting battery voltage sag, or shifting ambient illuminance alters the physical state transition function without changing a single weight. The sampling distribution of operational conditions $\mathcal{D}$ is an inseparable part of the quantity being estimated.
2. **Distinction:** Unlike open-loop validation metrics (such as validation loss or mean squared trajectory error) which score predictions against frozen recorded states, the closed-loop estimand integrates over the endogenous feedback loop where past actions physically move the machine, directly changing the future sensor observations it will receive.
3. **Common pitfall:** Reporting a single success rate percentage (e.g., “92 percent task success”) without predeclaring the support, density, and environmental bounds of the operational design domain $\mathcal{D}$. Any physical policy can be made arbitrarily successful by testing exclusively in benign, unperturbed states.

The physical mechanisms that alter c between trials operate independently of the policy:

- **Mobile systems:** A lithium-ion battery pack at 90 percent state of charge provides higher terminal voltage to the motor drivers than a depleted pack at 20 percent state of charge. As terminal voltage sags under high load, the reduced voltage headroom limits the current the inverter can push against back-EMF, truncating the available torque at high speeds according to the electromechanical torque-speed curve. If a trajectory demands a peak acceleration of 1.2 m/s^2 to clear a moving obstacle, the robot clears the obstacle when the battery is fully charged, but understeers and collides when the battery is depleted. Similarly, ambient sunlight shifting across an afternoon changes camera exposure times from 2 ms to 20 ms. This tenfold increase introduces motion blur during turning maneuvers, corrupting feature extraction in the visual backbone and causing state estimation drift. Floor dust accumulation lowers the static friction coefficient from $\mu = 0.80$ to $\mu = 0.55$, extending the stopping distance from an initial speed of 2.0 m/s from 0.25 m to 0.37 m.
- **Manipulator systems:** Continuous high-speed pick-and-place cycles generate Joule heating ($I^2 R$) in actuator stator windings and viscous frictional heating in harmonic drive gearboxes, heating lubricant and altering viscous damping coefficients by over 25 percent. As structural linkages warm, thermal expansion shifts the tool-center-point (TCP) SE(3) kinematic calibration by 0.15–0.30 mm. A policy trained to insert a peg into a chamfered hole with 0.20 mm clearance succeeds during early morning cold starts but jams repeatedly in the afternoon as thermal growth exhausts kinematic tolerance margins.
- **Process & energy systems:** Ambient temperature swings alter the thermal gradient across cooling heat exchangers. A neural policy regulating chiller pump speeds that maintains coolant temperature below 45°C in a 20°C laboratory environment fails when ambient air hits 38°C , triggering high-pressure compressor trips despite executing identical control laws.

In all these cases, the policy weights and input tensor shapes are identical, yet the physical trajectory transitions from success to failure.

Non-model variation also enters through the reset procedure and operator handling. When human operators manually reposition a machine between trials, they introduce subtle spatial biases. If an operator instinctively resets the robot facing open space after a difficult run, the empirical distribution of initial heading angles shifts toward favorable states. Conversely, an automated reset procedure that repeatedly drives the machine along a fixed path induces cyclic heating in the actuator windings, increasing electrical resistance and reducing actuator response speed across successive trials. If an operator intervenes to catch a falling machine and discards the run as an unrecorded reset, the hardest failure modes are erased from the sample history, artificially inflating the observed success rate.

Statistical evaluation requires distinguishing ordinary sampling variation from a change in the underlying estimand. When N trials are drawn from a fixed distribution $\mathcal{D}$, the sample proportion $\hat{p} = \frac{1}{N} \sum_{i=1}^N Y_i$ fluctuates around the true parameter $p(\mathcal{D})$ due to random sampling noise. This uncertainty can be bounded using standard statistical estimators. However, if an evaluation changes the payload mass from 0.0 kg to 5.0 kg, or shifts testing from dry concrete to wet asphalt, the experiment is no longer drawing samples from $\mathcal{D}$. It is sampling from an entirely different distribution $\mathcal{D}'$, defined by a new parameter $p(\mathcal{D}')$. Treating data collected under $\mathcal{D}'$ as an extension of the original sample pool conflates sampling variance with structural bias, combining observations from two different physical machines.

Because success probability is defined only with respect to a specific joint distribution $\mathcal{D}$, the intended operating envelope must be declared before trials are sampled. In systems engineering, this envelope is the **operational design domain** (ODD). If an engineering team executes fifty physical runs across ad hoc trajectories and reports a 90 percent pass rate, the denominator corresponds to an undefined population. A measured success rate supports a target-distribution claim only when the support and sampling weights of $\mathcal{D}$ are specified in advance, establishing explicit bounds on illuminance, payload ranges, surface friction coefficients, and allowable initial state offsets.

Extracting valid confidence bounds from empirical trials requires three operational conditions:

1. *Independent and Identically Distributed Sampling*: Trials must be independent realizations drawn from $\mathcal{D}$. If thermal accumulation in motor windings or progressive mechanical wear degrades hardware efficiency across consecutive runs, trial outcomes become negatively correlated over time, violating independence.
2. *Fixed Outcome Predicates*: The binary evaluation function ϕ and the termination criteria κ must remain fixed throughout the campaign. Modifying the time horizon $T_{\max}$ or adjusting the spatial tolerance of the goal region g midway through testing alters the definition of success.
3. *Full-Denominator Accounting*: Every initiated run that aborts due to hardware faults, communication timeouts, or safety-rated emergency stops (e.g., exceeding ISO 10218 / ISO/TS 15066 power and force limits) must be recorded as a failure ($Y = 0$). Dropping aborted trials substitutes a measure of conditional execution on surviving hardware for the true operational reliability of the machine.

When these conditions are strictly enforced, physical trials yield an unbiased sample mean for the closed-loop system under the tested distribution $\mathcal{D}$. Yet physical testing remains constrained by hardware wear, operator labor, and wall-clock execution time. To bypass these constraints, evaluation workflows often turn to open-loop evaluations on recorded datasets, calculating how accurately a candidate policy reproduces logged expert actions offline. That substitution assumes that a model that predicts demonstrated actions with high statistical accuracy will exhibit equivalent competence when granted real-time authority over a physical machine.



Checkpoint 14.1: Condition tuples and operational design domains

Before examining closed-loop dynamics, verify your understanding of experimental condition spaces and trial accounting:

- ☐ **Coupled condition tuple**: Can you explain why an empirical success rate measures the experimental tuple $(s_0, e, m, \pi_\theta, \kappa)$ rather than neural network weights in isolation?

- ❑ **Hardware non-stationarity:** Can you describe how battery voltage sag or thermal expansion shifts physical state transitions while policy weights remain completely frozen?
- ❑ **Full-denominator accounting:** Can you explain why dropping aborted runs or emergency stops inflates empirical reliability metrics?

In section 14.3, we analyze the mathematical and physical mechanisms that break this assumption, proving why open-loop dataset accuracy decouples from closed-loop hardware survival.

14.3 Closed-Loop Dynamics

In an **open-loop evaluation**, a candidate policy π_θ is scored across a static dataset of recorded demonstrations $\mathcal{D}_{\text{demo}} = \{(s_t^*, a_t^*)\}_{t=1}^T$. At each discrete timestep t , the policy receives a recorded demonstrator state s_t^* and outputs a predicted action $\hat{a}_t = \pi_\theta(s_t^*)$. An error metric, such as mean squared error or cross-entropy, measures the distance between $\hat{a}_t$ and the recorded expert action a_t^* . At step $t+1$, the evaluation pipeline supplies the next recorded state s_{t+1}^* from the dataset. The action $\hat{a}_t$ produced by the policy is discarded without executing on hardware, keeping the sequence of input states independent of the policy's past outputs. In a **closed-loop evaluation**, the policy proposes $a_t = \pi_\theta(s_t)$; an independent permission tier checks it against measured state and constraints, applies u_t (or a fallback), and the plant evolves as $s_{t+1} = f(s_t, u_t)$. The applied command changes the physical state and later observations, crossing the causal boundary in section 2.2.

Open-loop evaluation tests actions along the demonstrator's state trajectory; a policy with low offline prediction error can fail when its biased proposals are admitted to physical motion (Ross et al. 2011; Pomerleau 1989). Consider an analytical example of a planar moving platform tracking a straight corridor of length $L = 10$ m at a constant forward speed of $v = 1.0$ m/s. The nominal expert demonstration follows the centerline with heading $\theta^* = 0$ rad and lateral offset $y^* = 0$ m for a duration of 10 s. The policy runs at a control frequency of 10 Hz, executing updates every $\Delta t = 0.1$ s across 100 discrete steps. Suppose the candidate policy has a systematic heading bias of $\epsilon_\theta = 1.0^\circ$ (0.01745 rad) on every step and the illustrative permission tier admits these individually feasible proposals.

In an open-loop evaluation, the policy receives the demonstrator state $s_t^* = (x_t^*, 0, 0)$ at each step and predicts a forward velocity with a 1.0° steering offset. The offline scoring pipeline records a mean absolute angular error of 1.0° (0.01745 rad) and an extremely low mean squared error of $\text{MSE}_\theta = (0.01745)^2 \approx 3.05 \times 10^{-4} \text{ rad}^2$.⁴ A small angular mean squared error on recorded centerline states does not measure off-center recovery or justify deployment. In closed-loop execution, however, because the machine physically moves based on previously admitted commands, uncorrected heading errors compound continuously over time. Under planar unicycle kinematics ($\dot{y} = v \sin \theta \approx v \epsilon_\theta$), this persistent angular offset integrates into a cumulative lateral displacement:

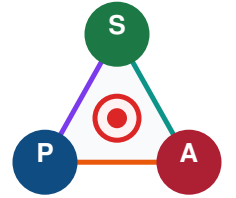
$$y(t) = \int_0^t v \sin \epsilon_\theta d\tau \approx v \epsilon_\theta t$$

Over the 10 s transit (10 m forward distance at $v = 1.0$ m/s), accumulated lateral drift reaches $y(10) = 1.0 \text{ m/s} \times 0.01745 \text{ rad} \times 10 \text{ s} \approx 0.175 \text{ m}$ (17.5 cm).⁵ If the corridor provides $W_{\text{clear}} = 0.15$ m (15.0 cm) of lateral clearance to the side wall, the clearance envelope is exhausted at timestamp:

$$t_{\text{impact}} = \frac{W_{\text{clear}}}{v \sin \epsilon_\theta} \approx \frac{0.15 \text{ m}}{1.0 \text{ m/s} \times 0.01745 \text{ rad}} \approx 8.6 \text{ s} \quad (x = 8.6 \text{ m})$$

Without an additional clearance barrier, the vehicle strikes the wall before completing the transit. This deterministic trial fails despite the small open-loop angular error.

Prerequisite: Irreversible physical effects after motor actuation are established in section 2.2.



The closed-loop evaluation core: empirical survival testing across coupled Sense, Plan, and Act dynamics.

4 | **Metric scale masking in regression loss:** Aggregating squared steering errors across thousands of validation frames obscures severe, intermittent directional mistakes. A single localized prediction spike of 15° is mathematically washed out when averaged across miles of straight highway data. A brief angular spike can matter if its integrated motion exhausts clearance before the independent permission path responds; the outcome requires a plant and timing model.

5 | **Kinematic integration of heading bias:** This example holds heading at a constant 1° offset, so $y(t) = v \sin(\epsilon_\theta)t$ is linear. A different example with constant yaw-rate bias ω begins with $\theta(t) = \omega t$ and has the small-angle approximation $y(t) \approx \frac{1}{2} v \omega t^2$. Neither relationship predicts all steering-controller errors.

🔍 Systems Perspective 14.2: Open-loop blindness

Open-loop loss measures dataset mimicry; closed-loop dynamics decide physical survival. A small offline prediction error can still produce a large physical excursion if a bias persists and the permission path admits it.

The **offline-online evaluation disconnect** is the fundamental decoupling of open-loop dataset loss metrics (such as action mean squared error or imitation log-likelihood on static logs) from closed-loop physical stability and task survival, caused by the inability of static evaluation to reflect compounding kinematic integration errors ($y(t) \approx \int_0^t v \sin \epsilon_\theta d\tau$) and self-induced distribution shift during embodied execution (the endogenous cascade where early control errors steer the machine into unfamiliar physical states, provoking degraded control responses that compound trajectory divergence).

This divergence occurs because open-loop evaluation resets the machine to an expert state at every clock cycle, masking the compounding nature of integration errors. In closed loop, the state at index t is the expanded composition of all previous actions:

$$s_t = f(f(\dots f(s_0, u_0) \dots, u_{t-2}), u_{t-1})$$

The first admitted action error shifts the physical machine from the demonstrated state s_1^* to an unvisited state s_1 . Once the platform drifts 2.0 cm off nominal, it enters a state that does not appear anywhere in $\mathcal{D}_{\text{demo}}$. Because the policy was evaluated only on demonstrator states where $y = 0$ m, the offline metric contains no evidence regarding what action $\pi_\theta(s)$ the policy will produce when $y \neq 0$ m. If the training distribution lacked demonstrations of recovery maneuvers from off-center coordinates, the policy can output larger corrective errors, accelerating the divergence instead of arresting it.

This disconnect manifests across all physical machine archetypes:

- *Mobile Systems:* As derived above, a 1.0° angular error integrates across 10 m into a 17.5 cm lateral offset, breaching corridor clearance limits.
- *Manipulator Systems:* An end-effector position error of 0.5 mm per step during a contact assembly task (e.g., chamfered pin insertion) appears negligible on offline Cartesian MSE. In closed-loop contact, that same 0.5 mm lateral displacement acts as a physical wedge, driving the pin against the hard metal edge of the hole (the chamfer rim). This generates immense lateral reaction forces ($F_{xy} = k_{\text{env}} \Delta_{xy}$) that overload the joint actuators and trip the robot's **ISO 10218 protective stop** for exceeding maximum torque limits ($\tau_i > \tau_{\text{max}}$).
- *Process & Energy Systems:* A temperature prediction error of 0.5°C per minute in an industrial reactor cooling policy seems minor on offline logs. In closed loop, failing to appropriately increase the coolant fluid's mass flow rate ($\dot{m}$) chokes off heat extraction ($Q = \dot{m} c_p \Delta T$). Because the chemical reaction generates its own heat (exothermic), this error feeds on itself, compounding thermal accumulation until the reactor core breaches the safe 95°C hardware trip threshold and triggers a shutdown within twenty minutes.

The mapping between offline scores and physical task performance is not a simple mirror image; it can be deeply misleading in both directions. While open-loop evaluation is optimistically blind to compounding trajectory divergence, its action metrics can conversely be pessimistic when the physical environment permits multiple valid actions. If a human demonstrator avoided an obstacle by steering left, an open-loop evaluation penalizes a candidate policy that steers right, recording a large action error against a_t^* . In closed-loop execution, passing on the right clears the obstacle and reaches the target state with equal validity. An action-level prediction error measures divergence from a specific demonstration recording, not the satisfaction of the physical task.

Figure 14.2 separates the two protocols. Open-loop evaluation scores candidate actions only on demonstrator states where errors are immediately discarded, obscuring the cumulative trajectory divergence that drives closed-loop physical execution into catastrophic wall collisions.

In physical reality, uncorrected steering biases do not require gross software bugs; they arise naturally from unmodeled hardware asymmetries such as differential tire wear, subtle camera mast

Contrast: Open-loop prediction loss contrasts with the photon-to-actuation age-of-information bounds analyzed in section 16.4.

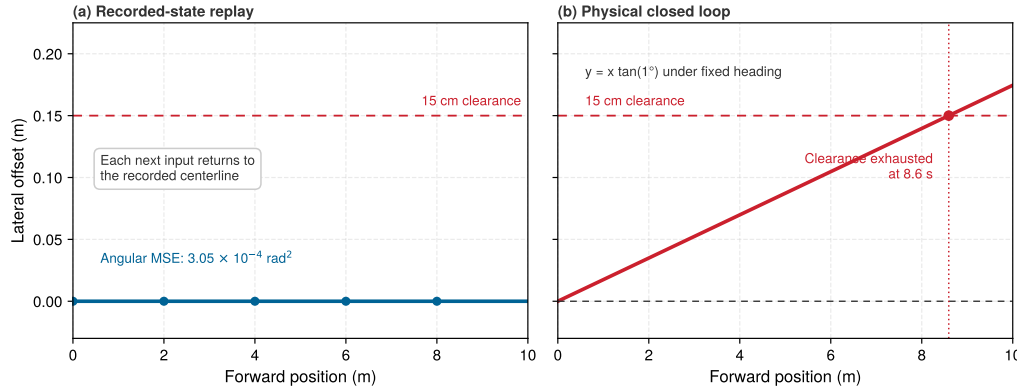


Figure 14.2: Open versus closed loop divergence: A stipulated persistent 1.0° heading offset gives $3.05 \times 10^{-4} \text{ rad}^2$ angular mean squared error on recorded centerline states. In the physical corridor example, admitted motion with that fixed heading drifts linearly and reaches the 15 cm side clearance at 8.6 s. The example makes no claim about recovery by a different controller.

vibration, or floor camber. As demonstrated by the unicycle derivation above and illustrated in figure 14.2, a stipulated persistent bias of $\epsilon_\theta = 1.0^\circ$ (0.0175 rad) has a small open-loop angular error ($\text{MSE}_\theta \approx 3.05 \times 10^{-4} \text{ rad}^2$), yet in this corridor example exhausts the 15.0 cm clearance envelope at $t_{\text{impact}} \approx 8.6 \text{ s}$, collapsing closed-loop task success to zero. This decoupling shows why open-loop error alone cannot establish physical survival. Closed-loop evaluation adds path excursion, intervention timing, and the measured performance of an independent permission path; a control barrier check must be validated against sensing error and stopping margin.

14.3.1 Why learned policies break analytical stability proofs

For control engineers steeped in classical dynamical systems theory, relying on statistical trial counting in section 14.1 feels incomplete. In classical control theory, safety and stability are not certified by counting successful runs. Instead, control theory constructs analytical **Lyapunov functions** $V(\mathbf{x}) > 0$ that satisfy $\dot{V}(\mathbf{x}) \leq -\alpha V(\mathbf{x})$, establishing exponential decay for the specified disturbance-free closed-loop dynamics on the region where the inequality holds. With disturbances, a separate Input-to-State Stability (ISS) argument can bound state error as a function of a bounded input, under its own model and gain assumptions.

Why can systems engineers not simply apply this analytical machinery directly to deep learned policies $\pi_\theta(\mathbf{x})$?

Four physical and mathematical barriers complicate broad classical Lyapunov proofs for deep neural policies:

1. **Non-Smooth Activation Functions and Intractable Optimization:** Modern neural networks use piecewise-linear or non-smooth activations (ReLU, GELU). Evaluating whether $\dot{V}(\mathbf{x}) = \nabla V(\mathbf{x})^\top f(\mathbf{x}, \pi_\theta(\mathbf{x})) \leq 0$ holds across an entire continuous state space requires solving a high-dimensional non-convex optimization problem. For neural networks with millions or billions of parameters, searching for polynomial storage functions via Sum-of-Squares (SOS) programming is NP-hard and computationally intractable beyond toy systems of a few dozen neurons.
2. **High-Dimensional Latent Representations:** Classical control assumes state vectors $\mathbf{x} \in \mathbb{R}^n$ composed of measurable physical quantities (joint angles, velocities, pressures). Deep Vision-Language-Action policies consume high-dimensional pixel arrays ($> 10^6$ dimensions) and map them into uncalibrated latent token spaces $\mathbf{z}_t \in \mathbb{R}^d$. A physically interpretable Lyapunov function is difficult to formulate directly over an unobservable latent manifold whose metric distances have no physical SI units or energy conservation laws.
3. **Discontinuous Contact Mechanics:** Physical manipulation and legged locomotion involve making and breaking contact with the physical world. Mechanical contact transitions introduce non-smooth complementarity constraints and instantaneous velocity jumps ($v^+ \neq v^-$).

Differentiating a Lyapunov candidate across non-Lipschitz, discontinuous contact manifolds coupled with non-convex neural function approximators destroys the smoothness assumptions that underpin LaSalle’s invariance principle.

4. **Out-of-Distribution (OOD) Fragility:** A linear controller has analyzable behavior within the plant and disturbance bounds used in its design; outside those bounds it can also fail. A neural policy can additionally exhibit abrupt output changes under sensory shift: a subtle shift in sensory input (such as an unexpected optical shadow or camera glint) can cause policy outputs to jump discontinuously, proposing hazardous torque in the wrong direction; the independent permission path must assess the proposal before actuation.

General stability proofs for an entire high-dimensional learned policy and physical plant are difficult; thus, physical AI systems engineers must adopt a rigorous two-pronged methodology: First, we deploy **non-asymptotic statistical metrology** (exact Clopper-Pearson confidence bounds, Butler & Finelli exposure testing, and Extreme Value Theory) to characterize empirical failure risks across the operational design domain. Second, we enforce hard real-time safety through **deterministic runtime safety shields** (such as Control Barrier Functions in chapter 24), moving the burden of mathematical proof from the multi-billion-parameter neural brain to an isolated, low-dimensional microcontroller enforcer.



Checkpoint 14.2: Kinematic bias compounding versus open-loop error

Before evaluating alternative evidence regimes, verify your understanding of open-loop error masking and kinematic integration:

- ☐ **Open-loop evaluation masking:** Can you explain why low angular mean squared error on recorded centerline states does not establish off-center recovery?
- ☐ **Kinematic bias compounding:** Can you describe how a sub-degree steering bias integrates through continuous unicycle kinematics to cause catastrophic boundary departure?
- ☐ **Closed-loop evaluation metrics:** Can you distinguish offline Mean Squared Error from Time-to-Intervention (T_{TI}) and control barrier function monitoring during physical rollouts?

This disconnect between open-loop statistical agreement and physical closed-loop survival is systematized in table 14.1, mapping common machine learning validation metrics to their underlying physical failure modes and closed-loop evaluators.

Table 14.1: Offline open-loop prediction metrics versus closed-loop physical survival disconnect matrix: Comparing standard machine learning benchmark metrics evaluated on static demonstration logs against the physical dynamics of embodied execution. Open-loop metrics implicitly assume that demonstrator state distributions remain invariant to policy errors; in closed loop, unmodeled actuator biases integrate into compounding spatial drift, multimodal action alternatives incur spurious penalties, and distribution shifts trigger unrecoverable boundary violations.

Offline Evaluation Metric	Open-Loop Statistical Assumption	Physical Disconnect Mechanism	Closed-Loop Hardware Failure Mode	Mandatory Closed-Loop Evaluator
Action Mean Squared Error	Small prediction residuals produce proportionally small trajectory deviations.	Unmodeled kinematic bias integrates over time, driving states out of distribution.	Catastrophic boundary collision, curb strike, or lane departure.	Closed-Loop Rollout Tracking Error: Maximum spatial deviation from reference path and Time-to-Intervention (T_{TI}) .
Top-1 / Discrete Action Accuracy	High classification accuracy on logged states ensures identical execution.	Equal penalty for benign alternative maneuvers vs catastrophic boundary excursions; blind to temporal error correlation.	High-frequency actuator chatter, rapid control sign-flipping, or sudden structural joint over-torque.	Smoothness & Dynamic Feasibility: Peak jerk and adherence to actuator torque limits.

Offline Evaluation Metric	Open-Loop Statistical Assumption	Physical Disconnect Mechanism	Closed-Loop Hardware Failure Mode	Mandatory Closed-Loop Evaluator
Cosine Similarity / Directional Agreement	Collinear action proposals produce dynamically stable physical paths.	Metric ignores control magnitude errors; under-actuation fails to overcome static friction or over-actuation triggers tire slip.	Limit-cycle hunting, velocity stall on inclines, or spin-out during dynamic cornering maneuvers.	Phase Margin & Slip Ratio: Effective damping ratio and wheel slip boundaries (see Appendix B).
Negative Log-Likelihood (NLL)	High likelihood on expert demonstrations captures the true physical action distribution.	Density is evaluated strictly at the logged expert action, heavily penalizing valid multimodal bypass trajectories (e.g., navigating right when the expert went left).	Spurious rejection of robust multimodal policies; deployed models overfit to specific demonstrator noise, inducing high-frequency actuator jitter.	Closed-Loop Task Success Rate: Terminal goal completion over randomized goal offsets.
Sensory Reconstruction / Chamfer Loss	Accurate world-model state prediction ensures safe receding-horizon plan synthesis.	Visual/feature reconstruction loss weights background pixels over sub-pixel spatial obstacles and contact manifolds.	Phantom obstacle braking, clipping thin obstacles (cables, glass), or false clearance estimation.	Spatial Clearance & CBF Invariance: Strict maintenance of minimum obstacle distance and control barrier safety conditions.

Recorded replay remains widely used because it evaluates model checkpoints across stored arrays in seconds, consuming compute cycles without occupying physical test cells or incurring mechanical wear. Closed-loop simulation restores a modeled proposal-permission recurrence $s_{t+1} = f_{\text{sim}}(s_t, u_t)$, allowing compounding drift to manifest and testing whether a policy can stabilize off-nominal states without risking physical hardware. Yet simulation replaces the open-loop distribution assumption with a dynamical modeling assumption. The simulation transition operator f_{sim} approximates real physics f_{real} , but it abstracts away unmodeled friction non-linearities, contact mechanics, and physical transport latencies. Establishing that a policy stabilizes f_{sim} demonstrates mathematical convergence inside the simulator, but it does not establish how the policy behaves when executed against physical hardware.

Extending the proposal-permission architecture from section 6.1, this open-loop versus closed-loop disconnect maps directly across the compute boundary. When the Linux MPU runs offline trace replay, it measures only whether neural inference executes within its memory and compute budgets. In physical closed-loop execution, the Linux MPU continuously dispatches candidate action proposals via dedicated memory mailboxes (RPMMSG) to the bare-metal MCU. If the policy's proposals drift off-target or vibrate erratically (chatter), this real-time MCU enforcer detects the spatial boundary breach or a missed timing deadline (lease expiration), forcefully rejecting the bad proposals and asserting an emergency deceleration. Evaluating a policy offline without the physical plant and the real-time permission referee in the loop evaluates tensor mathematics, not cyber-physical machine survival.

To navigate the trade-offs between static trace replay and unconstrained hardware testing, section 14.4 classifies testing environments into four formal evidence regimes, bounding what each environment can and cannot prove about physical autonomy.

14.4 Evidence Regimes

Evaluating a learned policy requires selecting an execution environment, divided into four evidence regimes: recorded replay, closed-loop simulation, shadow operation, and constrained physical testing, as shown in figure 14.3 and table 14.2. Each regime establishes a different relationship between the policy, its sensory inputs, and the state of the machine. Consequently, each regime supports a strictly bounded claim. Recorded replay evaluates numerical prediction error over static data. Simulation evaluates closed-loop stability against a synthetic transition operator (building on the causal boundary established in section 2.2). Shadow operation evaluates onboard execution timing

Downstream: Deterministic gatekeeping and stopping envelopes for unverified neural policies are derived in section 24.5.

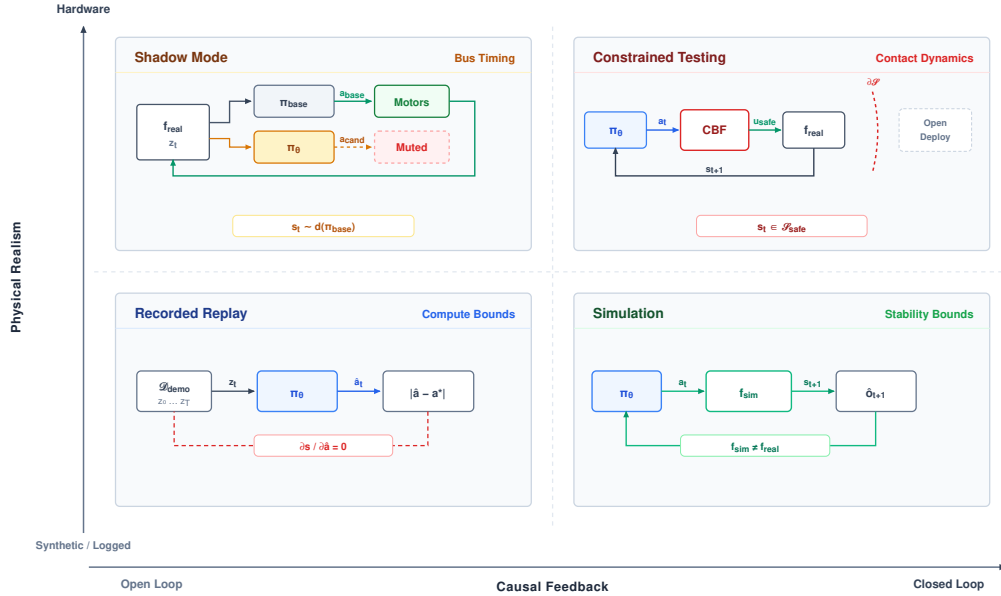


Figure 14.3: Evaluation evidence regimes across the causal-physical manifold: Testing environments span a two-dimensional trade-off space parameterized by causal feedback (open-loop versus closed-loop) and physical realism (synthetic/logged versus live hardware). Recorded replay (bottom left) evaluates computational throughput on static demonstration buffers $\mathcal{D}_{\text{demo}}$, but cannot observe compounding drift ($\partial s_{t+1}/\partial \hat{a}_t = 0$). Closed-loop simulation (bottom right) restores dynamic state transitions via synthetic physics f_{sim} , testing behavior under modeled dynamics while leaving the unmodeled reality gap unresolved. Shadow operation (top left) subjects candidate inference to real sensor jitter and bus timing, but locks the physical trajectory to the active baseline controller ($s_t \sim d^{\pi_{\text{base}}}$). Constrained physical testing (top right) closes the embodied control loop on real hardware f_{real} , but truncates tail failure dynamics at the safety supervisor boundary ($\partial \mathcal{S}$). Open-world deployment beyond the safety boundary remains untestable in any single isolated regime.

and behavioral divergence against an active baseline. Constrained physical testing evaluates closed-loop dynamics within an artificially restricted operational envelope. None of these **four evidence regimes** establishes how the unassisted policy behaves during unconstrained physical deployment, because each regime alters either the causal feedback of the control loop or the distribution of states the machine encounters.

In recorded replay, the policy executes as a feedforward computation on stored sensor buffers. The environment does not respond to the policy outputs, and the state sequence remains fixed to the trajectory recorded during data collection. This regime supports claims regarding computational throughput, memory allocation (Williams et al. 2009), and the statistical loss of the model relative to a logged dataset. It establishes whether the policy executes within a target latency budget, such as 18 ms on an onboard accelerator, and whether its numerical outputs match the recorded actions. It cannot support any claim about closed-loop stability, because the difference between the predicted action and the logged action never alters the subsequent state observation. If the policy erroneously outputs a hard left turn, the next pre-recorded camera frame will still show the machine driving straight.

Closed-loop simulation restores the causal feedback loop by substituting an explicit mathematical model, f_{sim} , for the physical machine and its surroundings. When the policy proposes an action and the modeled permission path selects u_t , the simulation integrates the equations of motion, updates the scene, and renders synthetic sensor inputs for the next cycle at index $t + 1$. This regime supports claims about the policy’s algorithmic behavior under the modeled dynamics. An engineer can run thousands of parallel simulator instances to test whether a mobile base recovers when initialized 0.5 m off its nominal path, or that an articulated arm avoids self-collision across its workspace. However, this evidence holds only with respect to f_{sim} . Unmodeled mechanical backlash, tire slip non-linearities, contact friction transitions, and lighting variations are excluded by construction.

Shadow operation mounts the candidate policy onto an active physical machine while a validated baseline controller, or a human operator, retains authority over the actuators. The candidate policy receives live sensor frames, runs onboard inference under real mechanical vibration and bus traffic, and computes candidate actions a_t^{cand} .⁶ These candidate actions are logged but never dispatched to the motor drives. Shadow operation supports claims about physical execution timing, validating whether the inference pipeline sustains a 50 Hz update rate under operational thermal conditions, and quantifying how often a_t^{cand} diverges from the baseline action a_t^{base} by more than a specified threshold, such as a steering difference of 0.1 rad. Yet shadow operation shares the fundamental limitation of recorded replay. Because the baseline controller steers the platform, the platform remains on the baseline's trajectory. If the candidate policy outputs an action that would have driven the wheels off a curb, the physical platform does not leave the road, and the sensor stream never reflects the candidate's impending failure.

Constrained physical testing closes the causal loop on real hardware by allowing candidate proposals through an independent permission tier within a bounded envelope. Boundaries take the form of physical tethers, velocity clamps, padded test cells, or an automated safety supervisor at a validated response rate in the nervous system (as architected in section 6.1) that vetoes unsafe commands. This regime supports claims about physical contact dynamics, actuator saturation, and sensor noise within the tested envelope. It establishes whether the machine can track a trajectory at 1.5 m/s on concrete without breaking traction. What it cannot establish is the tail behavior of the policy in open deployment. Every time a safety tether tensions or a supervisory controller intervenes to arrest a 2.0 m/s skid, the evaluation truncates the state trajectory. The observed data measures the performance of the combined policy-plus-supervisor system within the constraint boundary, not the unassisted policy in an unconstrained environment.

Distinguishing between these regimes requires tracking what properties transfer across regime boundaries and what properties stay behind. Numerical outputs for identical input tensors can transfer, but measured timing and memory pressure must be retested under each regime's bus, thermal, and scheduling load. Algorithmic invariants also transfer. A numerical overflow that causes an output to saturate in simulation will manifest identically on physical hardware given the same input tensor. What does not transfer across regimes is the state distribution. The moment the environment model changes from f_{sim} to f_{real} , or the tested permission path changes, the sequence of states $s_0, s_1, \dots, s_t$ diverges.

Viewing these regimes as a ladder of fidelity creates an engineering trap. Teams frequently assume that advancing from replay to simulation, from simulation to shadow mode, and from shadow mode to constrained testing progressively strengthens a single unified confidence claim. In reality, each transition trades one experimental distortion for another. Simulation trades real sensor noise and physical contact for closed-loop causality. Shadow mode trades closed-loop causality back for real sensor streams and real hardware execution. Constrained testing restores causality and physical reality, but introduces state censorship through safety boundaries—artificially preventing the machine from experiencing the true, potentially catastrophic consequences of its own mistakes. A policy that achieves zero interventions across one hundred trials in a constrained test cell has not established a low failure rate outside the tested envelope. It has established that its trajectory deviations remained within the envelope where the constraint mechanisms did not engage.

Rigorous evaluation requires matching each engineering question to the specific regime capable of answering it, rather than defaulting to the most expensive apparatus available. Questions about inference latency, memory footprint, and numerical consistency belong in recorded replay or on target hardware in shadow mode. Questions about kinematic envelope coverage, recovery from synthetic disturbances, and multi-agent coordination belong in simulation, where millions of boundary conditions can be tested without hardware wear. Questions about electrical noise, bus jitter, and mechanical vibration belong in shadow operation. Questions about traction, contact compliance, and actuator thermal rise under load belong in constrained physical trials. Each regime provides valid evidence for its own domain, while remaining blind to the failure modes of the others.

⁶ | **Hardware-in-the-loop bus contention:** While neural accelerator inference latency may exhibit low variance in isolation, shared PCIe backplanes, direct memory access (DMA) transfers, and CAN bus arbitration introduce variable transport jitter under production workloads. A nominal inference cycle can stretch under memory bus saturation, depleting the phase margin of high-rate control loops. In shadow mode, this latency jitter is logged across real bus traffic without exposing the physical chassis to the resulting trajectory oscillations.

Table 14.2: Systematic comparison of evaluation evidence regimes: Trade-offs between causal feedback coupling, environmental realism, supported empirical claims, and experimental boundary distortions across the four testing regimes. No single environment evaluates unconstrained wild deployment; rigorous certification requires combining orthogonal regimes rather than assuming a linear fidelity ladder.

Evaluation Evidence Regime	Causal Feedback Coupling	Environmental Dynamics Grounding	Supported Empirical Evidence	Masked Failure Mode / Boundary Distortion
Recorded Replay	Open Loop ($\partial s_{t+1} / \partial \hat{a}_t = 0$)	Static Demonstration Buffers $\mathcal{D}_{\text{demo}}$	Latency budget, DRAM footprint, numerical accuracy	Covariate shift; errors never alter subsequent observations
Closed-Loop Simulation	Closed Loop ($s_{t+1} = f_{\text{sim}}(s_t, u_t)$)	Synthetic Physics Operator f_{sim}	Kinematic reachability, massive parallel stress suites	Sim-to-real gap; unmodeled contact non-linearities and tire slip
Shadow Mode Operation	Open Loop for candidate (actions muted)	Live Physical Iron f_{real}	Bus contention, thermal throttling, real sensor noise	Trajectory confinement; states locked to baseline controller $d^{\pi_{\text{base}}}$
Constrained Physical Testing	Closed Loop with runtime veto (u_{safe})	Live Physical Iron f_{real} within envelope	Contact dynamics, compliance, friction, actuator limits	State censorship; safety supervisor and tethers truncate failure tails



Checkpoint 14.3: Trade-offs across evidence regimes

Before evaluating physical trial limits, verify your understanding of evidence regimes and fidelity trade-offs:

- ☐ **Regime causality trade-offs:** Can you contrast the causal feedback properties of recorded trace replay against closed-loop simulation?
- ☐ **Shadow-mode state censorship:** Can you explain why shadow-mode evaluation on an active vehicle fails to test candidate policy recovery maneuvers?
- ☐ **Constrained testbed validity:** Can you describe what property transfers reliably across regimes and what property stays behind?

While selecting an appropriate evidence regime clarifies what each test can prove, executing closed-loop trials on physical hardware introduces severe physical, temporal, and wear constraints. In section 14.5, we confront the mathematics of finite trial budgets, extreme value tail risks, and the Butler & Finelli exposure wall.

14.5 Physical Trial Limits

A physical trial consumes more than the simulated step or log-replay time. It consumes wall-clock time, stresses mechanical linkages, heats motor windings, and risks physical damage to the hardware. Consider an analytical example of a mobile robot evaluating a trajectory policy over a 50 m course at a nominal velocity of 1.0 m/s. The active run occupies 50 s. Budgeting the trial by active run time ignores the physical mechanics of the test cell:

- *Active Navigation Transit:* 50 m / 1.0 m/s = 50 s.
- *Return & Starting Pose Reset:* Autonomous or teleoperated return to initial dock waypoint consumes 120 s.
- *Telemetry Flush & Sensor Re-Zeroing:* Committing high-rate ring buffers to NVMe storage and re-calibrating IMU biases consumes 30 s.
- *Actuator Thermal Rest Window:* Building on the actuator thermal limits established in section 4.5, dissipating Joule heating ($I^2 R$) to respect the 80°C insulation ceiling requires 60 s.

The true wall-clock cost of one 50 s physical trial is approximately 4.33 minutes.⁷ Completing one hundred runs requires over 7.2 hours of continuous test cell occupancy and 14.4 person-hours from two safety operators monitoring manual emergency stops.

⁷ | **Physical trial cycle budget:** The total duration of a physical trial includes transit, automated docking, telemetry serialization, and thermal rest windows: $T_{\text{trial}} = 50 \text{ s} + 120 \text{ s} + 30 \text{ s} + 60 \text{ s} = 260 \text{ s} \approx 4.33 \text{ minutes}$. In multi-trial qualification campaigns, the overhead of re-zeroing sensors and cooling actuator windings accounts for over 80 percent of total test cell occupancy. Neglecting these reset phases produces unrealistic testing timelines and tempts teams to truncate cooldown intervals, causing thermal non-stationarity across runs.

The physical plant changes while these trials accumulate. Statistical evaluation assumes that repeated trials are independent and identically distributed draws from a stationary data-generating process. On physical machinery, mechanical wear can violate stationarity. Repeated runs can change gearhead backlash, wheel radius, and tire friction; the rate and magnitude require measurement on the test hardware. Structural joints loosen, damping characteristics change, and motor commutator brushes abrade. If monitored hardware parameters shift during a long campaign, the policy may encounter a different plant than in the first trial. Pooling results across one thousand trials treats a nonstationary progression of degrading hardware states as if it were a repeated test of a single invariant machine. Early successes do not guarantee that the worn machine can execute the same trajectory, and late failures often reflect physical degradation rather than an algorithmic deficiency.

This physical degradation is compounded by the problem of state censorship—a phenomenon where critical failure data remains hidden because safety mechanisms intervene before the machine can fully crash. The operational states that an engineering team most needs to evaluate are those at the stability boundary of the vehicle, including sudden loss of traction, high-speed collision avoidance, and sensor blinding. Yet these are precisely the states that operators cannot permit the physical machine to explore freely. Entering an unrecoverable skid or colliding with an obstacle damages the chassis, destroys external sensing rigs, and takes the test facility offline for repairs. Consequently, physical testing relies on safety cages, geofences, and human operators with physical emergency stops who intervene whenever a trajectory drifts outside an arbitrary conservative envelope. These protective measures keep the hardware intact, but they censor the empirical evidence. The trial records an external intervention, but the true downstream consequence of the policy output remains unobserved. Constrained physical trials deliberately truncate some severe outcomes; their intervention counts must remain in the denominator while unobserved downstream consequences remain unknown.⁸

For physical survival, a mean is an inadequate description of a peak. A short impact may exceed a structural limit even if average contact force is modest; a long bus delay may consume stopping margin even if mean latency is small. Extreme value theory (EVT) offers candidate statistical models for maxima, but a particular Gumbel, Fréchet, or Weibull family must be fitted to a defined sample of block maxima or threshold exceedances, checked against alternatives, and accompanied by uncertainty. The physical mechanism alone does not determine the tail family. Without those data, the following table states what to measure and what protection to validate rather than predicting rare-event frequencies.

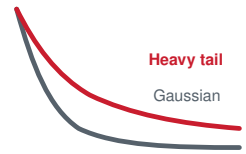
Table 14.3: Peak-disturbance evidence and protection: Mechanisms identify measurements and safeguards, not an EVT family or event rate. Any quantitative tail claim needs a defined population, fitted model, goodness-of-fit checks, and uncertainty.

Disturbance channel	Peak quantity and sampling record	Physical consequence to test	Candidate protection to validate
Compute and bus latency	Per-cycle end-to-end age and deadline misses	Stale proposals erode braking margin	Freshness lease and independently timed fallback
Contact shock	Force-time trace, impulse, and peak at the load path	Yield or overload before a reactive command takes effect	Passive compliance and precontact speed limit; reactive limits address later load
Low traction	Local friction estimate and stopping response	Slip extends stopping distance	Conservative speed ceiling and tested fallback on low-grip surfaces
Optical glare	Saturation duration and perception misses	False clearance or lost state estimate	Redundant sensing and independent physical constraints
Gusts	Measured disturbance and remaining control authority	Attitude or path excursion	Tested thrust reserve and emergency recovery

The mechanisms in table 14.3 frame the evidence needed for each disturbance. Consider an **illustrative** bipedal ankle with a stipulated 1800 N shear-pin ceiling. Suppose a captured rigid-contact trace peaks at 2350 N after 1.5 ms, while a separately measured compliant-contact trace peaks at 920 N over 22 ms. These numbers describe two assumed test traces, not a measured population or a fitted exceedance rate. Passive compliance can reduce the initial peak by lengthening

⁸ | **Extreme value disturbance modeling:** Block maxima or threshold exceedances can be modeled with extreme value methods when the sampling process, threshold, fit, and uncertainty are reported. The Fisher–Tippett–Gnedenko theorem describes possible limiting forms under assumptions; it does not assign a tail family to impact, glare, latency, or wind by physical mechanism alone.

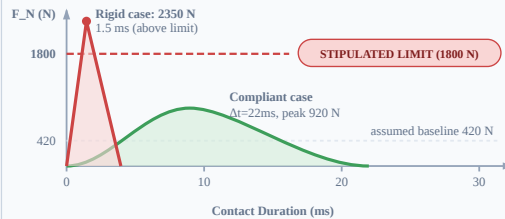
deceleration; its response must be established from impulse, geometry, and material tests. A force monitor that reacts only after contact cannot undo the first peak. Its measured end-to-end detection and actuator response may limit subsequent loading, while a precontact speed governor reserves the initial impact margin.



Illustrative tail-model comparison; field data and uncertainty are needed before selecting either curve.

Checkpoint 14.4: Contact peaks and response timing

ILLUSTRATIVE CONTACT TRACES



FIRST-PEAK VS LATER-LOAD DEFENSE

PRECONTACT / PASSIVE

- Speed cap and tested compliance
- Assumed peak: 2.35 ~ 0.92 kN

POST-CONTACT MONITOR

- Measure force and response delay
- **May limit later loading**
- Cannot undo first force peak

! No event rate without fitted data

- ☐ **Peak evidence:** What measured force traces and population would be needed before fitting an impact exceedance rate?
- ☐ **First-impact protection:** Why can passive compliance and precontact speed limits affect the initial peak while a post-contact monitor cannot undo it?
- ☐ **Reactive response:** Which measured detection and actuator delays determine whether the monitor can limit later load?

Attempting to bypass these physical bottlenecks by distributing trials across a fleet of identical machines introduces spatial and environmental correlation. When ten mobile robots run concurrently across different bays of a testing warehouse, their operational histories are not independent. The machines share identical manufacturing tolerances from the same production batch, run on identical concrete floor finishes, share ambient temperature profiles governed by a single building ventilation system, and interact with human operators trained under identical protocols. If an environmental anomaly such as floor glazing or ambient lighting glare is absent from the facility, none of the ten machines will experience it. As illustrated by the multi-bay robotic qualification test facility in figure 14.4, standardized physical test fixtures allow teams to isolate specific locomotion or perception capabilities under repeatable physical geometry, but they also bound the state space to that precise fixture envelope. Accumulating ten thousand fleet hours across ten machines in one facility yields ten parallel observations of a narrow environmental distribution, not ten thousand hours of independent operational diversity. Treating aggregated fleet hours as a direct substitute for independent statistical exposure obscures the common-mode biases that dominate physical operations.

The mismatch between physical trial budgets and rare-event claims makes the choice of interval method consequential. In standard machine learning benchmarks, practitioners frequently compute confidence intervals using the asymptotic Wald normal approximation (for derivations, see Appendix A). When testing high-reliability physical systems where zero failures are observed across n runs ($\hat{p} = 1.0$), the estimated variance collapses to zero ($\hat{p}(1 - \hat{p}) = 0$), yielding the absurd and degenerate confidence interval $[1.0, 1.0]$. At this boundary, the plug-in Wald calculation is invalid and hides sampling uncertainty.

For this zero-failure worked example, we choose the exact one-sided Clopper-Pearson method (Clopper and Pearson 1934). Other interval methods may also be valid when their assumptions and coverage are stated. Inverting this binomial limit gives the minimum iid zero-failure trial count needed for a specified success lower confidence limit; it does not certify deployment.⁹

⁹ | **Required zero-failure sample bound:** Inverting the cumulative binomial relation $(p_{\text{target}})^n \leq \alpha$ determines the minimum number of consecutive zero-failure trials required to establish a one-sided success lower limit at confidence level $1 - \alpha$: $n = \lceil \ln \alpha / \ln p_{\text{target}} \rceil$. Establishing a 99.9 percent one-sided success lower limit ($p_{\text{target}} = 0.999$) at 95 percent confidence requires $n = 2,995$ consecutive error-free runs. At 260 s per trial, this example occupies 216.3 h of test-cell time; whether components can sustain it depends on their measured load, duty cycle, maintenance, and replacement plan. Statistical evidence alone does not establish physical safety.



Figure 14.4: Standardized robotic evaluation facility: NIST response-robot test bays photographed in 2013 (Linda Joy/NIST); NIST described its then-current methods under ASTM E54.08. The image documents a structured facility, not a particular result or every test fixture. Repeatability is bought by bounding the state space, which is what limits the conclusion: a zero-failure run across standardized bays establishes competence inside the arena and says nothing about specular glare, surface contaminants, or novel obstacle dynamics outside it.



Napkin Math 14.2: Statistical scaling of assurance

Problem: Suppose an evaluation target calls for a one-sided $P_{99.9}$ success lower limit ($p_{\text{target}} = 0.999$) at 95 percent confidence ($\alpha = 0.05$). If each physical trial requires $T_{\text{trial}} = 260$ s, how many zero-failure runs and how much wall-clock testing time are required?

Calculations: Using the Clopper-Pearson bound formula, meeting this statistical target after zero failures requires $n = \lceil \ln(0.05)/\ln(0.999) \rceil = 2,995$ consecutive zero-failure trials. At 260 s per trial, this demands $T_{\text{total}} = 2,995 \times 260 \text{ s} \approx 216.31$ hours (9.01 days) of continuous testing.

With a 2-operator safety protocol, this consumes $216.31 \times 2 \approx 432.6$ operator hours, or over 54 eight-hour shifts.

Systems insight: Establishing a 99.9 percent one-sided success lower confidence limit by this method requires nearly 3,000 iid zero-failure trials and over nine continuous 24-hour days of test cell occupancy. The campaign must track wear and recalibration throughout; a component replacement or changing dynamics may require a new population definition rather than pooling incompatible runs.

When reliability requirements scale to continuous-time physical systems governed by an operational failure rate λ (failures per operating hour), the exposure deficit becomes insurmountable. The required zero-failure physical exposure time is governed by the Butler & Finelli exposure bound (Butler and Finelli 1993). As shown in the detailed callout box, at standard 95 percent statistical confidence, bounding an operational failure rate requires testing for roughly three times the target mean-time-to-failure continuous hours ($T \approx 3.0/\lambda$).¹⁰

The **Butler & Finelli infeasibility bound** is the mathematical theorem establishing that bounding ultra-low physical failure rates (e.g., catastrophic failure rates $\lambda \leq 10^{-6}$ to 10^{-9} h^{-1}) through black-box operational exposure alone is physically impossible due to astronomical duration requirements ($T = -\ln(1-C)/\lambda$), motivating independently validated runtime constraints and structured fault injection.¹¹

¹⁰ | **Butler-Finelli exposure bound:** Butler and Finelli (1993) proved that certifying ultra-high reliability ($\lambda \leq 10^{-9}$ failures/hour) purely through black-box operational testing requires more than 10^9 operating hours ($> 114,000$ years). In commercial aerospace and life-critical robotics, physical exposure can never clear this bound before hardware reaches wear exhaustion. Systems must therefore establish safety through deterministic runtime architectural supervisors, formal invariant filtering, and active hardware-in-the-loop fault injection.

¹¹ | **Functional safety scope:** Functional safety standards define context-specific hardware metrics and assurance processes; a single failure-rate number cannot be assigned to all Physical AI systems or certified by mileage alone. The Poisson examples here are analytical targets, not claims of compliance with IEC 61508 or ISO 26262.

🔍 Systems Perspective 14.3: Limits of testing for ultra-reliability

Result. This is a theoretical result rather than an accident. Butler and Finelli argued that validating ultra-high reliability requirements for life-critical cyber-physical systems (such as commercial fly-by-wire flight control software requiring a catastrophic failure rate $\lambda \leq 10^{-9}$ failures/hour) through empirical operational testing alone is physically impossible (Butler and Finelli 1993).

Mathematical Proof & Exposure Deficit: Under a continuous-time Poisson failure process, the probability of observing zero catastrophic failures over duration T is $e^{-\lambda T}$, so certifying that the true failure rate is at most λ at confidence $C = 1 - \alpha$ requires a zero-failure physical exposure time of $T = \frac{-\ln(1-C)}{\lambda}$. At standard 95 percent statistical confidence ($\alpha = 0.05$), $T = \frac{\ln(20)}{\lambda} \approx \frac{3.0}{\lambda}$, roughly three times the target mean-time-to-failure. Three stipulated hourly targets show the exposure scaling; they are teaching scenarios, not thresholds assigned by IEC 61508, ISO 26262, or DO-178C.

Figure 14.5 uses a constructed mileage series to show the scale of RAND physical-exposure examples. It is not a record of accumulated fleet evidence, and simulated miles do not count toward those thresholds.

- *Illustrative target* $\lambda = 10^{-6} \text{ h}^{-1}$. Requires $T = 3.0 \times 10^6$ hours (342 years) of continuous zero-failure execution.
- *Illustrative target* $\lambda = 10^{-8} \text{ h}^{-1}$. Requires $T = 3.0 \times 10^8$ hours (34,200 years).
- *Illustrative target* $\lambda = 10^{-9} \text{ h}^{-1}$. Requires $T = 3.0 \times 10^9$ hours (342,000 years).

Fleet Testing Fallacy: At a stipulated 10^{-9} h^{-1} Poisson target, even one hundred continuously operated rigs would require roughly 3,420 calendar years for a zero-failure 95 percent upper rate limit, before maintenance and changes in the tested population. Furthermore, fleet testbeds share common-mode software builds, environmental biases, and fixture setups, violating the statistical independence assumption.

Implication: The derivation above is this book's, following their argument; they reach the same conclusion by a different route and their published figure for the flight-control case is on the order of 10^9 hours. It shows that ultra-reliability cannot be tested into a machine through operational burn-in, fleet mileage accumulation, or passive sampling. High-assurance physical

AI systems must instead enforce safety through deterministic runtime architectural supervisors, formal invariant filtering, and active unannounced fault injection.

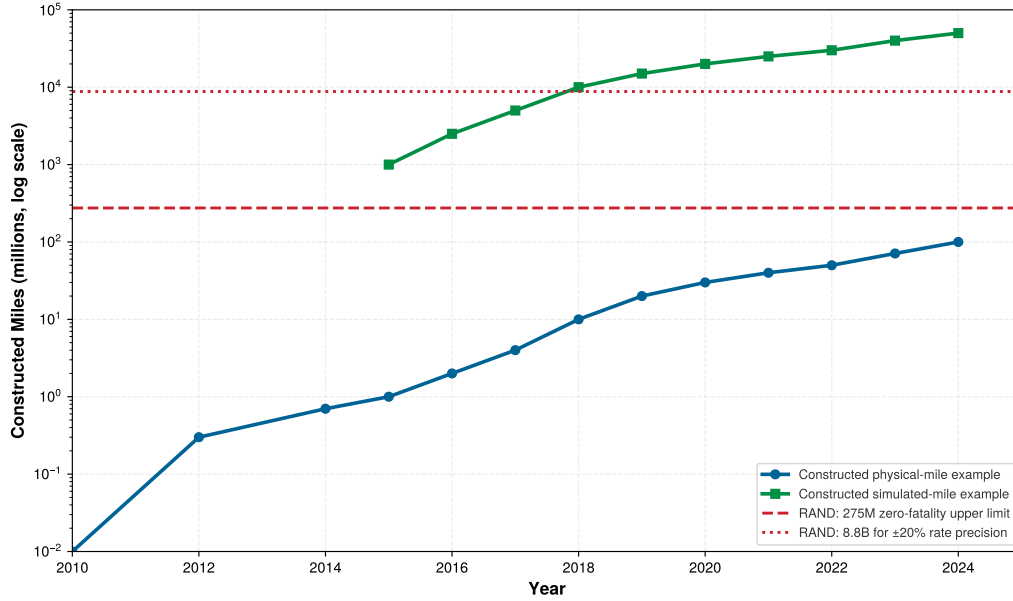


Figure 14.5: Autonomous miles scaling deficit: Constructed mileage series illustrate the difference between physical exposure and simulation; these curves are not audited fleet totals. RAND Table 1 (Kalra and Paddock 2016) gives 275 million zero-failure physical miles for a 95 percent upper confidence limit at the human fatality rate, 8.8 billion miles for ± 20 percent rate-estimate precision, and about 11 billion miles to detect a 20 percent lower rate at 95 percent confidence and 80 percent power. Simulated miles do not add physical exposure.

Consider an analytical example of an autonomous mobile manipulation platform requiring certification for an industrial manufacturing facility requiring a catastrophic failure rate $\lambda \leq 1.0 \times 10^{-4}$ failures/hour (Mean Time Between Critical Failures MTBF $\geq 10,000$ hours) at 95 percent statistical confidence ($\alpha = 0.05$). Each discrete warehouse retrieval mission requires $t_{\text{mission}} = 360$ s (6.0 minutes = 0.10 hours), traveling 432 m at an average velocity of $v = 1.20$ m/s. Automated dock recharge and telemetry reset requires $t_{\text{reset}} = 120$ s (480 s total per trial cycle). Wheel diameter is $D_{\text{wheel}} = 0.20$ m, and drive gearhead L_{10} bearing fatigue life is rated at 5.0×10^7 revolutions.

Under continuous Poisson arrivals, certifying failure rate λ at confidence $1 - \alpha$ without observing any failures demands continuous exposure time $T_{\text{continuous}} = -\ln(\alpha)/\lambda = -\ln(0.05)/(1.0 \times 10^{-4} \text{ h}^{-1}) \approx 29,957$ hours (3.42 years). Discretizing this requirement into missions with per-trial failure budget $p_{\text{fail}} \leq \lambda \cdot t_{\text{mission}} = 1.0 \times 10^{-5}$ ($p_{\text{target,success}} \geq 0.99999$) requires $N_{\text{req}} = \ln(\alpha)/\ln(p_{\text{target,success}}) \approx 299,572$ consecutive zero-failure trials. On a single physical rig, executing these trials consumes $T_{\text{calendar}} = 299,572 \times 480 \text{ s} \approx 39,943$ hours (4.56 years) and accumulates $N_{\text{rev}} = (N_{\text{req}} \cdot v \cdot t_{\text{mission}})/(\pi D_{\text{wheel}}) \approx 2.06 \times 10^8$ wheel revolutions. The wheel-rotation total is $4.12 \times$ a **stipulated** 5.0×10^7 -revolution L_{10} bearing rating; wheel revolutions are not automatically gearhead or bearing revolutions, and L_{10} is a population-life statistic at specified load and speed, not a destruction deadline. A real campaign must map drivetrain ratios, loads, maintenance, and replacements before combining trials. Physical testing alone also leaves hazards outside the sampled distribution, so an independent permission path and fault-injection evidence remain necessary.

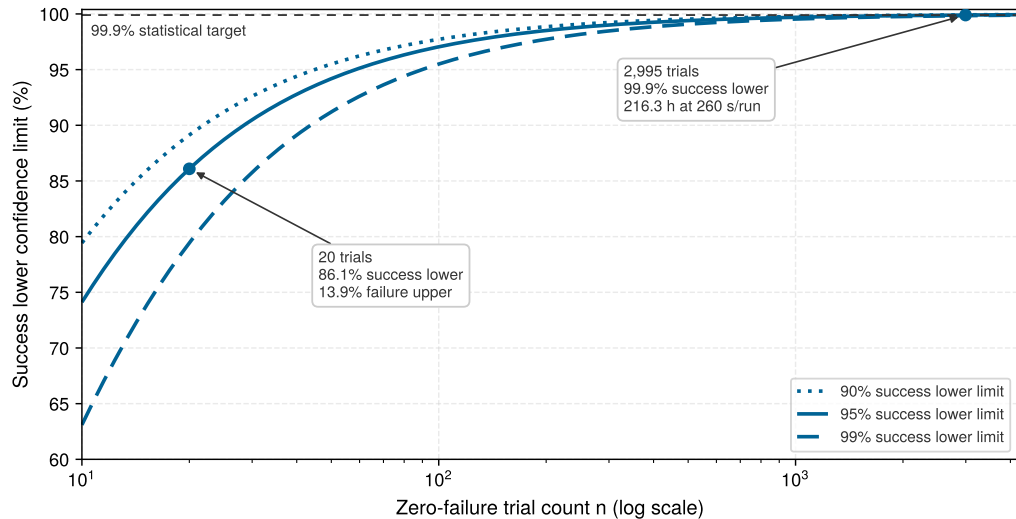


Figure 14.6: Confidence bounds and testing budgets: One-sided Clopper-Pearson success lower limits after zero failures at 90, 95, and 99 percent confidence. At 95 percent confidence, 20 iid runs give an 86.1 percent success lower limit and a 13.9 percent failure upper limit under the tested distribution. Reaching a 99.9 percent success lower limit requires 2,995 zero-failure draws under the same assumptions, or 216.3 hours at the illustrative 260-second cycle. Neither threshold grants deployment permission or proves component wear life.



Checkpoint 14.5: Sample size and MTBF evidence target

Before examining exact confidence lower bounds, verify your understanding of sample size scaling and physical test limits:

- ☐ **Butler–Finelli exposure deficit:** Under a stationary Poisson failure model, why does a one-sided 95 percent upper rate limit corresponding to a 10,000-hour MTBF target require over 29,000 hours of zero-failure physical exposure?
- ☐ **Mechanical fatigue exhaustion:** Can you explain why wheel revolutions cannot be compared directly with a bearing L_{10} rating without drivetrain mapping and load conditions?
- ☐ **Architectural containment versus statistical sampling:** Can you explain why high-assurance physical AI systems substitute deterministic runtime monitors for empirical black-box trial accumulation?

The mathematical scaling of these exact confidence lower bounds across trial counts and confidence levels is visualized in figure 14.6, highlighting the widening gap between achievable testing budgets and high-assurance reliability targets.

As summarized in table 14.4, the non-asymptotic scaling of confidence bounds reveals the severe evidentiary deficit of small testing budgets across common trial sample sizes, establishing the mathematical ceiling of empirical certification.

Table 14.4: One-sided binomial confidence limits after zero failures: Success lower limits are $\alpha^{1/n}$ and the corresponding failure upper limit is $1 - 0.05^{1/n}$. Test times use the illustrative 260-second cycle. These limits concern independent draws from one declared distribution; they do not estimate an observed failure interval, authorize deployment, or establish component wear life.

Zero-failure trials n	90% lower success limit	95% lower success limit	99% lower success limit	95% upper failure limit	Test time at 260 s/run	Interpretation under iid test draws
10	79.43%	74.11%	63.10%	25.89%	0.7 h	Wide uncertainty; no observed failures

Zero-failure trials n	90% lower success limit	95% lower success limit	99% lower success limit	95% upper failure limit	Test time at 260 s/run	Interpretation under iid test draws
20	89.13%	86.09%	79.43%	13.91%	1.4 h	Bound applies only to the sampled distribution
50	95.50%	94.18%	91.20%	5.82%	3.6 h	More evidence; untested conditions remain
100	97.72%	97.05%	95.50%	2.95%	7.2 h	No deployment permission follows from the interval
250	99.08%	98.81%	98.17%	1.19%	18.1 h	Check hardware and sampling stationarity
500	99.54%	99.40%	99.08%	0.60%	36.1 h	Bound remains tied to declared conditions
1,000	99.77%	99.70%	99.54%	0.30%	72.2 h	Below a 99.9% lower-limit target
2,995	99.92%	99.90%	99.85%	0.10%	216.3 h	Meets that statistical target if assumptions hold

These physical and statistical ceilings force an unavoidable architectural conclusion: physical AI safety cannot rest on statistical confidence in unconstrained model weights. When the exposure required to validate an autonomous policy cannot be purchased through physical testing because of wall-clock time, mechanical wear, and safety hazards, systems engineering mandates two foundational strategies:

1. **Dual-Brain Runtime Containment:** Bounding the physical authority of the learned policy by delegating ISO 10218 / ISO/TS 15066 safety envelopes, kinematic acceleration limits, and emergency stops to an independent, deterministic nervous system (Brooks 1986b) (extending the proposal-permission architecture from section 6.1). Bounding policy authority shifts the protection claim to a separately validated permission and fallback path, whose state estimates, plant and actuator models, response timing, and reserved stopping margin must hold in the stated operating envelope.
2. **Unannounced Fault Injection on Physical Benches:** Rather than waiting millions of operational hours for passive disturbances to strike, verification engineers actively attack system invariants on hardware testbeds, deterministically injecting unexpected anomalies during active rollouts to evaluate real-time supervisor responses.

Unannounced fault injection is the active evaluation methodology of deterministically introducing hardware, timing, and sensory disturbances (such as bus transport latency spikes, packet corruption, clock skew, and localized friction collapse) during live test bench execution without prior model notification, testing whether real-time supervisors respond within the measured physical margins for the injected conditions.

To operationalize this methodology while enforcing full-denominator accounting and non-asymptotic stopping rules without optional-stopping inflation, physical trials adhere to the structured qualification protocol formalized below:

Execution Context & Invariants:

- Automated Physical Test Cell / Hardware-in-the-Loop (HIL) Qualification Rig.
- *Statistical Invariant:* Pre-registered required sample size $N_{\text{req}} = \lceil \ln(\alpha) / \ln(p_{\text{target}}) \rceil$; zero informal looks to prevent optional stopping inflation ($P(\text{false alarm}) \leq \alpha$).
- *Physical Invariant:* Complete-denominator ledger accounting; every trial initiated under policy authority increments $n_{\text{attempted}}$ and cannot be discarded.

Execution Protocol:

1. **Pre-Registration & Exposure Sizing (Stage 1):** Compute required zero-failure sample count $N_{\text{req}} \leftarrow \lceil \ln(\alpha) / \ln(p_{\text{target}}) \rceil$, initialize ledger counters ($n_{\text{attempted}} \leftarrow 0, n_{\text{success}} \leftarrow 0, n_{\text{supervisor}} \leftarrow 0, n_{\text{timeout}} \leftarrow 0, n_{\text{fault}} \leftarrow 0$), and generate an immutable experiment pre-registration record with cryptographic digest $\text{SHA256}(\pi_{\theta} \parallel \Theta_{\text{safe}} \parallel N_{\text{req}})$.
2. **Thermal & Fixture Reset (Stage 2):** For each trial $i = 1, 2, \dots, N_{\text{req}}$:

- For the pooled binomial bound, draw each condition tuple c_i independently from the predeclared target distribution $\mathcal{D}_{\text{ODD}}$. A fixed quota grid instead needs separate per-stratum inference and declared weights before forming an ODD-wide claim.
- Poll motor winding temperatures T_{motor} ; if $T_{\text{motor}} > 40^\circ\text{C}$, enter thermal rest loop until cooled.
- Reposition physical chassis to initial pose s_0 , zero IMU biases, and flush telemetry ring buffers.

3. Closed-Loop Execution & Real-Time Supervision (Stage 3):

- Increment attempted trial count: $n_{\text{attempted}} \leftarrow n_{\text{attempted}} + 1$.
- Launch closed-loop trajectory execution under policy π_θ at 50 Hz control cadence.
- Concurrently, execute deterministic supervisor $\mathcal{M}_{\text{referee}}$ at its verified control rate on the independent permission tier:
 - Check SE(3) pose tracking error (the physical deviation in both 3D position and orientation) $\|s(t) - s_{\text{ref}}(t)\|_{\text{SE}(3)} \leq e_{\text{max}}$.
 - Check proposal freshness lease against bus jitter, verifying that command transmission latency does not exhaust valid execution leases, $\tau_{\text{age}} \leq \Delta t_{\text{max}}$.
 - Check spatial obstacle clearance $d_{\text{clear}}(t) \geq d_{\text{min}}$.
 - Check actuator current/torque limits, bounded by stator electrical time constants and thermal derating limits, $\|\tau(t)\| \leq \tau_{\text{max}}$.

4. Trial Outcome Classification (Stage 4):

- If spatial goal g reached within T_{max} with zero supervisor overrides: record success ($Y_i \leftarrow 1$; $n_{\text{success}} \leftarrow n_{\text{success}} + 1$).
- Else if $\mathcal{M}_{\text{referee}}$ asserted safety override: record censored failure ($Y_i \leftarrow 0$; $n_{\text{supervisor}} \leftarrow n_{\text{supervisor}} + 1$).
- Else if execution elapsed time $t > T_{\text{max}}$: record timeout failure ($Y_i \leftarrow 0$; $n_{\text{timeout}} \leftarrow n_{\text{timeout}} + 1$).
- Else if bus dropped or hardware faulted: record hardware failure ($Y_i \leftarrow 0$; $n_{\text{fault}} \leftarrow n_{\text{fault}} + 1$).
- Append immutable trial metadata record to ledger $\mathcal{L}$.

5. Predeclared Statistical Stopping Decision (Stage 5):

- If $Y_i = 0$ (any failure during the zero-failure target protocol): terminate evaluation early. Set $p_{\text{lower}} \leftarrow 0$ when $n_{\text{success}} = 0$; otherwise compute $p_{\text{lower}} \leftarrow \text{BetaQuantile}(\alpha; n_{\text{success}}, n_{\text{attempted}} - n_{\text{success}} + 1)$. Return (ZERO_FAILURE_TARGET_NOT_MET, p_{lower} , $\mathcal{L}$).
- If $i = N_{\text{req}}$ and $n_{\text{success}} = N_{\text{req}}$: compute exact zero-failure lower bound $p_{\text{lower}} \leftarrow \alpha^{1/N_{\text{req}}}$ and return (STATISTICAL_TARGET_MET, p_{lower} , $\mathcal{L}$).

12 | Non-Lipschitz contact

bifurcation: Rigid-body mechanical contact is fundamentally non-Lipschitz continuous due to set-valued friction cones and impact velocity jumps ($v^+ \neq v^-$). A sub-millimeter deviation in gripper contact location can abruptly alter contact constraints, shifting the system trajectory from stable force regulation to complete slip and jamming. Because small perturbations trigger discontinuous state jumps, open-loop evaluations fail to capture how minor trajectory errors destabilize downstream physical interactions.

Even under rigorous qualification protocols and automated test cells, statistical sampling alone cannot establish ultra-reliable autonomy. In section 14.6, we examine the structural boundaries of empirical evaluation, demonstrating why finite testing cannot overcome endogenous distribution drift or eliminate the necessity for deterministic runtime containment.

14.6 What Evaluation Cannot Establish

In a conventional software benchmark, the test framework evaluates a model against a fixed dataset with a distribution independent of the model's predictions. When an image classifier labels a stop sign as a speed limit sign, the test runner records the error, loads the next image from disk, and continues along its predetermined sequence. In a physical machine, the input distribution is endogenous, wherein the machine's own control outputs determine the physical states and sensory observations it will subsequently encounter through closed-loop interaction with the physical world.¹² Building on the causal boundary established in section 2.2, the observation at the next step

is affected by the command u_t admitted by the independent permission tier, which may differ from the policy proposal a_t .

Unrolling this recursive interaction across a finite sequence of actions reveals the compounding nature of embodied execution: the sequence of admitted commands determines the physical states it encounters, and those states dictate the subsequent sensory inputs it must process. Because the distribution of states depends on the policy itself, two policies starting from the same state can generate different future state distributions when their admitted commands differ (for formal derivations of trajectory distributions and state marginals, see Appendix A). In an analytical example where a mobile robot travels forward at 2.0 m/s with a control frequency of 50 Hz (20 ms per step), a steering discrepancy of 0.05 rad at step $t = 0$ alters the lateral position by 2.0 mm at step $t = 1$. That displacement changes the next camera perspective and may change the policy's subsequent proposal. How far the trajectories separate after 50 steps depends on the controller, permission decisions, and plant dynamics; the one-step calculation alone gives no support-overlap estimate. A static predeployment evaluation benchmark, whether an offline log of expert demonstrations or a fixed series of test injections, samples from a preselected distribution that need not match the candidate's closed-loop state distribution. Evaluating a policy against a fixed dataset measures prediction error on states visited by an external logging policy, providing no direct evidence about new states reached when candidate proposals are admitted on hardware.

Executing the policy in closed-loop simulation allows the model to generate its own trajectory, but it replaces one distributional assumption with another. In simulation, the next state is generated by an approximate transition model $\hat{\mathcal{T}}(s_{t+1} | s_t, u_t)$ and a synthetic sensor model $\hat{h}(s)$. The simulator samples from the simulated trajectory distribution $p_{\pi, \hat{\mathcal{T}}}$, not the physical deployment distribution $p_{\pi, \mathcal{T}}$. If the simulator models surface traction with a constant Coulomb friction coefficient of $\mu = 0.80$ while the physical floor has a localized oil patch with $\mu = 0.25$, or if the simulated actuator responds instantaneously while the physical motor exhibits a 15 ms electrical time constant in its stator windings, the physical trajectory can diverge from $\hat{\mathcal{T}}$. Mechanical discrepancies can compound across steps and lead to states absent from the tested simulations. Closed-loop simulation measures performance on specified trajectories and disturbances in the chosen simulator; without a separate formal argument it proves neither simulator-wide stability nor stability of the physical machine under $\mathcal{T}$.

Because the deployed distribution is endogenous and tied to physical mechanics, the strongest conclusion an empirical evaluation can support is narrow and conditional. A predeployment evaluation establishes that the specific software build and hardware revision tested achieved a measured outcome rate under the exact sampled initial conditions, within the specific environmental envelope present during the trial, subject to the stated confidence interval. For an analytical example of 500 completed evaluation runs yielding zero catastrophic incidents, the arithmetic of one-sided interval estimation establishes with 95 percent confidence that the true incident rate under those exact test conditions is below $1 - 0.05^{1/500} \approx 0.597$ percent per run (a failure probability under 6.0×10^{-3}). Evaluation does not establish that the failure rate remains below 0.597 percent when the ambient temperature drops by 10°C causing thermal derating, when gearbox compliance and mechanical backlash increase by 0.2 mm through wear, or when optical lens contamination degrades camera contrast by 15 percent.

Accumulating larger sample counts reduces sampling uncertainty, which tightens the statistical error bounds around the tested distribution, but it cannot reduce structural uncertainty about unmodeled physics or unvisited domains. If a mobile robot completes 10,000 consecutive navigation trials inside a climate-controlled test facility with an illumination of 500 lux, the resulting 95 percent upper confidence bound confirms a failure probability below 3.0×10^{-4} per trial in that facility. That statistical confidence is mathematically valid for the tested distribution d_{test} , but it provides zero coverage for an unmodeled condition such as a low-angle sun glare at 45 lux entering through an open doorway. More trials inside the test envelope merely measure the known distribution with finer precision. They cannot repair missing environmental variations, unmodeled actuator dynamics (such as bus jitter or SerDes bandwidth limits), or the physical wear of mechanical bearings over the operating life of the machine.

Each monitorable boundary of the evaluation claim should have a runtime signal, threshold, and tested response. Conditions that cannot be sensed reliably require ODD restrictions, passive

protection, or other independent barriers. Since evaluation cannot establish how the policy behaves outside the sampled distribution, the nervous system must monitor measured operating conditions and enforce independent physical constraints. Distribution support is only partially observable, and an OOD detector has measurable false negatives. This evaluation contract names five monitoring channels whose coverage and response must be measured:

1. **Input Support / Out-of-Distribution (OOD) Monitor:** Estimates whether incoming observations depart from the tested distribution using a calibrated score and reports false negatives on held-out shifted conditions. A detector cannot recognize every unseen hazard; physical permission barriers remain necessary.
2. **Execution Latency Monitor:** Measures each inference age against the declared control deadline (for example, 20 ms at 50 Hz), records misses, and checks that lease expiry invokes the validated fallback. A P_{99} summary alone is not a hard deadline bound.
3. **Constraint Margin Monitor:** Tracks the distance between the current physical state and hardware safety limits, including joint torque limits, motor winding temperature, and obstacle clearance against limits specified for the hardware and stopping model.
4. **Intervention Rate Monitor:** Calculates the frequency of safety supervisor overrides, detecting when policy instability causes excessive corrections.
5. **Environmental Assumption Monitor:** Measures external operating conditions, such as surface traction estimated from wheel slip and ambient illumination measured by photometric sensors.

A runtime monitor that merely records an anomaly to a disk log provides no protection to a moving body. When a monitor detects that a deployment parameter has crossed outside the evaluated envelope, the nervous system must act on the actuators by scaling down policy authority, switching to a deterministic fallback controller, or executing a controlled stop. Conceptually, this acts as a runtime reject-option gate (for control systems fallback logic, see Appendix B). A timely, high-confidence proposal is still subject to independent state, clearance, and actuator checks before any u_t is applied. However, if the input support monitor detects that sensor observations have drifted into an uncharacterized regime (epistemic uncertainty is too high), or if execution latency exceeds the deadline, the nervous system discards the inference output. It instead requests a validated fallback, such as a speed restriction or controlled stop, provided the sensed state, actuator capability, and reserved clearance make that response feasible, and requests human intervention.

Certain failure modes cannot be detected by runtime monitoring. When a learned policy proposes an action that satisfies every latency deadline, respects all joint torque limits, and operates on familiar sensor inputs, but nevertheless represents an incorrect action, such as steering into a glass partition that the vision-language backbone (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023) misidentified as free space, the monitors may observe no violation before the physical margin is spent. The detector can miss a novel condition, computation can finish on schedule, and the measured clearance may be wrong until contact occurs. A semantic error may evade an OOD detector and other monitors; their false-negative rates must be measured against held-out hazards. Safety for undetectable failures cannot be established through empirical evaluation or software monitors. It requires physical zoning, mechanical current limiters, and independent safety barriers (e.g., ISO 10218 / ISO/TS 15066 safety envelopes) whose protective capability is validated independently of the learned model under specified physical assumptions.

Downstream: Physical failures undetectable by sensory telemetry are formally cataloged in section 34.4.

Systems Perspective 14.4: Fall metrics and multi-contact stability

Evaluating a humanoid robot exposes the fatal flaws of conventional machine learning benchmark metrics:

- **The Fallacy of the 95 Percent Success Rate:** In manipulation, an assembly policy with a 95 percent success rate is a usable prototype: nineteen out of twenty trials succeed, and the twentieth trial simply fails to insert a peg and resets. For an upright 70 kg bipedal humanoid, the consequence of the failed 5 percent must be measured separately. If every failed task were a balance loss, the corresponding fall probability would be 5 percent per

task under the tested distribution; converting that to minutes also requires a measured task cadence. A ground impact can shock drives and structures and create injury risk. Humanoid evaluation therefore needs a separate fall count and exposure denominator, with uncertainty, alongside task completion.

- **The Denominator Problem in Locomotion Benchmarks:** Autonomous vehicle evaluation long ago recognized that reporting “1,000 miles without a crash” is meaningless if the miles were logged on straight, sunny highways. Similarly, evaluating humanoid locomotion on flat, pristine warehouse linoleum tells an engineer nothing about real-world balance stability. The evaluation denominator must be stratified across disturbance regimes: steps taken across slippery surfaces ($\mu < 0.3$), compliance changes (carpet vs concrete), curb transitions, and stochastic multi-directional impulse shoves ($> 150 \text{ N} \cdot \text{s}$).
- **Multi-Contact Stability Margin Certification:** Open-loop trajectory tracking errors ($\|\mathbf{q}_{\text{ref}} - \mathbf{q}\|$) do not predict falls. A humanoid policy can track joint trajectories with millimeter precision while its Zero Moment Point (ZMP) hovers on the razor-thin edge of the support foot boundary. Evaluating humanoid survivability requires monitoring dynamic stability reserves: capture point margin (the distance from the capture point to the edge of the support polygon), instantaneous centroidal angular momentum, and friction cone utilization across dynamic contact phases.



Checkpoint 14.6: Runtime monitoring channels and semantic limits

Before examining evaluation failure modes and denominator audits, verify your understanding of runtime monitoring boundaries:

- ☐ **Endogenous distribution divergence:** Can you explain why two policies starting from identical states encounter non-overlapping sensory distributions within seconds?
- ☐ **Five-channel runtime defense:** Can you identify the five real-time monitoring channels required to police predeployment evaluation boundaries?
- ☐ **Undetectable semantic failures:** Can you explain why vision-language misidentifications (such as clear glass partitions) bypass telemetry margin monitors and mandate physical zoning?

Recognizing what evaluation cannot establish clarifies where testing protocols are vulnerable to systemic distortion. In section 14.7, we diagnose the procedural and statistical pathologies that corrupt physical evaluation campaigns.

14.7 How Evaluation Goes Wrong

Systematic errors undermining policy evaluation rarely stem from deliberate fraud. They arise when an evaluation protocol measures a well-chosen environment, terminates on a well-timed run, and tracks a surrogate metric that answers an easier question than physical task completion. In a physical trial, environment selection occurs whenever the operating domain of the test bench is narrower than the claimed deployment envelope. Consider a mobile robot whose navigation policy is evaluated down a central 3.0 m wide hallway with diffuse overhead lighting of 450 lx, uniform concrete flooring, and zero dynamic obstacles. When the evaluation report aggregates twenty consecutive runs along this route into a single success rate, the claim silently expands to cover the entire facility, including 1.2 m wide side corridors with direct afternoon glare and dense pedestrian traffic. The policy succeeds on the bench because its sensory representations are never tested outside a benign, high-contrast slice of state space. Environment selection is detected by comparing a structured condition ledger, which records measured illumination, passage width, surface friction, and obstacle velocities for every trial, directly against the boundaries of the claimed operational design domain. If the ledger covers only a tight cluster within that envelope, the evaluation establishes nothing about the remaining volume.

A second failure mode occurs when the evaluation metric drifts away from physical task completion toward offline surrogate quantities. In open-loop evaluation, which refers to a system where outputs are not fed back to adjust inputs, a policy is scored based on an ‘action agreement’ metric. This metric measures the mean squared error or cosine distance between the predicted control commands and expert demonstrations recorded on a static dataset. A policy can achieve an action agreement metric above 98 percent on recorded sensor logs, yet fail immediately when its proposals are admitted on a physical machine due to unmodeled stator delays, gearbox compliance, and bus jitter (as established in section 4.4 and section 8.4). Action agreement does not account for the closed-loop compounding of physical execution error. As derived in section 14.3, an unmodeled heading bias of 1.0° (0.0175 rad) produces negligible loss on an offline validation batch, but over a 10 m traversal, the uncorrected lateral drift integrates through continuous unicycle kinematics ($y(t) \approx v\epsilon_\theta t$) into a 17.5 cm displacement, colliding with a boundary before completing the maneuver. Metric drift is detected by requiring the primary evaluation metric to measure closed-loop terminal outcomes, such as final pose error within 0.05 m and zero supervisor interventions, rather than open-loop prediction error. When an evaluation relies on open-loop metrics, it detects whether the network reproduced demonstration vectors, not whether the closed-loop physical system can reach a goal state.

Even with closed-loop physical trials, repeated tuning against a fixed evaluation benchmark degrades result independence. When an engineering team adjusts policy hyperparameters, observation filtering, or neural network checkpoints after every trial on a designated evaluation course, the nominal test set ceases to be an independent holdout. Each configuration change incorporates information about the specific obstacle positions, wall textures, and lighting angles of that exact track. After forty design iterations, the final model checkpoint reflects implicit training on the test course itself, masking how the policy responds to novel geometry. This test set contamination cannot be detected by inspecting the final model weights or running a single verification trial on the same course. It is detected exclusively through provenance tracking, analyzing the chronological history of policy versions, tuning commits, and test dates to confirm that the reported evaluation was executed on a fresh, quarantined configuration that was never exposed to iterative tuning.

Evaluation protocols are equally vulnerable to unintended cherry-picking through optional stopping. When an engineer monitors a continuous sequence of physical runs and halts the experiment as soon as a target performance threshold or statistical significance criterion is reached, the reported confidence is distorted. If an evaluation team conducts an experiment where each look carries a nominal false-positive threshold of 5 percent, stopping early upon observing an apparent success across multiple evaluation checkpoints inflates the true error rate. Five independent 5 percent looks would have a $1 - 0.95^5 \approx 22.6$ percent chance of at least one false positive. Actual repeated looks on one accumulating run are dependent, so that arithmetic is only an illustration; their error rate must be computed for the stated stopping rule. Unplanned repeated looks can exceed the nominal 5 percent false-acceptance rate. Optional stopping cannot be detected from summary tables or final pass rates. It is detected only by auditing the pre-registered trial protocol, verifying that sample sizes, stopping conditions, and evaluation milestones were fixed before the first physical run began. When valid early stopping is required to prevent exceeding actuator torque limits, manage Joule heating ($I^2 R$) thermal derating (detailed in section 4.1), and conserve testbed wear without inflating Type I error, evaluation teams employ Abraham Wald’s **sequential probability ratio test (SPRT)**¹³ (Wald 1945). Rather than fixing sample size in advance or naively peeking at intermediate results, SPRT evaluates an accumulation of evidence after each physical rollout, computing the cumulative likelihood ratio between an acceptable reliability hypothesis (H_1) and an unacceptable failure hypothesis (H_0). Testing halts dynamically the moment the accumulated evidence crosses predeclared likelihood-ratio thresholds. For two specified simple hypotheses and a declared sequential design, SPRT can reduce expected sample count compared with a fixed-horizon test. The common Wald boundary formulas are approximations affected by likelihood-ratio overshoot; actual Type I and Type II errors and testbed savings must be calculated or simulated for the chosen hypotheses and outcome model. For the exact log-likelihood martingale formulations, Bernoulli updates, and Wald boundary derivations, see section A.9.

A related statistical distortion occurs when the denominator of an evaluation is quietly filtered. In physical testing, runs frequently terminate abnormally due to software timeouts, emergency stop

¹³ | **Sequential Probability Ratio Test (SPRT):** Predeclared likelihood-ratio boundaries allow valid sequential stopping for specified hypotheses when their actual error probabilities are calibrated, including boundary overshoot. The number of saved runs is path- and hypothesis-dependent; no universal wear saving follows.

actuators, sensor communication drops, or safety supervisor interventions. When an evaluation team discards these aborted runs as invalid test artifacts, calculating success as the ratio of completed goals to non-aborted trials, the reported metric misrepresents physical reliability. If a mobile base completes nine runs out of ten attempts, but the tenth attempt required an operator takeover 4.0 s into execution to prevent a collision with a wall, removing that attempt raises the apparent success rate from 90 percent to 100 percent. Every initiated trial, operator takeover, emergency stop, and hardware timeout must remain in the denominator. When physical constraints or safety limits truncate a run before the nominal time limit of 60 s expires, the trial must be recorded explicitly as a censored failure rather than omitted from the ledger. This systematic artificial inflation of empirical reliability metrics by discarding aborted runs, mistaking conditional survival on truncated trajectories for genuine operational competence, is known as the **censored trial fallacy**. Denominator filtering is detected by cross-referencing low-level actuator telemetry and safety supervisor logs against the trial ledger, confirming that every power-on cycle and autonomy engagement corresponds to a recorded outcome.

⚠ War Story 14.1: The 2017 NHTSA Autopilot crash rate mirage

Context: In January 2017, the National Highway Traffic Safety Administration (NHTSA) closed its Office of Defects Investigation report (PE 16-007) examining Tesla's Autosteer system following a fatal Florida highway collision (National Highway Traffic Safety Administration (NHTSA) 2017). The report prominently concluded that vehicle crash rates plunged by nearly 40 percent (from 1.3 to 0.8 crashes per million miles) following the installation of Autosteer.

Mechanism: The government and manufacturer analysis computed crash rates by dividing total airbag deployment events across model years by cumulative fleet mileage before and after the Autosteer software rollout. However, the exposure denominator was fundamentally corrupted: mileage was aggregated across vehicles with active Autosteer hardware regardless of whether Autosteer was engaged, while mileage data was missing for vast portions of the fleet. Subsequent independent forensic re-analyses of the raw underlying data (Quality Control Systems Corp. 2019) revealed that for more than 82 percent of the surveyed vehicles, mileage was unknown or misattributed, and airbag deployments were concentrated in vehicles with incomplete exposure records.

Impact: When the missing-data bias and unmeasured exposure miles were reconstructed with rigorous statistical controls, the apparent 40 percent crash reduction vanished; on the subset of vehicles with verifiable mileage before and after activation, crash rates exhibited an increase rather than a decrease. Yet for years, the flawed 40 percent safety improvement figure was cited across industry and regulatory policy as definitive empirical proof of automated driving safety.

Response: NHTSA faced widespread methodological scrutiny and subsequent federal audits, prompting stricter data collection mandates—including the 2021 Standing General Order requiring autonomous vehicle manufacturers to report detailed, standardized incident telemetry with verified operational design domains and time-synchronized engagement logging.

Systems lesson: In Physical AI evaluation, an unverified denominator creates dangerous statistical mirages. Aggregating open-world mileage without precise hardware engagement telemetry, time-correlated operational design domain boundaries, and complete survival records transforms missing data into false safety claims. Evaluation metrics must audit denominator integrity with the same mathematical rigor applied to numerator failure events.

The temptation to substitute visual plausibility for statistical sampling represents a pervasive vulnerability in physical systems engineering. A single video recording of a robotic arm grasping an irregular workpiece or a mobile robot navigating a cluttered room is a demonstration, not a measurement. A visually convincing demonstration proves that the system state trajectory intersects the success set for at least one sequence of initial conditions and environmental disturbances, but it provides no information about the volume of that success set within ISO 10218 and ISO/TS 15066 safety envelopes. Because human perception is tuned to smooth motion and coordinated SE(3) kinematics, an observer easily mistakes kinematic fluidity for policy competence. An execution that



Figure 14.7: Rigid barrier impact limits: High-speed capture of a full-scale vehicle impact against an instrumented rigid barrier, with optical fiducials tracking deceleration and crumple at millisecond resolution. The test marks where runtime monitoring stops being able to help, since once contact occurs the kinetic energy $E_k = \frac{1}{2}mv^2$ is dissipated by structure rather than by software, and every margin the policy was supposed to hold has already been spent.

appears deliberate and smooth can operate within 10 mm of a catastrophic kinematic singularity, exceed SerDes bandwidth limits on camera feeds, or carry an unmeasured P_{95} tracking error that produces a collision in nine out of ten repeats. A representative demonstration has no statistical standing. A representative campaign needs predeclared sampling from the target ODD or separate per-stratum analysis, full-denominator accounting, and a sample size tied to its stated claim. As Jacob Cohen formalized in his seminal statistical power analysis¹⁴ (Cohen 1988), comparing two policies (e.g., Policy A vs. Policy B) requires pre-specifying Type I error (α), Type II error (β), and target effect size h . With twenty trials per policy, statistical power cannot be specified without the baseline success rate, target effect size, test, and error level. A nominal percentage improvement alone does not establish superiority.

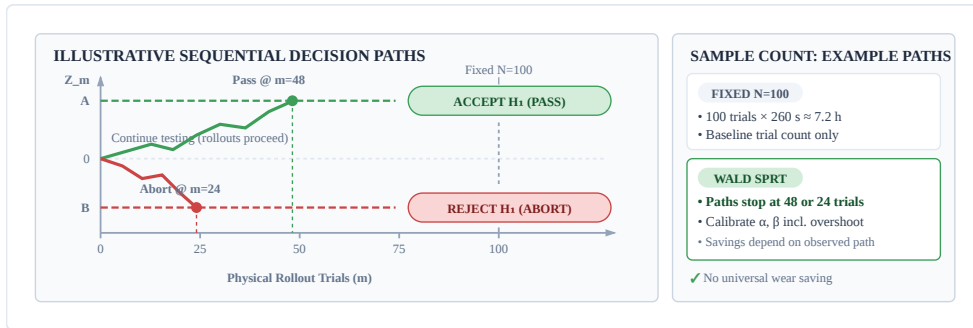
As captured in full-scale impact barrier testing (figure 14.7), physical kinetic energy transfers into destructive structural deformation within milliseconds when spatial clearance is lost. Predeployment evaluations that censor near-misses or test only benign, high-contrast environments leave physical machinery vulnerable to unrecoverable collisions the moment real-world operating conditions diverge from the tested envelope.

Understanding the mechanisms by which evaluation fails clarifies the purpose of physical testing. An empirical trial is not a demonstration to convince an observer that a model works. It is an instrument to measure where policy authority fails, what physical conditions trigger the failure, and how much statistical confidence the evidence supports. Documenting those boundaries requires an immutable record that binds every trial, condition, and intervention to a verifiable physical quantity.

¹⁴ | **Statistical Power in Physical Trials:** The probability $1 - \beta$ of correctly rejecting a false null hypothesis. For a two-policy comparison, compute power from the baseline rate, effect size, sample allocation, and chosen test before collecting physical trials (Cohen 1988).



Checkpoint 14.7: Evaluation pitfalls and denominator auditing



Before reviewing structured evaluation logging, verify your understanding of evaluation failure modes and denominator integrity:

- ☐ **Optional stopping error inflation:** Can you explain why unplanned repeated looks can inflate Type I error, while 22.6 percent assumes independent looks?
- ☐ **Sequential probability ratio testing:** Can you describe the hypotheses and error calibration needed before SPRT can support an early-stop claim?
- ☐ **Censored trial fallacy:** Can you explain how the 2017 NHTSA Autopilot investigation manufactured an apparent 40 percent crash reduction through corrupted denominator accounting?

Preventing these evaluation pathologies requires an uncorrupted audit trail that binds every trial, condition, and intervention to immutable physical quantities. In section 14.8, we specify the evaluation record schema and cryptographic provenance standards required for defensible qualification.

14.8 Evaluation Logs

Chapter 8 established the baseline schema, cryptographic hashing rules, and provenance invariants for physical run ledgers. An **evaluation record** extends that ledger with the formal fields required to audit a learned policy. It links measured outcomes to declared conditions and preserves human observations separately with their source. It serves as a self-contained evidence record that states what was measured, under what physical conditions, with what statistical power, and which operational claims the resulting evidence explicitly does not support.¹⁵

¹⁵ | **Safety of the intended functionality (SOTIF):** ISO 21448 and ANSI/UL 4600 provide frameworks for arguing about intended-functionality and autonomous-system hazards. This chapter's immutable record is a proposed engineering evidence artifact, not a specific form mandated by either standard.

Table 14.5: Physical policy predeployment evaluation record schema: Immutable schema fields added to the base engineering notebook to record target operational boundaries, hardware/model cryptographic hashes, complete-denominator trial ledgers, non-asymptotic statistical confidence bounds, and unsupported coverage gaps.

Record Schema Field	Physical / Statistical Modality	Cryptographic & Provenance Guarantee	Downstream Verifier & Usage
<code>target_odd_boundary</code>	Operational Design Domain bounds: passage width $w \geq 1.5$ m, illuminance [200, 1000] lx, surface friction $\mu \geq 0.60$, payload $m \in [0, 15]$ kg	Signed ODD specification and configuration hash	Defining tested conditions and calibrated OOD alerts; physical gating remains independent
<code>component_hashes</code>	Frozen neural network weights ($H(\theta)$), inference engine binary, chassis serial number, tire batch, sensor calibration matrices, and firmware build	SHA-256 binary digests, hardware UUID, ISO 9001 calibration certificate references	Supporting configuration traceability; preventing unrecorded hardware or driver substitutions
<code>sampling_protocol</code>	Target-distribution draw weights or per-stratum quotas, PRNG seed, reset procedure, and independence audit	Automated test harness configuration script hash, telemetry sequence log	Auditing against optional stopping and testing against hidden autocorrelation across trials
<code>complete_outcome_ledger</code>	Full-denominator raw outcome counts: $n_{\text{attempted}}$, n_{success} , n_{estop} , n_{timeout} , $n_{\text{supervisor}}$, and T_{TTI} time-series	Monotonic microsecond hardware timestamps, uninterrupted ring-buffer telemetry	Preventing censored trial fallacy; calculating uncorrupted empirical reliability
<code>statistical_evidence</code>	Exact binomial limit with sampling assumptions; Poisson rate only for separately modeled physical-hour exposure	Cryptographic verification signature, reproducible Python evaluation script	Passing bounded evidence and explicit assumptions to chapter 30
<code>unsupported_gap_catalog</code>	Explicit catalog of unvisited state spaces, omitted dynamics (surface water, dynamic crowds), and required runtime monitoring channels	Declared non-support manifest, signed safety engineering sign-off	Parameterizing real-time safety supervisors, CBF guardrails, and fallback policies in Part III

The evaluation record begins with the operational claim and its formal predicates, as specified in the predeployment evaluation record schema (table 14.5). It defines the mathematical success criterion, such as completing a trajectory between dock stations while maintaining a lateral lane error below 0.05 m and an end-point velocity below 0.1 m/s. It bounds the operational design domain across physical parameters, including ambient illuminance, surface friction coefficients, payload mass distributions, and obstacle velocities. Finally, it names the target population of physical environments to which the result is intended to generalize, distinguishing between the specific warehouse bays tested and the broader class of industrial facilities. Defining these boundaries in the record before trials begin prevents post-hoc adjustments to the pass criteria when anomalous behaviors appear.

The record captures immutable cryptographic identifiers for every software and hardware component to enable independent replication of the physical execution. These identifiers include the content hashes of the neural network weights, sensor preprocessing pipelines, policy configuration

files, simulation engine builds when synthetic runs are incorporated, and the evaluation execution scripts. On the physical system, the record logs the machine chassis serial number, wheel tire compound batch, sensor calibration matrices, motor controller firmware revisions, and communication bus configurations. Swapping a camera lens, altering a driver firmware version, or replacing worn drive belts changes the physical transfer function of the machine (shifting the causal boundary established in section 2.2), invalidating the historical data if the change is unrecorded.

The record preserves the trial-sampling protocol and its operational execution. It documents the stratification strategy used to cover the operating envelope across payload increments from 0.0 kg to 15.0 kg, the pseudorandom seed sequence governing initial robot poses, and the reset sequence executed between consecutive runs. When a mobile base returns to a starting waypoint under autonomous control, battery thermal rise (building on the actuator thermal limits established in section 4.5) and odometry drift (which subtly skews position estimates) introduce statistical dependence between adjacent attempts. The record specifies whether trials were statistically independent or chained in series, and it registers every manual reset, hardware abort, or deviation from the pre-specified protocol.

Aggregate percentages obscure the physical failure modes that determine system risk. The evaluation record reports raw outcome counts across all categories, separating completed trials, kinematic boundary violations (such as scraping a guardrail), safety supervisor torque overrides (extending the proposal-permission architecture from section 6.1), communication timeouts, and manual emergency stops. In addition to summary tallies, the record preserves per-trial time-series metadata, including peak motor winding temperatures, maximum trajectory tracking errors, and task completion durations. Logging this telemetry ensures that nominal successes operating near thermal limits or within 5 mm of a hard boundary are visible during audit. Every trial initiated under policy authority remains in the denominator.

The statistical specification defines the point estimator, confidence method, nominal confidence level, sampling distribution, and underlying mathematical assumptions. Alongside the supported claim, the record documents unsupported claims and known coverage gaps. If an evaluation suite tested navigation on dry concrete at an ambient temperature of 20°C, the record explicitly notes that it provides zero empirical support for performance on wet epoxy flooring, at temperatures below 5°C, or in the presence of unmappped obstacles taller than 0.15 m.

Consider an analytical example using the twenty-trial evaluation introduced earlier in this chapter. A policy controls a 25.0 kg differential-drive robot navigating a 30.0 m indoor corridor at 1.5 m/s. The test fixture executes twenty trials across clean concrete with ambient illumination between 250 lx and 350 lx, recording twenty completed runs with zero safety supervisor interventions. Rather than applying the degenerate Wald interval $[1.0, 1.0]$, an engineer audits this record against the exact Clopper-Pearson bounds established in section 14.5 and tabulated in table 14.4, confirming a 95 percent lower confidence bound of $p_{\text{lower}} \approx 86.1$ percent.

The completed record exposes the structural limits of the claim. As derived in table 14.4, 13.9 percent is the one-sided upper confidence limit on failure probability under the tested iid distribution, not an observed rate or expected interval between failures. When the engineer inspects the operational parameters in the record, the documentation further reveals that payload mass was fixed at 0.0 kg and ambient illuminance was confined to a narrow 100 lx band. The evaluation record proves that twenty successful runs cannot establish high reliability, providing an immutable basis to reject any assertion that the policy is ready for unmonitored deployment.

An evaluation record does not certify that an autonomous machine is safe. It establishes the empirical boundaries and statistical uncertainty of a single learned component operating under controlled conditions. In chapter 30, the verification argument ingests these evaluation records as formal evidence nodes, combining policy bounds with physical guardrail guarantees and hardware supervisor latencies to evaluate whether the integrated machine satisfies its operational safety case.



Checkpoint 14.8: Evaluation record schemas and provenance guarantees

Before examining common evaluation fallacies, verify your understanding of structured qualification logs:

Downstream: Non-asymptotic Clopper-Pearson trial bounds supply the empirical evidence nodes for safety cases in section 32.3.

- ❑ **Cryptographic component hashing:** Can you explain why evaluation records must capture SHA-256 digests of neural weights, motor firmware, and sensor calibration matrices?
- ❑ **Full-denominator telemetry audit:** Can you describe how logging raw per-trial outcome counts prevents silent filtering of safety supervisor interventions?
- ❑ **Unsupported coverage catalogs:** Can you distinguish supported empirical claims within an evaluated ODD from explicit catalogs of unvisited state spaces?

With evaluation logging protocols established, section 14.9 reviews the operational fallacies and pitfalls that emerge when practitioners misinterpret benchmark metrics or extend evidence beyond tested operational envelopes.

14.9 Fallacies and Pitfalls

An evaluation supports claims only within its tested conditions and limits. These mistakes arise when teams extend a result beyond those limits or attribute an architectural safeguard to the learned policy.

Fallacy: *Repeated success along a single nominal test route proves broad operational readiness.*

A delivery rover completes 20 trials on the same dry path (friction coefficient $\mu \approx 0.8$) with fixed lighting and payload. Even under the independent-trial assumptions of the binomial model, zero failures give a one-sided lower bound of $p_{\text{lower}} = 0.05^{1/20} \approx 0.861$ (86.1 percent) at 95 percent confidence. That bound describes the tested regime; it does not extend the evidence to wet pavement ($\mu < 0.3$), high-inertia payloads exceeding nominal actuator torque margins, or moving obstacles. Repeating the route can improve an estimate for nominal conditions while leaving deployment-critical safety envelopes untested. A readiness claim must identify the covered SE(3) workspace and operational envelope, allocating trials to its relevant physical variations alongside reporting uncertainty within each tested regime.

Fallacy: *Zero failures in 1,000 trials proves compliance with a 99.9 percent reliability specification.*

A pick-and-place policy completes 1,000 independent trials without a payload drop, but its requirement is 99.9 percent success at 95 percent confidence. The observed success fraction is perfect; the one-sided lower bound is only $p_{\text{lower}} = 0.05^{1/1000} \approx 0.9970$, below the required rate. With no failures and the same statistical assumptions, approximately 2,995 trials ($n = \ln(0.05) / \ln(0.999)$) are needed to meet that confidence requirement. The gap is an evidence deficit, not a policy failure. Engineers must choose the trial budget from the reliability claim and confidence level in advance, then report the statistical lower bound rather than equating a zero-failure record with absolute certainty in the physical system.

Fallacy: *High offline action agreement on static logs guarantees closed-loop physical stability.*

A corridor-navigation policy achieves 98.8 percent agreement with expert actions on 50,000 recorded examples, yet exceeds its cross-track error envelope and collides in three of five physical runs. Offline evaluation supplies each observation from the expert's trajectory regardless of the policy's previous prediction. Deployment instead feeds back the consequences of the steering commands admitted by the independent permission tier. A persistent steering bias of even $+0.05$ rad moves the robot into SE(3) states that the static comparison never tested, where subsequent errors compound via covariate shift. High action agreement therefore supports a claim about imitation on those records, not closed-loop physical stability. Physical evaluation must let the policy influence later observations and assess whether it recovers from the deviations it creates within the physical control loop.

Fallacy: *Surviving hardware transport stalls is an emergent property of learned neural weights.*

A robot completes 50 nominal runs, and the team credits its learned policy with tolerance to SerDes bus jitter. A qualification test then injects a 15.0 ms mailbox stall. Extending the proposal-permission architecture from section 6.1, the proposal's $\tau_{\text{lease}} = 10.0$ ms freshness lease expires, and the real-time microcontroller detects the stale lease and initiates a Category 1 stop via maximum braking torque. That response comes from the enforcement architecture; nominal completion records do not

establish that the neural weights detect or compensate for the stall. Fault injection must measure the hardware supervisor's detection and stopping behavior against the available stopping distance margin, and the evaluation report must attribute the protection to the hardware mechanism that supplied it.

14.10 Summary

Closed-loop evaluation measures the coupled policy, permission path, plant, environment, and trial protocol. A zero-failure confidence limit describes the tested distribution under its sampling assumptions; it does not guarantee that unseen conditions are safe. Independent physical constraints must be validated against sensing error, response latency, and stopping clearance.

The twenty-run result is a one-sided 86.1 percent success lower confidence limit, or 13.9 percent failure upper limit, only for independent draws from the declared test distribution. It is neither an observed failure frequency nor permission to deploy. A fixed quota across lighting or friction conditions requires per-stratum inference and declared target weights; wear or shared thermal state can invalidate a pooled iid calculation.

The corridor example shows the different question closed-loop testing answers: a persistent one-degree heading offset scores a small error on recorded centerline states, yet admitted motion reaches the wall in 8.6 seconds. Trials must include the permission tier, log every intervention, and report what remains untested. A runtime OOD detector can miss unfamiliar hazards; independent physical constraints and a feasible fallback must carry the safety argument.

The evaluation handoff has four reviewable artifacts:

1. *Bounded Operational Claim*: Specifying the precise operational design domain (ODD) boundaries where performance was empirically characterized.
2. *Reproducible Evaluation Record*: Documenting the complete physical and computational configuration of every test, including cryptographic digests of weights, firmware, and calibration matrices.
3. *Explicit Coverage Gap Catalog*: Detailing unverified corner cases, omitted environmental variations, and known failure modes.
4. *Runtime Monitoring Specifications*: Defining the telemetry channels, latency bounds, OOD reject gates, and divergence thresholds required to detect anomalous behavior at runtime.



Key Takeaways: Closed-loop evaluation and survival reliability

- **Success Rates Belong to Closed-Loop Systems, Not Weight Files**: A measured 98 percent completion rate reflects the coupled dynamics of sensor exposure latency, tire friction, battery charge state, and mechanical backlash. Changing physical conditions changes the estimand and may require a new evaluation.
- **Closed-Loop Feedback Unmasks Compounding Error**: An open-loop validation error of 1.0° (0.0175 rad) evaluates static predictions without state perturbation. As derived in section 14.3, a stipulated persistent steering error yields linear lateral drift (17.5 cm over 10 m), reaching the illustrative wall clearance before the transit ends unless another barrier intervenes.
- **The Fleet Testing Fallacy**: Pooling physical hours across a fleet still requires the Butler-Finelli exposure assumptions ($T \geq -\ln(1 - C)/\lambda$), and shared environmental, manufacturing, or thermal conditions violate statistical independence, limiting what passive mileage alone can establish about rare hazards.
- **Finite Samples Leave Wide Failure-Probability Bounds ($3/N$ Rule)**: Achieving zero failures in $N = 1000$ iid trials gives an approximate one-sided 95 percent failure upper confidence limit of $3/N \approx 0.3$ percent—a gap relative to a 99.9 percent success lower-limit target (2,995 zero-failure runs under the same assumptions).
- **Physical Fail-Safety is an Architectural Property**: When injected communication stalls or out-of-distribution observations occur, safety depends on a separately validated per-

mission path, feasible fallback, and physical barriers, never by the adaptability of learned neural weights.

- **The Evaluation Deliverable Quadrant:** A policy evaluation handoff should include four reviewable artifacts: Bounded Operational Claim (ODD), Reproducible Evaluation Record, Explicit Coverage Gap Catalog, and Runtime Monitoring Specifications.

A release decision needs the bounded evaluation record and evidence that the independent permission path can reject hazardous proposals or reach a feasible fallback within the measured physical margins.



What's Next: From closed-loop benchmarks to runtime perception

With this evaluation discipline in place, **Part II (Teaching the Machine)** concludes. Across data collection (chapter 10), policy training (chapter 12), and closed-loop qualification in this chapter, we have established that empirical mileage accumulation cannot eliminate catastrophic tail risk. We now enter **Part III (Running the Machine)**, shifting from training to the real-time runtime pipeline that governs execution when a permitted command moves physical mass. The journey begins in chapter 16, where raw sensory photons captured at camera SerDes (Serializer/Deserializer) lanes and IMU registers are ingested, timestamped, and converted into metric $SE(3)$ spatial claims (rigid 3D positions and orientations) under strict age-of-information budgets.



RUNNING THE MACHINE

Running the Machine

As a mobile robot approaches an obstacle, a camera frame locates a person near its path. The planner proposes a turn, but the permission path must decide whether that observation is still fresh and whether the remaining clearance permits the turn or a stop. The decision has a deadline: another polished trajectory is useless if the robot has already crossed the braking margin. Running a physical AI machine means reconciling the timescales of sensing, planning, and actuation with that physical deadline. Human goals may change over seconds, trajectories over tens of milliseconds, and a particular motor controller may update every millisecond. The required rates and response times come from the plant and its measured delays, not from the names of the software tiers.



Principle 7: The multi-rate authority hierarchy

Invariant: A learned policy or planner proposes motion; an independent permission path controls whether that proposal reaches the actuators. Its deadline must account for observation age, computation, communication, actuator response, and available braking or clearance margin. A 1 kHz microcontroller loop is one implementation with a 1 ms period, not a microsecond response guarantee.

Implication: Deliberation can run more slowly than local feedback, provided the permission path can reject stale or infeasible proposals in time. A control-barrier-function (CBF) check implemented with quadratic programming (QP) can support this design only within a validated dynamics model and state estimate, with a feasible intervention, sufficient actuator authority, and a demonstrated deadline.

Part III follows the observation, proposal, permission decision, and physical response through six stages. Their interfaces implement the Sense–Plan–Act (S·P·A) decisions introduced in section 2.6 and developed in Appendix D. The center, $S \cap P \cap A$, is the permission decision that brings sensing age, proposed motion, and physical response into one budget; compute placement supports that decision across all three tasks.

- **Perception (chapter 16, Sense):** Timestamp and calibrate camera and motion measurements so the robot can decide whether an object’s location is current enough to use.
- **Memory (chapter 18, $S \cap P$):** Track an object through occlusion, carry its uncertainty forward, and require a new observation when the motion margin no longer supports the estimate.
- **Intent (chapter 20, $S \cap P$):** Turn a human goal into a spatial, time-limited proposal without granting the language model actuator authority.
- **Planning (chapter 22, $P \cap A$):** Generate continuous motion and check any smooth trajectory or stopping suffix against available braking and clearance before it is admitted.
- **Enforcement (chapter 24, $A \cap S$):** Use fresh state and a separately validated permission path to admit, modify, or reject proposed motion before the actuator command crosses the causal boundary.
- **Placement (chapter 26):** Allocate compute, memory, power, and thermal resources so inference traffic cannot make the permission path miss its measured deadline.

III Principle 8: Age-limited physical belief

Invariant: An unobserved dynamic object's position estimate must carry an age and uncertainty bound; uncertain motion can widen that bound. Under an illustrative constant process-noise model,

$$\sigma^2(t) = \sigma_0^2 + Q \cdot (t - t_{\text{meas}})$$

where σ_0^2 is position variance at measurement time t_{meas} , $t - t_{\text{meas}}$ is the nonnegative observation age ($t \geq t_{\text{meas}}$), and $Q \geq 0$ is the assumed position-variance growth per unit time. This model does not prescribe exponential confidence decay or apply unchanged to every object and sensor.

Implication: Revalidate or expire a belief when its position-uncertainty bound exceeds the task's clearance or contact margin. Time-limited intent proposals similarly require renewal before use. On expiry, the permission path rejects the stale proposal and selects a validated fallback; a safe stop is available only if the current speed, clearance, braking capability, and response delay still permit one.

III Principle 9: Heterogeneous substrate isolation

Invariant: Shared compute, memory, power, or thermal resources must not delay a safety-critical permission decision beyond its validated deadline. Bursty neural inference can interfere with a real-time path on a shared system-on-chip, so the interference must be measured under the intended workload.

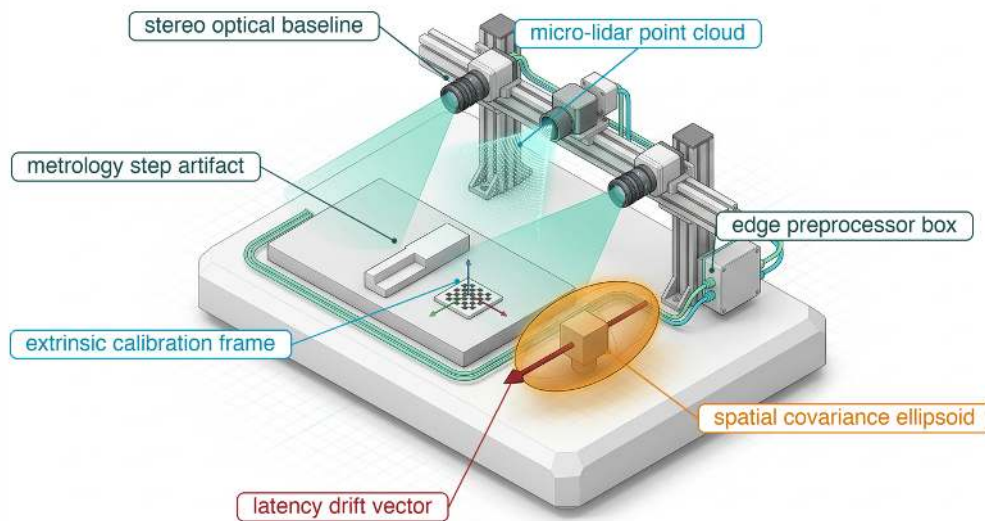
Implication: Choose scheduling, memory quality-of-service controls, cache reservation, separate rails, bare-metal execution, or a separate controller according to measured worst-case timing and power budgets. Verify that the chosen isolation preserves the permission deadline during inference bursts and faults.

The six stages keep learned proposals connected to measurements and constrained by the machine's available physical margin.

Sensor Perception

DRAFT

Sensor Perception

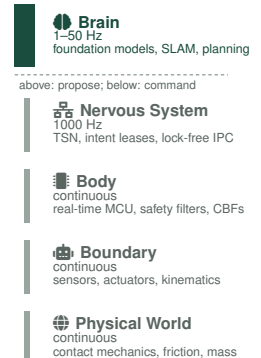


- 16.1 Photons to Spatial Claims
- 16.2 What Perception Hands Over
- 16.3 Sensor Transduction and Calibration
- 16.4 The Cost of Ingestion
- 16.5 Spatial Error and Clearance Bounds
- 16.6 How Observations Go Wrong
- 16.7 The Observation Contract
- 16.8 Fallacies and Pitfalls
- 16.9 Summary

Purpose

Why does an obstacle detection with 99 percent recognition accuracy still cause a collision if its capture timestamp, coordinate frame, and spatial uncertainty are ignored?

An obstacle location aids control only when its time, coordinate frame, and uncertainty are known. The world continues moving while a sensor captures an image, transfers it through memory, and waits for a model to interpret it. An accurate estimate can describe a position that is no longer current when the controller acts. Calibration errors and vibration introduce a different uncertainty, changing the relationship between a measurement and the physical space it represents. Reporting an object label without those qualifications hides information needed to judge clearance. Perception must therefore preserve when and where an observation originated and communicate the limits of the resulting spatial estimate. Recognition alone cannot repair stale or poorly calibrated measurements. In physical AI systems architecture, perception connects sensor behavior and processing delay to the geometric claims that the brain uses and the nervous system must evaluate before permitting motion.





Learning Objectives

- Specify observation records that preserve acquisition time, coordinate frame, and measurement uncertainty
- Calculate information age across the physical sensing, transport, preprocessing, and inference pipeline
- Diagnose spatial errors caused by sensor timing, motion during capture, and misaligned coordinate frames
- Estimate capture bandwidth requirements and their effect on shared memory and computation resources
- Evaluate whether observation uncertainty leaves enough physical clearance for a proposed action
- Distinguish perception failures that runtime checks can detect from errors that remain silent

16.1 Photons to Spatial Claims

A vision foundation model on an autonomous mobile robot can achieve a 98.4 percent mean Average Precision score yet still drive into a steel stanchion. The world moves while the camera reads out and the inference pipeline processes its image. If those stages consume eighty milliseconds while the vehicle travels at two meters per second, the robot advances sixteen centimeters before the updated obstacle estimate arrives. Recognition accuracy does not say whether that estimate is fresh enough for the proposed motion. In physical AI, an observation has an expiration date.

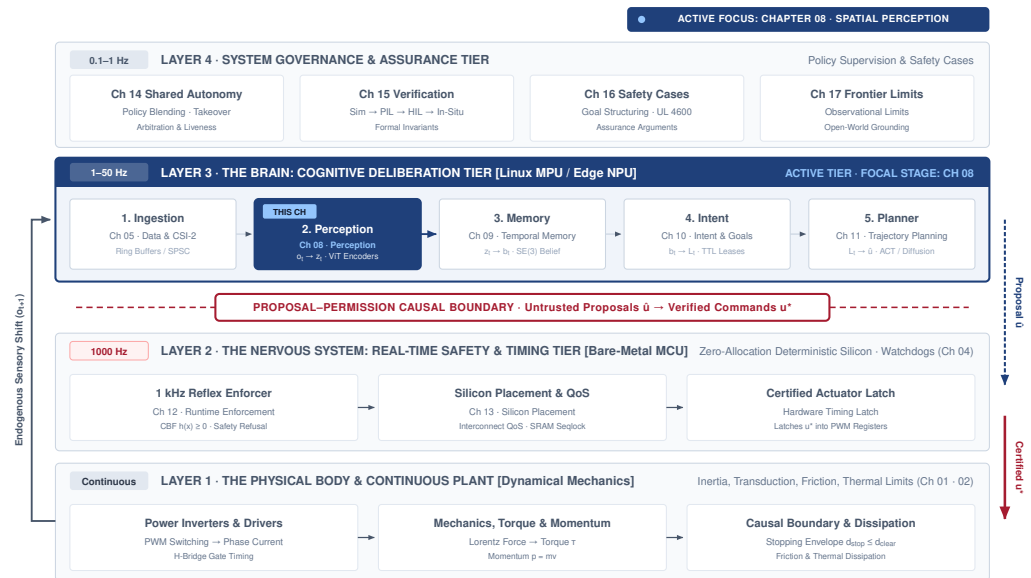


Figure 16.1: Perception stage architecture focus: The four-tier physical AI systems architecture highlights the cognitive deliberation tier, with perception as the second brain stage and this chapter's active focus. The stage converts raw observations o_t into latent state z_t , bounded upstream by sensor transduction it cannot improve and downstream by the memory that must store whatever it infers.

Prerequisite: The dual-brain proposal-permission privilege boundary is architected in section 2.9.

Within the four-tier systems hierarchy (figure 16.1), **Perception** represents the sensory gateway of Layer 3: the Cognitive Brain. It transforms sensor energy—photons, time-of-flight pulses, and reflected radio waves—into dated spatial claims, but holds no actuator authority. One implementation

runs that pipeline on an application processor at 10–50 Hz and places an independent permission path on an MCU with a 1 ms cycle.¹ The permission path must reject a claim after its freshness limit and select a validated response that remains feasible with the current sensing, braking, and clearance margin.²

 Definition 16.1: Transduction-to-actuation information age

Transduction-to-actuation information age is the total physical time elapsed from the midpoint of photon integration or acoustic sensing on a physical transducer to the mechanical current rise in the motor stator windings, defining the true physical freshness of an observation.

1. **Significance:** Quantifies the full end-to-end cyber-physical loop delay, determining the phase margin erosion and maximum stable operating speed of the embodied machine.
2. **Distinction:** Unlike compute latency or sensor FPS, transduction-to-actuation age spans the entire physical interval from sensor photon integration through neural inference, network bus transport, and motor inverter PWM switching to electromagnetic torque generation.
3. **Common pitfall:** Optimizing neural network inference time on an accelerator while ignoring sensor exposure time, DMA serialization latency, and motor current rise time, which often dominate the total delay budget.

A physical architecture can take many concrete shapes. An end-to-end vision-to-action policy (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023; Zhao et al. 2023) maps camera observations to proposed joint torque actions within a single neural network; an independent permission path must still decide which commands reach the actuators. A classical modular pipeline partitions that same computation across independent processes for sensor ingestion, state estimation, mapping, trajectory generation, and motor control (a paradigm historically challenged by behavior-based subsumption architectures (Brooks 1986b)). While these implementations look entirely distinct on a software architecture diagram, they represent the same machine analyzed at different levels of granularity. As David Marr established in his foundational formulation of computational vision (Marr 1982), visual perception must be understood across three distinct levels: the computational theory (what problem the visual system solves and why), representation and algorithm (how sensory inputs are transformed from raw 2D intensity maps into intermediate 2.5D sketches and 3D spatial models), and hardware implementation (how physical silicon, photodiode arrays, and neural circuits physically execute the algorithms). In physical AI, these responsibilities represent non-negotiable physical tasks that must be discharged somewhere along the signal path between physical stimulus and physical response.³

When an engineering team combines these responsibilities in an end-to-end model, the responsibilities remain, but their intermediate representations become harder to inspect independently. An explicit interface can expose a coordinate transform for checking, carry an expiration deadline, or let another component reject an inconsistent estimate. A hidden representation requires additional instrumentation or external checks to provide comparable evidence. If perception mistakes a wet-floor reflection for open clearance, the engineer needs a way to detect or contain the resulting error regardless of how many software modules implement the pipeline.

That evidence begins where physical energy converts into digital data. From transduction at the detector array to consumption by a state estimator, every processing stage adds information age. Consider an autonomous mobile base operating in an industrial facility at 2.0 m/s. Photons striking the photodiode array of a CMOS camera do not instantly appear as feature embeddings in an inference engine. In an analytical breakdown of a typical embedded vision system, the sensor array requires 33 ms for exposure and pixel readout at 30 Hz. Direct memory access transfers across a serial bus into system memory add 8 ms. Operating system kernel scheduling and driver buffer handling introduce 4 ms of queuing delay. Image debayering, rectification, and color correction require 5 ms of preprocessing time. Finally, running a vision backbone on an onboard accelerator requires 25 ms at P_{99} latency.

¹ | **Proposal-permission partition:** An independently clocked permission controller with its own state and memory path can continue checking actuator requests when host inference stalls. A 1 ms loop period does not establish its full detection-to-brake response; that response must be measured with bus and actuator delays. See section 6.1.

² | **Control Barrier Functions (CBFs):** A CBF check can constrain admitted motion under a validated model, state estimate, feasible control set, actuator authority, and deadline. Stale or unbounded perception requires refusing the affected proposal; the controller selects a validated fallback, which may be a stop only if remaining clearance and braking capability permit it. See section 24.7.

³ | **Marr's tri-level hypothesis:** Marr (1982) established that information processing systems must be understood at three distinct tiers: computational theory, representation and algorithm, and physical hardware implementation. In embodied AI, optimizing perceptual representations without evaluating physical bus and silicon memory constraints violates this tri-level hierarchy. Deploying dense vision backends on embedded edge NPUs requires co-designing spatial representations with available memory bandwidth.

By the time the perception stage yields an observation tensor, 75 ms of total latency have accumulated between photon capture and model consumption. Over that 75 ms window, the vehicle has traversed 0.15 m of physical space. If the perception stage hands downstream components an object detection bounding box without an explicit timestamp and a calibrated spatial transform, a downstream planner will evaluate that obstacle relative to the current position of the chassis, assuming the obstacle sits 0.15 m closer or further than its true coordinate. Downstream software cannot safely consume a perception output that does not explicitly declare when it was captured, what coordinate system it references, and how wide its error distribution remains.

Prerequisite: Transduction freshness and sensor measurement aging are derived in section 4.2.

Definition 16.2: Spatial information age

Spatial information age (τ_{age}) is the total elapsed physical time between the physical energy integration midpoint ($t_{\text{exp}}/2$) at the transducer and the downstream consumption of the resulting observation tensor by a state estimator or control policy.

1. **Significance:** Because physical mass moves while digital computation executes, sensory latency translates directly into unmodeled spatial displacement (equation 16.1):

$$\Delta \mathbf{x} = \int_{t_0}^{t_0 + \tau_{\text{age}}} \mathbf{v}(t) dt \quad (16.1)$$

This continuous physical shift depletes geometric safety buffers before an obstacle is even classified.

2. **Distinction:** Unlike software pipeline latency (wall-clock duration of a forward pass), spatial information age accounts for the full physical journey: photon exposure time, rolling shutter readout, SERDES bus transport, ISP debayering, OS buffer queuing, and neural tokenization.
3. **Common pitfall:** Treating the frame arrival timestamp (t_{recv}) in userspace memory as the observation time (t_0). Using arrival time hides upstream capture and transport delays, introducing severe phase lag and high-frequency tracking instability into feedback controllers.

16.1.1 Temporal batching and phase margin erosion

Batching improves accelerator throughput by amortizing work across images, but collecting a batch across successive capture times ages its earliest member. At a 30 Hz camera cadence ($T_{\text{sample}} \approx 33.3$ ms), four consecutive frames wait approximately 100, 67, 33, and 0 ms, respectively, before the batch is ready. At 10.0 m/s the oldest frame corresponds to another meter of forward travel before processing begins. Such temporal batching is inadmissible when its resulting end-to-end age exceeds the motion-dependent freshness budget. Images captured together by several cameras may instead be batched if measured age, synchronization error, and deadline remain within that budget.

Phase margin is the phase delay a feedback design can tolerate before its nominal gain-crossover condition reaches zero margin; it is not a universal property of a perception model. For a loop with nominal margin 45° (0.785 rad) at crossover $\omega_c = 10$ rad/s, the delay-only ceiling is $0.785/10 = 78.5$ ms. Exposure, processing, transport, permission, and actuator delay all spend that same budget. The transfer-function derivation is developed in Appendix B. In the Laplace domain, an additional pure delay contributes $e^{-s\tau_{\text{age}}}$. Substituting $s = j\omega$, its frequency response has unit gain ($|e^{-j\omega\tau_{\text{age}}}| = 1$) but contributes negative, frequency-proportional phase lag:

$$\Delta\phi(\omega) = -\omega\tau_{\text{age}} \quad [\text{rad}] \quad (16.2)$$

At the open-loop gain crossover frequency ω_c (the frequency where open-loop gain $|L(j\omega_c)| = 1$), the phase lag in equation 16.2 directly erodes the system's nominal phase margin (PM_{nom}):

$$\text{PM}_{\text{effective}} = \text{PM}_{\text{nom}} - \omega_c\tau_{\text{age}} \quad (16.3)$$

Under the fixed-loop, pure-delay approximation in equation 16.3, $\tau_{\text{age}} \geq \text{PM}_{\text{nom}}/\omega_c$ exhausts the nominal phase margin. That is a warning to re-evaluate the complete closed-loop response, not a

proof of a particular oscillation or pole location. Reducing one latency component from 80 ms to 40 ms restores $0.040\omega_c$ radians of modeled phase margin only if crossover and the rest of the loop remain unchanged.

The handoff to control starts with a timestamp, a coordinate frame, and a usable error bound. The handoff record in section 16.2 defines that contract; the following sections trace how transduction, ingress, and geometric estimation can invalidate it before the permission path admits a motion.

16.2 What Perception Hands Over

Raw sensor hardware yields electrical measurements such as photodiode voltages, time-of-flight counter values, or capacitive displacements. These unstructured signals differ significantly from the state variables needed for downstream autonomy. An actuator controller cannot track a desired force using a two-dimensional grid of raw pixel intensities, and a trajectory optimizer cannot plan collision-free paths using unorganized time-of-flight returns. Downstream estimation, planning, and memory subsystems require structured representations of physical state, including metric positions, velocities, bounding volumes, surface normals, and semantic classifications. Translating raw sensor streams into these representations is the operational responsibility of the perception pipeline. However, downstream consumers cannot safely interpret a perceived state without explicit metadata that anchors the measurement in physical reality. An observation must carry three properties for downstream software: a capture timestamp, a reference frame, and an uncertainty envelope.

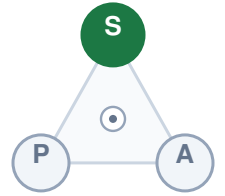
The first mandatory property is the **capture timestamp** t_0 , which specifies the temporal midpoint of the physical energy integration window (the effective moment the camera shutter was open or the laser pulse fired). Sensors do not take instantaneous snapshots; a camera integrates photons across an exposure duration, and a mechanical lidar sweeps laser pulses over an azimuth arc across several milliseconds. Downstream state estimators require t_0 to integrate the measurement into the machine's dynamic equations of motion. A common architectural failure in embedded systems is recording the arrival timestamp t_{recv} , the moment the operating system kernel completes the direct memory access transfer, and passing t_{recv} downstream as the observation epoch. Because pipeline execution, serial bus transfers, and driver scheduling introduce a transport delay $\tau = t_{\text{recv}} - t_0$, substituting arrival time for capture time corrupts the state estimator.

The magnitude of this corruption is a direct function of platform kinematics. Consider a vehicle traversing a trajectory with instantaneous velocity $v(t)$. When an observation taken at t_0 is evaluated by a downstream filter at t_{recv} , the physical state of the machine has shifted. The resulting spatial tracking error Δx is the integral of the platform velocity over the transport delay, $\Delta x = \int_{t_0}^{t_0+\tau} v(t) dt$. In an analytical case of constant linear velocity v , this relationship simplifies to $\Delta x = v \cdot \tau$. The Tempe crash analysis (section 2.7) shows why perception and prediction failures matter; the following robot calculation is separate and illustrative. At $v = 3.0$ m/s with an assumed $\tau = 80$ ms uncompensated observation delay, the robot moves $\Delta x = 0.24$ m between capture and use. In a feedback loop, that delay adds phase lag; whether it produces oscillation or instability depends on the controller and its remaining phase margin.

The second mandatory property is the **reference frame** identifier $\mathcal{F}_{\text{sensor}}$. Every spatial measurement is natively resolved in the local coordinate system of the physical transducer. A camera measures bearing angles relative to its optical center, while a radar measures range and Doppler velocity relative to its antenna array. Downstream trajectory planners and obstacle maps operate in a chassis body frame $\mathcal{F}_{\text{body}}$ or an inertial navigation frame $\mathcal{F}_{\text{world}}$. Transforming an observation from $\mathcal{F}_{\text{sensor}}$ to $\mathcal{F}_{\text{body}}$ requires applying a rigid-body spatial transformation consisting of a rotation matrix and a translation vector that define the sensor mounting extrinsics (the exact physical location and orientation where the sensor is bolted onto the machine).⁴ Across the three machine archetypes, this spatial binding takes distinct physical forms: in Mobile Systems, $\mathcal{F}_{\text{body}}$ represents the mobile chassis `base_link` tracking through an odometric frame `odom`; in Manipulator Systems, it maps across serial kinematic chains from the mounting flange `too10` through articulated joint angles to the base origin `base_link`; in Process / Energy Systems, it maps local pyrometry or laser galvanometer scan coordinates to the fixed workcell or reactor vessel datum $\mathcal{F}_{\text{vessel}}$. As structured in the coordinate frame transformation tree (figure 16.2), the standard spatial hierarchy formalizes this spatial progression by nesting coordinate frames: $\mathcal{F}_{\text{earth}} \rightarrow \mathcal{F}_{\text{map}} \rightarrow \mathcal{F}_{\text{odom}} \rightarrow \mathcal{F}_{\text{body}} \rightarrow \mathcal{F}_{\text{sensor}}$.



Under a fixed crossover model, additional age spends the loop's nominal phase margin.



The Sense axis: transforming un-compressed sensor photons into calibrated spatial claims under strict freshness budgets.

Downstream: Extrinsic sensor frames feed the hierarchical spatial transform tree in section 18.4.

⁴ | **REP 105 spatial frames:** REP 105 assigns distinct roles to global map, locally continuous odom, and body-fixed `base_link`; earth and a sensor frame extend the example where used. A local controller should consume an appropriate continuous estimate and reject unexpected transform jumps. The effect of a jump on torque depends on the controller and its limits.

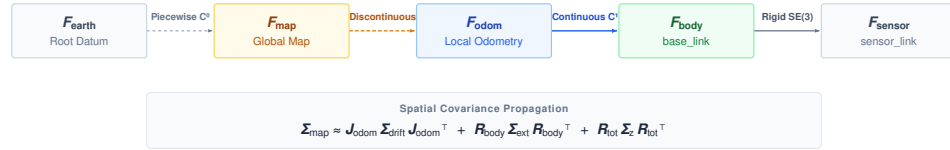


Figure 16.2: Coordinate frame tree and uncertainty: The spatial coordinate frame hierarchy formalizes the separation between continuous local dead reckoning ($\mathcal{F}_{odom} \rightarrow \mathcal{F}_{body}$) consumed by high-frequency tracking loops and discontinuous global map corrections ($\mathcal{F}_{map} \rightarrow \mathcal{F}_{odom}$) consumed by global planners. Spatial covariance compounds across the kinematic chain ($\Sigma_{map} \approx J_{odom} \Sigma_{drift} J_{odom}^T + R_{body} \Sigma_{ext} R_{body}^T + R_{tot} \Sigma_z R_{tot}^T$), requiring downstream consumers to reject observations that lack an explicit coordinate key $\mathcal{F}_{sensor}$, acquisition epoch t_0 , and covariance envelope Σ_z . Attribution: Book kinematic systems diagram, CC BY 4.0.

When perception outputs omit the frame identifier or rely on nominal CAD dimensions rather than calibrated physical extrinsics, small angular offsets create large spatial errors that scale with distance. An uncalibrated pitch or yaw error of $\theta = 1.0^\circ$ on a sensor detecting an object at a range $d = 15.0$ m induces a transverse position error of $\Delta y \approx d \cdot \sin(1.0^\circ) \approx 0.26$ m. In tight warehouse aisles or cluttered physical environments, a lateral error of 0.26 m causes the planner to steer into physical racking or reject valid corridors as obstructed.

The REP 105 frame convention separates a locally continuous odom pose from a globally corrected map pose. The roles in table 16.1 keep those roles visible without assigning a universal covariance law or differentiability class. An implementation still has to check transform timestamps, estimator uncertainty, and whether its controller consumes a frame that may jump.

Table 16.1: REP 105 frame roles for a mobile robot: A compact example of global, local, body, and sensor frames. REP 105 requires odom-relative pose to remain continuous but does not prescribe C^1 smoothness, a covariance growth rate, or fixed accuracy; these require estimator-specific evidence.

Frame	Parent in this example	Meaning	Consumer rule
earth	root	Optional geodetic datum	Verify datum before global routing
map	earth if present	Global localization; corrections may jump	Keep relocalization jumps out of local feedback
odom	map	Locally continuous pose estimate; may drift	Track motion locally; monitor uncertainty and discontinuities
base_link	odom	Body-fixed origin	Body pose still inherits odometry uncertainty
sensor_link	base_link	Calibrated sensor mounting frame	Apply time-valid extrinsics and their uncertainty

The third mandatory property is an **uncertainty representation**, such as covariance Σ_z together with a validated tail or hard error bound when clearance depends on it. Low illumination, atmospheric scattering, specular surfaces, and model error change the residual distribution. A covariance describes second moments, not a region guaranteed to contain the true state. Estimators can use it to weight innovations against a motion model; an underestimated covariance can pull a state estimate toward a spurious measurement and lead to poor downstream commands. Safety decisions must also account for bias, missed detections, and validation limits.



Systems Perspective 16.1: The three invariants of observation

Pass a perception claim with its capture time and clock domain, coordinate frame, and qualified uncertainty. Refuse a claim when any of those cannot be checked for the proposed motion.

These three properties define the **perception handoff contract**, the formal interface invariant between perception and the remainder of the autonomy stack. Downstream state estimators, planners, and safety watchdogs must reject any observation packet that lacks an explicit capture timestamp

t_0 , a verified coordinate frame $\mathcal{F}_{\text{sensor}}$, and an uncertainty covariance Σ . Dropping an incomplete or unbounded observation allows a downstream estimator to propagate its state forward using physical dynamics, degrading uncertainty smoothly over time. Accepting an ungrounded packet, by contrast, injects unmodeled delays, spatial distortions, and false confidence directly into the control loop. Enforcing this contract requires tracing how an observation originates, beginning at the hardware interface where physical phenomena are first sampled.

16.3 Sensor Transduction and Calibration

Lag accumulates before software receives anything at all, beginning at the photosite where physical phenomena are converted into electrical charge. A camera does not capture an instantaneous snapshot. Photons collect on a photodiode over a finite exposure duration t_{exp} , which means the recorded pixel intensity represents the temporal integral of scene radiance $I(t)$ over that interval:

$$I_{\text{pixel}} = \int_0^{t_{\text{exp}}} I(t) dt$$

To build intuition for exposure latency, consider mounting an open **rain gauge on the roof of a moving train**. To measure rainfall intensity, the gauge must remain open to collect falling raindrops over a finite time window (t_{exp}). If the train is stationary, the collected water reflects the rainfall rate at that location. But if the train is hurtling down the tracks at high speed, the water gathered in the bucket represents an average of raindrops collected across an entire stretch of track! You cannot assign the water in the gauge to the train's final arrival position; at best, the measurement reflects the physical midpoint where the train was halfway through the exposure interval ($t_{\text{exp}}/2$). In physical AI, a CMOS photodiode is a photon bucket collecting charge over time. At 2.0 m/s with a 33 ms exposure duration, the camera does not capture where the robot *is*; it captures where the robot *was* 16.5 ms ago, smeared across 6.6 cm of physical motion blur.

When the sensor or the observed environment moves at velocity v during exposure, a sharp physical boundary—such as a rigid door frame—smears across multiple photosites. The resulting motion blur width in the direction of motion is given by $\Delta x_{\text{blur}} = v \cdot t_{\text{exp}}$. Because light accumulates continuously across the exposure window, the effective point in time that corresponds to the accumulated charge is the temporal midpoint of the interval, introducing an unavoidable physical measurement lag of $t_{\text{exp}}/2$ relative to the start of exposure.

16.3.1 Rolling shutter distortion and motion compensation

The exposure window schedule across the pixel array determines the spatial geometry of the image. In a **global shutter** sensor, every pixel photodiode accumulates charge during the identical time interval and transfers the accumulated electrons to shielded storage nodes simultaneously. In a **rolling shutter** sensor, silicon area and cost constraints dictate that rows are reset, exposed, and read out sequentially from the top scanline $y = 0$ to the bottom scanline $y = H - 1$. Each successive scanline begins exposure after a row period t_{row} , creating a progressive temporal offset across the image plane:⁵

$$t(y) = t_0 + y \cdot t_{\text{row}}$$

where t_0 is the start of exposure for the top row. When the body translates at linear velocity v perpendicular to a vertical feature, scanline y captures it at a spatial displacement:

$$\Delta x(y) = v \cdot y \cdot t_{\text{row}}$$

Under rotational motion, rolling shutter distortion becomes non-linear. If the camera undergoes angular rotation at angular velocity vector $\omega = [\omega_x, \omega_y, \omega_z]^T$ (e.g., rapid yaw or pitch during aggressive robot maneuvers), the effective camera attitude changes continuously as scanlines are read. Physically, the camera points in a slightly different direction for every single row. For a scanline at vertical coordinate y , the differential rotation relative to the frame start is $\Delta\theta(y) \approx \omega \cdot y \cdot t_{\text{row}}$. This scanline-dependent rotation warps straight physical lines into curves, induces focal-length dilation (scaling distortion during pitch), and shears vertical walls into slanted surfaces, as demonstrated by the propeller blade curvature in figure 16.3.

⁵ | **Rolling shutter epipolar geometry:** Rolling shutter CMOS image sensors reset, integrate, and read pixel scanlines sequentially across the array rather than exposing all photosites simultaneously. When the camera rotates during this readout interval, straight physical edges project into curved geometric arcs across the image plane. This progressive scanline skew violates rigid multi-view epipolar constraints, causing visual-inertial odometry factor graphs to compute spurious feature depths unless continuous-time B-spline camera trajectories interpolate the instantaneous pose of every scanline.



Figure 16.3: Rolling shutter distortion geometry: A photograph of rotating propeller blades captured by a rolling shutter CMOS sensor. Because sensor scanlines are exposed and read sequentially from top to bottom rather than simultaneously, the rapid physical movement of the blades relative to the row period (t_{row}) shears and curves straight rigid structures into non-linear, disconnected geometric arcs. In high-speed embodied systems, uncompensated rolling shutter skew introduces catastrophic 3D triangulation errors and phantom obstacle intrusion in downstream visual-inertial odometry. (Credit: Wikimedia Commons, CC BY-SA 3.0).

As an analytical example, consider a ground robot traversing an aisle at $v = 3.0$ m/s equipped with a rolling shutter camera operating at 30 Hz with vertical resolution $H = 1080$ rows. If the active readout time across all rows is $t_{\text{readout}} = 33.3$ ms, the inter-row readout interval is $t_{\text{row}} = 33.3 \text{ ms}/1080 \approx 30.8 \mu\text{s}$. A vertical pallet edge spanning the full height of the frame does not appear vertical in the captured array. The bottom scanline ($y = 1079$) is sampled 33.3 ms after the top scanline ($y = 0$), producing a total physical spatial skew of:

$$\Delta x = 3.0 \text{ m/s} \times 0.0333 \text{ s} \approx 0.10 \text{ m} = 100 \text{ mm}$$

With an exposure duration of $t_{\text{exp}} = 10.0$ ms, each individual scanline also suffers a motion blur width of $\Delta x_{\text{blur}} = 3.0 \text{ m/s} \times 0.010 \text{ s} = 0.03 \text{ m} = 30 \text{ mm}$. If downstream geometry reconstruction treats this image as an instantaneous rigid projection, the perceived 100 mm tilt causes the motion planner to identify an artificial intrusion into the free corridor. Correcting this distortion requires **rolling-shutter motion compensation**, the continuous-time geometric unwarping algorithm that utilizes high-rate (1000 Hz) inertial angular velocities (ω) and linear velocities ($\mathbf{v}$) to interpolate the instantaneous 6-DoF camera pose $\mathbf{T}_{wb}(t(y))$ for each individual scanline y , restoring rigid epipolar geometry prior to 3D triangulation.

16.3.2 Camera intrinsic calibration and planar homographies

Each optical sensor requires intrinsic calibration to model the projective geometry mapping 3D physical coordinates to 2D pixel coordinates before multi-sensor calibration. Following the classical **pinhole camera model** (Hartley and Zisserman 2003), a metric 3D point $\mathbf{P}_c = [X_c, Y_c, Z_c]^\top$ defined in the camera optical frame (where $+Z_c$ points forward along the optical axis, $+X_c$ points right, and $+Y_c$ points down) projects onto the 2D pixel coordinate $[u, v]^\top$ via the **camera intrinsic matrix** $\mathbf{K}$:⁶

⁶ | **Camera intrinsic projection:** The pinhole intrinsic matrix $\mathbf{K}$ projects continuous 3D rays in the camera coordinate frame onto discrete 2D photosite coordinates, parameterizing focal lengths (f_x, f_y) and the optical center (c_x, c_y). Because projective mapping collapses the depth dimension, metric distance cannot be recovered from a single monocular ray without auxiliary multi-view triangulation, depth sensors, or learned scale priors. Small focal length calibration errors scale radial projection errors quadratically with distance, corrupting 3D bounding box dimensions. For the rigorous derivation of projective geometry, refer to Appendix C.

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim \frac{1}{Z_c} \mathbf{K} \mathbf{P}_c = \frac{1}{Z_c} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} \quad (16.4)$$

where f_x, f_y represent the focal lengths expressed in pixel dimensions ($f_x = F/p_x$, with focal length F in millimeters and pixel pitch p_x in millimeters per pixel), and (c_x, c_y) is the principal point marking the piercing intersection of the optical axis with the pixel sensor grid (nominally located near the array centroid $(W/2, H/2)$).

To reconstruct metric physical space from an image, physical AI models invert this projective relationship. While forward projection collapses 3D space into a 2D plane (losing absolute distance), the inverse transformation maps each 2D pixel coordinate (u, v) to an **optical bearing ray** $\mathbf{r}(u, v) \in \mathbb{R}^3$ originating from the camera optical center $\mathbf{O}_c = [0, 0, 0]^\top$:

$$\mathbf{r}(u, v) = \mathbf{K}^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{u-c_x}{f_x} \\ \frac{v-c_y}{f_y} \\ 1 \end{bmatrix} \quad (16.5)$$

Any 3D physical surface point projecting onto pixel (u, v) must lie somewhere along this 1D geometric ray, parameterized by its scalar metric depth $d = Z_c$:

$$\mathbf{P}_c(u, v, d) = d \cdot \mathbf{r}(u, v) = \begin{bmatrix} d \frac{u-c_x}{f_x} \\ d \frac{v-c_y}{f_y} \\ d \end{bmatrix}$$

Real-world optical assemblies deviate from ideal pinhole geometry due to lens curvature and misalignment, introducing radial and tangential distortion. In embedded MLSys architectures, these non-linear geometric distortions are parameterized via polynomial coefficients (k_1, k_2, p_1, p_2) and rectified directly in silicon by the hardware Image Signal Processor (ISP) using pre-calculated lookup tables during direct memory access (DMA) ingress, ensuring undistorted frames enter main memory with near-zero CPU overhead ($t_{\text{ISP}} \approx 2.5$ ms). The universal methodology for recovering intrinsic matrix $\mathbf{K}$ and distortion parameters was established by Zhang (Zhang 2000): by imaging a planar calibration target (such as a precision checkerboard or AprilTag grid) across diverse unconstrained orientations, planar homographies provide algebraic constraints that seed a non-linear optimization minimizing reprojection error across all fiducial corners.⁷

16.3.3 Camera-to-IMU temporal and extrinsic calibration

Visual perception does not operate in isolation; it functions alongside an Inertial Measurement Unit (IMU) running at high frequencies (1000 Hz). Fusing high-rate angular velocity ω and linear acceleration $\mathbf{a}$ from the IMU with low-rate visual frames (30–60 Hz) in **Visual-Inertial Odometry (VIO)** requires rigorous spatial and temporal calibration (Furgale et al. 2013; Li and Mourikis 2014; Forster et al. 2017).

The spatial relationship between the IMU reference frame $\mathcal{F}_{\text{imu}}$ and the camera reference frame $\mathcal{F}_{\text{cam}}$ is defined by a rigid extrinsic transformation. The physical separation of the camera and IMU by a lever arm induces additional tangential and centripetal accelerations at the camera due to angular acceleration or velocity experienced by the robot.⁸

Equally critical is the **temporal calibration offset** $\tau_{\text{cam-imu}} = t_{\text{cam}} - t_{\text{imu}}$. Standard commercial crystal oscillators on independent sensor breakout boards drift by roughly ± 50 ppm, accumulating $50 \mu\text{s}$ of error every second (4.3 s per day). Moreover, analog anti-aliasing filters in the IMU, electronic exposure delays in the camera, and DMA driver scheduling inject fixed and variable temporal offsets between the two data streams.

If the temporal offset Δt is uncalibrated, platform rotation couples directly into catastrophic spatial estimation errors. Suppose a robot rotates at an angular velocity of $\omega = 2.0$ rad/s ($115^\circ/\text{s}$).

⁷ | **Zhang's planar homography:** Zhang's calibration method estimates the intrinsic camera matrix $\mathbf{K}$ and radial lens distortion parameters by observing a planar fiducial pattern across multiple unconstrained orientations. Each observation establishes a projective planar homography $\mathbf{H} = \mathbf{K}[\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}]$ that imposes two algebraic orthogonality constraints on the image of the absolute conic. Solving these constraints across at least three non-parallel views yields a closed-form initialization that seeds non-linear Levenberg–Marquardt optimization over all reprojection residuals. For the full mathematical formulation, see Appendix C.

⁸ | **Kinematic lever arm acceleration:** When a rigid body undergoes rotational motion with angular velocity ω and angular acceleration $\dot{\omega}$, a sensor offset by lever-arm displacement $\mathbf{r}$ experiences additional tangential ($\dot{\omega} \times \mathbf{r}$) and centripetal ($\omega \times (\omega \times \mathbf{r})$) accelerations. Failing to compensate for this offset in visual-inertial sensor fusion causes false linear acceleration estimates during sharp turns. This fictitious acceleration corrupts gravity vector estimation, producing systematic pitch and roll drift in the platform attitude filter. For a rigorous derivation of this kinematic lever-arm acceleration, refer to Appendix B.



Figure 16.4: Multimodal autonomous sensor suite: An integrated perception pod carrying: (1) top perimeter LiDAR puck (spinning ToF laser array at 10–20 Hz), (2) downward proximity LiDAR (ground plane clearance and curb detection), (3) multi-exposure high-dynamic-range (HDR) CMOS cameras (>120 dB), and (4) rigid machined bracket and microsecond hardware synchronization bus (< 1, μ s). Extrinsic calibration and hardware trigger lines bound phase skew and rotational projection errors ($\Delta p \approx z \cdot \omega \cdot \Delta t$). Attribution: Open-source robotics testbed platform, CC BY-SA 4.0.

An undetected temporal lag of $\Delta t = 10$ ms between the camera frame and the IMU integration causes a rotational orientation mismatch of:

$$\Delta\theta = \omega \cdot \Delta t = 2.0 \text{ rad/s} \times 0.010 \text{ s} = 0.020 \text{ rad} \approx 1.15^\circ$$

At an operating distance of $z = 10.0$ m, this angular misalignment projects into a transverse spatial error of:

$$\Delta p \approx z \cdot \Delta\theta = 10.0 \text{ m} \times 0.020 \text{ rad} = 0.20 \text{ m} = 200 \text{ mm}$$

This artificial 200 mm displacement corrupts visual feature triangulation, causing visual-inertial estimators to compute false scale factors and driving the filter toward mathematical divergence. Eliminating these timing errors requires hardware-level synchronization: a hardware GPIO trigger strobe latches exposure start times across all cameras and IMU sampling registers simultaneously,⁹ while online continuous-time calibration algorithms estimate residual sub-millisecond offsets $\tau_{\text{cam-imu}}$ within the joint factor graph (Furgale et al. 2013). As shown in the physical multi-modal perception pod in figure 16.4, co-locating multiple high-resolution cameras and LiDAR modules on a rigid vehicle bracket requires routing these deterministic hardware sync strobes alongside high-speed GMSL2/FPD-Link serializer links to bound clock drift and transport latency across all sensing channels.

Camera-to-IMU spatial-temporal calibration is the joint determination of the rigid 6-DoF extrinsic transformation $\mathbf{T}_{\text{imu}}^{\text{cam}} \in SE(3)$ and the temporal latency offset $\tau_{\text{cam-imu}} = t_{\text{cam}} - t_{\text{imu}}$, bounding lever-arm centripetal accelerations and rotational misalignment errors ($\Delta p \approx z\omega\Delta t$) during dynamic motion.

16.3.4 The latency waterfall

Summing these stages yields the total age of sensory data in the perception pipeline. That information age is governed by the **latency waterfall**:

$$t_{\text{age}} = \frac{t_{\text{exp}}}{2} + t_{\text{readout}} + t_{\text{transport}} + t_{\text{DMA}} + t_{\text{ISP}} + t_{\text{backbone}} + t_{\text{IPC}}$$

9 | **Hardware synchronization timers:** Hardware timers (such as Advanced Control Timers TIM1/TIM8 on ARM Cortex-M or ePWM modules on TI C2000) feature dedicated input-capture and output-compare registers that latch microsecond counter values on external trigger edges without CPU software intervention. In multi-sensor perception pipelines, timer trigger lines synchronize image sensor exposure starts with IMU sampling interrupts to maintain sub-microsecond cross-sensor phase alignment, preventing the velocity estimation errors and scale drift that occur when asynchronous visual and inertial data are fused.

Napkin Math 16.1: Transduction latency and information age

The scenario. An embodied platform navigates a dynamic workspace. We evaluate the cumulative information age τ_{age} across the complete sensory-to-inference pipeline and compute the resulting unmodeled physical displacement $\Delta x = v \cdot \tau_{\text{age}} + \frac{1}{2} a \cdot \tau_{\text{age}}^2$ across three distinct physical machine archetypes.

Stage-by-Stage Latency Waterfall:

1. *Exposure Integration Midpoint* ($t_{\text{exp}}/2$): CMOS camera running at 30 Hz with auto-exposure $t_{\text{exp}} = 16.0 \text{ ms} \Rightarrow \Delta t_1 = 8 \text{ ms}$.
2. *Rolling Shutter Line Readout* (t_{readout}): Assume a 1080-row rolling readout contributes $\Delta t_2 = 16.6 \text{ ms}$ for the selected row. This is a scenario timing assumption, not an IMX477 measured readout specification.
3. *MIPI CSI-2 Serialization & Transport* ($t_{\text{transport}}$): The Sony IMX477 flyer specifies a four-lane option up to 2.1 Gbps/lane. This illustrative serial accounting assigns $\Delta t_3 = 0.8 \text{ ms}$ to residual transport after streaming readout; it is an assumption, not a measured device latency or a quotient of peak lane rate.
4. *DMA Ingress & Kernel V4L2 Buffer Queue* (t_{DMA}): Scatter-gather DRAM write and driver interrupt $\Rightarrow \Delta t_4 = 1.2 \text{ ms}$.
5. *Hardware ISP Debayering & Rectification* (t_{ISP}): Dedicated on-chip ISP hardware pass $\Rightarrow \Delta t_5 = 2.5 \text{ ms}$.
6. *Vision Backbone Tensor Forward Pass* (t_{backbone}): INT8 quantized Vision Transformer on edge NPU $\Rightarrow \Delta t_6 = 22 \text{ ms}$.
7. *Covariance Packaging & Zero-Copy IPC* (t_{IPC}): Shared-memory circular ring buffer dispatch $\Rightarrow \Delta t_7 = 0.5 \text{ ms}$.

Total Spatial Information Age:

$$\tau_{\text{age}} = 8\text{ms} + 16.6\text{ms} + 0.8\text{ms} + 1.2\text{ms} + 2.5\text{ms} + 22\text{ms} + 0.5\text{ms} = 51.6\text{ms}$$

Archetype Displacement Analysis:

- **Mobile Systems (Autonomous Mobile Robot):** At nominal cruising speed $v = 2.5 \text{ m/s}$ under acceleration $a = 0 \text{ m/s}^2$:

$$\Delta x_{\text{AMR}} = (2.5\text{m/s}) \times (0.0516\text{s}) = 0.129\text{m} = 129\text{mm}$$

Systems Impact: In the stated forward-obstacle scenario, consumes 129 mm of a 200 mm pre-brake travel reserve, leaving 71 mm for permission, bus, and actuator delay before braking begins. At unchanged speed, those delays together must be below 28.4 ms; braking distance and safety clearance are already reserved separately.

- **Manipulators (High-Speed Industrial Arm):** At end-effector Cartesian velocity $v = 1.8 \text{ m/s}$ during precision peg-in-hole assembly:

$$\Delta x_{\text{arm}} = (1.8\text{m/s}) \times (0.0516\text{s}) = 0.0929\text{m} = 92.9\text{mm}$$

Systems Impact: This displacement exceeds the stated 1–5 mm chamfer tolerance; an uncorrected pose estimate could cause a collision when the tool path passes that close to the fixture.

- **Process / Energy Systems (Laser Metal Additive Melt-Pool Scanner):** Galvanometer scan mirror traversing at $v_{\text{scan}} = 0.8 \text{ m/s}$ with thermal pyrometry:

$$\Delta x_{\text{melt}} = (0.8\text{m/s}) \times (0.0516\text{s}) = 0.0413\text{m} = 41.3\text{mm}$$

Systems insight: The thermal feedback loop may associate laser power with a melt-pool position displaced by > 80 assumed melt-pool diameters ($d_{\text{pool}} \approx 0.5 \text{ mm}$); the process consequence depends on scan geometry and thermal response.

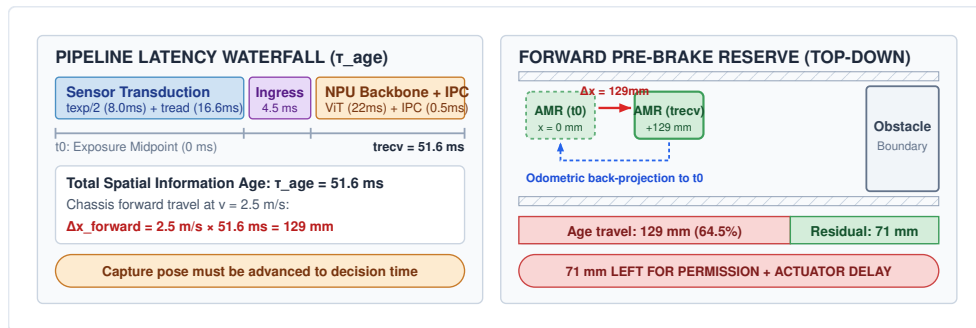
The serial latency example in table 16.2 yields 51.6 ms from capture midpoint to observation dispatch. At 2.5 m/s, the AMR travels 129 mm **forward** in that time. Suppose an obstacle ahead leaves a 200 mm pre-brake travel reserve *after* the required braking distance and final safety clearance have been set aside. Perception age consumes 129 mm of that forward reserve, leaving 71 mm, or 28.4 ms at unchanged speed, for the permission check, bus delivery, and actuator response before braking starts. The vehicle must reject the motion if measured end-to-end age or any of those remaining delays exceeds the validated reserve. A historical observation also needs timestamped pose propagation to the current frame; odometry cannot restore the distance already traveled. Lateral aisle clearance is a separate geometric constraint and is not reduced by forward travel alone.

Table 16.2: Illustrative serial information-age budget: These assumed, nonoverlapping increments produce 51.6 ms for the stated scenario; they are neither measured percentiles nor hard bounds. A real pipeline can overlap readout, transfer, and processing and can acquire different rows at different times. Timestamp the relevant exposure epoch and measure the integrated capture-to-consumer age, including queueing and correlated tails. At 2.5 m/s, this scenario age corresponds to 129 mm of forward travel.

Stage	Illustrative increment	Serial age after stage	What to verify
Exposure midpoint	8 ms	8 ms	Exposure and row timestamp convention
Rolling readout	16.6 ms	24.6 ms	Selected row and overlap with streaming
Residual transport	0.8 ms	25.4 ms	Selected sensor mode and host receiver
DMA and queue	1.2 ms	26.6 ms	Arbitration and queue tail
ISP	2.5 ms	29.1 ms	Mode and competing jobs
Vision backbone	22 ms	51.1 ms	Workload, clock, thermal state
Dispatch	0.5 ms	51.6 ms	End-to-end capture-to-consumer age



Checkpoint 16.1: Spatial information age and corridor depletion



- ☐ **Spatial information age:** Can you explain how physical latency from photon exposure to shared-memory dispatch converts directly into unmodeled kinematic displacement, eroding clearance margins before a control policy can react?
- ☐ **Exposure midpoint versus arrival timestamp:** Can you describe why treating kernel receive timestamps rather than exposure integration midpoints as the measurement epoch induces phase lag and destabilizes closed-loop feedback controllers?
- ☐ **Odometric back-projection:** Can you distinguish between evaluating spatial claims at current chassis coordinates versus back-projecting observations to their historical capture pose using high-frequency IMU dead reckoning?

Bounding this information age requires examining the physical hardware channels that move gigabytes of sensory data from camera serial links into compute memory. In section 16.4, we analyze the cost of sensor ingestion, quantifying direct memory access bandwidth, zero-copy buffers, and memory bus contention against real-time safety deadlines.

16.4 The Cost of Ingestion

Raw pixel streams arriving over MIPI CSI-2 or Ethernet consume DMA and memory service before inference can read them. On a shared application processor, camera writes can also delay other host reads or drop frames. Whether a safety-critical read is affected depends on its actual memory path and arbitration policy. In the independent-controller architecture of section 8.2, the permission tier keeps its own state and deadline; a congested host instead delivers a late or missing observation or proposal, which the permission tier must refuse.

16.4.1 MIPI CSI-2 DMA ingress and zero-copy buffers

Once analog-to-digital converters on the sensor die digitize the pixel charges, the frame must travel across a physical interconnect to host memory. Camera streams enter the unprivileged application processor (Linux MPU) via the **Mobile Industry Processor Interface Camera Serial Interface (MIPI CSI-2)** bus.¹⁰

Systems Perspective 16.2: Silicon ingestion dataflow architecture

1. **D-PHY / C-PHY physical layer:** Physical lanes serialize pixel packets at the selected sensor and receiver's supported rate; packet headers carry error checks.
2. **System-on-chip (SoC) CSI-2 host controller:** Deserializes byte packets, strips headers, and validates CRC integrity.
3. **Scatter-gather DMA controller:** A dedicated DMA engine transfers pixel byte words directly into DRAM over the internal system interconnect (e.g., AXI crossbar).
4. **Kernel DMA ring buffers:** A Linux v4l2 driver can map capture buffers into user space; buffer allocation, cache handling, and later copies depend on the driver and consumer.

Once DMA write completion occurs, the kernel driver issues an interrupt.¹¹ A driver may export a DMA buffer through `dma_buf` so compatible ISP or accelerator paths can share storage without an extra CPU copy. Format conversion, layout changes, or device constraints can still require a copy or transform. MIPI CSI-2, USB3, and GigE Vision each have mode- and implementation-dependent transport, assembly, and queueing delays; measure capture-to-consumer age and its tail on the selected hardware.

16.4.2 Memory bus contention and deadline checks

To see the scale of this competition across physical archetypes, consider their respective ingress demands: an industrial manipulator synchronizing dual high-speed eye-in-hand cameras with tactile arrays (600 MB/s burst ingress); a laser powder-bed fusion process scanner streaming coaxial NIR pyrometry at 1000 Hz (1.2 GB/s continuous ingress); or an autonomous mobile machine equipped with four high-resolution surround cameras and dual lidars. In the mobile configuration, suppose the four global shutter cameras capture 3840×2160 images at 30 Hz using a raw 12-bit format packed into 1.5 B per pixel. Each uncompressed frame occupies $3840 \times 2160 \times 1.5 \text{ B} \approx 12.44 \text{ MB}$ of memory. Across four cameras at 30 Hz, the sustained write traffic generated by frame ingestion alone is:

$$B_{\text{ingest}} = 4 \times 12.44 \text{ MB} \times 30 \text{ s}^{-1} \approx 1.493 \text{ GB/s}$$

When the perception stack reads these frames to feed vision backbones, driver or application paths may copy or transform intermediate buffers, and neural network accelerators read weights and write activations, the memory controller experiences traffic several times larger than the raw ingestion rate. If the host platform uses a dual-channel LPDDR5 memory subsystem with a theoretical peak bandwidth of 51.2 GB/s, the raw camera ingress consumes roughly 2.9 percent of theoretical capacity. Evaluated purely as a steady-state average, this ingress appears well within the limits of the hardware interface.

Average bandwidth does not determine the latency of one read. Synchronized cameras can produce concentrated DMA writes, but arbitration granularity, read priority, and bank placement

¹⁰ | **MIPI CSI-2 bus protocol:** CSI-2 transmits serialized pixel streams over D-PHY or C-PHY lanes to a host receiver and DMA controller. It can avoid a network stack, but transport and queueing delay still depend on sensor mode, lane rate, controller, and competing traffic. Measure the full capture-to-consumer age for the selected design; see Appendix C.

¹¹ | **Video4Linux2 direct memory access:** A V4L2 driver may use scatter-gather or physically contiguous DMA storage and expose a capture buffer through `V4L2_MEMORY_MMAP`. A compatible `dma_buf` importer can share that storage, but format conversion or device constraints may still copy or transform pixels. Saved host bandwidth affects observation and proposal latency; the independent permission controller uses its own memory path. See Appendix C.

determine how long another request waits. The following shared-bus thought experiment is intentionally **unpartitioned**: it illustrates what a designer would have to measure if a safety monitor used the same DRAM path. It is not the chapter's independent-MCU architecture. There, camera traffic can delay the host's observation or proposal, while the MCU continues its local permission loop and rejects an expired input.

For a loop that really shares this path, let T_{enforce} be its period, t_{exec} its execution time without contention, and $S = T_{\text{enforce}} - t_{\text{exec}}$ its slack. The relevant interference is a **measured or justified upper bound on one critical read's added arbitration delay**, not total frame service time. The admission check is:

$$\Delta t_{\text{read,max}}^{\text{measured}}(\text{camera mode, arbitration}) \leq S \quad (16.6)$$

The admission inequality in equation 16.6 also needs scheduler, cache, and actuator delays accounted for in the complete response deadline. Camera payload divided by aggregate DRAM service rate cannot establish $\Delta t_{\text{read,max}}$; no admissible payload limit follows from bandwidth alone.

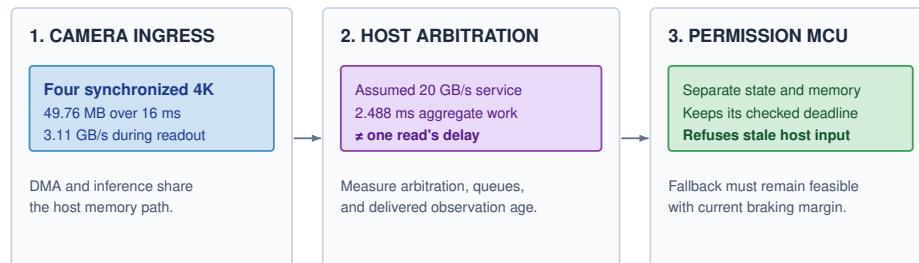
🔍 Systems Perspective 16.3: Ingress bandwidth as a safety parameter

Sensor resolution, frame rate, and channel count affect observation age and shared-resource demand. They affect a safety refusal path only when that path shares the contended resource; its added delay must be measured against deadline slack.

Consider four synchronized cameras producing packed 12-bit 3840×2160 frames at 30 Hz. One frame set is 49.76 MB; if those bytes arrive over 16 ms, their average arrival rate during readout is 3.11 GB/s. Dividing by an assumed 20 GB/s aggregate memory service rate gives 2.488 ms of *aggregate service under monopoly*, not a continuous stall of a safety read and not two dropped 1 ms control cycles. The read delay and any missed deadline require measured arbitration behavior for the actual shared path. In the intended isolated-MCU design, congestion can make a host observation or proposal stale; the MCU continues enforcing its own deadline and refuses the stale input, choosing a validated fallback when one remains physically feasible.¹² The format comparison in figure 16.5 shows the raw ingress demand of selected hypothetical camera configurations, not historical deployment measurements.

Contention can also delay camera DMA writes. If measured service delay exceeds a particular receiver FIFO's capacity, that receiver may flag an overrun or drop a frame; the threshold depends on its buffer and burst pattern. A dropped frame doubles the interval between two otherwise periodic samples, but the effect on a controller's phase margin depends on its state estimator and fallback. The independent permission path treats the missing update as an age or sequence fault rather than assuming that a new image will arrive on schedule.

📌 Checkpoint 16.2: Camera DMA bursts and memory stalls



- ☐ **Ingress burst contention:** Can you distinguish aggregate payload service time from the measured worst-case arbitration delay of one critical read?

Downstream: Crossbar bus arbitration and DMA QoS isolation are formally evaluated in section 26.3.

12 | **Arm Memory System Resource Partitioning and Monitoring (MPAM):** MPAM provides mechanisms for identifying and partitioning traffic where implemented and configured. It does not by itself establish a worst-case read latency or a control deadline. The design must measure or bound interference under the intended camera and accelerator load; a separate MCU memory path avoids this particular shared-DRAM dependence. See Appendix C.



Payload service time alone does not determine the latency of a safety-critical read.

- ❑ **Sensor resolution and freshness:** Can you explain how higher-resolution ingress can age or drop a host observation while an independently clocked permission MCU continues to refuse expired proposals?
- ❑ **UMA crossbar contention and FIFO overruns:** Can you explain how large GPU/NPU tensor operations can starve MIPI CSI-2 ingress DMA on unified memory buses, causing hardware FIFO overruns and dropped sensor frames?
- ❑ **Traffic shaping and bus isolation:** Can you identify what must be measured after configuring QoS or a separate memory path before claiming a deadline is met?

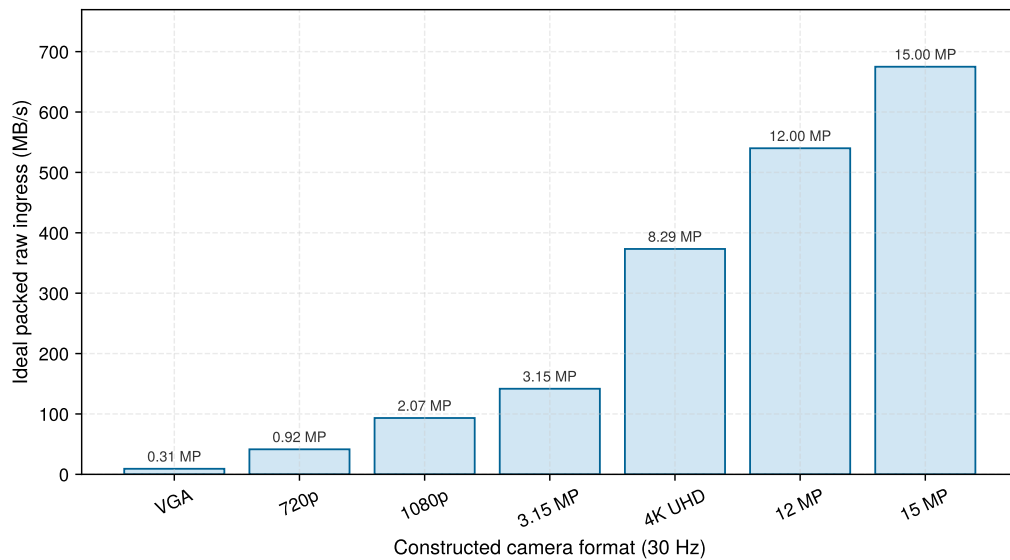


Figure 16.5: Illustrative camera format ingress: Packed raw bytes per second for selected constructed formats, all at 30 Hz. The rates exclude interface overhead, copies, ISP traffic, and arbitration; they are not historical fleet measurements or safety-read delay.

The hardware mechanisms governing frame delivery—including DMA rings, IOMMU mappings, and DRAM arbitration—are treated by Hennessy and Patterson (2019) and Bryant and O’Hallaron (2015). For this chapter’s decision, measure capture-to-consumer age and, if a critical path shares memory, its worst-case added read latency under the intended camera workload. In the independent-MCU architecture, host contention chiefly threatens observation freshness; it does not grant a stale proposal actuator authority.

Even when frames enter memory within latency and bandwidth bounds, converting raw 2D pixel grids into spatial geometry introduces irreducible measurement uncertainty. In section 16.5, we examine spatial error and clearance bounds, deriving how sensor noise and 3D lifting operations consume physical safety margins.

16.5 Spatial Error and Clearance Bounds

A spatial error is not merely a numerical representation. When downstream control logic uses an estimated coordinate to authorize motion, that error becomes a physical displacement, and it is spent out of the same clearance that keeps the machine off the obstacle. Transforming a sensor observation into physical authority requires **metric grounding**, which maps raw detector measurements into geometric space. This grounding relies on unprojecting two-dimensional sensor coordinates into three dimensions, an inversion that assumes known optics, rigid mounting, and predictable surface properties.

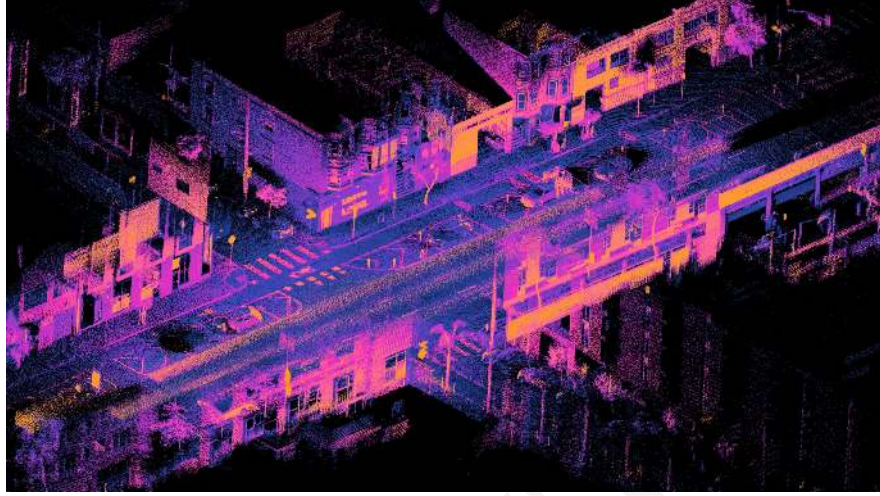


Figure 16.6: Registered urban LiDAR point cloud: Twenty-three million time-of-flight returns from an Ouster OS1-64 on a moving vehicle, registered into the world frame by SLAM and colorized by intensity. A specified time-of-flight sensor may have a flatter range-error curve than the stated passive-stereo model over a qualified distance and return-strength region; this photograph alone does not establish that error curve. Motion compensation across sweeping scanlines is what keeps vehicle velocity from shearing the reconstructed geometry. Attribution: Ouster Inc. public dataset, CC BY 4.0.

The mathematical principles of projection and triangulation, detailed by Hartley and Zisserman (2003) as well as Szeliski (2022), govern how measurement noise converts into spatial displacement. In passive stereo vision, depth z is computed from baseline b , focal length f , and disparity d through the relationship $z = \frac{fb}{d}$. Differentiating this unprojection with respect to disparity yields the depth uncertainty (equation 16.7):

$$\delta z \approx \frac{z^2}{fb} \delta d \quad (16.7)$$

where δd is a small disparity error.¹³ If δd denotes a disparity standard deviation σ_d , first-order propagation gives $\sigma_z \approx z^2 \sigma_d / (fb)$; it says nothing by itself about a hard error bound or a tail percentile. Doubling range quadruples this modeled standard deviation at fixed f , b , and σ_d . A pulsed time-of-flight sensor instead estimates $z = c\Delta t/2$; its range error depends on its timing and detection mechanism and may be approximately flat over a specified operating region.

Figure 16.6 shows a registered time-of-flight sweep after its returns have been placed in a common world frame. The reconstructed geometry survives the vehicle's own motion only because each sweeping scanline is compensated before registration, and the intensity coloring that makes the scene legible encodes return strength rather than range error.

Inverse-depth metric grounding is the geometric parameterization of visual range observations in inverse-distance coordinates ($\rho = 1/z$). Because depth uncertainty scales quadratically at long range, tracking Cartesian distance z directly causes standard estimators (such as Extended Kalman Filters) to destabilize when approximating non-Gaussian error tails. By tracking inverse depth instead, the measurement error remains roughly linear and well-behaved, preventing covariance divergence under the heavy-tailed, quadratic spatial error distributions inherent to passive stereo triangulation (equation 16.7) and monocular unprojection.

To quantify the geometry, assume $b = 0.20$ m, $f = 800$ px, and disparity standard deviation $\sigma_d = 0.5$ px over the tested range. First-order propagation gives $\sigma_z = 12.5$ mm at 2.0 m and $\sigma_z = 312.5$ mm at 10.0 m (figure 16.8). Those are **one-standard-deviation scales**, not clearance insets or 99th-percentile bounds. A usable inset requires a tail quantile of the *full spatial residual*, calibrated on the intended sensor, range, texture, motion, and lighting conditions, with systematic bias and missed detections handled separately. A $C_{\text{safe}} = 100$ mm physical margin therefore cannot be converted to 112.5 mm or 412.5 mm merely by adding one σ_z . Speed must be reduced or the proposal refused whenever the validated range-error and stopping margins do not fit; no universal 3–4 m cutoff

¹³ | **Inverse-depth parameterization:** In visual state estimation, parameterizing 3D landmarks by inverse depth ($\rho = 1/z$) converts the non-linear, heavy-tailed depth error distribution ($\delta z \propto z^2$) into an approximately Gaussian error space over the image plane. Under inverse depth, points at optical infinity ($z \rightarrow \infty$) exhibit finite variance ($\rho \rightarrow 0$), enabling standard Extended Kalman Filters and bundle adjustment solvers to track distant landmarks without numerical covariance divergence. For the rigorous derivation and factor graph formulation, see Appendix A.

follows. A time-of-flight device may have a flatter error curve over its qualified range, but its tail and failure modes likewise require measurement.



Checkpoint 16.3: Stereo depth divergence vs. LiDAR time of flight

- **Quadratic depth divergence:** Can you explain why passive stereo triangulation uncertainty grows quadratically with distance ($\delta z \propto z^2$) while a specified time-of-flight device may have a flatter error curve over its tested operating region?
- **Geometric clearance insetting:** Can you describe how expanding spatial uncertainty envelopes force downstream planners to defend wider clearance buffers, and why this requires dynamic speed derating in narrow physical passages?
- **Inverse-depth representation:** Can you distinguish between tracking range directly in Cartesian coordinates versus parameterizing visual observations in inverse depth ($\rho = 1/z$) to preserve linear error distributions in state estimators?

16.5.1 3D foundation models in physical AI: DINOv2, SAM, and ConceptGraphs

Modern physical AI architectures avoid classical hand-engineered geometric detectors and closed-set bounding box classifiers. Instead, they increasingly leverage **3D vision foundation models** that combine zero-shot open-vocabulary semantic understanding (building on language-driven affordances such as those in SayCan (Ahn et al. 2023)) with metric spatial grounding:

1. **Self-Supervised Vision Backbones (DINOv2):** DINOv2 (Oquab et al. 2023) uses ViT architectures trained via self-distillation on massive visual datasets.¹⁴ Unlike supervised models that reduce an image to a single class label, DINOv2 divides the image into a grid, producing dense patch token representations—high-dimensional vectors for each small region. Because they are not forced into rigid categories, these feature vectors preserve powerful geometric and semantic properties. Patch tokens from DINOv2 naturally encode dense cross-view correspondences, part-level semantics, and relative depth cues without any task-specific fine-tuning.
2. **Promptable Zero-Shot Segmentation (SAM):** The Segment Anything Model (SAM) (Kirillov et al. 2023) delivers class-agnostic instance mask segmentation across arbitrary visual scenes. By running a heavyweight image encoder once per frame and evaluating fast, promptable mask decoders, SAM partitions complex physical scenes into coherent object surfaces.
3. **Open-Vocabulary 3D Scene Graphs (ConceptGraphs):** To translate 2D foundation model representations into actionable physical representations, systems such as ConceptGraphs (Kuwajerwala et al. 2024; Jatavallabhula et al. 2023) and Hydra (Hughes et al. 2022) lift 2D instance masks and dense semantic features into 3D metric space:
 - For every video frame, 2D segmentation masks from SAM and dense semantic feature embeddings from DINOv2 / CLIP are extracted.
 - Using calibrated depth maps (from active stereo or LiDAR), camera extrinsics $\mathbf{T}_{bc}$, and odometry poses $\mathbf{T}_{wb}(t)$, the 2D masked pixels are unprojected into 3D point cloud clusters in the body or map frame.
 - Redundant 3D object proposals across sequential camera viewpoints are merged by evaluating both geometric overlap (3D bounding box Intersection over Union, IoU_{3D}) and semantic cosine similarity in the foundation embedding space.
 - The resulting output is an **object-centric 3D scene graph**, where each node represents a distinct physical object carrying an oriented 3D bounding box, a centroid position $\mathbf{p} \in \mathbb{R}^3$, a spatial covariance $\mathbf{p} \in \mathbb{R}^{3 \times 3}$, and an open-vocabulary text-aligned embedding vector.

3D foundation metric lifting is the multi-rate systems architecture that projects dense 2D foundation model representations (such as DINOv2 self-supervised patch tokens and SAM instance segmentation masks) into 3D metric entities (such as ConceptGraphs object-centric nodes) with calibrated spatial covariances ($\mathbf{p}$) and temporal freshness bounds.

Foundation-model features spend a compute and age budget (Williams et al. 2009). As a transparent sizing example, an assumed 3 TFLOP encoder at 30 Hz would require 90 TFLOP/s

¹⁴ | **Vision transformer patchification:** Vision Transformers decompose an input image $\mathbf{I} \in \mathbb{R}^{H \times W \times C}$ into a non-overlapping grid of patches of size $P \times P$, linearly projecting each flattened patch into a D -dimensional token embedding. Self-attention layers compute all-to-all token interactions across the grid, preserving dense spatial correspondence and semantic relationships across scales without discrete category downsampling. In Physical AI pipelines, these continuous token representations provide geometric and semantic grounding for 3D lifting operations. For more on self-supervised representation learning, see Appendix A.

of sustained useful throughput before preprocessing or memory traffic. NVIDIA lists up to 43 dense FP16 TFLOP/s for a Jetson AGX Orin configuration in its technical brief; that is peak arithmetic capability, not sustained end-to-end model throughput. The illustrative workload cannot meet that rate on this configuration even at the arithmetic ceiling. Smaller encoders such as MobileSAM change the work per frame, but their latency and power on a selected edge device must be measured; no universal 1–2 TFLOP cost or 50–150 ms latency follows from the model name. A slow semantic update is admissible only when fresh metric sensing and the independent permission path can maintain the physical margin between updates.

Physical AI architectures solve this by partitioning perception into a **multi-rate hierarchy**:

- **High-Rate Metric Ingress (30–60 Hz):** The Linux MPU converts raw sensor voltages into color pixels (hardware ISP debayering) and executes fast metric depth unprojection directly from the camera’s memory stream (MIPI CSI-2 DMA buffers). This pipeline can bypass a heavy neural backbone to generate metric obstacle points and occupancy grids at the stated rate, subject to sensor coverage, calibration, and measured age.
- **Asynchronous Foundation Graph Updates (1–5 Hz):** In parallel worker threads, the NPU evaluates DINOv2 patch embeddings and SAM segmentation masks on keyframes, incrementally updating the semantic nodes of the 3D ConceptGraph.
- **Independent Permission (illustrative 1000 Hz period):** The real-time controller checks proposals against a qualified clearance estimate and its age. Segregated state and memory keep host bus jitter off its local execution path, but stale or missing host observations still constrain what motion it can safely admit.

16.5.2 Ray frustum voxel lifting and metric splatting

Physical AI systems construct metric Birds-Eye-View (BEV) representations to ground learned 2D representations in the physical world and avoid perspective scale distortion. The end-to-end dataflow—from photodiode ingress to metric 3D memory—is structured in figure 16.7 across four computational stages:

1. **Stage 1: 2D Sensor Ingress & Feature Extraction:** Raw image arrays arrive over MIPI CSI-2 DMA ring buffers into shared memory. A vision backbone (such as DINOv2 (Oquab et al. 2023) or a lightweight convolutional net) processes image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$, extracting high-dimensional 2D feature patch tokens:

$$\mathbf{f}_{u,v} = \text{Backbone}(\mathbf{I})[u, v] \in \mathbb{R}^C$$

However, because forward pinhole projection collapses 3D rays onto 2D photodiodes (equation 16.4), a single 2D feature token $\mathbf{f}_{u,v}$ corresponds to an infinite 1D ray in physical space: the true metric distance of that visual feature is fundamentally ambiguous.

2. **Stage 2: Ray Unprojection and Categorical Depth Discretization (Lift):** The **Lift-Splat-Shoot (LSS)** architecture (Phillion and Fidler 2020) represents this ambiguity by lifting each 2D feature token into depth hypotheses along a 3D viewing frustum; it does not observe which depth is true. Using camera intrinsic matrix $\mathbf{K}$, the system evaluates the optical bearing ray $\mathbf{r}(u, v) = \mathbf{K}^{-1}[u, v, 1]^\top$ (equation 16.5). The network predicts scores across D discrete depth bins spanning $[d_{\min}, d_{\max}]$ (e.g., $D = 64$ bins spaced linearly or logarithmically from 0.5 m to 20.0 m):

$$\mathbf{p}_{u,v} = \text{softmax}(\mathbf{d}_{u,v}) \in \Delta^{D-1}$$

The continuous 3D feature representation is synthesized via an outer product, lifting 2D feature $\mathbf{f}_{u,v}$ into a 3D frustum point cloud:

$$\mathbf{c}_{u,v,k} = \mathbf{p}_{u,v}(k) \cdot \mathbf{f}_{u,v} \in \mathbb{R}^C \quad \text{at 3D coordinate } \mathbf{P}_c(u, v, d_k) = d_k \mathbf{K}^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

For a 128×128 feature map with $D = 64$ depth bins and $C = 64$ feature dimensions, this frustum tensor comprises $128 \times 128 \times 64 \times 64 \approx 6.71 \times 10^7$ FP16 values (134 MB per frame).

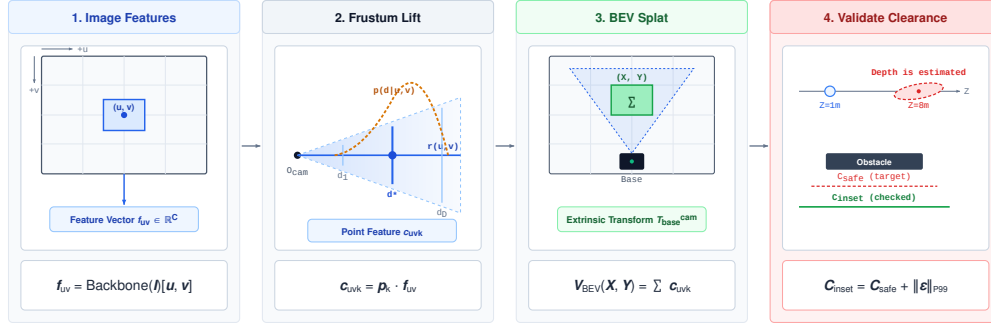


Figure 16.7: Camera ray frustum lifting and BEV splatting: LSS (Phillion and Fidler 2020) places learned depth hypotheses from each camera in a shared metric BEV frame; that coordinate transform does not resolve monocular depth ambiguity. The final clearance inset requires separately validated metric residuals, bias, and a chosen tail quantile or deterministic error bound before permission. (Architecture adapted from Phillion and Fidler (2020).)

Managing this data throughput without bus starvation demands zero-copy memory buffers and high-throughput NPU tensor cores.

3. **Stage 3: SE(3) Frame Transformation and Metric BEV Pillar Splatting (Splat):** Every frustum point $\mathbf{P}_c$ is mapped from the sensor optical frame into the shared robot base frame $\mathcal{F}_{\text{base}}$ using calibrated extrinsic transformation $\mathbf{T}_{\text{base}}^{\text{cam}} = [\mathbf{R}_{\text{base}}^{\text{cam}} | \mathbf{t}_{\text{base}}^{\text{cam}}] \in SE(3)$:

$$\mathbf{P}_{\text{base}} = \mathbf{R}_{\text{base}}^{\text{cam}} \mathbf{P}_c + \mathbf{t}_{\text{base}}^{\text{cam}}$$

To structure these unorganized 3D points for real-time motion planning, the physical environment is discretized into a metric Birds-Eye-View (BEV) pillar grid (e.g., 200×200 cells covering a $20 \text{ m} \times 20 \text{ m}$ workspace at 0.1 m resolution). In the “splat” step (implemented via accelerated cumulative sum operations or point-pillar pooling (Phillion and Fidler 2020; Z. Liu et al. 2023)), all 3D frustum features falling within vertical pillar (X, Y) are aggregated:

$$\mathbf{V}_{\text{BEV}}(X, Y) = \sum_{(u, v, k) \in \text{Pillar}(X, Y)} \mathbf{c}_{u, v, k}$$

This splat operation places multi-camera hypotheses in a shared metric BEV canvas. It aligns coordinates but cannot fill an unseen region or remove depth-scale errors: the footprint assigned to a pedestrian still depends on the accuracy of the depth hypotheses and calibration.

4. **Stage 4: Spatial Covariance Propagation & Clearance Insetting (The Safety Contract):** Metric features cannot be treated as deterministic truth. Stereo disparity uncertainty propagates approximately as $\sigma_Z \approx Z^2 \sigma_d / (fB)$; a learned depth softmax instead expresses model-assigned bin weights and its entropy is **not** calibrated metric covariance. A separate residual-calibration procedure must establish spatial error, bias, and tail behavior under the intended conditions. Downstream safety monitors can use a validated error quantile to expand the nominal obstacle boundary:

$$C_{\text{inset}} = C_{\text{safe}} + \|\epsilon_{\text{spatial}}\|_{P_{99}}$$

The 99th-percentile inset is a probabilistic design margin with residual tail risk, not a deterministic guarantee. If dropout, occlusion, or glare invalidates calibration, the permission path refuses the affected proposal and selects a feasible fallback.

16.5.3 Clearance inset margins and transform chain propagation

Spatial error extends beyond the sensor optical frame. To govern motion, the estimated point must propagate through coordinate transformations to the robot base frame and the end effector. The mechanics of rigid body transformations and SE(3) kinematic frame composition are standard results treated by Craig (Craig 2005) as well as Lynch and Park (2017). Each link in this transform chain contributes scale errors, mounting calibration drift, joint encoder quantization, and mechanical

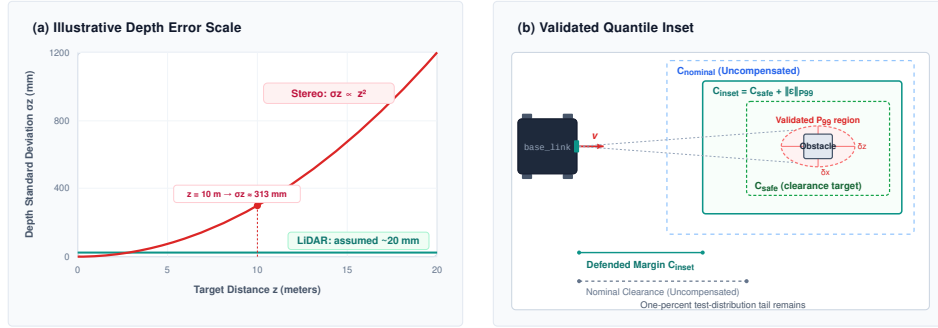


Figure 16.8: Depth error scale and clearance insets: (A) An illustrative stereo one-standard-deviation depth scale grows with squared range for fixed disparity spread; a qualified time-of-flight sensor can be flatter. (B) A validated 99th-percentile *norm-error* quantile can enlarge the nominal clearance inset, leaving residual tail and bias risk. A hard clearance guarantee requires a justified deterministic error bound and feasible stopping response. Attribution: Book geometric error analysis, CC BY 4.0.

compliance under load (such as a robot arm sagging slightly under gravity). Lever-arm effects amplify a microscopic angular twist at the sensor mount linearly over distance, sweeping projected coordinates across a wide arc. In an analytical case where an extrinsic calibration shifts by $\theta = 0.005$ rad (0.29°) between a camera mount and the vehicle chassis, the resulting lateral displacement error is $\delta x \approx z\theta = 50.0$ mm at a target distance of $z = 10.0$ m. When combined with quadratic depth uncertainty, the total spatial error vector $\epsilon_{\text{spatial}}$ at long range is dominated by depth along the optical axis and angular lever-arm effects orthogonal to it.

Every physical safety barrier operates by defending a geometric boundary around the machine. For position error vector $\epsilon_{\text{spatial}}$, the triangle inequality gives a conservative relation between nominal and true clearance:

$$C_{\text{true}} \geq C_{\text{nominal}} - \|\epsilon_{\text{spatial}}\|$$

For a validated 99th-percentile norm-error quantile, the safety enforcer can choose an **inset clearance** C_{inset} (equation 16.8):

$$C_{\text{inset}} = C_{\text{safe}} + \|\epsilon_{\text{spatial}}\|_{P_{99}} \quad (16.8)$$

The quantile must come from a specified, checked residual distribution for the full spatial error, including calibration bias. Even then it leaves a one-percent tail under the test distribution and cannot certify never breaching C_{safe} . The illustration in figure 16.8 shows how stereo's growing depth-error scale can require a larger inset at longer range. The comparison with LiDAR is illustrative: each sensor's actual error curve and missed-detection behavior need validation. A deterministic guarantee would instead require a justified hard bound on the relevant error, a feasible stopping path, and a permission response within the remaining delay budget.

Because clearance depends on propagated error, downstream planners and safety monitors cannot consume raw coordinate tuples. An observation crossing the perception boundary must specify its target coordinate frame, acquisition time, and uncertainty representation, such as $\in \mathbb{R}^{3 \times 3}$ together with an independently validated tail or hard bound. Covariance alone does not say whether a 10 mm or 500 mm clearance inset is warranted.

When spatial error is qualified, the permission path can expand the clearance inset, trading speed or workspace for separation. A textureless wall, glare, or lost calibration can invalidate that qualification without making a numerical variance literally infinite. The permission path must then refuse motion that depends on the affected claim and select a feasible fallback.

The complete operational protocol for metric unprojection, kinematic covariance propagation, and dynamic clearance inset sizing is formalized below:

16.5.4 Spatial clearance inset and depth covariance projection protocol

Cadence & Invariants:

- *Execution Cadence:* Synchronous with perception frame ingress (30–60 Hz on Linux MPU / Hardware ISP).

Downstream: Spatial covariance envelopes inflate the forward-invariant safety barrier sets derived in section 24.7.

- *Memory Invariant*: Zero dynamic heap allocation in inner unprojection loops; bounded matrix scratchpad buffers.
- *Interface Contract*: Emits bounded spatial claims $(\mathbf{p}_{\text{body}}, \text{body}, C_{\text{inset}}, \tau_{\text{age}})$ to the shared-memory spatial scene graph.

Execution Protocol:

1. **Information Age & Temporal Freshness Verification (Stage 1)**: Convert the capture timestamp to the consumer's clock with bounded synchronization error, then compute $\tau_{\text{age}} \leftarrow t_{\text{curr}} - t_0$. If age plus clock uncertainty exceeds the admissible budget, flag a stale observation and refuse proposals that require it. Otherwise, propagate position and uncertainty to the decision time using a validated motion model.
2. **Optical Metric Unprojection & Sensor Covariance Synthesis (Stage 2)**: Convert each 2D pixel measurement into 3D optical coordinates and synthesize a diagonal sensor covariance matrix representing the depth and lateral uncertainty.¹⁵
3. **Rigid Kinematic Transform & Spatial Covariance Projection (Stage 3)**: Transform the local sensor coordinates to the body frame, using a first-order SE(3) error propagation to account for uncertainty in the mounting extrinsics and velocity-induced displacement errors.
4. **Bounding Error Ellipsoid & Dynamic Clearance Inset Sizing (Stage 4)**: Use validated residuals, including bias, to obtain a task-relevant spatial error quantile or a justified deterministic bound. If no such bound remains valid, refuse the affected motion. Otherwise, add the chosen radius to the nominal safe boundary, recognizing any residual tail risk.
5. **Record Packaging & Shared Memory Emission (Stage 5)**: Populate claim record $\mathcal{C}_{\text{body}} \leftarrow (\mathbf{p}_{\text{body}}, \text{body}, C_{\text{inset}}, \tau_{\text{age}})$ and commit to the lock-free shared memory ring buffer with memory barrier (DMB ISH).

¹⁵ | **Metric unprojection and covariance propagation**: First-order propagation maps measurement covariance to $c = \mathbf{J}_{\text{unproj}} \text{meas} \mathbf{J}_{\text{unproj}}^T$ and then to $b = \mathbf{R}_{\text{bcc}} \mathbf{R}_{\text{bc}}^T + \text{ext}$. Its largest eigenvalue gives a principal variance scale, **not** a 99th-percentile error radius. A quantile requires an assumed and calibrated error distribution, including bias; see Appendix C.



Checkpoint 16.4: Foundation model rooflines and multi-rate perception

- ☐ **Roofline compute bottlenecks**: For the assumed 3 TFLOP encoder at 30 Hz, can you compare required sustained throughput with a device's peak ceiling and identify what must still be measured?
- ☐ **Lift-Splat-Shoot BEV pooling**: Can you explain why projected depth hypotheses share a metric frame but still require independent range evidence or residual calibration before a clearance decision?
- ☐ **Silent optical saturation versus detectable bus faults**: Can you distinguish between hardware transport dropouts that trip driver flags and silent optical clipping that produces valid tensors representing physically corrupted scenes?

Mathematical propagation of spatial uncertainty establishes the boundary of reliable perception. In section 16.6, we analyze the real-world failure modes that disrupt this chain, diagnosing how physical degradation, clock desynchronization, and out-of-distribution environments corrupt sensory observations.

16.6 How Observations Go Wrong

Perception failures divide into two distinct operational classes. **Detectable faults** occur when the physical sensor or the communication transport reports an explicit error condition. A serializer-deserializer link drops a lock (losing clock synchronization), a cyclical redundancy check detects a corrupted payload, a frame buffer overflows, or a transport bus times out. These failures are straightforward to handle since the hardware driver asserts an error flag at the nervous system boundary. Downstream planning algorithms immediately register the missing frame, invalidate stale state estimates, and transition the machine into a degraded operating mode or command a controlled stop. In contrast, **silent failures** occur when the sensor hardware and transport pipeline function without error, delivering a fully populated, structurally valid tensor whose numerical values misrepresent the physical world.

¹⁶ | **LiDAR range walk and detector blooming:** Analogous silent degradation occurs in active time-of-flight LiDAR receivers utilizing Avalanche Photodiodes (APDs) or Single-Photon Avalanche Diodes (SPADs). When an emitted laser pulse reflects off a retroreflective surface, intense return energy drives the photodetector into non-linear charge saturation, accelerating leading-edge discriminator threshold crossing via *range walk* (detector blooming). The resulting timing distortion artificially shortens the calculated round-trip time (t_{TOF}), shifting the apparent obstacle boundary several centimeters toward the sensor without asserting a hardware fault.

Physical degradation of the optical channel represents the most common source of silent error.¹⁶ When direct sunlight enters an optical aperture, incident photons exceed the physical charge capacity (the “full-well capacity”) of the CMOS photodiodes, clamping pixel intensities at their maximum quantization limit (clipping the image to pure white) across large patches of the array. This **dynamic range saturation** destroys local spatial gradients. Feature extractors, optical flow estimators, and disparity matching algorithms fail because mathematical optimization on a uniform flat field yields zero information, yet the driver emits a valid frame at nominal line rate. Low-light environments introduce the opposite failure mode. As ambient illumination drops, the discrete, random arrival of individual photons (**photon shot noise**) and the intrinsic electrical fluctuations of the sensor’s amplifier (**sensor read noise**) dominate the measurement. The signal-to-noise ratio collapses, causing downstream neural networks to extract geometric features from random thermal fluctuations rather than physical scene boundaries. Environmental contaminants on the protective lens or dome window (such as water droplets, airborne dust, or mud splatters) act as irregular refractive elements and optical low-pass filters. They attenuate contrast, distort spatial geometry, and obscure obstacles while the transport bus continues to operate without asserting a single fault flag. Across physical archetypes, these silent degradation modes take distinct manifestations: in Mobile Systems, low-angle sun glare saturates forward cameras on highway asphalt or rain droplets refract LiDAR beams; in Manipulators, specular reflections off polished metallic workpieces blind active stereo depth cameras and coolant mist coats eye-in-hand lenses; and in Process / Energy Systems, metallic vapor plumes, incandescent spatter, and soot deposits attenuate pyrometer optical sight glasses in laser additive powder-bed fusion, causing temperature estimators to report nominal states while melt pool temperatures surge uncontrollably.

Timing and transport anomalies can also corrupt an observation silently. If an unshielded physical interface experiences electromagnetic interference, deserializer bit slips (where the hardware drops or duplicates a bit, misaligning the entire subsequent data stream) can alter payload semantics without invalidating standard packet lengths. Far more destructive than a lost packet is an observation delivered with a timestamp that misreports its true acquisition epoch. If an image sensor captures photons at epoch t_0 , but an unbuffered PCIe interrupt handler or an unsynchronized system clock timestamps the resulting memory buffer at $t_0 + \delta t$, the downstream state estimator integrates spatial features under an incorrect body pose. Consider a machine traveling with instantaneous velocity v and linear acceleration a along a given trajectory. When perception suffers an undetected timing desynchronization δt , the state estimator assigns the detected position of an obstacle to the body coordinates at $t_0 + \delta t$ rather than t_0 . Over the unmodeled time interval δt , the physical state estimation error Δp between the asserted position and the true position is governed by the kinematic drift relation (equation 16.9):

$$\Delta p = v \cdot \delta t + \frac{1}{2} a \cdot (\delta t)^2 \quad (16.9)$$

In an analytical example of an autonomous vehicle operating at a velocity of $v = 20.0$ m/s under an active braking deceleration of $a = -4.0$ m/s², an undetected timing delay of $\delta t = 50$ ms (the duration of a single delayed frame across a congested internal bus) produces a positional estimation error of:

$$\Delta p = (20.0 \text{ m/s})(0.050 \text{ s}) + \frac{1}{2}(-4.0 \text{ m/s}^2)(0.050 \text{ s})^2 = 1.0 \text{ m} - 0.005 \text{ m} = 0.995 \text{ m}$$

An undetected positional error of 0.995 m exceeds the nominal safety clearance of an industrial aisle or a standard roadway lane. Because the pipeline delivers a continuous stream of valid tensors, the state estimator assumes a fresh observation, computing safe actuator trajectories from stale physics. A missing frame causes a known information loss that a Kalman filter can handle by expanding its state covariance. A frame that arrives on time with a corrupt timestamp causes the filter to contract its covariance around an incorrect mean, driving actuators toward an obstacle that the model believes has already been cleared.

Defending against silent failure requires dedicated runtime health monitors at the sensor ingestion boundary. A pixel intensity histogram monitor measures the distribution of raw digitized values immediately after direct memory access transfer, identifying over-exposure saturation when values concentrate at the maximum bit depth or identifying lens occlusion when spatial variance drops below an empirical noise floor. A hardware watchdog timer tracks inter-arrival times across the direct

memory access channel, flagging transport jitter when inter-frame intervals exceed $T_{\text{nominal}} + 3\sigma_{\text{jitter}}$. Higher up the stack, an optical flow plausibility monitor compares computed pixel displacement vectors against the physical velocity envelope of the body, rejecting observations that imply accelerations beyond mechanical capability. In multi-modal configurations, a cross-sensor consistency check compares high-rate (1000 Hz) inertial measurement unit integrations against visual odometry. If the visual pipeline reports that the machine is stationary while the accelerometer measures a sustained 2.0 m/s^2 specific force (the physical acceleration felt by the sensor body), the monitor detects the discrepancy and trips a perception fault.

Every runtime health monitor operates under the causal boundary established in section 2.2. A single sensing channel cannot validate its own semantic interpretations. A camera assembly with clean optics, balanced exposure, jitter-free transport, and synchronized microsecond clocking can still present an observation that leads a learned model to hallucinate a clear path or ignore an obstacle. Because deep vision models project high-dimensional inputs through complex non-linear manifolds, in-distribution optical anomalies (such as a high-contrast shadow across a road, a painted mural on the side of a transport container, or specular reflection from wet asphalt) generate plausible pixel statistics, valid optical flow fields, and high model confidence. A single-sensor monitor evaluates whether the data collection process operated within nominal limits, not whether the inferred semantics match physical reality. A second modality, such as lidar or radar, can corroborate some optical claims when its coverage and failure dependence have been checked. A separately validated permission path can enforce clearance only within its sensing, model, timing, and actuator assumptions; neither route establishes unrestricted semantic truth.

The comparison in table 16.3 shows what each sensing mechanism can claim and what must be checked before combining claims. A second modality can reduce a shared failure mode only when its own coverage, timing, calibration, and fault correlation are established.

Table 16.3: Spatial sensor modality comparison: Sensor mechanisms provide complementary but fallible claims. Numeric range, accuracy, refresh, and link budgets require a named device and mode. For example, Prophesee EVT2 encodes an event in 32 bits, so one million events per second alone is about 32 Mbps before transport overhead; a 50 Mbps CAN-FD path cannot be inferred from an unrelated radar row. Fusion reduces risk only if its failure dependence and timing are validated.

Modality	Direct physical information	Main limitation	Permission-relevant check
RGB camera	Bearing, appearance, and image motion	Monocular depth scale is ambiguous; glare, darkness, and occlusion can hide objects	Fresh capture time and validated metric corroboration
Calibrated stereo	Disparity-derived range over a qualified baseline and texture region	Range error grows with distance for fixed disparity error; calibration and matching can fail	Range-dependent residual and bias validation
Pulsed LiDAR	Time-of-flight range at sampled directions	Precision, angular coverage, and weather response depend on the device and scene	Qualified range, return quality, and scan timing
FMCW radar	Range and radial velocity of detected returns	Multipath, angular ambiguity, and detection thresholds vary by configuration	Track association and coverage of crossing targets
Event camera	Asynchronous brightness-change events and motion cues	Static objects may yield few events; output bandwidth depends on event rate and encoding	Event-rate health and independent metric depth source

War Story 16.1: Silent perception omission (2016)

Context: A 2015 Tesla Model S traveling at about 74 mph with Traffic-Aware Cruise Control and Autosteer active struck a refrigerated semitrailer turning left across its path near Williston, Florida (National Transportation Safety Board 2017).

Mechanism: The crossing-path collision lay outside the front-to-rear crash modes for which the 2015 warning and automatic braking systems were designed. NHTSA reported that automatic braking required agreement between camera and radar; unusual vehicle shapes could impede



Figure 16.9: Williston collision damage: This NTSB presentation slide shows damage on the right side of the semitrailer and the roofless Tesla after the 2016 crash. The crossing-path geometry and absence of pre-impact braking come from the NTSB accident report, not from the photographs alone. Photographs credited on the NTSB slide to Florida Highway Patrol.

camera classification. The investigation does not establish that bright sky or overhead-sign filtering caused this particular miss (NHTSA ODI PE16-007).

Impact: The NTSB found no recorded indication that forward collision warning or automatic emergency braking detected an object, and no pre-impact braking occurred. The car passed under the trailer, which sheared off its roof (figure 16.9).

Response: The NTSB examined operational design domain limits and driver engagement as well as the collision-avoidance systems. The available record establishes non-detection by those functions, not the internal sensor-level reason.

Systems lesson: An absent detection is not evidence that a crossing path is clear. A permission design must account for detection coverage and unsupported scenarios; a health flag cannot certify that every relevant object was perceived.

🔍 Systems Perspective 16.4: The timestamp integrity law

A plausible but wrong timestamp can misalign a measurement without triggering a dropout flag. A validated consumer rejects timestamps outside its clock and age bounds; missing frames and timestamp faults both require uncertainty-aware fallback.

When an observation crosses from perception into the nervous system, a tensor alone does not reveal sensor saturation, capture age, or calibration status. The producer must attach a dated, framed record with checked transport integrity and explicit health flags. Silent semantic errors still require separate evidence.

The contract in section 16.7 lets the consumer check clock conversion, age, frame, calibration, payload integrity, and health before a spatial claim influences a proposal.

16.7 The Observation Contract

A downstream tracker or safety monitor cannot reconstruct exposure timing, mounting calibration, or record integrity from a tensor alone. The perception producer therefore packages each observation with the physical context needed to check it. This typed record extends the transport envelope in

chapter 8; its firmware hash identifies the generating build but does not authenticate the observation bytes.

Table 16.4: Physical AI observation record schema: Compact fields needed to date and frame a claim, evaluate its uncertainty, and check the bytes and generating revision. A keyed authenticator is a separate requirement if the threat model includes malicious modification.

Record field	Meaning	Consumer check
<code>capture_timestamp_ns</code> , <code>capture_clock_id</code>	Exposure midpoint in one shared, monotonic capture clock	Convert to MCU time with a valid bounded-skew mapping before checking age
<code>sync_error_ns</code>	Bound on capture-to-MCU clock conversion error	Add to worst-case age; refuse if missing or expired
<code>sequence_index</code>	Monotonic camera frame counter	Detect gaps and reordering
<code>sensor_frame_id</code>	Transform-tree key and calibration revision	Select the valid extrinsic transform
<code>tensor_payload</code>	Format, dimensions, length, and buffer	Validate bounds and record checksum before reading
<code>spatial_covariance</code>	Positive-semidefinite spatial error scale	Use with separately calibrated bias and tail bounds
<code>calibration_metadata</code>	Intrinsics, extrinsics, coefficients, temperature, validity range	Reject calibration outside its validated region
<code>provenance_digest</code>	SHA-256 of firmware and model revisions	Match the approved build; this is not payload integrity
<code>payload_crc32</code>	CRC over serialized header (excluding this field) and payload	Detect accidental record corruption; not authentication
<code>health_bitmask</code>	Hardware and perception validity flags	Refuse claims with safety-critical faults

The fields in table 16.4 date each observation and its generating revision. All cameras in this example latch exposure midpoints in a shared monotonic capture domain; the timestamp is **not** nanoseconds since the UNIX epoch. If the MCU has a different clock, a synchronization service supplies a versioned conversion $t_{\text{MCU}} = at_{\text{capture}} + b$, its validity interval, and a measured error bound ϵ_{sync} . The consumer uses the upper age estimate $t_{\text{MCU,now}} - t_{\text{MCU,capture}} + \epsilon_{\text{sync}}$ and refuses an observation if the conversion is invalid or its age exceeds the motion budget. A rolling-shutter claim must also identify the row or region whose acquisition time it represents. `sequence_index` detects missing frames independently of clock conversion.

The transform key and calibration revision anchor metric coordinates.¹⁷ `spatial_covariance` describes second moments, so a clearance decision additionally needs a validated residual quantile or hard error bound, including bias. The firmware/model digest supports version provenance; `payload_crc32` detects accidental changes to the actual header and payload after serialization. Neither a checksum nor a build hash proves semantic truth, and neither authenticates an adversarial producer without a keyed check.

Before reading the tensor payload, the consumer validates the record checksum, clock conversion, and 64-bit health bitmask. The lower bits report hardware faults such as synchronization loss or transport errors; the upper bits can report perceptual validity concerns such as occlusion. A set safety-critical bit requires refusal of the affected claim. Bitmask testing is cheap, but its full end-to-end latency and effectiveness depend on record delivery, memory access, and which faults the producer actually detects; one CPU instruction is not a subnanosecond safety response.

The explicit record layout also supports fault injection. A test can alter the capture time, omit a sequence number, inflate reported covariance, or set an occlusion flag, then check whether the consumer rejects or degrades that claim within its budget. Such mutations verify response to the injected record faults; physical glare, miscalibration, and actuator behavior still require separate tests in chapter 30.

An observation record carries the fields needed to interpret a measurement under a stated sensor and clock contract. A structurally valid record can still be stale, biased, or incomplete. The permission path must check whether it remains admissible before acting. Once the record is committed to

¹⁷ | **Mechanical thermal expansion:** For a stereo baseline $b = 0.20$ m and an unmodeled change $|\delta b| = 0.1$ mm, first-order geometry gives $|\delta z|/z \approx |\delta b|/b$. At $z = 20$ m, the corresponding depth bias is about 10 mm, assuming focal length and disparity are otherwise unchanged. Temperature compensation must be checked against the actual mount and calibration.

Downstream: Observation contract mutations feed the hardware-in-the-loop verification harness in section 30.1.

memory, its value decays with every microsecond of transport delay, queuing stall, and processing latency across the system bus.

Systems Perspective 16.5: Foot-strike shock and sensor vibration

Humanoid perception operates in a uniquely hostile biomechanical vibration environment:

- **Periodic Dynamic Shockwaves:** Unlike wheeled AMRs rolling smoothly across level floors, a walking or running humanoid undergoes violent, periodic impact decelerations at every foot strike. Heel impact with a rigid floor transmits transient shock loads exceeding $3\text{--}6g$ vertically through the pelvis, torso, and neck to head-mounted cameras. These impacts excite mechanical resonance in camera mounts, causing high-frequency angular jitter ($\dot{\theta} > 2.0$ rad/s).
- **Rolling Shutter Warp and Epipolar Epicycles:** Head-mounted CMOS rolling shutter sensors integrate pixel rows sequentially over 15–30 ms. High-frequency torso bobbing and pitch oscillations distort vertical lines into curved S-shapes, breaking epipolar geometry assumptions required for stereo triangulation and feature-based visual SLAM. If downstream depth estimation models (such as foundation depth encoders or feature matchers) process these distorted frames without IMU-guided continuous rolling-shutter unwarping, estimated 3D object positions oscillate by tens of centimeters between successive strides.
- **Camera-IMU Hardware Time Synchronization:** Compensating for foot-strike shock requires sub-millisecond temporal synchronization between head IMUs (sampled at 1 kHz) and camera exposure midpoints ($t_{\text{exp-mid}}$). An uncalibrated timestamp offset of just 10 ms during a rapid head turn (1.5 rad/s) introduces an unmodeled angular error of 0.015 rad (0.86°), displacing perceived obstacle coordinates by 7.5 cm at a 5 m lookahead distance—completely wiping out safe stepping clearance margins.

Checkpoint 16.5: Observation record contracts and reflex triage

- ☐ **Record triage:** Can you explain why a health bit, checksum, valid clock conversion, and freshness check must precede consumption of a spatial claim?
- ☐ **Timestamp corruption versus dropped packets:** Can you describe why receiving a sensory frame with a desynchronized timestamp degrades state estimation far more severely than dropping the packet entirely?
- ☐ **Deterministic fault injection:** Can you distinguish between physically perturbing optical sensors versus deterministically mutating fields in the shared-memory observation record during automated hardware-in-the-loop verification?

With the observation contract established, section 16.8 examines the recurring fallacies and architectural pitfalls that arise when systems conflate semantic recognition scores with spatial timeliness and geometric certainty.

16.8 Fallacies and Pitfalls

Perception must describe the world within the strict temporal deadlines required for control actuation. Semantic confidence and high spatial accuracy cannot compensate for pipelines that deliver stale observations, fuse asynchronous exposures, or delay the critical safety refusal loop.

Fallacy: *Accepting an observation because its semantic classification confidence is high, regardless of its age.*

A transport robot receives a high-confidence observation that a doorway is clear, delayed by $\Delta t = 220.0$ ms in the vision pipeline. During that interval, a crossing forklift advances 0.55 m into the robot's path. The confidence score describes the earlier image; it does not establish present clearance. Accepting the packet as current can therefore authorize motion into an occupied space.

The perception interface must check measurement age against the motion-dependent freshness limit and reject observations that exceed it. Any continued state propagation without new data must carry expanding kinematic uncertainty covariances that bound the reachable sets of dynamic obstacles, allowing the controller to enforce safety envelopes as historical evidence ages.

Pitfall: *Fusing multi-sensor spatial observations without sub-millisecond hardware clock synchronization.*

Two cameras observe a vehicle moving at $v_{\text{target}} = 30.0$ m/s, but their clocks differ by $\Delta t_{\text{skew}} = 15.0$ ms. Treating the exposure times as equal can introduce a 0.45 m position discrepancy into track association, making a single physical vehicle appear as two distinct, conflicting object tracks (“ghosting”). Spatial calibration cannot correct an unknown temporal offset. The synchronization requirement must follow the tolerated position error and maximum relevant speed, expressed by $\Delta t_{\text{skew}} \leq \epsilon_{\text{pos}}/v_{\text{max}}$. Hardware synchronization at exposure time makes that bound enforceable; selecting a timing technology (like IEEE 1588 PTP) without first checking the spatial error budget leaves the fusion assumption unverified.

Pitfall: *Inferring a critical read’s deadline from average camera bandwidth.*

If camera DMA and a critical controller read share a memory path, an acceptable average utilization does not prove the read’s deadline. The designer must bound arbitration delay under synchronized capture and inference bursts, then compare that delay with measured slack. QoS can help only after its configuration is validated under load. With the independent MCU path used in this book, camera contention instead threatens the observation or proposal deadline; the MCU refuses stale input and selects a feasible fallback.

16.9 Summary

A perception result is usable only when its capture time, clock conversion, coordinate frame, and error qualification fit the proposed motion. The forward-obstacle example shows why: a 51.6 ms illustrative capture-to-dispatch age at 2.5 m/s spends 129 mm of a forward travel reserve. Permission, bus, and actuator delay still need room before braking begins. A lateral aisle margin is a separate geometric quantity.

Camera payload size sets an ingress and compute budget, but aggregate memory-service time does not prove the stall of one safety-critical read. On a shared bus, measure arbitration delay against the critical path’s slack. With a separate MCU state and memory path, host contention instead ages or drops observations; the permission tier must reject stale proposals and use a validated feasible fallback.

Stereo geometry gives an error scale that grows with squared range under fixed disparity noise. A covariance or learned depth softmax does not supply a 99th-percentile clearance bound by itself. The consumer needs residual calibration, bias checks, a stated tail risk or hard bound, and a synchronized capture clock. Those dated claims feed the persistent state estimator in chapter 18.



Key Takeaways: Spatial perception and age of information

- **Observation Age Begins at Capture:** Hardware-latched exposure time, row timing when relevant, and bounded clock conversion let the consumer compute an upper age before admitting motion.
- **Ingress Costs Depend on the Shared Path:** Camera payload and burst shape affect host observation age. A shared safety read needs a measured arbitration bound; an isolated permission MCU rejects stale host input without relying on that host bus.
- **Softmax Probability Is Not Geometric Uncertainty:** A classification or depth-bin score does not supply a calibrated spatial tail. Metric covariance, bias, and residual validation are separate inputs to a clearance decision (section 4.1).
- **Calibration Can Change:** Shock or temperature can alter $T_{\text{sensor}}^{\text{body}}$. An assumed 0.01 rad angular error at 5.0 m projects to about 50 mm laterally, so the implementation must validate its calibration interval.

- **Perception Provides Dated Snapshots:** Persistent spatial memory propagates state and uncertainty between observations and through occlusion; the permission path still checks whether that propagated claim fits the motion margin.

Sensors record physical energy interactions, not geometric truth. The permission path must therefore check the dated claim, its calibration, and the remaining response margin before allowing that claim to support motion.



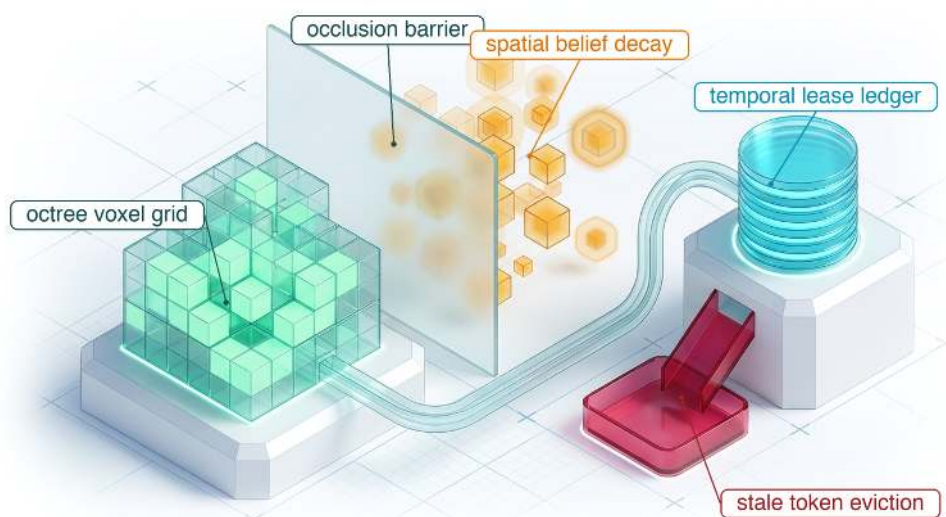
What's Next: From sensory photons to spatial memory

Perception delivers dated observations; chapter 18 carries their spatial estimates and uncertainty across the intervals between them. Its expiration rules help the permission path refuse a claim when occlusion or age has exhausted the validated motion margin.

Spatial Memory

DRAFT

Spatial Memory



- 18.1 Preserving Spatial Belief
- 18.2 What Memory Hands Over
- 18.3 A State Has an Epoch
- 18.4 Spatial Reference Frames
- 18.5 Volumetric Scene Memory and Attention Buffers
- 18.6 Belief Through Occlusion
- 18.7 The Rate of Belief Decay
- 18.8 State Estimation Bounds
- 18.9 How Belief Goes Wrong
- 18.10 The Spatial State Schema
- 18.11 Fallacies and Pitfalls
- 18.12 Summary

Purpose

How fast does a spatial belief turn into dangerous fiction once a sensor loses sight of an object?

A moving obstacle remains a constraint after disappearing behind a pillar. The machine must remember it, yet its last observed position becomes less reliable as time passes. Treating that position as current can send a planner into occupied space; discarding it can erase a nearby hazard. Spatial memory must therefore carry both an estimate and the limits of what that estimate supports. Observation age, uncertainty, and coordinate alignment determine whether a remembered object can still guide motion. A frozen coordinate transform is dangerous as it can make an old estimate appear current to downstream consumers. Expiration must withdraw that estimate's authority without treating the unseen region as empty. In the physical AI stack, memory connects the brain's account of the world to the nervous system's permission to move, with validity windows set by how quickly uncertainty consumes the body's available clearance.

Brain

1–50 Hz
foundation models, SLAM, planning

above: propose; below: command

Nervous System

1000 Hz
TSN, intent leases, lock-free IPC

Body

continuous
real-time MCU, safety filters, CBFs

Boundary

continuous
sensors, actuators, kinematics

Physical World

continuous
contact mechanics, friction, mass



Learning Objectives

- Specify belief records that preserve measurement time, coordinate frame, uncertainty, and conditions for use
- Set belief expiry from measured uncertainty growth and the task's physical error budget
- Compare spatial representations by memory cost, query time, and information needed for action
- Design spatial updates that preserve evidence age and distinguish missing observations from verified free space
- Diagnose stale transforms and other belief failures that downstream consumers may not detect

18.1 Preserving Spatial Belief

A memory register can preserve a coordinate exactly while the workpiece it describes moves. If a fixture drifts at 20 mm/s during occlusion, a 2 mm tooling tolerance is spent in 100 ms. Republishing the stored coordinate with a new software timestamp cannot restore the lost physical evidence. Spatial memory must therefore carry the last measurement time and an error-growth rule into the motion-permission decision.

A physical machine cannot rely solely on raw sensory observations or static memory registers. It requires a **belief**—an estimate of physical state carried forward from historical sensory evidence, coupled explicitly to an expanding model of uncertainty.¹ Perception supplies a dated, calibrated observation (as established in section 16.2). Spatial memory must determine how long that observation, combined with dynamic motion models, can continue to authorize physical action before uncertainty consumes the machine's remaining clearance.

Within the four-tier physical AI systems architecture, the memory stage highlighted in figure 18.1 connects perception to intent and trajectory planning. Operating as Stage 3 of the deliberative brain, its **spatial memory** carries geometric and physical state estimates forward across sensor dropouts, occlusions, and variable processing delays.

Prerequisite: Observation age and calibrated spatial covariances are generated in section 16.7.

¹ | **Belief state formulation:** In a Partially Observable Markov Decision Process (POMDP), a belief represents a probability distribution over latent states. A physical state interface can additionally carry a justified bounded-disturbance error envelope, or a calibrated error quantile with its declared tail risk; the latter is not a deterministic upper bound. Without the value, frame, evidence epoch, and applicable uncertainty model, a downstream optimizer cannot evaluate clearance during a sensor dropout.

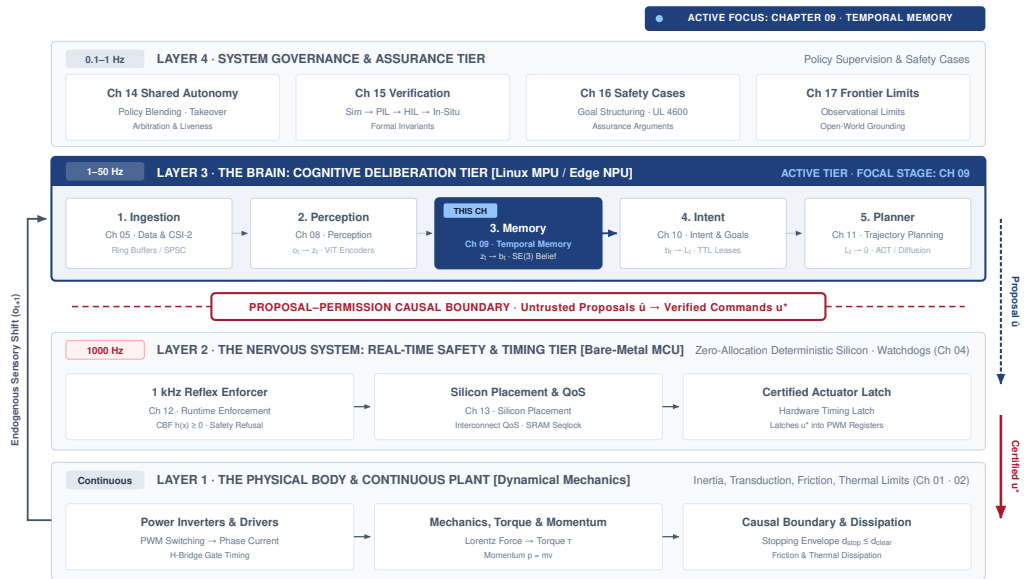


Figure 18.1: Memory stage architecture focus: The four-tier physical AI systems architecture highlights the cognitive deliberation tier, with memory as the third brain stage and this chapter's active focus. The stage carries latent state z_t into a persistent spatial belief b_t , making it the only place the system holds what its sensors can no longer see.

Crucially, memory operates directly across the **proposal-permission boundary**.² The unprivileged application processor (Linux MPU) maintains high-dimensional spatial belief structures, occupancy voxels, and coordinate transform trees. However, because the MPU experiences unpredictable scheduling jitter, memory bus contention, and inference latency, it cannot be trusted to self-report temporal freshness. The real-time safety processor (MCU) acts as the permission referee, evaluating every candidate motion proposal against independent hardware timers. When an unrefreshed belief breaches task safety margins, the nervous system unilaterally revokes execution permission.

Consider an analytical example of a manipulator tasked with touching a workpiece and regulating contact force against a moving assembly fixture. The fixture drifts relative to the robot base at a bounded velocity of at most 20 mm/s. The tooling compliance of 1 mm/N permits an allowable positioning error of 2 mm before generating a 2 N reaction force that overloads the wrist sensor. Dividing the allowable error by the maximum drift velocity yields an operational lifetime of 100 ms, calculated as $2 \text{ mm} / (20 \text{ mm/s}) = 0.100 \text{ s}$. For the first 50 ms after the camera captures an image, the physical position of the workpiece remains safely within the tolerance window. At 100 ms, the unobserved displacement reaches the outer boundary of the error budget. At 101 ms, without a new measurement, the modeled displacement exceeds the stated tolerance; a close approach using the stale coordinate would no longer satisfy the contact margin.

In software systems, data age is frequently measured from the instant a message was serialized, published over an inter-process bus, or updated in a shared memory table. In a physical AI system, this convention is dangerous. Propagating a state estimate through an extended Kalman filter at 1000 Hz or republishing an $SE(3)$ kinematic transform at 200 Hz creates a stream of fresh software timestamps, but it does not refresh the underlying physical information. The true age of a belief is determined strictly by the acquisition timestamp t_0 of the last physical measurement that constrained it. A downstream tracking loop that reads a position estimate generated by dead reckoning or policy rollouts may observe a publish timestamp only 2 ms old, yet the physical error envelope continues to expand at the velocity of the unobserved drift. Timestamping an inference output or a cache lookup records when compute occurred, but it tells the receiver nothing about when reality was last checked.

The stored bits can remain intact while the physical claim they represent loses support. A register preserves the last observed pose even when the fixture has moved, so memory integrity alone cannot establish geometric validity (Durrant-Whyte and Bailey 2006). To prevent a machine from acting on stale data, memory systems must treat expiry as an explicit interface event. The **belief state invalidation horizon** ($t_{\text{exp}} = \inf\{\Delta t \geq 0 : E(\Delta t) \geq E_{\text{max}}\}$) is the deadline under the declared error-growth model and operating envelope. When a justified hard bound reaches the physical margin, tracking authority expires; a calibrated quantile instead defines a risk-limited deadline with residual tail risk. Either case sends dependent motion proposals to the independent permission path and a validated, feasible fallback when they cannot be admitted.

Systems Perspective 18.1: The evidence epoch invariant

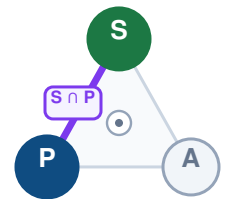
Extrapolation advances timestamps without refreshing evidence age. Unmonitored belief drifts into physical failure. Never let an intermediate predictor, filter, or transport bridge reset the evidence acquisition epoch t_0 ; compute freshness is a software artifact, but physical freshness belongs strictly to the hardware transducer.

The first handoff question is what a consumer needs to evaluate a stored value after its sensor loses sight of the object.

18.2 What Memory Hands Over

In the context of AI systems, a ‘belief’ refers to a representation of state or information that is subject to uncertainty. A belief passed across a real-time bus cannot be a naked array of floating-point numbers. It embodies a decaying interpretation of physical inputs and must explicitly carry its epoch, physical uncertainty, and reference frame to remain valid. An unannotated vector such as $[0.142, -0.051, 0.812]$ conveys nothing about what physical entity is being described, what physical dimension is represented, or how that quantity is scaled. A downstream controller receiving raw

² | **Dual-brain spatial partitioning:** The dual-brain architecture assigns high-dimensional spatial mapping (OctoMap, VDB voxels, factor graphs) to Linux application processor (MPU) cores while reserving deterministic hardware countdown timers and barrier evaluations for the real-time microcontroller (MCU). An independently clocked permission controller with a segregated memory path can continue its local check during MPU stalls; its complete detection-to-brake response still depends on validated sensing, bus, and actuator timing. See section 6.1 for the foundational interface specifications governing this boundary.



The Sense $\cap$ Plan belief interface: preserving spatial object state under line-of-sight occlusions.

coordinates without metadata must rely on implicit conventions established at compile time. If an estimator produces positions in meters while an actuator controller expects millimeters, or if an angular rate is mistaken for an absolute orientation, the resulting command registers as a massive tracking error, driving motor current to the actuator thermal limits detailed in section 4.3. To prevent misinterpretation across the causal boundary established in section 2.2, a belief record must first specify the entity identity, the physical quantity, and the measurement units. The consumer must be able to verify that the value represents the center of mass of a specific workpiece in meters, or the normal contact force on a fingertip in newtons, before routing that number into a control law.

Specifying the quantity and units remains incomplete without binding the value to a concrete reference frame. Identical spatial coordinates describe entirely different physical realities depending on the origin and orientation of the coordinate system. A Cartesian position $[0.142, -0.051, 0.812]$ m designates a point suspended in free space when evaluated in the robot base frame, but represents a point several hundred millimeters inside a rigid fixture when interpreted in the tool flange frame. Because kinematic links flex, joints move, and coordinate transforms update at varying rates, a belief record must carry an explicit frame identifier. Carrying the frame allows the receiving node to look up the corresponding spatial transformation tree and project the stored geometry into the local frame required by the actuator, preventing the controller from acting on geometric misalignments caused by moving kinematic chains.

Geometry and identity describe the physical state at a single moment, but memory must also capture the temporal anchor of that state. The timestamp attached to a belief must record the exact epoch t_0 of the physical interaction or optical exposure that last constrained the estimate. In distributed architectures, intermediate nodes often republish data, execute Kalman filter prediction steps, or run forward kinematic propagations to maintain high-rate control streams (for example, estimating where a moving target is right now based on its last known velocity). A prediction step or cache refresh updates the transmission timestamp, but it does not add new physical evidence from sensors. If an intermediate process overwrites t_0 with the current system clock during a forward rollout, downstream safety monitors observe a deceptive stream of fresh timestamps. The robot system falsely believes it has recent physical confirmation of the object, while the underlying physical state continues to drift unchecked. Preserving the original measurement epoch t_0 across all serialization and filtering stages ensures that consumers measure the age of the information against the physical world rather than against the scheduling frequency of the local computer.

Knowing the measurement epoch t_0 establishes elapsed time $\Delta t = t - t_0$, but elapsed time alone cannot determine whether a belief is valid for a given action. Two estimates of identical age diverge at different rates depending on physical constraints. A workpiece firmly held in a pneumatic vise maintains sub-millimeter positional certainty after several seconds of sensor occlusion, whereas the same workpiece sliding along a moving conveyor belt accumulates substantial position error in fifty milliseconds. Therefore, a belief record must include both the initial uncertainty bound e_0 established at the epoch of measurement and an explicit growth law that quantifies how that uncertainty expands over time.

The minimum record fields necessary for a consumer to evaluate a stored state can be derived directly from the requirement to bound current error. Let $e(t)$ represent the upper bound of the discrepancy between the stored value and true physical state at time t . If an estimator establishes a state at time t_0 with an initial measurement uncertainty e_0 , and external disturbances or unmodeled velocities cause the true state to diverge at a maximum rate r , the upper bound on error at current time t obeys the linear expansion $e(t) = e_0 + r(t - t_0)$. As an analytical example, consider a tactile sensor array that establishes the initial slip displacement of a grasped object with an uncertainty $e_0 = 0.40$ mm at $t_0 = 1.200$ s. If the object slides beneath the gripper at a maximum unmeasured velocity $r = 15.0$ mm/s, the error bound after an unobserved interval of $\Delta t = 80$ ms expands to $e(t) = 0.40$ mm + $(15.0$ mm/s) $(0.080$ s) = 1.60 mm. Omitting either e_0 or r leaves the consumer unable to compute $e(t)$, demonstrating that an initial bound and an expansion rate are both structurally required.

Expressing uncertainty through physical bounds and expansion rates distinguishes a physical AI belief from a probabilistic confidence score. An object detection network might output a class confidence of 0.94, but a downstream position controller cannot convert a unitless probability into a safe actuator displacement margin (i.e., how far the arm can physically move before risking a

collision). A confidence score of 0.94 provides no information about whether an obstacle boundary is uncertain by 2.0 mm or by 200 mm, nor does it indicate how rapidly that boundary moves when occlusion occurs. A physical memory record must express its initial bound e_0 and expansion rate r in concrete physical dimensions such as meters, radians, or newtons per second. Furthermore, these rates are valid only within a defined operating envelope, such as a maximum assumed acceleration or a verified contact friction coefficient. If physical operating conditions exceed that envelope, the expansion rate is no longer guaranteed, and the belief record must be treated as invalid.

Extending the proposal-permission architecture from section 6.1, when memory hands over a tuple containing value, reference frame, measurement epoch t_0 , initial bound e_0 , and growth rate r , the consumer executes a deterministic verification step before applying the data to physical control. The consumer possesses an application tolerance ϵ_{tol} dictated by the physical task. In a precision insertion operation with a radial clearance of 0.80 mm, the controller requires that positional uncertainty remain below $\epsilon_{\text{tol}} = 0.80$ mm. Upon reading the belief record at current time t_{now} , the consumer calculates the elapsed time $\Delta t = t_{\text{now}} - t_0$, evaluates the present error bound $e(t_{\text{now}}) = e_0 + r\Delta t$, and compares the result directly against ϵ_{tol} . If $e(t_{\text{now}}) \leq \epsilon_{\text{tol}}$, the belief passes this freshness check and can enter the independent motion-permission check. Otherwise the dependent proposal is refused; the controller selects a validated hold, search, or stop response only when it remains feasible.

Definition 18.1: The spatial belief contract

Spatial belief contract is a typed state record that binds an identified physical quantity and SI unit to a frame, acquisition and estimate epochs, clock conversion, initial error, growth law, operating envelope, provenance, and lifecycle status. These fields let a consumer evaluate freshness before a separate motion-permission check.

1. **Significance:** A state without an error bound and growth rate cannot be safely evaluated by downstream controllers. As time elapses ($\Delta t = t_{\text{now}} - t_0$), unobserved motion or sensor drift expands the spatial uncertainty envelope ($e(t) = e_0 + r\Delta t$). When uncertainty breaches task tolerance ($e(t) > \epsilon_{\text{tol}}$), the contract expires, requiring the permission tier to reject dependent proposals and choose a feasible validated response.
2. **Distinction:** Unlike a unitless classification probability (such as $P(\text{object}) = 0.94$), a spatial belief contract provides physically grounded metric units (meters, radians) and explicit time-decay mechanics.
3. **Common pitfall:** Relying on external publishers to broadcast invalidation signals upon occlusion. If the communication channel drops or the sensor driver hangs, the downstream consumer continues executing open-loop motions on frozen beliefs unless it independently evaluates the expiration contract.

Evaluating the spatial belief contract depends on calculating the elapsed time $\Delta t = t_{\text{now}} - t_0$ against physical reality. If the consumer cannot verify the exact physical instant when energy transduction occurred, the calculated error bound $e(t_{\text{now}})$ becomes an unanchored fiction, mistaking software compute cycles for physical evidence.

18.3 A State Has an Epoch

Every state reported by a sensor or estimated by a model corresponds to a specific point in time, termed its **epoch**. In a physical computing pipeline, a state passes through three temporal boundaries before a controller can act. The **acquisition epoch** (t_0) is the nanosecond-precise timestamp recording the exact physical instant when energy transduction occurred at the causal boundary (section 2.2), such as photons striking a photodiode array or a digital register latching an encoder count.³ The **estimation epoch** (t_{est}) marks the instant the computational pipeline (whether a state estimator or neural network) completed its mathematical inference over those measurements, representing the moment the computational pipeline resolves sensor measurements into a concrete state claim. The **publication epoch** (t_{pub}) marks the instant the serialized message was committed to the physical transport bus for dispatch to downstream consumers.

³ | **Hardware timer input-capture:** Microcontroller timer input-capture channels latch counter values into dedicated shadow registers upon external signal transitions, bypassing operating system interrupt latency. Reading capture registers without atomic synchronization against timer overflow flags causes 16-bit counter aliasing, injecting artificial timestamp jumps equal to the rollover period (65.5 ms on a 1 MHz timer). In downstream visual-inertial odometry, this temporal discontinuity introduces false acceleration spikes that destabilize platform attitude filters.

Collapsing these three timestamps into a single receipt or transmission timestamp destroys the physical validity of the state. If an inference accelerator requires 45 ms to execute an object detection model and the operating system incurs another 5 ms of scheduling and serialization latency before publication, tagging the resulting object pose with the publication time hides a 50 ms physical displacement. During that 50 ms interval, gravity, momentum, and contact forces continue to act on the physical system without pause. If a conveyor belt carries a workpiece at 0.60 m/s, a downstream gripper that treats the publication timestamp as the measurement epoch assumes the workpiece is located where it was 50 ms in the past, introducing an unmodeled position error of $(0.60 \text{ m/s})(0.050 \text{ s}) = 30.0 \text{ mm}$. A state that records only when data became available tells the consumer nothing about the physical instant the measurement represents, making temporal alignment impossible.

Prerequisite: Synchronized hardware timestamping across fieldbuses is established in section 8.4.

4 | Fieldbus clock synchronization jitter: Fieldbus architectures exhibit distinct hardware clock synchronization bounds: IEEE 1588 PTP over Ethernet achieves $\delta t < 1.0 \mu\text{s}$, EtherCAT Distributed Clocks achieve $< 100 \text{ ns}$ hardware SYNC0 jitter, and CAN-FD bit-phase synchronization yields $\pm 10\text{--}50 \mu\text{s}$. Because spatial uncertainty scales with target relative velocity ($e_{\text{sync}} = v_{\text{rel}} \delta t$), microsecond-level synchronization is essential to keep temporal errors within micrometer mechanical budgets. Uncalibrated clock drift between distributed sensor and motor nodes degrades multi-axis tool coordination, inducing high-frequency chatter in joint torque loops.

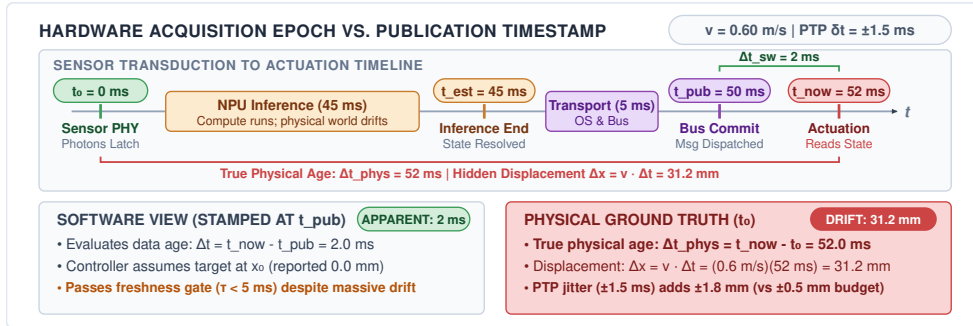
Even when a system records true acquisition epochs, physical AI architectures rely on distributed processing nodes whose local oscillator clocks drift independently, where each compute node operates on a distinct, time-varying local oscillator clock. Synchronizing these distributed clocks over standard bus networks introduces a bounded synchronization error, and that temporal uncertainty translates directly into spatial uncertainty.⁴ Consider an analytical example where an autonomous manipulator tracks a moving assembly target. The vision processor and the joint motion controller run on separate compute boards synchronized across an Ethernet bus using the Precision Time Protocol (PTP, IEEE 1588) to maintain a bounded clock offset of $\delta t = \pm 1.5 \text{ ms}$. If the target moves relative to the camera at an operational velocity of $v = 1.20 \text{ m/s}$, the clock offset alone creates an irrecoverable spatial positioning uncertainty of $e_{\text{sync}} = v \cdot \delta t = (1.20 \text{ m/s})(0.0015 \text{ s}) = 1.80 \text{ mm}$. In an assembly operation with a mechanical clearance tolerance of 0.50 mm, an uncalibrated clock synchronization bound of 1.5 ms causes physical interference and tool collision before any controller tracking latency is added. The physical error budget must therefore allocate margin for clock synchronization jitter alongside sensor resolution and structural deflection.

Real physical systems never sample all relevant state variables simultaneously. A tactile array might latch normal force readings at $t_1 = 12.0 \text{ ms}$, an optical tracking camera might finish sensor integration at $t_2 = 18.5 \text{ ms}$, and joint optical encoders might sample angular positions at $t_3 = 21.0 \text{ ms}$. These observations occur at different epochs and cannot be combined through direct algebraic operations. Fusing a camera observation at t_2 with an encoder observation at t_3 requires the consumer to project one state across the temporal gap $\Delta t = |t_3 - t_2| = 2.5 \text{ ms}$ using explicit kinematic models and bounded velocities (for example, propagating the historical camera observation along the joint velocity vector to reconcile it with the instantaneous encoder reading). Without the true acquisition epoch for each individual signal, the alignment calculation cannot determine the sign or magnitude of Δt . Extrapolating state without an accurate epoch causes the fusion algorithm to compute spatial offsets from fictitious temporal baselines, compounding errors across every fusion cycle.

For a consumer to use a state safely, the memory record must provide four explicit quantities: the physical state value, the precise acquisition epoch referenced to a synchronized system clock, the upper bound on clock synchronization uncertainty, and the maximum rate of physical state change. Given these quantities, the consumer computes the exact elapsed duration between the acquisition epoch and the execution deadline, evaluates the worst-case expansion of the error bound, and determines whether the state remains within task tolerances. Handing over a state without its acquisition epoch leaves the consumer blind to transport delays and scheduling jitter, turning deterministic control into open-loop guesswork. Because physical geometry also changes as bodies move through space, spatial transforms between reference frames are subject to the same temporal decay, carrying their own distinct measurement epochs.



Checkpoint 18.1: Hardware epochs and clock synchronization



Before analyzing coordinate frame hierarchies and lever-arm error propagation, verify your understanding of acquisition epochs and network clock jitter:

- ❑ **Epoch classification:** Can you distinguish between acquisition epoch (t_0), estimation epoch (t_{est}), and publication epoch (t_{pub}), explaining why stamping inference output with the transmission time hides physical displacement?
- ❑ **Fieldbus clock skew:** Can you calculate how bounded clock synchronization jitter over Ethernet PTP ($\delta t = \pm 1.5 \text{ ms}$) translates directly into physical positioning uncertainty ($e_{\text{sync}} = v \cdot \delta t$) for a high-speed moving target?
- ❑ **Multi-rate temporal alignment:** Can you explain why direct algebraic fusion of asynchronous sensor frames (e.g., 1000 Hz encoders vs 30 Hz depth cameras) requires explicit forward kinematic projection rather than simple timestamp matching?

18.4 Spatial Reference Frames

Geometric coordinates lack meaning without their reference frame, which moves in a machine. **Spatial transformation matrices** express translations and rotations between coordinate frames, allowing an onboard computer to map a target detected by a camera into joint torques applied at an end-effector (figure 18.2). In rigorous spatial mechanics, these rigid-body poses are parameterized as elements of the Special Euclidean group $SE(3)$.⁵

Standardized across robotics by ROS Enhancement Proposal 105 (REP 105),⁶ a robot maintains its spatial relationships as a **directed acyclic coordinate transform tree** $\mathcal{T} = (\mathcal{V}, \mathcal{E})$, where each vertex denotes a physical or virtual coordinate frame (such as `earth`, `map`, `odom`, `base_link`, camera sensors, or tool center points), and each directed edge stores a time-varying transform parameterized by joint angles (Lynch and Park 2017; Craig 2005).⁷ Classical mechanics treats geometric relationships as timeless, but in a physical computing system, they are time-stamped messages subject to transport delays, processing latency, and clock drift. A coordinate transform is valid only at the exact epoch when its underlying physical geometry was measured.

When an onboard computer chains together links across a robot arm, it multiplies a series of coordinate transforms to figure out where the end-effector is. If these transforms are measured at different times due to delays, asynchronous sampling, or dropped frames, the resulting position does not reflect the machine at any instant. It evaluates a synthetic, hybrid pose that never actually existed. For the rigorous matrix derivation of kinematic chains, see Appendix B.⁸

The cost of an expired transform is direct spatial displacement (figure 18.3). Applying the delay-as-distance principle from chapter 4 to kinematics, uncompensated latency translates directly into geometric positioning error through the lever arm of the linkages. For the rigorous cross-product derivation of this kinematic error propagation, see Appendix B.

Consider an analytical example of a manipulator holding a tool point at a distance of $\ell = 0.60 \text{ m}$ from its wrist assembly. Suppose the base and wrist rotate at an angular velocity of $\omega = 1.50 \text{ rad/s}$ while the arm translates through space at $v = 0.80 \text{ m/s}$. If a tracking pipeline publishes a camera-tool transform with an uncompensated staleness of $\Delta t = 40 \text{ ms} = 0.040 \text{ s}$, the translation of the

⁵ | **Special Euclidean group $SE(3)$:** The Special Euclidean group $SE(3)$ represents rigid-body spatial transformations as the semi-direct product $\mathbb{R}^3 \rtimes SO(3)$, parameterized by 4×4 homogeneous matrices combining rotational orientation with translational displacement. Operating directly on the Lie group avoids gimbal lock and singular parameterizations inherent in Euler angles. For the matrix exponential and Lie algebra $\mathfrak{se}(3)$ kinematics governing transform interpolation, refer to Appendix B.

⁶ | **Coordinate frame conventions (REP 105):** ROS Enhancement Proposal 105 (REP 105) formalizes kinematic frame hierarchies for mobile platforms ($\mathcal{F}_{\text{earth}} \rightarrow \mathcal{F}_{\text{map}} \rightarrow \mathcal{F}_{\text{odom}} \rightarrow \mathcal{F}_{\text{base}}$), isolating continuous odometric dead reckoning from discrete global map corrections. Confining non-continuous map updates to the $\text{map} \rightarrow \text{odom}$ transform prevents step discontinuities in local tracking loops. Injecting discontinuous coordinate jumps directly into inner motor controllers introduces derivative spikes that saturate inverter current and trip mechanical emergency stops.

⁷ | **Spatial DMA bus contention:** Traversing pointer-based spatial trees and hierarchical voxel grids induces severe CPU L1/L2 cache misses due to irregular memory strides. On unified-memory system-on-chip (SoC) architectures, concurrent direct memory access (DMA) transfers from multi-camera ingestion pipelines saturate shared LPDDR5 memory arbiters. This crossbar contention inflates spatial query latency tails from sub-microsecond expectations to tens of milliseconds, stalling real-time collision checks on the trajectory planner.

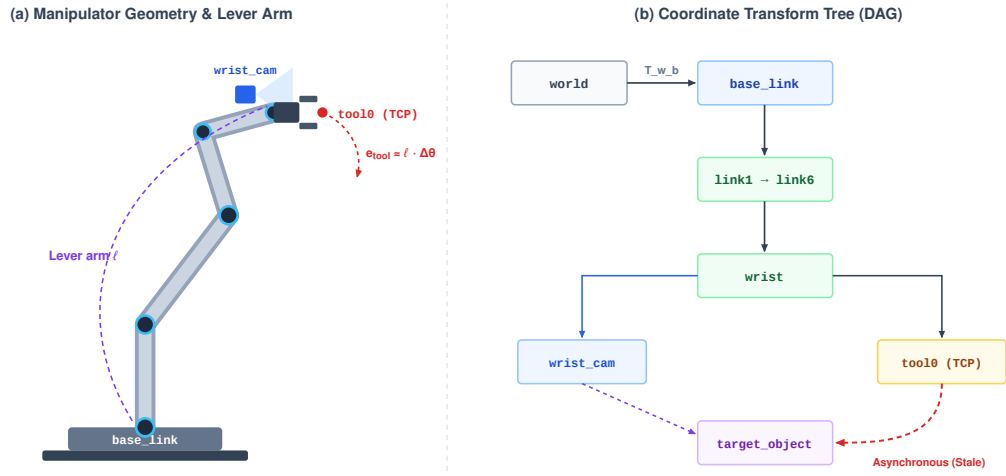


Figure 18.2: Manipulator kinematic frame tree and temporal dynamics: Articulated manipulator geometry (a) and corresponding directed acyclic coordinate transform tree in ROS tf2 (b). Each directed edge stores a homogeneous transformation matrix parameterized by joint state. Composing transforms across disparate measurement epochs injects lever-arm positioning error $e_{\text{tool}} \approx \ell \Delta\theta$, while an uncompensated asynchronous edge references stale coordinates (adapted from Craig 2005 and Foote 2013 (Craig 2005; Foote 2013)).

8 | Sequence locks for 6-DoF transforms: Because a 6-DoF spatial transform comprises multiple 64-bit floating-point registers, non-atomic concurrent reads during an in-flight write produce torn state records that combine old rotations with new translations. Lock-free sequence locks (seqlocks) paired with data memory barriers (DMB) protect transform handoffs by allowing reader threads to detect concurrent modifications via monotonic sequence counters without blocking writer threads. This lockless synchronization prevents low-priority perception publishers from inducing priority inversion on high-rate control threads. For memory barrier formalisms, see Appendix C.

frame drifts by $e_{\text{trans}} = v \cdot \Delta t = (0.80 \text{ m/s})(0.040 \text{ s}) = 0.032 \text{ m} = 3.20 \text{ cm}$. Over the same interval, the angular orientation rotates through $\Delta\theta = \omega \cdot \Delta t = (1.50 \text{ rad/s})(0.040 \text{ s}) = 0.060 \text{ rad}$. Through the lever-arm relation $e_{\text{rot}} \approx \ell \Delta\theta$, this angular shift swings the tool point across an arc of $e_{\text{rot}} \approx (0.60 \text{ m})(0.060 \text{ rad}) = 0.036 \text{ m} = 3.60 \text{ cm}$. Summing the translational shift and the lever-arm deflection yields a total tool-point positioning error of $e_{\text{tool}} \approx e_{\text{trans}} + \ell \Delta\theta = 3.20 \text{ cm} + 3.60 \text{ cm} = 6.80 \text{ cm}$. Comparing this 68.0 mm error against a physical task clearance tolerance of $\epsilon_{\text{tol}} = \pm 0.50 \text{ mm}$ reveals that an uncompensated staleness of just 40 ms breaches clearance by a factor of $68.0 \text{ mm} / 0.50 \text{ mm} = 136\times$. Because the transform tree silently evaluates without mathematical error, the controller commands the motor drives to push through solid metal before tactile sensors can detect the collision.



Checkpoint 18.2: Transform staleness and lever-arm deflection

Before exploring continuous volumetric mapping and attention buffer bounds, verify your understanding of kinematic frame staleness:

- ☐ **Transform staleness amplification:** Can you explain how uncompensated latency in base or wrist kinematic transforms amplifies across structural link lengths to induce large lever-arm displacements at the tool center point?
- ☐ **Silent evaluation failure:** Can you describe why a stale coordinate transform that continues to evaluate cleanly in linear algebra libraries is far more dangerous than a transform that raises a software exception?
- ☐ **Kinematic error allocation:** Can you distinguish between allocating error budgets purely to sensor calibration versus bounding end-to-end transport and thread scheduling jitter in distributed transform trees?



Systems Perspective 18.2: The leverage of staleness

Transform staleness multiplies through kinematic link lengths; an uncompensated millisecond of lag at the robot base swings the tool center point through centimeters of unobserved deflection.

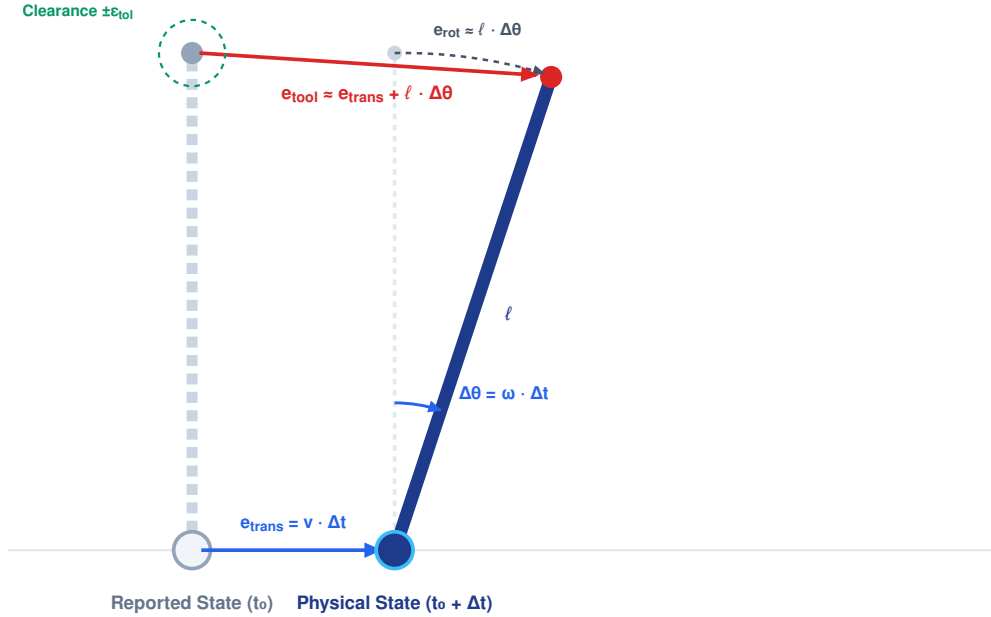


Figure 18.3: Transform staleness and lever-arm error propagation: Geometric error propagation under uncompensated transform latency Δt . Base translation shifts the origin by $e_{\text{trans}} = v \Delta t$, while joint rotation across link length ℓ swings the tool tip by $e_{\text{rot}} \approx \ell \omega \Delta t$, accumulating into total tool displacement $e_{\text{tool}} \approx e_{\text{trans}} + \ell \Delta \theta$ that breaches clearance tolerance $\pm \epsilon_{\text{tol}}$.

While kinematic trees govern the relative poses of articulated robot links, navigating and manipulating within unconstrained environments requires tracking the surrounding world itself. Beyond discrete kinematic vertices, an embodied machine must maintain continuous volumetric scene memory to represent unobserved surfaces and enforce free-space clearance.

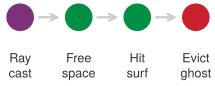
18.5 Volumetric Scene Memory and Attention Buffers

Beyond discrete kinematic frames, an embodied agent must also maintain continuous volumetric spatial memory to track unobserved obstacles, plan collision-free paths, and support real-time reactive grasping (figure 18.4). Formulated originally by Elfes (Elfes 1989), **occupancy grid mapping** models the world as a tessellation of spatial cells, recursively estimating occupancy state under sensor beam models. By executing **volumetric free-space raycasting** along sensor sightlines, spatial memory actively updates 3D voxel hierarchies (such as OctoMap or OpenVDB (Museth 2013)) and updates observed free cells under a sensor model. The foundational algorithm for volumetric raycasting is the 3D Differential Digital Analyzer (3D-DDA) fast voxel traversal (Amanatides and Woo 1987).⁹

Crucially, as the ray traverses free space from the sensor to the obstacle, the pipeline asserts **negative evidence** to clear unoccupied voxels. At the terminal measurement point where the obstacle is detected, positive evidence is integrated.¹⁰ This active assertion of negative evidence along valid sensor sightlines can invalidate a phantom obstacle track when the same region is observed free; occluded space still needs expiry.

⁹ | **Branchless DDA raycasting:** On Single Instruction, Multiple Threads (SIMT) GPU architectures, parallel 3D-DDA raycasting encounters execution divergence when adjacent threads within a 32-thread warp traverse spatial voxels of differing depths. High-throughput engines implement branchless DDA traversal, cache active voxel blocks in GPU Shared Memory, and apply atomic integer bitmasks (`atomicOr`) to register spatial updates without global DRAM lock contention. Eliminating warp serialization preserves memory bus bandwidth for concurrent neural network inference.

10 | Voxel traversal and log-odds updates: The 3D Differential Digital Analyzer (3D-DDA) evaluates ray-voxel intersections along optical sightlines in discrete volumetric grids. Traversed voxels in front of measured surfaces receive negative occupancy log-odds updates to clear transient obstacles, while voxels intersecting measured surface returns receive positive updates. This active clearing prevents ghost obstacles from permanently corrupting traversability graphs after dynamic obstacles depart. For the complete derivation of recursive log-odds Bayesian mapping, see Appendix A.



Active negative evidence along sightlines clears phantom obstacles without waiting for timeout decay.

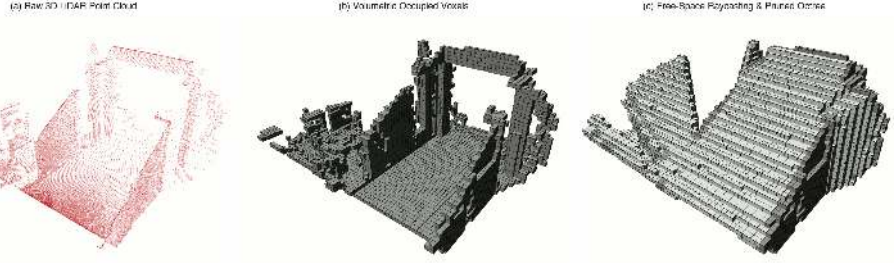


Figure 18.4: Probabilistic octree volumetric mapping: A corridor traversal reconstructed from raw point cloud into a probabilistic octree and then into explicit free, occupied, and unknown space (Hornung et al. 2013). The third panel is what distinguishes spatial memory from an accumulated point cloud, because raycasting records evidence of emptiness along sightlines, letting the map actively retire a phantom obstacle track instead of waiting for a timeout to expire it. (Credit: Hornung et al. / Autonomous Intelligent Systems Lab, University of Freiburg, CC BY 3.0).

For local surface reconstruction, **truncated signed distance fields (TSDFs)** (Curless and Levoy 1996; Newcombe et al. 2011; Oleynikova et al. 2017) store a *metric projective distance* $D(\mathbf{v}) \in [-\mu, +\mu]$ and fusion weight $W(\mathbf{v})$. The convention here is positive in front of an observed return along its sensing ray. Occupancy is a separate field: a return supports free-space updates before the hit and occupied evidence at the hit; space behind it remains unknown unless another observation reaches it. A projective TSDF is not the nearest-surface Euclidean distance needed for a conservative clearance decision.

For a calibrated depth image, transform voxel center $\mathbf{v}$ into camera coordinates $\mathbf{p} = [p_x, p_y, p_z]^\top$ and project it to pixel (u, v) . Its projective sample is $s = Z_{\text{meas}}(u, v) - p_z$. For a range ray, use the equivalent along-ray sample $s = r_{\text{hit}} - r_{\text{voxel}}$; both are positive before the hit. A negative sample just behind the measured return contributes to the local zero crossing under the surface model; it does not certify an unseen obstacle interior. Fusion uses a valid capture-time pose and refuses a sample older than a voxel's last evidence epoch. The observation weight w_t is dimensionless and the stored weight is capped at W_{max} . Near the return, clamp in **meters**:

$$d_\mu(\mathbf{v}) = \max(-\mu, \min(\mu, s)), \quad D_t(\mathbf{v}) = \frac{W_{t-1}(\mathbf{v})D_{t-1}(\mathbf{v}) + w_t d_\mu(\mathbf{v})}{W_{t-1}(\mathbf{v}) + w_t}$$

Interpolation of observed TSDF samples can supply a local approximate gradient near a zero crossing. Unknown voxels, projective bias, discretization, and dynamic objects limit that gradient. Before using distance for motion permission, a system must construct and validate a conservative occupancy or Euclidean signed distance field (ESDF), account for unknown space and error margins, and measure query latency on its chosen hardware. A surface-rendering or interpolation lookup alone supplies no collision guarantee.

For view synthesis, **3D Gaussian splatting (3DGS)**¹¹ (Kerbl et al. 2023; Matsuki et al. 2024) stores anisotropic primitives whose positions, opacity, and color can be optimized to render new camera views. That is valuable for recognition and inspection, but a Gaussian density or its derivative does not identify conservative occupied space or a signed clearance. Before a Gaussian map can inform motion permission, the system needs a validated geometry/occupancy conversion, an ESDF with unknown-space policy, or an independent range sensor. Rendering rate and collision-query latency are separate measurements. The handoff in figure 18.5 makes this distinction; figure 18.6 compares constructed storage scenarios rather than equal-accuracy benchmarks.

11 | Continuous 3D Gaussian splatting: A Gaussian primitive includes a center, shape, opacity, and color coefficients. Differentiable rasterization optimizes visual reconstruction; its spatial derivative is not a conservative surface normal or collision bound. Storage per primitive depends on packing and spherical-harmonic degree.

Contrast: A dense cubic grid stores $O(N^3)$ cells. Sparse occupancy and Gaussian maps store only allocated structure, but their required cell or primitive count depends on observed geometry and the accuracy target.

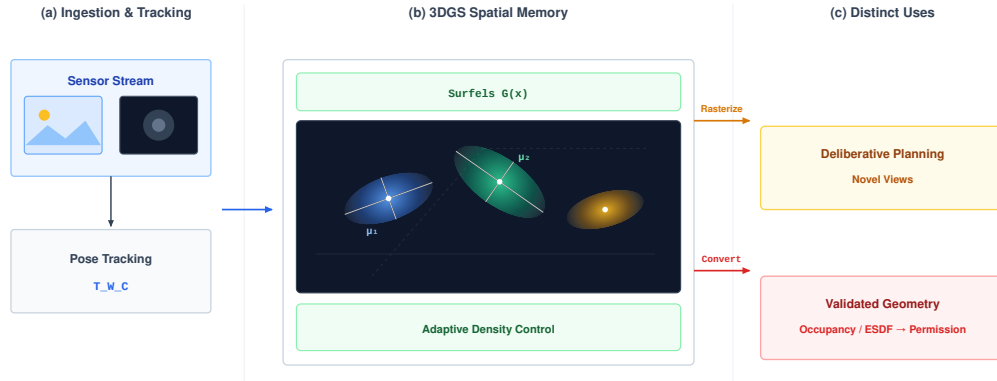


Figure 18.5: Gaussian memory and clearance handoff: Camera observations update a Gaussian scene for novel-view rendering. Its visual output does not directly supply conservative collision distance; a separately validated geometry or occupancy conversion and permission check are required (Kerbl et al. 2023; Matsuki et al. 2024).

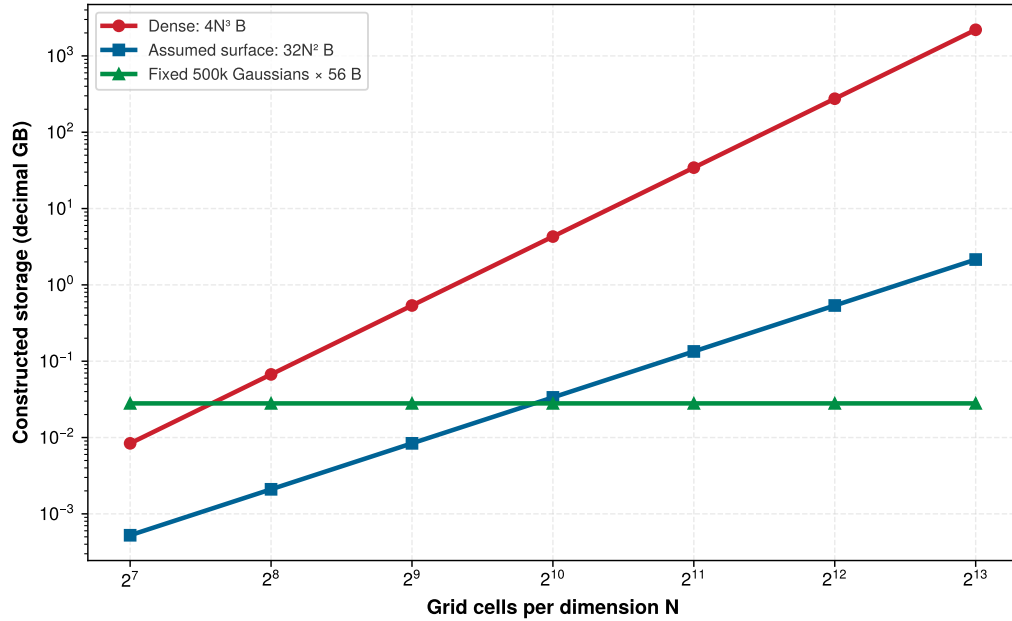


Figure 18.6: Constructed spatial-memory storage scenarios: A dense cube uses four bytes per voxel, the illustrative octree allocates one 32-byte surface cell per squared grid dimension, and the Gaussian scene holds a fixed count and packing while grid resolution varies. These are storage models, not equal-fidelity or measured runtime benchmarks; Gaussian count must rise if a stricter geometry target requires it.

For the constructed Gaussian packing in the plot, $K = 500,000$ and 56 bytes per primitive require 28 MB. Holding K fixed while the grid resolution changes shows a storage assumption, not equal surface accuracy. A deployment chooses its representation after measuring map completeness, update time, query time, and the conversion needed for conservative clearance.

Table 18.1: Spatial-memory representation decisions: Storage structure and useful output differ; this table intentionally omits unverified device-independent update and query microsecond ranges. The figure’s storage models are illustrative and do not hold accuracy constant.

Representation	Storage driver	Useful output	Permission limit
Octree occupancy (OctoMap)	Allocated free and occupied cells plus tree metadata	Observed free/occupied/unknown evidence	Unknown and aged cells require explicit policy; distance needs a separate transform
Sparse voxel blocks (VDB)	Allocated blocks, including traversed occupancy cells	Local updates and compact spatial indexing	Sparsity does not eliminate free-ray traversal or establish cache hit rate
Metric projective TSDF (Voxblox)	Allocated surface-band blocks and weights	Local reconstructed zero crossing	Projective distance and interpolation are not conservative Euclidean clearance; validate an ESDF or occupancy layer
Neural SDF	Network and feature/hash parameters	Approximate continuous field	Training coverage, error, and query deadline need validation
3D Gaussian splatting (Kerbl et al. 2023; Matsuki et al. 2024)	K primitives $\times$ implementation-dependent packing	Novel-view rendering	Appearance gradients need validated geometry/occupancy conversion or independent sensing

The representation trade-offs in table 18.1 explain why a rendered view is not a clearance bound. Beyond raw metric volumes and radiance fields, foundation-model-driven physical AI requires spatial memory to organize space into semantic hierarchies. The **3D dynamic scene graph (DSG)**, formulated by Rosinol et al. (2021) in Kimera, bridges metric SLAM with symbolic reasoning by structuring spatial memory into a five-layer hierarchical graph:

1. *Metric-Semantic Mesh (Layer 1)*: High-resolution triangular mesh capturing fine surface geometry with vertex semantic labels.
2. *Object and Agent Nodes (Layer 2)*: Bounded 3D geometric entities with oriented bounding boxes, visual embeddings, and tracked dynamic trajectories.
3. *Places and Freespace Topology (Layer 3)*: 3D Voronoi topology and navigable freespace spheres for planning collision-free paths.
4. *Rooms and Structural Enclosures (Layer 4)*: Functional architectural subdivisions partitioned by semantic walls and portals.
5. *Buildings and Global Topology (Layer 5)*: Top-level semantic graph governing long-horizon mission allocation.

By grounding high-level cognitive tokens in explicit metric coordinates through this hierarchical graph, 3D dynamic scene graphs allow foundation policies to query spatial memory at varying levels of abstraction without exhaustive metric searches across millions of voxels (figure 18.7).

The computational advantage of this bidirectional hierarchy lies in how it resolves the memory-compute tension on edge SoCs. Upward abstraction collapses gigabytes of dense triangular surface meshes into kilobyte-scale symbolic subgraphs, allowing foundation deliberators to evaluate long-horizon task plans without exhausting LPDDR5 bandwidth or choking KV caches with millions of raw geometric tokens. Conversely, downward grounding lets high-level spatial targets reach local geometric evidence. A TSDF zero crossing can help estimate a surface,¹² but a separately validated occupancy or ESDF layer, unknown-space policy, and deadline check must supply conservative clearance to the permission tier. However, grounding symbolic intent in continuous geometry requires the underlying metric layer to remain computationally tractable; as demonstrated in the following case study, naive dense voxelization quickly overwhelms edge memory architectures.

¹² | **Continuous TSDF surface integration:** A projective TSDF stores truncated signed displacement along a measurement ray, with a local zero crossing near an observed surface. Weighted fusion can reduce sensor noise, but interpolation and gradients are approximate and do not directly give nearest-surface Euclidean distance or a conservative collision force. Convert to a validated ESDF/occupancy layer before using the map for clearance.

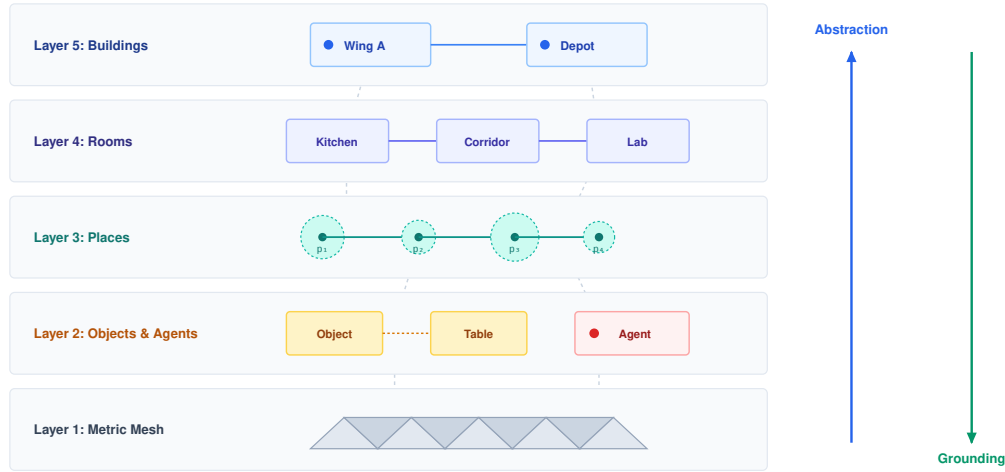


Figure 18.7: Hierarchical 3D dynamic scene graph architecture: Five-layer spatio-temporal memory organization bridging metric SLAM to symbolic foundation reasoning. Layer 1 captures triangular metric mesh geometry; Layer 2 extracts 3D objects and dynamic agents; Layer 3 models topological places via Voronoi freespace clearance spheres (p_i); Layer 4 structures architectural rooms and portals; and Layer 5 establishes global building mission topology. Vertical inter-layer edges coordinate upward abstraction (compressing metric surfaces into symbolic entities) and downward grounding (translating task tokens into metric coordinates) (adapted from Rosinol et al. (Rosinol et al. 2021) and Hughes et al. (Hughes et al. 2022)).

Napkin Math 18.1: DRAM bandwidth of 3D TSDF voxel grids

Practical Engineering Question: What are the distance-and-weight payload and modeled DRAM bandwidth of sparse versus dense TSDF representations, and how can the workload contend with policy inference on a shared host bus?

1. *Naïve Dense Volumetric Allocation:* Consider an active robotic manipulation workcell measuring $10 \text{ m} \times 10 \text{ m} \times 3 \text{ m} = 300 \text{ m}^3$.

- At isotropic resolution $\Delta v = 1.0 \text{ cm} = 0.01 \text{ m}$:

$$\text{Voxel Count} = \left(\frac{10}{0.01}\right) \times \left(\frac{10}{0.01}\right) \times \left(\frac{3}{0.01}\right) = 1000 \times 1000 \times 300 = 3 \times 10^8 \text{ voxels}$$

yielding 300,000,000 voxels.

- Distance-and-weight payload at 4 bytes per voxel (16-bit TSDF distance + 16-bit fusion weight), excluding occupancy, per-cell evidence epochs, and allocator overhead:

$$M_{\text{dense, 1cm}} = 3 \times 10^8 \times 4 \text{ B} = 1.20 \times 10^9 \text{ B} = 1.20 \text{ GB}$$

yielding 1.20 GB.

- If resolution is refined to $\Delta v = 5.0 \text{ mm} = 0.005 \text{ m}$ for precision tool contact:

$$\text{Voxel Count} = 2000 \times 2000 \times 600 = 2.4 \times 10^9 \text{ voxels} \Rightarrow M_{\text{dense, 5mm}} = 9.60 \text{ GB}$$

yielding 2,400,000,000 voxels and a partial payload of 9.60 GB. This payload alone exceeds an illustrative 8 GB memory budget.

2. *Sparse Voxel Hashing (Assumed TSDF-Band Fraction ≈ 3 percent):* Physical obstacles and workcell surfaces occupy only a thin 2D manifold within 3D volume. Grouping voxels into $8^3 = 512$ voxel leaf blocks and allocating only within the TSDF truncation band ($\mu = \pm 3 \text{ cm}$):

- Active occupied volume: ≈ 3.0 percent of $300 \text{ m}^3 = 9.0 \text{ m}^3$.
- Active voxel count at 1 cm: $9.0 \times 10^6 \text{ voxels} \Rightarrow \approx 17,578 \text{ blocks (17,579 blocks)}$.

- Distance-and-weight payload plus the stipulated 32-byte leaf header:

$$M_{\text{sparse}, 1\text{cm}} = 17,578 \text{ blocks} \times (512 \text{ voxels} \times 4 \text{ B} + 32 \text{ B header}) \approx 36.5 \text{ MB}$$

yielding 36.6 MB.

- At 5.0 mm resolution: this partial allocation is $M_{\text{sparse}, 5\text{mm}} \approx 292 \text{ MB}$ (yielding 292.5 MB). A full map must also budget the separate occupancy layer, per-cell epochs, block lookup and allocator metadata, and capacity for unknown-space policy.

3. *Traversal, Fusion, and DRAM Traffic at 60 Hz:* A 640×480 depth stream at 60 Hz supplies 18.432 million rays per second. In this constructed 3.0 m ray at 1 cm pitch, DDA visits 300 cells. Suppose 294 cells before the hit receive separate occupancy free-ray updates and six cells near the return receive metric TSDF fusion. Sparse TSDF allocation saves *map capacity*; it does not remove 300 traversals or the 294 occupancy updates.

At eight bytes of assumed read/write traffic per update, sending all 300 updates to DRAM costs 44.2 GB/s. If measurement on the selected architecture establishes a 95 percent cache hit fraction for **both** occupancy and TSDF updates, the modeled DRAM transaction demand is 2.2 GB/s. The corresponding reduction is 20× under that cache assumption. Different ray overlap, atomics, evictions, and writeback behavior change it; the 95 percent hit rate is an input to validate, not a VDB property. The independent permission controller's memory remains segregated from this host workload.

Systems insight: Sparse allocation yields 33× less *modeled TSDF payload* in this surface-fraction scenario; it is not a full-map memory ratio. Bus demand instead depends on measured locality and all traversed-cell work. Reserve host bandwidth and deadline margin using the observed transaction tail rather than a storage ratio.



Checkpoint 18.3: Volumetric spatial memory and memory bus saturation

- ☐ **Volumetric scaling penalty:** Can you explain why cubic voxel scaling ($\mathcal{O}(N^3)$) makes dense 3D grids silicon-infeasible on embedded edge SoCs when voxel resolution is refined from 1 cm to 5 mm?
- ☐ **DRAM bus starvation:** Can you describe how un-cached 60 Hz RGB-D raycasting across dense voxels consumes over 85 percent of shared LPDDR5 bandwidth, and why a 95 percent measured hit rate, including free-ray occupancy updates, is needed for the illustrated DRAM reduction?
- ☐ **Negative evidence raycasting:** Can you explain how active line-of-sight raycasting using 3D-DDA clears phantom obstacles without relying on arbitrary software timeout decay?

To integrate incoming sensory point clouds while actively evicting stale obstacle geometry, the spatial memory engine executes the structured raycast update detailed in algorithm 18.1.



In this assumed workload, caching reduces modeled DRAM transactions; ray traversal and occupancy clearing remain.

Algorithm 18.1 Occupancy Raycast and Metric TSDF Fusion

Require: returns $\mathcal{P}$ captured at t_0 in one capture clock; valid pose; occupancy $\mathcal{O}$; metric TSDF $\mathcal{V}$; truncation μ ; evidence half-life $t_{1/2}$; per-cell serialized updates

Ensure: observed free/occupied cells and near-surface TSDF samples updated without refreshing unseen cells

```

1:  $(t_{0,c}, \epsilon_{\text{sync}}) \leftarrow \text{CONVERTCAPTURETOCONTROLLERCLOCK}(t_0)$ 
2: if clock conversion invalid or  $t_{\text{now},c} - t_{0,c} + \epsilon_{\text{sync}} > \tau_{\text{max,age}}$  then
3:   return STATUS_FRAME_TOO_STALE
4: end if
5: if any target occupancy or TSDF cell has  $t_{\text{last}} > t_0$  then
6:   return STATUS_OUT_OF_ORDER  $\triangleright$  preflight before modifying the map
7: end if
8: for each return  $\mathbf{p}_k \in \mathcal{P}$  do
9:    $(\mathbf{r}_{\text{dir}}, r_{\text{hit}}) \leftarrow \text{RAYFROMPOSEANDRETURN}(\mathbf{p}_k)$ 
    $\triangleright$  Occupancy: measured free ray and hit, not TSDF distance
10:  for each cell  $c$  traversed before the hit do
11:     $c \leftarrow \text{GETORALLOCATEOCCUPANCY}(\mathcal{O}, c)$ 
12:     $c.L \leftarrow \text{CLAMPLOGODDS}(c.L + L_{\text{free}}); c.t_{\text{last}} \leftarrow t_0$ 
13:  end for
14:  update the observed hit cell with  $L_{\text{occupied}}$  and epoch  $t_0$ 
    $\triangleright$  TSDF: fuse metric samples only in a narrow surface band
15:  for each voxel  $v$  with  $|r_{\text{hit}} - r_v| \leq \mu$  do
16:     $\text{vox} \leftarrow \text{GETORALLOCATETSDF}(\mathcal{V}, v)$ 
17:     $d_\mu \leftarrow \text{CLAMP}(r_{\text{hit}} - r_v, -\mu, +\mu) \triangleright$  positive before hit
18:     $W_{\text{old}} \leftarrow 0$  if  $\text{vox}.W = 0$ , else  $\text{vox}.W 2^{-(t_0 - \text{vox}.t_{\text{last}})/t_{1/2}}$ 
19:     $w_{\text{obs}} \leftarrow 1 \triangleright$  dimensionless illustrative confidence
20:     $\text{vox}.D \leftarrow (W_{\text{old}} \text{vox}.D + w_{\text{obs}} d_\mu) / (W_{\text{old}} + w_{\text{obs}})$ 
21:     $\text{vox}.W \leftarrow \min(W_{\text{old}} + w_{\text{obs}}, W_{\text{max}}); \text{vox}.t_{\text{last}} \leftarrow t_0$ 
22:  end for
23: end for
24: return STATUS_MAP_UPDATED

```

The map manifest in table 18.2 fixes units and update metadata. A motion consumer still checks each cell used by its path, including unknown cells, and derives conservative clearance from a separately validated layer.

Table 18.2: Volumetric belief manifest and cell evidence: The map header describes layout and update time, while cell-local acquisition epochs determine whether the particular path region is fresh. The consumer treats unobserved or expired cells as unknown unless a separately validated policy permits traversal.

Map field	Unit or type	Meaning and consumer check
voxel_grid_origin, voxel_resolution_m	m; m	Spatial anchor and pitch; match the active frame and calibration
tsdf_trunc_margin_m	m	Metric projective distance clamp μ ; not an ESDF clearance
active_block_count	count	Allocated TSDF blocks; occupancy may have separate blocks
map_update_epoch	capture-clock ns	Last map publication/update; not freshness of every cell
cell_evidence_epoch	capture-clock ns per cell	Last physical observation for that TSDF or occupancy cell; never refreshed by prediction or unrelated map updates
free_space_sweep_epoch	capture-clock ns per occupancy cell	Last valid observed free ray through that cell
decay_half_life_s	s	If exponential weight decay is enabled, $W(t) = W_0 2^{-\Delta t/t_{1/2}}$; this does not renew evidence
clock_conversion, sync_error_ns	mapping; ns	Convert capture epochs to the consumer's monotonic clock with bounded error

While volumetric occupancy grids, TSDF maps, and coordinate trees defend metric clearance and geometric relationships, modern Physical AI systems increasingly deploy foundation models to maintain task context. In recurrent world models (such as DreamerV3 (Hafner et al. 2025)), temporal belief across transitions is compressed into fixed-size latent state vectors with $\mathcal{O}(1)$ memory. In autoregressive Vision-Language-Action (VLA) foundation policies (such as RT-2 (Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023), OpenVLA (Kim et al. 2024), and Octo (Octo Model Team et al. 2024)), however, temporal context is maintained through the transformer attention Key-Value (KV) cache. In these autoregressive architectures, every interaction step appends new multimodal tokens—including multi-view visual patch embeddings, proprioceptive joint states, and predicted action tokens—directly into memory.

The physical memory footprint of this KV-cache scales linearly with cumulative sequence context length L :

$$\text{Memory}_{\text{KV}} = 2 \times N_{\text{layers}} \times N_{\text{heads}} \times d_{\text{head}} \times L \times b_{\text{bytes}}$$

where $b_{\text{bytes}} = 2$ for 16-bit half-precision floating point (fp16 or bf16).

For an illustrative 32-layer decoder with 32 heads of width 128 in 16-bit storage, the cache adds 524,288 bytes (about 0.524 MB) per token. At an assumed 100 multimodal tokens per second, that is 52,428,800 bytes/s (about 52.4 MB/s). A 4,294,967,296-byte allocation fills in 81.92 s if nothing is evicted. This is a capacity calculation for a stated architecture and tokenization rate, not a measured Jetson policy latency. Cached decoding avoids recomputing all prior keys and values, but attention work for the next token grows as $\mathcal{O}(L)$ with live context length L ; cumulative work over an uncapped rollout grows as $\mathcal{O}(L^2)$.

Streaming attention retains a small number N_{sink} of initial attention-sink tokens and a rolling window of W recent tokens (Xiao et al. 2024). Sink tokens help the attention mechanism maintain normalization even when those tokens carry no task semantics. Task instructions and older physical facts therefore need separately managed task/spatial state; pinning four early tokens does not guarantee instruction retention or long-term memory. At 100 tokens per second, $W = 512$ spans only 5.12 s of recent tokens. The window evicts earlier observations even when their physical consequences persist, so spatial belief records must carry validated evidence across that eviction.

The cache ceiling is

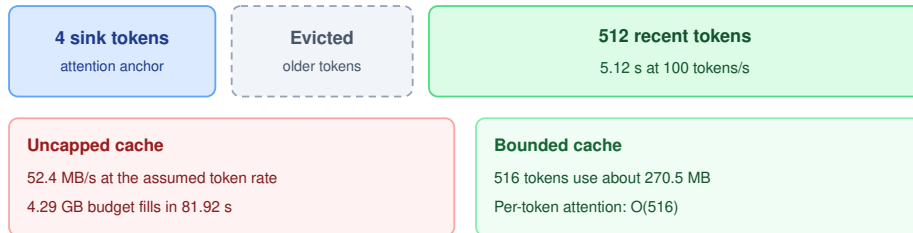
$$\text{Memory}_{\text{KV,max}} = 2N_{\text{layers}}N_{\text{heads}}d_{\text{head}}(N_{\text{sink}} + W)b_{\text{bytes}} \quad (18.1)$$

Applying equation 18.1 with $N_{\text{sink}} = 4$ and $W = 512$ yields a 516-token cache consuming 270,532,608 bytes (about 270.5 MB). Per-token cached attention work is then $\mathcal{O}(N_{\text{sink}} + W)$ and cumulative work over T new tokens is $\mathcal{O}(T(N_{\text{sink}} + W))$, aside from model layers and other costs. This bounds cache allocation, not the quality of long-horizon decisions or the deadline of a particular accelerator.



Checkpoint 18.4: VLA key-value cache growth and ring buffers

KV cache: capacity and evidence horizon



Sink tokens do not preserve task meaning; keep spatial and task state separately.

- **Autoregressive memory leakage:** Can you explain why unconstrained Key-Value caching in vision-language-action policies grows linearly in memory and, at the stated token rate and budget, fills the available cache unless tokens are evicted?

- ❑ **Attention sinks:** Can you describe why attention-sink tokens support the attention mechanism but cannot by themselves preserve task instructions or physical history?
- ❑ **Complexity bounding:** Can you distinguish between $O(L)$ cached attention work for one new token and $O(L^2)$ cumulative uncapped work from a fixed-window $O(W)$ per-token computation?

Definition 18.2: VLA KV-cache ring buffer

VLA KV-cache ring buffer is an embedded transformer memory architecture that caps autoregressive attention cache allocation to a constant ceiling ($\text{Memory}_{\text{KV,max}} \propto N_{\text{sink}} + W$) by retaining initial attention-sink tokens (N_{sink}) while cycling continuous sensory and action tokens through a rolling circular FIFO buffer (W).

1. **Significance:** Unbounded key-value caching in vision-language-action (VLA) models consumes memory linearly with execution time ($\mathcal{O}(L)$), which can exhaust a finite edge-device allocation during long-horizon tasks.
2. **Distinction:** Unlike naive token pruning that discards arbitrary historical states, ring buffering preserves structural attention sinks alongside a recent token window; explicit task state and spatial memory must preserve older semantics.
3. **Common pitfall:** Evicting initial prompt tokens ($N_{\text{sink}} = 0$). Removing attention sinks can degrade a model trained to rely on them; retaining them does not guarantee task coherence after middle tokens are evicted.

For motion permission, spatial representations must carry evidence ages and error limits appropriate to the task and operating region. Extending the proposal-permission architecture from section 6.1, a representation without an epoch is an unverified assertion, and an estimate whose age exceeds task tolerance must be excluded from motion proposals before the independent permission tier considers actuator commands. When direct geometric observations become unavailable, the machine cannot simply halt every operation at the first missed sensor frame. Extending action across brief sensing gaps requires estimators that propagate physical belief forward until new measurements arrive to confirm or correct the state.

18.6 Belief Through Occlusion

When direct sensing fails, a physical AI system uses an internal transition model to update its state estimate. A Kalman filter updates state estimates using plant dynamics,¹³ while a learned policy might predict the next relative pose using an autoregressive network. In both implementations, propagating a state across an unobserved interval does not generate new information. Prediction mathematically extrapolates old evidence forward in time. It preserves the utility of an observation by compensating for expected physical motion, but it does not reset the age of the underlying measurement. If an optical sensor captured the position of a workpiece at time $t = 0$ ms, an estimator projecting that position to $t = 100$ ms still relies entirely on evidence that is 100 ms old. Treating a predicted state as an active observation confuses mathematical continuity with physical truth.

This distinction becomes critical during physical contact. Consider an articulated arm tasked with inserting a hardened steel pin into a close-tolerance bearing sleeve. A wrist-mounted stereo camera tracks the pin and the sleeve during the approach phase. When the parallel-jaw gripper descends to seat the pin, the mechanical structure of the gripper and the fingers completely occludes the camera line of sight 40 mm above the fixture. For the remaining distance, the nervous system receives no direct optical measurements of the mating interface. The tracking pipeline switches to forward propagation, computing the relative pose between the tool center point and the sleeve bore from the last verified camera frame combined with joint encoder integration.

Every physical model simplifies reality, causing error between the projected state and the physical world to expand continuously during the blind interval. In state estimation, this dispersion is

¹³ | **Bayesian state estimation provenance:** Kalman (1960) introduced recursive minimum mean-square error state estimation for linear dynamic systems. Moravec and Elfes (1985) generalized these recursive Bayesian updates to spatial occupancy grids, modeling sensor beam uncertainty through explicit range-bearing distributions. In embodied systems, uncompensated propagation across extended sensor dropouts causes Bayesian belief distributions to disperse, requiring explicit invalidation horizons to prevent open-loop control divergence.

governed by **occlusion covariance expansion**—the continuous, physics-governed widening of uncertainty during sensor line-of-sight dropouts. Mathematically formalized by stochastic models (such as the continuous Lyapunov / Riccati covariance propagation equation, detailed in section B.6), this expansion quantifies the growing doubt about where a part actually is, bounding the unobserved divergence caused by unknown drift velocities and unexpected physical disturbances before the sensor regains visibility. For this deterministic worked model, suppose sensor characterization and disturbance limits establish an initial position-error bound $E_0 = 0.30$ mm at the moment of occlusion ($\Delta t = 0$ ms). During the unobserved descent, two bounded disturbance terms degrade the prediction. First, unobserved mechanical vibration and workpiece thermal expansion introduce an unmodeled drift velocity bounded by $v_{\text{drift}} = 4.0$ mm/s. Second, minor compliance in the arm joints and unmodeled drive backlash create an unobserved disturbance acceleration bounded by $a_{\text{dist}} = 30.0$ mm/s². Conditional on those velocity and acceleration bounds remaining valid, the error-growth envelope $E(\Delta t)$ over an occlusion duration Δt follows the quadratic expansion:

$$E(\Delta t) = E_0 + v_{\text{drift}}\Delta t + \frac{1}{2}a_{\text{dist}}(\Delta t)^2 \quad (18.2)$$

The insertion operation requires a maximum radial alignment error of $\epsilon_{\text{tol}} = 0.80$ mm to clear the lead-in chamfer of the sleeve. Substituting the parameters into the envelope (equation 18.2) yields the quadratic relation $15.0(\Delta t)^2 + 4.0\Delta t - 0.50 = 0$. Solving for the positive root gives a maximum admissible rollout duration of $\Delta t_{\text{max}} \approx 93$ ms. If the descent velocity is 150 mm/s, the arm travels 14.0 mm during this 93 ms window. Throughout this interval, the internal simulation runs smoothly, floating-point coordinates update without numerical instability, and trajectory generators report valid setpoints. Yet at $\Delta t = 94$ ms, the bound of the predicted error exceeds the physical clearance of the chamfer. The prediction has stopped functioning as evidence. Allowing the motor drivers to continue downward motion on predicted coordinates past 93 ms means accepting that the pin may strike the rim of the sleeve, generating lateral forces that can fracture the tooling. The independent permission tier must refuse a descent that relies on this belief by that boundary and select a guarded response feasible within the remaining stopping and clearance margin.

When the sensor pipeline reacquires line of sight, the system cannot simply overwrite the propagated state with the new measurement, nor can it blindly pass the raw measurement into the downstream controller. Sensor readings obtained immediately after occlusion often suffer from transient lighting reflections, partial edge visibility, or correspondence errors. Before blending or resetting the state, the estimator must calculate the **innovation**, a mathematical reality check defined as the difference between the new measurement and where the internal model predicted the pose would be. If the new measurement falls inside the calculated error envelope $E(\Delta t)$, the measurement confirms the model's drift assumptions, allowing the estimator to collapse the uncertainty and reset the evidence clock. If the incoming measurement falls outside the envelope, either the physical object moved unexpectedly or the sensor returned an outlier. Snapping the coordinate frame directly to an erroneous outlier tells the robot it has instantly teleported, introducing an instantaneous position step into the trajectory planner. The resulting derivative demand requests physically impossible, infinite joint acceleration to compensate, instantly triggering the actuator thermal limits and overcurrent faults detailed in section 4.3. The nervous system must isolate the discrepancy, evaluate whether the measurement is valid, and reject discontinuous jumps before updating actuation targets.

The identical failure mechanism governs continuous process machines that regulate fluids or gases. In an automated chemical dispensing loop, a motor-driven needle valve regulates fluid delivery into a reactor. A Coriolis mass flow meter measures liquid throughput, but because the sensor requires fluid mass to transit its measurement tubes, it sits 1.5 m downstream of the valve orifice. This physical distance introduces a 120 ms transport delay—the lag between the valve moving and the fluid actually reaching the sensor—at nominal flow rates. To maintain stable closed-loop pressure without waiting for the delayed sensor frame, an internal flow model predicts real-time throughput from valve stem displacement and differential pressure. This short model-based propagation allows the valve actuator to make rapid micro-adjustments during normal operation. If downstream viscosity shifts or an inline filter begins to clog, the physical plant diverges from the static model. If the controller relies on its internal model beyond the 120 ms transport window without reconciling

the prediction against delayed meter readings, the calculated flow diverges from physical reality. Because the internal prediction does not reflect the physical clog, the controller continues to open the valve to satisfy a phantom deficit, over-pressurizing the manifold and physically bursting the upstream fitting.

In both discrete manipulation and continuous fluid control, model prediction serves as a temporary bridge across missing or delayed observations. The bridge holds only while the accumulated model error remains smaller than the mechanical tolerance of the task. Forward propagation does not manufacture certainty out of thin air, and its authority over physical actuators must expire the moment the error envelope outgrows the clearance of the physical world.

18.7 The Rate of Belief Decay

Physical tasks impose strict clearance and error limits. In a precision peg-in-hole insertion, the chamfer geometry defines an allowable radial alignment error of 0.8 mm before the tool collides with the shoulder of the bore. In a continuous dispensing process, a fluid temperature error exceeding 2.5 °C alters viscosity enough to double the bead width and flood the channel. These physical bounds define the maximum tolerable error budget $E_{\max}$. The task of **state estimation** is to track an error envelope $E(\Delta t)$ whose meaning is explicit: a bounded-disturbance reachable set under stated assumptions or a calibrated quantile with declared residual risk.

The moment an observation frame ends, that envelope expands as unmeasured dynamics accumulate. Between sensor updates, the state estimation subsystem propagates the state forward by integrating commanded forces through an internal model of the plant—extrapolating kinematic displacement from commanded actuator torques and assumed plant dynamics without feedback confirmation.¹⁴

Napkin Math 18.2: Continuous-time covariance propagation

In continuous-time estimation theory, spatial uncertainty during unobserved intervals evolves through coupled state dynamics and stochastic disturbance injection. Errors in velocity estimation propagate directly into positional displacement over elapsed time, while unmodeled physical disturbances—such as air currents, structural vibrations, or friction fluctuations—continually inject stochastic energy that accumulates monotonically across the blind window. For the formal stochastic differential equations and the full derivation of the covariance matrix expansion, see Appendix B.

This closed-form derivation exposes the three distinct physical mechanisms governing spatial memory decay:

1. **Initial Velocity Dispersion** ($\sigma_v^2(0)(\Delta t)^2$): An unmeasured velocity estimation error at the onset of occlusion propagates linearly into positional displacement ($\sigma_v(0)\Delta t$), driving quadratic variance expansion over elapsed time.
2. **Stochastic Acceleration Diffusion** ($\frac{1}{3}q_c(\Delta t)^3$): Unmodeled stochastic forces—such as motor torque ripple, structural vibration, or ground reaction perturbations (modeled mathematically as continuous white noise q_c)—undergo double temporal integration from acceleration to velocity and position. This double integration induces cubic variance growth, causing rapid spatial dispersion over extended sensing dropouts.
3. **Deterministic Disturbance Acceleration** ($\frac{1}{2}a_{\text{dist}}(\Delta t)^2$): Any unmodeled steady force—such as a dragging power cable or persistent sliding friction—bounded by $|a_{\text{dist}}|$ acts as a constant acceleration, deflecting the physical body quadratically with time.¹⁵

As an analytical example (a bench realization will depend on specific motor thermal curves and bearing preload), consider an articulated arm holding a tool tip where the operation requires positioning error to remain below $E_{\max} = 1.0$ mm (figure 18.8). For this deterministic illustration, assume initial position error is at most $E_0 = 0.1$ mm, unobserved velocity error at most $v_{\text{drift}} = 12.0$ mm/s, and acceleration disturbance at most $a_{\text{dist}} = 50$ mm/s² throughout the stated operating region. Under those stipulated bounds, position error follows the kinematic envelope in equation 18.2:

¹⁴ | **Dead-reckoning cubic variance expansion:** In inertial dead reckoning, continuous double integration of unmodeled white noise acceleration into position state induces cubic variance growth ($_{pp}(t) \propto \frac{1}{3}q_c t^3$). Because positional uncertainty compounds rapidly over unobserved intervals, inertial estimation without external visual or tactile anchoring quickly violates platform clearance budgets. This stochastic variance model alone does not set a hard navigation deadline; a task-specific bound or risk criterion and feasible response are still required.

¹⁵ | **Non-smooth contact dynamics:** Physical surface contact introduces set-valued Coulomb friction and non-smooth stick-slip transitions that break linear covariance propagation models. Transitioning between stick and slip modes injects discontinuous velocity steps that instantly invalidate continuous kinematic drift bounds. Managing these contact regime transitions requires hybrid state estimators that switch between distinct error growth matrices upon detecting normal force thresholds.

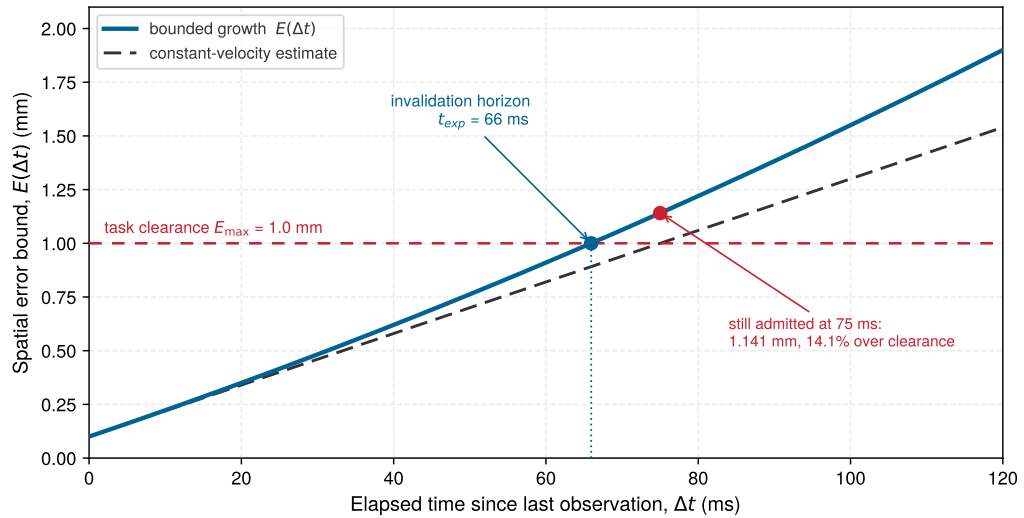


Figure 18.8: Belief uncertainty and invalidation horizon: Spatial error bound $E(\Delta t)$ expanding quadratically under unobserved forward propagation until intersecting task clearance $E_{\max} = 1.0$ mm at invalidation horizon $t_{\text{exp}} = 66$ ms. A lower lifecycle timeline marks transitions from valid through degraded to expired operational regimes. Belief has a shelf life governed by physical disturbance dynamics; sensor re-acquisition resets the evidence clock, whereas republishing cached matrices conceals physical drift.

Evaluating this growth bound with the stated parameters yields $E(\Delta t) = 0.1 + 12.0\Delta t + 25.0(\Delta t)^2$, with Δt in seconds and $E(\Delta t)$ in millimeters. Setting $E(\Delta t) = E_{\max} = 1.0$ mm gives the quadratic equation $25.0(\Delta t)^2 + 12.0\Delta t - 0.9 = 0$. The single positive real root gives $\Delta t = 0.066$ s (66 ms), so a proposal depending on this belief must expire by 66 ms under the stipulated bound. If the task is upgraded to a precision micro-clearance bore with $E_{\max} = 0.30$ mm, the allowable unobserved duration shrinks to $\Delta t \approx 16.1$ ms ($25.0(\Delta t)^2 + 12.0\Delta t - 0.20 = 0$). Tightening mechanical tolerance by $3.3\times$ shrinks the allowable unobserved duration by $4.1\times$, requiring sensor re-acquisition or compliant impedance holding within two 100 Hz control frames. In contrast, if an estimator naively assumes constant velocity ($a_{\text{dist}} = 0$), it predicts a valid horizon of $\Delta t_{\text{linear}} = 0.90/12.0 = 75$ ms. At $\Delta t = 75$ ms, the modeled bound reaches $E(0.075) = 1.141$ mm, exceeding the clearance by 14.1 percent. A constant-velocity-only admission rule would keep accepting the estimate for 9 ms beyond the bounded-model deadline; whether contact occurs depends on the path and geometry.



Checkpoint 18.5: Occlusion covariance and invalidation horizon

Before examining empirical covariance calibration and operational decay tables, verify your understanding of occlusion horizons:

- ☐ **Physical invalidation horizon:** Can you explain why the shelf life of a spatial belief during sensor occlusion is a physical property set by task clearance and disturbance dynamics rather than an arbitrary software timeout?
- ☐ **Disturbance acceleration hazards:** Can you describe why linear constant-velocity extrapolation underestimates physical error growth, causing controllers to continue executing open-loop motions past the true expiration deadline?
- ☐ **Tolerance sensitivity:** Can you distinguish between the gentle invalidation horizons permitted by generous macro-clearances versus the rapid, sub-control-frame expirations imposed by precision micro-clearance assemblies?

An estimator's internal **covariance matrix** reflects its stochastic model, not a measured hard maximum. A safety argument has two different choices. A **bounded-disturbance** envelope needs

justified limits on initial error, velocity, acceleration, clock conversion, and mode changes; then the 66 ms and 93 ms worked roots are conditional worst-case deadlines within that region. Alternatively, staged dropouts and external ground truth can calibrate a specified residual quantile (for example P_{99} or $P_{99.9}$) under a named test distribution. That yields a probabilistic admission limit with a one-percent or 0.1-percent tail before other model and coverage errors, not a guarantee. The record and consumer must declare which regime applies, monitor its operating envelope, and refuse dependent motion or choose a validated feasible fallback when evidence exceeds it.

The **belief state invalidation horizon** (formalized as the expiry time t_{exp} in equation 18.3) is defined as the earliest time at which the validated error envelope reaches the physical task tolerance:

$$t_{\text{exp}} = \inf\{\Delta t \geq 0 : E(\Delta t) \geq E_{\text{max}}\} \quad (18.3)$$

Under this definition, expiry is neither an arbitrary communication watchdog timer nor a fixed 100 ms heuristic chosen for scheduling convenience. It is a physical deadline dictated by the intersection of task geometry, actuator authority, and disturbance dynamics. When the last valid observation occurs at timestamp t_0 , the belief state carries an absolute expiration timestamp $t_{\text{deadline}} = t_0 + t_{\text{exp}}$. Beyond t_{deadline} , downstream control loops cannot assume the estimated pose lies within the required mechanical clearance.

Disturbance magnitudes and plant dynamics vary with operating conditions, so t_{exp} is not a static scalar for the entire machine. A robotic arm traversing free space at 0.2 m/s incurs small disturbance torques, yielding a validated error growth that permits an unobserved window of 140 ms. When the same arm moves at 1.5 m/s carrying an unmodeled 5.0 kg payload, centrifugal forces and joint compliance accelerate error accumulation, shrinking t_{exp} to 22 ms. Transitions between free motion and surface contact introduce violent, impulsive contact forces—such as the sudden jolt of striking a rigid table—that instantly alter velocity and invalidate smooth kinematic drift models within milliseconds. A production state interface must therefore evaluate decay rates across distinct operating regimes, selecting the active expiry limit based on payload mass, velocity, and contact state.

The derived expiration deadline t_{exp} spans multiple orders of magnitude depending on the entity kinematics and task clearances, as illustrated in table 18.3. Its six roots use stipulated, different disturbance and task models; they show how an expiry can range from milliseconds to hundreds of milliseconds without asserting those numbers for every fixture, person, or robot.

Table 18.3: Illustrative belief-expiry roots: Each root solves $E(t) = E_{\text{max}}$ for the stipulated entity-specific model, not a measured universal deadline or standards-prescribed clearance. To use one operationally, validate its initial error and disturbance bounds (or declare a calibrated quantile and residual risk), clock conversion, task geometry, and feasible response.

Illustrative entity	Stipulated error envelope $E(t)$ (mm; t in s)	Task limit (mm)	Root	Response to qualify
Fixed fixture	$0.02 + 1.0t^2$	0.20	424 ms	Hold approach; remeasure the datum
Conveyor workpiece	$0.30 + 25t + 75t^2$	2.00	58 ms	Refuse descent; keep a validated standoff
Moving coworker	$15 + 1200t + 3000t^2$	150	92 ms	Revoke dependent motion; stop only if braking clearance remains
Mobile robot	$30 + 1800t + 1250t^2$	300	137 ms	Refuse corridor advance; use a feasible fallback
Tool center point	$0.05 + 50t + 9000t^2$	0.80	6.8 ms	Refuse precision feed; validate compliance response
Dropped component	$2 + 4905t^2$	25	68.5 ms	Cancel intercept; protect the manipulator

18.8 State Estimation Bounds

A learned model predicting a physical quantity does not prove the quantity is observable. In state estimation, a state trajectory is **observable** if and only if the history of available sensor outputs over a

finite time window uniquely determines the initial state of the machine—guaranteeing that physical trajectory variations produce distinct sensor trajectories rather than mapping into indistinguishable measurement sequences. Building on the causal boundary established in section 2.2, when two distinct physical state histories produce the exact same sequence of sensor readings, no mathematical operation, filter, or neural network running on the nervous system can determine which physical trajectory the body actually followed. The machine receives identical bitstreams from its sensor interfaces across both realities.

To see why predictability differs from observability, consider an analytical example of a robotic end-effector regulating contact force against a rigid surface. Let the true normal contact force be $x(t)$, and let the strain-gauge conditioning electronics introduce an unknown, slowly changing sensor bias $b(t)$. The raw measurement delivered to the estimator at each sample interval is $y(t) = x(t) + b(t)$. Suppose the sensor outputs a constant stream $y(t) = 5.0$ N. This observation is satisfied by a true physical force $x(t) = 5.0$ N with zero offset $b(t) = 0.0$ N. It is equally satisfied by a true force $x(t) = 2.0$ N with a sensor bias $b(t) = 3.0$ N, or by $x(t) = 0.0$ N with a bias $b(t) = 5.0$ N. For any arbitrary offset trajectory $\Delta(t)$, the state pair $(x(t) - \Delta(t), b(t) + \Delta(t))$ produces the exact same measurement sequence $y(t)$. The difference between the true force and the sensor offset lies in an **unobservable subspace** of the measurement system—a structural blind spot where physical variations perfectly cancel each other out in the sensor data.

Because the measurement stream cannot distinguish between physical load and sensor drift, an estimator must rely on prior assumptions about how each state evolves. If a Kalman filter or a learned recurrent estimator is configured with a belief that the sensor offset $b(t)$ is static with near-zero variance, it attributes every variation in $y(t)$ to changes in true contact force $x(t)$. As new samples arrive at 1000 Hz, the filter incorporates the incoming data, repeatedly updating its internal state. Because the incoming measurements match the forward projection of the state dynamics, the calculated state covariance contracts, and the estimator reports an increasingly narrow uncertainty bound, publishing an estimate of $5.0 \text{ N} \pm 0.05 \text{ N}$ at P_{99} . Building on the actuator thermal limits detailed in section 4.3, I^2R Joule heating from the joint stator coils warms the sensor bridge (altering the metal's electrical resistance as it expands), causing the physical bias $b(t)$ to drift at a thermoelastic rate of 0.2 N/min. Over 15 min, $b(t)$ shifts by 3.0 N. The true force on the surface has fallen from 5.0 N to 2.0 N, yet the estimator reports high confidence in a state that has drifted by 60 percent of its nominal value.

The downstream control policy responds to this false certainty by commanding motor torques to maintain what it believes is a 5.0 N preload. Because the true force is only 2.0 N, the friction force at the contact patch drops below the shear load of the workpiece, causing the part to slip from the gripper and fall into the tooling fixture. This failure occurs without triggering a single internal software diagnostic. The estimator monitors the **innovation sequence**, which serves as a running measure of surprise representing the difference between incoming measurements $y(t)$ and predicted measurements $\hat{y}(t)$. In this drift scenario, the estimator predicted $\hat{y}(t) = \hat{x}(t) + \hat{b}(t) = 5.0$ N, and the physical sensor returned $y(t) = 5.0$ N. The innovation residual is zero at every time step. An innovation check measures only whether the physical sensors disagree with the forward projection of the internal state. It cannot expose an unobservable error mode, because along the unobservable manifold the sensor output is invariant to the drift.

Recovering observability requires the system architecture to provide an additional constraint that breaks the symmetry along the unobservable subspace. An engineering design accomplishes this in one of three ways. First, independent sensing can introduce a second measurement channel with different failure physics, such as monitoring joint motor current or an optical deflection sensor that does not share the strain gauge's thermal sensitivity. Second, the machine can perform deliberate physical excitation, commanding the gripper to briefly break contact with the workpiece for 40 ms, creating a known boundary condition of $x(t) = 0.0$ N where the measured sensor value directly exposes $b(t)$. Third, the machine can rely on periodic external anchoring, bringing the tool into contact with a calibrated reference load cell during scheduled tool-change cycles to reset the bias estimate.

When a physical state cannot be constrained by complementary sensing or active excitation, the nervous system must treat that state as unobservable and refuse to publish false precision. The estimation layer must mark the corresponding state dimensions as unavailable, or expand their

uncertainty envelope according to the worst-case physical drift rate of the underlying hardware rather than the contracted covariance of a filtering algorithm. Formal observability analysis for linear and nonlinear dynamic systems is an established foundation of control theory, treated rigorously by Kalman (1960) for linear state spaces and by Hermann and Krener (1977) for nonlinear systems via **Lie derivatives** (see section B.6). A systems engineer does not rederive these rank conditions at runtime. The engineering task is to identify which physical states lack full observability rank under the current sensor suite, and to enforce that the state interface never reports statistical convergence on an unconstrained degree of freedom.

State estimation guarantees are imported results that hold only under four explicit assumptions. First, the sensor forward model must capture all physical couplings without unmodeled algebraic dependencies. Second, physical parameters such as component stiffness and thermal coefficients must remain within pre-calibrated envelopes. Third, sensor noise must remain zero-mean and uncorrelated with actuator commands. Fourth, the operating trajectory of the machine must avoid singular configurations that cause the **observability matrix** to lose rank, a mathematical collapse indicating that certain physical states have temporarily become invisible to the sensors. The nervous system validates these assumptions during operation by cross-checking redundant measurement channels, checking environmental temperature sensors, and enforcing active re-zeroing routines before uncertainty timers expire. When an operating condition violates these assumptions, such as when an optical sensor becomes occluded or a kinematic configuration reaches a singular alignment, the nervous system revokes the valid status flag of the estimate and forces downstream controllers to fall back to conservative velocity and torque limits.

18.9 How Belief Goes Wrong

A failure in a state estimation pipeline takes one of two forms. In a **loud failure**, a component halts execution, drops a communication packet, clears an output buffer, or asserts an explicit error bit. Downstream consumers immediately observe the absence of data or the cleared validity flag, enabling deterministic fallback routines to halt motion or transition to open-loop compliance. In a **silent failure**, the memory layer continues to publish a complete, numerically well-formed state record with valid status bits, low variance indicators, and continuous timestamps, while the numbers inside the record diverge from the physical configuration of the machine. Downstream control loops consume these corrupted records without raising an exception, computing feedback corrections against a phantom geometry. Because physical actuators execute commands derived from these valid-looking records—driving an arm full-speed toward a target that now physically intersects a wall, for instance—silent state corruption produces high-velocity collisions and unconstrained force delivery that breaches the causal boundary established in section 2.2 before software safety monitors register an anomaly.¹⁶

The most deceptive silent failure is the **frozen transform**, which occurs when software replaces an aging spatial relationship with a synthetic stream of current timestamps—effectively masking a dead sensor by continuously republishing a stale spatial snapshot as if it were just observed. Consider an optical tracking node that computes a transformation matrix $T_{\text{camera}}^{\text{object}}$ at timestamp $t_0 = 0$ ms. If the optical sensor disconnects, drops frames, or suffers thread starvation at $t = 10$ ms, the underlying tracking algorithm ceases to produce new spatial evidence. If the message-passing middleware or an intermediate transport node retains the last computed matrix in an unexpired buffer and republishes it at a loop rate of 1000 Hz, generating a new transmission timestamp $t_k = t_0 + k \cdot (1 \text{ ms})$, the temporal provenance of the measurement is erased. A downstream tracking controller verifies freshness by computing data age as $\Delta t = t_{\text{now}} - t_{\text{msg}}$. Because the publisher stamped the message with the current system clock, Δt evaluates to less than 1 ms, satisfying a strict freshness threshold of $\tau_{\text{max}} = 5$ ms. The controller accepts the transform as current evidence. If the physical object moves at 0.5 m/s while the transform remains static, the controller accumulates 50 mm of spatial tracking error in 100 ms, yet reports zero estimated tracking divergence.

Phantom-object tracks arise when the nervous system decouples the lifetime of spatial coordinates from the lifetime of existence evidence. A perception module maintains an object track comprising a kinematic state vector $\mathbf{x} = [p, v]^T$ (encoding spatial position and velocity) and an existence probability $p_{\text{exist}} \in [0, 1]$ (encoding confidence that the object is physically real). When the physical object is removed from the workspace or an occlusion begins, raw detector hits drop to zero. If the estimation

¹⁶ | **AMR Safety Standards:** ISO 3691-4 addresses safety requirements for driverless industrial trucks and their systems. This chapter does not assert a clause-specific uninterrupted field of view or a universal braking distance. In a given design, occlusion and stopping distance must be assessed for the operating domain and validated sensor/actuator configuration.

layer continues to propagate x forward using process dynamics while existence probability decay is governed by a long time constant (for instance, an analytical decay rate where p_{exist} remains above an operating threshold for 1500 ms without reinforcement), the tracker maintains a persistent phantom entity. The path planner routes the manipulator around empty space, or an automated grasping routine closes its fingers on an empty coordinate. Detecting a phantom track cannot rely on passive measurement absence. It requires an **active negative observation**, where a line-of-sight sensor sweeps the volume and confirms that the expected occupancy is empty, or an unfulfilled contact expectation when a force sensor registers zero load at the predicted contact boundary.¹⁷

Coordinate frame misattributions divide along a similar boundary between syntax and semantics. A **schema-detectable frame mismatch** occurs when a publisher labels a transform with an invalid or undefined frame identifier in the transformation tree graph. A message bus with static schema enforcement rejects the packet at the serialization boundary because the named frame cannot be resolved to a known parent link. In contrast, a **semantically wrong frame assignment** occurs when an engineer or configuration file assigns a valid, type-correct frame identifier that represents the wrong physical link. For example, commanding a tool center point trajectory using the transform of the bare mounting flange T_{flange} instead of the extended tool tip T_{tool} satisfies every schema validation check, type assertion, and tree traversal test. In a hypothetical setup with a tool tip 150 mm beyond the flange along the tool z -axis, a flange-as-tip error shifts the intended contact point by that offset. Whether contact occurs depends on the fixture geometry and admitted path. Detecting a semantic frame error requires physical consistency checks, such as verifying measured motor torques against an analytical dynamic model of the arm, or cross-referencing multi-camera optical tracker observations against joint encoder kinematics.

When sensor updates drop out, an estimator relies entirely on its predictive forward model to advance state estimates across the blind interval. A well-formulated filter updates the mean state estimate $\hat{x}(t)$ while monotonically expanding the estimation covariance (t) to reflect the growth of unconstrained process noise. An **overconfident predictor** understates process variance Q , holding (t) artificially compact over an extended prediction duration exceeding 200 ms. When fresh sensor measurements eventually arrive, the artificially small covariance causes the filter to discount incoming observations; it weights its internal dead-reckoning so heavily that it classifies valid measurements as statistical outliers and rejects them. Engineers identify overconfidence during system qualification by executing staged synthetic occlusions, systematically dropping sensor frames for intervals between 50 ms and 500 ms and evaluating the reacquisition **innovation residual** $y(t) - \hat{y}(t)$ —the mathematical discrepancy between what the sensor physically observes upon waking up and what the predictor expected to see. If the reacquisition residual consistently exceeds the predicted 3σ covariance bounds across fifty analytical trials, the process noise model is invalid. However, when drift occurs along unobservable degrees of freedom (such as workpiece slippage inside compliant gripper pads during high-acceleration transfers), no reacquisition residual can develop because no sensor directly measures the displacement. Unobservable physical drift evades both residual checks and staged occlusion detectors, persisting until the machine makes physical contact with an external reference surface.



War Story 18.1: A rover trapped in sand (2009)

Context: *Spirit* first sank into soft terrain near Troy on Sol 1886 and was embedded by Sol 1899, as the JPL navigation-camera record dates the approach and embedding. Earlier imagery showed a surface crust; the rover encountered lighter, slippery material beneath it.

Mechanism: Wheel rotations did not produce the expected translation as the rover sank. JPL reported that a rock beneath the vehicle might have contacted its underside; contact was a possibility, not an established mechanism. Surface appearance alone could not establish the subsurface load-bearing state.

Impact: Recovery teams tested extraction maneuvers, but *Spirit* did not regain mobility. The event illustrates a terrain-belief limit: the physical interaction supplied evidence that the previous traversability estimate could not safely preserve.

¹⁷ | **Autonomous Safety Standards:** UL 4600 addresses safety evaluation of autonomous products, and ISO 26262 addresses automotive functional safety. The exact data-age, track-invalidation, and transform checks here are proposed design requirements for this worked architecture, not verified quotations of clauses in either standard.

Response and lesson: Slip checks based on visual odometry already existed before Troy, including work motivated by *Opportunity*'s Purgatory event (JPL technical account). Troy does not establish a new universal protocol or a particular slip threshold. A controller should compare wheel rotation with verified motion, expire a terrain assumption when that evidence conflicts, and admit further driving only through a feasible permission check.

Preventing stale, misattributed, and unbounded estimates from compromising the physical machine requires formal structure at the boundary between estimation and control. An ad hoc dictionary of floating-point values and raw transformation matrices cannot distinguish between active evidence and cached momentum. Extending the proposal-permission architecture from section 6.1, every element of spatial memory must reside within an explicit state record that couples the numerical value to its base coordinate frame, its sensor provenance, its decay envelope, and a hard invalidation deadline that forces downstream consumers to act on physical truth rather than numerical habit.

18.10 The Spatial State Schema

A consumer cannot reconstruct physical meaning from a position tensor alone. The record below extends the transport check in chapter 8 with the evidence and growth information needed for a proposed tactile insertion. It is one **illustrative bounded-disturbance record**, not a claim that its numbers fit every gripper. The estimate epoch records a computation; the evidence epoch records the last physical measurement. The capture clock is converted to the permission controller's monotonic clock with a bounded error before age is evaluated.

Table 18.4: Complete illustrative spatial-belief record: Value, unit, entity, frame, two epochs, clock conversion, error model, envelope, provenance, and lifecycle travel together. A map publication timestamp is separate from each cell's evidence epoch and cannot refresh an occluded region.

Field	Example record value	Consumer check
Entity and quantity	peg_17, lateral position	Match the target entity and task quantity
Value and SI unit	0.142 m	Decode as meters, not millimeters or force
Frame	assembly_frame, transform revision T42	Resolve a time-valid transform chain
Evidence epoch	10.000 s on capture clock cam_A	Preserve across prediction and republishing
Estimate epoch	10.005 s on capture clock cam_A	Date the propagated estimate; do not renew evidence
Clock conversion	Versioned $t_{MCU} = at_{capture} + b$, error at most 0.2 ms	Refuse if conversion expires; add error to upper age
Initial error	$E_0 = 0.00005$ m	Check the stipulated initial bound and calibration
Growth law	$E(t) = E_0 + (0.001 \text{ m/s})t + \frac{1}{2}(0.005 \text{ m/s}^2)t^2$	Compare at evidence age with the task's lateral tolerance
Operating envelope	Arm acceleration at most 2.0 m/s ² ; contact mode unchanged; residual drift and acceleration bounds above validated within this region	Expire immediately on a mode or residual-bound violation
Provenance	Observation ID, sensor/calibration revision, estimator build hash	Match approved sources; hash is version provenance, not payload integrity
Lifecycle	VALID, DEGRADED, EXPIRED, or UNOBSERVABLE	Never restore VALID without fresh physical evidence
Map cell evidence, when applicable	Per-cell capture epoch and free/occupied/unknown state	Check every cell used by the candidate path; unknown stays unknown

The record in table 18.4 separates publication time from cell evidence. For this example, the error law has meaning as a hard envelope only while the stated initial, velocity, acceleration, and contact-mode limits hold. If it is instead calibrated as a $P_{99.9}$ residual quantile, the record must say so and carry its 0.1-percent tail risk and test region; it cannot be called a worst-case bound. The consumer

converts the capture epoch, includes clock-error margin, checks status and operating envelope, and evaluates $E(t)$ against the proposed motion's tolerance. Passing these checks admits the belief to the **separate permission decision**; it does not actuate the robot. On expiry or unobservability, the controller rejects dependent proposals and selects a validated fallback whose braking, hold, or search behavior remains physically feasible.

A voxel map also has a publication time, but that time does not date each voxel. A path query checks the evidence age and state of every occupancy or clearance cell it uses, including gaps marked unknown. A new scan on the far side of the workcell cannot renew a cell hidden behind the robot. Negative free-space evidence updates only cells actually traversed by a valid ray; it does not clear a volume behind its terminal return.

Systems Perspective 18.3: Blind-spot terrain memory for footsteps

For a bipedal humanoid walking over irregular or cluttered terrain, spatial memory is not an abstract database—it is a non-negotiable prerequisite for physical balance:

- **Self-Occlusion and Blind-Spot Stepping:** A humanoid's forward-facing head cameras cannot see the terrain directly beneath its feet or behind its pelvis due to occlusion by its own chest, arms, and thighs. Consequently, every forward footstep descends blindly onto terrain that was observed 1.5 to 3.0 seconds earlier when it was still within the camera's forward field of view. The robot must rely completely on a local 2.5D elevation map and affordance memory stored in spatial belief records.
- **Kinematic Drift and False Footing:** In the seconds between when a stair edge or rock was visually observed and when the swing foot makes contact, accumulated floating-base odometry drift ($e_{\text{drift}} > 3$ cm) distorts the remembered terrain elevation. If the spatial memory overconfidently reports the ground surface 3 cm higher than it physically is, the swing foot initiates ground contact deceleration prematurely in mid-air, throwing off swing phase momentum. Conversely, if memory reports the ground lower, the foot strikes the surface with high velocity, transmitting an unbudgeted impact impulse that causes the support leg to buckle.
- **Per-Voxel Expiry and Dynamic Obstacles:** If a rolling ball or another person moves into the humanoid's blind spot after the head camera has panned away, static voxel grids fail catastrophically. The humanoid memory architecture must associate every elevation cell with an explicit evidence epoch (t_0) and confidence decay envelope (τ_{decay}). When evidence decays beyond threshold before foot touchdown, the stepping planner must widen its impedance margins or execute an exploratory tactile foot search rather than committing full weight to an unverified memory.

Checkpoint 18.6: Observability bounds and state lifecycle contracts

Before examining common spatial belief fallacies and failure modes, verify your understanding of observability limits and state schema invariants:

- ☐ **Unobservable manifold hazards:** Can you explain why zero innovation residuals ($\tilde{y} = 0$) fail to detect sensor bias drift caused by motor thermal heating ($I^2 R$ Joule heating) along an unobservable degree of freedom?
- ☐ **State lifecycle state machine:** Can you trace the deterministic transitions between VALID, DEGRADED, EXPIRED, and UNOBSERVABLE, explaining why a state record must never revert to VALID without a fresh hardware evidence epoch t_0 ?
- ☐ **Provenance validation:** Can you describe why intermediate compute nodes (planners, filters, message bridges) must be forbidden from updating t_0 when forward-propagating estimates across occlusions?

18.11 Fallacies and Pitfalls

Valid records do not form a consistent account of the workspace. A controller must reconcile their times, coordinate frames, and physical sources before using them together to authorize motion.

Pitfall: *Evaluating spatial clearance between moving bodies using asynchronous timestamps.*

Two arms bring a shaft and collar together at a relative closing velocity of $v_{\text{rel}} = 0.50$ m/s. Their position records differ in acquisition time by 20.0 ms, although both remain within a 50.0 ms expiry window. Comparing the records directly combines positions that never described the same instant. During the time gap, the relative SE(3) configuration changes by 10.0 mm, enough to cause a damaging collision during a precision insertion. Individual freshness checks cannot establish joint alignment. The controller must propagate both estimates to a common evaluation time or bound their allowable skew using the interaction's motion and clearance requirements.

Fallacy: *Averaging conflicting spatial estimates resolves uncertainty when measurements disagree.*

An overhead camera and an in-hand profilometer locate the same workpiece feature, but their means differ by 4.0 mm while their stated error bounds sum to 2.5 mm. A fusion routine averages the reports and returns a narrower uncertainty interval without first investigating the disagreement. The cutter then acts on a position estimate whose apparent precision conceals inconsistent evidence. Averaging cannot repair a violated measurement assumption. The estimator must apply its innovation gate and withhold the fused estimate when the consistency check fails, so the independent permission path can refuse the cut and select a feasible validated stop or defer the operation until the discrepancy is resolved.

Fallacy: *Bounded spatial covariance proves that a physical entity still exists in the workspace.*

An operator removes a component from a moving feed belt before it reaches the pick zone. The tracker receives no further detections but continues blindly predicting the component's location along the belt, keeping its spatial covariance below the rejection threshold for 300.0 ms. At $t = 200.0$ ms after removal, the gripper descends toward the predicted position and strikes the empty belt. The position estimate was conditional on the component still being present. A tracker must maintain evidence of existence separately from spatial covariance and update it when expected detections are absent, so a precise extrapolation does not authorize a pick on a vanished object.

Fallacy: *Agreement between redundant estimators implies independent physical evidence.*

A press relies on two force estimators that share an uncompensated strain-gauge channel. As with the actuator thermal limits established in section 4.5, Joule heating introduces a +15.0 N bias into that channel, so both estimators report 200.0 N while the panel actually receives only 185.0 N. Their agreement reveals nothing about the shared error, and the resulting bond can remain under-compacted. Comparing outputs detects only faults that produce different results. An independence claim must trace each estimate back through its physical transducers and analog processing, identifying common sources of bias. Different algorithms alone do not supply independent evidence when both depend on the same affected measurement.

Pitfall: *Applying an unexpired kinematic transform from an asynchronous epoch to sensor data.*

A mobile manipulator scans a turbine blade at $t_{\text{scan}} = 50.0$ ms, then transforms the points using its pose at $t_{\text{kin}} = 70.0$ ms. Both records satisfy their expiry checks, but the base has moved 4.0 mm between those instants. The resulting surface reconstruction incorporates machine motion as though it were workpiece geometry, shifting the grinder's intended contact. An SE(3) transform is physically valid only for a particular instant in time, not just a spatial location. The lookup must match the acquisition epoch within the allowed temporal tolerance, $|t_{\text{scan}} - t_{\text{kin}}| \leq \epsilon$, rather than selecting any transform that has not expired.

18.12 Summary

Spatial memory maintains geometric permanence across discrete, asynchronous perceptions by structuring physical scenes into hierarchical voxel grids, truncated signed distance fields (TSDFs), or differentiable 3D Gaussian splats. Physical environments evolve outside the sensor horizon,

creating spatial representations as ephemeral beliefs whose geometric error and validity decay under dynamic uncertainty until confirmed by new evidence.

Building on the causal boundary established in section 2.2, a state estimate handed across the interface to the planning and control architecture is not a bare array of floating-point coordinates. It is a bound data structure that carries its physical semantics, the exact timestamp of the physical measurement that produced it, an explicit physical-error bound, and the hardware provenance of its sensing path. When a tactile probe or vision sensor measures the surface of a workpiece, the value produced is meaningful only within the coordinate frame where the physical interaction occurred, at the precise microsecond of measurement. If a downstream consumer receives a position vector without knowing that the underlying optical measurement occurred 45 ms in the past and relies on a kinematic chain calibrated at a different epoch, the consumer acts on an illusion of geometry. The policy and the nervous system must never consume a coordinate without knowing which sensor captured it, through which bus it traveled (and its associated bus jitter), and when that evidence ceased to be fresh.

State propagation algorithms, such as numerical integration through an $SE(3)$ kinematic model or a forward dynamic filter, advance the timestamp of a state vector to match the current execution cycle. This forward projection bridges the latency gap between discrete sensor arrivals, but it generates zero new physical evidence. Extrapolating a position from an encoder reading taken at $t_0 = 0$ ms to $t_1 = 40$ ms under an assumption of constant velocity yields a computational state for t_1 , yet the evidence supporting that value remains rooted entirely at t_0 . The age of the belief is 40 ms, not 0 ms. Calling a kinematic predictor repeatedly without fresh measurements cannot refresh the machine's memory. It merely inflates the spatial variance envelope—the mathematical “blur” defining where the tool might physically reside—driven strictly by the limits of unmodeled acceleration. Confusing the prediction horizon with evidence arrival is the root mechanism of stale-state execution, where a controller acts with mathematical certainty while tracking an extrapolation that has drifted away from the physical workpiece, risking the task-specific power and force limits set for a collaborative application.

The lifetime of a state estimate cannot be governed by convenient software timeouts, such as an operating system watchdog or a SerDes video transport buffer set to a round 100 ms. Expiry is a physical calculation dictated by the mechanical tolerances of the workpiece and the validated growth rate of unobserved error. If a milling cutter operates with an allowable geometric tolerance of 0.5 mm, and the tool carriage can accelerate under unexpected load disturbances at up to 4.0 m/s^2 , then after an observation dropout of $\Delta t = 20$ ms, the unmeasured displacement can reach 0.8 mm via the second-order expansion $\frac{1}{2} a_{\text{dist}} (\Delta t)^2$. At the threshold of 15.8 ms, the unobserved error envelope crosses the 0.5 mm tolerance limit and the belief expires, regardless of whether the software message queue remains healthy. A frozen transform or an expired state held in a transport cache is far more hazardous than an explicit missing frame, because a recognized missing frame can trigger a validated fallback, whereas a stale transform disguised as fresh can let an unsafe proposal pass an inadequate freshness check.

A systems engineer must maintain the distinction between modeled uncertainty and unobservable error. Modeled uncertainty expands predictably, widening the covariance matrix as process noise accumulates during prediction steps. Unobservable error, in contrast, occurs when the available sensors cannot distinguish between physical states, such as when redundant force sensors share an elastic mounting plate that deforms under thermal load (e.g., $I^2 R$ Joule heating, extending the actuator thermal limits explored in section 4.3), or when an in-hand scanner experiences unmeasured structural vibration along the tool arm. In these regimes, the estimator may report a tightly bounded covariance and negligible residual between redundant channels, falsely indicating high agreement while the physical tool is off target by several millimeters. Guarding against unobservable drift requires cross-modal validation across mechanically independent sensing channels and explicit hardware lineage records, verifying that redundant estimators do not share common-mode physical failure paths or uncalibrated gearbox compliance under reversing torques.

A spatial memory map is an active epistemic representation of a decaying world, not a passive data cache. The instant a physical obstacle moves behind an occluding barrier or an optical sensor drops a frame, spatial coordinates diverge under unobserved physical dynamics. Treating historical coordinates as static geometric truth converts sensor dropouts into catastrophic structural collisions.

Downstream: Forensic recording of expired spatial transforms populates the authority logs in section 28.8.

Key Takeaways: Spatial memory and dynamic occupancy

- **Prediction Advances State but Generates Zero Evidence:** Forward-propagating a state vector through kinematic differential equations advances the timestamp, but evidence age (t_{age}) remains rooted at the raw measurement epoch. Calling a forward predictor without new measurements inflates spatial variance ($((t))$).
- **Temporal Skew Across Moving Bodies Destroys Spatial Clearance:** In multi-entity relative motion ($v_{\text{rel}} = 0.5 \text{ m/s}$), an unsynchronized measurement skew of $\Delta t = 20 \text{ ms}$ induces $\Delta x = 10 \text{ mm}$ of unmodeled geometric error, requiring all relative states to be strictly reconciled to the identical microsecond epoch.
- **Innovation Gating Must Precede Sensor Fusion (The Midpoint Fallacy):** When two sensors disagree by more than their combined uncertainty margins (e.g., a camera sees a block at 145 mm and a tactile probe feels it at 149 mm with 1 mm bounds), blindly averaging them computes a mathematically optimal midpoint (147 mm) where no physical object actually exists. Systems must reject fusion and halt motion when Chi-squared (χ^2) innovation gates (formal statistical tests evaluating measurement consistency against the predicted innovation covariance) fail, preventing the controller from actuating toward an ungrounded hallucinated midpoint.
- **Spatial Covariance Does Not Represent Physical Existence:** Covariance matrices quantify geometric variance assuming an object exists. If a tracked object is removed or occluded, spatial extrapolation continues blindly, requiring an explicit existence probability field ($P_{\text{exist}}(t)$) that decays upon negative visual evidence.
- **State Expiry is Dictated by Physics, Not Software Timeouts:** Belief validity windows are bounded by mechanical tolerances and disturbance acceleration ($\tau_{\text{expire}} = \sqrt{2\delta_{\text{tol}}/a_{\text{dist}}}$). Stale state records held in transport buffers are far more hazardous than explicit missing frames.

Spatial representations in embodied systems are fundamentally transient beliefs, not static maps. Because the physical world moves independently of sensor scans and unmodeled errors compound quadratically over time, retaining geometric certainty without fresh observational evidence is an operational failure mode. The nervous system must enforce explicit physical expiration bounds, ensuring that stale state estimates decay into conservative uncertainty volumes rather than driving actuators into occluded obstacles.

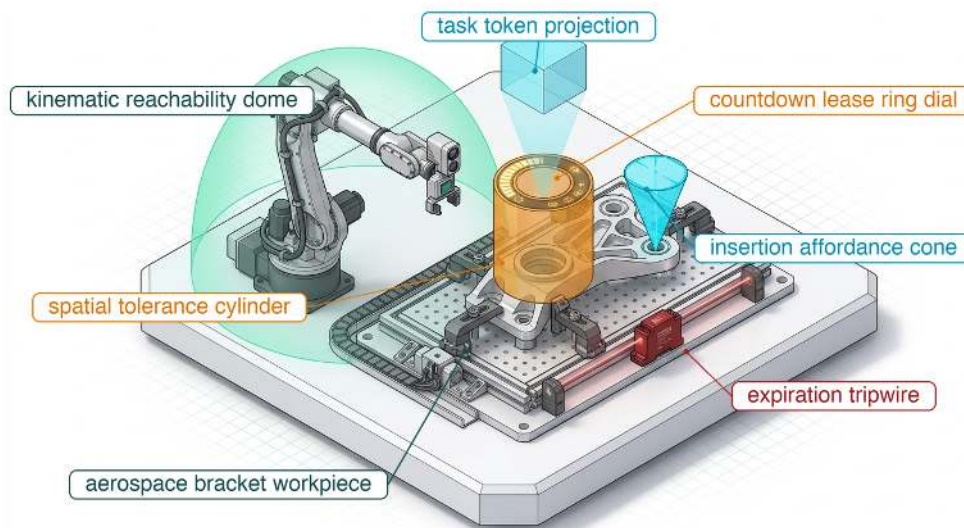
What's Next: From volumetric maps to behavioral intent

Where spatial memory structures the dynamic environment into metric representations and bounds epistemic decay during sensing dropouts, upstream foundation models operate at slow deliberative cadences (1–5 Hz) with high latency variance. In chapter 20, we examine how the intent architecture consumes grounded spatial state records to translate high-level task semantics into bounded, expiring geometric leases ($\mathcal{L}_{\text{intent}}$)—decoupling cognitive deliberation from real-time motor execution.

Grounded Intent

DRAFT

Grounded Intent

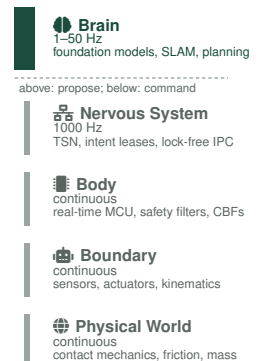


- 20.1 Grounded Affordances
- 20.2 What Intent Hands Over
- 20.3 From Request to Expiring Geometric Proposal
- 20.4 Goals That Expire
- 20.5 Admissible Goal Sets
- 20.6 How Intent Goes Wrong
- 20.7 The Intent Lease
- 20.8 Fallacies and Pitfalls
- 20.9 Summary

Purpose

What must an autonomous goal carry to prevent a slow reasoning model from driving a machine into a situation that no longer exists?

A semantic request like 'retrieve the tool' specifies an outcome but leaves physical decisions—path, force, and duration—unresolved. Because the physical scene changes while a slow reasoning model deliberates, we cannot blindly execute its delayed output. A goal that was reachable when proposed can become obstructed before execution, even when the model's interpretation of the request was correct. Intent must connect a task to an admissible region of physical action and a deadline. When that deadline passes, continued motion cannot depend on the same delayed model noticing that its proposal has expired. The execution layer needs enough information to withdraw permission and enter a defined fallback on its own. Grounded intent links the brain's task choices to the body's changing constraints through an interface that grants the nervous system bounded authority to act.



Learning Objectives

- Construct grounded goal records with spatial tolerances, motion limits, and finite validity windows
- Set goal validity windows from scene dynamics and the physical response needed before expiry
- Evaluate whether a proposed goal is reachable within the machine's physical and temporal limits
- Design goal expiration that revokes motion permission even when the issuing process cannot cancel it

20.1 Grounded Affordances

HIGH-LEVEL directives like “pick up the blue vial” or “clear the conveyor junction” specify outcomes but leave path, force, and deadline unresolved. A motor inverter accepts electrical commands, not language. Vision-Language-Action (VLA) models (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023) can propose actions or targets, but their update cadence depends on the model and hardware and may be much slower than local control. If a target moves during inference, a stale proposal can consume clearance. The system therefore needs dated geometric evidence, bounded requests, and an independent permission path that can withdraw authority while the model is silent.

Memory can supply a spatial belief with a validity window (chapter 18); it cannot decide what action that belief authorizes. **Intent** bridges this chasm by connecting high-level task semantics to a bounded physical goal. As highlighted in figure 20.1, Intent occupies Stage 4 of the deliberative cognitive brain, positioned directly between Spatial Memory (Chapter 09) and Trajectory Planning (Chapter 11). It specifies an admissible, time-bounded target before the trajectory optimizer constructs a continuous motion profile to reach it.

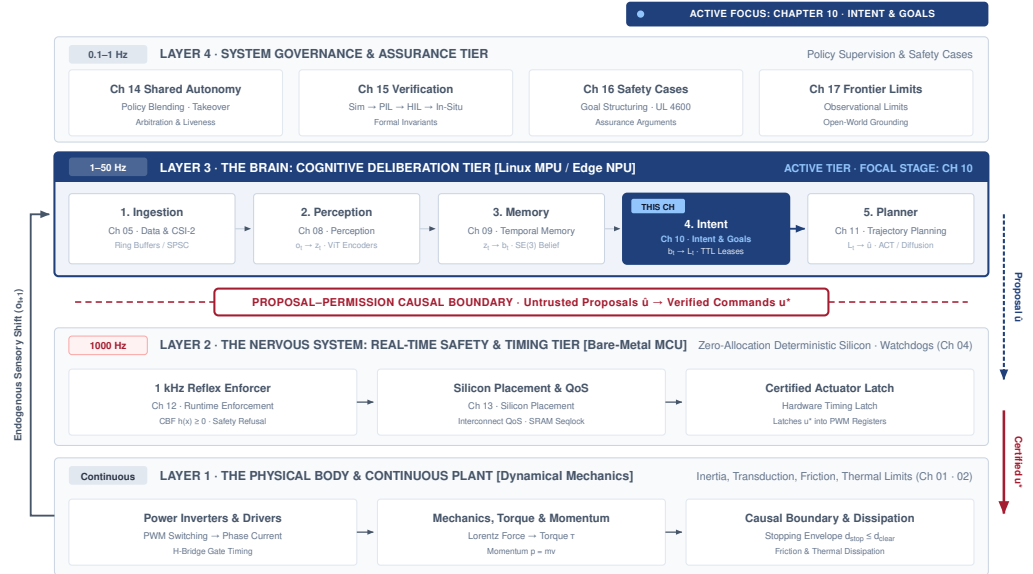


Figure 20.1: Intent stage architecture focus: The four-tier physical AI systems architecture highlights the cognitive deliberation tier, with intent as the fourth brain stage and this chapter's active focus. The stage turns belief b_t into a goal ℓ_t , projecting semantic objectives across the privilege boundary as time-bounded geometric leases that expire on their own unless the nervous system receives a refresh.

This fundamental challenge is the modern robotics manifestation of what Harnad termed the **symbol grounding problem** (Harnad 1990): purely symbolic or linguistic tokens within a computational system have no intrinsic physical meaning unless causally grounded in non-symbolic, sensorimotor interactions with the physical environment.¹ Whereas early artificial intelligence systems such as Winograd's SHRDLU (Winograd 1972) operated inside closed-world blocks domains with synthetic, analytic geometries,² modern physical AI systems must ground open-vocabulary instructions into continuous, dynamic, and uncertain physical spaces.³



Definition 20.1: Intent grounding refusal

Intent grounding refusal is the deterministic safety primitive by which an embodied policy explicitly rejects ambiguous semantic directives or unreachable spatial targets (ERR_GROUNDING_AMBIGUITY), refusing to execute non-deterministic argmax guesses through clearance volumes.

1. **Significance:** Rejects a proposal when the specified ambiguity test fails, before that proposal reaches trajectory generation. Protection also depends on the independent motion-permission and fallback checks.
2. **Distinction:** Unlike a trajectory planning failure that cannot find a collision-free path to a valid goal, grounding refusal rejects the goal itself as physically ungrounded or ambiguous before the planner is invoked.
3. **Common pitfall:** Forcing an argmax token decode or nearest-neighbor guess when semantic entropy is high, driving the physical plant toward an arbitrary or hazardous target.

That permission must remain bounded in time. Figure 20.2 gives a constructed schedule in which a slow deliberative model, local tracker, permission loop, and drive run at different rates. Its numbers are scenario assumptions, not model-family benchmarks. A target whose evidence expires in 60 ms cannot be renewed by a model updating every 200–1000 ms; a separate validated local observation path or task pause is required.

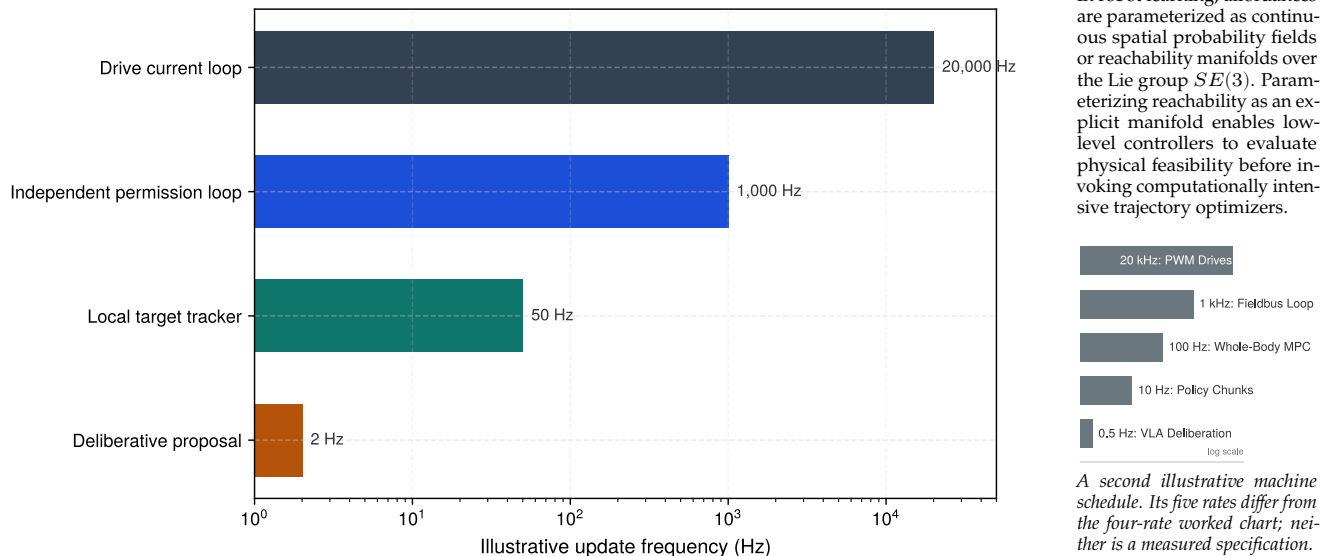


Figure 20.2: Illustrative multi-rate schedule: Four stipulated rates show why a deliberative proposal cannot act as a motor or permission-loop heartbeat. They are a constructed schedule, not historical measurements or a law relating model size to frequency. The local tracker can renew target evidence only if its sensor and error bounds are independently validated.

1 | Symbol Grounding Problem: Stevan Harnad's 1990 formulation argues that syntactic manipulation alone does not give symbols intrinsic meaning. In physical AI, grounding binds a proposed label to dated sensor evidence, calibrated metric frames, and task-specific physical checks. Without that binding, a planner may optimize toward the wrong physical target.

2 | Classical STRIPS planning: Classical planning represents goals as logical predicates in a modeled world. Dynamic obstacles and sensor dropouts can invalidate a physical target before its planned use. An independent timer revokes tracking authority at expiry; live evidence and clearance checks must detect invalidation that occurs earlier.

3 | Affordance space parameterization: In ecological psychology, an affordance defines an action-relative physical relationship between an agent's kinematic capabilities and environmental geometry. In robot learning, affordances are parameterized as continuous spatial probability fields or reachability manifolds over the Lie group $SE(3)$. Parameterizing reachability as an explicit manifold enables low-level controllers to evaluate physical feasibility before invoking computationally intensive trajectory optimizers.

A second illustrative machine schedule. Its five rates differ from the four-rate worked chart; neither is a measured specification.

4 | **Inter-core communication across privilege boundaries:** RPMSG over a statically bounded shared SRAM ring is one implementation of this proposal-permission path. An independent timer can detect loss of host renewals despite a host kernel panic; the complete detection-to-brake response still needs validated bus, sensing, drive, and actuator timing. See section 24.3 and Appendix C.

Prerequisite: The physical proposal-permission boundary and hardware referee are introduced in section 2.9.

Across the privilege boundary, the Linux MPU proposes a dated **intent record** and requested validity window.⁴ In the running implementation, the independent MCU grants a bounded interval only after checking current limits and stopping feasibility. A host stall then prevents fresh proposals, so the local timer revokes tracking without waiting for a cancellation packet. Deceleration can protect clearance only if its total response was budgeted before the motion was admitted.

In this proposed interface, each intent packet from the learned MPU requests three bounds for the independent permission controller to check against configured limits and the current physical state:

1. **3D spatial tolerance envelope:** This term refers to an explicit target pose defined in the mathematical space known as $SE(3)$, which represents the set of all possible positions and orientations in three-dimensional space. The spatial tolerance envelope includes a bounded metric tolerance volume, which defines the allowable dispersion within which the low-level controller can maneuver without compromising stability or safety.
2. **Expiring temporal lease (TTL):** An explicit time-to-live τ measured in milliseconds. If no renewal packet arrives before the deadline passes, hardware timers automatically revoke motion authority.
3. **Admissible command and force envelope:** Explicit dynamic limits that cap the peak velocity ($v_{\max}$), acceleration ($a_{\max}$), and contact wrench ($F_{\max}$) the machine may exert while pursuing the target.

Definition 20.2: Expiring intent lease

Expiring intent lease ($\mathcal{L}_{\text{intent}}$) is a finite target-evidence validity request derived from a dated observation. The independent permission tier admits a shorter or equal execution interval only after checking configured ceilings, current clearance, and a feasible stopping response; it revokes tracking authority when evidence or the admitted interval expires.

1. **Significance:** Momentum and gravity continue when the high-level planner stalls. An independent timer can revoke tracking without an abort packet; safe stopping additionally requires a validated response and enough remaining clearance.
2. **Distinction:** A target-evidence horizon follows workspace drift and task tolerance. A separate stopping budget covers detection, communication, brake onset, and physical deceleration.
3. **Common pitfall:** Sizing the lease duration too generously to mask high inference latency percentiles (P_{99}). An oversized lease leaves a wide unmonitored window where the machine operates open-loop in an environment that may have changed.

Grounding an open-ended directive therefore requires a proposal that carries the target, its evidence age, spatial tolerance, and requested effort limits. The independent permission path must test that proposal against current state and stopping clearance before motion begins.

20.2 What Intent Hands Over

Every autonomous action delivered across the causal boundary (section 2.2) needs a checkable proposal before the real-time nervous system accepts it. Instead of streaming continuous joint trajectories or raw voltages, the deliberative cognitive layer issues a **conceptual intent contract** requesting bounded operational authority over space, time, and effort.

The **conceptual intent contract** states the checks: target tolerance, evidence validity, requested effort, and a feasible fallback. The **wire-level schema** in section 20.7 shows one fixed-size serialization of those requests and their sensor-record lineage. The lower tier grants permission only after validating them against independent limits and current state.

In physical manipulation, an isolated coordinate is incomplete. An affordance is an action-relative relation among geometry, robot reach, and contact limits (Gibson 1979). The contract therefore proposes a target pose in $SE(3)$ and a **task tolerance envelope** within which the task counts as complete. This tolerance differs from the estimator's covariance, which describes uncertainty about the target. For a tactile probe, an approved task profile might allow 1.5 mm translational and

0.05 rad rotational error. A controller can then terminate within that region rather than integrating forever toward an exact noisy point; it still checks that estimated target error fits inside the tolerance.

Spatial validity in the physical world is transient. A physical goal therefore carries an evidence epoch and a finite requested validity window rather than relying on an asynchronous cancellation message⁵ from the reasoning layer. A 4 Hz model produces a new result only every 250 ms before jitter; that cadence cannot renew a target whose evidence is valid for less than 250 ms. The task then needs a qualified faster local tracker, a slower or paused physical process, or refusal. An independent permission controller evaluates the admitted deadline on each control cycle and drops tracking when the evidence or lease expires. It selects a configured hold or stop only when the measured state and remaining clearance make that response feasible.⁶ The local timer removes dependence on a host abort packet; the stopping distance remains a separate physical obligation.

A target bounded in space and time is hazardous if the controller is allowed unlimited physical effort. The intent packet therefore requests a command envelope for contact force, joint velocity, and power. A model may place a target slightly beneath a surface because of stereo disparity error; a stiff position controller can then command excessive restoring torque. For an illustrative tactile search, the proposal might request a 6.0 N normal-force ceiling, 0.10 m/s speed cap, and 25 W joint-power cap. The permission tier compares each request with independently configured actuator and task ceilings, then admits only a feasible subset. The low-level **impedance controller**⁷ enforces the admitted limits subject to sensor, actuator, and response-time validation.

Omitting any of these three bounds leaves a machine exposed during communication dropouts or perception stalls. Consider an analytical mobile manipulator moving at 1.2 m/s toward a target 3.0 m ahead. It covers 120 mm every 100 ms. A 400 ms unserved stall at constant speed therefore adds 480 mm before any braking. For the protected case, stipulate that lease expiry, detection, communication, and actuator engagement together fit within a **150 ms total pre-brake delay**, and that the measured state supports at least 3.0 m/s² deceleration. The actual lease must be shorter than 150 ms by the reserved post-expiry delays. Under those assumptions the base travels 180 mm before braking, then needs another 400 ms and 240 mm to stop:

$$d_{\text{stop}} = d_{\text{react}} + d_{\text{brake}} = v\tau + \frac{v^2}{2a_{\text{max}}} \quad (20.1)$$

Evaluating equation 20.1 gives $d_{\text{stop}} = 180 \text{ mm} + 240 \text{ mm} = 420 \text{ mm}$. This response can preserve clearance only if an independently checked obstacle boundary lies farther than 420 mm away, with model and sensing margins added. In the stalled case, a 150 kg chassis still cruising at 1.20 m/s at $t = 400 \text{ ms}$ has $E_k = \frac{1}{2}mv^2 = 108 \text{ J}$. If braking begins at $t = 150 \text{ ms}$ as stipulated, then by $t = 400 \text{ ms}$ its speed is 0.45 m/s and energy is 15.2 J, about 86 percent lower; complete rest occurs at $t = 550 \text{ ms}$. The timer removes dependence on a host abort packet, while the physical stop remains conditional on the total delay, deceleration, and available clearance.



Checkpoint 20.1: Deceleration vs. runaway during communication loss

Before exploring open-vocabulary grounding and geometric proposal synthesis, verify your understanding of multi-rate fail-safety:

- ☐ **Fail-safe lease expiration:** Can you explain why the loss of upstream heartbeats must autonomously transition actuators into a verified safe state via hardware countdown timers rather than waiting for an explicit software abort packet?
- ☐ **Kinetic containment:** Can you combine lease expiry, detection, actuation delay, and feasible deceleration to check whether available clearance exceeds total stopping distance?
- ☐ **Dual-brain timing asymmetry:** Can you distinguish a local 1 ms control period from the complete physical stopping response?

⁵ | **Asynchronous Safety Event Dispatch:** Relying on asynchronous cancellation messages or network event dispatch to abort motion introduces non-deterministic queue serialization delays. When a mobile platform moves at 2 m/s, a 50 ms queue dispatch latency translates to 100 mm of unbraked travel before the motor drive can drop torque. A local countdown can revoke tracking authority on missed renewal; the selected holding or braking response must be validated for the load and clearance.

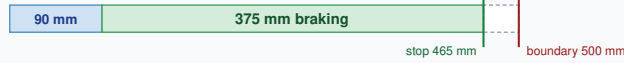
⁶ | **Hardware watchdog safety monitors:** A safety design can use an independently clocked watchdog to detect missed renewals and initiate a validated response without the host. Safe Torque Off removes drive torque; it does not by itself hold a suspended load or stop moving mass within a specified clearance. The safety function and response time must be validated for the particular machine.

⁷ | **Mechanical impedance regulation:** Impedance control regulates the relation between contact force and motion through effective mass, damping, and stiffness. It can reduce loads within a validated contact envelope; the first impact and peak force still depend on approach speed, passive compliance, sensing, and actuator response.

Napkin Math 20.1: Evidence expiry and physical response

Total pre-brake delay: 60 ms

Evidence horizon is separately 60 ms; lease must reserve post-expiry response.

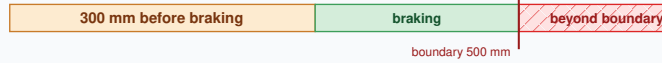


35 mm remains

1.5 m/s; 3.0 m/s² braking
Clearance before other margins

Total pre-brake delay: 200 ms

The slower response starts braking after 300 mm of travel.



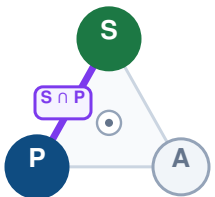
175 mm breach

Stops differ by 210 mm.
That is not separation margin.
stop 675 mm

The three models below answer different questions. A target-evidence horizon limits how long a proposed coordinate remains justified; a stopping calculation additionally needs the total delay to brake onset and the deceleration available from the *current* state. Contact and thermal thresholds also need validated physical envelopes.

1. **Mobile robot:** An illustrative 150 kg AMR travels at 1.5 m/s with 3.0 m/s² available braking. A target tolerance of 30 mm, initial error bound of 6 mm, and relative drift bound of 0.40 m/s give an evidence horizon of 60 ms. This horizon does not establish stopping clearance. Separately suppose the current obstacle boundary is 500 mm away and *all* pre-brake delay, including lease expiry, detection, bus, and actuator response, fits within 60 ms. Travel before braking is 90 mm; braking adds 375 mm. The modeled stop at 465 mm leaves 35 mm before the boundary, prior to additional model and sensing margins. If total pre-brake delay instead reaches 200 ms, the stop at 675 mm breaches it by 175 mm. The 210 mm difference between the two stops is not a separation margin. A feasible controller must reserve post-expiry time inside the 60 ms budget; equality between the evidence horizon and total delay is only a numerical coincidence in this scenario.
2. **Manipulator contact:** At 0.03 m/s against an assumed 4.0×10^5 N/m fixture, modeled force rises at 12 N/ms. A 15 N tripwire and 2 ms loop-response delay yield 39 N **when the brake command begins**. This is not the peak during deceleration: that peak also depends on moving mass, passive compliance, and achieved braking. Unbraked motion for 200 ms advances 6 mm and gives 2,400 N in the stipulated linear-spring model; no component failure rating has been supplied.
3. **Thermal process:** For an illustrative lumped capacitance of 900 J/K and uncompensated 3600 W input, temperature rises at 4 K/s. A stipulated 2 K task overshoot allowance is consumed in 500 ms; ten seconds at the same net heat input yields a 40 K modeled rise. This calculation alone cannot establish battery thermal runaway or venting. That verdict needs initial temperature, chemistry, cooling, pack geometry, and validated failure thresholds.

8 | **Subsumption architecture principles:** Rodney Brooks's subsumption architecture decomposed autonomous robotics into asynchronous, priority-arbitrated behavioral layers operating on sensory feedback. Here, the learned layer proposes a bounded target while an independent controller checks permission and executes a validated response on expiry. The lease alone does not establish closed-loop stability.



The Sense $\cap$ Plan grounding interface: translating semantic language goals into bounded, expiring spatial intent leases.

These three examples clarify the division of authority. High-level reasoning can propose a target, time window, and effort ceiling. The independent permission tier determines which limits are admissible from measured state, configured ceilings, and a feasible fallback. Local control then tracks only an admitted proposal and withdraws authority on invalid evidence or expiry. This resembles the priority separation of the **subsumption architecture**⁸ (Brooks 1986a), while physical containment still depends on sensing, timing, and actuator capability.

As illustrated by the manipulator in figure 20.3, language can generate 3D affordance and constraint value maps for a trajectory optimizer. Those maps are proposals; a separate implementation must validate path geometry, limits, evidence age, and permission before physical execution.

Grounding an intent packet into these three bounds requires a formal transformation from ambiguous sensory inputs into structured, verifiable proposals. When a system processes a high-

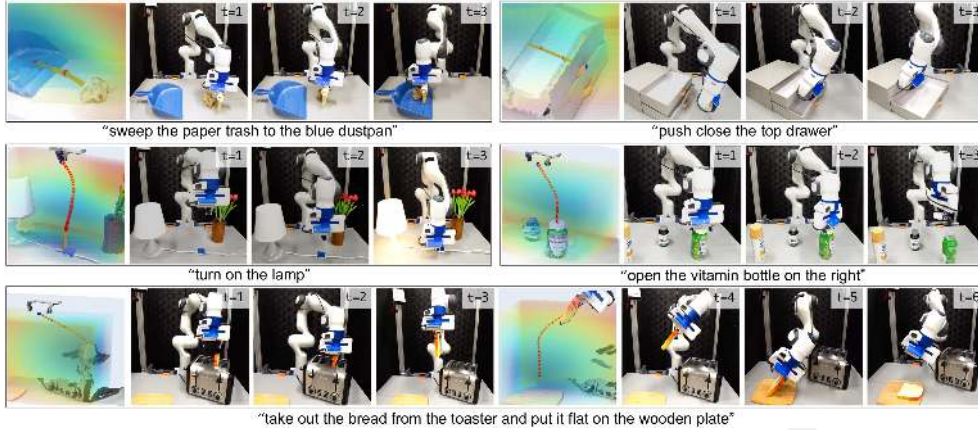


Figure 20.3: Manipulator 3D affordance grounding: VoxPoser composes language-guided 3D value maps for manipulation tasks (Huang et al. 2023). The illustrated research system proposes spatial objectives; an independent permission and stopping wrapper would require separate design and validation. (Source: Huang et al., 2023 (Huang et al. 2023), CC BY 4.0).

level directive or a sequence of visual tokens, the initial proposal is rarely a single, unambiguous set of coordinates. It begins as an uncertain geometric hypothesis that must carry its own spatial ambiguity and its own temporal limits before it can be admitted to the execution pipeline.

20.3 From Request to Expiring Geometric Proposal

A natural language instruction becomes dangerous when an unverified semantic hypothesis commands motion. When an operator or high-level supervisor issues a semantic directive such as “grasp the brass valve,” an onboard vision-language model parses the character stream, projects image features into a shared embedding space, and selects a candidate visual bounding region. From those pixels, the network extracts an estimated pose in $SE(3)$ (representing full 3D position and orientation) and writes coordinate values into a memory buffer. At the causal boundary established in section 2.2, an ungrounded string of tokens becomes a physical destination that commands mass, velocity, and actuator currents.⁹ If the visual backbone associates the language token with an adjacent fixture or an unlabelled calibration target, the resulting coordinate vector $\mathbf{p} \in \mathbb{R}^3$ enters the trajectory pipeline as an authoritative physical goal. Because a low-level motor drive measures only encoder pulses, winding temperatures (tracking Joule heating (I^2R) for thermal derating, as established in section 4.1), and phase currents, it cannot evaluate whether the coordinate it tracks corresponds to the intended component or to an empty tabletop.¹⁰ The grounding process must produce an explicit geometric proposal that includes spatial uncertainty, selection evidence, and temporal expiration before any motion planner accepts it.

To understand how open-vocabulary language directives translate into actionable geometric targets, modern physical AI systems rely on three foundational grounding paradigms:

1. **Affordance-grounded skill selection (SayCan):** Ahn et al. (Ahn et al. 2023) combine a language model’s semantic score with a learned estimate of a skill’s success¹¹ (see Appendix A). A closed drawer may lower the estimated success of “pick up the sponge” and favor “open the drawer” first. This is a task-dependent ranking, not a deterministic veto on kinematically impossible or obstructed motion.
2. **3D value-map affordance fields (VoxPoser):** Rather than selecting from discrete, pretrained skill policies, VoxPoser¹² (Huang et al. 2023) leverages vision-language models and LLM-generated Python code to synthesize continuous 3D spatial affordance and constraint value maps directly in the robot’s physical workspace. By querying 2D visual grounding backends (e.g., OWL-ViT, SAM, Grounding DINO) across multi-view RGB-D streams, the system back-projects visual activations into a 3D voxel grid $\mathcal{V} \subset \mathbb{R}^3$. The LLM writes code that computes 3D potential fields: an **attraction value map** $\mathbf{V}_{\text{attr}}(\mathbf{x})$ that pulls the end-effector toward target contact affordances (e.g., grasping handles or rim geometries), and a **repulsion constraint map** $\mathbf{V}_{\text{rep}}(\mathbf{x})$ that penalizes trajectories passing near obstacles. A downstream **model-predictive**

⁹ | **Semantic-to-metric transformation:** Projecting discrete linguistic embeddings into continuous $SE(3)$ coordinates requires an unbroken coordinate calibration chain from image pixels to actuator base frames. Unmodeled camera mounting compliance or lens distortion maps semantic attention peaks into systematic metric offsets. If the transformation chain lacks rigid extrinsic calibration, high semantic confidence produces large physical tracking errors at the end effector.

¹⁰ | **Learned-output sensitivity:** A learned pose estimate can change substantially when the image or scene changes, and an empirical confidence score does not bound that change. The permission tier limits the admitted target and motion using independently checked geometry, state, and actuator constraints; the bound is conditional on those checks and the available fallback.

¹¹ | **SayCan affordance value functions:** SayCan multiplies language preference by a learned skill-success estimate. The estimate is conditioned on training and observations; it can misrank an unseen or changed scene. Independent geometry and permission checks remain necessary.

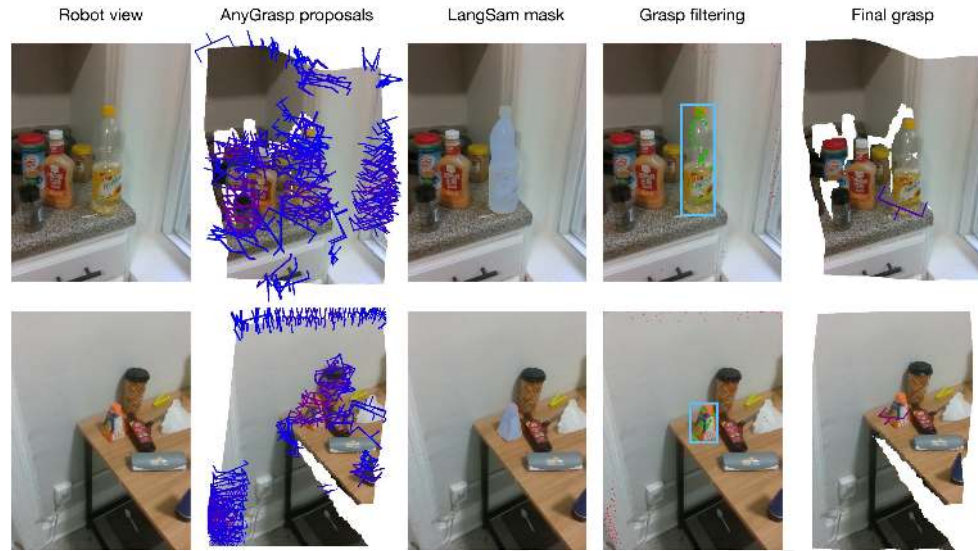


Figure 20.4: Open-vocabulary grasp grounding pipeline: OK-Robot combines candidate grasps with language-conditioned object selection and geometric filtering (Liu et al. 2024). The pictured research pipeline proposes a grasp; a bounded tolerance, evidence deadline, and independent permission check are an architectural wrapper developed in this chapter, not a reported feature of the image.

¹² | **VoxPoser 3D value maps:** VoxPoser synthesizes 3D affordance and constraint value maps from language and RGB-D observations. A trajectory optimizer can use the maps as costs; their gradients alone do not certify collision-free paths or torque limits. A separate checked path and actuator envelope supply those constraints.

¹³ | **Model predictive trajectory control:** Model predictive control (MPC) repeatedly solves a constrained finite-horizon problem. A feasible solution can respect modeled torque, velocity, and obstacle constraints if the model, state estimate, solver deadline, and actuator response are valid; soft penalties alone do not establish hard feasibility. See chapter 22.

controller (MPC)¹³ or trajectory optimizer then computes dynamic paths directly over this composed 3D cost manifold (an invisible spatial landscape of attraction valleys and repulsion hills) without requiring task-specific reinforcement training. A separate permission path must check actuator limits and the proposed path.

3. **Language model programs for embodied control (Code as Policies):** Introduced by Liang et al. (Liang et al. 2023), Code as Policies (CaP)¹⁴ prompts large language models to generate executable Python programs that compose perception APIs, geometric coordinate transformations, and parameterized motion primitives. Instead of outputting single static setpoints, the LLM writes algorithmic control structures containing spatial arithmetic, conditional branching (`if is_grasped(): ...`), and reactive loops that query real-time sensor feedback.

Open-vocabulary affordance grounding uses language and sensor observations to propose a task-relevant skill, spatial field, or program. SayCan ranks learned skill-success estimates, VoxPoser proposes 3D value maps, and Code as Policies composes callable primitives. These outputs are not themselves verified targets; any geometric request passed downstream needs a named frame, SI units, dated evidence, and independent admission checks.

One concrete grasp-proposal path appears in figure 20.4: OK-Robot generates candidate grasps and uses language-conditioned object selection to filter them. Other grounding methods need not use that pipeline. Where a task requires a metric grasp target, the producer can combine a candidate generator with segmentation such as LangSAM, calibrated depth, and a named-frame pose.¹⁵ The resulting pose still needs evidence, uncertainty, path, and permission checks before actuation.

Discrete tokens do not all represent the same physical quantity. RT-1 (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023) and OpenVLA (Kim et al. 2024) discretize each **robot action dimension** into 256 bins over a model-specific action range. By contrast, PaliGemma’s location tokens provide 1024 levels for normalized **image coordinates**, not 3D robot targets. A separate depth measurement, calibration chain, and uncertainty model are needed before image coordinates can become a base-frame pose.¹⁶ Action bins likewise do not themselves encode a target’s metric clearance.

For an explicitly hypothetical metric target encoder with 1000 equal bins over a 1.0 m axis, each bin is 1.0 mm wide. Nearest-bin decoding contributes at most 0.5 mm error along that axis when the true value is inside the encoded range, before camera, calibration, model, and motion errors. The producer must combine those other errors using a justified model and pass a conservative

representation to the permission tier. It cannot label a 20–50 mm localization error as a consequence of this binning alone.

To prevent a learned output from masquerading as a deterministic setpoint, the producer must retain the referent identity and the dated evidence that selected it. Its observation record may contain bounding boxes, attention weights, class probabilities, and a full oriented covariance estimate. The compact 168-byte proposal below links to that trusted record by digest and capture epoch; it carries a candidate pose, conservative base-frame error bounds, task tolerances, and requested expiry, not those full producer-side fields. The independent permission tier checks the linked evidence and request against its own limits. An object can shift or become occluded after camera exposure, so a coordinate without a valid evidence link and deadline cannot remain an authorized target.

When a vision-language model sees multiple candidate objects, an argmax can hide an unresolved choice. A digital recommendation can still mislead a user, but it has no direct actuator path; in a robot, an admitted spatial proposal can move mass. If two similar components are 180 mm apart, a poorly grounded guess may select the wrong part. The system should preserve candidate identity and calibrated uncertainty, refusing a proposal when a task-specific ambiguity threshold is exceeded. The permission tier then maintains a feasible hold or stop while the producer seeks a clearer observation or instruction.

Formally, **open-vocabulary grounding ambiguity** is the condition where multimodal feature projections yield multi-peak posterior distributions, overlapping geometric hypotheses, or unresolvable evidential ties across candidate physical referents ($H(\mathbf{z}) > H_{\text{thresh}}$), requiring an explicit grounding refusal (ERR_GROUNDING_AMBIGUITY) rather than optimistic argmax tie-breaking.

Systems Perspective 20.1: Ambiguity demands refusal

An argmax hides an evidential tie. In a physical task, an admitted target at the wrong coordinate can move mass toward the wrong object. When calibrated ambiguity exceeds the task threshold, refuse that proposal and use the independently validated fallback.

War Story 20.1: AutoRT task filtering and supervision (2024)

Context: AutoRT used language models to propose and filter tasks for mobile manipulators in office environments (Ahn et al. 2024). The authors collected 77,000 robot episodes across more than 20 robots; they did not report the mirror-grasp, shattered-object, or chassis-strike sequence previously attributed to this study.

Evidence: In a labeled sample of 259 proposed tasks, 228 were initially judged acceptable. After an LLM affordance filter, 200 of 214 retained tasks were acceptable, raising the acceptable fraction from about 88 to 93 percent while leaving some unsuitable tasks. The paper explicitly says constitutional prompting cannot guarantee compliance.

Response: The reported deployment supplemented prompts with joint-force pausing, physical emergency stops, and human line-of-sight supervision unless a work area was barricaded. These are distinct controls with different coverage; the prompt itself was not a certified geometric or physical safety supervisor.

Systems lesson: A learned task filter can improve the proposal stream, while an independent permission and fallback path must still validate each physical action for its stated operating region.

The complete computational pipeline from multimodal patch tokenization to the entropy-gated expiring intent lease is mapped in figure 20.5.

Consider a gripper target with nominal base-frame position $\mathbf{p}_0 = [0.450, -0.120, 0.220]^T$ m. In an illustrative calibrated Gaussian localization model, the principal sensor-frame standard deviations are 4.0, 6.0, and 15.0 mm. A **95 percent statistical region** uses Mahalanobis squared distance $(\mathbf{p} - \mathbf{p}_0)^T (\mathbf{p} - \mathbf{p}_0) \leq \chi_{3,0.95}^2 \approx 7.815$, yielding principal semi-axes 11.18, 16.77, and 41.93 mm. Figure 20.6 contrasts that uncertain target with a separately specified 12.0 mm mechanical capture

14 | **Code as Policies execution:** Code as Policies prompts language models to synthesize executable Python control scripts that compose perception APIs and parameterized motion primitives. Generating structured algorithmic code rather than raw joint positions enables recursive error recovery and conditional branching on sensor feedback. A sandbox can limit generated code, while an independent watchdog and permission path must retain control if that code stalls.

15 | **Open-vocabulary segmentation cascades:** Language Segment-Anything (Lang-SAM) combines text-grounded detection with class-agnostic segmentation. Its mask is a candidate image region, not a verified 3D hull; depth, calibration, occlusion policy, and geometric checks are still needed for a physical grasp proposal.

16 | **Metric camera unprojection:** Pinhole unprojection maps image coordinates (u, v) and measured depth z through the inverse intrinsic matrix $\mathbf{K}^{-1}$, then transforms the result into the robot base frame. Extrinsic rotation and depth bias can dominate sub-millimeter token quantization, especially at long standoff distance.

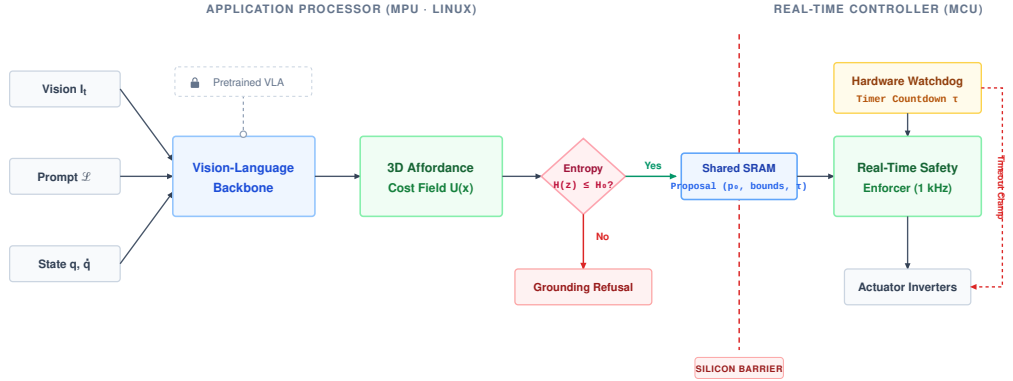


Figure 20.5: Grounded intent pipeline architecture: This proposed implementation turns multimodal inputs into a candidate spatial field, applies an ambiguity filter, and passes a bounded proposal across an MPU-to-MCU permission boundary. The MCU validates evidence age, configured limits, and feasible fallback before admitting a local countdown. Cross-attention and prompting supply proposals, not certified physical constraints (synthesized from Huang et al. (2023), Ahn et al. (2023), and Gray (1989)).

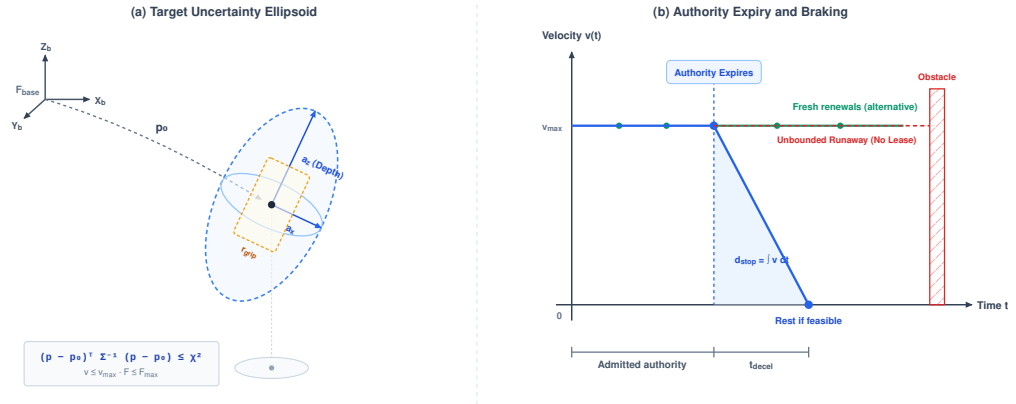


Figure 20.6: Localization uncertainty and authority expiry: Panel (a) sketches a statistical target-uncertainty ellipsoid beside a separate gripper capture tolerance. Panel (b) compares counterfactual traces: blue expires and brakes, green continues only with fresh renewals, and red shows motion without a lease. Rest before the obstacle is conditional on total pre-brake delay, deceleration, and checked clearance.

tolerance along the approach axis. The depth uncertainty exceeds that tolerance, so the proposal needs better sensing, a different approach, or refusal. A 95 percent region also leaves a residual tail and cannot by itself certify clearance; the wire example below carries conservative base-frame error bounds rather than claiming this covariance is encoded in six floats.

A detection with a large uncertainty estimate can be rejected by a configured statistical gate. A confident grounding to the wrong physical entity can pass that gate. If a vision-language model evaluates an image containing an exposed glass beaker adjacent to a machined steel housing and assigns a 99.2 percent confidence score to the glass beaker when instructed to “secure the housing,” the model emits a sharp, low-variance geometric proposal centered on the glass. The trajectory planner receives the coordinate, computes an $SE(3)$ inverse kinematics solution, verifies joint limits, and confirms that the path clears all obstacles in the static environment map. The permission tier may find the proposed motion within its configured geometric and force bounds. If those bounds still allow steel-grasp force, a smooth, geometrically valid trajectory can damage the glass despite passing the checks.

This failure reveals the boundary between what a downstream safety enforcer can verify and what it must accept as an unvalidated premise. A deterministic safety enforcer checks coordinate bounds, kinematic reachability, joint torque limits, and geometric collisions against known occupancy grids. Those physical checks do not establish that the coordinate names the intended object. Semantic classification needs separate task evidence, and the wrong-object risk remains within the declared operating envelope. The mechanics of multimodal feature extraction, cross-attention alignment, and open-vocabulary phrase grounding belong to the computer vision and language literature (Kamath et al. 2021; Gu et al. 2022; Radford et al. 2021; S. Liu et al. 2023). A physical AI architecture consumes these models without relying on their internal activations for safety. The responsibility of the systems engineering contract is to govern the boundary where these models meet the physical machine, requiring a tolerance envelope, ambiguity test, and evidence deadline before a proposal can enter the independent motion-permission check.

To evaluate how different open-vocabulary perceptual frontends interface with deterministic systems contracts, table 20.1 compares the dominant architectural categories of language grounding.

Table 20.1: Grounding representations and admission questions: Outputs from different model families require different conversions before a physical permission check. Latency, memory use, and metric precision depend on the particular model, sensor, hardware, and task; this table does not assign universal ranges.

Proposal source	Representation sent downstream	Early check	Remaining evidence needed
2D box or mask	Image region plus measured depth and camera calibration	Reject absent depth or a target outside a validated region	3D pose error, occlusion, object identity, and swept-path clearance
3D feature or value field	Candidate target or cost field in a stated frame	Reject out-of-workspace targets and stale map cells	Conservative occupancy, unknown-space policy, actuator and timing bounds
Action-token VLA	Decoded action in its trained action range	Reject out-of-range or inadmissible proposals	State-dependent physical limit and independent permission; token bins provide no clearance proof
Language-selected skill	Skill identifier with estimated success	Rank or decline low-confidence candidates	Current preconditions, path feasibility, and separate physical enforcement



Checkpoint 20.2: Open-vocabulary grounding and ambiguity refusal

Before admitting an unverified semantic hypothesis to the physical trajectory pipeline, verify your understanding of how multimodal proposals are geometrically bounded and evidentially gated:

- ☐ **Affordance feasibility gating:** Can you explain what SayCan’s learned skill-success score ranks and which physical checks still belong to an independent permission path?
- ☐ **Spatial token quantization:** Can you distinguish 256-bin robot action tokens from image-location tokens and calculate the nearest-bin error of a hypothetical metric target encoder?
- ☐ **Grounding ambiguity refusal:** Can you distinguish between optimistic argmax tie-breaking in digital web search versus mandatory grounding refusal (ERR_GROUNDING_–AMBIGUITY) when posterior entropy exceeds safety thresholds in physical AI?

20.4 Goals That Expire

An intent lease must expire autonomously, because the same software fault requiring cancellation may prevent the abort signal from being sent. An explicit abort requires a working sender, scheduler, and communication path. If a learned model stalls, an independent countdown can withdraw that proposal’s tracking authority without waiting for the model. This adapts the lease primitive of Gray and Cheriton (1989) to a physical controller: expiry triggers a separately validated fallback, whose ability to stop before a hazard depends on current state, clearance, actuator authority, and total response delay.¹⁷

¹⁷ | **Hardware watchdog down-counters:** An independent counter can detect a missed renewal without waiting for the host scheduler. Detection time, drive command propagation, brake engagement, and physical deceleration require separate measured budgets; an interrupt alone does not establish a safe stop.

Prerequisite: Kinetic momentum budgets and stopping distance constraints originate in section 4.3.

The target-evidence deadline follows the rate at which the physical target can diverge from its last measurement. Suppose a validated operating envelope bounds relative target drift by v_{drift} , initial position error by e_0 , and the task's allowable target error by r_{tol} . After evidence age t , the error bound is $e_0 + v_{\text{drift}}t$. Its validity horizon is:

$$\tau_{\text{evidence}} \leq \frac{r_{\text{tol}} - e_0}{v_{\text{drift}}} \quad (20.2)$$

When $r_{\text{tol}} \leq e_0$, the target is inadmissible at capture. The controller refuses dependent motion. Even when equation 20.2 gives a positive horizon, a separate motion-permission check must verify current clearance against total sensing, permission, communication, and brake-onset delay plus physical braking distance, as in chapter 8.

If a validated disturbance acceleration bound a_{dist} is also needed, section 18.6 gives $E(t) = e_0 + v_{\text{drift}}t + \frac{1}{2}a_{\text{dist}}t^2$. The positive root at $E(t) = r_{\text{tol}}$ shortens the **evidence** horizon. It does not compute how long the robot needs to stop.

For a conveyor moving at a bounded relative speed of 0.20 m/s, a 15 mm grasp tolerance and 3 mm initial error leave a 60 ms **target-evidence horizon** by equation 20.2. Inflating it to 650 ms to match an illustrative model tail would allow modeled target error to reach $3 + 200(0.650) = 133$ mm, or 118 mm beyond the tolerance. That arithmetic shows invalid targeting, not a proven collision without path geometry. A 1–5 Hz model, especially one with a 650 ms tail, cannot by itself refresh a 60 ms target for continuous motion. A qualified local tracker may supply fresh evidence within its own bounds; otherwise the conveyor must slow or pause, or the task must be refused. At 20 mm/s drift the same model gives a 600 ms evidence horizon (figure 20.7), still subject to a separate stopping-clearance check.



Checkpoint 20.3: Intent lease duration vs. workspace drift

Before analyzing dynamic reachability envelopes and early ingestion filters, verify your understanding of physics-derived temporal leases:

- ☐ **Physical lease derivation:** Can you explain why the validity duration of an intent lease is strictly bounded by task tolerance and scene drift velocity rather than the execution period of the reasoning model?
- ☐ **Conflating latency with stability:** Can you describe the physical failure mode that occurs when an engineer inflates intent lease durations to mask tail inference latency (P_{99}) in vision-language models?
- ☐ **Passive revocation mechanics:** Can you distinguish between relying on active network cancellation packets during an inference crash versus enforcing passive hardware countdown expiration on an embedded microcontroller?

Coupling target-evidence validity to the reasoning model's cadence creates a drift hazard. Suppose the declared drift and error envelope supports a 250 ms evidence horizon, while a nominal 4 Hz model has an illustrative 650 ms high-tail inference delay. Extending the target record to 650 ms solely to hide that delay would leave its estimate outside the declared error envelope for 400 ms. The 250 ms limit is an **evidence-validity** deadline, not a safe-stop time: the independent permission path also needs a separately feasible physical stopping budget. Compute throughput cannot justify renewing either bound without new evidence and a new motion check.

Continuous tracking requires **fresh physical evidence**, not merely a new packet from the same stale frame. A renewal based on a new capture must arrive before the current evidence and admitted authority expire, with capture, processing, transfer, clock uncertainty, and permission time included in its budget. The simple condition $T_{\text{period}} + t_{\text{latency}} < \tau_{\text{evidence}}$ is only a necessary scheduling check; for the 60 ms conveyor case it fails for the stated slow model. A validated local tracker, task slowdown, or pause is required. When a fresh proposal arrives, the planner may compute a jerk-limited splice from the measured state. It admits that splice only if the swept path, new evidence, and stopping reserve remain feasible; an abrupt target change can instead require refusal or a checked stop.

In the running MPU/MCU implementation, the host sends a proposal over a bounded shared mailbox.¹⁸ The MCU validates the evidence epoch, requested limits, and current stopping reserve

¹⁸ | **Shared SRAM mailbox topologies:** One asymmetric multiprocessing implementation exchanges proposals through a bounded shared-memory mailbox. RPMSG is one possible transport. Copy count, cache maintenance, coherency, and worst-case hand-off time depend on the SoC and driver; they must be measured for the selected design.

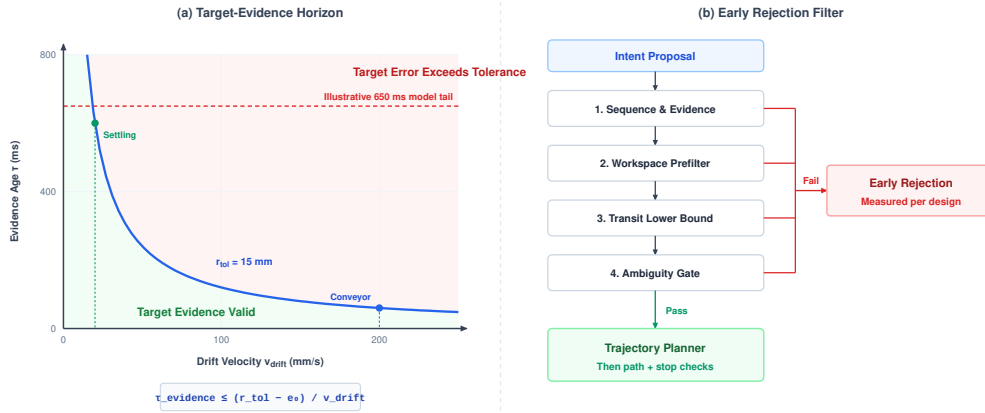


Figure 20.7: Target-evidence horizon and early filters: Panel (a) plots the illustrative bound $\tau_{\text{evidence}} \leq (r_{\text{tol}} - e_0) / v_{\text{drift}}$; its green region means only that the target remains within the stipulated error tolerance. A 650 ms model cadence cannot renew the 60 ms conveyor case. Panel (b) shows conceptual early rejection stages before full path and stopping checks; no universal runtime is claimed.

before admitting a local countdown. If the host stops renewing, the timer revokes tracking without waiting for host software. A timer interrupt or drive-disable signal does not itself stop moving mass; the reserved detection, command, brake-onset, and deceleration intervals determine whether the configured fallback stays inside physical clearance.

On expiry, the independent controller withdraws the proposal’s authority and selects a previously validated response for the measured load and contact mode. An active position hold needs torque and can protect a suspended payload; controlled deceleration needs brake capability and clearance; torque disable can be suitable for some drives but can release a load or leave momentum uncontrolled. The proposer may request a mode, but it cannot choose the terminal action for the MCU. Admission fails when no response remains feasible from the current state.

Executing an expiring lease safely assumes that the commanded geometric proposal is actually achievable by the physical body. If a proposed coordinate lies beyond kinematic reach or demands joint velocities that exceed actuator limits during the lease window, granting the lease wastes control authority and risks triggering hardware fault trips—such as blowing a digital fuse when a motor attempts to draw excessive current to achieve an impossible speed. Determining whether a goal is kinematically and dynamically admissible belongs before the trajectory generator accepts the lease, establishing a physical filter between high-level intent and motion execution.

20.5 Admissible Goal Sets

Filtering a goal against the physical capability of the machine belongs at the intent ingestion boundary, before the proposed target enters the trajectory optimization queue. When an upstream vision-language model or neural policy emits a coordinate, treating that proposal as an unverified candidate protects the real-time motion pipeline from executing or even evaluating impossible motions. Kinematic workspace analysis, imported from classical formulations (Craig 2005; Lynch and Park 2017), defines the closed spatial manifold of reachable end-effector configurations (the physical 3D envelope the robot arm can reach) given link dimensions, joint range limits, and mounting geometry.¹⁹ An analytical reachability check evaluates whether the target $SE(3)$ pose $\mathbf{p}_{\text{target}}$ (encoding both 3D position and orientation) resides inside this workspace manifold $\mathcal{W}$, while an occupancy check verifies that the bounding volume of the target does not intersect known persistent obstacles. If a proposed touch target lies outside $\mathcal{W}$, no sequence of joint torques can reach it. Passing an out-of-reach goal to a trajectory optimizer forces the solver to search an empty solution space, converting geometric impossibility into computing failure.

Cheap workspace and bounding-volume checks can reject obvious failures before trajectory optimization (figure 20.7, panel b). Their execution time and planner savings depend on the hardware

¹⁹ | **Kinematic workspace boundaries and singularities:** The kinematic workspace defines the continuous manifold of task-space poses attainable by the manipulator given link geometry and joint travel limits. Near workspace boundaries and internal singular configurations, the kinematic Jacobian $J(\mathbf{q})$ loses rank, requiring infinite joint velocities to maintain finite Cartesian end-effector motion. Enforcing workspace manifold filters at the ingestion stage prevents trajectory optimizers from driving actuators into kinematic lockup.

Downstream: Admissible goal envelopes specify the boundary conditions for spline generation in section 22.1.

and scene. Passing a bounding sphere or scalar transit test is only a necessary prefilter: joint orientation, swept obstacles, torque, thermal state, unknown space, and stopping clearance still require a checked path and permission decision.

🔍 Systems Perspective 20.2: Reject at the ingestion gate

A cheap workspace or static-occupancy prefilter can reject some impossible targets before a more costly trajectory search. Its measured runtime and coverage belong to the chosen implementation; passing it does not prove a feasible path.

A target that lies within the static kinematic workspace may still be physically unachievable within the operational window. Static reachability establishes only that the mechanism is geometrically long enough to touch the target coordinate if given infinite time. **Dynamic reachability** is the physical condition wherein an end effector can physically transition from its instantaneous initial state $(\mathbf{p}_0, \mathbf{v}_0)$ to a target $SE(3)$ pose within the remaining lease validity duration $\tau = t_{\text{exp}} - t_{\text{now}}$, while strictly respecting actuator velocity ceilings (v_{max}), acceleration limits (a_{max}), jerk constraints (j_{max}), and thermal torque envelopes.

For a rest-to-rest displacement $d = \|\mathbf{p}_{\text{target}} - \mathbf{p}_0\|$ under maximum allowable acceleration a_{max} and cruise velocity v_{max} , motion bifurcates between two physical regimes based on whether the mechanism has sufficient travel to reach its velocity ceiling:

- **Low-speed moves ($d \leq d_{\text{crit}}$) are acceleration-limited:** The mechanism never reaches the velocity ceiling v_{max} . Instead, the actuator accelerates to the midpoint and immediately decelerates back to rest in a triangular velocity profile, yielding un-jerked traversal time $t_{\text{min}} = 2\sqrt{d/a_{\text{max}}}$ (where reachable displacement scales quadratically with available time, $d \propto \tau^2$).
- **High-speed moves ($d > d_{\text{crit}}$) are velocity-limited:** When displacement exceeds the acceleration runway, motion saturates at cruise velocity v_{max} . The trajectory comprises acceleration, constant-velocity cruise, and deceleration phases in a trapezoidal velocity profile, yielding un-jerked traversal time $t_{\text{min}} = d/v_{\text{max}} + v_{\text{max}}/a_{\text{max}}$ (where displacement scales linearly with available time, $d \propto \tau$).

The physical boundary between these two regimes is governed by the critical displacement:

$$d_{\text{crit}} = \frac{v_{\text{max}}^2}{a_{\text{max}}}$$

Commanding instantaneous step changes in acceleration assumes infinite jerk ($j = \|\dot{\mathbf{a}}\| \rightarrow \infty$). In physical machines, infinite jerk corresponds to an instantaneous step in motor torque, inducing gearbox backlash shock, exciting structural resonance modes, and generating motor current spikes that risk tripping inverter fault interlocks. Physical AI systems therefore constrain the maximum allowable jerk ($\|\dot{\mathbf{a}}\| \leq j_{\text{max}}$), which introduces a finite acceleration ramp-up delay $t_j = a_{\text{max}}/j_{\text{max}}$.

This finite ramp rounds the sharp corners of triangular and trapezoidal profiles into smooth S-curves (C^2 -continuous in position). For a preliminary schedule, adding $a_{\text{max}}/j_{\text{max}}$ to the idealized time gives an **illustrative allowance**,²⁰ not an exact seven-segment S-curve solution. For a velocity-limited move the estimate is:

$$t_{\text{est},j} = \frac{d}{v_{\text{max}}} + \frac{v_{\text{max}}}{a_{\text{max}}} + \frac{a_{\text{max}}}{j_{\text{max}}} \quad (20.3)$$

For acceleration-limited moves ($d \leq d_{\text{crit}}$), the analogous estimate is:

$$t_{\text{est},j} \approx 2\sqrt{\frac{d}{a_{\text{max}}}} + \frac{a_{\text{max}}}{j_{\text{max}}} \quad (20.4)$$

Geometrically, a **dynamic reachability envelope** $\mathcal{R}(\tau)$ contains states reachable from the measured state within the admitted time $\tau = t_{\text{exp}} - t_{\text{now}}$. The idealized rest-to-rest formulas above provide necessary time lower bounds under their stated acceleration and speed limits. A jerk cap further contracts the envelope, but its exact boundary requires the selected trajectory model and actuator

²⁰ | **Jerk-limited reachability envelopes:** A finite jerk cap changes acceleration ramp times. The additive $a_{\text{max}}/j_{\text{max}}$ term is a schedule illustration; exact reachability depends on initial state and the chosen S-curve profile. See Appendix B for the piecewise treatment.

constraints (see Appendix B). If the idealized lower bound already exceeds the remaining target-evidence horizon, the proposal can be rejected before a full solve. Passing this scalar test proves neither a feasible path nor an available safe stop.

In an analytical scenario, consider a manipulator resting at $\mathbf{p}_0$ with limits $v_{\max} = 0.60$ m/s, $a_{\max} = 2.0$ m/s², and jerk limit $j_{\max} = 20.0$ m/s³, yielding critical displacement $d_{\text{crit}} = (0.60)^2/2.0 = 0.18$ m and jerk ramp time $t_j = 2.0/20.0 = 0.100$ s = 100 ms. For a long displacement $d = 0.30$ m and an offered 450 ms window, the no-jerk trapezoidal formula alone exceeds the window; the illustrative jerk allowance in equation 20.3 gives 900 ms. For a short displacement $d = 0.08$ m and a 350 ms window, the no-jerk triangular formula also exceeds the window; the illustrative estimate in equation 20.4 is 500 ms. Both proposals fail even the necessary idealized test. The 900 ms and 500 ms estimates are not certified travel times for a particular joint trajectory. These scalar calculations reject proposals whose required idealized transit time already exceeds the remaining evidence window, returning `ERR_DYNAMIC_TIME_INSUFFICIENT`. Passing them only permits a fuller path, torque, and stopping check. Their runtime and any solver savings must be measured on the implementation.



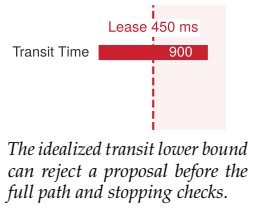
Checkpoint 20.4: Dynamic reachability vs. lease horizon

Before moving to the physical failure modes of learned perception, verify your ability to derive dynamic reachability limits and evaluate admission thresholds:

- ❑ **Dynamic reachability vs. static workspace:** Can you explain why an end-effector coordinate that lies well inside the static kinematic workspace can still be physically unreachable within a given temporal lease?
- ❑ **Triangular vs. trapezoidal profiling:** Can you derive the bifurcation threshold $d_{\text{crit}} = v_{\max}^2/a_{\max}$ and explain why short displacements follow $t_{\min} = 2\sqrt{d/a_{\max}}$ while longer displacements follow $t_{\min} = d/v_{\max} + v_{\max}/a_{\max}$?
- ❑ **Jerk-bounded envelope contraction:** Can you explain why finite jerk requires a fuller trajectory model beyond the no-jerk transit lower bound?
- ❑ **Early ingestion rejection:** Can you explain how a cheap analytical lower-bound check rejects some impossible targets before a full solver, and why its own latency must still be measured?

These dynamic reachability bounds rest on imported rigid-body mechanics under three assumptions, namely that actuator torque-speed curves remain within nominal thermal envelopes, joint encoders accurately report the initial state $(\mathbf{p}_0, \mathbf{v}_0)$, and the payload mass matches the inertial model used in the acceleration bounds. A representative nervous-system implementation monitors joint current, motor temperature, and communication delay at rates chosen from the plant and sensor budgets. These observations can detect some departures from the model but cannot verify every assumption, such as payload inertia, on every tick. Building on the actuator thermal limits established in section 4.3, when motor winding temperatures exceed thermal thresholds, or when an external force measurement indicates payload mass deviation, the nervous system imposes thermal derating, scaling down the maximum allowed acceleration ($a_{\max}$) and velocity ($v_{\max}$) in the parameter registry. The intent filter immediately evaluates subsequent incoming leases against these reduced physical limits. A derating event can make a previously admitted path infeasible; the controller must recheck it and select a feasible fallback if the margin is lost.

When the ingestion filter rejects an intent proposal, it must not simply discard the packet. Discarding without feedback leaves the upstream reasoning model unaware of why motion failed to occur, forcing it to wait for an end-to-end task timeout. The intent filter generates an immediate, structured diagnostic packet returned directly to the reasoning layer over the inter-core mailbox. This diagnostic payload contains the rejection classification, such as a kinematic workspace violation (`ERR_WORKSPACE_OUT_OF_BOUNDS`), dynamic time insufficiency (`ERR_DYNAMIC_TIME_INSUFFICIENT`), or swept-volume collision intersection (`ERR_OCCUPANCY_BLOCKED`), along with the numerical margins of failure. For a dynamic reachability failure, the feedback returns the shortfall $t_{\min} - t_{\text{rem}}$ and the maximum reachable distance along the commanded vector within the allotted lease. For a static workspace violation, the feedback provides the projection of the target onto the boundary of $\mathcal{W}$ (the



The idealized transit lower bound can reject a proposal before the full path and stopping checks.

closest physical point the arm can actually reach). The reasoning layer uses this structured signal to adjust its proposal on the subsequent inference cycle, by obtaining fresher evidence, choosing an alternate approach, slowing or pausing the process, or proposing base relocation. It cannot extend the evidence horizon without a new justification.

Early reachability filters remove known impossible targets but do not prove that every surviving goal has a feasible, collision-free trajectory. The planner and independent permission path must still check the complete motion. Geometry also cannot determine whether the selected coordinate names the intended object.

20.6 How Intent Goes Wrong

When a vision-language model outputs an $SE(3)$ target pose for a physical manipulator, that pose can fail in several distinct ways. It may be a hallucinated target generated from visual attention patterns on reflective background textures. It may be a stale target representing an object that a human coworker bumped or displaced 300 ms earlier while the camera pipeline was buffering frames. It may be a reference ambiguity where the model arbitrarily selects one of two identical machined parts on a tray without resolving which one the task requires. Or it may stem from an out-of-distribution scene shift, such as unexpected lighting or dust on an optical lens, that degrades the spatial prediction accuracy of the network. Despite differing causal origins, both hallucinated targets and stale correct targets fail downstream in the same way.

The planner receives a dated target proposal, not an actuator command. A pose on an empty location may lead to a futile reach; a pose inside an occupied fixture may lead to contact if the later path and permission checks fail. The wrong-beaker case is different: geometry can be valid while the selected object is semantically wrong. The independent permission path therefore checks what it can measure—freshness, coverage, geometry, current state, and actuator limits—without claiming to verify the task’s meaning.

A fresh depth observation can reject a proposed grasp whose required object evidence is absent in a covered, visible region. No return in an occluded or unobserved region leaves the region unknown. When several visually similar parts are present, a task-specific ambiguity test can refuse a low-confidence proposal and request clarification. A confidently wrong classification remains possible, so the test is a filter rather than a physical safety proof.

Table 20.2 separates these admission checks from responses after motion or contact has begun. In the manipulator-contact example above, the assumed 2 ms response delay leaves 39 N at brake-command onset, not a bound on the later peak. Precontact speed and force limits, actuator response, compliance, and available clearance determine whether the full contact trace stays within the validated envelope. A stop command or Safe Torque Off request alone cannot establish that result.

Table 20.2: Grounding failure modes and their evidence limits: Each check addresses a different failure class. The table separates proposal rejection from a response to an already moving plant; timing and force limits require configuration-specific validation.

Failure mode	Evidence or check	Admission or response	Physical limit to verify
Phantom target: image artifact in empty space	Fresh depth evidence and target-region occupancy, with coverage and occlusion checked	Reject the proposal when the required object evidence is absent; request another view if the region is unknown	Rejection precedes motion only if the proposal has not already been admitted. Missing returns alone do not prove empty space.
Stale target: workpiece moved after capture	Converted capture time, bounded clock error, local tracking, and target-error envelope	Revoke dependent tracking when the evidence limit fails; select the validated fallback	Total detection, communication, actuation, and braking delay must fit the measured clearance.
Semantic ambiguity: two plausible workpieces	Task-specific confidence or disagreement test on the candidate identities	Refuse admission and request clarification	A confidence test can miss a confidently wrong identity; independent physical checks remain necessary.
Misclassified affordance: bolted fixture selected as graspable	Contact-force and motion residuals after contact; precontact task/geometry checks where possible	Limit precontact speed and force, then invoke the validated contact response on a threshold crossing	A reactive trip does not cap the first impact peak; test the full force trace, response delay, and stopping energy.
Dynamic unreachability: target cannot be reached in its valid window	No-jerk transit lower bound followed by a complete path and stopping check	Reject a proposal whose lower bound exceeds the remaining evidence window	Passing the lower bound proves neither collision-free motion nor a feasible stop.

The proposed target must carry its spatial envelope, provenance, and expiry. A rejected proposal returns a reason to the learned layer; an admitted proposal remains subject to independent permission checks as the plant moves.



Checkpoint 20.5: Intent failure taxonomy and detection limits

- ☐ **Geometry and meaning:** Why can an occupancy check reject some phantom targets yet miss a confidently selected wrong object?
- ☐ **Staleness and stopping:** Which delays belong in the physical stopping budget after a target becomes invalid?
- ☐ **Contact response:** Why does the 39 N force at brake-command onset not bound the peak load?

20.7 The Intent Lease

This section formalizes the *concrete wire-level schema* and real-time execution protocol crossing the dual-brain boundary, building on the *conceptual intent contract* from section 20.2.

The wire example uses a fixed-size, statically parsed message in a bounded mailbox. A particular deployment may use FlatBuffers or a packed structure, but allocation and parse time must be measured against its control deadline. The 168-byte example below is an **implementation**, not a requirement of all robots. It transports a proposal; the independent permission controller owns the configured force ceilings, deadline ceilings, and state-dependent fallback.

The full covariance ellipsoid in the earlier grounding example is a producer-side localization estimate. The packet does not pretend that six floats encode a general oriented 3×3 covariance. Instead, `bounds_xyz` carries three conservative **position-error envelopes** (e_x, e_y, e_z) and three requested **task tolerances** (r_x, r_y, r_z) in the named robot-base frame. These have different meanings: uncertainty describes where the target could be, while tolerance describes error the task can accept. The producer must justify any conversion from covariance to an envelope, including calibration bias and declared tail risk. A 95 percent Gaussian ellipsoid alone is a probabilistic region, not a hard physical bound. Orientation tolerances and contact-mode rules in this compact example come from an independently approved task profile; the packet's quaternion remains the proposed orientation.

The timestamp `t_source_mcu` is the **physical capture epoch converted into the MCU monotonic domain** by a validated capture-to-MCU clock mapping with bounded synchronization error. The MCU matches both the parent digest and timestamp to a trusted sensor-record ring and adds the conversion error to its age test. UTC may be logged separately but is never subtracted from the MCU clock. The proposed `t_expire_mcu` uses that same monotonic domain; the MCU shortens it to the earliest of target-evidence expiry, configured lease ceiling, and any state-dependent stopping deadline. A parent SHA-256 digest links the proposal to an observation. It is **lineage, not message authentication**: it does not prove who chose the target, limits, or requested terminal mode. This example relies on the independent bounds and a protected mailbox writer policy; a deployment requiring sender authentication must add a keyed MAC over the command fields with protected key ownership and recalculate its layout NIST FIPS 198-1.

Table 20.3: Illustrative 168-byte intent proposal: The offsets sum to 168 bytes. The MCU validates capture age in its own clock domain, treats bounds as explicit base-frame envelopes rather than an encoded full covariance, and retains exclusive permission and fallback authority.

Offset	Field (bytes)	Meaning	MCU admission check
0x00..07	<code>sequence_id</code> (8)	Increasing proposal sequence	Newer than last accepted sequence
0x08..0F	<code>t_source_mcu</code> (8)	Converted capture epoch, MCU monotonic ns	Match trusted ring epoch; age plus sync error within evidence limit
0x10..2F	<code>parent_hash</code> (32)	SHA-256 observation digest	Match trusted sensor record; lineage only
0x30..67	<code>target_pose_SE3</code> (56)	Base-frame position, m; unit quaternion	Finite values, normalized quaternion, workspace prefilter
0x68..7F	<code>bounds_xyz</code> (24)	<code>float32[6]</code> : error $e_{x,y,z}$ and task tolerance $r_{x,y,z}, m$	$0 \leq e_i \leq r_i$; check each against approved profile; no covariance eigentest
0x80..87	<code>t_expire_mcu</code> (8)	Requested MCU-domain monotonic deadline, ns	Shorten to evidence, policy, and feasible stop ceilings
0x88..9F	<code>wrench_request</code> (24)	Requested three forces, N, and three torques, N m	Clamp or reject against independent task/actuator limits
0xA0..A7	<code>terminal_request</code> (8)	Fallback enum plus padding	Select a feasible approved response from current load/contact state

The fields in table 20.3 support layered checks. Sequence and parent-ring lookup reject stale or mismatched observations but do not authenticate the sender. Cheap workspace and target-envelope tests reject obvious failures before planning; passing them does not prove a collision-free path. A planner and permission check must still evaluate the swept path, unknown space, actuator constraints, total pre-brake delay, and current stopping clearance.

The admitted intent moves through Pending, Active, Renewed, Expired, and Aborted states. The MCU enters Active only after checking the request against independently configured ceilings and a feasible response from the measured state. A renewal needs a newer sequence, a fresh trusted observation, and a new permission check; it does not automatically extend authority. Meeting the approved task tolerance completes the goal. Unexpected contact or loss of a required belief aborts the dependent motion.

At expiration, the timer revokes tracking authority. The packet's `terminal_request` is advisory: the MCU selects among approved hold, controlled deceleration, or torque-disable responses according to load, contact mode, brake capability, and clearance. A suspended payload may need powered holding; disabling torque can drop it. A moving base needs enough room to decelerate. Neither a 1 kHz check period nor a hardware interrupt establishes a one-millisecond physical stop.

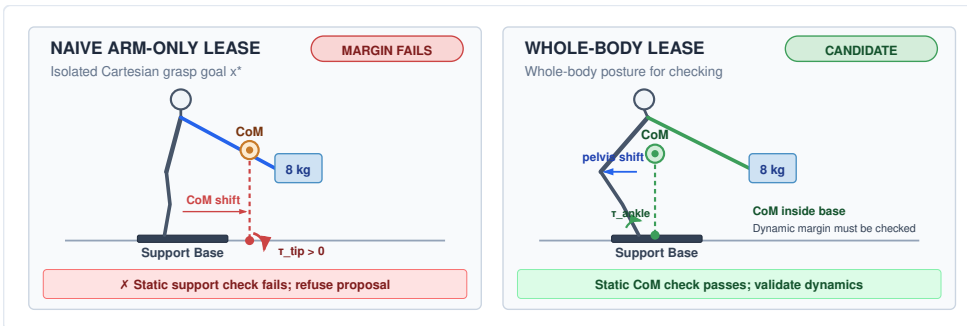
One implementation follows this bounded admission sequence:

1. **Evidence and sequence:** Read a complete packet from the protected mailbox. Reject an old sequence or a parent digest and converted capture epoch that do not match the trusted sensor ring. Compute capture age in the MCU monotonic domain and add the bounded clock-conversion error. Reject when that age exceeds the target-evidence horizon.

2. **Proposal limits:** Check finite pose values, quaternion normalization, base-frame error and task-tolerance axes, and requested wrench against approved task and actuator limits. Check known occupancy and unknown-space policy. These are early filters, not a swept-path proof.
3. **Dynamic admission:** Use current velocity, clearance, and actuator state to reject proposals that already fail a lower bound on transit time. Require the planner's checked path and the separate total sensing-to-brake-onset plus braking-distance condition before granting motion. If no feasible fallback exists from this state, refuse the proposal.
4. **Authority latch:** Store only the MCU-approved target, effort ceilings, expiry no later than the evidence deadline, and selected fallback. Arm the local timer for the admitted interval. The untrusted requested expiry and terminal mode do not control the drive directly.
5. **Runtime check:** On each configured control tick and relevant hardware fault event, compare measured state with the admitted limits. On evidence expiry, missed renewal, or an observed violation, revoke the proposal and start the previously validated response. The ensuing braking and hold have their own measured delay and physical margin.

This schema gives chapter 30 a concrete interface for fault-injection tests: corrupted quaternions, stale capture records, oversize force requests, and incompatible terminal requests should be rejected under the tested state and timing conditions. Such tests check this implementation's response; they do not establish safety outside its validated sensing, dynamics, and actuator envelope.

Systems Perspective 20.3: Affordance grounding and dynamic balance



Grounding high-level semantic intent on a bipedal humanoid forces the intent interface to couple manipulation targets with whole-body balance feasibility:

- **Whole-body affordances:** An 8 kg battery pick requires a target pose and a whole-body support check. The figure draws one posture with a CoM projection outside the support region and another inside. The payload mass alone cannot determine either CoM displacement or a corrective pelvis shift; those depend on the robot's link masses, joint pose, and contact geometry. The permission tier admits a posture only after those measurements and dynamic balance margins are checked.
- **Contact schedule:** Walking and pushing change support contacts. For an illustrative single foot with $F_N = 350$ N and $\mu = 0.5$, the nominal Coulomb friction limit is 175 N. A proposed 120 N horizontal push is **below** that nominal limit, so the given numbers do not prove slip. Dynamic load transfer, force direction, and other disturbances can reduce usable margin; a request exceeding the validated margin needs another support contact or refusal.
- **Expiry while walking:** A mid-stride lease expiry cannot be handled by freezing joints without checking balance. The controller needs a separately validated recovery for the current support phase, available contact forces, actuator authority, and remaining time. If it cannot reserve such a response, it must refuse the task before entering that state.



Checkpoint 20.6: Intent wire contracts and hardware watchdog enforcement

Before exploring the fallacies and pitfalls of grounded autonomy, evaluate your understanding of the byte-level serialization and real-time execution invariants of the intent lease:

- ☐ **Bounded parsing:** Can you explain why this example uses a fixed-size, measured parse path across the permission boundary?
- ☐ **Cross-core clock domain safety:** Why must this example convert the capture timestamp into the MCU monotonic domain with bounded error before checking age and the requested absolute expiry?
- ☐ **Fallback selection:** Can you explain why the MCU must select a feasible approved hold, deceleration, or torque-disable response from the measured load state rather than obeying the packet's terminal request?
- ☐ **Whole-body balance:** Can you explain why a bipedal payload pick needs measured CoM and contact geometry beyond an end-effector target?

20.8 Fallacies and Pitfalls

An intent can become unsafe despite unchanged contents. Local checks must withdraw tracking authority when the goal's temporal validity window expires or its physical justification dissolves, even when no status updates or abort packets arrive across the SerDes bus.

Fallacy: *Upstream host crashes or hangs will reliably emit cancellation packets to downstream actuators.*

A manipulator approaches a fixture under intent records with a $\Delta t_{\text{lease}} = 100.0$ ms lease. The host accelerator deadlocks and sends neither a renewal nor an error packet. Continuing the last trajectory leaves the arm approaching the fixture, while abruptly removing stator current can cause the robot to go limp and release its suspended payload due to a sudden loss of holding torque. The real-time controller needs a local expiry check and an achievable transition to controlled deceleration and a gravity compensation hold. Its response cannot depend on cooperation from the failed host. Lease duration, detection delay, and the kinematic stopping envelope must be budgeted together so that communication timeouts revoke authority while the machine still has sufficient clearance to decelerate into the validated holding state.

Pitfall: *Allowing an unexpired temporal lease to continue commanding motion when target spatial bounds are violated.*

An arm lowers its gripper toward a bracket under an intent with tolerance radius $r_{\text{tol}} = 12.0$ mm. Conveyor vibration shifts the bracket 40.0 mm laterally while 75.0 ms remain on the lease. The timestamp still permits execution, but the target has left the $SE(3)$ region that justified the approach. Continuing until expiry would preserve an obsolete, potentially colliding command. The local perception and enforcement loops must compare the observed target with the intent's spatial bounds and withdraw permission when those bounds fail. Encoding that geometric condition in the contract allows the controller to interrupt the descent without waiting for a new high-level plan.

Pitfall: *Oscillating between emergency braking trips and uncontrolled execution during inference latency jitter.*

A mobile manipulator's coprocessor undergoes thermal derating, delaying intent renewals beyond its $\Delta t_{\text{lease}} = 100.0$ ms lease. Repeated hard braking and immediate restarts can excite payload oscillation through gearbox compliance, while automatically extending the lease would allow motion on stale intent. The controller must reserve enough remaining authority to decelerate along a jerk-limited (C^2 -continuous) trajectory and enter a stationary hold. It can then wait for a valid renewal without alternating between full execution and abrupt bus-jitter-induced stops. The lease and stopping horizon must shape normal degradation behavior, so predictable latency variation does not repeatedly trigger a mechanical emergency maneuver.

Fallacy: *Neural networks can resolve bimodal spatial ambiguity by averaging candidate target locations.*

A sorting robot sees two identical connectors 35.0 mm apart and receives their spatial mean as a grasp target. The midpoint lies in empty space, so averaging has destroyed the $SE(3)$ distinction the task requires. In this example, the reported state covariance expands beyond the permitted targeting uncertainty, allowing the admission gate to reject the proposal and maintain a hold. That check is useful only when the estimate correctly captures bimodal ambiguity. The intent interface must preserve statistical uncertainty rather than collapse competing object hypotheses into an apparently precise Cartesian coordinate, and the local controller must defer the pick when the resulting target distribution cannot meet its physical admission bounds.

20.9 Summary

A learned model proposes a dated target and an uncertainty description. The independent permission tier checks the target's evidence age, configured motion bounds, current state, path clearance, and feasible fallback before applying motion. The target-evidence horizon $\tau_{\text{evidence}} \leq (r_{\text{tol}} - e_0)/v_{\text{drift}}$ limits how long the target estimate may be used under its stated drift envelope. It does not provide time to stop. A separate total-delay and braking check must fit within the available physical clearance.

The 60 ms conveyor example makes the cadence constraint concrete: a model that updates every 200–1000 ms cannot maintain that target record by itself. A qualified local tracker can refresh it from new evidence; otherwise the machine slows, pauses, or refuses the task. Expiry revokes the proposal's tracking authority and invokes a response validated for the measured state. It cannot undo motion already executed.

An early reachability lower bound rejects some impossible requests before full trajectory planning. Passing it does not establish semantic correctness, collision-free motion, or a safe stop. The wrong-beaker case remains a semantic failure even when geometry and timing checks pass. Conversely, a force trip after contact limits later loading only according to the measured response and available actuator authority; precontact limits are needed to bound the first impact.



Key Takeaways: Grounded intent at the permission boundary

- **Proposal:** The brain supplies a target, evidence epoch, error envelope, task tolerances, and requested expiry; it does not write actuator setpoints.
- **Evidence deadline:** Target drift and task tolerance set a finite validity horizon under a declared operating envelope. A fresh timestamp alone does not establish a correct target.
- **Motion deadline:** The permission path separately budgets sensing, communication, enforcement, actuator delay, and braking distance for the current state and clearance.
- **Refusal:** Ambiguous identity, stale evidence, insufficient reachability, or infeasible stopping can each reject a proposal. Each check has its own evidence limit.



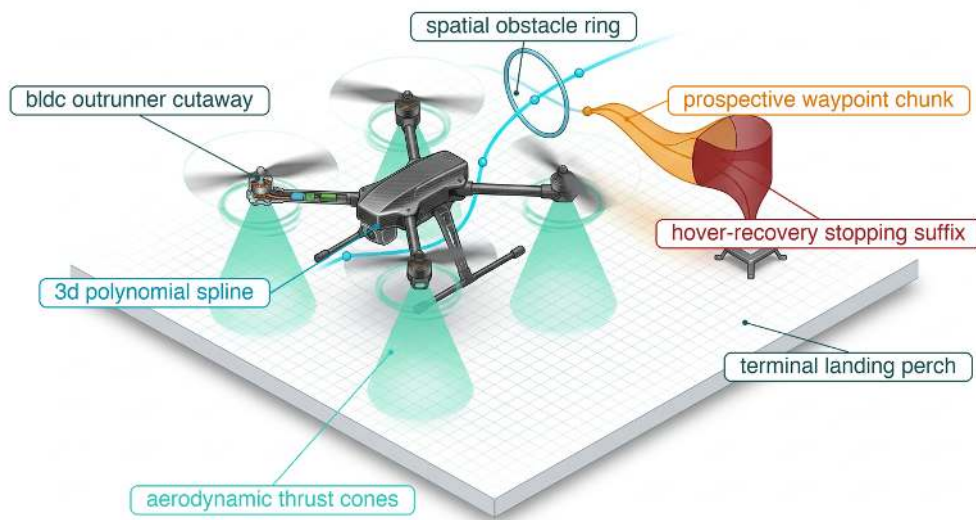
What's Next: From intent leases to trajectory planning

The intent boundary turns an uncertain semantic request into a dated geometric proposal. Ambiguity and wrong-target risk can remain, so the independent tier admits motion only within checked evidence, effort, and stopping limits. The next decision, developed in chapter 22, is a trajectory that reaches an admitted target with feasible jerk, torque, path clearance, and time while preserving a checked fallback.

Trajectory Planning

DRAFT

Trajectory Planning

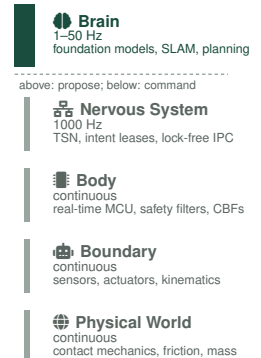


- 22.1 Continuous Trajectories
- 22.2 What Planning Hands Over
- 22.3 From Goal to Trajectory
- 22.4 Kinodynamic Feasibility
- 22.5 Behavior at the Seam
- 22.6 How Plans Go Wrong
- 22.7 The Trajectory Contract
- 22.8 Fallacies and Pitfalls
- 22.9 Summary

Purpose

What does a physical machine do during the unbudgeted milliseconds when the next neural trajectory chunk is late?

A moving machine retains momentum while its policy computes the next action. If the current trajectory ends before its replacement arrives, waiting impacts clearance and stability. Abruptly substituting a new command can also demand accelerations that the actuators cannot deliver or excite vibration in the mechanism. Planning must convert successive proposals into motion that remains feasible during transitions and absence. Smooth connections between trajectory segments protect the body during normal execution, while a stopping continuation gives the execution layer a defined response to an overdue plan. That continuation must remain feasible from the states the current plan can reach; attaching a stop command after the planning horizon has expired is too late. Trajectory planning therefore couples the brain's variable computation time to the nervous system's continuous execution and the body's capacity to change motion.





Learning Objectives

- Select trajectory horizons from inference latency, observation freshness, and the time needed for fallback
- Evaluate trajectory feasibility and continuity against the body's actuator and transmission limits
- Specify physically feasible continuation behavior when a replacement trajectory arrives late or never arrives
- Distinguish trajectory failures detectable before execution from those requiring monitoring during physical motion

22.1 Continuous Trajectories

An articulated robotic arm lowering a cutting tool onto a moving workpiece possesses physical mass, structural inertia $\mathbf{M}(\mathbf{q})$, and continuous kinetic energy $E_k = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}}$. It cannot change velocity or direction instantly. Even when upstream autonomy grants a time-bounded intent lease—authorizing motion toward a grounded semantic goal—that semantic permission alone cannot turn motor shafts. The machine must synthesize continuous nominal motion while target evidence remains valid, plus a separately authorized stopping continuation if tracking authority ends. In the running implementation, a motor tracking loop evaluates feedback and setpoints at 1000 Hz; that period does not specify the physical stopping response. Above this loop sits the cognitive deliberative brain, where modern robot learning policies—such as Action Chunking with Transformers (ACT) (Zhao et al. 2023) or score-based Diffusion Policies (Chi et al. 2024)—process multimodal observations to synthesize future reference paths on a Linux accelerator.¹ Inference latency varies with workload and contention. In an illustrative measured trace, a nominal 40 ms forward pass could extend beyond 150 ms under memory or scheduling interference; the deployed tail needs its own measurement.

If an architecture predicts actions one step ahead ($H = 1$), the motor tracking loop starves. If it generates a multi-step **action chunk** across a finite future horizon ($H > 1$), the machine encounters the peril of the **execution seam**. During the unbudgeted milliseconds when an active trajectory chunk finishes executing before its replacement clears the tail of the inference distribution, the controller faces an execution discontinuity. If the controller clamps to the final position, the reference velocity steps to zero and the drive may saturate; actual torque and damage depend on current limits, compliance, and protection. If a newly arrived chunk begins with a velocity or acceleration that diverges from the physical state of the moving mechanism, the step discontinuity in commanded torque violently shakes the chassis. A trajectory proposal must define nominal motion, boundary derivatives, and a stopping continuation admitted before motion begins.



Definition 22.1: Kinodynamic feasibility envelope

Kinodynamic feasibility envelope is the subset of state space from which a physical machine can track a candidate trajectory while strictly satisfying simultaneous actuator torque limits, velocity ceilings, and third-order jerk bounds (C^2 continuity) without exceeding mechanical stopping margins.

1. **Significance:** Geometric collision freedom is necessary but insufficient for physical execution; an arm may follow a collision-free path that nonetheless demands motor torques exceeding continuous thermal ratings or produces accelerations that tip an underactuated base. The envelope states the conditions under which the model predicts trackable motion; runtime sensing and actuator checks must still validate them.
2. **Distinction:** Unlike pure kinematic reachability (which evaluates whether the robot's linkages can attain a spatial configuration), kinodynamic feasibility incorporates time parameterization, actuator force/torque envelopes, reflected inertia, and mechanical braking runways.

¹ | **Action chunking representations:** Action Chunking with Transformers (ACT) predicts sequences of future joint setpoints using a conditional variational autoencoder with temporal ensembling, whereas Diffusion Policy generates continuous action trajectories via iterative score-based denoising solvers. Both paradigms decouple slow, stochastic neural inference from high-rate motor servoing loops. However, because multi-step chunk outputs lack intrinsic boundary derivative constraints, downstream controllers must synthesize transition splines to prevent acceleration shocks across replanning seams.

Prerequisite: Kinetic energy (E_k) and joint torque limits are established in section 4.3.

3. **Common pitfall:** Checking feasibility only at discrete waypoints. Interpolation splines between sample points can produce intra-sample velocity overshoots and torque spikes that breach motor limits even when every individual waypoint passes validation.

As shown in figure 22.1, the planner is the fifth and final deliberative stage of the cognitive brain. Intent defines the admissible goal; planning supplies the continuous motion, temporal parameterization, and dynamic limits needed to pursue that goal.

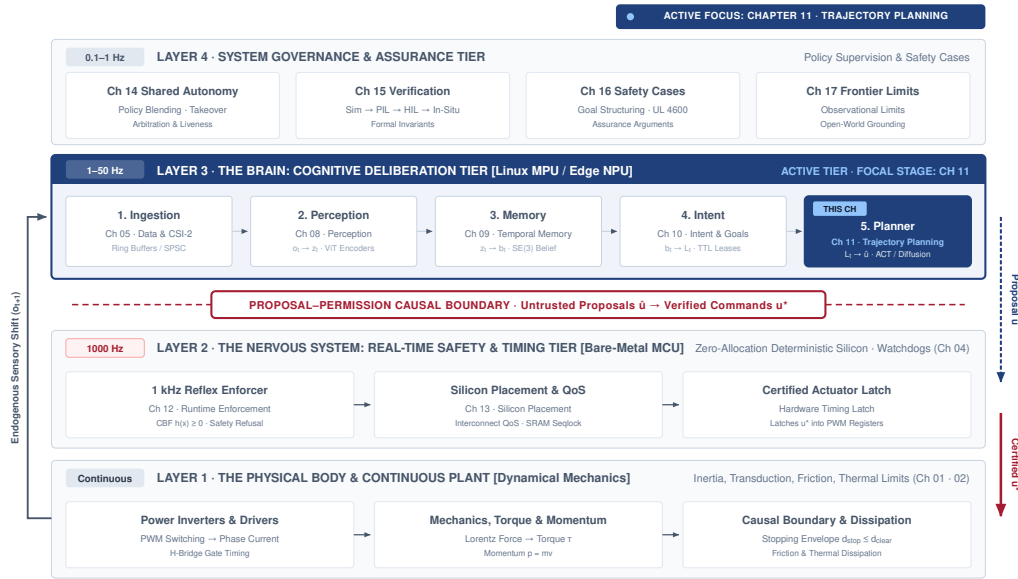


Figure 22.1: Planner stage architecture focus: In the physical AI systems architecture, the deliberative brain proposes time-parameterized action chunks across an unprivileged boundary to the deterministic nervous system. Because neural inference latencies exhibit heavy-tailed stochastic distributions, every plan must define its own replacement timing, boundary derivatives, and precomputed stopping suffix before execution begins.

Across the **proposal-permission boundary**, a plan is not a direct command written to actuator registers, nor is it a guaranteed physical path. It is an unprivileged proposal submitted across the MPU-to-MCU inter-processor communication bus to the tracking controller and the real-time safety enforcer (chapter 24).² Because measured inference latencies can have consequential tails, the downstream controller in this design consumes admitted references on its own clock and cannot suspend physical motion while the model finishes. A plan must therefore be an atomic, self-contained contract: it must establish second-order derivative continuity (C^2 continuity) at its admission seam,³ budget its replacement dispatch epoch against extreme tail quantiles, and include a stopping suffix admitted with the nominal motion, subject to the current state, clearance, and actuator assumptions.

Connecting variable inference latency between slow, variable neural inference and high-frequency, continuous actuator tracking requires treating trajectory generation as a time-bounded contract with explicit seam continuations. Rather than transmitting raw geometric waypoints or unconstrained torque arrays across the system bus, the deliberative planner must assemble a self-contained operational packet that encapsulates validity intervals, dynamic boundary derivatives, and validated fallback profiles before releasing authority to physical hardware.

The first decision is what the planner must hand to the independent execution tier while the body is still moving.

² | **Deterministic proposal verification:** The deterministic nervous system isolates unprivileged trajectory proposals within hardware-managed staging buffers, evaluating kinematic feasibility and tracking invariants before latching setpoints into motor control registers. A bounded admission pass checks the complete candidate before activation; a separate per-tick check monitors the active state. The measured admission budget depends on the maximum chunk size. For runtime safety filter architectures, see section 24.1.

³ | **C^2 derivative continuity:** Enforcing C^2 continuity guarantees that position, velocity, and acceleration remain unbroken functions of time across trajectory boundaries. In permanent-magnet synchronous motors (PMSM), electromagnetic torque couples directly to quadrature stator current ($\tau = K_t I_q$). An acceleration step asks for a finite torque and current step under this model, while a velocity step implies an ideal acceleration impulse. Neither C^2 matching nor a polynomial name alone bounds interior jerk or resonance response.

Prerequisite: Expiring intent leases provide the bounded time windows discharged by trajectories in section 20.7.

22.2 What Planning Hands Over

For a two-joint arm, one pair of joint angles (q_1, q_2) is one point in **configuration space** $\mathcal{C}$. An obstacle excludes some angle pairs, leaving $\mathcal{C}_{\text{free}} = \mathcal{C} \setminus \mathcal{C}_{\text{obs}}$. A curve through that free set says where the arm may move, but not how quickly it may move or whether its motors can supply the required torque (Lozano-Pérez 1983). A **trajectory** assigns time to the curve and exposes $\mathbf{x}(t) = [\mathbf{q}(t)^T, \dot{\mathbf{q}}(t)^T, \ddot{\mathbf{q}}(t)^T]^T$. The planner proposes that time-indexed reference, and the independent permission tier checks it before any setpoint reaches the drive. The tracking controller interpolates an admitted reference at its configured cadence; the enforcer checks current state and limits during execution. A geometric path alone supplies neither timing nor fallback clearance.

The packet names its coordinate frame $\mathcal{F}$, generation epoch t_0 , initial reference state, and four deadlines in the **MCU monotonic clock domain**: nominal tracking begins at t_{start} , the nominal setpoints end at $t_{\text{plan_exp}}$, replacement closes at t_{commit} , and the admitted stop finishes at $t_{\text{stop_end}}$. It also links to the active intent expiry $t_{\text{intent_exp}}$. Conversion from the sensor and host clocks carries a bounded error. The independent tier permits nominal **TRACK** only while the intent and plan are valid. Without a successor it transfers to MCU-owned **STOP** no later than t_{commit} ; the protected stop may finish after the intent's tracking permission expires. Earlier lease expiry or unexpected invalidation requires a separate state-matched MCU fallback, because the suffix matched at t_{commit} cannot be played from an arbitrary earlier state.

The initial reference state matters while the plant moves during inference. Suppose the host samples at $t = 0$ and spends 60 ms planning while the arm advances 20 mm. Starting the new chunk from that historical pose would create a 20 mm reference error. Before admission, the MCU compares the candidate seam with its current encoder state, checks position, velocity, and acceleration, and either admits a feasible bridge or retains the active plan and fallback. Projecting the active *reference* forward helps form a candidate, but it is not a measurement of the plant.

The candidate declares payload, contact, effort, and obstacle assumptions. The MCU first validates its finite header and lineage, then checks the complete bounded trajectory and suffix, including interior extrema, clearance, stopping distance, and a feasible response for reachable states. A scan over N setpoints is $O(N)$, even with a fixed maximum N and measured worst-case execution time. After admission, a distinct $O(1)$ per-tick check reads the current state and local reference. The MCU latches the validated payload and stop coefficients into protected memory before enabling **TRACK**; a host crash or shared-memory corruption cannot remove the resident fallback. A separately validated immutable baseline covers earlier invalidation or a state outside the matched suffix's entry envelope.

The two consumers have different jobs. The tracker interpolates the admitted reference and feedforward terms. The independent permission path monitors current limits and authority; it does not inherit the planner's feasibility claim as proof. A fresh obstacle, changed payload, or contact event can revoke nominal tracking and invoke the state-matched fallback. The packet therefore binds nominal motion, the commitment point, and a physically checked stopping continuation into one reviewable request.⁴

Structuring the plan handoff as a bounded, hardware-grounded contract shifts the burden of safety and timing from the non-deterministic planner to the deterministic nervous system. The tracking controller receives the dense state targets, timestamps, and feedforward terms required to drive the actuators along a smooth path. The safety enforcer receives the generation metadata, dynamic assumptions, validity bounds, and terminal suffix needed to guard the machine against latency tails, state divergence, and physical model mismatches. When each field in the handoff corresponds to an explicit runtime verification, the nervous system can execute high-speed motions without relying on the assumption that the next inference cycle will always succeed.



Checkpoint 22.1: The trajectory packet contract

Before exploring the generation of trajectories from high-level intent, verify your understanding of the interface boundary between planning proposals and real-time execution:

- ☐ **Dual consumer roles:** Can you distinguish between the operational requirements of the tracking controller and the real-time safety enforcer when consuming a trajectory packet?

⁴ | **Inter-core shared-memory contracts:** A shared-memory implementation needs an ownership protocol, cache maintenance where required, and release/acquire ordering. The MCU validates a completed candidate and copies the admitted plan and stop into protected memory before activation. The protocol must be tested on the selected platform.

- **State alignment and drift:** Can you explain why a trajectory packet must declare both its generation timestamp t_0 and initial state $\mathbf{x}_0$, and how the nervous system detects execution drift?
- **Lockless buffer mechanics:** Can you describe how a double-buffered execution queue with atomic pointer swaps prevents race conditions between an asynchronous planning thread and a 1 kHz servo loop?

22.3 From Goal to Trajectory

An intent specifies a desired physical outcome—such as touching a workpiece with a specified contact force or placing an end-effector at a target pose (chapter 20)—but an actuator cannot execute a goal. A goal defines where the machine should arrive or what constraint it should satisfy, without providing joint velocities, torque bounds, intermediate kinematic states, or a continuous time schedule to move physical mass through space. The nervous system requires a trajectory, which assigns concrete state targets, feedforward actuation terms, and explicit timestamps across a bounded temporal interval. If a learned policy could evaluate multimodal state observations and output valid motor currents instantaneously at the hardware control period (1000 Hz), planning as an architectural stage would not exist; the machine would operate purely as a high-frequency feedback controller. In physical systems, learned neural policies require tens or hundreds of milliseconds to compute a motion sequence, while the tracking controller and safety enforcer (extending the proposal-permission architecture from section 6.1) must execute deterministically every few milliseconds. A plan assigns time and physical limits to a goal; the independent permission path decides whether its proposed motion may run.

In classical robotics systems, bridging this gap was achieved through sampling-based path planning. Rather than constructing explicit algebraic representations of the high-dimensional configuration obstacle manifold $\mathcal{C}_{\text{obs}}$, algorithms randomly sample the free configuration space $\mathcal{C}_{\text{free}}$. Multi-query architectures, pioneered by Kavraki et al. (1996) through the **Probabilistic Roadmap (PRM)**, construct a global topological roadmap of interconnected collision-free configurations during an offline preprocessing phase, enabling rapid online multi-query path lookups. For dynamic, single-query real-time navigation, LaValle (1998) introduced the **Rapidly-Exploring Random Tree (RRT)**, which incrementally grows a space-filling tree biased toward unexplored Voronoi regions of $\mathcal{C}_{\text{free}}$, providing probabilistic completeness under its stated assumptions without requiring a precomputed roadmap under the usual sampling assumptions.

To span this temporal gap between slow model inference and high-frequency actuator control, modern robot learning architectures formulate policy execution via **receding-horizon action chunking**, predicting multi-step trajectory sequences over a forward horizon while committing only an initial temporal prefix before replanning on fresh observations.⁵ Two dominant paradigms characterize this generation:

1. **Action Chunking with Transformers (ACT):** Formulated as a Conditional Variational Autoencoder (CVAE) paired with a Transformer encoder-decoder (Zhao et al. 2023), ACT predicts a discrete sequence of future joint setpoints $\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_H] \in \mathbb{R}^{H \times D}$ simultaneously across a prediction horizon H . To mitigate compounding trajectory drift across successive chunks, ACT applies *temporal ensembling*, maintaining an exponential moving average over overlapping predicted chunks ($w_i = \exp(-m \cdot i)$). Temporal ensembling averages overlapping setpoints but does not enforce derivative continuity. If a downstream tracker admits such a seam unchecked, it may ask for high-rate reversals that must be assessed against the actuator thermal and vibration limits in section 4.5.
2. **Diffusion Policy:** Formulating trajectory synthesis as an iterative score-based denoising process (Chi et al. 2024), Diffusion Policy rolls out future action trajectories by solving a score Stochastic Differential Equation (SDE) or Denoising Diffusion Implicit Model (DDIM) ODE conditioned on visual and proprioceptive histories. A diffusion policy predicts a chunk of horizon T_p (16 steps in the real Push-T setup), executes a subset T_a (six steps there), and replans on subsequent sensor ingress (figure 22.2). While diffusion models capture rich

⁵ | **Dynamic Programming Lineage:** The principle of optimality and discrete dynamic programming algorithms were formalized by Bellman (1957) for sequential decision processes. While DP guarantees globally optimal trajectory synthesis over discretized state lattices, computational complexity scales exponentially as $O(K^d)$ where d is state dimension and K is grid resolution. In high-dimensional physical systems, this exponential curse of dimensionality forces trajectory planners to use continuous trajectory optimization or randomized sampling.

multimodal distributions, their iterative denoising schedules can add workload-dependent inference latency and variance.

3. **Learned warm-start for trajectory optimization:** A learned model can propose an initial path for a constrained solver. For state sequence $\mathbf{x}_{1:H}$ and controls $\mathbf{u}_{1:H}$, the numerical stage solves:

$$\min_{\mathbf{x}_{1:H}, \mathbf{u}_{1:H}} \sum_{t=1}^H \ell(\mathbf{x}_t, \mathbf{u}_t) \quad \text{subject to} \quad \mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t), \quad g(\mathbf{x}_t, \mathbf{u}_t) \leq 0$$

A useful proposal can reduce iterations when it lies near a feasible basin. It may also choose the wrong side of an obstacle or fail to converge by the deadline. The learned guess is never a feasibility certificate: admit only a converged candidate whose full path, seam, torque, and stop checks pass. Iteration count and milliseconds depend on solver, scene, hardware, and workload and need a measured trace.

The scheduling quantity is **replacement latency** L : elapsed time from the observation needed for a new proposal through transfer, inference, full-trajectory validation, and admission. A representative stage ledger has 15, 5, 65, 2, 18, and 1 ms for sensing, DMA, inference, deserialization, validation, and activation, respectively, totaling 106 ms. These are scenario assumptions, not a hard bound; overlapping stages require a measured end-to-end trace rather than a sum of stage percentiles.

$$L = t_{\text{sense_ingress}} + t_{\text{ipc_dma}} + t_{\text{infer}} + t_{\text{deser}} + t_{\text{validate}} + t_{\text{admit}} \quad (22.1)$$

The component sum in equation 22.1 is an elapsed replacement path, not an absolute timestamp. Consider $K = 3000$ handoffs in a ten-minute, 5 Hz illustrative mission. If each handoff independently has a 0.01 chance of exceeding a P_{99} budget, the expected number of late handoffs is 30 and the probability that *all* arrive on time is $(0.99)^{3000} \approx 8 \times 10^{-14}$. Correlated contention can cluster misses, so this binomial all-on-time value is not a measured mission reliability. Allocating a mission exceedance target $\epsilon_{\text{mission}} = 10^{-3}$ equally gives a per-handoff target $\epsilon \leq \epsilon_{\text{mission}}/K \approx 3.3 \times 10^{-7}$ by the union bound. The corresponding $P_{99.99997}$ is a **design quantile target**, not a validated latency guarantee. A deployment must establish a conservative bound with representative load testing and uncertainty treatment, or accept a higher measured fallback rate.

The next request is sent T_{request} after the active chunk starts. Admission must finish **before** t_{commit} , not merely before the last nominal setpoint. With a replacement-latency target $Q_{1-\epsilon}(L)$ and reserve T_{reserve} , require $t_{\text{commit}} - t_{\text{start}} \geq T_{\text{request}} + Q_{1-\epsilon}(L) + T_{\text{reserve}}$. The admitted stop then needs another T_{stop} under MCU authority. The complete **time horizon**, regardless of encoding, has the conditional lower bound:

$$T_{\text{record}} \geq T_{\text{request}} + Q_{1-\epsilon}(L) + T_{\text{reserve}} + T_{\text{stop}} \quad (22.2)$$

The complete duration in equation 22.2 includes the stop suffix. For the conveyor-tracking arm, stipulated $T_c = 10$ ms, request delay 60 ms, replacement target 160 ms, reserve 20 ms, and a separately checked 300 ms stop require at least 540 ms of complete motion authority, or 54 ten-millisecond **duration equivalents**. This does not mean 54 stored setpoints: N in the packet counts only nominal samples; the stop is represented by its separately validated polynomial coefficients. The nominal replacement deadline must be at least 240 ms from start; the worked 360 ms commitment below leaves 120 ms beyond that target. A 240 ms *total* authority horizon would omit the stop runway. In the worked arm, nominal setpoints end at 400 ms and the admitted stop completes at 660 ms, so the complete authority window spans 66 ten-millisecond duration equivalents.

This timing lower bound competes with target-evidence age. Under the illustrative conveyor drift model $a_{\text{drift}} = 0.50 \text{ m/s}^2$ and 15 mm remaining tolerance, $\Delta x = \frac{1}{2} a_{\text{drift}} t^2$ reaches the allowance in about 245 ms. A single observation cannot justify nominal tracking to the 360 ms commitment. The machine needs a qualified local tracker and fresh intent evidence before that limit; otherwise TRACK ends at evidence expiry and the MCU uses its separately validated state-matched stop. Extending a target timestamp because the compute tail is long would invert the permission rule.



The tail target allocates replacement risk; observed latency and confidence determine whether the chosen deadline is credible.

🔍 Systems Perspective 22.1: The replacement and stop horizon

Budget replacement admission before t_{commit} and the complete stopping continuation after it. A tail percentile allocates risk; it does not replace observed timing and physical clearance validation.

📋 Checkpoint 22.2: Action chunk horizon and tail latency budgeting

Before evaluating real-time disturbance recovery and future-state conditioning, verify your ability to dimension action chunk horizons against extreme latency tails:

- ❑ **Mission exposure and tail quantiles:** In the stated independent-handoff illustration, what is the chance of one or more missed P_{99} deadlines across 3000 handoffs, and why is that not a measured mission failure rate?
- ❑ **Horizon sizing trade-off:** Can you distinguish between the risk of trajectory buffer starvation from too short an action chunk and the risk of unmodeled state drift and stale perception from an excessively long chunk?
- ❑ **Reserve margin allocation:** Can you describe why the horizon equation requires an explicit reserve term T_{reserve} dedicated to trajectory splicing, safety filtering, and timer jitter beyond raw inference latency?

When a generative policy serves as the planner, iterative score evaluation and denoising steps govern the shape of this latency distribution. The internal mechanics of diffusion schedules, attention caching, and token generation sit below the first budgetable quantity of the physical AI system. Building on the causal boundary established in section 2.2, the systems interface treats the learned model strictly as a stochastic producer of motion chunks characterized by an empirical latency profile and an output horizon N . In physical hardware deployments of receding-horizon action chunking, the execution buffer absorbs inference latency while allowing the controller to react dynamically to disturbances (figure 22.2). Sizing N against a validated latency distribution lowers the risk of a late replacement; the protected stop handles a miss. Yet ensuring that a replacement trajectory arrives in time addresses only the temporal continuity of the plan. The state samples contained within the admitted chunk must also match the kinematic capabilities of the joints and remain continuous with the physical velocity and acceleration of the mechanism at the handoff seam.

A replacement planner must account for motion during its computation. In an illustrative 80 ms delay at a constant joint speed of 0.50 rad/s, a stale request-epoch pose lags the moving reference by 0.040 rad (2.29°). This is a reference displacement, not a measured tracking error or a guaranteed inverter trip. A proposed replacement can instead start from the active plan's projected reference at the intended seam:

$$\Delta q_{\text{reference}} = \int_{t_0}^{t_{\text{seam}}} \dot{q}_A(t) dt, \quad \hat{\mathbf{x}}_{\text{seam}} = \mathbf{x}_A(t_{\text{seam}}) \quad (22.3)$$

The projected seam in equation 22.3 is useful for constructing a candidate; contact, tracking error, or scheduling changes can make the plant differ. The MCU therefore measures current encoder state, validates the actual seam $(q, \dot{q}, \ddot{q})$ and any bridge, and refuses a candidate outside the admitted envelope. Algebraic matching of two *reference* trajectories gives C^2 continuity only for those references; it does not establish zero physical tracking gap.

📖 Definition 22.2: Future-state conditioning

Future-state conditioning uses the active plan's projected reference at an anticipated handoff time to form the next candidate. It reduces the expected reference seam gap when timing and

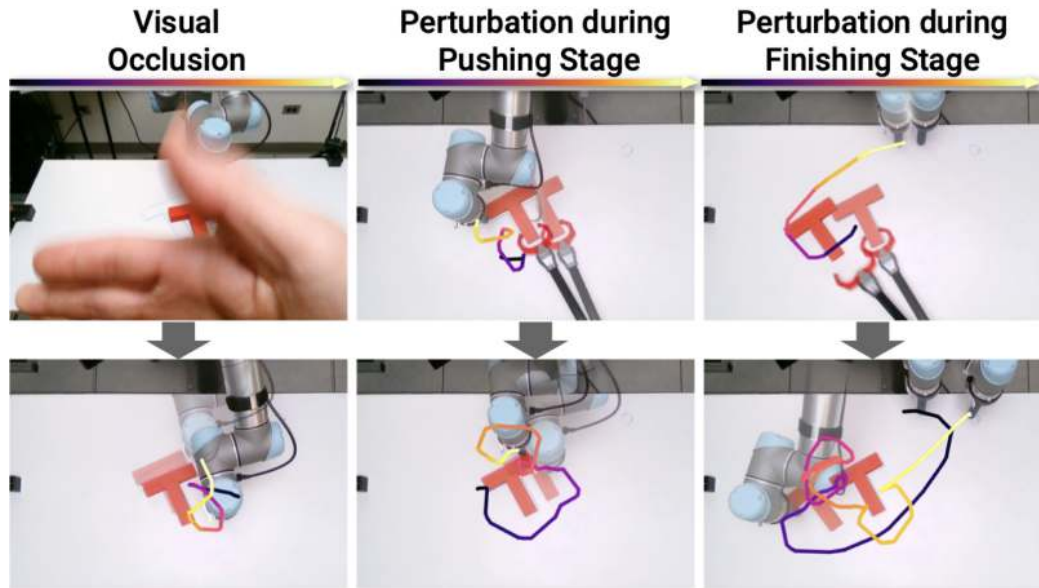


Figure 22.2: Push-T disturbance recovery: Chi et al. report a real Push-T configuration with 16 predicted and 6 executed action steps; 10 Hz commands were interpolated to 125 Hz. The illustrated recoveries show closed-loop replanning in those tests. They do not establish a validated stopping suffix or immunity to inference starvation (Chi et al. 2024).

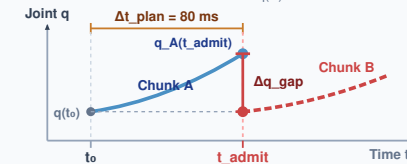
tracking remain close to the model. Admission still compares the proposal with measured state and checks a feasible bridge; a projection cannot certify the actual plant state.



Checkpoint 22.3: Execution delay gaps and future-state conditioning

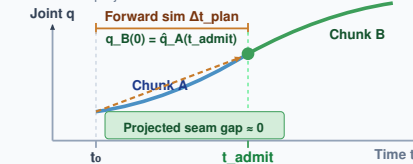
Naive State Conditioning

Conditions rollout on stale measurement $q(t_0)$



Future-State Conditioning

Rollout uses projected active reference



Before analyzing kinodynamic feasibility and transmission dynamics, evaluate your understanding of computational latency compensation at the handoff seam:

- ☐ **Execution delay gap origin:** Can you calculate the spatial offset injected into a robot joint if a planner conditions on the current measured state rather than projecting forward across computational latency?
- ☐ **Forward simulation protocol:** Can you explain why projected reference matching still requires encoder-state and bridge checks at admission?
- ☐ **Disturbance vulnerability:** Can you identify the failure mode that occurs when a physical disturbance displaces the mechanism while an upstream planner is solving a future-conditioned rollout?

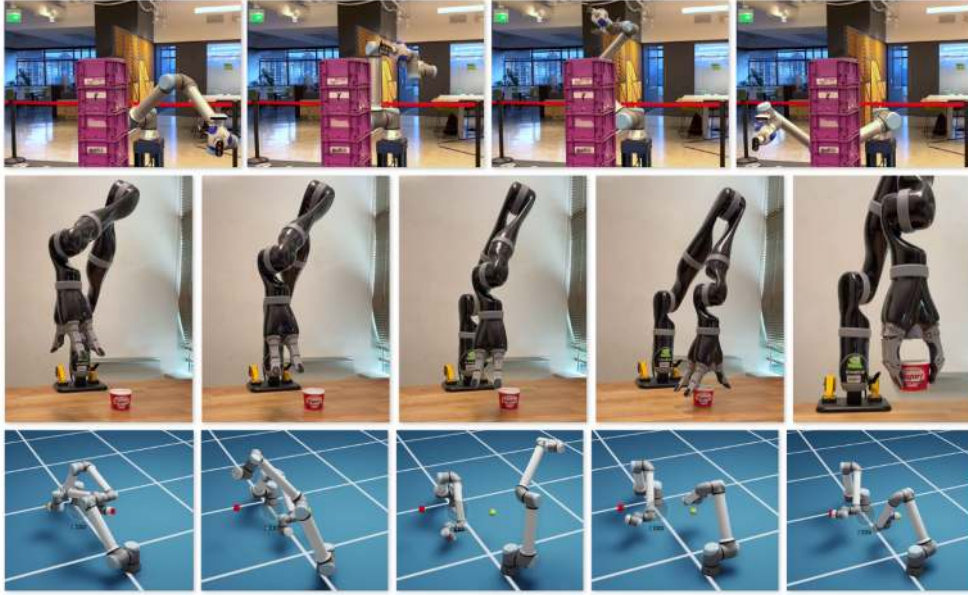


Figure 22.3: Real-time trajectory optimization: GPU-accelerated motion generation executing minimum-jerk splines on hardware, avoiding a dynamic obstacle against a signed distance field, executing a constrained pick-and-lift, and coordinating two arms in a shared workspace. Collision distance is evaluated in continuous time rather than at waypoints, which is what allows the trajectory to be re-optimized while it is being executed. Adapted from (Sundaralingam et al. 2023).

22.4 Kinodynamic Feasibility

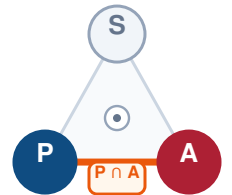
A candidate trajectory is a geometric and timed proposal, not permission to move. Before admission, the planner-side check evaluates joint position and velocity, predicted actuator torque and current, contact limits, obstacle clearance, and the state-matched stopping suffix over the proposed horizon. For an arm, the rigid-body model maps $\mathbf{q}(t)$, $\dot{\mathbf{q}}(t)$, and $\ddot{\mathbf{q}}(t)$ to $\tau(t) = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q})$ and $I_i(t) = \tau_i(t)/k_{\tau,i}$ (Craig 2005; Lynch and Park 2017). The MCU admits the bounded trajectory only if its configured limits and complete stop clearance hold for the verified initial state; it then checks tracking, freshness, and the remaining stopping budget during execution. An optimizer may smooth a path or reduce obstacle proximity, but the admitted trajectory and resident stop—not the optimizer’s objective value—define the motion permission (figure 22.3).

Validating feasibility only at discrete waypoints creates a false sense of compliance. When a planner outputs trajectory points separated by a sampling interval T_s , the physical mechanism moves along an interpolated path between those samples. If the nervous system checks constraints only at the discrete sample timestamps $t_k = kT_s$, high-order trajectory derivatives can exceed physical bounds during the inter-sample interval $[t_k, t_{k+1}]$. Consider an analytical example of a linear joint with a maximum acceleration limit $a_{\max} = 20.0 \text{ m/s}^2$ and a velocity ceiling $v_{\max} = 2.00 \text{ m/s}$. If the planner outputs consecutive velocity samples $\dot{q}_k = 1.95 \text{ m/s}$ and $\dot{q}_{k+1} = 1.95 \text{ m/s}$ separated by $T_s = 20 \text{ ms}$, a cubic spline interpolator that applies a peak acceleration of $a_{\max}$ produces an intra-sample velocity overshoot bounded by $\Delta v \leq \frac{1}{4}a_{\max}T_s$ (integrating a worst-case triangular acceleration reversal across the sampling midpoint). Evaluating this bound yields:

$$\Delta v \leq \frac{1}{4}(20.0 \text{ m/s}^2)(0.020 \text{ s}) = 0.10 \text{ m/s}$$

The true physical velocity peaks at $1.95 \text{ m/s} + 0.10 \text{ m/s} = 2.05 \text{ m/s}$ between the two valid waypoints, exceeding the 2.00 m/s limit and triggering an over-speed clamp. To guarantee continuous-time feasibility without evaluating every infinitesimal point, the admission filter must evaluate an interior contracted bound, $|\dot{q}_k| \leq v_{\max} - \frac{1}{4}a_{\max}T_s$. This acts as a conservative safety margin that artificially shrinks the allowed velocity limit to absorb the maximum possible intra-sample overshoot.

Prerequisite: Actuator torque saturation limits and rotor inertia scaling (N^2J) are derived in section 4.4.



The Plan $\cap$ Act motion interface: synthesizing kinodynamically feasible C^2 splines and verified stopping suffixes.

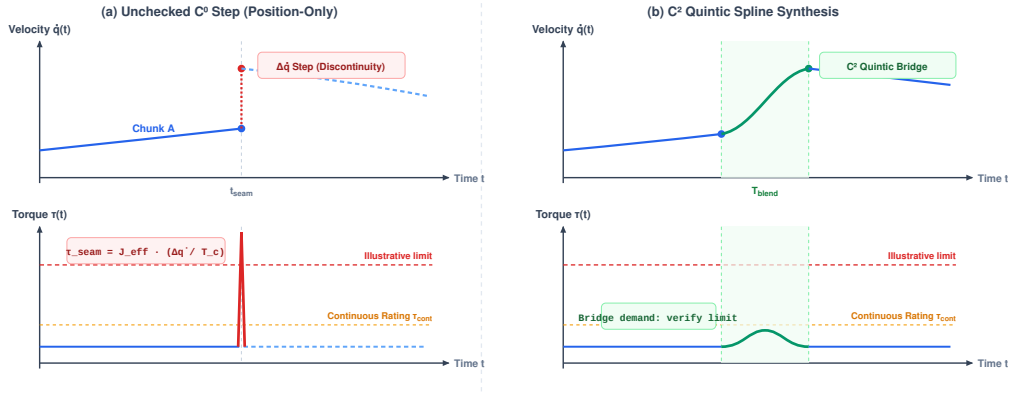


Figure 22.4: Seam demand and checked bridge: A position-matched velocity step creates a large one-tick inertial torque request. A finite C^2 bridge removes the reference velocity step, but its interior jerk, torque, and joint response still require validation.

Concatenating individually feasible chunks can create an infeasible seam. Position-only matching (C^0) can leave a velocity step; matching velocity (C^1) can leave a finite acceleration and torque step. In a rigid-inertia approximation, a commanded velocity change $\Delta\dot{q}$ spread over servo period T_c asks for the torque in equation 22.4. A physical drive saturates or filters that request according to its current limits and compliance; the arithmetic is a demand estimate, not a measured impact load. Figure 22.4 places that request beside its remedy, a finite C^2 bridge that removes the reference velocity step without thereby certifying the interior jerk and torque the bridge itself introduces.

Definition 22.3: The C^2 spline seam

A C^2 *seam* matches position, velocity, and acceleration of two admitted reference segments at their join. A quintic can satisfy six endpoint conditions. The controller must still check interior speed, acceleration, jerk, torque, and clearance and compare the candidate with measured plant state.

For the stipulated joint with $J_L = 1.5 \text{ kg m}^2$, $J_m = 1.2 \times 10^{-4} \text{ kg m}^2$, and $N_g = 100$, the output-side effective inertia is $J_{\text{eff}} = J_L + N_g^2 J_m = 2.7 \text{ kg m}^2$. A 0.15 rad/s reference velocity gap corrected in 2 ms requests 75 rad/s² and 202.5 N m of inertial torque. A separate model with a 120 N m peak ceiling would reject that request; this does not predict fracture or a particular inverter trip.⁶ The product-specific Harmonic Drive example below uses its actual 51 N m rated and 127 N m repeated-peak specifications.

$$\tau_{\text{request}} = J_{\text{eff}} \frac{\Delta\dot{q}}{T_c} \quad (22.4)$$

The transmission response depends on rotor and link inertia, gear ratio, stiffness, damping, and the selected drive protection. For a simple two-inertia model with load inertia J_L , motor inertia reflected to the load $J_{m,r} = N_g^2 J_m$, and coupling stiffness k_θ , the antiresonance and resonance frequencies for this model are $\frac{1}{2\pi} \sqrt{k_\theta/J_L}$ and $\frac{1}{2\pi} \sqrt{k_\theta(1/J_L + 1/J_{m,r})}$, respectively. With stipulated $k_\theta = 2.0 \times 10^4 \text{ N m/rad}$, $J_L = 1.5 \text{ kg m}^2$, and $J_{m,r} = 1.2 \text{ kg m}^2$, these are about 18.4 and 27.6 Hz. Real joints require identified inertia, stiffness, damping, load, and boundary conditions. The comparison in table 22.1 names checks, not universal frequency or jerk limits.⁷

6 | **Drive protection depends on the device:** Current limiting, desaturation detection, DC-bus protection, and braking behavior depend on the driver and its configured thresholds. A reference torque request above a rating is grounds for rejection; it does not prove the specific fault mode or damage outcome.

7 | **Sampling and mechanical modes:** A control period faster than a mechanical mode can help resolve that mode but is not a stability proof. Controller phase margin, sensor delay, damping, filtering, and actuator bandwidth must be validated for the identified joint.

Table 22.1: Architecture-dependent seam checks: These are mechanisms and validation tasks; no frequency, stiffness, or jerk range transfers between products without measured parameters.

Architecture	Relevant state and seam risk	Admission and runtime check
Strain-wave geared joint	Reflected rotor inertia and compliant transmission can amplify a sharp reference change.	Identify load-dependent modes; check bridge extrema, torque and drive limits.
Cycloidal geared joint	Input and output compliance, load, and backlash affect response.	Check measured torque and oscillation margin for the chosen reducer.
Quasi-direct-drive leg	Lower ratio changes reflected inertia, but contact and electrical limits remain.	Check current, DC bus and contact response for the actual drive.
Direct-drive gantry	No gear reduction, but moving motor mass and payload still contribute inertia.	Check force, travel, jerk, and structural modes.

Smoothing this boundary discontinuity requires online C^2 **trajectory spline blending**, which must be treated as the synthesis of an entirely new motion plan rather than as cosmetic filtering. When the nervous system inserts a polynomial bridge $\theta_{\text{blend}}(t)$ over a transition interval T_{blend} to connect chunk A and incoming chunk B , the resulting bridge segment must undergo the exact same dynamic feasibility checks as the primary chunks.⁸

From a physical AI perspective, why must the bridge be a fifth-order (quintic) polynomial? In an actuator drivetrain, motor torque is directly coupled to joint acceleration:

$$\tau = J_{\text{eff}} \ddot{\theta}$$

Matching position alone (C^0) leaves a velocity step; matching velocity (C^1) still leaves an acceleration step. An acceleration step asks for a finite torque step $\Delta\tau = J_{\text{eff}} \Delta\ddot{\theta}$ in this rigid-inertia model; an ideal velocity step instead implies an acceleration and torque impulse. Actual drive current limits and compliance determine the response. Matching acceleration at the reference seam (C^2 continuity) removes the commanded acceleration step; interior torque and plant response still require validation.

To guarantee C^2 continuity across the seam, the bridge must match position, velocity, and acceleration at both endpoints—a total of six boundary conditions:

$$\begin{aligned} \theta(0) &= \theta_0, & \dot{\theta}(0) &= \dot{\theta}_0, & \ddot{\theta}(0) &= \ddot{\theta}_0 \\ \theta(T_{\text{blend}}) &= \theta_1, & \dot{\theta}(T_{\text{blend}}) &= \dot{\theta}_1, & \ddot{\theta}(T_{\text{blend}}) &= \ddot{\theta}_1 \end{aligned}$$

where $(\theta_0, \dot{\theta}_0, \ddot{\theta}_0)$ is the terminal physical state of chunk A at the handoff instant, and $(\theta_1, \dot{\theta}_1, \ddot{\theta}_1)$ is the target state on incoming chunk B at $t = T_{\text{blend}}$. Matching six boundary conditions requires six independent coefficients ($a_0 \dots a_5$), uniquely dictating a fifth-degree polynomial:

$$\theta(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5$$

Solving this linear boundary value problem yields exact closed-form expressions for the six coefficients.⁹ Differentiating three times reveals the dynamic **jerk profile**:

$$\ddot{\ddot{\theta}}(t) = 6a_3 + 24a_4 t + 60a_5 t^2$$

The jerk is a quadratic function of time in general, and finite over a finite blend. Admission must check its extrema and the measured joint response; C^2 endpoints alone do not suppress all resonant energy.

A fixed-degree polynomial can be evaluated with bounded work per axis on a chosen controller. Coefficient synthesis and whole-trajectory extrema checks occur at admission; the measured worst-case execution time, including memory access and monitoring, must fit the configured servo period.

Representation affects what can be proved at admission. Table 22.2 separates continuity supplied by the representation from the dynamic checks still required. A B-spline of degree p has C^{p-r} continuity at an interior knot of multiplicity r ; its control-point convex hull bounds position, not automatically velocity, torque, or clearance. Minimizing an integrated jerk objective requires solving that optimization; choosing a B-spline does not itself solve it.

⁸ | **Trajectory jerk:** A jerk objective can make a particular bridge smoother. A finite jerk bound alone does not exclude energy near a measured resonance; the complete command spectrum and closed-loop response must be checked for the joint and load.

⁹ | **Quintic spline boundary value solutions:** Matching position, velocity, and acceleration at both boundary endpoints yields a non-singular system of six linear algebraic equations. The unique polynomial solution matches acceleration at the endpoints; its jerk is generally quadratic within the blend. For the complete closed-form matrix inversion and coefficient derivations, see Appendix B.

Table 22.2: Trajectory representations and required checks: Smooth parameterization helps form a candidate but does not by itself prove dynamic feasibility. Latency depends on workload and hardware and must be measured.

Representation	Native seam property	Checks before execution
Quintic bridge	Can match $q, \dot{q}, \ddot{q}$ at both ends; jerk is generally quadratic.	Check interior extrema, torque, clearance, and timed admission.
B-spline	Continuity depends on degree and knot multiplicity.	Check derivatives, dynamics, clearance, and optimizer result.
ACT action chunk	Discrete setpoints; zero-order hold jumps at ticks, linear interpolation is C^0 .	Construct and admit a feasible bridge and stop.
Diffusion action chunk	Discrete sampled actions; no intrinsic actuator authority.	Validate timing, state, bridge, limits, and stop.



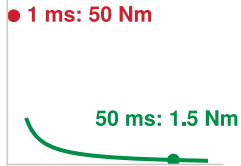
Napkin Math 22.1: Seam acceleration and torque requests

For the registered CSG-25-50-2UH teaching example, reflected rotor inertia is 0.25 kg m^2 . Correcting a 0.2 rad/s reference velocity gap in one 1 ms tick asks for $50 \text{ N}\cdot\text{m}$ of inertial torque. The manufacturer's specification gives $51 \text{ N}\cdot\text{m}$ rated L10 and $127 \text{ N}\cdot\text{m}$ repeated peak; this one-tick **request** is below the rated value, not a predicted fracture. A symmetric C^2 velocity blend over 50 ms has $1 \text{ N}\cdot\text{m}$ mean and $1.5 \text{ N}\cdot\text{m}$ peak inertial torque, a $33.3\times$ peak-request reduction. Total torque still includes load, gravity, friction, and control response; it needs a full peak and thermal check.



Systems Perspective 22.2: The C^2 continuity invariant

C^2 reference continuity avoids a commanded velocity impulse; admission must still check the bridge's interior demands and the measured plant state.



In this specified inertia model, the C^2 blend cuts peak inertial torque request by $33.3\times$; neither request predicts damage by itself.

¹⁰ | **Actuator Thermal Derating:** Continuous peak current commands induce rapid Joule heating ($I^2 R$) in motor windings, driving stator core temperatures past safe thermal thresholds and degrading permanent magnet flux density. To prevent irreversible demagnetization and winding insulation breakdown, motor controllers enforce thermal derating curves that dynamically throttle allowable continuous torque. In trajectory planning, ignoring this thermal governor causes aggressive velocity profiles to trigger mid-trajectory current limits, leading to dynamic tracking failure.

A related physical failure arises from output chatter in learned policy proposals. High-frequency oscillations across successive planning chunks are frequently produced by stochastic sampling, score-matching noise, or autoregressive variance in generative policies. Even when every discrete sample in a chattering trajectory satisfies absolute position and velocity limits, rapid sign reversals in commanded acceleration can request repeated torque reversals. Building on the actuator thermal limits established in section 4.4, an analytical duty cycle where an actuator reverses a torque of $30 \text{ N}\cdot\text{m}$ at 25 Hz would create current transitions and $I^2 R$ heating if the drive tracks them; backlash impact and wear depend on the transmitted load and joint response.¹⁰ The systems engineer budgets this high-frequency variation not as a perceptual roughness metric, but as an actuator thermal rise and transmission fatigue cost. A trajectory that excites structural resonances or exceeds the motor drive's thermal dissipation limits is physically infeasible, regardless of how accurately its waypoints follow the task objective. Even when a candidate chunk satisfies every internal and boundary feasibility check, the nervous system must still account for the reality that the next replacement chunk may fail to arrive before the active plan reaches its terminal sample.



Checkpoint 22.4: Seam continuity and reflected inertia

Before examining seam timelines and fallback deceleration hierarchies, verify your understanding of reflected drivetrain inertia and higher-order spline continuity:

- ☐ **Continuity orders across seams:** Can you explain why C^0 position matching or C^1 velocity matching between consecutive action chunks can request a velocity impulse or torque step, and what determines the actual drivetrain response?

- **Reflected inertia amplification:** Can you describe how a gearbox reduction ratio (N_g) squares the motor rotor inertia, and explain why high-ratio strain wave drives are far more vulnerable to velocity steps than direct-drive or quasi-direct-drive actuators?
- **Spline blending physics:** Can you distinguish between cosmetic trajectory smoothing and online fifth-order (C^2) polynomial spline synthesis across a replanning seam?

22.5 Behavior at the Seam

Because replacement latency varies, the execution tier must know when a candidate can still be admitted and how it will stop if no candidate arrives. The relevant times are distinct: t_{blend} is the preferred seam; t_{commit} is the matched stop-suffix entry; $t_{\text{plan_exp}}$ is the end of nominal reference setpoints; $t_{\text{stop_end}}$ is the end of the protected stopping continuation; and $t_{\text{intent_exp}}$ ends the current target's tracking permission. All are compared in one MCU monotonic clock domain with bounded conversion error. The suffix duration is $T_{\text{stop}} = t_{\text{stop_end}} - t_{\text{commit}}$. A replacement must pass the full admission check before the measured handoff cutoff; a late host progress report has no authority.

The controller has two authority modes. In **TRACK**, it follows an admitted nominal path only while the plan and intent evidence remain valid. If no successor is admitted before commitment, it transfers to MCU-owned **STOP** and follows the validated suffix already latched in protected memory. STOP may finish after $t_{\text{intent_exp}}$ because it no longer pursues the target. If intent expires or a fault invalidates the path *before* the matched commitment state, the MCU uses a separately validated state-matched baseline; playing the suffix designed for a later state would violate its seam conditions. Admission therefore checks stopping clearance from all reachable pre-commit states and the specific suffix entry at commitment.

For a cruise entry with $q(t_{\text{commit}}) = q_c$, speed $v_c > 0$, and zero entry acceleration, a C^2 position-and-velocity-matched stop over duration T uses $u = (t - t_{\text{commit}})/T$:

$$q(u) = q_c + v_c T \left(u - u^3 + \frac{1}{2} u^4 \right), \quad 0 \leq u \leq 1 \quad (22.5)$$

Differentiating equation 22.5 gives speed $v_c(1 - 3u^2 + 2u^3)$ and acceleration $-6v_c u(1 - u)/T$, peak deceleration is $1.5v_c/T$, and stopping distance is $v_c T/2$. Unlike a constant-deceleration switch, its acceleration begins and ends at zero. A nonzero measured entry acceleration requires a general quintic matching the actual $(q, \dot{q}, \ddot{q})$ at entry and zero terminal velocity and acceleration; the enforcer checks its interior speed, acceleration, jerk, torque, and clearance. Neither the polynomial degree nor C^2 matching by itself certifies those limits.

In the illustrative arm case, the blend deadline is 300 ms, matched commitment 360 ms, nominal setpoint end 400 ms, and stop end 660 ms. Thus $T = 300$ ms. At $v_c = 1.2$ m/s, the suffix's peak deceleration is 6.0 m/s² and it travels 180 mm. From the 300 ms clearance check to physical brake onset at 360 ms, continued cruise uses 72 mm; the full 252 mm leaves 48 mm of the stipulated 300 mm obstacle clearance. These are conditional geometric margins, with sensing and model error still to reserve. The stated 360 ms is **physical suffix onset**: the MCU must close replacement admission and pretrigger any communication or actuator delay δ by $360 \text{ ms} - \delta$. If it does not, add 1.2δ of travel and shift the stop end; the 48 mm margin is then smaller. Intent tracking may expire at 400 ms while MCU STOP continues to 660 ms. Figure 22.5 places those instants on a single axis and separates the two ownership regimes, so the point where nominal reference tracking ends can be read against the separately authorized stop that carries the arm to rest.

For a second illustrative AMR, choose blend at 300 ms, commitment at 360 ms, nominal end at 500 ms, and a 450 ms stop ending at 810 ms. From $v_c = 1.5$ m/s, the same suffix has peak deceleration 5.0 m/s² and travels 337.5 mm. The 60 ms pre-stop cruise adds 90 mm; 427.5 mm total leaves 72.5 mm of a stipulated 500 mm corridor. Waiting until the nominal setpoints end at 500 ms would add 300 mm of cruise before the same stop: 637.5 mm total, breaching the corridor by 137.5 mm. A later stop must still match its actual entry state; this counterfactual holds cruise speed constant only to show the lost clearance.

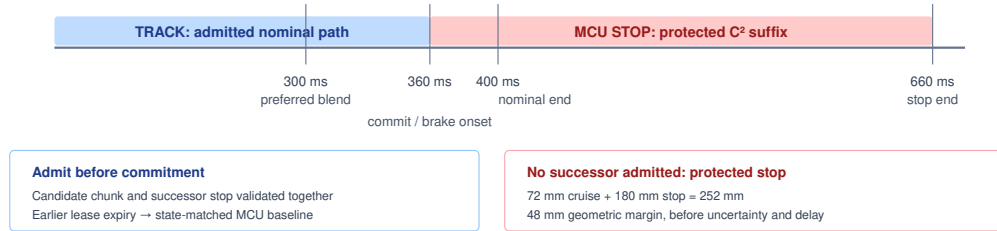
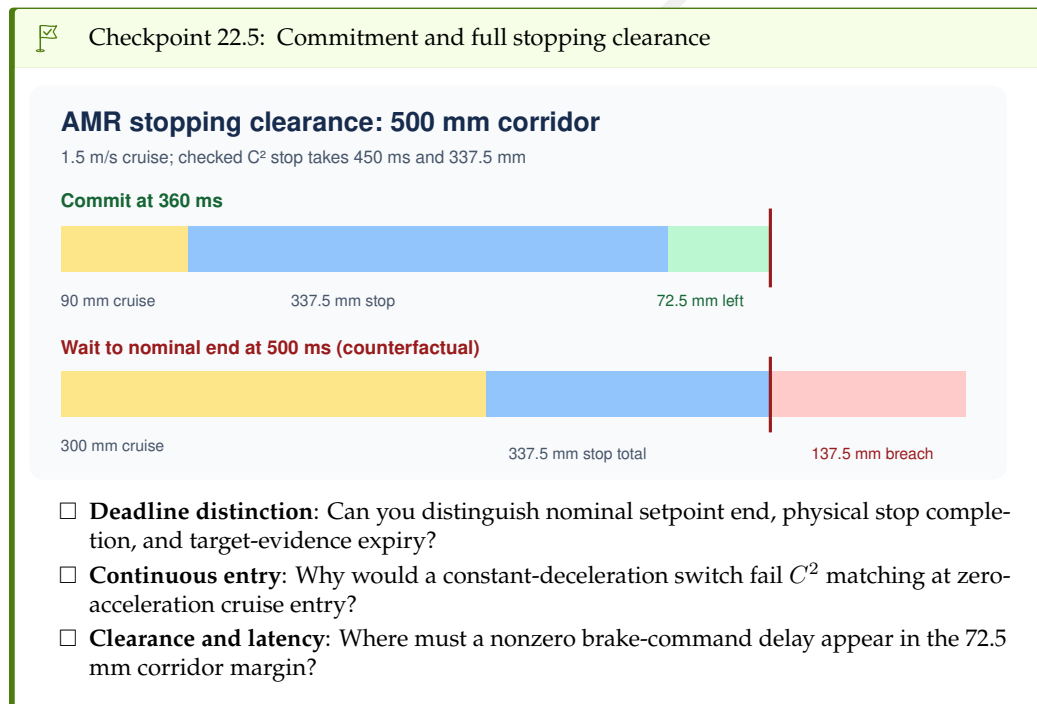


Figure 22.5: Nominal tracking and MCU-owned stop: The worked arm permits a checked replacement before the 360 ms physical stop onset. Nominal references would end at 400 ms, while the separately authorized, C^2 stop reaches rest at 660 ms. The suffix is matched to the commitment state; earlier invalidation needs a state-matched MCU baseline. All deadlines use the MCU monotonic clock, and physical clearance includes the full response and stop.



A replacement received after the preferred blend deadline can still be admitted only if its full bridge and suffix validate before the commitment cutoff. A compressed bridge can raise acceleration and torque; the controller evaluates its polynomial extrema and current encoder discrepancy before latching it. If the check cannot finish, it keeps the current plan and executes STOP. A candidate received after commitment cannot retrospectively restore the spent runway.

The planner proposes the suffix, but the MCU validates and latches its coefficients and immutable assumptions into protected local memory **before** nominal motion begins. At commitment, the MCU selects that resident path without reading host DRAM. A host crash, bus stall, or corrupted shared packet therefore does not erase the stop. If the measured state falls outside the latched suffix's entry envelope, the MCU uses the separately validated baseline appropriate to that state; torque disable or a spring brake is not a universal substitute for a controlled stop, especially with a suspended load.

The admission and fallback decisions in table 22.3 are distinct from an emergency power cut. Each row is conditional on sensed state, available actuation, and the clearance checked for that response.

Table 22.3: Tracking and MCU-owned fallback decisions: Full candidate admission is $O(N)$ for N samples and precedes commitment; the per-tick state check is bounded separately. A host crash does not remove a latched MCU stop.

State	Trigger	MCU action and required evidence
TRACK, ordinary handoff	Replacement passes the complete admission check before commitment.	Latch the new nominal path and its own stop, then transfer at a checked C^2 seam.
TRACK, late candidate	Candidate arrives after preferred blend but before commitment.	Admit only if the compressed bridge, full path, stop, and timing still fit; otherwise keep the current record.
STOP, matched suffix	No successor admitted by commitment.	MCU follows its protected suffix from the validated entry state; target tracking may expire while STOP continues.
STOP, earlier invalidation	Intent expiry, contact, state mismatch, or fault before commitment.	MCU selects a separately validated state-matched baseline with reserved clearance.
Fault response	Controlled stop cannot be assured from measured state.	Use the machine-specific risk response (which may include drive inhibit, brake, or compliance) only under its validated load and timing assumptions.

The following protocol describes the running MCU implementation. Its configured servo period is 1 ms; that period is not a measured response time or a guarantee of physical stopping.

1. **Sample and compare absolute deadlines.** Read encoder state and t_{now} in the MCU monotonic domain. In TRACK, transfer to STOP at the earliest of commitment without a successor, intent expiry, or earlier invalidation. The matched suffix is allowed only when its entry-state envelope holds; earlier transfer uses the resident baseline.
2. **Validate a candidate before admission.** In a bounded background admission budget, verify sequence, active intent lineage, clock conversion, immutable payload and sidecar digest, sample count, every setpoint/interpolated extremum, torque, obstacle clearance, and both fallback paths. Compare the proposed seam against measured $(q, \dot{q}, \ddot{q})$ and recheck entry state at activation. The complete scan is $O(N)$ and must finish before t_{commit} ; a candidate merely marked ready has no authority.
3. **Latch then transfer.** Copy the validated trajectory and stop into protected MCU memory. Only after the copy and final state check may an atomic active-record switch enable TRACK on the successor. If validation times out or the bridge is infeasible, keep the current record and its stop.
4. **Execute locally.** Each tick performs bounded reference interpolation and state/limit checks. At commitment without a successor, the MCU runs the latched C^2 suffix. On earlier invalidation it runs the appropriate state-matched baseline. Neither path reads host DRAM or asks a stalled optimizer for a new command.
5. **Log actual transitions.** Record request, validation, admission, activation, TRACK-to-STOP, and stop-completion times. Drive inhibit or mechanical braking is a separate device-specific response, not an automatic proof of a controlled stop.

The invariant is that an admitted trajectory has a resident and feasible response for its reachable states; it is not that the next inference call will finish.

22.6 How Plans Go Wrong

A plan can become infeasible by execution time. Building on the causal boundary established in section 2.2, the latency between trajectory synthesis in the learned policy and physical actuation by the motor drives (including magnetic stator delays and digital communication bottlenecks such as SerDes serialization latency) creates a blind spot. During this temporal window, the physical state of the machine drifts, the environment shifts, or the geometric interface between consecutive trajectory chunks violates dynamic constraints. Failures in planned motion distribute across several distinct points of first visibility. Establishing where a failure first becomes visible dictates which subsystem possesses the signal and the time budget required to detect and reject it.

The earliest failure occurs when a generated chunk is invalid before reaching the motor drivers. An unconstrained neural policy or an imperfectly sampled trajectory can propose waypoint sequences

that exceed physical hardware limits. A **pre-execution bound checker** evaluates candidate chunks against fixed kinematic and dynamic limits during the admission phase. Rather than evaluating arbitrary safety thresholds, the bound checker calculates the exact margin $M(t) = L_{\text{limit}} - |x(t)|$ across every discrete trajectory point for position q , velocity $\dot{q}$, and acceleration $\ddot{q}$. In an analytical example of a manipulator joint governed by a velocity limit of $\dot{q}_{\text{limit}} = 2.50$ rad/s, a candidate trajectory containing a velocity sample of $|\dot{q}(120 \text{ ms})| = 2.65$ rad/s produces a negative margin of $M(120 \text{ ms}) = -0.15$ rad/s. The pre-execution checker intercepts the chunk before admission, emitting an explicit fault record containing the violated joint channel, the numerical limit, the timestamp of the violation, and the negative margin. Rejecting an invalid plan at the admission boundary incurs zero dynamic disturbance because the machine remains on its previously validated trajectory.

When two consecutive plans are each internally feasible, their concatenation can still produce an infeasible seam. A **boundary checker** compares the candidate with the active plan and measured joint state at t_{handover} , including Δq , $\Delta \dot{q}$, and $\Delta \ddot{q}$. A velocity step implies an ideal acceleration impulse; spreading the requested change over one servo period Δt gives the simplified inertia estimate $\tau_{\text{inertial}} \approx J \Delta \dot{q} / \Delta t$. This is a **request**, not a prediction that the drive supplies it within one tick.

For an illustrative $J = 0.080 \text{ kg m}^2$, $\Delta t = 1.0 \text{ ms}$, and 40.0 N m configured peak limit, allocating the entire limit to this acceleration gives an *idealized ceiling* $\Delta \dot{q} \leq (40.0 \text{ N m})(0.001 \text{ s}) / (0.080 \text{ kg m}^2) = 0.50 \text{ rad/s}$. Available margin is smaller after gravity, payload, friction, other commanded torque, current rise time, and thermal or bus limits are accounted for. The enforcer must use that state-dependent margin and the measured drive response when evaluating a finite bridge. It rejects a direct velocity discontinuity even if its one-tick arithmetic falls below the illustrative ceiling; a bounded C^2 bridge and its interior checks are still required.¹¹

Even with all pre-execution and boundary checks passed, changes in the physical world can render the plan infeasible. A grasped workpiece can slip from its intended SE(3) pose, an unmodeled obstacle can enter the workspace, or structural tracking error can accumulate beyond allowable tolerances. **Runtime monitors** detect changed-world infeasibility by observing four dedicated signals, comprising joint state divergence, tracking error velocity, unexpected contact wrenches, and the invalidation of geometric assumptions recorded in the plan header. The tracking error allowance ϵ_{pos} is derived directly from the geometric clearance $d_{\text{clearance}}$ between the planned trajectory envelope and the nearest obstacle boundary minus the structural deflection and gearbox compliance allowance δ_{deflect} . In an analytical packaging cell with a budgeted clearance of $d_{\text{clearance}} = 0.020 \text{ m}$ and an unmodeled payload deflection and gearbox compliance allowance of $\delta_{\text{deflect}} = 0.005 \text{ m}$, the position error trip threshold evaluates to $\epsilon_{\text{pos}} = d_{\text{clearance}} - \delta_{\text{deflect}} = 0.015 \text{ m}$. If measured Cartesian tool-position error $\|\mathbf{x}_{\text{tool,actual}}(t) - \mathbf{x}_{\text{tool,planned}}(t)\|$ exceeds 0.015 m , or a six-axis force-torque sensor reports an unmodeled 10.0 N contact in this free-space example, the monitor revokes nominal tracking and requests the validated state-matched fallback. Actual deceleration begins after the measured detection, dispatch, and actuator response delays.

A plan can also exhaust its nominal references before the next candidate passes admission. The MCU must not wait for the execution pointer to run off the array or treat a final-position hold as a stop. It compares the absolute current time with the admitted commitment and intent deadlines in its own monotonic domain. At the earliest transfer event, it selects the matched protected suffix if its entry state is valid, or a separately validated state-matched baseline. Complete stopping clearance includes detection, dispatch, actuator response, and motion under the selected profile; an assumed constant deceleration is insufficient for the C^2 worked suffix.

War Story 22.1: A fleet stop in fog (2023)

Waymo reported a heavy-fog interruption in San Francisco's Balboa Terrace neighborhood in 2023; its public account says subsequent software changes were tested against the conditions. This was a Waymo event, not a Cruise incident. The public account does not establish a LiDAR-planner deadlock, a particular stopping-suffix design, or a postincident suffix redesign.

¹¹ | **Drive and resonance response:** A sharp command can excite a joint's *identified* flexible modes, but their frequencies and damping depend on the transmission and load. Current limiting, desaturation detection, DC-bus response, and shutdown depend on the selected driver and its configuration; see the TI gate-driver protection description. Neither a requested torque nor this simplified model establishes a particular trip, bearing damage, or gear failure.

Systems lesson: For a machine already in motion, an unexpected pause in upstream planning is a useful *hypothetical* stress test: what local plan remains executable, when does tracking authority end, and can the independent fallback stop within the measured clearance? The answers require the system’s own logs and validation, not an inferred mechanism from the fog report.

Every detector in the planning pipeline relies on an explicit contract between the trajectory generator and the execution runtime. If a plan is delivered as a raw list of coordinates without metadata defining its valid lifespan, its geometric clearance bounds, its boundary derivatives, and its intended fallback behavior, the nervous system cannot determine whether an ongoing motion remains safe or has drifted into infeasibility. Making plan execution verifiable requires encapsulating waypoints within a structured record that binds kinematic trajectories to their temporal validity windows and monitoring thresholds.



Checkpoint 22.6: Trajectory failure modes and runtime detection

Before examining the wire-level schema and execution buffers of the trajectory contract, evaluate your understanding of real-time monitoring and failure detection:

- ☐ **Point-of-visibility classification:** Can you categorize trajectory failures into pre-execution invalidity, seam boundary steps, runtime world divergence, and horizon exhaustion?
- ☐ **Boundary torque limits:** Can you derive the maximum admissible velocity step $\Delta \dot{q}_{\text{allow}}$ across a single control cycle from actuator peak torque and reflected joint inertia?
- ☐ **Cross-plan oscillation mechanics:** Can you explain why single-plan validators fail to detect policy dithering between competing local minima, and how sequence monitors track thermal derating budgets?

22.7 The Trajectory Contract

The **trajectory record** is the immutable data structure that the planning system hands to the execution runtime at every replanning cycle. While the general schema, serialization format, and global provenance rules for system logs are established in chapter 8, trajectory management introduces domain-specific fields that enforce physical continuity and temporal deadlines. Within its kinematic definition, the record stores the geometric motion profile either as discrete kinematic setpoint arrays or as continuous polynomial spline coefficients. It pairs this representation with the sample cadence (such as $\Delta t = 10$ ms for a 100 Hz setpoint stream), the total planned horizon T_{plan} (such as 400 ms), the target coordinate frame identifier $\mathcal{F}$, the engineering SI units, and the exact initial-state timestamp t_0 matching the state measurement from which the path was computed. Recording the coordinate frame and units directly in the handoff prevents frame mismatches at the tracking layer, while the explicit cadence and timestamp allow the low-level joint controller to interpolate setpoints and evaluate kinematic derivatives without relying on ambient system clocks.

Physical feasibility depends on the scene and load used to validate the plan. The immutable sidecar therefore identifies obstacle-map epochs, payload and friction assumptions, limits, validator version, measured margin, and the source observation. A digest in the fixed header binds both setpoints and sidecar. The MCU compares live state with those assumptions and revokes TRACK when they no longer hold; a recorded margin is evidence for a stated scene, not a permanent safety flag. Solver seed and model hash belong in the audit sidecar for reproducibility where the toolchain supports it, not in the servo’s per-tick path.

For the six-axis worked implementation, table 22.4 gives a **344-byte header**. All time fields are absolute MCU-monotonic nanoseconds after a validated bounded conversion. The active intent record, referenced by digest, supplies $t_{\text{intent_exp}}$ separately. A variable-length setpoint payload and immutable provenance/assumption sidecar are outside the header. The MCU verifies their digest, validates the complete bounded candidate in staging memory, then latches the admitted payload and suffix in protected memory before TRACK. The digest binds content and lineage but is not a keyed authentication tag; transport authentication, if required, is a separate mechanism.

Downstream: Trajectory feasibility proposals are separately checked against conditional control barrier functions in section 24.7.

Table 22.4: Worked six-axis trajectory header: Offsets are contiguous and total 344 bytes; the separately bounded setpoint payload and assumptions/provenance sidecar are digested, validated, and latched before activation.

Bytes	Header field	Shape	Admission purpose
0x00–07	sequence_id	u64	Reject replay.
0x08–0F	generation	u64	Bound age after clock conversion.
0x10–2F	intent_digest	u8[32]	Match active intent lineage.
0x30–37	cadence,N,DOF,version	u32,u16,u8,u8	Bound payload size and decoder.
0x38–3F	t_start	u64	First nominal sample.
0x40–47	t_commit	u64	Latest matched-suffix entry.
0x48–4F	t_plan_exp	u64	Nominal setpoint end.
0x50–57	t_stop_end	u64	Protected stop completion.
0x58–5F	frame_id	u64	Check coordinate frame.
0x60–7F	payload_sidecar_digest	u8[32]	Bind immutable evidence and samples.
0x80–C7	entry ($q, \dot{q}, \ddot{q}$)	f32[18]	Six-axis seam state.
0xC8–157	suffix coefficients	f32[36]	Six-axis quintic stop.

The deadline relation is $t_{\text{commit}} = t_{\text{stop_end}} - T_{\text{stop}}$, with $t_{\text{start}} \leq t_{\text{commit}} < t_{\text{plan_exp}} < t_{\text{stop_end}}$ in the worked case. The MCU permits TRACK only while the active intent and nominal plan remain valid. The protected suffix may continue after either expires. Earlier invalidation requires the separately admitted baseline; a header timestamp cannot make a mismatched suffix safe. A log records request, generation, validation, admission, activation, transfer, and stop-completion times in a common converted domain so actual response delay can be measured.

🔍 Systems Perspective 22.3: Centroidal momentum and multi-contact QP

Whole-body trajectory planning on a humanoid robot pushes real-time numerical optimization to its computational limits:

- **Centroidal Momentum and Underactuated Floating Base:** Unlike fixed manipulators where every degree of freedom has an active motor anchoring it to the floor, a humanoid possesses an underactuated 6-DoF floating base. Acceleration of the robot's center of mass (CoM) is governed strictly by external contact wrenches:

$$\dot{\mathbf{h}}_{\text{lin}} = m\ddot{\mathbf{c}} = \sum_k \mathbf{f}_k + m\mathbf{g}, \quad \dot{\mathbf{h}}_{\text{ang}} = \sum_k (\mathbf{p}_k - \mathbf{c}) \times \mathbf{f}_k$$

A planned trajectory cannot arbitrarily accelerate limbs without verifying that the required ground reaction forces $\mathbf{f}_k$ remain strictly inside friction cones ($\|\mathbf{f}_{k,\perp}\| \leq \mu f_{k,\parallel}$) and unilateral contact constraints ($f_{k,\perp} \geq 0$). For the complete mathematical formulation of floating-base dynamics and hierarchical WBC-QP, see section B.7.

- **Whole-Body Control Quadratic Programming (WBC-QP):** One implementation solves a hierarchical Quadratic Program (QP) at a configured 500 Hz to track tasks while respecting underactuated balance constraints. The QP optimizes joint accelerations $\ddot{\mathbf{q}}$ and contact forces subject to rigid-body kinematics ($\mathbf{J}\ddot{\mathbf{q}} + \dot{\mathbf{J}}\dot{\mathbf{q}} = \ddot{\mathbf{x}}$), actuator torque limits ($|\tau| \leq \tau_{\text{max}}$), and Center of Pressure (CoP) boundaries. If that implementation misses its measured computation budget, the controller must switch to a separately admitted fallback for its current contact state.
- **Commitment Horizons and Dynamic Deceleration Seams:** If high-level trajectory re-planning stalls, a wheeled AMR needs an admitted stop path and brake capability within remaining clearance. A walking humanoid cannot stop instantaneously without tipping forward over its toes. The planned trajectory chunk must include a **validated stopping response** matched to the current contact state and balance region. A learned capture-step proposal needs independent checks; a double-support outcome is not automatic.

Checkpoint 22.7: Trajectory contract schema and timing ledgers

Before exploring the fallacies and pitfalls of robotic motion planning, verify your understanding of real-time trajectory serialization and hardware provenance:

- ☐ **Fixed-size memory invariance:** Can you explain why the trajectory handoff structure uses bounded staging and protected execution storage instead of allocating in the servo interrupt?
- ☐ **Hardware timestamp chains:** Can you trace how logging monotonic hardware timestamps from request to expiration enables automated detection of bus contention and queue starvation?
- ☐ **Deterministic replay hashing:** Can you describe why a logged seed, model hash, and solver configuration help reproduce a stochastic trajectory proposal?
- ☐ **Humanoid momentum contracts:** Can you explain why whole-body trajectory contracts on an underactuated bipedal robot must constrain ground reaction forces to friction cones while budgeting capture-step stopping suffixes?

22.8 Fallacies and Pitfalls

A trajectory remains feasible only while its physical assumptions hold at chunk boundaries and during fallback. Admission and execution checks must address those transitions as well as the nominal path.

Fallacy: *Static admission checks protect a machine against obstacle displacements during active execution.*

An arm begins an admitted trajectory with 50.0 mm clearance from an assembly jig. During execution, another mechanism shifts the jig 70.0 mm into the tool's path. The admission check was correct for the earlier geometry but no longer supports continued motion. Execution monitoring must detect the changed clearance and initiate deceleration while sufficient stopping room remains. A fixed check at the start cannot establish that condition throughout the trajectory. The design must bound how quickly workspace changes become visible via sensor bus latency and reserve the stopping margin needed to respond before the updated geometry becomes impossible to avoid safely, under the application's validated separation and stopping assumptions.

Pitfall: *Stitching individually feasible action chunks without checking derivative continuity across seams.*

A manipulator receives consecutive 200.0 ms chunks that repeatedly reverse a joint's direction. Each chunk passes its own acceleration and jerk checks, yet the sequence produces oscillatory motion that can excite the drivetrain's resonant frequencies due to gearbox compliance, causing violent shaking. Local limits do not establish that one chunk connects correctly to the next or that a repeated pattern remains acceptable. Admission must compare derivatives across each seam and retain enough recent motion history to detect recurring reversals that could induce excessive Joule heating (I^2R) in the stator. These checks address the composed trajectory, which the motor and transmission actually execute. Treating chunks as independent requests hides behavior that appears only when the requests are joined.

Pitfall: *Delaying fallback transition past the hard stopping horizon in the hope that delayed compute finishes.*

A planner supplies a 300.0 ms chunk whose stopping suffix needs $T_{\text{stop}} = 120.0$ ms. The controller must therefore commit to fallback at $t_{\text{commit}} = 180.0$ ms unless it has admitted a valid replacement. At that point, the coprocessor reports inference nearly complete, but it has not delivered an executable plan over the SerDes link. Waiting consumes the physical runway reserved for deceleration and can leave insufficient stopping clearance. The controller must follow the admitted stopping suffix at the commitment point. Progress reports cannot extend a mechanical stopping budget; only a newly validated trajectory can justify continued execution.

Fallacy: *Feasible boundary conditions at spline endpoints guarantee feasibility throughout the interior blend.*

A gantry blends perpendicular motion segments while carrying an 8.0 kg fixture. Its admission routine checks the endpoint conditions and accepts the blend without evaluating the interior motion.

Centrifugal forces inside the blend can demand a dynamic torque that still exceeds the motor's available output, even when the checked endpoints are within bounds. The endpoint test has omitted the part of the trajectory where the constraint becomes active. Admission must establish velocity, acceleration, and SE(3) wrench feasibility over the whole blend using the applicable trajectory model. Checking boundary values is sufficient only when that model also establishes the required interior bounds.

Pitfall: *Relying solely on feedforward dynamic models when physical contact or payload mass can change abruptly.*

An arm lifts a 6.0 kg casting using feedforward torque computed for the loaded body. The casting slips, but the drives continue applying the same lifting torque to the now unloaded system, causing the arm to violently accelerate upward. The admitted trajectory has not changed; the physical dynamics that made its commands appropriate have. This resulting unmodeled acceleration and tracking error expose the mismatch only if the controller observes the physical response. Feedback monitoring must detect that divergence and constrain the commands within the available margin to prevent thermal derating or mechanical limit strikes. A feedforward model improves nominal tracking, but it cannot replace runtime checks for abrupt changes in payload or contact.

Fallacy: *A trajectory can omit an achievable terminal stopping suffix if replacement plans are expected to arrive in time.*

A gantry travels at $v = 2.0$ m/s with deceleration limited to $a_{\text{dec}} = 4.0$ m/s² due to actuator torque limits, requiring a minimum braking time of $T_{\text{min}} = v/a_{\text{dec}} = 500.0$ ms to stop. A proposed suffix allows only $T_{\text{stop}} = 200.0$ ms, so it cannot stop the machine from that boundary speed without demanding an impossible amount of torque, which would exceed the stipulated deceleration capability. The drive response depends on its configuration. Expecting the next plan to arrive does not make this fallback executable. The admission gate must check $T_{\text{stop}} \geq v_{\text{boundary}}/a_{\text{dec}}$ and reject a packet that fails. Every accepted trajectory needs a physically achievable termination path for the case in which no replacement becomes available.

22.9 Summary

A planner turns spatial intent into a timed motion proposal. The execution tier admits that proposal only after checking current state, the complete nominal path, the seam bridge, and a stopping response. C^2 continuity removes reference position, velocity, and acceleration jumps at a seam; it does not by itself bound interior jerk, torque, vibration, or clearance.

Replacement timing has two ends. Nominal setpoints end at $t_{\text{plan_exp}}$, while the MCU-owned stop can finish later at $t_{\text{stop_end}}$. The decision to switch from TRACK to STOP occurs at commitment without a successor, intent expiry, or earlier invalidation. A matched suffix handles its validated commitment state; earlier transfer requires a state-matched resident baseline. Both require full response-delay and braking clearance. The worked arm's $72 + 180 = 252$ mm stop leaves 48 mm of geometric margin before uncertainty and unreserved delay.

An inference-latency quantile is a scheduling risk target, not a safety certificate. The packet carries explicit time fields, frame, entry state, payload lineage, and stop coefficients. The bounded full-candidate admission scan is $O(N)$; after latching, the MCU performs bounded per-tick checks without depending on a host reply. Future-state projection helps construct the next candidate, while encoder measurements decide whether that candidate fits the real seam.



Key Takeaways: Timed motion and independent stopping

- Admit a full trajectory and protected stop before nominal tracking begins.
- Compare absolute deadlines in one MCU-monotonic domain and reserve response plus stopping clearance.
- Check polynomial extrema and actual drive response; representation names alone prove neither feasibility nor resonance immunity.
- Retain a state-matched MCU baseline for early invalidation and a measured, logged TRACK-to-STOP transition.



What's Next: From trajectory optimization to safety enforcement

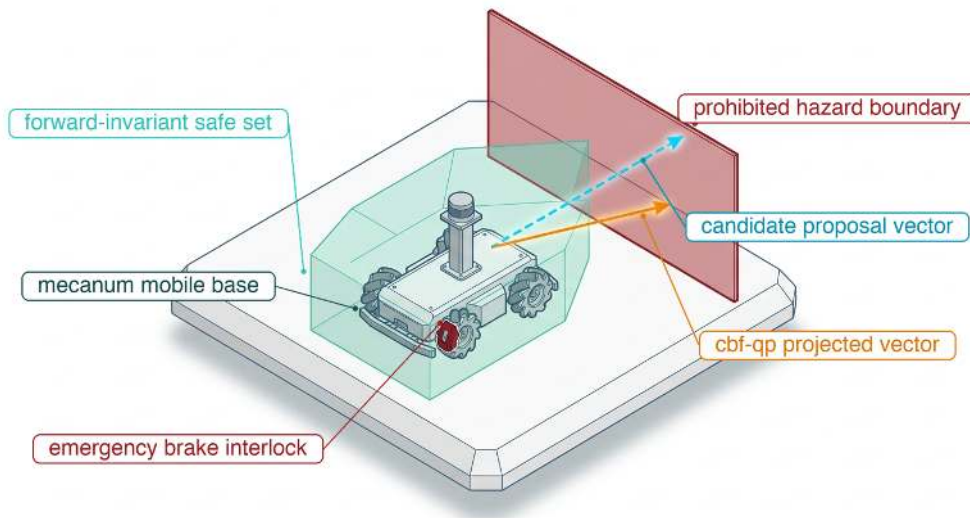
Although the planning subsystem constructs kinodynamically feasible motion contracts, an unprivileged predictive optimization pipeline can never be permitted to certify its own physical execution. In chapter 24, we construct the deterministic safety filter: checking proposed motion against state and model-dependent safe-set conditions. A feasible, timely safety-filter solution and a validated fallback are still required before unverified proposals reach motor drives.

DRAFT

Safety Enforcement

DRAFT

Safety Enforcement



- 24.1 The Deterministic Gatekeeper
- 24.2 What Control Hands Over
- 24.3 The Authority to Refuse
- 24.4 Where the Enforcer Sits
- 24.5 Stopping Envelopes
- 24.6 When the Enforcer Runs Out of Authority
- 24.7 Safe Sets as Conditional Permission
- 24.8 Minimal Intervention
- 24.9 The Fallback Ladder
- 24.10 How Enforcement Goes Wrong
- 24.11 Enforcer Telemetry
- 24.12 Fallacies and Pitfalls
- 24.13 Summary

Purpose

How does an unprivileged software proposal become physical torque, and what lets the gatekeeper refuse it in time under the stated plant and timing limits?

A policy can propose commands exceeding the machine's clearance or actuator capacity. Once that command becomes motor current, diagnosing the model's mistake cannot undo the resulting motion. An independent enforcement path must evaluate proposals before they reach the actuators and retain the authority to reject them within a bounded time. Timely refusal is insufficient. The replacement command must also produce the intended physical response. A brake that has reached its current limit cannot deliver additional force because the safety calculation assumes it can. Enforcement therefore depends on measured state and on the tracking error of the feedback loops beneath it. When those conditions no longer support the requested motion, the machine needs a fallback whose limits are explicit. The nervous system combines permission and execution, linking the brain's proposed behavior to the body's tracking and stopping capabilities.

Brain
1–50 Hz
foundation models, SLAM, planning

above: propose; below: command

Nervous System
1000 Hz
TSN, intent leases, lock-free IPC

Body
continuous
real-time MCU, safety filters, CBFs

Boundary
continuous
sensors, actuators, kinematics

Physical World
continuous
contact mechanics, friction, mass



Learning Objectives

- Explain the relationship between tracking accuracy, command permission, and independent enforcement at the actuator boundary
- Diagnose how update rate, feedback delay, and actuator saturation limit corrective control
- Calculate stopping margins accounting for state uncertainty, tracking error, response delay, and available braking
- Evaluate when measured safety constraints require accepting, modifying, or refusing a proposed actuator command
- Select fallback responses when sensing, computation, or actuator limits invalidate normal enforcement

24.1 The Deterministic Gatekeeper

HIGH-CAPACITY learned neural policies—vision-language-action foundation models, deep reinforcement learning agents, and trajectory diffusion networks—excel at open-ended manipulation and adaptive navigation in unmodeled environments. Even when an upstream deliberative brain synthesizes smooth splines to guide the robot, candidate trajectories remain unprivileged proposals across the causal boundary. Neural policies are fundamentally unverified, non-deterministic optimizers. They can hallucinate infeasible paths, mistake specular reflections for open corridors, or emit joint commands that exceed mechanical torque limits. In standard software engineering, an algorithmic bug or invalid pointer can be trapped by a kernel exception handler or contained inside an application sandbox. In physical AI, software isolation alone is useless once motion begins. The instant a digital command triggers pulse-width modulation (PWM) switching in a motor inverter, electrical current energizes stator windings, generating magnetic flux that accelerates physical mass. At that point, software diagnosis is powerless: momentum cannot be trapped by an exception handler, and kinetic energy cannot be recalled once released into the physical plant.

The central dilemma of physical AI systems engineering is reconciling two irreconcilable realities: *How do we harness the task intelligence of stochastic neural policies while ensuring the physical machine never breaches geometric clearance, thermal limits, or human safety invariants?*

Solving this dilemma requires stepping across the **proposal-permission boundary** from Layer 3 (The Cognitive Brain) down to Layer 2 (The Deterministic Nervous System), as highlighted in figure 24.1. The **reflex enforcer** is the physical gatekeeper of the machine: an edge-deployed, hard real-time component positioned directly in front of the actuator drive electronics. It evaluates candidate proposals, verifies invariant state bounds, and holds the exclusive, irrevocable hardware authority to permit current flow or command autonomous fallback braking.



Definition 24.1: Reflex enforcer

Reflex enforcer is an edge-deployed deterministic gatekeeper positioned between unverified deliberative software proposals and physical actuator drive electronics, executing at a rate chosen from plant dynamics to check validated state and torque constraints, project feasible proposals, and command a state-matched fallback when the tracking contract fails.

1. **Significance:** Physical momentum and actuator saturation make late software intervention catastrophic. An independent permission path at the drive interface can reject uninspected commands; its actual braking response must be tested.
2. **Distinction:** Unlike high-level trajectory planners that optimize multi-step paths over hundreds of milliseconds, a reflex enforcer performs instantaneous, single-step feasibility checks and projections under strict sub-millisecond execution deadlines.
3. **Common pitfall:** Treating the enforcer as an emergency exception handler invoked only during crashes. In practice, the enforcer is a continuous, per-tick gatekeeper; if its state

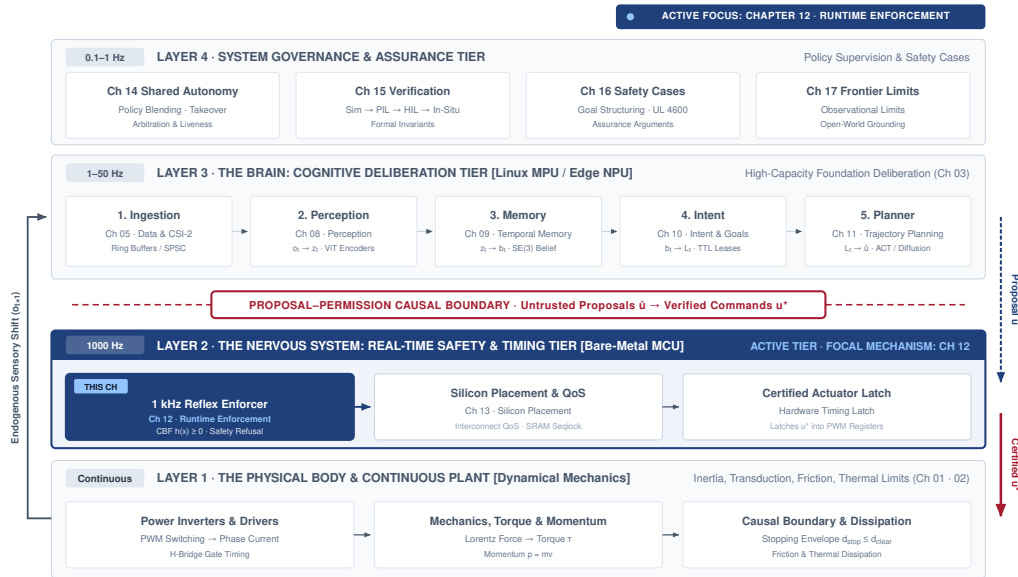


Figure 24.1: Reflex enforcer architecture focus: In the physical AI systems architecture, the deterministic nervous system sits between the unverified deliberative brain and the dynamical body, serving as the sole authority permitted to translate advisory commands into physical force or refuse them outright. Operating on a bounded millisecond control period, the enforcer projects candidate actions onto forward-invariant safe sets and manages a multi-tier fallback hierarchy to retain a conditional, tested fallback path.

estimate or execution deadline leaves the validated envelope, it must transfer to a feasible fallback before the reserved stopping margin is consumed.

This division creates an asymmetric authority relationship between the brain and the nervous system, formalized in systems engineering as the **Simplex architecture pattern**¹ (Sha 2001). Upstream neural policies emit candidate control proposals across time-bounded intent leases (section 20.7). But only the nervous system, executing on bare-metal microcontroller hardware directly wired to motor drives and joint encoders, possesses the physical authority to turn a proposal into mechanical force. In the illustrated design, the enforcer evaluates incoming setpoints each millisecond control cycle against validated constraints using **Control Barrier Functions (CBFs)**² (Ames et al. 2019). When a proposal is safe, it passes to the motor drives unaltered. When a proposal marginally threatens a boundary, an active-set Quadratic Program (CBF-QP) minimally modifies the command, minimizing torque deviation while enforcing the derived constraint under the stated model and timing assumptions.³ When a proposal is structurally invalid or the cognitive brain falls silent, the gatekeeper severs the command channel and triggers an autonomous fallback deceleration sequence.

However, an enforcer cannot guarantee safety beyond the physical tracking fidelity of the control loop beneath it. Every mathematical barrier certificate assumes that the closed-loop tracking controller follows commanded references within a bounded error. The enforcer consumes a single foundational contract from the controller: an explicit bound on the **worst-case tracking error** (ϵ_{track}) between where the machine was commanded to be and where its physical mass actually resides. Every geometric clearance and dynamic velocity ceiling evaluated by the enforcer must be strictly inset by this error bound plus transport lag displacement.

Constructing this deterministic protective shield requires establishing the exact contractual boundaries between continuous tracking controllers, optimization-based safety filters, and bare-metal fallback actuators. The system cannot enforce safety in the abstract; it must ground barrier certificates in the verified tracking error and transport lag of the physical plant, structure asymmetric authority to reject ungrounded neural proposals, and position dual-stage filters across application processors

Prerequisite: The dual-brain proposal-permission privilege split was established in section 2.9.

¹ **Simplex architecture and safety switching:** Formulated by Sha (2001), the Simplex architecture provides runtime safety assurance by pairing an unverified advanced controller with a certified baseline safety controller, arbitrated by a deterministic safety manager. The hardware multiplexer switch intercepts output channels, instantly diverting authority to the baseline controller if state invariants degrade. Bypassing unverified software in hardware preserves the validated baseline controller's authority after the primary policy hangs, provided the baseline remains powered, observable, and physically feasible.

² | **Control barrier functions and forward invariance:** Control Barrier Functions define a continuously differentiable certificate $h(\mathbf{x}) \geq 0$ over an admissible safe set $\mathcal{C}$, enforcing the directional clearance derivative constraint $\dot{h}(\mathbf{x}, \mathbf{u}) \geq -\gamma(h(\mathbf{x}))$. Enforcing this inequality renders $\mathcal{C}$ forward-invariant: states initialized in a viable set can remain there while the barrier is feasible and the stated model and timing assumptions hold. When approaching the boundary, the constraint forces opposing actuation inputs that arrest momentum before physical contact can occur.

³ | **Active-set quadratic programming for safety shields:** Active-set Quadratic Programming solvers operating on bare-metal microcontrollers compute $\min_{\mathbf{u}} \frac{1}{2} \|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2$ subject to linear barrier constraints $\mathbf{a}(\mathbf{x})^\top \mathbf{u} \leq b(\mathbf{x})$. By restricting linear constraints to active boundary contacts and eliminating dynamic memory allocation, a fixed iteration limit and tested platform can bound termination; no universal 150 μs limit follows from the solver type. If numerical ill-conditioning prevents convergence before the loop deadline, the solver aborts gracefully to an autonomous deceleration fallback.

⁴ | **Closed-loop tracking stiffness and disturbance bounds:** Sustained force and effective closed-loop stiffness determine static deflection. Transient tracking also depends on inertia, damping, command shape, and saturation. Disturbances outside the validated envelope invalidate the claimed tube; they need not produce exponential divergence.

and real-time silicon. When finite actuator torque and dynamic traps contract the admissible state space, minimal-intervention quadratic programs project candidate actions onto forward-invariant safe sets or cascade down an escalation ladder to bring kinetic energy safely to zero.

The first quantity the enforcer needs is the tracking error that remains after a permitted command reaches the physical plant.

24.2 What Control Hands Over

The tracking contract isolates the enforcer from the plant's internal mechanics. A feedback controller makes many internal guarantees, but only one critical metric passes upward across the systems interface. Within a validated envelope of reference acceleration, load, temperature, and disturbance, the controller may provide a transient bound $\|x(t) - r(t)\| \leq \epsilon_{\text{track}}$ over the complete maneuver. The enforcer does not inspect transfer functions, root loci, or proportional-integral-derivative (PID) gains. To the software sitting above the loop, the controller is a black box that turns reference setpoints into physical motion, guaranteed to stay within the error bound ϵ_{track} of the commanded path.

This bound is a physical distance, not a mathematical abstraction. A physical plant is governed by rotor inertia, transmission compliance, and actuator torque limits. The steady-state position error—the persistent physical gap between the commanded destination and the actual resting place—balances unmodeled external disturbances against the restoring control effort of the closed-loop stiffness. A steady-state stiffness calculation alone does not bound transient overshoot or settling; the required bound also needs validated dynamics or full-maneuver testing.⁴

Consider an analytical example of a linear positioning stage operating in an environment where friction and payload variations introduce disturbance forces. If the maximum theoretical disturbance translates to a steady-state error of 5.0 mm, and the reference command executes a step of 100.0 mm, the carriage accelerates and eventually settles. The settled position alone does not bound its transient excursion. At steady state, the physical carriage does not sit precisely at 100.0 mm. It rests anywhere within the interval [95.0 mm, 105.0 mm].

The steady-state calculation does not establish a transient bound. For the following clearance example, separately assume full-maneuver validation establishes $\epsilon_{\text{track}} = 5.0$ mm within defined operating conditions; this numerical equality with the settled error is an additional scenario assumption. The controller can provide that bound only under the validated reference, load, and thermal conditions. First, the reference trajectory cannot demand accelerations beyond what the actuators can deliver. Second, external disturbances must remain strictly below d_{max} . Third, the actuator must operate below the continuous thermal derating limits established in section 4.5 (governed by Joule heating) and voltage bus rails. If an upstream policy commands a trajectory that demands an acceleration of 15.0 m/s² from an actuator whose peak torque caps acceleration at 10.0 m/s², the feedback loop saturates. The integrator winds up, phase lag accumulates, and the physical mechanism falls behind the reference by an amount that grows with every millisecond of saturation. If an unexpected external load exceeds the validated disturbance envelope, the controller can leave the assumed ± 5.0 mm transient corridor. Outside its rated envelope, the controller provides no bound at all.

Because physical reality can diverge from the reference command by up to ϵ_{track} , every geometric and dynamic margin evaluated by an enforcer must be inset by that exact distance. If a robot arm must maintain a physical clearance of at least 20.0 mm from a fragile fixture, and the underlying tracking loop has separately established a full-maneuver bound $\epsilon_{\text{track}} = 5.0$ mm for this operating envelope, the enforcer cannot evaluate safety against the nominal geometry. It must inset the admissible state space by adding a virtual buffer of 5.0 mm, checking that the commanded reference maintains a clearance of at least 25.0 mm from the obstacle surface. Compounding tracking error, transport delay displacement, and state estimation residual yields the composite inset margin invariant:

$$\delta_{\text{margin}} \geq \epsilon_{\text{track}} + v \cdot \tau_{\text{delay}} + \epsilon_{\text{est}} \quad (24.1)$$

As formalized in equation 24.1, if worn bearings, thermal expansion, or altered motor parameters loosen the true tracking error from 5.0 mm to 12.0 mm without the enforcer updating its internal model, the true physical clearance to the fixture shrinks from 20.0 mm to 13.0 mm. The enforcer believes it is operating with a comfortable safety margin while the physical hardware moves danger-

ously close to collision. A degraded tracking bound silently erodes safety margins across the entire software stack.

A tracking bound never checked at runtime is an assumption, not a contract. Because environmental conditions fluctuate and mechanical components degrade, the nervous system must publish ϵ_{track} alongside the explicit operational conditions that validate it, while monitoring the instantaneous tracking error $e(t) = \|x(t) - r(t)\|$ on every tick of the control cycle. If sensor feedback reveals that $e(t)$ approaches or exceeds ϵ_{track} , the tracking contract has broken. When a disturbance force spikes to 40.0 N or an actuator enters thermal throttling, the nervous system cannot treat the resulting divergence as a normal transient. It must detect the contract violation immediately, invalidating the spatial assumptions used by downstream filters before unmodeled deflection causes physical damage.

Prerequisite: Spatial clearance inseting from sensor covariance and transport delay originates in section 16.5.

Systems Perspective 24.1: The inset margin invariant

Every spatial and dynamic safety boundary evaluated by an enforcer must be inset by the underlying tracking loop's verified worst-case tracking error plus the transport lag displacement according to equation 24.1: $\delta_{\text{margin}} \geq \epsilon_{\text{track}} + v \cdot \tau_{\text{delay}} + \epsilon_{\text{est}}$. An enforcer that evaluates barrier certificates against nominal geometric boundaries without insetting will permit trajectories that physically collide with obstacles whenever tracking error reaches its certified bound.

The mathematical synthesis of tracking controllers, observer design, and disturbance rejection proofs are treated in depth by the control literature (Slotine and Li 1991; Åström and Murray 2021). For the systems engineer building physical AI architectures, the responsibility begins where control guarantees end. The tracking controller provides the mechanism to translate permitted references into physical currents, but it has no semantic awareness of obstacles, task goals, or system-level hazards. It will execute a command into a solid wall with the same closed-loop fidelity that it brings to free-space motion. Before any proposed action is handed to this tracking machinery, the system must determine whether the proposal is permitted to become force at all.

24.3 The Authority to Refuse

The permission path is a safety gate that admits actuation only when specific physical criteria are satisfied. It grants no authority to an incoming learned proposal until that proposal passes the configured state, input, freshness, and stopping-clearance checks. In conventional automation, control pipelines operate on trust, executing commands under the assumption that the upstream software component produces valid setpoints. In physical AI, that assumption fails by design. A learned policy operating in high-dimensional perception space has no mathematical guarantees of stability, no bounded output variance, and no internal representation of actuator current limits or mechanical stress. The learned model cannot be held accountable for physical destruction; thus, the nervous system must not treat its outputs as authoritative. Within this architecture, upstream models produce advisory proposals, but only the nervous system possesses the physical authority to turn a proposal into force.

This division creates an asymmetric authority model between the brain and the nervous system, formalized in systems engineering as the **Simplex architecture pattern** (Sha 2001). The Simplex pattern resolves the tension between high performance and safety by decomposing the control system into four structural components:

1. **The advanced controller:** An unverified, high-capacity learned model (such as a Vision-Language-Action network, a residual reinforcement learning policy (Johannink et al. 2019), or a diffusion policy executing on a Linux MPU) that optimizes for complex, open-ended task performance.
2. **The baseline safety controller:** A certified, high-assurance deterministic controller (such as a Lyapunov-based posture regulator or precomputed quintic deceleration spline executing on bare-metal MCU firmware) that acts as the trusted low-level stabilization baseline of the physical plant, providing tested stability and stopping behavior within a stated envelope.

3. **The safety manager:** Deterministic arbitration logic that continuously evaluates the state of the machine against forward-invariant barrier sets, tracking error bounds, and execution deadlines.
4. **The Simplex switch:** An atomic hardware or real-time firmware multiplexer that routes actuation signals to the motor bridge. The switch admits a checked proposal when conditions hold and transfers to a resident baseline within its qualified response delay when those conditions fail.

Under this Simplex contract, the upstream policy never commands the plant directly. Instead, communication across the causal boundary operates through time-bounded **intent leases**—a mechanism analogous to an industrial dead-man watchdog, where permission to move automatically expires unless continuously renewed by a verified, healthy brain. The Linux MPU transmits candidate control vectors as structured packets over shared SRAM mailboxes through a configured interprocessor channel, such as RPMsg.⁵ Each proposal payload contains a monotonic sequence identifier, a microsecond timestamp, an explicit lease duration (τ_{lease}), candidate setpoints, a transmission integrity checksum (CRC32), and, across unprivileged operating system boundaries, a cryptographic Message Authentication Code (MAC) (such as AUTOSAR SecOC with AES-128 CMAC) to prevent unauthorized command spoofing. In this implementation, the MCU checks command age on each control cycle using a bounded conversion into its clock domain. If the proposal is fresh, the state remains in the validated viable region, and tracking error remains bounded, the Simplex switch permits the proposal (or its minimally projected variant) to drive the actuators. If the lease expires without renewal, tracking diverges, or the proposal threatens physical boundaries, the Simplex switch revokes permission and engages the Baseline Safety Controller unilaterally.⁶ The upstream policy is not consulted, does not negotiate, and possesses no hardware mechanism to override the switch.

When a proposal conflicts with physical constraints, the enforcer chooses between two interventions: command filtering and outright veto. Command filtering applies when the proposal is structurally sound but marginally exceeds an admissible kinematic or dynamic boundary. If an analytical model of a six-axis manipulator shows that a proposed end-effector velocity vector demands a joint speed 4 percent above a rated motor limit of 3.0 rad/s, the nervous system can mathematically shave off the excess by projecting the reference onto the boundary of the safe set using Control Barrier Functions or quadratic programming (CBF-QP).

At the physical layer, clearance must cover momentum as well as position. For a torque-controlled arm, raw clearance d has relative degree two: $\dot{d}$ contains approach speed but no immediate torque input. The velocity-dependent stopping barrier derived in section 24.7 has relative degree one while the arm approaches the boundary. Its derivative, not $\dot{d}$ alone, yields an affine torque constraint for the real-time solver.⁷

$$\mathbf{a}(\mathbf{x})^\top \mathbf{u} \leq b(\mathbf{x})$$

Forward invariance requires that this clearance rate constraint holds continuously. Physically, this means that whenever momentum carries the machine toward a boundary, the actuators must exert sufficient opposing torque to decelerate before clearance vanishes. When a proposal marginally threatens a boundary, the CBF-QP safety filter minimally adjusts the commanded torques to satisfy $\mathbf{a}(\mathbf{x})^\top \mathbf{u} \leq b(\mathbf{x})$. This Euclidean projection minimizes change in the torque vector; it may change motion direction and requires separate path and tracking validation. Projection is permissible only when the deviation is local, continuous, and cleanly separated from catastrophic hazards. When a proposal exhibits structural invalidity—such as a discontinuous jump in commanded position across consecutive frames, a request for acceleration that exceeds maximum physical torque capabilities, or a trajectory vector aimed directly into an immovable obstacle—geometric projection becomes dangerous. A discontinuous or wildly infeasible proposal indicates that the upstream perception or inference pipeline has entered an unmodeled failure mode. Attempting to project an ungrounded hallucination onto the nearest safe boundary executes a synthetic motion that has no relationship to task intent. In such cases, the enforcer must veto the proposal entirely, tripping the Simplex switch and dropping authority to a local fallback.

⁵ | **Remote Processor Messaging:** Linux RPMsg provides virtio-based messaging between processors. Buffer ownership, copies, scheduling delay, and worst-case handover time depend on the platform; OpenAMP offers a separate no-copy API. The lease requires a measured, bounded delivery path.

⁶ | **Hardware break circuits and PWM trip zones:** A configured external timer break input such as TIMx_BKIN can force PWM outputs to a specified state without waiting for an interrupt. EPWM_TZFRM is a software force mechanism, not the external trip input. Output state and response time depend on the timer, gate drive, wiring, and qualification tests; the trip still needs a physically suitable stop path.

⁷ | **Control barrier functions and Lie derivatives:** In nonlinear control theory, the unforced drift and actuator control authority are formalized using Lie derivatives ($L_f h(\mathbf{x})$ and $L_g h(\mathbf{x})$). When the safety boundary has relative degree one, the control input appears directly in the first time derivative of the barrier function, yielding an instantaneous affine constraint on commanded torque. If the relative degree is two, the enforcer must evaluate higher-order derivatives to prevent actuator saturation from rendering boundary violations inevitable; see Appendix B.

24.3.1 Constrained reinforcement learning versus runtime projection shields

Machine learning practitioners question why an architecture needs an independent hard real-time quadratic programming solver on a microcontroller to enforce physical boundaries: *Why not train the neural policy to be safe using Constrained Reinforcement Learning (Constrained RL) or Lagrangian penalty formulations?*

In Constrained RL, the optimization problem maximizes task reward subject to expected cost bounds across rollout episodes:

$$\max_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \gamma^t R(s_t, a_t) \right] \quad \text{subject to} \quad \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \gamma^t C(s_t, a_t) \right] \leq d$$

Here π_{θ} is the learned policy, τ an episode of state-action pairs (s_t, a_t) , R the task reward, C a violation cost, γ a discount factor, and d the permitted expected episode cost. Lagrangian updates can optimize this average training objective; they do not enforce each physical tick.

While mathematically elegant in simulation, training-time constraint penalties fail on physical hardware for four fundamental reasons:

1. **Expectation versus Pointwise Guarantees:** Constrained RL enforces constraints *in expectation* over the stationary policy distribution ($\mathbb{E}[C] \leq d$). An expectation allows the robot to violently collide with an obstacle in 1 percent of trials, provided it behaves with extreme conservatism in the remaining 99 percent to keep the statistical average below d . An expected cost limit does not bound the consequence of an individual high-momentum collision. The permission path therefore checks each candidate at its sampled actuation boundary; any forward-invariance claim additionally depends on a viable state, accurate estimates, feasible braking, and bounded sensing-to-actuation delay.
2. **The Reward Shaping Dilemma and the Freezing Robot Trap:** Balancing task reward against constraint penalties is notoriously brittle. If the Lagrange multiplier λ is initialized too low, exploratory policy gradients prioritize task completion, causing destructive physical breaches. Increasing λ may reduce sampled violations but can also suppress useful motion or make optimization unstable near a constraint boundary. Training penalties alone provide no per-tick containment guarantee.
3. **Destructive Exploratory Rollouts:** In reinforcement learning, the policy learns constraint boundaries by experiencing violations and backpropagating the resulting penalties. In a simulated environment, a catastrophic crash is resolved by invoking an instantaneous software reset. On physical hardware, exploratory collisions destroy transmissions, strip gearboxes, and trigger hardware E-stops, rendering trial-and-error constraint discovery physically infeasible.
4. **Uncertainty Under Test-Time Distribution Shift:** Even if a policy achieves zero violations across billions of simulated training steps, real-world deployment introduces unmodeled friction variations, sensor lens contamination, and payload fluctuations. Because deep neural policies lack certified Lipschitz continuity outside their training support, a minor perceptual shift can produce an out-of-distribution action that violates physical invariants without warning.

By contrast, the physical AI architecture decouples task optimization from runtime safety enforcement. The learned policy (whether trained via unconstrained RL, imitation learning, or diffusion) proposes task-oriented nominal controls $\mathbf{u}_{\text{nom}}$. Downstream, the deterministic CBF-QP safety shield executing on the real-time microcontroller solves the minimal convex projection:

$$\mathbf{u}^* = \arg \min_{\mathbf{u} \in \mathcal{U}} \frac{1}{2} \|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2 \quad \text{subject to} \quad \mathbf{a}(\mathbf{x})^\top \mathbf{u} \leq b(\mathbf{x})$$

When the policy's proposal is safe, the shield is completely transparent ($\mathbf{u}^* = \mathbf{u}_{\text{nom}}$), leaving nominal task execution unhindered. When a candidate violates the current input half-space, the shield projects it onto that constraint; a conditional forward-invariance claim still requires a viable state, verified model, state estimate, feasible torque, and deadline.

To maintain authority when the upstream host stalls, consider a constructed timing contract. A policy proposes at 20 Hz, every 50 ms; allowing 10 ms of renewal jitter gives a maximum proposal

interval of 60 ms, below a 75 ms proposal lease. A separate 100 Hz host heartbeat has a validated maximum interval of 15 ms and a 30 ms silence timeout. The 1 kHz MCU loop feeds its own 5 ms self-watchdog only after a successful cycle; a separate external trip covers MCU failure. At each timeout the MCU revokes the proposal and executes a resident, state-matched fallback, provided the reserved clearance and available braking still suffice. For the AMR parameters below, a lease expiry followed by a 50 ms brake response requires about 478 mm of clearance at 1.2 m/s: $1.2(0.075 + 0.050) + 1.2^2/(2(2.5)) + 0.040 = 0.478$ m, before any additional estimate bound. This lease-expiry case assumes the separate heartbeat remains live while fresh admissible proposals stop arriving; a total host stall triggers the 30 ms heartbeat timeout earlier.

Severing upstream authority requires a defined physical response at the lowest hardware interface. The **zero-command floor** is the configured response when active software control disappears; its safety depends on the plant state and available restraint. In software systems, halting an execution pipeline often corresponds to returning a null pointer or clearing a status register. In physical systems, writing zero to a motor drive register produces different mechanics depending on actuator electrical and mechanical topology. Setting current commands to zero in a low-friction, direct-drive motor removes motor torque, allowing a gravity-loaded arm to descend unless a separate restraint carries the load. Conversely, commanding zero velocity in a stiff position controller can request a large transient; actual transmitted load and damage depend on the drive, compliance, and mechanism. The system designer must select the zero-command response to match the plant. In geared transmissions with high backdrivability, dynamic braking shorts the motor windings through low-resistance shunts or MOSFET bridges. This converts the back-electromotive force (the voltage a spinning motor naturally generates by acting as a generator) into electrical current that resists motion, generating braking torque that falls with speed. This damping can dissipate energy as heat, but it must be validated for the load and thermal endpoint and cannot hold at standstill by itself. When static holding or power-off safety is required, a separate spring-applied friction brake can hold the load only within its verified engagement speed and rating.

Refusal is therefore a steady-state decision, not only an emergency exception. On each illustrated millisecond control tick, the enforcer checks proposal validity, separate communication heartbeats, the applicable kinetic constraint, and tracking error against validated limits. If a condition fails, it revokes the proposal and selects the resident response appropriate to the current state and available clearance. That permission path must remain able to act when the upstream proposal source fails.

24.4 Where the Enforcer Sits

The enforcer sits at a specific physical and logical boundary in the signal pipeline, determined by the dynamics' timescale and the latencies of the feeding channels. Two natural boundaries exist in every machine that translates learned proposals into motion. One boundary sits upstream of the feedback control loop, where the system executes reference checking. A reference checker validates planned trajectory waypoints, coordinate targets, or velocity vectors emitted by a high-level planner before those targets enter the tracking loop. Because it operates in kinematic space over a forward horizon, a kinematic reference checker operating at 50 Hz can evaluate a 300 ms trajectory segment against static geometric envelopes, workspace boundaries, and self-collision models. The strength of reference checking is wide spatial foresight, but its fundamental limitation is complete blindness to the actual physical state of the plant. If a planned path maintains a clearance of 5.0 mm from a steel bracket, the reference checker marks the command valid. Yet if the downstream tracking controller exhibits a transient position overshoot of 10.0 mm when accelerating a heavy load, the physical arm strikes the bracket despite passing every upstream validation step.

The second boundary sits downstream of the feedback controller, near the power electronics, where the system executes command checking. A command checker inspects the raw physical quantities produced by the tracking loop, evaluating motor phase currents, hydraulic valve spool displacements, pulse-width modulation duty cycles, and joint torque references immediately before writing them to hardware registers. Operating at a plant-derived inner-loop frequency, illustrated here by 1000 Hz, the command checker enforces hard instantaneous limits. It prevents electrical overcurrent, structural over-torque, and thermal runaway by clipping or zeroing signals that exceed the physical ratings of the plant. What the command checker gains in physical certainty, it surrenders in temporal foresight. A scalar command clamp lacks future-state information; a kinetic barrier at

Downstream: Heterogeneous silicon partitioning of the reflex enforcer onto isolated microcontroller enclaves is mapped in section 26.2.

the same tick can use a validated stopping model. An instantaneous torque command of $18 \text{ N} \cdot \text{m}$, which is well within a thermal continuous rating of $45 \text{ N} \cdot \text{m}$, appears entirely safe to a post-control monitor. However, if that torque accelerates a $10 \text{ kg} \cdot \text{m}^2$ linkage toward a rigid stop at 1.5 rad/s , the rotational kinetic energy of the mechanism will exceed the maximum deceleration capacity of the actuator over the remaining travel distance. The command checker detects no fault until the linkage is inches from impact, at which point no allowable reverse torque can prevent a collision.

This split reveals the trade-off between latency and visibility. Reference checkers possess multi-step geometric visibility but suffer from high latency and blind spots regarding closed-loop tracking dynamics. In this example, operating at planning rates between 10 Hz and 50 Hz , a reference filter cannot respond to unmodeled plant resonance, sensor noise amplification, or sudden control instability occurring at frequencies above its update rate. If a feedback controller develops a 60 Hz limit cycle oscillation, the reference setpoints remain stationary while the physical linkages shake. Conversely, command checkers can run each millisecond tick with a separately measured input-to-output delay, but their one-step horizon creates dynamic traps where every instantaneous torque is legal until an irrecoverable state is reached.

The placement of the enforcer also determines how it integrates into the computing fabric. In a sequential in-line architecture, every signal passes synchronously through the enforcement logic before reaching the next stage. A reference checker filters the trajectory before the controller reads it, or a command clamp modifies the torque register before the motor drive samples it. Sequential execution guarantees zero uninspected commands, but it inserts the worst-case execution time of the enforcer directly into the critical latency path. For a digital 1000 Hz control loop with a total deadline of $1000 \mu\text{s}$, allocating a P_{99} latency of $250 \mu\text{s}$ to an in-line safety check shrinks the budget available for feedback computation. If the enforcer overruns its execution budget, the entire cycle faults. In a parallel asynchronous architecture, the enforcer executes on a separate co-processor or an isolated processor core, monitoring the state of the machine and the command stream concurrently without intercepting every cycle. When the parallel monitor detects an invariant violation, it asserts a hardware line to open a safety contactor (physically severing high-voltage power to the actuators) or switch the motor drive to dynamic braking (shorting the motor phases to rapidly dissipate kinetic energy). Parallel execution preserves control loop timing, but the asynchronous detection and communication introduce an enforcement lag bounded by the monitor cycle time and bus jitter, with empirical EtherCAT measurements showing 3.0 ms to 8.0 ms of communication transport lag. To compensate for this reaction delay, the system designer must expand the physical safety margins, slowing the machine or increasing its minimum clearance buffers.

Because neither single location provides both dynamic foresight and instantaneous protection, physical AI architectures resolve the tension through a **dual-stage enforcement pattern**. This pattern maps cleanly onto the heterogeneous dual-brain System-on-Chip (SoC) architecture (section 6.1):

- **Upstream Stage (Pre-Control Corridor Filter on Application MPU):** Operating at $20\text{--}50 \text{ Hz}$ on an unprivileged application microprocessor core, this stage accepts multi-step trajectory chunks emitted by the learned policy. It verifies that the planned path maintains a dynamic stopping corridor under worst-case braking parameters and validates geometric clearance against static workspace envelopes before packing the waypoints into an RPMSG intent lease.
- **Downstream Stage (Instantaneous CBF-QP Safety Shield on Bare-Metal MCU):** Operating inside a strict 1000 Hz ($1000 \mu\text{s}$) timer interrupt loop on an isolated hard real-time safety microcontroller, this stage receives the proposed setpoint from shared SRAM, verifies tracking error bounds, solves an active-set Control Barrier Function (CBF-QP) quadratic program in pre-allocated static SRAM, and clamps raw current and torque commands directly before writing to PWM timer registers. Because a velocity-dependent stopping barrier can yield a linear torque inequality $\mathbf{a}(\mathbf{x})^\top \mathbf{u} \leq b(\mathbf{x})$ under the stated approach dynamics, this safety check reduces to a single-step convex projection whose execution time must be bounded on the selected hardware.

The pre-control stage checks that a feasible stop remains from admitted states. The drive-side stage checks current observations and commands, but neither stage can recover a state that has already exhausted braking clearance.

Placing the enforcer correctly is useless if its inputs share failure modes with the component it checks. An enforcer may only evaluate quantities that it can establish independently of the

proposal path. If a pre-control corridor filter calculates collision distances using a depth map generated by the same neural perception model that planned the motion, an out-of-distribution visual artifact will deceive both the planner and the enforcer simultaneously. Independent enforcement requires independent sensing and isolated computation. On heterogeneous dual-brain SoCs and industrial robotics controllers, the bare-metal safety MCU reads secondary optical encoders and hardware limit switches wired directly to its own GPIO and hardware timer capture registers over dedicated buses, completely isolated from the Linux virtual memory space and operating system scheduler. A collision enforcer must read dedicated safety lidar or ultrasonic proximity sensors whose signal conditioning relies on deterministic threshold logic rather than learned inference. Where no physically independent sensor exists, such as when estimating the friction coefficient of a wet roadway or the variable mass of an unmodeled payload, the systems engineer cannot construct an independent physical guarantee. In those operating regimes, the safety assertion is conditional, its validity bounded by the statistical uncertainty of the learned estimator rather than guaranteed by the nervous system. Every invariant that the enforcer checks, whether evaluated upstream across a geometric corridor or downstream at the motor terminals, ultimately assumes that the physical plant retains the control authority to arrest its own motion before crossing a forbidden boundary.



Systems Perspective 24.2: The Simplex independence rule

A safety monitor must never evaluate a state estimate produced by the component it is designed to constrain, nor share an un-preemptible communication bus, memory matrix, or clock tree with unverified processes. Where independent sensing is physically impossible, the safety claim is conditional on model uncertainty rather than guaranteed by hardware.



Checkpoint 24.1: Zero-jitter enforcement boundaries and hardware trip paths

Before analyzing dynamic stopping equations, verify your architectural understanding of where safety enforcement must execute relative to the operating system and motor hardware:

- ☐ **Co-location vulnerability:** Can you explain why running safety filtering inside the same Linux user-space process as a vision-language policy exposes the machine to unbudgeted kernel scheduling jitter (> 20 ms)?
- ☐ **Hardware break circuits:** How can a configured break input such as TIMx_BKIN put PWM outputs in the device-specific safe state without waiting for a software interrupt, and what drive and physical-stop delays still need measurement?
- ☐ **Asymmetric privilege boundaries:** Can you contrast the fault domain of an unprivileged Linux Application Processor (MPU) with a bare-metal microcontroller (MCU) running deterministic 1 kHz safety loops?
- ☐ **Shared-bus starvation:** Can you describe how high-throughput direct memory access (DMA) bursts from camera image sensors can stall real-time SPI torque command packets across shared system crossbars?

Keep the independent permission and trip paths within verified execution and response deadlines; a preemptible application scheduler cannot own the final actuator permission decision.

24.5 Stopping Envelopes

The stopping envelope is the set of states from which a validated braking response can arrest motion before a boundary. Reserving that response costs achievable speed: velocity increases reaction travel linearly and braking travel quadratically. The machine must remain inside this viable envelope before a fault or unsafe proposal occurs.

The stopping envelope geometry derives from Newtonian kinematics and the nervous system's signal latency. Consider a body moving along a single axis at instantaneous velocity v . When an enforcer detects a boundary violation or receives an unsafe proposal, a finite transport delay

τ_{delay} elapses before actuators can generate opposing force. This total delay compounds perception exposure time, transport latency over the internal communications bus (e.g., EtherCAT bus jitter, which introduces slight variations in message delivery time), enforcer evaluation cycle time, and the physical current rise time (L/R time constant) in the stator windings (section 4.1). During τ_{delay} , the body continues to travel under its pre-existing velocity, traversing a reaction distance of $v \cdot \tau_{\text{delay}}$. Once braking torque engages, assuming a validated lower bound on available deceleration a_{max} within the stated operating envelope, the time required to arrest the remaining velocity is v/a_{max} , during which the body traverses a braking distance of $v^2/(2a_{\text{max}})$.

A pure kinematic calculation assumes that the underlying feedback controller tracks the reference trajectory with zero error. By physical AI first principles, a real physical plant diverges from the reference trajectory due to external disturbances, unmodeled joint friction, and the sensor quantization limits (section 4.2). This instantaneous tracking error is bounded by ϵ_{track} (section 24.2). If an obstacle sits at a coordinate x_{obs} , placing the nominal stopping target exactly at x_{obs} causes a collision whenever the tracking error reaches its positive bound. The enforcer must therefore inset the allowable reference boundary away from the obstacle by ϵ_{track} . Combining reaction displacement during transport lag, active braking deceleration, and the low-level tracking error bound yields the fundamental stopping envelope invariant:⁸

$$d_{\text{stop}}(v) = v\tau_{\text{delay}} + \frac{v^2}{2a_{\text{max}}} + \epsilon_{\text{track}} + \epsilon_{\text{est}} \quad (24.2)$$

Under those delay, estimate, and braking assumptions, any trajectory proposal that places the machine at a distance less than $d_{\text{stop}}(v)$ in equation 24.2 from a physical constraint along the direction of motion is unsafe and physically inadmissible. The stopping envelope is fundamentally anisotropic; an obstacle behind the instantaneous velocity vector requires clearance only for the positional tracking error ϵ_{track} .

Consider an analytical example of an autonomous mobile robot (AMR) navigating an industrial facility with an onboard safety LiDAR sensor offering a clear detection horizon $d_{\text{sense}} = 2.50$ m. The nervous system exhibits a total command transport and electrical rise delay $\tau_{\text{delay}} = 50$ ms = 0.050 s (accounting for sensor ingress, fieldbus packet exchange, safety filter solve, and motor inverter rise time). Within the declared load and surface envelope, the chassis braking system provides a validated minimum available deceleration $a_{\text{max}} = 2.50$ m/s², and the low-level tracking controller provides a validated transient error bound $\epsilon_{\text{track}} = 40$ mm = 0.040 m. This idealized worked calculation takes the additional position-estimate bound $\epsilon_{\text{est}} = 0$; a nonzero bound subtracts directly from available clearance. The deceleration is a lower bound on achievable braking in this scenario, not a motor nameplate maximum.

At a nominal cruising speed $v_1 = 1.20$ m/s, the reaction drift distance during transport lag is $d_{\text{react}} = v_1 \tau_{\text{delay}} = (1.20 \text{ m/s})(0.050 \text{ s}) = 0.060 \text{ m} = 60 \text{ mm}$. The active braking distance is $d_{\text{brake}} = v_1^2/(2a_{\text{max}}) = (1.20 \text{ m/s})^2/(2 \times 2.50 \text{ m/s}^2) = 0.288 \text{ m} = 288 \text{ mm}$. Summing these with the tracking margin yields a total required stopping clearance of $d_{\text{stop}}(1.20) = 0.060 \text{ m} + 0.288 \text{ m} + 0.040 \text{ m} = 0.388 \text{ m} = 388 \text{ mm}$.

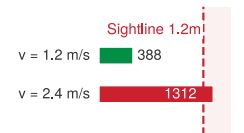
If an optimization policy doubles vehicle velocity to $v_2 = 2.40$ m/s to increase warehouse throughput, the reaction drift expands linearly to $d_{\text{react}} = (2.40)(0.050) = 0.120 \text{ m} = 120 \text{ mm}$, while the active braking distance quadruples to $d_{\text{brake}} = (2.40)^2/(2 \times 2.50) = 1.152 \text{ m} = 1152 \text{ mm}$. Under the same zero-additional-estimate assumption, the total required stopping clearance surges to $d_{\text{stop}}(2.40) = 0.120 \text{ m} + 1.152 \text{ m} + 0.040 \text{ m} = 1.312 \text{ m} = 1312 \text{ mm}$. While velocity increased by 100 percent ($2.0\times$), the required stopping clearance expanded by 238 percent ($3.38\times$), demonstrating how the quadratic scaling of kinetic braking energy ($E_k \propto v^2$) dominates spatial clearance demands.

Furthermore, if the robot approaches a blind warehouse rack aisle where structural shelving limits the unobstructed sightline to $d_{\text{corner}} = 1.20$ m, the stopping envelope constraint from equation 24.2 requires:

$$\frac{1}{2a_{\text{max}}}v^2 + \tau_{\text{delay}}v + (\epsilon_{\text{track}} - d_{\text{corner}}) \leq 0$$

Substituting the physical parameters yields $0.20v^2 + 0.050v - 1.160 \leq 0$, which solves to a positive quadratic root of $v_{\text{rooeline}} \approx 2.29$ m/s. Regardless of neural inference throughput, the kinematic

⁸ | **Kinematic stopping distance and latency components:** The total stopping distance in this model compounds four terms: response-delay travel, kinetic braking distance under a validated minimum available deceleration, transient tracking error, and position-estimate error. Omitting computational transport delay or tracking error results in premature encroachment on physical obstacles before peak braking can engage. See Appendix B for formal kinematic derivations.



Doubling velocity quadruples braking distance, burning completely through the blind-corner clearance envelope.



Figure 24.2: Industrial safety laser scanner: A scanner can provide an early warning field and a protective field. The warning field requests earlier deceleration; the protective field's safety output feeds a controller that selects a risk-assessed stop. Protective-field length must include scanner response, controller and drive delay, braking travel, and uncertainty. A moving AMR does not become safe merely because motor torque is removed.

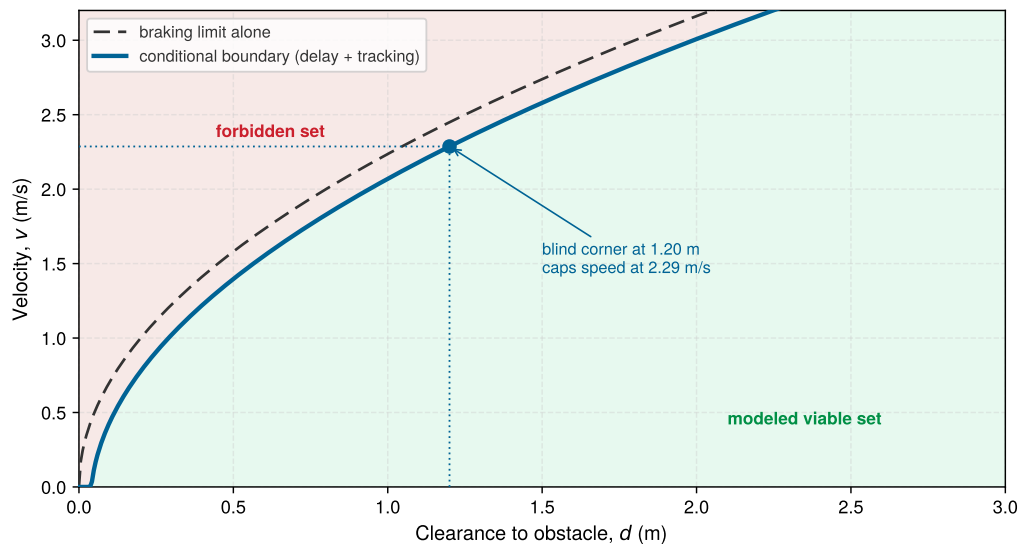


Figure 24.3: Conditional stopping envelope: A parabolic boundary separates states with modeled braking clearance from states that have already exhausted it. The inset includes bounded response delay, transient tracking error, and any nonzero position-estimate bound. The curves are valid only for the stated lower bound on braking deceleration and operating envelope.

roofline imposes a hard velocity ceiling of $v \leq 2.29$ m/s. Any policy requesting higher speed around the blind corner violates the forward-invariant stopping contract and must be vetoed by the enforcer. For an AMR, a risk-assessed protective-field design may use a safety scanner under the applicable vehicle and machinery standards, including ISO 3691-4 and ISO 13849, such as the time-of-flight safety laser scanner shown in figure 24.2.

The physical necessity of maintaining a clear stopping envelope levies an inevitable tax on operational throughput. In a state-space phase portrait plotting velocity against distance to the nearest

obstacle (figure 24.3), the stopping equation defines a parabolic boundary separating safe operation from inescapable collision. The margin between the physical obstacle boundary and the certified envelope represents the spatial budget consumed by perception delay, transport lag, and tracking error before peak braking engages. In any deployment where sensing range is finite or environmental visibility is restricted by geometry, the maximum detection horizon d_{sense} dictates a hard ceiling on legal velocity. Just as the Patterson Roofline model (Williams et al. 2009) in computer architecture relates peak compute performance to memory bandwidth limits, the stopping envelope dictates a strict kinematic roofline: if an onboard optical sensor can confirm an unobstructed corridor only up to 2.0 m ahead around a blind warehouse corner, the system cannot permit a speed exceeding 3.01 m/s (or 2.68 m/s for a 1.61 m sightline) under the parameters of the earlier example, regardless of how quickly the underlying machine learning model can compute actions. To run faster, the engineer must either increase braking torque a_{max} , compress signal latency τ_{delay} , or tighten the tracking error ϵ_{track} through higher loop gains. When hardware limits prevent further reduction in those terms, the only remaining parameter is velocity.



Checkpoint 24.2: Stopping envelopes and transport lag

Consider how the stopping envelope dictates the boundaries of permissible autonomous motion:

- ☐ **Quadratic envelope scaling:** Can you explain why a doubling of operating velocity requires more than triple the stopping clearance, and why higher-order planning policies cannot bypass this kinematic floor?
- ☐ **Transport lag versus braking power:** Can you distinguish between the spatial cost of computational transport delay ($v \cdot \tau_{\text{delay}}$) and active braking ($v^2 / 2a_{\text{max}}$), identifying which term dominates at low crawling speeds versus high transit speeds?
- ☐ **The kinematic roofline principle:** Can you describe how finite sensor detection horizons and occluded sightlines (such as warehouse blind corners) impose an unyielding ceiling on safe operating speed, regardless of how fast neural inference can propose trajectories?

The parameters governing the stopping envelope are rarely stationary during physical operation. When a robotic manipulator grasps an unmodeled 15 kg steel billet, the combined inertia of the arm increases, reducing the maximum angular deceleration a_{max} that joint actuators can produce without exceeding their peak mechanical torque ratings. Similarly, on a mobile robot, moisture or oil on a concrete floor reduces the tire friction coefficient, lowering the maximum tractive deceleration before wheel slip occurs and widening the tracking error ϵ_{track} . Repeated hard decelerations also drive up rotor and inverter temperatures (Joule heating via $I^2 R$ losses), triggering the actuator thermal derating limits (section 4.5). An enforcer that treats a_{max} as a static datasheet constant will underestimate the stopping envelope precisely when the machine is under heavy load. The enforcer must use independently bounded braking capability and reduce permitted speed when validated friction, payload, or thermal conditions change; an uncertain estimate cannot by itself restore a guarantee.

Dynamic envelope adaptation functions only as long as the enforcer possesses accurate state estimates and the actuators retain the physical capacity to supply the requested forces. If surface contamination drops effective traction to near zero, or if a sustained duty cycle pushes winding temperatures past their thermal ceilings, the deceleration capability assumed in the safety calculation evaporates. Under those conditions, the mathematical envelope expands toward infinity while the physical machine continues along its ballistic path. The moment the required deceleration torque exceeds what the physical hardware can produce, the enforcer transitions from a deterministic guardian into an impotent observer.

24.6 When the Enforcer Runs Out of Authority

In control engineering, **actuator saturation** is a bounded nonlinearity that introduces phase lag, degrades stability, and causes windup in feedback controllers. In the architecture of a physical AI system, saturation is more immediate and physical: it is the moment a motor is giving everything

it has and simply cannot push any harder. It is the instant the permission path loses the physical capacity required to keep its safety warranty. The enforcer makes guarantees on the premise that it can interpose a corrective control action u_{corr} whenever the machine approaches a forbidden state boundary. When an actuator reaches its mechanical limit (peak torque), its electrical limit (bus voltage or current clipping), or its thermal limit (the I^2R Joule heating threshold, section 4.5), that corrective force cannot be delivered at the commanded magnitude. The enforcer cannot protect a boundary using force that the physical body cannot generate.

A stopping envelope computed under nominal assumptions becomes invalid as soon as authority is consumed. From first principles, actuator capacity is finite, and any authority expended to maintain steady-state motion (like climbing an incline or holding a payload against gravity) directly subtracts from the force available for emergency braking.⁹ Under a steady-state bias torque τ_{bias} , the effective deceleration available to arrest angular motion is:

$$\alpha_{\text{eff}} = \frac{\tau_{\text{max}} - \tau_{\text{bias}}}{J} \quad (24.3)$$

where J is the mechanism inertia and τ_{max} is the peak dynamic torque limit. In linear translational systems with mass m and bias force u_{bias} , equation 24.3 takes the equivalent form $a_{\text{eff}} = (u_{\text{max}} - |u_{\text{bias}}|)/m$. For example, if a mobile base requiring 0.50 m to stop consumes 75 percent of its motor torque simply to maintain steady speed on an incline, only one-quarter of its force remains for braking, quadrupling its stopping distance to 2.00 m.

Consider an analytical example of an articulated industrial manipulator lowering a payload in a joint-level scenario at joint angular velocity $\omega_0 = 1.20$ rad/s toward an assembly fixture. The effective linkage length is $\ell = 0.80$ m, the total joint inertia is $J = 3.0$ kg · m², and the joint motor drive provides a peak dynamic torque limit $\tau_{\text{max}} = 120$ N · m. In horizontal free space where gravitational bias is absent ($\tau_{\text{bias}} = 0$), the entire peak torque is available for braking, yielding a nominal maximum angular deceleration of $\alpha_{\text{nom}} = 120 \text{ N} \cdot \text{m} / 3.0 \text{ kg} \cdot \text{m}^2 = 40.0 \text{ rad/s}^2$. The nominal stopping angle from $\omega_0 = 1.20$ rad/s is $\Delta\theta_{\text{nom}} = \omega_0^2 / (2\alpha_{\text{nom}}) = (1.20)^2 / (2 \times 40.0) = 0.018 \text{ rad} = 1.03^\circ$, which produces a Cartesian tool-point linear stopping displacement along arc of $d_{\text{tool,nom}} = \ell\Delta\theta_{\text{nom}} = (0.80 \text{ m})(0.018 \text{ rad}) = 0.0144 \text{ m} = 14.4 \text{ mm}$.

However, during vertical downward descent, the specified load and arm geometry produce a downward joint bias of $\tau_{\text{bias}} = 90$ N · m, consuming 75 percent of the motor's total torque capacity. According to equation 24.3, the net upward torque available for emergency deceleration collapses to $\tau_{\text{net}} = \tau_{\text{max}} - \tau_{\text{bias}} = 120 - 90 = 30$ N · m. The effective angular deceleration collapses by a factor of four to $\alpha_{\text{eff}} = 30/3.0 = 10.0 \text{ rad/s}^2$, causing the stopping angle to expand fourfold to $\Delta\theta_{\text{eff}} = (1.20)^2 / (2 \times 10.0) = 0.072 \text{ rad} = 4.13^\circ$. The tool stopping displacement surges to $d_{\text{tool,eff}} = (0.80 \text{ m})(0.072 \text{ rad}) = 0.0576 \text{ m} = 57.6 \text{ mm}$, an unbudgeted overrun of $\Delta d = 57.6 \text{ mm} - 14.4 \text{ mm} = 43.2 \text{ mm}$. If the enforcer evaluates barrier envelopes using datasheet torque τ_{max} rather than net dynamic headroom $\tau_{\text{max}} - \tau_{\text{bias}}$, the predicted stop travels 43.2 mm farther than budgeted. Whether contact occurs or causes damage depends on actual initial clearance and fixture mechanics.

When tracking controllers beneath the enforcer employ integral action, saturation introduces an additional latency penalty. If an actuator reaches its limit while tracking error $\epsilon(t) = x_{\text{des}}(t) - x(t)$ persists, the integrator continues accumulating error, driving the internal control state far beyond the physical ceiling. Physically, the controller accumulates an unexecutable corrective command integral because the saturated plant cannot eliminate the error. When the enforcer intervenes to command full braking, the motor does not reverse force immediately. The physical output remains pinned at the forward saturation limit while this accumulated integrator error unwinds back below the clamping threshold. In an analytical example where tracking error accumulates over 200 ms, the integrator unwinding phase can introduce a recovery delay $\tau_{\text{unwind}} = 150$ ms before net negative force actually develops at the motor. At $v_0 = 2.0$ m/s, this delay adds an unbraked coasting distance $\Delta d_{\text{delay}} = v_0\tau_{\text{unwind}} = 0.30$ m directly to the stopping envelope. Control literature addresses this behavior through anti-windup clamping, back-calculation, and conditional integration (see Åström and Hägglund (2006); Franklin et al. (1998)). From the perspective of systems enforcement, the requirement is not merely to stabilize the control loop, but to measure this unwinding delay and budget its unbraked distance penalty inside the forward stopping horizon.

Actuator saturation is one of the few hardware failures with a direct, deterministic detector inside the nervous system. At each execution cycle of the motor tracking loop, typically running at 1000 Hz

⁹ | **Actuator bias torque and stopping margin loss:** When an actuator allocates continuous baseline torque to counter gravitational loading or steady-state friction, available emergency braking headroom collapses proportionally. In vertical and heavily loaded axes, this depletion of net dynamic torque causes stopping distances to quadruple under seemingly modest bias loads. For a detailed derivation of saturated authority and stopping margin contraction, see Appendix B.

(1 ms tick interval), the nervous system compares the commanded effort u_{cmd} with the realizable actuator limit $u_{\text{act}} = \text{clip}(u_{\text{cmd}}, u_{\text{min}}, u_{\text{max}})$. A nonzero clipping difference, $|u_{\text{cmd}} - u_{\text{act}}| > 0$ —meaning the software requested more torque than the hardware could physically deliver—or an inverter status register indicating current limit saturation, provides an immediate signal that the actuator has exhausted its authority. By tracking the duration and magnitude of this clamping signal, the nervous system monitors available control authority on every millisecond tick without relying on noisy external state estimation.

Because authority consumption alters stopping distance immediately, the enforcer cannot treat actuator limits as static parameters in an offline calculation. Instead, the nervous system must fold real-time authority consumption directly into the stopping envelope. When an actuator allocates baseline torque u_{bias} to hold a payload against gravity or overcome an opposing grade, the enforcer recalculates effective deceleration as $a_{\text{eff}} = (u_{\text{max}} - |u_{\text{bias}}|)/m$ before verifying the next trajectory segment. As authority is consumed, the allowable speed ceiling decreases continuously. If authority depletion causes the calculated stopping distance to exceed the clear sensing horizon, the system reaches an authority deficit. The enforcer cannot allow the machine to maintain its speed. It must command deceleration while sufficient physical authority still remains.

Authority depletion is state information that must be exported upstream to the planner. A learned policy operating at 20 Hz that continues commanding accelerations to a motor saturated at 1000 Hz operates completely divorced from physical reality. If the planner does not receive saturation feedback, it will continue generating trajectories based on acceleration profiles the body cannot execute, rapidly widening the divergence between the model's expected state and the machine's true physical state. The nervous system communicates saturation events to the policy layer as an active constraint violation. When an authority alarm triggers, the planner's active trajectory is revoked, requiring the model to re-plan from the machine's true, authority-constrained state.

Every dynamic stopping envelope and safety margin depends on the physical capacity of the machine to generate corrective force within the remaining distance. When that capacity is restricted by load, grade, or thermal derating, the space of executable safe trajectories contracts. An enforcer cannot verify safety simply by checking whether the current state is collision-free. It must confirm that the current state permits an executable sequence of future control actions that terminate safely under the actuator authority that actually exists.

Systems Perspective 24.3: The actuator authority invariant

A Control Barrier Function, as formulated in section 20.3, is mathematically valid only while physical actuators retain sufficient current, voltage, and thermal headroom to deliver the computed corrective force: $u^* \in [u_{\text{min}}, u_{\text{max}}]$. When steady-state gravity or friction consumes the actuator's operating envelope, the enforcer must dynamically contract the safe set before control authority at the margin vanishes.

Checkpoint 24.3: Actuator headroom and barrier contraction

TORQUE HEADROOM LOSS & STOPPING DISTANCE EXPANSION

Datasheet Rating: $\tau_{\text{max}} = 120 \text{ N}\cdot\text{m}$

(a) Horizontal Free Space

NOMINAL · $\tau_{\text{b}} = 0$

Available Dynamic Braking Headroom:

120 N·m Dynamic Braking Headroom (100%)

Stopping Displacement (Scale: 1 mm = 3 px):

14.4 mm

Free Space Clearance Buffer

$d_{\text{stop}} = 14.4 \text{ mm}$

Requires $\geq 14.4 \text{ mm}$ clearance

Decel: $\alpha_{\text{nom}} = 40.0 \text{ rad/s}^2$ · Arrest Angle: 1.03°

Nominal stop: 14.4 mm under stated assumptions.

(b) Vertical Descent (Loaded)

LOAD · $\tau_{\text{b}} = 90$

Torque Allocation (75% Steady-State Gravity):

Gravity Holding: 90 N·m (75%)

30 N·m (25%)

Stopping Displacement (4× under Load Bias):

14.4 mm

+43.2 mm beyond budget

Nominal stop

Loaded stop: 57.6 mm

Decel: $\alpha_{\text{eff}} = 10.0 \text{ rad/s}^2$ (1/4th) · Arrest Angle: 4.13°

Contact if clearance < 57.6 mm; recheck the margin.

Consider how steady-state gravitational and external loads reshape actuator control authority:

- **Datasheet limits versus dynamic headroom:** Can you explain why evaluating safety barriers using nominal datasheet torque ratings creates a false sense of security during vertical or loaded maneuvers?
- **Effective deceleration collapse:** Can you describe the physical mechanism that causes stopping distance to quadruple when steady-state gravity consumes 75 percent of peak actuator torque?
- **Dynamic safe set contraction:** Can you explain why the nervous system must actively shrink the admissible state space when an actuator approaches thermal or electrical saturation, rather than waiting for tracking error to breach a hard limit?

24.7 Safe Sets as Conditional Permission

A safe set gives the enforcer a concrete predicate that can be evaluated on every tick of the control clock. In control theory, a **safe set** $\mathcal{C}$ is defined as the zero-superlevel set of a continuously differentiable scalar certificate $h(x) : \mathcal{X} \rightarrow \mathbb{R}$, such that

$$\mathcal{C} = \{x \in \mathcal{X} \mid h(x) \geq 0\}, \quad \partial\mathcal{C} = \{x \in \mathcal{X} \mid h(x) = 0\}.$$

The mathematical foundation of **forward invariance**, established by Blanchini (Blanchini 1999), formulated through Hamilton-Jacobi reachability analysis and viability kernels by Mitchell, Bayen, and Tomlin (Mitchell et al. 2005), and extended through Control Barrier Functions by Ames et al. (2014) and Ames et al. (2019),¹⁰ characterizes conditions under which an admissible controller can keep a state within a viable set. Geometric clearance alone is not that set when stopping torque is finite.

¹⁰ | **Nagumo viability theorem and boundary tangent cones:** Nagumo's theorem establishes that a closed set is forward-invariant if and only if the system velocity vector belongs to the contingent cone of the set at every boundary point. Control Barrier Functions synthesize this geometric condition into real-time affine inequality constraints on actuator torque. Violating this sub-tangentiality condition causes immediate set exit, rendering future boundary enforcement unachievable; see Appendix B.



Napkin Math 24.1: Forward invariance and safety boundaries

Forward invariance formalizes safety by bounding trajectories within an admissible safe set $\mathcal{C}$ where kinematic, electrical, and thermal limits remain unbreached. Rather than forecasting the machine's multi-step future path through expensive non-convex trajectory optimization, Control Barrier Functions evaluate an instantaneous rate-of-approach condition: whether the commanded control input vectors the state toward the safe set boundary faster than available braking deceleration can arrest momentum.

By constraining the temporal derivative of the barrier function along the system vector fields, the nonlinear dynamics of the physical plant collapse into an instantaneous linear inequality constraint on actuator torque. If the barrier inequality remains feasible and the model, sensing, actuation, and timing bounds hold, this condition supports forward invariance from an initially viable state.

For formal mathematical derivations of Control Barrier Functions, Lie derivatives, and Nagumo's viability theorem, see Appendix B.



Definition 24.2: Control barrier function (CBF-QP)

Control barrier function safety filter (CBF-QP) is a conditional safety filter that projects a proposed input through a bounded convex quadratic program:

$$\min_{\mathbf{u}} \frac{1}{2} \|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2 \quad \text{s.t.} \quad \mathbf{a}(\mathbf{x})^\top \mathbf{u} \leq b(\mathbf{x}), \quad \mathbf{u}_{\min} \leq \mathbf{u} \leq \mathbf{u}_{\max}$$

that projects unverified learned proposals $\mathbf{u}_{\text{nom}}$ onto linear braking constraints $(\mathbf{a}(\mathbf{x})^\top \mathbf{u} \leq b(\mathbf{x}))$, leaving candidates that satisfy the current torque half-space untouched and projecting candidates that violate it, even while the state remains inside the viable region.

1. **Significance:** Rather than attempting the impossible task of proving formal safety across billions of uninterpretable neural network weights, a CBF-QP filter decouples high-level policy innovation from low-level safety enforcement. The learned model is free to explore complex manipulation or navigation strategies, while the deterministic enforcer can preserve a validated viable set while its physical assumptions remain true.
2. **Distinction:** Unlike a heuristic clamp that saturates control channels independently (destroying multi-axis coordination), a CBF-QP filter performs an optimal orthogonal projection in control space, minimizing the chosen torque-space distance; it does not automatically preserve a Cartesian path.
3. **Common pitfall:** Formulating barrier constraints using raw geometric coordinates without accounting for relative degree or braking inertia. If the barrier does not incorporate stopping distance ($\Delta d = v^2/2a_{\max}$), the filter intervenes too late, after the vehicle's momentum has already made a boundary collision inevitable.

When a learned neural policy proposes an unverified action $\mathbf{u}_{\text{nom}}$, the enforcer does not need to forecast the entire future path of the machine. It evaluates a single half-space inequality—a linear constraint representing the maximum safe approach speed toward the boundary—at the current state $\mathbf{x}$. If the proposed action satisfies this constraint, mathematically expressed as $\mathbf{a}(\mathbf{x})^\top \mathbf{u}_{\text{nom}} \leq b(\mathbf{x})$, the enforcer passes the command to the motor untouched. If the proposed action violates a feasible half-space, the enforcer projects its torque vector. The resulting plant path still depends on coupled dynamics and tracking.

The MCU evaluates this barrier on sampled cycles; the admission margin must cover intersample travel and measured sensing-to-torque delay. Consider a heavy cart moving down a track toward a wall. If the enforcer only looks at the distance to the wall, it realizes there is a problem too late—when the cart is already too close to stop given its current momentum.¹¹ Instead, the enforcer's safety barrier must factor in both the cart's position and its kinetic stopping distance ($v^2/2a_{\max}$).

This dynamic exposes the foundational physical concept of **relative degree**. The physical problem is that actuators command electrical current and torque, not velocity or position. Torque produces acceleration ($F = ma$). As a result, position is separated from the commanded control input by two time integrals:

$$\text{Actuation } u \xrightarrow{\int \frac{1}{m} dt} \text{Velocity } v \xrightarrow{\int dt} \text{Position } x$$

This two-step integration cascade defines relative degree $r = 2$. If an enforcer monitors only geometric distance without accounting for kinetic energy, actuator braking cannot overcome physical inertia once the stopping envelope is breached. Differentiating clearance once with respect to time yields velocity:

$$\dot{d} = -v$$

At this first derivative, the actuator control input u (motor torque or phase current) has not yet appeared, because an electric motor cannot produce an instantaneous step change in physical velocity. Only when clearance is differentiated a second time does Newton's second law ($m\ddot{x} = u - d_{\text{ext}}$) enter the physical dynamics:

$$\ddot{d} = -\dot{v} = -\left(\frac{u - d_{\text{ext}}}{m}\right)$$

Because the commanded control input u physically appears only on the second time derivative, geometric clearance has relative degree $r = 2$.

A raw-distance rule $\dot{d} \geq -\gamma d$ cannot directly constrain this torque input: $\dot{d} = -v$ contains no u . For an approaching arm, let $D = d_{\text{tool}}/\ell$ be the remaining angular clearance, $\omega > 0$ its approach speed, $J\dot{\omega} = \tau + \tau_{\text{bias}}$, and α a validated lower bound on available angular braking. Define the kinetic barrier

$$h(D, \omega) = D - \frac{\omega^2}{2\alpha}, \quad \dot{h} = -\omega - \frac{\omega(\tau + \tau_{\text{bias}})}{J\alpha}. \quad (24.4)$$

¹¹ | **Relative degree and inertia-aware barrier certificates:** When a safety barrier monitors position while the actuator commands torque, the control input is separated from the constraint by two time integrals ($r = 2$). Enforcing a zero-order spatial barrier without embedding kinetic stopping distance produces an optimization problem with zero control authority on the boundary. A velocity-dependent barrier can support a conditional forward-invariance argument while braking remains feasible and the state, delay, and tracking margins hold; see Appendix B.

The raw distance D has relative degree two; this velocity-dependent h has relative degree one for $\omega > 0$. Requiring $\dot{h} + \gamma h \geq 0$ produces the actual affine torque half-space

$$\tau \leq -\tau_{\text{bias}} + J\alpha \left(\frac{\gamma h}{\omega} - 1 \right), \quad -\tau_{\text{max}} \leq \tau \leq \tau_{\text{max}}. \quad (24.5)$$

At standstill ($\omega = 0$), the controller uses a separate no-approach and acceleration admission check; it does not divide by zero. Delay, estimate error, and torque-tracking error must consume additional clearance before applying this ideal continuous-time barrier.

In the loaded-arm scenario, take $J = 3 \text{ kg} \cdot \text{m}^2$, $\tau_{\text{bias}} = 90 \text{ N} \cdot \text{m}$ toward the fixture, $\tau \in [-120, 120] \text{ N} \cdot \text{m}$, $\alpha = (120 - 90)/3 = 10 \text{ rad/s}^2$, $\ell = 0.80 \text{ m}$, $\omega = 1.2 \text{ rad/s}$, and $\gamma = 10 \text{ s}^{-1}$. These are stated joint-level scenario inputs; the $90 \text{ N} \cdot \text{m}$ bias is not inferred from the payload mass alone. At 80 mm tool clearance, $D = 0.10 \text{ rad}$ and $h = 0.028 \text{ rad}$, so equation 24.5 permits at most $-113 \text{ N} \cdot \text{m}$. At 57.6 mm , $h = 0$ and the limit reaches $-120 \text{ N} \cdot \text{m}$. At 50 mm , it demands $-122.375 \text{ N} \cdot \text{m}$, outside the available torque interval: this state is already outside the modeled viable set. The enforcer must prevent entry into it; a fallback started there cannot restore the lost clearance. These results assume immediate torque realization, constant bias, and exact state. A deployed checker must inset the boundary for bounded response delay, measurement error, and torque-tracking deviation.

A real machine does not execute continuous, ideal forces on an uncorrupted plant. The nominal barrier assumes exact state feedback and instantaneous torque realization, but the nervous system operates subject to physical latency, including stator delays (the electromagnetic lag before current builds in the motor coils), bus jitter, and gearbox compliance (the mechanical springiness and slop in the transmission). Because of this, the enforcer cannot evaluate the barrier at the geometric edge of the safe set. It must mathematically inset the boundary to swallow the worst-case stopping deficit caused by these delays and mechanical compliances, forcing it to intervene earlier in the trajectory.

The safety warrant provided by the barrier calculation is strictly conditional. A forward-invariance claim requires a feasible barrier and the following bounded conditions; the list is necessary for this design, not an exhaustive if-and-only-if test:

1. **Model Fidelity:** The nominal plant dynamics $f(x)$ and $g(x)$ must capture the true system dynamics within a certified parameter error bound.
2. **Disturbance Bounds:** External forces and unmodeled friction acting on the mass must remain strictly within the modeled disturbance envelope $|d(t)| \leq d_{\text{max}}$.
3. **Actuation Authority:** Physical actuators must possess sufficient voltage, current, and thermal headroom to realize the commanded corrective force without saturation ($u^* \in [u_{\text{min}}, u_{\text{max}}]$).
4. **Solver Determinism:** The optimization solver or projection algorithm must terminate strictly within its allocated execution budget ($T_{\text{solve}} \leq T_{\text{budget}} < T_{\text{loop}}$).
5. **Discretization Invariance:** The discrete sampling interval T_{ctrl} must be sufficiently small that inter-sample state drift does not breach the continuous-time invariance boundary.

Each of these five conditions maps directly to physical system budgets, separating empirical bench characterization from assumptions that the release argument must explicitly carry. Model fidelity and disturbance bounds are measured on the bench through system identification and load testing, while operational envelopes assume these bounds hold. Actuation authority is governed by dynamometer curves under continuous load and actuator thermal limits (section 4.5). Solver execution is governed by the latency budget of the nervous system processor, verified through static worst-case execution time analysis. Sampling interval validity is governed by the loop frequency budget, ensuring that the discrete control step $T_{\text{ctrl}} = 1.0 \text{ ms}$ limits the unobserved physical drift that occurs between sensor readings (known mathematically as zero-order hold integration error) to a fraction of the safety margin δ_{margin} .

Because a conditional guarantee whose conditions are unmonitored is an ungrounded assertion, the nervous system must attach a dedicated hardware or software detector to each condition. A **disturbance observer** calculates the residual between predicted acceleration and measured accelerometer data on every cycle, flagging unmodeled external forces that exceed the disturbance budget. Inverter status registers and current shunt monitors track **actuator saturation**, reporting when commanded torque cannot be delivered. A dedicated hardware watchdog and a high-resolution cycle counter monitor solver execution latency against the sub-millisecond deadline, transferring

authority to an independently validated fallback if the core halts or misses a deadline. Timestamp counters on the sensor bus monitor communication jitter and missed sample deadlines.

When any runtime detector trips, the mathematical premise of the safe set collapses. If an actuator overheats and drops its torque limit, or if an unexpected payload disturbance exceeds the observer threshold, the enforcer cannot continue claiming that tangential projection along $\partial\mathcal{C}$ guarantees safety. The correct system response is not to recompute the barrier with optimistic parameters, but to execute an immediate transition to a dedicated fallback mode via the **Simplex switch**. Triggering this pattern revokes the neural policy and requests a state-matched fallback whose response time and braking margin were reserved before the fault; later rungs may limit harm if the viable set has already been lost. The safe set provides a valid permission path only while its operating contract is intact, and the nervous system enforces safety by knowing precisely when that contract has failed.

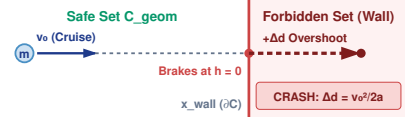


Checkpoint 24.4: Forward invariance, Lie derivatives, and viability kernels

DISTANCE AND KINETIC STOPPING BARRIERS

(a) Raw distance: relative degree 2

Evaluates Spatial Clearance: $h(x) = x_{\text{wall}} - x \geq 0$

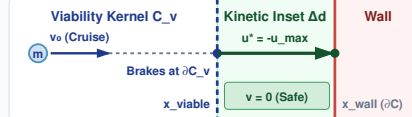


Failure: Torque commands acceleration, not velocity ($r = 2$).
Braking at boundary overshoots by kinetic stopping distance Δd .

Dynamics: $m\ddot{x} = u$ (Relative Degree $r = 2$)

(b) Kinetic h: relative degree 1

Embeds Stopping Distance: $h_z(x, v) = (x_{\text{wall}} - x) - v^2/(2a) \geq 0$



Condition: Safe set contracted to viability kernel: $C_v \subset C_{\text{geom}}$.
Brake before the wall if model, delay, and torque bounds hold.

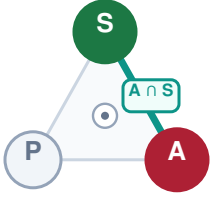
Before studying minimal intervention filters, verify your mathematical understanding of how forward invariance guarantees containment under worst-case plant dynamics:

- ☐ **Nagumo viability condition:** Can you explain why the system velocity vector must belong to the contingent cone of the safe set at every boundary point ($\dot{h}(x, u) \geq -\alpha(h(x))$) to guarantee set invariance?
- ☐ **Relative degree mismatch:** Can you describe why applying a first-order Control Barrier Function directly to an acceleration-controlled plant with relative degree two creates an uncontrolled boundary overshoot?
- ☐ **Lie derivative projection:** Can you demonstrate how Lie derivatives ($L_f h(x) + L_g h(x)u$) linearize nonlinear plant dynamics into an instantaneous affine half-space constraint on actuator torque?
- ☐ **Viability kernel shrinkage:** Can you explain why the topological safe set $\mathcal{C}$ must be strictly smaller than the kinematic free-space manifold to account for finite actuator deceleration capacity?

A state is not safe merely because it is currently collision-free; it is safe only if there exists an executable sequence of future control actions that terminates within the admissible boundary under physical torque limits.

24.8 Minimal Intervention

The principle of **minimal intervention** leaves an admitted proposal unchanged when it satisfies every active constraint and changes it when projection is both needed and feasible, even while the state remains inside the viable set. Replacing the neural network's commands with a hand-crafted conservative controller during nominal operation discards the behavioral competence justifying the learned model. Overriding a proposed action with an aggressive braking command at the first boundary encounter creates acceleration spikes, destabilizes balance, and interrupts task progress. Minimal intervention treats safety enforcement as a **safety shield** (projection operator). The nervous



The Act $\cap$ Sense reflex shield: 1 kHz Control Barrier Functions and hardware veto authority.

Prerequisite: Nominal trajectory setpoints and feedforward torques are synthesized by the planner in section 22.3.

12 | Worst-case execution time and active-set QP complexity: On bare-metal safety MCUs, QP safety filters execute within static memory arrays using fixed iteration limits. While typical active-set solve times range between 60–120 μ s, worst-case execution time spikes to 600 μ s under ill-conditioned or nearly degenerate constraint geometries. Hard real-time control loops must budget for this maximum execution envelope to prevent timing overruns from causing uncommanded actuator hold states.

13 | Control barrier functions and Lie derivatives: In nonlinear control theory, the unforced drift and actuator control authority are formalized using Lie derivatives ($L_f h(x)$ and $L_g h(x)$). When the safety boundary has relative degree one, the control input appears directly in the first time derivative of the barrier function, yielding an instantaneous affine constraint on commanded torque. If the relative degree is two, the enforcer must evaluate higher-order derivatives to prevent actuator saturation from rendering boundary violations inevitable; see Appendix B.

system accepts the policy’s proposal as the default intent and computes the smallest physical correction that keeps the system within the invariant set.

A **minimal-intervention QP projection (safety shield)** is a sub-millisecond convex optimization safety filter ($\min_{\mathbf{u}} \frac{1}{2} \|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2$ s.t. $\mathbf{a}(\mathbf{x})^\top \mathbf{u} \leq b(\mathbf{x})$, $\mathbf{u}_{\text{min}} \leq \mathbf{u} \leq \mathbf{u}_{\text{max}}$) that passes a proposal unchanged when it satisfies the current torque constraint and computes the least-squares projection when it violates that constraint, including at viable interior states; torque-space proximity alone does not preserve the task path.

This safety filtering mechanism is framed as a **control barrier function quadratic program (CBF-QP)** (Ames et al. 2016). In standard matrix form for the real-time solver, the CBF-QP evaluates a convex filter: checking a model-derived torque half-space before the stopping margin is exhausted. At each discrete control interval $T_{\text{ctrl}} = 1.0$ ms on the bare-metal microcontroller, the enforcer receives a nominal torque command $\mathbf{u}_{\text{nom}} \in \mathbb{R}^m$ from the policy mailbox and solves the convex optimization problem:¹²

$$\min_{\mathbf{u}} \frac{1}{2} \|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2 \quad \text{s.t.} \quad \mathbf{a}(\mathbf{x})^\top \mathbf{u} \leq b(\mathbf{x}), \quad \mathbf{u}_{\text{min}} \leq \mathbf{u} \leq \mathbf{u}_{\text{max}} \quad (24.6)$$

where $\mathbf{a}(\mathbf{x})^\top \mathbf{u} \leq b(\mathbf{x})$ represents a torque constraint derived from a velocity-dependent barrier such as equation 24.5,¹³ and $[\mathbf{u}_{\text{min}}, \mathbf{u}_{\text{max}}]$ represents the verified actuator torque envelope.

Napkin Math 24.2: Minimal intervention through optimization

Objective: Check each candidate against the active torque constraint, projecting a violating proposal with the least torque change while the state remains viable.

The safety filter receives unverified candidate torques and tests them against model-derived affine constraints. A feasible proposal passes unchanged; an infeasible proposal is projected when the admissible set is nonempty. Euclidean projection minimizes torque-vector change, not Cartesian path deviation.

For a nonempty convex feasible set, the quadratic objective has a unique minimizer. A platform-specific iteration cap and measured worst-case execution bound are still needed for a hard deadline.

For a rigorous derivation of the closed-form projection and Lagrange multipliers used in the CBF-QP formulation, see Appendix B.

When enforcing a single active linear barrier constraint $\mathbf{a}^\top \mathbf{u} \leq b$ in equation 24.6, the optimization admits an exact closed-form orthogonal projection:

$$\mathbf{u}^* = \mathbf{u}_{\text{nom}} - \frac{\mathbf{a}^\top \mathbf{u}_{\text{nom}} - b}{\|\mathbf{a}\|^2} \mathbf{a} \quad (24.7)$$

where the optimal Lagrange multiplier $\lambda^* = (\mathbf{a}^\top \mathbf{u}_{\text{nom}} - b) / \|\mathbf{a}\|^2$ scales the minimal intervention vector $\Delta \mathbf{u} = -\lambda^* \mathbf{a}$.

Consider an analytical example of a 2-axis robotic manipulator governed by an unconstrained neural policy proposing joint torques $\mathbf{u}_{\text{nom}} = [3.5, 3]^\top \text{ N} \cdot \text{m}$. To isolate the projection arithmetic, suppose an abstract safety half-space has already been derived and validated: $\mathbf{a}^\top \mathbf{u} \leq b$ with $\mathbf{a} = [2, 1]^\top$ and $b = 5 \text{ N} \cdot \text{m}$, with per-joint actuator saturation limits of $[-5.0, 5.0] \text{ N} \cdot \text{m}$. Evaluating the constraint on the candidate command yields $\mathbf{a}^\top \mathbf{u}_{\text{nom}} = 2.0(3.5) + 1.0(3.0) = 10 \text{ N} \cdot \text{m}$. Because $10 > 5$, the policy proposal breaches the barrier boundary by $\Delta b = 5 \text{ N} \cdot \text{m}$, demanding intervention.

Applying the closed-form projection from equation 24.7, the optimal Lagrange multiplier is $\lambda^* = (10 - 5) / (2^2 + 1^2) = 5/5 = 1$. The minimal-intervention command is:

$$\mathbf{u}^* = \begin{bmatrix} 3.5 \\ 3.0 \end{bmatrix} - 1.0 \begin{bmatrix} 2.0 \\ 1.0 \end{bmatrix} = \begin{bmatrix} 1.5 \\ 2.0 \end{bmatrix} \text{ N} \cdot \text{m}$$

yielding $\mathbf{u}^* = [1.5, 2]^\top \text{ N} \cdot \text{m}$. Evaluating the barrier boundary confirms $\mathbf{a}^\top \mathbf{u}^* = 2.0(1.5) + 1.0(2.0) = 5 \text{ N} \cdot \text{m}$, exactly satisfying the stipulated half-space while both joints remain well within their $[-5.0, 5.0] \text{ N} \cdot \text{m}$ limits. The intervention norm is $\|\Delta \mathbf{u}\| = \sqrt{(-2.0)^2 + (-1.0)^2} = \sqrt{5} \approx 2.236 \text{ N} \cdot \text{m}$.

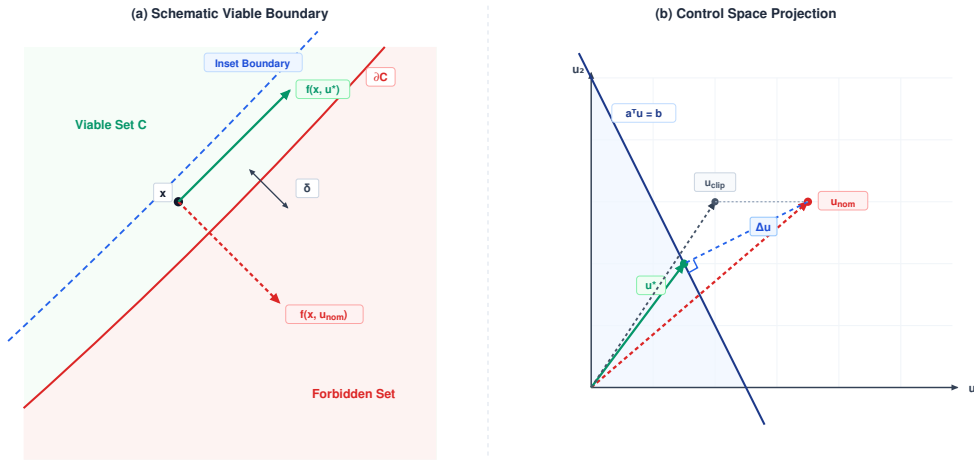


Figure 24.4: Kinetic barrier and torque projection: Panel (a) schematically shows a velocity-dependent viable boundary inset from geometric clearance; the raw distance alone has relative degree two for torque. Panel (b) illustrates Euclidean projection onto a supplied torque half-space. The $[2, 1]$ normal and $5 \text{ N} \cdot \text{m}$ threshold are abstract arithmetic inputs, not coefficients derived from the arm example. Projection changes torque direction and does not alone certify the Cartesian path.

The geometric necessity of this projection becomes obvious when contrasted with heuristic per-axis clamping. If an embedded controller naively clamps Joint 1 at $2 \text{ N} \cdot \text{m}$ while passing Joint 2 at $3 \text{ N} \cdot \text{m}$ ($\mathbf{u}_{\text{clip}} = [2, 3]^T \text{ N} \cdot \text{m}$), evaluating the barrier yields $\mathbf{a}^T \mathbf{u}_{\text{clip}} = 2.0(2.0) + 1.0(3.0) = 7 \text{ N} \cdot \text{m} > 5 \text{ N} \cdot \text{m}$. The heuristic clamp fails this abstract torque constraint by $2 \text{ N} \cdot \text{m}$. Furthermore, while the nominal command had an actuation angle of $\theta_{\text{nom}} = \arctan(3.0/3.5) \approx 40.6^\circ$, the heuristic clamp shifts the torque vector to $\theta_{\text{clip}} = \arctan(3.0/2.0) \approx 56.3^\circ$ (a 15.7° uncoordinated skew). The clipped torque vector changes direction. The projected vector also rotates relative to the proposal, so Cartesian direction and mechanical response must be checked separately. The quadratic projection minimizes Euclidean torque change under the given constraint. Neither operation alone establishes the end-effector path; that requires a coupled plant model and trajectory check.

Figure 24.4 works the minimal-intervention architecture simultaneously across state space $\mathcal{X}$ and control input space $\mathcal{U}$. In state space (panel a), unconstrained policy proposals vectoring across the barrier into the unsafe region are deflected tangentially along the forward-invariant safe set boundary $\partial \mathcal{C}$ inset by the tracking error margin δ_{margin} . In control space (panel b), the enforcer orthogonally projects the unconstrained command $\mathbf{u}_{\text{nom}}$ onto the linear half-space boundary $\mathbf{a}^T \mathbf{u} \leq b$, satisfying the illustrative half-space while the independently clamped vector violates it; neither operation alone proves a Cartesian path.

Filter choice joins a mathematical condition to a measured execution budget and the machine’s available braking authority. As summarized in table 24.1, architectural choices vary dramatically in their handling of relative degree, solver worst-case execution time (WCET), forward invariance guarantees, and directional fidelity. Independent clamping can miss coupled constraints; a CBF-QP can enforce a derived half-space if it is feasible and finishes within the verified cycle. Predictive filters (Wabersich and Zeilinger 2021) trade a longer horizon for configuration-dependent compute cost.

Table 24.1: Safety-filter decisions: A filter’s physical claim depends on the chosen barrier and validated plant, not on the algorithm name. Nominal solve measurements are not worst-case deadlines.

Filter	Input and condition	What it can establish	Main validation task
Independent torque clamp	Each axis checked separately	Hardware amplitude limit, not coupled stopping clearance	Test coupled wrench and stopping behavior
First-order CBF-QP	Barrier whose first derivative contains the input	Conditional invariance from a viable initial state	Validate model, feasible torque set, and cycle bound
Kinetic / higher-order barrier	Position constraint with speed and braking authority included	Conditional stopping margin before geometric contact	Bound bias torque, delay, estimate error, and discretization
Precomputed viable region	State-to-permitted-input lookup	Containment for the modeled region	Verify lookup coverage and interpolation
Predictive safety filter	Finite-horizon trajectory and terminal set	Recursive feasibility only under terminal and timing assumptions	Bound full solve time and fallback on timeout

Operating near an active safe-set boundary can produce chattering. If a learned policy repeatedly proposes an unsafe command, sampled enforcement may switch the active constraint across successive 1.0 ms ticks. Depending on the plant and controller, rapid torque changes can excite a structural mode or increase motor heating and transmission wear. A tested boundary layer or hysteresis can reduce switching, but its added tracking error must fit inside the validated stopping margin; smoothing alone does not establish invariance.

Beyond enforcing safety on each instantaneous cycle, the intervention vector $\Delta u = u^* - u_{\text{nom}}$ provides a continuous diagnostic measurement of upstream policy health. During nominal operation within the training distribution, a well-trained policy operates inside the safe set, producing zero intervention norm across most cycles. An increase in intervention frequency or magnitude can indicate proposal drift, a changed task, or a changed plant; the signal needs diagnosis before it is used as a release decision. By tracking the cumulative intervention metric over a rolling window of $W = 500$ ms, calculated as the discrete sum $\sum_{k=0}^{W/\Delta t-1} \|u_k^* - u_{\text{nom},k}\| \Delta t$, the nervous system can flag a pattern for diagnosis while separate per-tick limits still protect the plant. When the P_{95} intervention metric over the window exceeds a predetermined threshold, the system flags policy divergence, allowing the host architecture to trigger retraining data logging or operator notification.

The conditional barrier result requires a validated control-affine plant model, bounded state and delay error, feasible actuator authority ($\mathcal{U}_{\text{safe}} \neq \emptyset$), and a sampled implementation whose measured tracking and execution errors fit the reserved margin. The 10 kHz current loop beneath the illustrative 1 kHz torque loop is one design to test; its nominal rate alone does not establish delivered-torque accuracy.



Napkin Math 24.3: Microcontroller CBF-QP execution limits

Design Problem: *Can an embedded microcontroller evaluate convex safety optimizations at kilohertz frequencies without missing hard real-time deadlines?*

Systems Analysis: In this constructed schedule, acquisition, validation, and a typical active-set solve occupy about 175–225 μs . That is a nominal timing illustration, not a full-cycle bound.

The qualified full path instead budgets up to 650 μs for the solver plus bounded acquisition, validation, write, and dispatch time within the 1.0 ms cycle; a timeout transfers authority before an overrun. For a clock-cycle-accurate worst-case execution time (WCET) analysis, see Appendix C.

In this illustrative heterogeneous design, the shield executes inside a 1000 Hz (1000 μs) timer interrupt routine on the real-time microcontroller core. The offsets below describe a nominal sequence; the separate full-cycle WCET must include every stage:

1. **Interrupt Entry & State Acquisition (0–15 μs):** Hardware timer triggers ISR; firmware reads raw optical encoder registers and ADC current shunts.

2. **Proposal & Lease Verification (15–25 μs):** The MCU inspects the shared SRAM ring buffer populated by the Linux MPU over RPMSG or shared memory. It validates the CRC32 checksum (verifying transport integrity) and authenticates the cryptographic MAC (verifying origin authenticity), confirms monotonic sequence order, and verifies that the intent lease has not expired ($t_{\text{age}} \leq 75$ ms, with renewal no more than 60 ms apart). This proposal lease limits command age under the stated clock-conversion and delivery bounds. A separate 100 Hz host heartbeat times out after 30 ms of silence. The MCU feeds its 5 ms self-watchdog only after successful control cycles; an external trip covers MCU loss of execution.
3. **Tracking Contract Verification (25–35 μs):** Firmware evaluates instantaneous tracking error $e(t) = \|x(t) - r(t)\|$ against the certified bound ϵ_{track} .
4. **CBF-QP Safety Shield Solve (35–150 μs):** An active-set QP solver running in pre-allocated static DTCM SRAM (leveraging the low-latency memory architecture detailed in section 8.2) evaluates active barrier inequalities and computes $u^* = \arg\min_u \frac{1}{2} \|u - u_{\text{nom}}\|^2$. Nominal solve latency spans 35–150 μs for 1-DoF and 2-DoF active constraints, with an assumed qualified solver WCET capped at 650 μs under maximum constraint iteration. Dynamic memory allocation (`malloc`) is strictly forbidden, as searching for free memory blocks introduces unpredictable delays and risks non-deterministic heap fragmentation that could cause the solver to miss its strict sub-millisecond deadline.
5. **Simplex Switch Arbitration (150–165 μs):** If the QP is feasible and constraints hold, u^* is written directly to the PWM timer compare registers. If the intent lease has expired, tracking error exceeds ϵ_{track} , or actuator saturation renders the feasible control set empty ($\mathcal{U}_{\text{safe}} = \emptyset$), the Simplex switch revokes the policy and selects the state-matched fallback. If the state is already outside the viable set, that fallback can limit harm but cannot restore a forward-invariance claim.
6. **Output Dispatch & Cycle Completion (165–225 μs nominal):** The MCU checks the register write and services its own watchdog only after the cycle succeeds. The 650 μs solver WCET shifts the later stages; their qualified total must still finish before the 1000 μs tick.

Checkpoint 24.5: CBF-QP projection vs. heuristic clamping

Consider the mathematical and physical differences between structured optimization filters and heuristic saturation:

- ☐ **Torque-space change versus path:** What does the minimum-norm projection ($\mathbf{u}^* = \mathbf{u}_{\text{nom}} - \lambda^* \mathbf{a}$) minimize, and why does it not by itself preserve the Cartesian path?
- ☐ **Multi-axis dynamic coupling:** Can you describe how clamping individual joint torques independently can fail to enforce a coupled safety barrier, allowing the physical state to cross into forbidden territory despite local saturation?
- ☐ **Sub-millisecond solver determinism:** Can you distinguish between the computational requirements of an active-set quadratic program in static SRAM and a general numerical optimizer, explaining why dynamic heap allocation is strictly prohibited in real-time safety shields?

24.9 The Fallback Ladder

The fallback must be selected before the admissible set becomes empty. If it is already empty, escalation can contain a fault or limit harm but cannot promise to avoid a boundary crossing. A physical machine cannot treat safety as a single binary switch. If an enforcer cuts bus power whenever an upstream proposal brushes against a barrier margin, the system suffers uncoordinated deceleration, thermal cycling, and continuous mission aborts. If it instead relies on software optimization when an inverter bridge experiences a shoot-through current (section 4.4), the gate drivers burn before an operating system can schedule an interrupt handler. The nervous system resolves this tension through a deterministic escalation ladder, from feasible mathematical projection to plant-specific controlled stopping or torque removal.

Echoing the layered competence hierarchies of behavior-based robotics and the subsumption architecture (Brooks 1986b), the **four-tier fallback ladder** is a graded, deterministic escalation hierarchy of defensive actions (Level 1: Minimal-Intervention QP Projection; Level 2: Active Position Hold; Level 3: Controlled Dynamic Stop; Level 4: Drive Torque Inhibit) that transitions across independent execution substrates as control authority, solver feasibility, or hardware invariants degrade.

The first rung of the fallback ladder is least-squares projection, executed on every control cycle while the feasible control set remains non-empty ($\mathcal{U}_{\text{safe}} \neq \emptyset$). When a nominal proposal u_{nom} points outside the admissible half-spaces, the enforcer computes the minimal-norm modification $u^* = \arg \min_u \|u - u_{\text{nom}}\|^2$ subject to the active barrier inequalities. In the constructed 1000 Hz loop, a $175 \mu\text{s}$ solve is nominal; the admitted full-cycle worst-case budget is larger. Projection supports forward invariance only from a viable state under the validated model, measurement, actuation, and deadline bounds. The projection succeeds only while the intersection of barrier constraints and achievable actuator torques contains a solution and the system remains in the validated viable region. The admission path must preserve viability before saturation makes $\mathcal{U}_{\text{safe}} = \emptyset$. If infeasibility occurs anyway, the MCU revokes the proposal and invokes the best available state-matched response without claiming restored invariance.

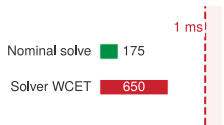
One possible second response is a controlled deceleration into an energized position hold, analogous to a Category 2 stop.¹⁴ Rather than attempting to navigate along the planned trajectory, the nervous system commands the actuators to arrest motion and maintain the current coordinate x_{hold} under closed-loop feedback. Inverter gate drivers remain actively modulated by pulse-width modulation, driving stator currents to generate the reaction torques needed to resist gravitational load and payload inertia. This rung costs electrical energy and generates continuous Joule heating ($I^2 R$) across the motor windings, accumulating thermal debt against actuator limits (section 4.5), but it incurs zero friction wear on mechanical brake pads and leaves the kinematic state fully preserved. After a fresh proposal and state check, the enforcer may resume without homing if the plant and position estimate remained valid. If the proposal lease expires or tracking exceeds its bound, the MCU selects a validated stop; holding remains appropriate only while available torque and feedback support it.

A third response is a controlled dynamic stop, analogous to a Category 1 stop under IEC 60204-1. In this illustrative regenerative drive, the nervous system overrides task goals and commands a validated braking profile, using the energy path detailed in section 4.6 with $a_{\text{brake}} = a_{\text{max}}$. For an arm operating at a linear velocity of 2.0 m/s with an assumed validated minimum braking acceleration of $a_{\text{max}} = 4.0 \text{ m/s}^2$ and an actuation response delay of 12 ms , stopping requires a deterministic clearance budget of $d_{\text{stop}} = v\tau + v^2/(2a_{\text{max}}) = 0.524 \text{ m}$. As counter-torque slows the moving mass, mechanical kinetic energy regenerates through the inverter bridge into the DC bus capacitors.¹⁵ The power stage must absorb this electrical surge through sized bus capacitance and dynamic brake chopper resistors to prevent DC-bus overvoltage faults. The profile must be checked for torque, regenerated energy or mechanical dissipation, and stopping clearance before use. Power removal, if specified, follows the measured standstill condition.

A separate last-resort drive function can inhibit motor-generated torque. Drive Safe Torque Off (STO) does not itself disconnect the DC bus, apply a mechanical brake, or stop a moving motor; the motor may coast. In the illustrated architecture, an external safety controller may also command a separately rated contactor and spring-applied brake. Their response and stopping ability must be measured as one plant-specific path. Removing torque while a gravity-loaded arm or moving vehicle still depends on powered braking can worsen the hazard.

Crucially, physical AI systems engineers must distinguish between **fail-safe** and **fail-operational** plants when configuring terminal fallbacks. In a machine already at a mechanically restrained standstill, removing drive torque may be safe after the restraint has been verified. In underactuated, open-loop unstable, or high-momentum machines, however, suddenly de-energizing actuators is itself a catastrophic hazard:

1. **Aerial Platforms (Multirotors and eVTOLs):** Cutting motor torque drops aerodynamic thrust to zero, transforming an airborne robot into a ballistic projectile in uncontrolled free fall.
2. **Dynamic Legged Systems (Bipedal Humanoids):** During dynamic locomotion at 1.5 m/s , the center of mass is outside the base of support. Abruptly removing the torque needed for



Illustrative nominal solve: 175 microseconds. Qualification budgets the full cycle and a longer solver worst case.

¹⁴ | **Machine stop functions:** IEC 60204-1 distinguishes uncontrolled power removal (Category 0), controlled stopping followed by power removal (Category 1), and controlled stopping with power retained (Category 2). IEC 61800-5-2 drive functions such as STO, SS1, and SS2 may implement parts of those responses. The required performance level, circuit architecture, and suitable stop are determined by the machine-specific risk assessment; ISO 13849-1 does not impose one universal PL or channel count.

¹⁵ | **Dynamic brake choppers and DC-bus overvoltage clamping:** In this illustrative regenerative Category 1 stop, mechanical kinetic energy flows through the inverter bridge into DC bus capacitors ($\Delta E = \frac{1}{2}mv^2$). A suitably rated chopper and resistor can dissipate energy when the DC bus rises; switching thresholds, pulse-energy rating, and thermal behavior must be validated for the stop profile. Other designs use mechanical dissipation or a battery that can accept regeneration.

Prerequisite: Hardware trip zones and low-side MOSFET dynamic shunt braking are introduced in section 8.5.

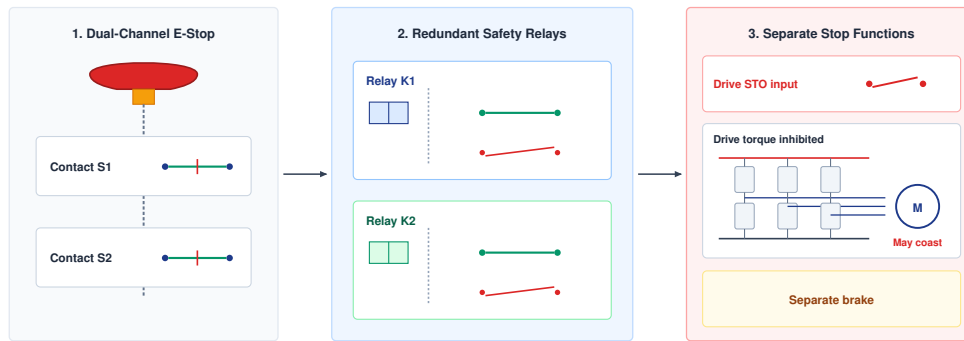


Figure 24.5: Illustrative emergency-stop circuit: Two emergency-stop contacts feed safety relays and drive STO inputs. A spring-applied brake is shown as a separate optional path; an external supply contactor, if fitted, is another independent function. The diagram specifies no response time, performance level, or stopping distance. A moving motor can coast after STO, so the physical stop must be validated for the load and state.

balance can cause a fall; the direction and consequence depend on the gait phase and available restraint.

3. **High-Speed Autonomous Vehicles:** Removing powered steering or braking at highway speed can eliminate the authority needed for a controlled stop; the outcome depends on vehicle dynamics and backup actuation.

For such dynamic systems, the terminal fallback cannot simply be an uncoordinated power cut. Instead, the nervous system must execute a deterministic **Minimal Risk Maneuver (MRM)** running on an independent, qualified controller with the required sensing and actuator authority. For aerial and legged robots, any controlled descent or balance recovery must be validated for remaining propulsion, contact, energy, and state-estimate authority. A road vehicle needs a risk-assessed powered braking and steering path. Drive STO is requested only when a risk assessment shows that torque removal is safe in the current state; any supply isolation or brake engagement is a separate, checked action. The wiring that carries such a request is deliberately plain. Figure 24.5 traces two emergency-stop contacts through safety relays into the drive's torque-inhibit input and keeps the spring-applied brake on its own path, because removing torque and arresting a moving load are different functions carrying different validation evidence.

The rungs trade intervention time, available control authority, and recovery cost. A controlled stop requires reserved clearance and a working drive; an STO request removes torque and may leave a moving machine coasting. The plant-specific response must be validated for the state at which each rung can be entered.

The deterministic escalation hierarchy across these four defensive tiers is structured in figure 24.6. The diagram orders possible responses by fault condition; it is not a guarantee that each lower rung stops faster or protects a moving plant better.

🔒 War Story 24.1: Cruise post-impact fallback (2023)

Context: In October 2023, a human-driven vehicle struck a pedestrian into the path of a Cruise robotaxi in San Francisco (Exponent, Inc. 2023).

Mechanism: The Cruise vehicle initially braked and stopped. Its system then misclassified the event as a side impact and initiated a pullover about 1.83 s after contact while tracking of the pedestrian was intermittent. The vehicle dragged the pedestrian roughly 20 ft at a maximum reported speed of 7.7 mph.

Response: An independent impact and underbody-occupancy response should revoke ordinary policy authority, retain braking and steering needed for a controlled stop, and hold the

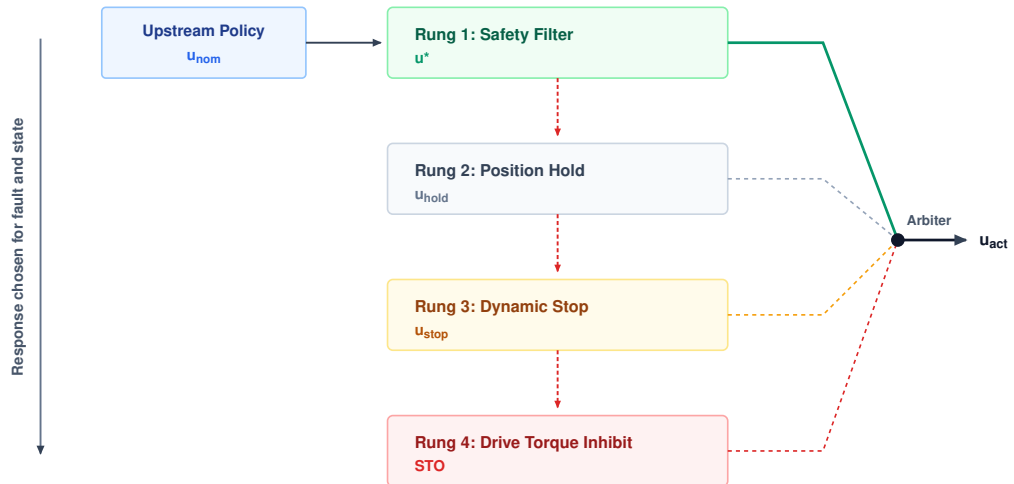


Figure 24.6: Available enforcement responses: A deterministic arbiter can select feasible torque projection, controlled hold, controlled stop, or drive torque inhibit according to the current fault and measured plant state. The responses are alternatives with different prerequisites; descending the diagram does not imply increasing physical safety or faster arrest.

vehicle once stationary. Safe Torque Off is appropriate only when removing drive torque leaves that vehicle in a validated safe state.

Systems lesson: A post-contact maneuver cannot infer clear space from a missing tracked object. The fallback must use independent evidence and preserve the ability to brake and hold.

The stop categories describe power and control behavior, not a universal sequence of safer rungs. Table 24.2 separates the drive function from the plant response that must be tested.

Table 24.2: Stop functions and physical response: Illustrative mapping of stop behavior. No stop category alone supplies a safety integrity rating, a brake, galvanic isolation, or a guaranteed stopping distance.

Response	Motor torque and power	Physical stop obligation	Appropriate use
Admitted filtering	Drive remains active	Recheck the feasible kinetic barrier and tracking bound every tick	Nominal operation inside the validated envelope
Controlled hold (Category 2 / SS2-like)	Power remains for braking and hold	Verify stopping distance, thermal headroom, and static load support	Hold when feedback and torque authority remain available
Controlled stop then torque removal (Category 1 / SS1-like)	Power remains through deceleration; torque is removed afterward	Verify the full braking and transition time under load	Moving machine with enough reserved clearance
Torque removal (Category 0 / STO-like)	Drive cannot produce commanded torque; DC bus isolation is a separate function	A moving motor may coast; a separate brake or mechanical restraint needs its own timing and rating	State for which loss of torque has been shown safe

Every rung of the ladder assumes that the hardware executing the fallback remains independent of the fault that triggered the escalation. If an enforcer shares its power rail, crystal oscillator, or memory bus with the host processor producing the unsafe commands, a single electrical disturbance can compromise both the nominal planner and the defensive fallback. A deterministic state machine in firmware cannot provide safety if the physical substrate executing it lacks genuine isolation.

Checkpoint 24.6: Choosing the physical fallback

- ☐ **State-dependent stop:** Why can powered braking be safer than STO for a moving AMR, while drive torque inhibit may be appropriate after a separately verified restraint holds a stationary arm?
- ☐ **Independent functions:** What does STO inhibit, and which separate functions would remove bus power or apply a spring brake?
- ☐ **Timing and clearance:** Which measured detection, dispatch, drive, and brake delays belong in the stop reservation?
- ☐ **Lost viability:** What can a fallback still accomplish when the admissible torque set is empty?

24.10 How Enforcement Goes Wrong

A secondary processor alongside the primary compute module creates an illusion of architectural independence that physical packaging undermines. When a machine hosts a trajectory planner on a high-throughput system-on-chip and an enforcement monitor on a real-time microcontroller, both chips almost always share a common circuit board. That shared board introduces common-mode coupling across power distribution networks, clock sources, and shared copper ground planes. If a single 5.0 V to 3.3 V step-down regulator powers both processors, an electrical short, transient latch-up, or thermal shutdown on the host side drops the supply rail below the microcontroller operating threshold of 2.7 V, resetting both chips simultaneously. Similarly, routing clock lines from a single crystal oscillator or distribution tree creates a shared failure point where a crystal fracture or line ringing halts both execution pipelines on the exact same cycle. High return currents from pulse-width-modulated motor drivers elevate local potential across the digital ground plane, inducing ground bounce that shifts reference potentials and corrupts logic thresholds on shared board-level communication buses such as I2C and SPI. These hardware failure modes are invisible to software running on the chips. A microcontroller brownout detector can pull a reset line when rail voltage collapses, but if the reference supply to the detector itself drops, the enforcer fails silently without executing its defensive routine.

Even when physical isolation across silicon and power supplies is complete, an enforcer fails if its computational delay falls out of phase with plant dynamics, causing the control action to reflect a stale historical state rather than current physical coordinates. A safety filter evaluates admissibility by projecting state trajectories or evaluating control barrier functions at each discrete control tick. If the calculation budget is allocated 1.0 ms within a 1000 Hz control loop, but complex boundary geometry or numerical conditioning issues push solver execution to 12.0 ms, the resulting command arrives eleven cycles late. For an articulated link moving at an angular velocity of 3.0 rad/s, a 12.0 ms delay represents an unmodeled angular displacement of 0.036 rad, or approximately 2.1°. When the enforcer finally asserts an emergency braking torque computed for the earlier state, that torque acts against physical coordinates the machine has already passed. In an underdamped mechanical assembly, an intervention executed out of phase with plant velocity injects energy into the structural resonance instead of damping it, accelerating the body toward the obstacle the filter was constructed to avoid. A hardware watchdog or real-time timer interrupt detects the deadline overrun and aborts the cycle, but it cannot undo the physical momentum accrued during the overrun period.

An enforcer cannot observe physical mechanics directly; it observes digital state estimates reconstructed from sensor data. When the sensing pipeline introduces phase lag, calibration drift, or undetected mechanical slip, the enforcer becomes blind to boundary violations occurring in the physical world. Consider an optical encoder measuring motor shaft angle across an elastomeric timing belt. If dynamic belt stretch under high torque produces 4.0 mm of linear compliance, or if the drive pulley slips by several gear teeth during an aggressive acceleration transient, the joint state reported to the enforcer remains within allowable geometric limits while the physical link has breached a virtual wall. Low-pass filters applied to eliminate high-frequency sensor noise introduce group delay—an unavoidable time lag where the smoothed signal trails behind reality. A second-order filter with a 20 Hz cutoff frequency delays the velocity estimate by approximately

15 ms. If an unexpected contact begins at $t = 0$ ms, the filtered velocity signal does not cross the over-speed fault threshold until impact forces have already peaked and loaded the mechanical transmission. Kinematic cross-validation against redundant absolute encoders or joint torque sensors can detect large structural discrepancies, but slow thermal bias drift in inertial measurement units and sub-threshold mechanical slip remain undetected until physical contact occurs.

A common engineering misconception equates deterministic software execution with system safety. Determinism guarantees that a computation finishes in an exact number of clock cycles and produces bit-identical outputs for identical inputs, but it provides no guarantee regarding the physical validity of the underlying equations. A real-time operating system executing a control barrier filter on an isolated core with negligible scheduling jitter is fully deterministic. Yet if the plant model programmed into that filter assumes a nominal friction coefficient of $\mu = 0.80$ when oil contamination on the rail has dropped the actual friction to $\mu = 0.15$, the enforcer calculates an analytical stopping distance of 0.40 m for a deceleration maneuver that physically requires 2.13 m of travel. The enforcer executes its intervention with microsecond precision, asserts the calculated braking current on schedule, and deterministically crashes the carriage into the mechanical end-stop. When paired with an unmodeled payload mass, an inaccurate inertia tensor, or an uncalibrated force constant, determinism simply guarantees that the failure is perfectly repeatable. Open-loop timing monitors cannot detect model inaccuracy, which reveals itself only when closed-loop tracking error diverges in downstream physical states.

Even when algorithms are verified and control threads are scheduled deterministically, architectural resource contention within the underlying SoC interconnect routinely defeats safety enforcement. Modern physical AI SoCs route high-bandwidth data streams—such as neural network weights, camera frame buffers, and real-time motor commands—across shared multi-layer bus fabrics (such as ARM CoreLink AXI or AHB crossbar matrices). When high-throughput hardware peripherals initiate bulk direct memory access (DMA) transfers, they compete directly with real-time motor peripherals for memory bus access. For example, when an image sensor processor (ISP) streams raw high-resolution video frames or a background black-box diagnostic thread executes an 8 KB burst write to non-volatile flash memory, an unweighted round-robin DMA arbiter allows the bulk logging traffic to saturate the bus matrix. This **DMA memory contention** saturates available bus bandwidth and starves the real-time SPI or CAN-FD channels responsible for transmitting updated torque commands to motor inverters.¹⁶ Although the microcontroller CPU core may finish solving its barrier quadratic program well within its 150 μ s deadline, the resulting command word remains stalled at the bus interface. During this **bus starvation** dead-time, the motor inverter either retains its last commanded torque setpoint or trips an uncommanded communication fault, allowing mechanical momentum to carry the machine across the safety boundary before physical braking can engage.¹⁷

At the operating system and middleware layers, runtime enforcement is equally vulnerable to **priority inversion** and synchronization hazards. In embedded architectures where real-time control routines execute under a real-time operating system (RTOS) or Linux PREEMPT_RT kernel, enforcement tasks share synchronization primitives with diagnostic logging and communication daemons. Priority inversion occurs when a low-priority thread (such as an asynchronous telemetry serializer) acquires a shared mutual exclusion lock (mutex) guarding a shared SPI peripheral or memory buffer, and is subsequently preempted by an unrelated medium-priority thread (such as an unprivileged Ethernet networking stack or garbage collection daemon). When the high-priority 1000 Hz reflex enforcer interrupt service routine executes and attempts to acquire the locked mutex, it is blocked. Because the medium-priority thread prevents the low-priority thread from running and releasing the mutex, the high-priority safety enforcer remains indefinitely stalled. Without strict priority inheritance protocols (PTHREAD_PRIO_INHERIT) or ceiling priority locking, the enforcer misses its hard real-time execution deadline, failing to refresh the intent lease or assert corrective braking before mechanical limits are violated.

To prevent catastrophic blind spots, physical AI systems engineering requires auditing runtime enforcement against a comprehensive **failure taxonomy**:

1. **Substrate and Packaging Coupling:** Common-mode electrical failures where high return currents from PWM motor switching induce ground bounce, shared DC-DC converters suffer

¹⁶ | **Direct memory access bus contention and telemetry starvation:** On SoC interconnects (such as ARM AXI or AHB crossbar matrices), unweighted round-robin arbitration allows bulk telemetry dumps to saturate available bus bandwidth. This contention blocks high-priority SPI DMA transfers destined for motor drive digital-to-analog converters, introducing multi-millisecond command latencies that breach stopping boundaries even while processor cores remain completely unburdened.

¹⁷ | **Hardware safety interlock isolation:** An independently wired trip path avoids the illustrated shared-bus scheduling failure. The safety case must also test common power, wiring, and actuator faults; IEC 61508 and ISO 13849 do not impose one universal physical partition or stop response.

transient brownouts, or shared crystal oscillator fractures halt both cognitive planning and reflex enforcement simultaneously.

2. **Interconnect and Memory Contention:** Hardware bus saturation where bulk camera DMA ingestion or background flash logging starves real-time actuator communication channels at the crossbar matrix.
3. **Scheduling and Concurrency Hazards:** RTOS priority inversion, unbounded kernel preemption latency, or mutex deadlocks that cause safety enforcement tasks to miss microsecond timer deadlines.
4. **State Observability and Feedback Latency:** Unmodeled group delay in low-pass filtering, mechanical transmission compliance (such as timing belt stretch or gearbox backlash), encoder slip, and thermal sensor bias drift that mask true boundary violations.
5. **Model and Environmental Mismatch:** Actuator saturation, thermal derating (I^2R heating), sudden friction coefficient collapse (such as wet or oily surfaces), and unmodeled payload mass that invalidate forward-invariant barrier certificates.

⚠ War Story 24.2: Therac-25 software interlock elimination

Context: The AECL Therac-25 was a dual-mode (25 MeV electron / X-ray) medical linear accelerator and automated radiotherapy patient table, controlled by a single PDP-11 minicomputer running uniprocessor assembly firmware (Leveson and Turner 1993).

Mechanism: In preceding models (Therac-6 and Therac-20), physical safety was guaranteed by independent hardware interlocks: electromechanical microswitches, mechanical turntable detents, and analog diode steering circuits that physically prevented the accelerator from firing high-current beams unless the tungsten beam-flattening target was mechanically locked in place. In the Therac-25, engineers removed all physical hardware interlocks to reduce manufacturing costs, delegating exclusive safety enforcement to software. Concurrently, two latent software bugs defeated the software checks: an 8-bit shared variable (C1ass3) rolled over from 255 to 0 (0xFF → 0x00) during setup, bypassing interlock evaluation; and a race condition allowed fast terminal keystrokes (< 8 seconds) to update operating modes without reconciling the physical turntable position with the commanded high-current beam energy.

Impact: Between 1985 and 1987, six patients received massive radiation overdoses up to 25,000 rad (more than 100 times the intended therapeutic dose), resulting in catastrophic radiation burns, severe physical trauma, and several patient fatalities.

Response: Regulatory agencies halted machine operations, prompting comprehensive investigations that led to modern medical device software engineering standards, mandatory independent hardware safety interlocks, and formal verification of safety-critical concurrent states.

Systems lesson: Software-only safety filters executing on complex, non-deterministic compute stacks cannot replace independent bare-metal hardware trip zones. The Therac-25 disaster is the canonical historical proof that removing physical interlocks in favor of software decision logic transforms transient concurrency bugs and arithmetic overflows directly into catastrophic kinetic and physical energy releases. For the illustrated physical AI design, an independent controller and a device-qualified hardware trip path can revoke software authority without waiting on an application scheduler. A configured TIMx_BKIN input can force PWM outputs to a specified safe state; drive torque removal and physical stopping time must be measured separately.

Every failure mode in the enforcement layer traces back to a breakdown of independence or a divergence between the internal plant representation and physical reality. The nervous system cannot treat its safety claims as unconditional truths. A mathematical barrier certificate, a dual-core silicon package, and a zero-jitter real-time scheduler establish safety only within the bounded envelope where power supplies remain isolated, state estimates match physical mechanics, and execution deadlines are met without exception.

24.11 Enforcer Telemetry

The **enforcement record** is the immutable specification contract and runtime audit ledger that defines the physical safety boundary for system integration, deployment, and task placement. The record explicitly establishes what the permission path guarantees: verified tracking error bounds (ϵ_{track}), stopping envelopes ($t_{\text{stop}}, d_{\text{stop}}$, bounding the maximum physical duration and displacement required to arrest plant momentum), Control Barrier Function parameters, discrete fallback escalation thresholds, and certified environmental operating assumptions. This record appends temporal, spatial, and dynamic fields unique to closed-loop force regulation and physical intervention.

The specification contract grounds its guarantees in four primary quantified parameters: the verified tracking error bound ϵ_{track} , the maximum stopping time t_{stop} , the barrier function coefficients, and the loop period T_{loop} . Consider an analytical example of a linear gantry stage that regulates contact force before translating a payload. The nervous system executes its inner regulation loop at a fixed period of $T_{\text{loop}} = 1.0$ ms (1000 Hz). In this constructed record, the permission inset assumes a separately validated transient bound $\epsilon_{\text{track}} = 5.0$ mm over the declared load and disturbance envelope. A measured $P_{99.99}$ tracking error of 4.2 mm would support evaluation but would not establish that bound. If the safety filter intervenes, the downstream braking mechanism brings the carriage to rest from its maximum allowable velocity of 1.5 m/s within $t_{\text{stop}} = 85$ ms, traversing an analytical stopping distance of 0.064 m under peak deceleration. When the intervention relies on a control barrier function $h(x) \geq 0$ with class- $\mathcal{K}$ linear parameter $\alpha = 12.0$ s⁻¹ (specifying the rate of decay of allowable approach velocity near the safe boundary), the record binds these exact values directly to the physical plant model. Upstream trajectory generators cannot treat the enforcer as an unconstrained safety net, because they must budget a minimum spatial clearance of $\Delta x_{\text{margin}} \geq \epsilon_{\text{track}} + v_{\text{max}} t_{\text{stop}}$ to prevent an intervention from contacting a physical limit.

Each numerical entry in the record carries a provenance tag indicating whether the bound originates from analytical derivation, physics simulation, or hardware stress testing. These three sources offer vastly different levels of real-world confidence. An analytical derivation based on rigid-body equations provides a mathematical baseline, such as an ideal stopping time of $t_{\text{stop}} = 62$ ms that assumes instantaneous current reversal and constant traction. A multibody physics simulation sweeping parameter variations across fifty thousand runs might incorporate thermal winding degradation and calculate a $P_{99.9}$ tail of $t_{\text{stop}} = 78$ ms. Direct hardware injection testing on the physical rail under cold-temperature lubricant conditions reveals contactor bounce and electromagnetic latching latency. A $t_{\text{stop}} = 85$ ms bound would require a justified worst-case fault and operating envelope; a measured percentile alone cannot establish it. Documenting provenance prevents an integrator from mistaking an uncalibrated simulation boundary for an empirically validated hardware invariant.

The record maps separately qualified thresholds to protective actions. Escalation does not rely on coarse error strings or unquantified health flags. In our analytical positioning stage, Level 1 nominal operation permits quadratic programming (QP) filter adjustments as long as tracking error satisfies $e(t) \leq \epsilon_{\text{track}} = 5.0$ mm and optimization solver latency remains below $T_{\text{solve}} \leq 650$ μ s. If tracking error diverges into the intermediate window between 5.0 mm and $\epsilon_{\text{crit}} = 8.0$ mm, or if the full-cycle deadline is at risk, the system escalates to Level 2, overriding policy proposals with a deterministic safe setpoint that smoothly decelerates the robot along a precomputed trajectory. If tracking error exceeds $\epsilon_{\text{crit}} = 8.0$ mm or the kinetic barrier reaches zero, the controller must use the prevalidated controlled stop while clearance remains. Passive winding shunting is a separate design option whose torque decays with speed. The 5 ms MCU self-watchdog detects missed successful cycles; this design also assumes an independently wired supervisor output or external trip that requests drive torque inhibit if the MCU fails. Its total trip response must be measured. A 30 ms host-heartbeat timeout revokes upstream authority but does not itself justify power removal. A contactor, if fitted, is an independently timed supply-isolation function.

To formalize this specification contract across the dual-brain boundary, table 24.3 defines the abstract data schema for the safety invariant envelope. Every field is bound to an explicit physical SI unit, a real-time validation invariant, an independent hardware verification substrate, and a deterministic fallback escalation tier.

Downstream: High-rate enforcer refusal events populate the forensic evidence logs analyzed in section 28.8.

Table 24.3: Abstract data schema for the real-time safety invariant envelope (SafetyInvariantEnvelope): Specification contract layout (68 bytes total), binary data types, physical SI units, mathematical verification invariants, hardware audit substrates, and deterministic escalation tiers governing real-time safety enforcement across the proposal-permission boundary.

Schema Field Identifier	Binary Type & Layout	Physical SI Units	Verification Invariant	Hardware Audit Substrate	Fallback Escalation Tier
seq_id	uint64_t (8 B)	Monotonic counter	$s_k = s_{k-1} + 1$	MCU DMA Packet Parser	Level 2 (Hold on gap)
timestamp_us	uint64_t (8 B)	Microseconds (μs)	MCU-clock age ≤ 75 ms after bounded conversion	Hardware Timer 64-bit	Level 2 (age > 75 ms)
pos_tracking_bound	float32_t (4 B)	Meters (m)	$ x_{\text{meas}} - r_{\text{cmd}} \leq \epsilon_{\text{track}}$	Dual Optical Encoders	Level 2 / Level 3
vel_ceiling_limit	float32_t (4 B)	Radians/sec (rad/s)	$ \dot{q} \leq \dot{q}_{\text{max}}$	Timer Input Capture	Level 1 (QP Clamp)
torque_slew_ceiling	float32_t (4 B)	N · m/s	$ \dot{\tau} \leq \dot{\tau}_{\text{max}}$	Inverter Current Loop	Level 1 (Slew Limiter)
cbf_barrier_margin	float32_t [K] (24 B)	Normalized distance	$h_k(\mathbf{x}) \geq \delta_{\text{margin}, k}$	Active-Set QP Engine	Level 1 / Level 3
actuator_curr_limit	float32_t (4 B)	Amperes (A)	$ I_{\text{phase}} \leq I_{\text{peak}}$	ADC Current Shunts	Level 3 (Dynamic Brake)
dc_bus_overnvolt_thresh	float32_t (4 B)	Volts (V)	$V_{\text{bus}} \leq V_{\text{max_safe}}$	Analog Comparator (COMP1)	Level 3 (Brake Chopper)
watchdog_timeout_us	uint32_t (4 B)	Microseconds (μs)	MCU feeds only after successful cycle; timeout 5 ms	Silicon Windowed Watchdog	External fault response (drive inhibit)
crc32_checksum	uint32_t (4 B)	CRC32-IEEE 802.3	$\text{CRC}(\text{payload}) == \text{checksum}$	Hardware CRC Accelerator	Level 2 (Reject packet)

The distinct dispatch and fallback timers appear in table 24.4. The schema records what the MCU checks; the state table records which authority path remains active after a failed check.

Table 24.4: Illustrative enforcement state contract: The 20 Hz proposal, 75 ms lease, 100 Hz host heartbeat, and 1 kHz MCU loop have distinct renewal and failure paths. Timing and physical arrest limits require configuration-specific validation.

State	Trigger and authority	Timing check	Physical response to validate
STATE_INLINE_SHIELD	Fresh proposal, healthy heartbeat, feasible kinetic barrier	1 ms cycle; assumed solver WCET $650 \mu\text{s}$ plus bounded acquisition and dispatch	Track admitted torque within qualified error and brake reserve
STATE_AUTONOMOUS_HOLD	Proposal age > 75 ms, host heartbeat silent > 30 ms, or failed validation	Transfer before reserved stopping delay expires	State-matched powered hold or controlled stop if clearance permits
STATE_CONTROLLED_STOP	Tracking or model envelope fails while powered braking remains available	Include detection, dispatch, drive rise, and brake response	Measured stop distance under load; no claim after viability is lost
STATE_DRIVE_INHIBIT	Independent wired supervisor detects MCU cycle loss or a separate electrical trip	Assumed 5 ms detection plus measured drive, contactor, or brake response	STO removes motor torque; motion may coast unless separately restrained

No enforcement guarantee holds when the machine operates outside its designed physical environment, so the record bounds the plant dynamics within an explicit operating envelope. For the gantry stage, the record specifies a Coulomb friction coefficient bounded by $\mu \in [0.65, 0.90]$, a total payload mass of $m_{\text{load}} \in [0.0 \text{ kg}, 5.0 \text{ kg}]$ attached to a carriage mass of 12.0 kg, and an ambient operating temperature range between 5°C and 45°C . If water contamination drops rail friction to $\mu = 0.20$, or

if an unmodeled payload mass of 9.0 kg is mounted to the end-effector, the assumed actuator torque and kinetic-barrier model can become invalid if the reduced tractive force is insufficient for the reserved stopping clearance. The record exposes these constraints directly to supervisory monitors, which must revoke policy dispatch and trigger a controlled shutdown whenever sensors detect that physical parameters have crossed outside the certified envelope.

An enforcement record connects proposed motion to validated assumptions, thresholds, and observed fallback response. The execution platform must still meet its timing and physical braking contract under load. If an enforcer is mathematically verified but stalls on a saturated memory bus during a high-speed transit maneuver, the failure occurs in time rather than in logic.



Systems Perspective 24.4: High-relative-degree barrier functions

Enforcing Control Barrier Functions (CBFs) on a bipedal humanoid exposes the severe mathematical challenges of underactuated floating-base dynamics:

- **High Relative Degree of Balance Constraints:** Obstacle distance and balance constraints can have relative degree greater than one with respect to joint torque; the degree depends on the chosen state, contact mode, and actuator model. On a floating-base humanoid, an arm collision constraint and a center-of-mass or Zero Moment Point (ZMP) balance constraint may require different controlled barriers. Torque changes propagate through coupled contact forces and foot motion. A filter built from raw geometric distance alone can intervene after the viable recovery margin has been consumed.
- **Nonlinear Friction Cone Polytope Projection:** Bipedal balance requires contact forces to remain inside a validated friction approximation at the feet ($\sqrt{f_x^2 + f_y^2} \leq \mu f_z$). An aggressive torso or arm proposal can demand more shear force than the contact can provide. A whole-body filter can include joint-torque and contact constraints, but its solve time, contact estimate, and state-matched fallback must be measured for the platform; a nominal 1 kHz loop does not establish a universal 500 μ s recovery.
- **Minimal Intervention without Posture Collapse:** Minimizing $\|\mathbf{u} - \mathbf{u}_{\text{policy}}\|^2$ changes torque as little as the chosen norm allows while satisfying the supplied constraints. Because the joint-space inertia matrix $\mathbf{M}(\mathbf{q})$ couples links, a hand-collision correction can alter the ground reaction wrench. The design must test the resulting whole-body motion and retain a feasible balance response; torque-space proximity alone does not protect posture.



Checkpoint 24.7: Enforcement record and response audit

Use the record above to check one intervention from trigger to physical response:

- ☐ **Bound provenance:** Which values are analytical assumptions, measured tails, and separately qualified worst-case limits?
- ☐ **Timing contract:** Which timestamp, lease, heartbeat, solver, and dispatch thresholds cause the MCU to revoke a proposal?
- ☐ **Physical fallback:** What measured state and remaining clearance justify the selected braking or hold response?
- ☐ **Audit trail:** Which sensor observation, barrier value, proposed torque, admitted torque, and fallback transition should the event record retain?

24.12 Fallacies and Pitfalls

Physical enforcement ensures a machine stops before a collision or failure, relying on timely execution, usable state feedback, and sufficient actuator authority. Fault tests must establish how the safety

filter responds when one of those physical assumptions fails, rather than checking its nominal mathematical equations alone.

Pitfall: *Allowing a barrier QP solver to overrun its hard execution deadline in the hope of convergence.*

A gantry's barrier QP solver normally finishes in $420.0\ \mu\text{s}$, but PCIe bus jitter stretches one solve beyond its $800.0\ \mu\text{s}$ hard deadline. Waiting for the mathematics to converge leaves the previous, now-stale torque command active while the heavy carriage continues toward the mechanical end stop at $2.4\ \text{m/s}$. The real-time executive must enforce the timeout and assert the validated Category 1 stop already stored in local SRAM. That fallback braking maneuver also needs to fit within the remaining kinematic motion margin. Mean solve time cannot establish this safety guarantee. Qualification must exercise solver overruns and verify the complete timed transition from missed computation to physical deceleration.

Fallacy: *Control Barrier Functions remain valid when actuators enter full current or voltage saturation.*

When an external load drives a gantry stator to its $I_{\text{peak}} = 28.0\ \text{A}$ limit, Joule heating ($I^2 R$) induces rapid thermal derating. If the tracking controller keeps accumulating integral error while fighting the load, suddenly removing that load can release an excessive "wound up" forward command just as the machine needs to brake. A barrier calculation that assumes physically unavailable torque will not correct that behavior. The physical enforcement model must include back-EMF voltage limits and inverter current saturation, while the tracking controller actively prevents integral windup. When operating conditions reduce available control authority, the admissible SE(3) kinematic envelope must reflect that physical reduction. The safety claim relies on achievable physical actuator torque, not just the ideal command calculated by the filter.

Pitfall: *Assuming high-level policy computers can be relied upon to execute graceful shutdown.*

A linear stage moves at $v = 2.0\ \text{m/s}$ when the AI application processor suffers a kernel panic. The failed host cannot generate a deceleration trajectory, and merely dropping the motor enable signal to zero torque leaves the heavy carriage freely coasting under its own momentum toward its physical hard stops. The local motion controller must detect the lost heartbeat locally and independently execute the validated Category 1 stop deceleration response stored in its local flash memory. Both the heartbeat detection delay and the worst-case physical stopping distance belong in the available-clearance safety envelope. A host shutdown handler may coordinate an orderly stop during normal operation, but protection against host failure requires a separately validated stopping path, sufficient clearance, and actuator torque authority that remain available after the high-level process disappears.

Fallacy: *A mathematically correct safety filter guarantees safety even when underlying state feedback has drifted.*

Particulate contamination, such as dust or grease, biases a motion stage's optical encoder, making the carriage appear farther from the safety boundary than it physically is. The CBF QP solver can satisfy its mathematical inequalities on that drifted state estimate while dangerously delaying necessary braking on the physical machine. Comparing the encoder position with the integrated IMU response and redundant resolver sensing can expose the divergence, provided the fault is observable through those independent channels. Qualification must bound the filtering detection interval and the kinematic error that accumulates before switching feedback or engaging emergency braking. Enforcement depends entirely on the validity of its physical state estimate, so the ISO/TS 15066 safety margin must strictly account for sensor estimation error as well as network SerDes delays, solver latency, and actuator rise times.

Pitfall: *Coupling safety braking authority to stochastic semantic classification rather than deterministic spatial clearance.*

When braking depends on upstream classification and path prediction, a detected object may still fail to trigger an effective stop. In the March 2018 Tempe collision (figure 24.7), the developmental automated driving system first detected the pedestrian $5.6\ \text{s}$ before impact but did not predict her path. It recognized an imminent collision $1.2\ \text{s}$ before impact, suppressed action for one second, then planned gradual slowing rather than emergency braking; the NTSB does not establish a Kalman-filter reset on each classification change. The vehicle's original Volvo collision-warning and emergency-braking functions were disabled during ADS operation. A proposed independent

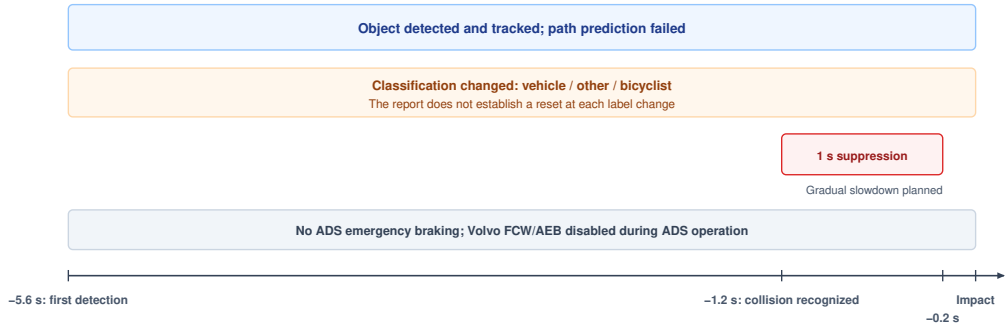


Figure 24.7: Tempe detection and braking timeline: The NTSB reports first detection 5.6 s before impact and recognition of imminent collision 1.2 s before impact. The ADS suppressed its motion plan for one second and then planned gradual slowing; it did not perform emergency braking. Classification and path prediction failed. The Volvo forward-collision warning and automated emergency braking functions were disabled while the developmental ADS operated. The report does not establish a Kalman-filter reset at each label change. (Adapted from NTSB/HAR-19/03 (National Transportation Safety Board 2019).)

geometric clearance check would need its own validated sensing, stopping authority, and timing; it is an architectural lesson, not a mechanism documented in that vehicle.

24.13 Summary

A physical AI system assigns actuator permission to a path independent of learned proposals. A kinetic barrier can help preserve a validated viable set when state error, model mismatch, actuation, and response delay stay inside their declared bounds. If braking authority or clearance is lost, the enforcer can revoke the proposal but cannot claim that a later fallback restores the original safe set.

The physical clearance protecting a machine is never a static geometric measurement. It is a composite safety margin that compounds feedback tracking bounds, transport latency, actuator saturation limits, and state estimation error. As an analytical example, consider a linear carriage translating at $v = 1.5 \text{ m/s}$ toward an end-stop. Four distinct physical realities compound to define the true stopping boundary. First, the current loop introduces a maximum tracking deviation $\epsilon_{\text{track}} = 4.0 \text{ mm}$. Second, the transport delay across the communication bus creates a $t_{\text{delay}} = 2.0 \text{ ms}$ lag, allowing the carriage to coast $d_{\text{delay}} = v \cdot t_{\text{delay}} = 3.0 \text{ mm}$ before the inverter even receives the brake command. Third, the motor's physical current limits cap the braking acceleration at $a_{\text{max}} = 5.0 \text{ m/s}^2$, dictating a kinematic stopping distance of $d_{\text{brake}} = v^2 / (2a_{\text{max}}) = 225.0 \text{ mm}$. Finally, the state estimator carries an unobservable residual uncertainty of $\epsilon_{\text{est}} = 3.5 \text{ mm}$. Under the stated worst-case bounds, a conservative required clearance is the sum of these terms: $d_{\text{clear}} \geq \epsilon_{\text{track}} + d_{\text{delay}} + d_{\text{brake}} + \epsilon_{\text{est}} = 235.5 \text{ mm}$. If an engineer budgets only the nominal stopping distance d_{brake} , the unmodeled transport lag, tracking jitter, and sensor uncertainty consume the remaining 10.5 mm of margin, and the carriage may reach the end-stop before braking completes.

A planner checks a future trajectory; a local kinetic barrier checks the current proposal against the modeled ability to stop. For torque-controlled motion, raw clearance has relative degree two. The velocity-dependent barrier in equation 24.4 produces the affine torque condition in equation 24.5 for positive approach speed. This local calculation still depends on a reserved stop, accurate state, and timely torque delivery.

The permission path needs an independent response when the proposal source stalls. In the constructed example, a 75 ms proposal lease, 30 ms host-heartbeat timeout, and 5 ms MCU self-watchdog detect different failures. The MCU revokes proposal authority and chooses a prevalidated state-matched response; torque removal is reserved for states where coasting or separate restraint is acceptable.



Key Takeaways: Conditional enforcement and physical stopping

- **Tracking is a bounded premise:** A measured steady-state error or percentile is evidence, while a clearance inset needs a justified transient bound in the operating envelope.
- **Stopping consumes clearance:** Response travel, braking distance under available deceleration, transient tracking error, and position-estimate error all enter the admission check.
- **The controlled barrier includes speed:** Raw distance has relative degree two for torque. A kinetic stopping barrier yields a torque half-space for positive approach speed, conditional on model, state, actuation, and timing.
- **Fallback is state dependent:** Feasible projection, powered hold, controlled braking, and drive torque inhibit have different prerequisites. An empty admissible set cannot guarantee a safe stop.
- **Independent clocks detect different failures:** Proposal age, host heartbeat, MCU cycle progress, and external trips are checked separately; no one timeout is a universal braking guarantee.

Safety enforcement is an architectural contract between a mathematical admission test and a physical response. When an autonomous system attempts to verify its own safety inside the same operating system thread running neural inference, any thread priority inversion or memory allocation stall disables the safety net at the exact instant of crisis. A viable permission path must retain authority to refuse proposals and execute its tested fallback within the reserved margin.



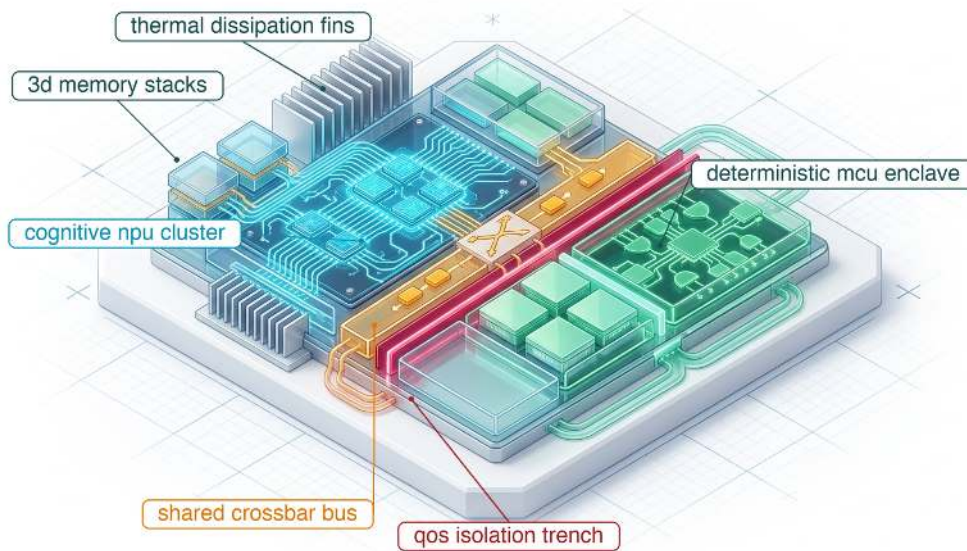
What's Next: From barrier certificates to heterogeneous SoC placement

A barrier result remains conditional on execution timing and the physical plant. In chapter 26, we transition from mathematical safety filtering to hardware isolation and heterogeneous SoC placement: examining how to partition cognitive deliberation and reflex enforcement across heterogeneous compute fabrics while testing whether memory starvation, clock throttling, or bus contention can delay permission or fallback.

Silicon Placement

DRAFT

Silicon Placement



- 26.1 The Silicon Substrate
- 26.2 Two Paths on One Die
- 26.3 Contention for Shared Resources
- 26.4 The Boundary Contract
- 26.5 Thermal and Timing Coupling
- 26.6 Testing the Separation
- 26.7 Silicon Isolation Decisions
- 26.8 Hardware Allocation
- 26.9 Fallacies and Pitfalls
- 26.10 Summary

Purpose

Why do the architectural boundaries drawn between brain and nervous system collapse the moment they share memory buses and power rails on the same die?

Blocks in an architecture diagram can compete for the same memory bus, power supply, and cooling capacity. A burst of neural computation may delay a control task even when the tasks run on different processor cores. The delay is critical as the machine moves while the safety path waits for data or recovers from a power disturbance. Compute placement must therefore establish how much interference each shared resource permits and whether the remaining response time fits the physical deadline. Software privilege boundaries alone cannot answer that question. Measurements under contention can expose a failed isolation claim, while resource reservations or separate hardware can reduce the coupling that caused it. Those choices also consume space and power that the machine must supply. Placing the brain and nervous system on silicon is a co-design decision about workload interference, execution deadlines, and tolerance for delayed correction.

Brain
1–50 Hz
foundation models, SLAM, planning

above: propose; below: command

Nervous System
1000 Hz
TSN, intent leases, lock-free IPC

Body
continuous
real-time MCU, safety filters, CBFs

Boundary
continuous
sensors, actuators, kinematics

Physical World
continuous
contact mechanics, friction, mass



Learning Objectives

- Inventory shared hardware resources used by learned proposal and safety enforcement paths
- Measure enforcement latency while competing workloads stress shared memory, power, and thermal resources
- Design tests that could disprove a claimed separation between proposal and enforcement
- Analyze how fluctuations in power supply (power transients) and heat distribution (thermal coupling) can affect the timing margins of safety-critical systems, leading to potential timing violations in real-time applications.
- Select isolation remedies using measured interference, resource availability, and the plant's response deadlines

26.1 The Silicon Substrate

RUNTIME safety filters, Control Barrier Functions, simplex architectures, and fail-operational fallback maneuvers can establish conditional invariance under plant, sensing, feasibility, and timing assumptions. One such assumption is that the computing machinery evaluates its quadratic program or projection filter strictly within its hard physical sampling deadline ($T_{\text{enforce}} \leq 1.0$ ms). Where do these algorithms actually execute on the physical machine? On a software architecture diagram, the boundary between the Cognitive Deliberation Tier and the Deterministic Nervous System is cleanly drawn as a neat, dashed privilege boundary. On physical silicon, that boundary collapses onto a single shared die. Consider a modern heterogeneous System-on-Chip (SoC) governing an autonomous mobile robot or industrial bypass valve. A learned vision-language-action policy executes on the embedded GPU or neural accelerator, while a deterministic safety enforcer executes on an adjacent real-time microcontroller core. When the neural policy initiates an inference pass across a multi-billion parameter model, billions of transistors activate simultaneously. This massive, instantaneous current surge (dI/dt) induces an inductive and resistive voltage droop ($\Delta V = L \cdot dI/dt + IR$) across the chip's internal Power Distribution Network (PDN), risking clock instability and timing closure failures in adjacent real-time logic.¹ Concurrently, dense tensor matrix multiplications saturate the shared LPDDR memory bus, streaming gigabytes of activation maps and evicting the safety enforcer's pinned sensor tables from the last-level cache.

When a real-time safety reflex reads a sensor buffer under a 1.0 ms control deadline, its requests can queue behind direct memory access traffic. The worked transaction-level example below shows how a subdeadline can fail under stated assumptions. The enforcer misses the modeled 120 μs sensor-read-plus-compute subdeadline because the proposal workload occupies the shared memory controller. The full sensor-to-actuator deadline still needs separate measurement; the missed subdeadline shows that logical privilege alone does not establish the required timing separation.

This chapter addresses the physical foundation of the Part III execution pipeline, highlighted in figure 26.1. All preceding software contracts—the sensor timestamps of chapter 16, the decaying spatial beliefs of chapter 18, the expiring intent leases of chapter 20, the continuous action chunks of chapter 22, and the reflex enforcers of chapter 24—rely on the bedrock assumption that software execution deadlines correspond to physical reality.

This shared-silicon reality governs every domain of physical embodiment across all **Three Physical Machine Archetypes**:

- In **Mobile Systems (Class 1)**, an autonomous chassis co-locates multi-camera vision-language-action navigation models (Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023) on an embedded GPU with 1000 Hz steer-by-wire and emergency braking reflexes on an adjacent real-time core. A memory bus stall during feature extraction delays an obstacle-avoidance brake command, allowing moving kinetic mass to breach safety stopping margins.
- In **Manipulators (Class 2)**, an articulated robot arm executes high-dimensional diffusion grasping policies (20–50 Hz) alongside high-gain $SE(3)$ kinematics and joint torque loops (1000 Hz). If an accelerator tensor burst evicts safety state tables from the shared last-level

¹ | **Inductive Voltage Droop Mechanics:** In high-performance VLSI design, inductive voltage droop ($\Delta V = L \cdot dI/dt$) occurs across the parasitic inductance (L) of power distribution package pins and on-chip power grids when massive numbers of logic gates switch simultaneously. If the local rail voltage drops below the minimum operating threshold ($V_{\min}$), digital flip-flops suffer setup-time violations, metastabilities, or brownout resets that disrupt deterministic real-time execution.

Prerequisite: Hardware execution cadences and microcontroller timing loops were defined in section 8.3.

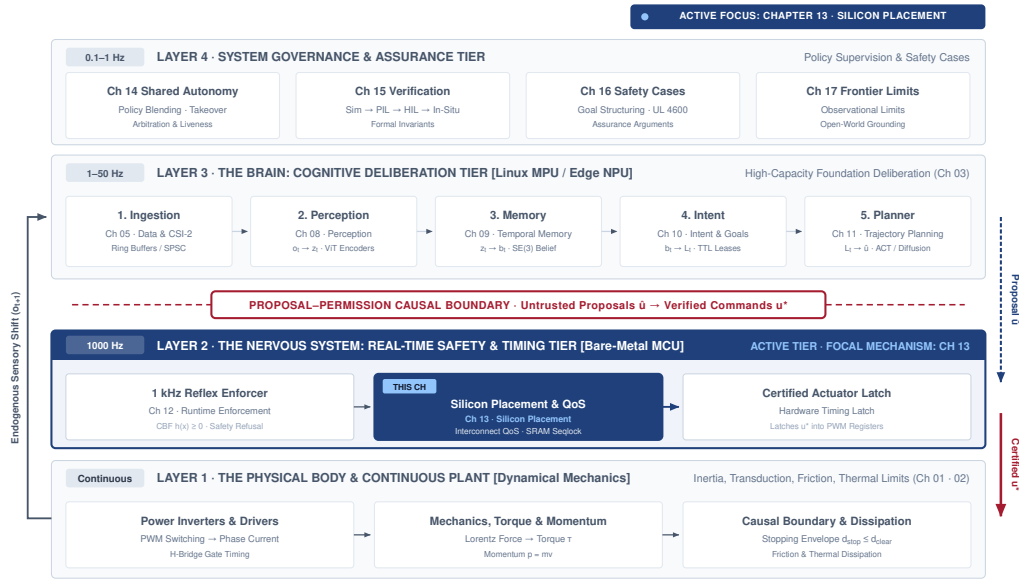


Figure 26.1: Silicon placement architecture focus: The four-tier physical AI systems architecture highlights the real-time safety tier, with silicon placement as this chapter’s active focus. Read the highlighted box as the physical co-location boundary where cognitive bursts and hard real-time reflex loops land on one system-on-chip and then compete for memory controllers, interconnect crossbars, power rails, and thermal mass.

cache, the resulting microsecond bus jitter degrades feedback phase margins, exciting gearbox compliance, inducing motor torque ripple, or breaching ISO 10218 safety envelopes.

- **In Process and Energy Systems (Class 3)**, industrial chemical bypass valves and high-voltage grid battery inverters co-locate continuous thermodynamic optimization with hard real-time overpressure and power-stage shoot-through interlocks. A delayed safety interrupt allows manifold pressure to spike beyond structural burst limits or power transistors to enter catastrophic thermal runaway.

Following the end-to-end argument in system design (Saltzer et al. 1984), memory protection and operating system process isolation do not by themselves establish operational independence. On a heterogeneous die,² **operational independence** requires that proposal-side activity cannot push enforcement execution beyond its hard physical deadline, and that proposal-side crashes or thermal overruns cannot disable the nervous system’s authority to refuse.

Because these couplings are governed by physical solid-state phenomena rather than software abstractions, architectural claims of isolation must be treated as hypotheses to be falsified through rigorous measurement. Verifying a system requires four distinct engineering steps: building an exhaustive claimant inventory of every shared silicon resource; subjecting the proposal path to synthetic worst-case compute, memory, and bus contention while measuring the tail latency distribution of the enforcement path at P_{99} and $P_{99.9}$; measuring thermal dissipation and voltage droop under sustained Joule heating; and recording a formal placement decision that bounds worst-case response times within physical safety margins.

Transforming logical software separation into verifiable physical isolation requires auditing and partitioning every shared hardware resource on the silicon die. We begin in section 26.2 by analyzing the co-location of cognitive proposals and deterministic safety enforcement across heterogeneous execution engines, mapping their competing timing demands and isolation requirements. In section 26.3, we quantify microarchitectural contention across interconnect crossbars, shared caches, and DRAM controllers, applying queuing models and arithmetic intensity to evaluate memory saturation. Section 26.4 establishes the formal boundary contract, examining lock-free state exchange mailboxes and bounded fallback behaviors that prevent reader stalling. We then analyze thermal and

² | **Heterogeneous System-on-Chip Architecture:** Heterogeneous SoCs integrate multiple distinct compute domains onto a single silicon substrate, combining high-throughput application processors and neural accelerators with deterministic real-time safety islands. Although memory management units isolate software address spaces across domains, hardware subcomponents continue to share power distribution networks, clock distribution trees, and DRAM controllers. Under peak cognitive burst workloads, uncoordinated resource contention across these shared physical structures can degrade real-time response times beyond critical physical deadlines.

Prerequisite: The sub-millisecond execution deadline of the CBF-QP reflex enforcer was established in section 24.8.

3 | Core pinning and processor affinity: The operating system binding of an execution thread to an explicit physical CPU core using interfaces such as `pthread_setsaffinity_np` or `taskset`. While core pinning prevents cross-core thread migration overhead, it does not guarantee real-time determinism if pinned cores share last-level caches (LLC), memory controllers, or internal interconnect crossbars with untrusted workloads.

4 | PREEMPT_RT: A Linux real-time configuration reduces some scheduler latencies, but its tail depends on kernel, drivers, workload, and shared hardware. No portable sub-50 μ s or bare-metal comparison follows without measurement.

5 | Inter-processor communication mailboxes and RPMs: Hardware FIFO registers and interrupt lines that signal state transitions between asymmetric compute cores (such as an ARM Cortex-A application processor and Cortex-R/M microcontroller). The Remote Processor Messaging (RPMs) protocol structures shared-memory ping-pong ring buffers (`vrings`) across these mailboxes, enabling lock-free state handoffs without requiring shared operating system primitives.

6 | Hardware memory protection and IOMMU domains: Physical silicon structures that enforce spatial address isolation. While an on-chip Memory Protection Unit (MPU) confines CPU thread execution to bounded memory segments, an Input-Output Memory Management Unit (IOMMU) translates and polices direct memory access (DMA) requests from peripheral bus masters (such as PCIe neural accelerators or camera deserializers), physically preventing an untrusted device from corrupting safety state buffers.

electrical coupling in section 26.5, modeling Joule heating, clock throttling hysteresis, and power-rail collapse. Section 26.6 develops loaded adversarial test harnesses to empirically falsify claimed architectural independence. Finally, section 26.7 through section 26.10 formulate the hierarchy of silicon isolation decisions, synthesize full-system hardware allocation budgets, review critical systems fallacies, and summarize the complete placement methodology.

26.2 Two Paths on One Die

Placement cannot defer to integration; the physical assignment of software functions to hardware execution engines determines separation. Consider a valve regulating high-pressure fluid flow in a chemical processing line, where a learned model adjusts the valve trim position to track a variable target flow rate while an enforcement filter prevents downstream pressure transients that would rupture the pipe. The machine partitions this control across two paths with conflicting requirements: the proposal path ingests downstream pressure histories, acoustic vibration signals, and temperature logs over a moving window of one hundred samples, running a learned neural policy at 20 Hz to calculate a recommended valve displacement. It operates under a soft deadline of 45 ms, holds no direct hardware authority over the motor driver, and runs in an unprivileged user space. The enforcement path samples raw differential pressure transducers at 1000 Hz, evaluates physical boundary constraints on pressure slew rates and cavitation limits, and generates electrical drive commands for the valve servo. It executes under a hard physical deadline of 1.0 ms, requires an execution budget bounded at 400 μ s to leave sufficient margin for mechanical valve transit, and runs in a privileged real-time execution domain. Only the enforcement path holds memory-mapped input-output (MMIO) access to the pulse-width modulation (PWM) registers to command the valve motor.



Definition 26.1: Heterogeneous silicon partitioning

Heterogeneous silicon partitioning is the architectural allocation of stochastic, high-capacity cognitive workloads (perception, learned policy evaluation) and deterministic reflex loops (safety invariant checks, actuator commutation) across specialized execution domains isolated in hardware.

- 1. Significance:** Placing high-capacity neural deliberation and real-time safety enforcement on the same general-purpose operating system or compute engine creates catastrophic common-mode failures. Kernel thread preemption, dynamic memory allocation, and GPU warp scheduling inject multi-millisecond tail jitter that causes the safety reflex to miss physical control deadlines.
- 2. Distinction:** Unlike logical software process isolation (such as Linux containers or POSIX process priorities), silicon partitioning enforces isolation across independent hardware registers, tightly-coupled static memories (TCM), and dedicated peripheral buses.
- 3. Common pitfall:** Assuming multi-core CPU affinity provides true failure-domain separation. Cores that share on-chip L3 caches, DRAM memory controllers, phase-locked loop (PLL) clock trees, or power distribution rails remain vulnerable to cross-core interference, thermal throttling, and voltage droop brownouts.

Concrete performance numbers require mapping these functions onto physical silicon domains. In a heterogeneous dual-brain system-on-chip, such as the canonical dual-brain architecture introduced in chapter 8, proposal inference maps onto an unprivileged Linux Application Processor (MPU) or dedicated neural accelerator running with standard CPU core affinity,³ which reduces migration overhead but remains vulnerable to PREEMPT_RT kernel scheduling jitter.⁴ State exchange between the paths maps onto a shared static memory mailbox⁵ or ring buffer in dynamic random-access memory (DRAM). Safety enforcement and sensor validation map onto a dedicated Real-Time Controller Subsystem (MCU) running bare-metal firmware or a deterministic real-time operating system protected by a hardware Memory Protection Unit.⁶ Device input-output maps onto dedicated serial peripheral interface controllers, analog-to-digital converters, and timer channels directly wired to the MCU. Telemetry logging, network communications, and fault recording map onto the Linux

MPU. Where shared-silicon interference and common-cause faults cannot be bounded for the application, a separately powered safety controller is one design option.⁷ As detailed in table 26.1, extending the classical cyber foraging and edge-placement paradigms of pervasive computing (Satyanarayanan 2001) to heterogeneous autonomous silicon, each compute domain on a modern physical AI platform exhibits distinct trade-offs across nominal clock rate, tail latency determinism, operating system isolation, and memory bus coupling.

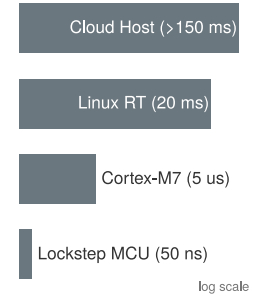
Table 26.1: Heterogeneous compute allocation and execution domain characteristics: Illustrative placement options; clock, power, jitter, and fault-containment values require a named device and measured operating envelope.

Compute Engine Domain	Nominal Clock & Power	Timing Behavior to Qualify	OS & Privilege Isolation	Memory & Interconnect Coupling	Target Workload Assignment
Application Processor Subsystem (MPU)	1.0–2.2 GHz, 2–10 W	Scheduler and memory tails depend on configuration	Rich OS (Linux / POSIX), Ring 3 user space vs. Ring 0 kernel	Shared coherent L3 cache, virtual memory via MMU, unified DRAM	Telemetry logging, edge VLMs/VLAs, PyTorch, ROS2 policy evaluation (10–50 Hz)
GPU Tensor Core Engine	0.8–1.3 GHz, 10–40 W	Driver queue and memory contention require measurement	Host driver queue, user-space runtime, unified virtual memory	High-throughput LPDDR5/HBM, AXI bursts limited by beats and 4,096-byte boundary	High-throughput perception, multi-camera feature extraction, dense VLM inference
Dedicated Neural Processing Unit (NPU / Systolic Array)	0.6–1.0 GHz, 2–8 W	Graph cadence and DMA tail require measurement	Hardware mailbox, fixed-depth systolic pipeline, pinned memory buffers	Dedicated on-chip SRAM (8–32 MB), periodic DMA bursts to DRAM	Learned policy evaluation, low-latency trajectory chunk generation (20–100 Hz)
Real-Time Controller Subsystem (MCU)	200–800 MHz, 0.1–1.5 W	Bounded only with qualified clocks, memory, and interrupts	RTOS (FreeRTOS / Zephyr) or Bare-Metal, MPU memory regions	Tightly-Coupled Memory (TCM / SRAM), private peripheral bus, zero paging	Deterministic safety invariants, 1 kHz CBF safety filters (Ames et al. 2019), direct PWM timers
Discrete Safety Microcontroller	200–400 MHz, 0.2–1 W	Lockstep detects selected faults; response bound is device-specific	Bare-metal / AUTOSAR Classic, hardware Fault Collection Unit (FCCU)	Isolated SRAM/Flash, independent external power rails, dedicated crystal	Emergency mechanical clamping, hardware interlocks, supervisory watchdog trips

In an analytical evaluation of the unloaded system, the computational budgets appear generous. In the enforcement path on the real-time MCU, reading four pressure sensors over a high-speed peripheral bus requires $40\mu\text{s}$, evaluating the constraint envelope and computing the actuator signal takes $80\mu\text{s}$, and writing the resulting register values to the motor controller timer takes $15\mu\text{s}$. The total unloaded execution demand is $135\mu\text{s}$. Against the $400\mu\text{s}$ computational deadline selected for the 1.0 ms physical cycle (which reserves $600\mu\text{s}$ for servo motor settling), the enforcement path exhibits an unloaded slack of $265\mu\text{s}$. In the proposal path on the Linux MPU, a policy requiring 3.2 GFLOP per step executes on an accelerator capable of 2.0 TFLOPS sustained throughput, yielding an inference time of 1.6 ms. Adding 0.8 ms for tensor formatting and 0.1 ms to publish the proposal to the shared ring buffer, the proposal path requires 2.5 ms of computation per 50 ms cycle, leaving 47.5 ms of unloaded slack under the assumed sustained accelerator rate. Evaluated in isolation, both loops operate with comfortable margins.

Assigning tasks to separate processor cores does not create independent failure domains if the silicon infrastructure is shared.⁸ While the real-time core executes instructions independently from the application processor, both cores rely on common hardware services:

7 | Dual-core lockstep: Two cores can execute matching instruction streams while comparator logic detects selected divergences. Detection coverage, delay, shared-rail vulnerability, and diagnostic response depend on the device and application safety case; lockstep alone does not establish an ASIL D system.



Candidate execution domains differ in their timing controls; measured tails determine placement.

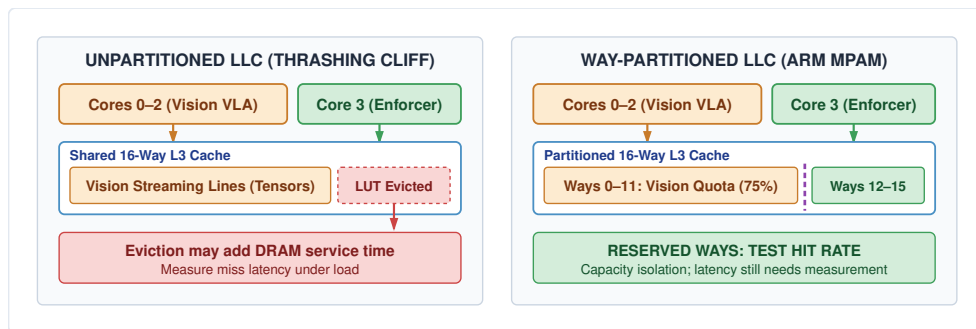
8 | Avionic Multicore Certification: FAA Advisory Circular AC 20-193 and CAST-32A formalize multicore interference analysis for safety-critical avionics. The standard requires rigorous bounding of worst-case execution times by demonstrating that shared caches, interconnect crossbars, and memory controllers cannot cause critical tasks to overrun execution deadlines. Without hardware-enforced quality-of-service partitions or dedicated interconnect channels, cross-core memory traffic invalidates deterministic timing guarantees.

⁹ | **Cache coloring and way partitioning:** Techniques that divide shared cache capacity among competing tasks. Software cache coloring partitions physical page allocations so that addresses from distinct processes map to non-overlapping cache sets, while hardware partitioning mechanisms (such as ARM Memory System Resource Partitioning and Monitoring, MPAM, or Intel Cache Allocation Technology, CAT) dynamically restrict the cache ways an untrusted neural accelerator can allocate, preventing line evictions in critical safety threads.

- *Shared interrupt routing:* The Generic Interrupt Controller (GIC) routes peripheral and inter-processor notifications across a shared fabric, where a flood of Ethernet packet interrupts on the Linux MPU can delay the timer interrupt that triggers the 1000 Hz MCU safety loop.
- *Shared memory controllers:* Both paths access the same dynamic memory controller and interconnect crossbar, where large matrix tile loads generated by the neural accelerator saturate DRAM command queues, turning an expected 20 ns level-two cache refill on the real-time core into a 350 ns round-trip to main memory.⁹
- *Shared clock trees:* The cores share phase-locked loops (PLLs) for clock generation, where dynamic voltage and frequency scaling (DVFS) transients on the application processor introduce clock jitter across adjacent real-time domains.
- *Shared power rails:* A single Power Distribution Network (PDN) delivers current to both domains, where switching surges from the neural accelerator drop the internal voltage rail, risking logic state corruption or brownout resets.
- *Shared reset domains:* A kernel panic on the Linux MPU that triggers a hardware watchdog reset can pull down the shared peripheral bus or system interconnect, disabling PWM outputs and leaving the valve unpowered.

For the preceding 1.0 ms valve-loop case, compare three *constructed* placements under their stated loads. Shared application cores give an illustrative $P_{99.9} = 2.8$ ms, missing the deadline. A same-die MPU/NPU plus real-time core has an observed loaded execution value of 380 μ s against the chosen 400 μ s compute allowance, but a modeled thermal clock reduction raises it to 540 μ s. A separate MCU adds an assumed 150 μ s serial transfer to its 135 μ s unloaded execution. Each option still needs a measured sensor-to-valve response, fault response, and worst-case interference analysis; separate packages alone do not guarantee a one-tick safe transition.

Checkpoint 26.1: Asymmetric core allocation and shared silicon services



Before partitioning workloads across processor cores, evaluate the hidden physical coupling paths that breach logical CPU boundaries:

- ☐ **Core pinning versus silicon independence:** Can you explain why assigning a real-time thread to a dedicated CPU core via `pthread_setaffinity_np` fails to guarantee isolation if that core shares an L3 cache or memory controller with an accelerator?
- ☐ **Generic Interrupt Controller serialization:** Can you describe how high-frequency Ethernet or camera DMA interrupts on an application core can introduce latency jitter into real-time microcontroller interrupt service routines?
- ☐ **Cache line thrashing:** Can you quantify the latency penalty when an untrusted vision model evicts the real-time enforcer's lookup tables from a shared last-level cache into dynamic RAM?
- ☐ **DMA address protection:** Can you explain why an MCU MPU constrains CPU accesses while an IOMMU or bus firewall must constrain DMA masters that could overwrite safety state?

Logical core affinity is not physical isolation; if two tasks share cache lines, interrupt controllers, or DRAM channels, they belong to the same failure domain.

26.3 Contention for Shared Resources

Every shared physical resource has multiple claimants; a utilization metric that ignores concurrent competition is functionally meaningless. On a system-on-chip regulating a fluid pipeline, physical wires do not respect functional boundaries drawn on a software architecture diagram. The proposal path, the enforcement path, and the system infrastructure share the same silicon die, competing directly for the same memory controllers, crossbar switches, and power distribution rails. Digital claimants on the main memory bus include sensor DMA streams depositing pressure transducer readings and acoustic telemetry into DRAM, the neural accelerator fetching model weight matrices and intermediate activation tensors, the real-time core reading pipeline invariants and historic sensor buffers, actuator write buffers for PWM commands to the flow valve driver, cache controllers executing dirty line writebacks, and diagnostic logging processes writing state records to nonvolatile storage.

In physical AI hardware design, **crossbar bus and DRAM bank contention** is queuing delay when memory and DMA transactions share an interconnect and controller. One in-flight AXI4 transaction may finish before a newly arrived safety request; blocking by a whole model tile requires a specified repeated-grant or FIFO policy. Row-buffer scheduling can add delay, so the actual bound depends on transaction size, queue capacity, QoS semantics, bank mapping, and new arrivals.¹⁰

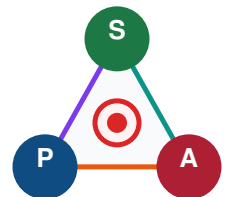
Each claimant imposes a demand that can be derived from its transfer size per event and its event rate. In an analytical example of a flow-regulation controller sharing a single 64-bit LPDDR4 memory channel rated at 12.8 GB/s peak bandwidth, sensor direct memory access transfers a 64 kB buffer at 1000 Hz, generating an average demand of 64.0 MB/s. The learned proposal model executes an inference pass every 20 ms (50 Hz), streaming 60 MB of parameters and activation data for an average demand of 3.0 GB/s. The enforcement path executes every 1.0 ms (1000 Hz), reading 4 kB of pipeline state and historical sensor logs, consuming 4.0 MB/s on average. Actuator writes require 1 kB per millisecond (1.0 MB/s), cache maintenance evictions contribute 128.0 MB/s in writebacks, and diagnostic telemetry logging transfers 32 kB records at 100 Hz (3.2 MB/s). Summing these rates gives an aggregate demand of 3.2002 GB/s, which represents 25.0 percent of the channel capacity. An engineer who inspects only average utilization would conclude that three-quarters of the bus capacity remains available. But memory arbiters do not schedule average rates, they arbitrate discrete bursts, and two workloads with identical average throughput create distinct interference profiles if one streams continuously in small packets while the other arrives in long, saturating blocks.

Arithmetic intensity, defined as the ratio of arithmetic operations to memory bytes transferred (Williams et al. 2009), indicates whether a computational routine is bounded by execution units or by memory bandwidth during steady-state execution. For a convolution or dense matrix multiplication on the proposal path, an arithmetic intensity of 120 FLOP/byte indicates that arithmetic units will be fully occupied once data resides in local cache or static memory. However, during the initial phase where weights are transferred from dynamic memory, the operational arithmetic intensity drops to zero. Physically, the accelerator ceases to be a computation engine and instead acts as a blunt, high-bandwidth direct memory access engine that aggressively saturates the shared memory bus. Arithmetic intensity reveals whether a workload can hide memory transfer latency behind arithmetic work in isolation, but it provides no information about queue contention, transfer priority, burst duration, or which request waits when two paths access the bus simultaneously. The same imbalance that governs a data center accelerator governs the memory channel on a robot, only with less headroom. Figure 26.2 shows compute outrunning bandwidth roughly two to one across four generations, which is why an arithmetic-intensity argument decides placement more often than a raw throughput number does.

The latency experienced by the enforcement path depends on transaction arbitration and DRAM scheduling (figure 26.3 a). In this constructed system, the accelerator moves weights in 262,144-byte tiles, but a tile is not one AXI4 burst: Arm's AXI4 specification permits at most 256 beats per burst and forbids a burst from crossing a 4,096-byte address boundary. Assume an aligned internal 128-bit

Prerequisite: Multi-camera direct memory access ingress traffic volumes were calculated in section 16.4.

¹⁰ | **DRAM bank conflicts:** A different row in the same active bank may require precharge and activation before column service. Actual bank mapping, overlap, and scheduling determine the added delay; the three switches in the worked case are explicit assumptions.



The silicon substrate: co-locating Sense, Plan, and Act across shared heterogeneous System-on-Chip crossbars.

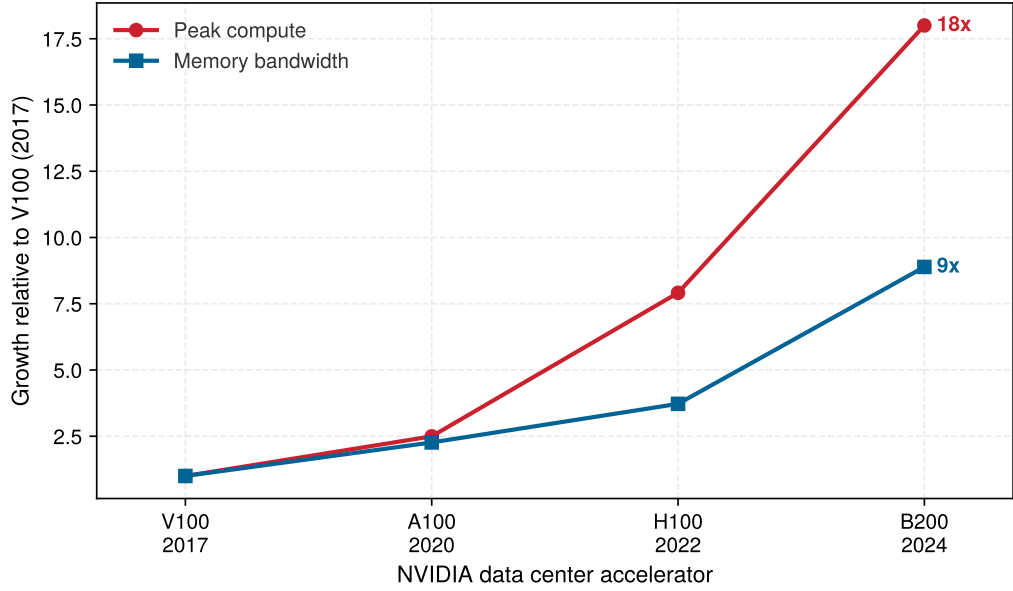


Figure 26.2: Memory bandwidth deficit: Peak compute and memory bandwidth for data center accelerators, normalized to the V100. Compute rises 18× against bandwidth’s 8.9×, and a placement decision works against that ratio because an on-robot budget cannot buy its way past a bandwidth limit. Selected vendor peak or up-to specifications are recorded with source links in the local `gpu_memory_wall.csv` (B200 dense FP16/BF16 and HBM bandwidth); the plotted ratios do not measure sustained or on-robot latency.

AXI path with 16-byte beats, so each 4,096-byte transaction uses 256 beats and a tile requires 64 transactions. The 64-bit LPDDR channel width does not establish that internal AXI width. An in-flight transaction may be non-preemptible; whether the remaining 63 tile transactions block a later safety read depends on arbiter and controller policy, including whether priority/QoS bypass is implemented. Dynamic random-access memory also has bank row buffers. In the following *assumed* bank-switch model, a switch adds precharge $t_{RP} = 14$ ns and activation $t_{RCD} = 14$ ns before column service. These are scenario inputs, not measured values for a named device.

To visualize DRAM row buffers, imagine a records clerk working in front of a filing cabinet where **only one row per bank can be active at a time**. If an incoming request asks for a folder in that bank’s currently open drawer (a row-buffer hit), the clerk grabs it instantly ($t_{CAS} \approx 14$ ns). But if an urgent request asks for a folder inside a different, closed drawer (a bank conflict or row-buffer miss), the clerk cannot just reach in. They must first slide the open drawer completely shut and latch it (precharge, $t_{RP} \approx 14$ ns), pull the newly requested drawer open (activation, $t_{RCD} \approx 14$ ns), and only then extract the document.

Some DRAM controllers use **First-Ready First-Come-First-Served (FR-FCFS)** scheduling to favor ready row hits; that policy can reorder requests and must be measured or bounded separately. For the worked calculation, assume instead a FIFO queue that does not let a late safety transaction bypass already queued traffic, with no new arrivals during service. If the queued transactions induce three bank switches, their idealized queuing time is:

$$t_{\text{queue}} = \sum_{k \in Q} \frac{B_k}{\text{BW}_{\text{peak}}} + N_{\text{conflicts}}(t_{RP} + t_{RCD}) \quad (26.1)$$

In equation 26.1, Q contains the transactions ahead of the safety read, B_k is each transaction’s byte count, and BW_{peak} is an optimistic service-rate assumption. This equation is a FIFO scenario model, not a controller-independent worst-case bound; arbitration overhead, refresh, and further arrivals require separate allowance.

Consider a separate 65,536-byte diagnostic sensor-history transfer for a 1000 Hz controller with a 120 μs read-plus-compute subdeadline. The per-tick 4,096-byte state read used in the preceding

average-bandwidth calculation is a different workload. In this diagnostic case, compute takes an assumed $95\ \mu\text{s}$ and an unloaded $65,536$ -byte read takes at least $65,536 / (12.8 \times 10^9) = 5.120\ \mu\text{s}$ at ideal peak bandwidth, leaving $19.880\ \mu\text{s}$ of the subdeadline. Immediately before the read arrives, assume the controller can admit all 88 earlier transactions and its FIFO contains 64 aligned $4,096$ -byte tile transactions, 16 camera transactions ($65,536$ bytes), and eight telemetry transactions ($32,768$ bytes): 88 transactions or $360,448$ bytes total. Their ideal transfer time is $28.160\ \mu\text{s}$; three assumed bank switches add $0.084\ \mu\text{s}$. Thus $t_{\text{queue}} = 28.244\ \mu\text{s}$, $t_{\text{sensor,loaded}} = 33.364\ \mu\text{s}$, and $t_{\text{exec,loaded}} = 128.364\ \mu\text{s}$, exceeding the $120\ \mu\text{s}$ subdeadline by $8.364\ \mu\text{s}$. If priority service bypasses every queued transaction after the single in-flight $4,096$ -byte transaction, the same idealized model gives $100.44\ \mu\text{s}$ total; actual bypass may be unavailable. Additional arrivals or controller overhead could make either path longer. A dedicated DMA route into a coherent MCU snapshot and a $131,072$ -byte Tightly Coupled Memory (TCM) allocation for the $65,536$ -byte buffer plus $8,192$ -byte state table remove this DRAM read from the critical path only if transfer completion and coherence are validated. TCM access latency is per access, not per buffer. At an assumed $800\ \text{MHz}$ and eight bytes per cycle, 8192 accesses take at least $10.24\ \mu\text{s}$, giving $105.24\ \mu\text{s}$ total and $14.76\ \mu\text{s}$ slack; at $200\ \text{MHz}$ the $40.96\ \mu\text{s}$ read floor exceeds the subdeadline after $95\ \mu\text{s}$ compute.



Systems Perspective 26.1: The memory contention invariant

Average memory bandwidth cannot bound a deadline. Admission depends on bounded transaction sizes, queue policy, bank timing, and measured end-to-end response under the permitted load.



Checkpoint 26.2: DRAM burst queuing and real-time control overrun

- ☐ **Average bandwidth fallacy:** Can you explain why average memory bus bandwidth utilization can appear safely under-utilized while a hard real-time safety task experiences catastrophic deadline overruns on the same bus?
- ☐ **Scheduling policy:** Can you distinguish the worked FIFO queue from FR-FCFS row-hit reordering and an AXI QoS bypass?
- ☐ **Queuing delay sizing:** Can you reproduce $28.244\ \mu\text{s}$ of assumed FIFO delay from 88 aligned transactions and three bank switches, then test whether the resulting $128.364\ \mu\text{s}$ fits the $120\ \mu\text{s}$ subdeadline?
- ☐ **TCM SRAM isolation:** Can you evaluate why provisioning dedicated Tightly-Coupled Memory (TCM) SRAM eliminates bus-induced tail jitter more effectively than software thread priority or cache-pinning alone?

Contention extends to the power distribution network, where claimants compete for charge rather than bandwidth (figure 26.3 b). When the neural accelerator starts a matrix multiply across thousands of parallel multiply-accumulate units, its current draw spikes rapidly. Across a shared power distribution network with an effective loop inductance L and equivalent series resistance R , an abrupt current transient dI/dt with peak step ΔI induces an instantaneous rail droop ΔV_{droop} governed by:¹¹

$$\Delta V_{\text{droop}} = L \frac{dI}{dt} + \Delta I \cdot R \quad (26.2)$$

PDN voltage droop coupling is the instantaneous electrical rail collapse induced across shared on-die power distribution networks. Across a shared power delivery network, when steep, high-amplitude current steps from parallel neural execution units suddenly deplete local decoupling capacitance, the operating voltage drops precipitously. This transient voltage sag stretches CMOS logic gate propagation delays, risking clock frequency degradation, flip-flop setup-time violations, or uncommanded brownout resets across co-located real-time cores.

¹¹ | **Inductive Power-Rail Droop Mechanics:** Rapid current transients (dI/dt) across the parasitic inductance of package balls, bond wires, and printed circuit board traces induce instantaneous supply voltage sags ($\Delta V = L \cdot dI/dt$). Because off-chip voltage regulator modules respond on microsecond timescales, nanosecond current spikes deplete local on-die decoupling capacitance. If the resulting droop breaches the minimum operating voltage threshold, logic cells experience setup-time timing violations or trigger uncommanded brownout resets.

On a nominal 0.85 V core supply, a sufficiently large droop could cross the minimum operating voltage of a co-located core; that threshold requires a device specification and rail measurement. The off-chip Voltage Regulator Module (VRM) operates with a control loop bandwidth that is several orders of magnitude too slow to counteract a nanosecond transient—it simply cannot push more current into the chip fast enough to catch the falling voltage. Therefore, first-stage charge delivery relies entirely on integrated on-die deep-trench decoupling capacitors (DTCs), which supply localized charge directly adjacent to switching logic blocks. If local capacitance is depleted before the current transient settles, supply voltage collapses.

To ground this physical vulnerability on silicon, consider an analytical heterogeneous SoC powering a systolic array neural accelerator and a real-time safety processor from a shared $V_{\text{core}} = 0.85$ V power distribution network. The PDN package loop exhibits parasitic inductance $L = 30$ pH and equivalent series resistance $R = 8$ m Ω . When the neural accelerator starts a matrix multiply, its current demand surges by $\Delta I = 6$ A with rise time $t_r = 2$ ns ($dI/dt = 3.0$ A/ns). Evaluating the electrical collapse via equation 26.2 yields inductive drop $\Delta V_{\text{ind}} = (30 \times 10^{-12} \text{ H}) \times (3.0 \times 10^9 \text{ A/s}) = 0.090$ V = 90 mV and resistive drop $\Delta V_{\text{res}} = 6 \text{ A} \times 0.008 \Omega = 48$ mV, producing a total rail droop of $\Delta V_{\text{droop}} = 138$ mV = 0.138 V. The drooped core voltage falls from 0.850 V to 0.712 V—an instantaneous 16.2 percent voltage drop.

Under the stated alpha-power *gate* model ($V_{\text{th}} = 0.42$ V, $\alpha = 1.3$), the 16.2 percent rail droop predicts a 39 percent propagation-delay increase.¹² This is not a measured thread-runtime expansion or proof that V_{min} was crossed. Whether it consumes a loop's slack, trips a watchdog, or affects a valve depends on the device threshold, circuit path, clock policy, and measured end-to-end response. A separate assumed 22 A step in 5 ns with the same L and R would instead give $132 + 176 = 308$ mV; the 138 mV result cannot stand in for that case.

¹² | **Alpha-Power CMOS Delay Model:** The illustrative model uses $t_{\text{prop}} \propto V/(V - V_{\text{th}})^\alpha$ with assumed $V_{\text{th}} = 0.42$ V and $\alpha = 1.3$. Its gate-delay ratio does not directly determine software execution time, setup failure, or brownout behavior; those require device characterization.



Napkin Math 26.1: Power rail voltage droop and timing margins

Practical Engineering Question: Under which measured rail threshold and timing margin would an accelerator current step threaten an adjacent enforcement core on a shared supply? When thousands of multiply-accumulate units in a neural accelerator suddenly turn on, they demand a massive, instantaneous surge in current. Even though the overall power supply might be large enough, the tiny physical wires and package bumps connecting the silicon to the supply have microscopic amounts of resistance and inductance.

Because of this physical reality, the sudden rush of current causes the voltage supplied to the entire chip to briefly drop, a phenomenon known as **voltage droop**. If the voltage sags below the minimum operating level of the adjacent real-time core, the microscopic transistors inside that core cannot switch fast enough to meet their clock timings.

For the rigorous mathematical derivation of inductive rail droop and the alpha-power law governing gate delay stretch, see Appendix C.

Systems insight: The modeled gate-delay increase motivates rail and execution-time measurements. Decoupling, current-slew limits, separate power domains, or separate silicon are candidate remedies selected after checking the device threshold and physical deadline.



Checkpoint 26.3: PDN voltage droop and propagation stretch

- ☐ **Transient droop mechanics:** Can you distinguish between steady-state bulk power delivery and high-frequency $L \cdot dI/dt$ transient voltage droop on a shared power distribution network?
- ☐ **Rail droop sizing:** Can you compute the instantaneous voltage sag on a 0.85 V rail when a systolic array surge ($\Delta I = 6.0$ A in 2.0 ns) hits a PDN with $L = 30$ pH and $R = 8.0$ m Ω ?
- ☐ **Alpha-power delay scaling:** Can you explain why an unmitigated voltage droop causes logic gate propagation delay to stretch nonlinearly according to the Sakurai-Newton alpha-power law rather than a simple linear scaling?

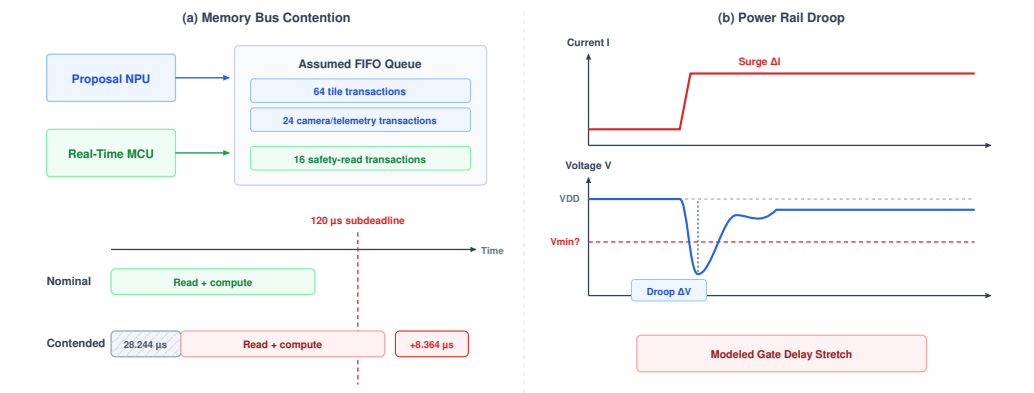


Figure 26.3: Heterogeneous SoC interference: (a) In the constructed FIFO example, 88 queued 4,096-byte transactions delay a 65,536-byte safety read and push read plus compute to 128.364, μ s against a 120, μ s subdeadline; priority bypass would change the result. (b) A separate assumed 6 A in 2 ns current step gives 138 mV PDN droop. The drawn V_{min} crossing is a conditional hazard, not established by that droop calculation.

❑

Hardware vs. privilege boundary: Can you explain why software privilege rings (such as ARM TrustZone or hypervisors) are fundamentally incapable of preventing setup-time violations caused by adjacent accelerator current surges?

Detecting these interactions requires dedicated hardware instrumentation, because software running on a core that has been stalled cannot record the duration of its own stall. Memory-controller performance counters log queue occupancy, bank conflicts, and per-master read and write transaction counts. Performance monitoring units on the real-time core record cache miss counts and stall cycles against an invariant free-running counter. On the electrical side, on-die analog-to-digital converters and fast comparator circuits record voltage excursions on internal rails, thermal diodes log junction temperature gradients across the floorplan, and power-management logs record dynamic frequency throttling states. When an uncommanded reset occurs, reset-cause registers distinguish a software watchdog expiration from an under-voltage brownout or a thermal emergency trip.

When contention is budgeted through measured burst times, queue structures, and power supply impedances rather than assumed from isolated benchmarks, the boundary between proposal and enforcement ceases to be a conceptual line on a block diagram. Table 26.2 summarizes these physical interference channels and their required hardware mitigations.

Table 26.2: Shared SoC resource contention channels and physical coupling mechanisms: Breakdown of five physical interference vectors on unified silicon. Contention operates at microsecond and nanosecond timescales beneath operating system visibility, requiring architectural partitioning or dedicated hardware isolation to prevent cognitive bursts from violating reflex timing bounds.

Contention Channel	Physical Coupling Mechanism	Contention Trigger & Workload Dynamics	Nominal vs. Contended Delay	Systems Failure Manifestation	Hardware Mitigation & Interceptor
DRAM Memory Bus & Controller	FIFO transaction queue and assumed bank switches	262,144-byte weight tile split into 64 aligned 4,096-byte AXI transactions	Constructed 65,536-byte read: 5.120 $\rightarrow$ 33.364 μ s	128.364 μ s read plus compute exceeds 120 μ s subdeadline	Validate QoS bypass, reserve channel, or use coherent local snapshot

Contention Channel	Physical Coupling Mechanism	Contention Trigger & Workload Dynamics	Nominal vs. Contended Delay	Systems Failure Manifestation	Hardware Mitigation & Interceptor
Shared Last-Level Cache (L3)	Cache-capacity thrashing	Streaming tensors can evict safety tables if cache is shared	Latency depends on cache, mapping, and DRAM configuration	Possible invariant-check stall; quantify under load	Cache way partitioning, locking, or private TCM; validate remaining bus path
On-Chip Interconnect Crossbar	Arbiter head-of-line blocking	Concurrent camera DMA and diagnostic flushes	Transaction latency is configuration dependent	Possible dispatch delay or missed heartbeat	Segregated channels and measured priority routing
Power Distribution Network (PDN)	Inductive ($L \cdot dI/dt$) and resistive ($I \cdot R$) voltage droop	Assumed 6 A step in 2 ns, 30 pH and 8 m Ω	Model: $90 + 48 = 138$ mV droop, 0.712 V remaining	Gate-delay model predicts stretch; device $V_{\min}$ and runtime response unmeasured	Measure rail and path; qualify decoupling and current-slew limits
Shared Thermal Package	Substrate heat diffusion through package spreader ($\theta_{JA} = 2.0$ K/W)	Sustained peak inference (15–25 W) driving junction past $T_{\text{trp}} = 95^\circ\text{C}$	Assumed governor halves clock (1.2 $\rightarrow$ 0.6 GHz); compute $2\times$	Enforcement execution expands (0.70 ms $\rightarrow$ 1.40 ms), deadline missed	Independent PLL domains, NPU thermal throttling caps, physical dual-die split

The interaction between two paths on one die cannot be left to uncoordinated arbitration. What crosses the physical boundary must be defined by explicit guarantees on production time, validity, and failure response.

26.4 The Boundary Contract

The boundary contract crosses between the proposal and enforcement paths, not an unqualified pointer into shared memory. Passing a raw memory address across the trust boundary couples the reader directly to the writer’s internal memory layout, execution timing, and failure domain. If a vision-language model or deep neural policy running on an accelerator provides only a pointer to its private output buffer, the real-time core that regulates a gas valve has no mechanism to determine whether the writer is still modifying the buffer, has stalled mid-inference, or has encountered an unhandled fault. The boundary payload must therefore be an immutable, self-contained record. For a fluid flow regulator, this record contains the target mass flow rate command $\dot{m}_{\text{cmd}}$, a monotonically increasing sequence counter k , a generation timestamp t_{prod} sampled from an invariant hardware timer, an expiration lease duration Δt_{valid} , and an integrity checksum calculated over the payload bytes. In a canonical heterogeneous dual-brain architecture (as formalized in the intent contract in chapter 8), this interface manifests as a packed struct exchanged across zero-copy shared SRAM mailboxes, binding spatial target vectors, velocity/acceleration limits, epistemic model confidence γ_{conf} , and an IEEE 802.3 32-bit CRC checksum. The enforcement path reads the record as an isolated snapshot, evaluating its temporal freshness and physical validity before any value reaches the valve actuator.

Preserving temporal isolation across a shared die requires eliminating blocking synchronization at the exchange interface. If the real-time enforcement loop acquires a lock held by the proposal generator, an inference stall can delay a physical decision. A 25 ms host stall must therefore leave the 1000 Hz enforcer running each tick: it may use the last validated chunk only while its lease and current state remain admissible, then transfer to its independently validated fallback. Allocation and logging on the enforcement path likewise need measured time bounds. The producer may stall or drop updates; it cannot backpressure the plant’s enforcement cadence.

The enforcement reader needs one bounded snapshot attempt per tick, as in the single-slot mailbox of chapter 8. A separate, noncritical logging consumer may use a ring: with 50 Hz proposal production and an assumed 60 ms *logging-consumer* pause, $\lceil 50 \times 0.060 \rceil + 1 = 4$ slots preserve the

Prerequisite: The byte-level intent lease schema and monotonic sequence IDs are defined in section 20.7.

three arrivals plus one publication slot if phase and write timing match the assumption. Overflow can overwrite the oldest audit sample while recording a gap. This ring neither preserves a proposal lease nor rescues a paused enforcement reader; the valve controller continues reading current sensor state and applying permission or fallback every 1 ms.

Preventing the enforcement reader from observing a partially written or mixed-version record requires a lock-free publication protocol with memory barriers (figure 26.4 a).¹³ The writer and reader coordinate through a single shared sequence counter S . Before modifying the payload buffer, the writer increments S from an even value to an odd value, executes a store-release memory barrier to ensure that the updated sequence is visible to interconnect arbiters, writes the payload fields, executes a second store-release barrier, and increments S to the next even integer. The enforcement reader reads S , storing the initial value S_1 . If S_1 is odd ($S_1 \bmod 2 = 1$), the writer is actively modifying the buffer, and the reader rejects the slot without touching the payload. If S_1 is even, the reader executes a read memory barrier, reads the payload fields, executes a second read memory barrier, and samples the counter a second time as S_2 . The reader accepts the data if and only if $S_1 = S_2$. If $S_1 \neq S_2$, the writer updated or started modifying the record during the read operation, indicating that the data is torn. Tracing the writer across all preemption points demonstrates why a mixed-version record cannot be exposed. If the writer halts before the first increment, the reader samples the previous even sequence and obtains the prior intact record. If the writer halts after setting S odd or during payload writes, the reader observes an odd sequence on every subsequent cycle and rejects the buffer. If the writer halts after the final even increment, the reader receives the new, fully committed record. At no point can a stalled or preempted writer force the reader to consume inconsistent bytes.

¹³ | **Data Memory Barrier Semantics:** Out-of-order processor pipelines and non-blocking memory interconnects can reorder load and store transactions across distinct compute masters. Hardware data memory barriers (such as ARM DMB ISH) enforce explicit memory consistency by ensuring that preceding memory transactions commit before subsequent operations become visible across the inner-shareable domain. In lock-free state exchange mailboxes, these barriers prevent real-time readers from observing torn or stale payload structures.

Systems Perspective 26.2: Lock-free asymmetric decoupling

Never share a blocking synchronization primitive across proposal and enforcement domains. One bounded mailbox attempt can reject an in-flight write without waiting; its execution bound must be measured on the chosen hardware.

The architectural behavioral specification for hardware crossbar QoS arbitration and autonomous thermal supervision is formalized in algorithm 26.1. Rather than an operating system software scheduler, this specification defines the cycle-by-cycle hardware arbitration logic and state-machine transitions executed directly by the on-chip interconnect switch fabric and the Power Management Controller (PMC).

Algorithm 26.1 Architectural behavioral specification for hardware crossbar QoS arbitration and thermal supervision

Require: Ingress transaction request $\text{req} = \{\text{master_id}, \text{addr}, \text{burst_len}, \text{qos_tag}\}$, junction temperature T_j , credit token register C_{tokens}

Ensure: Interconnect channel grant signal grant , routed bus transaction txn , clock frequency control registers f_{target}

```

1: Hardware State Machine 1: Interconnect Crossbar Arbiter (Cycle Clock  $\tau_{\text{clk}}$ )
2: if  $\text{req.qos\_tag} = \text{QOS\_CRITICAL}$  then
3:   Ingress transaction to high-priority FIFO:  $Q_{\text{realtime}} \leftarrow \text{req} \triangleright \text{MCU safety transaction}$ 
4: else
5:   Ingress transaction to best-effort FIFO:  $Q_{\text{besteffort}} \leftarrow \text{req} \triangleright \text{accelerator or DMA burst}$ 
6: end if
7: if  $\text{InterconnectIdle} = \text{true}$  and  $\text{NOTEMPTY}(Q_{\text{realtime}})$  then
8:    $\text{txn} \leftarrow \text{DEQUEUE}(Q_{\text{realtime}})$ 
9:   Route  $\text{txn}$  to dedicated low-latency channel:  $\text{ASSERTGRANT}(\text{channel} = \text{CH\_DIRECT})$ 
10: else if  $\text{InterconnectIdle} = \text{true}$  and  $\text{NOTEMPTY}(Q_{\text{besteffort}})$  then
11:    $\text{txn} \leftarrow \text{DEQUEUE}(Q_{\text{besteffort}})$ 
12:   if  $C_{\text{tokens}} \geq \text{txn.burst\_len}$  then
13:     Deduct bandwidth credit:  $C_{\text{tokens}} \leftarrow C_{\text{tokens}} - \text{txn.burst\_len}$ 
14:     Route  $\text{txn}$  to shared channel:  $\text{ASSERTGRANT}(\text{channel} = \text{CH\_SHARED})$ 
15:   else
16:     Deassert AXI ready signal:  $\text{DEASSERTREADY}(\text{txn.master\_id}) \triangleright \text{hardware backpressure}$ 
17:   end if
18: end if

19: Hardware State Machine 2: Autonomous Thermal and Clock Governor (PMC)
20: if  $T_j \geq T_{\text{trip}}$  then
21:   Scale cognitive PLL divider:  $f_{\text{target}}(\text{DOMAIN\_NPU}) \leftarrow f_{\text{npu\_nom}}/2 \triangleright \text{hardware thermal throttling}$ 
22:   Enforce hardware lock on safety PLL:  $f_{\text{target}}(\text{DOMAIN\_MCU}) \leftarrow f_{\text{nom}} \triangleright \text{protect reflex clock}$ 
23:   Assert status register:  $\text{REG\_THERMAL\_STATUS} \leftarrow \text{STATUS\_DEGRADED}$ 
24: else if  $T_j \leq T_{\text{clear}}$  then
25:   Restore cognitive PLL divider:  $f_{\text{target}}(\text{DOMAIN\_NPU}) \leftarrow f_{\text{npu\_nom}}$ 
26:   Clear status register:  $\text{REG\_THERMAL\_STATUS} \leftarrow \text{STATUS\_NOMINAL}$ 
27: end if

```

Each enforcement tick makes one bounded mailbox snapshot attempt. An odd or changed sequence rejects the snapshot; a consistent snapshot still needs checksum, age, and physical-limit checks before it is eligible for permission. A missing sequence is an audit gap, not by itself evidence that the current proposal is unsafe. If no admissible proposal remains, the MCU uses a separately validated hold or stop response matched to sensed pressure and actuator capability; a fixed valve ramp or ten-millisecond hold is only a candidate design until tested against the plant.

Table 26.3: Boundary contract failure classes and deterministic fallback matrix: Systematic classification of exchange faults across the proposal-enforcement boundary. Fallback paths are selected without waiting on host recovery; their physical response remains conditional on the validated plant and actuator.

Failure Class	Reader Detection Condition	Physical Systems Fallback Action	Bounded Response
Torn / In-Flight	Odd S_1 or changed S_2	Reject current snapshot; use admissible prior chunk or fallback	One bounded attempt; measure duration
Malformed	Checksum or configured command bound fails	Reject proposal; use independently checked fallback	Measure check and response

Failure Class	Reader Detection Condition	Physical Systems Fallback Action	Bounded Response
Stale	Converted local age exceeds lease	End tracking authority; enter feasible MCU fallback	Deadline from plant budget
Missing	Sequence gap	Record audit gap; evaluate current record on its own merits	One tick for flag

The failure classes in table 26.3 keep a failed producer from forcing the enforcer to wait. It cannot by itself ensure physical containment: the independent sensor, compute, drive, and fallback paths still need bounded timing and plant validation.

26.4.1 Windowed watchdog timers and execution liveness

Software memory boundaries and lock-free seqlocks protect against torn reads, but cannot guarantee that the safety enforcer code is alive and progressing through its intended operations. In embedded robotics and automotive controllers, supervisory hardware relies on watchdog timers to catch execution stalls. Yet standard countdown watchdogs introduce a treacherous architectural vulnerability.

A conventional watchdog timer (WDT) is serviced before a configured timeout; its device-specific expiration may reset the processor, while a separately wired supervisor can request drive inhibit. A timer feed proves only that the feed path ran. If a timer interrupt continues to service it after a control task stalls, the watchdog may miss the task failure. In *Bookout v. Toyota* (2013), a plaintiff expert argued that a timer-tick ISR serviced the watchdog while other tasks could die; this is a reported trial mechanism, not a finding that electronic faults caused unintended acceleration in every investigated vehicle.

To defeat this pathology, high-integrity physical AI architectures enforce **Windowed Watchdog Timers (WWDT)**.¹⁴ A windowed watchdog rejects any refresh that occurs either too early or too late. The timer hardware defines a service window parameterized by a lower bound (the closed window, T_{lower}) and an upper bound (the open window, T_{upper}):

1. **Early-kick violation** ($t_{\text{service}} < T_{\text{lower}}$): If software attempts to refresh the watchdog before the lower threshold has elapsed, the hardware immediately triggers a fault reset. An early kick can indicate that the software has bypassed required algorithmic stages, branched into a pathological short loop, or suffered execution speedup from clock corruption.
2. **Late-kick violation** ($t_{\text{service}} > T_{\text{upper}}$): If the timer counts down past T_{upper} without receiving a valid refresh, the counter underflows, asserting a hardware reset. A late kick can indicate that the enforcer was starved of compute cycles by interconnect contention, deadlocked on an unhandled exception, or delayed by excessive cache evictions.
3. **Valid service window** ($T_{\text{lower}} \leq t_{\text{service}} \leq T_{\text{upper}}$): A refresh inside the window shows the feed path met its timing condition; task progress still needs a suitable challenge or independent supervision.

If the controller feeds once after each successful tick, the watchdog measures the interval *between feeds*. Let completion offset c_n from tick n start lie in $[c_{\min}, c_{\max}]$, and bound tick-phase variation by J_{phase} . A conservative inter-feed envelope is

$$T_{\text{feed}} \in [T_{\text{period}} - (c_{\max} - c_{\min}) - J_{\text{phase}}, T_{\text{period}} + (c_{\max} - c_{\min}) + J_{\text{phase}}].$$

Configure T_{lower} and T_{upper} around that *measured or justified* envelope, while keeping the late-trip response within the plant's separately derived fault budget. A one-tick execution time alone does not set a watchdog service interval.

Some designs add **challenge-response checkpointing**: the feed depends on fresh sensor validation, permission evaluation, and output-state checks rather than a timer ISR alone. Its fault coverage depends on where the challenge is generated, who can compute or replay the response, and whether failed enforcer states can still execute the feed path. The scheme needs fault injection; a token by itself does not prove task progress.

However, establishing clean boundaries in memory, interconnect arbitration, and watchdog execution liveness solves only the logical and timing half of co-location. When the accelerator

¹⁴ | **Windowed watchdog timers:** An autonomous hardware timer that enforces both an upper and lower time bound on software refresh operations (“pets”). Unlike standard watchdogs that reset only on underflow, a windowed watchdog asserts a hardware system reset if serviced too early (which may indicate execution loop corruption or clock runaway) as well as if serviced too late (which may indicate task deadlock or starvation).

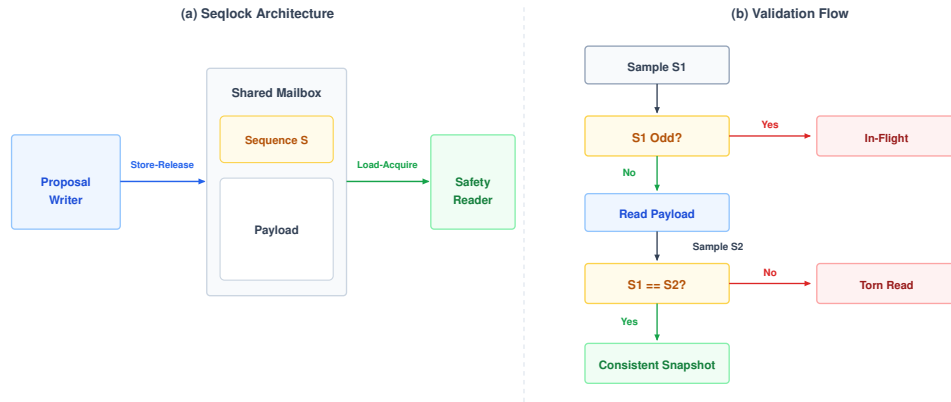


Figure 26.4: Lock-free boundary contract: (a) Inter-core lock-free state exchange architecture where an untrusted proposal writer and a real-time safety reader coordinate through a shared mailbox using atomic memory barriers and a monotonic sequence counter S . (b) Non-blocking validation decision flow where sampling sequence snapshots (S_1, S_2) rejects in-flight updates on one bounded attempt (S_1 odd) or torn reads ($S_1 \neq S_2$) without acquiring locks or waiting on the writer.

and the real-time processor share the same silicon die, the high power dissipated during dense matrix operations conducts directly through the substrate, elevating local junction temperatures and altering transistor switching speeds across adjacent cores.



Checkpoint 26.4: Lock-free state exchange and boundary contracts

- ☐ **Seqlock protocol mechanics:** Can you trace how atomic store-release and load-acquire memory barriers coordinate state exchange through a monotonic sequence counter S without blocking?
- ☐ **Torn read detection:** Can you explain why one bounded sequence check rejects an odd or changed record without relying on an unmeasured universal microsecond latency?
- ☐ **Ring buffer capacity sizing:** Can you size a four-slot audit ring for a 50 Hz producer and a 60 ms *logging* pause while explaining why the enforcement reader still runs every tick?
- ☐ **Windowed watchdog mechanics:** Can you derive why standard countdown watchdogs fail against infinite spin-loops, and how the open window $[T_{\text{lower}}, T_{\text{upper}}]$ catches both starved and runaway execution?
- ☐ **Deterministic fallback matrix:** Can you map the four boundary contract failure classes (malformed, stale, missing, torn) to their bounded physical actuation fallbacks?

26.5 Thermal and Timing Coupling

Every watt dissipated on a silicon die flows into a single thermal domain. In co-located architectures, heat generation extends beyond the compute cores executing the proposal policy. Power dissipation divides across four coupled mechanisms, comprising dense matrix units performing neural inference, general-purpose cores executing the real-time enforcement loop, memory interface transceivers driving high-bandwidth external memory traffic, and parasitic losses inside integrated voltage regulators. While the enforcement loop consumes a predictable, low-power baseline, the proposal path exhibits large dynamic swings depending on batch dimensions and model activation patterns. The package heat spreader, thermal interface material, and protective enclosure present finite thermal conductivity to ambient air, causing heat from peak accelerator activity to raise the junction temperature across adjacent circuits on the die.

In physical AI systems engineering, **thermal clock throttling** is the automatic, hardware-enforced reduction of processor clock frequency triggered when sustained high-power cognitive inference elevates silicon junction temperature (T_j) past thermal trip thresholds (T_{trip}). To prevent damage to



Figure 26.5: Enclosed edge-compute kit: The photograph shows a Jetson AGX Orin Developer Kit box and a closed, vented enclosure. The SoC, memory, cooling hardware, and clock domains are not visible; the thermal model in the text is a separate illustrative scenario. Photograph: Auledas, CC BY 4.0, via Wikimedia Commons; local copy re-encoded.

the die, the hardware abruptly cuts clock speed. This introduces a lingering, temperature-dependent slowdown (hysteresis) that inflates execution times even after the cognitive workload subsides, forcing co-located real-time safety loops to overrun their hard physical deadlines.

Device temperature differs from ambient room conditions. A machine operating in an industrial flow-control enclosure at an ambient temperature of 45.0°C experiences an internal temperature rise governed by total power dissipation and the thermal resistance of the assembly. In a shared-SoC design, neural accelerators, application cores, and real-time processing clusters can share a thermal path. Figure 26.5 shows an enclosed developer kit; its visible enclosure does not reveal the internal heat path or clock domains. Consider an analytical example where a shared system-on-chip is mounted on a conduction-cooled enclosure with an effective **junction-to-ambient thermal resistance** of $\theta_{JA} = 2.0 \text{ K/W}$ and a **thermal time constant** of $\tau_{th} = 8.0 \text{ s}$. When the enforcement loop runs in isolation, drawing $P_{enf} = 1.5 \text{ W}$ alongside baseline idle and memory power of $P_{base} = 2.5 \text{ W}$, total dissipation is 4.0 W , producing a steady-state temperature rise of 8.0 K and maintaining the junction at 53.0°C . When a high-rate vision-language-action policy begins continuous proposal inference, the accelerator draws an additional $P_{prop} = 16.0 \text{ W}$, memory traffic adds $P_{mem} = 3.5 \text{ W}$, and internal power regulator inefficiencies contribute $P_{reg} = 2.5 \text{ W}$. Total power steps to $P_{total} = 26.0 \text{ W}$. The steady-state junction temperature rises by $\Delta T = P_{total} \times \theta_{JA} = 26.0 \text{ W} \times 2.0 \text{ K/W} = 52.0 \text{ K}$, elevating the silicon substrate to $T_j = 97.0^\circ\text{C}$.

Thermal management controllers protect the die from structural degradation through automated clock throttling. When the internal temperature sensor registers a junction temperature crossing a hardware trip threshold $T_{trip} = 95.0^\circ\text{C}$, the clock distribution network forcibly reduces operating frequencies. In this analytical example, the real-time core cluster shifts from a nominal frequency of $f_{nom} = 1.20 \text{ GHz}$ down to a throttled frequency of $f_{throt} = 600 \text{ MHz}$. For an enforcement cycle requiring $N_{cycles} = 8.4 \times 10^5$ cycles to parse telemetry, verify sequence records, compute pressure derivatives, and evaluate safety polygons, nominal execution time is $t_{exec} = 0.70 \text{ ms}$. Within a 1000 Hz control period (1.00 ms tick deadline), nominal execution leaves a timing margin of 0.30 ms . Under the throttled state, execution time scales inversely with frequency, expanding to $t_{exec,throt} = 8.4 \times 10^5 / (600 \times 10^6 \text{ s}^{-1}) = 1.40 \text{ ms}$. The modeled execution path overruns its period by 0.40 ms (a

40 percent period overrun). A qualified independent fallback is needed before this path misses a required control update; the calculation alone does not establish how the machine responds.

Thermal throttling introduces **state-dependent hysteresis** that extends deadline violations long after the triggering load subsides. Hardware thermal managers do not restore nominal clock frequencies immediately when temperature drops below T_{trip} . To prevent the system from frantically toggling between fast and slow clocks as the temperature hovers near the trip line (thermal oscillation), the controller enforces a cooler clearance threshold, such as $T_{\text{clear}} = 85.0^\circ\text{C}$. This requires the junction to cool by a full 10.0 K before restoring the 1.20 GHz clock. After power returns to baseline, Newton's law of cooling governs substrate temperature recovery:

$$T_j(t) = T_{j,\text{target}} + (T_{j,\text{start}} - T_{j,\text{target}})e^{-t/\tau_{\text{th}}} \quad (26.3)$$

where $T_{j,\text{start}}$ is junction temperature when throttling begins, $T_{j,\text{target}}$ is the asymptotic equilibrium temperature under baseline power, and τ_{th} is the package thermal time constant.

In this analytical system, reaching 95°C from 53°C after the power step takes $8\ln[(97 - 53)/(97 - 95)] \approx 24.73$ s under the same first-order model. Once the supervisory system detects the deadline overrun and halts the VLA policy (dropping power to $P_{\text{base}} = 4.0$ W), the target equilibrium drops to $T_{j,\text{target}} = 45.0^\circ\text{C} + 4.0 \text{ W} \times 2.0 \text{ K/W} = 53.0^\circ\text{C}$. Under equation 26.3, the die cannot clear throttling until the junction cools from $T_{j,\text{start}} = 95.0^\circ\text{C}$ down to $T_{\text{clear}} = 85.0^\circ\text{C}$. Solving for the recovery duration yields the minimum clearance interval:

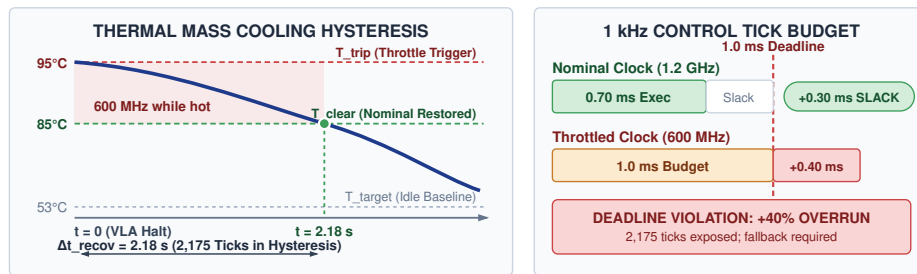
$$t_{\text{recov}} = \tau_{\text{th}} \ln \left(\frac{T_{j,\text{start}} - T_{j,\text{target}}}{T_{\text{clear}} - T_{j,\text{target}}} \right) = 8.0 \text{ s} \cdot \ln \left(\frac{42.0}{32.0} \right) \approx 2.18 \text{ s}$$

At 1000 Hz, the modeled 2.175 s cooldown spans 2,175 scheduled ticks exposed to the throttled clock. It does not authorize 2,175 missed updates: a separately timed fallback or workload limit must be active once the 1.40 ms path cannot meet its 1.00 ms period.

Systems Perspective 26.3: Thermal mass as latency memory

Thermal mass acts as a low-pass filter on power, but a long-duration memory on execution timing: once a heavy cognitive workload trips thermal clock throttling, the real-time core remains trapped at degraded clock frequencies for multiple seconds after the cognitive workload has halted.

Checkpoint 26.5: Junction temperature and clock throttling



- ☐ **Thermal mass latency memory:** Can you explain why physical thermal mass acts as a long-duration memory on real-time task execution latency even after the offending cognitive workload has been terminated?

- ❑ **Cooling recovery sizing:** Can you derive the clearance time t_{recov} required for an SoC with thermal time constant $\tau_{\text{th}} = 8.0$ s to cool from $T_{\text{trip}} = 95^\circ\text{C}$ to $T_{\text{clear}} = 85^\circ\text{C}$ under a 53°C baseline equilibrium?
- ❑ **Governor hysteresis purpose:** Can you explain the purpose of temperature hysteresis (T_{trip} vs. T_{clear}) in hardware thermal governors and describe how it prevents high-frequency clock oscillation?
- ❑ **Physical segregation decision:** What measured clock, power, and timing evidence would justify a shared die, and when would a separate controller be needed?

Diagnosing execution-time expansion requires separating thermal degradation from **electrical power-rail coupling**. Both failure modes originate in heavy accelerator activity, but they operate on distinct physical timescales. Power-rail coupling manifests as instantaneous supply rail depressions across shared power distribution networks within microseconds of a parallel accelerator burst. If the supply voltage sags below minimum operating margins, logic gates lack the energy to switch states fast enough. This causes pipeline timing violations or triggers immediate **brownout resets**, where the processor abruptly reboots to prevent data corruption. Thermal coupling develops over hundreds of milliseconds to several seconds as heat diffuses through silicon and packaging. Correlating execution-time drift against high-frequency oscilloscope captures of core supply voltage $V_{\text{core}}(t)$ and low-frequency telemetry from on-die thermal diodes $T_j(t)$ isolates the root cause. A timing fault that tracks the envelope of temperature rather than instantaneous current transients confirms thermal throttling rather than rail collapse.

Relocating the proposal model to a physically separate processor or discrete companion accelerator eliminates on-die thermal interference, but it alters the timing and resource balance of the entire machine. Moving the neural workload off-chip replaces intra-die shared memory transfers with board-level serial buses such as PCIe or Ethernet. Squeezing wide data streams through serial links (SerDes bandwidth limits), packaging data into network frames, and waiting for congested buses to clear (jitter) collectively add hundreds of microseconds to several milliseconds of latency to proposal delivery. This physical transport delay increases the required validity window Δt_{valid} defined in the causal boundary established in section 2.2. Off-chip transceivers also increase board-level power consumption, shift heat dissipation toward peripheral power converters, and introduce new contention points on external memory buses. Partitioning computation to resolve a thermal deadline violation redistributes electrical, communication, and timing constraints, requiring the complete placement budget to be recalculated from physical measurements.

26.6 Testing the Separation

An independence claim on shared silicon is a testable bound on enforcement latency and refusal availability across declared proposal workloads, chip temperatures, and modes. If contention delays a refusal past the physical deadline, the claimed operational separation fails.

The pass threshold for this experiment derives directly from physical dynamics. Consider a high-pressure gas regulation manifold where downstream pipe compliance and fluid compressibility dictate a hard physical deadline $t_{\text{deadline}} = 15.0$ ms to initiate valve closure before line pressure breaches safe containment limits. Fixed sensing overhead consumes $t_{\text{sense}} = 3.5$ ms for pressure transducer sampling, analog-to-digital conversion, and direct memory access ingest into local buffers. The assumed bounded output overhead is $t_{\text{act}} = 2.1$ ms: 1.6 ms for serial transmission, command cyclic redundancy checks, and coil energization, plus a measured-for-this-scenario 0.5 ms from coil current to first valve motion. Setting aside an instrumentation uncertainty margin $t_{\text{uncert}} = 0.4$ ms to account for measurement jitter and timer resolution, the maximum duration permitted for enforcement computation is derived by subtraction in equation 26.4:

$$t_{\text{enf,max}} = t_{\text{deadline}} - t_{\text{sense}} - t_{\text{act}} - t_{\text{uncert}} = 15.0 \text{ ms} - 3.5 \text{ ms} - 2.1 \text{ ms} - 0.4 \text{ ms} = 9.0 \text{ ms} \quad (26.4)$$

This 9.0 ms boundary is the **falsification threshold**. If any enforcement cycle under test exceeds this value, the independence hypothesis is refuted.

Refuting an independence claim requires synthetic workloads engineered to induce worst-case resource contention rather than standard benchmarks that report average throughput.¹⁵ High

Downstream: Adversarial microarchitectural contention tests inform the hardware fault injection suite in section 30.1.

15 | **Microarchitectural Timing Anomalies:** In out-of-order and speculative processing architectures, a local performance improvement—such as a cache hit or accelerated branch evaluation—can alter instruction scheduling and resource contention, resulting in a net increase in global execution time. This counterintuitive behavior invalidates simple compositional worst-case execution time analysis. Safety verification needs a justified bound for the declared workload, arbitration, temperature, and fault envelope; measurements alone and a non-speculative core alone do not establish a hard end-to-end bound.

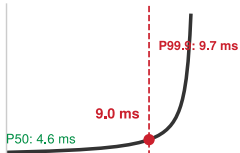
average utilization schedules memory accesses evenly across clock cycles, making the system look safely underloaded, whereas actual physical interference peaks during synchronized bursts when multiple components demand the same wires at the exact same instant. The test framework must deliberately flood shared memory channels with unaligned direct memory access transfers, thrash shared cache capacity with dirty line writebacks from transformer weight tensors, trigger accelerator matrix multiplications that induce rapid supply rail voltage droops ($L \cdot di/dt$), and stream peak-rate sensor telemetry while diagnostic logging writes to non-volatile flash storage. These stress profiles reproduce the transient contention that occurs when a proposal model initiates a large attention pass at the exact microsecond the flow enforcement loop reads pressure telemetry.

Instrumentation records enforcement entry after the sensor snapshot is coherent (t_0), permission completion (t_1), and command queue publication (t_2). These diagnose compute and dispatch, but t_2 is not valve closure. A drive-pin or coil-current edge (t_3) and measured valve-motion onset (t_4) complete the physical chain. In the separate 15.0 ms manifold scenario, compare measured t_4 minus physical pressure-event onset, including clock error and actuator response, against the declared closure-start deadline; the 9.0 ms computation allowance alone is insufficient.

Coupling mechanisms compound across physical, electrical, and logical domains, requiring a full factorial experimental matrix. The evaluation sweeps across cold junction conditions ($T_j = 35^\circ\text{C}$) and warm junction conditions ($T_j = 85^\circ\text{C}$), an idle proposal path versus a saturated proposal engine executing maximum-batch inference, diagnostic circular logging disabled versus streaming at full non-volatile memory bandwidth, and nominal tracking commands versus boundary-violating commands that trigger refusal routines. Testing refusal paths under full stress is mandatory because executing a safe clamping trajectory or entering emergency shutdown involves cold branch paths, non-cached error-handling code, and secondary bus transactions that never execute during steady-state flow regulation.

Consider a *constructed* 10^7 -invocation stress trace for the 15 ms manifold scenario. If bench probes are spaced 15 ms apart, the trace spans 150,000 s, or 41 h 40 min, before reset or setup time. This bench spacing is not the deployed sampling period and does not test worst-phase pressure-event response. The illustrative cold trace has $P_{50} = 1.8$ ms, $P_{99} = 2.4$ ms, and observed maximum 2.9 ms; the warm, loaded trace has $P_{50} = 4.6$ ms, $P_{99} = 8.1$ ms, $P_{99.9} = 9.7$ ms, and observed maximum 11.4 ms. The loaded observations cross the 9.0 ms compute threshold and refute the claimed bound for this constructed test. A percentile describes this trace, not a confidence level or an always-valid bound; a real qualification must measure the complete pressure-event-to-valve-motion endpoint.

Completing millions of test cycles without a timing violation cannot prove absolute independence. It supports only that observed interference stayed below the threshold in the tested states. A report gives the latency distribution, observed maximum, sample count, duration, and unexercised conditions, including supply degradation, electromagnetic interference, and aging.¹⁶



Under memory bus contention, P99.9 tail latency breaches the 9.0-millisecond threshold, refuting independence.

¹⁶ | **Silicon Substrate Aging and Timing Margins:** Long-term substrate degradation mechanisms, such as Negative Bias Temperature Instability (NBTI) and Hot Carrier Injection (HCI), gradually increase transistor switching delays over years of operation. When coupled with transient supply-voltage droop under sudden multi-core current steps ($\frac{di}{dt}$), aged silicon experiences propagation delays that exceed design margins. In hard real-time systems, these physical timing degradations cause intermittent race conditions and missed scheduler deadlines that never manifest during clean bench qualification.

Checkpoint 26.6: Empirical isolation stress testing and WCET bounds

Verify whether your timing qualification methodology actively falsifies independence claims under adversarial hardware workloads:

- ☐ **Synthetic microarchitectural contention:** Can you identify the specific stress patterns (e.g., streaming DMA bursts, pseudo-random cache thrashing, power-rail current steps) required to provoke worst-case bus blocking?
- ☐ **Average versus tail latency fallacies:** Can you explain why proving that average memory bus utilization is below 25 percent provides zero guarantee against a multi-millisecond deadline overrun during coincident burst traffic?
- ☐ **Microarchitectural timing anomalies:** Can you describe how out-of-order execution pipelines and speculative execution can cause a cache hit to paradoxically increase overall worst-case execution time?
- ☐ **Predeclared falsification criteria:** Can you define the measured endpoint and threshold that would require a placement remedy, with relocation if shared-device controls cannot meet the bound?

Nominal benchmarks cannot establish interference bounds. A loaded pass supports only the tested workload, environment, sample size, and fault conditions; untested conditions remain open.

⚠ War Story 26.1: Apollo 11 radar load and computer restarts (1969)

Context: On July 20, 1969, the Apollo 11 Lunar Module *Eagle* was executing its powered descent toward the lunar surface. The Apollo Guidance Computer (AGC)—a revolutionary 1.024 MHz core-rope memory processor—executed hard real-time guidance, navigation, and descent engine throttle loops under an asynchronous, priority-scheduled executive designed by Margaret Hamilton’s team at MIT (NASA Manned Spacecraft Center 1969; Mindell 2008).

Mechanism: During descent the rendezvous-radar switch was in SLEW, and the radar resolver produced counter activity that consumed computer time. The 800 Hz figure describes resolver excitation, not an interrupt rate. NASA’s mission report, section 16 describes up to 12,800 counter pulses per second, about 15 percent of AGC computer time at $11.7\ \mu\text{s}$ per pulse. Recurring jobs then accumulated and exhausted Executive work areas; the switch position alone does not establish that AUTO would have avoided every mismatch.

Impact: The overload produced 1201/1202 Executive alarms during descent. The primary account does not establish a missed descent-engine throttle deadline, an inevitable abort, or a crash.

Response: The AGC restarted and selectively resumed essential guidance and display work according to priority. The crew and mission control assessed the alarms while descent continued. This was a priority-based recovery from excess computer-time demand, not evidence of a particular memory-bus burst.

Systems lesson: A peripheral can consume execution capacity at the same moment guidance needs it. Modern placement tests should account for asynchronous peripheral load and verify that the critical path and its recovery behavior remain within their declared timing envelope.

Ariane 5 Flight 501 (Lions 1996) illustrates a different failure: an unprotected conversion of the SRI alignment variable BH (horizontal bias, related to horizontal velocity) from 64-bit floating point to 16-bit signed integer overflowed outside its assumed envelope. Both identical SRI units stopped; the active unit sent a diagnostic bit pattern over the data bus, which the on-board computer treated as flight data and commanded extreme nozzle deflection. The inquiry report identifies a software specification and exception-handling failure, not a DRAM scheduling failure.

In contrast, Apollo 11’s Executive alarms and the Mars Pathfinder priority inversion (examined in section 8.4) illustrate distinct forms of real-time resource contention. Apollo 11 recovered by restarting and selectively resuming priority work; the episode does not identify a modern DRAM queue or prove an actuator deadline miss.

On modern safety silicon, placing high-throughput cognitive policies and hard real-time safety reflexes on a single die exposes the physical machine to the hardware analog of both pathologies. The following analytical scenario demonstrates how unexercised coupling paths breach physical boundaries on shared silicon. On the bench, the proposal path executed steady-state flow optimization while the enforcement loop processed nominal valve adjustments. In the field, an abrupt upstream pressure spike caused the enforcement monitor to trigger an emergency refusal and command rapid valve closure. Simultaneously, the policy engine detected an anomaly and initiated a high-priority diagnostic memory dump over the shared direct memory access bus. The diagnostic transfer saturated the memory arbiter for 8.9 ms, expanding the refusal calculation time from its nominal 3.2 ms to 12.1 ms. Because the bench test suite had evaluated refusal logic and diagnostic memory logging in isolation rather than in combination, the arbiter lockup went undetected until the delayed valve command allowed line pressure to rupture the downstream burst disc. The numbers here are illustrative; what is not illustrative is the structure, in which two subsystems that pass every test separately contend for one memory controller the moment they run together.

A failed interference test identifies where the assumed separation may collapse: cache, memory arbitration, thermal clocks, power, or output queues. Further instrumentation localizes the mechanism before the team reserves, partitions, or relocates the shared resource.

26.7 Silicon Isolation Decisions

An empirical test that falsifies an independence claim—showing that two software tasks intended to run in isolation physically interfere—demonstrates that a boundary on an architectural diagram does not exist in the silicon. Resolving that failure requires an explicit engineering choice among four distinct interventions, comprising static reservation, spatial partitioning, workload reduction, and physical relocation to separate silicon. As structured in the decision framework (table 26.4), each intervention trades specific performance margins and physical costs to restore predictable bounds to the nervous system.

Table 26.4: Workload re-placement and isolation decision matrix: Systematic evaluation of remediation pathways when empirical testing falsifies independence on shared silicon. Each tier trades distinct engineering currencies (throughput, capacity, latency, board footprint) to eliminate contention channels before escalating to physically segregated silicon.

Remediation Strategy	Target Coupling Layer	Implementation Mechanism	Systems Engineering Trade-Off	Residual Vulnerability	Physical Silicon Migration Trigger
Static Reservation	CPU execution time, OS scheduling preemption	Fixed-priority preemptive scheduling (SCHED_FIFO), static time-slot slicing (ARINC 653)	Idle capacity (20–40% throughput loss); unallocated cycles wasted during quiescent states	Micro-architectural contention beneath OS visibility (shared L3, DRAM bank conflicts)	Memory bus or cache stalls violate real-time deadlines despite 100% CPU core allocation
Spatial Partitioning	Cache capacity thrashing, DRAM bus contention	Cache way-partitioning (ARM MPAM, Intel CAT), MPU/IOMMU domains, AXI QoS rate limiters	Structural fragmentation; reduced L3 working set for perception (+15–30 ns DRAM access penalty)	Unpartitionable DRAM controller queues, row-buffer locality reordering, PDN voltage droop	Matrix engine current steps ($\Delta I > 10$ A) trigger rail droop, or DRAM queue stalls exceed slack
Workload Reduction	Memory throughput saturation, peak thermal power	Model quantization (INT8/FP8), context downsampling, reduced policy rate (50 Hz $\rightarrow$ 20 Hz)	Degraded policy tracking fidelity; increased control phase lag near dynamic stability boundaries	Common clock distribution trees, shared Power Management Units (PMU), unified reset domains	Model accuracy drops below operational limits, or shared clock/reset faults propagate across domains
Physical Relocation	Shared power, clock, reset, or unbounded controller interference	Separate enforcer MCU with independently assessed power, clock, memory, and sensor/actuator path	More board area and an external communication budget	Harness, transceiver, and common-supply faults remain	Choose when the shared die cannot provide evidence for required timing and fault containment

17 | **Automotive Freedom From Interference:** ISO 26262 Part 9 formalizes Freedom From Interference (FFI) as the absence of cascading faults between software elements with different safety integrity levels. In heterogeneous robotics SoCs, achieving FFI requires hardware-enforced spatial partitioning (memory protection units, IOMMU page tables) and temporal partitioning (fixed-priority bus arbitration, TDMA interconnects) to prevent unprivileged perception models from degrading safety-critical enforcement tasks.

Static reservation preserves the physical placement of workloads on the shared chip while allocating a guaranteed fraction of a shared resource to the critical path. Reservation assigns dedicated execution slots or fixed memory bus bandwidth to the nominal tracking loop. The cost of reservation is idle capacity. If the tracking loop requires 2.0 ms of memory arbiter access during every 10 ms control window to guarantee deadline compliance during fluid pressure transients, that 20 percent slice of memory bandwidth remains unavailable to the background model during steady-state operation, even when the valve position is stationary and no fluid correction is needed.

Spatial partitioning divides physical hardware structures so that workloads no longer compete for the same physical cells.¹⁷ Cache way-partitioning, core pinning, and dedicated memory channels establish hardware barriers within a single die. The cost of partitioning is lost flexibility and structural fragmentation. Splitting an eight-megabyte shared cache into a six-megabyte partition for the policy

model and a two-megabyte partition for flow safety logic prevents cache line evictions across the boundary. However, it permanently restricts the working set of the learned model, forcing higher DRAM access rates and increasing mean memory latency by tens of nanoseconds on every inference step.

Systems Perspective 26.4: Substrate failure domain invariant

Separate packages are an architectural choice when timing or fault containment cannot be evidenced on a shared device. Integrity targets require an application safety case, not a universal silicon count.

Workload reduction resolves contention by shrinking the computational or memory footprint of the non-critical workload until its peak demand falls within the uncontended margins of the shared system. An engineer might downsample telemetry logging from 1000 Hz to 100 Hz, quantize policy weights from sixteen-bit floating point to eight-bit integers to cut memory bus traffic in half, or lower the execution frequency of the flow optimization model. The cost of reduction is degraded functional authority. Lowering the update rate of the policy increases the phase lag of its control proposals, while weight quantization can degrade the accuracy of flow predictions near non-linear valve operating limits.

If reservations, partitioning, and workload reduction cannot establish the required response bound, physical relocation is a candidate remedy. An external controller adds transport and common-supply obligations, so its sensor and actuator path must be measured as a whole.

Electrical or thermal coupling may motivate relocation when the required rail and timing independence cannot be evidenced on the shared die. When high-performance neural compute cores draw transient currents of tens of amperes during complex inference bursts, localized voltage droop ($L \cdot di/dt$) on shared power rails can force the system-on-chip power management unit to throttle clock frequencies across the entire die. If that thermal or electrical throttling slows the nervous system's flow monitoring loop from 1000 Hz to 500 Hz, the safety mechanism violates its control deadline through physical rail coupling rather than bus contention. Furthermore, sharing a single substrate couples the failure domains. If policy and enforcer share a reset or clock domain, a fault in that domain can halt both; a partitioned SoC may instead contain the fault, which must be shown by design and injection tests. A separate controller can reduce these common-cause paths if power, clock, reset, sensor, and actuator independence are designed and tested. The two SoC packages visible in figure 26.6 alone do not establish those properties. Tesla's 2019 Hot Chips presentation describes two SoCs and redundant supplies in its FSD computer, while clock, reset, and failover timing still need design evidence.

Relocating a workload to separate silicon is rarely a localized modification. It triggers a cascade of placement and communication adjustments across the entire machine. In this *illustrative valve design*, moving the enforcement monitor off-chip replaces an assumed $0.2 \mu\text{s}$ internal exchange with an assumed 1.5 ms one-way external transfer. Actual SPI, CAN, or Ethernet latency depends on payload, schedule, transceiver, and load; the full command and sensor paths must be measured.

This added transport latency consumes a substantial portion of the total reaction budget. For the same constructed design, a 10.0 ms response limit minus two assumed 1.5 ms transfers leaves 7.0 ms for sensing, computation, and valve response. The engineering team must then re-evaluate the placement of upstream sensor interfaces, deciding whether pressure transducers should wire directly to the safety microcontroller or remain on the primary board. Moving the sensor interface offloads the main chip and eliminates communication latency for the safety loop, but it simultaneously forces the learned policy model to receive its observation stream over the slower external link. Every re-placement decision propagates through the physical interconnects, shifting bottlenecks from on-die memory arbiters to board-level pin counts, wiring runs, and serial buses.



Figure 26.6: Automotive domain-controller board: The image identifies two SoC regions, nearby memory and power circuitry, and camera-interface hardware. Separate power, clock, reset, or failover behavior cannot be established from the visible board alone; those properties require schematics and fault tests. Photograph: Votpuske, CC BY 4.0 via Wikimedia Commons; local copy re-encoded.



Checkpoint 26.7: Silicon isolation trade-offs and physical migration

- ☐ **Remediation hierarchy trade-offs:** Can you evaluate the systems engineering trade-offs between static reservation, spatial partitioning, workload reduction, and physical relocation?
- ☐ **Hardware unpartitionability:** Can you identify the specific microarchitectural structures beneath operating system visibility that render software-only partitioning ineffective under heavy AXI burst contention?
- ☐ **Physical migration penalties:** Can you recompute the constructed 10.0 ms response budget with two assumed 1.5 ms transfers and identify what needs hardware measurement?
- ☐ **Common-cause failure isolation:** When can a shared SoC provide measured fault-containment evidence, and when does the application require a separate controller?

26.8 Hardware Allocation

Every placement choice trades off communication latency and resource contention; engineering teams must avoid unwritten assumptions about boundary conditions. The **placement record** formalizes this physical topology in the systems engineering notebook. Building upon the baseline schema, versioning rules, and provenance requirements established in chapter 8, this chapter introduces the specific descriptors required to capture on-die co-location and silicon contention.

The placement record combines a fixed-size topology header with a linked empirical test ledger. The illustrative `HeterogeneousPlacementRecord_t` header in table 26.5 binds ten fixed-width identifiers, configuration values, and observed timing fields; the digest at its end identifies the external evidence. The ledger holds inventories, distributions, sample counts, test conditions, mitigations, and open limits.

Table 26.5: Fixed-size heterogeneous SoC placement header (HeterogeneousPlacementRecord_t): The 128-byte header records configuration and observed test maxima; its digest links to a separate immutable ledger holding distributions, sample count, physical endpoints, provenance, and untested conditions. It does not by itself establish Freedom From Interference.

Offset & Field Name	Type & Size	Units & Topology	Structural Invariant & Bound	Hardware Audit Substrate
0x00: execution_domain_id	uint64_t (8B)	Core Enum	Must match declared topology	Memory Protection Unit (MPU)
0x08: privilege_and_ring	uint64_t (8B)	Privilege Level	Proposal $\leq$ EL0, Enforcer $\geq$ EL1/Secure	ARM TrustZone / HSM
0x10: nominal_period_ns	uint64_t (8B)	ns	$T_s \in [10^6, 5 \times 10^7]$ ns (1.0 ms enforcer)	Free-running monotonic timer
0x18: hard_deadline_ns	uint64_t (8B)	ns	Example: 1.0 ms period less 0.2 ms reserved transit = 0.8 ms	Monotonic timer and output probe
0x20: unloaded_observed_max_ns	uint64_t (8B)	ns	Example observed maximum 135,000 ns	Timestamped test ledger
0x28: loaded_observed_max_ns	uint64_t (8B)	ns	Example observed maximum 380,000 ns under declared load	Timestamped test ledger
0x30: shared_axi_channel_qos	uint64_t (8B)	Configured QoS	Example tags: MCU 0xF, NPU 0x4; verify controller semantics	Register capture and stress test
0x38: max_pdn_droop_thermal_trip	uint32_t[2] (8B)	mV & °C	$\Delta V_{\text{droop}} \leq 70$ mV, $T_{\text{trip}} \leq 95.0^\circ\text{C}$	On-die comparator & thermal diode
0x40: hardware_mitigation_flags	uint8_t[32] (32B)	Bitfield	Active: TCM SRAM, MPAM, Isolated Rail	Register lock (SYSCON_LOCK)
0x60: test_ledger_sha256	uint8_t[32] (32B)	SHA-256	Digest of immutable external evidence ledger, including model and firmware hashes	Verified ledger store

As a separate 1 kHz placement-record example, an MCU may have a 0.8 ms computation deadline after reserving 0.2 ms for output and actuator transit. Example domain IDs are MPU, NPU, GPU, MCU, and lockstep controller. The illustrative header above records 135 μs unloaded and 380 μs largest observed in a declared synthetic load. Neither sample maximum is a proven WCET; the external ledger must specify the tested distribution and unexercised states. This case is distinct from the 120 μs diagnostic subdeadline and the 15 ms manifold example.

The external ledger inventories memory channels, cache partitions, DMA engines, power rails, and their competing claimants. For each it records the arbitration configuration and measured interference. A constructed 45 $\rightarrow$ 310 ns LPDDR read inflation would motivate testing a local TCM snapshot, but one access-latency observation does not by itself prove a loop deadline or mandate that remedy.

An interference measurement needs its operating state: model and firmware hashes, clock policy, enclosure and thermal interface, ambient and junction temperatures, power rail, arbitration settings, traffic generators, sampling rates, instrumentation, sample count, and test duration. The fixed-size header links to a separate immutable test ledger carrying those variable-length fields, latency distributions, observed maxima, and unexercised states. A chosen 10^7 cycles at 1 kHz span 2 h 46 min 40 s; that count alone does not establish confidence in a $P_{99.99}$ tail.

Every placement entry requires a predeclared falsification criterion written before the test executes. This criterion specifies the physical threshold that constitutes architectural failure, such as any measured valve command endpoint beyond the declared 0.80 ms limit during simultaneous neural accelerator bursts and DMA transfers. The record logs the measured empirical distribution against this criterion, noting whether the tested claim was not falsified or was falsified. When interference violates the target budget and forces an engineering change, the record details the exact mitigation applied, such as enabling memory bus bandwidth throttling on the neural accelerator or moving the pressure sensor DMA buffer to an isolated Tightly Coupled Memory (TCM) bank. The entry must also document the boundary conditions that were not exercised during the test run, such as

operating states above 45°C ambient temperature, multi-bit memory error injection, or simultaneous sensor transceiver fault states.

Documenting these boundaries during integration provides the foundation for safety analysis in later chapters. The placement record cannot be generated during the logical architecture design in chapter 24, because logical block diagrams do not contain physical pin assignments, clock trees, or memory arbiter contention profiles. Instead, this record is handed forward directly to the verification and release workflows. Chapter 30 consumes the recorded arbitration points and shared power rails as the target map for hardware and software fault injection, deliberately corrupting shared structures to test whether fault propagation matches the predicted failure domains. Chapter 32 incorporates the recorded loaded execution bounds and unexercised test limits into the formal safety case, transforming empirical measurements into evidentiary arguments for operational deployment.

A placement record applies to its declared workload, temperature, supply, fault, and arbitration envelope. New operating conditions require additional evidence.

Downstream: Hardware placement safety cases provide evidentiary warrants for production deployment in section 32.3.

🔍 Systems Perspective 26.5: Torso thermal and power bus packaging

Silicon placement within a humanoid torso presents an unforgiving co-design problem across compute density, thermal dissipation, and electrical transient stability:

- **Enclosed Torso Thermal Packaging:** A humanoid torso may place cameras, neural compute, real-time control, and DC-DC power stages within one thermal enclosure. In an illustrative high-power design, sustained inference can raise junction temperature and cause clock derating. The cooling and balance trade-off depends on the actual module power, chassis geometry, and measured thermal path.
- **Transient Power Bus Coupling under Multi-Joint Lunges:** A shared motor-and-compute supply may sag during multi-axis current steps. The example 48 V bus and 150 A/ms slew are scenario inputs; whether a compute reset follows requires battery impedance, wiring inductance, hold-up energy, and undervoltage thresholds. Separate regulators or local hold-up are options to validate against the full maneuver.
- **Cross-Die Contention on Heterogeneous SoCs:** Shared LPDDR and cache can delay a balance controller when inference traffic is concurrent. Place time-critical state on a verified local path or separate controller if the shared-device interference bound cannot be met; neither topology is sufficient without tested power, clock, and sensor/actuator paths.

26.9 Fallacies and Pitfalls

Placement determines the physical resources shared by complex proposal algorithms and critical enforcement routines. Traces of normal operation cannot prove these systems are truly independent when rare physical events—such as power voltage transients, clock faults, memory contention, or a stalled producer—stress those shared hardware pathways.

Fallacy: *Logical privilege rings protect real-time enforcers from shared-rail electrical droop.*

A gas-flow controller places a neural accelerator and a lockstep enforcement core on one SoC. An accelerator current burst (di/dt) depresses their shared power rail, causing the power-management circuitry to reduce the enforcer clock from 400.0 MHz to 200.0 MHz to prevent a complete brownout. The enforcement task exceeds its physical pressure-response deadline, despite its highest-priority software privileges. Access control prevents unauthorized memory writes, but it cannot preserve execution timing through the physical electrical transient. Placement qualification must test shared-rail interference and establish adequate capacitance decoupling, di/dt slew limits, or physical supply separation so accelerator activity does not consume the enforcer's timing margin.

Pitfall: *Sharing Phase-Locked Loops or reset controllers across proposal and permission domains.*

A propellant regulator uses one shared Phase-Locked Loop (PLL) and one reset controller for both the proposal and permission domains (section 6.1). A thermal clock fault or a watchdog reset of the stalled policy cluster can therefore abruptly halt the critical enforcement domain as well. The nominal

block diagram shows separate processors, but both depend on fragile silicon infrastructure that a single event can silently disable. Trace clock and reset trees through the physical implementation to ensure independent resources for continuous enforcement. The physical valve drivers also need a defined hardware transition to their passive return state (e.g., via pull-down resistors) when active PWM control becomes unavailable; restarting the policy processor cannot be the only recovery mechanism.

Fallacy: *Low average memory bus utilization guarantees bounded read latency for real-time safety tasks.*

In a constructed metering example, 28 percent average bus use hides camera and logging bursts that coincide with enforcement reads. An assumed $840\mu\text{s}$ read tail in a 1.0 ms tick leaves only $160\mu\text{s}$ for all remaining work, so this configuration fails if compute and output need more. A coherent TCM snapshot may remove the shared-DRAM read, provided its DMA completion, capacity, and local transfer time are qualified. Average utilization alone cannot establish the deadline.

Pitfall: *Using blocking software mutexes across inter-processor communication boundaries with untrusted neural policies.*

A gas-blending policy crashes halfway through publishing a 256 byte flow schedule to shared memory. If the physical enforcer must acquire a hardware mutex (or spinlock) held by that producer, the policy failure effectively freezes the enforcer indefinitely. A wait-free publication protocol (such as a sequence lock or seqlock) allows the consumer to identify an incomplete or “torn” data record and cleanly reject it without ever blocking, but its retry behavior must remain tightly temporally bounded. The safety controller must proceed with its defined fallback actuation instead of waiting for the crashed producer to finish or repair the record. The physical communication contract must specify timing behavior for interrupted publications as rigorously as it specifies the payload format.

26.10 Summary

Heterogeneous SoC architectures resolve the tension between deep learning throughput and real-time determinism by partitioning workloads across distinct fault domains, isolating untrusted neural proposal policies on Linux application processors (MPUs) from deterministic safety enforcers on bare-metal microcontrollers (MCUs).

Physical isolation must be supported by measured bounds on memory queues, crossbar arbitration, power transients, and the actuator path. A placement result is an empirical claim scoped to the measured envelope. It does not establish that two software components are independent in an absolute sense. Instead, the placement record demonstrates that under an analytical stress workload executing at a specific ambient temperature, with fixed clock governance policies and documented fault domains, the shared physical substrates of the die did not cause the untrusted path to violate the timing constraints of the safety path. On a gas blending manifold regulating high-velocity flows at 25.0 m/s , drawing the policy and the safety enforcer in separate boxes on an architectural diagram does not isolate them from a shared voltage regulator, a common memory crossbar, or the heat sink clamped across both processor clusters. If the matrix multiplication core executing the learned flow proposal draws peak current while all four memory channels service direct memory access transfers, the resulting voltage drop and thermal dissipation affect the enforcer core immediately. The boundaries drawn during system design exist on the silicon only when verified against these shared electrical, thermal, and architectural limits.

The placement record gathers five kinds of evidence: loaded enforcement latency and observed maximum for the declared test envelope; shared-resource queue and power headroom; common power, clock, reset, and memory failure paths; measured malformed-record and stalled-producer response; and explicitly untested conditions. For the constructed FIFO diagnostic example, read plus compute reaches $128.364\mu\text{s}$ against a $120\mu\text{s}$ subdeadline, while the assumed 800 MHz local snapshot path reaches $105.24\mu\text{s}$. Neither value transfers automatically to the separate manifold or placement-record cases. The ledger records the arbitration policy, clocks, sample count, and physical endpoint before any release claim.

With the completion of this placement analysis, the five architectural handoffs of Part III form a single measured machine. Data flow from sensor ingress to state estimation, from estimated state to policy proposal, from proposal to reflex enforcement, and from enforcement to valve actuation can no longer be treated as independent software modules communicating across idealized channels. The

logical boundaries established across earlier chapters provide the structure needed to reason about authority and failure propagation, but physical reality governs their execution. A memory controller follows configured arbitration and row scheduling, which need not reflect software privilege, and an interrupt can arrive during either workload. When a valve controller commands a proportional solenoid to alter gas mixture ratios, the action is the culmination of a pipeline whose timing, power, and memory footprints have been measured together on one die.



Key Takeaways: Heterogeneous silicon placement and memory contention

- **Logical Privilege Rings Do Not Isolate Shared Electrical Rails:** The worked 6 A in 2 ns current step gives 138 mV modeled droop. Device rail thresholds and execution response must be measured; access control does not remove electrical coupling.
- **Clock Distribution Networks and Reset Lines Are Common-Mode Vectors:** Sharing on-chip Phase-Locked Loops (PLLs) or reset controllers across proposal and permission domains allows an unhandled exception in an unverified policy engine to reset or glitch the real-time enforcer's register files.
- **Phase-Aligned Bus Collisions Threaten Real-Time Timing:** Average utilization cannot bound a queued read. The explicit 88-transaction FIFO scenario gives $128.364\ \mu\text{s}$ read plus compute against a $120\ \mu\text{s}$ subdeadline; test the actual arbiter and local snapshot path.
- **Untrusted Policy Engines Need Bounded Publication:** One sequence check rejects a torn mailbox snapshot without waiting; checksum, age, state, and fallback checks still require measured time bounds.
- **The Placement Deliverable Quintet:** Before deploying an architecture, five physical invariants must be measured under concurrent maximum load: loaded enforcement latency ($P_{99.99}$), shared-resource headroom, common-mode failure paths, detector fault coverage, and cataloged untested boundaries.

Logical boundaries do not isolate shared rails or memory queues. The placement decision must preserve the independent permission path under the tested interference envelope and record where its timing evidence ends.



What's Next: From compute placement to supervisory intervention

With physical placement and bus contention quantified across the silicon substrate, **Part III (Running the Machine)** concludes having integrated the end-to-end runtime pipeline. **Part IV (Governing the Machine)** steps back from execution mechanics to confront the disciplines of justification, verification, and operational authority. Chapter 28 opens this governance framework by addressing supervisory intervention: examining how the nervous system arbitrates authority between learned policies and human supervisors, executing bumpless, deterministic transfers that preserve plant stability and acceleration limits when physical reality diverges from the autonomous plan.

IV

GOVERNING THE MACHINE

Governing the Machine

DEMONSTRATING a physical AI model in a controlled laboratory is a milestone. Outside the workbench, dust obscures lenses, lighting changes, wireless networks drop packets, gearboxes wear, and human coworkers share the machine's path. Each change can invalidate a premise used to permit motion.

Governing the machine means deciding who may command it, what evidence supports that authority, and how the machine responds when the evidence expires.



Principle 10: The bumpless authority handshake

Invariant: A handoff can command a torque step if human and autonomous controllers disagree about the current state or target. For a switch from autonomous to human proportional position control, assume both controllers use the same measured joint position and gain k_p , with q_{human} and q_{auto} denoting their target positions. The *requested* human-minus-autonomous torque difference is

$$\Delta\tau_{\text{cmd}} = k_p(q_{\text{human}} - q_{\text{auto}}).$$

The resulting motion depends on actuator limits, inertia, load, and the response delay; a command step alone does not establish damage or an impulse in the plant.

Implication: Admit a transfer only after the controllers agree on a fresh state, the intended mode, and a feasible transition. Chapter 28 develops a four-phase handshake and C^2 reference blend as one implementation; its timing and smoothness limits must be checked against the particular plant. One permission path retains actuator authority throughout the handoff.

Part IV follows three decisions:

- **Shared authority with humans (chapter 28):** Bound the delay and state mismatch in a handoff, keep one actuator writer, and record which authority was active.
- **Testing the machine (chapter 30):** Use software, processor, hardware, and physical tests to challenge the claimed timing, sensing, and fallback envelope under injected faults.
- **Deciding to operate (chapter 32):** Link each operating claim to evidence and limits, then decide which conditions the machine may enter. ISO 26262, ISO 21448, and UL 4600 are examples for road-vehicle or autonomous-vehicle applications; other machines need standards and hazards matched to their use.

The book then examines the remaining evidence limits in chapter 34.

III Principle 11: Hardware-in-the-loop qualification

Invariant: A simulation tests the physical effects represented by its model. Bus arbitration, deserializer jitter, thermal throttling, rail droop, and backlash may be absent or miscalibrated; their absence in a simulation says nothing about whether they will occur on a particular machine.

Implication: Select software-in-the-loop, processor-in-the-loop, hardware-in-the-loop, and in-situ tests for the claims each can challenge. Inject degraded sensing, bus, rail, and actuator conditions, and measure the physical response. A passing test supports the declared configuration and tested envelope; untested conditions remain open.

III Principle 12: Goal-structured operational evidence

Invariant: A “99.9% success rate” lacks a safety meaning until the event, denominator, operating conditions, and uncertainty are defined. Zero observed failures give a conditional statistical bound, not proof of safe deployment. Some operational design domain (ODD) limits can be monitored online; others require passive protection or an operating restriction.

Implication: A claim-argument-evidence safety case can connect operating limits, tests, fault coverage, and residual risk; Goal Structuring Notation (GSN) is one way to organize it. Record field events and software versions so a release decision can be revisited. A software rollback cannot undo physical motion, so an ODD exit also needs a separately validated, physically feasible response.

III Principle 13: The epistemic containment principle

Invariant: Suppose independent operating hours have the same hazardous-event probability $p = 10^{-9}$, or a constant-rate Poisson model applies. With zero observed events, a one-sided 95 percent upper-bound argument needs approximately

$$N \geq \frac{-\ln(0.05)}{10^{-9}} \approx 3.0 \times 10^9 \text{ failure-free hours.}$$

Correlated exposure or a changed hardware/software configuration limits how much those hours say about the next deployment. Empirical exposure alone is therefore impractical for this particular target.

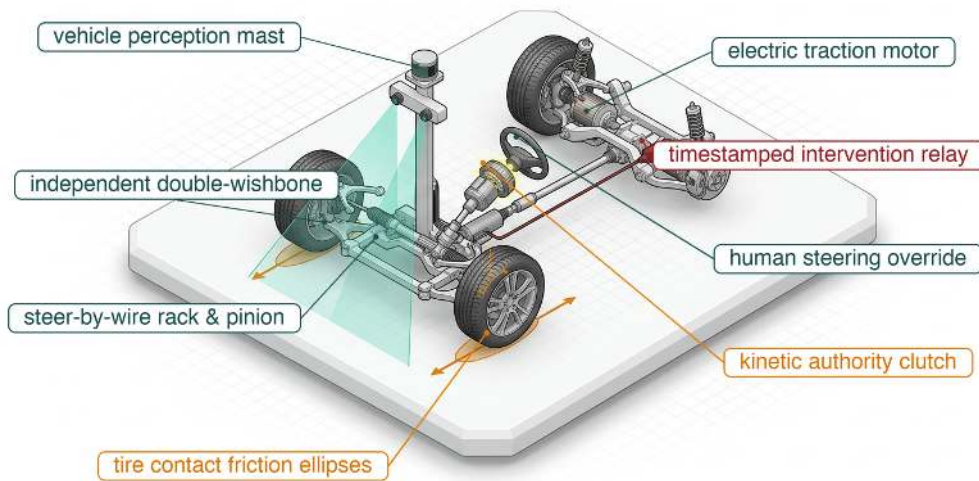
Implication: Bound learned authority with a permission and fallback path whose sensing, state estimate, model, actuator capacity, timing, and stopping clearance have been validated for the operating envelope. When those conditions cannot be established, restrict the task or stop where a stop is physically feasible. A runtime barrier is a conditional protection, not an unconditional guarantee.

These decisions make authority, evidence, and the machine’s response reviewable before it operates among people.

Supervisory Intervention

DRAFT

Supervisory Intervention



- 28.1 Shared Authority
- 28.2 Human Authority Over Actuation
- 28.3 Authority Transitions
- 28.4 Transfer Without Discontinuity
- 28.5 Authenticating an Intervention
- 28.6 Trustworthy Operational Logs
- 28.7 How Handover Goes Wrong
- 28.8 The Authority Log
- 28.9 Fallacies and Pitfalls
- 28.10 Summary

Purpose

When a human grabs the controls of a running machine, how do you transfer kinetic authority without causing the very collision you meant to prevent?

A request to take control reaches a machine already moving under another command. Combining commands without a defined handover can produce unintended motion. Abruptly switching between them can also impose a mechanical shock. Intervention must assign authority to specific operations, establish when the transfer takes effect, and preserve a feasible command throughout the transition. Human reaction time and communication delay consume the same clearance budget as automated computation, so an operator cannot serve as an assumed instantaneous fallback. The machine needs a defined response when a takeover request or acknowledgment arrives late. Recording the transfer matters as well, because a successful human rescue must not appear in the evidence as autonomous success. Supervisory intervention extends the contract among brain, nervous system, and body to a human authority whose commands remain subject to physical limits and execution deadlines.

Brain
1–50 Hz
foundation models, SLAM, planning

above: propose; below: command

Nervous System
1000 Hz
TSN, intent leases, lock-free IPC

Body
continuous
real-time MCU, safety filters, CBFs

Boundary
continuous
sensors, actuators, kinematics

Physical World
continuous
contact mechanics, friction, mass



Learning Objectives

- Specify operator permissions that bind authorized actions to scope, deadlines, and authentication requirements
- Design authority transfers that preserve command continuity within the machine's physical limits
- Distinguish policy failure, authority transfer, and human recovery in timestamped intervention records
- Evaluate whether operational logs can reconstruct command authority and physical outcomes after an incident

28.1 Shared Authority

A human supervisor is often drawn as one arrow into the plant. At 5 m/s, even a two-second pause before braking consumes ten meters of clearance. An operator request reaches a machine that is already moving under an admitted command; it cannot erase momentum or make an actuator respond instantaneously. The arbiter must decide who may propose the next command, the enforcer must admit or reject it, and the plant must have enough clearance to execute the chosen stop. An abrupt setpoint switch can request a large torque-rate change; whether that causes slip or damage depends on actuator and load dynamics. A delayed switch can exhaust stopping room even when both controllers are individually well behaved.

Prerequisite: The dual-brain proposal-permission boundary establishes the baseline for supervisory preemption in section 2.9.

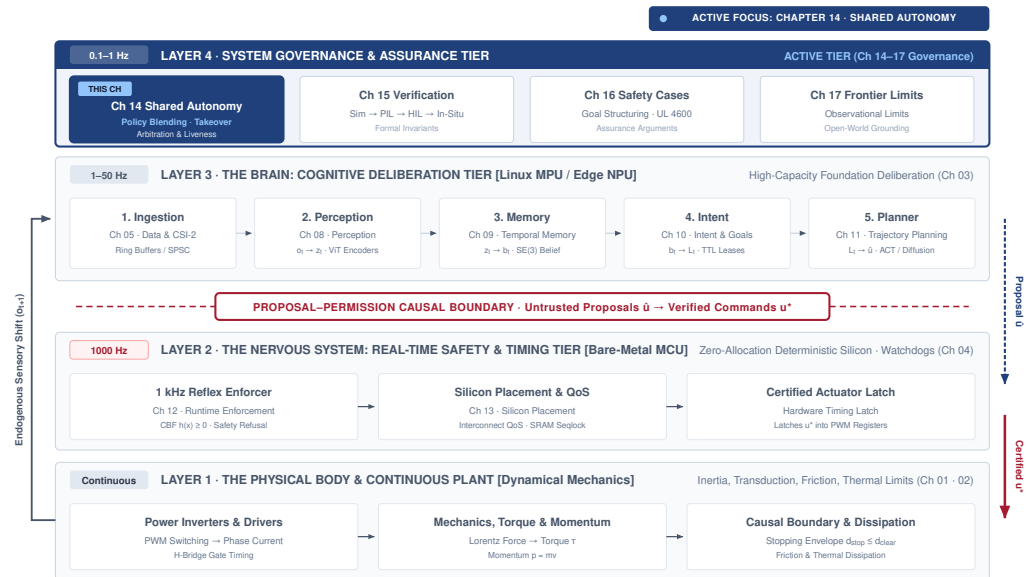


Figure 28.1: Shared autonomy architecture focus: The four-tier physical AI systems architecture highlights the system governance and assurance tier, with shared autonomy as this chapter's active focus. Read the highlighted box as the layer that decides who holds actuator authority at each instant, because concurrent human and machine commands produce split-brain execution and mechanical shock without a deterministic arbiter.

This governance dilemma marks the entry into Part IV of this book. As illustrated in the architecture in figure 28.1, Shared Autonomy resides within Tier 4 (System Governance & Assurance). Operating at supervisory cadences of 0.1 to 1 Hz, governance sits at the apex of the physical AI stack, establishing the operational rules and privilege boundaries under which all lower tiers must execute. Yet supervisory decisions cannot remain abstract software policies; they must govern the continuous physical interaction between human operators and the machine across the entire stack:

- **The Brain (Tier 3):** where high-capacity neural networks, vision-language-action (VLA) foundation models, and deliberative planners generate continuous trajectory proposals at 1 to 50 Hz;¹
- **The Nervous System (Tier 2):** where hard real-time fieldbuses (EtherCAT, CAN-FD), hardware state arbiters, and deterministic safety enforcers execute at 1000 Hz to enforce timing and physical invariants;² and
- **The Body (Tier 1):** where motor inverters, winding thermal dissipation limits (I^2R Joule heating), mechanical gearboxes, and payload inertia dictate the continuous laws of motion.³

Bainbridge's classic *ironies of automation* (Bainbridge 1983) observe that an autonomous system tends to leave human operators responsible for the rarest, most complex edge cases while stripping them of the continuous manual engagement necessary to maintain situational awareness. When an operator intervenes to stabilize an oscillating 500 kg crane payload translating at 1.0 m/s, supervisory authority is effective only if it maps to an executable physical response bounded by actuator torque and thermal derating limits. Taking control of a machine already in motion forces the system to resolve two tightly coupled challenges simultaneously: discrete state arbitration without ambiguity in the software domain, and continuous physical trajectory transition without mechanical shock in the continuous domain. In discrete software, the architecture must prevent split-brain execution where the learned policy and the human operator issue concurrent, conflicting torque setpoints. In the illustrative 1000 Hz controller, the arbiter revokes the policy execution token and latches its last approved reference at commit; the independent enforcer remains the sole actuator writer during the subsequent blend. In continuous dynamics, the nervous system must execute a **bumpless transfer**, blending reference trajectories in $SE(3)$, matching initial velocities, and ramping joint torques within strict jerk bounds and checking measured plant limits during the transition. ISO 10218 and ISO/TS 15066 apply to their specified industrial-robot contexts.

Beyond immediate real-time control, intervention events introduce a treacherous failure mode into the machine learning lifecycle. When an operator intervenes to steer a mobile robot away from a blind drop-off or steady a slipping robotic gripper, the telemetry recorder captures a sudden trajectory discontinuity. If the training data pipeline ingests these rescued episodes as nominal expert demonstrations without recording the exact millisecond of handover, imitation learning policies and offline reinforcement learning algorithms learn to reproduce the exact near-failure states that necessitated the rescue (Ross et al. 2011; Zhao et al. 2023). The policy optimizes toward the boundary where the human intervened because the subsequent manual recovery contains high task rewards. Preventing this policy corruption requires rigorous data hygiene: the logging pipeline must capture immutable, hardware-synchronized timestamps of authority transitions, mark the pre-intervention trajectory as a terminated failure, and isolate recovery maneuvers from nominal behavioral datasets.

The first design decision is operational: which named intervention may reach which physical channel, and how long can the machine wait for it?

28.2 Human Authority Over Actuation

System block diagrams often depict a human supervisor with an arrow pointing toward the plant, treating oversight as an unexamined safety backstop. In the physical world, drawing a person in the loop is not an authority model. Authority cannot attach to a generic human role or abstract supervision block. It must attach to specific named operations with explicit timing budgets, physical degrees of freedom, and deterministic preemption rules. Rather than granting an operator broad administrative authority, the nervous system maintains an explicit **capability matrix**—a strict digital permissions table mapping individual operations (such as an emergency power cutoff, a velocity trim adjustment, a trajectory path veto, or full manual steering) to authenticated roles, reflecting classic taxonomies of human supervisory control⁴ (Sheridan 1992) and multi-level automation (Parasuraman et al. 2000). Each entry in this matrix defines the exact actuator channels the operation governs, the preemption priority it carries over the learned policy, and the maximum propagation latency permitted from physical input to actuator response.

Naming the operations separates **intervention** from **safety enforcement**, two mechanisms that system architectures routinely conflate. Enforcement withholds permission from a candidate action

¹ | **Proposal-Permission Privilege Boundary:** As established in section 6.1, Tier 3 neural planners and vision-language-action policies operate as unverified proposal generators. They emit candidate trajectory setpoints that must never bypass deterministic safety enforcement to command motor registers directly. Direct register dispatch from unverified models risks unconstrained current spikes, motor stator saturation, and uncontrollable plant runaway.

² | **Deterministic Nervous System Enforcement:** As detailed in section 8.1 and section 24.1, Tier 2 executes at a hard real-time cadence of 1000 Hz. It hosts runtime barrier certificates, hardware watchdog interlocks, and fieldbus arbiters capable of atomically revoking model execution tokens within a single 1 ms control cycle. Token revocation blocks further policy proposals at the actuator path; separate state, timing, and drive checks must establish thermal and mechanical protection.

³ | **Actuator Dynamics and Kinodynamic Limits:** As analyzed in section 4.1 and section 22.4, abrupt command changes can demand current or torque transients whose thermal and mechanical effects depend on the drive and load. Kinodynamic feasibility requires trajectory setpoints to observe strict acceleration, torque rate, and jerk limits. Violating validated limits can cause mechanical or electrical damage; a command discontinuity alone does not establish a particular failure.

⁴ | **Supervisory Control Levels:** Sheridan and Verplank (1978) formulated the foundational ten-level taxonomy of supervisory control, demonstrating that human authority degrades into hazardous ambiguity without explicit decision boundaries and strict timing contracts. In standardized autonomous architectures (SAE J3016), supervisory authority is formalized through defined operational design domains and deterministic fallback windows. Failure to enforce these operational boundaries leaves the plant vulnerable to unmonitored trajectory drift during operator latency intervals.

proposed by a learned model, modifying, clamping, or rejecting the setpoint while leaving the machine under autonomous control. Intervention transfers authority away from the model to an external entity, terminating autonomous operation and placing actuation under direct supervisory command. An **enforcement filter** sits in the fast control path to prevent the policy from demanding an over-current maneuver or violating a barrier certificate (Ames et al. 2019) during nominal tracking, respecting the actuator thermal limits established in section 4.1 and the kinodynamic feasibility boundaries from section 22.1. An intervention mechanism strips the policy of its execution token entirely—revoking its right to command the hardware—when an operational boundary is breached or an operator demands control. A system equipped with enforcement but lacking intervention cannot be redirected when human intent changes or external conditions fall outside the policy design envelope. Conversely, a system equipped with intervention but lacking enforcement permits a lagging operator to drive the plant into structural failure during nominal manual flight. A system with one mechanism and not the other is incomplete in a way no amount of the other fixes.

For example, a remote assistant may hold a signed **trajectory veto** capability for an AMR planner channel: role `remote_assistant`, operation `veto_path`, channel `planner_proposal`, priority above the learned planner but below the local enforcer, authenticated session and nonce, and a clearance-derived acknowledgment deadline. The enforcer responds to a valid veto with a checked local hold or stop. A steering-torque packet from that same credential is denied because its capability grants no actuator channel. The permission record binds actor, operation, channel, deadline, authentication, rank, and fallback; a valid signature alone grants no motion authority.

When an operator attempts an intervention, human biology imposes a hard real-time latency budget before any physical corrective action can occur. Human response time is not a single point event, but a cascaded pipeline of physiological and mechanical stages. In one illustrative, additive budget for an operator monitoring a vehicle telemetry stream, the sensory recognition stage requires 150 ms to 300 ms for light to strike the retina, travel through the visual pathway, and register as an optical anomaly. The situational assessment stage, in which the operator recognizes that the learned policy is drifting toward a hazard and selects an appropriate counter-action, demands 500 ms to 1500 ms. Finally, the motor actuation stage, covering the transmission of motor signals down the spinal cord, muscle recruitment in the hand, and the physical displacement of an override switch, consumes 200 ms to 400 ms. Summing these sequential budgets establishes the physiological **takeover reaction floor**. Accounting for network transport and communication handshake latency T_{comm} , total supervisory takeover latency T_{Σ} decomposes into:

$$T_{\Sigma} = T_{\text{sensory}} + T_{\text{cognitive}} + T_{\text{motor}} + T_{\text{comm}} \quad (28.1)$$

During this reaction interval, the machine coasts blind, traversing an unmonitored drift distance $d_{\text{drift}} = v_0 \cdot T_{\Sigma}$ under unchecked policy authority before the electrical intervention signal ever reaches the hardware arbiter. Once the operator engages physical braking, the vehicle requires an additional stopping distance governed by its maximum deceleration a_{max} . Integrating the constant velocity phase during human reaction with the uniform deceleration phase produces the minimum safe intervention distance:⁵

$$d_{\text{stop}} = v_0 \cdot T_{\Sigma} + \frac{v_0^2}{2a_{\text{max}}} \quad (28.2)$$

The total distance for a human operator to detect an obstacle, actuate an override, and bring the plant to rest is dominated by neurological, muscular, and communication delays before the brakes engage. Because the reaction distance scales linearly with velocity while the deceleration distance scales quadratically, human intervention becomes physically incapable of averting close-range collisions as operational speed increases.

To see the direct physical consequences of equation 28.1 and equation 28.2, consider a mobile robot traveling at forward velocity $v_0 = 5.0 \text{ m/s}$ (18.0 km/h) toward an obstacle with an initial clearance $d_{\text{clearance}} = 10.0 \text{ m}$, equipped with chassis braking capable of maximum deceleration $a_{\text{max}} = 2.5 \text{ m/s}^2$. Once mechanical braking engages, the vehicle requires $d_{\text{brake}} = \frac{v_0^2}{2a_{\text{max}}} = \frac{(5.0 \text{ m/s})^2}{2 \times 2.5 \text{ m/s}^2} = 5.00 \text{ m}$ of deceleration runway regardless of intervention mode. However, the total stopping envelope diverges dramatically based on supervisory latency:

1. **Mode 1: Direct Bilateral Teleoperation:** Under continuous haptic teleoperation with an active proprioceptive loop, total closed-loop reaction and bus transit delay is bounded to $T_{\Sigma,1} =$

⁵ | **Minimum Safe Intervention Distance:** Total stopping distance d_{stop} integrates the unmonitored spatial drift during human reaction latency ($v_0 T_{\Sigma}$) with the mechanical braking distance ($v_0^2 / (2a_{\text{max}})$). At an operational speed of 15 m/s, a 2.0 s out-of-the-loop reaction delay consumes 30 m of clearance before friction brakes can even engage. Systems lacking adequate spatial clearance margins inevitably strike obstacles at full operating velocity.

Prerequisite: Actuator torque limits and back-EMF dynamics governing safe handovers originate in section 4.4.

150.0 ms (0.15 s). Free-running reaction drift is $d_{\text{drift},1} = v_0 \cdot T_{\Sigma,1} = (5.0 \text{ m/s})(0.15 \text{ s}) = 0.75 \text{ m}$. Applying equation 28.2 yields a total stopping distance of $d_{\text{stop},1} = 0.75 \text{ m} + 5.00 \text{ m} = 5.75 \text{ m}$, safely halting the machine with a comfortable 4.25 m buffer remaining.

2. **Mode 2: Passive Out-of-the-Loop Supervisory Takeover:** When an operator monitors remote telemetry passively, sensory recognition consumes $T_{\text{sensory}} = 300.0 \text{ ms}$, cognitive situational assessment requires $T_{\text{cognitive}} = 1200.0 \text{ ms}$, motor switch actuation takes $T_{\text{motor}} = 300.0 \text{ ms}$, and network transport and handshake protocol overhead adds $T_{\text{comm}} = 200.0 \text{ ms}$. By equation 28.1, the cumulative reaction latency expands to $T_{\Sigma,2} = 2000.0 \text{ ms} = 2.0 \text{ s}$. During this cognitive re-engagement window, the unmonitored chassis covers $d_{\text{drift},2} = (5.0 \text{ m/s})(2.00 \text{ s}) = 10.00 \text{ m}$. Total stopping distance under equation 28.2 expands to $d_{\text{stop},2} = 10.00 \text{ m} + 5.00 \text{ m} = 15.00 \text{ m}$ —a spatial expansion ratio of $\frac{d_{\text{stop},2}}{d_{\text{stop},1}} = \frac{15.00 \text{ m}}{5.75 \text{ m}} \approx 2.61$ ($2.61\times$), while reaction drift alone expands by $\frac{10.00}{0.75} \approx 13.33\times$.

In Mode 2, the vehicle breaches the 10.0 m clearance boundary by 5.00 m, striking the obstacle at the full initial velocity of $v_{\text{impact}} = 5.0 \text{ m/s}$ because impact occurs before mechanical braking ever begins. For an out-of-the-loop supervisor in Mode 2 to avert collision within that same 10.0 m clearance given the 2.0 s reaction time, solving the quadratic kinematic constraint $d_{\text{clearance}} = v_{\text{max}} T_{\Sigma} + \frac{v_{\text{max}}^2}{2a_{\text{max}}}$ bounds the maximum allowable velocity to:

$$v_{\text{max}} \approx 3.66 \text{ m/s} \quad (13.2 \text{ km/h})$$

This demonstrates that human re-engagement latency establishes a hard physical ceiling on operational velocity that cannot be circumvented by higher-gain actuators or emergency friction pads.



Napkin Math 28.1: Takeover budget and clearance margin

Concept: Calculating the total stopping distance during a human intervention requires adding up every source of delay before the machine comes to a complete halt across four distinct phases:

1. **Reaction Drift:** The distance the machine travels while the human perceives the obstacle and physically reacts ($v_0 \cdot T_{\text{reaction}}$).
2. **Handshake Drift:** The distance covered while the machine's internal software verifies the human's command ($v_0 \cdot T_{\text{handshake}}$).
3. **Actuator Blending:** The distance traveled while the machine executes the C^2 bumpless authority transfer ramp to safely smooth the transition from autonomous to manual control (section 22.1).
4. **Physical Braking:** The final distance required for the brakes to decelerate the mass to rest ($v_0^2/(2a_{\text{max}})$).

Worked Comparison: At nominal speed $v_0 = 5 \text{ m/s}$ under maximum braking $a_{\text{max}} = 2.5 \text{ m/s}^2$ ($d_{\text{brake}} = 5 \text{ m}$):

- **Direct Bilateral Teleoperation** ($T_{\Sigma} = 150 \text{ ms}$):

$$d_{\text{stop,haptic}} = (5 \text{ m/s}) \times (150 \text{ ms}) + \frac{(5 \text{ m/s})^2}{2 \times (2.5 \text{ m/s}^2)} = 0.75 \text{ m} + 5 \text{ m} = 5.75 \text{ m}$$

- **Out-of-the-Loop Takeover** ($T_{\Sigma} = 2000 \text{ ms}$):

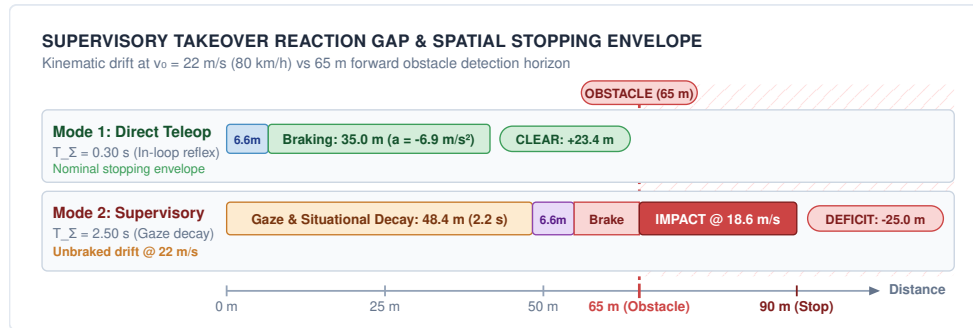
$$d_{\text{stop,ool}} = (5 \text{ m/s}) \times (2000 \text{ ms}) + \frac{(5 \text{ m/s})^2}{2 \times (2.5 \text{ m/s}^2)} = 10 \text{ m} + 5 \text{ m} = 15 \text{ m}$$

Systems Insight: Passive monitoring inflates reaction delay from 150 ms to 2000 ms, expanding total stopping distance by 3× (from 5.75 m to 15 m). When adding these up for an autonomous vehicle on a highway, the combined delays often consume nearly the entire safety margin. Even

a fraction of a second drop in human attention can turn a safe stop into a collision. For the rigorous kinematic derivation of these stopping distances, see Appendix C.



Checkpoint 28.1: Takeover reaction floor and clearance expansion



Consider an embodied mobile robot operating near physical obstacles with a human supervisor:

- ❑ **Reaction floor decomposition:** Can you explain why total supervisory takeover latency T_Σ cannot be treated as a single uniform software timer, breaking it down into physiological sensory recognition, cognitive situational assessment, motor actuation, and digital transport stages?
- ❑ **Linear drift versus quadratic braking scaling:** Can you describe how the linear velocity scaling of reaction drift ($v_0 \cdot T_\Sigma$) interacts with the quadratic scaling of mechanical braking ($\frac{v_0^2}{2a_{\max}}$) as operating speed increases, and explain why out-of-the-loop human takeover becomes physically incapable of averting collisions at highway speeds regardless of brake clamping force?
- ❑ **Velocity ceilings enforced by supervisory latency:** Can you explain why human re-engagement latency ($T_\Sigma \geq 2.0$ s) enforces a hard mathematical ceiling on allowable autonomous operating velocity within bounded sensing horizons?



Systems Perspective 28.1: The spatial takeover budget law

Handover is an allocation of physical distance, not digital clock cycles. If the spatial travel during human reaction time $v \cdot T_{\text{reaction}}$ plus the kinematic braking distance $\frac{v^2}{2a_{\max}}$ exceeds the sensing clearance margin, no software handshake or arbitration algorithm can avert collision. The handover deadline T_{timeout} must be dynamically bounded by physical stopping distance, not fixed in configuration files. Figure 28.2 sweeps that calculation across the operating envelope, and the three bands make the failure mode legible: the two latency terms scale linearly with velocity while braking scales with its square, so a handover margin that is comfortable in an urban corridor disappears at highway speed.

The same 5 m/s vehicle and validated 2.5 m/s² braking floor need 5.0 m after brake onset. Table 28.1 uses **selected illustrative delays**, not measured modality guarantees. At the upper end of a 3.7 s out-of-loop delay, drift is 18.5 m and total stopping distance is 23.5 m; the 15.0 m worked case uses 2.0 s. Delayed haptic feedback can destabilize a particular human-machine loop, so passivity and stability must be checked for its controller and transport.

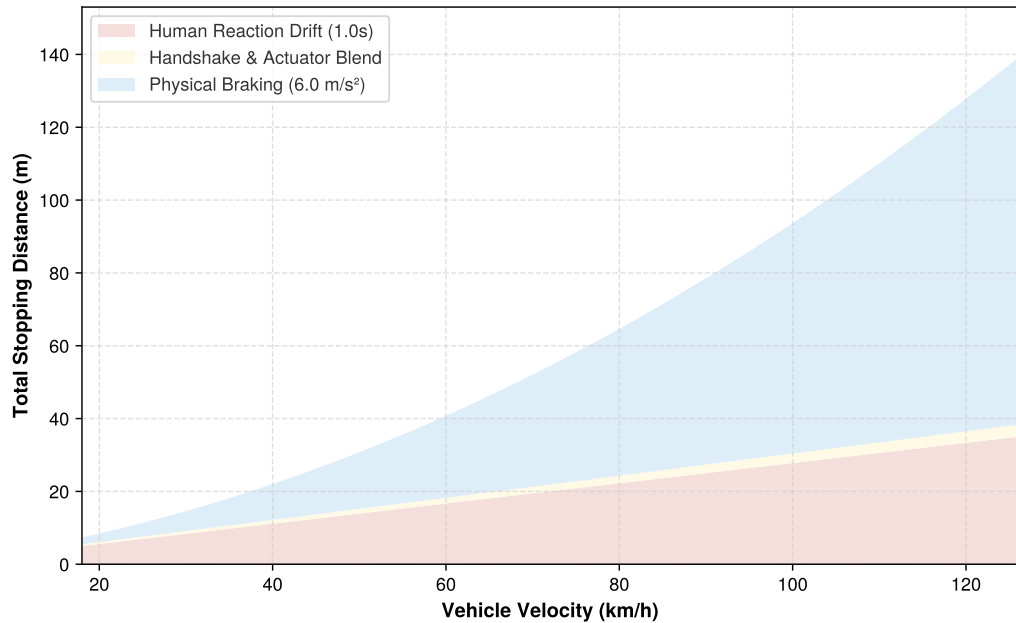


Figure 28.2: Spatial takeover budget: Constructed stopping distance against vehicle velocity, decomposed into human reaction drift, protocol handshake and actuator blend, and physical braking. The reaction and handshake terms grow linearly with speed while braking grows quadratically under the stated constant-deceleration model.

Table 28.1: Illustrative takeover delay and stopping distance: Each distance uses the delay in its own row, $v = 5 \text{ m/s}$, and an assumed validated braking floor of 2.5 m/s^2 after the stated delay. Actual reaction, transport, scanner/drive, and brake-response distributions must be measured for the machine; a stop function is selected by the hazard assessment.

Intervention mode	Selected total delay	Drift at 5 m/s	Total stop with 5 m brake	Local protection
Direct bilateral teleoperation	150 ms	0.75 m	5.75 m	Qualified passivity/rate checks
Continuous velocity guidance	450 ms	2.25 m	7.25 m	Onboard velocity/clearance gate
Supervisory waypoint dispatch	1,150 ms	5.75 m	10.75 m	Local obstacle veto and fallback
Out-of-loop takeover	2,000 ms	10.0 m	15.0 m	Independent stop while waiting
Physical protective input	250 ms human-plus-device response	1.25 m	6.25 m	Risk-assessed protective stop



Figure 28.3: Industrial teach pendant: Front view of an ABB pendant showing the display, buttons, emergency-stop actuator, and cable. Its rear enabling grip, internal channel wiring, response time, and stop behavior are not visible here. The ABB manual documents a three-position grip and a 250 mm/s manual-mode limit for its specified configuration. Photograph by Auledas, CC BY-SA 4.0, via Wikimedia Commons; local copy is re-encoded.

Human response is slow and prone to interruption; thus, manual authority cannot be an open-ended state.⁶ If an operator assumes control and subsequently experiences wireless packet loss, physical incapacitation, or cognitive distraction, the machine cannot remain adrift under stale manual setpoints. The architecture protects against operator dropouts by bounding manual authority with configured and validated temporal leases—countdown timers that expire without active, periodic renewal—and hardware **enabling devices**, which require continuous physical engagement to maintain actuation channels (figure 28.3). In a lease-based communication model, a particular vehicle may use a 50–100 ms delegation window if its communication and stopping budgets support it. The operator console must continuously stream authenticated keep-alive packets to renew the lease before the deadline expires. If a single lease window lapses without renewal, or if an operator releases a spring-loaded physical deadman switch on the controller, manual proposals are revoked at the qualified detection deadline, and the safety controller commands the plant-specific validated stop or hold. Drive inhibit, friction braking, and holding brakes are distinct functions.⁷ Authority is not a persistent condition. It is a perishable resource that requires active, periodic renewal to survive.

Even while an operator holds an active lease, human authority remains subordinate to the kinematic and dynamic limits enforced at the hardware boundary. A common engineering mistake assumes that an emergency manual override must bypass all downstream safety filters to guarantee direct, unmediated actuator control. Bypassing safety enforcement at the motor level creates severe mechanical failure modes. If an operator panics and commands a steering rate that induces vehicle rollover, or requests a joint torque that would physically shear the gear teeth of a robotic actuator, the low-level safety enforcer sitting at the hardware floor rejects the command. The enforcer evaluates human setpoints against the same control barrier certificates, current limits, and kinematic floor boundaries applied to the learned model. Human intervention transfers operational discretion away from the learned policy, but it cannot suspend the laws of physics or dismantle the hardware-enforced envelopes that preserve the physical integrity of the machine.

⁶ | **NTSB Autonomous Vehicle Investigation HAR-19/03:** The NTSB describes operator takeover and ADS disengagement procedures in the 2018 Tempe test, but found inadequate oversight of operator attentiveness. The development ADS suppressed its own emergency braking for one second after emergency recognition without alerting the operator to the hazard; the factory Volvo forward-collision warning and AEB were disabled in test mode. The report does not establish a diagnostic-bus rationale for the suppression. The four-phase handshake in this chapter is a proposed design response to the reliance on operator availability, not a procedure attributed to the NTSB investigation.

7 | Three-Position Enabling Devices: The cited ABB operating manual documents a three-position enabling grip and a 250 mm/s manual-mode limit for that robot. The required performance level and stop behavior depend on the machine risk assessment; ISO 13849-1 does not assign PL e to every teach operation. Grip release or over-squeeze is a request into the configured safety circuit, whose actual stop time and load behavior must be measured.

8 | Inter-Core Authority Multiplexing: Authority arbitration requires hardware-enforced exclusivity across distributed fieldbuses. Dual-writing to CAN-FD or EtherCAT segments from autonomous and manual controllers induces non-deterministic actuator chatter and inverter current oscillations. A qualified arbiter and bus permissions must prevent two software sources from writing one actuator channel; the physical consequence of a command conflict depends on the plant and drive.

Downstream: Deterministic takeover state transitions furnish the core behavioral models tested in section 30.1.

28.3 Authority Transitions

Control ownership must be a discrete state, never an inference from a blended command. When an engineering team attempts to soften transitions by continuously averaging the control vectors of an autonomous policy and a human operator, the resulting command may represent neither intent. If a human steers left to avoid a ditch while a neural network policy steers right to follow a lane marker, a linear interpolation between the two commands directs the steering angle straight forward, driving the vehicle into the obstacle at full speed. The original Boeing 737 MAX MCAS case is a separate warning about repeated automated trim after pilot interruption, not an example of averaging two live commands (Joint Authorities Technical Review 2019).⁸ To prevent this ambiguity, control authority at every discrete time tick t must resolve to a single, mutually exclusive state $S_t \in \{\text{Autonomous, Handover-Pending, Blend, Human-Manual, Degraded-Fallback, Emergency-Stop}\}$. The nervous system evaluates arbitration logic, determines the active state, and commits that identity to the bus before dispatching setpoints to low-level motor controllers. At no point do two masters write the actuator. During BLEND, the enforcer alone owns the actuator channel and interpolates between a latched last-approved policy reference and a checked human target.

The complete state transition logic, guard preconditions, execution time bounds (T_{trans}), and bumpless actuator blending profiles governing this arbiter are formalized in table 28.2. Every edge in the state machine defines an unambiguous trigger condition, mutual exclusion enforcement across actuator channels, and a deterministic fallback path should an active communication lease lapse.

Table 28.2: Authority state machine transition matrix: Each row names the single actuator owner and the state-dependent fallback. The example uses a 1 ms arbiter tick; switch and stop times need measured platform bounds. The BLEND state interpolates only latched, checked endpoints, under enforcer ownership.

From → to	Guard and deadline	Actuator owner / command	Failure response
Autonomous → Pending	Policy requests supervision; deadline computed from clearance	Enforcer admits policy proposals while feasible	MCU fallback if deadline reached
Autonomous or Pending → Blend	Authenticated operator ACK, fresh state, feasible target and stopping reserve; commit on a control tick	At commit, revoke policy token, latch last approved command and checked manual target; enforcer alone writes actuator	Deny invalid target; use resident fallback
Blend → Human-Manual	Profile completes and manual lease remains valid	Enforcer admits live human proposals within physical limits	MCU fallback on lease or enabling-device loss
Pending, Blend, or Manual → Degraded-Fallback	Timeout, stale lease, or invalid state	MCU executes separately validated stop/hold	Re-arm only after validated recovery procedure
Any state → Emergency response	Independent protective input or qualified critical fault	Safety circuit/controller selects plant-specific protective stop	STO alone may coast; brake/hold where validated

Shifting authority between these discrete states requires a deterministic **four-phase handshake protocol** (Request → Acknowledge → Commit → Confirm). Executed by the **real-time safety enforcer**, this four-stage transaction is designed to enforce mutual exclusion, validates cryptographic freshness to ensure commands are not stale network echoes, and atomically shifts actuation authority between discrete holders across a synchronous tick boundary. When implemented and tested, this sequencing prevents split-brain command contention, where two masters attempt to drive the hardware in conflicting directions during the same compute cycle. In the first phase, designated as the *Request* phase, one entity signals an intent to transfer control. This occurs either when an operator asserts an override through a physical input like a button press or steering torque exceeding a calibrated threshold, or when the autonomous policy detects that sensor confidence has degraded and requests human assumption of control. In the second phase, the *Acknowledge* phase, the safety enforcer inspects the target state prerequisites, verifying that sensor data is valid, communications are active, and the recipient is physically capable of receiving command authority. In the third phase, the *Commit* phase, the enforcer executes the state transition at a synchronous control tick boundary, revoking the policy token, latching its last admitted endpoint and the checked human

target, and assigning the enforcer sole actuator ownership throughout BLEND. In the final phase, the *Confirm* phase, the nervous system broadcasts a state confirmation packet across the network and triggers physical feedback, such as haptic vibration⁹ or optical indicators, to tell the operator whether blending or manual control is active; lost confirmation does not change the committed owner.

The protocol primitives, timing deadlines, cryptographic verification artifacts, and fallback actions for each stage of this four-phase sequence are specified in table 28.3. This structured transaction rejects spoofed override injections and routes interrupted handovers to the separately validated fallback path.

Table 28.3: Constructed four-phase handover timing: $10 + 5 + 1 + 15 = 31$ ms in the stated configuration. These are assumed design budgets, not portable CAN, cryptographic, or HMI specifications. Confirmation and physical blending are distinct from the atomic ownership commit.

Phase	Bound in constructed example	Guard and owner	Failure handling
1. Request	10 ms	Authenticated role and fresh, clock-converted nonce; no owner change	Reject invalid packet; keep current qualified mode
2. Acknowledge	5 ms	Check state, manual target, and full stop reserve	Reject request; invoke fallback if current mode cannot continue
3. Commit	1 ms tick	Revoke policy token, latch both checked endpoints, assign enforcer sole actuator write path	No partial commit; fallback on failed guard
4. Confirm	15 ms	Announce BLEND, then MANUAL only after profile and lease check	Lost notice alarms operator but does not undo commit

Because human attention is volatile, a handover request initiated by the autonomous policy cannot remain open indefinitely. If a machine encounters an operational design domain boundary at speed and requests intervention, an operator who is distracted, asleep, or incapacitated will fail to complete the handshake. The state machine enforces the earlier of a configured protocol ceiling and a clearance-derived latest fallback time. For forward speed $v > 0$, verified available deceleration $a_b > 0$, clearance d_c , uncertainty reserve e , and dispatch/brake-onset delay δ , the latest time after the request at which fallback may be dispatched is

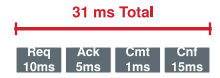
$$T_{\text{latest}} = \frac{d_c - e - v^2 / (2a_b)}{v} - \delta, \quad T_{\text{timeout}} = \max(0, \min(T_{\text{config}}, T_{\text{latest}})).$$

The MCU initiates fallback when elapsed time is **at least** this deadline; a nonpositive T_{latest} calls for immediate fallback and may mean stopping is already infeasible. In a constructed 100 Hz example at 15 m/s, with 20 m clearance, 6.87 m/s^2 validated braking, 0.30 m uncertainty reserve, and 10 ms dispatch delay, $T_{\text{latest}} = 211.6$ ms. Round down to the 10 ms tick: fallback begins by 210 ms, before the 500 ms configuration ceiling. The former is a state-dependent deadline; the latter alone is not a deployable safety rule.

Once authority resides in Human-Manual, the machine must verify the engagement signal required by its delegated-control design. A steering interface might use grip, gaze, or torque sensing; each needs measured coverage and false-loss behavior. In this illustrative design, a 100 ms loss-of-engagement lease transfers authority to a validated, state-matched fallback. The state machine then blocks a return to Autonomous or Human-Manual until its specified reset conditions are met. Those conditions may require a full stop, fault review, and a new handshake; the physical outcome still depends on available braking and clearance.

When multiple control sources propose commands, the nervous system resolves source selection through a fixed arbitration hierarchy (Brooks 1986a) evaluated each cycle, extending the proposal-permission architecture from section 6.1. The independent enforcer admits, modifies, or rejects the selected candidate under its configured physical limits. In Human-Manual with a live lease, the human proposal takes precedence over a learned proposal, but remains subject to that check. In Autonomous, the learned policy may propose actions for admission. Fixed-state arbitration takes bounded work per tick in this design; it does not require live negotiation between the two proposers.

⁹ | **Assistive Haptic Guidance vs. Authoritative Override:** Shared control architectures must strictly distinguish between assistive haptic cues (which apply informational torque within human muscular impedance limits) and authoritative mechanical overrides (which lock or redirect the column). Blurring this boundary causes operators to perceive advisory compliance forces as an autonomous takeover, inducing panic counter-steering that excites column resonance.



This constructed four-phase budget totals 31 ms; the enforcer owns the blend after commit.

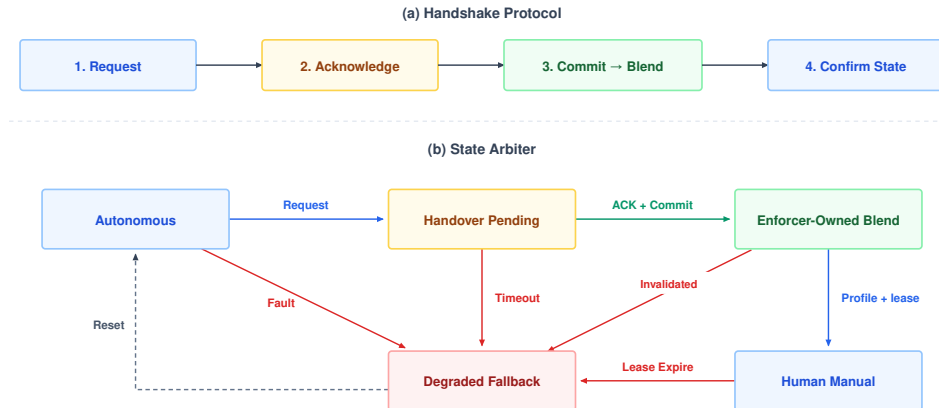
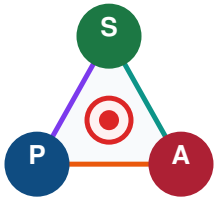


Figure 28.4: Authority handshake and owner states: Request and acknowledgment precede atomic commit. At commit the policy token is revoked and the enforcer owns the blend between checked, latched endpoints; manual proposals begin only after profile completion with a live lease. Timeout invokes a separately validated fallback.

As synthesized in figure 28.4, the runtime intervention interface joins the four-phase handshake to an arbiter with bounded timeouts and explicit fallback transitions.

One writer makes authority unambiguous at each control tick, but an unblended switch may still request an abrupt command change. If the policy's last admitted drive torque is $15.0 \text{ N} \cdot \text{m}$ and the operator target is zero, directly swapping those setpoints requests a $15.0 \text{ N} \cdot \text{m}$ step. The resulting torque, slip, and structural response depend on the drive and load. The committed enforcer instead uses the latched endpoints to generate a checked transition profile with a reserved stopping margin (section 22.1).



Supervisory arbitration: dynamic authority transitions across the physical proposal-permission boundary.

Checkpoint 28.2: Authority transition protocols and timeout bounding

Before designing handshake protocols for human-robot authority handoffs, verify your understanding of transition state machines and timeout dynamics:

- ☐ **Finite-state transition semantics:** Can you trace the deterministic progression between Autonomous, Request, Blending, and Manual states, and explain why transition aborts must yield to a verified safe-stop envelope rather than reverting to a degraded policy?
- ☐ **Kinematically bounded timeouts:** Can you calculate why the timeout window T_{timeout} for an unacknowledged takeover request must be dynamically bounded by the remaining physical clearance distance rather than fixed as an arbitrary operating-system timer?
- ☐ **Asymmetric preemption hierarchy:** Can you explain why the qualified local protection path must meet its measured deadline, while restoring autonomous authority requires a fresh admission handshake?
- ☐ **Fallback degradation trajectories:** Can you distinguish between active dynamic deceleration to a controlled stop and passive actuator de-energization under different mechanical impedance configurations?

If any transition paths permit unhandled timeouts or undefined intermediate states, the authority protocol is vulnerable to mode confusion and runaway actuation. Review this section before proceeding to smooth command blending.

28.4 Transfer Without Discontinuity

At commit time t_0 , the enforcer latches the last admitted policy command $\mathbf{u}_0$ and a checked human target $\mathbf{u}_1$. A direct switch would request $\Delta \mathbf{u} = \mathbf{u}_1 - \mathbf{u}_0$ in one tick. Its ideal command derivative is discontinuous, but the actual plant filters the command through finite drive, brake, and mechanical

dynamics. A large change may excite a gearbox, exceed a tire's available traction, or create hydraulic pressure shock in a valve system; none follows without the relevant plant model and load. The transfer therefore checks both rate and full stopping clearance rather than assuming a software tick maps instantaneously into physical force (section 2.2).

Eliminating this step discontinuity requires smoothly mixing the old and new commands across a finite time window τ_{blend} , a technique known as **bumpless transfer**. A linear blend has endpoint slope changes; whether those matter depends on the commanded variable and actuator response. For this fixed-endpoint acceleration-command example, a C^2 **bumpless transfer ramp** eases in and out, removing endpoint jumps in the command and its first two derivatives. A standard curve used for this is the quintic spline (see Appendix B for the mathematical derivation).

 Definition 28.1: The C^2 bumpless authority transfer ramp

C^2 **bumpless authority transfer ramp** is an enforcer-owned command profile between two checked, latched references $\mathbf{u}_0$ and $\mathbf{u}_1$, with a monotonic weight $\alpha(t) \in [0, 1]$ over a finite window τ_{blend} . Authority already transferred atomically to the enforcer at commit.

1. **Significance:** A setpoint step has an unbounded ideal derivative. A C^2 command profile removes that discontinuity; measured actuator tracking and plant limits still decide physical feasibility.
2. **Distinction:** Unlike a linear interpolation ramp that exhibits derivative discontinuities at its endpoints, a C^2 quintic spline enforces vanishing first and second derivatives ($\dot{\alpha} = \ddot{\alpha} = 0$) at both entry and exit, making the commanded acceleration continuous with zero endpoint jerk for fixed endpoints.
3. **Common pitfall:** Using $\tau_{\text{blend}} < 1.875|\Delta\mathbf{u}|/j_{\text{allowable}}$ for a fixed acceleration-command change breaches the stated command-jerk limit. Resonance, slip, and stopping distance require plant tests.

Actuator authority is blended using a normalized C^2 quintic smoothstep weighting profile $\alpha(\hat{t}) = 6\hat{t}^5 - 15\hat{t}^4 + 10\hat{t}^3$ (where $\hat{t} = \frac{t-t_0}{\tau_{\text{blend}}}$). For a fixed change in *commanded acceleration* and a chosen command-jerk limit $j_{\text{allowable}}$, the profile requires:¹⁰

$$\tau_{\text{blend},\min} = \frac{1.875|\Delta\mathbf{u}|}{j_{\text{allowable}}} \quad (28.3)$$

This command-rate bound holds when $\mathbf{u}$ denotes acceleration, the endpoints remain fixed, and $j_{\text{allowable}}$ is a validated command-jerk limit. It does not by itself prove physical traction or stopping safety.

This latency-jerk trade-off applies to the stated acceleration-command profile. Increasing τ_{blend} reduces the profile's peak commanded jerk, but delays the full human target and can lengthen the stopping trajectory. Consider an autonomous vehicle traveling at $v_0 = 10.0$ m/s experiencing an emergency operator takeover. The human asserts full emergency braking ($a_{\text{tgt}} = -4.0$ m/s²) while the autonomous policy was commanding steady cruising ($a_{\text{policy}} = 0.0$ m/s²), yielding an acceleration step discrepancy $|\Delta a| = 4.0$ m/s². If chassis dynamics and tire traction constrain maximum allowable jerk to $j_{\text{allowable}} = 10.0$ m/s³, blending actuator authority via the C^2 quintic polynomial enforces a minimum blending window per equation 28.3 of:

$$\tau_{\text{blend},\min} = \frac{1.875 \times 4.0 \text{ m/s}^2}{10.0 \text{ m/s}^3} = 0.750 \text{ s} \quad (750 \text{ ms})$$

Under mean blending deceleration $\bar{a}_{\text{blend}} = \frac{1}{2}a_{\text{tgt}} = -2.0$ m/s², vehicle velocity at the conclusion of the blend phase drops to $v_1 = v_0 + \bar{a}_{\text{blend}}\tau_{\text{blend}} = 10.0 - 1.50 = 8.50$ m/s, having traversed $d_{\text{blend}} = v_0T + a_{\text{tgt}}T^2 \int_0^1 (1-s)\alpha(s) ds = 10(0.75) - 4(0.75)^2/7 = 7.178571$ m, because the smoothstep integral is $1/7$. Full deceleration from $v_1 = 8.50$ m/s to rest subsequently requires $d_{\text{brake}} = \frac{v_1^2}{2|a_{\text{tgt}}|} =$

10 | **Smoothstep Jerk Bound:** For normalized time $s = (t-t_0)/T$, the maximum $|d\alpha/ds| = 1.875$ occurs at $s = 0.5$. Since $ds/dt = 1/T$, a fixed acceleration-command difference has peak commanded jerk $1.875|\Delta\mathbf{u}|/T$. Time-varying endpoints add their own derivative terms. For a latched acceleration target, this bounds the *commanded* jerk; plant jerk, torque response, traction, and resonances require separate validation (derived in section B.3).

Prerequisite: Acceleration-bounded bumpless transfer profiles build upon the physical stopping constraints enforced in section 24.5.

Bumpless Transfer Braking Penalty: This 3.709821 m difference belongs to the specified 10 m/s, -4 m/s^2 , 0.75 s acceleration profile. It is not a universal cubic jerk formula. A real vehicle also needs measured tire, brake, and actuator response to compare this idealized blend against a fast brake request.

$\frac{72.25}{8.0} = 9.03 \text{ m}$, yielding a total blended stopping distance of $d_{\text{stop,blended}} = 7.178571 \text{ m} + 9.03125 \text{ m} = 16.209821 \text{ m}$.

The executed example yields 7.178571 m during the blend and 16.209821 m to rest.

Compared to a theoretical instantaneous step takeover ($d_{\text{stop,step}} = \frac{v_{\text{tgt}}^2}{2|a_{\text{tgt}}|} = \frac{100.0}{8.0} = 12.50 \text{ m}$), bumpless blending imposes a spatial penalty of 3.709821 m ($16.209821 - 12.50 = 3.709821 \text{ m}$).¹¹ However, attempting to bypass this blend window by applying a step change in acceleration over a single 1.0 ms control cycle would request a command jerk of about 4000 m/s^3 over that tick. Whether the actuator can follow it, traction is lost, or stopping distance increases depends on the measured brake and tire dynamics; the idealized step is only a comparison baseline.



Systems Perspective 28.2: The smoothstep jerk-latency invariant

For the specified fixed acceleration endpoints, $\tau_{\text{blend}} \geq 1.875|\Delta \mathbf{u}|/j_{\text{allowable}}$ enforces the chosen command-jerk ceiling. Clearance must include this delay. Faster transfer is possible only with a different validated profile, actuator response, or limit; slip and resonance cannot be inferred from the inequality alone.



Checkpoint 28.3: Bumpless blending and transition stopping

Consider an embodied vehicle or robotic manipulator transferring authority under emergency conditions:

- ☐ **$\mathcal{C}^2$ continuity versus linear interpolation:** Can you explain why linear command interpolation is insufficient for mechanical actuators, and describe how enforcing vanishing first and second derivatives ($\dot{\alpha} = \ddot{\alpha} = 0$) at trajectory endpoints removes the ideal endpoint command-rate discontinuity under fixed endpoints?
- ☐ **The jerk-latency trade-off:** Can you describe why lengthening the blending duration τ_{blend} reduces commanded jerk while possibly increasing physical stopping distance?
- ☐ **Physical failure of step overrides:** Can you distinguish between theoretical step-response stopping calculations and the measurements needed to decide whether a faster brake request causes slip or changes total stopping distance?

Command blending removes the chosen output discontinuity, but a standby human feedback controller can still accumulate integral error while disconnected from the plant. **Initial condition matching** pre-biases that controller against the measured state before commit, avoiding an idle-integrator transient. It does not force the deliberate human target to equal the last policy command. The enforcer latches both endpoints, sizes the blend for their actual difference, and checks the resulting command and stopping reserve throughout transfer.¹²

28.4.1 Admittance versus impedance control under neuromuscular resonance

Executing physical interventions requires deciding whether the machine responds to human motion or human force. In physical AI and haptic teleoperation (Hogan 1985), this distinction separates **impedance control** (responding to motion by rendering a force) from **admittance control** (responding to force by yielding in motion):

1. **Impedance Control (motion $\rightarrow$ force):** The controller measures kinematic displacement or velocity ($\mathbf{x}, \dot{\mathbf{x}}$) and commands an interaction force or joint torque (τ):

$$\tau = \mathbf{K}(\mathbf{x}_{\text{des}} - \mathbf{x}) + \mathbf{D}(\dot{\mathbf{x}}_{\text{des}} - \dot{\mathbf{x}})$$

Impedance control behaves like a virtual spring-damper. It is naturally suited for direct-drive or highly backdrivable mechanisms where the mechanical linkage can be moved by hand with minimal unmodeled friction.

¹² | **Initial condition matching:** A standby controller can track the current measured plant state and bound its integral term. The admitted human target may still differ; the residual command difference is handled by the validated blend and permission checks.

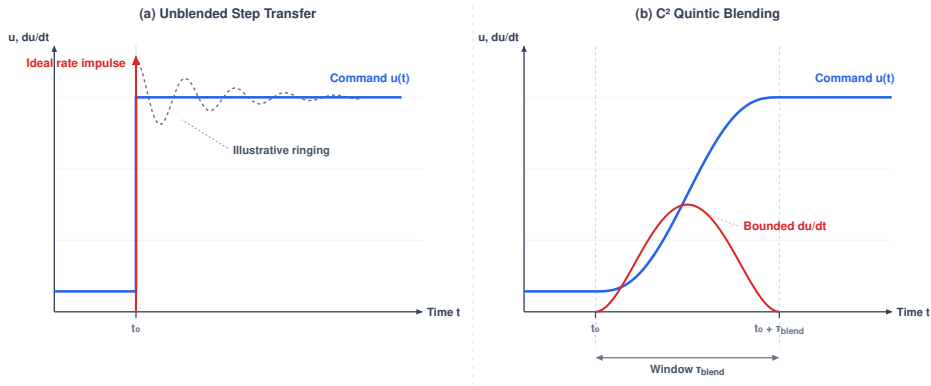


Figure 28.5: Illustrative command transfer: A step and fixed-endpoint quintic blend are compared. Blue is the command; red is its ideal derivative, and gray ringing is schematic. The profile bounds the commanded rate for the assumed endpoints; plant response needs measurement.

2. **Admittance Control (force $\rightarrow$ motion):** The controller measures external interaction forces ($\mathbf{F}_{\text{ext}}$) exerted by the human (via a six-axis wrist force-torque sensor or steering column strain gauges) and calculates a compliant kinematic trajectory that the actuator tracks under stiff position control:

$$\mathbf{M}_d \ddot{\mathbf{x}} + \mathbf{D}_d \dot{\mathbf{x}} + \mathbf{K}_d (\mathbf{x} - \mathbf{x}_0) = \mathbf{F}_{\text{ext}}$$

Admittance control is essential for non-backdrivable, high-gear-ratio industrial robots (such as harmonic or cycloidal drives with large reflected rotor inertia $N^2 J$, as established in section 4.4).

The human arm, actuator, and communication channel form a coupled feedback loop. Their resonance and phase margin depend on the operator, grip, mechanism, controller gains, and transport delay; a universal 4–8 Hz human resonance or automatic limit cycle cannot be assumed. If measured manual torque and drive current oppose one another during a handover, the engineer can reduce stiffness, add validated damping, or change the control mode while retaining clearance and torque limits. Impedance and admittance designs must be tested for passivity or stability over their specified operating range.

To verify a transfer, record the command profile, motor current, encoder motion, and chassis inertial response on a shared timebase with bounded alignment error. A step command has a derivative discontinuity in the ideal model; ringing, high jerk, and slip are possible *observations*, not inevitable waveforms. A fixed-endpoint quintic bounds the commanded rate. The measured plant response establishes whether the handover met its physical limits.

The physical waveforms comparing an unblended step handover against $\mathcal{C}^2$ quintic bumpless blending are illustrated in figure 28.5.

The complete algorithmic governor executing discrete four-phase state handshakes, integrator pre-biasing, and continuous $\mathcal{C}^2$ quintic spline blending on bare-metal microcontroller silicon is formalized below:

Cadence & invariants:

- *Execution Cadence:* Hard real-time periodic timer interrupt ($f = 1000$ Hz, period $T_{\text{loop}} = 1.0$ ms, hardware timing jitter $< 1.0 \mu\text{s}$).
- *Memory Invariant:* Zero dynamic heap allocation (`malloc/free` strictly forbidden; all state variables, mailboxes, and circular log buffers statically pre-allocated in DTCM SRAM).
- *Hardware Authority:* Exclusive direct register write lock to motor inverter PWM compare registers (TIMx_CCR / ePWM CMPA), hydraulic proportional valve DACs, and Safe Torque Off (STO) hardware cutoffs.

Execution protocol (illustrative 1 kHz enforcer):

1. Sample and validate plant state, available stopping reserve, independent protective inputs, and the current authority lease. An independent protective input selects its validated plant-specific stop path; STO by itself can allow coasting.
2. In AUTO, a manual request or policy handover request enters PENDING; the policy may propose only while its existing permission remains valid. Set T_{timeout} to the smaller of the protocol ceiling and clearance-derived deadline. At elapsed time $\geq T_{\text{timeout}}$, transfer to the MCU-owned fallback. An invalid network packet is rejected and logged, not interpreted as a stop request.
3. In PENDING, verify the operator credential, nonce, enabling input, target freshness, actuator limits, and full transfer-plus-stop clearance. On ACK, at the next qualified **Commit** tick, revoke the policy execution token, latch $\mathbf{u}_0$ as its last *admitted* command, latch a checked human target $\mathbf{u}_1$, and set BLEND. The enforcer remains the only actuator writer. If admission fails, retain or enter the independently validated fallback.
4. In BLEND, evaluate $s = \min(1, (t - t_0)/T)$ and $\alpha(s) = 6s^5 - 15s^4 + 10s^3$. The enforcer proposes $\mathbf{u}_{\text{blend}} = (1 - \alpha)\mathbf{u}_0 + \alpha\mathbf{u}_1$, then checks that candidate against current state, torque/rate, and stopping limits before dispatch. These are **latched endpoints**, not simultaneous live policy and human actuator streams. If the lease, clearance, or tracking assumption fails, use a state-matched MCU fallback. At $s = 1$ with a live manual lease, set MANUAL and confirm that state to the operator.
5. In MANUAL, the enforcer checks each live human proposal. Lease expiry or enabling-device loss revokes manual proposals and transfers to the validated fallback. Log request, ACK, commit, candidate, admitted command, measured response, and clock uncertainty separately; the bounded high-rate record must not block the control tick.

Calculating blending filters and pre-biasing controller states assumes that the incoming signal originates from a verified, legitimate operator entitled to assume physical control. If the nervous system accepts a transfer request without validating identity, an unauthenticated command injection or a malfunctioning subsystem can seize authority and override the machine while it is in motion. The mechanisms that make an intervention smooth must therefore be paired with mechanisms that establish who is allowed to intervene.

28.5 Authenticating an Intervention

Authenticating an intervention requires preventing an unauthorized entity from seizing physical authority while ensuring that an authorized operator can halt the machine before collision. These requirements conflict, necessitating classification of intervention operations by the mechanical work they induce. Removing power or asserting a mechanical brake is physically distinct from injecting active motion commands, and the two actions do not warrant identical verification mechanisms. A protective stop may need controlled braking and load support before torque inhibition; removing motor torque alone can leave a moving plant coasting or a suspended load falling. In contrast, injecting active steering angles, wheel torques, or joint velocity setpoints introduces fresh mechanical power into the system. An unauthorized or corrupted motion command can accelerate a moving mass into an obstacle, whereas a trusted out-of-band protective input requests the plant-specific minimum-risk stop.

Motion commands introduce active kinetic hazards; thus, the verification mechanism must meet both latency and freshness bounds from the kinematics of the maneuver. If an obstacle is detected at clearance $d_{\text{clearance}}$ while the platform travels at velocity v_0 , and mechanical brakes apply deceleration a_{brake} after mechanical buildup lag t_{mech} and bus serialization delay t_{bus} , the maximum permissible latency for cryptographic signature verification and replay checking is bounded by:

$$T_{\text{auth,max}} = \frac{d_{\text{clearance}} - \frac{v_0^2}{2a_{\text{brake}}}}{v_0} - t_{\text{mech}} - t_{\text{bus}} \quad (28.4)$$

To see the physical consequences of equation 28.4, consider an autonomous transport robot ($m = 250$ kg) traveling at forward velocity $v_0 = 2.0$ m/s toward an obstruction detected at initial clearance $d_{\text{clearance}} = 1.20$ m. When an emergency override command is broadcast, mechanical brakes apply constant deceleration $a_{\text{brake}} = 4.0$ m/s² once actuated, requiring mechanical braking distance

$d_{\text{brake}} = \frac{v_0^2}{2a_{\text{brake}}} = \frac{(2.0 \text{ m/s})^2}{2 \times 4.0 \text{ m/s}^2} = 0.50 \text{ m}$. The displacement available before braking must commence is $d_{\text{margin}} = d_{\text{clearance}} - d_{\text{brake}} = 1.20 \text{ m} - 0.50 \text{ m} = 0.70 \text{ m}$. At $v_0 = 2.0 \text{ m/s}$, total elapsed time before braking must engage is $T_{\text{max}} = \frac{0.70 \text{ m}}{2.0 \text{ m/s}} = 350.0 \text{ ms}$. If mechanical actuator lag and hydraulic buildup consume $t_{\text{mech}} = 160.0 \text{ ms}$, and CAN-FD bus serialization consumes $t_{\text{bus}} = 40.0 \text{ ms}$, equation 28.4 yields a cryptographic verification deadline of:

$$T_{\text{auth,max}} = 350.0 \text{ ms} - 160.0 \text{ ms} - 40.0 \text{ ms} = 150.0 \text{ ms}$$

For a **constructed** Cortex-M4 timing comparison, assume the following verification times; they are not measured or portable benchmarks:

- **Algorithm A (Asymmetric RSA-2048):** Assumed verification takes $t_{\text{auth,A}} = 180.0 \text{ ms}$, exceeding the 150.0 ms budget. Total delay before braking commences expands to $T_{\text{total,A}} = 180.0 + 160.0 + 40.0 = 380.0 \text{ ms}$ (0.380 s). The robot traverses $d_{\text{travel}} = (2.0 \text{ m/s})(0.380 \text{ s}) = 0.76 \text{ m}$ before brakes actuate. The remaining runway to the obstacle is only $d_{\text{rem}} = 1.20 \text{ m} - 0.76 \text{ m} = 0.44 \text{ m} < 0.50 \text{ m}$. The machine collides with the obstacle at impact velocity:

$$v_{\text{impact}} = \sqrt{v_0^2 - 2a_{\text{brake}}d_{\text{rem}}} = \sqrt{(2.0)^2 - 2(4.0)(0.44)} = \sqrt{4.0 - 3.52} = \sqrt{0.48} \approx 0.693 \text{ m/s} \quad (2.5 \text{ km/h})$$

- **Algorithm B (Elliptic-Curve Ed25519):** Assumed verification takes $t_{\text{auth,B}} = 15.0 \text{ ms}$, well within the 150.0 ms budget. Total delay before braking is $T_{\text{total,B}} = 15.0 + 160.0 + 40.0 = 215.0 \text{ ms}$ (0.215 s). Free-running displacement before braking is $d_{\text{travel}} = (2.0 \text{ m/s})(0.215 \text{ s}) = 0.43 \text{ m}$, yielding a total stopping distance of $d_{\text{stop}} = 0.43 \text{ m} + 0.50 \text{ m} = 0.93 \text{ m} < 1.20 \text{ m}$. The robot halts safely, preserving a 27 cm clearance buffer.

Freshness requirements arise from the same kinematics. If an authentication window permits a timestamp drift or message age threshold of $\tau_{\text{fresh}} = 100 \text{ ms}$, an authenticated packet replayed from the boundary of that window commands an action based on a chassis position displaced by $\Delta x = v \cdot \tau_{\text{fresh}} = (2.0 \text{ m/s})(0.10 \text{ s}) = 0.20 \text{ m}$. In a precision navigation envelope, that spatial displacement turns a safe clearance margin into a physical collision.

A trusted, independently wired protective input is evaluated by its safety circuit without waiting for a network signature. Its response is the stop or hold selected and validated for the plant; drive torque inhibition alone does not necessarily brake a moving vehicle or support a biped. Remote motion or takeover packets follow the capability and authentication checks. A corrupted checksum, stale nonce, or failed signature on that network is rejected and logged; malformed traffic does not acquire the authority to command a brake. If valid command renewal is lost, the separately monitored lease expires and the safety controller invokes its validated fallback. This separation preserves an available local protective path while preventing arbitrary network noise from forcing an unsafe stop.

CAN bus priority: A transmitting CAN frame is non-preemptive. A higher-priority identifier wins only at the next arbitration opportunity; controller buffer policy and worst-case current-frame serialization must be included in the measured delivery bound. An independent protective input avoids this shared queue where the safety case requires it.



Checkpoint 28.4: Cryptographic verification and spatial margin

Consider an embedded real-time controller authenticating emergency intervention commands over a shared fieldbus:

- ☐ **Physical bounds on cryptographic deadlines:** Can you explain how kinematic clearance margins and mechanical actuator lag dictate hard real-time execution ceilings for cryptographic signature verification on moving platforms?
- ☐ **Fail-safe versus fail-closed under kinetic momentum:** Can you contrast standard cybersecurity fail-closed design (dropping unverified packets) with physical AI fail-safe requirements, explaining why a trusted out-of-band protective input is distinct from an invalid network packet, and how lease expiry selects the validated fallback?
- ☐ **Signature strength versus execution latency:** Can you describe the architectural trade-off between cryptographic verification latency (e.g., asymmetric RSA versus elliptic-curve Ed25519 or symmetric GMAC) and physical stopping margins on embedded compute nodes?

A training simulator that always supplies immediate human rescue can underprice risky states. If deployed intervention instead incurs authentication, transport, and actuation delay, a policy trained on that mismatch may visit states from which rescue is no longer feasible. The remedy is to model the qualified latency and fallback path in training and hardware-in-the-loop evaluation, retain failed learner-visited states, and test the actual clearance distribution. Neither dataset aggregation nor a particular signature algorithm by itself establishes this failure mode.

The authority inventory must include diagnostic, debug, bootloader, and wiring paths that can affect actuation. A production design may disable debug access, authenticate maintenance commands, protect boot updates, and enforce bus permissions according to its threat and fault model. The verification task is to show that each reachable actuator-write path is either gated by the permission authority or independently protected; a universal fuse, enclosure, or coprocessor prescription does not follow.

Prerequisite: Microsecond override masking registers on real-time fieldbuses are implemented in section 8.5.

🔍 Systems Perspective 28.3: Cryptographic primitives and physics

Cryptographic primitives: The mathematical formulation of cryptographic primitives, including public key signatures such as Ed25519, symmetric message authentication codes such as AES-GMAC, and key distribution hierarchies, is treated comprehensively in foundational security literature. Systems engineering for physical AI is responsible for mapping those primitives onto physical deadlines, managing the trade between latency and authority, and identifying the residual risks that cryptographic verification cannot address. The architecture presented here defends against remote packet injection over wireless links, replay attacks on internal buses, and unauthorized commands originating from compromised high-level application processors. Physical tampering defenses depend on the threat model, inspection access, and platform packaging; they are separate from the remote-packet argument here.

When an intervention occurs, whether triggered by a legitimate operator, an unauthenticated command injection, or electrical noise on a safety line, the nervous system must record the transition alongside the physical state of the plant. Diagnosing why an intervention occurred, establishing whether an emergency stop was caused by a genuine boundary violation or an invalid cryptographic token, and proving that the machine behaved correctly under handover deadlines requires an immutable record preserved across physical power cycles.

28.6 Trustworthy Operational Logs

A record supporting an argument months later has requirements that a debug log lacks. A debug log may be buffered in volatile user space and dropped under backpressure. If an asynchronous disk write drops three lines of trace data during an operating system stall, an engineer loses minor diagnostic context while the application continues running. Missing records weaken reconstruction across the causal boundary established in section 2.2. When a moving manipulator strikes an assembly jig, establishing whether the learned policy commanded an illegal torque, whether the safety enforcer failed to clamp it, or whether an operator intervened too late requires a bounded, integrity-checked record plus independent plant observations.¹³ A **forensic operational log** therefore does not exist to assist a developer with line-by-line code tracing. Its purpose is to preserve what was sampled, proposed, admitted, and dispatched. Motor current, encoder motion, and contact-force sensors provide separate physical observations; commands alone do not prove delivered force or collision cause.

The chosen 1 kHz control design records a structured authority and action tuple each tick, with declared loss handling. The minimal **evidence tuple** takes the form $\langle t_{\text{PTP}}, S_{\text{auth}}, \mathbf{x}_{\text{state}}, \mathbf{u}_{\text{prop}}, \mathbf{u}_{\text{enf}}, \mathbf{u}_{\text{act}}, \text{ID}_{\text{operator}} \rangle$, extending the proposal-permission architecture from section 6.1. In this structure, t_{PTP} provides a hardware-synchronized timestamp, S_{auth} records the active authority state indicating whether autonomous policy, manual override, or fault mitigation holds control, and $\text{ID}_{\text{operator}}$ identifies the authenticated source of any active intervention. The kinematic state and action vectors $(\mathbf{x}_{\text{state}}, \mathbf{u}_{\text{prop}}, \mathbf{u}_{\text{enf}}, \mathbf{u}_{\text{act}})$ retain their established roles from the safety enforcer. Omitting one element can limit a particular causal question, so the record declares what each field actually measures. If the log records $\mathbf{u}_{\text{act}}$ without $\mathbf{u}_{\text{prop}}$, investigators lose the policy-command comparison, though driver corruption still

¹³ | **Precision Time Protocol Logging:** Unlike legacy event data recorders logging at coarse 10 Hz intervals, physical AI incident reconstruction demands sub-millisecond precision. A qualified PTP installation can align the named device clocks with a measured conversion-error bound. Event ordering requires the intervals around timestamps, sensor exposure and sampling, and actuator response not to overlap; sub-microsecond timestamp precision alone does not timestamp physical contact.

needs independent drive evidence. If $\mathbf{u}_{\text{enf}}$ is omitted, the log loses the proposed-versus-admitted comparison; actual deadline compliance needs timestamped execution evidence.

A **forensic authority record** is a sampled ledger entry with protected integrity and clock-conversion metadata that logs the discrete authority state S_{auth} , raw proposed action $\mathbf{u}_{\text{prop}}$, enforcer-modified action $\mathbf{u}_{\text{enf}}$, delivered actuator command $\mathbf{u}_{\text{act}}$, and operator identity across every control cycle to support incident reconstruction and separate policy, blend, and reviewed human-recovery data.

Ordering these tuples requires physical timestamp integrity rather than software clock queries. Software timestamps derived from operating system calls inherit scheduling jitter, context-switch delays, and variable interrupt servicing latencies that can exceed 5 ms on loaded application processors. Over a 5 ms interval, a mobile base traveling at 2.0 m/s covers 10 mm, and an internal control loop running at 1000 Hz executes five complete control cycles. If an operating system logs the arrival of a manual intervention command with a 5 ms delay, the record places the operator's command after the actuator response, inverting physical cause and effect. A hardware timestamp records a defined electrical event, such as frame ingress or a protective-input edge, after clock conversion with bounded error. Separate drive-current, encoder, and contact measurements are needed to infer physical response. Investigators can order command and detected contact only when their timestamp uncertainty, sampling windows, and sensor delays do not overlap; a command log cannot prove the exact instant of impact.

Persisting full sensor streams and high-rate telemetry continuously to non-volatile storage exceeds both the bandwidth and the endurance limits of embedded hardware. Consider an analytical mobile platform equipped with four depth sensors generating 1280×720 16-bit depth frames at 30 Hz. Each camera produces approximately 55.3 MB/s, totaling 221 MB/s across four 30 Hz streams. Adding the control telemetry of the 128-byte evidence tuple recorded at 1000 Hz contributes another 128 kB/s. At that rate, ten seconds of raw pre-trigger depth alone occupy about 2.21 GB and a separate ten-second post-trigger segment another 2.21 GB, or 4.42 GB before indexing and other streams. This is not an MCU SRAM ring. The 128-byte, 1 kHz control tuple needs only 128 kB/s (2.56 MB over twenty seconds) and can use a bounded small-memory ring. If full raw depth is required, provision a separate DRAM/storage tier of at least the measured stream budget, or retain selected/compressed frames with a declared loss of detail. At a trigger, seal the pre-trigger segment and continue writing post-trigger samples into a different allocated segment; freezing the sole ring would lose the promised post-event evidence. Frame records retain their native 30 Hz capture times, while control tuples retain their 1 kHz issue and sample times. Background persistence must not block the enforcer.

The pre-trigger window must capture the moment of policy divergence rather than merely the moment of physical takeover. When a learned policy commands an erroneous trajectory, human operators and external monitoring processes exhibit reaction latencies ranging from hundreds of milliseconds to multiple seconds before recognizing the deviation and asserting authority. If the ring buffer only recorded data from the instant an operator pulled an override switch, the telemetry would capture the rescue while omitting the anomalous commands that made the rescue necessary. Capturing a declared pre-trigger interval lets investigators inspect policy proposals and states before intervention; whether a proposal was unsafe needs the contemporaneous model, state estimate, and physical evidence. As established in chapter 10, without the complete pre-divergence state history, evaluation pipelines cannot separate a necessary rescue from an unnecessary or premature human interruption. Losing that divergence window corrupts the statistical metrics used to validate policy safety cases and distorts the imitation data collected for offline model retraining.

Integrity and retention can use a protected hash chain and independently tested power-loss storage. Within the nervous system, logging routines batch evidence tuples into fixed blocks corresponding to 100 ms of execution. Each block is hashed using SHA-256, and its hash is concatenated with the hash of the preceding block to form an append-only cryptographic chain. With protected key and anchor custody, changing a committed block is detectable against the retained chain; loss before commit and compromised keys remain separate failure modes. One implementation periodically anchors root hashes in protected nonvolatile storage; the safety case must test key custody, power-loss retention, and missing intervals. Power-loss testing must establish which committed blocks survive and how interrupted writes are marked; a hash chain alone supplies neither persistence nor complete event capture.

A retained, integrity-checked log can establish recorded authority and command values for its sampled intervals. Delivered force and contact require physical sensor evidence and uncertainty analysis. Yet possessing a mathematically sound record of authority does not prevent physical instability during the transition itself. When authority transfers while a mechanical body is carrying momentum and actuators are delivering active torque, the machine faces the immediate physical problem of coordinating two independent control sources. The central danger of an intervention is not that the log will fail to record the event, but that the handover itself will leave the machine in a state where neither the policy nor the operator has coherent control.

28.7 How Handover Goes Wrong

The most common handover failure is uncertainty over control. When an autonomous policy and a human operator share authority, system failures rarely arise from mechanical failures or software exceptions. They stem from mode confusion and authority ambiguity, where the machine assumes the human is managing an operational axis while the human assumes the machine retains jurisdiction. In multi-axis physical systems, authority is frequently decomposed across degrees of freedom, separating lateral steering from longitudinal throttle and braking. If a learned policy encounters an unmodeled surface reflection and quietly yields lateral tracking without disengaging the longitudinal speed regulator, an operator who notices the lateral drift may grasp the wheel to steer while assuming the automated system will continue to brake for downstream traffic. Conversely, if the operator taps the brake pedal to initiate longitudinal deceleration, firmware that disconnects only the longitudinal controller leaves the lateral steering policy actively hunting for lane boundaries against the driver's intended exit trajectory. In telemetry logs, this failure presents as disjointed control flags where authority bits toggle asynchronously across execution threads. At the human-machine interface, however, this state is undetected until kinematic error exceeds the physical clearance margin, because neither claimant receives an immediate, unambiguous signal of the other's partial withdrawal.

Even with defined authority transfer protocols, the human nervous system cannot match the response deadlines of moving mass. An active human operator in direct physical control reacts to sudden tactile or visual hazards with a motor reflex latency of roughly 0.5 s. When the same operator is placed in a passive supervisory role while a neural policy guides the platform, situational awareness decays. In an analytical scenario of an autonomous vehicle traveling at 30 m/s (108 km/h), nominal cruising for several minutes causes operator visual gaze to fixate away from the forward collision envelope, while cognitive readiness drops. When the onboard system encounters an unresolvable perceptual distribution shift and issues an emergency handover request, human reaction time inflates from 0.5 s to between 3.0 s and 5.0 s. During this transition gap, a platform moving at 30 m/s covers 90 m to 150 m without coherent guidance. By the time the operator perceives the obstacle, re-establishes spatial orientation, and applies physical input, the available stopping distance has been entirely consumed. Cabin-facing infrared cameras can track pupil dilation and gaze direction at 60 Hz, detecting eye closure or head rotation. They cannot measure cognitive comprehension of dynamic scene geometry, meaning this latency expansion is detected only after the response deadline has elapsed and impact occurs.

When an intervention occurs while an actuator applies force, manual input and the enforcer can oppose each other. For example, a human may request 15 N · m of steering torque while a local rate or rollover limit admits less. The measured column torque and motor current can then oppose each other. If the human-device loop has insufficient phase margin, repeated corrections may oscillate; the frequency, saturation, and physical result depend on gains, delay, grip, and steering mechanics. The enforcer must retain the physical limit while the interface makes the opposing force and active mode legible to the operator. A torque-sign comparison is evidence of opposition, not proof that an inverter will trip or a column will lock.

Uncoordinated handovers damage both physical interactions and learned models from fleet logs. When an intervention occurs, the telemetry stream contains several seconds of anomalous states where the automated policy drifted, followed by high-amplitude, high-jerk recovery actions executed by the human operator. If an automated data ingestion pipeline incorporates these raw telemetry logs into offline behavioral cloning or diffusion training datasets without explicit intervention masking, the training distribution is corrupted. The learned policy optimizes its weights against a

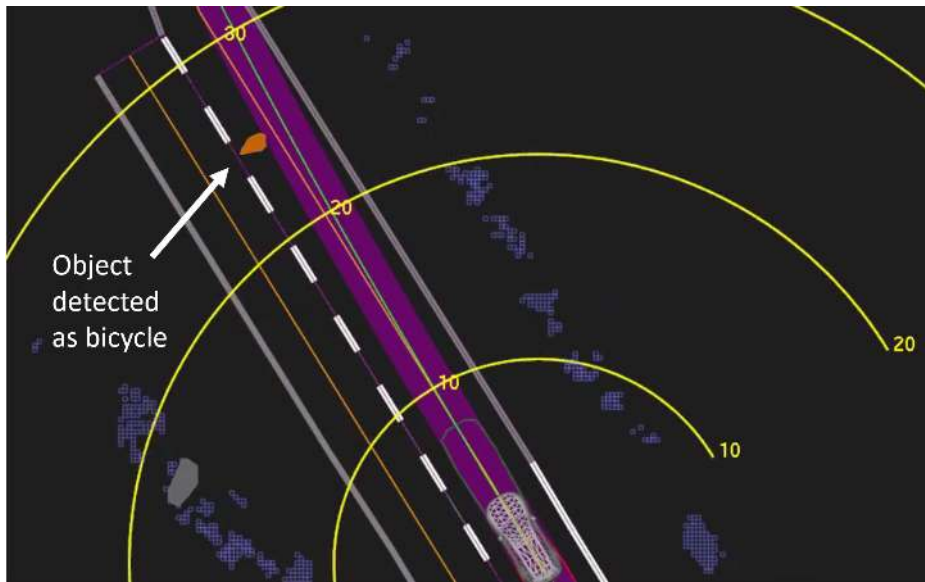


Figure 28.6: Tempe preliminary playback still: One overhead NTSB playback frame about 1.3 seconds before impact, not a classification or braking telemetry trace. The NTSB final report separately records first detection at 5.6 seconds, emergency recognition at 1.2 seconds, and a one-second ADS emergency-brake suppression through about 0.2 seconds before impact. Image: NTSB preliminary Figure 2, via Wikimedia Commons, U.S. public domain.

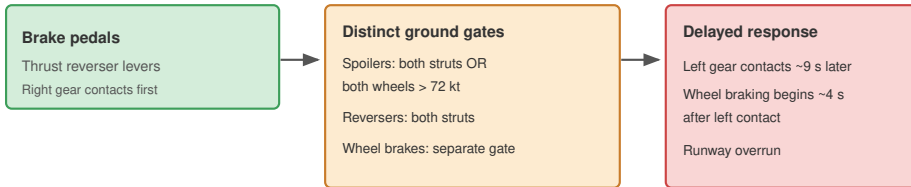
dataset where nominal lane features are paired with aggressive, non-equilibrium steering inputs. In subsequent training epochs, the model associates ordinary highway geometry with emergency recovery dynamics, causing the next-generation policy to hallucinate abrupt steering steps in nominal operating conditions. This failure is undetected during training, as standard validation losses (such as mean squared error on trajectory chunks) show nominal numerical convergence. The dataset corruption is discovered only during deployment, when production machines execute sudden, uncommanded swerves on open paths.

The Tempe still in figure 28.6 illustrates the scene; the final NTSB report supplies the timeline. The developmental ADS detected the pedestrian 5.6 seconds before impact, recognized the need for emergency braking 1.2 seconds before impact, and suppressed emergency braking for one second. The factory Volvo forward-collision warning and automatic emergency braking were disabled during testing. These are distinct paths, and the report does not document a Kalman-filter reset as the cause. The safety operator did not intervene in time. The failure asks whether an independent stop path and operator-monitoring contract would have been effective under the measured remaining clearance (National Transportation Safety Board 2019).

A separate October 2023 Cruise incident shows why post-contact authority matters. The vehicle initially braked and stopped after a pedestrian was thrown into its path. It then misclassified the collision as a side impact and began a pull-over, dragging the pedestrian about 20 feet (6.1 m); a later wheel-speed mismatch initiated a degraded stop (Quinn Emanuel Urquhart and Sullivan, LLP 2024). Remote assistants received video, so lack of floorpan visibility should not be asserted as the cause. The architectural question is whether contact uncertainty should have revoked pull-over authority until an independently validated hold or controlled stop was established.

War Story 28.1: Automation lockout on flight 2904

Crew requests deceleration



A crew input and each actuator effect require separate state and timing evidence.

Context: Two distinct aviation control interactions show why a crew command cannot be treated as proof of its physical effect: original Boeing 737 MAX MCAS trim logic (Joint Authorities Technical Review 2019) and Lufthansa Flight 2904 landing-ground interlocks (Main Commission Aircraft Accident Investigation Warsaw and German Federal Bureau of Aircraft Accident Investigation 1994).

Mechanism: Original MCAS could command repeated nose-down trim about five seconds after pilot electric trim was released. Pilot electric trim interrupted an active MCAS command, and cutout switches were available; high manual trim-wheel forces depended on mistrim and airspeed, not a mechanically locked wheel. At Warsaw, the right gear contacted the wet runway about nine seconds before the left. Spoiler deployment required both struts compressed **or** both wheel speeds above 72 knots; thrust reversers required both struts. Wheel brakes had a separate wheel-speed gate and alternate mode. Those distinct conditions delayed the requested deceleration; the aircraft was not simply classified airborne by one register.

Outcome and lesson: Wheel braking began roughly four seconds after the left-gear contact, about thirteen seconds after the first right-gear contact. The aircraft overran the runway; the inquiry reports 72 knots at the runway end, not embankment impact speed. These cases motivate explicit annunciation, independent sensor checks, and tested crew-command paths for each actuator function. The applicable response still depends on the plant and available stopping distance.

Systems Perspective 28.4: The invariant priority invariant

Manual takeover grants authority to select among safe physical trajectories, never authority to command the plant outside its physical survival envelope. Human inputs remain unprivileged proposals subject to deterministic barrier gating and kinematic floor limits.

Preventing these concurrent failures requires an explicit, unambiguous mechanism that adjudicates command authority before physical signals reach the actuator stage. The control architecture cannot rely on implicit handoffs, human attentiveness, or ad hoc torque overrides. It requires an immutable authority record that specifies exactly who holds permission over each physical axis, under what operational state conditions that permission transfers, and what strict temporal deadlines govern the release of mechanical control.

28.8 The Authority Log

The engineering notebook established in chapter 8 defines the base schema, hardware provenance records, cryptographic hashing, and monotonic versioning for all machine configurations. This chapter adds a single table to this manifest, the **forensic authority record**. The static manifest is loaded and verified before actuator drivers energize. The per-tick forensic event record in table 28.4

is a separate stream documenting decisions and measured response; neither substitutes for the other. It binds every physical actuator channel to an explicit matrix of authorized actors, operational preconditions, preemption priorities, and transition deadlines.

The static authority manifest specifies every permitted interaction as a hard contract between the physical platform and its potential operators. Each entry identifies a distinct physical authority domain through a hardware actuator mask, an authorized entity class (such as a local physical operator, a remote teleoperation console, an autonomous trajectory policy, or an emergency stop loop), and an integer **preemption tier**. An entity presenting a command vector must prove its authority through cryptographic role credentials verified by the nervous system before its input can enter the actuation pipeline. Alongside the credential, the entry specifies the mutual exclusion policy for the channel, the maximum allowed command gradient (how rapidly requested torque can change to prevent physical jerking), the active blending duration τ_{blend} enforcing a $\mathcal{C}^2$ bumpless authority transfer ramp to maintain the kinodynamic feasibility bounds established in section 22.4, and the terminal fallback state that the hardware enters if an active session is severed.

To formalize this verification and forensic contract across the hardware boundary, table 28.4 defines the abstract data schema for the forensic authority and takeover record. The compact table counts bytes and states what the sampled record establishes; the versioned static manifest supplies vector units, field order, timebase, credential rules, and hardware access controls.

Table 28.4: Forensic authority event record (ForensicAuthorityRecord_t): The listed fields total 114 bytes packed; a 128-byte slot leaves 14 bytes of padding in this illustrative layout. Field units, vector order, clock conversion, signal provenance, credential rules, and stopping limits belong to the separately versioned static manifest. The record is a sampled decision trace, not proof of delivered force or contact.

Per-tick field	Packed bytes	Recorded evidence and limit
timestamp_ptp_ns	8	Converted event time; store measured clock-error bound separately.
actuator_channel_mask	2	Channels to which this decision applies; bus/DMA permissions enforce write scope.
authority_state	1	AUTO, PEND, BLEND, MAN, FALLBACK, or ESTOP.
operator_role_id	1	Authenticated source role, interpreted against the static capability manifest.
preempt_trigger_code	2	Request, timeout, enabling-device loss, or protective input cause.
blend_factor_alpha	4	Unitless $[0, 1]$ progress of the enforcer-owned command profile.
policy_proposed_cmd	16	Four example command components before permission; not a motor write.
enforcer_clamped_cmd	16	Four components admitted after the current control constraint check.
actuator_delivered_cmd	16	Four dispatched setpoint components; drive delivery needs separate evidence.
measured_plant_state	16	Four sampled state components, with sensor timing and units in the manifest.
prev_block_hmac_chain	32	Protected-key digest link for tamper evidence, subject to key/anchor custody.

Mapping these entries to the physical dynamics of the machine requires bounding four timing parameters against platform limits across the three archetypes: the **operator reaction budget** T_{react} , the **handshake protocol latency** $T_{\text{handshake}}$, the **timeout threshold** T_{timeout} , and the minimum **blending window** τ_{blend} . The formal kinematic and timing specifications for Mobile Systems, Manipulators, and Process/Energy Systems are detailed in table 28.5.

Table 28.5: Illustrative intervention timing across three plants: The rate ceiling r bounds the derivative of each *command*, not physical chassis jerk or valve pressure without tracking evidence. Human and protocol times are scenario assumptions. Every lease and pending deadline must fit the state-dependent stopping or holding reserve.

Constructed plant	Command change and chosen rate ceiling	Smooth-step window $T \geq 1.875 \Delta u /r$	Assumed human / protocol time	Fallback contract
Ground vehicle steering	25 N · m; torque rate $r = 500 \text{ N} \cdot \text{m/s}$	94 ms	1.0 s / 31 ms	100 ms manual lease; 500 ms pending ceiling shortened by clearance
Industrial arm joint	15 N · m; torque rate $r = 750 \text{ N} \cdot \text{m/s}$	38 ms	0.25 s / 2 ms	Enabling-device loss invokes validated stop; 50 ms is an example lease
Process valve	2.5 MPa; pressure-command rate $r = 15 \text{ MPa/s}$	313 ms	2.5 s / 10 ms	Remote-link loss invokes plant-specific controlled response; 1 s is an example lease

Consider an analytical ground vehicle operating at 22 m/s. The operator reaction budget T_{react} sets the maximum duration the nervous system permits between issuing a takeover demand and detecting manual steering torque, budgeted at $T_{\text{react}} = 1.0 \text{ s}$ for an attentive seated driver (a separate illustrative attentive-driver assumption). The handshake latency $T_{\text{handshake}}$ defines the round-trip deadline for mutual authentication and state acknowledgment over the bus, budgeted at $T_{\text{handshake}} = 31 \text{ ms}$ per the four-phase protocol in table 28.3. The timeout threshold T_{timeout} dictates the maximum communication silence tolerated from an overriding entity before the nervous system declares a link failure and reverts to the fallback trajectory (enforcing the watchdog lease and heartbeat patterns established in section 8.3). In this scenario, it is set to $T_{\text{timeout}} = 100 \text{ ms}$ per the deadman lease in table 28.5. Finally, the blending window τ_{blend} governs the rate of authority transfer to prevent mechanical shock. For an electric power steering column with an assumed commanded torque-rate ceiling of $500 \text{ N} \cdot \text{m/s}$, transitioning an automated $25 \text{ N} \cdot \text{m}$ torque bias to zero requires an analytical blending duration of $\tau_{\text{blend}} \geq \frac{1.875 \times 25 \text{ N} \cdot \text{m}}{500 \text{ N} \cdot \text{m/s}} = 93.75 \text{ ms} \approx 94 \text{ ms}$.

These four timing parameters determine how much of the assumed clearance remains. At 22 m/s, the constructed transition interval $T_{\text{react}} + T_{\text{handshake}} + \tau_{\text{blend}} = 1000 \text{ ms} + 31 \text{ ms} + 94 \text{ ms} = 1125 \text{ ms}$ consumes 24.75 m of travel before the human target reaches full command weight. If an obstacle is detected 65 m ahead and validated braking under these conditions needs 35 m, the total 59.75 m budget leaves 5.25 m. Additional delay uses that margin; a delay exceeding about 239 ms at this speed would exhaust it under the same constant-speed and braking assumptions.

This static manifest and event schema form test inputs and evidence for verification. Chapter 30 uses the authority record to construct hardware-in-the-loop fault-injection cases for specified transition failures. The suite checks four design rules: one enforcer path writes each actuator channel; a lower-tier proposal cannot bypass an active higher-tier source and the independent permission check; an unacknowledged handshake invokes the validated fallback at the earlier configured or clearance-derived deadline; and invalid authentication locks out the affected network channel. Dropped CAN frames, a frozen operator input, and replayed override packets test the response to those injected conditions. Broader physical protection still requires measured actuator, timing, and plant evidence under the stated operating envelope.

When assembling the safety case for operational certification, chapter 32 evaluates the authority record as primary evidence for release. A safety assessment examines the stated authority constraints, implementation evidence, fault coverage, and tested operating envelope. The certification package pairs the static authority ledger with empirical execution traces gathered from the fault-injection

campaign, showing measured preemption latency and any observed gaps in the audit trail. A P99.99 statistic is evidence for a declared distribution, not a worst-case guarantee.

A verified authority record documents the intended command-contention rule; implementation tests must establish that the arbiter follows it. But verifying that the nervous system switches authority according to specification does not guarantee that the resulting physical vehicle dynamics remain stable under real-world tarmac friction, worn actuators, and split-second driver reflexes. A handover that executes flawlessly at the bus protocol level can still induce dynamic rollover or road departure if the physical machine and the human operator are pushed beyond their combined operational limits.

Downstream: Forensic intervention records provide evidentiary artifacts for the deployment safety cases in section 32.3.

Systems Perspective 28.5: Active compliance and fall arrest limits

Intervening on a failing humanoid robot reveals why industrial emergency stop paradigms can be physically destructive on an underactuated biped:

- **The Catastrophic Cost of “Coast Mode”:** In some industrial workcells a Category 0 response uses STO to inhibit motor-generated torque; moving loads may coast, and STO does not itself remove all electrical power. If an engineer hits an STO button on an upright 70 kg humanoid, motor-generated torque is inhibited after the device-specific response time; a moving or unsupported body may coast or fall. Without support or a separately powered stabilizing response, gravity can make the body fall. A fall may damage the robot or endanger nearby people; impact speed and injury depend on posture, height, contact geometry, and any mechanical support. For an upright bipedal machine, unpowered coasting is an active hazard rather than a safe state.
- **Active Compliant Fall Arrest:** Where the platform has sufficient power, state estimation, and actuator authority, a validated active response may reduce fall consequences. When a human supervisor presses an emergency pause or an autonomy watchdog trips, one possible response uses impedance damping:

$$\tau_{\text{safe}} = -\mathbf{D}_{\text{virt}}\dot{\mathbf{q}} + \tau_{\text{gravity}}(\mathbf{q})$$

If dynamic balance is already lost, the nervous system may use a validated **fall-arrest trajectory**—retracting arms to protect delicate wrist load cells, rotating the torso to take the impact across compliant rear backpack dampers, and actively modulating knee and hip damping to absorb the impact energy over tens of milliseconds rather than a microsecond collision impulse.

- **Bilateral Tele-Intervention and Force Reflection:** A remote humanoid intervention may benefit from bilateral haptic feedback, but the safe mode depends on the platform and measured delay. If the operator feels only visual feedback over a 100 ms video stream, the operator may command corrective angles that exceed the robot’s real-time Zero Moment Point (ZMP) stability envelope. Smooth authority handover requires rate-limited blending where the robot’s autonomous low-level reflex controller retains continuous authority to reject any manual operator command that violates foot friction cones.

Checkpoint 28.5: Forensic authority logging and hardware isolation

Before relying on operational logs for post-incident liability and safety verification, evaluate your implementation against hardware isolation and audit invariants:

- ☐ **Actuator-write isolation:** Can you explain how register permissions restrict CPU writers and how an IOMMU or bus firewall, if DMA or another processor can reach the drive, enforces the same channel mask?

- ❑ **Precision PTP hardware timestamping:** Can you combine qualified clock conversion, sensor sampling windows, and actuator response delays to determine when two observed events can be ordered?
- ❑ **Monotonic tamper-evident chaining:** Can you explain how protected key custody, hash chaining or authenticated storage, and power-loss tests support tamper evidence and retention?
- ❑ **Protective-input path:** Can you explain when the hazard analysis calls for an independent protective signal, and how its measured response avoids the lower-tier serialized queue?

An unverified authority log creates the illusion of accountability while permitting silent fault masking. Review this section before examining recurring engineering fallacies.

28.9 Fallacies and Pitfalls

Changing an authority register does not complete a safe handover. The machine must preserve its stopping margin and enforcement functions through the transfer, while the resulting logs must distinguish intervention from expert recovery.

Pitfall: *Treating human takeover timeout as a static time delay rather than a dynamic spatial stopping budget.*

A transport platform travels at $v = 15.0$ m/s with 42.0 m of clearance when it requests human takeover. Waiting 2.00 s for an unresponsive operator consumes 30.0 m of physical runway, leaving only 12.0 m. Under the example's braking conditions ($\mu = 0.70$, $a_{\text{brake}} = 6.87$ m/s²), stopping still requires $d_{\text{stop}} = v^2 / (2a_{\text{brake}}) = 16.38$ m. The fixed temporal timeout has silently eaten into the physical stopping budget. Handover timing must therefore follow current speed, spatial clearance, and available friction-limited deceleration, with autonomous fallback beginning while stopping remains physically feasible. A fixed temporal acknowledgment window cannot establish a safe margin across different physical operating states.

Fallacy: *Human manual takeover should bypass automated safety barriers and grant unconstrained actuator authority.*

A delivery robot approaches a pedestrian, and its operator takes over steering to avoid the person. The chosen turn leads toward a glass partition only 2.5 m away. Switching to human control only changes who is sending the command; it does not change the physical realities of momentum, friction, or nearby obstacles. Within this architecture, the safety enforcer must continue checking the manual proposal against the admissible motion set and constrain it when a kinodynamically feasible alternative exists (building on the feasibility principles from section 22.4). If the request cannot be admitted, the defined autonomous fallback still applies. Takeover lets the operator guide the task planning; it does not make force, clearance, and stopping limits optional at the joint actuator boundary.

Pitfall: *Relying on software-level cryptographic authentication without hardware rate limiting on real-time fieldbuses.*

A manipulator verifies cryptographic message authentication codes (MACs) on takeover frames, but a faulty fieldbus node floods the network at an illustrative 8000 frames/s. At an assumed 18.5 μ s of interrupt and verification work per frame, this traffic alone consumes $8000(18.5 \mu\text{s}) = 148$ ms of CPU time each second, or 14.8% of one core. Whether it causes a deadline miss depends on the remaining utilization, priorities, and interrupt budget. Authentication can preserve command integrity while leaving an availability hazard. Mailbox filtering and rate limiting should bound traffic before it reaches the critical execution path; qualification must test both command rejection and enforcement timing under admitted load.

Fallacy: *Training directly on raw human intervention logs captures expert demonstrations.*

A learning pipeline treats every depot-tractor takeover as an expert demonstration and trains on the preceding action window. The logs combine genuine policy failures, cautious early interventions, and operator kinematic mistakes, while the control transition itself often includes jerky physical corrections, delayed neuromuscular responses, and hesitant steering as the human orients to the

scene. Cloning those segments can reproduce the unstable behavior that made recovery difficult rather than the recovery the team wants to teach. Dataset curation must identify why intervention occurred, who held the authority register, and which commands actually restored acceptable operation. Reviewed recovery trajectories can supply useful demonstrations, but neither an intervention timestamp nor full manual authority alone establishes that the recorded behavior is suitable for imitation.

28.10 Summary

Human authority over an embodied physical AI system needs an engineered real-time contract that arbitrates takeover, revokes autonomous policy authority at commit, and selects a validated fallback within the available physical budget. Supervisory intervention governs the transition between proposers while the independent enforcer checks the admitted command and executes the selected stop when a lease lapses. Consider an analytical vehicle traveling at 15.0 m/s (54 km/h). If takeover negotiation takes 200 ms, the vehicle travels 3.0 m under its previously admitted command before the transfer begins. That travel belongs in the stopping-clearance budget.

Transferring control from an active policy takes time. Abruptly replacing an admitted policy setpoint with a different human target in one 1 ms cycle requests a torque step; the resulting shock or slip depends on the plant, load, and drive response.

The C^2 bumpless authority transfer ramp varies an authority parameter $\alpha(t)$ smoothly from zero to one between latched command endpoints. Its sampled implementation must be checked against command-rate limits, measured plant response, and the stopping reserve developed in section 22.4.

In an analytical example of a 2000 kg vehicle chassis with an actuator rate limit of 35 Nm/s and a lateral jerk bound of 2.5 m/s³, a proposed 150 ms blend can limit commanded rate if it meets the chosen torque and jerk constraints; the stated mass and rate limits alone do not establish traction. The enforcer admits the blend only with a validated stopping reserve and monitors the plant response; command smoothness alone cannot guarantee stability.

Intervention telemetry informs incident analysis and model retraining. A logged commit marks the recorded boundary where autonomous policy authority was revoked, subject to the logger's clock and sampling limits. Hash chaining and protected keys can reveal some later modification; they do not prove that every physical event was sampled. Command and authority records support reconstruction when paired with independent sensor, current, and contact evidence and their timing uncertainty.

For model training, the pre-commit interval is a **policy-owned** segment even when a human is deciding to intervene. Retain its learner-visited states, unsafe proposals, and authority flags as failure evidence and candidates for expert relabeling; exclude unsafe policy commands from expert imitation targets. The post-commit blend is enforcer-owned transition data. Only reviewed post-transfer human recovery states and actions that meet the task's validity criteria should become demonstrations. An intervention timestamp alone does not establish expert behavior.



Key Takeaways: Supervisory intervention and bounded authority transfer

- **Clearance governs waiting:** At 15 m/s, a two-second wait uses 30 m. Dispatch fallback by the earlier of the configuration ceiling and the state-dependent stopping deadline.
- **One actuator writer:** Commit revokes the policy token and latches its last admitted command. The enforcer owns BLEND and checks human proposals against the same physical limits.
- **Smooth commands need plant evidence:** A fixed-endpoint C^2 acceleration profile bounds commanded jerk. Its extra stopping distance must be reserved; tire, brake, and actuator behavior still need measurement.
- **Distinct stop paths:** Trusted protective wiring invokes a risk-assessed plant response. Invalid network packets are rejected; lost valid renewal invokes the lease fallback.
- **Causal intervention data:** Keep policy-owned failure states for audit and possible expert relabeling. Use only reviewed post-transfer recovery actions as imitation targets.

Supervisory handover is not an instantaneous discrete toggle; it is a physical dynamic transition governed by plant momentum, actuator bandwidth, and human reaction latency. Relying on an unmonitored human operator to catch a drifting autonomous policy without reserved spatial clearance and validated transition dynamics can make intervention too late or physically infeasible. Engineering trustworthy shared autonomy requires structuring intervention as a bounded, verifiable contract that enforces dynamic limits regardless of operator panic.



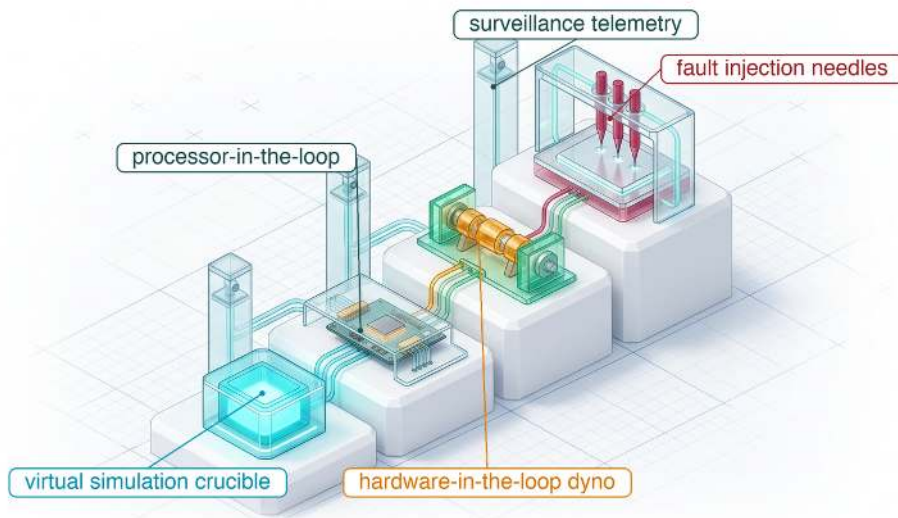
What's Next: From authority revocation to physical verification

A release safety argument needs fault-injection evidence for the specified authority and fallback boundaries. In chapter 30, we turn the authority and fallback claims into fault-injection tests. Packet drops, bus flooding, and actuator saturation test whether the integrated system follows the specified transition and reveal limits of the exercised operating envelope.

Adversarial Verification

DRAFT

Adversarial Verification



- 30.1 Fault Injection
- 30.2 The Operating Envelope
- 30.3 Deriving the Fault List
- 30.4 The Qualification Ladder
- 30.5 Fault Oracles and Acceptance
- 30.6 Fault Injection on Hardware
- 30.7 The Fault Manifest
- 30.8 Fallacies and Pitfalls
- 30.9 Summary

Purpose

Why is a robot that has operated for weeks without a single crash not evidence that the machine is safe?

Weeks of successful operation establish behavior under encountered conditions but leave untested scenarios like sensor freezes or late plans. A machine may seem dependable if its operating environment has never challenged its safety assumptions. Verification in physical AI systems involves transforming assumptions into specific testable conditions, such as simulating sensor failures or unexpected environmental changes, which engineers can deliberately manipulate to observe the system's response and ensure safety. The fault list derives from the design's claims about freshness, timing, and physical response, not from generic stress tests. Each experiment must record whether the integrated machine detects the violation and contains its consequences within the stated envelope. Simulation can expose an inconsistent contract, while hardware tests reveal effects that the simulated plant omits. Neither supports claims beyond the conditions it represents. Whole-system verification connects the brain's assumptions, the nervous system's responses, and the body's measured behavior into evidence that can support, narrow, or reject an operating claim.

Brain
1–50 Hz
foundation models, SLAM, planning

above: propose; below: command

Nervous System
1000 Hz
TSN, intent leases, lock-free IPC

Body
continuous
real-time MCU, safety filters, CBFs

Boundary
continuous
sensors, actuators, kinematics

Physical World
continuous
contact mechanics, friction, mass

Prerequisite: Physical failure modes and measurement limits providing verification targets originate in section 4.7.



Learning Objectives

- Define operating envelopes and acceptance criteria that make the machine's physical claims falsifiable
- Design fault-injection tests that challenge timing, data validity, and authority claims on physical hardware
- Evaluate which simulation and hardware tests support each claim and where their evidence ends
- Construct reproducible fault records linking injected conditions, detected violations, response times, and physical outcomes

30.1 Fault Injection

An incident-free operating log answers only the conditions it encountered. Three quiet weeks in a warehouse do not exercise a frozen level sensor, a brownout, or a bus occupied by DMA traffic. Whole-system verification therefore turns each safety claim into a deliberate fault and an observable physical outcome. The sampling cost of waiting for a rare event also grows: under a stipulated constant event rate, a tenfold rarer event takes ten times as many miles to observe on average (figure 30.2). This is a sampling illustration, not a trend inferred from vehicle disengagement reports; a disengagement is not itself a crash or a uniform safety anomaly. Injected faults can expose the untested mechanism directly, with a test oracle and limits declared before the run.

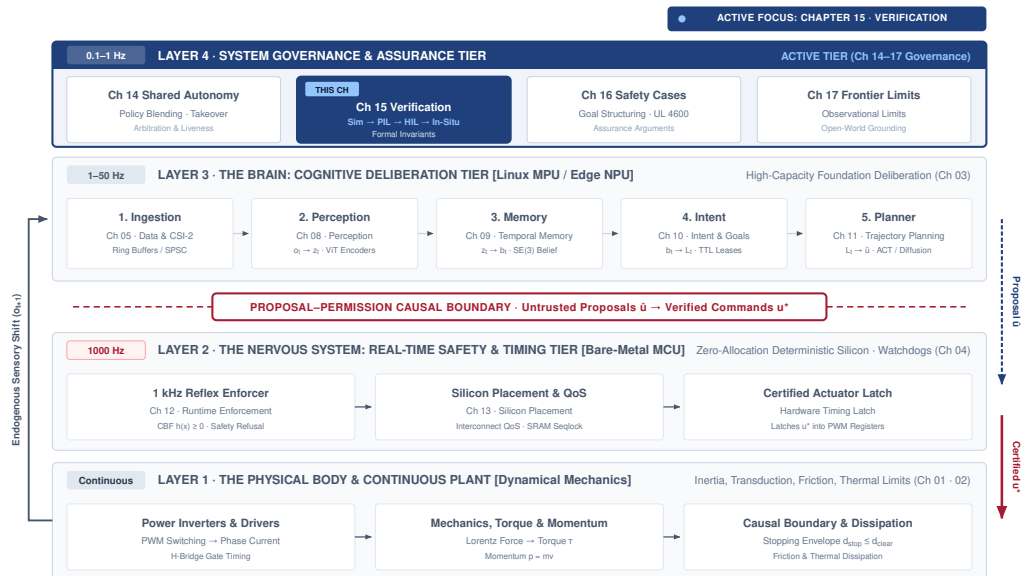


Figure 30.1: Verification architecture focus: The four-tier physical AI systems architecture highlights the system governance and assurance tier, with verification as this chapter's active focus. Read the highlighted box as the qualification ladder that walks a safety claim from simulation through processor-in-the-loop and hardware-in-the-loop stages to in-situ trials, attacking one cyber-physical boundary at each rung.

As located in the architecture stack in figure 30.1, whole-system verification is the second pillar of Tier 4 (System Governance & Assurance). Operating at supervisory timescales, verification provides the empirical evidence required to justify or reject operational release claims. Its duty is not to evaluate isolated model accuracy benchmarks or unit-test isolated software components, but to assess how the assembled cyber-physical machine behaves when stressed across all tier boundaries:

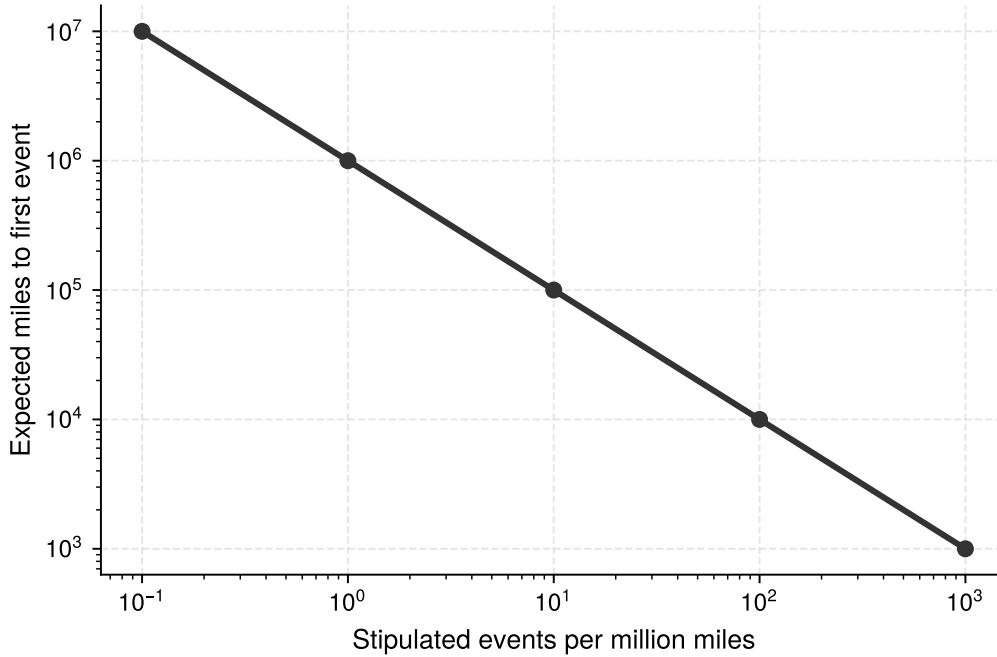


Figure 30.2: Illustrative rare-event exposure: For independent miles with a stipulated constant event rate r per million miles, expected distance to the first event is $10^6/r$ miles. The constructed points show why passive observation becomes costly as the target event becomes rarer; they are not measured fleet disengagement or crash data.

- **The Brain (Tier 3):** evaluating whether cognitive perception models, semantic memory lookups, and neural trajectory planners degrade predictably under corrupted sensor feeds, frame drops, and execution latency spikes;¹
- **The Nervous System (Tier 2):** verifying whether fieldbuses, watchdogs, and independent enforcers detect contract violations and start validated fallback within deadlines derived for the machine (a 1 ms enforcement tick in the worked architecture);² and
- **The Body (Tier 1):** measuring whether physical actuators, motor drives, mechanical brakes, and pressure relief valves physically contain kinetic energy and mass without exceeding thermal derating or structural dissipation limits.³

Consider a flow regulator designed to keep a 100.0 L buffer tank below a 95.0 L overflow weir. Under nominal conditions, liquid enters at 40.0 L/min, an ultrasound sensor reports level, and an electric actuator adjusts the discharge valve at 50 Hz. Routine operation near 80.0 L says little about a combined inlet surge and stale observation. Suppose an upstream switch raises inlet flow to 80.0 L/min while discharge remains at 40.0 L/min, and a bus conflict adds 140 ms to a 10 ms sensing delay. The net 40.0 L/min fill rate adds only 0.10 L during those 150 ms; from 90.0 L, continued net filling would take 7.5 s to reach the weir. The test must therefore inject a sustained response delay or a failed valve to challenge overflow containment. Rapid valve closure is a separate pressure-surge hypothesis: a 12 bar water-hammer claim would require a measured or modeled wave speed, pipe geometry, and closure profile. The two hazards lead to different fault injections and physical oracles.

Converting such failure modes into verification evidence requires structured methods designed to challenge specific system properties. Five essential ingredients turn a physical run into a test result that can support a safety case:

1. **A bounded claim:** stating the exact invariant the machine must maintain (e.g., maintaining liquid volume below 95.0 L and transient line pressure below 6.0 bar);

¹ | **Cognitive Trajectory Contracts:** As developed in section 6.1 and section 22.7, deliberative planners and neural policies operate under explicit assumptions of state freshness and kinematic feasibility. Verification challenges these assumptions with latency stalls and perceptual corruption. When an upstream decision expires, the independent enforcer must revoke its tracking authority and execute a state-matched fallback whose sensing, actuator, and clearance assumptions have been tested.

2 | Watchdogs and Real-Time Seams: As detailed in section 8.1 and section 24.9, Tier 2 safety enforcers and hardware watchdog interlocks provide deterministic fallback paths. Verification evaluates whether these enforcers trip predictably within their budgeted deadlines when upstream software hangs or fieldbuses stall. Any unhandled preemption delay in the safety monitor permits transient actuation commands to propagate, causing physical overshoots that breach mechanical yield limits.

3 | Actuator Dynamics and Energy Dissipation: As analyzed in section 2.2 and section 4.1, physical safety outside the nominal operating envelope is delivered by mechanical energy dissipation, thermal margins, and structural compliance rather than by software optimization. When destructive kinetic or hydraulic energy exceeds these passive hardware absorption limits, structural deformation or component rupture occurs regardless of policy intent. Whole-system verification must therefore characterize physical containment boundaries directly on instrumented dynamometer and destructive testbeds.

Prerequisite: Non-asymptotic Clopper-Pearson statistical confidence intervals are formulated in section 14.1.

2. **Defined test conditions:** specifying the initial state space, fluid viscosity, ambient temperature, and supply pressure range;
3. **A precommitted oracle:** an automated evaluator that inspects recorded telemetry against the claim without human intervention or post-hoc adjustments;
4. **A stated sampling and injection procedure:** deliberately driving the machine toward its performance limits and injecting faults into sensors, buses, and actuators; and
5. **Provenance:** the complete metadata record of model weights, calibration tables, sensor serial numbers, and raw bus logs permitting independent reproduction.

The overflow and pressure-surge hypotheses require separate envelopes, injection points, and pass conditions. The first decision is where nominal regulation must yield to an independent containment response.

30.2 The Operating Envelope

The **operating envelope** defines the complete set of simultaneous physical states, environmental disturbances, sensing conditions, execution latencies, compute loads, electrical supply variations, and operator response times under which the machine is claimed to satisfy its performance specifications. Defining this envelope requires precise numerical boundaries for an automated test framework to drive the system deterministically across each constraint. For a fluid regulation system, this means specifying not merely a nominal liquid level, but the exact allowable ranges for fluid temperature, kinematic viscosity, supply line pressure fluctuations, ambient temperature, acoustic transducer signal-to-noise ratios, CAN bus arbitration jitter, CPU thermal throttling states, supply rail voltage droop, and the maximum permissible time before an operator acknowledges an alert.

An operating envelope is distinct from the larger **survival envelope** that encompasses it. Inside the operating envelope, the learned policy and primary tracking controllers fully regulate the physical process, maintaining closed-loop tracking within tight tolerances. When external disturbances, sensor failures, or compute stalls push the system across the boundary of the operating envelope, nominal regulation is conceded. At that threshold, the machine enters the survival envelope, where the primary objective shifts entirely to bringing the physical body to a bounded, safe state before structural limits are breached. In the flow regulator, the survival envelope defines the broader region of volume, pressure, and temperature within which emergency shutoff valves and mechanical relief mechanisms can successfully vent pressure and arrest inflow before a pipe ruptures or fluid overflows secondary containment. Beyond the boundary of the survival envelope lies catastrophic physical failure, where stored energy or incoming mass exceeds the mechanical dissipation capacity of the hardware. This boundary between control and survival manifests directly in destructive vehicle crash qualification (figure 30.3; see also section 2.2): while nominal autonomous policies operate strictly within the dynamic control envelope, catastrophic perception or planning failures push the machine across the boundary into the survival envelope, where structural crumple zones and deterministic passive or pyrotechnic enforcers must physically dissipate kinetic energy before passenger compartment survival limits are breached.

Defining the boundary of the operating envelope requires deriving physical limits from first principles rather than asserting convenient software timeouts. Consider a pumped chemical buffer reservoir with an overflow weir at $V_{\max} = 95.0$ L and a nominal operating target of $V_0 = 80.0$ L. Under an upstream valve failure disturbance, the inlet flow rate surges from nominal 40.0 L/min to $Q_{\text{in}} = 1.50$ L/s (90.0 L/min), while a downstream pump trips, reducing discharge to $Q_{\text{out}} = 0.10$ L/s (6.0 L/min), yielding a net liquid accumulation rate of $dV/dt = Q_{\text{in}} - Q_{\text{out}} = 1.40$ L/s. Conservation of volume dictates that when the liquid crosses an initial trigger threshold $V(t_0) = 88.0$ L, the remaining volume margin is $\Delta V = V_{\max} - V(t_0) = 95.0$ L $-$ 88.0 L = 7.00 L. The total physical time-to-breach before containment loss is governed by equation 30.1:

$$t_{\text{breach}} = \frac{V_{\max} - V(t)}{Q_{\text{in}} - Q_{\text{out}}} \quad (30.1)$$

which yields $t_{\text{breach}} = \frac{7.00 \text{ L}}{1.40 \text{ L/s}} = 5.00$ s.

Every delay in the independent emergency path consumes part of that five-second budget. Assume a validated upper bound of $t_{\text{detect}} = 80 + 20 + 100 + 10 = 210$ ms for level acquisition, bus delivery,



Figure 30.3: Crashworthiness test after loss of nominal control: Post-test side view of a 2018 Honda Odyssey after NHTSA NCAP moving-deformable-barrier test 10132 (NHTSA record; photo provenance). Its side structure is deformed; contrasting targets and test fixtures remain attached. The image illustrates a passive crashworthiness test, but does not establish the sensors' roles, measurement resolution, or this chapter's control-response times. Photo: NHTSA/MGA.

filtering, and MCU safety evaluation. Assume the isolation valve seats within $t_{\text{valve}} = 1.20$ s and cuts the modeled inflow then. The direct protective path therefore seats it by $t_{\text{stop}} = 0.21 + 1.20 = 1.41$ s after the 88.0 L trigger; under the stipulated constant 1.40 L/s net inflow, volume reaches $88 + 1.40(1.41) = 89.974$ L. A separate 0.30 s pressure-settling assumption may matter to a pressure oracle, but it does not add volume after a valve that has stopped this inflow. The learned policy may continue to propose nominal valve settings; its inference latency has no authority to delay this MCU trip.

For contrast, deliberately route the protective trip through a stalled neural path in a **failed architecture**. A 3.60 s stall before the same detection and valve delays would seat the valve at $3.60 + 0.21 + 1.20 = 5.01$ s, after the five-second breach time. The constant-flow model predicts $88 + 1.40(5.01) = 95.014$ L, or 14 mL beyond the weir. This is precisely the dependency the independent MCU path must eliminate. With a separately reserved 1.00 s buffer, a conservative direct-trip threshold follows from the validated delay bounds:

$$V_{\text{trip}} \leq V_{\text{max}} - (Q_{\text{in}} - Q_{\text{out}})(t_{\text{detect}} + t_{\text{valve}} + t_{\text{buffer}}) \quad (30.2)$$

Using equation 30.2 for the stated envelope gives $V_{\text{trip}} \leq 95.00 - 1.40(0.21 + 1.20 + 1.00) = 91.626$ L. The chosen 88.0 L threshold is below this ceiling. A measured $P_{99.9}$ compute latency can inform a risk analysis, but it cannot substitute for a justified bound on this protective path. The level threshold and valve seating claim also depend on the flow envelope, level-estimate error, and actual isolation effectiveness.



Checkpoint 30.1: Inlet surge reaction and sensing latency

Consider an embodied fluid regulation or chemical containment plant operating under boundary disturbances:

- ☐ **Protective reaction budget:** Can you sum the independently bounded detection and valve-seat delays and calculate the resulting volume at the 88.0 L trigger?
- ☐ **Inference independence:** Can you explain why a 3.60 s neural stall causes overflow only in the deliberately failed design that routes isolation through inference?

- ❑ **Trip threshold:** Can you derive the threshold from bounded net inflow, detection, valve travel, and a declared reserve rather than treating a percentile as a hard deadline?

Tabulating separate minimum and maximum values for individual variables fails to capture the true operating boundary because physical and computational dimensions are coupled. In isolation, an inlet pressure of 6.0 bar, a fluid viscosity of 45.0 mPa · s, a valve slew rate of 30.0 deg/s, and an observation age of 120 ms might each fall within their respective single-variable ranges. When these conditions coincide, the higher viscosity increases fluid shear across the valve spool, slowing mechanical travel below the nominal slew rate. Simultaneously, the elevated inlet pressure accelerates volume accumulation, while the 120 ms observation age hides the initial pressure surge from the policy. The physical state crosses into the survival envelope long before any single parameter exceeds its isolated maximum. The true operating boundary is an interconnected hypersurface in state, parameter, and latency space—a multidimensional boundary where temperature, pressure, and timing all physically affect one another. A valid verification suite must probe this combined surface rather than checking isolated scalar limits.

To prevent envelope boundaries from deteriorating into optimistic assumptions, every defined parameter must specify its physical units, its measurement provenance, its calibrated uncertainty, and a designated subsystem engineer who remains accountable for its validity. A boundary stated as a maximum pressure of 8.0 bar is incomplete without recording whether that limit derives from hydrostatic burst testing of the flanged joints, finite-element pipe stress models, or pump stall characteristics. The boundary specification must also record the measurement uncertainty of the sensors used during characterization, such as ± 0.15 bar at P_{99} . When system components change, such as swapping an elastomeric seal or upgrading a CAN transceiver, the designated owner must formally re-evaluate whether the physical boundary has shifted.

Verification of the operating envelope requires a disciplined sampling strategy that deliberately attacks the boundary surface. Testing nominal interior points, where the tank sits at 80.0 L, ambient temperature is 21.0°C, and sensing latency is 12 ms, demonstrates only that the machine functions under ideal conditions. To generate rigorous verification evidence, the test framework must place test points directly on, immediately inside, and immediately outside each consequential boundary. Evaluating points just inside the boundary verifies that the policy maintains closed-loop stability under maximum permissible stress. Evaluating points on the boundary confirms that safety margins do not collapse under nominal sensor noise. Evaluating points just outside the boundary confirms that safety enforcers intervene predictably, taking authority from the learned policy and bringing the system to a safe halt within the survival envelope before containment is lost.

🔍 Systems Perspective 30.1: The hardware falsification principle

If a safety property cannot be deterministically falsified by injecting a physical or electrical fault on target hardware, the property is an unverified assumption rather than an engineering invariant.

Formally documenting the operating and survival boundaries converts the design claims of the preceding chapters into concrete targets for fault injection. When every physical constraint, latency budget, and environmental limit is stated as a verifiable number, the safety engineer knows precisely what conditions the machine must withstand, what thresholds must trigger protective intervention, and what dynamic claims must be subjected to deliberate failure.

30.3 Deriving the Fault List

The initial fault list derives from inverting design records across the architecture. Chapter 10 through chapter 14 supply the properties of the learned policy, including training distribution bounds, state-action assumptions, and inference latency ceilings. Chapter 16 through chapter 26 provide interface and execution properties, such as bus bandwidth allocations, serialization overheads, observation freshness guarantees, and state estimation bounds. Chapter 28 supplies the properties of the authority-transfer mechanism, specifically the maximum latency to disconnect a failing policy and

the independent override authority of the safety enforcer. Inverting these records systematically generates an initial candidate set for verification. If an analytical model of a flow control loop records that the learned policy produces a valid valve command every 20 ms under all flow rates between $0.10 \text{ m}^3/\text{s}$ and $2.50 \text{ m}^3/\text{s}$, the verification suite inverts that claim by injecting stalled command streams, delayed observations, and out-of-range sensor readings to test whether the downstream nervous system detects the violation within its budget.

Every recorded design property becomes a testable fault only when structured as a five-element chain comprising the negated condition, the physical injection point, the expected detector, the bounded response, and an observable pass condition. In a high-pressure fluid delivery system, consider a flow meter that publishes volumetric discharge rates across a Controller Area Network (CAN) fieldbus with an analytical latency bound of 5.0 ms. The negated property injects an artificial transport delay of 25 ms at the network interface buffer. The expected detector is the stale-frame monitor in the safety enforcer, the required response is a transition to a recirculating bypass within 10 ms, and the observable pass condition is that fluid pressure at the manifold remains below 6.0 bar at P_{99} . When any element of this chain cannot be defined or observed, the system contains a **verification gap**. A verification gap indicates that the claim cannot be falsified by test. Such gaps cannot be ignored; they require dedicated hardware instrumentation, a narrower operational claim, or a redesign of the execution path.

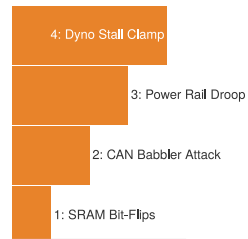
This inversion process must challenge data provenance, simulation validity, and offline evaluation assumptions with physical runtime packets. Distribution shift and misleading optimization metrics are system faults even when every communication frame arrives without corruption. If a learned policy was trained on hydrodynamic simulations that assumed a constant fluid viscosity of $1.0 \times 10^{-3} \text{ Pa} \cdot \text{s}$, cooling the fluid to increase viscosity to $3.5 \times 10^{-3} \text{ Pa} \cdot \text{s}$ alters the pump head curve and introduces cavitation noise. Inverting the evaluation claim requires injecting these unmodeled fluid dynamics, simulated sensor bias, and synthetic data perturbations where a high nominal reward hides high-frequency actuator jitter. Because software and physical mechanics remain coupled across the system boundary (section 2.2), a physical AI system fails when valid, uncorrupted commands drive the physical body into an unstable mechanical regime because training assumptions were violated.

At the structural boundaries established across chapter 16 through chapter 26, the fault generator inverts each inter-module contract at its physical point of handoff. These tests negate observation freshness, state validity, intent lifetime, planning deadlines, enforcement independence, and resource allocations. If a trajectory planner issues an actuation profile with an intent lifetime of 100 ms, the test environment injects an otherwise valid actuation frame at 125 ms to confirm that the actuator interface rejects it as expired. If the safety enforcer claims execution independence from the primary policy computer, the test saturates the shared memory bus with maximum DMA traffic (simulating a runaway background process hogging memory bandwidth) to confirm that the enforcer still meets its 1.0 ms control deadline. Conversely, fault injection must remain disciplined, because bus starvation, clock skew, and supply voltage fluctuations should be injected only where the design explicitly claims tolerance. Testing arbitrary failure modes from a generic menu invents requirements that the machine was never specified to meet, diverting engineering effort away from validating actual architectural contracts.

The structural limit of record inversion is absolute. Inverting recorded properties produces only the faults that the design team already modeled, which is precisely the set of behaviors that will not surprise the creators of the machine. The completeness of an inverted fault list is bounded by the completeness of the original design claims. If the engineering records never documented how an electric motorized valve behaves when ambient temperatures freeze the mechanical seals, no record exists to invert, and the failure mode will not appear in the verification suite. Inversion guarantees that every documented claim is tested, but it remains blind to unstated assumptions, overlooked physical hazards, and unmodeled failure dependencies.

Overcoming this limitation requires a second, independent source that works forward from the physical machine rather than backward from design documents. This forward analysis enumerates what the physical body, the power distribution network, the fluid dynamics, and the operating environment can do regardless of what the software believes. Physical pipes experience destructive shockwaves when rapid valve closure abruptly halts fluid momentum, a phenomenon known as

⁴ | **STAMP/STPA Hazard Analysis:** Developed at MIT by Nancy Leveson, STAMP treats system safety as a dynamic control problem rather than a component reliability failure. In autonomous physical AI, emergent hazards arise at hardware-software seams where asynchronous bus latency or filter lag invalidates low-level stability guarantees. A rigorous verification campaign must therefore inject boundary stress across these seams to verify that safety interlocks trip before dynamic envelope limits are breached.



Fault injection climbs from software bit-flips to physical dyno shaft jamming.

water hammer; mechanical valve seats accumulate particulate deposits that prevent full seating, elastomeric seals wear under repeated pressure cycling, and pump vibration induces harmonic resonance in downstream sensor brackets. These physical phenomena originate in mechanics, materials science, and fluid dynamics, operating entirely outside the digital abstractions of the nervous system—a reality formalized in Leveson's **Systems-Theoretic Accident Model and Processes (STAMP)** framework⁴ (Leveson 2011).

For example, start with the physical hazard of a valve that cannot seat while the tank fills. The unsafe control action is continued inlet pumping after the 88 L trip. Inject a seat obstruction or motor stall on the rig, observe valve position and motor current independently of the commanded close bit, and require the MCU to isolate the upstream pump or use a separately validated containment path. The precommitted pass condition is measured volume below 95 L throughout the declared flow and sensor-error envelope; an unmeasured valve position makes the trial unobservable for the seating claim. This forward chain yields an injection and oracle even if the original software requirements omitted seat obstruction.

Working forward from the physical hardware exposes entire fault classes that no software property record implies. These include total clock loss, supply rail brownouts where digital logic levels blur unpredictably between 0.8 V and 2.0 V, electromagnetic interference from high-current pump contactors coupling into analog pressure lines, floating high-impedance output pins subject to stray electromagnetic coupling during power-on reset transitions, corrupted non-volatile configuration memory, and shared reset lines that accidentally bridge independent domains. If an emergency pressure-relief circuit and a learned policy accelerator share a single power management integrated circuit, an undervoltage transient on the accelerator can reset the safety circuit, disabling the relief mechanism at the exact moment the policy loses control. No software record contains this failure mode because each software module assumes its underlying power rail is an ideal, independent DC voltage source.

Physical forward analysis also challenges the common-cause independence claims established in chapter 8 and chapter 26. Two control paths that appear isolated in an architectural diagram often share a single physical vulnerability. A primary flow sensor and a redundant pressure switch may route their wiring through a single conduit exposed to mechanical abrasion, or share a common analog reference voltage within a multi-channel converter, or rely on identical timer silicon vulnerable to the same manufacturer errata. If an electromagnetic transient induces simultaneous parasitic PNP latch-up conditions across both channels, treating them as statistically independent failures invalidates the entire safety argument. **Common-cause analysis** traces physical routing, power paths, and shared clock trees to ensure that no single physical event can disable both the operational policy and the intervention mechanism simultaneously.

The physical source must also encompass operator error modes and foreseeable misuse from chapter 28, treating human interaction as an operational hazard of the machine. Field operators force bypass switches during maintenance, misconfigure manual throttling valves upstream of automated manifolds, disconnect telemetry cables to silence acoustic alarms, and request setpoint changes that exceed the thermal dissipation capacity of pump motors. When an operator commands an instantaneous step increase in flow from 0.20 m³/s to 2.00 m³/s, the verification framework must verify that the nervous system clamps the command to a safe acceleration rate, preventing cavitation and hydraulic shock while maintaining closed-loop stability.

Finally, the fault inventory must directly challenge the authority-transfer claims established in chapter 28. A policy driving a motorized valve at full speed builds up mechanical momentum and electrical back-EMF that actively resist rapid deceleration. In an analytical test scenario where a valve actuator rotates at 90°/s, the fault injection framework triggers an unrecoverable policy fault and measures whether the safety enforcer de-energizes the motor driver within 4.0 ms, engages the mechanical brake, and halts flow before line pressure exceeds the pipe burst threshold of 10.0 bar. The intervention mechanism is not an unassailable arbiter; it is a physical subsystem whose claims of speed, independence, and override capability must be deliberately attacked, measured, and falsified before the assembled machine can be judged ready for structured qualification.

Working forward from the physical machine adds representative fault classes to the record-derived list in table 30.1. Completeness still requires a separate hazard-analysis argument for the specified plant and operating envelope.

Table 30.1: Representative hardware and cyber-physical fault classes: Physical and logical failure modes mapped to injection points, detector, response, and observable pass criteria; the table does not establish exhaustive coverage for a specific machine.

Failure Domain & Fault Mode	Physical Mechanism & Injection Point	Target Manifestation & Causal Impact	Detection Primitive & Latency Budget	Enforced Safe Response & Bounded Recovery	Observable Pass Criterion
Sensor Corruption & Signal Bias	Piezoelectric transducer drift, stuck-at-zero ADC register, injected via DAC inline interceptor	Stale or unphysical state estimate $\hat{x}(t)$; policy commands erroneous slew rate	χ^2 innovation / plausibility monitor ($\Delta t_{\text{det}} \leq 20$ ms)	Isolate faulty sensor channel; switch to dead-reckoning or secondary estimator	Process state remains inside survival envelope: $P(t) \leq P_{\text{max}} + \Delta P_{\text{margin}}$
Bus Contention & Frame Loss	Babbling-idiot CAN transceiver, real-time Ethernet queue overflow, injected via CAN disturbance node	Missing observation frames; heartbeat starvation; outdated plan execution	Cyclic redundancy check (CRC) & frame arrival watchdog ($\Delta t_{\text{det}} \leq 10$ ms)	Reject expired proposal; freeze actuator to zero-torque safe hold or clamp valve	End-to-end reaction time $t_{\text{react}} \leq t_{\text{budget}}$; zero unhandled bus exceptions
Power Brownout & Voltage Droop	Motor contactor inrush transient, LDO rail sag ($V_{\text{core}} < 0.85$ V), injected via programmable DC load	Indeterminate logic states; corrupted SRAM registers; asynchronous MCU reset	Hardware Undervoltage-Lockout (UVLO) monitor ($\Delta t_{\text{det}} \leq 15$ μ s)	Hardware crowbar circuit fires; de-energize gate drivers; assert fail-safe spring return	Logic pins never float in undefined state; secondary relief valve seats within 50 ms
Actuator Jam & Mechanical Stall	Particulate contamination in valve, bearing seizure under shock load, injected via hydraulic brake clamp	Locked rotor; severe back-EMF collapse; rapid stator winding heating ($I^2 R$)	Motor phase overcurrent & Hall-effect velocity stall monitor ($\Delta t_{\text{det}} \leq 5$ ms)	De-energize H-bridge power stage; engage electromechanical brake; route bypass	Stator temperature $T_{\text{coil}} < 180^\circ\text{C}$ (Class H insulation limit); line pressure relieved before burst disc rupture
Watchdog Panic & Priority Inversion	DMA bus-master lockup on GPU, deadlock in high-priority RTOS task, injected via bus stall emulator	Interrupted control loop cadence; un-serviced safety heartbeat; hung NPU	Independent windowed hardware watchdog timer ($\Delta t_{\text{det}} \leq 2.0$ ms)	Assert Non-Maskable Interrupt (NMI); hardwired authority transfer to safety MCU	Takeover completed in ≤ 4.0 ms; zero command bleed-through from hung processor

30.4 The Qualification Ladder

Subjecting an assembled machine to an inventory of faults requires choosing an execution environment for each test. Each evaluation environment exercises a different subset of the physical and computational causal loop that connects sensor inputs, inference pipelines, communication buses, actuator drives, plant dynamics, and environmental feedback. Qualification environments do not form a strict hierarchy. Because each mode closes, breaks, or substitutes different physical and computational causal links, higher rungs cannot completely subsume lower ones. Instead, they establish distinct categories of evidence while remaining blind to others.

Definition 30.1: The four-stage qualification ladder

Four-stage qualification ladder is the structured progression of cyber-physical evaluation environments—Software-in-the-Loop (SIL), Processor-in-the-Loop (PIL), Hardware-in-the-Loop (HIL), and In-Situ Physical Fault Rigs—that systematically closes, breaks, or substitutes specific arcs in the causal loop to balance combinatorial test throughput against physical and temporal execution fidelity.

1. **Significance:** No single test environment can validate a physical AI system. Software simulators allow massive combinatorial sweeps ($> 10^8$ cycles/day) but omit target silicon

bus contention and material wear; physical rigs expose authentic mechanics and electrical transients but cannot execute destructive fault sweeps at scale.

2. **Distinction:** Unlike a cumulative testing staircase where higher rungs subsume lower ones, each qualification rung exercises complementary, non-overlapping causal mechanisms while remaining blind to others.
3. **Common pitfall:** Assuming that passing extensive Software-in-the-Loop (SIL) simulation implies safety on embedded hardware. SIL abstracts away microcontroller clock drift, DMA bus lockups, interrupt jitter, and power supply voltage sag.

Offline trace replay feeds recorded sensor data into the computational pipeline, evaluating how the software responds to historical operational sequences. Because the input signals come from physical sensors operating in real environments, replay preserves the true noise profiles, quantization anomalies, and electrical transients of operational hardware, such as the high-frequency ripple from a turbine flowmeter operating at 1000 Hz. Replay verifies whether the software produces deterministic outputs, meets internal latency deadlines, and handles historical edge cases without crashing. Yet replay leaves the feedback loop broken. The recorded inputs are frozen in time, completely decoupled from the commands the policy emits under test. If an updated policy alters a valve opening command at an analytical test offset of $t = 50$ ms, the replay log continues to supply the pressure measurements that resulted from the original historical command. Replay establishes algorithmic reproducibility and computation time under real input distributions, but it cannot establish closed-loop dynamic stability or evaluate how the physical plant responds when the policy diverges from the recorded path.

Software-in-the-loop (SIL) simulation restores the closed feedback loop by substituting mathematical models for the physical plant, actuators, sensors, and environmental dynamics on high-performance workstation or cloud clusters. Within this synthetic environment, the simulation clock is decoupled from physical wall-clock time—meaning a ten-minute physical process can be computed in seconds—allowing an engineering team to execute massive parallel parameter sweeps exceeding 10^8 cycles/day. SIL enables dense adversarial testing across millions of operating conditions, systematically sweeping line pressures from 1.0 bar to 25.0 bar, injecting tensor bit-flips in neural network weight matrices, or commanding instantaneous fluid flow steps without risking physical hardware destruction. SIL can expose failures of the integrated control logic, estimation filters, and enforcers at sampled model states and disturbances; a stability claim over a continuous region needs a separate proof or justified analysis. However, this evidence’s validity is strictly bounded by the fidelity of the underlying numerical ODE or PDE models. If the simulator models a hydraulic valve as a pure first-order lag with a fixed time constant of 30 ms, it omits messy physical phenomena such as fluid-induced hydrodynamic torque, dynamic seal friction hysteresis, actuator torque limits, and cavitation-induced vibration. Furthermore, running software on a workstation host abstracts away the harsh realities of embedded target silicon, such as physical clock drift, direct memory access contention, cache thrashing, and dynamic thermal throttling. Passing simulation trajectories supports behavior under the chosen numerical model and sampled conditions; it neither proves the controller correct for all model states nor validates the plant model.

Processor-in-the-loop (PIL) evaluation closes the architectural gap between host compilation and embedded execution by cross-compiling the neural policy and safety control algorithms for the target Instruction Set Architecture (ISA)—such as ARM Cortex-A/M cores or RISC-V microcontrollers—and executing the resulting binary on actual target silicon or cycle-accurate core emulators. The target processor is interfaced to a host-simulated plant via high-speed co-simulation links (such as JTAG, background debug interfaces, or cycle-synchronized shared memory). PIL verification directly validates compiler optimization levels (−O3 versus −O0), target vector acceleration libraries (such as ARM Helium or CMSIS-NN), fixed-point and quantized tensor scaling artifacts (e.g., int8 versus float32 truncation error), stack memory headroom, register spills, and worst-case execution time (WCET) bounds. PIL exposes subtle software faults that never appear in host SIL, such as compiler-induced register aliasing, unaligned memory access faults, and arithmetic overflow in fixed-point state estimators. Nevertheless, PIL operates across a synthetic peripheral boundary: it simulates external buses and timers, remaining blind to physical transceiver electrical contention,

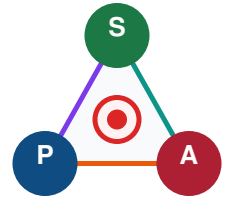
bus voltage droop, SerDes bandwidth limits, electromagnetic interference, and dynamic actuator back-EMF.

Hardware-in-the-loop (HIL) bench testing mounts the physical target computing hardware (the proposal-permission Dual-Brain architecture from section 6.1), communication fieldbuses, power supplies, and actuator drive stages onto an instrumented laboratory stand, interfacing them in hard real-time to high-bandwidth field-programmable gate array (FPGA) plant emulators and dynamic dynamometer testbeds. This mode subjects the real target electronics to physical electrical loads and authentic temporal bus jitter. HIL testing exposes real-world bus arbitration delays across physical CAN and Ethernet transceivers, microcontroller interrupt preemption latencies, DMA bus lockups, and the thermal derating of motor driver power stages delivering 10 A under continuous load.

By coupling the actuator drive to a high-bandwidth four-quadrant regenerative dynamometer ($f > 1$ kHz), HIL benches dynamically synthesize mechanical counter-torques. This setup simulates aerodynamics and friction reversals, and injects sudden joint stalls and high-rate load steps to measure physical back-EMF collapse, inverter Joule heating (I^2R), gearbox compliance and backlash, stator delays, and gate-driver switching spikes. HIL establishes that the embedded control software executes reliably on actual silicon, that clock drift between distributed microcontrollers remains within budgeted bounds, and that physical safety switches disconnect actuator power within the required ISO 10218 / ISO/TS 15066 safety envelopes, such as a 4.0 ms deadline following an overpressure trigger. The limitation of the HIL bench lies in its synthesized plant boundary: an FPGA model or mechanical dynamometer cannot reproduce microscopic material fatigue, fluid viscosity degradation under chemical shear, seal wear, or destructive structural burst limits.

In-situ physical fault rigs and shadow deployments evaluate the fully assembled physical machine—complete with production mechanics, hydraulic conduits, wiring harnesses, and primary sensors—operating under authentic operational stresses. Physical fault rigs operate within reinforced laboratory containment cells equipped with dedicated destructive actuators, including pyrotechnic valve jammers, pneumatic line clamps, and magnetic brake clamps that physically seize rotating shafts or obstruct high-pressure fluid lines. In-situ testing captures the full unmodeled complexity of physical reality: nonlinear Coulomb friction reversals, spatial kinematics in SE(3), turbulent fluid cavitation noise, structural resonant vibrations, and thermal runaway. Alongside destructive rigs, shadow operation deploys the fully assembled software and computing hardware into the field alongside an incumbent controller, processing live sensor telemetry from an operating industrial plant without exerting physical control over the actuators. Shadow deployment establishes that the perception and control stack operates reliably under true production workloads without software exceptions or false-positive fault declarations. Yet shadow testing cannot establish the physical consequences of the commands it generates. Because the shadow policy commands are intercepted and discarded at the actuator fieldbus boundary while the legacy controller drives the plant, the plant dynamics reflect the legacy controller actions rather than the candidate policy. If the candidate policy issues an unstable valve trajectory that would induce severe water hammer, the shadow log records only the mathematical command, while the physical pipe remains protected by the legacy controller.

Each qualification mode alters or severs different arcs in the causal loop, making it dangerous to treat these rungs as a cumulative staircase. A live shadow deployment cannot intentionally command a valve to slam shut against a 15.0 bar head to measure the resulting pressure surge, because doing so would destroy the production facility. Conversely, a software simulator cannot reveal a race condition in physical CAN transceiver silicon, and a hardware bench cannot execute the tens of millions of randomized parameter variations required to establish statistical confidence in edge-case handling. High-assurance qualification relies on treating these four modes as complementary evidence scopes, directing each fault injection test to the specific environment capable of exercising the relevant causal mechanisms (figure 30.4, table 30.2).



Closed-loop falsification: stress-testing the integrated S-P-A pipeline against microarchitectural and physical faults.

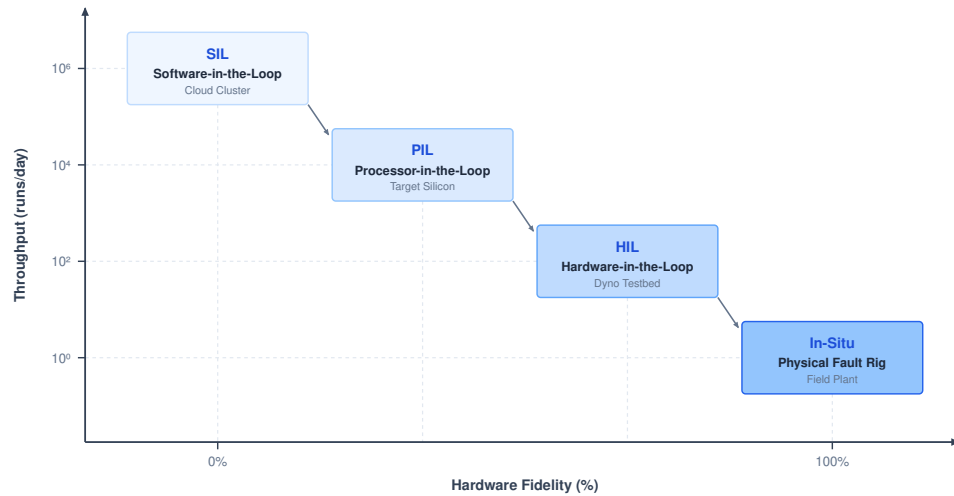


Figure 30.4: Four-stage qualification ladder: Four rungs trading throughput for fidelity, from roughly 10^6 software-only runs a day at zero hardware realism down to a handful of in-situ physical fault injections at full realism. The ladder is not a progression toward accuracy but a sequence of different blind spots, because each stage establishes evidence only within its own causal loop and stays blind to every mechanism outside it.

Table 30.2: The four-stage qualification ladder across cyber-physical boundaries: Systematic decomposition of qualification environments across computing substrates, exercised causal-loop elements, fault injection primitives, execution throughput, physical and temporal fidelity, and definitive test oracle mechanisms.

Qualification Rung & Substrate	Causal Loop Realism (Compute, Bus, Actuator, Plant)	Fault Injection Mechanism & Scope	Throughput & Sample Scaling	Physical & Temporal Fidelity	Primary Test Oracle & Evidentiary Limit
Software-in-the-Loop (SIL) (Host Workstation / Cloud Cluster)	Compute: Synthetic / Host x86; Bus: Idealized zero-copy buffer; Actuator: Linear / saturation model; Plant: Numerical ODE/PDE solver	Tensor bit-flips, synthetic sensor dropout, hydrodynamic parameter sweeps (μ, ρ, T)	10^3 – $10^5 \times$ real-time parallel sweeps ($> 10^8$ cycles/day)	High mathematical flexibility; zero target silicon, bus timing, or electrical realism	Oracle: Mathematical assertion corridors and state invariants. Limit: Cannot validate target execution timing or unmodeled physical dynamics.
Processor-in-the-Loop (PIL) (Target SoC / MCU Core Emulation)	Compute: Real target silicon (ARM/RISC-V); Bus: Simulated bus interconnect; Actuator: Emulated command register; Plant: Real-time host model	Instruction cache evictions, register corruption via JTAG/BIST, interrupt jitter injection	0.1 – $1.0 \times$ real-time; bound by cross-platform synchronization	Instruction-set and compiler accurate; abstracts analog bus lines and actuator back-EMF	Oracle: Target register state and WCET execution bounds. Limit: Ignores physical transceiver bus contention, voltage droop, and thermal rise.
Hardware-in-the-Loop (HIL) (Target ECUs + Real-Time Plant Rig)	Compute: Production ECU / NPU; Bus: Real physical CAN/Ethernet; Actuator: Real motor drives / dyno; Plant: Hard real-time FPGA plant rig	Pin break/short, CAN frame stuffing / babbling idiot, supply rail droop, load steps	$1.0 \times$ strict real-time (24 h/day wall-clock limit)	High electrical, timing, and protocol fidelity; bounded by FPGA plant model accuracy	Oracle: Protocol CRC, bus deadline monitors, and interlock timings. Limit: Cannot execute destructive physical overload or simulate micro-cavitation wear.

Qualification Rung & Substrate	Causal Loop Realism (Compute, Bus, Actuator, Plant)	Fault Injection Mechanism & Scope	Throughput & Sample Scaling	Physical & Temporal Fidelity	Primary Test Oracle & Evidentiary Limit
Physical Fault Rig & Shadow (Dynamometer Cell / Field Plant)	Compute: Production ECU / NPU; Bus: Production harness & sensors; Actuator: Real physical mechanisms; Plant: Real fluid / mechanical body	Pyrotechnic valve jammers, magnetic brake stall, thermal heating, transducer bias drift	$1.0 \times$ wall-clock; constrained by mechanical wear and thermal reset ($< 10^3$ cycles/day)	Full authentic physics (Coulomb friction, acoustic wave shock, thermal dissipation, wear)	Oracle: Independent physical containment instrumentation and yield transducers. Limit: High financial cost per trial; destructive tests cannot scale to statistical tail bounds.

To prevent false confidence, every reported verification result must explicitly state which elements of the causal loop were physically real, which were simulated, which were replayed from static logs, which were bypassed, and which remained unobserved. An analytical engineering claim that an intervention mechanism halted an overpressure event within 12 ms means nothing without specifying whether the pressure signal was generated by a differential equation, transmitted over a physical analog current loop, or replayed from a digital buffer. Specifying this causal mapping exposes the assumptions underlying each test, ensuring that unmodeled physics or artificial boundary conditions do not disguise latent hazards.

The fidelity boundary of a given test environment defines the exact threshold where synthetic models, emulated buses, or replayed sensor traces sever the continuous bidirectional energy and information exchange between compute and mechanics (section 2.2).

To quantify this reality gap, consider the divergence between rigid-body physics simulation and physical hardware when an articulated joint encounters unmodeled Coulomb friction and actuator lag (30.1).⁵

Example 30.1: Coulomb friction and stator lag reality gap

Consider a short-window manipulator example comparing an idealized SIL actuator with a physical joint. The calculation neglects viscous damping in both paths over 50 ms, assumes a constant $4.50 \text{ N} \cdot \text{m}$ breakaway/Coulomb threshold on hardware, and models applied torque as $\tau(t) = 6.00(1 - e^{-t/0.005}) \text{ N} \cdot \text{m}$. The 5.00 ms lag is a stipulated effective model, not a sum of independently proven electrical and sampling time constants. Both paths start at rest with inertia $J = 0.050 \text{ kg} \cdot \text{m}^2$:

1. In simulation, with zero Coulomb threshold, the full torque accelerates the inertia at $t = 0^+$, yielding $\ddot{\theta}_{\text{sim}}(0^+) = \frac{\tau_{\text{cmd}}}{J} = \frac{6.00 \text{ N} \cdot \text{m}}{0.050 \text{ kg} \cdot \text{m}^2} = 120 \text{ rad/s}^2$. On physical hardware, motion is impossible until electromagnetic torque overcomes τ_c . Once torque reaches the commanded $6.00 \text{ N} \cdot \text{m}$, net accelerating torque is reduced by Coulomb friction to $\tau_{\text{net}} = \tau_{\text{cmd}} - \tau_c = 1.50 \text{ N} \cdot \text{m}$, yielding steady acceleration of $\ddot{\theta}_{\text{real}} = \frac{1.50}{0.050} = 30 \text{ rad/s}^2$ —a fourfold reduction (4×) from simulation.
2. Motion initiates only when electromagnetic torque overcomes static stiction (τ_c). With first-order torque buildup $\tau(t) = \tau_{\text{cmd}}(1 - e^{-t/\tau_{\text{lag}}})$, solving $\tau(t_{\text{dead}}) = \tau_c$ yields the exact deadband duration:

$$t_{\text{dead}} = \tau_{\text{lag}} \ln \left(\frac{\tau_{\text{cmd}}}{\tau_{\text{cmd}} - \tau_c} \right) \quad (30.3)$$

Evaluating equation 30.3 gives $t_{\text{dead}} = 5.00 \text{ ms} \times \ln \left(\frac{6.00}{6.00 - 4.50} \right) = 5.00 \text{ ms} \times \ln(4.0) \approx 6.93 \text{ ms}$. During these initial 6.93 ms, the real joint remains stationary ($\theta_{\text{real}} = 0$), whereas the simulation claims the joint has already reached $\theta_{\text{sim}}(t_{\text{dead}}) \approx \frac{1}{2} \ddot{\theta}_{\text{sim}} t_{\text{dead}}^2 \approx 2.88 \times 10^{-3} \text{ rad}$ (0.165°).

Prerequisite: Microarchitectural stress patterns build directly on the memory contention benchmarks from section 26.3.

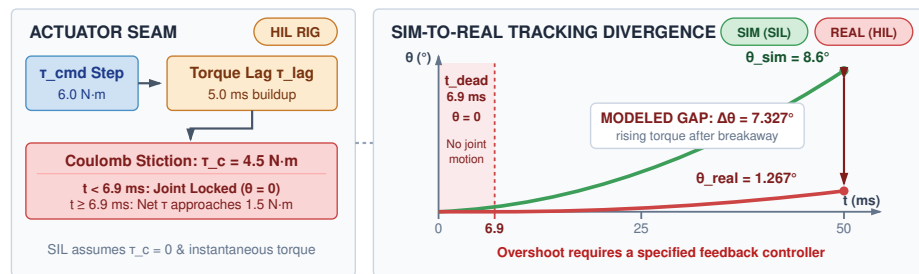
⁵ | **Actuator Stiction Deadband:** Under the stated first-order torque model and constant breakaway threshold, the joint remains still until $t_{\text{dead}} = \tau_{\text{lag}} \ln \left(\frac{\tau_{\text{cmd}}}{\tau_{\text{cmd}} - \tau_c} \right)$. This delay can reduce phase margin in a high-gain feedback loop; chattering or overshoot requires the loop gains, integral action, and load dynamics to be specified and tested.

3. At $t = 50.0$ ms, the idealized path reaches $\theta_{\text{sim}} = \frac{1}{2}(120)(0.050)^2 = 0.1500$ rad (8.594°). The physical path starts at $t_{\text{dead}} = 6.93$ ms, then accelerates according to $\ddot{\theta}(t) = [6(1 - e^{-t/0.005}) - 4.5]/0.050$, not immediately at its final 30 rad/s². Integrating this acceleration twice with zero position and velocity at breakaway gives $\theta_{\text{real}}(50 \text{ ms}) = 0.0221$ rad (1.267°), a gap of 0.1279 rad (7.327°). The constant- 30 rad/s² shortcut would overstate physical displacement because torque is still rising.

The SIL test misses the initial no-motion interval and overpredicts position at 50 ms. If the deployed controller also has integral feedback that accumulates error through the deadband without anti-windup, breakaway may produce additional overshoot. That separate controller behavior must be modeled and measured; the open-loop torque calculation alone does not establish it.



Checkpoint 30.2: Sim-to-real reality gap and actuation lag



Consider a robotic manipulator or articulated chassis transitioning from physics simulation to physical hardware:

- ☐ **Non-linear friction and dead-time latency:** Can you explain why omitting Coulomb breakaway friction in rigid-body simulation creates an unmodeled physical dead-time t_{dead} during actuator torque buildup?
- ☐ **Sim-to-real spatial divergence:** Can you describe how actuator stator lag and friction thresholds combine to induce tracking divergence between simulation and physical hardware, even when identical torque setpoints are applied?
- ☐ **Controller-dependent overshoot:** What integral-feedback and anti-windup assumptions would be needed before the modeled deadband could predict breakaway overshoot?

Once the test environment is defined and its causal boundaries are accounted for, the qualification process requires concrete mechanisms to evaluate whether the machine behavior during a fault injection constitutes a pass or a failure.

30.5 Fault Oracles and Acceptance

When a fault is injected into a running machine, the system will inevitably react, producing telemetry transients, actuator motions, and acoustic emissions. Without a pre-established oracle, a test is merely an anecdote. Engineers are left to subjectively evaluate time-series plots post-run, deciding if the machine survived based on catastrophic symptoms—whether Joule heating melted a phase wire, a component burst, or a controller threw an unhandled exception. An **oracle** is the mathematical predicate that evaluates a test execution and returns a binary verdict of pass or fail, defined in full before the execution begins.

In physical AI systems, evaluating whether a machine satisfied its safety claims requires moving beyond static scalar bounds to continuous temporal specifications. Physical systems do not fail

at single isolated instants; they fail through dynamic transients over time. To express and verify these dynamic properties rigorously, verification engineering uses **Signal Temporal Logic (STL)**. By combining simple physical constraints (like keeping pressure below a limit) with logical combinations (AND, OR, NOT) and strict time windows, STL can formally state that a condition must “always” hold during a specified interval, must “eventually” happen within a deadline, or must persist “until” another event occurs (for formal syntax, see Appendix A).

Crucially, STL equips verification with **quantitative robustness semantics**. Instead of a simple pass/fail flag, robustness is a continuous numerical score measuring *how far* the system remained from violating its safety margins.⁶

Consider a high-pressure manifold regulating coolant flow at 10.0 bar with a nominal throughput of 5.0 L/s. When a downstream valve jams under a 15.0 bar head, the physical safety requirement might state that line pressure must never exceed a 16.5 bar containment limit, and whenever a disturbance occurs, the bypass flow must exceed 3.5 L/s within 45 ms. If an injected fault causes the peak pressure to reach 17.2 bar, the robustness score evaluates to -0.7 bar (the difference between the limit and the peak), immediately quantifying the failure severity without manual inspection.

Quantitative semantics transforms whole-system verification from unguided random testing into **temporal logic falsification as adversarial optimization**. Because neural policies operate in high-dimensional state spaces—governed by SE(3) kinematics and gearbox compliance—uniform random sampling requires an intractable number of trials to find rare failures. Instead, automated falsification engines actively search for the worst-case physical disturbances—such as sensor noise, voltage drops, and friction—that drive the robustness score as low as possible.

Falsification engines employ global black-box optimization algorithms (such as covariance matrix adaptation evolution strategies or Bayesian optimization) to navigate messy, discontinuous cyber-physical landscapes. In simulation environments that support it, they can even use exact gradients computed directly through the physics models and neural network layers to steer the system toward failure faster. When the engine successfully drives the robustness score below zero, it produces a concrete counterexample trajectory, exposing the exact combination of disturbances and delays that breaks the machine.

Beyond black-box falsification, formal neural network verification methods—exemplified by the Reluplex SMT solver (Katz et al. 2017) and abstract interpretation frameworks—attempt to formally prove that piecewise linear networks satisfy state-space safety invariants (e.g., proving that for all states in an admissible set $\mathcal{X}$, the network output never intersects an unsafe control domain $\mathcal{U}_{\text{unsafe}}$ defined by the machine-specific hazard assessment, actuator torque limits, and applicable industrial robot requirements such as ISO 10218 / ISO/TS 15066). Formal SMT solvers face NP-complete complexity due to exponential state-space expansion with network depth and scale poorly beyond small feedforward networks or closed-loop dynamics. Consequently, physical AI verification cannot rely on formal proof alone; it must combine formal bounds with automated temporal logic falsification, empirical hardware fault injection, and out-of-band runtime enforcement.

Alongside the pass and fail criteria, automated verification requires an **invalid-test rule** that identifies when an experimental run must be discarded rather than scored. A test run is invalid whenever the execution environment fails to deliver the specified fault conditions or violates the required operational preconditions. If an upstream supply pump cavitates before the planned fault injection timestamp, lowering the baseline manifold pressure to 6.0 bar instead of the required $10.0 \text{ bar} \pm 0.2 \text{ bar}$, the test did not evaluate the safety envelope under high-head conditions. Similarly, if an injection framework injecting corrupted CAN frames suffers an internal queue overflow and drops the fault packet before it reaches the transceiver, the system under test was never subjected to the disturbance. Scoring an unexercised run as a pass manufactures false confidence, while scoring it as a failure penalizes the machine for an error in the test fixture. The invalid-test rule codifies the boundary conditions, rejecting the run and resetting the test sequence whenever preconditions fail.

Precommitment is the engineering discipline that separates verifiable evidence from post-hoc storytelling. When an assembled machine undergoes physical or electrical fault injection, the resulting behavior is almost always noisy, complex, and surprising. If acceptance criteria are negotiated post-observation, engineers rationalize anomalies. An unexpected pressure oscillation of 17.5 bar is explained away as an acceptable structural surge, or a 60 ms reaction delay is excused because the system eventually brought the flow back to steady state without tripping the mechanical

⁶ | **Quantitative Robustness Semantics:** Each predicate has a signed margin in its own physical unit: pressure may use bar, clearance meters, and jerk meters per second cubed. A negative margin falsifies that predicate. A minimum across unlike units is meaningless; a combined dimensionless STL score requires each margin to be divided by a predeclared positive scale of the same unit. The record retains raw margins and scales so a pass or failure remains physically interpretable.

burst disc. Post-hoc evaluation converts safety verification into a negotiation, where the definition of success drifts to match whatever the machine happened to do.

Precommitment fixes the STL predicates, physical units and any normalization scales, response-time bounds, and invalidation triggers before the first injection. The evaluator retains a signed margin for each predicate and returns PASS only if every required margin and timing bound passes. If pressure exceeds its limit by 0.1 bar or response misses its deadline by 2 ms, the record states which measured predicate failed; the trace and follow-up analysis determine whether control margin, actuator lag, derating, or a diagnostic delay caused it.

Establishing precommitted oracles and quantitative temporal logic boundaries transforms fault injection from a qualitative demonstration into a quantitative measurement of system margins. Yet the validity of that measurement remains entirely dependent on whether the injected disturbances and the monitored reactions interact through real physical dynamics or through mathematical approximations. Defining the exact boundaries of a valid test and the criteria for success prepares the engineering team to confront the physical machine itself, where synthetic models give way to the unyielding mechanics of real hardware.



Checkpoint 30.3: Temporal logic oracles and test precommitment

- ☐ **Quantitative robustness interpretation:** Can you explain how Signal Temporal Logic replaces binary pass/fail assertions with a signed distance metric, and how a negative score quantifies physical failure depth?
- ☐ **Adversarial optimization of disturbance trajectories:** Can you describe how black-box optimizers leverage quantitative robustness scores to discover worst-case combinations of latency, noise, and friction?
- ☐ **Test invalidation versus failure criteria:** Can you distinguish between an execution that falsifies a safety invariant and an invalid trial where testbed preconditions or fault injection mechanisms failed?

30.6 Fault Injection on Hardware

A fault injected in simulation tests the simulator's response. When a disturbance is synthesized inside a mathematical model, the software reacts against the numerical assumptions that the modeler wrote down. Digital simulations assume ideal clock synchronization, perfectly isolated memory addresses, deterministic context switches, and linear communication delays across shared interconnects. In the physical machine, faults do not execute inside isolated functions. They propagate across physical silicon, where memory access contention on a shared system bus, clock drift between asynchronous microcontrollers, direct memory access stalls, and voltage drops across inductive actuator drivers dictate whether an intervention executes in time. Fault injection must occur directly on target hardware during production handoff, driving disturbances through physical transceivers, timers, shared buses, and the hardware enforcement path, while an independent containment rig protects the test facility.

In-situ fault injection is the deliberate, deterministic introduction of physical, electrical, and computational disturbances (including sensor register latches, clock drift, bus contention,⁷ supply brownouts, and actuator load stalls) directly into target production silicon and mechanics while operating under authentic electrical, thermal, and dynamic loads.

Hardware-in-the-loop (HIL) facilities can bridge part of this reality gap by connecting representative control hardware through physical harnesses and power distribution to a real-time plant simulator and fault-injection interfaces. The photographed ring (figure 30.5) shows hardware placement and cabling; its timing and fault-injection capability must be established from a facility specification or test record.

⁷ | Silent Data Corruption from Single-Event Upsets:

High-density neural accelerators manufactured on sub-7 nm FinFET processes have critical node charges (Q_{crit}) so low that atmospheric neutron strikes and alpha particles flip register bits. In legacy planar CMOS, single-bit upsets (SBUs) were effectively mitigated by Single-Error Correction Double-Error Detection (SECCDED) Hamming codes. However, as physical cell pitch contracts below 30 nm in sub-7 nm FinFET and GAA architectures, a single ionizing particle strike deposits a charge cloud spanning multiple adjacent bit cells, producing **Multi-Bit Upsets (MBUs)**. When two or more bits within the same code word flip simultaneously, standard SECCDED ECC either detects an uncorrectable fault and halts execution or suffers silent bit aliasing. In consumer-grade accelerators lacking physical bit interleaving (spreading logically adjacent code word bits across physically separated memory columns) and dual-core lockstep (DCLS) logic with temporal staggering, soft errors silently corrupt policy tensors without triggering hardware exceptions. In-situ fault injection frameworks must inject transient single-bit and multi-bit flips directly into accelerator memory spaces to verify that downstream kinematic shields detect uncommanded torque jumps before mechanical yield occurs.



Figure 30.5: Avionics integration ring: The photograph shows a circular assembly of labeled electronic boxes and cable harnesses in a laboratory. The visible hardware motivates tests of electrical and timing behavior, but the image alone does not establish loop rate, fallback deadline, or fault-injection capability. Source: NASA Marshall Space Flight Center.

🔍 Systems Perspective 30.2: Testbed instrumentation architecture

Physical testbed verification requires dedicated bench instrumentation engineered to intercept, corrupt, and observe signals without altering circuit impedance or timing characteristics. A fully instrumented physical AI verification testbed integrates five specialized hardware subsystems:

1. **High-Bandwidth Digital Storage Oscilloscopes (DSOs) and Logic Analyzers:** Synchronized across common 10 MHz reference clocks to capture microsecond-level transitions across SPI, CAN, and gate-driver PWM lines.
2. **Inline Breakout Injection Boxes:** Equipped with solid-state Field-Effect Transistor (FET) switches capable of sub-microsecond pin-level open-circuit disconnections, short-to-ground (simulating a chassis ground fault), and short-to-power (simulating cross-rail short circuits).
3. **Programmable Fieldbus Disturbance Nodes:** Dedicated microcontrollers capable of injecting bit-level CAN frame stuffing, cyclic redundancy check (CRC) corruptions, and unthrottled babbling-idiot message floods (where a malfunctioning node continuously transmits unthrottled frames, choking the network).
4. **Precision Programmable DC Power Supplies and Electronic Loads:** Capable of synthesizing fast supply rail droops ($V_{\text{core}} < 0.85 \text{ V}$), ground bounce (momentary reference plane potential shifts), and peak current surges during rotor startup under dynamic motor loading.
5. **External Truth Transducers:** Laser Doppler vibrometers, optical shaft encoders, and high-speed photogrammetric cameras wired to isolated digitizers, recording ground-truth physical motion independent of the embedded controllers under test.

Consider a **separate, sealed, liquid-filled** 0.10 m^3 manifold upstream of the vented buffer tank. Under the illustrative rigid-volume model, bulk modulus $K = 1.6 \times 10^9 \text{ Pa}$, density $\rho = 1000 \text{ kg/m}^3$, and a 0.50 kg/s flow imbalance give $\dot{p} = K\dot{m}/(\rho V) = 80 \text{ bar/s}$. A 0.8 bar pressure allowance is

8 | Silent Data Corruption from Single-Event Upsets:

High-density neural accelerators manufactured on sub-7 nm FinFET processes have critical node charges (Q_{crit}) so low that atmospheric neutron strikes and alpha particles flip register bits. In legacy planar CMOS, single-bit upsets (SBUs) were effectively mitigated by Single-Error Correction Double-Error Detection (SECCDED) Hamming codes. However, as physical cell pitch contracts below 30 nm in sub-7 nm FinFET and GAA architectures, a single ionizing particle strike deposits a charge cloud spanning multiple adjacent bit cells, producing **Multi-Bit Upsets (MBUs)**. When two or more bits within the same code word flip simultaneously, standard SECCDED ECC either detects an uncorrectable fault and halts execution or suffers silent bit aliasing. In consumer-grade accelerators lacking physical bit interleaving (spreading logically adjacent code word bits across physically separated memory columns) and dual-core lockstep (DCLS) logic with temporal staggering, soft errors silently corrupt policy tensors without triggering hardware exceptions. In-situ fault injection frameworks must inject transient single-bit and multi-bit flips directly into accelerator memory spaces to verify that downstream kinematic shields detect uncommanded torque jumps before mechanical yield occurs.

9 | Non-Intrusive Hardware Tracing:

Non-intrusive tracing utilizes dedicated silicon hardware debug blocks (such as ARM CoreSight Embedded Trace Macrocell) and external FPGA bus sniffers that snoop internal bus transactions without stealing CPU execution cycles. By buffering execution traces in dedicated high-speed SRAM decoupled from the system memory bus, hardware analyzers capture cycle-accurate fault-reaction timestamps without perturbing real-time thread schedules. Relying on software logging daemons introduces Heisenberg observer effects, where logging overhead masks the exact race conditions under test.

consumed in 10 ms if that rate persists. Neither a valve-stem displacement nor the vented tank alone establishes this pressure rise. The hardware test holds an ADC value stale while preserving its original acquisition timestamp, then measures detector assertion and pressure on independent instruments. A measured P_{99} detection time of 4 ms is distributional evidence; a containment claim needs a justified bound over the declared manifold, bus-load, and actuator envelope.

Testing the heterogeneous Dual-Brain architecture (section 6.1) demands physical fault injection directly at their inter-processor boundary:

- **Stale Observation Injection:** An inline FPGA interceptor latches an ADC register or suppresses CAN frame updates while preserving the original hardware timestamp. This evaluates whether the MCU's temporal freshness monitor detects the frozen observation within its $\Delta t_{det} \leq 4.0$ ms budget before stale state estimates degrade actuator commands.
- **Acoustic Transducer Resonance Spoofing:** An ultrasonic acoustic emitter targets the micro-machined resonant frequency (20–35 kHz) of MEMS gyroscopes. The acoustic pressure waves excite physical oscillations in the micro-silicon capacitive combs, overriding Coriolis deflection and saturating the analog front-end. The IMU reports false zero-rotation or rails at maximum angular velocity. The test verifies that the sensor fusion estimator detects cross-modal discrepancy against wheel odometry or optical flow within its 10 ms budget, rejecting the corrupted IMU channel before attitude control destabilizes.
- **Sub-7nm Accelerator Soft Error (SEU) Injection:** Direct register injection or scan-chain tools flip bits in the neural accelerator's activation SRAM or weight caches, replicating single-event upsets caused by atmospheric cosmic neutron strikes in sub-7 nm FinFET and Gate-All-Around (GAA) silicon.⁸ A single bit flip in a float16 exponent converts a minor path correction into a catastrophic full-scale torque setpoint. The test verifies that the downstream Layer 2 safety shield intercepts and clamps the uncommanded torque impulse within a single 1.0 ms control period before motor stator current can stress mechanical gear teeth.
- **Late Plan Injection:** The injection testbed delays neural trajectory proposals across the MPU-to-MCU shared memory bridge. If a proposal scheduled for execution at $t = 100$ ms arrives at $t = 106$ ms, the test verifies that the MCU's deterministic scheduler instantly refuses the late submission, clamps the actuator to a precommitted safe hold position, and transitions the loop to deterministic fallback before physical vessel margins expire.
- **Expired Intent Lease Revocation:** The test framework suppresses a renewal packet. At the illustrative $t = 20.0$ ms expiry in figure 30.6, the MCU revokes policy proposal authority and runs its resident, previously admitted fallback; the separate stale-sensor detector asserts at 24.0 ms and the drive override is confirmed at 26.0 ms. The test measures each event on its own wire and clock.

Recording the resulting state trajectory, the detector identity, the exact refusal timestamp, and the time required to restore the bounded state requires instrumentation that is electrically and logically independent of the machine under test. If the test engineer relies on the onboard logging daemon to measure reaction times, the measurement is corrupted by the very failure it attempts to capture, because a faulted CPU or a saturated bus delays log serialization.⁹ Furthermore, independent measurement prevents the acceptance of false passes where the machine survived for reasons absent from the safety claim. The Apollo 11 landing alarms illustrate a different overload path: radar-related counter activity consumed computer time, the Executive ran out of work areas, and priority restart resumed essential jobs.¹⁰ An even simpler failure mode on modern test benches occurs when an injected overpressure fault triggers an unmonitored thermal breaker or an auxiliary ground loop that drops power to the valve drive coil. The valve springs shut mechanically, keeping pressure inside the containment bound, while the software enforcer has actually deadlocked in a priority inversion.¹¹ An automated oracle that only inspects downstream pressure registers a pass, manufacturing false confidence from an unverified hardware artifact. The test framework must verify that the enforcer path in the safety claim executed the corrective action.

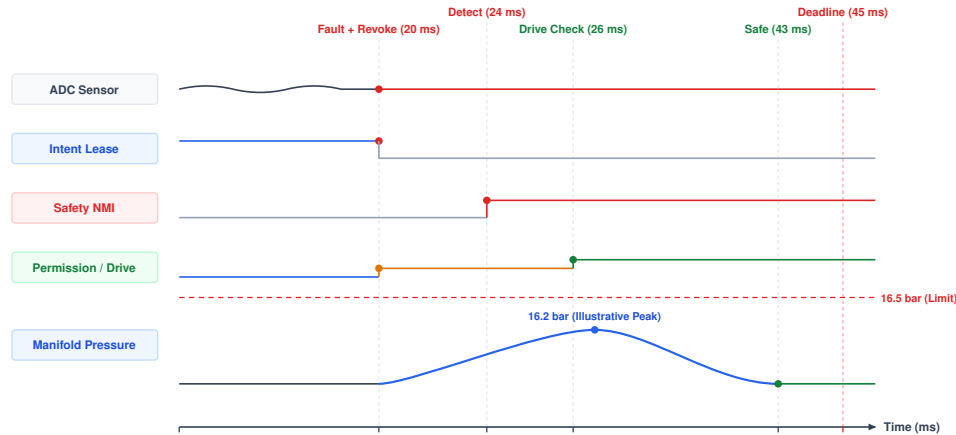


Figure 30.6: Illustrative fault-injection timeline: At 20 ms, lease expiry revokes policy proposal permission and the MCU enters its resident fallback. A separate state-sensor fault asserts a diagnostic NMI at 24 ms; the drive override is confirmed at 26 ms, two milliseconds later. Thus fault-to-detection is 4 ms, NMI-to-drive confirmation is 2 ms, and fault-to-bounded state is 23 ms at 43 ms. The 16.2-bar pressure peak under a 16.5-bar scenario limit is a constructed acceptance trace pending plant and sensor validation.



War Story 30.1: Testing a task-blind watchdog hypothesis

Context: The Toyota ETCS-i litigation raised questions about whether a throttle-control task could fail while a timer-tick interrupt continued to service the watchdog (Barr 2013).

Mechanism to test: Plaintiff expert Michael Barr proposed a task-death path associated with memory corruption. The test vector kills the control task while leaving the timer interrupt active, then independently measures throttle and watchdog outputs. The NHTSA/NASA investigation did not establish an electronic cause of unintended acceleration, so this is an attributed hypothesis rather than a universal incident explanation.

Oracle: A running watchdog is insufficient evidence if the safety claim requires timely task-specific control. Record whether the throttle returns to a validated state and whether the brake response meets the vehicle's tested limit. The result depends on the implemented monitor and plant; initial brake application can overcome full engine torque in the NHTSA tests.

Systems lesson: Inject the failure mode that the liveness signal might miss, then measure the response. Memory protection, independent supervision, and brake-override designs are options whose coverage must be demonstrated for the target vehicle; no universal hardware gate-driver cutoff mandate follows from this case.



Systems Perspective 30.3: The watchdog liveness invariant

A watchdog fed by a timer tick shows that its feed path ran, not that every critical task met its deadline. A qualified design identifies the task progress and physical response signals needed for its fault model and tests those signals under task death and stalled-output injections.

The intervention mechanism must be subjected to fault injection under the exact operational conditions that make intervention necessary. A safety enforcer that takes over cleanly when the system sits idle on a test bench provides no evidence of safety in production. During an operational crisis, the learned policy is typically running at maximum computational throughput, memory buses are saturated with high-resolution sensory streams, direct memory access channels are transferring frame buffers, and the actuator is experiencing maximum hydrodynamic load. The test framework injects faults under the declared thermal and electrical load envelope and measures both lease-

¹⁰ | **Apollo 11 AGC Executive Priority:** The NASA mission report attributes excess computer load to rendezvous-radar-related counter interrupts, consuming up to about 15 percent of AGC time. The overloaded Executive issued 1201/1202 restart alarms; the NASA engineer account describes selective resumption of essential steering and display jobs. These sources do not establish a missed throttle deadline or a simple memory-bus theft mechanism.

¹¹ | **Toyota watchdog hypothesis:** In *Bookout v. Toyota*, plaintiff expert Michael Barr proposed that task death could leave an unwanted throttle command while a timer-tick interrupt continued servicing the watchdog. This is an expert failure hypothesis, not an established cause of all unintended-acceleration events; the NHTSA/NASA investigation did not find an electronic cause of unintended acceleration. The test-design lesson is to inject loss of each critical task and observe the physical response rather than infer coverage from a running CPU.

expiry revocation and the separate diagnostic-to-drive response. In figure 30.6, the constructed diagnostic-to-drive interval is 2.0 ms; a measured P_{99} alone would not establish a hard response bound. If the intervention path shares an internal bus arbiter with the neural accelerator, the hardware takeover can stall behind an un-preemptible burst transfer. A machine whose safety case assumes instantaneous takeover will fail in the field if that takeover was only ever verified on a quiet bus.

The complete deterministic protocol for conducting automated hardware-in-the-loop fault injection, verifying real-time detection invariants, and evaluating quantitative Signal Temporal Logic robustness on target silicon is formalized below:

Substrate & Invariants:

- *Execution Substrate:* Hard real-time FPGA plant rig and testbed orchestrator ($f = 1000$ Hz, hardware timestamping jitter < 500 ns).
- *Memory Invariant:* Pre-allocated static DMA buffers in host test orchestrator; zero runtime dynamic heap allocation on safety MCU.
- *Safety Interlock Authority:* Independent optical/laser truth sensors wired to hardware crowbar relays capable of cutting target actuator power within 2.0 ms upon physical envelope breach.

Execution protocol:

1. **Pre-Execution Invariant & Envelope Precondition Audit (Stage 1):** Sample baseline environmental sensors via $\mathcal{J}_{\text{truth}}$ (temperature, rail voltage, and fluid pressure). Compare each measurement with its declared same-unit tolerance; do not take one norm across kelvin, volts, and pascals. If any precondition fails, log `ERR_PRECONDITION_OUT_OF_SPEC`, set Verdict $\leftarrow$ `INVALID_PRECONDITION`, and abort.
2. **Baseline Closed-Loop Synchronization & Steady-State Arming (Stage 2):** Arm FPGA plant simulator and power up target Dual-Brain system-on-chip (SoC); initiate periodic control loop (1000 Hz). Establish synchronized reference clock across logic analyzer, FPGA plant, and truth digitizers via 10 MHz PXI clock. Allow physical plant state $\mathbf{x}(t)$ to stabilize in nominal operating envelope for $t_{\text{settle}} = 2.000$ s. If baseline tracking error $\|e_{\text{track}}\| > \epsilon_{\text{nom}}$, assert `ERR_BENCH_INSTABILITY`, set Verdict $\leftarrow$ `INVALID_PRECONDITION`, and abort.
3. **Deterministic In-Situ Fault Injection Trigger (Stage 3):** At monotonic test epoch t_{inject} , hardware test orchestrator asserts trigger line to inline injection rig:
 - *Sensor Latency/Bias:* FPGA interceptor latches ADC register or holds stale CAN payload while preserving valid hardware timestamp.
 - *Acoustic Sensor Resonance:* Ultrasonic transducer pulses resonant acoustic frequency (25 kHz) directly at MEMS IMU die.
 - *Accelerator Single-Event Upset:* Scan-chain fixture injects pseudo-random bit flip into neural accelerator activation SRAM.
 - *Bus Contention / Frame Stuffing:* Fieldbus disturbance node floods CAN bus with dominant identifier bits (98 percent bus load).
 - *Power Rail Droop:* Programmable electronic load pulls V_{core} down from 1.10 V to 0.82 V for $\Delta t_{\text{duration}} = 5.0$ ms.
 - *Actuator Mechanical Stall:* 4-quadrant dynamometer applies step counter-torque $\tau_{\text{stall}} = 3.5 \times \tau_{\text{rated}}$. Record microsecond injection timestamp $t_{\text{fault_start}} \leftarrow t_{\text{now}}$.
4. **Real-Time Safety Enforcer Response & Takeover Audit (Stage 4):** Monitor target MCU safety enforcer pins, CAN bus telemetry, and gate-driver PWM lines via isolated logic analyzer:
 - Detect assertion of Non-Maskable Interrupt (NMI) / diagnostic flag: $t_{\text{det}} \leftarrow t_{\text{now}} \implies \Delta t_{\text{det}} = t_{\text{det}} - t_{\text{fault_start}}$.
 - Detect hardware multiplexer preemption of motor driver: $t_{\text{takeover}} \leftarrow t_{\text{now}} \implies \Delta t_{\text{takeover}} = t_{\text{takeover}} - t_{\text{fault_start}}$.
 - Detect physical plant transition to safe bounded regime: $t_{\text{bounded}} \leftarrow t_{\text{now}} \implies \Delta t_{\text{bounded}} = t_{\text{bounded}} - t_{\text{fault_start}}$. If external containment rig trips due to structural envelope breach: hardware crowbar cuts actuator power; assert `ERR_PHYSICAL_BREACH`.

Downstream: Hardware-in-the-loop stress results furnish the evidentiary warrants for safety cases in section 32.3.

5. **Quantitative STL Robustness Evaluation & Ledger Serialization (Stage 5):** Ingest truth trajectory $x_{\text{obs}}(t)$ from independent instrumentation over the declared trial window. If required truth frames are missing, set $\text{Verdict} \leftarrow \text{UNOBSERVABLE_TELEMETRY}$ and retain the partial trace. For an observable trial, compute each precommitted predicate's signed margin in its own unit (see Appendix A). Return PASS only when every required margin is nonnegative and both measured fault-to-detection and fault-to-drive-takeover times satisfy their declared bounds; otherwise return FAIL. A dimensionless aggregate may be computed only after predeclared per-predicate normalization. Link the fixed `FaultExecutionRecord_t` header to the variable-length evidence bundle by digest, sign the header under the specified custody scheme, and retain both.

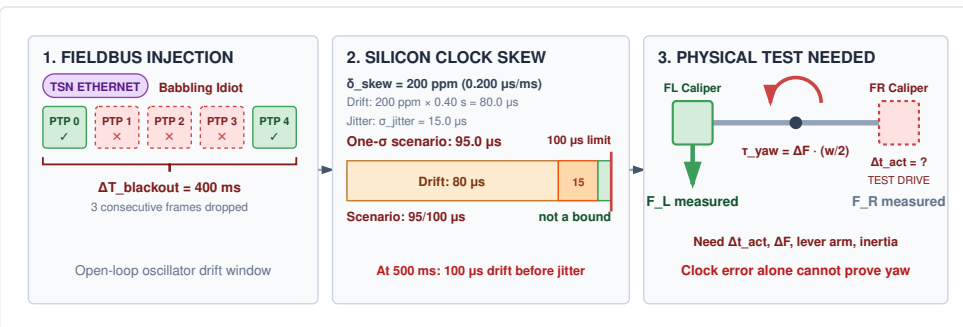
Testing distributed loops exposes clock-conversion risk even when each processor meets its local schedule. Consider an MPU and an MCU with stipulated oscillator limits of ± 100 and ± 50 ppm. Assume an additional 50 ppm thermal contribution is independently bounded and not already included in those specifications. The worst relative rate in this constructed case is 200 ppm = 200 $\mu\text{s/s}$. A 500 μs clock-phase allowance would be consumed by drift alone after 2.50 s without resynchronization. A healthy 100 ms sync period limits that drift contribution to 20 μs ; network and conversion error must be budgeted separately.

Suppose good sync frames occur at $t = 0$ and $t = 400$ ms, while the three scheduled frames at 100, 200, and 300 ms are dropped. The blackout lasts four sync periods. The bounded drift contribution is 200 $\mu\text{s/s} \times 0.40 \text{ s} = 80 \mu\text{s}$. If the observed synchronization jitter has standard deviation $\sigma_{\text{jitter}} = 15 \mu\text{s}$, adding one sigma produces only the illustrative value

$$\Delta t_{\text{one-sigma scenario}} = 80 \mu\text{s} + 15 \mu\text{s} = 95 \mu\text{s} \quad (30.4)$$

The one-sigma scenario in equation 30.4 is not a peak or hard bound: a separate justified jitter envelope and clock-conversion test are needed for a deterministic deadline. A fifth missed period gives 100 μs of drift alone, before jitter. If a caliper controller requires front/rear command alignment within 100 μs , the timebase could consume most of that command-timing budget in this scenario. Whether command skew becomes actuation skew, asymmetric force, or yaw requires measured drive and hydraulic response, lever arms, braking forces, and vehicle yaw inertia. Clock phase error by itself does not establish a spin.

Checkpoint 30.4: HIL clock drift and sensor jitter budget



- ☐ **Distributed clock drift accumulation:** Can you explain why independent crystal oscillators subject to thermal gradients diverge linearly over time, and how an unmitigated skew rate consumes state estimation phase margins?
- ☐ **Network packet drop and timing blackouts:** Can you describe the mechanism by which network congestion or adversarial packet flooding transforms periodic PTP synchronization into an open-loop drift window?
- ☐ **Clock-to-actuator bridge:** What measured scheduling, valve or caliper response, force, and inertia evidence would be needed before a timebase discrepancy could support a physical yaw claim?

30.7 The Fault Manifest

The **fault record** links a small fixed header to a retained evidence bundle by digest, following section 8.9. Together they identify the claim, operating point, injection, hardware and software builds, raw trace, detector, response timing, physical outcome, and anomalies. For flow-regulator claim C-04, a record could specify a 24 V valve-driver override for 120 ms at 350 kPa and 45°C, with firmware 3.4.1 on board B. Its bundle would name the pressure-gradient detector, measured 1.8 ms response, peak pressure below 380 kPa, and observed thermal, breaker, and bus status. Table 30.3 defines the fixed portion; the variable trace stays in the bundle.

The verdicts must remain distinct. A measured containment violation is **FAIL**. A breaker that prevents the requested injection makes the trial **INVALID**; lost truth frames make it **UNOBSERVABLE**, with the partial trace retained. Neither may be counted as a pass or silently dropped from coverage.

Table 30.3: Fixed fault-record header: The thirteen fields total 177 bytes packed in this illustrative layout. The linked evidence bundle carries fault duration, hardware and software build identifiers, raw trace references and clock domains, detector identity, every physical-margin value and scale, measured physical outcome, secondary anomalies, and partial traces for non-pass verdicts. The header and bundle together form the reproducible record; neither alone proves an unmeasured physical effect.

Fixed-header field	Packed bytes	Purpose and interpretation
<code>manifest_id</code>	8	Unique trial identifier in the versioned ledger.
<code>claim_ref_id</code>	4	Index of the precommitted physical claim and its units.
<code>envelope_point</code>	16	Four declared <code>float32</code> values for this flow-regulator example: pressure (Pa), temperature (K), viscosity (Pa·s), speed (m/s); another plant needs a versioned field definition.
<code>fault_injection_mode</code>	2	Injected fault class.
<code>fault_magnitude</code>	4	<code>float32</code> amplitude; its unit is fixed by the fault-class manifest.
<code>injection_target_node</code>	2	Hardware boundary receiving the injection.
<code>stl_spec_hash</code>	32	Digest of the immutable predicate and oracle configuration.
<code>measured_t_det</code>	4	Microseconds from fault trigger to detector assertion.
<code>measured_t_takeover</code>	4	Microseconds from fault trigger to measured gate takeover; subtract detection time for the post-detection interval.
<code>stl_robustness_rho</code>	4	One <code>float32</code> raw signed margin for <code>claim_ref_id</code> , in that claim's declared unit. Other predicate margins remain in the bundle.
<code>verdict_status</code>	1	PASS, FAIL, INVALID, or UNOBSERVABLE.
<code>evidence_bundle_hash</code>	32	Digest of the separately retained trial evidence bundle.
<code>provenance_sig</code>	64	Signature over canonical header fields excluding this signature, including the bundle digest; algorithm and key ID are in the versioned manifest.

For a trial aborted before injection, detector time, takeover time, and robustness margin are unavailable rather than zero. This illustrative wire format stores `0xFFFFFFFF` in an unavailable `uint32` time slot and a canonical quiet NaN in an unavailable `float32` margin slot; the signed evidence bundle records a validity flag for each measured field and why it is absent. The oracle checks those flags and the verdict before doing arithmetic, and excludes unavailable slots from pass counts and latency or margin statistics. The versioned manifest fixes the NaN bit pattern and byte order so hashing remains reproducible.

Reporting verification progress as a single aggregate percentage obscures critical engineering vulnerabilities. A safety case cannot rely on a summary metric stating that a machine survived 98 percent of five thousand fault trials if all five thousand injections were conducted at room temperature on a quiescent memory bus. Coverage must instead be reported across separate operational dimensions, including functional safety requirements, physical envelope boundaries, physical and logical injection sites, timing tails, hardware build revisions, and multi-fault combinations. For a flow regulator, boundary coverage tracks whether faults were injected at maximum fluid viscosity, minimum back-pressure, and maximum pump ripple. Timing tail coverage isolates injections targeting the P_{99} scheduling latency of the nervous system and peak direct memory access utilization. A machine achieves acceptable verification not when an average coverage index exceeds an arbi-

trary threshold, but when the minimum coverage across every individual dimension satisfies the requirements of the safety case.¹²

Interpreting repeated trials requires distinguishing between deterministic execution and stochastic sampling. Repeating the exact same deterministic fault script five hundred times on a digital test bench yields no more information than running it once. If initial conditions, processor register states, clock dividers, and input sequences are fixed, identical deterministic software executes the identical branch sequence every time. Repetition adds evidentiary value only when sampling physical and computational distributions that exhibit stochastic variance. In a physical flow loop, repeated trials sample turbulent velocity fluctuations, asynchronous interrupt arrivals, bus arbitration contention, and phase jitter between the high-frequency sensor clock and the policy inference cycle. In these stochastic regimes, repetition measures the spread and tail of the response-time distribution rather than re-confirming a deterministic path.

When n independent Bernoulli trials sampled from the declared fault distribution have zero missed mitigations, the result sets a one-sided upper confidence limit on the miss probability for that distribution.¹³ To place that upper limit at most $p_* = 1 - C_*$ with confidence $1 - \alpha$, the exact count and useful small- p_* approximation are:

$$N_{\text{trials}} = \left\lceil \frac{\ln \alpha}{\ln(1 - p_*)} \right\rceil \approx \frac{-\ln \alpha}{p_*}. \quad (30.5)$$

Applying equation 30.5 at 95 percent confidence, $p_* = 0.01$ requires 299 zero-miss trials; $p_* = 0.001$ requires 2,995. These are limits on the sampled fault population, not proof that every fault mode is covered. To infer a FIT-weighted hardware coverage, injections must represent the raw-fault weights λ_i or use a justified stratified weighting and per-stratum analysis.¹⁴

The bound applies only when the injection outcomes meet the declared independence and sampling assumptions. Repeated solenoid injections can heat a coil ($I^2 R$), change resistance, and couple later trials to earlier ones. A campaign at steady laminar flow and 20°C cannot establish a limit for cavitation or a -10°C cold start. A zero-miss count is a confidence limit under its test distribution, not a per-tick safety guarantee.



Napkin Math 30.1: Fault coverage and exposure under declared assumptions

Scenario: Assume component fault rates sum to $\lambda_{\text{raw}} = 9,800$ FIT, where one FIT is one fault per 10^9 operating hours. The four illustrative contributions are sensors 1,500, compute 1,000, transceivers and power management 2,500, and gate drivers 4,800 FIT. Let C_{diag} be the fraction of these faults whose hazardous effect is successfully mitigated, weighted by the applicable fault rates. If the simplified partition is valid,

$$\lambda_{\text{unmitigated}} = \lambda_{\text{raw}}(1 - C_{\text{diag}}).$$

A stipulated 100 FIT target needs $C_{\text{diag}} \geq 1 - 100/9800 = 98.9796\%$; a 10 FIT target needs $C_{\text{diag}} \geq 99.8980\%$. These are engineering requirements for the assumed fault model, not measured coverage or thresholds assigned by a safety-integrity label.

For a fault population sampled in proportion to its λ_i weights, zero missed mitigations in 299 independent injections puts the one-sided 95 percent miss-probability limit at 1 percent; 2,995 are needed for 0.1 percent. Uniform injections across unequal-rate components estimate a different distribution unless outcomes are weighted and each stratum has adequate evidence. The counting result does not itself verify fault rates, common-cause failures, detector latency, or physical containment. A test must record which mode was injected, whether it was detected, and whether the plant actually reached its allowed state.

For operating exposure, make a separate constant-rate assumption: if incidents occur as a Poisson process with rate λ , zero observed events over T independent, representative operating hours gives the 95 percent upper limit $\lambda_U = -\ln(0.05)/T$. To put that limit at a selected 10^{-9} h^{-1} target requires about 3.0×10^9 event-free aggregate hours. With 100 units operating continuously, that is roughly 3,420 years before maintenance or distribution shifts are considered. This is impractical for the

¹² | **Scope of assurance standards:** ISO 26262-5 addresses road-vehicle electrical and electronic hardware. Its single-point and latent fault metrics concern the architecture and its allocated safety goals; they are not this chapter's single aggregate C_{diag} . Fault injection is one way to test diagnostic assumptions, alongside analysis and other evidence. ANSI/UL 4600 frames an autonomous-vehicle safety case from multiple kinds of evidence. Neither standard makes a fixed number of these illustrative HIL trials sufficient for release.

Contrast: Simulated fault injection contrasts with real-time fieldbus bus-off recovery dynamics in section 8.4.

¹³ | **Clopper-Pearson Exact Confidence Intervals:** For zero failures in n independent Bernoulli trials, the exact upper Clopper-Pearson bound at confidence level $1 - \alpha$ is $p_{\text{upper}} = 1 - \alpha^{1/n}$. Unlike Gaussian approximations (which collapse to zero width at $k = 0$), the Clopper-Pearson bound correctly accounts for sampling uncertainty in extreme-reliability regimes.

¹⁴ | **Trial sampling and aging:** The binomial count assumes independent, identically distributed Bernoulli outcomes under the declared injection distribution; it does not require Poisson event arrivals. Repeated heating, wear, or contact changes can break independence and require a state-stratified test design. A separate Poisson or constant-rate model is used below only to illustrate zero-event operating-hour exposure.

illustrated program, not a mathematical impossibility. As Butler and Finelli (1993) explain, such exposure demands call for an argument built from more than black-box operating hours. DO-178C is software development assurance, while aircraft catastrophic-condition probability objectives are separate system safety targets; ISO 26262 applies to road-vehicle electrical and electronic hazards.

Table 30.4: Illustrative zero-event exposure under a constant-rate model: $T = -\ln(0.05)/\lambda$ gives a one-sided 95 percent event-rate upper limit after zero observed events in representative independent exposure. Rows are chosen hazard-rate targets, not SIL, ASIL, or DAL assignments. Fleet durations assume 100 units operate continuously and share the same target population; they do not account for aging, maintenance, or correlated failures.

Illustrative target rate	95% zero-event hours	Duration for 100 continuous units	Interpretation
10^{-3} h^{-1}	3.0×10^3	30 h	Representative exposure may be practical for this selected target.
10^{-6} h^{-1}	3.0×10^6	3.4 years	Distribution and aging assumptions become consequential.
10^{-8} h^{-1}	3.0×10^8	342 years	Exposure alone is impractical in the assumed fleet.
10^{-9} h^{-1}	3.0×10^9	3,420 years	Other analysis and targeted evidence must carry the argument.

The exposure targets in table 30.4 are assumptions, not assurance levels. A defensible safety case requires that untested operational envelopes, unobservable test results, and unsupported probability claims be presented with the exact same visual and structural prominence as successful verifications. Hiding an untested envelope boundary or an unverified hardware permutation in an appendix manufactures a false impression of completeness. The fault record must explicitly list every parameter combination where fault injection was omitted due to test bench limitations, sensor bandwidth constraints, or destructive hardware limits. Documenting what was never tested prevents unverified operational zones from silently masquerading as verified safety margins when the record is transferred to the safety case in chapter 32. Furthermore, every isolated entry in the record documents the response of the machine to a single, localized disturbance. In physical operation, however, disturbances rarely occur in strict isolation, and verifying how a machine responds to an individual fault leaves open the question of how it behaves when multiple coupled physical disturbances (e.g., stator delays, gearbox compliance, and bus jitter) strike the system at the same time.



Checkpoint 30.5: Fault manifests and statistical qualification limits

- ☐ **Fault manifest evidence chain:** Can you trace a claim, injected disturbance, detector response, plant outcome, and raw evidence bundle through the signed record?
- ☐ **Zero-miss confidence limit:** Why do 299 representative independent injections with no misses give a 95 percent upper miss-probability limit of 1 percent for that injection population, without proving universal diagnostic coverage?
- ☐ **Exposure model:** Under what constant-rate and representative-exposure assumptions does a 10^{-9} h^{-1} target need about 3.0×10^9 zero-event hours for a 95 percent upper limit?



Systems Perspective 30.4: Gantry tether and dynamic HIL testing

Validating a bipedal humanoid under destructive hardware faults requires specialized physical verification infrastructure:

- **Fall-risk test:** On a bipedal platform walking at an assumed 1.5 m/s, an encoder dropout or missed fieldbus packet could invalidate balance estimation. The fault may cause a fall

The zero-event exposure requirement depends on the selected rate model and test population.

depending on phase, remaining sensing, and controller response. A safety case needs those conditions tested while protecting people and equipment.

- **Instrumented support:** A gantry or tether can limit test damage if its detection, travel, force, and attachment limits are independently measured. A proposed 15 ms catch response is an acceptance target for a specified rig, not a property of gantries in general; tether loading can also change the gait being measured.
- **Compound injection:** One constructed campaign might combine 50 μ s fieldbus jitter, 10 V droop on a 48 V rail, and a 100 ms intent stall. Its oracle would measure whether the configured supervisor starts a feasible holding or capture response within the declared clearance and current limits, and retain the trace when it does not.

30.8 Fallacies and Pitfalls

A machine can pass isolated fault tests and still fail when shared physical resources—such as power distribution buses, which distribute electrical power to various components, and high-speed SerDes communication channels, which facilitate rapid data transfer—couple the faults or disable their detectors. These components significantly impact system reliability and fault detection. Useful compound tests track these dependencies and preserve enough galvanically or logically independent instrumentation to establish the sequence of events.

Fallacy: *Independent local watchdog timers guarantee end-to-end physical deadline compliance.*

A proportional flow regulator must arrest a pressure spike within 250.0 ms. Its sensing, inference, and valve actuation stages each pass their local watchdog limits, but their execution delays combine with fieldbus queuing jitter to reach 75.0 ms + 115.0 ms + 38.0 ms + 45.0 ms = 273.0 ms. No individual timeout detects the excess because the unbudgeted handoff delay occurs between the local checks. Verification must trace the complete sensing-to-actuation interval under coincident load and include communication jitter in the budget. Local watchdogs remain useful, but their bounds support an end-to-end claim only when the composition, including every handoff and buffer delay, fits the physical response envelope.

Pitfall: *Allowing diagnostic logging and high-throughput telemetry to share fieldbuses with real-time safety loops.*

A platform shares a single EtherCAT fieldbus among sensing, actuation, enforcement, and evidence logging. A diagnostic memory dump raises bus utilization to 98 percent, inducing network transmission jitter that delays both the SE(3) kinematic commands and the telemetry needed to judge their physical effect. The same bandwidth contention also delays safety heartbeats and the logs that would explain a failure. Testing each stream separately misses this common dependency. Critical traffic requires bounded service through validated time-sensitive networking (TSN) scheduling and hardware priority, or separate physical channels, while diagnostic traffic must remain within its assigned budget. Compound-load testing must confirm that logging cannot prevent either the safety response or the observation of that response.

Pitfall: *Relying on software anomaly detectors that share electrical power or failure modes with monitored actuators.*

A power rail brownout freezes a Coriolis flow meter at a plausible 4.2 kg/s reading while a downstream valve blockage physically reduces actual flow to 0.8 kg/s. The ADC packets remain well formed and the value stays within nominal limits, so the anomaly detector relying on that signal does not recognize the blockage. An isolated blockage test would not reveal this common-cause failure if the meter's power remained healthy. Verification must examine whether one fault can disable the evidence needed to detect another. Galvanically isolated sensing and explicit checks for specified stuck-signal behavior must support the response, rather than assuming a plausible measurement is a functioning measurement.

Fallacy: *Treating an unobservable test outcome where diagnostic loggers dropped packets as evidence of safety.* During a compound-fault test, bus contention drops logger packets and a watchdog reset erases the volatile SRAM trace. The machine does not strike a physical hard stop, but the team can no longer reconstruct its peak impact forces, its SE(3) trajectory, or its response timing. That observation cannot establish compliance with ISO 10218 safety envelopes that were not empirically measured. The trial must remain inconclusive for those claims, with the recording failure treated as an instrumentation defect. Before repeating the test, the team must make the required evidence path survive the injected electrical conditions. A visually favorable outcome cannot substitute for the quantitative measurements specified by the safety acceptance criteria.

30.9 Summary

A flow regulator that holds 4.0 MPa for three shifts has shown that its nominal loop worked under those encountered conditions. To test its containment claim, the team must also jam the relief valve, delay pressure observations, and measure whether the independent trip seats the isolation valve before the stated vessel limit. Each test links a design claim, injection, detector, response, and measured plant outcome. Passing one condition narrows uncertainty; it does not prove every combination of faults and aging states safe.

The nominal operating envelope and the separately declared survival envelope give these tests meaning. Crossing the nominal boundary can be a **valid** survival test if the injected state, instrumentation, and acceptance oracle remain inside the test plan. A trial becomes **INVALID** when the injector or preconditions fail; it becomes **UNOBSERVABLE** when required truth data are lost. A valid failure beyond the nominal envelope can expose an inadequate fallback or passive containment limit. These verdicts preserve evidence that a simple pass percentage would hide.

Simulation can sample many cavitation and timing scenarios but cannot establish unmodeled seal wear or electrical response. A physical test samples those effects under its particular hardware and environmental conditions. Zero-miss injections bound a miss probability only for their declared fault distribution; zero-event operating hours bound an incident rate only under the declared exposure model. Neither substitutes for measured actuator response and an independently admitted fallback.

Downstream: Untested operational regimes identified during falsification feed the epistemic boundary analysis in section 34.1.



Key Takeaways: Hardware-in-the-loop verification and fault injection

- **Trace the complete response:** Precommit the fault, detector, deadline, physical oracle, and evidence path. A local watchdog pass does not establish the end-to-end limit.
- **Challenge shared resources:** Coincident bus, power, and sensing faults can disable both a protective response and its witness.
- **Keep verdicts distinct:** A valid violation is a failure; broken preconditions are **INVALID**; lost required telemetry is **UNOBSERVABLE**.
- **State the evidence population:** Report tested operating and survival conditions, fault weights, timing tails, builds, and gaps. A zero-event count is a conditional confidence limit.
- **Carry the record forward:** The versioned fault record supplies one part of the release safety case, alongside analysis, architecture, and field evidence.



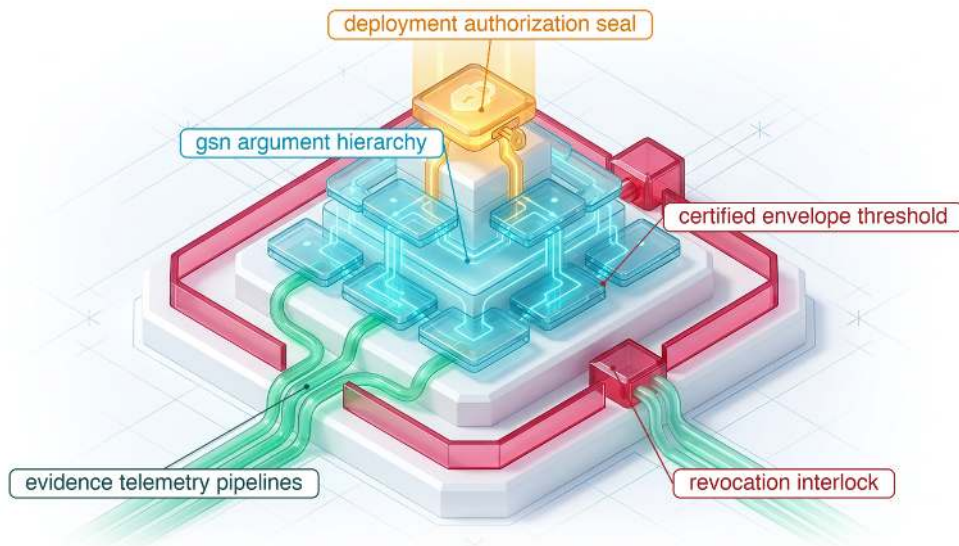
What's Next: From hardware-in-the-loop to deployment release

The versioned fault record generated by verification provides the essential empirical foundation for release certification in chapter 32. Rather than relying on ungrounded statistical optimism, chapter 32 consumes these auditable fault-injection traces, timing margins, and quantified coverage gaps to construct formal Claim-Argument-Evidence safety cases: sealing neural network weights, FPGA bitstreams, and safety invariants under a hardware Root of Trust before granting operational authority.

Deployment Release

DRAFT

Deployment Release



- 32.1 The Decision to Deploy
- 32.2 The Release Question
- 32.3 Safety Cases and Claims
- 32.4 The Three Verdicts
- 32.5 The Verdict's Standing Conditions
- 32.6 A Case Worked in Full
- 32.7 Adversarial Review
- 32.8 How Release Arguments Go Wrong
- 32.9 The Release Manifest
- 32.10 Fallacies and Pitfalls
- 32.11 Summary

Purpose

Under what auditable chain of physical evidence is an autonomous machine permitted to move among humans?

Releasing a machine allows its software to produce physical consequences outside the test environment. That permission requires an argument connecting the intended operation to evidence about the assembled system. Model accuracy alone cannot ensure the machine stops within its clearance, nor can a demonstration confirm the fallback's effectiveness under required faults. The release decision must identify which measurements support each claim and which assumptions limit its scope. A change in payload or operating conditions can invalidate the argument even when the software remains unchanged. Operating authority carries conditions the deployed system must monitor and specify responses when they no longer hold. Release brings the brain's capabilities, the nervous system's enforcement, and the body's measured limits under one accountable judgment about where the machine may operate and what evidence must remain valid for that permission to continue.

Brain
1–50 Hz
foundation models, SLAM, planning

above: propose; below: command

Nervous System
1000 Hz
TSN, intent leases, lock-free IPC

Body
continuous
real-time MCU, safety filters, CBFs

Boundary
continuous
sensors, actuators, kinematics

Physical World
continuous
contact mechanics, friction, mass



Learning Objectives

- Construct a release argument linking a bounded operating claim to auditable physical evidence
- Evaluate whether evidence supports operation, conditional operation, or refusal within the proposed envelope
- Specify standing conditions that withdraw operating authority when measurements or configuration changes invalidate a release
- Diagnose unsupported assumptions and weak evidence through adversarial review of a release argument

Prerequisite: The empirical verification evidence backing release claims is produced in section 30.4.

1 | Privilege Separation in Cognitive Architecture: As established in section 6.1, Tier 3 cognitive models are probabilistic and unverified. Release governance ensures that learned policies are never granted direct actuator authority, requiring all proposed setpoints to pass through deterministic enforcers. Granting direct actuator access to unverified latent representations risks commanding trajectory setpoints that exceed mechanical joint limits or induce destructive high-speed collisions.

2 | Enforcement and Safe Sets: As detailed in section 8.1 and section 24.7, a sampled control barrier can constrain admitted commands under its model, state-estimation, input, and timing assumptions. The release case checks those assumptions and independent preemption under the claimed faults. Bus starvation or software lockups can otherwise consume the stopping margin.

3 | Causal Boundaries and Contact Criteria: As analyzed in section 2.2 and section 4.1, the contact assessment must name effective moving mass, geometry, stiffness, body region, pressure, and velocity. ISO/TS 15066 supplies body-region-dependent guidance; 50 N is the conservative criterion selected for this worked cell, not a universal transient-contact threshold.

4 | Fault Evidence: Section 30.1 and section 30.7 describe signed, reproducible fault records. They document responses to injected conditions; a finite campaign does not by itself establish the worst possible response or coverage of untested fault modes.

32.1 The Decision to Deploy

A directory of passing test records and hardware fault traces does not answer the ultimate systems question: under what auditable chain of physical evidence can an autonomous machine move among humans? In enterprise software or web services, deploying a new build carries operational and financial risk—a regression may corrupt a customer database or degrade service latency. In physical AI, the decision to release delegates physical authority over mass, force, and velocity into an unstructured world populated by human beings. An unhandled edge case, a perceptual hallucination, or an unbudgeted compute latency spike does not terminate in a benign software stack trace; it converts stored electrical, chemical, or hydraulic energy into crushing kinetic work against physical structures and human bodies. Even after adversarial verification harnesses have systematically stressed cyber-physical assumptions—injecting clock drift, memory bus contention, sensor resonance spoofing, and hardware fault conditions into target silicon and mechanics—the engineering challenge shifts from uncovering defects to proving bounds. Yet in engineering practice, development teams routinely attempt to justify deployment using ungrounded statistical optimism: quoting 99.9 percent benchmark accuracy on static validation datasets, or showcasing thousands of error-free cycles recorded during controlled laboratory demonstrations. Neither metric addresses the release question. High validation accuracy merely demonstrates that a learned policy can reproduce training correlations under stationary data distributions, while a flawless laboratory demonstration proves only that a single sequence of states executed safely under the particular environmental conditions present during that run. Deciding to deploy is not a statistical prediction; it is an evidence-based engineering adjudication that formalizes whether empirical data bounds the physical risk of operation.

As illustrated in the architecture stack in figure 32.1, Safety Cases and Release Governance form the third pillar of Tier 4 (System Governance & Assurance). Operating at supervisory cadences of 0.1 to 1 Hz, release governance serves as the constitutional gatekeeper of the physical AI stack. It does not generate control actions; rather, it evaluates whether the mechanisms constructed and tested across every lower tier cohere into an unbroken, auditable argument:

- **The Brain (Tier 3):** evaluating whether the uninterpretable representations and probabilistic trajectory proposals of vision-language-action policies are bounded by provable constraints;¹
- **The Nervous System (Tier 2):** verifying that independent enforcers, watchdogs, and fallbacks can revoke learned-policy tracking within the deadlines derived for this plant; the worked 1 ms control period is distinct from full stopping response;² and
- **The Body (Tier 1):** grounding all safety claims in the hard physics of contact mechanics, effective moving inertia, motor winding thermal limits, and biomechanical human injury thresholds (such as ISO/TS 15066).³

The worked cell sorts and transfers metal castings beside assembly workers. Its proposed contact criterion is 50 N under a specified contact model; at 12.0 kg effective mass and 1.71 m/s, the unbraked elastic model predicts about 725 N. Release therefore depends on pre-contact clearance, measured brake capability, a bounded full response path, and fault evidence.⁴ A measured $P_{99.99} \leq 1.8$ ms reflex latency is useful evidence, but a percentile alone cannot serve as the hard response bound.

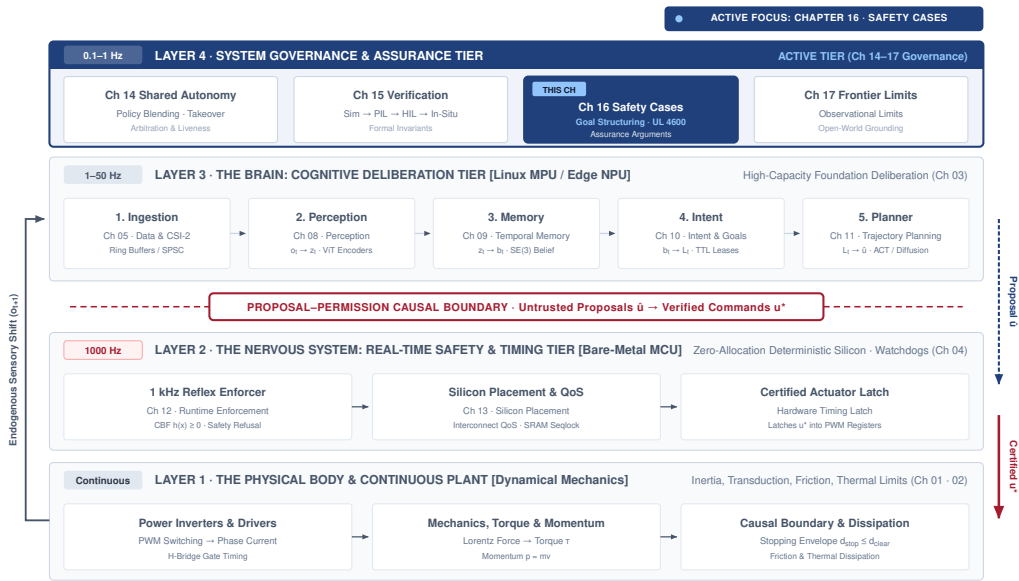


Figure 32.1: Safety cases architecture focus: The four-tier physical AI systems architecture highlights the system governance and assurance tier, with safety cases as this chapter’s active focus. Read the highlighted box as the point where measurements and runtime enforcers from every lower tier are assembled into an auditable argument, because deciding that a machine may operate requires proof rather than statistical optimism.

A release verdict is a bounded, revocable judgment. The chapter builds that judgment from the ODD, a claim-argument-evidence tree, three signed verdicts, standing conditions, and a manifest whose invalid evidence withdraws permission.

32.2 The Release Question

The release question is whether evidence justifies a particular machine’s motion in a defined operating domain. The argument names the physical bounds, specified fault set, and validated permission and fallback path. It can support a revocable grant of tracking authority while those conditions hold; it cannot infer protection against every possible fault from finite tests (32.1).

 Definition 32.1: Goal-structured physical safety case

Goal-structured physical safety case is an explicit claim, argument, and evidence chain for a stated machine, operating envelope, hazard criteria, and fault set. Its warrants identify how tests, analysis, and runtime envelope support the bounded claim and where they remain conditional.

1. **Significance:** Transforms subjective, informal deployment confidence into an auditable, falsifiable engineering argument that connects high-level safety goals to concrete empirical evidence across the entire physical stack.
2. **Distinction:** Unlike software unit test suites or static model validation scorecards, a physical safety case explicitly links operational design domain boundaries, hardware fault injection logs, and runtime enforcer tripwires to demonstrate bounded residual risk.
3. **Common pitfall:** Treating the safety case as a retrospective compliance checklist completed after development, rather than a living architectural contract that governs feature integration and release gating.



Figure 32.2: Large-machine context: A loaded Caterpillar 785C haul truck at the Kosse lignite mine illustrates the mass and clearance a release case may need to address. The photograph does not establish autonomous operation, sensing, or braking performance. Photo: Roy Luck, Wikimedia Commons (CC BY 2.0); original on Flickr.

The **operational design domain (ODD)** is the set of physical and environmental states covered by the release case. The worked cell permits a reach no greater than 1.2 m and height 0.10–1.20 m, payload 0.5–12.0 kg within an inertia bound, measured contact friction $\mu \geq 0.35$, ambient 15–40°C, illumination 200–3500 lx, and a measured human separation of at least 0.30 m. Parts of those ranges are conditional rather than nominal, as specified in table 32.1. Unknown measurements do not establish ODD membership.

Inside these boundaries the learned policy may propose actions across the nominal task envelope, but each action still requires independent permission. Conditional zones restrict that envelope; for example, the worked cell slows when illumination falls below 500 lx or measured friction falls below $\mu = 0.50$. Unknown measurements and states outside the signed envelope refuse autonomous motion and invoke the separately validated, state-dependent stop. Figure 32.2 illustrates the scale of machinery for which this distinction matters; the photograph alone does not establish its sensors or braking architecture.

As formalized in table 32.1, each monitored dimension is partitioned into nominal, conditional, and refusal states. The rows are an illustrative cell configuration; rates, trip response, and stop feasibility require validation on the assembled plant. A conditional state restricts admitted proposals. A refusal state withdraws tracking permission and selects a validated fallback; cutting torque alone can allow a moving load to coast.

Prerequisite: Absence of ledgers defining the operational design domain originate in section 10.8.

Table 32.1: Operational design domain (ODD) boundary example: Each listed dimension has disjoint nominal, conditional, and refusal intervals. Conditional operation requires every dimension to remain in a signed admissible interval; missing or invalid measurements refuse. Sensor rates and stopping response are plant validation obligations.

ODD Dimension / Invariant	Nominal Envelope ($\mathcal{S}_{\text{nom}}$)	Degraded Boundary ($\mathcal{S}_{\text{cond}}$)	Hard Refusal ($\mathcal{S}_{\text{refuse}}$)	Runtime Monitor & Rate	Enforcer Action & Latency
Workspace Reach & Height	$0 \leq r \leq 1.00 \text{ m}$, $0.10 \leq z \leq 1.20 \text{ m}$	$1.00 < r \leq 1.20 \text{ m}$ with nominal height	$r < 0$ or $r > 1.20 \text{ m}$; $z < 0.10$ or $z > 1.20 \text{ m}$	Encoders, configured sample rate	Restrict speed in buffer; refuse outside reach or height
Payload Mass & Inertia	$0.5 \leq m \leq 8.0 \text{ kg}$, $0 \leq I_{zz} \leq 0.08 \text{ kg m}^2$	$8.0 < m \leq 12.0 \text{ kg}$ with $I_{zz} \leq 0.08 \text{ kg m}^2$	Other mass or inertia; invalid load estimate	Calibrated load estimate	Derate for heavier payload; refuse invalid estimate
Contact Surface Friction	Measured $\mu \geq 0.50$	$0.35 \leq \mu < 0.50$	$\mu < 0.35$ or unverified surface	Tactile shear and scheduled inspection	Restrict grasp; stop on loss of validated traction
Ambient Illumination	$500 \leq E \leq 2000 \text{ lx}$	$200 \leq E < 500$ or $2000 < E \leq 3500 \text{ lx}$	$E < 200$ or $E > 3500 \text{ lx}$	Calibrated light sensor	Restrict sensing- dependent tasks or refuse
Thermal Actuator State	$15 \leq T_{\text{amb}} \leq 35^\circ\text{C}$ and $0 \leq T_{\text{coil}} \leq 353 \text{ K}$	$35 < T_{\text{amb}} \leq 40^\circ\text{C}$ or $353 < T_{\text{coil}} \leq 383 \text{ K}$, and neither variable in refusal range	$T_{\text{amb}} < 15$ or $> 40^\circ\text{C}$; $T_{\text{coil}} < 0$ or $> 383 \text{ K}$	Thermistors and age checks	Derate only in validated combined states; otherwise refuse
Human Co-Presence	$d_{\text{sep}} \geq 1.50 \text{ m}$	$0.30 \leq d_{\text{sep}} < 1.50 \text{ m}$	$d_{\text{sep}} < 0.30 \text{ m}$ or invalid range	Safety-rated separation sensing	Restrict speed; command assessed protective stop

A defensible release claim states its plant, contact geometry, and fault set.⁵ The proposed cell uses a chosen 50 N peak-force criterion for an assessed contact case and a 40 ms stopping target under validated braking and response assumptions. The release argument must demonstrate which human contacts are excluded before impact, which detected contacts the model covers, and which single faults the independent fallback handles. A stop after contact does not by itself cap the first force peak.⁶

Consider an effective moving mass (manipulator link, wrist, and gripped casting) of $m_{\text{eff}} = 12.0 \text{ kg}$ traveling at peak transit velocity $v_{\text{max}} = 1.71 \text{ m/s}$. The initial translational kinetic energy carried by the payload evaluates to:

$$E_k = \frac{1}{2} m_{\text{eff}} v_{\text{max}}^2 = \frac{1}{2} (12.0 \text{ kg}) (1.71 \text{ m/s})^2 = 17.54 \text{ J}$$

In an unbraked elastic impact against human tissue modeled with effective compliance stiffness $k_{\text{tissue}} = 15,000 \text{ N/m}$, the peak contact force follows equation 32.1:

$$F_{\text{unbraked}} = v_{\text{max}} \sqrt{k_{\text{tissue}} m_{\text{eff}}} = 1.71 \text{ m/s} \times \sqrt{15,000 \text{ N/m} \times 12.0 \text{ kg}} \approx 725.5 \text{ N} \quad (32.1)$$

This ideal elastic model predicts 725.5 N, or 14.5× the worked case's selected 50 N criterion. It establishes why this speed and mass cannot be admitted into an unseparated contact zone without a stronger, measured protection argument.

To maintain the release claim, the safety reflex incorporates an optical-tactile contact detector with latency $t_{\text{detect}} = 2 \text{ ms}$, followed by electromechanical spring brakes providing sustained deceleration $a_{\text{brake}} = 45 \text{ m/s}^2$. The total stopping distance combines reaction drift and active braking displacement, formalized in equation 32.2:⁷

$$d_{\text{stop}} = d_{\text{drift}} + d_{\text{brake}} = v_{\text{max}} \cdot t_{\text{detect}} + \frac{v_{\text{max}}^2}{2a_{\text{brake}}} \quad (32.2)$$

Under nominal execution, reaction drift spans $d_{\text{drift}} = (1.71 \text{ m/s})(0.0020 \text{ s}) = 3.42 \text{ mm}$, active braking requires $t_{\text{brake}} = 1.71/45.0 = 38 \text{ ms}$ across $d_{\text{brake}} = (1.71)^2/(2 \times 45.0) \approx 32.5 \text{ mm}$, yielding a total stopping time $t_{\text{stop}} = 2.0 \text{ ms} + 38.0 \text{ ms} = 40 \text{ ms}$ and stopping envelope $d_{\text{stop}} \approx 35.9 \text{ mm}$.

⁵ | **ISO/TS 15066 Biomechanical Limits:** The specification gives body-region- and contact-type-dependent force and pressure data (ISO/TS 15066 2016). A site assessment must select the applicable body region, contact geometry, and measurement procedure. The 50 N figure here is a conservative case criterion, not a universal ISO/TS 15066 threshold or an injury boundary.

⁶ | **Contact Mechanics Force Model:** The elastodynamic model couples translational kinetic energy, effective moving inertia, and contact interface stiffness to determine transient peak impact force during an unmitigated collision. Impacting a rigid boundary concentrates stored kinetic energy into a sub-millisecond impulse with severe force amplification, whereas elastic compliance extends deformation time and dissipates energy over a longer stroke. Failing to account for contact stiffness leads to severe underestimation of peak impact force during rapid decelerations.

⁷ | **Kinematic Braking Integration:** Computing total stopping distance requires piecewise integration of the velocity profile across initial reaction latency and active mechanical deceleration phases. The moving mass drifts linearly at maximum velocity during the sensing and brake solenoid engagement interval before following a quadratic trajectory as mechanical friction dissipates stored momentum. Underestimating reaction latency or brake engagement lag causes the machine to penetrate safety zones before deceleration begins.

If optical surface contamination (e.g., airborne oil mist on tactile covers) delays impact detection by an unbudgeted $\Delta t_{\text{delay}} = 12$ ms (expanding perception latency to $t'_{\text{detect}} = 14$ ms), reaction drift expands to $d'_{\text{drift}} = (1.71)(0.014) = 23.94$ mm. The total stopping envelope widens to $d'_{\text{stop}} = 23.94 \text{ mm} + 32.49 \text{ mm} = 56.43$ mm, while stopping time extends to $t'_{\text{stop}} = 14.0 \text{ ms} + 38.0 \text{ ms} = 52$ ms. Because $t'_{\text{stop}} = 52.0 \text{ ms} > 40.0 \text{ ms}$ and $d'_{\text{stop}} = 56.4 \text{ mm} > 36.0 \text{ mm}$, the modeled stop exceeds its assumed clearance budget by 20.5 mm (a 57 percent expansion). Contact depends on the actual obstacle geometry, but this result falsifies the proposed clearance warrant (32.1).



Checkpoint 32.1: Biomechanical force and stopping envelope

- ☐ **Biomechanical injury thresholds:** Can you explain why unbraked kinematic momentum transforms into transient peak forces exceeding human tissue pain limits by more than an order of magnitude?
- ☐ **Braking reaction drift vs active deceleration:** Can you distinguish between the linear reaction displacement during sensor detection latency and the quadratic dissipation of kinetic energy during active friction braking?
- ☐ **Sensing latency tails and envelope expansion:** Can you describe how unbudgeted perceptual latency tails (such as optical contamination) expand the physical stopping envelope and invalidate a release claim?

Accountability for the release decision must reside with an identifiable **lead systems adjudicator** rather than being diffused across development teams. In complex physical AI architectures, diffusion of responsibility is a primary organizational failure mode. A vision team signs off on mean average precision scores, an actuator supplier signs off on motor torque curves, and a reinforcement learning team signs off on policy convergence. Each group assumes that another layer provides the margin of safety. The lead systems adjudicator alone assumes personal technical authority for the release verdict, attesting that the integrated system satisfies the safety case. This attestation binds the verdict to a set of immutable technical artifacts, including the cryptographic hash of the verified policy weights, the calibration logs of the tactile sensors, the measured $P_{99,99} \leq 1.8$ ms reflex tail together with a separately justified worst-response bound, and the physical dynamometer records establishing the 45 m/s^2 braking deceleration. Signing records the adjudicator's bounded judgment about that evidence and the specified operating domain.

Every release verdict carries an expiration boundary defined by physical degradation and environmental change. A machine whose release case was accepted at commissioning can become unsafe through operational drift, out-of-distribution physical interactions, and unmodeled mechanical wear. Over months of operation in a manufacturing plant, particulate contamination on optical tactile covers degrades force sensing resolution, ambient temperature swings alter motor coil resistance (as established in section 4.1), and gear backlash in the arm joints increases tracking errors while degrading braking efficiency. Out-of-distribution interactions, such as an operator presenting an unmodeled workpiece with unfamiliar surface compliance, invalidate the momentum transfer assumptions of the contact proof. When sensor diagnostics detect parameter drift beyond tolerance, or when total operating cycles reach the service limit of the mechanical friction brakes, the **release verdict** is invalidated automatically. The release question must then be asked again, demanding a renewed chain of evidence before physical authority can be restored.

32.3 Safety Cases and Claims

A safety case is a structured argument that a physical machine meets its safety claims within a specified operational design domain. Adapting Stephen Toulmin's structural argumentation framework (Toulmin 1958), Goal Structuring Notation (GSN)⁸ (Kelly 1998; Kelly and Weaver 2004), and the UL 4600 autonomy evaluation standard (Koopman 2020), this argument takes the form of a **claim-argument-evidence (CAE) safety case**. This framework decomposes a top-level safety proposition (e.g., non-injurious kinetic force dissipation) into falsifiable sub-claims, linking them through architectural and inductive warrants to empirical measurements, hardware logs, and containment proofs.

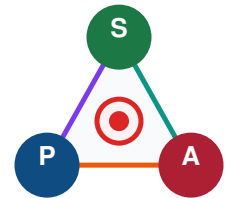
⁸ | **Goal Structuring Notation:** Developed by Tim Kelly at the University of York, GSN provides an explicit graphical grammar for structuring and auditing safety assurance cases (Kelly 1998; Kelly and Weaver 2004). The notation decomposes top-level system safety goals into contextual sub-claims, argumentation strategies, and verifiable solutions grounded in empirical test artifacts. Omitting rigorous hierarchical decomposition conceals unverified inductive leaps between high-level policies and low-level physical dynamics.

The root of the tree is the top-level claim, which asserts a measurable physical property required to prevent harm, such as guaranteeing that the kinetic energy of an articulated manipulator dissipates before the tool center point penetrates a human safety zone, or that an autonomous haul truck brings its 240-ton moving mass to rest within corridor containment. This top-level claim cannot be validated directly by inspection. Instead, it is broken down through strategies or intermediate arguments that partition the verification problem across operational sub-regimes, physical hazard classes, or architectural subsystems. Each sub-claim terminates either in supporting sub-claims or in concrete evidentiary artifacts, bounded by explicit context and assumptions that define the physical conditions under which the links hold.

The structure's validity hinges on distinguishing genuine evidence from demonstration. A **demonstration** is a record of successful task execution over an uncharacterized test distribution, such as recording ten thousand error-free bin-picking cycles under nominal factory conditions. Demonstrations measure mean performance but do not bound tail behavior or reveal failure modes outside the sampled trajectory. Genuine evidence consists of falsifiable, calibrated artifacts with quantified bounds. In a physical AI system, evidence comprises worst-case timing distributions from real-time hardware execution traces (including stator delays characterized in section 4.1), calibrated sensor noise profiles under degraded environmental states, dynamic load cell measurements recorded across deceleration regimes, and targeted fault injection trials that deliberately force the policy to the boundary of its training distribution. An evidentiary artifact lacking quantified uncertainty or failing to test the physical boundary remains a demonstration and cannot support an inferential link.

Connecting an item of evidence to a claim requires a **warrant**, which is the inferential rule stating why the data proves the assertion. In software systems governed by learned policies, this link is inductive, asserting that because the system avoided failure across finite test episodes, it will continue to avoid failure during future deployment. In physical systems operating in continuous state spaces, this inductive link is fundamentally incomplete without a deterministic enforcer.

Finite black-box testing cannot validate high-consequence physical claims, as illustrated by a target hazard rate in failures per operating hour. Establishing that a system's true hazard rate meets an acceptable threshold with high statistical confidence requires a duration of testing that is mathematically intractable.⁹ For an industrial manipulator operating beside human assembly workers, an acceptable catastrophic hazard rate might be set at one catastrophic failure per million operating hours ($\lambda = 10^{-6}$ failures/hour). To demonstrate this bound empirically with a standard 95 percent confidence level, the test campaign must complete approximately 3,000,000 hours (or 342 years) of continuous, failure-free testing on a single machine. Even a fleet of one hundred test cells running continuously in parallel would require more than three years of uninterrupted operation without encountering a single unmitigated fault. Empirical testing cannot cover the continuous state space within practical schedules, making an inductive link based solely on sampling insufficient for establishing safety. The argument must bridge the inductive gap through an architectural warrant, grounding the claim in a deterministic physical enforcer (chapter 24) that monitors reflex latencies and SE(3) kinematics in hardware, bounding actuator torque limits and interrupting motor power whenever learned commands violate ISO 10218 safety envelopes (32.1).



Release assurance: structured safety cases bounding the complete operational envelope across Sense, Plan, and Act.

⁹ | **Poisson Stationarity Assumptions:** A homogeneous Poisson model assumes a constant hazard rate λ over the stated exposure. Wear and thermal cycling can make that rate age- and load-dependent; a Weibull model is one possible alternative when supported by data. Applying a constant-rate estimate to later life without checking those conditions can understate risk.

Napkin Math 32.1: ODD coverage and Bayesian safety claims

Engineering Question: How many failure-free test hours N must an autonomous machine operate to substantiate a catastrophic hazard rate of $\lambda = 10^{-6}$ /h at 95 percent statistical confidence through empirical black-box testing alone, and how does adding a deterministic hardware enforcer with coverage $c = 0.9999$ alter the required testing duration?

1. The Mathematical Intractability of Pure Empirical Testing: Proving with 95 percent statistical confidence that an autonomous system's true catastrophic hazard rate does not exceed $\lambda = 10^{-6}$ failures/hour requires approximately 3.0×10^6 hours (342 years) of continuous, failure-free operation on physical hardware. If even a single failure occurs during the test campaign, the required operational exposure increases substantially. In high-consequence embodied domains, verifying safety solely through black-box endurance testing is physically

and economically intractable within practical engineering schedules. For the formal Poisson zero-failure testing derivations, Clopper-Pearson confidence intervals, and Bayesian risk formulations, see section 14.3.

2. Conditional Architectural Decomposition: Suppose hazardous *proposals* follow an assumed constant rate and an independent enforcer successfully contains 99.99% of that proposal population. Then the residual rate from that path is $\lambda_{\text{proposal}}(1 - c)$; at a proposal rate of 0.01/h it equals 10^{-6} /h. About 300 failure-free hours would put a one-sided 95 percent upper limit near 0.01/h for the proposal rate under a constant-rate exposure model. This does not measure c , detector coverage, common-cause failures, or hardware failures. Those need separate evidence and must enter the combined system claim.

Systems insight: Architectural containment can shift part of the evidence burden from rare observed system harms to separately testable proposal, detection, and fallback mechanisms. The plotted reduction is conditional on those mechanisms, not a release verdict from two weeks of testing.



Systems Perspective 32.1: The inductive safety fallacy

Finite testing constrains rates only for the sampled population and model. Figure 32.3 compares a black-box exposure calculation with a conditional proposal-rate calculation that assumes independently justified 99.99% containment. The vertical separation shows how the evidence question changes; it does not prove that the enforcer attains that coverage.

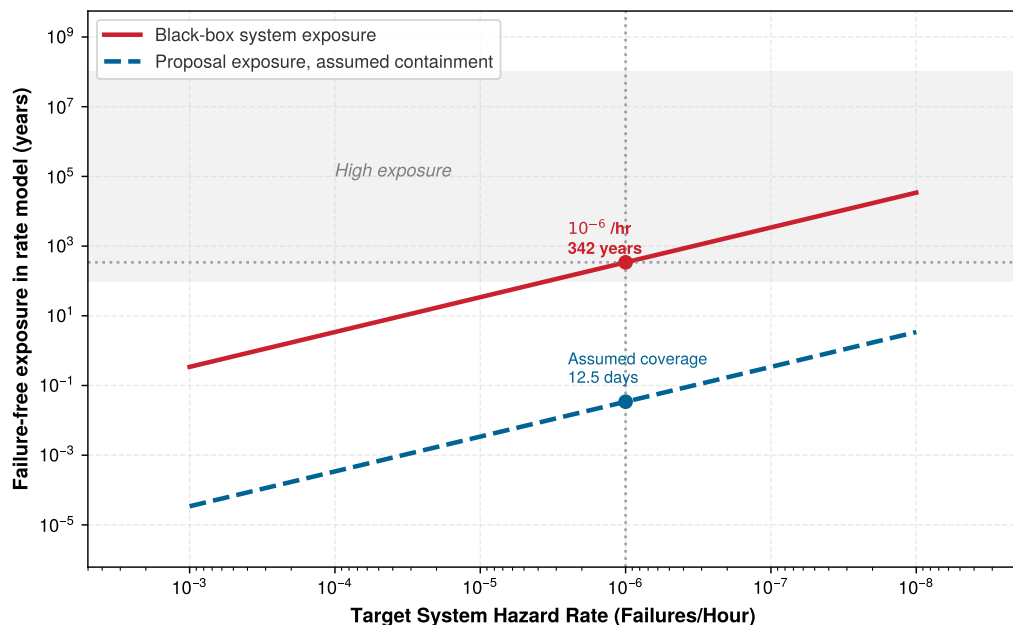


Figure 32.3: Constructed exposure comparison: Red is one-sided 95 percent zero-event exposure for an assumed constant system hazard rate. Blue is exposure to bound the *policy's hazardous-proposal rate* if independent, representative evidence establishes 99.99 percent containment of those proposals. Hardware failures, ODD gaps, and common-cause faults are additional terms; the blue curve is not an observed release result.

Every argument link relies on contextual assumptions, with unstated assumptions posing the greatest vulnerability in a safety case. A warrant that derives safe stopping distance from measured optical camera frame rates implicitly assumes that ambient illumination remains above the sensor

noise threshold, that lens covers remain free from airborne grease, that SerDes (serializer/deserializer) delivery remains within its measured loss and delay budget, and that internal bus arbitration introduces negligible jitter. In a constructed low-light mode, camera exposure lengthens from 5 ms to 30 ms, adding 25 ms of unmodeled latency to the perception pipeline. At a tool speed of 1.8 m/s, this unstated latency shift adds 0.045 m (45 mm) to the displacement of the arm before the braking command can be issued. The link between the vision benchmark and the stopping distance claim breaks silently because the physical environment drifted across an unstated boundary. A sound argument explicitly defines every physical dependency as a bounded context element linked to runtime diagnostic monitors that detect assumption violations.

The weakest link governs every claim-argument-evidence tree. A safety case is not an aggregate metric where statistical over-performance in one branch compensates for evidentiary deficits in another. High accuracy on object recognition benchmarks does not offset an uncharacterized $P_{99.9}$ latency spike in the CAN bus transceiver, nor does extensive policy simulation offset the absence of physical dynamometer brake wear data. If a top-level claim decomposes into sub-claims for kinetic energy dissipation, thermal limit protection (e.g., bounding I^2R Joule heating below stator thermal derating limits established in section 4.5), and SE(3) trajectory bounding, a failure in the evidence or warrant of any single branch invalidates the root claim. When an adjudicator examines the tree, the goal is not to count the number of supporting artifacts, but to identify the single weakest link whose failure would permit physical damage. Evaluating whether every branch maintains an unbroken chain of evidence from physical measurements to verified limits determines the verdict that governs machine release.

Downstream: Epistemic assumptions embedded within safety case warrants are critically interrogated in section 34.1.

32.3.1 Match the safety argument to the machine

For this industrial manipulation cell, ISO 10218-1 and ISO 10218-2 frame the robot and integration assessment; ISO/TS 15066 informs the assessed collaborative-contact case (*ISO/TS 15066* 2016). The 50 N force ceiling in this example is a selected, site-specific design criterion, not a universal limit from that specification. The release argument must name the contacted body region, pressure, geometry, and measurement method before using a biomechanical value.

ISO 26262 and ISO 21448 address road-vehicle electrical/electronic safety and intended-function limitations, respectively (*ISO 26262* 2018; *ISO 21448* 2022). Their fault-versus-performance distinction is useful here as an analogy, but neither supplies an ASIL D certificate for this factory cell. An independent permission path can bound learned proposals only where its sensor coverage, plant model, actuator authority, timing, and fallback are validated. A shared bus, power rail, or sensor can defeat that independence; fault analysis and stress tests must examine those paths. Unknown obstacles or contact mechanics absent from the safety model remain release exclusions or reasons for conservative separation, not automatically solved by a control barrier.

The Claim-Argument-Evidence scorecard in table 32.2 separates physical warrants from the evidence needed to support them. Claim C3 shows the decision: oil contamination can defeat the tactile branch, so nominal model accuracy cannot justify an unrestricted verdict.

Table 32.2: Claim-argument-evidence review for the worked cell: These are proposed claims and evidence obligations, not certified results. A failed branch cannot be compensated by nominal model accuracy.

Claim	Warrant and operating assumptions	Evidence needed	Adjudication
C1. Contact and stopping	Chosen 50 N criterion for specified contact geometry; bounded mass, speed, stiffness, and pre-contact clearance	Calibrated force traces and a justified full-response/deceleration bound, including worn brakes	Conditional until the measured case covers the envelope
C2. Permission timing	Independent detector, bus, drive, and brake path under defined contention and faults	Logic-analyzer traces plus response analysis for untested states; percentile traces are supporting evidence	Refuse if the hard delay budget lacks a bound
C3. Slip detection	Dry contact and validated traction; oily material is a defeater	Paired clean/contaminated trials, false-negative estimate, and independent torque trip evidence	Restrict to inspected dry parts or refuse
C4. Perception	Lighting, cable, compute, and age stay inside their measured envelope	Hardware profiling across declared conditions and a fallback for stale data	Restrict the sensing-dependent task on degradation
C5. Permission independence	Policy cannot bypass or starve the enforcer under the claimed faults	Fault injection for bus, DMA, rail, software, and supervisor paths	Refuse if a common cause defeats permission



Checkpoint 32.2: Claim-argument-evidence decomposition and inductive warrants

Before presenting a safety case to external adjudicators, test the logical integrity of your Claim-Argument-Evidence structure against formal assurance criteria:

- ☐ **Top-level claim falsifiability:** Can you formulate the top-level safety goal as an unambiguous, mathematically falsifiable proposition (e.g., bounding kinetic energy transfer below tissue fracture limits) rather than an aspirational qualitative assertion?
- ☐ **Inductive warrant validity:** Can you identify the explicit physical or probabilistic warrants connecting low-level empirical test benches to high-level system safety claims, and document the operational premises under which those warrants hold?
- ☐ **Defeater and rebuttal enumeration:** Can you show where potential defeaters (e.g., sensor surface fouling, unmodeled ambient temperature swings, or CAN bus arbitration collisions) are explicitly represented in the Goal Structuring Notation graph?
- ☐ **Evidentiary provenance binding:** Can you trace every terminal evidence node in the argument tree back to an immutable, cryptographically signed test execution trace with calibrated sensor certificates?

A safety argument with unstated inductive leaps is an operational liability disguised as formal diligence. Ensure that every claim-argument link is reinforced by empirical test manifests before submitting the dossier for adjudication.

¹⁰ | **Automotive Safety Analogy:** ISO 26262 Part 10 provides guidance for interpreting the road-vehicle functional-safety series (ISO 26262 2018). ISO 21448 addresses hazards from insufficiencies of intended functions, including perception limitations, in road vehicles (ISO 21448 2022). Their distinction between faults and performance limits helps frame this factory-cell argument; neither standard supplies this cell's release criteria or a certificate.

32.4 The Three Verdicts

The adjudication of a safety case resolves the claim-argument-evidence tree into three distinct verdicts, comprising **unconditional operation**, **conditional operation**, and **refusal to operate**.¹⁰ Each verdict establishes an explicit operating contract and mandates concrete engineering actions. The first verdict, unconditional operation, grants the full envelope named in a signed release case while its standing conditions remain true. Each hazard claim in that case needs evidence for the specified plant, load, operational design domain, and fault set, including independent permission checks where learned proposals can reach actuators (section 2.2). Brake thermal capacity, stopping clearance, and tracking-error bounds must be qualified for that envelope (section 4.5). The verdict does not eliminate residual or newly discovered hazards; it authorizes operation under the assessed claims and requires revocation when their premises fail.



Figure 32.4: Physical separation example: A yellow industrial arm works behind yellow mesh fencing. Fencing can be one condition of a restricted release when a validated access interlock and stopping test support it; the photograph does not establish those functions or their timing. Source credit: U.S. Army Chemical Materials Activity / PEO ACWA.

The second verdict, conditional operation, applies when the evidence demonstrates that the machine is safe within a restricted operational subset, but lacks the warrants required for unconstrained deployment. Rather than treating release as an all-or-nothing threshold, engineering release can enforce physical, kinematic, or environmental boundaries to eliminate unverified hazard states. These operational boundaries include de-rating peak actuator torque or joint velocity, restricting ambient operating conditions such as temperature ranges or illumination levels, imposing physical exclusion barriers that keep human personnel outside the maximum reach radius of the manipulator, or activating runtime telemetry guardrails within the nervous system. In heavy industrial robotics, when kinetic energy and force dissipation cannot be bounded unconditionally for unseparated human co-presence, a restricted case may require an assessed physical fence and validated access interlocks (figure 32.4). The access design must be tested so the protective stop completes before a person reaches the hazardous workspace, converting an unmodeled contact risk into a deterministic physical exclusion boundary.

Every condition attached to a release verdict is a binding contract. Consider an illustrative packaging arm with $m_{\text{eff}} = 40.0 \text{ kg}$ at an unrestricted speed of 1.50 m/s , carrying 45.0 J . The site assessment selects a 15.0 J energy ceiling for its contact case. An initial calculation considers 0.86 m/s , close to the mathematical ceiling; the actual command and trip limits must be lower after accounting for sampled detection and speed growth. The energy calculation is:

$$E_k = \frac{1}{2} m_{\text{eff}} v_{\text{derated}}^2 \leq E_{k,\text{allow}} \quad (32.3)$$

Under equation 32.3, 0.86 m/s gives $E_k = \frac{1}{2} (40.0) (0.86)^2 = 14.792 \text{ J}$, only 0.208 J below the chosen ceiling. This is the maximum sampled-speed case, not the commanded speed.

The absolute energy ceiling gives $v_{\text{safe}} = \sqrt{2(15 \text{ J}) / (40 \text{ kg})} = 0.8660 \text{ m/s}$. The prior 0.90 m/s trip would allow 16.2 J and is therefore rejected even though its nominal stopping distance is $18 + 27 = 45 \text{ mm}$ inside the 50 mm clearance. For this illustrative corrected contract, the proposed speed is at most 0.80 m/s and the 1000 Hz independent monitor trips at 0.84 m/s . A validated bound of 0.02 m/s on additional speed from sample phase, encoder error, confirmation, and drive response gives a worst sampled speed of 0.86 m/s , or 14.792 J . The assumed total pre-brake response remains 20 ms and validated minimum deceleration is 15 m/s^2 . Thus $d_{\text{drift}} = 0.86(0.020) = 17.20 \text{ mm}$, $d_{\text{brake}} = 0.86^2 / (2 \cdot 15) = 24.65 \text{ mm}$, and $d_{\text{stop}} = 41.85 \text{ mm}$, leaving 8.15 mm of the 50 mm clearance. These bounds require measurement on the assembled cell; a one-kilohertz sample rate alone establishes neither the speed overshoot nor the full response time.

Prerequisite: Deterministic fallback coverage supporting conditional release verdicts builds on section 24.9.

If wear reduces the validated deceleration floor to 10.0 m/s^2 and extends total pre-brake response to 30.0 ms, the same 0.86 m/s worst sampled speed travels 25.80 mm before braking and 36.98 mm while braking, totaling 62.78 mm. This exceeds the 50.0 mm reserve by 12.78 mm; contact follows only if the obstacle lies at that boundary. The worn case must fail the release standing condition during scheduled brake inspection or a validated brake self-test, before the machine enters the shared workspace. The original 0.90 m/s test would have stopped in 67.5 mm with worn brakes, a useful diagnostic contrast but never an authorized contact-zone speed (32.3).



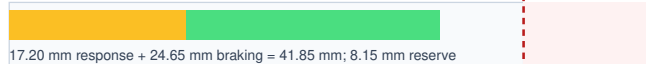
Checkpoint 32.3: Velocity derating and braking clearance

Release budget: energy and stopping clearance

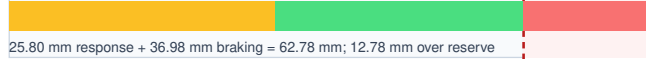
40 kg · 15 J ceiling · command $\leq 0.80 \text{ m/s}$ · trip 0.84 m/s · bounded overshoot 0.02 m/s

Worst sampled speed $0.86 \text{ m/s} \rightarrow 14.792 \text{ J}$. Earlier 0.90 m/s trip $\rightarrow 16.2 \text{ J}$: rejected.

Validated brakes



Worn brakes



Yellow: pre-brake travel · green: braking within reserve · red: braking beyond reserve. Refuse worn state.

- ☐ **Quadratic kinetic energy scaling:** Can you explain why velocity derating is an exceptionally powerful lever for satisfying biomechanical energy constraints compared to mass reduction?
- ☐ **Hardware supervisor trip thresholds and reaction lag:** Can you describe how confirmation cycle windows and brake engagement delays consume physical clearance before mechanical deceleration begins?
- ☐ **Mechanical brake wear and automatic invalidation:** Can you explain how physical degradation of friction surfaces expands the stopping envelope over operating cycles, and how runtime telemetry monitors detect this drift to revoke operating permits?

The third verdict, refusal to operate, is the mandatory outcome whenever the safety case contains missing warrants, uncharacterized tail latencies, incomplete fault-injection coverage, or unmonitored failure modes. If a system exhibits a $P_{99.99}$ control loop latency of 85 ms against a physical stability deadline of 20 ms, or if memory corruption tests under bus contention were omitted, the hazard remains unquantified and cannot support release. Refusal is likewise required when empirical observations contradict analytical physical models, such as measured motor winding temperatures rising 15 K above the lumped-parameter thermal predictions established in section 4.5 during repetitive braking cycles. Under these conditions, operating the machine is an unmodeled gamble rather than an engineered deployment.

A refusal verdict is an engineering deliverable, not an administrative failure. In industrial environments, release evaluations can degrade into political friction between delivery schedules and unquantified safety apprehensions. A formal refusal verdict eliminates this friction by isolating the precise broken link in the argument tree, converting organizational impasse into an actionable, falsifiable testing mandate. If release is refused because transient bus arbitration causes uncharacterized packet loss during simultaneous camera transmission and motor feedback updates, the engineering team does not debate subjective risk tolerances. Instead, they construct a deterministic bus schedule or partition **direct memory access**¹¹ channels, and measure communication performance until they demonstrate a P_{100} transmission latency below 4.0 ms under maximum synthetic bus load. The refusal verdict defines the exact evidentiary test that must pass before re-adjudication can occur.

None of the three verdicts represents a permanent, static grant of authority. Operating under unconditional or conditional release creates ongoing obligations that must be maintained throughout the active deployment lifecycle. Mechanical linkages develop backlash, friction surfaces wear, optical sensor lenses accumulate particulate film, and ambient distributions change. A valid release

¹¹ | **Direct Memory Access Contention:** Direct memory access subsystems transfer data between peripheral interfaces and system memory independently of the central processing unit. When high-bandwidth peripherals such as high-resolution vision cameras flood shared memory buses without quality-of-service arbitration, concurrent real-time safety control loops experience severe read-after-write stalls. Unmanaged bus contention expands worst-case control loop latency, causing deterministic safety trip-wires to miss their physical intervention deadlines.

verdict specifies mandatory recalibration intervals, such as recalibrating joint torque sensors every 500 operating hours or verifying brake pad clearance every 250000 actuation cycles. It also establishes explicit drift budgets for runtime telemetry. If the P_{99} trajectory tracking error of the nervous system drifts from 0.004 m at initial release to 0.012 m under accumulated mechanical wear, the drift budget is exhausted, and the release authorization automatically lapses until maintenance restores the baseline. High-frequency ring buffers in the nervous system must continuously record pre-trigger and post-trigger states during every enforcer trip, preserving the audit trail required to confirm that the empirical assumptions underlying the verdict remain true over time.

The adjudicator signs one of the three decisions in table 32.3. The table concerns the industrial cell under ISO 10218 and the assessed collaborative-contact criteria informed by ISO/TS 15066. ISO 26262 and ISO 21448 provide road-vehicle analogies, not a statutory classification for this cell. A conditional verdict needs a separately evidenced restricted envelope; a known hole in the nominal argument cannot simply be relabeled as a safe restriction.

Table 32.3: Three signed release verdicts for the worked industrial cell: Full and restricted operation both retain independent per-action permission. Refusal is an actionable engineering result.

Signed verdict	Evidence required	Permission and runtime limits	Expiry or refusal trigger
UNCOND	Full ODD claim, measured plant and timing bounds, independently validated permission and fallback	Policy proposes across full <i>signed</i> task envelope; every action passes the enforcer	Evidence, configuration, calibration, or standing premise fails
COND	Restricted ODD claim and its own evidence, limits, and fault coverage	Policy proposes only within signed reduced limits; every action passes the enforcer	Any restricted premise becomes false or unknown
REFUSE	Missing, contradictory, expired, or unauthenticated critical evidence	No autonomous tracking permit; execute the assessed stop or hold	Re-adjudication with repaired evidence

32.5 The Verdict's Standing Conditions

A release verdict depends on physical premises with different observation schedules. A mobile manipulator carrying 10.0 kg steel billets may rely on $\mu \geq 0.60$ floor friction, but a one-time assumption cannot cover spills or surface wear. The case needs a floor inspection schedule and a stop or exclusion rule after contamination, with any on-board traction monitor specified separately. A torque-sensor drift limit of $\pm 0.5 \text{ N} \cdot \text{m}$ can be re-established at a documented calibration interval; a 12.0 kg payload limit is checked before each lift. Each premise must remain valid at the cadence required by its physical failure mode.

Every monitored premise requires both a deterministic runtime detector and a pre-allocated response. If a safety case claims that image exposure latency remains under 16.0 ms or that CAN bus frame drops remain below 1×10^{-5} , but the embedded software lacks a hardware timestamp comparator or a frame error counter, that premise has not been monitored. It has silently degraded into an unverified assumption. An unmonitored premise blinds the machine to its own boundary crossings, converting an explicit safety argument into an unmodeled gamble. The runtime detector must evaluate the premise against physical bounds at a frequency set by the physical dynamics of the failure mode. For example, a high-torque brushless actuator risks stator winding insulation breakdown at temperatures exceeding 393 K (section 4.2). If the motor thermistor samples temperature at 10 Hz, the detector measures the thermal rise within 100 ms, identifying excessive Joule heating ($I^2 R$) before breakdown occurs, commanding an immediate current derating to cool the motor.

Evaluating monitored premises at runtime exposes a fundamental trichotomy in **envelope membership** (32.2). Software systems outside physical domains often treat safety conditions as binary booleans, presuming that in the absence of an explicit fault flag, execution proceeds normally. In physical AI, envelope membership occupies three distinct states, namely known-true, known-false, and unknown. When sensor telemetry is verified, timestamps fall within deadline bounds, and state estimators converge with bounded covariance, envelope membership is known-true. When a physical limit is breached, such as trajectory tracking error exceeding 0.050 m, the state is known-false. When an optical encoder drops communication, a camera frame arrives with a cyclic redundancy check

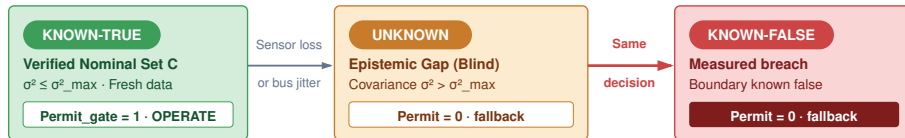
(CRC) mismatch, or an extended Kalman filter diverges, the machine cannot determine whether it is inside or outside the verified ISO/TS 15066 safety envelope. In the presence of mass and momentum, an unknown state must be treated as outside the envelope rather than inside. Presuming safety during an epistemic gap—a period where the machine is physically moving but computationally blind—allows unmeasured kinetic energy to accumulate in unmodeled configurations.



Definition 32.2: The envelope trichotomy

RUNTIME ENVELOPE: THREE EVIDENCE STATES

Unknown and known-false have different causes; both refuse tracking permission.



● **Software Fallacy:**

`is_safe == (fault)` treats telemetry drops as benign → frozen commands risk collision

● **Physical Invariant:**

UNKNOWN and KNOWN-FALSE → refuse tracking; select validated stop or hold

Envelope trichotomy is the three-state classification of runtime physical system health relative to its certified safety envelope: *known-true* (verified nominal state with bounded telemetry covariance), *known-false* (verified safety boundary violation triggering immediate hardware refusal), and *unknown* (epistemic state uncertainty due to sensor dropouts, bus jitter, or estimator divergence).

1. **Significance:** An unknown state is not evidence of a boundary breach, but neither supports continued tracking. The permission decision is refusal in both unknown and known-false states; the logged cause remains distinct and the response depends on the plant's validated stop or hold capability.
2. **Distinction:** Unlike a binary boolean status check (`is_safe == true`), the envelope trichotomy isolates epistemic blindness from active violations, ensuring telemetry loss is never mistaken for safe clearance.
3. **Common pitfall:** Defaulting to “fail-open” or holding last-known commands when communication fails. Frozen commands in the presence of unmodeled dynamics rapidly result in mechanical collisions.



Systems Perspective 32.2: The epistemic invariant

An unknown state cannot justify a new tracking command. Revoke tracking permission and use the independently validated response appropriate to the measured plant state.

The response to an unevaluable premise must be derived directly from the physical consequence of continuing motion without telemetry, rather than from subjective severity classifications. If an autonomous mobile robot (AMR) with total mass $m = 80.0$ kg travels at 1.5 m/s, it carries 90.0 J of kinetic energy. If its primary lidar sensor stops transmitting updates, each millisecond of blind travel carries the machine 1.5 mm closer to an unmapped obstacle, exhausting the stopping distance available to the mechanical brakes. The system must select among three deterministic fallback responses, established during the safety case rather than at the moment of confusion. A **degraded mode** continues operation when redundant sensors preserve sufficient state observability to bound positional uncertainty, such as falling back to wheel odometry and halving the maximum commanded velocity to 0.75 m/s. **Restricting authority** clamps actuator torque limits or constrains policy outputs to a precomputed passive corridor when secondary telemetry fails. A **protective stop**

uses the configured drive and brake sequence when observability is lost; its ability to stop within clearance depends on the current speed, response delay, and measured braking capability.

Even when every monitored premise remains known-true and periodic recalibrations pass, the verdict carries an expiration timestamp and cycle budget. Operating hours and mechanical cycles can change the plant in ways that runtime telemetry does not fully observe. Gear teeth may develop surface pitting,¹² elastomer seals stiffen, and structural fasteners lose clamp preload. A software or firmware change requires a new signed case because it may invalidate prior timing, tracking, and response evidence. The load-specific fatigue data and inspection plan set the signed operating-hour and cycle limits; reaching either limit requires re-adjudication before further motion is permitted.

Standing conditions that affect motion need an enforcement path with a measured detection and response budget; post-event logging cannot supply that protection. In the architecture of chapter 24, the independent enforcer checks proposals and monitored premises before issuing actuator permission.¹³ This worked cell uses a 1000 Hz control update, but its full sensor-to-brake response must be measured separately. When a premise is unknown or false, the enforcer revokes tracking and commands the state-appropriate validated stop. Torque inhibition alone can let a moving load coast; whether controlled braking or a holding brake is required follows the plant and hazard assessment. The signed release case must include that stop choice and its measured timing.

32.6 A Case Worked in Full

An autonomous workstation inserts machined steel splines near assembly workers. Its six-axis arm has compliant fingertips and a learned insertion policy. The proposed release goal in figure 32.5 uses this site's selected 50 N normal and 20 N shear criteria for an assessed contact geometry (*ISO/TS 15066* 2016). The claim must be bounded by payload, speed, contact location, sensor coverage, and faults; it cannot extend to every unexpected obstacle or every model error. The first review challenges whether either the tactile reflex or the motor-current trip can actually enforce the goal.

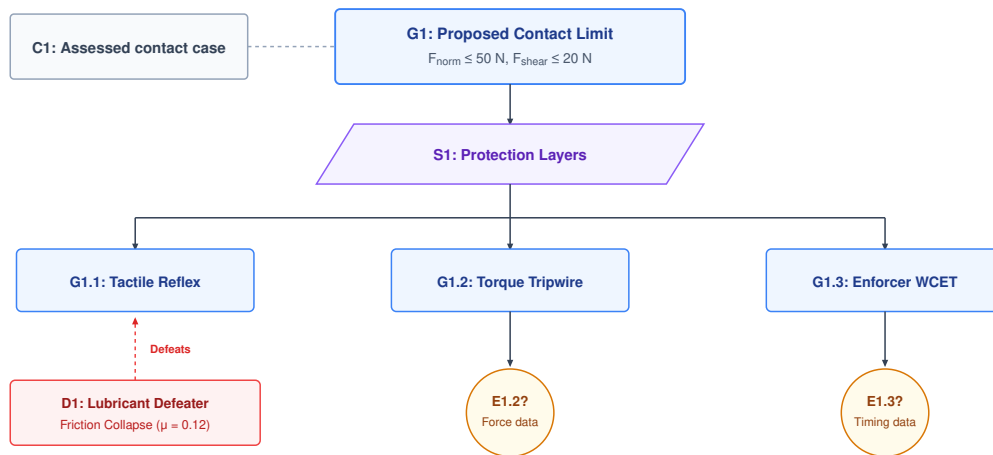


Figure 32.5: Challenged safety-case draft: The proposed contact-force goal branches to tactile, torque-trip, and timing claims. Lubricant defeats the tactile branch. The other two leaves still need calibrated contact-force and full-response evidence; generic crash logs or sampled loop times cannot close the goal (GSN structure adapted from (Kelly 1998; Toulmin 1958)).

A clean, calibrated fingertip may detect surface shear and trigger the independent brake path. The nominal proposal assumes a 0.40 m/s approach, 4.0 N/mm effective contact stiffness, a tactile detection tail of 12 ms, an 8 ms enforcer response, and a further 5 ms brake interval. These selected delays sum to 25 ms, during which 0.40 m/s would traverse at most 10 mm without slowing. They do not establish a 36 N peak: force can rise during detection, and the response percentile is not a hard upper bound. Pre-contact clearance, a justified full-response limit, and calibrated force traces are required before the tactile branch can support release.

¹² | **Mechanical Fatigue Budget:** A cumulative-damage model such as Miner's rule ($\sum_i n_i/N_i$) can organize a measured load spectrum against component-specific fatigue data; it is an approximation, not a universal failure deadline. The assessed loads, uncertainty, inspection evidence, and stopping consequences determine any signed payload derating and cycle limit. An expired limit revokes the applicable release case rather than silently extending service.

¹³ | **Hardware Watchdog Interlocks:** A watchdog can detect a missed configured heartbeat. In this illustrative design, an independently wired supervisor then requests the validated drive stop; the detector, wiring, drive, brake, and load response all enter the timing and fault analysis. A watchdog reset or torque inhibit by itself does not prove that moving mass stops.

The tactile warrant breaks under a plausible contamination test. Surface markers in the elastomer report shear only when friction transfers workpiece motion into the skin. In the constructed oily-part case, friction falls from $\mu = 0.85$ to 0.12, so a slipping metal part may fail to deform those markers enough for the learned detector to respond. The policy can continue proposing forward motion, but it never owns the actuator: the independent current trip remains the next protection layer. The release review must measure whether that layer arrests motion before the specified force criterion is exceeded.

With the primary tactile reflex disabled, the safety of the workstation falls to a secondary safety mechanism, specifically a motor-current threshold tripwire running in the joint servo drives at 1000 Hz. When mechanical resistance impedes motion, motor current rises until the measured joint torque exceeds the expected feedforward torque by a set margin, translating to an end-effector trip force threshold of $F_{\text{trip}} = 35$ N. To evaluate whether this secondary mechanism satisfies the safety claim, the physical stopping profile must be derived from first principles.

Consider an analytical impact where the tool center point moves at $v_0 = 0.40$ m/s with moving mass $m_{\text{eff}} = 4.50$ kg against a compliant human hand and fixture with effective stiffness $k = 4.0$ N/mm = 4000 N/m. Before the tripwire threshold is reached, contact force ramps up at $dF/dt = kv_0 = 4000$ N/m $\times$ 0.40 m/s = 1600 N/s. The arm requires $t_{\text{trip}} = F_{\text{trip}}/(kv_0) = 35.0$ N/(1600 N/s) $\approx$ 21.9 ms to register the disturbance, indenting compliant tissue by $x_{\text{trip}} = v_0 \cdot t_{\text{trip}} = 8.75$ mm. Once tripped, sensor filtering, bus arbitration, and brake solenoid drop add an unmitigated latency $t_{\text{lat}} = 10.0$ ms, during which the arm continues driving forward unbraked by an additional $\Delta x_{\text{lat}} = v_0 \cdot t_{\text{lat}} = 4.0$ mm. The contact force at the exact instant mechanical brakes commence deceleration is:

$$F_{\text{lat}} = F_{\text{trip}} + kv_0 t_{\text{lat}} \quad (32.4)$$

Evaluating equation 32.4 gives $F_{\text{lat}} = 35.0$ N + 4000 N/m $\times$ 0.40 m/s $\times$ 0.010 s = 51.0 N, already exceeding this cell's chosen 50.0 N criterion before deceleration. Once the brakes engage, both brake work and elastic contact oppose motion. The resulting indentation is:

$$x_{\text{max}} = x_{\text{lat}} + \Delta x_{\text{brake}} = (x_{\text{trip}} + v_0 t_{\text{lat}}) + \Delta x_{\text{brake}} \quad (32.5)$$

In equation 32.5, Δx_{brake} balances the initial $\frac{1}{2} m_{\text{eff}} v_0^2$ against brake work $m_{\text{eff}} a_{\text{brake}} \Delta x_{\text{brake}}$ and added elastic energy $\frac{1}{2} k (x_{\text{max}}^2 - x_{\text{lat}}^2)$. With $a_{\text{brake}} = 2.50$ m/s², this ideal one-dimensional model gives $\Delta x_{\text{brake}} = 4.985$ mm, $x_{\text{max}} = 17.735$ mm, and $F_{\text{peak}} = 70.94$ N. Even that favorable model exceeds this case's chosen 50 N contact criterion by about 42%. Rotor inertia, torque rise, and geometry would require measured parameters and a separate dynamic trace. Unconditional release is refused on the favorable model alone.¹⁴

An adjudicator cannot issue an unconditional release verdict based on this evidence tree. The primary evidence link supporting the tactile slip reflex is broken by fluid contamination, and the secondary motor-current tripwire cannot prevent excessive contact impulse under the nominal payload mass of 2.0 kg. The adjudication therefore issues a rejection of the unconditional proposal and constructs an amended contract for an **operate with conditions** verdict. To restore mathematical bounds on contact force, the operational envelope is restricted along three dimensions:

1. The maximum allowable payload mass is capped at 1.5 kg, lowering the total moving mass to $m_{\text{eff}} = 4.0$ kg and reducing kinetic energy.
2. An automated optical inspection station is added to the feeding fixture to perform a surface-dryness check before part acquisition, rejecting any component displaying fluid sheen.
3. The motor-current tripwire threshold is tightened from 35 N to $F'_{\text{trip}} = 18.0$ N by routing torque estimation over a dedicated hard real-time safety bus, slashing unmitigated latency to $t'_{\text{lat}} = 2.0$ ms and boosting braking deceleration to $a'_{\text{brake}} = 8.0$ m/s².

The restricted case still needs measured evidence for its premises. Under the stated ideal one-dimensional model, the secondary tripwire reaches 18.0 N after 11.25 ms and 4.5 mm indentation. A bounded 2.0 ms delay adds 0.8 mm, so force at brake onset is 21.2 N. For $m_{\text{eff}} = 4.0$ kg, $v_0 = 0.40$ m/s, $k = 4000$ N/m, and available brake deceleration 8.0 m/s², solve $0.32 = m_{\text{eff}} a d + \frac{k}{2} [(0.0053 + d)^2 - 0.0053^2]$ in SI units. The positive root is $d = 5.055$ mm, giving $x_{\text{max}} = 10.355$ mm and $F_{\text{peak}} = 41.42$ N, below this case's 50 N criterion. The free-space brake distance $v_0^2/(2a) =$

¹⁴ | **Reflected Rotor Inertia in Impact Dynamics:** A geared joint can reflect rotor inertia approximately as $N^2 J_{\text{rotor}}$ into the joint coordinate. Its contribution at the contact point also depends on linkage geometry and compliance. The ideal one-dimensional calculation excludes that contribution and brake torque rise; the release case needs measured or modeled bounds before treating its force result as a physical maximum.

10.0 mm is a different quantity; contact spring work shortens the idealized post-onset travel. A measured response bound, contact geometry, and independent pre-contact clearance must support the release; these calculations alone do not establish an all-contact guarantee (32.4).



Checkpoint 32.4: Secondary torque tripwire under contamination

- ☐ **Surface contamination and friction collapse:** Can you explain why residual cutting fluid disabling optical-tactile shear strain detection creates a silent failure mode in what appeared to be a verified reflex safety argument?
- ☐ **Secondary tripwire force accumulation:** Can you explain why a current trip without a full response bound can exceed this case's contact criterion before braking begins, as in equation 32.4?
- ☐ **Conditional contract mitigation via mass and threshold tightening:** Can you describe how an Operate with Conditions verdict restores safety margins by simultaneously restricting payload mass, introducing an upstream dryness check, and migrating torque observation to a hard real-time bus?

32.7 Adversarial Review

The key question during adjudication is not whether the engineering team believes its safety argument, but where an independent reviewer can break it. An **adversarial review** is a structured audit that treats the safety case not as a defense brief compiled to justify deployment, but as an attack surface designed to be systematically dismantled. When engineers assemble a safety case, confirmation bias steers data collection toward nominal operating envelopes where learned models succeed and monitoring tripwires trigger on schedule. The adversarial reviewer inverts this posture, searching for unsupported warrants, uncharacterized physical transitions, and edge conditions that invalidate the mathematical bounds of the release contract. If an argument cannot withstand an adversary aiming to force a physical violation on paper, the machine will not survive in a mechanically governed environment.

The review evaluates every evidentiary node in the argument tree using four targeted probe vectors. The first probe challenges **empirical sufficiency** by testing whether verification datasets covered the physical extremes of the operational envelope. For an analytical example, consider a benchmark demonstrating ten thousand successful manipulation cycles at an ambient temperature of 20°C on clean workpieces, supporting a warrant that grip slip probability remains below $P(\text{slip}) \leq 0.001$. The reviewer probes the boundary by introducing airborne oil mist and an ambient temperature of 40°C, conditions that accelerate **Joule heating** (I^2R), force **thermal derating** (section 4.1), and reduce the friction coefficient of the elastomer gripper pads from $\mu = 0.80$ to $\mu = 0.35$. If the test suite contains zero data under degraded friction, the warrant is unsupported. This failure mode is detected through distribution coverage audits of physical parameters during review, but on the factory floor it goes undetected until a dropped payload strikes the workstation. The second probe examines **domain transfer**, identifying claims where simulation data substitutes for hardware measurement. An optical classifier may demonstrate a 99.9 percent detection rate for surface contamination under synthetic rendering, but specular reflections from micro-machining marks on physical steel surfaces can drop the hardware accuracy to 86 percent. This defect is detected by paired hardware-in-the-loop cross-validation on physical workpieces, and without those physical benchmarks, it remains unmonitored until contaminated stock enters the cell.

The third probe attacks **structural independence**, asking whether the learned model and safety enforcer share a resource whose contention could delay sensor delivery. For a constructed shared-bus case,¹⁵ a 14 ms delay would defeat a claimed 2 ms observation budget. Stress tests and timing analysis must establish the actual interference bound; idle-bus unit tests cannot do so. The fourth probe identifies **unmonitored state drift**. Wear can change gearbox compliance¹⁶ and the response between motor current and link motion. The reviewer therefore asks for paired wear-state braking and contact traces before accepting the 41.42 N restricted-case model above. A motor-current trip alone does not establish that the worn plant remains inside the case's measured force and stopping limits.

15 | **System-on-Chip Resource Sharing:** A system-on-chip integrates central processing cores, graphics accelerators, memory controllers, and peripheral buses onto a single silicon die. While highly power-efficient, sharing internal interconnects and memory caches between high-throughput neural inference and hard real-time safety monitors introduces severe cross-core contention. Unmanaged bus contention from cognitive vision tasks can starve real-time safety threads, causing tripwire latencies to spike beyond physical stability limits.

16 | **Cycloidal Drive Dynamics:** Cycloidal speed reducers employ eccentric cam mechanisms and rolling pins to achieve high torque reduction ratios with minimal backlash in compact joints. Over extended operating life-cycles, rolling contact wear increases internal torsional compliance and introduces phase lag between the actuator motor and link sensors. This unmonitored structural compliance degrades closed-loop disturbance rejection, delaying emergency braking response and increasing peak collision forces.

Reviewers operationalize these vulnerabilities by constructing explicit **defeaters**, which are physically realizable scenarios that invalidate a specific argument warrant. Standard diagnostic error handlers identify binary faults like disconnected sensors or memory segmentation panics, but cannot detect coupled, sub-threshold degradations. An adversary constructs combinations of minor anomalies that each remain within allowable operational bands yet collectively defeat a safety claim. Consider an optical lens with a 10 percent particulate haze paired with structural cable vibration and a 4 mm mechanical misalignment from fixture bolt relaxation. In isolation, the optical sensor reports normal visibility, the fixture monitor registers within tolerance, and the execution thread meets its nominal timing budget. Concurrently, the degraded contrast forces the neural feature-matcher into iterative retry loops (introducing compute jitter), while mechanical vibration on the camera harness induces cyclic redundancy check (CRC) errors and packet retransmissions across the physical serializer/deserializer link (**SerDes bandwidth**). This combination of compute jitter and transmission delay extends perception latency by 20 ms and forces the policy to command an uncharacterized **SE(3) kinematic trajectory** that positions the gripper off-center. The slip reflex arrives too late to halt the falling mass. Because no individual sensor crossed an alarm threshold, software diagnostics report normal operation while the physical invariant fails.

The adversarial review also audits the execution integrity of the permission path, verifying that neither model inference tasks nor auxiliary background services can bypass, corrupt, or starve safety enforcer assertions (chapter 24) or human authority overrides (chapter 28). Under the proposal-permission architecture (section 6.1), safety assertions and human overrides must maintain non-preemptible execution priority over learned models. An adversary evaluates whether a runaway policy thread, an unhandled exception in an accelerator runtime, or a high-volume telemetry stream can saturate memory channels, freeze the operating system by locking a kernel spinlock, or exhaust physical power delivery rails. If an onboard graphics accelerator draws peak transient current that causes a voltage droop, resetting an adjacent micro-controller on the same power plane, the nervous system loses its ability to command actuator brakes within the required 5 ms deadline. Validating **permission path** independence requires physical fault injection, including bus saturation, power rail glitching, and forced kernel panics. If the permission path is implemented in user-space software rather than isolated hard real-time threads and dedicated hardware interrupt lines, the reviewer marks the link as broken.

When an adversarial attack succeeds, the safety case cannot be salvaged by editing claim descriptions or inserting arbitrary safety margins. The protocol requires decomposing the broken warrant into its underlying systems failure, whether that represents an uncharacterized physical boundary, an inadequate verification dataset, or an unmodeled hardware latency. Remediation routes the defect directly back to the owning subsystem. If an adversary proves that the motor-current observer fails under gearbox compliance, the mechanical specification must introduce hard wear limits and runtime compliance estimation. If shared bus contention compromises enforcer latency, the compute architecture must enforce physical memory partitioning and real-time scheduling guarantees. If perception transfer fails across lighting variations, the verification pipeline must collect hardware-in-the-loop datasets under physical extremes. The safety case returns to the engineering team with broken warrants explicitly identified, requiring updated hardware boundaries, re-derived stopping distances, and renewed empirical verification before the machine can be submitted for a new release verdict.



Checkpoint 32.5: Adversarial review vectors and margin erosion

Before approving a deployment contract, evaluate the safety case through the adversarial posture of an independent auditor seeking to dismantle its claims:

- ☐ **Empirical sufficiency stress-testing:** Can you verify that testing datasets deliberately attacked the physical boundaries of the Operational Design Domain (such as extreme temperature, oil mist contamination, and voltage droop) rather than sampling only nominal interiors?

Contrast: Algorithmic capability claims contrast with physical actuator saturation limits established in section 4.4.

- ❑ **Sim-to-real transfer challenge:** Can you identify every claim that relies on synthetic simulation data and verify whether physical HIL dynamometer experiments have confirmed the transfer boundaries under non-linear friction and contact dynamics?
- ❑ **Normalization of deviance detection:** Can you audit field telemetry records for recurring, non-fatal anomalous transients (such as bus jitter spikes or brief tracking errors) that have been improperly rationalized as acceptable operational noise?
- ❑ **Proxy metric gaming resistance:** Can you prove that optimizing for secondary performance metrics (such as cycle time, trajectory smoothness, or power conservation) has not silently eroded the physical braking clearance and reaction buffers?

If an adversarial review cannot identify the exact physical conditions that would invalidate the release verdict, the audit is incomplete. Review this section before examining structural argument failure modes.

32.8 How Release Arguments Go Wrong

Accepted arguments often follow a few flawed patterns. A flawed safety case that permits a hazardous machine to operate rarely stems from an unclassifiable anomaly. It traces back to one of four recurring structural pathologies in the claim-argument-evidence tree: the simulation-to-reality leap, the proxy metric fallacy, the unmonitored condition, and the organizational normalization of deviance. Each pathology introduces a specific flaw into the argument graph, creating an illusion of empirical verification while leaving the physical machine unprotected against mechanics, timing jitter, and wear.

The *simulation-to-reality leap* occurs when high-volume synthetic or simulated pass rates are substituted for physical verification without bounding unmodeled contact dynamics or sensor noise floors. An engineering team may run 1×10^6 collision-free simulated episodes with a 99.9 percent task completion metric in a rigid-body physics engine. Yet virtual physics engines calculate contact forces using idealized **Coulomb friction**,¹⁷ rigid geometries, instantaneous surface normals, and noise-free coordinate frames. On physical hardware, mechanical interaction involves elastomeric deformation, lubricant film breakdown, micro-slip, optical sensor noise, and thermal expansion in actuator housings. If an argument treats simulation rollouts as direct empirical evidence for a physical force or trajectory bound without measuring the unmodeled dynamics on hardware test benches, the warrant is invalid. When the deployed machine encounters physical contact, unmodeled compliance and sensor latencies cause state trajectories to diverge exponentially, converting a simulated gentle touch into a damaging mechanical impact.

The *proxy metric fallacy* occurs when statistical loss functions, mean average precision, or cumulative reward are presented as evidence for physical safety without measuring the hazardous states and response path. An object detector can score well on a validation benchmark while missing a rare obstacle at the wrong time. Neither an average score nor a $P_{99.99}$ percentile establishes a worst-case physical bound. In a vehicle traveling at 19.2 m/s, a 1.2 s delay before braking adds 23.0 m of travel if speed remains approximately constant. The release argument must test that causal path and its declared envelope.

The *unmonitored condition* arises when a conditional verdict depends on a premise that the deployed machine cannot establish at the required time. This cell, for example, must know whether ambient temperature and payload remain in the signed ranges of table 32.1. A collaborative task that relies on operator attention also needs an evidenced way to assess that reliance and a response when attention is unavailable; a particular gaze sensor or motor-power cutoff does not follow from the premise alone. Unknown or false standing conditions refuse tracking permission, and the independent path executes the assessed fallback within its validated response budget.

The *normalization of deviance* occurs when safety margins are degraded over time because non-catastrophic edge-case anomalies are observed without triggering argument invalidation or formal re-review (Leveson 2011; Vaughan 1996) (32.1).¹⁸ In deployed fleets, machines routinely encounter minor boundary excursions, such as transient 40 ms communication jitter spikes on an internal bus, momentary false-positive sensor drops, or sub-threshold tracking errors during high-speed maneuvers. Because these anomalies did not immediately cause a collision or mechanical fracture, engineering teams gradually treat the anomalies as acceptable operational noise rather than evidence

¹⁷ | **Coulomb Friction Idealization:** Classical Coulomb friction models assume a constant kinetic friction coefficient that is independent of contact velocity, surface micro-roughness, and lubricant contamination. Real physical interfaces exhibit Stribeck velocity-weakening effects, stick-slip limit cycles, and elastomeric shear deformations that alter traction under dynamic loads. Substituting idealized dry friction models for measured physical contact mechanics produces simulation trajectories that diverge drastically from hardware behavior during boundary manipulation.

¹⁸ | **Normalization of Deviance in High-Consequence Systems:** The Presidential Commission on the Space Shuttle Challenger Accident (Presidential Commission on the Space Shuttle Challenger Accident 1986) revealed that engineering management repeatedly observed secondary O-ring erosion on previous launches and misclassified it as an acceptable operational trend. This incremental erosion of safety margins exemplifies Vaughan's normalization of deviance (Vaughan 1996), where anomalous physical signals are rationalized into routine background noise. In physical AI, treating transient sensor dropouts or bus latency spikes as benign operational quirks inevitably invites catastrophic failure when unmodeled disturbances strike.

of broken warrants. Software thresholds are widened, safety margins are trimmed to improve task throughput, and warning alarms are suppressed. Over months of field operation, the gap between the verified safety envelope and actual machine behavior widens until a routine operational disturbance exhausts the remaining physical margin.



War Story 32.1: Challenger and experience as a boundary (1986)

Context: NASA Space Shuttle Solid Rocket Booster (SRB) field joints sealed by dual fluoroelastomer O-rings (Presidential Commission on the Space Shuttle Challenger Accident 1986). No ML or software appears in this chain, but the adjudication failure it records is the exact failure mode this chapter exists to prevent in AI systems: treating an unverified Operational Design Domain (ODD) as an acceptable risk.

Mechanism: Ambient launch temperature at Pad 39B was 36°F, plunging the SRB joint temperature far below the certified minimum qualification baseline of 53°F. The frozen O-rings lost their elastic resilience. Upon ignition, rapid pressurization caused the joint to flex open; the stiffened O-rings failed to expand and seal the gap in time, allowing 3100°C combustion gas to escape.

Impact: Catastrophic in-flight structural breakup resulting in the total loss of the STS-51-L orbiter and all seven crew members.

Response: Strict mathematical and physical ODD boundary tripwires were enforced. Mechanical field joints were redesigned with integrated heaters to guarantee temperature constraints regardless of the external environment.

Systems lesson: Shifting the burden of proof from *proving safety within evidence* to *proving impending disaster without prior failure data* is fatal (the “normalization of deviance” (Vaughan 1996)). For physical AI, operating outside an empirically certified ML ODD without hardware evidence is not an act of calculated risk—it is operating in an unmonitored state where physical laws will ruthlessly enforce reality.

The 2018 Uber Advanced Technologies Group (ATG) collision in Tempe (section 2.7) shows why those premises need scrutiny. The factory Volvo forward-collision warning and automatic emergency braking were disabled during ADS operation over interference concerns. ATG’s ADS suppressed emergency action for one second to reduce false positives and allow operator takeover, while oversight of operator attentiveness was inadequate (National Transportation Safety Board 2019). A release review should have asked what would brake when neither the ADS nor the operator acted, and what evidence supported that takeover premise. An independently validated brake or engagement monitor is a possible counterfactual design response, not an NTSB finding that a particular gaze-to-actuator interlock was required. The release manifest must bind the chosen response and its tested physical limits.

32.9 The Release Manifest

A release verdict needs a signed artifact that records the decision, physical conditions, and expiration. Table 32.4 gives a 13-field illustrative manifest. The signed GSN graph identifies its evidence leaves; `calib_trace_pointers` binds the calibration references, and `fault_manifest_root_hash` binds a canonical aggregate of the signed fault records from section 30.7. Evidence links must resolve to the reviewed record versions before permission is granted.

Table 32.4: Illustrative cryptographic release record schema: Thirteen fields bind the reviewed images, evidence graph, fault-ledger root, verdict, standing limits, and lifecycle budget. A new signed version can supersede a record only after the current physical conditions are checked; the hardware root anchors keys rather than storing all mutable hashes.

Field Identifier	Data / Representation Type	Physical SI Unit / Invariant	Hardware Storage Substrate	Architectural Verification & Trip Invariant
claim_manifest_id	UUID-128 / Hex Digest	Top-level safety claim binding	Secure Flash (Bank A/B)	Cryptographically linked to GSN root node hash
gsn_topology_digest	SHA-384 Hash (48 bytes)	Complete CAE argument graph	Secure Flash / TPM NVRAM	Verified against lead adjudicator digital signature
policy_weights_sha384	SHA-384 Hash (48 bytes)	Neural network model weights	Signed manifest in secure flash	Mismatch refuses release
enforcer_bitstream_hash	SHA-384 Hash (48 bytes)	FPGA real-time safety logic	Signed manifest in secure flash	Mismatch refuses release
rtos_kernel_hash	SHA-384 Hash (48 bytes)	Microkernel image & drivers	Secure Flash (A/B)	Verified by hardware Root of Trust at power-on
calib_trace_pointers	Signed graph leaf references	Dynamometer / load cell certificates	Signed manifest in secure flash	Missing or stale evidence refuses unless separately signed restricted case covers it
fault_manifest_root_hash	SHA-256 digest of canonical signed-record ledger (32 bytes)	Verified HIL fault record ledger (section 30.7)	Signed manifest in secure flash	Missing, unauthenticated, or mismatched ledger refuses
verdict_classification	Enum (UNCOND, COND, REFUSE)	Granted operational envelope	Signed manifest in secure flash	Verified enum selects permitted envelope
standing_conditions	Struct Array (Premise, Rate, Trip)	Physical sensor trip thresholds	FPGA Block RAM / Real-Time MCU	Worked-cell 1000 Hz check; unknown requests assessed stop
telemetry_drift_budget	Signed task-specific error limits and review triggers	Tracking and timing drift	Signed manifest; trace in NVRAM	Bound violation revokes; percentile trend prompts inspection
verdict_expiry_timestamp	UTC Timestamp (ISO 8601)	Seconds ($s \leq t_{\text{expiry}}$)	Tamper-Proof RTC / Crypto-Timer	$\text{RTC} > t_{\text{expiry}}$ voids operating permit automatically
lifecycle_budget	Signed hours/cycles pair	$T \leq T_{\text{max}}$ and $N \leq N_{\text{max}}$	Signed manifest; trusted counters	Exceeded or invalid counter refuses motion
adjudicator_signature	Key ID plus Ed25519 signature	Technical approval of bounded claim	Signed manifest in secure flash	Verify under authorized, nonrevoked adjudicator key

The compiled result is the **cryptographic release record** (or **release manifest**), the formal, immutable engineering artifact that cryptographically binds this verification ledger to the signed verdict of the lead systems adjudicator.¹⁹

Figure 32.6 places authentication and physical-standing checks before any operating permit. The hardware root anchors an authorized verification key, while protected storage carries the signed manifest and evidence references. A changed weight file, enforcer bitstream, or fault-ledger root refuses permission. Successful boot authentication is necessary but insufficient: current calibration, brake condition, ODD membership, and the signed verdict must also pass before the independent enforcer admits motion.

The formal procedure for attesting hardware silicon state, verifying cryptographic manifest signatures, validating firmware image hashes, and executing pre-flight standing condition audits prior to energizing motor power stages is formalized below:

Substrate and signed bytes: The hardware root of trust anchors an authorized, revocable adjudicator verification-key chain. Protected storage holds the signed manifest and signed evidence graph. Serialize versioned manifest fields in a canonical byte order, including the adjudicator key

¹⁹ | **Cryptographic Manifest and Silicon Attestation:** OTP/eFuse storage can anchor a verification key or its digest. Mutable signed manifests and image hashes reside in protected storage; TPM PCRs accumulate measured boot state rather than directly storing a verdict. On failed validation, revoke permission and execute the assessed stop. Bank B is only an eligible software fallback if it has its own valid signature, release case, and current physical standing conditions; restoring software cannot restore worn brakes.

Downstream: Cryptographic release manifests enforce runtime configuration locking against field drift in section 34.8.

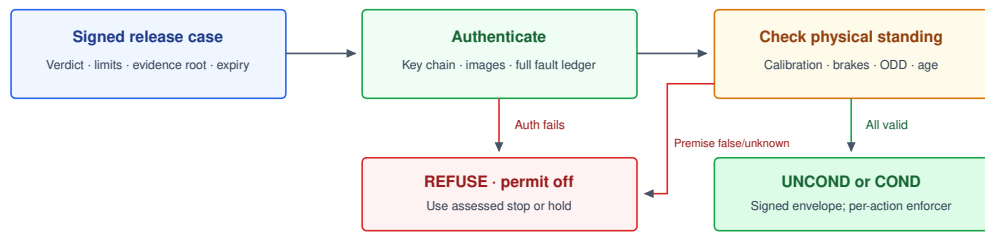


Figure 32.6: Fail-closed release pipeline: The signed verdict and evidence root pass key, image, and complete fault-ledger authentication, then current calibration, brake, and ODD checks. Either check can refuse permission. Only a valid signed full or separately signed restricted case reaches per-action enforcement. Attribution: Textbook production team (CC BY-NC-ND 4.0).

ID but excluding the signature bytes; sign the SHA-384 digest of those bytes with Ed25519. This is Ed25519 over a digest message, not the distinct Ed25519ph scheme. Verify the key chain and revocation state before trusting the signature. The individually signed `FaultExecutionRecord_t` records from section 30.7 are serialized in a declared order; the release producer computes SHA-256 over the complete canonical ledger and binds that digest as `fault_manifest_root_hash` in the signed manifest. The verifier checks each record signature, ordering, completeness, and root against the reviewed evidence index. A digest alone establishes neither record validity nor test adequacy.

Fail-closed authorization protocol: Begin with `Permit_gate = 0` and the physical plant in its assessed nonoperating state. All returned decisions use the signed enum UNCOND, COND, or REFUSE; an implementation may log a reason code separately.

1. Verify the adjudicator key chain and revocation status, canonical manifest signature, expiry, signed operating-hour and cycle budgets, and every active image digest. Any invalid, missing, or expired item returns REFUSE. A different software bank requires its own signed case and a new check of physical conditions.
2. Authenticate the complete fault ledger and calibration references against the signed graph. A missing ledger, bad record signature, or root mismatch returns REFUSE, even if the manifest says UNCOND. Stale calibration also returns REFUSE under the full case. It can support COND only when a *separately signed* restricted case explicitly covers that calibration state and all of its evidence and limits still validate.
3. Check the state-dependent standing conditions, including brake test, separation, timebase, and monitoring health. KNOWN_FALSE and UNKNOWN both return REFUSE while retaining distinct logged causes. Apply the approved stop or hold; torque removal alone is not a universal moving-load stop.
4. If every check passes and the signed verdict is UNCOND, load the signed full envelope and return UNCOND. If every restricted condition passes under a signed COND case, load its signed reduced envelope and return COND. A signed REFUSE case or any other state returns REFUSE. In both operating modes the learned policy only *proposes* actions; the independent enforcer checks each candidate and controls actuator permission. Force trip thresholds must include measured detection and response headroom below the contact criterion.

Table 32.5: Release authorization truth table: A restricted mode is a distinct authenticated claim, never an unsigned downgrade of the full case.

Signed case	Evidence and standing checks	Restricted case covering stale calibration	Gate result
UNCOND	All valid and current	Not needed	Full <i>proposal</i> envelope under enforcer; UNCOND
COND	All restricted evidence valid and current	Included if calibration stale	Signed restricted envelope under enforcer; COND
UNCOND or COND	Fault-ledger root/signature mismatch, expiry, image mismatch, or unknown physical premise	Any	Permit off; REFUSE
UNCOND	Calibration stale	No	Permit off; REFUSE
Separately signed COND	Only the named calibration is stale; restricted evidence and limits valid	Yes	Restricted envelope; COND
REFUSE	Any	Any	Permit off; REFUSE

Systems Perspective 32.3: The silicon root-of-trust invariant

The gate outcomes in table 32.5 depend on an authenticated case and live standing checks. A hardware root of trust anchors authorized verification keys; the signed safety case and its changing evidence hashes live in protected storage. Operating permission requires validation of both the signature and the current physical standing conditions.

Checkpoint 32.6: Cryptographic release manifest and silicon root of trust

Before examining provenance tracking and invalidation triggers, verify your understanding of hardware-anchored release manifests:

- ☐ **Hardware Root of Trust verification:** Can you explain how the authorized key chain and image digests are checked before the drive is permitted to energize, and which protected memory the implementation uses?
- ☐ **Envelope trichotomy and epistemic gaps:** Can you distinguish between known-false and unknown envelope states, and explain why unmeasured momentum demands treating epistemic uncertainty as a safe-state trigger?
- ☐ **Decaying cryptographic leases and parameter drift:** Can you describe how hardware lifecycle counters and telemetry covariance drift budgets automatically revoke operating permits without software changes?

Every evidence leaf needs an identifier for its raw trace, instrument, calibration, configuration, and test script. A $P_{99,99} \leq 1.8$ ms measured tail must be labeled as a percentile; a hard response claim needs a separate justified bound. The chosen 50 N contact criterion needs load-cell time series for the specified body-region surrogate and geometry. Linked records let an auditor repeat the test and identify where the claim stops applying.

The release record names the configuration and physical limits for which its verdict applies. A changed policy weight file, real-time priority, driver, or unverified component revision requires a new case or an explicitly approved equivalent. For this illustrative cell, signed wear limits may include backlash below 0.10° and brake lining thickness above 2.0 mm; the actual limits need measured stopping and load evidence. Monitored ODD refusal conditions include ambient temperature above 40°C and payload above 12.0 kg, consistent with table 32.1. A voltage floor such as 21.6 V belongs in the signed standing conditions only if drive and brake response have been validated there. When a measured condition crosses a signed limit, the permission path revokes tracking at its specified detection and response time and commands the assessed fallback. Unmeasured conditions remain unknown and cannot establish a valid operating verdict.

Field telemetry records torque residuals, loop timing, and enforcer trips for drift review, while the permission path checks the signed live thresholds at the cadence the plant requires. Scheduled inspections may compare braking and tactile measurements with the release baseline. A measured increase in brake response beyond the signed clearance budget or stale calibration revokes the current verdict. Restricted motion is possible only if a separately signed COND case covers that physical state; otherwise the plant uses its assessed stop and awaits maintenance and re-adjudication. A percentile trend can prompt inspection but is not a substitute for a hard stopping bound.

The signed release record binds the reviewed configuration and evidence to the permitted envelope. Runtime monitors and scheduled inspections can revoke that permission as hardware wears; a software rollback alone cannot restore a changed plant (32.4).



Systems Perspective 32.4: Staged fleet updates across joint wear

Releasing software updates and deploying neural locomotion policies across a fleet of bipedal humanoids exposes the fragility of fleet-wide assumptions:

- **Mechanical Heterogeneity across Production Units:** Joint friction, compliance, and backlash differ across units and change with use. A policy evaluated on the factory testbed cannot inherit the same release verdict on a worn warehouse unit without a unit-specific standing check and evidence that the shared limits still hold.
- **Canary Deployment with Dynamic Stability Gating:** A staged software update can expose a small, separately authorized cohort in shadow or restricted mode while collecting joint tracking and balance evidence. A worsening measured trend prompts inspection or revocation against signed limits; a percentile trend alone cannot establish a hard fall-prevention bound. Reverting the software requires its own valid signed case and a fresh check of the current plant.
- **Non-Stationary Wear Invalidation Triggers:** Scheduled diagnostics can measure sole friction and joint backlash against the unit's validated limits. If a wear limit is crossed, the dynamic-running permit is revoked. Slow walking is available only under a separately signed restricted case covering that measured state; otherwise the assessed stop remains in force until maintenance and re-adjudication.

32.10 Fallacies and Pitfalls

Fallacy: *Statistical coverage substitutes for deterministic architectural warrants.*

A team presents millions of simulated hours and thousands of incident-free cycles as its release argument. The record supplies evidence about the tested conditions, but demonstrating a catastrophic failure rate as low as 10^{-6} per hour can require physical exposure beyond the project budget. Simulation also retains the assumptions of its physical models, such as idealized Coulomb friction or infinitely rigid gearboxes. The release case must explain how architectural enforcement—such as independent hardware watchdogs that physically cut power—constrains harmful policy behavior, and it must provide evidence for that mechanism under its stated assumptions. Empirical performance and architectural warrants answer different questions; a large trial count cannot replace the missing argument that connects actuator authority to absolute physical limits.

Fallacy: *A release verdict is a permanent certification of safety.*

A machine passes validation and enters production under a signed safety case. Over time, brake pad wear lengthens stopping distances, kinematic calibration drifts due to thermal cycling, and optical contamination degrades sensing. The software can remain unchanged while the physical assumptions supporting the release fail. Operating authority must therefore remain conditional on the verified physical configuration and limits. The release record needs maintenance requirements and invalidation triggers tied to relevant lifecycle counters or measured drift, with defined restrictions when those conditions fail. A signature records responsibility for a bounded release decision; it does not make the tested body's hardware capabilities permanent.

Pitfall: *Relying on unstated environmental assumptions to support a safety claim.*

A manipulation cell verifies stopping performance under nominal lighting and clean test conditions. In the field, dimmer light lengthens camera exposure from 5 ms to 30 ms, adding 25 ms of latency to the perception pipeline that the original stopping budget omitted. At a Cartesian velocity of 2 m/s, this unmodeled delay adds 50 mm to the stopping distance, potentially violating the ISO/TS 15066 safety envelope. The mechanical brake itself still performs as tested, but the total sensing-to-response time no longer matches the release assumption. The safety case must state the environmental conditions on which its claims depend and explain how operation remains within them. Where monitoring can detect a violation, it must trigger the defined restriction or fallback; an unstated assumption supplies neither a testable release condition nor a usable runtime boundary.

32.11 Summary

Production release binds a claim-argument-evidence case to a measured operational design domain, a signed configuration, and a permission path that can revoke motion. The signature establishes the authorized case; runtime checks determine whether its physical premises still hold.



Key Takeaways: Deployment release and defensible safety cases

- **Bound the claim:** The ODD names mass, friction, speed, thermal state, and contact geometry for which evidence was gathered.
- **Bind evidence to the configuration:** The case needs calibrated traces and reproducible fault records, with a signed root that authenticates their reviewed versions.
- **Keep permission independent:** In both UNCOND and COND, the model proposes; the enforcer admits each action under the signed envelope and validated plant assumptions.
- **Fail closed:** Bad signatures, a failed fault-ledger root, expired evidence, or false or unknown standing conditions return REFUSE. A restricted mode needs its own signed case.
- **Retest wear:** Brake and traction assumptions need scheduled checks and signed thresholds. The 1000 Hz monitor and any statistical drift trigger in this chapter are design examples, not universal release rules.

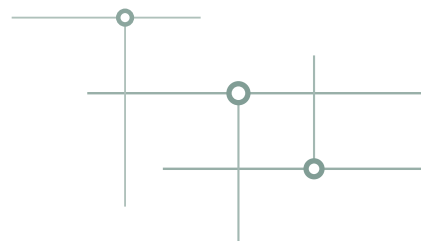
A release decision remains valid only while its evidence, configuration, and physical standing conditions remain valid. On invalidation, the permission path withdraws tracking and invokes the assessed fallback; a new signed case is required to resume.



What's Next: From certified release to the physical frontier

With the cryptographic release manifest in place, **Part IV (Governing the Machine)** completes the operational governance framework developed across this book. Yet even the most rigorous empirical release process operates within fundamental epistemic boundaries. In chapter 34, the curriculum concludes with an overarching synthesis of physical AI systems engineering: confronting the astronomical exposure wall of open-world deployment, examining the epistemic limits of empirical testing, and framing the physical AI frontier across the systems discipline.

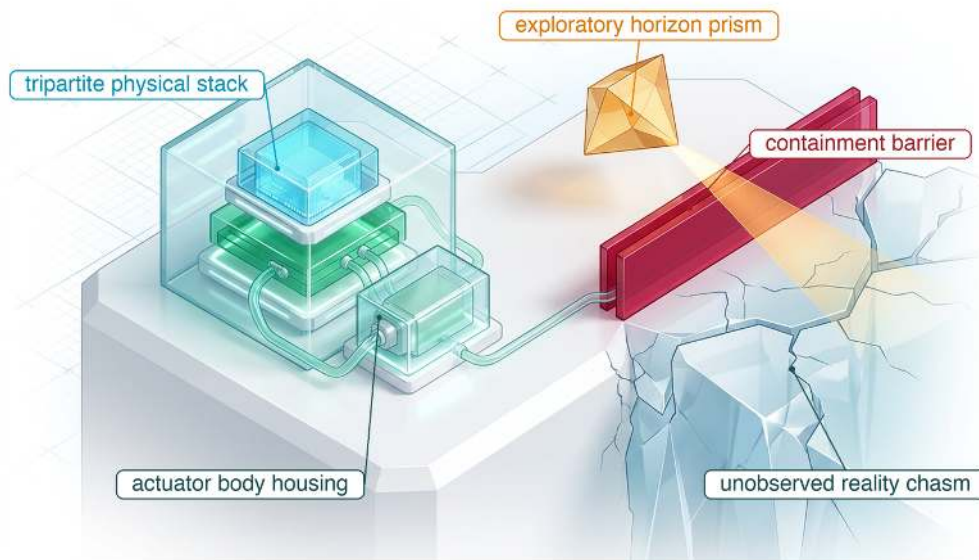
SYNTHESIS



The Epistemic Frontier

DRAFT

The Epistemic Frontier

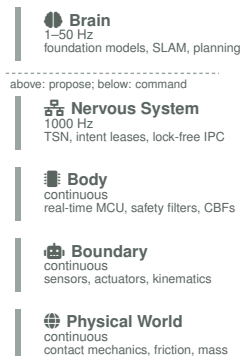


- 34.1 Epistemic Limits
- 34.2 What the Method Cannot Establish
- 34.3 A Residual-Claims Register
- 34.4 Failures Without Detectors
- 34.5 What Would Change These Answers
- 34.6 Generalizing to Unfamiliar Systems
- 34.7 Architectural Synthesis Across the Physical AI Stack
- 34.8 Epistemic Governance
- 34.9 Fallacies and Pitfalls
- 34.10 Summary

Purpose

What physical failure modes remain invisible to modern runtime detectors, and how must an engineer govern what evidence cannot settle?

An engineering record can depend on a physical condition the machine cannot observe. A stopping claim may assume sufficient friction, while the available sensors cannot establish that assumption before every maneuver. Successful trials do not supply the missing measurement or show machine behavior outside tested conditions. Such gaps require an explicit account of what remains unsupported and how that uncertainty limits operation. A runtime detector can support a safety argument only for failures it can reveal in time for an effective response. Where that evidence is unavailable, engineers must evaluate whether reduced speed or restricted access provides adequate containment, or whether operation should be withheld. These limits belong in the architecture and release decision. The physical AI stack remains accountable only when claims about the brain, nervous system, and body stay within the reach of the evidence supporting them.



Prerequisite: The four foundational physical laws synthesize the opening principles from section 2.8.



Learning Objectives

- Assess which safety claims exceed the evidence available for a specific physical machine
- Evaluate whether a detector can identify a hazard before effective intervention becomes impossible
- Define operational restrictions and closure tests for unsupported claims in the engineering record
- Apply physical budgets, information age, authority separation, and evidence requirements to an unfamiliar machine

34.1 Epistemic Limits

CONSIDER a transport robot whose stopping calculation assumes tire friction $\mu \geq 0.60$. An unmonitored oil patch lowers friction to $\mu = 0.15$; under the same speed and response delay, the friction-limited braking term becomes four times larger. The controller can meet its timing deadline and still overrun the reserved clearance. The missing evidence is the local friction floor before braking begins. Which physical premises can the machine establish in time, and what operation remains justified when it cannot?

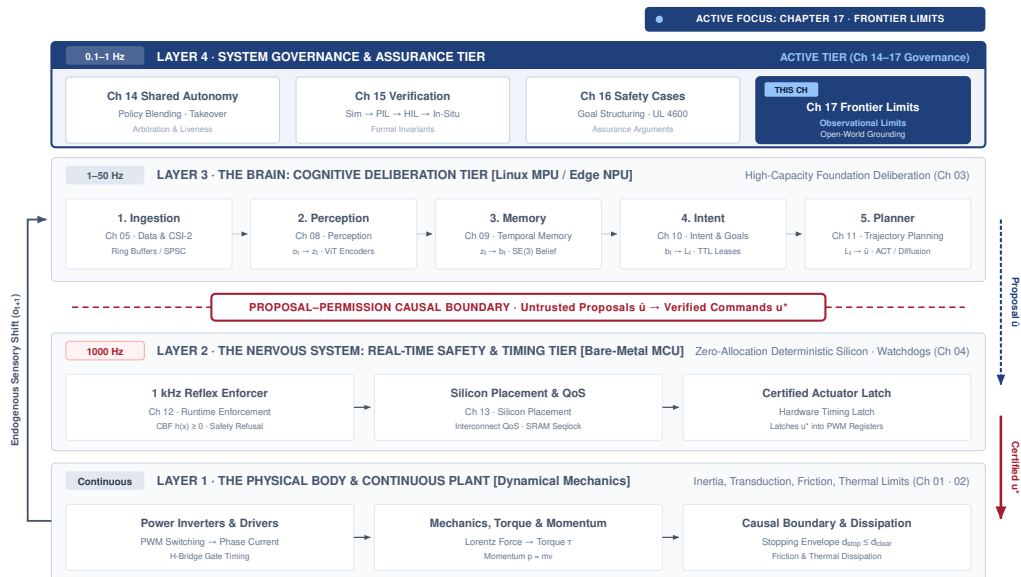


Figure 34.1: Frontier limits architecture focus: The four-tier physical AI systems architecture highlights the system governance and assurance tier, with frontier limits as this chapter's active focus. Read the highlighted box as the edge of the assurance argument, where cataloged failure modes outrun the runtime detectors built to catch them and architectural containment has to carry whatever risk remains.



Definition 34.1: The embodied latency scaling law

Embodied latency scaling law is the physical constraint dictating that scaling neural parameter count yields diminishing operational returns unless inference latency decreases proportionally, because computational delay consumes the mechanical reaction time constant of the physical plant.

1. **Significance:** Establishes that intelligence in physical systems cannot be scaled purely by adding model parameters, because computational delay directly degrades dynamic stability and spatial stopping clearance.
2. **Distinction:** Unlike compute scaling laws in static language modeling where latency affects only token generation speed, embodied latency scaling directly couples computational delay to mechanical reaction time constants ($t_{\text{react}} \geq t_{\text{inference}} + t_{\text{actuation}}$).
3. **Common pitfall:** Deploying multi-billion parameter foundation models directly in the closed-loop control path under the assumption that greater semantic reasoning compensates for an unbudgeted response delay.

As the final chapter of this book, Frontier Limits represents the capstone of Tier 4 (System Governance & Assurance) and the outer boundary of the four-tier architecture stack in figure 34.1. Throughout this text, we have architected an end-to-end discipline designed to take physical AI from experimental prototypes to dependable machines:

- **The Body (Tier 1):** governing continuous plant dynamics, actuator torque/speed curves, friction limits, and thermal dissipation;¹
- **The Nervous System (Tier 2):** providing deterministic real-time communication, microsecond scheduling, hardware isolation, and barrier-certificate enforcement;²
- **The Brain (Tier 3):** generating learned representations, multimodal perception, and long-horizon deliberative planning;³ and
- **System Governance & Assurance (Tier 4):** orchestrating shared authority, whole-system verification, and auditable release safety cases.⁴

Yet every engineering method has a boundary where its guarantees expire. As captured by the Embodied Latency Scaling Law in 34.1, simply scaling the parameter count of foundation models cannot circumvent physical mechanics: computational delay directly consumes the physical reaction time constant of the plant. Furthermore, as governed by Ashby's Law of Requisite Variety (Ashby 1956), a controller cannot regulate disturbances that its sensory architecture cannot perceive. Consider three canonical machines whose line items close perfectly on paper:

1. **A mobile transport platform:** moving at 1.5 m/s, whose stopping claim of 0.49 m assumes tire friction $\mu \geq 0.60$. When unmonitored surface liquid drops friction to $\mu = 0.15$, braking distance quadruples from 0.19 m to 0.76 m, expanding total stopping distance to 1.06 m and breaching the 0.80 m clearance buffer;
2. **An articulated manipulator:** commanded to limit contact force to 40 N, where the guarantee assumes structural stiffness across its end-effector mount. When fatigue micro-cracking introduces unmodeled mechanical compliance, the mount stores strain energy under load and releases it abruptly during surface transitions, delivering an unmeasured 120 N transient peak while joint torque sensors register nominal values; and
3. **A chemical flow regulator:** maintaining line pressure below 200 kPa, whose control loop assumes an undrifted diaphragm pressure transducer. When chemical deposition induces a steady calibration drift of 0.5 kPa/h, the nervous system throttles against a phantom pressure spike or opens into an actual overpressure state of 240 kPa, unaware that its reference sensor has drifted from physical ground truth.

In each constructed case, the machine can leave the assumed envelope while its available telemetry remains nominal. A one-sided 95 percent zero-event bound at 10^{-9} h^{-1} under a stationary Poisson model needs about three billion failure-free hours. This **astronomical exposure wall** makes fleet testing alone inadequate for such a release cadence; it does not make the hours mathematically impossible to accumulate. The safety case must name the missing premise, restrict operation while it remains open, and state what evidence could close it.

The chapter tests that discipline against finite exposure, unobservable friction and material states, and an unfamiliar packaging cell. Each example asks what the machine can measure, what its actuators can still prevent, and which claim must remain open.

1 | **Continuous Plant Dynamics and Mechanical Margins:** As established in section 2.2 and section 4.1, the physical plant imposes hard thermodynamic and kinematic limits. When software detectors fail, mechanical dissipation, passive compliance, and structural margins are candidate protections whose limits still need evidence. Operating without these physical margins risks uncontained kinetic impacts during sensor blind spots.

2 | **Microsecond Determinism and Real-Time Seams:** As detailed in section 8.1 and section 26.1, Tier 2 bridges the asynchronous world of neural inference to the hard real-time demands of actuation. Its watchdogs and fieldbus arbitration support timing only within measured response and interference bounds. Loss of observability requires a separately validated state-dependent fallback while sufficient clearance remains.

3 | **Deliberative Reasoning and Proposal Boundaries:** As developed in section 6.1 and section 22.4, Tier 3 models propose complex behaviors in continuous state spaces. Their performance may improve with compute, while admitted motion remains contingent on external verification and enforcement. Allowing unverified neural trajectories to bypass deterministic enforcement risks driving actuators into kinematic singularities or destructive collisions.

4 | **Assurance Frameworks and Governance Seams:** As structured in section 28.1, section 30.1, and section 32.1, Tier 4 binds authority arbitration, qualification ladders, and goal-structured safety cases into an accountable engineering discipline. This governance tier ensures that empirical verification records and residual assumptions remain auditable across continuous model retrains. Without structured lifecycle governance, silent distribution shift can erode safety margins without triggering telemetry alarms.

Prerequisite: The statistical impossibility of exhaustive testing expands upon the Butler & Finelli limits in section 14.5.



Checkpoint 34.1: Epistemic horizons and unobservable physical dynamics

Before asserting safety claims over high-capacity neural policies in open physical environments, verify your understanding of fundamental epistemic limits:

- ☐ **Observationally indistinguishable world states:** Can you identify failure modes where benign and catastrophic physical conditions produce mathematically identical sensor telemetry ($y_A(t) \equiv y_B(t)$) up to the system's action deadline?
- ☐ **Scale-invariant epistemic blind spots:** Can you explain why increasing neural network parameter counts, dataset size, or inference compute cannot resolve latent physical states that emit zero observable physical energy prior to harm?
- ☐ **Embodied shortcut learning in latent representations:** Can you describe how deep perception models exploit non-causal visual features as proxies for unmeasured physical properties, commanding destructive actions with near-zero softmax entropy?
- ☐ **Proposal-permission architectural containment:** Can you explain why unmodeled physical risks demand structural, out-of-band permission barriers rather than relying on internal policy uncertainty or self-assessment?

Confusing high statistical confidence with physical truth is the quintessential failure mode of physical AI. Ensure all safety-critical invariants are anchored in observable physical mechanics rather than latent policy representations.

34.2 What the Method Cannot Establish

A safety release claim for a physical AI system is not a monolithic assertion. Its necessary links depend on the hazards and protection chosen. The argument must establish the operating domain, measured actuator and stopping bounds, component contracts, and independent permission for learned proposals (section 6.1). Hazards controlled by runtime detection also need measured detector coverage and timely monitoring. Hazards controlled by passive limits or operating restrictions instead need evidence that those measures hold over the declared load, wear, and exposure envelope. Generating extensive evidence for one link cannot compensate for an unsupported premise in another.⁵

Every link in a safety case requires an explicit **epistemic classification**, identifying each premise as measured directly through physical instrumentation, derived analytically from measured premises, assumed on physical or operational grounds, decided by accountable judgment, left unsupported by existing data, or explicitly declared outside the threat model. In the three reference machines examined throughout this text, these classifications reveal the exact boundary where evidence ends. For the wheeled mobile robot, the stopping-distance guarantee relies on an assumed friction coefficient between tire and floor that cannot be measured before braking begins. For the articulated manipulator, the collision-containment argument depends on an assumed material-damage threshold for surrounding equipment, which is an engineering judgment about structural yield rather than a measured property of the controller. For the aerial vehicle, the state-estimation envelope assumes statistical independence between optical flow and inertial sensors, an assumption that collapses under sudden illumination changes or frame resonance. Naming these premises explicitly does not weaken the release record. It establishes the physical envelope within which the release verdict is justified, and outside of which the verdict ceases to apply.

Separating measurement from judgment is fundamental to systems engineering discipline. Empirical testing and statistical characterization can estimate physical exposure, quantify sensor noise, and bound parametric uncertainty, but no measurement can choose an acceptable **residual risk** or decide which individuals in an operational environment must bear that risk. Those choices are accountable human decisions that must be recorded as explicit judgments rather than disguised as empirical findings. In the same way, the **threat model** establishes an absolute boundary on the validity of the record. Empirical testing across nominal fleet operations and accidental component faults provides no evidence regarding intentional adversarial attacks. A safety record demonstrating that a vision pipeline handles sensor noise, lens blur, and partial occlusions provides zero statistical support

⁵ | **Popperian Falsification:** Karl Popper formalized the asymmetry of empirical induction: no finite sequence of safe observations can prove universal safety, whereas a single counterexample refutes it. Operational fleet hours establish survival rates strictly under observed stationary conditions, not out-of-distribution guarantees. Extrapolating zero observed incidents to unbounded environments produces an illusory sense of safety that fails upon encountering unmodeled edge cases.

for claims about resilience against adversarial optical projection, model parameter tampering, or intentional denial of service across the internal communication bus.

Before counting hours, define the event, exposed population, operating conditions, rate target λ , confidence level $1 - \alpha$, and dependence assumptions. For the illustrative 10^{-9} h^{-1} target,⁶ a stationary homogeneous Poisson model gives $P(N(n) = 0 | \lambda) = e^{-\lambda n}$. With zero events, the one-sided upper rate is $\lambda_U = -\ln \alpha / n$; reaching $\lambda_U \leq \lambda$ requires equation 34.1:

$$n \geq \frac{-\ln \alpha}{p} \quad (34.1)$$

Evaluating equation 34.1 for $p = 10^{-9}$ failures/h at 95 percent confidence ($\alpha = 0.05$) yields $n \geq -\ln(0.05)/10^{-9} \approx 3.00 \times 10^9$ hours. Accumulating 3.0×10^9 operating hours without a single observed failure requires approximately 341,744 machine-years of continuous execution (1 year = 8,766 h). For an operational fleet of $M = 100$ physical units running continuously, accumulating this empirical exposure requires $T_{\text{fleet}} \approx 3,417$ calendar years (figure 34.5, panel b). This threshold represents the **astronomical exposure wall**: the mathematical scaling limit where a rare rate target requires exposure inversely proportional to that target, beyond this fleet's proposed release interval.⁷ Figure 34.2 extends that relation across the full range of target rates, and the curve is the argument: because required exposure grows as the reciprocal of the probability being bounded, each additional order of magnitude of claimed safety costs an order of magnitude more operating time than the last.

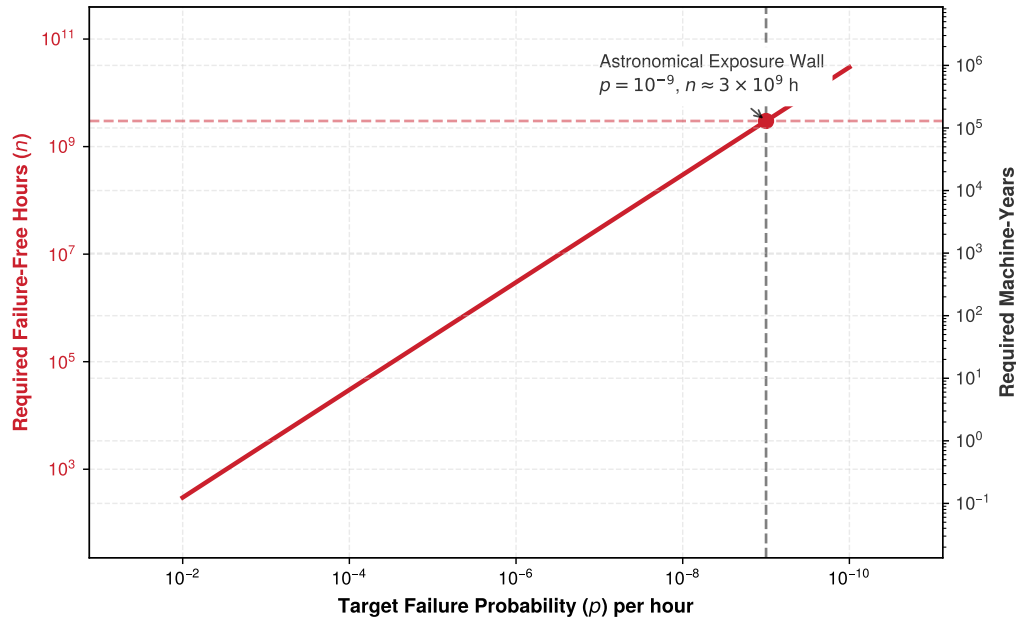


Figure 34.2: Astronomical exposure wall: Required failure-free exposure hours against target failure probability per hour, under a Poisson zero-event model at 95 percent confidence, with the equivalent machine-years on the opposing axis. Certifying 10^{-9} per hour demands roughly three billion failure-free hours, which this fleet cannot accumulate within its 90-day update interval.



Napkin Math 34.1: Fleet exposure and a conditional shield calculation

Problem: One hundred robots operate for 90 days with a shield. No catastrophic system event is observed. What does that exposure establish? Separately, suppose a counterfactual oracle could identify hazardous policy proposals, and a named shield's interception coverage were measured for that hazard class. Would an assumed coverage of 0.9999 suffice for an illustrative 10^{-9} h^{-1} system target?

⁶ | **Scope of the Rate:** The 10^{-9} h^{-1} rate is an illustrative system hazard target here. The IEC 61508 high-demand SIL 4 dangerous-failure band is $\geq 10^{-9}$ to $< 10^{-8} \text{ h}^{-1}$ in the UK HSE functional-safety guide; it does not prescribe this chapter's target for every machine.

⁷ | **Evidence Across Updates:** An updated model may alter the proposal distribution. Earlier data can be pooled only with a justified configuration-equivalence and exposure argument; an update does not automatically erase every observation. The Poisson model also requires a defensible treatment of shared fleet conditions and correlated faults.

Observed system record: Under a stationary homogeneous Poisson model, $100 \times 90 \times 24 = 216,000$ fleet-hours and zero incidents give a one-sided 95 percent upper bound $\lambda_{\text{system},U} = -\ln(0.05)/216,000 = 1.39 \times 10^{-5} \text{ h}^{-1}$. This is $13,870\times$ above the illustrative target. Prevented hazardous proposals are absent from this incident count, so this result cannot be relabeled as a policy-proposal rate.

Separate conditional design case: Imagine a *different*, fully adjudicated proposal log with zero hazardous proposals in 2.16×10^5 hours and a validated counterfactual hazard oracle. Its numerical upper bound is also $1.387 \times 10^{-5} \text{ h}^{-1}$ only because its assumed exposure and count match. If class-specific shield interception coverage were independently established at $c = 0.9999$, and hardware plus common-cause residual were assumed zero for this calculation, the modeled residual bound would be $1.387 \times 10^{-9} \text{ h}^{-1}$, **above** the target. The required coverage in that hypothetical is at least 0.9999279; this is a design target, not measured coverage.

Required proposal evidence at $c = 0.9999$: The proposal-rate bound must be at most 10^{-5} h^{-1} . Zero adjudicated hazardous proposals would require about 299,573 independently modeled proposal-hours, or 124.822 days for 100 units, 34.8 days beyond this 90-day cycle. A policy update needs an explicit configuration-equivalence argument before older proposal evidence can be pooled; a changed policy does not automatically erase every earlier observation.

Systems insight: A shield changes the system claim only for hazard classes it can detect and intercept in time. Proposal frequency, conditional interception coverage, hardware faults, and common-cause failures need separate evidence. Oil or material failure outside the shield's sensing coverage stays in the residual-claims register.

Operational fleet data may also violate the model behind the bound. Fleet-wide updates, shared layouts, and correlated weather can create common-cause events; the observation process and exposure model must account for them. A changed checkpoint cannot silently inherit older evidence, but an explicit configuration-equivalence argument may justify pooling relevant data. Human interventions should remain in the log as visited states and intercepted proposals, with authority flags and counterfactual adjudication where possible. A simulation contributes evidence only for the dynamics and disturbances it actually represents.

The shield calculation is a *conditional* architectural argument, not a multiplication applied to observed system incidents. For a defined hazard class h , let $\lambda_{\text{proposal},h}$ be the rate of hazardous proposed actions before permission, and let $c_h = P(\text{intercept} \mid \text{hazardous proposal}, h)$ be measured interception coverage for that class. With a separately bounded hardware and common-cause term λ_{other} , a model may use equation 34.2:

$$\lambda_{\text{system},h} \leq \lambda_{\text{proposal},h} (1 - c_h) + \lambda_{\text{other}} \quad (34.2)$$

The relation requires a valid proposal oracle, timely detection, actuator authority, and a justified treatment of correlated failures. It gives no credit for an oil patch or brittle material state that the shield cannot observe. The illustrative 90-day proposal record and $c_h = 0.9999$ in the notebook would still miss the 10^{-9} h^{-1} target. No release follows from the arithmetic until each premise is supported.⁸

⁸ | **Physical Trip Path:** Some configured comparator and gate-driver paths can remove commanded torque without waiting for the host scheduler. The pin response, drive behavior, mechanical stopping time, and clearance must be measured for the particular plant. Torque removal alone may permit coasting or gravity-driven motion.



Checkpoint 34.2: Exposure wall and distinct event rates

- ☐ Can you state which event the zero-incident fleet record bounds and why it cannot measure prevented hazardous proposals?
- ☐ Can you identify the stationary Poisson and fleet-dependence assumptions behind $-\ln(0.05)/T$?
- ☐ For a named hazard class, what separate proposal, interception, hardware, and common-cause evidence would justify equation 34.2?
- ☐ Why does the illustrative 90-day proposal scenario with $c = 0.9999$ still miss 10^{-9} h^{-1} ?



Figure 34.3: Distant contact operation: Perseverance’s arm and turret work at a Martian rock outcrop. Communication delay makes the onboard force, torque, weight-on-bit, and tilt checks described by NASA/JPL important to this operation; the photograph does not demonstrate a particular stop bound. Image: NASA/JPL-Caltech/MSSS.

A planetary rover provides a different example. A long Earth–Mars communication delay rules out human correction of a fast drilling contact event. NASA/JPL describes software checks on force and torque, drill weight on bit, and tilt, with settings analyzed and tested for that operation. Those checks depend on their sensing, actuation, and geological assumptions; the cited account does not establish a passive torque clutch or guaranteed arrest before every rock failure (NASA/JPL). Figure 34.3 shows the arm and turret at the rock face where those onboard checks must hold. The photograph fixes the contact geometry and settles nothing about the response time behind it.

34.3 A Residual-Claims Register

An unproven premise remains even if an engineer omits it from a safety report. When empirical testing cannot establish a safety bound, the unproven premise enters a formal **residual-claims register**. This auditable engineering ledger maps every unproven premise or unmonitored invariant to its missing physical observable, its active containment or operating restriction, an accountable human owner, and a predeclared closure test. An engineering team cannot treat an unverified physical assumption as an unwritten risk or temporary convenience. Without these four components, an unresolved claim is an open release claim.

Each entry links the unsupported claim to the specific unmeasurable physical observable. In an embodied system with physical authority over mass, a learned policy fails to settle a safety property because sensors cannot transduce the relevant environmental state, the computing substrate cannot process the measurement within the control deadline, or the physical interaction cannot be observed before contact occurs. An engineer must define the **missing observable** with physical precision rather than statistical abstractions. Characterizing an unknown failure as an “out-of-distribution latent state” provides no actionable signal for an enforcement loop to trigger a physical intervention. By contrast, stating an unmeasured surface friction coefficient, an undetected optical flare across a camera lens array, or an unobservable micro-fracture in an actuator housing identifies the exact physical deficit. Naming the missing observable establishes the boundary where learned inference stops and physical uncertainty begins.

In an analytical example, consider a 450 kg transport robot on depot concrete. Its proposed stopping claim assumes local tire–floor friction that a forward camera cannot establish beneath a clear oil film. With $v = 0.9$ m/s and the example’s 17.1 ms response delay, $d = v\tau + v^2/(2\mu g)$ is about 0.36 m if $\mu \geq 0.12$ is a validated floor. At $\mu = 0.05$, braking alone is 0.826 m and the full stop is about 0.841 m, exceeding the 0.50 m clearance. The proposed speed clamp is therefore conditional on measured or controlled floor conditions; it cannot establish safety on an unbounded oil slick. A lateral limit of 1.0 m/s² also exceeds μg at $\mu = 0.05$.

An open entry requires an **operational response**. For this depot, the operators can inspect and control the floor to support a friction lower bound, maintain physical separation barriers, and admit the 0.9 m/s restricted mode only within that validated envelope. If the lower bound cannot be supported, the release must exclude the route or withhold motion. The firmware limit reduces kinetic energy, but the response delay, achievable braking, and available clearance remain separate premises.⁹

The restricted mode needs an owner able to keep its physical premises true. The drivetrain lead controls the signed velocity and braking configuration; the depot operations manager maintains the inspected floor and separated walkways. Both record the inspection interval and the condition that withdraws permission. A claim about friction cannot be maintained by a firmware speed setting alone.

Every entry needs a **closure test** declared before collecting data. For the depot, a candidate forward oil-film sensor must identify the specified contaminants early enough that the complete detection, permission, and braking path fits the measured clearance. A zero-miss result in 1,000 independent trials gives only a one-sided 95 percent miss-probability upper bound of $1 - 0.05^{1/1000} \approx 0.002991$. Establishing a bound at or below 0.001 under the same Bernoulli assumptions requires at least 2,995 independent, failure-free trials. The trial distribution, distance, latency, false alarms, and friction after detection must be declared. A wheel-slip reflex can limit later motion but cannot prove pre-contact friction. Only a validated end-to-end detector and stopping result could justify lifting the route restriction.

Specifying a closure test forces an engineering organization to determine whether a gap can be resolved with improved instrumentation and testing, or whether it represents a permanent unobservable state. When a closure test requires a physical transducer that is unavailable or cannot meet the response deadline, the register entry remains open. The operational restriction remains until the evidence changes. The danger arises when an organization treats an unclosable entry as a temporary backlog item, assuming that every physical failure mode can eventually be trapped by an onboard runtime detector.

The structured anatomy of these register entries is formalized across representative physical AI archetypes in table 34.1, illustrating how unmonitored hazards are systematically bounded by operational constraints until falsifiable closure tests are satisfied.

⁹ | **Velocity Kinetic Derating:** $E_k = mv^2/2$ decreases quadratically with speed. Stopping distance still depends on a validated braking floor and total response delay; a speed register alone cannot compensate for arbitrarily low friction.

Table 34.1: Residual-claims register matrix: Auditable ledger of unproven physical premises across representative physical AI archetypes. For each open claim, the register defines the missing physical observable, the physical hazard mechanism, the active operational containment enforced by the nervous system or mechanical hardware, the accountable human owners, and the falsifiable closure test required to retire the operational restriction.

Claim and missing state	Hazard mechanism	Active containment and owner	Predeclared closure test
Transport robot: $d_{\text{stop}} \leq 0.50$ m; local μ beneath clear oil is unknown.	At 0.9 m/s and a 17.1 ms delay, the modeled stop is ≈ 0.36 m at $\mu = 0.12$ but ≈ 0.841 m at $\mu = 0.05$.	Restrict speed only where inspection and floor controls support $\mu \geq 0.12$; otherwise use barriers or withhold motion. <i>Owners:</i> drivetrain and depot operations leads.	Characterize an independent forward contaminant sensor and full stop across a declared film distribution; at least 2,995 zero-miss independent trials for a 95% upper miss probability ≤ 0.001 , plus timing and clearance tests.
Ceramic gripper: brittle workpiece voids are not visible before contact.	With $k = 500$ N/mm and jaw speed 2 mm/s, force rises 1000 N/s and reaches a 15 N fracture threshold at 15 ms.	A calibrated 6 N preload, 2.5 N/mm compliant spring, and 3 mm hard stroke would limit the modeled force to 13.5 N if that force path is maintained. <i>Owner:</i> tooling lead.	Inspect internal voids with an independently validated method, or test the passive force path across the specified stroke, temperature, and wear envelope.
Flow regulator: diaphragm zero-point drift is not observable from its own pressure reading.	A biased reading can permit true manifold pressure to exceed the configured limit.	Independently rated pressure relief and scheduled zero checks. <i>Owners:</i> process safety and plant reliability leads.	Compare two independently calibrated pressure channels in kPa, with a justified 5 kPa disparity threshold and tested response. A flow meter would need a validated pressure–flow model.
Indoor drone: visual features disappear during lighting transitions.	Pose error can grow before a visual policy recovers.	Restrict flight to a validated clearance and illumination envelope, with a platform-tested landing or hover response. <i>Owner:</i> flight autonomy lead.	Demonstrate bounded state-estimate error and actuator response over a declared blackout distribution; a proposed event camera alone is not proof.

To ensure auditable tracking of unmonitored invariants across software deployments and physical revisions, safety-critical systems represent open residual claims through a formal, structured data contract (table 34.2). The abstract schema below defines the mandatory fields, scalar types, physical SI units, and structural invariants required for every unresolved physical assumption:

Table 34.2: Epistemic residual-claims register: Concrete data schema for residual claims.

Manifest Category	Parameter & Type	Physical Unit	Invariant & Operational Contract
Manifest Header	system_identifier (str)	—	Unique embodied platform ID (e.g., AMR-450-DEPOT)
Manifest Header	hardware_build_rev (str)	—	Target silicon and chassis revision (HW-REV-3.2)
Manifest Header	software_digest (str)	—	Cryptographic binary hash (SHA-256)
Manifest Header	validity_limit_hours (float)	hours	Configured operating window before review
Residual Claims	entry_id (str)	—	Unique claim tag (e.g., CLAIM-FRIC-001)
Residual Claims	target_safety_claim (str)	—	Explicit safety invariant ($d_{\text{stop}} \leq 0.50 \text{ m}$)
Residual Claims	missing_observable (str)	SI units	Unmeasured physical state (e.g., surface friction μ)
Physical Hazard	governing_law (str)	—	Newton-Euler, Navier-Stokes, Joule heating
Physical Hazard	time_to_harm (float)	seconds	Time from unmodeled transition to irreversible yield
Operational Containment	containment_type (enum)	—	FIRMWARE_PARAM_CLAMP, PASSIVE_MECHANICAL, or OPERATING_RESTRICTION; validate the selected limit
Operational Containment	authority_limit (float)	m/s,N	Hard numeric setpoint ceiling ($v_{\text{max}}, F_{\text{max}}$)
Operational Containment	kinetic_energy_ceiling (float)	J	Maximum allowable kinetic energy under fault ($E_k \leq E_{\text{max}}$)
Assurance & Inspection	proof_test_interval (float)	hours	Scheduled test interval; evidence between tests still depends on the operating restriction
Predeclared Closure	closure_modality (str)	—	Transducer technology or formal method required
Predeclared Closure	sample_trials (int)	count	Case-specific independent exposure (2,995 zero misses for a 95% upper miss rate ≤ 0.001)
Predeclared Closure	confidence_level (float)	—	Declared confidence level (95% in the worked detector test)



Checkpoint 34.3: Residual claims and operational containment

An auditable residual-claims register turns unsupported physical assumptions into explicit operating restrictions with named owners and evidence. Hardware or controlled route exclusion can enforce a restriction. Scheduled inspection supports its standing evidence, with permission revoked when inspection is overdue.

- ☐ **Missing observable identification:** Can you identify the specific unmeasured physical state that prevents an empirical safety case from closing for a given robot, rather than relying on abstract terms like out-of-distribution shift?
- ☐ **Operational containment vs operational advice:** Can you identify who enforces each restriction, when it is checked, and what prevents operation when its premise expires?
- ☐ **Closure test falsifiability:** Can you define the criteria that distinguish a genuine, predeclared physical closure test from an ad-hoc post-deployment evaluation?

34.4 Failures Without Detectors

A runtime detector is itself a physical subsystem. To operate as a **practical detector**, it must satisfy three coupled parameters: comprehensive coverage across the failure distribution, a bounded false-

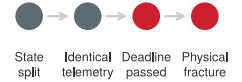
alarm rate, and a detection latency strictly shorter than the physical system’s time-to-harm. If a mobile transport base traveling at 1.5 m/s requires 180 ms of braking actuation to avoid colliding with an obstacle, a perception pipeline requiring 250 ms of sensor aggregation and compute latency is not a detector for that hazard. It is a post-incident logging mechanism. Producing an accurate classification label after crossing an irreversible threshold generates diagnostic evidence but cannot serve as a runtime safeguard.

The fundamental limit of runtime detection arises from observational indistinguishability. Consider two physical states of the world, S_A and S_B . In state S_A , a commanded actuator trajectory $u(t)$ is safe. In state S_B , the identical trajectory $u(t)$ causes structural damage or human injury, requiring the nervous system to intervene and execute a stopping reflex. The dynamics of the body impose a hard deadline t_{deadline} , determined by actuator response time and system momentum, after which physical contact or mechanical yield cannot be prevented. If the telemetry history $y(t)$ provided by onboard instrumentation is identical for S_A and S_B across the entire interval $t \leq t_{\text{deadline}}$, no algorithm operating on those observations can reliably distinguish the benign state from the hazardous one. Identical traces rule out a state-specific response before that deadline. A conservative action chosen for both states or a passive mechanical limit may still protect them, if its own clearance and force assumptions are valid.

Definition 34.2: Observational indistinguishability

Observational indistinguishability is the physical condition where two distinct world states—one nominal (S_A) and one catastrophic (S_B)—generate identical pre-harm sensor telemetry ($y_A(t) \equiv y_B(t)$) up to the system’s physical action deadline ($t_{\text{deadline}} = t_{\text{harm}} - t_{\text{act}}$).

1. **Significance:** Without distinguishing physical energy prior to the action deadline, timely algorithmic differentiation is physically and mathematically impossible. Scaling neural model parameters, increasing dataset size, or accelerating inference compute cannot separate states that emit zero distinguishing physical energy before the time-to-harm.
2. **Distinction:** Unlike epistemic uncertainty or high classifier entropy (which signals that the model knows it lacks evidence), observational indistinguishability often pairs with high output confidence and near-zero entropy because the neural network maps corrupted observations into nominal latent manifolds.
3. **Common pitfall:** Relying on software anomaly detectors or VAE reconstruction loss to catch detectorless degradation. When sensor physics blinds the transducer—through piezoresistive crystal creep or pre-contact subsurface voids—the anomaly detector consumes the corrupted telemetry and fails silently.



Identical pre-contact telemetry prevents software intervention before irreversible physical fracture occurs.

In learned perception-action policies, this detectability limit manifests as silent distribution shift and embodied shortcut learning. High-capacity neural networks, such as Vision-Language-Action (VLA) models (Brohan, Brown, Carbajal, Chebotar, Dabis, et al. 2023; Brohan, Brown, Carbajal, Chebotar, Chen, et al. 2023) or deep imitation policies (Zhao et al. 2023), minimize empirical loss by discovering opportunistic statistical correlations across high-dimensional demonstration datasets. In embodied tasks, unmeasured physical properties—such as contact friction, material yield strength, or payload inertia—are unobserved latent variables. Deep models routinely learn to exploit non-causal visual features (such as ambient illumination angles, floor specular highlights, or camera frame-rate synchronization) as computational proxies for these physical properties. When operating under open-world distribution shift, these non-causal correlations collapse. However, because high-dimensional non-linear encoders compress the out-of-distribution observation $\mathbf{x}$ directly into a nominal latent manifold $\mathbf{z} = \phi(\mathbf{x}) \in \mathcal{Z}_{\text{nom}}$, the policy outputs action distributions $\pi(\mathbf{a} | \mathbf{z})$ with near-zero softmax entropy ($H(\pi) \approx 0$, indicating absolute mathematical certainty) and minimal ensemble disagreement. The model commands aggressive, damaging physical trajectories while its internal representations indicate nominal performance. Crucially, any epistemic uncertainty estimator implemented in software—whether evaluating latent Mahalanobis distance, Monte Carlo dropout variance, or autoencoder reconstruction residuals—evaluates uncertainty using the very same corrupted representations $\mathbf{z}$. Because the confidence estimator and the policy share the broken

perceptual representations, internal software metrics cannot serve as an independent detector of shift. This failure mode is termed **embodied shortcut learning and latent epistemic shift**: the process where a high-capacity neural policy exploits non-causal statistical correlations in training demonstrations as computational proxies for unmeasured physical states, causing the model to command destructive actions with maximal confidence while rendering internal uncertainty metrics completely blind to the failure. The learned Brain is fundamentally blind to its own shortcut failure, necessitating architectural proposal-permission separation to enforce physical containment.

Physical sensing hardware exhibits an identical lack of identifiability over longer operating durations. In a fluid regulation system, an inline thermal mass flow sensor subjected to surface fouling or thermal aging drifts downward at an analytical example rate of 0.02 L/min per 100 h of operation. Over the same timescale, an incipient pipe constriction or supply pressure drop produces an identical, genuine reduction in mass flow. A single measurement channel measuring heat dissipation reports a declining scalar signal $q(t)$. The nervous system receives the telemetry stream, but no algorithm processing that single signal can identify whether the physical process has changed or the transducer calibration has degraded. Both physical conditions generate the exact same observation sequence. Without an independent reference transducer or a direct differential measurement, the drift is undetectable at runtime.¹⁰

Mechanical manipulation gives a faster example. A sound ceramic workpiece tolerates 50 N in the constructed case, but a hidden void lowers the fracture threshold to 15 N. Surface sensing does not distinguish them before contact. With rigid-jaw stiffness $k = 500$ N/mm and closing speed $v = 2$ mm/s, the modeled force slope is $kv = 1000$ N/s; the threshold is reached in 15 ms. The stated force-filter, bus, and MCU delays total $15 + 2 + 1 = 18$ ms before a new drive command can begin changing current. A winding $L/R = 24$ ms is one decay time constant, leaving about 37 percent of the initial current in the ideal exponential model; it is not a measured cutoff or complete stop. This reactive path cannot prevent the modeled fracture without a different early measurement or passive force limit.

At the 18 ms first possible command change, the unmitigated rigid-jaw model has reached 18 N, already past the 15 N threshold. At 42 ms of continued closure it reaches 42 N; that is an illustrative no-intervention trajectory, not the motor force after one time constant. This is a detectorless-for-this-path failure: the observation and actuation chain cannot alter force before the workpiece's deadline. A different sensor or a validated passive force path is required.¹¹

For the constructed passive alternative, a 6 N preload and 2.5 N/mm spring over a mechanically limited 3 mm stroke give $6 + 2.5 \times 3 = 13.5$ N < 15 N. The stiffness ceiling in equation 34.3 holds only if the detent carries the entire contact load, the stroke cannot exceed 3 mm, and wear and dynamic impact tests support the model.

$$k_{\text{spring}} \leq \frac{F_{\text{allow}} - F_{\text{preload}}}{\Delta x_{\text{stall}}} \quad (34.3)$$

¹⁰ | **Piezoresistive Transducer Drift:** Mechanical stress relaxation, surface oxidation, and temperature cycles alter the piezoresistive coefficient of silicon strain gauges over extended campaigns. Monolithic software filters cannot separate slow baseline voltage drift from true process pressure increments, creating an unobservable state transition. Preventing catastrophic overpressure therefore requires independent physical zero-point proof-tests or dual-dissimilar sensing architectures.

Prerequisite: Failures undetectable by sensory telemetry challenge the observation contracts formulated in section 16.7.

¹¹ | **Contact Boundary:** Rigid impact and friction transitions can invalidate a smooth free-motion model. Peak contact force and component damage require measured stiffness, damping, geometry, and actuator response; neither a controller nor a passive part alone supplies a universal bound.



Checkpoint 34.4: Contact fracture deadline and compliance

(a) Rigid-jaw reaction deadline

500 N/mm $\times$ 2 mm/s = 1000 N/s



15 N fracture threshold at 15 ms

First command change at 18 ms

24 ms L/R = one current time constant

42 N at 42 ms only if closure continues

(b) Passive force path

Preload 6 N; spring 2.5 N/mm

Rigid path: 15 N reached at 15 ms

Reactive path is late for this workpiece.

$6 + 2.5 \times 3 = 13.5$ N

Requires ≤ 3 mm stroke and full load path through the validated compliant element.

The specified reactive path misses the 15 ms fracture deadline; the passive example requires a validated load path and stroke limit.

- **Observational indistinguishability and pre-contact blindness:** Can you explain why internal structural voids yielding identical pre-contact telemetry ($y_A(t) \equiv y_B(t)$) prevent any software-based perception policy from classifying the defect before impact?
- **Reaction deadline:** Can you distinguish the 15 ms fracture deadline, 18 ms first command change, and 24 ms winding time constant?
- **Passive containment:** Which stroke, load-path, wear, and impact assumptions must hold for the 13.5 N spring calculation?

Legged robots face a related uncertainty on rubble: a camera can show the shape of a block without measuring how it will bear a footstep. A contact response therefore needs measured joint, sensing, and support limits; a photographed course alone cannot establish millisecond reflex timing, torque capacity, or a safe fall. Passive foot mechanics may help under tested load conditions. Figure 34.4 shows the geometry that makes this hard, a field of loose and stacked cinderblocks in which every candidate foothold carries a load-bearing capacity that no camera reports.

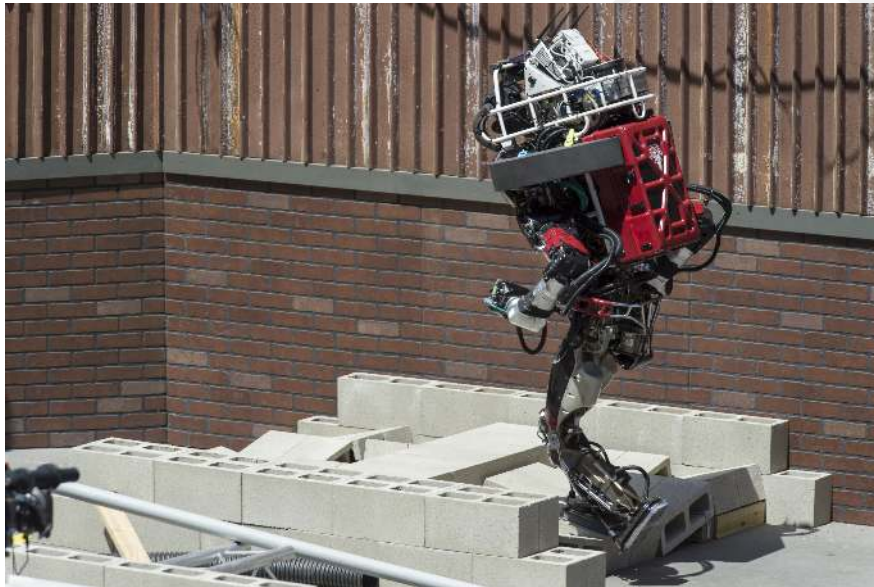


Figure 34.4: Rubble support uncertainty: A humanoid test platform steps among cinderblocks at the 2015 DARPA Robotics Challenge. The image shows irregular support geometry; it does not measure block strength, controller rate, joint torque, or recovery time. U.S. Navy photograph (public domain).

📖 War Story 34.1: Deepwater horizon blind shear ram failure (2010)

Context: On April 20, 2010, the Macondo well in the Gulf of Mexico suffered a catastrophic blowout on the *Deepwater Horizon* drilling rig. The ultimate physical containment safeguard was the subsea Blowout Preventer (BOP)—a 450-ton stack of hydraulic valves and blind shear rams situated 1,500 m below sea level (U.S. Chemical Safety and Hazard Investigation Board 2014).

Mechanism: Following explosions on the surface platform, the BOP automatic deadman system triggered to close the blind shear rams and sever the drill pipe. However, high-pressure wellbore flow had placed the drill pipe into severe axial compression while the well was shut in, inducing elastic-plastic Euler buckling ($P_{cr} = \pi^2 EI / L_{eff}^2$). The pipe buckled laterally, pinning itself against the BOP casing wall. The shear rams had been certified assuming a concentrically centered pipe; when the V-shaped blades advanced, they crushed and folded the off-center pipe against the wall rather than cleanly shearing it, jamming the ram blocks and preventing elastomeric seal closure.

Impact: Containment failed completely. The blades punctured the drill pipe, opening a massive flow path that allowed hydrocarbons to surge unchecked, killing 11 workers, sinking the rig, and discharging 4.9 million barrels of crude oil into the Gulf of Mexico (U.S. Chemical Safety and Hazard Investigation Board 2014).

Response: The U.S. Chemical Safety Board recommended changes to blowout-preventer design, testing, and oversight. Its investigation also documented battery and wiring failures. Recommendations should not be presented as an already enacted universal mandate.

Systems lesson: The disaster represents the catastrophic manifestation of an unobservable, detectorless physical state transition ($y_A(t) \equiv y_B(t)$). The investigation found that the pipe was off center when the shear ram acted, so its qualified cutting geometry did not cover the actual condition. The broader lesson is to test protection across the physical states it may encounter, including pipe position and the independent battery and wiring faults documented by the CSB.

When a failure mode lacks a practical detector, a software proxy can still miss the physical state. Responses include limiting actuator authority under a validated force or clearance model, restricting the operating domain, adding independent sensing, and scheduling proof tests. ISO 10218 and ISO/TS 15066 apply to their industrial robot scopes; they do not supply a universal force limit for this machine. If none of the proposed measures can support the claim, operation must be withheld.

The safety case must document these gaps. Figure 34.5 marks where the argument runs out. When safe state S_A and hazardous state S_B produce identical sensor observations prior to the physical deadline $t_{deadline}$,¹² software cannot choose a state-specific response before the deadline; a safe common response or passive limit may remain possible. The residual-claims register records the detector's blind region, defining the set of physical states that produce indistinguishable observations, along with the maximum exposure duration permitted between physical proof tests. Treating an undetectable failure as a solved runtime problem undermines the validity of the safety case. Preserving the exact boundary where sensing stops forces the engineering organization to rely on physical constraints rather than software optimism.

An active probe can be proposed when it fits a separately validated energy, contact, clearance, and timing envelope. A dither is still physical actuation. It cannot be used to discover friction if the probe itself could slip beyond the reserved clearance. The independent permission path must admit each probe command and choose a state-matched fallback when the needed bound is unavailable. The candidate procedure in algorithm 34.1 is not proof that a 125 Hz ripple is safe on every plant.

¹² | **Serialization Latency Limits:** Serializing parallel sensor frames across SerDes links and fieldbuses imposes irreducible transmission delays before telemetry arrives in processor memory. This transport lag consumes a significant fraction of the available stopping distance before software safety checks can even execute. Failure to budget for bus serialization jitter can cause high-speed platforms to breach physical clearance envelopes during emergency stops.



Figure 34.5: Two limits on the claim: Panel (a) shows states with identical pre-deadline telemetry. A speed clamp is useful only with a supported braking and clearance envelope. Panel (b) shows the zero-event exposure required for a rare-rate claim under the chosen Poisson model; a barrier argument also needs observable state, a feasible control action, and valid timing assumptions.

Algorithm 34.1 Active Epistemic Probe and Invariant Degradation Monitor

Require: telemetry stream $\mathbf{y}_t$, learned proposal $\mathbf{u}_{\text{policy}}$, plant parameters $\mathcal{M}_{\text{nominal}}$, threshold limits $\{\Delta_{\text{disparity}}, \tau_{\text{deadline}}, E_{\text{kin,max}}\}$

Ensure: permitted actuator command $\mathbf{u}_{\text{cmd}}$, trip flag register_trip_flag

```

1:  $\mathbf{y}_t \leftarrow \text{READADCSANDENCODERSDMA}; t_{\text{now}} \leftarrow \text{READMONOTONICHARDWARETIMER}$ 
2:  $\delta \mathbf{u}_t \leftarrow \text{PROPOSEPROBE}(\mathbf{y}_t, \mathcal{M}_{\text{nominal}}) \triangleright \text{Candidate actuation}$ 
3:  $\mathbf{u}_{\text{probe}} \leftarrow \mathbf{u}_{\text{policy}} + \delta \mathbf{u}_t$ 
4: if  $\neg \text{INDEPENDENTPROBEGATE}(\mathbf{u}_{\text{probe}}, \mathbf{y}_t, E_{\text{kin,max}}, \tau_{\text{deadline}}, \text{contact}, \text{clearance})$  then
5:   return (STATEMATCHEDFALLBACK, TRUE)
6: end if
7:  $\mathbf{u}_{\text{cmd}} \leftarrow \text{INDEPENDENTPERMISSIONFILTER}(\mathbf{u}_{\text{probe}}, \mathbf{y}_t, \mathcal{C}_{\text{safe}})$ 
8:  $\text{WRITEHARDWAREPWMREGISTERS}(\mathbf{u}_{\text{cmd}}) \triangleright \text{Only the admitted probe is applied}$ 
9:  $\mathbf{y}_{t+\Delta} \leftarrow \text{READRESPONSEWITHINDEADLINE}$ 
10:  $\hat{\mathbf{Z}} \leftarrow \text{COMPUTEPROBEIMPEDANCE}(\delta \mathbf{u}_t, \mathbf{y}_{t+\Delta})$ 
11:  $\Delta_{\text{disp}} \leftarrow \|\hat{\mathbf{Z}} - \mathcal{M}_{\text{nominal}}.\text{impedance}\|$ 
12: if  $\Delta_{\text{disp}} > \Delta_{\text{disparity}}$  or  $\neg \text{REMAININGCLEARANCEVALID}(\mathbf{y}_{t+\Delta})$  then
13:    $\text{SETHARDWAREFAULTREGISTER}(\text{ACTIVE\_HIGH})$ 
14:    $\text{LOGCLAIMSREGISTEREVENT}(\text{"DEGRADATION\_TRIP"}, \Delta_{\text{disp}}, t_{\text{now}})$ 
15:   return (STATEMATCHEDFALLBACK, TRUE)
16: end if
17: return ( $\mathbf{u}_{\text{cmd}}$ , FALSE)

```

These unobservable physical failure modes are compiled in table 34.3.¹³

¹³ | **Motor Thermal State:** Winding resistance and magnet flux can change with temperature. Whether that changes braking authority depends on the drive, current limits, load, and measured thermal model; a separate holding path may be required for gravity-loaded axes.

Table 34.3: Detectorless physical failure taxonomy: Comparison of critical failure modes across transduction, contact mechanics, electromagnetic drives, and learned vision-action policies where physical observables are indistinguishable before the time-to-harm. The table details the physical degradation mechanism, the root cause of observational blindness, the timing budget deficit ($\tau_{\text{det}} + \tau_{\text{act}} > t_{\text{harm}}$), the failure of software proxies, and the required physical hardware or architectural interceptors.

Failure and missing state	Why the existing channel can miss it	Deadline or evidence deficit	Defensible response
Pressure-sensor drift: zero offset changes with aging or deposition.	One channel cannot separate true pressure change from its own offset drift.	Drift may persist until the process reaches its pressure limit; a detector cannot be credited without a second reference.	Independent calibrated pressure channel, proof test, and rated relief path.
Brittle contact: an internal void lowers the workpiece threshold.	Exterior sensing gives identical pre-contact observations.	In the worked gripper, 15 N is reached at 15 ms; the first possible command change is at 18 ms.	Validate the passive load path and stroke limit, or add an independent pre-contact material measurement.
Rotor flux loss: delivered torque differs from current-based estimate.	Stator current alone does not measure the torque constant after magnet damage.	A gravity-loaded axis may have little time to arrest once available torque is lost.	Measure torque or rotor state, derate within tested limits, and validate a holding or controlled-stop path.
Learned perception shift: an optical artifact is mistaken for a known scene.	Model confidence may remain high even while the physical scene differs.	Confidence or reconstruction loss alone does not bound clearance or stopping response.	Independent range/clearance checks and an admitted motion envelope, with residual blind regions recorded.

34.5 What Would Change These Answers

Every entry in the residual-claims register marks an unsupported link in the safety case where argument gives way to assumption. Transforming these recorded caveats into an engineering agenda requires sorting them by what is physically or theoretically missing. A recorded gap falls into one of four classes, defined by the missing element. It stems from an **unmeasured observable** when the true physical state (such as a hidden internal defect) cannot be distinguished from sensor data before the deadline for action; from an **evaluation deficit** when the tail probability of failure cannot be bounded because these rare events require intractable physical exposure; from a **theoretical deficit** when the learned policy lacks a bounded relationship between training conditions and execution stability; or from a **scope mismatch** when the target claim exceeds what the system architecture can physically support. Naming the missing capability defines the exact physical or mathematical requirement that would discharge the entry.

Closing an observability gap requires a new physical measurement that separates previously indistinguishable states within the nervous system's deadline. For the ceramic grasping failure where an internal void is undetectable prior to contact, an advance cannot be a software filter or a learned estimator operating on the same surface sensors. The advance must introduce a transducer operating on an independent physical channel, such as an ultrasonic pulse-echo transducer or an active acoustic resonance detector. To close the gap, the measurement must satisfy a budgetable engineering threshold: it must resolve the internal defect within a latency that leaves sufficient margin for the actuator to halt, while maintaining a bounded false-alarm rate and demonstrating failure independence from the primary optical stream under variable ambient illumination. For the worked gripper above, fracture occurs at 15 ms, so a proposed 12 ms measurement plus 10 ms braking path would be too late. Closure would require a faster validated path or the passive force limit, including measurement age and actuator response.

Closing a rare-event evaluation gap requires replacing raw sample counts with structured estimations that can be analytically bounded for bias and variance. A safety claim asserting a tail failure rate below 10^{-6} failures/hr cannot be verified by logging 10^6 hours of unperturbed physical operation, nor by running 10^8 simulated episodes whose fidelity in the tail—specifically regarding extreme sliding friction and contact bifurcations—is unverified. An advance that closes this gap must construct an importance sampling or adaptive stress-testing scheme that demonstrates conservative probability estimation on known, analytical benchmark distributions. The evaluation must report effective independent exposure rather than raw trial counts. If ten thousand simulated trajectories explore one local basin of a collision surface, their effective evidence about other basins may be far

smaller than their raw count; effective sample size must be estimated from weights and dependence. An advance closes the evaluation gap only when it provides an estimator whose variance decreases at a verifiable rate and whose assumptions about the tail geometry are independently checkable.

Closing a theory gap requires a bounded analytical claim paired with a runtime mechanism that checks its premises. When a policy is trained by imitation or reinforcement learning, the gap is the absence of an inductive bias that guarantees bounded outputs under distribution shift. A theoretical advance cannot close this gap by proving asymptotic convergence or by deriving bounds that depend on uncheckable Lipschitz constants across high-dimensional activation spaces. The theory must state the precise invariance it guarantees, define the support over which that invariance holds, and provide an operational detector that flags when physical state trajectories leave that support. If a learned tracking controller is analyzed for payload mass in [1.0 kg, 2.5 kg] and a specified actuator bandwidth, a candidate online monitor must show that mass is observable under available excitation, bound estimator error and decision delay, and validate a state-matched brake path before relying on that analysis.

To prevent research optimism from corrupting the safety case, each candidate advance must undergo evaluation against a predeclared closure test. The test defines the missing argument link, the numerical threshold required for sufficiency, and the independent evidence necessary to verify it. An advance fails the test if it merely relocates an assumption to a less visible layer of the system. Replacing a neural network controller with a control-barrier function whose certificate assumes perfectly known linear dynamics does not close a theory gap. It merely relocates the unmodeled dynamics from the opaque network weights to the barrier derivative. A candidate advance is stopped before it can alter the safety verdict whenever its validation relies on the exact unverified properties it was designed to establish.¹⁴

Research directions in physical machine learning must be judged solely by whether they convert an open residual claim into verified evidence capable of altering the operational verdict. General improvements on benchmark leaderboards, such as a 3 percent reduction in average tracking error or higher reward on a standard task suite, provide zero evidence toward closing an unsupported safety link. A research project becomes relevant to physical deployment only when its deliverable targets a specific entry in the claims register, whether by providing the missing transducer, bounding the tail estimator, or furnishing the runtime-checked stability proof. If an advance improves mean performance but leaves the detector blind spot or the tail uncertainty unchanged, the authority limits of the machine remain exactly where they were.

Certain gaps cannot be closed by any known sensor, sampling regime, or mathematical framework. When physical laws, computational limits, or economic constraints preclude the necessary measurement or proof, the safety case cannot remain suspended in anticipation of future discovery. Narrowing the actuator's physical authority, restricting the operational design domain, or refusing to operate the machine is not an admission of research defeat. It is the definitive engineering resolution. As established in classical fault-tolerant systems engineering, reliable cyber-physical systems cannot eliminate unmodeled open-world disturbances through unconstrained scale; rather, dependable operation requires explicit hardware self-monitoring thresholds and structural isolation mechanisms that contain catastrophic faults. A machine whose actuator authority is physically bounded below destructive force levels does not require an ultrasonic sensor to survive an undetected void. The residual-claims register records the boundary of verifiable knowledge, and the nervous system enforces the physical constraints that render the remaining unknown harmless.

34.5.1 Research against an open claim

A new sensing or modeling method matters here when it supplies evidence for a named open entry. An event camera can report local brightness changes with low latency under a specified sensor mode; it does not measure oil friction or hidden ceramic voids. Compute-in-memory may reduce data movement in a qualified implementation, but its power and timing benefits must be measured on the target workload and cannot replace the independent permission path. Neither is a universal closure test.

¹⁴ | **Structural Fault Bounds:** In classical fault-tolerant architectures, system dependability does not require eliminating every internal component error, but ensuring that fault propagation is bounded by structural invariants. When runtime detectors cannot resolve latent physical degradation, passive mechanical limiters or firmware parameter clamps prevent localized faults from escalating into catastrophic failure. Such isolation supports containment only for the faults, load paths, and actuator states included in its validation.

Hamiltonian neural networks (Greydanus et al. 2019) encode an energy-like prior for a learned conservative dynamics model. For a time-independent learned Hamiltonian $\mathcal{H}$ with its ideal canonical dynamics,

$$\dot{\mathbf{q}} = \frac{\partial \mathcal{H}}{\partial \mathbf{p}}, \quad \dot{\mathbf{p}} = -\frac{\partial \mathcal{H}}{\partial \mathbf{q}},$$

the model conserves its *learned* $\mathcal{H}$ along those modeled trajectories. Actuated torque, dissipation, contact, parameter error, and discretized execution can violate the model's premises. This representation can improve a dynamics estimate; it cannot establish physical force, stopping clearance, or safe actuator authority without separate evidence. A research claim closes only when the relevant model error and permission limits are checked over the stated plant envelope.

34.6 Generalizing to Unfamiliar Systems

Applying the framework developed across this book to a machine not previously examined demonstrates how the four properties, consisting of physical bounds, information-age budgets, the authority split, and the evidence record, isolate hazards before hardware is energized. Consider an automated packaging cell designed for prepared meals. The machine moves rigid plastic trays along a linear track, presses a heated sealing head onto tray perimeters to bond plastic barrier film, and injects pressurized steam into product cavities to blanch ingredients immediately prior to sealing. This cell exercises all three modalities of physical consequence within a single enclosure, spanning kinetic mass transport (moving the loaded tray), mechanical contact force (pressing the sealing head), and thermodynamic energy transfer (injecting hot steam). Evaluating this system requires assessing the physical envelope, computing timing deadlines, enforcing strict boundaries on learned software interactions, and determining the available evidence.

Every evaluation begins by cataloging the irreversible state changes the cell can produce. A loaded food tray traversing a linear transfer rail carries momentum. When a tray strikes an obstruction, the kinetic energy is dissipated through plastic deformation of the container and structural deflection of the frame, a state transition that cannot be reversed by commanding the drive motor in reverse. The sealing mechanism introduces a second irreversible boundary. An actuator drives an aluminum sealing head at 180°C against the tray flange to melt polymer bonding layers. If the actuator exerts force beyond the compressive yield strength of the container wall, the flange buckles, destroying the hermetic geometry. The steam subsystem introduces a third irreversible boundary through enthalpy exchange. Steam supplied at 120°C delivers heat to the product; the product-zone temperature can remain lower only under a measured heat-transfer and dwell model. Once proteins denature and cell walls rupture under excessive thermal exposure, cooling the chamber cannot restore product texture. Software retries do not exist in thermodynamic or structural domains: a software loop cannot uncrush a buckled plastic tray or uncook a heat-damaged vegetable. Once an actuator delivers energy beyond the yield threshold of a material, the physical state is altered permanently.

State changes cannot be undone; thus, the nervous system must intervene before consuming physical clearances. In an analytical baseline, consider a linear carriage of mass $m_{\text{carriage}} = 12.0$ kg transporting a loaded tray at a nominal velocity of $v_{\text{nom}} = 1.50$ m/s toward an optical inspection station whose mechanical gate provides a physical clearance margin of $d_{\text{clear}} = 60.0$ mm before contacting a rigid frame stop. An overhead vision system detects a closing inspection gate; the stated 60.0 mm is the measured distance to that gate, and any other obstruction closer than the gate must be separately detected or excluded. The controller commands an electromagnetic track brake with a validated available-deceleration floor of $a_{\text{brake}} = 15.0$ m/s² in the stated envelope. Determining whether an onboard vision system can halt the carriage prior to an impact requires computing the end-to-end sensorimotor information age across the full observation-to-enforcement path: optical exposure and rolling-shutter integration ($t_{\text{exp}} = 16.0$ ms), direct memory access (DMA) transfer and neural detection inference ($t_{\text{inf}} = 36.0$ ms), CAN-FD safety arbitration bus latency ($t_{\text{bus}} = 2.0$ ms), and brake solenoid flux buildup rise time ($t_{\text{sol}} = 10.0$ ms), summing to a total delay of $\tau_{\text{info}} = 16.0 + 36.0 + 2.0 + 10.0 = 64.0$ ms = 0.064 s. Over this sensing and actuation lag, the carriage travels completely unbraked for a lag distance $d_{\text{lag}} = v_{\text{nom}} \tau_{\text{info}} = 1.50$ m/s $\times$ 0.064 s = 96.0 mm. Once the solenoid builds flux and mechanical pads engage, deceleration consumes an additional distance $d_{\text{brake}} = \frac{v_{\text{nom}}^2}{2a_{\text{brake}}} = \frac{(1.50 \text{ m/s})^2}{2 \times 15.0 \text{ m/s}^2} = 75.0$ mm. The total kinematic stopping envelope is defined by equation 34.4:

$$d_{\text{stop}} = d_{\text{lag}} + d_{\text{brake}} = v\tau_{\text{info}} + \frac{v^2}{2a_{\text{brake}}} \quad (34.4)$$

Evaluating equation 34.4 yields a stopping envelope of $d_{\text{stop}} = 96.0 \text{ mm} + 75.0 \text{ mm} = 171.0 \text{ mm}$.¹⁵ Because 171.0 mm exceeds the 60.0 mm physical clearance by a catastrophic deficit of $\Delta d = d_{\text{stop}} - d_{\text{clear}} = 111.0 \text{ mm}$, the machine cannot stop before the modeled gate at this velocity by relying on that vision path. Indeed, because the unbraked lag distance alone ($d_{\text{lag}} = 96.0 \text{ mm}$) exceeds the available clearance ($d_{\text{clear}} = 60.0 \text{ mm}$), the physical deadline is missed before the brake generates its first newton of retardance, crashing into the barrier while the vision model is still processing the video frame.

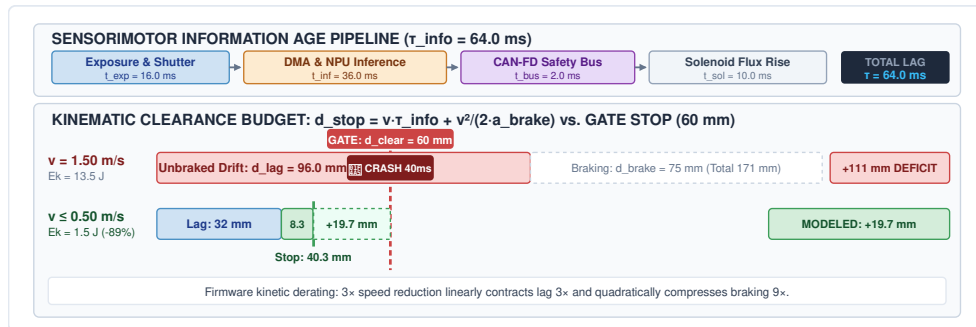
Resolving this hazard requires an explicit authority boundary. In this cell, learned models propose actions across all three domains. A vision model infers tray contents to propose carriage transit speeds, a multimodal network estimates packaging film thickness to propose sealing forces and dwell durations, and a thermal estimator proposes steam valve pulse widths based on incoming product temperature. The nervous system prevents these proposals from reaching actuators directly. An independent kinematic governor operating at 1000 Hz monitors optical encoders along the track, admitting carriage velocity only where rangefinder coverage, measured gate clearance, and a validated stop path support the envelope. For the sealing head, a proposed mechanical force limiter caps transmitted force at a selected 220 N, pending tests of its response and the tray flange threshold. For the steam delivery system, the proposed independent temperature path closes the steam valve when a sensed product-zone temperature reaches 105°C or dwell reaches 400 ms. Sensor lag, residual heat after valve closure, and the product limit still require testing. The learned policy retains authority to optimize throughput and seal quality within these bounds, but the nervous system retains sole authority to permit physical execution.

With the physical bounds, information-age budget, and authority split established, the safety case is compiled into a formal release claim. The evidence record can support a restricted kinetic claim after tests of rangefinder coverage, gate clearance, and braking. The firmware velocity clamp is sized from equation 34.4. Requiring the stopping envelope to fit within the physical clearance with a safety margin of $d_{\text{margin}} \geq 15.0 \text{ mm}$ imposes the constraint $d_{\text{stop}}(v) \leq 45.0 \text{ mm}$. Because unbraked lag distance scales linearly with velocity ($v\tau_{\text{info}}$) while braking distance scales quadratically ($v^2/(2a_{\text{brake}})$), reducing operational velocity yields compounding physical returns. Derating velocity threefold—from $v_{\text{nom}} = 1.50 \text{ m/s}$ down to a firmware clamp of $v_{\text{clamp}} \leq 0.50 \text{ m/s}$ —compresses unbraked lag travel linearly from 96.0 mm to $d_{\text{lag}} = 0.50 \text{ m/s} \times 0.064 \text{ s} = 32.0 \text{ mm}$, while braking distance collapses ninefold from 75.0 mm down to $d_{\text{brake}} = \frac{(0.50 \text{ m/s})^2}{2 \times 15.0 \text{ m/s}^2} \approx 8.3 \text{ mm}$. Under the declared timing and braking bounds, the composite envelope $d_{\text{stop}} = 40.3 \text{ mm}$ fits within the 45.0 mm threshold, preserving a 19.7 mm physical margin. Simultaneously, carriage kinetic energy ($E_k = \frac{1}{2}mv^2$) drops quadratically by 88.9 percent—from 13.5 J down to 1.50 J—reducing impact energy; whether a collision damages the tray or frame still requires a tested material threshold. The proposed 220 N force limiter and 105°C cutoff still need measured valve response, material thresholds, and fault coverage before they can support crushing or thermal claims. However, assembling the evidence record also uncovers claims that remain unsupported. To assess seal hermeticity, the engineering team evaluates a vision model that inspects optical scattering through the steam chamber for micro-tears in the sealing film. Product sterility is a separate thermal-process and microbiology claim with its own validated evidence. Within the chamber, localized steam condensation creates optical refraction patterns that mimic film defects while masking genuine punctures. Because no independent physical sensor exists in the cell to measure film seal integrity in real time under dense vapor, this claim has no practical detector. Marking this link as detectorless and unsupported in the claims register is not an omission in the documentation, but an accurate statement of what the evidence can justify.

15 | **Kinematic Stopping Envelope Scaling:** While mechanical braking displacement scales quadratically with velocity ($v^2/2a_{\text{brake}}$), the unbraked drift distance scales linearly with velocity multiplied by end-to-end sensorimotor lag ($v\tau_{\text{info}}$). At high operational speeds, this unbraked drift dominates the stopping envelope, rendering faster mechanical brakes ineffective if sensor exposure and neural inference latencies remain unoptimized. Failing to bound sensorimotor transport latency exhausts physical clearance buffers before braking torque can begin decelerating the plant.



Checkpoint 34.5: Cell information age and clearance sizing



Evaluating an unfamiliar embodied machine requires computing sensorimotor lag across every pipeline stage to verify that physical stopping envelopes fit within available clearance margins.

- ☐ **Information age lag consumption:** Can you explain why sensor exposure, inference latency, bus transport, and actuator rise time consume physical clearance linearly with velocity before mechanical braking can begin per equation 34.4?
- ☐ **Kinematic clearance deficit and crash inevitability:** Can you describe why an unbraked lag distance exceeding available clearance ($d_{lag} > d_{clear}$) makes physical impact inevitable regardless of how powerful the mechanical brakes are?
- ☐ **Firmware velocity derating and kinetic containment:** Can you explain how solving the stopping envelope equation for a safe velocity clamp (v_{clamp}) drastically reduces kinetic energy and supports clearance under the validated detection, timing, and braking premises?

The constructed record supports a *candidate restricted mode* for transport if the stated 64 ms end-to-end delay, 15 m/s² available braking floor, and 60 mm clearance are validated together; at 0.50 m/s its modeled stop is 40.3 mm. Sealing and thermal processing remain conditional on tests of the relief valve, cutoff, material limits, and independent fault path. Autonomous package sign-off is refused because vapor can conceal film defects from the available vision sensor and the example has not established sterility independently. That link could be closed by post-cooling leak or vacuum-decay inspection, a tested material-process argument, a self-indicating film, or mandatory downstream inspection. A throughput model cannot replace that physical evidence.

This evaluation illustrates the discipline that governs every physical machine operating under learned control. The engineering process does not begin with model architecture, training losses, or benchmark scores. It begins at the physical body, identifying every point where mass, force, or energy creates an irreversible state change. From those physical mechanics, the engineer derives the freshness deadlines that the nervous system must meet, verifies that the end-to-end information age fits within physical clearances, and enforces a strict authority split that denies learned software unmediated access to actuators. Finally, the engineer assembles the evidence record, acknowledging where mathematical proofs and sensor capabilities end, marking open links without equivocation, and restricting operational authority to match the boundaries of what has been proven. Building physical systems that learn and act requires accepting that gravity, inertia, and thermodynamics do not yield to statistical expectations; a neural network predicting a high probability of a safe stop will still crash if the physical brakes cannot engage before the clearance distance is consumed.

34.7 Architectural Synthesis Across the Physical AI Stack

This discipline unfolds across every layer of the physical AI stack, as shown in figure 34.6. Part I (chapter 2 through chapter 8) establishes the foundational systems premise: *the Causal Boundary*. Crossing this boundary marks the transition from decoupled software to continuous physical reality, where bits modulate motor drives, accelerate inertial mass, and dissipate irreversible Joule heat. We

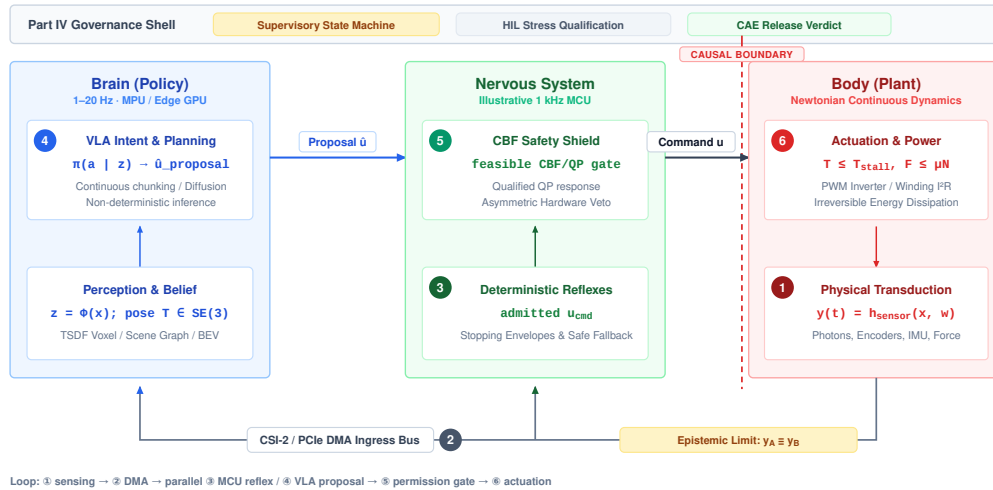


Figure 34.6: Closed causal loop: Learned planning submits proposals; an illustrative 1 kHz MCU checks a feasible permission constraint under its state, plant, actuator, and deadline assumptions. Governance records the tests and release conditions. The drawing is an architecture, not an ASIL certification or a universal controller rate.

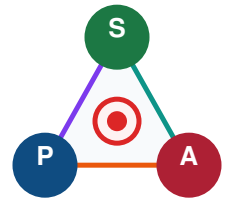
decomposed the embodied machine into its three constitutive organs: the *Body*, the *Brain*, and the deterministic *Nervous System*.

In Part II (chapter 10 through chapter 14), we demonstrated why closing the sensorimotor loop breaks the foundational independent and identically distributed (i.i.d.) assumptions of classical machine learning. Physical action can alter pose, contact, temperature, or other state and shape later observations, making observation streams endogenous and non-stationary, which requires rigorous demonstration budgeting, distribution-shift characterization, and finite-sample exposure arithmetic.

In Part III (chapter 16 through chapter 26), we constructed the online real-time execution engine. We traced perception latencies across DMA SerDes transfers, maintained temporal state freshness in structured memory, negotiated multi-cadence intent horizons, planned kinematically bounded trajectories, enforced deterministic invariant shields at the hardware interface, and placed compute across heterogeneous edge silicon fabrics to satisfy hard physical deadlines.

In Part IV (chapter 28 through chapter 34), we governed the physical machine in operation: managing shared human-machine interventions, verifying fault tolerance through empirical stress testing, compiling Claim-Argument-Evidence (CAE) trees into auditable release verdicts, and now, at the Frontier, confronting what finite empirical exposure can never settle.

Table 34.4 condenses the four decisions that recur across the book. The numbers needed for each decision come from the particular machine, not from a chapter label.



The physical AI synthesis: closing the loop across Sense, Plan, and Act at the epistemic frontier.

Prerequisite: Continuous authority revocation expands the supervisory state machines from section 28.3.

Table 34.4: Physical AI architectural decisions: The four recurring questions connect body dynamics to observation, permission, and release. Each row names evidence required for a particular plant rather than prescribing a universal hardware floor.

Decision	Physical question	Evidence and permission consequence
Body and timing (chapter 4; chapter 8)	How much energy can the plant deliver before detection and braking take effect?	Measure actuator, load, delay, and clearance bounds; size a feasible state-dependent fallback.
Observation and proposal (chapter 10; chapter 16; chapter 18; chapter 20; chapter 22)	Which states are fresh and distinguishable before action?	Preserve evidence epochs and uncertainty; treat learned outputs as proposals rather than actuator commands.
Permission and placement (chapter 24; chapter 26)	Can an independent path enforce the limit on the actual silicon and plant?	Test state estimation, feasibility, timing, interference, and physical response together.
Governance and frontier (chapter 28; chapter 30; chapter 32)	Which remaining premises support the signed operating envelope?	Record handoffs, fault tests, restricted modes, owners, and closure tests; withhold operation where the residual claim is unsupported.

🔍 Systems Perspective 34.1: Epistemic containment and governance

A detector whose observation and response finish after the body’s harm deadline cannot provide a state-specific intervention for that hazard. A common fallback, passive limit, or restricted operating envelope can support the claim only when its load path, friction, timing, and actuator assumptions are validated. Fleet exposure bounds the events actually observed; a shielded-system incident log cannot by itself measure hazardous policy proposals or conditional shield coverage. Learned commands remain proposals until the independent permission path admits them.

34.8 Epistemic Governance

An open claim becomes an operating decision. The release record should say which physical premise remains unsupported, which monitored or passive measure limits its consequence, who owns the restriction, and which change triggers review or revocation. ISO 26262 and ISO 21448 provide road-vehicle examples of scoped safety reasoning; UL 4600 addresses autonomous-system safety cases. They do not prescribe one microcontroller rate, one force threshold, or one release decision for every machine.¹⁶

Four decisions keep the residual register operational. First, preserve each open entry across model updates until a predeclared closure test supplies evidence. Second, keep learned outputs as proposals and maintain an independently validated permission and fallback path. Its processor, scheduling period, sensor coverage, and actuator response follow the plant’s measured deadlines; bare-metal execution is one design choice. Third, use passive limits only where their load path, stroke, wear, and stopping consequences are tested. Fourth, require named engineering and operations owners to sign the release conditions and define the measurements that withdraw authority.

¹⁶ | **Standards Scope:** ISO 26262 and ISO 21448 concern road vehicles; UL 4600 provides an autonomous-product safety-case framework. Apply the standard relevant to the machine and its jurisdiction, then state this case’s actual requirements rather than treating these examples as universal mandates.

Systems Perspective 34.2: Foundation models and balance limits

A learned whole-body policy may propose footholds and trajectories for an unfamiliar task. A support block can still crumble before a camera reveals its load capacity. The release case must therefore bound proposed motion through a measured contact, balance, and fall-response envelope, with a state-dependent fallback. A 1 kHz controller or a control-barrier function is useful only if its state estimate, actuator authority, feasible control set, and complete response deadline support that envelope. Model scale supplies none of those premises by itself.

The same distinction holds in three familiar plants. A mobile robot's wheel-slip detector acts *after* entry onto a low-friction patch; permission for the approach needs a justified friction floor, shorter stopping envelope, barrier, or route exclusion. A delta robot whose planner stalls can transfer to a resident stop only if each reachable entry state has an admitted stop and enough clearance. A chemical reactor whose feed optimization policy sees a temperature rise needs an independent temperature and pressure path with a tested shutdown and cooling response; removing pump torque alone does not arrest an ongoing exothermic reaction. These are proposal-permission decisions grounded in the body's measured dynamics, not guarantees from the word "deterministic."

Checkpoint 34.6: Epistemic governance and operational revocation

- ☐ Which residual claim and physical premise determine the permitted mode?
- ☐ Who maintains its inspection, calibration, and actuator limits?
- ☐ Which measured change revokes tracking permission, and which validated fallback remains available?
- ☐ Which new evidence would close the claim without simply moving the assumption to a new sensor or model?

34.9 Fallacies and Pitfalls

Some hazards are unobservable through available sensors, and some claims require more exposure than the design can accumulate. The following examples show why those limits must shape operating authority rather than disappear into a confidence score.

Pitfall: *Relying on vision-based perception alone when the physical hazard is observationally indistinguishable.*

A pallet robot approaches a transparent hydraulic-fluid spill that its available cameras and range sensors cannot distinguish from dry concrete, producing observational indistinguishability. After the wheels enter the patch, the sudden drop in friction causes the wheels to spin without grip. This slip reveals the changed surface, but the physical loss of deceleration authority means stopping distance has already expanded to 2.51 m against 0.80 m of clearance. A visually clear path does not establish the required friction for the admitted speed. The operating envelope must account for that unobserved condition through defensible speed restrictions or independently validated sensing. The residual-claims register must retain the friction assumption and its containment measure instead of treating missing visual evidence as evidence of traction.

Fallacy: *Stator temperature and current measurements are sufficient to detect rotor magnet flux degradation.*

A manipulator's rotor magnets degrade during repeated duty cycles. Because output torque follows $\tau = K_t I_q$, a reduced torque constant K_t lowers the actual mechanical torque produced for any given commanded quadrature current I_q . If the drive commands its rated limit, telemetry will report nominal current delivery and expected $I^2 R$ Joule heating, but available torque may fall from 50 N·m to 35 N·m, leaving the joint unable to securely hold its rated payload. The existing electrical telemetry cannot observe the missing mechanical parameter. Qualification and maintenance must independently verify torque capability, while the holding architecture needs passive protection appropriate to its loss. Nominal current indicates electrical delivery, not proof that the original mechanical authority is available.

Prerequisite: The gap between verified models and unmodeled friction challenges the safety cases in section 32.3.

Fallacy: *Empirical fleet testing can statistically validate ultra-rare catastrophic failure rates in open-world environments.*

A delivery-vehicle developer presents 500,000 hours without accidents to support a claimed catastrophic rate of $p \leq 10^{-9}$ /hour at 95 percent confidence. The stated zero-failure bound requires $n \geq 3.0 \times 10^9$ h under its exposure assumptions, far beyond the submitted record. Correlated fleet conditions and configuration changes require an argument about which observations are applicable. The record cannot support the proposed rate merely because no accident occurred. Release must narrow the claim or provide additional architectural evidence for bounded physical authority, with explicit assumptions and validation of the protective mechanism. Calling that mechanism deterministic does not itself close the evidence gap.

Pitfall: *Assuming high-capacity learned models can evaluate their own perceptual and mechanical limits.*

Glare from a workbench corrupts a manipulator's view of a glass ampoule and packaging tray. The visual encoder compresses the image into a nominal latent representation, discarding the glare as background noise. Consequently, both the action policy and its latent-space uncertainty monitor remain confident because they evaluate the corrupted latent vector, while the proposed descent trajectory targets the wrong physical surface. The uncertainty monitor shares the representation that produced the error. Protection must therefore draw on independently justified contact and proximity limits at the actuation boundary, with mechanical compliance where the available response time is insufficient. Confidence within the learned representation cannot establish that a fragile physical object will survive the proposed motion.

Fallacy: *If a failure mode is documented, a neural network can be trained to detect and prevent it.*

In the constructed ceramic-gripper case, the rigid force ramp reaches 15 N at 15 ms, while filter, bus, and MCU delay permit the first new command only at 18 ms. The 24 ms winding time constant then describes current decay rather than a completed stop. Improving classification accuracy without shortening the physical response cannot prevent that modeled fracture. The proposed software safeguard fails its physical timing requirement even if it identifies every event perfectly. The system design must instead modify the physical exposure or protection mechanism—for example, through series elastic actuators or passive compliant mechanisms that bound contact forces below a case-specific, anatomically relevant contact threshold within the available actuation interval. Documenting a hazard supplies a software detection objective; it does not guarantee that the available compute pipeline and actuator dynamics can support a timely physical intervention.

Fallacy: *Massive cloud simulation can substitute for physical exposure in certifying open-world safety.*

A team runs billions of simulated hours in the cloud to bypass the cost of physical exposure. These runs extensively sample the simulator's synthetic disturbances, contact models, and sensing assumptions, but they do not establish safety against omitted physical realities—such as unmodeled Stribeck friction transitions, gearbox compliance, or I^2R Joule heating that induces thermal derating. Repetition increases evidence within the closed model; it does not validate sim-to-real fidelity against every deployment condition. Simulation remains useful for logic checks and regressions, while physical safety claims also require measurements or validated operating envelopes for the machine. The release case must distinguish simulated evidence from evidence that its assumptions hold on the physical system.

34.10 Summary

At the causal boundary (section 2.2), a computational decision becomes part of a continuing, irreversible physical process. The architecture developed throughout this book connects this physical irreversibility to measured budgets, explicit interfaces, independent enforcement, and accountable release decisions. Its frontier lies in the premises those mechanisms still depend on but cannot fully observe or verify. An engineering record must expose these limits and state how they constrain operation.



Key Takeaways: The epistemic frontier and master synthesis

- **Residual Claims Must Constrain Operating Authority:** Empirical verification cannot test the full combinatorial open world. A **Residual-Claims Register** identifies unsupported premises, the restrictions or containment measures proposed for them, and the evidence needed to justify those measures; where that justification is unavailable, operation must be withheld.
- **Observational Indistinguishability Demands Structural Containment:** When safe and catastrophic physical states emit identical pre-harm sensor signals ($y_A(t) \equiv y_B(t)$) until the reaction window is exhausted ($t_{\text{distinguish}} + \tau_{\text{det}} + \tau_{\text{act}} > t_{\text{harm}}$), a detector using those signals cannot distinguish them in time. Passive compliance, slip clutches, or velocity restrictions must then be evaluated against the hazard rather than assumed sufficient.
- **The Exposure Wall Limits Empirical Assurance:** Under the stated statistical assumptions, validating a safety claim of $p \leq 10^{-9}$ /hour requires $> 3.0 \times 10^9$ h (342,000 machine-years) of failure-free exposure. This impractical testing burden motivates combining empirical evidence with architectural isolation and runtime enforcement, whose coverage and physical assumptions must also be justified.
- **The Proposal–Permission Architectural Invariant:** A learned model may propose actions; a separately validated permission path owns actuator admission and state-dependent fallback. Its processor and cycle time are chosen from measured plant deadlines.
- **Delay Consumes Physical Margin:** While a machine moves, computational delay consumes clearance ($\Delta x = \int v dt$), and released kinetic energy ($\frac{1}{2}mv^2$) cannot be recalled. The operating argument must account for detection, response time (including SerDes bandwidth and bus jitter), and the mechanical body’s ability to safely dissipate that kinetic energy within the hazard-assessed envelope; ISO 10218 / ISO/TS 15066 apply where their industrial robot scope fits.
- **The Systems Architecture Arc:** This book establishes the architectural foundations for physical machine learning systems, grounding high-capacity digital neural models in real-time execution fabrics where proposals must adhere to immutable physical conservation laws.

Engineering embodied intelligence requires knowing when the available evidence is sufficient to act and when it is not. Sensor degradation such as piezoresistive drift, non-stationary environmental disturbances, and mechanical wear such as progressive gearbox compliance can invalidate physical assumptions even while software executes nominally. The physical AI systems engineer must keep those assumptions visible, test the mechanisms that depend on them, and restrict or withhold operation when the remaining uncertainty cannot be contained. That judgment connects the entire path from a learned proposal to a machine’s next physical action.



What’s Next: From physical AI to the systems discipline

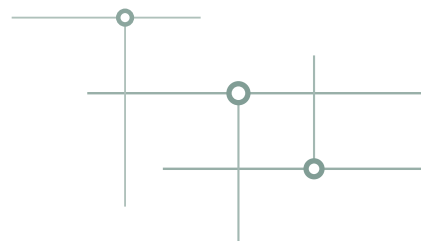
This book completes an investigation into the full architecture of machine learning systems. Across the systems curriculum, we explored the single machine (mastering the silicon contract, arithmetic intensity, memory hierarchies, and local hardware acceleration), distributed fleets (navigating collective communication fabrics, fault tolerance, cluster orchestration, and high-throughput serving infrastructure), and agentic software trajectories (constructing sandboxed execution environments, dynamic context working sets, and deterministic verification harnesses for autonomous multi-step reasoning). Here, machine learning leaves the digital domain to inhabit embodied physical intelligence—where learned foundation models act as unprivileged proposal engines, real-time nervous systems enforce validated operating bounds, and artificial intelligence at last meets the unyielding conservation laws of the physical universe. This work

is dedicated to the students, researchers, and systems engineers who take up the challenge of giving machines safe and capable hands in an open physical world.

Prof. Vijay Janapa Reddi, Harvard University

DRAFT

BACK MATTER



References

- Ahn, Michael, Anthony Brohan, Yevgen Chebotar, et al. 2023. "Do as I Can, Not as I Say: Grounding Language in Robotic Affordances." *Proceedings of the 6th Conference on Robot Learning (CoRL)*, Proceedings of machine learning research, vol. 205: 287–318.
- Ahn, Michael, Debidatta Dwibedi, Chelsea Finn, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Karol Hausman, Brian Ichter, et al. 2024. "AutoRT: Embodied Foundation Models for Large Scale Orchestration of Robotic Agents." *arXiv Preprint arXiv:2401.12963*.
- Amanatides, John, and Andrew Woo. 1987. "A Fast Voxel Traversal Algorithm for Ray Tracing." *Eurographics '87*, 3–10.
- Ames, Aaron D, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. 2019. "Control Barrier Functions: Theory and Applications." *European Control Conference (ECC)*, 3420–31.
- Ames, Aaron D, Jessy W Grizzle, and Paulo Tabuada. 2014. "Control Barrier Function Based Quadratic Programs with Application to Adaptive Cruise Control." *53rd IEEE Conference on Decision and Control*, 6271–78.
- Ames, Aaron D, Xiangru Xu, Jessy W Grizzle, and Paulo Tabuada. 2016. "Control Barrier Function Based Quadratic Programs for Safety Critical Systems." *IEEE Transactions on Automatic Control* 62 (8): 3861–76.
- An, Dong, Yuankai Qi, Yang Huang, Yan Wei, Liang Wang, Tieniu Tan, et al. 2023. "ETPNav: Evolving Topological Planner for Vision-Language Navigation in Continuous Environments." *arXiv Preprint arXiv:2304.03047*.
- Anderson, Robert J., and Mark W. Spong. 1989. "Bilateral Control of Teleoperators with Time Delay." *IEEE Transactions on Automatic Control* 34 (5): 494–501. <https://doi.org/10.1109/9.24201>.
- Andrychowicz, Marcin, Bowen Baker, Maciek Chociej, Rafal Jozefowicz, et al. 2020. "Learning Dexterous in-Hand Manipulation." *The International Journal of Robotics Research* 39 (1): 3–20.
- Ashby, W Ross. 1956. *An Introduction to Cybernetics*. Chapman & Hall.
- Åström, Karl Johan, and Tore Hägglund. 2006. *Advanced PID Control*. ISA-The Instrumentation, Systems,; Automation Society.
- Åström, Karl Johan, and Richard M Murray. 2021. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton university press.
- Bainbridge, Lisanne. 1983. "Ironies of Automation." *Automatica* 19 (6): 775–79. [https://doi.org/10.1016/0005-1098\(83\)90046-8](https://doi.org/10.1016/0005-1098(83)90046-8).
- Barr, Michael. 2013. *Bookout v. Toyota Motor Corp.: 2005 Camry L4 Software Analysis*.
- Bellman, Richard. 1957. *Dynamic Programming*. Princeton University Press.
- Black, Kevin, Homer Walke, Kevin Bishop, Lee Zhang, Albert Fang, Brent Chen, Steven Morad, et al. 2024. " π_0 : A Flow-Matching Foundation Model for Generalist Robots." *arXiv Preprint arXiv:2410.24164*.
- Blanchini, Franco. 1999. "Set Invariance in Control." *Automatica* 35 (11): 1747–67.
- Brohan, Anthony, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, et al. 2023. "Rt-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control." *arXiv Preprint arXiv:2307.15818*.
- Brohan, Anthony, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, et al. 2023. "RT-1: Robotics Transformer for Real-World Control at Scale." *Robotics: Science and Systems (RSS)*. <https://doi.org/10.15607/RSS.2023.XIX.025>.
- Brooks, Rodney. 1986a. "A Robust Layered Control System for a Mobile Robot." *IEEE Journal on Robotics and Automation* 2 (1): 14–23.
- Brooks, Rodney. 1986b. "A Robust Layered Control System for a Mobile Robot." *IEEE Journal on Robotics and Automation* 2 (1): 14–23.

- Brooks, Rodney A. 1991. "Intelligence Without Representation." *Artificial Intelligence* 47 (1–3): 139–59. [https://doi.org/10.1016/0004-3702\(91\)90053-M](https://doi.org/10.1016/0004-3702(91)90053-M).
- Bryant, Randal E., and David R. O'Hallaron. 2015. *Computer Systems: A Programmer's Perspective*. 3rd ed. Pearson.
- Butler, Ricky W., and George B. Finelli. 1993. "The Infeasibility of Quantifying the Reliability of Life-Critical Real-Time Software." *IEEE Transactions on Software Engineering* 19 (1): 3–12. <https://doi.org/10.1109/32.210303>.
- Cadene, Rémi, Simon Alibert, Alexander Toshev, Thomas Sen, Quentin Gallouédec, Adil Zouitine, et al. 2024. "LeRobot: State-of-the-Art Machine Learning for Real-World Robotics in PyTorch." *arXiv Preprint arXiv:2410.09312*.
- Chi, Cheng, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. 2024. "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion." *The International Journal of Robotics Research* 44 (10–11): 1684–704. <https://doi.org/10.1177/02783649241273668>.
- Clopper, C. J., and Egon S. Pearson. 1934. "The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial." *Biometrika* 26 (4): 404–13.
- Cohen, Jacob. 1988. *Statistical Power Analysis for the Behavioral Sciences*. 2nd ed. Routledge.
- Coles, Stuart. 2001. *An Introduction to Statistical Modeling of Extreme Values*. Springer.
- Craig, John J. 2005. *Introduction to Robotics: Mechanics and Control*. 3rd ed. Pearson Prentice Hall.
- Cruise LLC. 2023. *Part 573 Safety Recall Report 23E-086: Cruise LLC Automated Driving System Collision Detection Subsystem*. Nos. 23E-086. U.S. Department of Transportation, National Highway Traffic Safety Administration.
- Curless, Brian, and Marc Levoy. 1996. "A Volumetric Method for Building Complex Models from Range Images." *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '96)*, 303–12.
- Driess, Danny, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, et al. 2023. "PaLM-e: An Embodied Multimodal Language Model." *International Conference on Machine Learning (ICML)*, 8469–88.
- Durrant-Whyte, Hugh, and Tim Bailey. 2006. "Simultaneous Localization and Mapping: Part I." *IEEE Robotics & Automation Magazine* 13 (2): 99–110. <https://doi.org/10.1109/MRA.2006.1638022>.
- Elfes, Alberto. 1989. "Using Occupancy Grids for Mobile Robot Perception and Navigation." *Computer* 22 (6): 46–57. <https://doi.org/10.1109/2.30720>.
- ESA Investigation Board. 2017. *ExoMars 2016 - Schiaparelli Anomaly Inquiry*. European Space Agency.
- Exponent, Inc. 2023. *Cruise AV SF Incident—Pedestrian Collision: Technical Root Cause Analysis*. Exponent Project 2310645.000. Exponent, Inc.
- Florence, Pete, Corey Lynch, Feryal Behbahani, Tianhe Yu, Wenlong Huang, Fei Xia, et al. 2022. "Implicit Behavioral Cloning." *Conference on Robot Learning (CoRL)*, 158–68.
- Foote, Tully. 2013. "Tf: The Transform Library." *IEEE International Conference on Technologies for Practical Robot Applications (TePRA)*, 1–6.
- Forster, Christian, Luca Carlone, Frank Dellaert, and Davide Scaramuzza. 2017. "On-Manifold Preintegration for Real-Time Visual-Inertial Odometry." *IEEE Transactions on Robotics* 33 (1): 1–21.
- Franklin, Gene F, J David Powell, and Michael L Workman. 1998. *Digital Control of Dynamic Systems*. 3rd ed. Addison-Wesley.
- Furgale, Paul, Joern Rehder, and Roland Siegwart. 2013. "Unified Temporal and Spatial Calibration for Multi-Sensor Systems." *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1280–86.
- Gibson, James J. 1979. *The Ecological Approach to Visual Perception*. Houghton Mifflin.
- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press.
- Gray, Cary, and David Cheriton. 1989. "Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency." *ACM SIGOPS Operating Systems Review* 23 (5): 202–10.
- Greydanus, Samuel, Misko Dzamba, and Jason Yosinski. 2019. "Hamiltonian Neural Networks." *Advances in Neural Information Processing Systems (NeurIPS)* 32: 15379–89.
- Gu, Xiuye, Tsung-Yi Lin, Weicheng Kuo, and Yin Cui. 2022. "Open-Vocabulary Object Detection via Vision and Language Knowledge Distillation." *International Conference on Learning Representations*.
- Hafner, Danijar, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. 2025. "Mastering Diverse Control Tasks Through World Models." *Nature* 640 (8059): 647–53. <https://doi.org/10.1038/s41586-025-08744-2>.
- Hanley, James A., and Abby Lippman-Hand. 1983. "If Nothing Goes Wrong, Is Everything All Right? Interpreting Zero Numerators." *JAMA* 249 (13): 1743–45. <https://doi.org/10.1001/jama.1983.03330370053031>.

- Harnad, Stevan. 1990. "The Symbol Grounding Problem." *Physica D: Nonlinear Phenomena* 42 (1-3): 335–46.
- Hartley, Richard, and Andrew Zisserman. 2003. *Multiple View Geometry in Computer Vision*. Cambridge university press.
- Hennessy, John L., and David A. Patterson. 2019. *Computer Architecture: A Quantitative Approach*. 6th ed. Morgan Kaufmann.
- Hermann, Robert, and Arthur J. Krener. 1977. "Nonlinear Controllability and Observability." *IEEE Transactions on Automatic Control* 22 (5): 728–40.
- Hogan, Neville. 1985. "Impedance Control: An Approach to Manipulation: Part i–Theory, Part II–Implementation, Part III–Applications." *Journal of Dynamic Systems, Measurement, and Control* 107 (1): 1–24.
- Hornung, Armin, Kai M Wurm, Maren Bennewitz, Cyrill Stachniss, and Wolfram Burgard. 2013. "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees." *Autonomous Robots* 34 (3): 189–206.
- Huang, Wenlong, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wong, Li Fei-Fei, and Yuke Zhu. 2023. "VoxPoser: Composable 3D Value Maps for Everyday Robot Manipulation with Language Models." *arXiv Preprint arXiv:2307.05973*.
- Hughes, Nathan, Yun Chang, and Luca Carlone. 2022. "Hydra: A Real-Time Spatial Perception Engine for 3D Scene Graph Construction and Optimization." *Proceedings of Robotics: Science and Systems (RSS)*.
- ISO 21448: *Road Vehicles, Safety of the Intended Functionality*. 2022. International Organization for Standardization.
- ISO 26262: *Road Vehicles, Functional Safety*. 2018. International Organization for Standardization.
- ISO/TS 15066: *Robots and Robotic Devices, Collaborative Robots*. 2016. International Organization for Standardization.
- Jatavallabhula, Krishna Murthy, Alihusein Kuwajerwala, Qiao Gu, et al. 2023. "ConceptFusion: Open-Set Multimodal 3D Mapping." *Proceedings of Robotics: Science and Systems (RSS)*.
- Johannink, Tobias, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. 2019. "Residual Reinforcement Learning for Robot Control." *IEEE International Conference on Robotics and Automation (ICRA)*, 6023–29. <https://doi.org/10.1109/ICRA.2019.8794127>.
- Joint Authorities Technical Review. 2019. *Boeing 737 MAX Flight Control System: Observations, Findings, and Recommendations*. Federal Aviation Administration.
- Kalman, Rudolph Emil. 1960. "A New Approach to Linear Filtering and Prediction Problems." *Journal of Basic Engineering* 82 (1): 35–45.
- Kalra, Nidhi, and Susan M. Paddock. 2016. "Driving to Safety: How Many Miles of Driving Would It Take to Demonstrate Autonomous Vehicle Reliability?" *Transportation Research Part A: Policy and Practice* 94: 182–93. <https://doi.org/10.1016/j.tra.2016.09.010>.
- Kamath, Aishwarya, Mannat Singh, Yann LeCun, Gabriel Synnaeve, Ishan Misra, and Nicolas Carion. 2021. "Mdetr-Modulated Detection for End-to-End Multi-Modal Understanding." *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 1780–90.
- Katz, Guy, Clark Barrett, David L. Dill, Kyle Julian, and Mykel J. Kochenderfer. 2017. "Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks." *International Conference on Computer Aided Verification (CAV)*, 97–117. https://doi.org/10.1007/978-3-319-63387-9_5.
- Kavraki, Lydia E., Petr Švestka, Jean-Claude Latombe, and Mark H. Overmars. 1996. "Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces." *IEEE Transactions on Robotics and Automation* 12 (4): 566–80. <https://doi.org/10.1109/70.508439>.
- Kelly, Timothy Patrick. 1998. "Arguing Safety: A Systematic Approach to Managing Safety Cases." PhD thesis, Department of Computer Science, University of York.
- Kelly, Tim, and Rob Weaver. 2004. "The Goal Structuring Notation: A Safety Argument Notation." *Proceedings of the Dependable Systems and Networks Workshop on Assurance Cases*.
- Kerbl, Bernhard, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. "3D Gaussian Splatting for Real-Time Radiance Field Rendering." *ACM Transactions on Graphics* 42 (4): 139:1–14.
- Kerr, Justin, Chung Min Kim, Ken Goldberg, Angjoo Kanazawa, and Matthew Tancik. 2023. "LERF: Language Embedded Radiance Fields." *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 19729–39.
- Khazatsky, Alexander, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, et al. 2024. "DROID: A Large-Scale in-the-Wild Robot Manipulation Dataset." *Robotics: Science and Systems (RSS)*.
- Kim, Moo Jin, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, et al. 2024. "OpenVLA: An Open-Source Vision-Language-Action Model." *arXiv Preprint arXiv:2406.09246*.

- Kirillov, Alexander, Eric Mintun, Nikhila Ravi, Hanzi Mao, et al. 2023. "Segment Anything." *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 4015–26.
- Koopman, Philip. 2020. *How Safe Is Safe Enough? Measuring and Predicting Autonomous Vehicle Safety*. BookBaby.
- Kopetz, Hermann. 2011. *Real-Time Systems: Design Principles for Distributed Embedded Applications*.
- Krotkov, Eric, David Hackett, Lawrence Jackel, Michael Perschbacher, James Pippine, Craig Strauss, Gill Pratt, and Charles Orillat. 2017. "The DARPA Robotics Challenge Finals: Results and Perspectives." *The DARPA Robotics Challenge Finals: Humanoid Robots to the Rescue*, Springer tracts in advanced robotics, vol. 121: 1–26.
- Kumar, Aviral, Aurick Zhou, George Tucker, and Sergey Levine. 2020. "Conservative q-Learning for Offline Reinforcement Learning." *Advances in Neural Information Processing Systems (NeurIPS)* 33: 1179–91.
- Kuwajerwala, Alihusein, Qiao Gu, Sacha Morin, et al. 2024. "ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning." *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 14170–77.
- Landauer, Rolf. 1961. "Irreversibility and Heat Generation in the Computing Process." *IBM Journal of Research and Development* 5 (3): 183–91. <https://doi.org/10.1147/rd.53.0183>.
- LaValle, Steven M. 1998. *Rapidly-Exploring Random Trees: A New Tool for Path Planning*. TR 98-11. Computer Science Department, Iowa State University.
- Leveson, Nancy. 2011. *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press.
- Leveson, Nancy G., and Clark S. Turner. 1993. "An Investigation of the Therac-25 Accidents." *Computer* 26 (7): 18–41.
- Levine, Sergey, Peter Pastor, Alex Krizhevsky, Julian Ibarz, and Deirdre Quillen. 2018. "Learning Hand-Eye Coordination for Robotic Grasping with Deep Learning and Large-Scale Data Collection." *The International Journal of Robotics Research* 37 (4-5): 421–36.
- Li, Mingyang, and Anastasios I. Mourikis. 2014. "Online Temporal Calibration for Camera-IMU Systems: Theory and Algorithms." *The International Journal of Robotics Research* 33 (7): 947–64.
- Liang, Jacky, Wenlong Huang, Fei Xia, et al. 2023. "Code as Policies: Language Model Programs for Embodied Control." *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 9493–500.
- Lions, Jacques-Louis. 1996. *ARIANE 5 Flight 501 Failure: Report by the Inquiry Board*. European Space Agency; Centre National d'Études Spatiales.
- Lipman, Yaron, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nicklas, and Matt Le. 2022. "Flow Matching for Generative Modeling." *arXiv Preprint arXiv:2210.02747*.
- Liu, C. L., and James W. Layland. 1973. "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment." *Journal of the ACM* 20 (1): 46–61.
- Liu, Peiqi, Yaswanth Orru, Jay Vakil, Chris Paxton, Nur Muhammad Mahi Shafiullah, and Lerrel Pinto. 2024. "OK-Robot: What Really Matters in Integrating Open-Knowledge Models for Robotics." *arXiv Preprint arXiv:2401.12202*.
- Liu, Shilong, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jianwei Yang, Chunyuan Li, et al. 2023. "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection." *arXiv Preprint arXiv:2303.05499*.
- Liu, Zhijian, Haotian Tang, Alexander Amini, Xinyu Yang, Huizi Mao, Daniela Rus, and Song Han. 2023. "BEVFusion: Multi-Task Multi-Sensor Fusion with Unified Bird's-Eye View Representation." *IEEE International Conference on Robotics and Automation (ICRA)*, 2774–81.
- Lozano-Pérez, Tomás. 1983. "Spatial Planning: A Configuration Space Approach." *IEEE Transactions on Computers* C-32 (2): 108–20. <https://doi.org/10.1109/TC.1983.1676196>.
- Lynch, Kevin M, and Frank C Park. 2017. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press.
- Ma, Yingdong, Yifan Song, Ziyuan Cao, Yu Zhang, et al. 2024. "A Survey on Vision-Language-Action Models for Embodied AI." *arXiv Preprint arXiv:2405.14093*.
- Main Commission Aircraft Accident Investigation Warsaw and German Federal Bureau of Aircraft Accident Investigation. 1994. *Report on the Accident to Airbus A320-211 Aircraft in Warsaw on 14 September 1993*. Polish Ministry of Transport; Maritime Economy / BFU.
- Mandlekar, Ajay, Danfei Xu, Roberto Mart'in-Mart'in, Yuke Zhu, Li Fei-Fei, and Silvio Savarese. 2020. "Human-in-the-Loop Imitation Learning Using Remote Teleoperation." *arXiv Preprint arXiv:2012.06733*.
- Marr, David. 1982. *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*. W. H. Freeman; Company.
- Mason, Matthew T. 2001. *Mechanics of Robotic Manipulation*. MIT Press.

- Matsuki, Hidenobu, Ryo Murai, Paul H. J. Kelly, and Andrew J. Davison. 2024. "Gaussian Splatting SLAM." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 18039–48.
- Maxwell, J Clerk. 1868. "On Governors." *Proceedings of the Royal Society of London* 16: 270–83.
- Mindell, David A. 2008. *Digital Apollo: Human and Machine in Spaceflight*. MIT Press.
- Mitchell, Ian M., Alexandre M. Bayen, and Claire J. Tomlin. 2005. "A Time-Dependent Hamilton-Jacobi Formulation of Reachable Sets for Continuous Dynamic Games." *IEEE Transactions on Automatic Control* 50 (7): 947–57. <https://doi.org/10.1109/TAC.2005.851439>.
- Moravec, Hans. 1988. *Mind Children: The Future of Robot and Human Intelligence*. Harvard University Press.
- Moravec, Hans, and Alberto Elfes. 1985. "High Resolution Maps from Wide Angle Sonar." *IEEE International Conference on Robotics and Automation* 2: 116–21.
- Museth, Ken. 2013. "VDB: High-Resolution Sparse Volumes with Dynamic Topology." *ACM Transactions on Graphics* 32 (3): 1–22. <https://doi.org/10.1145/2487228.2487235>.
- NASA Manned Spacecraft Center. 1969. *Apollo 11 Mission Report*. MSC-01855. National Aeronautics; Space Administration.
- National Highway Traffic Safety Administration (NHTSA). 2017. *PE 16-007: Investigation of Tesla Motors Model S Automatic Emergency Braking and Autopilot Systems*. PE 16-007. Office of Defects Investigation (ODI), U.S. Department of Transportation.
- National Highway Traffic Safety Administration, Office of Defects Investigation. 2023. *ODI Resume, Investigation PE23-018: Automated Driving System*. Nos. PE23-018. U.S. Department of Transportation.
- National Transportation Safety Board. 2017. *Collision Between a Car Operating with Automated Vehicle Control Systems and a Tractor-Semitrailer Truck Near Williston, Florida, May 7, 2016*. HAR-17/02. National Transportation Safety Board.
- National Transportation Safety Board. 2019. *Collision Between Vehicle Controlled by Developmental Automated Driving System and Pedestrian, Tempe, Arizona, March 18, 2018*. HAR-19/03. National Transportation Safety Board.
- Newcombe, Richard A, Shahram Izadi, Otmar Hilliges, David Molyneaux, David Kim, Andrew J Davison, Pushmeet Kohi, Jamie Shotton, Steve Hodges, and Andrew Fitzgibbon. 2011. "KinectFusion: Real-Time Dense Surface Mapping and Tracking." *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 127–36.
- Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees Nair, et al. 2024. "Octo: An Open-Source Generalist Robot Policy." *arXiv Preprint arXiv:2404.08839*.
- Oleynikova, Helen, Zachary Taylor, Marius Fehr, Roland Siegwart, and Juan Nieto. 2017. "Voxblox: Incremental 3D Euclidean Signed Distance Fields for on-Board MAV Planning." *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1366–73.
- Open X-Embodiment Collaboration, Abhishek Padalkar, Acorn Pooley, et al. 2024. "Open X-Embodiment: Robotic Learning Datasets and RT-X Models." *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 6892–903.
- Oquab, Maxime, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, et al. 2023. "Dinov2: Learning Robust Visual Features Without Supervision." *arXiv Preprint arXiv:2304.07193*.
- Parasuraman, Raja, Thomas B. Sheridan, and Christopher D. Wickens. 2000. "A Model for Types and Levels of Human Interaction with Automation." *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans* 30 (3): 286–97. <https://doi.org/10.1109/3468.844354>.
- Philon, Jonah, and Sanja Fidler. 2020. "Lift, Splat, Shoot: Encoding Images from Arbitrary Cameras by Predicting Depth." *Proceedings of the European Conference on Computer Vision (ECCV)*, 272–85.
- Pomerleau, Dean A. 1989. "ALVINN: An Autonomous Land Vehicle in a Neural Network." *Advances in Neural Information Processing Systems (NeurIPS)* 1: 305–13.
- Presidential Commission on the Space Shuttle Challenger Accident. 1986. *Report of the Presidential Commission on the Space Shuttle Challenger Accident*. U.S. Government Printing Office.
- Quality Control Systems Corp. 2019. *NHTSA's Implausible Safety Claim for Tesla's Autosteer Driver Assistance System*. Quality Control Systems Corp.
- Quinn Emanuel Urquhart and Sullivan, LLP. 2024. *Report to the Boards of Directors of Cruise LLC, GM Cruise Holdings LLC, and General Motors Holdings LLC Regarding the October 2, 2023 Accident in San Francisco*. Quinn Emanuel Urquhart; Sullivan, LLP.
- Radford, Alec, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, et al. 2021. "Learning Transferable Visual Models from Natural Language Supervision." *International Conference on Machine Learning*, 8748–63.
- Reeves, Glenn E. 1997. *What Really Happened on Mars?* Electronic mail message, Jet Propulsion Laboratory.

- Rosinol, Antoni, Andrew Violette, Marcus Abate, Nathan Hughes, Yun Chang, and Luca Carlone. 2021. "Kimera: From SLAM to Hierarchical 3D Dynamic Scene Graphs." *The International Journal of Robotics Research* 40 (12–14): 1410–44. <https://doi.org/10.1177/02783649211056674>.
- Ross, Stéphane, Geoffrey Gordon, and Drew Bagnell. 2011. "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning." *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 627–35.
- Saltzer, Jerome H, David P Reed, and David D Clark. 1984. "End-to-End Arguments in System Design." *ACM Transactions on Computer Systems (TOCS)* 2 (4): 277–88.
- Satyanarayanan, Mahadev. 2001. "Pervasive Computing: Vision and Challenges." *IEEE Personal Communications* 8 (4): 10–17. <https://doi.org/10.1109/98.944002>.
- Seok, Sangok, Albert Wang, Meng Yee Chuah, Dong Jin Hyun, JaeJun Lee, David M. Otten, Jeffrey H. Lang, and Sangbae Kim. 2015. "Design Principles for Energy-Efficient Legged Locomotion and Implementation on the MIT Cheetah Robot." *IEEE/ASME Transactions on Mechatronics* 20 (3): 1117–29.
- Sha, Lui. 2001. "Using Simplicity to Control Complexity." *IEEE Software* 18 (4): 20–28.
- Sha, Lui, Ragunathan Rajkumar, and John P. Lehoczky. 1990. "Priority Inheritance Protocols: An Approach to Real-Time Synchronization." *IEEE Transactions on Computers* 39 (9): 1175–85.
- Sheridan, Thomas B. 1992. *Telerobotics, Automation, and Human Supervisory Control*. Wiley.
- Sheridan, Thomas B., and William L. Verplank. 1978. *Human and Computer Control of Undersea Teleoperators*. Technical Report N00014-77-C-0256. Man-Machine Systems Laboratory, Massachusetts Institute of Technology.
- Siciliano, Bruno, Lorenzo Sciacivco, Luigi Villani, and Giuseppe Oriolo. 2009. *Robotics: Modelling, Planning and Control*. Springer Science & Business Media.
- Slotine, Jean-Jacques E, and Weiping Li. 1991. *Applied Nonlinear Control*. Prentice Hall.
- Spencer, Jonathan, Sanjiban Choudhury, Arun Venkatraman, Brian Ziebart, and J. Andrew Bagnell. 2021. "Feedback in Imitation Learning: The Three Regimes of Covariate Shift." *arXiv Preprint arXiv:2102.02872*.
- Spong, Mark W. 1987. "Modeling and Control of Elastic Joint Robots." *Journal of Dynamic Systems, Measurement, and Control* 109 (4): 310–19.
- Stephenson, Arthur G. et al. 1999. *Mars Climate Orbiter Mishap Investigation Board Phase i Report*. National Aeronautics; Space Administration.
- Sundaralingam, Balakumar, Siva Kumar Sastry Hari, Adam Fishman, Caelan Garrett, Karl Van Wyk, Valts Blukis, Alexander Millane, et al. 2023. "CuRobo: Parallelized Collision-Free Robot Motion Generation." *IEEE International Conference on Robotics and Automation (ICRA)*, 11124–31.
- Sutton, Richard. 2019. "The Bitter Lesson." *Incomplete Ideas (Blog)* 13 (1): 38.
- Sutton, Richard S., and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press.
- Szeliski, Richard. 2022. *Computer Vision: Algorithms and Applications*. 2nd ed. Springer.
- Tedrake, Russ. 2023. *Robotic Manipulation: Perception, Planning, and Control*. Massachusetts Institute of Technology.
- Toulmin, Stephen E. 1958. *The Uses of Argument*. Cambridge University Press.
- U.S. Chemical Safety and Hazard Investigation Board. 2014. *Investigation Report: Drilling Rig Explosion and Fire at the Macondo Well (Deepwater Horizon)*. Report No. 2010-10-I-OS. U.S. Chemical Safety Board.
- Vaughan, Diane. 1996. *The Challenger Launch Decision: Risky Technology, Culture, and Deviance at NASA*. University of Chicago Press.
- Wabersich, Kim P., and Melanie N. Zeilinger. 2021. "A Predictive Safety Filter for Learning-Based Control of Constrained Nonlinear Dynamical Systems." *Automatica* 129: 109647.
- Wald, Abraham. 1945. "Sequential Tests of Statistical Hypotheses." *The Annals of Mathematical Statistics* 16 (2): 117–86. <https://doi.org/10.1214/aoms/1177731118>.
- Wensing, Patrick M., Albert Wang, Sangok Seok, David Otten, Jeffrey H. Lang, and Sangbae Kim. 2017. "Proprioceptive Actuator Design in the MIT Cheetah: Impact Mitigation and High-Bandwidth Physical Interaction for Dynamic Legged Robots." *IEEE Transactions on Robotics* 33 (3): 509–22.
- Wiener, Norbert. 1948. *Cybernetics: Or Control and Communication in the Animal and the Machine*. John Wiley & Sons.
- Williams, Samuel, Andrew Waterman, and David Patterson. 2009. "Roofline: An Insightful Visual Performance Model for Multicore Architectures." *Communications of the ACM* 52 (4): 65–76.

- Winograd, Terry. 1972. *Understanding Natural Language*. Elsevier.
- Wu, Philipp, Yiyang Shentu, Zhonghe Yi, Xingyu Lin, Ken Goldberg, and Pieter Abbeel. 2024. "GELLO: A General, Low-Cost, and Intuitive Teleoperation Framework for Robot Manipulation." *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.
- Xiao, Guangxuan, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. "Efficient Streaming Language Models with Attention Sinks." *International Conference on Learning Representations (ICLR)*.
- Yun, Heechul, Gang Yao, Rodolfo Pellizzoni, Marco Caccamo, and Lui Sha. 2013. "MemGuard: Memory Bandwidth Reservation System for Efficient Performance Isolation in Multi-Core Platforms." *Proceedings of the 19th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 55–64.
- Zhang, Zhengyou. 2000. "A Flexible New Technique for Camera Calibration." *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22 (11): 1330–34. <https://doi.org/10.1109/34.888718>.
- Zhao, Tony Z., Vikash Kumar, Sergey Levine, and Chelsea Finn. 2023. "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware." *Robotics: Science and Systems (RSS)*. <https://doi.org/10.15607/RSS.2023.XIX.016>.

Machine Learning Background

This appendix collects the machine learning models, statistical bounds, and spatial representations used throughout this book. Each calculation states what its assumptions support and what still requires measurement on the physical machine.

A.1 How to Use This Appendix

CONSULT this appendix when encountering specific learning systems bottlenecks, statistical verification limits, or representation boundaries across chapters in this book:

When you encounter this systems symptom	Consult	For this quantitative tool
A vision-language model hallucinates actions under perceptual ambiguity	section A.2	Posterior entropy metrics, Shannon uncertainty, and refusal thresholds
Overconfident neural outputs command aggressive actions out-of-distribution	section A.2.3	Softmax simplex geometry and Expected Calibration Error (ECE)
Multi-camera extrinsics, IMU lever arms, or 3D scene geometry drift	section A.3	$SE(3)$ homogeneous transforms and inverse-depth parameterization
Differentiable 3D scene reconstruction diverges during real-time tracking	section A.3.3	3D Gaussian Splatting parameters and covariance projection
Occupancy mapping retains phantom obstacles after objects move	section A.3.4	Recursive Bayesian log-odds voxel updates
Small sensor noise produces explosive, unconstrained actuator commands	section A.4.1	Lipschitz continuity bounds and spectral normalization
A behavioral cloning policy drifts into catastrophic off-trajectory states	section A.5	$\mathcal{O}(T^2\epsilon)$ compounding error and DAGger linear reduction
L2 regression averages bimodal demonstrations into fatal center collisions	section A.5.1	Conditional expectation proof and diffusion symmetry-breaking
Offline reinforcement learning selects ungrounded, destructive motor setpoints	section A.6	Bellman value overestimation and SayCan affordance grounding
Verifying time-varying physical safety properties across continuous traces	section A.7	Signal Temporal Logic (STL) syntax and quantitative robustness degree
Black-box fleet testing fails to certify ultra-rare catastrophic failure rates	section A.8	Binomial Rule of Three ($3/p$) and Astronomical Exposure Wall
Physical testbed wear explodes during fixed-sample policy qualification	section A.9	Sequential Probability Ratio Test (SPRT) and Wald stopping boundaries
Contact collision shocks and latency spikes exceed Gaussian 6-sigma bounds	section A.10	Extreme Value Theory (GEV/GPD) and high-quantile hardware sizing

A.2 Probability, Information Theory, and Calibration

Autonomous machines must reason quantitatively about uncertainty across perception, intent estimation, and world modeling.

A.2.1 Conditional probability and bayes' rule

The conditional probability $P(Y | X)$ characterizes the probability of event Y given observed evidence X . By Bayes' rule:

$$P(\theta | \mathcal{D}) = \frac{P(\mathcal{D} | \theta)P(\theta)}{P(\mathcal{D})}$$

where $P(\theta)$ represents prior belief over model parameters θ , $P(\mathcal{D} | \theta)$ represents the likelihood of physical dataset $\mathcal{D}$, and $P(\theta | \mathcal{D})$ represents the updated posterior belief.

A.2.2 Shannon entropy and ambiguity thresholds

Entropy $H(X)$ quantifies the expected uncertainty or information content in a probability distribution. For a discrete random variable X taking values in alphabet $\mathcal{X}$:¹

$$H(X) = - \sum_{x \in \mathcal{X}} P(x) \log_2 P(x)$$

For two equally scored referents, the *model's* candidate-action entropy is high. A calibrated, task-specific threshold can trigger a grounding refusal (ERR_GROUNDING_AMBIGUITY) and a validated hold while better evidence is sought. A confidently wrong model can have low entropy, so this threshold is only one input to permission.

A.2.3 Softmax overconfidence and calibration error

Standard neural classification heads normalize raw logit vectors $\mathbf{z} \in \mathbb{R}^K$ into categorical distributions using the softmax function:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i/T}}{\sum_{j=1}^K e^{z_j/T}}$$

where $T > 0$ is a temperature hyperparameter. The softmax operator projects logits onto the probability simplex Δ^K strictly through algebraic exponentiation. A large output probability indicates only relative separation between logits, not genuine epistemic certainty.

To quantify whether predicted probabilities match empirical physical frequencies, reliability engineering computes the **Expected Calibration Error (ECE)**:²

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)|$$

where the interval $[0, 1]$ is partitioned into M equal-width bins B_m , $|B_m|$ is sample count in bin m , N is total sample count, $\text{conf}(B_m)$ is mean confidence, and $\text{acc}(B_m)$ is empirical accuracy. In physical autonomy, an uncalibrated model with high ECE reports false certainty during out-of-distribution conditions, misleading safety monitors into withholding supervisory intervention.

A.3 Spatial Representations and Scene Parameterization

Embodied physical agents perceive, manipulate, and navigate within Euclidean 3D space.

A.3.1 Special euclidean group SE(3) transforms

Rigid body transformations in 3D space belong to the Special Euclidean group $SE(3)$, combining a 3×3 rotation matrix $\mathbf{R} \in SO(3)$ with a translation vector $\mathbf{t} \in \mathbb{R}^3$:

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \in SE(3)$$

A homogeneous point $\tilde{\mathbf{p}} = [\mathbf{p}^\top, 1]^\top \in \mathbb{R}^4$ transforms across frames according to linear matrix-vector multiplication:

$$\tilde{\mathbf{p}}' = \mathbf{T} \tilde{\mathbf{p}}$$

Let $\mathbf{T}_{AB}$ map coordinates expressed in frame B into frame A . For world W , robot base B , end effector E , and camera C , the chain is $\mathbf{T}_{WC} = \mathbf{T}_{WB} \mathbf{T}_{BE} \mathbf{T}_{EC}$. A valid rigid transform has inverse $\mathbf{T}^{-1} = [\mathbf{R}^\top, -\mathbf{R}^\top \mathbf{t}; \mathbf{0}^\top, 1]$; numerical accuracy still depends on calibrated transforms and their uncertainty.

¹ | **Shannon Information Entropy:** Entropy quantifies average uncertainty within a specified discrete distribution. Its physical meaning depends on whether that distribution is calibrated to the relevant referents and observations. Low model entropy cannot prove that a physical target was identified correctly.

² | **Expected Calibration Error:** Binned calibration metrics quantify the divergence between neural prediction confidence and realized empirical accuracy across confidence intervals. Standard unregularized deep networks exhibit substantial overconfidence because cross-entropy minimization drives logits to extreme values. Relying on uncalibrated confidence scores to gate safety fallbacks permits undetected distribution shifts to corrupt closed-loop actuation.

A.3.2 Inverse-depth parameterization

In visual-inertial odometry and SLAM, representing distant 3D landmarks in standard Cartesian coordinates (x, y, z) induces extreme non-linearity: small pixel tracking errors produce quadratic depth errors ($\delta z \propto z^2$).

Under **inverse-depth parameterization**, a 3D feature is parameterized by the 6D vector $\mathbf{y} = [x_0, y_0, z_0, \theta, \phi, \rho]^\top$, where (x_0, y_0, z_0) is the optical center when first observed, (θ, ϕ) are azimuth and elevation angles, and $\rho = 1/d$ is inverse depth:³

$$\mathbf{p}(d) = \begin{bmatrix} x_0 \\ y_0 \\ z_0 \end{bmatrix} + \frac{1}{\rho} \begin{bmatrix} \cos \phi \sin \theta \\ -\sin \phi \\ \cos \phi \cos \theta \end{bmatrix}$$

At optical infinity ($d \rightarrow \infty$), inverse depth approaches zero ($\rho \rightarrow 0$). This parameterization can improve estimator conditioning for distant features; covariance shape and numerical stability still depend on the measurement model, noise assumptions, and estimator implementation.

A.3.3 3D Gaussian splatting parameters

Continuous neural scene representations parameterize physical environments through collections of 3D Gaussians (Kerbl et al. 2023). Each Gaussian $G_i(\mathbf{x})$ is defined by four core attributes:

1. **Position** ($\mu_i \in \mathbb{R}^3$): The 3D spatial center.
2. **Covariance matrix** ($\Sigma_i \in \mathbb{R}^{3 \times 3}$): Defining ellipsoidal shape and orientation. To guarantee positive semi-definiteness during gradient descent, Σ_i is factorized into a rotation matrix $\mathbf{R}_i \in SO(3)$ (parameterized by a unit quaternion $\mathbf{q}_i$) and a scaling diagonal matrix $\mathbf{S}_i = \text{diag}(s_{x,i}, s_{y,i}, s_{z,i})$:

$$\Sigma_i = \mathbf{R}_i \mathbf{S}_i \mathbf{S}_i^\top \mathbf{R}_i^\top$$
3. **Color representation** (c_i): Parameterized via Spherical Harmonics (SH) coefficients to model view-dependent specular radiance.
4. **Opacity** ($\alpha_i \in [0, 1]$): Modeled via a sigmoid activation $\alpha_i = \sigma(o_i)$ to ensure continuous density gradients.

During real-time rendering, 3D Gaussians are projected onto camera coordinate frames via Jacobian approximation $\mathbf{J} \in \mathbb{R}^{2 \times 3}$, yielding projected 2D covariance:

$$\Sigma_{2D} = \mathbf{J} \mathbf{W} \Sigma \mathbf{W}^\top \mathbf{J}^\top$$

where $\mathbf{W} \in SO(3)$ is the rotational part of the world-to-camera transform; the translation enters the projected mean, not this covariance. Differentiable tile-based rasterization sorts Gaussians by depth and evaluates alpha-blended color accumulation:

$$C = \sum_{i \in \mathcal{N}} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$

These differentiable parameters support novel-view rendering, not a conservative collision surface by themselves. Clearance permission needs a separately validated geometric extraction, occupancy/ESDF conversion, or independent range sensor that accounts for unknown space and geometric error, as developed in section 18.5.

A.3.4 Recursive Bayesian log-odds occupancy mapping

Discrete volumetric grids model occupancy uncertainty through recursive Bayesian estimation. Let m_i denote the binary occupancy state of voxel i ($m_i = 1$ occupied, $m_i = 0$ free).

Using the log-odds transformation $L(x) = \ln \frac{P(x)}{1-P(x)}$, recursive Bayesian filtering collapses nonlinear probability multiplication into additive linear updates:⁴

³ | **Inverse-Depth Parameterization:** Civera and colleagues used inverse depth to represent distant visual landmarks in monocular estimation. It is a useful coordinate choice, not a guarantee of Gaussian uncertainty or convergent filtering for every sensor configuration.

⁴ | **Recursive Log-Odds Occupancy:** The log-odds formulation maps probability values from the bounded unit interval to the unbounded real line, turning nonlinear Bayesian product updates into fast additions. Dynamic ranges are clamped by upper and lower saturation bounds to prevent saturation lockup. Failure to clamp log-odds values delays the clearing of dynamic obstacles when physical objects vacate the workspace.

$$L_t(m_i) = L_{t-1}(m_i) + \ln \frac{P(m_i | z_t)}{1 - P(m_i | z_t)} - \ln \frac{P(m_i)}{1 - P(m_i)}$$

where $P(m_i | z_t)$ is the inverse sensor model and $P(m_i)$ the prior. Along a reliable range ray, traversed cells receive free-space evidence and the measured return receives occupied evidence; space behind the return remains unknown unless observed separately. The additive update can use bounded fixed-point log odds, but sensor error, moving objects, and evidence age still matter.

A.4 Function Approximation and Sensitivity Bounds

Neural networks deployed as physical controllers must satisfy strict input-output stability guarantees.

A.4.1 Lipschitz continuity and spectral normalization

A neural policy $\pi : \mathcal{X} \rightarrow \mathcal{U}$ is L -Lipschitz continuous if there exists a non-negative scalar constant L such that for all $\mathbf{x}_1, \mathbf{x}_2 \in \mathcal{X}$:

$$\|\pi(\mathbf{x}_1) - \pi(\mathbf{x}_2)\|_{\mathcal{U}} \leq L \|\mathbf{x}_1 - \mathbf{x}_2\|_{\mathcal{X}}$$

For a feedforward network with K layers $\pi(\mathbf{x}) = \phi_K(\mathbf{W}_K \dots \phi_1(\mathbf{W}_1 \mathbf{x}))$, let L_{ϕ_k} bound each activation's slope over the input domain. The composed bound is:

$$L \leq \prod_{k=1}^K L_{\phi_k} \|\mathbf{W}_k\|_2.$$

ReLU and Tanh are 1-Lipschitz in their usual scalar forms; exact GeLU has maximum derivative about 1.1289, so assigning it a factor of one would understate the bound.

A.4.2 Perturbation amplification and supervisory clamping

In physical systems, input state $\mathbf{x}$ is corrupted by sensor measurement noise $\delta \mathbf{x}$ bounded by $\|\delta \mathbf{x}\| \leq \epsilon$. The resulting deviation in commanded control action is bounded by:

$$\|\delta \mathbf{u}\| = \|\pi(\mathbf{x} + \delta \mathbf{x}) - \pi(\mathbf{x})\| \leq L \epsilon$$

The constant has output-units per input-unit. For a constructed policy mapping position in meters to torque in newton-meters, $L = 10^3 \text{ N} \cdot \text{m/m}$ and $\epsilon = 1 \text{ mm}$ allow up to $1 \text{ N} \cdot \text{m}$ of command change. Whether that exceeds torque headroom depends on the joint and operating state. **Spectral normalization** can reduce an upper bound on network sensitivity; it does not establish plant stability, safe torque, or sensor validity. Those require a separate actuator limit and state-dependent permission check.

A.5 Imitation Learning and Compounding Covariate Shift

Behavioral cloning frames physical policy acquisition as supervised regression over demonstrator state-action pairs.

A.5.1 Multimodal regression and the mode-averaging proof

Consider an imitation learning objective minimizing mean squared error (L_2 loss) over demonstration distribution $P(o, a)$:

$$\min_{\pi} \mathbb{E}_{(o,a) \sim \mathcal{D}} [\|a - \pi(o)\|^2]$$

Expanding the objective shows that the optimal unconstrained regressor $\pi^*(o)$ evaluates to the **conditional expectation**:

$$\pi^*(o) = \mathbb{E}[a | o] = \int a P(a | o) da$$

When demonstrator behavior is inherently multimodal—such as dodging an obstacle by steering left ($a = +1.0$) or steering right ($a = -1.0$) with equal probability $P(a = +1) = P(a = -1) = 0.5$ —the conditional expectation computes the arithmetic mean:

$$\pi^*(o) = 0.5(+1.0) + 0.5(-1.0) = 0.0$$

Executing this averaged action commands zero steering deflection, driving the physical robot directly into the central obstacle.

Diffusion policies (Chi et al. 2024) model a conditional distribution over action sequences rather than returning only its mean. Starting from Gaussian noise $\mathbf{a}_K \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, iterative denoising can sample one coherent mode. The sampled mode still needs geometric and actuator validation before execution.

A.5.2 Compounding quadratic error in behavioral cloning

Offline supervised evaluation samples from a fixed demonstration distribution. In closed-loop execution, current action a_t affects the subsequent physical state s_{t+1} and the next observation distribution.

Let π^* be the expert and $\hat{\pi}$ the learner. Suppose its error probability is at most ϵ at each step under the expert's time-indexed state distribution $d_{\pi^*}^t$:

$$\forall t \leq T: \quad \mathbb{E}_{s \sim d_{\pi^*}^t} [\ell(s, \hat{\pi})] \leq \epsilon.$$

An error can move the robot into states that the expert demonstrations rarely visited. Offline accuracy under $d_{\pi^*}^t$ gives no comparable bound on action error in those induced states.

Couple the learner and expert trajectories until their first disagreement. If the error bound applies at each expert-visited step, a union bound gives $P_t \leq t\epsilon$ for a disagreement by step t .⁵ If the additional downstream cost after divergence is at most $C_{\max}$ per step, then:

$$J(\hat{\pi}) - J(\pi^*) \leq C_{\max} \sum_{t=1}^T P_t \leq \frac{T(T+1)}{2} \epsilon C_{\max} = \mathcal{O}(T^2 \epsilon).$$

This is an upper bound under the stated cost and distribution assumptions (Ross et al. 2011). Single-step validation accuracy alone cannot predict a particular closed-loop failure probability.

A.5.3 Interactive dataset aggregation (Dagger) linear reduction

The **Dagger** (Dataset Aggregation) algorithm (Ross et al. 2011) addresses this distribution shift by collecting training data under the learner's induced state distribution $d_{\hat{\pi}_i}$.

At each training iteration i :

1. Execute policy $\hat{\pi}_i$ under the applicable safety envelope to collect trajectories $\mathcal{T}_i = \{s_1, s_2, \dots, s_T\}$.
2. Query the expert to label every visited state $s_t \in \mathcal{T}_i$ with the optimal recovery action $a^* = \pi^*(s_t)$.
3. Aggregate the recovery dataset $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a^*)\}$.
4. Retrain policy $\hat{\pi}_{i+1}$ on the aggregated dataset.

For the Dagger bound, make a stronger assumption than the earlier per-step $C_{\max}$: one wrong action followed by the expert changes expected remaining cost by at most $u_{\max}$, independent of horizon. With expert labels on learner-visited states, a no-regret online learner, sufficient hypothesis capacity, and a valid finite-sample bound δ_N , a selected policy can satisfy a bound of the form

$$J(\hat{\pi}) - J(\pi^*) \leq T u_{\max} (\epsilon_N + \gamma_N + \delta_N),$$

where ϵ_N is best-in-class loss on the induced-state distribution, γ_N is average online-learning regret, and δ_N covers finite-sample generalization and policy selection. It scales linearly in T only when $u_{\max} = \mathcal{O}(1)$ and those errors remain controlled. Appending recovery trajectories alone does not establish these conditions; the drift states need expert corrective labels.

A.6 Value Functions and Affordance Grounding

Reinforcement learning optimizes sequential physical decision-making through expected reward accumulation.

⁵ | **Compounding Error in Imitation Learning:** Ross and colleagues analyzed how learner actions shift the state distribution. The quadratic form bounds worst-case expected excess cost under bounded loss; it is not a measured task-failure rate. Expert corrective labels on learner-visited states can reduce this exposure, while independent permission still governs physical execution.

A.6.1 Bellman optimality and out-of-distribution overestimation

The optimal state-action value function $Q^*(s, a)$ satisfies the Bellman optimality equation:

$$Q^*(s, a) = R(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} \left[\max_{a' \in \mathcal{A}} Q^*(s', a') \right]$$

With static offline data, the maximization can query a learned critic on actions poorly represented in training. Value overestimation can then favor an out-of-distribution proposal. That proposal still needs an independently checked action envelope before it reaches an actuator.

A.6.2 SayCan affordance-grounded skill selection

The SayCan framework (Ahn et al. 2023) integrates semantic knowledge from high-capacity language models with physical affordance constraints from trained policies.

Let ℓ represent high-level language instruction and a represent an available low-level skill. The probability of executing skill a in current physical state s decomposes into the product of two probabilities:

$$P(a | s, \ell) \propto P_{\text{LLM}}(a | \ell) \cdot V_{\text{policy}}(s, a)$$

where:

- $P_{\text{LLM}}(a | \ell)$ represents the **semantic prior**, evaluating whether skill a contributes logically to completing instruction ℓ .
- $V_{\text{policy}}(s, a) \in [0, 1]$ represents the **physical affordance probability**, modeled by a learned value function that predicts the probability of successfully executing skill a from current state s .

If the learned value correctly assigns a near-zero score to an unavailable skill, the product deprioritizes it. The learned score is an estimate of success, not a path, contact-force, or clearance guarantee; a separate motion-permission check is needed.

A.7 Signal Temporal Logic and Continuous Robustness

Verifying whether a physical autonomous system satisfies dynamic safety requirements requires moving beyond instantaneous boolean assertions to continuous temporal specifications.

A.7.1 Syntax and semantics of signal temporal logic

Signal Temporal Logic (STL) evaluates continuous-time physical signals $\mathbf{x}(t)$ against temporal requirements defined over time intervals $[a, b]$:

$$\varphi ::= \top \mid \mu \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \Box_{[a,b]}\varphi \mid \Diamond_{[a,b]}\varphi \mid \varphi_1 \mathcal{U}_{[a,b]}\varphi_2$$

where $\mu = (g(\mathbf{x}) \geq 0)$ is an atomic predicate (e.g., clearance $d(t) - d_{\min} \geq 0$), $\Box_{[a,b]}$ denotes the **Always** operator, $\Diamond_{[a,b]}$ denotes the **Eventually** operator, and $\mathcal{U}_{[a,b]}$ denotes the **Until** operator.

A.7.2 Quantitative robustness degree

Beyond a boolean satisfaction verdict ($\mathbf{x} \models \varphi$), STL admits a signed **quantitative robustness metric** $\rho(\mathbf{x}, \varphi, t)$.⁶ For a formula combining pressure, speed, and force, first define dimensionless atomic margins $\bar{g}_i(\mathbf{x}) = g_i(\mathbf{x})/s_i$, where each positive scale s_i has the same SI unit as its predicate. Then the minimum and maximum below compare like quantities:

$$\begin{aligned} \rho(\mathbf{x}, \mu, t) &= \bar{g}(\mathbf{x}(t)) \\ \rho(\mathbf{x}, \neg\varphi, t) &= -\rho(\mathbf{x}, \varphi, t) \\ \rho(\mathbf{x}, \varphi_1 \wedge \varphi_2, t) &= \min(\rho(\mathbf{x}, \varphi_1, t), \rho(\mathbf{x}, \varphi_2, t)) \\ \rho(\mathbf{x}, \Box_{[a,b]}\varphi, t) &= \min_{t' \in [t+a, t+b]} \rho(\mathbf{x}, \varphi, t') \\ \rho(\mathbf{x}, \Diamond_{[a,b]}\varphi, t) &= \max_{t' \in [t+a, t+b]} \rho(\mathbf{x}, \varphi, t') \end{aligned}$$

The sign of ρ indicates satisfaction or violation, while its normalized magnitude is dimensionless and depends on the declared scales. Preserve each raw physical margin and unit separately in the fault record; a single minimum of meters, newtons, and kilopascals has no physical meaning.

⁶ | **Signal Temporal Logic Robustness:** Quantitative semantics map a trace and formula to a signed score. Its magnitude inherits the atomic predicate's units only when all combined predicates use commensurate units; otherwise a declared normalization gives a dimensionless score. Negative scores identify violations for falsification searches, subject to sampled-time and measurement limits.

A.8 Zero-Failure Testing and the Astronomical Exposure Bound

Suppose a release case selects a catastrophic-event target rate $\lambda_* = 10^{-9}/\text{h}$ and asks what zero-event exposure alone can establish at 95 percent confidence. This numerical target is a scenario assumption, not a rate universally assigned to an IEC SIL, an automotive ASIL, or a learned policy.

For n independent equal-duration Bernoulli trials with event probability p in each trial, the probability of zero events is:

$$P(\text{zero failures}) = (1 - p)^n \leq e^{-pn}$$

To exclude a hypothesized per-trial event probability p at one-sided confidence $1 - \alpha$, the zero-event probability under that hypothesis must be at most α :

$$(1 - p)^n \leq \alpha$$

Taking the natural logarithm of both sides:

$$n \ln(1 - p) \leq \ln \alpha$$

For rare events where $p \ll 1$, using the Taylor expansion $\ln(1 - p) \approx -p$ gives:⁷

$$-np \leq \ln \alpha \Rightarrow n \geq \frac{-\ln \alpha}{p} = \frac{\ln(1/\alpha)}{p}$$

For standard 95 percent confidence ($\alpha = 0.05$):

$$n \geq \frac{-\ln(0.05)}{p} = \frac{2.99573}{p} \approx \frac{3}{p}$$

This result is known as the **Rule of Three** in reliability engineering and biostatistics.

A.8.1 The astronomical exposure wall

For continuous operating time, suppose incidents follow a stationary homogeneous Poisson process of rate λ and fleet hours are applicable to the same configuration. Then $P(N_H = 0) = e^{-\lambda H}$, giving $\lambda_U = -\ln(0.05)/H$ after zero incidents. For the selected λ_* , the required exposure is:

$$H \geq \frac{-\ln(0.05)}{10^{-9}/\text{h}} \approx 3.00 \times 10^9 \text{ h} \approx 342,000 \text{ machine-years.}$$

The exposure scales inversely with the selected rate. A fleet could accumulate these hours in principle, but a 100-machine fleet would need about 3,420 years of continuous operation, impractical for the illustrated release cycle. A change to weights, calibration, or firmware requires a configuration-equivalence argument before earlier evidence can be pooled; it does not mechanically reset every observation to zero. Zero-event fleet hours bound incidents of the integrated system under the model, not hazardous proposals stopped by a shield. Architectural evidence and separately measured coverage are needed for claims beyond the observed operating envelope.

A.9 Sequential Probability Ratio Test

Evaluating embodied physical policies on physical hardware is expensive and induces mechanical wear. Fixed-sample-size testing requires pre-specifying sample count N and executing all N physical trials even if the system fails catastrophically on early runs or achieves overwhelming evidence of safety. Conversely, naive early stopping (halting testing as soon as a nominal threshold is breached) introduces optional stopping bias, drastically inflating Type I error (declaring an unsafe policy safe).

Abraham Wald's **Sequential Probability Ratio Test (SPRT)** (Wald 1945) handles optional stopping for two predeclared simple hypotheses by updating a likelihood ratio after each rollout and comparing it with stopping boundaries.

⁷ | **Statistical Rule of Three:** Zero events in n independent Bernoulli trials give the exact one-sided 95 percent upper probability $1 - 0.05^{1/n}$, approximately $3/n$ for small probabilities. Continuous operating hours instead require a declared event-rate model. Neither calculation covers changed configurations or unobserved hazard classes automatically.

A.9.1 Log-likelihood ratio and bernoulli updates

Let $x_1, x_2, \dots, x_m$ denote independent trial outcomes from consecutive physical rollouts. Consider two competing operational hypotheses:

- Null hypothesis H_0 : The system failure probability is unacceptable ($p = p_0$).
- Alternative hypothesis H_1 : The system failure probability meets the safety target ($p = p_1 < p_0$).

After observing m trials, the cumulative log-likelihood ratio Λ_m is:

$$\Lambda_m = \sum_{i=1}^m \ln \frac{P(x_i | H_1)}{P(x_i | H_0)}$$

For discrete Bernoulli trials where $x_i \in \{0, 1\}$ (with $x_i = 1$ denoting task failure and $x_i = 0$ denoting task success), let $k = \sum_{i=1}^m x_i$ be the cumulative failure count across m runs. The log-likelihood ratio simplifies to the exact closed-form evaluation:

$$\Lambda_m = k \ln \left(\frac{p_1}{p_0} \right) + (m - k) \ln \left(\frac{1 - p_1}{1 - p_0} \right)$$

A.9.2 Wald stopping boundaries and decision rules

Given declared Type I error rate α (the maximum allowable probability of falsely accepting H_1 when H_0 is true) and Type II error rate β (the maximum allowable probability of falsely rejecting H_1 when H_1 is true), testing proceeds according to Wald's stopping rule:

$$B < \Lambda_m < A$$

where the usual Wald boundary approximations are:

$$A = \ln \left(\frac{1 - \beta}{\alpha} \right), \quad B = \ln \left(\frac{\beta}{1 - \alpha} \right)$$

At each evaluation step m :

1. If $\Lambda_m \geq A$: Halt and favor H_1 over the specified H_0 under the trial model; this is not a general safety certification.
2. If $\Lambda_m \leq B$: Halt and favor H_0 over the specified H_1 ; do not release on this evidence.
3. If $B < \Lambda_m < A$: The accumulated evidence remains inconclusive; execute rollout $m + 1$.

A.9.3 Average sample number optimality

For independent identically distributed outcomes under these two simple hypotheses, SPRT has an average-sample-number efficiency result relative to comparable tests.⁸ Actual machine-hour savings depend on the two event probabilities, error allowances, overshoot at the stopping boundaries, and trial cost. The displayed thresholds are approximations; a finite-sample operating-characteristic calculation or calibrated simulation is needed for the chosen protocol.

A.10 Extreme Value Theory and Tail Risk

Under its conditions, the Central Limit Theorem describes normalized sums and averages, not the tail of each disturbance. A clearance or thermal design may instead depend on a maximum over a declared exposure block:

$$M_n = \max\{X_1, X_2, \dots, X_n\}$$

Even a true standard Gaussian has one-sided exceedance probabilities of about 2.87×10^{-7} at 5σ and 9.87×10^{-10} at 6σ . At 1,000 independent samples per second, the corresponding mean waits are about 0.97 hours and 11.7 days. Contact and bus-delay data may have different tails or dependence; their family and high quantiles must be estimated from relevant measurements, not assigned from the mechanism's name.

⁸ | **Wald Sequential Probability Ratio Test:** The efficiency result assumes the specified simple hypotheses and independent, identically distributed outcomes. Boundary crossing can overshoot the nominal threshold, so exact error rates and average sample count should be checked for the discrete trial model. A sequential design can stop an unpromising test early without claiming a fixed percentage of saved robot hours.

⁹ | **Tail Family Is an Empirical Claim:** A positive fitted GEV shape suggests a heavy-tail model for the sampled exposure blocks; it does not make all collision forces Fréchet. Block size, dependence, threshold choice, fit diagnostics, and parameter uncertainty affect a high-quantile estimate. A probabilistic quantile alone is not a worst-case structural guarantee.

A.10.1 The fisher-tippett-gnedenko theorem

For independent, identically distributed block observations whose normalized maxima have a nondegenerate limit, the Fisher-Tippett-Gnedenko theorem (Coles 2001) restricts that limit to the **Generalized Extreme Value (GEV)** family:⁹

$$G(z) = \exp \left(- \left[1 + \xi \left(\frac{z - \mu}{\sigma} \right) \right]^{-1/\xi} \right)$$

for $\xi \neq 0$ and $1 + \xi(z - \mu)/\sigma > 0$, with location μ , scale $\sigma > 0$, and shape ξ . Candidate limiting families are:

1. **Gumbel (Type I, $\xi = 0$):** The continuous limit as $\xi \rightarrow 0$, often considered for light-tailed observations:

$$G(z) = \exp \left(- \exp \left(- \frac{z - \mu}{\sigma} \right) \right)$$

2. **Fréchet (Type II, $\xi > 0$, with tail index $\alpha = 1/\xi$):** A possible limit for power-law tails ($P(X > x) \sim x^{-\alpha}$), if supported by the measured sample and fit. The general GEV expression above supplies its CDF without changing location and scale conventions.
3. **Weibull (Type III, $\xi < 0$):** A possible limit when the sampled quantity has a finite upper endpoint. A nominal mechanical stop alone does not establish that every measured force or latency is bounded.

A.10.2 Peaks-over-threshold and generalized Pareto distribution

For a suitable underlying distribution and sufficiently high threshold u , the excess $Y = X - u$ conditional on $X > u$ may be approximated by a **Generalized Pareto Distribution (GPD)**. Peaks-over-threshold analysis preserves more exceedances than one maximum per block but requires attention to clustered, dependent events:

$$H(y) = 1 - \left(1 + \frac{\xi y}{\tilde{\sigma}} \right)^{-1/\xi}$$

where $\tilde{\sigma} > 0$ is the threshold scale and the expression applies on its supported domain, with the $\xi = 0$ case defined by a limit.

A.10.3 Hardware margin and quantile sizing

For a fitted GPD with exceedance fraction N_u/n , one possible high-quantile estimate at per-sample exceedance probability $\epsilon < N_u/n$ and $\xi \neq 0$ is:

$$z_\epsilon = u + \frac{\tilde{\sigma}}{\xi} \left(\left(\frac{n}{N_u} \epsilon \right)^{-\xi} - 1 \right)$$

where N_u is the number of threshold exceedances across n sampled cycles. At $\xi = 0$, its continuous limit is $z_\epsilon = u + \tilde{\sigma} \ln[(N_u/n)/\epsilon]$. Either estimate must carry fit and sampling uncertainty; its quantile does not guarantee a hard physical bound.

The designer can use fitted tails to choose test levels and candidate margins. A safety-critical clearance or force limit additionally needs a validated operating envelope, actuator response, material threshold, and policy for observations outside the evidence range. Series elasticity, physical stops, or an independently timed clamp are possible protections when their own behavior has been measured for the machine.

A.11 Further Reading

For a deeper foundational understanding of machine learning theory, sequential statistics, and extreme value modeling, consult the following authoritative references:

- **Deep Learning** (Goodfellow et al. 2016) provides a comprehensive treatment of neural network optimization, regularized representation learning, and Lipschitz sensitivity bounds.

- **An Introduction to Statistical Modeling of Extreme Values** (Coles 2001) establishes the definitive mathematical foundations for GEV, GPD, and tail risk quantification.
- **Sequential Analysis** (Wald 1945) develops the first-principles mathematical framework and average sample number optimality proofs for the Sequential Probability Ratio Test.
- **A Reduction-Based Framework for Sequential Prediction** (Ross et al. 2011) establishes the theoretical foundations of compounding error in imitation learning and derives the DAgger algorithm.

Control and Dynamics

This appendix collects the control and dynamics models used throughout this book. Their guarantees apply only within a declared state, actuator, disturbance, and timing envelope.

B.1 How to Use This Appendix

CONSULT this appendix when encountering specific physical systems bottlenecks, control interface boundaries, or design sizing decisions across chapters in this book:

When you encounter this systems symptom	Consult	For this quantitative tool
A learned policy commands unsafe actions near physical obstacles or joint limits	section B.2	Control Barrier Function (CBF-QP) safety filters and kinetic stopping insets
Mode switching (policy handover, human intervention, or fallback) causes torque spikes	section B.3	C^2 quintic splines and bumpless transfer blending duration bounds
Sizing the validity horizon, spatial lease, or watchdog timeout for an action chunk	section B.4	Triangular, trapezoidal, and jerk-limited kinematic reachability envelopes
An articulated manipulator stalls, exceeds joint velocity limits, or loses control authority	section B.4.1	Manipulator Jacobian singularities and Yoshikawa manipulability index
Spatial sensor offsets or IMU-camera lever arms corrupt state estimation under rotation	section B.4.3	Spatial coordinate transforms ($SE(3)$) and lever-arm acceleration propagation
Discrete sampling and transport latency erode control loop stability margins	section B.3.3	Discrete zero-order hold (ZOH) phase lag and latency margin penalties
Physics simulation diverges or sim-to-real transfer fails during mechanical contact	section B.5	Unilateral contact complementarity and non-smooth impact mechanics
Sensor occlusions or unobservable states cause state estimation uncertainty to diverge	section B.6	Continuous Lyapunov covariance propagation and Lie derivative observability rank
Teleoperation or compliant contact chatters due to transport delay	section B.8	Passivity conditions, wave variables, and energy-bounded interaction
Underactuated legged or humanoid robots balance dynamic momentum across multi-contact points	section B.7	Centroidal momentum equations and hierarchical Whole-Body Control QP

B.2 Safety and Control Barrier Functions

An independent enforcer can check a learned policy proposal against a control barrier constraint before actuation. A Control Barrier Function (CBF) describes a candidate invariant set; its protection depends on observed state, feasible input, plant model, and sampled response time. Torque ceilings and obstacle clearance may require separate constraints.

Let the system state be $x \in \mathcal{X} \subset \mathbb{R}^n$ and the control input be $u \in \mathcal{U} \subset \mathbb{R}^m$. Suppose we define a safe operating set $\mathcal{S}$ as the superlevel set of a continuously differentiable scalar function $h(x)$:

$$\mathcal{S} = \{x \in \mathcal{X} \mid h(x) \geq 0\}, \quad \partial\mathcal{S} = \{x \in \mathcal{X} \mid h(x) = 0\}$$

- $h(x) > 0$ when the system is strictly inside the safe interior.
- $h(x) = 0$ on the boundary of the safe set.
- $h(x) < 0$ when the system violates safety.

B.2.1 Lie derivatives and forward invariance

Forward invariance means a trajectory starting in $\mathcal{S}$ remains there for the modeled time interval. This claim requires an initial state in $\mathcal{S}$ and an admissible control at every reachable state.

For a continuously differentiable relative-degree-one barrier under continuous-time control, one sufficient condition is:¹

$$\dot{h}(x, u) \geq -\alpha h(x)$$

where $\alpha > 0$ is a chosen gain in s^{-1} , not a measured braking bandwidth.

When $h(x)$ is large, the condition permits some decrease. At $h(x) = 0$, it requires $\dot{h} \geq 0$ in the continuous model. The selected barrier must first encode the plant's momentum and available braking; raw distance cannot be regulated directly by torque in a second-order plant.

For a control-affine dynamical system $\dot{x} = f(x) + g(x)u$, the time derivative $\dot{h}$ is evaluated via chain-rule directional derivatives (Lie derivatives):

$$\dot{h}(x, u) = \frac{\partial h}{\partial x} \dot{x} = \frac{\partial h}{\partial x} f(x) + \frac{\partial h}{\partial x} g(x)u = L_f h(x) + L_g h(x)u$$

where $L_f h(x) \in \mathbb{R}$ represents the unforced drift of clearance (momentum, gravity), and $L_g h(x) \in \mathbb{R}^{1 \times m}$ represents the directional control effectiveness matrix.

B.2.2 Relative degree and kinetic stopping insets

The **relative degree** r of a barrier function $h(x)$ is the number of times it must be differentiated with respect to time before the control input u explicitly appears:

$$L_g L_f^k h(x) = 0 \quad \forall k < r-1, \quad \text{and} \quad L_g L_f^{r-1} h(x) \neq 0$$

For a one-dimensional approach to a stationary obstacle, let raw clearance be $h_r = D$, approach speed $v > 0$, and commanded approach acceleration a_{cmd} . Then $\dot{D} = -v$ and $\ddot{D} = -a_{\text{cmd}}$. Acceleration enters raw clearance only at the second derivative, so h_r has relative degree two with respect to this input. One approach uses a kinetic inset. In a one-dimensional approach to a stationary obstacle, let D be measured clearance, $v > 0$ approach speed, and $a_b > 0$ a validated *lower bound* on available braking deceleration for the operating state. Then:

$$h_k(D, v) = D - \frac{v^2}{2a_b}, \quad \dot{D} = -v, \quad \dot{v} = a_{\text{cmd}}.$$

For immediate commanded acceleration, $\dot{h}_k = -v - (v/a_b)a_{\text{cmd}}$. Thus $\dot{h}_k + \gamma h_k \geq 0$ gives the affine constraint $a_{\text{cmd}} \leq (a_b/v)(\gamma h_k - v)$ for $v > 0$. At standstill, use a separate start/clearance rule. State error, sampling, and brake-onset delay need additional clearance reserves; the continuous calculation alone does not cover them.

B.2.3 Minimal-intervention quadratic program

For a barrier h_b whose first derivative actually depends on u , the enforcer can choose a command near the nominal proposal while respecting the feasible input set:

$$\begin{aligned} u^* &= \arg \min_{u \in \mathcal{U}} \frac{1}{2} \|u - u_{\text{nom}}\|^2 \\ \text{subject to: } & L_f h_b(x) + L_g h_b(x)u \geq -\alpha h_b(x), \quad u_{\min} \leq u \leq u_{\max} \end{aligned}$$

Writing its half-space as $a^\top u \leq b$, with $a = -L_g h_b(x)^\top$ and $b = L_f h_b(x) + \alpha h_b(x)$, gives this projection for one constraint when $a \neq 0$ and no box bound is active:

$$u^* = u_{\text{nom}} - \lambda^* a, \quad \text{where } \lambda^* = \max\left(0, \frac{a^\top u_{\text{nom}} - b}{\|a\|^2}\right)$$

If the feasible set is nonempty and convex, the strictly convex objective has a unique minimizer. Solver response time and total dispatch delay must be measured for the selected dimension and hardware.

¹ | **Boundary Viability:** Nagumo's tangent-cone condition characterizes invariance under regularity assumptions. The displayed CBF inequality is a sufficient design condition for the stated smooth control-affine model. A sampled implementation needs a separate bound for state error, computation, hold interval, and actuator response; failure of one candidate inequality does not prove contact within one integration step.

⚠ Check feasibility before relying on invariance

The projection formula does not solve the box-constrained problem. Clamping its result afterward can violate $a^\top u \leq b$. If no command satisfies both the barrier and actuator limits, the modeled invariant cannot be maintained by this filter. A separately validated fallback may still limit harm; it does not restore the lost forward-invariance claim. Admission must reserve stopping authority before the state reaches this boundary.

B.3 Smooth Trajectories and Bumpless Transfer

Physical systems possess inertia. A velocity step is an ideal acceleration impulse; an acceleration step is a finite torque step under a simple inertia model. Whether a commanded seam causes resonance or damage depends on the plant, load, and controller response.

B.3.1 Analytical quintic splines

A quintic (5th-degree) polynomial trajectory segment $\theta(t)$ over time interval $t \in [0, T]$ is defined as:

$$\theta(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5$$

The six coefficients $[a_0, \dots, a_5]$ are uniquely determined by matching six boundary conditions at $t = 0$ and $t = T$:

$$\begin{aligned} \theta(0) &= q_0, & \dot{\theta}(0) &= \dot{q}_0, & \ddot{\theta}(0) &= \ddot{q}_0 \\ \theta(T) &= q_1, & \dot{\theta}(T) &= \dot{q}_1, & \ddot{\theta}(T) &= \ddot{q}_1 \end{aligned}$$

Solving the resulting linear system yields the exact analytical formulas:

$$\begin{aligned} a_0 &= q_0 \\ a_1 &= \dot{q}_0 \\ a_2 &= \frac{1}{2} \ddot{q}_0 \\ a_3 &= \frac{10(q_1 - q_0) - (6\dot{q}_0 + 4\dot{q}_1)T - (3\ddot{q}_0 - \ddot{q}_1)\frac{T^2}{2}}{T^3} \\ a_4 &= \frac{-15(q_1 - q_0) + (8\dot{q}_0 + 7\dot{q}_1)T + (3\ddot{q}_0 - 2\ddot{q}_1)\frac{T^2}{2}}{T^4} \\ a_5 &= \frac{6(q_1 - q_0) - 3(\dot{q}_0 + \dot{q}_1)T - (\ddot{q}_0 - \ddot{q}_1)\frac{T^2}{2}}{T^5} \end{aligned}$$

The finite-duration polynomial has quadratic jerk $\ddot{\theta}(t) = 6a_3 + 24a_4 t + 60a_5 t^2$. Matching position, velocity, and acceleration at the endpoints gives a C^2 join to adjacent segments with those same endpoint values. Its peak jerk and resonant response still require an extrema check against the machine's limits.

B.3.2 Bumpless transfer and smoothstep blending

For **bumpless transfer**, let $s = (t - t_0)/\tau_{\text{blend}} \in [0, 1]$ and use the C^2 smoothstep $\alpha(s)$ to interpolate two commands:

$$\alpha(s) = 6s^5 - 15s^4 + 10s^3, \quad u(t) = (1 - \alpha(s))u_{\text{prior}}(t) + \alpha(s)u_{\text{new}}(t).$$

The endpoint first and second derivatives of α vanish. If u is commanded acceleration, its time derivative is commanded-acceleration jerk:

$$j_{\text{cmd}}(t) = \frac{\alpha'(s)}{\tau_{\text{blend}}} (u_{\text{new}} - u_{\text{prior}}) + (1 - \alpha(s))\dot{u}_{\text{prior}} + \alpha(s)\dot{u}_{\text{new}}.$$

Here $\alpha'(s) = 30s^2(1-s)^2$ differentiates with respect to *normalized* time and peaks at 1.875. Only for two fixed, latched endpoint accelerations ($\dot{u}_{\text{prior}} = \dot{u}_{\text{new}} = 0$) does the blend contribution satisfy

$$\max |j_{\text{cmd}}| = \frac{1.875|\Delta u|}{\tau_{\text{blend}}}, \quad \tau_{\text{blend}} \geq \frac{1.875|\Delta u|}{\dot{j}_{\text{allowable}}}.$$

This is a command-rate condition. Measured plant jerk, tracking error, and available stopping clearance remain separate admission checks.

B.3.3 Discrete-time sampling and zero-order hold phase lag

Real-time digital controllers execute periodically at sampling period $T_s = 1/f_s$. Actuator drivers hold commanded setpoints constant between clock ticks using a **Zero-Order Hold (ZOH)**.

At frequencies well below the sampling frequency, the zero-order hold's phase is equivalent to about half a sample period of delay:

$$\tau_{\text{ZOH}} = \frac{T_s}{2}$$

For a closed-loop control system operating at crossover frequency ω_c , this transport delay erodes the system's phase margin Φ_m by:²

$$\Delta\Phi = \omega_c \frac{T_s}{2} \quad (\text{radians}) \approx 28.65^\circ \cdot \left(\frac{\omega_c}{f_s} \right)$$

For fixed modeled compute and bus delays, the added phase lag near crossover is approximately $\Delta\Phi_{\text{total}} = \omega_c(\tau_{\text{comp}} + \tau_{\text{bus}} + T_s/2)$. The controller must validate phase margin over the measured latency and operating envelope; the formula alone does not predict a particular contact response.

B.4 Kinematic Reachability and Intent Envelopes

Kinematic reachability characterizes the spatial set of configurations or end-effector poses an embodied system can reach over a given planning or intent horizon τ .

B.4.1 Forward kinematics and singularities

For an n -joint arm, the **forward-kinematics** map $f(q)$ sends joint coordinates $q \in \mathbb{R}^n$ to an end-effector pose $T_{WE} \in SE(3)$:

$$T_{WE} = f(q).$$

In a declared spatial or body twist convention, the **manipulator Jacobian** $J(q) \in \mathbb{R}^{6 \times n}$ relates joint rates to the six-vector end-effector twist V_E :

$$V_E = J(q)\dot{q}.$$

For a six-dimensional task and $n \geq 6$, rank loss ($\det(JJ^\top) = 0$) marks a **kinematic singularity**. Tracking a nonzero twist along a lost direction would require unbounded joint rates in the ideal inverse model; real actuators saturate first. A task-specific design may monitor $w(q) = \sqrt{\det(JJ^\top)}$ against a validated threshold.³

B.4.2 Trajectory transit time and reachable distance

For a one-dimensional rest-to-rest move with symmetric available acceleration a_{max} and speed ceiling v_{max} , the shortest nominal transit has two regimes:

1. **Triangular profile (acceleration-limited):** For short horizons where peak velocity v_{max} is not attained ($d \leq d_{\text{crit}} = v_{\text{max}}^2/a_{\text{max}}$):

$$t_{\min} = 2\sqrt{\frac{d}{a_{\text{max}}}}, \quad d_{\text{reach}}(\tau) = \frac{1}{4}a_{\text{max}}\tau^2$$

² | **Phase Margin and Delay:** A pure delay contributes phase $-\omega\tau$. The half-sample approximation applies in the baseband; actual stability depends on the sampled plant, compensation, jitter, and measured total response. Consuming modeled phase margin predicts loss of the stated stability reserve, not a specific destructive failure mode.

³ | **Yoshikawa Manipulability Measure:** The measure summarizes the velocity ellipsoid for a chosen task Jacobian. Near rank loss, inverse-kinematics commands can exceed joint-rate or torque limits. The actual drive response and fallback depend on the specified controller and hardware.

2. **Trapezoidal profile (velocity-limited):** For larger horizons ($d > d_{\text{crit}}$):

$$t_{\min} = \frac{d}{v_{\max}} + \frac{v_{\max}}{a_{\max}}, \quad d_{\text{reach}}(\tau) = v_{\max} \left(\tau - \frac{v_{\max}}{a_{\max}} \right)$$

With $|\dot{a}| \leq j_{\max}$, reaching full acceleration from zero takes at least $a_{\max}/j_{\max}$, if the profile reaches it at all. The minimum-time trajectory may have four or seven jerk phases, so an unconditional additive offset to the trapezoidal time is invalid. Solve the constrained profile for the stated start/end velocity and acceleration, then include the stopping suffix and response delay when sizing a lease.

B.4.3 Spatial coordinate frames and lever-arm kinematics

Rigid-body pose in three-dimensional space is parameterized by the Special Euclidean group $SE(3)$. A rigid transform from frame B to frame A is represented by the homogeneous transformation matrix $\mathbf{T}_{AB} \in SE(3)$:

$$\mathbf{T}_{AB} = \begin{bmatrix} \mathbf{R}_{AB} & \mathbf{p}_{AB} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad \mathbf{R}_{AB} \in SO(3), \quad \mathbf{p}_{AB} \in \mathbb{R}^3$$

When sensors—such as IMUs or cameras—are displaced from the body center of rotation by lever arm $\mathbf{r}_{P/O}$, rotational motion induces apparent velocities and accelerations at sensor location P . The linear velocity $\mathbf{v}_P$ and linear acceleration $\mathbf{a}_P$ satisfy:

$$\mathbf{v}_P = \mathbf{v}_O + \boldsymbol{\omega} \times \mathbf{r}_{P/O}$$

$$\mathbf{a}_P = \mathbf{a}_O + \boldsymbol{\alpha} \times \mathbf{r}_{P/O} + \boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{r}_{P/O})$$

where $\boldsymbol{\omega}$ is the body angular velocity, $\boldsymbol{\alpha} = \dot{\boldsymbol{\omega}}$ is the angular acceleration, $\boldsymbol{\alpha} \times \mathbf{r}$ is the tangential acceleration, and $\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{r})$ is the centripetal acceleration. State estimation and perception pipelines must compensate for these kinematic lever-arm terms when synchronizing multi-camera extrinsics and fusing IMU data.

B.5 Rigid Body Contact Mechanics

Simulating and controlling physical interaction—such as locomotion, dexterous grasping, and tool assembly—requires modeling mechanical contact. The governing physical challenge of contact is its non-smooth complementarity: interaction forces arise strictly when bodies establish physical contact.

Let $\phi(q)$ be the signed distance between two bodies:

- $\phi(q) > 0$: Bodies are separated.
- $\phi(q) = 0$: Bodies are touching.
- $\phi(q) < 0$: Bodies are penetrating (a geometric interpenetration that rigid mechanics strictly forbids).

Let f_n be the normal contact force pushing the bodies apart:

- If $\phi(q) > 0$, there is no contact, so $f_n = 0$.
- If bodies are touching, they can only exert repulsive contact force, so $f_n \geq 0$.

Ideal unilateral contact is represented by **complementarity conditions**:

1. Non-penetration: $\phi(q) \geq 0$
2. Unilateral force: $f_n \geq 0$
3. Complementarity: $\phi(q)f_n = 0$

These conditions exclude tensile normal contact. A time-discretized rigid-contact problem becomes a linear complementarity problem only under suitable linearization and contact assumptions; frictional multi-contact models can be nonlinear or require other solvers. They do not establish sim-to-real fidelity by themselves.

B.5.1 Non-smooth collision manifolds and velocity jumps

For frictionless normal impact against a stationary rigid plane, let $v_n^- < 0$ be the pre-impact normal velocity and $e \in [0, 1]$ the restitution coefficient. The normal component obeys $v_n^+ = -ev_n^-$; tangential velocity and a moving boundary require additional equations. With effective normal mass m_{eff} , the normal impulse is $P_n = m_{\text{eff}}(v_n^+ - v_n^-)$. In an ideal zero-duration impact, velocity jumps and acceleration is an impulse rather than an ordinary finite derivative:⁴

$$\lim_{\Delta t \rightarrow 0} \frac{\|\dot{x}(t + \Delta t) - \dot{x}(t)\|}{\Delta t} = \infty$$

Smooth learned models and trajectory gradients may be inaccurate at contact-mode changes. A controller can use an explicit hybrid/contact model, while compliance, pads, or torque limiters can reduce force only within validated stroke, energy, wear, and load-path conditions. Neither modeling nor a passive element alone guarantees safe contact.

B.6 Continuous Covariance Propagation and Observability

When sensors suffer line-of-sight dropouts or environmental occlusion, internal state estimators must project the physical state forward in time. Quantifying how rapidly estimation confidence degrades across blind intervals requires analyzing continuous error covariance dynamics and systems observability.

B.6.1 Continuous lyapunov and riccati covariance dynamics

Consider a continuous-time linear dynamic system governed by:

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) + \mathbf{w}(t)$$

$$\mathbf{y}(t) = \mathbf{C}\mathbf{x}(t) + \mathbf{v}(t)$$

For the continuous white-noise model, $\mathbf{Q}$ and $\mathbf{R}$ are process and measurement noise *intensities*, with covariance kernels $E[\mathbf{w}(t)\mathbf{w}(s)^\top] = \mathbf{Q}\delta(t-s)$ and $E[\mathbf{v}(t)\mathbf{v}(s)^\top] = \mathbf{R}\delta(t-s)$. The model assumes suitable independence and $\mathbf{R} \succ 0$.

Under an optimal continuous-time estimator (the Kalman-Bucy filter (Kalman 1960)), the error covariance matrix $\mathbf{P}(t) = \mathbb{E}[(\mathbf{x} - \hat{\mathbf{x}})(\mathbf{x} - \hat{\mathbf{x}})^\top]$ evolves according to the continuous matrix differential Riccati equation:⁵

$$\dot{\mathbf{P}}(t) = \mathbf{A}\mathbf{P}(t) + \mathbf{P}(t)\mathbf{A}^\top + \mathbf{Q} - \mathbf{P}(t)\mathbf{C}^\top \mathbf{R}^{-1} \mathbf{C}\mathbf{P}(t)$$

When direct sensing is occluded or dropped ($\mathbf{C} = 0$), the measurement correction term vanishes. The covariance growth collapses to the open-loop **continuous Lyapunov differential equation**:

$$\dot{\mathbf{P}}(t) = \mathbf{A}\mathbf{P}(t) + \mathbf{P}(t)\mathbf{A}^\top + \mathbf{Q}$$

The solution propagates initial uncertainty and accumulated $\mathbf{Q}$ through $e^{\mathbf{A}t}$. Unstable modes can grow exponentially; a random walk with $\mathbf{A} = 0$ grows linearly in covariance. A state-memory record should carry the calibrated growth model and expire tracking permission before its declared error bound crosses the physical tolerance. Covariance alone is not a hard error envelope unless its tail and model assumptions are justified.

B.6.2 Observability rank and Lie derivative extensions

A linear system is observable on $[0, T]$ if its initial state can be uniquely recovered from outputs and known inputs. For the linear time-invariant pair $(\mathbf{A}, \mathbf{C})$, the **observability matrix** has full *column* rank n exactly when the state is observable:

$$\mathcal{O} = \begin{bmatrix} \mathbf{C} \\ \mathbf{C}\mathbf{A} \\ \vdots \\ \mathbf{C}\mathbf{A}^{n-1} \end{bmatrix}, \quad \text{rank}(\mathcal{O}) = n$$

⁴ | **Moreau Time-Stepping and Non-Smooth Mechanics:** Measure-differential models represent ideal impulses without resolving microscopic contact duration. Numerical contact solvers still require a validated geometry, friction law, and step-size study for the plant; a smooth optimizer that omits impact can miss a relevant force transient.

⁵ | **Continuous Matrix Riccati Equation:** The measurement term reduces modeled covariance when valid measurements arrive. During occlusion, propagation follows the plant and process-noise model. Its growth may be bounded, linear, polynomial, or exponential depending on the dynamics and noise; the estimator must compare the resulting bound with the task tolerance.

⁶ | **Hermann-Krener Nonlinear Observability:** Full rank of the observation codistribution is a local structural test, not a sensor-noise or finite-horizon accuracy bound. A missing state direction may require another sensor, an active maneuver with its own permission, or a restricted operating envelope.

For a smooth autonomous nonlinear system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$, $\mathbf{y} = \mathbf{h}(\mathbf{x})$, a local rank test builds the codistribution from differentials of successive Lie derivatives $dL_f^k \mathbf{h}(\mathbf{x})$ until its rank stops growing.⁶ Full local rank n is a sufficient local weak-observability condition under its regularity assumptions; a finite truncation at $n - 1$ is not universally sufficient.

If $\text{rank}(\mathcal{O}) < n$, $\ker(\mathcal{O})$ contains initial-state differences that produce identical ideal outputs. Measurement updates cannot resolve those directions from this sensor model, though a particular Kalman gain need not contain literal zero entries. The estimator should expose such unobservable components and maintain an independent growth bound or restrict the task.

B.7 Centroidal Dynamics and Floating-Base Control

Legged robots and dynamic humanoids differ fundamentally from fixed-base manipulators: they possess an underactuated 6-degree-of-freedom floating base that cannot be commanded directly by motors. The robot can only move its center of mass (CoM) and alter its spatial orientation by exchanging contact forces with the environment through its feet or hands.

B.7.1 Floating-base equations of motion

Let $\mathbf{q} = [\mathbf{q}_b^\top, \mathbf{q}_j^\top]^\top \in SE(3) \times \mathbb{R}^n$ represent the generalized coordinates, where $\mathbf{q}_b \in SE(3)$ is the floating-base pose and $\mathbf{q}_j \in \mathbb{R}^n$ are the actuated joint angles. The equations of motion are (Siciliano et al. 2009):

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{S}^\top \tau + \sum_{k=1}^{N_c} \mathbf{J}_k(\mathbf{q})^\top \mathbf{f}_k$$

where $\mathbf{S} = [\mathbf{0}_{n \times 6}, \mathbf{I}_{n \times n}]$ is the actuation selection matrix separating the 6 unactuated floating-base coordinates from the n actuated joints, and $\mathbf{J}_k(\mathbf{q}) = \partial \mathbf{p}_k / \partial \mathbf{q} \in \mathbb{R}^{3 \times (n+6)}$ is the contact point Jacobian for contact wrench $\mathbf{f}_k$.

B.7.2 Centroidal momentum dynamics

Projecting the floating-base dynamics onto the robot's center of mass $\mathbf{c} \in \mathbb{R}^3$ yields the Newton-Euler centroidal momentum equations (Lynch and Park 2017):

$$\begin{aligned} \dot{\mathbf{h}}_{\text{lin}} &= m\ddot{\mathbf{c}} = \sum_{k=1}^{N_c} \mathbf{f}_k + m\mathbf{g} \\ \dot{\mathbf{h}}_{\text{ang}} &= \sum_{k=1}^{N_c} [(\mathbf{p}_k - \mathbf{c}) \times \mathbf{f}_k + \tau_k] \end{aligned}$$

where $\mathbf{h}_{\text{lin}} \in \mathbb{R}^3$ is linear momentum, $\mathbf{h}_{\text{ang}} \in \mathbb{R}^3$ is angular momentum about the CoM, and $(\mathbf{p}_k, \mathbf{f}_k, \tau_k)$ are the position, force, and moment at each active contact patch.⁷

B.7.3 Physical contact feasibility constraints

Because the environment cannot pull on the robot and contact surfaces exhibit finite friction, admissible contact wrenches must satisfy three physical invariants:

1. **Unilateral Normal Force:** Feet can only push against the ground:

$$f_{k,\perp} = \mathbf{f}_k^\top \hat{\mathbf{n}}_k \geq 0$$

2. **Coulomb Friction Cone:** Tangential shear forces cannot exceed normal traction:

$$\|\mathbf{f}_{k,\parallel}\| = \|\mathbf{f}_k - f_{k,\perp} \hat{\mathbf{n}}_k\| \leq \mu f_{k,\perp}$$

3. **Support condition for coplanar contacts:** Under flat-ground assumptions, the Zero Moment Point (ZMP) should remain inside the effective support polygon with a case-specific inset.⁸

$$\mathbf{p}_{\text{ZMP}} \in \mathcal{P}_{\text{support}}, \quad \mathcal{P}_{\text{support}} = \text{Conv} \left(\bigcup_k \mathcal{A}_k \right),$$

where $\mathcal{A}_k$ is each active contact patch projected onto that plane.

⁷ | **Centroidal Momentum Matrix:** Centroidal dynamics summarize the external wrench needed for a proposed CoM and angular-momentum change. A controller still needs feasible contacts and actuator torque; omitting angular momentum can mispredict the required contact moments.

⁸ | **Zero Moment Point (ZMP):** ZMP is useful for a specified coplanar support model. Edge contact, friction loss, and multi-height supports require more general wrench feasibility checks; crossing a modeled polygon edge does not by itself predict a particular fall.

B.7.4 Hierarchical whole-body control quadratic program

One whole-body controller can track torso and foot tasks while constraining dynamics and contact forces. For an illustrative sampled implementation, a conservative *polyhedral* friction approximation makes the following convex quadratic program possible; the exact circular norm cone in the preceding equation instead gives a second-order cone constraint:

$$\begin{aligned} \min_{\dot{\mathbf{q}}, \mathbf{f}, \tau} \quad & \sum_i w_i \|\mathbf{J}_i \ddot{\mathbf{q}} + \dot{\mathbf{J}}_i \dot{\mathbf{q}} - \ddot{\mathbf{x}}_i^{\text{des}}\|^2 + w_\tau \|\tau\|^2 + w_f \|\mathbf{f}\|^2 \\ \text{subject to:} \quad & \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{S}^\top \tau + \sum_{k=1}^{N_c} \mathbf{J}_k^\top \mathbf{f}_k \\ & \mathbf{F}_k \mathbf{f}_k \leq \mathbf{0}, \quad f_{k,\perp} \geq 0 \quad (\text{conservative friction pyramid}) \\ & |\tau| \leq \tau_{\max}, \quad \mathbf{q}_{j,\min} \leq \mathbf{q}_j \leq \mathbf{q}_{j,\max} \end{aligned}$$

Here $\mathbf{F}_k$ describes the chosen inner approximation to the measured friction cone. The displayed joint-position bounds concern the current configuration; a real controller must also constrain predicted motion between samples. A 1 ms solve is an example deadline to validate with the selected contact count, solver, processor, friction estimate, and fallback. A feasible numerical solution does not itself prove that a foothold will support the load.

B.8 Passivity and Energy-Bounded Interaction

During teleoperation or stiff contact, transport delay can reduce the damping margin of a feedback loop. **Passivity** is an energy-accounting condition useful for analyzing a specified interconnection; it does not remove the need to verify actuator saturation, contact geometry, sampling, and stored energy.

B.8.1 Energy storage and the passivity definition

Consider a continuous-time dynamical system with state $\mathbf{x}(t)$, control input $\mathbf{u}(t)$ (an effort variable, such as force or torque $\mathbf{F}$), and measured output $\mathbf{y}(t)$ (a flow variable, such as velocity $\mathbf{v}$). The system is defined as **passive** if there exists a continuously differentiable, non-negative storage function $V(\mathbf{x}) \geq 0$ (representing the total internal physical energy stored in mass momentum and structural compliance) such that:

$$\dot{V}(\mathbf{x}) \leq \mathbf{u}(t)^\top \mathbf{y}(t) - g(\mathbf{x}) \quad (\text{B.1})$$

where $\mathbf{u}(t)^\top \mathbf{y}(t)$ represents instantaneous power injected into the system, and $g(\mathbf{x}) \geq 0$ denotes internal energy dissipation (such as mechanical damping or electrical resistance). Integrating equation B.1 over any operational duration $[0, T]$ yields the fundamental passivity energy inequality:

$$\int_0^T \mathbf{u}(t)^\top \mathbf{y}(t) dt \geq V(\mathbf{x}(T)) - V(\mathbf{x}(0)) + \int_0^T g(\mathbf{x}(t)) dt \geq -V(\mathbf{x}(0)) \quad (\text{B.2})$$

Under equation B.2 and the declared port orientation, the net energy extractable from this modeled component is bounded by its initial storage. A loaded mechanism can still release enough stored energy to injure or damage; passivity is not a contact-force limit.

Negative-feedback interconnection of compatible passive components preserves a passive energy balance when ports and signs match. Bounded or asymptotic state stability requires additional storage and detectability conditions. A human, powered tool, or active environment cannot be assumed passive without evidence.

B.8.2 The transport delay destabilization hazard

An untreated effort/flow communication channel need not remain passive when delayed. A fixed pure delay contributes phase $\Delta\phi = -\omega\tau_d$ at angular frequency ω , but no universal 90° threshold determines the sign of an arbitrary coupled mechanical impedance.

For a particular controller, late force feedback can oppose the operator's current motion poorly and inject energy into the closed loop. Whether this causes chatter depends on the port signs, damping, delay distribution, and contact stiffness; measure those conditions rather than inferring failure from latency alone.

B.8.3 Passivity preservation via wave variables

For a delay channel with specified port orientation, Anderson and Spong (1989)'s **wave variable transformation** maps effort $\mathbf{F}$ and flow $\dot{\mathbf{x}}$ into incoming and outgoing waves with characteristic impedance $b > 0$:

$$\mathbf{u}_m(t) = \frac{\mathbf{F}_m(t) + b\dot{\mathbf{x}}_m(t)}{\sqrt{2b}}, \quad \mathbf{w}_m(t) = \frac{\mathbf{F}_m(t) - b\dot{\mathbf{x}}_m(t)}{\sqrt{2b}} \quad (\text{B.3})$$

Computing instantaneous power from the wave transformation in equation B.3 reveals:

$$P_m(t) = \mathbf{F}_m(t)^\top \dot{\mathbf{x}}_m(t) = \frac{1}{2} \|\mathbf{u}_m(t)\|^2 - \frac{1}{2} \|\mathbf{w}_m(t)\|^2$$

For an ideal fixed delay, a received wave is a delayed transmitted wave. The channel can hold energy in flight, so a same-window inequality between master and slave port powers needs a channel-storage term: $E_{\text{ch}}(T) - E_{\text{ch}}(0) \leq \int_0^T (P_m - P_s) dt$ with consistent signs. The scattering construction can preserve channel passivity under its assumptions; time-varying delay, packet loss, sampling, saturation, and powered endpoints require additional compensation or tests. It does not guarantee contact-force safety or convergence of the complete human-robot system.

B.9 Further Reading

For a deeper theoretical foundation in control theory, robotics, and dynamical systems, consult the following authoritative references:

- **Modern Robotics: Mechanics, Planning, and Control** (Lynch and Park 2017) provides an elegant, rigorous treatment of kinematics and dynamics using screw theory and Lie groups.
- **Robotics: Modelling, Planning and Control** (Siciliano et al. 2009) serves as an authoritative reference for multi-body robot dynamics, trajectory generation, and operational space control.
- **Feedback Systems: An Introduction for Scientists and Engineers** (Åström and Murray 2021) establishes classical and state-space control theory, frequency-domain stability margins, and state estimation from first principles.

Systems and Hardware

This appendix collects the embedded timing, memory, power, and actuator models used throughout this book. Each bound depends on a specified device, workload, and operating envelope; the equations identify what must be measured before a physical deadline can be claimed.

C.1 How to Use This Appendix

CONSULT this appendix when diagnosing physical systems bottlenecks, timing jitter, electrical rail instability, or hardware interface boundaries across chapters in this book:

When you encounter this systems symptom	Consult	For this quantitative tool
A real-time safety task misses periodic execution deadlines	section C.2	Worst-case execution time (WCET) and Response Time Analysis (RTA)
A software deadlock or kernel panic freezes actuator control loops	section C.2	Watchdog detection and separately measured stop-path response
Inverter bridge MOSFETs burn out or trip overcurrent protections	section C.3	Microcontroller PWM complementary timers and dead-time insertion
Heavy neural inference bursts induce compute brownout resets	section C.4	Inductive voltage droop (LdI/dt) and alpha-power gate delay
High-rate sensor DMA streams cause safety threads to miss deadlines	section C.5	DRAM bank-conflict penalties and non-preemptive burst queuing
Sensor fusion streams suffer temporal registration skew under motion	section C.6	PTP hardware timestamping and bounded clock conversion
Shared memory queues between Linux and MCU return torn or stale data	section C.7	SoC hardware mailboxes, RPSMC, and hardware memory barriers
Joint actuators fail to track sudden torque reversals or overheat	section C.8	Stator electrical time constants (L/R) and lumped thermal ODEs
Emergency braking spikes DC bus voltage and damages power silicon	section C.9	Regenerative capacitive energy surges and dynamic brake choppers
Shared serial buses experience queuing and message jitter	section C.10	CAN FD priority blocking, EtherCAT cycle budgets, and bounded seqlocks
Monocular 3D bounding boxes distort and depth covariance diverges	section C.11	Pinhole intrinsic projection and metric unprojection covariance
A mobile base breaches clearance buffers under surface wetness	section C.12	Kinematic stopping envelopes, information age lag, and velocity clamps

C.2 Real-Time Timers, Watchdogs, and Schedulability

In embedded real-time systems, temporal predictability is as vital as computational correctness. A late control action is not merely degraded; on physical hardware, it is incorrect and potentially destructive.

C.2.1 Real-time task schedulability and response time analysis

A periodic real-time task τ_i is characterized by its worst-case execution time (WCET) C_i , relative deadline D_i , and period T_i . Under a single-core, fixed-priority preemptive model with bounded blocking

and no release jitter (such as Rate-Monotonic Scheduling (Liu and Layland 1973)), schedulability is checked using the worst-case **response time** R_i :

$$R_i = C_i + B_i + I_i \leq D_i$$

where B_i is the maximum blocking duration induced by lower-priority tasks accessing shared resources, and I_i is the interference from higher-priority tasks. The response time is computed via the fixed-point recurrence:

$$R_i^{(k+1)} = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j$$

starting with $R_i^{(0)} = C_i + B_i$ and iterating to a fixed point or until a deadline is exceeded. A finite fixed point with $R_i \leq D_i$ for every task establishes schedulability under these scheduling and interference assumptions. WCET C_i is one component of response time, not the end-to-end sensor-to-actuator latency.

C.2.2 Hardware watchdogs and fault response

To detect a missed execution checkpoint, an embedded design may use a **hardware watchdog timer**. Its clock and response path must be independent enough of the monitored task for the claimed fault set; an independent crystal is one implementation.

The timer is initialized to a threshold value W_{init} . Software execution threads must periodically refresh (or “kick”) the register before the count reaches zero. If $W(t)$ represents the register count at physical time t , and software executes a refresh at epoch t_k , the register resets:¹

$$W(t_k) = W_{\text{init}}$$

If the software misses the configured refresh window, the timer expires and raises its configured interrupt, reset, or external supervisor output. Expiry only detects the missed checkpoint. The safety case must bound expiry-to-drive response, wire any MCU-loss trip independently of the failed MCU, and show that the resulting state-matched brake or hold can stop the plant within its remaining clearance. A reset or safe torque off (STO) command alone may leave a moving load coasting.

C.3 Inverter Timers and Power Electronics Gate Drive

Actuator motor drives employ three-phase half-bridge inverters to convert DC bus voltage into alternating stator currents.

C.3.1 Microcontroller timer dead-time insertion

Each inverter phase comprises a high-side power MOSFET (or IGBT) and a low-side power MOSFET connected across the DC supply rail V_{bus} . Microcontroller timer peripherals generate complementary pulse-width-modulated (PWM) drive signals with hardware-enforced **dead-time insertion** t_{dead} :

$$t_{\text{dead}} \geq t_{\text{off, max}} + t_{\text{prop, skew}} - t_{\text{on, min}}$$

where $t_{\text{off, max}}$ is the maximum turn-off time of the power switch, $t_{\text{on, min}}$ is the minimum turn-on time, and $t_{\text{prop, skew}}$ is the propagation delay mismatch across gate drivers.²

For the specified device and gate-drive delays, adequate dead time reduces overlap between high-side and low-side conduction. If both switches conduct simultaneously, **shoot-through** creates a low-impedance path across the DC bus; current and damage depend on bus impedance, protection, and event duration.

C.4 Inductive Voltage Droop and Gate Delay Stretch

When deep neural network accelerators (such as systolic array matrix engines) transition from idle to full execution, current draw increases by tens of amperes within nanoseconds.

¹ | **Hardware Watchdog Down-Counters:** A configured watchdog expires when its expected service does not occur. Its oscillator, timeout, output wiring, and plant response differ by device. Feeding it merely from an unrelated interrupt does not prove that the monitored control computation progressed; a firmware-fed reset also need not signal a drive. See the Linux watchdog API for reset behavior and section 24.9 for stop selection.

² | **Inverter Dead-Time Insertion:** Timer dead time is selected from switch and driver timing over temperature and load, then verified with the actual gate waveforms. An overlap can create a damaging bus short; a numeric current or fuse outcome requires the circuit impedance and protection response.

C.4.1 Power delivery network impedance and rail droop

Every power delivery network (PDN) exhibits finite parasitic loop inductance L_{PDN} and equivalent series resistance R_{PDN} across PCB traces, package balls, and bond wires. When accelerator execution triggers a transient current step ΔI with rise time t_r (giving current slew rate $dI/dt \approx \Delta I/t_r$), Faraday's law of induction dictates an instantaneous voltage collapse ΔV_{droop} :

$$\Delta V_{\text{droop}} = L_{\text{PDN}} \frac{dI}{dt} + \Delta I \cdot R_{\text{PDN}}$$

If supply voltage $V(t) = V_{\text{nom}} - \Delta V_{\text{droop}}$ approaches a device's characterized minimum operating voltage V_{min} , timing margin may shrink or brownout protection may act.

C.4.2 Digital logic gate delay and the alpha-power law

In sub-micron CMOS silicon, logic gate propagation delay t_{prop} is governed by the Sakurai-Newton **alpha-power law**:³

$$t_{\text{prop}}(V) \propto \frac{C_L \cdot V}{(V - V_{\text{th}})^\alpha}$$

where C_L is the gate load capacitance, V_{th} is the transistor threshold voltage, and $\alpha \in [1.2, 1.5]$ is the velocity saturation index (contrasting with $\alpha = 2.0$ in long-channel theory).

When rail voltage droops from V_{nom} to V_{droop} , propagation delay stretches according to:

$$\frac{t_{\text{prop}}(V_{\text{droop}})}{t_{\text{prop}}(V_{\text{nom}})} = \frac{V_{\text{droop}}}{V_{\text{nom}}} \left(\frac{V_{\text{nom}} - V_{\text{th}}}{V_{\text{droop}} - V_{\text{th}}} \right)^\alpha$$

In a synchronous digital pipeline with clock period T_{clk} , setup time t_{setup} , and clock skew t_{skew} , timing closure requires:

$$t_{\text{prop}} + t_{\text{setup}} + t_{\text{skew}} \leq T_{\text{clk}}$$

Stretching t_{prop} reduces setup slack. A timing failure occurs only if the measured path crosses its slack limit; a brownout reset depends on a separately configured supervisor threshold. Local decoupling can reduce the rail excursion, subject to its impedance and placement.

C.5 DRAM Contention and Interconnect Arbitration

In shared-memory multi-core system-on-chip (SoC) platforms, direct memory access (DMA) transfers and vision prefetching compete with real-time safety monitors for memory controller command queues and physical DRAM banks.

C.5.1 DRAM bank organization and conflict latency

A DRAM chip is structured into multiple independent banks, each possessing a single row buffer. Accessing storage cells requires three sequential operations:

1. **Row activate** (t_{RCD}): Loads a row of storage cells into the bank's row buffer.
2. **Column access** (t_{CAS}): Reads or writes data from the open row buffer onto the data bus.
3. **Row precharge** (t_{RP}): Restores charge to storage capacitors and closes the active row.

When consecutive operations target different rows of the same bank (a *bank conflict*), the controller may need to precharge and activate a row before column access, adding approximately $t_{\text{RP}} + t_{\text{RCD}}$. Each bank can have its own open row. The penalty and number of switches depend on address mapping and the controller schedule.⁴

³ | **Sakurai-Newton Delay Modeling:** The alpha-power law models voltage-dependent gate delay; its parameters are technology dependent. The model predicts reduced setup slack during droop, while a missed deadline, register error, or reset must be established against measured path slack and device thresholds.

⁴ | **DRAM Bank-Conflict Penalties:** A row change in one bank can add precharge and activate time. Video DMA can also queue transactions ahead of a safety read. The resulting delay must be bounded for the specific address mapping, arbitration policy, and workload; a fixed penalty per request is generally unjustified.

C.5.2 Interconnect crossbars and burst contention

An accelerator tile is split into bounded bus transactions; it is not one atomic AXI burst. For an illustrative FIFO controller that admits all K transactions ahead of a safety request, with no priority bypass or new arrivals, the ideal bus-transfer component is

$$t_{\text{queue,ideal}} = \frac{\sum_{k=1}^K B_k}{\text{BW}_{\text{bus}}}.$$

Add the bank-switch penalties for the *actual* address sequence and then the safety request's own transfer and compute times. This is a constructed scheduling model, not a portable worst-case bound: refresh, protocol overhead, controller reordering, and competing traffic can increase latency, while qualified QoS can bypass queued work. Section 26.3 gives a recomputable case. A local, coherently published safety snapshot can avoid this DDR path, but TCM capacity alone does not provide the snapshot or bound its transfer time.

C.6 Clock Synchronization and Network Timestamps

Distributed perception must convert sensor timestamps into a common clock domain with a measured conversion-error bound.

C.6.1 Precision time protocol message exchange

The IEEE 1588 Precision Time Protocol (PTP) synchronizes a Slave clock (S) to a Master clock (M) by exchanging four physical timestamps across the communication link:

1. t_1 : Master transmits a *Sync* frame, recording its local transmission timestamp.
2. t_2 : Slave receives the *Sync* frame, recording its local arrival timestamp.
3. t_3 : Slave transmits a *Delay_Req* frame, recording its local transmission timestamp.
4. t_4 : Master receives the *Delay_Req* frame, recording its local arrival timestamp.

Assuming stable clocks during the exchange and symmetric forward and reverse propagation delay ($D_{M \rightarrow S} = D_{S \rightarrow M} = D$), the time relationships satisfy:

$$t_2 = t_1 + O + D$$

$$t_4 = t_3 - O + D$$

where O is the slave-minus-master clock offset. Under those assumptions, the estimated offset is:

$$O = \frac{(t_2 - t_1) - (t_4 - t_3)}{2}$$

The one-way network propagation delay is:

$$D = \frac{(t_2 - t_1) + (t_4 - t_3)}{2}$$

The slave estimates the mapping to the master domain by subtracting O . Asymmetric path delay, oscillator drift between exchanges, and timestamp error leave a residual that must be bounded for evidence-age checks.

C.6.2 Physical layer hardware timestamping

Software timestamping includes operating-system dispatch and queue variation, which must be measured for the selected network stack.

PTP-capable hardware can timestamp near the Ethernet PHY or MAC boundary, reducing host-dispatch error.⁵ It does not by itself establish a submicrosecond end-to-end clock bound; topology, asymmetry, synchronization interval, timestamp location, and MCU clock conversion must be characterized.

⁵ | **Hardware Timestamping:** A compatible PHY or MAC timestamps frames closer to the wire than a software interrupt handler. Hardware timestamping removes one source of dispatch variation, while path asymmetry and clock conversion remain. Timestamp uncertainty belongs in the physical belief record and permission budget.

C.7 Heterogeneous SoC Mailboxes and Memory Barriers

Some robotics SoCs couple an application processor running Linux with a real-time core or micro-controller. Their communication contract must specify memory ownership, cache behavior, message validity, and what the real-time side does when no new message arrives.

C.7.1 Remote processor messaging over shared SRAM

An illustrative shared-memory mailbox works as follows:

1. Core A writes a message payload to a fixed buffer in shared SRAM.
2. Core A makes the payload visible to Core B using the platform's coherent mapping or explicit cache maintenance and ordering.
3. Core A triggers an inter-processor interrupt (IPI) by writing to Core B's hardware mailbox register.
4. Core B checks publication order, integrity, age, and permission before using the payload.

Linux RPMsg is a virtio-based messaging framework that may use shared memory and a platform-specific kick. It does not prescribe this fixed-buffer protocol or a universal latency bound.

C.7.2 CPU pipeline reordering and memory fences

Weak memory ordering and noncoherent caches can let a receiver observe a notification without a current payload unless the producer and consumer follow one published protocol:

```
// Core A (Application Processor)
shared_buffer->target_torque = 15.0f; // Store 1: Payload
mailbox_doorbell = 1;                // Store 2: Interrupt trigger
```

If the CPU or interconnect reorders Store 2 ahead of Store 1, Core B receives the interrupt before the new torque setpoint is committed to physical memory, reading stale or invalid data.

A data memory barrier can order accesses within its selected shareability domain:⁶

```
shared_buffer->target_torque = 15.0f;
__asm__ volatile("dmb ish" ::: "memory"); // Only if both peers share this domain
mailbox_doorbell = 1;
```

The barrier in this example is sufficient only if both cores share the inner-shareable coherent domain and the doorbell has the specified device-memory ordering. It does not flush a noncoherent cache or make data visible to an unrelated DMA master. Otherwise, use the platform's appropriate barrier and DMA cache-ownership synchronization, or map the buffer coherently, and verify the receiver's acquire side. The real-time reader still needs a bounded validation attempt and a lease-limited fallback when publication is torn or stale.

C.8 Actuator Electrical and Thermal Dynamics

Actuators convert electrical energy into mechanical work through electromagnetic lorentz forces, subject to coupled electrical and thermal constraints.

C.8.1 Stator electrical rise time

A brushless DC (BLDC) motor phase winding exhibits phase resistance R_{phase} and phase inductance L_{phase} . Applying terminal voltage V_{terminal} against back-electromotive force $e_{\text{bemf}} = K_e \omega_m$ yields the first-order stator current ODE:

$$L_{\text{phase}} \frac{dI(t)}{dt} + R_{\text{phase}} I(t) = V_{\text{terminal}} - K_e \omega_m$$

For a voltage step applied from rest ($\omega_m = 0$), phase current rises exponentially:

$$I(t) = \frac{V_{\text{terminal}}}{R_{\text{phase}}} (1 - e^{-t/\tau_e}), \quad \tau_e = \frac{L_{\text{phase}}}{R_{\text{phase}}}$$

⁶ | **Hardware Memory Barriers:** Barriers order accesses only within their specified shareability and memory-attribute contract. On a noncoherent route, cache clean/invalidate operations or coherent mappings are also required. Arm's memory-ordering guide distinguishes ordering from coherence.

where τ_e is the electrical time constant: in this ideal step model, current reaches about 63 percent of its final value after one τ_e . Torque $\tau_m = K_t I$ follows this current only while the magnetic, voltage, and control assumptions hold; a command is not an instantaneous torque change.

C.8.2 Lumped thermal networks and duty cycle

Current flowing through stator windings dissipates resistive Joule heat:

$$P_{\text{loss}}(t) = I_{\text{rms}}^2(t) R_{\text{phase}}(T)$$

where copper resistance increases linearly with temperature:

$$R_{\text{phase}}(T) = R_0 [1 + \alpha_{\text{Cu}}(T - T_0)]$$

with temperature coefficient $\alpha_{\text{Cu}} \approx 0.00393 \text{ K}^{-1}$ for copper. In a single-node lumped thermal model with thermal resistance to ambient θ_{JA} (in K/W) and thermal heat capacity C_{th} (in J/K), temperature rise $\Delta T(t) = T_{\text{winding}}(t) - T_{\text{ambient}}$ satisfies:⁷

$$C_{\text{th}} \frac{d\Delta T(t)}{dt} = P_{\text{loss}}(t) - \frac{\Delta T(t)}{\theta_{JA}}$$

Under constant power, temperature rises with thermal time constant $\tau_{\text{th}} = \theta_{JA} C_{\text{th}}$:

$$\Delta T(t) = P_{\text{loss}} \theta_{JA} (1 - e^{-t/\tau_{\text{th}}})$$

Continuous torque capacity τ_{cont} is bounded by maximum allowable winding temperature ΔT_{max} :

$$\tau_{\text{cont}} \leq K_t \sqrt{\frac{\Delta T_{\text{max}}}{\theta_{JA} R_{\text{phase}}}}$$

An above-continuous torque may be permitted transiently if the device's temperature estimate and specified current-time limits allow it. The integral $\int_0^t I^2 dt$ is one possible protection measure, not a universal motor rating.

C.9 DC Bus Power Distribution and Energy Absorption

A robotic platform's electrical power distribution network must supply acceleration surges and absorb regenerative braking energy across finite bus capacitance and harness impedance.

C.9.1 Inductive droop and capacitive decoupling

When multiple actuators accelerate, harness resistance and inductance impede source current while local capacitance supplies the difference. For a specified waveform, the resistive and inductive terms are $\Delta I_{\text{source}} R_{\text{bus}}$ and $L_{\text{bus}} dI_{\text{source}}/dt$; capacitor discharge obeys

$$\Delta V_C(t) = \frac{1}{C_{\text{bus}}} \int_0^t (I_{\text{load}}(s) - I_{\text{source}}(s)) ds.$$

These terms cannot simply be added as simultaneous peaks without a circuit and waveform model. If the resulting voltage crosses a configured under-voltage lockout (UVLO) threshold for long enough, the supply may disable or reset; the threshold, filtering, and plant consequence require measurement.

⁷ | **Stator Thermal Time Constants:** Resistive Joule heating elevates motor phase winding temperatures during continuous high-torque maneuvers. Because copper electrical resistivity increases linearly with temperature, hot windings exhibit higher resistance, reducing torque output under terminal voltage saturation. Exceeding winding insulation temperature limits breaks down dielectric coatings, causing inter-turn short circuits and irreversible actuator destruction.

C.9.2 Regenerative braking surges and chopper sizing

During controlled regenerative deceleration, some mechanical kinetic energy $E_k = \frac{1}{2}mv_0^2$ (linear) or $E_k = \frac{1}{2}J_{\text{load}}\dot{q}_0^2$ (rotational) may return through the inverter. If the battery cannot accept it and an assumed fraction η_{regen} reaches the local bus capacitor C_{bus} before another sink acts, the ideal capacitor relation is:

$$\Delta E_C = \frac{1}{2}C_{\text{bus}}(V_{\text{final}}^2 - V_{\text{initial}}^2) = \eta_{\text{regen}}E_k$$

$$V_{\text{final}} = \sqrt{V_{\text{initial}}^2 + \frac{2\eta_{\text{regen}}E_k}{C_{\text{bus}}}}$$

For an implementation with an active **brake chopper**, the resistor and switch must absorb the modeled peak power and pulse energy when $V_{\text{bus}} \geq V_{\text{clamp}}$.⁸

$$R_{\text{brake}} \leq \frac{V_{\text{clamp}}^2}{P_{\text{regen, peak}}}$$

$$P_{\text{regen, peak}} \approx \eta_{\text{regen}} \cdot \tau_{\text{brake}} \cdot \omega_{\text{max}}$$

The resistor inequality is only a peak-power sizing check; minimum resistance for switch current, pulse energy, and thermal recovery impose additional constraints.

C.10 Real-Time Fieldbuses and Lock-Free Synchronization

Coordinating distributed drives requires a measured network schedule and a bounded local publication protocol.

C.10.1 Fieldbus serialization and schedulability

On a shared serial bus, N_{axes} drives transmit telemetry and receive setpoints cyclically. Serialization time depends on the complete frame, physical bit rates, and any stuffing or retransmission allowance.

CAN FD uses non-destructive priority arbitration at its nominal bit rate; a winning frame then sends its data phase at the configured data bit rate. An error-free serialization estimate for frame i is

$$t_{\text{frame},i} \approx \frac{B_{\text{arb},i}}{R_{\text{nom}}} + \frac{B_{\text{data},i}}{R_{\text{data}}} + t_{\text{stuff},i} + t_{\text{interframe},i}.$$

This estimate is not a response-time bound. A lower-priority frame already transmitting can block a newly ready high-priority frame until that transmission ends; arbitration losers defer without a collision-error retry. Bound each message's queuing and release jitter with the actual priority set, frame lengths, error assumptions, and bus load; Bosch's CAN FD description distinguishes the arbitration and data phases.

EtherCAT processes data as frames pass through slave controllers. For a specified topology, a cycle budget includes serialization of every configured frame, master and slave forwarding, PHY and cable propagation, processing, and a guard for jitter and error recovery:

$$T_{\text{cycle}} \geq \sum_f \frac{B_f}{R_{\text{link}}} + t_{\text{master}} + \sum_s t_{\text{slave},s} + t_{\text{prop}} + t_{\text{guard}}.$$

The achievable cycle period is configuration dependent; short advertised periods do not establish a deadline for a particular machine. The EtherCAT Technology Group describes the processing method and configurable process data.

⁸ | **Dynamic Braking Choppers:** A qualified chopper can dissipate returned energy when the battery is unavailable as a sink. Its threshold, switch and resistor ratings, pulse duration, and thermal recovery must be tested. Regeneration and chopper operation are not inherent to every emergency stop; safe torque off can instead leave a motor coasting.

C.10.2 Lock-free seqlocks for multi-rate shared memory

When a producer updates multiword state read by a deadline-bound enforcer, waiting on a producer-held mutex can violate the reader's deadline. The single-slot mailbox in algorithm 8.1 uses a sequence counter to detect a torn snapshot.

A **sequence counter** marks a single writer's publication in progress with an odd value and a completed publication with an even value:

1. **Writer Protocol:**

- Publish an odd sequence value, write the payload, and make it visible through the platform's coherent or cache-maintained memory contract.
- Publish the next even value with release ordering.

2. **Reader Protocol:**

- Read S_1 with acquire ordering. If it is odd, reject this publication without waiting.
- Make one bounded copy to scratch memory using the platform's required load ordering, then read S_2 .
- Accept only if $S_1 = S_2$ and both are even, and the copied frame passes integrity, timestamp, and lease checks. Otherwise retain the last validated local command only while its lease and stopping margin permit; then enter the MCU-owned fallback.

The reader's one-attempt execution time can be bounded on the target hardware; success probability depends on the writer's schedule and cannot be inferred from payload size alone. Conventional Linux seqlock readers may retry, as the kernel documentation shows, but a real-time permission gate cannot use an unbounded retry loop.

C.11 Camera Projective Geometry and Covariance Propagation

Perception pipelines map continuous Euclidean 3D scenes onto discrete 2D camera photosite grids.

C.11.1 Intrinsic projection matrix and radial distortion

Under the standard pinhole camera model, a 3D point $\mathbf{p}_c = [x_c, y_c, z_c]^\top$ in camera coordinates projects onto pixel coordinates $\mathbf{u} = [u, v]^\top$ via the intrinsic calibration matrix $\mathbf{K}$:⁹

$$\tilde{\mathbf{u}} = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{z_c} \mathbf{K} \mathbf{p}_c = \frac{1}{z_c} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix}$$

where (f_x, f_y) are focal lengths in pixel units and (c_x, c_y) is the principal point. Non-linear radial lens distortion is modeled by distortion coefficients (k_1, k_2) :

$$r^2 = x_n^2 + y_n^2, \quad x_d = x_n(1 + k_1 r^2 + k_2 r^4), \quad y_d = y_n(1 + k_1 r^2 + k_2 r^4)$$

where $(x_n, y_n) = (x_c/z_c, y_c/z_c)$ are normalized image coordinates.

C.11.2 Metric unprojection and spatial covariance propagation

When unprojecting a 2D pixel observation $\mathbf{u}$ with measured depth z into 3D camera coordinates, photosite measurement noise $\text{meas} = \text{diag}(\sigma_u^2, \sigma_v^2, \sigma_z^2)$ propagates through the unprojection Jacobian $\mathbf{J}_{\text{unproj}}$:

$$\mathbf{p}_c = \begin{bmatrix} (u - c_x)z/f_x \\ (v - c_y)z/f_y \\ z \end{bmatrix}, \quad \mathbf{c} = \mathbf{J}_{\text{unproj}} \text{meas} \mathbf{J}_{\text{unproj}}^\top$$

⁹ | **Pinhole Intrinsic Projection:** The pinhole model projects 3D points onto a 2D sensor through calibrated intrinsics. Under fixed pixel noise, the lateral variance after depth unprojection grows with squared distance; depth-error and calibration terms follow their own models. Uncalibrated intrinsics can bias a clearance estimate and must be included in its error envelope.

Transforming this 3D point into the robot body frame via rigid extrinsic transform $\mathbf{T}_{bc} = [\mathbf{R}_{bc}, \mathbf{p}_{bc}; \mathbf{0}^\top, 1]$ yields the spatial body covariance:

$$\mathbf{b} = \mathbf{R}_{bcc} \mathbf{R}_{bc}^\top + \text{ext}$$

where ext models independent extrinsic uncertainty under this first-order approximation; correlations require the corresponding cross terms. A probabilistic clearance buffer may scale with $\sqrt{\lambda_{\max}(\mathbf{b})}$ at a declared tail risk. A deterministic permission check instead needs a justified bounded-error envelope; covariance alone does not give a worst-case bound.

C.12 Kinematic Stopping Envelopes and Information Age Lag

Halting an embodied machine transporting mass m at velocity v_0 requires balancing sensorimotor information freshness and mechanical braking deceleration.

C.12.1 Sensorimotor information age and lag distance

For a specified hazard detector and stop path, let τ_{pre} bound the time from the physical state represented by the evidence until validated braking deceleration begins:

$$\tau_{\text{pre}} = t_{\text{age}} + t_{\text{detect}} + t_{\text{compute}} + t_{\text{bus}} + t_{\text{brake-onset}}$$

where:

- t_{age} converts the evidence timestamp to the controller clock and includes exposure/readout and clock error.
- t_{detect} bounds recognition or lease-expiry detection.
- t_{compute} is the admitted stop decision, including any required input transfer.
- t_{bus} bounds command dispatch and network queuing.
- $t_{\text{brake-onset}}$ ends when the measured actuator begins delivering the specified deceleration, not when a command register is written.

Under the explicit simplifying assumption that speed cannot increase before brake onset, the pre-brake travel is bounded by $d_{\text{lag}} = v_0 \tau_{\text{pre}}$. If propulsion or a slope can increase speed, use its validated acceleration bound in this interval instead.

C.12.2 Braking dynamics and friction limits

Tire-floor friction and actuator torque place upper limits on achievable deceleration; $a \leq \mu g$ does not establish what braking the machine will deliver. Let $a_{\text{floor}} > 0$ be a validated lower bound on *achieved* deceleration throughout the stop, for the declared load, temperature, slope, brake condition, and surface. A justified friction floor μ_{floor} may support that bound, but does not replace brake tests. With constant or stronger braking after onset, and a nonnegative clearance allowance ϵ_d for state and tracking error,

$$d_{\text{stop,bound}}(v_0) = v_0 \tau_{\text{pre}} + \frac{v_0^2}{2a_{\text{floor}}} + \epsilon_d.$$

Motion is admitted only when $d_{\text{stop,bound}}(v_0) \leq d_{\text{clear,lower}}$, where $d_{\text{clear,lower}}$ is a validated lower bound on available path clearance. The sampled controller must reserve intersample travel and a state-matched stop before the viable region is lost.

C.12.3 Kinetic energy derating and friction collapse

In an ideal friction-limited model, lowering μ from μ_{nom} to a positive μ_{fault} changes the braking-distance ratio to $\mu_{\text{nom}}/\mu_{\text{fault}}$. That ratio is illustrative; actual deceleration can also be limited by brake hardware or control. If surface friction is not sensed in time, derive a_{floor} from a controlled surface condition and inspection schedule, exclude degraded surfaces, or withhold motion. For the constant-speed pre-brake model above, solving the clearance inequality gives

$$v_{\text{clamp}} = -a_{\text{floor}} \tau_{\text{pre}} + \sqrt{(a_{\text{floor}} \tau_{\text{pre}})^2 + 2a_{\text{floor}}(d_{\text{clear,lower}} - \epsilon_d)}.$$

This clamp is valid only when $d_{\text{clear,lower}} > \epsilon_d$, the braking and timing lower bounds hold, and the speed bound is enforced before entering the hazard corridor. No positive clamp follows if the available deceleration floor is unknown or zero.

C.13 Further Reading

For a deeper foundational understanding of embedded architectures, real-time operating systems, and power electronics, consult the following authoritative references:

- **Computer Organization and Design: The Hardware/Software Interface** (Hennessy and Patterson 2019) provides the definitive treatment of processor architectures, cache hierarchies, and memory subsystem trade-offs.
- **Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment** (Liu and Layland 1973) establishes the mathematical foundations of rate-monotonic scheduling and real-time task schedulability.
- **Modern Operating Systems** by Andrew S. Tanenbaum and Herbert Bos provides comprehensive coverage of inter-process communication, lock-free synchronization, and memory management.
- **Real-Time Systems** by Jane W. S. Liu delivers the authoritative mathematical formulation of real-time scheduling algorithms, priority inversion, and response time analysis.

S·P·A Diagnostic Playbook

A mobile chassis that overshoots a pallet corridor could have stale sensor evidence (Sense), an infeasible waypoint (Plan), or a worn brake that delivers less deceleration than assumed (Act). The **S·P·A taxonomy** locates the first broken contract among **Sense**, **Plan**, and **Act**. Logs and physical measurements then test that diagnosis before a remedy is admitted.

How to Use This Appendix

Use this diagnostic playbook when an embodied system violates a safety envelope, misses an execution deadline, chatters during contact, or trips hardware interlocks. Work through the diagnosis in four systematic passes:

1. **State the physical symptom and binding budget:** Record the observed stopping-distance breach, contact-force transient, thermal trip, or enforcer deadline overrun, including units, load, operating state, and the limit configured for that machine.
2. **Localize the candidate axis or intersection:** Use table D.1 for a first pass across the three core axes. When the symptom involves interface timing, state representations, or actuation authority, move directly to the intersection landscape in table D.2.
3. **Test the quantitative hypothesis:** Measure the candidate constraint or budget using the instruments in table D.3, such as timestamp skew, bus queue time, phase current, or tracking error.
4. **Intervene and remeasure across the causal boundary:** Apply a remedy and rerun the measured test over the declared load, timing, and fault envelope.

The physical symptom determines the most effective entry point. Unstable contact oscillations should immediately examine the Act–Sense ($A \cap S$) boundary. A sudden reference discontinuity or measured torque transient points to the Plan–Act ($P \cap A$) boundary. Phantom obstacle braking or drift under occlusion points to the Sense–Plan ($S \cap P$) interface. Deadline jitter during concurrent neural inference points directly to the three-way causal boundary ($S \cap P \cap A$).

D.1 Diagnostic Summary: The S·P·A Axes

THE S·P·A taxonomy decomposes the cyber-physical loop into three axes. Table D.1 lists a constraint or budget to inspect for each one; these are not all conservation laws.

Table D.1: S-P-A axis reference: The axes organize diagnosis around measured constraints and budgets, with the chapter that develops each contract.

Axis	Operational role	Constraint or budget to test	Diagnostic action	Book locus
Sense (S)	Capture and estimate physical state	Evidence age, field of view, clock conversion, uncertainty, and sensor transport	Compare capture and per-cell evidence epochs with a measured state and the permission deadline	chapter 18
Plan (P)	Propose a target and trajectory	Intent validity, compute latency, dynamics, and stopping-suffix admission	Compare proposed targets and future states with independently checked limits	chapter 22
Act (A)	Apply and measure force	Current, torque, thermal headroom, brake response, and kinetic clearance	Measure command-to-force response and achieved stop under the declared load	chapter 24

While this single-axis separation is helpful for initial triage, production failures rarely originate within a single isolated component. More frequently, the failure sits at the **boundary** between two axes—such as an unblended trajectory step that excites high mechanical rotor inertia, or camera direct memory access (DMA) ingress that saturates the memory crossbar needed by real-time motor timers. Resolving these failures requires mapping the intersection landscape.

D.2 The Intersection Landscape

The foundational conceptual model is established in figure 2.8 (section 2.6), showing how Sense, Plan, and Act intersect to define the operational boundary of physical AI systems. In this appendix, we formalize this landscape into an operational systems diagnostic matrix.

Each intersection is a testable handoff. Table D.2 names the failure mode and the evidence needed before a proposed fix earns permission.

Table D.2: The S-P-A intersection taxonomy: Four handoffs locate the earliest broken physical contract and the evidence required to repair it; the axis reference above links to the underlying chapters.

Intersection	Handoff contract	Failure to test	Evidence and candidate remedy
$S \cap P$ — belief	Observations become timed, framed state estimates	A fresh map publication hides an old voxel or an unseen object remains unknown	Check per-cell evidence epochs, free versus occupied rays, and frame transforms; use a separately validated collision layer
$P \cap A$ — motion	A proposal becomes an admitted, state-matched trajectory	A seam requests a torque transient or the stop suffix no longer fits clearance	Check entry state, derivatives, lease, and clearance; reject an infeasible seam and use the resident stop
$A \cap S$ — feedback	Actuation changes the state that sensors report	Contact force rises before the next useful sample or an actuator saturates	Measure command-to-force and sensor-return delay; choose a local reflex and passive limit where justified
$S \cap P \cap A$ — permission	The independent gate turns a proposal into physical action	Compute, memory, power, or clock interference delays refusal	Measure response under contention; revoke tracking and qualify isolation or relocate the path if its bound fails

D.3 Physical AI Production Troubleshooting Matrix

When an operational machine violates physical limits or software invariants, use table D.3 to match observable diagnostic symptoms to the binding axis and high-leverage remedy.

Table D.3: Physical AI production troubleshooting matrix: Each row is a hypothesis, measurement, and candidate remedy; acceptance depends on the machine's declared limits.

Observable symptom	Candidate boundary	Discriminating measurement	Candidate response
Contact hunting during assembly	$A \cap S$	Measure phase margin and force rise against the sensor-to-torque delay under the actual contact stiffness	Qualify a local impedance loop, approach-speed limit, and passive compliance for that contact
Torque or current transient at an action-chunk seam	$P \cap A$	Compare encoder, phase-current, and requested acceleration traces at the measured entry state	Admit a state-matched C^2 bridge and reserve a stopping suffix; the reduction in chapter 22 is profile-specific
Phantom obstacle stop in a corridor	$S \cap P$	Distinguish observed free, occupied, and unknown rays; inspect per-voxel evidence age	Clear only observed free space and retest the occupancy-to-permission conversion
Enforcer misses its configured deadline during camera ingress	$S \cap P \cap A$	Correlate response-time misses with bus queue, bank switches, cache misses, and DMA ownership	Qualify QoS bypass or a coherent local safety snapshot; remeasure under contention
Link or workpiece yields during contact	A	Compare measured force and displacement with the tested material and tooling limits	Reduce admitted speed or force and qualify a compliant load path
Policy update arrives after its admitted chunk horizon	$P \cap A$	Measure the update tail, lease expiry, and fallback transfer state	Shorten the admitted action or use an MCU-owned stop; do not extend permission from a percentile alone
MCU rail droops during accelerator bursts	$S \cap P \cap A$	Measure rail waveform, path slack, supervisor threshold, and resulting response time	Characterize decoupling, current slew, or physical power isolation
Vehicle drifts laterally despite low offline prediction error	$A \cap S$	Compare closed-loop yaw bias, lateral clearance, and surface-dependent braking	Test closed-loop rollouts and impose a feasible corridor and stop bound

D.4 The S·P·A Systems Scorecard

Before release, use table D.4 to name the evidence needed for each handoff. Its checks are examples, not portable pass thresholds: the signed release case must supply the actual limits, operating envelope, confidence or hard-bound semantics, and fallback for the machine.

Table D.4: The S·P·A Physical AI systems scorecard: Example release checks across the three axes and their handoffs. Each row requires a case-specific threshold and evidence; the solver number is an assumed worked qualification target.

Boundary	Example acceptance check for a declared machine	Response if the check fails
Sense — capture clock	Bound capture-to-MCU conversion error and include it in evidence age	Resynchronize or withhold motion when the bound expires
Sense — ingestion	Measure worst admitted sensor/DMA queue occupancy, drop rate, and transfer time under concurrent load	Reserve capacity or reduce sensing load; dropped frames do not renew belief
Plan — chunk horizon	Admit a successor before the commit deadline, or transfer to a resident state-matched stop	Use measured update tails as evidence, not as a worst-case guarantee
Plan — dynamics	Check position, velocity, acceleration, torque, and stopping suffix against the actual joint and path limits	Reject or replan an infeasible chunk
Act — brake and contact	Validate achieved deceleration and force response for load, wear, surface, and temperature	Derate speed, change tooling, or refuse the untested state
Act — thermal	Keep measured winding and drive temperature inside device ratings under the duty cycle	Reduce duty or enter the validated stop/hold mode
$S \cap P$ — spatial evidence	Preserve each cell's evidence epoch and distinguish observed free, occupied, and unknown	Refuse unknown clearance or obtain a new observation
$P \cap A$ — seam	Check the matched entry state, derivative limits, and full stopping clearance before commit	Use the MCU-owned baseline stop if a successor is absent or invalid
$A \cap S$ — reflex	Bound sensing-to-force response against a contact-specific harm deadline	Reduce approach energy or add a validated passive limit
$S \cap P \cap A$ — compute	In the illustrative 1 ms cycle of chapter 24, qualify a solver WCET of at most $650 \mu s$ and budget acquisition, validation, write, and dispatch separately	Revoke tracking permission before the cycle overrun consumes stopping margin
$S \cap P \cap A$ — shared silicon	Measure enforcer response under worst admitted bus, rail, and thermal contention	Qualify QoS/local snapshots or relocate the path if isolation cannot be shown
$S \cap P \cap A$ — permission	Verify model/state assumptions, feasible CBF input, clock and estimate margins, and a state-matched fallback	Refuse the proposal when a premise is missing; a QP result alone proves no path safety

Glossary

This glossary defines key terms used throughout *Physical AI*. Terms are organized alphabetically.

A

Action Chunk Seam Discontinuity A mismatch in position, velocity, or acceleration where one admitted action chunk meets the next. An ideal velocity step implies an acceleration impulse; a finite acceleration step requests a finite torque step whose plant response depends on inertia and drive limits. The worked bridge matches position, velocity, and acceleration at its entry and reserves a separately admitted stop. (Chapter 22)

Astronomical Exposure Wall (Butler & Finelli Bound) Under a stationary Poisson model, zero observed system-harm events require about $-\ln(0.05)/10^{-9} \approx 3.0 \times 10^9$ independent failure-free machine-hours to place a one-sided 95% upper rate below $10^{-9}/\text{h}$. Aggregate exposure is possible, but is impractical within the illustrated fleet and policy-update interval; the bound says nothing by itself about unshielded hazardous-proposal rate or detector coverage. (Chapter 14, chapter 30, chapter 34)

B

Belief State Invalidation Horizon (t_{exp}) The time at which a physical estimate no longer meets its declared error tolerance. A worst-case deadline requires a justified bounded-disturbance growth law; a calibrated quantile instead carries an explicit tail risk. Each cell or entity retains its own evidence epoch, independent of map publication time. (Chapter 18)

C

Category 0 Stop (IEC 60204-1) A stop by immediate removal of power to machine actuators. It is uncontrolled after removal: a moving motor may coast under safe torque off. Any mechanical brake is a separate, validated component; Category 0 is selected only when its coast and load behavior suit the hazard. (Chapter 24)

Category 1 Stop (IEC 60204-1) A controlled stop with power available during deceleration, followed by removal of power after stopping. Its response and stopping distance depend on the implemented drive, brake, load, and monitoring path. (Chapter 24)

Category 2 Stop (IEC 60204-1) A controlled stop with power retained after the stop, for example to maintain a verified powered hold. Continued power does not itself establish that a suspended or moving load is safe. (Chapter 24)

Causal Boundary The transition from an independently admitted digital command to physical actuation. Subsequent energy transfer or contact can have irreversible effects even when software state is rolled back. The exact register, drive, and force timing are device-specific. (Chapter 2)

Claim-Argument-Evidence (CAE) A safety-case structure linking a claim to a reasoned warrant and supporting evidence. Evidence can include analysis, qualified tests, field observations, and fault injection, with assumptions and residuals explicit; a fault log alone is not a release verdict. (Chapter 32)

Covariate Shift Flywheel In imitation learning, policy errors can move the robot to states underrepresented in demonstrations, where further errors become more likely. A quadratic-horizon behavioral-cloning bound requires the stated error, state-distribution, and cost assumptions; it is not a claim that every deviation reaches a hazard. (Chapter 10, chapter 12)

D

Demonstration Provenance Tuple The 4-stream action logging protocol $(a_{\text{req}}, a_{\text{cmd}}, a_{\text{enf}}, a_{\text{meas}})$ that distinguishes intentional human teleop commands (a_{req}) from kinematic transformations (a_{cmd}), safety governor clamps (a_{enf}), and structural compliance/backlash (a_{meas}). (Chapter 10)

Detectorless Failure Mode A hazard mechanism for which the installed sensors provide no timely distinguishing signal before the required intervention deadline. Passive protection, operating restriction, or inspection may support a narrower claim when software detection cannot. (Chapter 34)

Dynamic Fallback Ladder (MRM) A menu of supervisor-owned responses such as command filtering, powered hold, controlled braking, and torque inhibit. The response is chosen from the current state and validated plant capability; these are not universally ordered by safety or latency, and torque inhibit can leave a moving load coasting. (Chapter 24, chapter 28)

Dynamic Stopping Distance (d_{stop}) A conservative path-clearance bound combining pre-brake travel, braking travel under a validated lower bound $a_{\text{floor}} > 0$ on achieved deceleration, and state/tracking allowance: $d_{\text{stop,bound}} = v\tau_{\text{pre}} + v^2/(2a_{\text{floor}}) + \epsilon_d$ under constant speed before brake onset. Unknown braking capability does not support a positive speed permission. (Chapter 4, chapter 24, section C.12)

E

Endogenous Sensory Distribution Shift Closed-loop actions alter the states the machine later observes, so deployed observations can depart from the demonstration distribution. A quadratic-horizon imitation bound holds only under its specified per-step error and downstream-cost assumptions. (Chapter 2, chapter 10, chapter 12)

Envelope Trichotomy Runtime evidence can support a claim as known true within a signed operating envelope, show it known false, or leave it unknown. Known false and unknown both refuse nominal tracking, but remain distinct records; the MCU chooses only a separately validated state-matched fallback. (Chapter 32)

Expiring Intent Lease A time-bounded request for tracking a grounded target under stated tolerance and evidence assumptions. The independent MCU checks the request and its own deadline. At expiry it withdraws tracking permission and uses the admitted fallback; physical stopping may finish after the tracking lease expires if that fallback has separate authority and reserved clearance. (Chapter 20, chapter 22)

F

Frozen Transform Trap A transport bridge republishes an old spatial transform with a new publication timestamp. The capture or evidence epoch must remain attached so downstream age checks do not mistake stale geometry for a fresh observation. (Chapter 18)

I

Inevitable Collision State (ICS) A state from which no admissible control can avoid collision under the specified dynamics, actuator, obstacle-motion, and delay bounds. An ICS claim is conditional on that model and the available stopping or evasive actions. (Chapter 4, chapter 22, chapter 24)

L

Lock-Free Seqlock Buffer A single-writer shared-memory publication protocol using an odd sequence value during mutation and an even value after publication. The real-time reader makes one bounded copy attempt, accepts only matching even values plus integrity/freshness checks, and otherwise retains a validated command only within its lease before fallback. Coherent memory or explicit cache maintenance is also required. (Chapter 8, chapter 26)

M

Minimal-Intervention Safety Shield An independent filter that tests each candidate command against a feasible input constraint derived from a validated barrier and plant model. A candidate can violate the derivative constraint while the state still has $h(x) \geq 0$; the filter then modifies or refuses it. A QP minimizes the selected command-space deviation, subject to actuator and timing limits, but does not by itself prove a Cartesian path safe. The 1 kHz loop is an illustrative implementation. (Chapter 24)

Multidimensional Sim-to-Real Reality Gap Vector (Δ_{gap}) A record of dynamics, sensing, appearance, and latency differences between a model and its physical plant. Its components have distinct units and can be combined into one score only after a stated normalization and risk rule. (Chapter 12)

O

Observational Indistinguishability Limit Two world states generate the same available sensor record until the intervention deadline, so no downstream computation can choose different actions from that record alone. A common conservative response or passive bound may still protect both states if its force, timing, and actuator premises are validated. (Chapter 34)

P

Physical AI System A system with learned live perception or policy whose admitted actions change physical state and later observations, and whose accepted proposals may actuate without case-by-case human approval. An independent veto can retain final permission authority while the machine still exercises delegated physical agency. (Chapter 2)

Proposal-Permission Privilege Boundary The separation in which a learned or other upstream controller proposes an action and an independent path retains authority to admit, modify, or refuse actuation under declared state and timing bounds. An MCU checking each millisecond is the worked example, not a universal implementation rate. (Chapter 2, chapter 6, chapter 24)

R

Residual-Claims Register An auditable record of claims whose evidence or sensor coverage remains incomplete, naming the hazard, operating restriction or passive protection, evidence needed, and accountable owner. Inspection supports the evidence for a separately enforced restriction; it does not itself stop motion between checks. (Chapter 34)

S

Spatial Information Age (τ_{age}) Elapsed time between a spatial record's capture or evidence epoch and the query or permission decision, after converting both clocks into one domain with bounded error. A full sensing-to-force response budget is a separate sum of detection, computation, communication, and actuator delays. (Chapter 16, chapter 18)

T

Two-Speed Embodied Compute Architecture An illustrative partition between slower proposal generation and a bounded local permission and control path. The required rates follow the plant and hazard deadlines; the volume's 1 kHz MCU and slower model rates are examples. (Chapter 6, chapter 26)

V

Volumetric Free-Space Raycasting Using a sensor ray to mark traversed cells before an observed hit as free in an occupancy layer, subject to range and occlusion validity. Near-surface TSDF fusion is a separate update; space behind the hit remains unknown unless observed from another ray. (Chapter 18)

W

Worst-Case Execution Time (WCET) A justified upper bound C on a specified task's execution on its target hardware and workload envelope. Response time runs from job release to completion and may include blocking, preemption, bus interference, and dispatch. Release jitter relative to a physical event must be handled separately when bounding event-to-actuation latency; C alone is not that deadline. (Chapter 8, chapter 26, section C.2)

Z

Zero-Allocation SRAM Invariant A design rule used by a particular real-time safety path to avoid unbounded heap work: required buffers and solver workspaces are preallocated in qualified local memory. Static allocation improves timing analyzability but does not by itself bound bus access, execution, or physical response. (Chapter 8, chapter 24, chapter 26)

Index

- 2018 Tempe Autonomous Collision
 - NTSB report, 23
- 3D Gaussian splatting, 296, 313
- 3D dynamic scene graph, 298
- 3D foundation metric lifting
 - definition, 273
- 3D vision foundation models
 - definition, 273
- Absence ledger principle, 169
- Action agreement, 242
- Action Chunk Representations, 8
- Action chunking, 183
 - architectures, 344
 - definition, 89
 - receding-horizon ensembling, 183
 - temporal horizon bounds, 89
 - temporal trajectory, 89
- Action tap
 - definition, 145
- Active negative observation, 310
- Actuation
 - braking current, 394
 - mechanical resonance, 212
 - PWM drivers, 393
- Actuator
 - back-EMF, 399
 - Bias torque, 380
 - Class F insulation, 191
 - current limiters, 240
 - electrical rise time, 587
 - intermittent torque, 57
 - Joule heating, 240, 399
 - saturation, 379
 - series elastic, 550
 - stator delay, 239
 - stator thermal dynamics, 541, 588
 - stiction deadband, 481
 - torque limit, 240
 - velocity limits, 331
- Actuator authority, 520
- Actuator authority budget
 - definition, 42
- Actuator bandwidth
 - definition, 43
- Actuator Dynamics, 550
- Actuator dynamics
 - kinodynamic feasibility, 441
 - Tier 1 verification, 472
- Actuator dynamics latency robustness
 - definition, 178
- Actuator saturation, 370, 384
- Actuator thermal derating
 - physics, 354
- Actuator thermal dynamics
 - in evaluation, 219
- Actuator torque limits, 242, 370
 - safety bounds, 483
- Adaptation wear ledger, 210
- Adaptive Stress-Testing, 542
- Admission gate
 - rejection, 339
- Adversarial review
 - definition, 513
- Adversarial vulnerability
 - perturbation bounds, 95
- Affordance grounding
 - open-vocabulary, 326
- Affordances
 - 3D potential fields, 326
 - SayCan value function, 568
 - spatial parameterization, 321
- Algorithm
 - 3D-DDA traversal, 296
- Alpha-Power CMOS delay law, 414
- ALVINN, 181
- AMR Safety Standards, 309
- ANSI/UL 4600
 - evaluation records, 245
- Antipatterns
 - asynchronous safety cancellation, 323
 - moving average filtering, 115
- Apollo 11
 - AGC 1202 alarm, 487
- Architectural boundaries, 76
- Architectural isolation, 551
- Architectural warrants, 520
- Architecture
 - dual-brain, 289
 - subsumption, 324
- Arithmetic intensity, 551
 - definition, 411
- ARM memory model, 112
- ARM MPAM
 - cache partitioning, 410
- Astronomical exposure wall
 - definition, 531
 - mathematical derivation, 569
 - retraining reset, 531
- Authentication
 - cryptographic MAC, 464
- Authority
 - ambiguity, 458
- Authority restriction
 - torque limits, 510
- Authority split, 544
- Automotive safety
 - FFI, 426
- Autonomous Safety Standards, 310
- Back-electromotive force, 374
- Back-EMF, 221
- Bandwidth
 - servo sampling, 352
- Barrier certificates, 371
- Base frame, 290
- Bayes' rule
 - parameter estimation, 564
- Behavioral cloning, 181, 458
 - compounding error, 567
 - quadratic error compounding, 160
- Belief, 288
- Belief contract
 - definition, 291
- Belief state invalidation horizon, 289, 307
- Bimodal ambiguity
 - spatial, 339
- Blending window, 461
- Bode delay theorem, 46
- Body
 - physical plant, 547
- Bound Checker
 - pre-execution, 358
- Boundary Checker, 358
- Boundary contract, 416
- Brain
 - cognitive policy, 547
- Brake
 - mechanical, 521
- Braking
 - friction limit, 591
- Braking distance, 47
- Braking dynamics, 122
- Brownout reset, 423
 - undervoltage, 61
- Brushless actuator
 - thermal monitoring, 509
- Budget enforcement, 42
- Bumpless transfer, 441, 451
 - definition, 451
 - handover blending, 575
- Bumpless transfer braking penalty, 452
- Bus impedance
 - power distribution, 61
- Bus jitter, 242, 274, 314, 492, 551
- Bus latency, 333
- Bus starvation, 394
- Butler & Finelli infeasibility bound
 - definition, 234
- Butler-Finelli exposure bound
 - systems consequence, 234
- C2 bumpless transfer ramp, 451
- C^2 bumpless authority transfer ramp
 - definition, 451
- C^2 continuity, 345
- C^2 jerk spline seam
 - definition, 352
- C^2 trajectory spline blending
 - definition, 353
- Cache coloring
 - memory isolation, 410
- Calibration
 - spatial, 283
- Camera model
 - pinhole projection, 590
- Camera-to-IMU spatial-temporal calibration
 - definition, 266
- CAN arbitration, 124
- Capability matrix, 441
- Capture timestamp
 - definition, 261
- Catastrophic forgetting
 - fine-tuning regression, 207
 - in physical fine-tuning, 204
- Causal boundaries
 - Tier 1 release, 498
- Causal Boundary, 546
- Causal boundary, 550
 - definition, 9
 - thermodynamic constraint, 142
- CBF-QP safety filter
 - definition, 382
- Censored trial fallacy
 - definition, 243

- Centroidal momentum
 - Newton-Euler projection, 579
 - Orin matrix, 579
- Claim-argument-evidence safety case, 547
 - definition, 502
- Claims register, 545
- Classification
 - latency tradeoff, 550
- Clopper-Pearson
 - safety margin, 220
- Clopper-Pearson Interval, 491
- Clopper-Pearson interval, 66
- Clopper-Pearson method, 232
 - sample size bound, 232
- Clopper-Pearson non-asymptotic bound
 - definition, 219
- Closed Causal Loop, 547
- Closed-loop evaluation
 - definition, 223
- Closed-loop evaluation estimand
 - definition, 221
- Closed-loop stability, 474
- Closed-loop stiffness, 370
- Closure test
 - definition, 534
- Code as Policies
 - generation, 327
- Cognitive architecture
 - algorithmic progression, 86
- Cognitive contracts
 - Tier 3 verification, 471
- Cognitive proposal boundary
 - Tier 3 frontier, 529
- Common-cause analysis
 - definition, 476
- Compounding covariate cost, 182
- Computational delay, 551
- Computer vision
 - Lang-SAM, 327
- Concurrency
 - sequence locks, 294, 590
- Confidence score, 282
- Configuration space
 - definition, 346
- Connectionist representations
 - statistical approximation, 95
- Conservative Q-Learning, 190
- Contact constraints
 - unilateral normal force, 579
- Contact fidelity invariant, 193
- Contact friction, 228
- Contact mechanics
 - bifurcation, 238
 - force model, 501
 - non-smooth impact, 538
- Continuous flow matching, 186
- Control
 - Closed-Loop Stability, 248
 - subsumption, 324
- Control architecture, 460
- Control barrier function, 128, 145, 372, 381, 393, 396
 - definition, 382
 - forward invariance, 370
 - forward-invariant filtering, 145
 - High-order, 383
 - Lie derivatives, 372, 386
 - QP solver, 399
 - quadratic program, 386
 - runtime filtering, 76
 - safety filtering, 573
 - safety invariance, 85
- Control Barrier Functions
 - safety filter, 30, 259
- Control loop
 - closed-loop stiffness, 370
 - digital, 375
- Control multiplexer, 126
- Control systems
 - impedance control, 323
 - model predictive control, 326
- Controller
 - windup, 379
- Copper resistance
 - temperature dependence, 56
- Core pinning
 - CPU affinity, 408
- Corrective aggregation, 160
- Coulomb friction
 - simulation limitations, 515
- Coulomb friction cone
 - traction boundary, 579
- Covariance
 - spatial, 313
- Covariance matrix, 306
- Covariate drift
 - physical AI, 142
- Covariate Shift
 - compounding drift, 27
- Covariate shift
 - endogenous feedback, 142
 - in behavioral cloning, 206
- Cross-Class Coupled Contention, 17
- Cross-embodiment compatibility
 - contract
 - definition, 162
- Crossbar bus and DRAM bank
 - contention
 - definition, 411
- Cruise SF incident
 - blind-spot trajectory execution, 167
- Cryptographic manifest
 - hardware Root of Trust, 517
- Cryptographic release record
 - definition, 517
- Curriculum architecture
 - Part III mapping, 86
- Cybernetic Feedback Loops, 6
- Cyclic redundancy check, 112, 423
- Cycloidal drive
 - fatigue life, 204
- Cycloidal drive gearbox
 - torsional compliance, 513
- C² bumpless authority transfer ramp, 465
- DAgger
 - linear error bound, 160
- Data
 - curation, 465
- Data collection
 - linear scaling fallacy, 173
- Data memory barrier, 112
- Dead-time delay, 110
- Deadlines
 - end-to-end, 493
- Deadman switch, 447
- Defeaters
 - definition, 514
- Degraded mode
 - velocity reduction, 510
- Demonstration
 - versus evidence, 503
- Demonstration provenance record
 - definition, 169
- Denominator filtering, 243
- Deployment
 - policy ranking inversion, 211
- Detectable faults, 277
- Detectorless failure
 - indistinguishability, 537
- Deterministic Disturbance Acceleration, 305
- Deterministic enforcement
 - Tier 2 release, 498
- Deterministic fault logging
 - non-volatile ring buffers, 161
- Deterministic fieldbuses
 - Tier 2 frontier, 529
- Diffusion sampling latency, 187
- Direct memory access, 508
 - Bus contention, 394
 - bus contention, 508
 - cache coherence, 109
- Directed acyclic coordinate transform
 - tree, 293
- Distributed clock, 118
- Distribution shift, 547
- Disturbance observer, 384
- Disturbance rejection
 - recovery data, 211
- DMA, 429, 547
- DMA memory contention, 394
- Domain randomization
 - corridor, 201
 - support boundary, 200
- Domain transfer, 513
- DRAM, 408
 - bank conflicts, 411
- Drivetrain
 - jerk dynamics, 345
 - resonant frequency, 358, 361
 - torsional resonance, 353
- Dual-brain
 - privilege boundaries, 322
- Dual-Brain Architecture, 259
- Dual-Brain architecture, 486
- Dual-stage enforcement pattern, 375
- Dynamic brake chopper
 - regenerative clamp, 61
- Dynamic braking, 374
 - DC bus overvoltage, 390
- Dynamic programming
 - provenance, 347
- Dynamic random-access memory, *see* DRAM
- Dynamic range saturation, 278
- Dynamic reachability, 332
- Dynamic voltage and frequency scaling, 109, 110
- Dynamics
 - inertia tensor, 394
 - non-smooth contact, 305
 - payload mass, 394
 - plant dynamics, 393
- Edge silicon fabrics, 547
- Electrical power integrity
 - definition, 43
- Embodied execution
 - compounding dynamics, 239
- Embodied latency scaling law
 - definition, 528
- Embodied shortcut learning and latent
 - epistemic shift
 - definition, 538
- Embodiment contract
 - definition, 162
- Emergency stop
 - mechanical friction brakes, 510
- Empirical physical safety margin
 - definition, 220
- Empirical state-action occupancy
 - distribution, 157
- Empirical sufficiency, 513
- Endogenous covariate shift
 - definition, 142
- Enforcement
 - manual override, 464
 - transfer boundaries, 464
- Enforcement filter, 442
- Enforcement record, 396
- Enforcer
 - tracking contract, 370
- Envelope membership
 - trichotomy states, 509
- Envelope trichotomy
 - definition, 510
- Environment selection, 241
- Epistemic classification, 530
- Epistemic gap
 - kinetic energy accumulation, 510
- Epoch, 291
 - acquisition, 291
 - estimation, 291
 - publication, 291
- Equations of motion
 - manipulator dynamics, 78
- Error expansion, 290
- Error-correcting code
 - SECDED, 484, 486
- EtherCAT, 377
- Evaluation
 - Action Agreement, 248
 - Binomial Model, 248
 - confidence bounds, 239
 - Confidence Intervals, 248

- Covariate Shift, 248
- Fault Injection, 249
 - metric masking, 223
- Reliability Bounds, 248
- Evaluation estimand
 - definition, 221
- Evaluation Gap, 542
- Evaluation record
 - definition, 245
- Evidence record, 544
- Evidence regimes
 - definition, 228
- Evidence tuple, 456
- Executable trajectory
 - definition, 346
- Execution monitoring
 - collision detection, 361
- Expected Calibration Error
 - binned accuracy metric, 564
 - distribution shift, 96
 - mathematical formulation, 96
- Expiring intent lease
 - definition, 322
- Exposure arithmetic, 547
- Exposure time, 283
- Extreme Value Theory
 - Frechet distribution, 570
- Extreme value theory, 231
 - physical disturbances, 231
- Failure modes
 - common-cause, 493
- Failure rate
 - catastrophic, 520
- Fallback trajectory
 - achievability, 362
 - commitment point, 361
- Falsification
 - Popperian induction, 530
- Falsification threshold
 - definition, 423
- Fatigue limits
 - cyclic derating, 511
- Fault coupling
 - shared resources, 493
- Fault Injection, 430
- Fault injection, 474
 - hardware, 474
- Fault manifest
 - Tier 4 release, 498
- Fault record
 - definition, 490
- Fault Tolerance
 - kernel panic, 399
- Fault tolerance
 - Simplex switch, 369
 - structural bounds, 543
- Feasibility
 - kinodynamic, 344
- Feedback control
 - error monitoring, 362
- Feedforward control
 - torque demand, 362
- Field-Oriented Control, 8
 - current loop limits, 50
 - vector control, 8
- Fieldbus
 - CAN-FD arbitration, 589
 - contention, 493
 - EtherCAT on-the-fly, 589
 - serialization latency, 540
 - synchronization jitter, 292
- Filter
 - Kalman, 303
- Fine-tuning
 - on physical hardware, 204
- Floating-base dynamics
 - equations of motion, 579
- Flux degradation, 549
- Forensic authority record, 457, 460
- Forensic operational log, 456
- Forward dynamic filter, 314
- Forward invariance, 382
 - Nagumo viability, 574
- Forward kinematics
 - mapping, 576
- Foundation model
 - embodied decoders, 89
- Four-phase handshake protocol, 448
- Four-stage qualification ladder
 - definition, 477
- Four-stream action tap tuple
 - definition, 145
- Four-tier fallback ladder, 390
- FR-FCFS scheduling
 - definition, 412
- Freedom from interference, 123
 - ISO 26262, 426
- Friction
 - Coulomb, 239
 - Stribeck effect, 550
- Frozen transform, 309
- Functional safety
 - Emergency stop categories, 390
 - fail-operational plants, 391
 - ISO 26262, 506
 - random hardware metrics, 234
- Future-state conditioning
 - definition, 349
- Gate delay
 - Sakurai-Newton model, 585
- Gate propagation delay
 - voltage headroom, 414
- Gearbox
 - compliance, 338, 361, 550
- Gearbox compliance, 242, 314, 492, 513, 551
 - dynamics, 483
- Generalized Pareto Distribution
 - peaks over threshold, 571
- Geometry
 - TSDF integration, 298
- Global shutter
 - definition, 263
- Goal Structuring Notation, 502
 - argumentation grammar, 502
- Goal-structured physical safety case
 - definition, 499
- Governance synthesis
 - Tier 4 frontier, 529
- GPU
 - memory bandwidth wall, 89
 - thermal design power, 28
- Gravity compensation
 - hold state, 338
- Grounding
 - ambiguity, 327
 - metric mapping, 325
- Handover
 - authority register, 464, 465
 - autonomous fallback, 464
 - takeover request, 464
- Handshake protocol latency, 461
- Haptic feedback
 - authority boundaries, 449
- Hardware
 - actuator boundary, 464
 - ARM MPAM, 270
 - branchless DDA, 295
 - bus jitter, 239
 - clock synchronization, 283
 - DMA bus contention, 293
 - fieldbus, 464
 - gearbox compliance, 239
 - LiDAR range walk, 278
 - MIPI CSI-2, 269
 - rolling shutter, 263
 - SerDes bandwidth, 239
 - SerDes Jitter, 248
 - silicon bandwidth, 283
 - synchronization timers, 266
 - thermal derating, 239
 - thermal expansion, 281
 - timer input-capture, 291
 - Trip zones, 372
 - V4L2 DMA, 269
 - watchdog timer, 323, 329
- Hardware acceleration, 551
- Hardware Architecture
 - common-mode coupling, 393
- DMA starvation, 394
 - ground bounce, 393
 - ground plane, 393
 - microcontroller, 393
 - PCIe bus jitter, 399
 - SerDes, 399
 - system-on-chip (SoC), 393
- Hardware fault register
 - trip, 532
- Hardware safety
 - WWDG, 419
- Hardware timer interrupt
 - DMA latching, 144
- Hardware timestamps
 - PHY capture, 44
- Hardware tracing
 - CoreSight ETM, 486
- Hardware trip zone, 122
- Hardware watchdog, 384
- Hardware wear
 - actuator thermal duty cycle, 142
 - bearing life, 191
- Hardware-in-the-loop, 479, 484
 - bus contention, 229
- Harmonic drive, 221
 - fatigue life under RL
 - exploration, 191
- Heterogeneous computing
 - IPC, 408
- Heterogeneous silicon partitioning
 - definition, 408
- Heterogeneous system-on-chip
 - architecture, 407
- Human-machine interventions, 547
- Humanoid robotics
 - capstone archetype, 16
- Hysteresis
 - state-dependent, 422
- Idempotency, 8
- IEC 60204-1
 - Category 1 stop, 399
 - Stopping categories, 390
- IEC 61508
 - SIL 4 testing, 531
- IEEE 1588 gPTP
 - hardware timestamping, 46
- IEEE 1588 Precision Time Protocol
 - distributed clock sync, 166
- IEEE 1588 PTP, 283
- Imitation learning
 - expert demonstration, 464
 - expert recovery, 464
- Impedance controller, 323
- Importance Sampling, 542
- In-situ fault injection
 - definition, 484
- In-situ physical fault rigs, 479
- Incidents
 - DARPA Robotics Challenge, 97
 - Uber Tempe collision, 96
 - Williston accident, 279
- Independent enforcement, 550
- Inductive voltage droop
 - power delivery network, 585
 - VLSI design, 406
- Inertial mass, 546
- Inference latency, 108
- Information age
 - budget, 544
 - definition, 260
 - end-to-end, 544
 - physical pipeline, 259
- Initial condition matching, 452
- Initial Velocity Dispersion, 305
- Injection attack
 - physical, 31
- Innovation, 304
- Innovation gate, 313
- Innovation Gating Must Precede Sensor
 - Fusion (The Midpoint Fallacy), 315
- Innovation residual, 310
- Innovation sequence, 308
- Input-to-State Stability
 - classical control, 225

- Inset clearance
 - definition, 276
- Integrative Capstone Embodiment, 16
- Integrator windup, 370
- Intent, 320
- Intent Grounding
 - SayCan affordances, 325
- Intent grounding refusal
 - definition, 321
- Intent ingestion, 331
- Intent lease, 372
 - definition, 322
 - temporal contract, 87
- Intent record
 - force envelope, 322
 - spatial tolerance envelope, 322
 - temporal lease, 322
- Inter-core authority multiplexing, 448
- Inter-core communication
 - RPMSG, 322
 - RPMSG framework, 587
- Inter-processor communication
 - RPMSG, 372
 - shared memory, 346
- Interconnect
 - RPMSG shared memory, 330
- Interconnect crossbar, 410
- Interconnect crossbar contention, 106
- Interference
 - shared power rail, 430
- Intervention, 441
- Intervention curation
 - definition, 160
- Intervention masking, 458
- Intervention record
 - definition, 161
- Invalid-test rule
 - definition, 483
- Invalidation triggers, 520
- Invariant shields, 547
- Inverse-depth metric grounding
 - definition, 272
- Inverse-depth parameterization
 - visual odometry, 565
- Inverter
 - desaturation trip, 352, 358
- IOMMU
 - DMA isolation, 408
- ISO 10218, 222
 - contact assessment, 206
 - Protective stop, 224
 - safety envelope, 483
- ISO 21448
 - SOTIF, 245, 506
 - SOTIF and UL 4600, 548
- ISO 26262
 - ASIL D, 234
 - Hardware Fault Injection, 491
- ISO/TS 15066, 222, 551
 - biomechanical limits, 501
 - power and force limiting, 206
 - safety envelope, 510
 - safety margin, 399
 - separation distance, 47
- Jerk
 - minimization, 353
- Jerk profile
 - definition, 353
- Joint current, 333
- Joule heating, 221, 313, 314, 333, 370, 379, 390, 513, 546, 549
 - actuator thermal limits, 204
 - component failure, 482
 - current derating, 509
 - fault coupling, 491
- Kinematic boundary violations, 247
- Kinematic braking
 - integration, 501
- Kinematic governor, 545
- Kinematic reachability
 - intent horizon, 576
- Kinematic Roofline Nomogram, 11
- Kinematic singularity
 - Jacobian rank deficiency, 576
- Kinematic stopping distance
 - information age, 591
 - velocity scaling, 545
- Kinematic trajectory planning, 547
- Kinematic velocity root
 - quadratic clearance, 47
- Kinematics
 - jerk limits, 332
 - lateral trajectory drift, 223
 - lever arm acceleration, 265
 - metric unprojection, 327
 - Pick-and-Place, 248
 - SE(3), 248, 313, 399
 - SE(3) calibration, 221
 - SE(3) trajectory, 494
 - SE(3) transform, 313
 - semantic-to-metric, 325
 - stopping envelope, 338
 - uncertainty, 283
 - workspace, 332
 - workspace boundaries, 331
 - workspace reachability, 331
- Kinetic energy, 551
 - braking dissipation, 24
- Kinetic momentum
 - definition, 43
- Kinetic stopping distance
 - barrier inset, 574
- Kinodynamic feasibility envelope
 - definition, 344
- KV-cache ring buffer
 - definition, 303
- Latency
 - bus jitter, 338, 375
 - open-loop chunk horizon, 89
 - Stopping distance contribution, 377
- Latency waterfall
 - definition, 266
- Lead systems adjudicator
 - accountability, 502
- Leader arm
 - teleoperation, 144
- Lease
 - spatial invalidation, 338
 - temporal expiry, 338
- Least-squares projection, 390
- Lever-arm effect
 - apparent acceleration, 577
- Lie derivative
 - directional clearance decay, 574
- Lie derivatives, 309
- Lie group
 - rigid-body kinematics, 86
- Lifecycle degradation
 - definition, 66
- Linear Complementarity Problem
 - contact physics, 196
- Linear complementarity problem
 - contact mechanics, 577
- Lipschitz continuity
 - policy sensitivity, 95
 - spectral normalization, 566
- Local checks
 - authority withdrawal, 338
- Lockstep processor
 - fault detection, 409
- Log-odds transformation
 - occupancy mapping, 565
- Loud failure, 309
- Lyapunov differential equation
 - covariance divergence, 578
- Lyapunov function
 - stability certification, 225
- Machine Learning
 - 3D Gaussian splatting, 296, 565
 - geometric uncertainty, 283
 - ViT patchification, 273
- Manipulability index
 - Yoshikawa formulation, 576
- Manipulator Jacobian
 - differential kinematics, 576
- Marr's tri-level hypothesis
 - perception systems, 259
- Mathematics
 - covariance propagation, 277, 590
 - cubic variance expansion, 305
 - intrinsic projection, 264
 - inverse-depth, 272
 - SE(3) group, 293
 - Zhang's homography, 265
- Measurement epoch, 290
- Measurement freshness
 - definition, 42
- Mechanical backlash, 220, 228
- Mechanical Systems
 - end-stop, 394
 - mechanical slip, 393
 - timing belt, 393
- Memory
 - DMA, 269
- Memory bandwidth
 - worst-case execution time, 431
- Memory barrier
 - instruction ordering, 587
- Memory barriers
 - data memory barrier ('DMB'), 417
 - read barrier, 417
 - store-release, 417
- Memory contention
 - DRAM bank conflicts, 585
- Memory controller, 410
- Memory hierarchies, 551
- Memory Protection Unit
 - spatial isolation, 408
- Memory-mapped I/O, 408
- MEMS Acoustic Resonance, 31
- Metric drift, 242
- Metric grounding
 - definition, 271
- Metric spatial grounding, 77
- Microcontroller
 - bare-metal, 431
- Microprocessor, 274
 - Linux, 431
- Minimal intervention, 385
- Minimal Risk Maneuver
 - fail-operational systems, 391
- Minimal risk maneuvers, 465
- Minimal-intervention QP projection
 - (safety shield), 386
- Minimum safe intervention distance, 442
- MIPI CSI-2
 - DMA buffers, 274
 - sensor readout interface, 46
- Missing observable
 - definition, 533
- Mobile Industry Processor Interface
 - Camera Serial Interface (MIPI CSI-2)
 - definition, 269
- Mode averaging
 - mathematical derivation, 182
- Mode confusion, 458
- Model calibration
 - binned calibration error, 96
- Model predictive control, 326
- Moravec's Paradox, 6
- Moravec's paradox, 6
- Motor
 - Joule heating, 361
 - thermal derating, 362
 - torque limit, 362
- Motor drive
 - pulse-width modulation, 546
- Multi-rate hierarchy
 - definition, 274
- Multicore interference
 - FAA AC 20-193 / CAST-32A, 409
- Multimodality catastrophe
 - loss function, 179
 - mode averaging, 566
- Mutex
 - inter-processor blocking, 431
- Nagumo theorem
 - Viability kernel, 382
- Nagumo viability theorem
 - boundary condition, 574

- Negative event registry, 209
- Negative evidence, 295
- Nervous System, 547
- Nervous system
 - definition, 107
 - deterministic enforcement, 441
 - gatekeeper, 345
- Network
 - autoregressive, 303
 - bus latency, 361
 - rate limiting, 464
 - SerDes bandwidth, 361
- Networking
 - heartbeat monitor, 399
- Neural network
 - Lipschitz continuity, 325
- Non-smooth mechanics
 - Moreau time-stepping, 578
- Nonlinear control
 - Lie derivatives, 372, 386
- Normalization of deviance, 515, 516
- Rogers Commission, 515
- NTSB Report HAR-19/03, 447
- Object-centric 3D scene graph
 - definition, 273
- Observability
 - Hermann-Krener criterion, 578
 - Kalman rank criterion, 578
- Observability Gap, 542
- Observability matrix, 309
- Observable, 307
- Observation
 - freshness limit, 283
- Observational indistinguishability, 549
- definition, 537
- Occlusion covariance expansion, 304
- Occupancy grid mapping, 295
- Offline reinforcement learning, 190
- Offline-online evaluation disconnect
 - definition, 224
- Open-loop evaluation
 - definition, 223
- OpenAI Dactyl
 - system identification, 195
- Operating envelope
 - definition, 472
- Operational debt
 - simulation transitions, 212
- Operational design domain, 246, 249, 516, 521, 543
 - definition, 222, 500
 - training boundaries, 210
- Operational envelope
 - trichotomy, 510
- Operational independence
 - definition, 407
- Operational recovery loop
 - asynchronous notification, 97
- Operational response
 - definition, 534
- Operator reaction budget, 461
- Optional stopping, 242
- Oracle
 - definition, 482
- Out-of-distribution action
 - overestimation, 190
- Out-of-order execution
 - memory consistency, 417
- Overconfident predictor, 310
- Overvoltage protection
 - dynamic braking, 61
- Painlevé paradox, 196
- Paired probing
 - simulator validation, 198
- Passive compliance, 551
- Passivity
 - energy storage, 580
- Passivity theorem
 - feedback stability, 580
- Payload
 - oscillation, 338
- PDN voltage droop coupling
 - definition, 413
- Perception, 282
 - calibration drift, 283
 - dated snapshots, 284
 - local loop, 338
 - observation age, 283
 - pipeline latency, 282
 - zero-shot segmentation, 327
- Perception handoff contract
 - definition, 262
- Perceptual chattering
 - state reset hazard, 96
- Permission path, 514
- Phantom-object tracks, 309
- Phase margin
 - delay degradation, 46
 - transport delay penalty, 576
- Phase-locked loop
 - shared dependency, 430
- Photon shot noise, 278
- Physical AI
 - definition, 8
- Physical AI stack, 546
- Physical Artificial Intelligence
 - definition, 8
- Physical body
 - definition, 44
- Physical bounds, 544
- Physical reality, 546
- Physical Wall, 28
- Piezoresistive drift, 551
- Piezoresistive transducer
 - drift, 538
- Pinhole camera model
 - definition, 264
- Placement record
 - definition, 428
- Planning
 - STRIPS, 321
- Plant dynamics
 - Tier 1 frontier, 529
- Plant mutation during training, 192
- Poisson process
 - stationarity violation, 503
- Policy learning
 - temporal horizons, 344
- Policy manifest
 - schema, 208
- Policy vulnerability inference chain, 206
- Power Distribution Network
 - inductive rail droop, 413
 - inductive voltage droop, 406
- Power Electronics
 - inverter saturation, 399
- Power electronics
 - Brake chopper, 390
- Power Inverter Timers
 - dead-time insertion, 9
- Power Systems
 - power distribution network (PDN), 393
- Power-rail coupling, 423
- Practical detector
 - definition, 536
- Practical Engineering Question, 299
- Pre-trigger ring buffer, 209
- Precision Time Protocol
 - clock synchronization, 21, 586
 - hardware timestamping, 586
- Precision time protocol logging, 456
- Prediction Advances State but Generates Zero Evidence, 315
- Preempt RT
 - kernel latency, 408
- Preemption tier, 461
- Priority inversion, 394
 - Mars Pathfinder, 117
- Privilege boundary
 - unprivileged proposals, 85
- Privilege separation
 - Tier 3 release, 498
- Probabilistic roadmap
 - definition, 347
- Probability calibration
 - algebraic normalization, 95
- Processor Mailboxes
 - shared SRAM, 29
- Processor-in-the-loop, 478
- Proposal-permission architecture
 - budget enforcement, 42
 - enforcer, 179
 - privilege separation, 143
 - Tier 3, 441
- Proposal-permission decoupling
 - definition, 76
 - DRC dynamic balance, 97
- Proposal-Permission Architecture, 28
- Proposal-Permission Privilege Boundary, 8
- Protective intervention, 474
- Provenance, 472
- Proxy metric fallacy, 515
- Pulse-width modulation
 - dead-time insertion, 584
 - Hardware clamp, 372
 - registers, 408
- Quadratic program
 - closed-form projection, 574
- Quadratic programming, 372, 396
 - active-set solver, 370
 - Worst-case execution time, 386
- Qualification ladder
 - definition, 477
- Quantile convolution
 - tail non-additivity, 45
- Quantitative robustness semantics
 - definition, 483
 - intuition, 483
- Quintic spline
 - analytical coefficients, 575
 - boundary solution, 353
- Rapidly-exploring random tree
 - definition, 347
- Rate-monotonic scheduling, 117
- Raymond Goertz
 - bilateral telemanipulation, 144
- Reachable Set
 - forward invariance, 31
- Reachable set, 31
- Real-machine fine-tuning debt, 204
- Real-time nervous system, 106
 - definition, 107
- Real-time scheduling
 - cache thrashing, 408
- Real-Time Systems
 - deadline enforcement, 399
- Real-time systems
 - memory barriers, 346
 - worst-case execution time, 583
- Reality gap
 - definition, 194
- Receding-horizon action chunking
 - definition, 347
- Recovery handshake, 124
- Reference checker
 - kinematic, 374
- Reference frame
 - definition, 261
- Reflected inertia
 - definition, 43
 - gearbox amplification, 512
- Reflected rotor inertia
 - definition, 51
- Reflex enforcer, 368
 - definition, 368
- Reflex monitor
 - safety enforcement, 76
- Refusal
 - grounding ambiguity, 321
- Regenerative braking
 - chopper resistor, 589
- Relative degree, 383
 - definition, 574
 - Inertia-aware certificates, 383
- Release claim, 545
- Release manifest
 - definition, 517
- Release verdict, 547
 - invalidation, 502
 - operate with conditions, 512
- Remote Processor Messaging
 - Shared SRAM leases, 372

- Remote Processor Messaging (RPMMsg)
 - mailbox, 408
- Reset controller
 - isolation, 430
- Residual risk, 530
- Residual-Claims Register, 542, 551
- Residual-claims register
 - definition, 533
- Resource contention
 - worst-case, 423
- Reward hacking
 - bang-bang current saturation, 206
 - solver exploitation, 179
- Riccati equation
 - covariance propagation, 578
- Rigid body transformations, 275
- Rigid Contact Mechanics
 - complementarity, 15
- Rigid-body dynamics
 - Euler-Lagrange formulation, 78
- Robot
 - transport, 282
- Rolling shutter
 - definition, 263
- Rolling-shutter motion compensation
 - definition, 264
- Rule of Three, 219
- Rule of three
 - zero-failure bound, 66
 - zero-failure testing, 569
- Runtime enforcement, 551
- Runtime monitor, 358
- Runtime monitoring, 249
 - constraint margin, 240
 - environmental assumptions, 240
 - execution latency, 240
 - intervention rate, 240
 - OOD detection, 240
- Runtime safety enforcement, 220
- Safe set, 382
- Safe stop 1, 390
- Safe stop 2, 390
- Safe Torque Off
 - hardware gate driver, 97
- Safe torque off, 122, 453
- Safety
 - admissible motion, 464
 - Category 1 Stop, 248
 - deterministic terminal state, 331
 - envelope, 283
 - Freshness Lease, 248
 - ISO 10218, 240, 494
 - Lipschitz bounds, 325
 - loop delay, 282
 - Operational Envelope, 248
 - physical zoning, 240
 - safety filter, 393
 - stopping margin, 464
- Safety Case, 430
- Safety case
 - claim-argument-evidence, 521
 - lifecycle, 520
 - physical AI, 499
- Safety enforcement, 441
 - execution buffer, 345
- Safety enforcer
 - hardware interlock, 511
 - real-time, 448
- Safety Engineering
 - safety envelope, 399
- Safety envelope
 - ISO 10218, 243
 - ISO/TS 15066, 521
- Safety filter
 - barrier certificate, 370
- Safety interlocks
 - hardware isolation, 394
- Safety shield, 385
 - control barrier function, 375
 - sub-millisecond QP, 370
- Safety silicon
 - hardware interlocks, 97
- Safety standards
 - ISO 13849, 323
- Safety supervisor
 - torque overrides, 247
- Safety-Critical Standards
 - autonomous systems, 26
- Scaling laws
 - embodied latency, 528
- Scenario ledger, 158
- Schema-detectable frame mismatch, 310
- Scope Mismatch, 542
- SE(3), 331, 332
 - spatial bounds, 338
 - target precision, 339
 - wrench feasibility, 362
- SE(3) Kinematics, 334, 339
- SE(3) kinematics, 243, 275, 514
 - dynamics, 483
 - error propagation, 277
- SE(3) pose, 220
- Self-Monitoring, 543
- Semantically wrong frame assignment, 310
- Sensing
 - absolute encoder, 394
 - calibration drift, 393
 - inertial measurement unit (IMU), 394
 - joint torque sensor, 394
 - optical encoder, 393
 - phase lag, 393
 - thermal bias drift, 394
- Sensor
 - camera intrinsics, 327
 - Coriolis mass flow, 304
 - IMU, 399
 - optical encoder, 399
 - resolver, 399
- Sensor read noise, 278
- Seqlock
 - sequence counter, 417
 - torn read, 417
- Sequence lock, 111
- Sequence lock (seqlock), 431
- Sequential Probability Ratio Test
 - Wald formulation, 569
 - Wald optimality, 570
- Sequential probability ratio test, 242
- SerDes
 - bus timeout, 338
 - sensor links, 10
- SerDes bandwidth, 244, 314, 514, 547, 551
- Serial bus transmission, 423
- Set invariance
 - Subtangentiality, 382
- Shannon entropy
 - information theory, 564
- Shared memory
 - SRAM mailboxes, 416
- Signal Temporal Logic
 - definition, 483
 - quantitative robustness, 568
 - syntax and semantics, 568
- Silent failure, 309
 - definition, 277
- Silicon aging
 - timing margins, 424
- Silicon contract, 551
- Silicon partitioning
 - definition, 408
- Sim-to-real
 - latency robustness, 178
- Sim-to-real contact exploit, 196
- Sim-to-real gap, 550
 - physical vector decomposition, 198
- Sim-to-real reality gap vector
 - definition, 194
- Simplex architecture
 - Advanced controller, 371
 - Baseline controller, 371
 - formal safety, 369
 - Safety manager, 372
 - Simplex switch, 372
- Simplex architecture pattern, 371
- Simplex switch, 385
- Simulation
 - timestep bounds, 209
- Simulation provenance
 - omission log, 210
- Simulation-to-reality (Sim2Real)
 - contact solver simplifications, 211
- Simulation-to-reality leap, 515
- Single-Event Upset
 - accelerator soft errors, 484, 486
 - multi-bit upsets, 484, 486
- Slip clutches, 551
- Smoothstep jerk bound, 451
- Smoothstep polynomial
 - jerk bounding, 575
- SoC architectures
 - heterogeneous, 431
- Softmax normalization
 - epistemic limitations, 95
- Software Architecture
 - real-time operating system (RTOS), 394
 - scheduling jitter, 394
- Software-in-the-loop, 478
- Spatial belief contract
 - definition, 291
- Spatial coordinate frames
 - SE(3) representation, 564
 - SE(3) transformation, 577
- Spatial Covariance Does Not Represent Physical Existence, 315
- Spatial information age
 - definition, 260
- Spatial memory, 288, 313
- Spatial partitioning, 426
- Spatial transformation matrices, 293
- Special Euclidean group
 - SE(3) grounding, 77
 - SE(3) rigid-body pose, 86
- Special Euclidean Group $SE(3)$, 314
- Speed and Separation Monitoring, 47
- SPRT
 - sequential testing, 242
- Stale-state execution, 314
- Standards
 - REP 105, 261, 293
- State
 - control vs. machine learning, 10
- State change
 - irreversible, 544
- State estimation, 305, 314
 - Bayesian provenance, 303
 - belief state, 288
 - propagation, 283
- State Expiry is Dictated by Physics, Not Software Timeouts, 315
- State freshness, 547
- State of charge (SoC), 221
- Static reservation, 426
- Statistical power
 - sample size, 244
- Statistical verification
 - collapse under fine-tuning, 204
- Stator, 377
- Stator current
 - abrupt removal, 338
- Stator delay, 242, 492
- Stator resistance
 - thermal coefficient, 56
- Stochastic Acceleration Diffusion, 305
- Stopping distance, 521
- Stopping envelope, 380
 - Anisotropy, 377
 - Kinematic distance, 377
 - Margin loss, 380
- Storage
 - Direct I/O, 152
- Streaming ring-buffer attention, 302
- STRIPS planning, 321
- Structural independence, 513
- Structural Isolation, 543
- Subsumption architecture, 324, 390
- Sum-of-squares optimization, 225
- Supervisory Control Levels, 442
- Support envelope relational join
 - definition, 169
- Survival envelope

- definition, 472
- Survivorship bias
 - in physical datasets, 173
- Symbol Grounding Problem, 321
- Symbol grounding problem, 321
- Symbolic logic
 - deterministic safety enforcement, 95
- Synchronous invariant checking, 125
- System-on-chip
 - resource contention, 513
- System-on-Chip (SoC)
 - heterogeneous compute, 407
- Systems-Theoretic Accident Model and Processes
 - definition, 476
- Systems-Theoretic Process Analysis
 - emergent hazards, 476
- Takeover reaction floor, 442
- Task planning
 - program synthesis, 327
- Teleoperation
 - demonstration yield, 155
 - kinematic asymmetry, 173
- Tempe fatal collision
 - investigation findings, 447
- Temporal action chunking
 - definition, 89
- Temporal calibration offset
 - definition, 265
- Temporal ensembling, 183
 - definition, 187
 - trajectory smoothing, 91
- Temporal logic falsification
 - definition, 483
- Temporal Skew Across Moving Bodies
 - Destroys Spatial Clearance, 315
- Test set contamination, 242
- Testing
 - Oracle, 472
- Theory Gap, 542
- Thermal capacity
 - definition, 43
- Thermal clock throttling
 - definition, 420
- Thermal derating, 242, 333, 379, 513
 - coprocessor, 338
- Thermal Dynamics
 - Joule heating, 550
- Thermal management
 - Joule heating, 212
- Thermal resistance
 - junction-to-ambient, 421
- Thermal time constant, 421
- Thermodynamic Limits of Computation, 12
- Threat model, 530
- Three Physical Machine Archetypes, 406
- Three-position enabling devices, 448
- Tightly coupled memory, 429, 431
- Time-sensitive networking, 119, 120
- Time-to-harm
 - definition, 537
- Time-to-Intervention
 - metric, 226
- Timeout threshold, 461
- Timestamp
 - asynchronous, 313
 - epoch skew, 313
- Timestamping
 - oscillator drift, 145
- Timing anomalies
 - worst-case execution time, 423
- Tolerance envelope
 - 3D spatial, 322
- Tool center point, 221
- Tool flange frame, 290
- Torque-speed curve, 221
- Torque-thermal horizon, 42
- Track association, 283
- Tracking error
 - disturbance bounds, 370
- Tracking error bound, 370
- Tracking history
 - classification changes, 24
- Training provenance record, 208
- Trajectory
 - braking time, 362
 - jerk-limited, 338
- Trajectory Generation
 - jerk-limited splice, 330
- Trajectory generation
 - S-curve profiles, 332
- Trajectory optimization
 - spline synthesis, 353
- Trajectory record
 - definition, 359
- Transducer
 - ultrasonic pulse-echo, 542
- Transduction timestamps, 44
- Transduction-to-actuation information
 - age definition, 259
- Translation lookaside buffer, 109
 - shootdown, 109
- Transport delay, 304
- Transport lag, 371
- Trial budget
 - wall-clock cost, 230
- Triple buffer
 - wait-free mailbox, 112
- Truncated signed distance field, 296, 313
- Two-speed brain architecture
 - hybrid control, 97
- Unannounced fault injection
 - definition, 237
- Uncertainty
 - covariance expansion, 339
- Uncertainty bound, 290
- Uncertainty envelope
 - definition, 262
- Unintended Acceleration
 - watchdog refresh, 487
- Unmonitored condition, 515
- Unmonitored state drift, 513
- Unobservable subspace, 308
- Unprivileged proposal
 - privilege boundary, 84
- Validation
 - data sanitization, 211
- Value map
 - attraction, 325
 - repulsion, 325
- Velocity clamp
 - kinetic derating, 534, 591
- Velocity restrictions, 551
- Verdicts
 - conditional operation, 506
 - refusal to operate, 506
 - unconditional operation, 506
- Verification gap
 - definition, 475
- Virtual spring-damper system
 - definition, 52
- Vision-language model, 331, 334
- Vision-language-action model, 537
 - trajectory synthesis, 89
- Visual-inertial odometry
 - definition, 265
- VLA KV-cache ring buffer
 - definition, 303
- Voltage droop, 414, 424
 - clock throttling, 430
 - transient response, 413
- Volumetric free-space raycasting, 295
- Voxel grid, 313
- VoxPoser
 - value maps, 326
- Vulnerability analysis
 - training regimes, 206
- Wait-free data structures
 - sequence locks, 431
- Warrant
 - inferential rule, 503
- Watchdog interlock
 - hardware safety, 511
- Watchdog lease, 108
 - bound, 124
- Watchdog timer
 - challenge-response, 419
 - conventional countdown, 419
 - hardware, 410
 - hardware downcounter, 584
 - Tier 2 verification, 472
 - windowed, 419
 - windowed service, 419
- Water hammer
 - definition, 476
- Wave variables
 - time delay passivity, 581
- Wear
 - bearings, 370
- Whole-body control
 - hierarchical QP, 580
- Windowed Watchdog Timer
 - hardware mechanics, 419
- Workload reduction, 427
- Worst-case execution time
 - Active-set QP, 386
 - multicore certification, 409
 - timing anomalies, 423
- Zero Moment Point
 - Vukobratovic criterion, 579
- Zero-allocation static architecture, 118
- Zero-command floor, 374
- Zero-failure testing
 - independence, 491
- Zero-order hold, 110
 - discrete sampling, 576